Synopsys Controller IP

25GMAC - 25G Ethernet MAC

Databook

DWC Ether 25GMAC – Product Code: H255-0
DWC XGMAC TSN1 Add-On – Product Code: D475-0
DWC XGMAC RxParser Add-On – Product Code: D476-0
DWC Data Integrity for 25GMAC Add-On – Product Code: H353-0
DWC AC Ether 1G to 25G TSN MAC – Product Code: G890-0

Version 4.20a
January 2025

Copyright Notice and Proprietary Information

© 2025 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at
https://www.synopsys.com/company/legal/trademarks-brands.html
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History **17**
Preface **19**
Databook Organization 19
Related Documentation 20
Web Resources 20
Reference Documentation 20
STAR on the Web (SotW) 22
Synopsys Statement on Inclusivity and Diversity 22
Customer Support 23
Chapter 1 Product Overview **25**
1.1. General Product Description 26
1.2. Standards Compliance 27
1.3. System Overview 28
1.3.1. Interfaces 28
1.3.2. Buffers and Memories 29
1.4. Features 31
1.4.1. MAC 31
1.4.2. MAC Transaction Layer (MTL) 34
1.4.3. DMA 36
1.4.4. AXI Interface 37
1.4.5. Audio Video Support 38
1.4.6. Time Sensitive Networking Features 38
1.4.7. Automotive Safety Features 38
1.4.8. Monitoring, Testing, and Debugging Support 39
1.5. Configurations 40
1.5.1. Configurability 40
Chapter 2 Architecture **43**
2.1. Introduction 44
2.2. CSR Bus Interface 46

2.2.1. AXI Subordinate Interface....46
2.2.2. APB Manager Interface....48
2.2.3. MAC Control Interface....49
2.3. Application Manager Interface....51
2.3.1. AXI4 Manager Interface....51
2.3.2. AXI Low-Power Interface....58
2.4. DMA Controller....62
2.4.1. Functions of the PDMA....64
2.4.2. Functions of the VDMA....65
2.4.3. Application Bus Burst Access....65
2.4.4. Application Data Buffer Alignment....66
2.4.5. Buffer Size Calculations....66
2.4.6. DMA Descriptor Fetch Operation....67
2.4.7. DMA Tx Data Transfer Operation....68
2.4.8. DMA Rx Data Transfer Operation....70
2.4.9. Rx DMA Data Transfer Flow....72
2.4.10. Rx DMA Data Flow....73
2.4.11. Mapping of VDMAs to PDMA....74
2.4.12. Tx DMA Data Flow....74
2.4.13. External SPRAM Memory for CSR Space of Per Channel Registers....75
2.4.14. Configurable Tx and Rx Cache Sizes....76
2.4.15. DMA Descriptor Write-Back Operation....76
2.4.16. DMA Start/Stop Operation....77
2.4.17. DMA Channel Number Sideband Signals....78
2.4.18. Memory Cache Size Requirements....78
2.4.19. Memory Cache Access Arbitration....79
2.4.20. DMA Error Handling....79
2.4.21. DMA Throughput Tuning....81
2.5. MAC Transaction Layer (MTL)....91
2.5.1. Transmit Path....91
2.5.2. Receive Path....108
2.6. The MAC Layer....120
2.6.1. PHY Interface Selection....120
2.6.2. Transmission....121
2.6.3. Reception....135

2.7. Interrupt Hierarchy in the MAC 144
2.7.1. Periodic Scheduling of Transmit and Receive Interrupt.... 149
2.7.2. Per channel Transfer Complete Interrupt....150
Chapter 3 Memory Organization for Tx and Rx Queue....153
3.1. Introduction.... 154
3.2. Asynchronous 2-Port RAM Feature....155
3.2.1. Low-Power, True Dual-Port SRAM Connection....155
3.2.2. High-Speed, Asynchronous Clocked 2-Port RAM Connection.... 156
3.3. Synchronous 2-Port RAM Feature.... 157
3.3.1. Data Flow.... 157
3.3.2. Application Clock Frequency....157
3.4. Synchronous Single-Port RAM Feature.... 159
3.4.1. Data Flow.... 159
3.4.2. Application Clock Frequency Requirements for Memory Transfers.... 160
Chapter 4 Optional Features.... 163
4.1. Source Address or VLAN Insertion, Replacement, or Deletion....164
4.1.1. Source Address Insertion or Replacement....164
4.1.2. VLAN Insertion, Replacement, or Deletion....165
4.1.3. Queue/Channel Based VLAN Insertion on Tx....165
4.2. Double VLAN Processing....167
4.2.1. Transmit Path.... 167
4.2.2. Receive Path....167
4.3. Packet Filtering....169
4.3.1. Source Address or Destination Address Filtering....170
4.3.2. VLAN Filtering....173
4.3.3. Extended VLAN Filtering....175
4.3.4. Rx VLAN Tag Lookup-Based Filtering....181
4.3.5. Layer 3 and Layer 4 Match Filtering....187
4.3.6. Hash Based Layer 3 and Layer 4 Filtering.... 189
4.4. Flexible Receive Parser....190
4.4.1. Overview of Flexible Receive Parser.... 190
4.4.2. Description of Flexible Receive Parser.... 190
4.4.3. Enabling the Flexible Receive Parser....202
4.4.4. Registers Related to the Flexible Receive Parser....202
4.4.5. Programming Guidelines for Flexible Receive Parser.... 203

4.5. IEEE 1588 Timestamps....204
4.5.1. Delay Request-Response Mechanism....204
4.5.2. Peer-to-Peer PTP Transparent Clock (P2P TC) Message Support....206
4.5.3. Clock Types....207
4.5.4. Reference Timing Source....209
4.5.5. System Time Register Module....209
4.5.6. Transmit Path Functions....212
4.5.7. Receive Path Functions....215
4.5.8. Timestamp Correction....216
4.5.9. PTP Processing and Control....219
4.5.10. Improving the Timestamping Accuracy Using the Pulse and Latency Input from PCS....224
4.5.11. Additional/Auxiliary Features....226
4.6. Media Clock Generation and Recovery....229
4.6.1. Overview of Media Clock Generation and Recovery....229
4.6.2. Description of Media Clock Generation and Recovery....229
4.6.3. Presentation Time Counter....231
4.6.4. Comparators....232
4.6.5. Media Clock Generation and Recovery Flow....232
4.6.6. Enabling Media Generation and Recovery....234
4.6.7. Signals Related to Media Generation and Recovery....234
4.6.8. Registers Related to Media Clock Generation and Recovery....235
4.6.9. Programming Guidelines for Media Clock Generation and Recovery....235
4.7. One-Step Timestamp....236
4.7.1. MAC Transmit PTP Mode....236
4.7.2. One-Step Time Stamping for PTP Over UDP....239
4.8. PTP Offload Function....242
4.8.1. PTP Packet Generation....244
4.8.2. PTP Message Specific Fields....246
4.9. IPv4 ARP Offload....247
4.10. Memory Access in Debug Mode....248
4.10.1. Subordinate Mode Access....248
4.10.2. Debug Mode Access....250
4.11. Flow Control....252
4.11.1. Transmit Flow Control....252
4.11.2. Receive Flow Control....256

4.12. Checksum Offload Engine....259
4.12.1. Transmit Checksum Offload Engine....259
4.12.2. Receive Checksum Offload Engine....263
4.13. TCP Segmentation Offload....267
4.13.1. Description of the TSO Feature....267
4.13.2. Tunnel Packet Handling....268
4.13.3. DMA Operation with TSO Feature....268
4.13.4. TCP/IP Header Fields....270
4.13.5. Header and Payload Fields of Segmented Packets....271
4.13.6. Context Descriptor Sequence....272
4.13.7. External TSO Memory Interface....273
4.14. Receive Side Scaling Function....274
4.14.1. RSS Function when VXLAN/NVGRE Mode is Enabled....275
4.14.2. External RSS Lookup-Based Filtering, Routing, and Packet Duplication....275
4.15. QoS Function....277
4.15.1. QoS Parser....277
4.15.2. Hash/Sequence Number Comparator....278
4.15.3. QoS Interrupt....278
4.16. Management Counters....280
4.16.1. Address Assignments....280
4.16.2. Rx Channel Mapping with Rx Frame....281
4.17. ITU-T G999.1 Function Support....286
4.17.1. Transmit Path....286
4.17.2. Receive Path....286
4.18. External Lookup Interface....287
4.18.1. Receive DMA Routing and Status....287
4.18.2. Packet Duplication....288
4.19. VxLAN/NVGRE Support....290
4.19.1. Overview....290
4.19.2. VxLAN/NVGRE Controllability....293
Chapter 5 PHY Interfaces....295
5.1. Station Management Agent Interface (MDIO)....296
5.1.1. MDIO Interface Format....296
5.1.2. MDIO Operating Clock Frequency....298
5.1.3. MDIO Preamble Suppression....299

5.1.4. MDIO Single Read/Write Operations....299
5.1.5. MDIO Continuous Write Operation....300
5.1.6. MDIO Continuous Scan Operation....301
5.1.7. Trailing Clocks and Back-to-Back Transactions....301
5.1.8. Enabling Station Management Agent....302
5.2. Reduced Gigabit Media Independent Interface (RGMII) to PHY....303
5.2.1. Functional Description of RGMII PHY Interface....303
5.2.2. RGMII Clocks....304
5.2.3. Signal Conversion....304
5.2.4. RGMII Transmit Timing....308
5.2.5. RGMII Receive Timing....308
5.2.6. Enabling the RGMII PHY Interface....308
5.3. Reduced Media Independent Interface (RMII)....310
5.3.1. Introduction to Reduced Media Independent Interface....310
5.3.2. Description of Reduced Media Independent Interface....310
5.4. Multiple PHY Interfaces....315
Chapter 6 Multiple Channels and Queues Support....317
6.1. Block Diagram....318
6.2. DMA....319
6.2.1. Transmit Path....319
6.2.2. Receive Path....320
6.2.3. Priority Scheme for TxDMA and RxDMA....320
6.3. Multiple Queues Support in the MAC....321
6.4. Multiple Queues Support in the MAC Transaction Layer....322
6.4.1. Queue Memory....322
6.4.2. Traffic Classes and Transmit Queues....322
6.4.3. Data Center Bridging....324
6.4.4. Audio Video Bridging....327
6.5. Tx Side Arbitration Between DMA and MTL....333
6.6. Rx Side Arbitration Between MTL and DMA....334
6.7. Rx Queue to DMA Mapping....335
6.7.1. Static Mapping....335
6.7.2. Dynamic (per packet) Mapping....335
6.8. Broadcast/Multicast Packet Duplication....339
6.8.1. Receive Channel-List Based Packet Duplication....339

6.8.2. Programmable Unicast Packet Duplication....339
6.9. Selection of Tag Priorities Assigned to Tx and Rx Queues....341
6.10. Distribution of Ingress Packets from MAC to Rx Queues....342
Chapter 7 Power Management and Energy Efficient Ethernet....345
7.1. Overview of Power Management....346
7.1.1. PMT Registers....346
7.2. Remote Wake-Up Mode....347
7.2.1. Remote Wake-Up Packet Filters....347
7.2.2. VLAN Frame Support....349
7.2.3. Enabling PMT through Remote Wake-Up Mode....349
7.3. Magic Packet Detection Mode....350
7.3.1. Enabling PMT through Magic Packet Mode....351
7.4. PMT Interrupt....352
7.5. Design Partition for Low-Power Support....353
7.5.1. Advantages of UPF....354
7.5.2. UPF Flow Block Diagram....355
7.5.3. Address Filtering with UPF....356
7.5.4. ARP Response Generation With UPF....356
7.5.5. Power-Down and Power-Up Sequence....357
7.6. Energy Efficient Ethernet....360
7.6.1. LPI Transmit Path Functions....360
7.6.2. LPI Receive Path Functions....363
7.6.3. LPI Timers....365
7.6.4. Power-Down Mode Support....365
7.6.5. Sideband Control to Change Speed....368
Chapter 8 Time-Sensitive Networking....371
8.1. Enhancements to Scheduled Traffic (EST)....372
8.1.1. Frequently Used Terms in EST....373
8.1.2. Updates to the Transmit Scheduling to Support EST....374
8.1.3. Transmission Gating Implementation....377
8.1.4. Idle Slope Computation Updates....377
8.1.5. Operational Details of GCL....378
8.1.6. Installing a New GCL....380
8.1.7. Impact of Transmit Scheduling Algorithms on EST....383
8.1.8. Error Reported in EST....384

8.1.9. Interrupts in EST for Stream-Gate Filtering....384
8.1.10. Programming Guidelines for EST....384
8.2. Frame Preemption (FPE)....385
8.2.1. Overview of the Frame Preemption....385
8.2.2. Description of the Frame Preemption....385
8.2.3. Enabling the Frame Preemption....386
8.2.4. GCL Modification to Support FPE....386
8.2.5. Impact of Preemption on CBS....387
8.2.6. mPacket Format....388
8.2.7. Transmit Preemption....392
8.2.8. Receive Preemption....393
8.2.9. Received Preemption Frames Support in Offload Engines....397
8.2.10. Verify and Respond mPackets....397
8.2.11. Frame Preemption and MMC Counter and Interrupt Registers....397
8.3. Time-Based Scheduling....399
8.3.1. Description of Time-Based Scheduling....399
8.3.2. Definitions Used....399
8.3.3. Enabling the Time-Based Scheduling Feature....405
8.3.4. Programming Guidelines for Time-Based Scheduling....405
8.4. Stream-Gate Filtering....406
8.4.1. Stream-Gate Filtering with EST....407
8.4.2. Stream-Gate Filtering with Frame Preemption....408
8.4.3. Enabling Stream-Gate Filtering....408
8.5. Policing Counters....409
8.5.1. Overview of Policing Counters....409
8.5.2. Description of Policing Counters....409
8.5.3. Enabling Policing Counters....412
8.5.4. Programming Guidelines for Policing Counters....412
8.6. Asynchronous Traffic Shaper (ATS)....413
8.6.1. Fetching Frame Arrival Time....413
8.6.2. Eligibility Time (ET Time) Computation....414
8.6.3. Transmission Selection Algorithm....415
8.6.4. DSTS Support with ATS feature....418
Chapter 9 Automotive Safety Features....419
9.1. AXI Interface Safety Protection....420

9.1.1. Overview ... 420
9.1.2. Description of AXI Interface Safety Protection ... 420
9.2. Error Correction Code (ECC) Protection for Memories ... 429
9.2.1. Overview ... 429
9.2.2. Description of ECC Protection for Memories ... 429
9.2.3. Enabling ECC Protection for Memories ... 436
9.2.4. Signals Related to ECC Protection for Memories ... 436
9.2.5. Registers Related to ECC Protection for Memories ... 437
9.2.6. Programming Guidelines for ECC Protection for Memories ... 437
9.3. On-Chip Data Path Parity Protection ... 438
9.3.1. Overview ... 438
9.3.2. Transmit Datapath Parity Protection ... 438
9.3.3. Receive Data Path Parity Protection ... 439
9.3.4. DMA Data Path Parity Protection ... 439
9.3.5. MTL-Tx Data Path Parity Protection ... 443
9.3.6. MTL-Rx Data Path Parity Protection ... 444
9.3.7. MAC-Tx Data Path Parity Protection ... 445
9.3.8. MAC-Rx Data Path Parity Protection ... 446
9.3.9. AXI Subordinate Data Path Parity Protection ... 447
9.3.10. Parity Error Injection ... 449
9.3.11. Interrupt for Parity Error ... 449
9.3.12. Recovering from a Data Path Parity Error ... 449
9.3.13. Enabling On-Chip Data Path Parity Protection ... 449
9.3.14. Signals Related to On-Chip Data Path Parity Protection ... 449
9.3.15. Registers Related to On-Chip Data Path Parity Protection ... 450
9.3.16. Programming Guidelines for On-Chip Data Path Parity Protection ... 450
9.4. FSM One-Hot and Timeout Protection ... 451
9.4.1. Overview ... 451
9.4.2. FSM One-Hot Protection ... 451
9.4.3. FSM Timeout Protection ... 451
9.4.4. Recovering From FSM Parity/Timeout Errors ... 453
9.4.5. Enabling FSM Parity and Timeout Protection ... 453
9.4.6. Signals Related to FSM Parity and Timeout Protection ... 453
9.4.7. Registers Related to FSM Parity and Timeout Protection ... 453
9.4.8. Programming Guidelines for FSM Parity and Timeout Protection ... 453

9.5. Application/CSR Interface Timeout Protection....454
9.5.1. Overview....454
9.5.2. Description....454
9.5.3. ATI and ARI Interface....454
9.5.4. AXI Manager Interface....454
9.5.5. AXI Subordinate Interface....455
9.5.6. Enabling Application/CSR Interface Timeout Protection....455
9.5.7. Signals Related to Application/CSR Interface Timeout Protection....455
9.5.8. Programming Guidelines for Application/CSR Interface Timeout Protection....455
9.6. Control Register Protection....456
9.6.1. Parity Generation....456
9.6.2. Parity Checking....456
9.6.3. Error Injection Capability....456
Chapter 10 Parameter Descriptions....457
Chapter 11 Register Descriptions....507
Chapter 12 Signal Descriptions....1629
Chapter 13 Descriptors....1795
13.1. Overview....1796
13.2. Descriptor Structure....1797
13.3. Descriptor Endianness....1799
13.4. Transmit Descriptor....1800
13.4.1. Transmit Normal Descriptor (Read Format)....1800
13.4.2. Transmit Normal Descriptor (Write-Back Format)....1805
13.4.3. Transmit Context Descriptor....1807
13.5. Receive Descriptor....1811
13.5.1. Receive Normal Descriptor (Read Format)....1811
13.5.2. Receive Normal Descriptor (Write-Back Format)....1813
13.5.3. Receive Context Descriptor (Write-Back Format)....1825
13.6. Enhanced Descriptor for Time-Based Scheduling....1829
13.6.1. Enhanced Normal Descriptor, Read Format....1829
13.6.2. Enhanced Normal Descriptor, Write-Back Format....1830
13.6.3. Enhanced Context Descriptor, Read Format....1831
13.6.4. Enhanced Context Descriptor, Write....1833
13.7. Header Payload Split Support....1834
13.7.1. Descriptor Structure....1836

Chapter 14 Programming Guidelines....1839
14.1. Initializing the MAC Registers....1841
14.2. Initializing the MTL Registers....1842
14.3. Initializing the DMA Registers....1843
14.4. Programming Guidelines for DMA....1845
14.4.1. Programming Guidelines for Idle Timer Function....1846
14.4.2. Programming Guidelines for Datapath Parity....1846
14.5. Performing Receive and Transmit Operation....1847
14.6. Stopping and Restarting Transmission....1848
14.7. Reconfiguring the DMA Registers....1849
14.8. Switching to New Descriptor List in Receive DMA....1850
14.9. Handling Bus Errors and Recovery....1851
14.9.1. Transmit DMA Channel....1851
14.9.2. Receive DMA Channel....1851
14.10. Setting Up Multi-Channel and Multi-Queue Operations....1853
14.10.1. Programming Sequence for the Transmit Channel/Queue....1853
14.10.2. Programming Sequence for the Receive Channel/Queue....1853
14.10.3. Programming Guidelines for Enabling Packet Duplication....1854
14.10.4. Programming Guidelines to Enable Broadcasting of Unicast Packet....1854
14.11. Programming Guidelines for XGMII Link State Transitions....1856
14.12. Enabling the Energy Efficient Ethernet....1857
14.13. Setting up TCP Segmentation Offload (TSO)....1859
14.14. Enabling the Header-Payload Splitting Offload....1860
14.15. Enabling the VxLAN/NVGRE Tunneled Packet Processing....1861
14.15.1. Tx....1861
14.15.2. Rx....1861
14.16. Programming Sequence for Queue/Channel Based VLAN Inclusion Register....1862
14.17. Programming Guidelines for External DA Hash Table-Based Filtering and Routing....1863
14.18. Setting up VLAN Filtering on Receive....1864
14.19. Setting up Extended VLAN Filtering....1865
14.20. Setting up External Rx VLAN Tag Lookup-Based Filtering and Routing....1866
14.21. Setting up the L3-L4 Filtering....1867
14.21.1. Writing to the Indirect Addressed Registers....1867
14.21.2. Reading the Indirect Addressed Registers....1867
14.22. Setting Up the Flexible Receive Parser....1868

14.22.1. Programming the Stream-Gate Filtering Registers....1868
14.23. MTL Indirect Read/Write Access....1870
14.23.1. MTL Single Indirect Read/Write Access....1870
14.23.2. MTL Continuous Indirect Read/Write Access....1870
14.24. MAC Indirect Read/Write Access....1872
14.24.1. MAC Single Indirect Read/Write Access....1872
14.24.2. MAC Continuous Indirect Read/Write Access....1872
14.25. Programming Guidelines for the Policing Counters....1874
14.26. Setting up the IEEE 1588 PTP Timestamping....1875
14.26.1. Initialization Guideline for System Time Generation....1875
14.27. Setting up the Flexible Pulse-Per-Second (PPS) Output....1877
14.28. Enabling the Audio Video Traffic Shaper....1879
14.28.1. Initializing the DMA....1879
14.28.2. Enabling Slot Number Checking....1880
14.28.3. Enabling Average Bits Per Slot Reporting....1880
14.28.4. Disabling Flow Control for AV Enabled Queues....1880
14.29. Programming the GCL and GCL Linked Registers....1882
14.30. Programming Guidelines for EST....1883
14.30.1. Programming the EST Registers....1883
14.31. Enabling the Frame Preemption Function....1885
14.31.1. Receive Side Programming....1885
14.31.2. Transmit Side Programming....1885
14.32. Setting up the Time-Based Scheduling Function....1886
14.33. Programming Guidelines for the Asynchronous Traffic Shaper (ATS)....1887
14.33.1. Initialization Flow....1887
14.34. Programming Guidelines for Media Clock Generation and Recovery....1888
14.34.1. Common Guidelines for Media Clock Generation and Recovery....1888
14.34.2. Programming Guidelines for Media Clock Generation....1888
14.34.3. Programming Guidelines for Media Clock Recovery....1888
14.35. Programming Guidelines for ECC....1890
14.35.1. Programming Guidelines for ECC Protection for Memories....1890
14.35.2. Programming Guidelines for ECC Protection for AXI Read/Write Operations....1890
14.35.3. Programming Guidelines for ECC Hardware Error Injection for Flexible Receive Parser Memory....1890

14.35.4. Programming Guidelines for ECC Hardware Error Injection in Debug Mode for AXI Read/Write Operations ... 1891
14.35.5. Programming Guidelines for ECC Hardware Error Injection in Debug Mode ... 1891
14.36. Programming Guidelines for On-Chip Datapath Parity Protection ... 1892
14.36.1. Programming Guidelines for Parity Protection of AXI Control Signals ... 1892
14.36.2. Programming Guidelines for On-Chip Datapath Parity Protection ... 1892
14.36.3. Programming Guidelines for Parity Hardware Error Injection in Debug Mode for AXI Control Operations ... 1892
14.37. Programming Guidelines for FSM and Timeout ... 1893
14.38. Programming Guidelines for Switching the AXI Clock Frequency ... 1894
14.39. Programming Guidelines for Dual System Time Source Support ... 1895
14.39.1. Initialization Guidelines for Local System Time Generation ... 1895
14.40. Programming Guidelines for 25G XGMII Speed Mode and Timestamping Accuracy ... 1896
14.41. Programming Guidelines for Enabling Tx Timestamp Status FIFO Streaming Interface ... 1897
Appendix A Area and Power ... 1899
A.1. Area ... 1900
A.1.1. DMA Area Estimates ... 1900
A.1.2. Gate Count Summary ... 1900
A.1.3. Area for Scan Ready and Clock Gating ... 1912
A.2. Power ... 1915
Appendix B Endian Support ... 1917
B.1. Introduction ... 1918
B.1.1. Little-Endian Mode ... 1918
B.1.2. Big-Endian Mode ... 1918
B.1.3. Byte-Invariant Big-Endian Mode ... 1918
B.2. 64-bit Data Bus Transfers ... 1919
B.2.1. Manager Interface ... 1919
B.2.2. Register Bus Interface ... 1919
B.3. 128-bit Data Bus Transfers ... 1922
B.3.1. Manager Interface ... 1922
B.3.2. Register Bus Interface ... 1923
Appendix C VxLAN / NVGRE Packet Format ... 1927
C.1. VxLAN Overlay Packet Structure ... 1928
C.1.1. Outer Header Using IPV4 ... 1928
C.1.2. Outer Header Using IPv6 ... 1930

C.2. NVGRE Overlay Packet Structure....1932
C.2.1. Outer NVGRE Header....1932
Appendix D 25GMAC Automotive Safety....1937
D.1. Overview....1938
D.1.1. Functional Safety Assessment....1938
D.1.2. Reference Configurations....1938
D.1.3. Failure Mode, Effects, and Diagnostic Analysis (FMEDA)....1938
D.1.4. Safety Manual....1939
D.2. Automotive Safety Package Documents....1940
D.3. Configuring the Automotive Safety Features....1941
Appendix E Internal Parameter Descriptions....1943

Revision History

Revision history for Synopsys Controller IP 25GMAC - 25G Ethernet MAC Databook:

Version Number	Date	Description
4.20a	January 2025	Added: ■ “OoS Interrupt” on page 278 ■ “Asynchronous Traffic Shaper (ATS)” on page 413 ■ “Programming Guidelines for the Asynchronous Traffic Shaper (ATS)” on page 1887 ■ “Programming Guidelines for Enabling Packet Duplication” on page 1854 Updated: ■ Instances of "DesignWare Cores", "DesignWare", and "DWC" to "Synopsys IP" to align with corporate naming conventions ■ Terminology as per Synopsys Statement on Inclusivity and Diversity ■ “Standards Compliance” on page 27 ■ “MAC” on page 31 ■ “MAC Transaction Layer (MTL)” on page 34 ■ “Periodic Scheduling of Transmit and Receive Interrupt” on page 149 ■ “Layer 3 and Layer 4 Match Filtering” on page 187 ■ “Ethernet Line Speed Dependency” on page 198 ■ “Transmit Path Functions” on page 212 ■ “Enhancements to Scheduled Traffic (EST)” on page 372 ■ “Descriptor Endianness” on page 1799 ■ “Programming Guidelines to Enable Broadcasting of Unicast Packet” on page 1854 ■ “Setting up TCP Segmentation Offload (TSO)” on page 1859 ■ “Enabling the Header-Payload Splitting Offload” on page 1860 ■ “Enabling the VxLAN/NVGRE Tunneled Packet Processing” on page 1861 ■ “Programming Sequence for Queue/Channel Based VLAN Inclusion Register” on page 1862

Version Number	Date	Description
		■ “Programming Guidelines for 25G XGMII Speed Mode and Timestamping Accuracy” on page 1896 ■ “Gate Count Summary” on page 1900 ■ “25GMAC Automotive Safety” on page 1937 ■ Table 2-6 on page 96, Table 2-7 on page 97, Figure 6-2 on page 323, Table 13-24 on page 1827, Figure 13-11 on page 1829, Table 13-25 on page 1829
4.10a	April 2023	Added: ■ “Tx Timestamp Status FIFO with Packet Identifier” on page 206 ■ “Improving the Timestamping Accuracy Using the Pulse and Latency Input from PCS” on page 216 Updated: ■ Appendix D, “DWC_25gmac Automotive Safety”
4.10a-lca00	November 2022	Initial Version

Preface

This document describes the Synopsys IP, 25GMAC - 25G Ethernet MAC (25GMAC). The 25GMAC implements the Ethernet MAC pertaining to the MAC layer. The term 25GMAC corresponds to 25G Ethernet MAC in the SolvNetPlus database.

This chapter contains the following sections:

- "Databook Organization" on page 19
- "Related Documentation" on page 20
- "Web Resources" on page 20
- "Reference Documentation" on page 20
- "STAR on the Web (SotW)" on page 22
- "Synopsys Statement on Inclusivity and Diversity" on page 22
- "Customer Support" on page 23

Databook Organization

The chapters of this databook are organized as follows:

- Chapter 1, "Product Overview" describes the overall architecture, and lists the product features, deliverables, and general requirements.
- Chapter 2, "Architecture" describes the 25GMAC architecture and its main blocks
- Chapter 3, "Memory Organization for Tx and Rx Queue"describes the memory organization in 25GMAC.
- Chapter 4, "Optional Features" describes about the optional features of 25GMAC.
- Chapter 5, "PHY Interfaces" describes the various PHY interfaces supported in 25GMAC controller.
- Chapter 6, "Multiple Channels and Queues Support" describes about the multiple channels and multiple queues of 25GMAC.
- Chapter 7, "Power Management and Energy Efficient Ethernet" describes the power management features supported by 25GMAC controller.
- Chapter 8, "Time-Sensitive Networking" describes the time sensitive networking features.
- Chapter 9, "Automotive Safety Features" describes the automotive safety features.
- Chapter 10, "Parameter Descriptions" describes 25GMAC configuration options and parameters
- Chapter 11, "Register Descriptions" describes the function of 25GMAC control and status registers.

- Chapter 12, "Signal Descriptions"provides the pinout and describes the I/O for 25GMAC.
- Chapter 13, "Descriptors" describes 25GMAC descriptors.
- Chapter 14, "Programming Guidelines" describes the initialization sequences for 25GMAC.
- Appendix A, "Area and Power" provides the information about area, power, and latency.
- Appendix B, "Endian Support" describes the various endian modes for various data bus widths in the 25GMAC controller.
- Appendix C, "VxLAN / NVGRE Packet Format" describes the packet formats for overlay encapsulation protocols supported in 25GMAC.
- Appendix D, "25GMAC Automotive Safety" describes the details of configuring the automotive features and the safety package documents in 25GMAC.
- Appendix E, "Internal Parameter Descriptions" 25GMAC describes internal configuration options and internal parameters.

Related Documentation

Verification IP (VIP)

VC Verification IP (VIP) for Ethernet has to be downloaded and installed separately as explained in the Installation Guide but is instantiated in the provided testbench. VC Verification VIP for Ethernet is written entirely in System Verilog enabling it to run natively in supported simulators for highest performance. Synopsys documentation for the VC VerificationVIP is available after product installation in the following locations:

- Overview of Synopsys Ethernet Verification IP Documentation

  <install_dir>/vip/svt/ethernet_svt/latest/doc/ethernet_svt_overview.pdf

Synopsys Tools

- coreConsultant User Guide, Synopsys, Inc. (included with the coreConsultant tool)

Web Resources

For more information, check the following Synopsys online resources:

- Synopsys IP product information: https://www.synopsys.com/designware-ip.html
- Your custom Synopsys IP page: https://www.synopsys.com/dw/mydesignware.php
- Documentation through SolvNetPlus: https://solvnetplus.synopsys.com (Synopsys password required)
- Synopsys Common Licensing (SCL): https://www.synopsys.com/keys

Reference Documentation

- Carrier Sense Multiple Access With Collision Detection (CSMA/CD) Access Method and Physical Layer Specifications, Standard 802.3-2018, Institute of Electrical and Electronics Engineers (IEEE)
- Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems, Standard 1588-2008, IEEE

- Standards for Local and metropolitan area networks—Virtual Bridged Local Area Networks, IEEE Standard, 802.1Q-2005
- Standards for Audio Video (AV) traffic, Standard 802.1AS-2020 and Standard 802.1Qav-2009, IEEE
- Standard for Energy Efficient Ethernet (EEE), IEEE Standard 802.3az-2010
- AMBA AXI and ACE Protocol Specification, https://developer.arm.com/documentation/ihi0022/latest
- AMBA APB Protocol Specification, https://developer.arm.com/documentation/ihi0024/latest
- RFC 768, User Datagram Protocol (UDP), DARPA Internet Program, August 1980
- RFC 791, Protocol Specification (Internet Protocol, Version 4 (IPv4) Specification), DARPA Internet Program, September 1981
- RFC 792, Internet Control Message Protocol Specification, DARPA Internet Program, September 1981
- RFC 793, Transmission Control Protocol (TCP), DARPA Internet Program, September 1981
- RFC 1071, Computing the Internet Checksum (memo), Network Working Group, September 1988
- RFC 2460, Internet Protocol, Version 6 (IPv6) Specification, The Internet Society, Network Working Group, December 1998
- RFC 2665: Definitions of Managed Objects for the Ethernet-like Interface Types, The Internet Society, Network Working Group, August 1999
- RFC 2819, Remote Network Monitoring Management Information Base, The Internet Society, Network Working Group, May 2000
- RFC 2965, HTTP State Management Mechanism, The Internet Society, Network Working Group, October 2000
- RFC 4443, Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification, The Internet Society, Network Working Group, March 2006
- XENPAK Multisource Agreement: A Cooperation Agreement for 10 Gigabit Ethernet Transceiver Package, XENPAK Group, September 2002
- 1588 Main Web Site, National Institute of Standards and Technology
- Hardware-Assisted IEEE 1588* Implementation in the Intel® IXP46X Product Line (white paper), Intel, Inc., March 2005
- A Frequency Compensated Clock for Precision Synchronization using IEEE 1588 Protocol and its Application to Ethernet, Sivaram Balasubramanian, Kendal R. Harris, and Anatoly Moldovansky, Rockwell Automation, November, 2003
- UNH Ethernet Clause 4 MAC Test Suite - Annex D, University of New Hampshire InterOperability Laboratory
- Magic Packet Technology White Paper, Advanced Micro Devices, Inc.
- Device Class Power Management Reference Specification, version 2.0 by Microsoft Corporation and Advanced Micro Devices, Inc.
- Synopsys Low-Power Flow User Guide, Version E-2010.12, December 2010 (SolvNetPlus ID required)
- Synopsys Low Power Verification Tools Suite User Guide, Version 2011.03, March 2011 (SolvNetPlus ID required)

- VXLAN: A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks (https://datatracker.ietf.org/doc/draft-mahalingam-dutt-dcops-vxlan/)
- NVGRE: Network Virtualization using Generic Routing Encapsulation draft-sridharan-virtualization-nvgre-04 (https://datatracker.ietf.org/doc/draft-sridharan-virtualization-nvgre/)
- IEEE 802.1Qbb 2011 IEEE Standard for Local and Metropolitan Area Networks Virtual Bridged Local Area Networks - Amendment: Priority-based Flow Control (PFC).
- IEEE P802.3cx/D2.0, Draft Standard for Ethernet Amendment: Media Access Control (MAC) service interface and management parameters to support improved Precision Time Protocol (PTP) timestamping accuracy.
- IEEE 802.1Qaz EEE Standard for Local and Metropolitan Area Networks-Virtual Bridged Local Area Networks - Amendment: Enhanced Transmission Selection (ETS).
- Microsoft Windows: Receive-Side Scaling Enhancements in Windows Server November 05, 2008
  - ITU-T Telecommunication Standardization Sector of ITU G.999.1 (10/2009). (Series G Transmission Systems and Media, Digital Systems and Networks. Digital Selection and Digital line system-Access Networks).
  - Microsoft windows: Offloading the Segmentation of Large TCP Packets (NDIS 5.1).
- Reduced Gigabit Media Independent Interface (RGMII), Version 2.6, Broadcom Corp., Hewlett-Packard Co., Marvell Technology Group, Ltd.

  RMII Specification, Revision 1.2, RMII Consortium.
- IEEE standards for Time-Sensitive Networking (TSN)
  - Enhancements for Scheduled Traffic, IEEE standard 802.1Qbv-2015
  - Specification and Management Parameters for Interspersing Express Traffic, IEEE standard 802.3br-2016
  - Frame Preemption, Standard 802.1Qbu-2016, IEEE
- Bridges and Bridged Networks - Amendment 28: Per-Stream Filtering and Policing, IEEE Standard 802.1Qci-2017

STAR on the Web (SotW)

You must review all STARs on the Web (SotWs) associated with your product. SotWs are considered a part of the Synopsys documentation suite, and show critical information related to your product. To review product SotWs, refer to the Synopsys IP product information:

https://www.synopsys.com/designware-ip.html

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive

language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Customer Support

To obtain support for your product, prepare the required files and contact the support center using one of the methods described:

- Prepare the following debug information, if applicable:
  - For environment set-up problems or failures with configuration, simulation, or synthesis that occur within coreConsultant or coreAssembler, select the following menu:
    - **File > Build Debug Tar-file**

    Check all the boxes in the dialog box that apply to your issue. This option gathers all the Synopsys product data needed to begin debugging an issue and writes it to the <*core tool startup directory*>/debug.tar.gz file.
  - For simulation issues outside of coreConsultant or coreAssembler:
    - Create a waveform file (such as VPD, VCD, or FSDB).
    - Identify the hierarchy path to the Design Under Test (DUT).
    - Identify the timestamp of any signals or locations in the waveforms that are not understood.
- For the fastest response, enter a case through SolvNetPlus:

  a. https://solvnetplus.synopsys.com

  Note SolvNetPlus does not support Internet Explorer.

  b. Click the **Cases** menu and then click **Create a New Case** (below the list of cases).

  c. Complete the mandatory fields that are marked with an asterisk and click **Save**.

  Make sure to include the following:
  - **Product L1:** DesignWare Cores
  - **Product L2:** Enterprise Ethernet

  d. After creating the case, attach any debug files you created.

  For more information about general usage information, refer to the following article in SolvNetPlus:

  https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources
- Or, send an e-mail message to support_center@synopsys.com (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
  - Include the Product L1 and Product L2 names, and Version number in your e-mail so it can be routed correctly.
  - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
  - Attach any debug files you created.

- Or, telephone your local support center:
  - North America:

    Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
  - All other countries:

    https://www.synopsys.com/support/global-support-centers.html

1

Product Overview

This chapter provides an overview of the DesignWare Cores 25GMAC (25GMAC) and describes its components and features.

This chapter contains the following sections:

- "General Product Description" on page 26
- "Standards Compliance" on page 27
- "System Overview" on page 28
- "Features" on page 31
- "Configurations" on page 40

1.1 General Product Description

The 25-Gigabit Ethernet Media Access Controller (DWC_25gmac) enables a host to transmit and receive data over Ethernet in compliance with the IEEE 802.3-2018 standard. The 25GMAC controller has the following major configurations:

- XGMAC-CORE: 25GMAC with a native interface
- XGMAC-MTL: 25GMAC with a transaction layer
- XGMAC-AXI: 25GMAC subsystem with AXI interface

The native application interface of XGMAC-CORE, XGMAC-MTL, and AXI4 interface of XGMAC-AXI enables easy integration with up to 25Gbps host applications. The 25GMAC supports 25Gbps/10Gbps/5Gbps/2.5Gbps/1Gbps/100Mbps/10Mbps line rates using XGMII/GMII/MII/RGMII/RMII interfaces. For more details, see “PHY Interface” on page 28.

25GMAC controller (also referred to as 25GMAC) can be used in numerous applications, such as Network Interface ports, backplane switches, and enterprise switches. The 25GMAC IP supports an optimized (in gate count, latency), configurable, and flexible product for multiple applications.

1.2 Standards Compliance

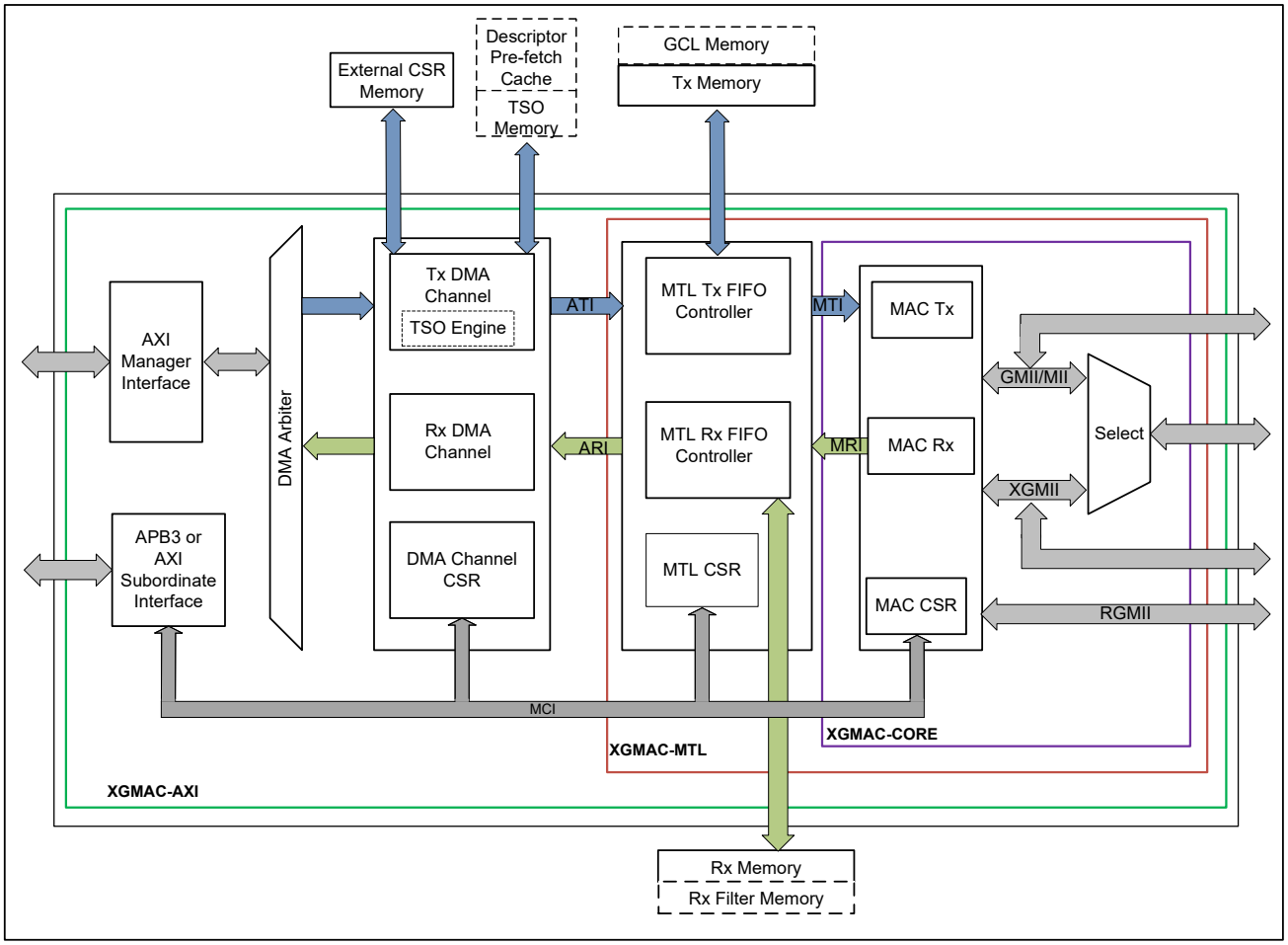
In addition to the default interfaces defined in the IEEE 802.3-2018 specifications, the 25GMAC supports several industry standard interfaces to the PHY. The 25GMAC is compliant with the following standards:

- IEEE 802.1AS-2020 and 802.1-Qav-2009 for Audio Video (AV) traffic
- IEEE 802.3-2018 for Ethernet MAC, Gigabit Media Independent Interface (GMII), 10G Media Independent Interface (XGMII).
- IEEE 1588-2008 for precision networked clock synchronization
- IEEE 802.3az-2010 for Energy Efficient Ethernet (EEE)
- AMBA 3.0 and APB3 subordinate ports
- AMBA4 AXI and ACE protocol specification, February 2013, ARM Ltd for AXI4 and APB4 interface
- NBASE-T Alliance 2.5 and 5 Gigabit Ethernet (USXGMII)
- Universal SXGMII PHY-MAC Interface for Multiple Network Port, Nov 2017 (Cisco)
- Reduced Gigabit Media Independent Interface (RGMII), Version 2.6, Broadcom Corp., Hewlett-Packard Co., Marvell Technology Group, Ltd.
- RMII specification version 1.2 from RMII consortium

1.3 System Overview

Figure 1-1 provides a system-level overview of the 25G/10G/2.5G/1G 25GMAC.

Figure 1-1 Overview of 25GMAC



1.3.1 Interfaces

1.3.1.1 AXI Interface

This interface is valid only for XGMAC-AXI configurations. The AXI interface transfers the data to and from the system memory through AXI4-compatible manager interface. When the AXI interface is selected, the host CPU uses the AXI subordinate interface to access the Control and Status registers (CSRs) of the 25GMAC subsystem. You can also select an APB3 port for CSR access instead of the AXI subordinate port.

1.3.1.2 Native Interface

In XGMAC-MTL or XGMAC-CORE configurations, the 25GMAC offers a simple interface on the Transmit and Receive side for transferring packet Data and packet Control to or from the application. The Native interface is a generic interface that provides simple handshake for Data or Control transfer and for extracting the status information.

1.3.1.3 PHY Interface

25GMAC supports the PHY Interfaces listed in Table 1-1.

Table 1-1 PHY Interfaces Supported by 25GMAC

Speed	Interface	Data Width	Interface Clock Frequency
25Gbps	XGMII	32-bit	781.25 MHz in SDR configuration and 390.625 MHz in DDW configurations
10Gbps	XGMII	32-bit	312.5 MHz in SDR configuration and 156.25 MHz in DDW configurations
5Gbps	XGMII	32-bit	156.25 MHz in SDR configuration and 78.125MHz in DDW configurations
2.5Gbps	XGMII	32-bit	78.125 MHz in SDR Configuration and 39.0625MHz in DDW Configurations
2.5Gbps	GMII	8-bit	312.5 MHz
1Gbps	GMII	8-bit	125 MHz
100Mbps	MII	4-bit	25 MHz
10Mbps	MII	4-bit	2.5 MHz
1Gbps	RGMII	4-bit	125 MHz
100Mbps	RGMII	4-bit	25 MHz
10Mbps	RGMII	4-bit	2.5 MHz
100Mbps	RMII	2-bit	50 MHz
10Mbps	RMII	2-bit	50 MHz

Note 25GMAC supports half duplex mode only for 10/100Mbps speed modes.

25GMAC also supports management data input/output (MDIO) interface capable of addressing MDIO devices that comply with clause 22 or clause 45 of the IEEE 802.3-2018 standard.

1.3.2 Buffers and Memories

1.3.2.1 Transmit and Receive FIFOs in XGMAC-MTL and XGMAC-AXI Configurations

The Transmit FIFO (Tx FIFO) buffers the data transferred from the application to the 25GMAC. Similarly, the Receive FIFO (Rx FIFO) stores the Ethernet frames received from the line until they can be transferred to the application.

These are asynchronous 2-port RAM, or synchronous 2-port RAM, or synchronous single-port RAM. Tx memory and Rx memory are 2-port RAMs (or two single-port RAM, each of half the depth in case of synchronous single-port RAM) of 68-bit or 133-bit width for 64 or 128 data bus widths, respectively.

The additional bits are used for storing the byte-enable information.

When multiple queues are selected, all Tx queue share the Tx FIFO memory and all Rx queue share the Rx FIFO memory. The application can program the size of the FIFO memory allocated to each Tx or Rx queue.

For ease of integration and testing, both memories are instantiated outside the controller. You must add technology-specific memory modules and related BIST logic outside the controller, and connect the data bus, address, and control signals to the 25GMAC. The 25GMAC provides a direct read or write access to the FIFO memories in Debug mode.

1.3.2.2 TCP/IP Header Buffer in XGMAC-AXI Configurations

This buffer is present only when Enable Separate Memory for TCP/IP Headers option is selected. A single-port SRAM is required for this memory and is external to the 25GMAC. The 25GMAC supports TCP/IP segmentation offloading, and the TCP/IP Header memory is used to store the TCP/IP headers for segmented packets.

The memory size can be configured from 64 bytes to 1KB in powers of 2 for each TSO channel. The total size gets multiplied by the number of DMA channels requiring TSO feature. The memory width is the same as the data-width configured (64 or 128).

1.3.2.3 Descriptor Cache in XGMAC-AXI Configurations

The memory (Single-Port RAM) is external to the 25GMAC and used to store the pre-fetched descriptors of all the DMAs active in the controller. Each Receive and Transmit VDMA is allocated space for the caching descriptors indicated by "Per Channel Receive Descriptor Cache Size (DWCXG_PCHRDC_SZ)" and "Per Channel Transmit Descriptor Cache Size (DWCXG_PCHTDC_SZ)" registers, respectively.

The memory width is same as the data-width configured (64 or 128) and the depth depends on the number of TxDMA and RxDMA channels selected during the configuration. This buffer can share the same memory as that used for the TCP/IP Header buffer (One memory instance) or can have a separate memory interface/instance if you select “Enable Separate Memory For TCP/IP Headers” option. The latter option is useful to avoid latencies and arbitration delays between the Descriptor cache accesses and the TSO Header accesses and thus get better performance throughput.

1.3.2.4 Gate Control List (GCL) Memory

The GCL memory (Single-Port RAM) is used for the Gate Control List. This list in the controller memory holds the gate controls and the associated time intervals, as specified in the 802.1Qbv EST function.

The GCL memory is present only when you enable the `DWCXG_AV_EST` option in the configuration. You can configure the depth of one instance of Gate Control List between 64 and 1024 in powers of 2, using the DWCXG_GCL_DEP parameter. The memory width is the sum of the width of the “Time Interval (ns)” for the validity of a schedule (DWCXG_GCL_WID) and the number of Traffic Class (DWCXG_NUM_TC).

1.3.2.5 MAC Receive Filter Lookup Memory

When single port RAM, extended VLAN is more than 32, RSS and DA Hash more than 256 are selected, these filters are implemented using the MAC Receive Filter Lookup Memory.

1.3.2.6 DMA CSR Memory in XGMAC-AXI Configurations

A single port RAM is external to the 25GMAC and is used to store the CSR registers of all the virtual DMAs that are active in the controller. The memory width is 128-bit and the depth depends on the number of virtual Tx DMA and Rx DMA channels selected while configuring the controller.

1.4 Features

The 25GMAC controller features are divided into the following categories:

- "MAC" on page 31
- "MAC Transaction Layer (MTL)" on page 34
- "DMA" on page 36
- "AXI Interface" on page 37
- "Audio Video Support" on page 38
- "Time Sensitive Networking Features" on page 38
- "Automotive Safety Features" on page 38

Attention

- Not every feature mentioned in the *Databook* is available in every possible configuration (of other parameters).
- You must always check the Features and Limitations[a] section in the *Databook* for general limits and restrictions on combinations of features.
- For a definitive confirmation, always contact Synopsys with your exact configuration and/or list of features you intend to use.

a. This list is not exhaustive.

1.4.1 MAC

The 25GMAC controller includes the following features:

- Full-duplex operation at 25/10/5/2.5 Gbps (XGMII), 2.5 Gbps/1 Gbps (GMII), 100Mbps/10Mbps(MII)
- Half-duplex operation in 10/100M modes. Supports
  - CSMA/CD protocol
  - Flow control using back pressure support
- Communication with an external 25/10/5/2.5 gigabit PHY provided by an XGMII interface that fully complies with Clause 46 of the IEEE 802.3-2018 standard
- Link fault detection in the XGMII receiver and automatic fault signaling in the XGMII Transmit interface
- Communication with an external 1Gbps/100Mbps/10Mbps PHY provided by a GMII/MII interface that fully complies with Clause 35/22 of the IEEE 802.3-2018 standard. Existing XGMII I/O can be used for GMII/MII operation, or a separate GMII/MII I/O can be configured
- 2.5-Gbps operation using an 8-bit GMII interface operating at 312.5 MHz
- Optional support for RMII or RGMII interface to connect to an external PHY, when operating in 10M/100M/1G modes of operation
- Clause 78 (Energy Efficient Ethernet (EEE) feature) of the IEEE 802.3az, standard support
- Separate transmission, reception, and configuration (control and status register) interfaces to the application

- Configurable big-endian or little-endian support for transmission and reception data paths. Support for dynamic big-endian or little-endian modes through sideband input pin
- 64-bit or 128-bit data transfer interface on the system side.
- Separate status word returned for each transmitted frame and up to 128-bit status word for each received frame
- Preamble, SFD, and other control character insertion (in transmit path) and deletion (in receive path)

Note Programmable minimum Inter-Packet Gap (IPG) from 64 bit times to 224 bit times.

- Supports transmit pace rate adjustment (through IDLE character addition) to match MAC to PHY SONET/SDH rates in 10G operation.
- Supports DIC (Deficit Idle Count) algorithm to maintain the average IPG gap of 12Bytes between frames in XGMII mode.
- Thirty-two-bit cyclic redundancy check (CRC) generation for transmit frames and checking for received frames. Programmable support for receive-side CRC stripping. CRC addition or replacement on transmit side controllable on a per-frame basis.
- Padding in transmit side for short (less-than-64 byte long) frames received from the application. Pad stripping on the receive side for short frames.
- Programmable source address inclusion or replacement for Transmit frames.
- Programmable frame length, supporting standard or jumbo (extendable to 16 KB) Ethernet frames
  - Support for jumbo frames up to 9 KB expandable up to 16 KB.
  - Truncation support for oversized frame reception and transmission.
- Supports IEEE 802.3 flow-control and priority based flow control (PFC)
  - Optional forwarding of received PAUSE/PFC control frames to the application
  - Automatic transmission of zero-quanta PAUSE/PFC frames on flow-control input deassertion
  - Support for unicast PAUSE/PFC frame reception
  - In the XGMAC-MTL and XGMAC-AXI configurations:

    Transmission of PAUSE/PFC frames triggered by either software controlled register bit or optional input signal or based on Queue fill level
- Support for VLAN-tagged frame processing in compliance with the IEEE 802.1Q standard
  - Support for VLAN-tagged frame detection, stripping, and filtering.
  - Support for VLAN tag inclusion, replacement, or deletion in transmit frames through per-frame sideband control signals and the CSR.
  - Support for up to two Stacked-VLAN tagged or QinQ tagged packets
- Support for the following flexible address filtering modes for received frames:
  - Up to 64 destination address (DA) perfect match address filters with masks for each byte. First 32 are selectable with granularity of 1, and option for 40, 48, or 64.
  - Up to 63 source address (SA) perfect match filters with masks for each byte.

  - 64 bit to 4096 bit (in steps for granularity of powers of 2) hash filter (optional) for multicast and unicast (DA) addresses.
  - Option to pass all multicast-addressed frames.
  - Promiscuous mode to pass all frames without any filtering for network monitoring.
  - Up to 4096 VLAN-based perfect matched selectable with option 4, 8, 16, 24, 32, 40, 48, 64, or 4096, and hash filtering.
  - VLAN-based perfect matched and hash filtering.
  - Up to 64 Layer 3 and Layer 4-based (TCP or UDP over IPv4 or IPv6) match filters, first 8 are selectable with granularity of 1, and option for 16, 24, 32, 40, 48, or 64.
  - Pass all incoming packets with a status report as determined by the filter.
- Optional network statistics with RMON or MIB Counters (RFC2819/RFC2665)
  - Option to select individual counters during configuration to save area.
  - Option to implement 64-bit RMON counters (default is 32-bit) to avoid fast wrap-up at 10 Gbps data rates. For example, octet counters.
- Optional features to support Ethernet packet timestamping as described in IEEE 1588-2008 (64-bit timestamps given in the Tx or Rx status of each packet).
  - In the XGMAC-AXI configurations:

    Supports Tx timestamps in CSR Status Register for 2-step timestamping as Tx status is not provided back to the host. Option to store up to 16 timestamps with packet identifier in the CSR.
  - One-step timestamping in Tx direction with the host providing a 64-bit timestamp along with TX packet. MAC parses PTP (over Ethernet) packets and insert or correct timestamp in appropriate header field.
  - Parses received frames for PTP packets and take timestamps as per configured node
  - Provides registers for asymmetric time correction
  - Supports internal System time generation and update along with option of taking reference time from input pins alternatively.
  - Support for dynamic selection of reference time source through control bit in CSR.
  - Flexibility to control the Pulse-Per-Second (PPS) output signal (ptp_pps_o)
  - Support for capturing timestamp based on external trigger and providing it in CSR.
  - Optional PTP offload module to support automatic generation and transmission of SYNC and Delay request/Response PTP packets
- Option to select up to 16 general purpose inputs and outputs. The general purpose inputs are programmable to generate interrupts on rising or falling edges.
- Loopback capability at XGMII/GMII (transmit data to receive path).
- Detection of LAN wake-up frames and AMD Magic Packet frames during the power-down mode. Wake-on LAN support up to 16 filters.
- MDIO Manager interface (optional) for multiple PHY devices and their configuration and management.

- Interrupt generation mechanism through MDIO for hot-plugged XENPAK.
- IPv4 header checksum offload on reception.
- TCP, UDP, or ICMP checksum offload (IPv4 and IPv6) for received packets.
- Receive-Side Scaling (RSS) based on Toeplitz HASH algorithm. The incoming traffic is segregated into (up to) 16 DMA channels based on the RSS hash. IP out-of-sequence detection for TCP/UDP over IP traffic in each channel based on the RSS hash. The out-of-sequence status is given along with the data.
- Option to interface with an external look-up engine, which can provide the DMA channel number to which the packet must be routed.

1.4.2 MAC Transaction Layer (MTL)

The MTL block consists of two sets of FIFOs: a Transmit FIFO with programmable threshold capability, and a Receive FIFO with a programmable threshold (default 64 bytes).

Common Transmit and Receive Features

- 64-bit or 128-bit, Transaction Layer block that bridges the application and the MAC. Optimization for packet-oriented transfers with packets delimiters.
- Multiple queues in single RAM on TX side (up to 16) and another on RX side (up to 16).
- RAM memory instantiation outside the top-level module to facilitate memory testing or instantiation. Disabling of Data Memory RAM chip-select when inactive to reduce power consumption.
- Option to select asynchronous 2-port RAM or synchronous 2-port RAM or Single-port RAM for the Transmit FIFO and Receive FIFO memory.
- Optional Synchronous 2-port RAM interface with same read and write clock to optimize memory area.
- Optional Synchronous Single Port RAM interface with same read and write clock for automotive applications.
- Programmable burst length, up to half the size of the MTL Rx queue or Tx queue size, to support burst data transfer in the XGMAC-MTL configuration
- Supports store and forward and threshold modes for data transfer in FIFO. Programmable threshold capability for each queue (default of 64 bytes)
- Programmable size for each queue buffer.
- CSR Secondary Port Read/Write access to Rx FIFO and Tx FIFO memory to enable boot-up sequence and to enable the host to send or receive packets directly without using the DMA.

Transmit Features

- Transmit memory size from 1 KB to 32 KB for single queue configurations and up to 256 KB for multiple queues configurations, with size configurable in powers of 2.

  Programmable size with granularity of 256 bytes per queue.
- Optional module to calculate and insert IPv4 (L3) header checksum and TCP, UDP, or ICMP (L4) checksum in each queue.
- Automatic retransmission of collision packets in half-duplex mode.

- Discard packets on late collision, excessive collision, excessive deferral, and under-run conditions with appropriate status.
- Multiple (up to eight) traffic classes are supported, with the following scheduling algorithms. The number of traffic classes and transmit queues can be independently selected in such a way that the number of transmit queues is always greater than or equal to the number of traffic classes.
  - Weighted Round Robin (WRR)
  - Deficit Weighted Round Robin (DWRR) (when DCB enabled)
  - Weighted Fair Queuing (WFQ) (when DCB enabled)
  - Strict Priority (SP)
- Optional packet level control for
  - VLAN tag insertion or replacement
  - Ethernet source address insertion
  - Checksum insertion
  - One-step timestamp correction
  - Timestamp control
  - CRC and pad control
- When audio-video bridging is enabled, supports Credit-based Shaper scheduling algorithms in configurations with multiple queues.
- Optional, forwarding the Transmit status to the application.
  - When time-sensitive networking is enabled, Time Based Scheduling (TBS).

Receive Features

- Rx FIFO size from 4 KB to 32 KB for single queue configurations and up to 256 KB for multiple queues configurations, with size configurable in powers of 2.

  Programmable size with granularity of 256 bytes per queue.
- Insertion of Rx Status vectors into the Rx queue after the EOP transfer (in Threshold mode) and before SOP (in Store-and-Forward mode) in XGMAC-MTL configuration. In store-and-forward mode, the status is available before the packet data. Therefore, the application can take the packet processing decision even before the packet is read from the queue. This enables multiple-frame storage in the Rx Queue without requiring additional memories blocks to store the Rx status of packets.
- Option to filter all error packets on reception and not forward them to the application in the store-and-forward mode.
- Option to forward the undersized good packets.
- Automatic generation of Pause packet control to the MAC based on the Rx Queue fill level with programmable Flow control thresholds in each Rx queue.
- Received packets are mapped into the multiple queues based on the Priority field of the VLAN tag or on Packet Type (DCB control packet, and so on).

1.4.3 DMA

The DMA block exchanges the data between the MTL block and the host memory. The host can use a set of registers (DMA CSR) to control the DMA operations. The DMA block supports the following features:

- 64-bit, 128-bit data transfers
- Multi-channel Transmit and Receive engines (up to 16 Transmit and 16 Receive channels)
- Separate DMA channel in the Transmit path for each queue in MTL. Single or multiple DMA channels for any number of queues in MTL Receive path
- Descriptor structure to support the following:
  - Byte-aligned addressing for data buffer
  - Dual-buffer descriptor ring and support for 64-bit addressing in descriptor structure. Expandable descriptor with context descriptor for additional control and status information
  - Descriptor architecture to allow large blocks of data transfer with minimum CPU intervention (each descriptor can transfer up to 8 KB of data)
  - TCP Segmentation Offload (TSO) and IEEE 1588 Timestamps
  - One-step PTP time correction in Tx, that is, input 64-bit Timestamp provided in descriptor
  - Option to provide 64-bit Timestamp for received packets
  - Option to give up to two VLAN tags in descriptor for insertion or replacement in Tx and stripped VLAN tags in Rx
  - Option to split the packet header (Layer 3 and Layer 4) and Layer 4 payload in different buffers
  - Delivers RSS hash and Out-of-Sequence information for packets received with TCP/UDP over IP payload
- Individual programmable burst size for TxDMA and RxDMA engines for optimal host bus utilization
- Selectable number of TxDMA channels with TCP Segmentation Offload (TSO) feature enabled. TSO engine to support TCP over IPv4/IPv6 segmentation
  - Local memory option for storing TSO headers and better performance
- Routing of received packets to the DMA channels based on RSS hash or the Ethernet DA or VLAN Priority in multi-channel DMA configurations
- Register programmable RX buffer sizes for each DMA channel
- Additional per-DMA channel interrupt pins for normal packet transfers
- Per-packet Transmit and Receive Interrupt control in descriptors
- Improved DMA architecture for better performance and throughput
- Supports pre-fetching and caching up to 32 descriptors for each DMA channel
- Descriptor fetches, Data transfers and Descriptor writes are de-linked and performed independently so as to reduce gaps between data transfers of each DMA
- TxDMA engines have capability to issue up to 16 outstanding requests for packet/burst transfer
- DMA descriptor writes can be “posted”, so that next descriptor can be processed immediately for next packet

- Descriptor memory cache can be either shared with TSO Header memory or can be a separate instance
- Option to support L2 Network Virtualization protocol (VXLAN/NVGRE) stateless offloading functions are as follows:
  - Recognition and processing of either VXLAN or NVGRE (selectable through programming) based Tunnelled packets
  - Checksum Offload functions acts on both outer UDP/IP header as well as the inner L3-L4 headers
  - TCP Segment Offload Engine segments a tunneled packet into multiple segments each with necessary tunneled packet headers
  - Routing of the incoming tunneled packets based on VNI/TNI, FlowID (for NVGRE), Inner DA and Optionally Inner VLAN Tag
  - Supports External Lookup based or Hash based mechanism to select the RxDMA channel for forwarding
  - Option to transfer the complete header (including Tunnelling header and Inner Packet header) and inner payload in to separate buffers
  - Support for processing both Tunneled (VxLAN/NVGRE) packets and Un-tunneled (Normal) packets simultaneously
- Time-sensitive conditional packet transmission by comparing the Slot Time information provided in the descriptor (useful for AV applications)

1.4.4 AXI Interface

This section lists the AXI Interface features.

1.4.4.1 AXI Manager Interface

The AXI manager interface is compatible to interface with AXI-4 subordinate ports. It supports the following features:

- 32-bit, 40-bit, or 48-bit address width, little-endian and byte-invariant big-endian modes
- AXI low-power interface
- 64-bit or 128-bit data
- OKAY, SLVERR, and DECERR responses from AXI subordinates
- Software-selected type of AXI burst (fixed and variable length burst) in AXI manager interface; Option to select address-aligned bursts
- AXI 4 KB boundary burst splitting and option to limit burst splitting boundary to 1 KB boundary
- Up to 256 outstanding Read and Write transactions with register programmable control. Minimum of 2 outstanding requests per channel in maximum configuration of 16 TxDMA and 16 RxDMA channels.
- One of the outstanding requests is reserved for high priority Descriptor operation.
- Posted writes during multiple data transfers from the AXI manager interface to maximize the bus utilization

- AXI-ID based on DMA channel number. Each TX and RXDMA has its own ID for data transfer
- ACE-LITE support in which user-programmable values are driven on AxDOMAIN and AxCACHE signals, depending on the type of data transfer (descriptor, header, payload, and so on)

The AXI manager interface does not generate the following:

- Atomic, Exclusive, Locked, and Protected accesses
- EXOKAY access

1.4.4.2 AXI Subordinate Interface

The AXI Subordinate interface supports the following features

- 32-bit, 64-bit, 128-bit for CSR access
- 32-bit address width
- Little-endian and byte-invariant big-endian modes
- Narrow burst and FIXED or INCR burst
- Option to select AXI4-LITE interface

The AXI subordinate interface does not support the Exclusive, Atomic, Locked, Cache, and Protected accesses.

1.4.5 Audio Video Support

25GMAC supports the following Audio Video (AV) features:

- Separate channels or queues for AV data transfer
- Up to eight queues on the Receive paths for AV traffic and seven queues on the Transmit path for AV traffic
- IEEE 802.1-Qav specified credit-based shaper (CBS) algorithm for Transmit channels

1.4.6 Time Sensitive Networking Features

25GMAC supports the following Time Sensitive Networking (TSN) features:

- IEEE 802.1Qbv-2015, Enhancements to Scheduling Traffic
- IEEE 802.1Qbu/802.3br, Frame preemption and Interspersing Express Traffic
- IEEE 802.1Qci-2017, Stream-Gate Filtering and Monitoring
- Time Based Scheduling
- IEEE Std 802.1Qcr™-2020, Asynchronous Traffic Shaping

1.4.7 Automotive Safety Features

The automotive safety features are targeted to improve the reliability and reduce the time taken to detect faults in the controller:

Note These features are available only with DWC-Ethernet-XGMAC-ASP license.

25GMAC supports the following features for automotive safety:

- AXI interface safety protection
- Error correction code (ECC) protection for memories
- On-chip data path parity protection
- FSM parity and timeout protection
- Application/CSR interface timeout protection

1.4.8 Monitoring, Testing, and Debugging Support

- Support for internal loopback on the XGMII/GMII for debugging
- DMA states (Tx and Rx) as status bits
- Current Tx or Rx Buffer pointer as status registers
- Current Tx or Rx Descriptor pointer as status registers
- Statistical counters to calculate the bandwidth served by each Transmit channel when DCB support is enabled
- Tx or Rx Memory accessible through Secondary port for debug
- Provides Debug status register that gives the status of FSMs in the Transmit and Receive data paths and FIFO fill-levels
- Provides Complete network Ethernet statistics (optional) with RMON-MIB counters (RFCs 2819 and 2665)

1.5 Configurations

The 25GMAC can be configured to any of the following major architectures using coreConsultant:

- **XGMAC-AXI**: Subsystem with DMA with AXI interface
- **XGMAC-MTL**: Subsystem with transaction (FIFO) layer only (without DMA) and with native FIFO interface
- **XGMAC-CORE**: MAC only with native FIFO interface (without DMA, FIFO layer, and AXI)

1.5.1 Configurability

Using coreConsultant, you can configure the following features to optimize the area and timing of your design:

- Number of Tx queues or DMA channels
- Layer2 Network Virtualization over L3 Network (VxLAN/NVGRE) offload
- Number of TSO engines with or without local memory storage
- Number of Rx queues in Receive FIFO
- Audio Video (AV) and Data Center Bridging (DCB) traffic queuing and forwarding support
- Number of RxDMA channels
- 32-bit or 64-bit addressing support on AXI interface.
- Size of receive memory in Transaction layer
- Size of transmit memory in Transaction layer
- Synchronous 2-port RAM memory for transaction layer
- 64-bit or 128-bit application interface data bus width;
- Hash address filters
- VLAN Tag based perfect and hash filtering in Rx
- SA and VLAN insertion in Tx
- Configurable number (up to 32) of MAC address registers for filtering
- RMON counters (MMC) with individual register selection
- Power Management Module (PMT) with magic packet and remote wake-up frame processing options
- Checksum Offload Engine (TCP/IP layer) for the Transmit and Receive paths.
- IEEE 1588 timestamping and PTP support
- Hash based RSS and out-of-sequence packet detection
- Endianness for all application-side data paths
- Energy Efficient Ethernet
- Station Management Agent (SMA) MDIO module
- Include GMII when support for 1-gigabit or 2.5-gigabit operation is required

- XGMII interface provides three data rate/width options:
  - Double Data Width (DDW): 64-bit data operating with a single 156.25-MHz clock edge
  - Single Data Rate (SDR): 32-bit data operating with single 312.5-MHz clock edge
- Transmit Packet Status in XGMAC-MTL configuration
  - Maximum number of packets in Transmit FIFO (2, 4, 8)
- (Optional) forwarding of transmit packet status to ATI interface
- Transmit Time stamping FIFO depth (1, 2, 4, 8, 16)
- Support for Priority Flow Control feature
- RGMII PHY interface support
- RMII PHY interface support
- AVB feature selection for Rx only

2 Architecture

This chapter describes the interfaces and functionality of 25GMAC.

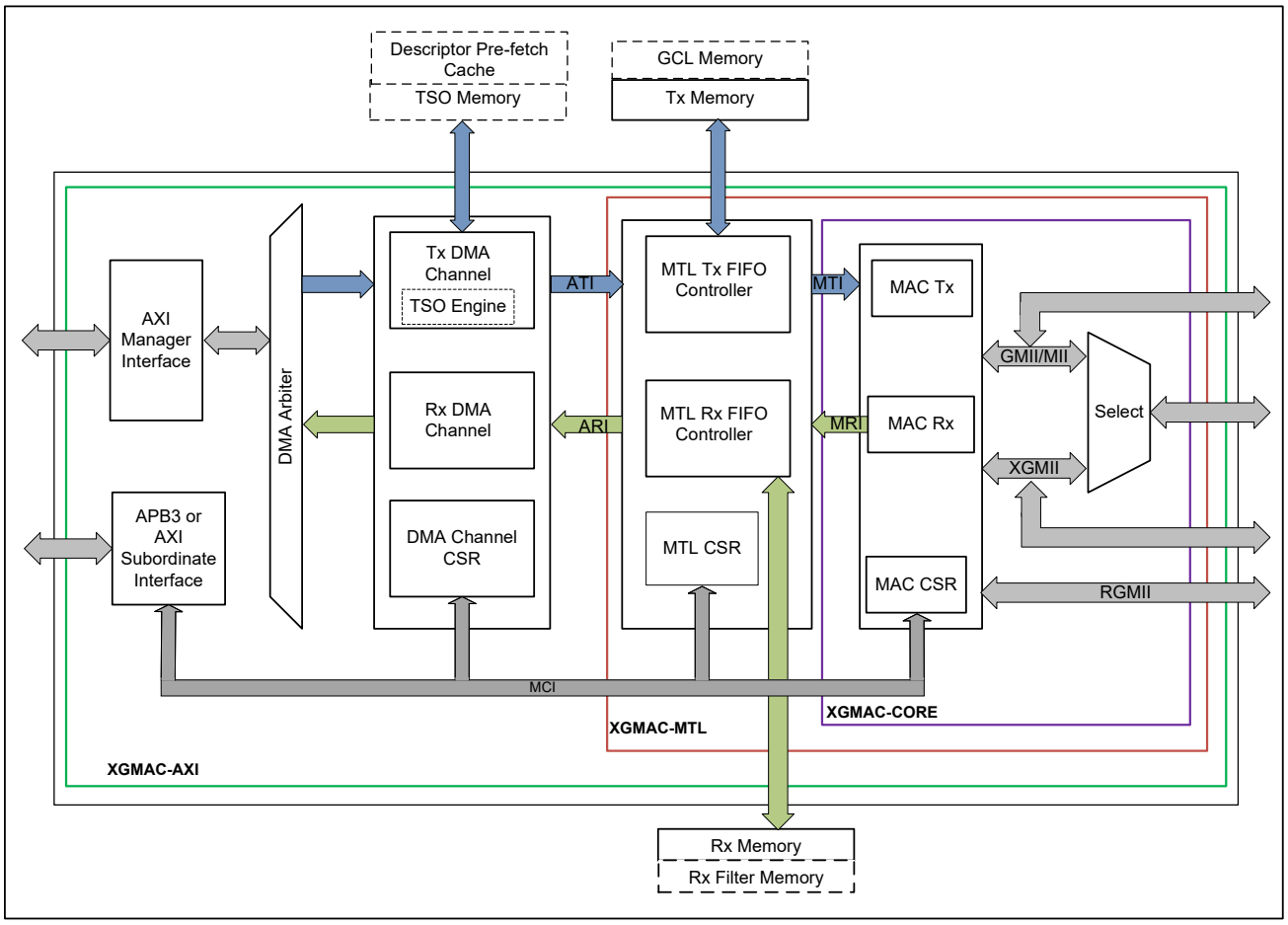
This chapter contains the following sections:

- "Introduction" on page 44
- "CSR Bus Interface" on page 46
- "Application Manager Interface" on page 51
- "DMA Controller" on page 62
- "MAC Transaction Layer (MTL)" on page 91
- "The MAC Layer " on page 120
- "Interrupt Hierarchy in the MAC " on page 144

2.1 Introduction

25GMAC is a configurable product with multiple interfaces towards the system side or the PHY side. Major modules and functions can be selected as required for your design, giving you an area-optimized design. Figure 2-1 provides a block diagram of major system configurations of 25GMAC.

Figure 2-1 25GMAC Configurations



- **XGMAC-CORE**

  Using CSR, software can configure the XGMAC-CORE to operate at 10/5/2.5G(XGMII), 1/2.5G (GMII), 100M/10M (MII and RMII) or 1G/100M/10M (RGMII) speeds. The XGMAC-CORE contains the MAC Transmitter and Receiver functions specified in the IEEE 802.3-2018/RMII/RGMII standard. It interfaces to the line side with the XGMII/GMII/MII/RGMII/RMII and with a native interface to the system side. These system-side interfaces, called the MAC Transmit Interface (MTI) and the MAC Receive Interface (MRI) are synchronous, respectively, to the XGMII/GMII/MII/RGMII/RMII Transmit and XGMII/GMII/MII/RGMII/RMII Receive clocks. In addition to these interfaces, the application can access the Control and Status Registers (CSR) through a MAC Control Interface (MCI) or APB3.

- **XGMAC-MTL**

The XGMAC-MTL configuration consists of the XGMAC-CORE and the MAC Transaction Layer (MTL). This layer introduces a FIFO buffer into the transmit and receive paths. It interfaces to the system with native interfaces called the Application Transmit Interface (ATI) and Application Receive Interface (ARI). These interfaces are synchronous to the system clock.

- **XGMAC-AXI**

  The XGMAC-AXI configuration consists of the XGMAC-CORE, MTL, DMA engines and the AXI interface.

2.2 CSR Bus Interface

The CSR bus interface provides access to the CSR space. You can select a register bus interface by using the CSR Interface option in coreConsultant. The 25GMAC can have one of the following register bus interfaces:

- "AXI Subordinate Interface" on page 46
- "APB Manager Interface" on page 48
- "MAC Control Interface" on page 49

2.2.1 AXI Subordinate Interface

The AXI subordinate is supported in the XGMAC-AXI configurations. The features supported for the AXI subordinate interface are listed in "AXI Subordinate Interface" on page 38. The simplified AXI4-LITE interface which has a reduced set of interface signals and no burst capabilities can also be selected.

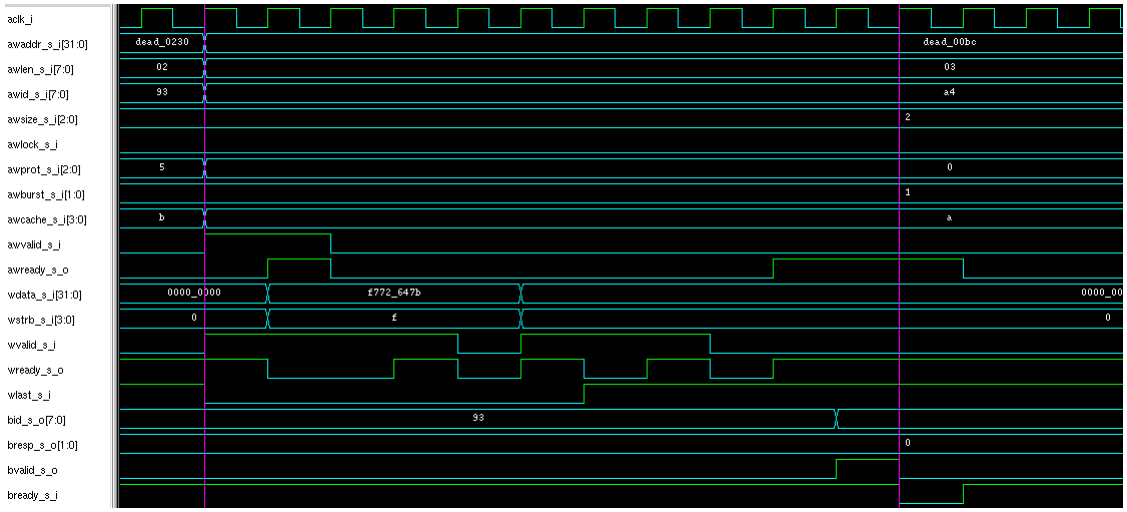
The subordinate interface does not support the following features:

- Data reordering
- Data interleaving
- WRAP burst transfers
- Signals awcache/awprot or arcache/arprot
- Atomic accesses
- Exclusive Access

Note DWC_25gmac supports SLVERR and OKAY response on bresp_s_o and rresp_s_o signals. DWC_25gmac does not respond with EXOKAY and DECERR responses.

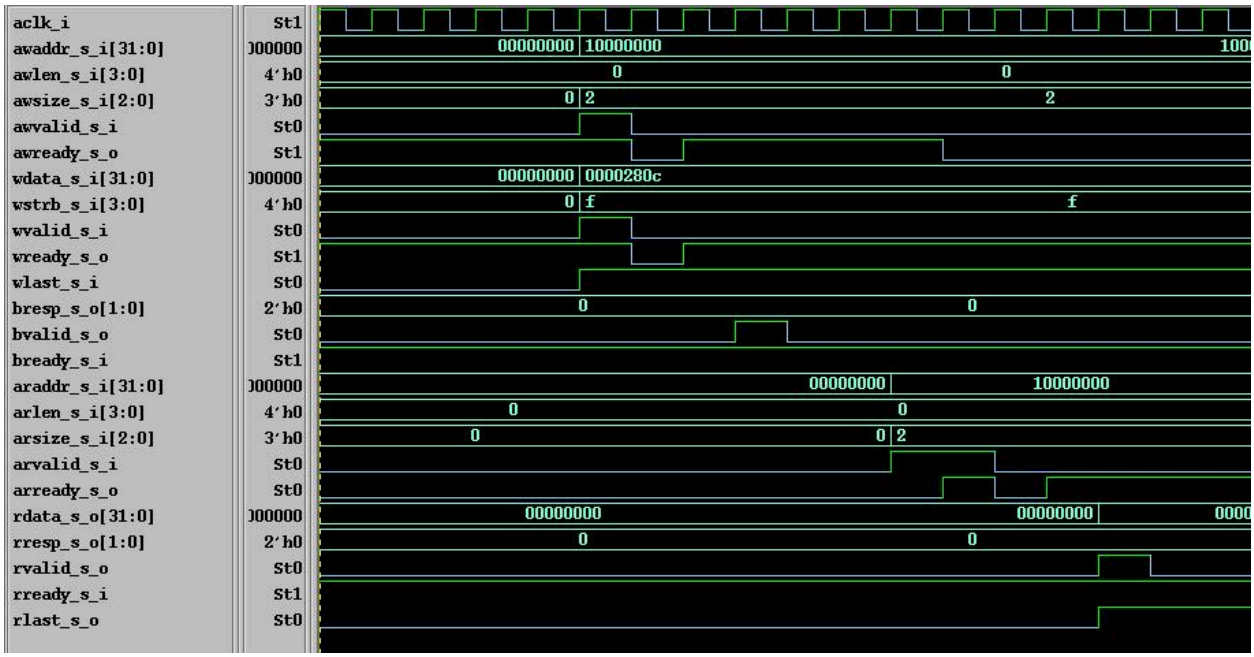
Figure 2-2 shows the timing on the AXI subordinate interface for a burst-write transfer.

Figure 2-2 Write Channel Timing of 32-Bit AXI Subordinate Interface



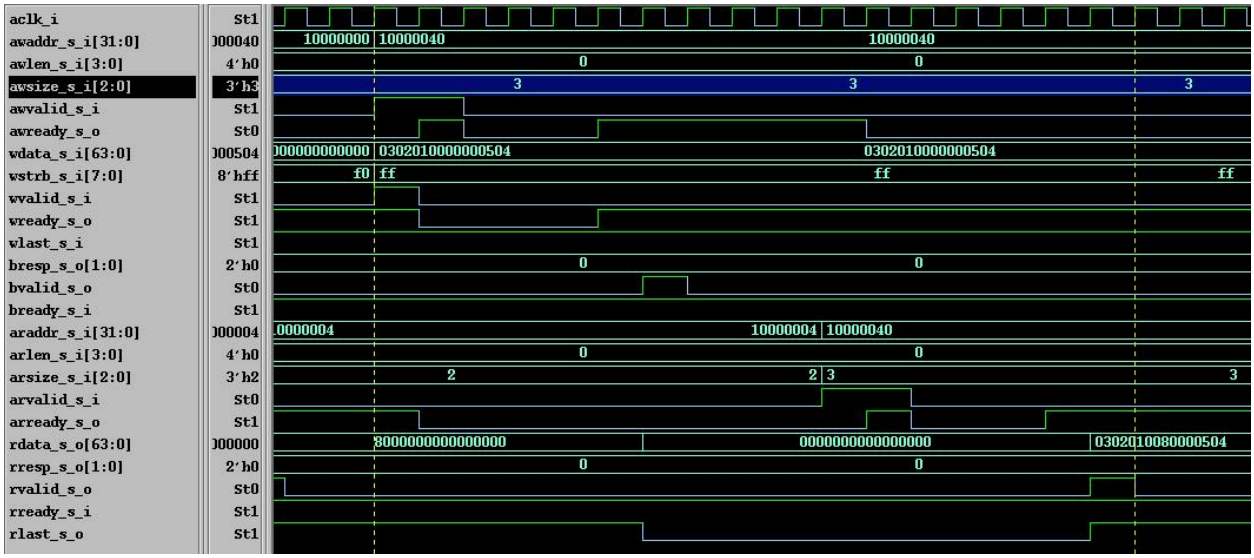
The AXI subordinate interface receives a write burst of length four, which gets accepted in seven clocks. The respective write response takes additional three clocks. Similarly, a single write transfer gets completed in four clocks, as shown in Figure 2-3.

Figure 2-3 Single Write and Read Transfer



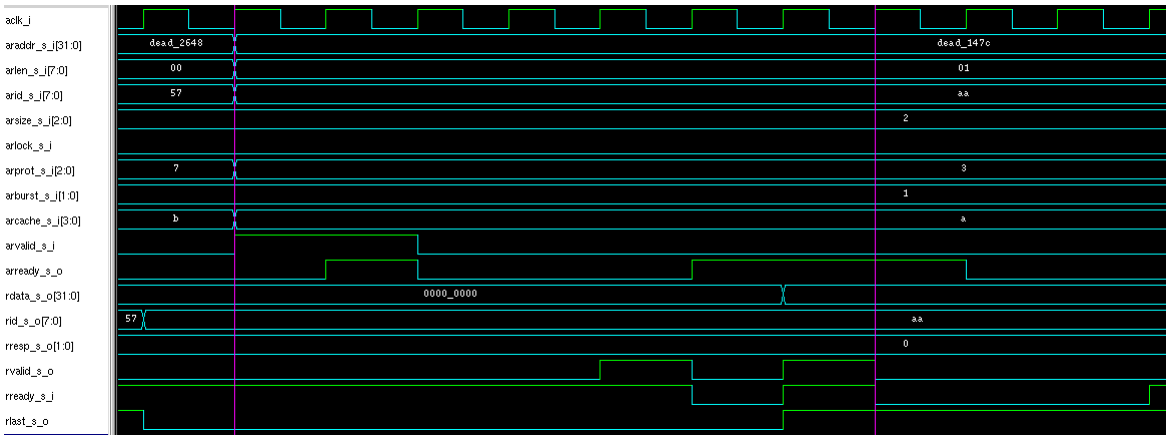
The AXI subordinate interface supports all types of byte, half-word, and word burst-size transfers. The latencies may vary for a 64-bit AXI subordinate interface, depending on the address offset and data bus width as shown in Figure 2-4.

Figure 2-4 Latencies for a 64-bit AXI Subordinate Interface



In Figure 2-5, the AXI subordinate receives a read request of burst length 2. For the first data to be output on the AXI bus, the latency is four clocks. The remaining beats are output two clocks after the AXI manager accepts the previous beat. In the AXI-read channel, the latencies depend on the burst length, data bus width, and the address offset.

Figure 2-5 AXI Subordinate Receives a Read Request of Burst Length 2



Note

- The XGMAC-AXI configuration supports the little-endian and byte-invariant big-endian modes on the AXI subordinate ports.
- There is an additional delay in accessing DMA CSR Space registers when "Use Different Clock for CSR" feature is selected, as the registers are implemented in the Application or AXI clock domain.

2.2.2 APB Manager Interface

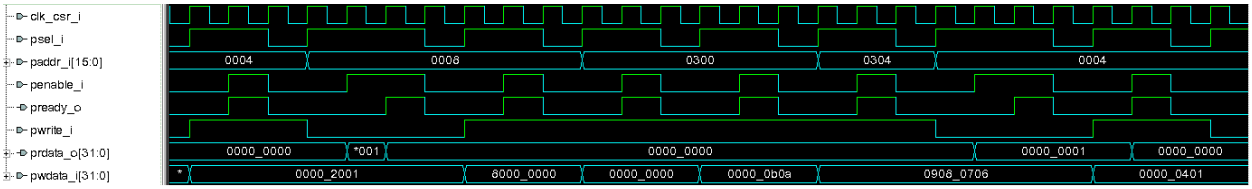
The APB3 or APB4 manager interface is supported in all configurations. You can select your choice of APB port to access the DMA, MTL, and MAC CSR space. Only 32-bit transfers are supported on this APB manager interface. All register accesses are completed in less than five clocks.

2.2.2.1 APB3 Port Timing

Figure 2-6 shows a register write access followed by a register read access on the APB3 port. The APB3 port signals behave as per the AMBA3 APB Protocol specification. Each write transfer completes in 2 clock cycles whereas each read transfer completes in 3 or 4 clock cycles.

There is an additional delay in accessing DMA CSR Space registers when "Use Different Clock for CSR" feature is selected, as the registers are implemented in the Application or AXI clock domain.

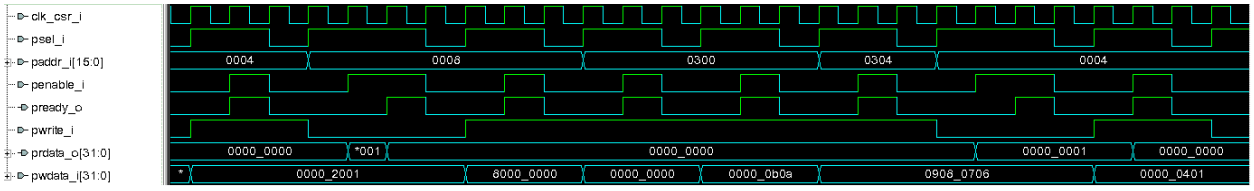
Figure 2-6 APB3 Port Timing



2.2.2.2 APB4 Port Timing

Figure 2-7 shows a register write access followed by a register read access on the APB4 port. The APB port signals behave as per the AMBA4 APB Protocol specification. Each write transfer completes in 2 clock cycles whereas each read transfer completes in 3 clock cycles. The `pstrb_i` signal enables the byte to be updated during WRITE transactions. The `pprot_i` has no impact and is ignored by 25GMAC.

Figure 2-7 APB4 Port Timing



2.2.3 MAC Control Interface

The MAC Control Interface (MCI) is supported in non-XGMAC-AXI configurations.

This interface connects the application with the Control and Status register (CSR) module of the Media Access Controller to provide the control path for programming the MAC registers. The application initiates a register write or register read by asserting the command valid signal `mci_val_i` (with the valid data `mci_wdata_i` during write operations and valid byte enables `mci_be_i`), along with the register address on `mci_addr_i`. The `mci_rdwn_i` control signal indicates a Write (1'b0) or Read (1'b1) operation. The application assumes a successful data transfer when it receives an acknowledgement (`mci_ack_o`) asserted for one clock cycle. The CSR module drives the Read data on `mci_rdata_o`. The application can de-assert the command valid after receiving the acknowledgement.

If the application continues to assert the command valid signal (on receiving an acknowledgement), the MAC considers it as the next access . The MAC completes the access to reserved registers with placeholder read or write operations. This means that the write operations to the reserved addresses have no effect and the read operations are completed with an all-zero data output on the `mci_rdata_o` bus.

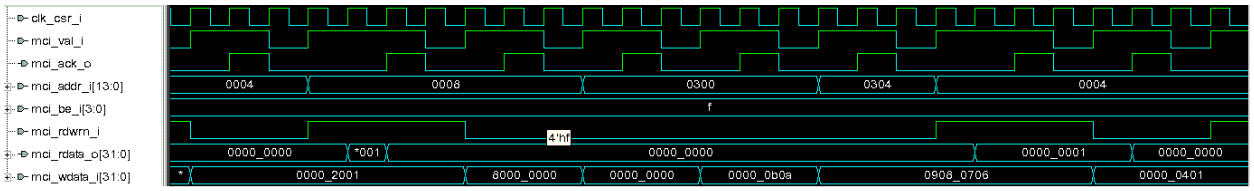
All write accesses to the CSR complete in two clock cycles, that is, the `mac_ack_o` signal is asserted after a delay of one clock cycle after the `mci_val_i` signal goes high. The `mci_rdata_o` signal is registered. Therefore, there is an additional delay of one clock in assertion of the `mci_ack_o` signal. Therefore, each read access is completed in 3 clock cycles for such configurations.

Note For the correct access, the input control signals must be valid at the same time.

2.2.3.1 MCI Port Timing

Figure 2-8 shows read and write accesses on the MCI port.

Figure 2-8 MCI Port Timing



The flow shown in Figure 2-8 is as follows:

1. The first access to address 0x0004 is a single write cycle which completes in two clock cycles. The `mci_val_i` signal and the corresponding address and control signals must be kept asserted for the whole duration of the transaction.
2. The second and third accesses are single read and write operation transactions to address 0x0008, respectively. The `mci_ack_o` signal is asserted two clocks after the `mci_val_i` signal is asserted for a read operation. The write operations are completed in two clock cycles.
3. The following accesses are burst write and burst read operations starting from address 0x0300.

   Each individual write or read transaction in the burst is identical to the single write or read transactions with respect to timing.

2.3 Application Manager Interface

The DMA Controller supports following interfaces to the application:

- "AXI4 Manager Interface" on page 51
- "AXI Low-Power Interface" on page 58

2.3.1 AXI4 Manager Interface

In the XGMAC-AXI configuration, the DMA controller interfaces with the application through the AMBA AXI4 interface (referred to as AXI Interface in remainder of this document). The AMBA AXI bus interface provides the characteristics to support highly-effective data traffic throughput. The system bus utilization is maximized by allowing simultaneous read and write transfers initiated from different DMA channels.

The AXI manager interface allows multiple numbers of burst requests to be queued for Read or Write operation. This gives the AXI manager the flexibility to dynamically and independently select multiple subordinates for handling Priority Requests and Slow peripheral subordinates. The XGMAC-AXI configuration also supports the low-power interface defined in the AXI specification.

This section covers the following topics:

- "Burst Splitting and Burst Selection" on page 51
- "INCR Burst Type" on page 52
- "INCR_ALIGNED Burst Type" on page 52
- "Outstanding Transactions" on page 52
- "Priority of AXI Requests" on page 53
- "Bursts Reordering and Data Interleaving" on page 53
- "AXI-ID Translation" on page 54
- "Endianness Support" on page 55
- "Posted Writes" on page 55
- "Error Response Handling" on page 55
- "AXI Memory Read" on page 56
- "AXI Memory Write" on page 56
- "AXI Early Burst Termination" on page 57

2.3.1.1 Burst Splitting and Burst Selection

The AXI manager splits the DMA requests into multiple bursts on the AXI system bus. Splitting is based on the DMA transfer count, the software-controllable burst enable bits (UBL, BLEN256, BLEN128, BLEN64, BLEN32, BLEN16, BLEN8, BLEN4), and the software-controllable burst types (INCR and INCR_ALIGNED). For more information, see `DMA_SysBus_Mode` register, in the *Register Descriptions* chapter.

The AXI manager also handles splitting the transfers that cross 4 KB address boundary, with an option to limit it to 1-KB address boundary. The Burst Length Select Priority is in the following sequence:

UBL, BLEN256, BLEN128, BLEN64, BLEN32, BLEN16, BLEN8, and BLEN4.

2.3.1.2 INCR Burst Type

If UBL bit is set, the AXI manager always chooses the maximum allowed burst length based on the BLEN256, BLEN128, BLEN64, and BLEN32 bits (possible maximum burst lengths are 256, 128, 64, 32, or 16 by default). The AXI manager chooses a burst length of any value less than the maximum enabled burst length (all lesser burst length enables are redundant) in the following cases:

- The DMA requests are not multiples of the maximum allowed burst length
- The 4K/1K address boundary crossing condition is enabled

For example, when only BLEN64, BLEN16, and BLEN4 are enabled, and the DMA requests a burst transfer of 102 beats size, the AXI manager splits it into two bursts of 64 and 38 beats size, respectively.

If UBL bit is set to zero, the burst length is based on the priority of the enabled bits in the following order: BLEN256, BLEN128, BLEN64, BLEN32, BLEN16, BLEN8, and BLEN4. When the DMA requests to transfer a burst, the AXI interface splits the requested bursts into multiple transfers by using only the enabled burst lengths. This splitting can occur in following cases:

- The remaining DMA transfer count is not a multiple of the maximum enabled burst
- The current start address and max allowed burst length is crossing the 4K/1K address boundary

If the AXI interface cannot choose any of the enabled burst lengths, it selects the burst length as 1. For example, if only BLEN64, BLEN16, and BLEN4 are enabled and the DMA requests a burst transfer of size 102 beats, the AXI interface splits it into multiple bursts of size 64, 16, 16, 4, 1, and 1 beats, respectively. The sequence is in decreasing burst lengths.

2.3.1.3 INCR_ALIGNED Burst Type

When the address-aligned burst type is enabled, then in addition to the burst splitting conditions explained in "INCR Burst Type", the AXI interface splits the DMA requested bursts such that each burst length is aligned to the least significant bits of the start address. The AXI interface initially generates smaller bursts so that the remaining transfers can be transferred with the maximum possible (enabled) fixed burst lengths. For example, in the same setting as explained earlier for UBL (BLEN64, BLEN16 and BLEN4 are enabled), the DMA requests a burst of size 102 beats at the start address of 0x0000_03A0 (128-bit bus). The AXI manager starts the first transfer with size 6 such that the address of the next burst is aligned (0x0000_0400) for a burst of 64. Therefore, the sequence of bursts is 6, 64 and 32, respectively.

When UBL is not set, the sequence of burst transfers is 1, 1, 4, 64, 16, and 16 in same scenario, respectively. The sequence of smaller bursts at the beginning aligns the address to the next higher enabled burst lengths programmed in the register.

2.3.1.4 Outstanding Transactions

The AXI manager supports up to 8, 16, 32, 64. 128, or 256 (configurable in coreConsultant) outstanding Read or Write requests on the AXI bus. You can also control these outstanding requests through software by programming Bits[29:24] and Bits[21:16] in the `DMA_SysBus_Mode` register. This is a useful workaround for any system-level issues with handling of multiple requests.

For more information, see `DMA_SysBus_Mode` register, in the Register Descriptions chapter.

DWCXG_MAX_RDBREQ and DWCXG_MAX_WRBREQ are the maximum number of outstanding data read/write requests that can be issued from each DMA data transmission module, without waiting for response. The configurable values supported are 2, 4, 8, 16, 32, and 64.

25GMAC supports programmable control to limit the maximum number of outstanding requests to value lesser than the defined DWCXG_MAX_RDBREQ and DWCXG_MAX_WRBREQ for read/write outstanding requests respectively.

When the register fields. ORRQ and OWRQ – Outstanding Read/Write Requests respectively of the `DMA_CH_RX_Control2(#i)` and `DMA_CH_TX_Control2(#i)` register is programmed to a non-zero value, processing of outstanding requests per DMA channel stops when outstanding requests are equal to the programmed value.

2.3.1.5 Priority of AXI Requests

The descriptor transfers have higher priority than the data transfers. Therefore, if there are two requests, Rx Descriptor Read and Tx Data Read, the Rx Descriptor Read is given higher priority so that the next Rx Data Write (subsequent to Rx Descriptor Read) need not wait for the Tx Data Read transfer to complete. If there are descriptor read requests from both DMAs, these requests are serviced based on the first-come first-serve basis. The RxDMA has higher priority if descriptor read requests, generated from both DMAs, are in the same clock. Similarly, in the write channel, descriptor writes from any DMA have higher priority than the data-write transfers for the RxDMA.

In the configurations where multiple DMA Channels are present, if the requests are generated from different channels simultaneously in the same clock, the request priority among the channels is: Channel 15 (if present), Channel 14,..., Channel 1, and Channel 0.

Note

The maximum outstanding requests on the AXI is limited to `DWCXG_AXI_GM_MAX_RD_REQUESTS` and `DWCXG_AXI_GM_MAX_WR_REQUESTS` parameters corresponding to Read and Write channels respectively.

You can limit the maximum outstanding requests on the AXI by programming the `RD_OSR_LMT` and `WR_OSR_LMT` fields of the `DMA_SysBus_Mode` register.

Out of the maximum outstanding requests, one of the requests is always reserved for Descriptor operation (Descriptors Fetch on Read Channel and Descriptors write-back on the Write channel). This ensures that high priority Descriptors operation is not blocked by the Packet Buffer transfers.

2.3.1.6 Bursts Reordering and Data Interleaving

The AXI protocol allows reordering of data transfers with different AXI-IDs with respect to the sequence of requests. It also allows data interleaving between transfers of different AXI-IDs for Read channels.

The AXI manager supports reordering and interleaving from the AXI subordinates on the Read channel. TxDMA uses unique ARID for each Tx channel. As each channel has dedicated Tx FIFO, the response data of each Tx channel (as indicated by RID) gets routed to the corresponding Tx FIFO, to ensure re-ordering responses across IDs are handled appropriately.

The RxDMA also uses unique AWID for each Channel for Data Write and Descriptor write-back and the order of Data write and Descriptor write-back on the same channel is always sequential. The XGMAC-AXI configuration does not support data re-ordering write channel which is compliant with AXI-4 specification.

The 25GMAC uses dedicated ARID(0x0) for Descriptor fetch for both Tx and Rx channels and read response re-ordering is not expected during descriptor fetch. Also, 25GMAC uses dedicated AWID (0x0) for Tx Descriptor write-back and 25GMAC.

> **Note** The XGMAC-AXI manager does not support data reordering and interleaving on the Write channel which is compliant with the AXI 4 specification.

2.3.1.7 AXI-ID Translation

The XGMAC-AXI selects the IDs as follows:

- AXI Read Channel ID usage
  a. For TxDMA and RxDMA Descriptor Fetches, the AXI Read ID = 0.
  b. For TxDMA Channel Data Transfers, the AXI Read ID = Tx_DMA_Channel_ID_Number + 1. For example the Tx_DMA_Channel0 uses ID=1, Tx_DMA_Channel1 uses ID=2 and Tx_DMC_Channel2 uses ID=3 and so on.
- AXI Write Channel ID usage
  a. For TxDMA Descriptor write-back, the AXI Write ID = 0.
  b. For RxDMA Channel Data transfers as well as for Rx Channel Descriptor write-back, the AXI Write ID = Rx_DMA_Channel_ID_Number + 1. For example the RxDMA_Channel0 uses ID=1, RxDMA_Channel1 uses ID=2 and Rx_DMC_Channel2 uses ID=3 and so on.

> **Note** The AXI system bus ID width is configurable. The default width for the AXI manager is 4 bits. The unused bits are hardwired to zeros. The default ID width for the AXI subordinate interface is 8 bits.
>
> The AXI system bus ID width selected must be at the least Log2 (1 + higher of the number of TxDMA channels and number of RxDMA channels, rounded to the nearest power of 2 value on the higher side).
>
> For example, if the number of TxDMA channels and number of RxDMA channels selected are 3 and 3 respectively, AXI system bus ID width selected must be at least 2 (i.e. Log2 (1+3=4).

2.3.1.8 Unique AXI ID for Descriptor Fetches

The DMA uses AXI ID=0 for all the Transmit and Receive Descriptor fetch. When you enable Unique ID for Descriptor Access by setting the DWCXG_UAXIID parameter, unique AXI IDs per Transmit and Receive DMA are used for Descriptor fetch. The minimum AXI ID width required is increased by 2.

Table 2-1 shows Unique AXI ID used for the AXI transactions:

Table 2-1 AXI ID for Descriptor Fetches

AXI Signal	Transmit/Receive	Type	AXI ID as per Translation	Unique AXI ID
AXI Read Channel IDs(arid_m_o, rid_m_i)	Transmit	Descriptor Read	0x00000	2'b10_"Tx_DMA_No.+1"
	Receive	Descriptor Read	0x00000	2'b11_"Rx_DMA_ No.+1"
	Transmit	Data Read	Tx_DMA_NUM+1	2'b00_"Tx_DMA_No.+1"

Table 2-1 AXI ID for Descriptor Fetches (continued)

AXI Signal	Transmit/Receive	Type	AXI ID as per Translation	Unique AXI ID
AXI Write/Response Channel IDs (awid_m_o, wid_m_o, bid_m_i)	Transmit	Descriptor Write	0x00000	2'b01_"Tx_DMA_No.+1"
	Receive	Descriptor Write	Rx_DMA_NUM+1	2'b00_"Rx_DMA_ No.+1"
	Receive	Data Write	Rx_DMA_NUM+1	2'b00_"Rx_DMA_ No.+1"

2.3.1.9 Endianness Support

The AXI manager supports the little-endian and byte-invariant big-endian modes on the AXI manager ports. The AXI manager does not support narrow transfers and always performs full data-width transfers.

2.3.1.10 Posted Writes

In posted writes, the write channel of the AXI manager transfers an OKAY response to the DMA as soon as the last cycle or beat of data is accepted by the AXI interconnect. The AXI manager generates this response without waiting for the response from the target AXI subordinate on the write response channel. In non-posted writes, the AXI manager interface transfers the response received from the AXI subordinate channel to the DMA.

For transferring the Ethernet data to system memory, the RxDMA always issues posted write requests to the AXI manager interface for both Data and Descriptor write-back. The AXI manager does not support out-of-order write transfers. However, the RxDMA ensures that the sequence is maintained for descriptor writes and data transfers.

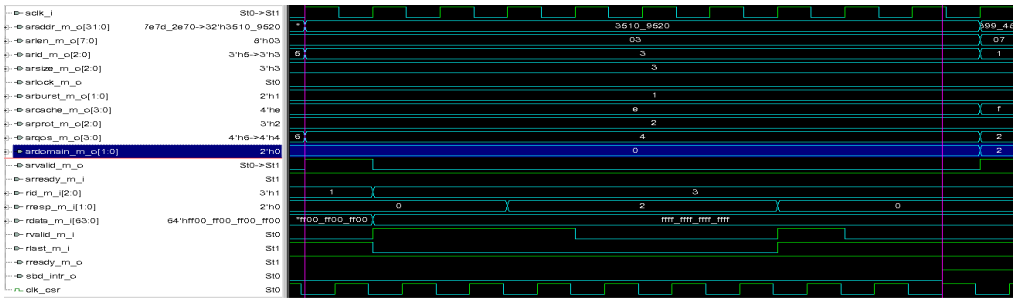
For Tx writing descriptors (status or timestamp), the DMA issues the non-posted write requests by default. This is needed because the Transfer Complete Interrupt is generated based on the descriptor writes for which the DMA needs completion response from the memory subordinate. This ensures no race condition between 25GMAC and the software because the interrupt is generated only after the data and descriptor are written to the subordinate memory.

Note For data transfer to system memory, the RxDMA always issues posted writes. So, after the data is accepted by AXI interconnect, DMA moves out of buffer data write state to descriptor write state without waiting for the response from the target AXI subordinate, on the write response channel. Therefore, after the ERROR response for the previous buffer write is received, DMA initiates a Descriptor Write and captures ERROR status (received during Descriptor Write) as bus error, into the CSR space.

2.3.1.11 Error Response Handling

Whenever there is an error response from the AXI subordinate (as shown in Figure 2-9 with rresp_m_i[1:0] = 2), the respective DMA channel which generated the request gets disabled. The XGMAC-AXI manager asserts the interrupt (sbd_intr_o in Figure 2-9) when the corresponding “Fatal Bus Error” interrupt is enabled. The traffic on the other channels does not get affected if an Error is detected for a DMA Channel.

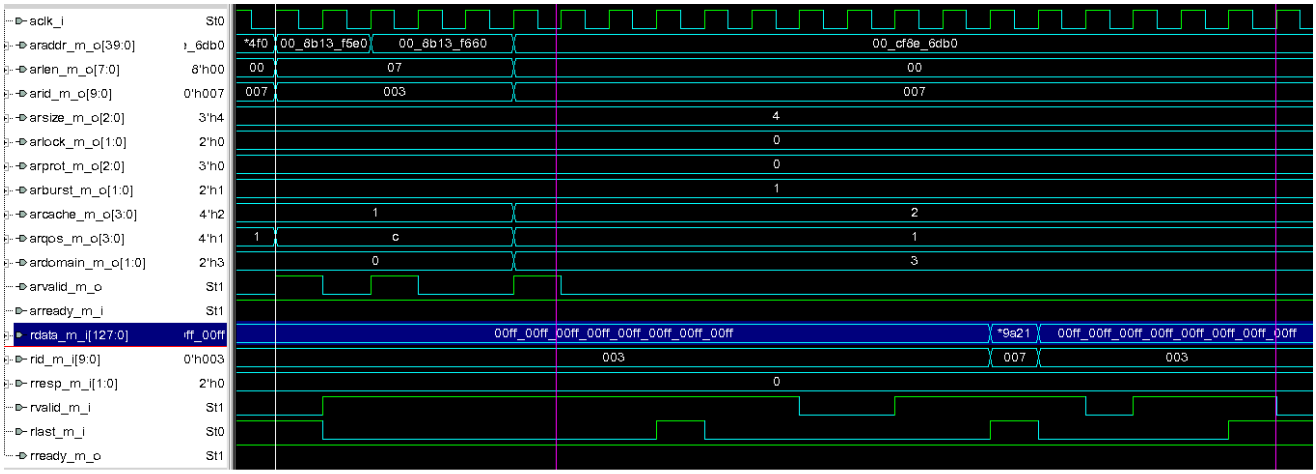
Figure 2-9 Error Response Timing



2.3.1.12 AXI Memory Read

Figure 2-10 shows the AXI manager read timings. Two requests are placed for ID 3 and one request for ID 7. The Read data is interleaved by the interconnect (by way of ID 3 and ID 7) and received by the AXI manager. The TxDMA in AXI manager always ensures that it has enough space in its FIFO to accept the requested burst of data. Therefore, the `r_ready_m_o` response is high except for some exceptions like when the Tx FIFO controller is not ready because it is writing internally generated data (for example, checksums) to the Tx FIFO.

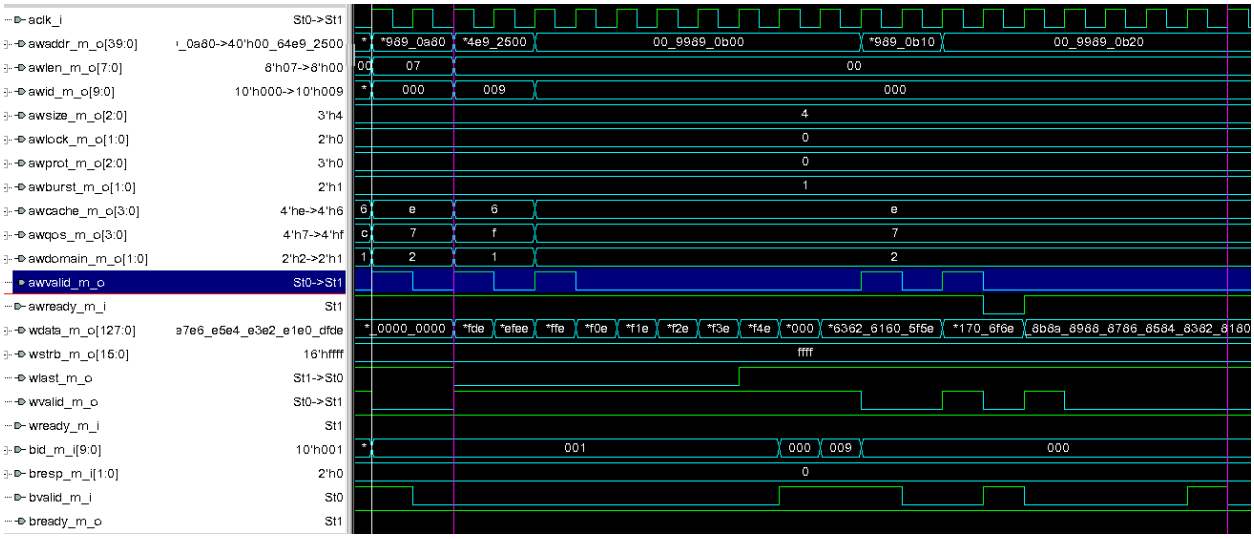
Figure 2-10 AXI Memory Read Timing



2.3.1.13 AXI Memory Write

Figure 2-11 shows the AXI write interface diagram. There are five requests: the first one is for ID0; second one for ID9; and remaining three for ID0. These requests are issued with burst lengths of 8 for the first one and 1 for all others. The latency between the first write request and the corresponding first data is two clocks. The RxDMA ensures that it always has the requested burst of data available in its FIFO before the request is generated. Therefore, there are no further latencies or delays in the data transfer provided there is no delay in the acceptance of the data by the AXI subordinate, as shown in Figure 2-11.

Figure 2-11 AXI Memory Write Timing



2.3.1.14 AXI Early Burst Termination

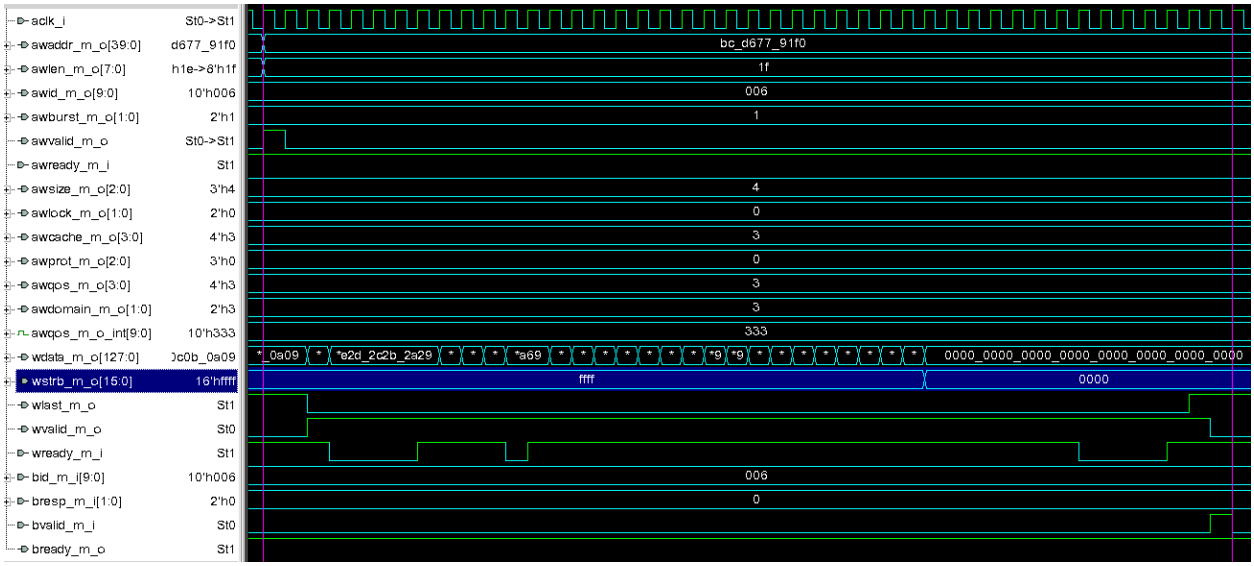
Figure 2-12 shows the early burst termination on Write channel. This can happen during data transfer when the EOP is read out of the Rx queue before the programmed burst transfer is completed when Rx queue operates in cut-through (threshold) mode. A request is issued, but the EOP gets transferred in the middle of this burst. The AXI manager still continues the data phases of the outstanding requested bursts but does not allow further data to be written to the subordinate memory by de-asserting the write-strobes (as indicated by wstrb_m_o[3:0] = 0).

Note

- When the Rx Queues are programmed in Threshold mode, after transferring the EOP data to the system memory, a lot of AXI bandwidth may be wasted because of test writes. Therefore, it is recommended to have two or less than two outstanding requests on the AXI manager write channel by appropriately programming DMA_SysBus_Mode.
- When Rx Queues are programmed in Store and Forward mode, the Burst requests issued on the AXI are adjusted based on the Packet Length such that there is no bandwidth wastage and no placeholder writes.

The following AXI Early Burst Termination diagram is applicable only when the Rx queues are programmed in Threshold Mode.

Figure 2-12 AXI Early Burst Termination Timing



2.3.2 AXI Low-Power Interface

The XGMAC-AXI configuration supports the low-power interface defined by the AXI specification. Bits[15:14] of `DMA_SysBus_Mode` register controls the behavior of the AXI manager in the low-power mode. When Bit 15 (EN_LPI) is enabled and the application requests a low-power initiation, the AXI manager checks the Idle status of the data paths before giving the response. For more information, see `DMA_SysBus_Mode` register, in the Register Descriptions chapter.

This means that the AXI manager accepts the low-power request and goes into the low-power mode in which the application can remove the AXI clock when the following conditions are true:

- The MAC transmitter and receiver are in the Idle mode.
- Both DMA engines are in Idle mode.
- There is no data in the Transmit or Receive data path or queue for a continuous period of 256 clock-cycles (`aclk_i`).

If the MAC goes to an inactive state within 255 clocks after the request, the AXI manager waits for confirmation of the inactive state for a further period of 256 clocks before giving the response. If some activity is found in the MAC during this period of 256 clock cycles, the AXI manager responds by denying the low-power request.

In the low-power mode, the AXI manager can initiate a request to come out of the low-power mode in the following scenario (depending on Bit 14 of `DMA_SysBus_Mode`):

- The MAC receives any packet
- The MAC receives a magic packet or remote wake-up packet

This request signal assertion (`cactive_o`) is synchronous to the `clk_rx_312pt5_i` clock in such cases because the `aclk_i` clock is assumed to be absent.

Note The timing and behavior of the XGMAC-AXI low-power interface signals is according to the Figure 12-2 of the AMBA AXI Specification.

2.3.2.1 Automatic AXI Low-Power Entry

In addition to the system initiated AXI low-power entry (where 25GMAC enters into low-power upon request from system controller), 25GMAC also supports AXI low-power when there is no activity for a configured interval of time. This feature makes the 25GMAC to automatically (i.e. without any request from system controller) enter into AXI low power state once the condition is reached.

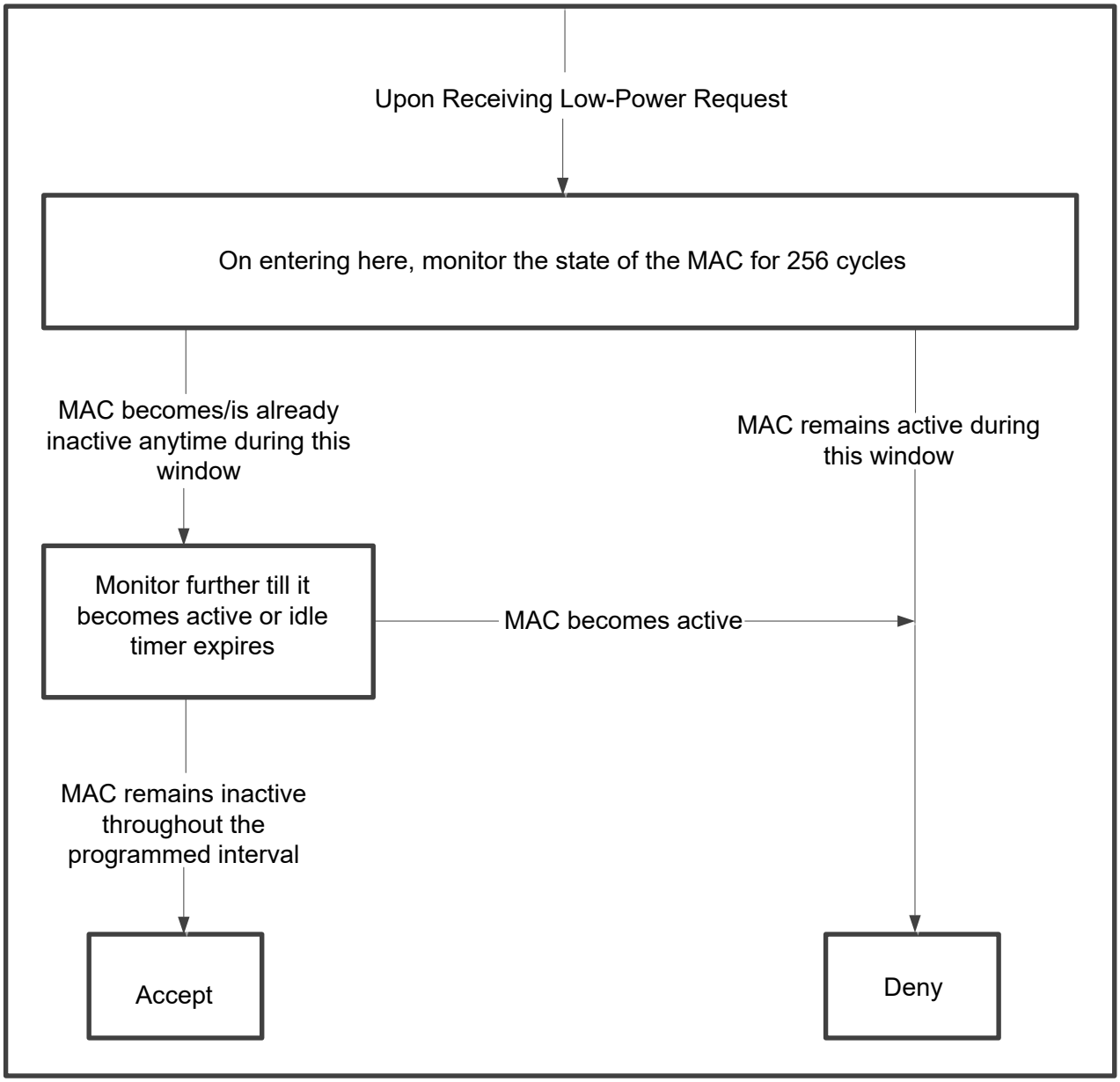
A timer (idle timer) is added to detect the MAC remaining idle continuously for the interval programmed in the `AXI_LPI_Entry_Interval` register. The timer is loaded every time the MAC becomes active and decrements when MAC becomes idle. The AALE bit in the `DMA_SysBus_Mode` Register when set enables the Automatic AXI LPI Entry.

On receiving a AXI low-power request, if the AXI manager sees that the MAC is already in inactive state, it waits till the idle timer expires and accepts the low power request. If the idle timer has already expired, it immediately accepts the Low Power Request. At any point during the idle timer window if it becomes active, the AXI manager denies the request.

The AXI manager accepts the low-power request and goes into the low-power mode in which the application can remove the AXI clock when the following conditions are true. These conditions are applicable for both the 256 clock cycle period and the idle timer value programmed in the AXI LPI Entry Interval Register.

- The MAC transmitter and receiver are in the idle mode
- Both DMA engines are in idle mode
- Subordinate interface is idle

Figure 2-13 AXI Low-Power Entry Flow



2.3.2.2 Coming out of Low-Power Mode

In the low-power mode without the AALE field set, the AXI manager can initiate a request to come out of the low-power mode in the following scenario (depending on Bit 14 of `DMA_SysBus_Mode`):

- The MAC receives any packet
- The MAC receives a magic packet or remote wake-up packet.

This request signal assertion (`cactive_o`) is synchronous to the `clk_rx_312pt5_i` clock in such cases because the `aclk_i` clock is assumed to be absent.

If the AALE bit is set then the AXI manager can come out of low power mode when one of the following occurs:

- The MAC receives any packet or magic/remote wake-up packet based on Bit 30 of `DMA_SysBus_Mode`
- Exit initiated by the system clock controller
- Activity in the register bus interface for register read/write access

  This is applicable only when DWCXG_CSR_SLV_CLK is selected.

This request signal assertion (`cactive_o`) is synchronous to the `clk_rx_312pt5_i`, `clk_csr_i` and `aclk_i` based on the type of exit.

Note

In case of exit due to csysreq_i assertion, the clock controller must enable the clock immediately along with csysreq_i. The cactive_o and csysack_o is synchronous to this clock.

If the system controlled LPI Exit is not used, the aclk_i can be asserted when cactive_o goes high. This might impact register fields that are synchronized to aclk_i especially when back to back writes performed on those fields.

2.4 DMA Controller

The DMA has independent Transmit (Tx) and Receive (Rx) engines, and a CSR space. The Tx engine transfers data from the system memory to the device port (MTL), whereas the Rx engine transfers data from the device port to the system memory. The controller uses descriptors to efficiently move data from source to destination with minimal application CPU intervention. The DMA is designed for packet-oriented data transfers such as packets in Ethernet. The controller can be programmed to interrupt the application CPU for situations such as Packet Transmit and Receive Transfer completion, and other normal or error conditions.

The DMA and the application communicate through the following two data structures:

- Control and Status registers (CSR)
- Descriptor lists and data buffers

The DMA transfers the data packets received by the MAC to the Rx Buffer in system memory and Tx data packets from the Tx Buffer in the system memory to the MAC for transmission. The descriptors that reside in the system memory contains the pointers to these buffers.

The base address of each list is written to the respective Tx Descriptor List Address register and Rx Descriptor List Address register. The descriptor list is forward linked and the next descriptor is always considered at a fixed offset to the current one. The offset is controlled by the programming of the DSL field of `VDMA_Ch(#i)_Control` register. The number of descriptors in the list is programmed in the respective Tx (or Rx) Descriptor Ring Length register. After the DMA processes the last descriptor in the list, it jumps back to the descriptor in the List Address register to create a descriptor ring.

The descriptor lists reside in the physical memory address space of the application. Each descriptor can point to a maximum of two buffers. This enables two buffers to be used, physically addressed, rather than contiguous buffers in memory.

Note

- When the 25GMAC is configured with an address greater than 32 bits, the DMA considers that all the descriptors in a list are present in a memory page with maximum size of 4GB. In other words, address bits greater than 32 are taken to be constant and equal for all the descriptors in the ring. However, the upper bits of the various buffer addresses can be different and independent.
- When the 25GMAC is configured with an address greater than 32 bits, the Tx Descriptor can have only one buffer address. However, the Rx Descriptor can still have two buffer addresses.

A data buffer resides in the application physical memory space and consists of an entire packet or part of a packet but cannot exceed a single packet. Buffers contain only packet data. Buffer status is maintained in the descriptor. Data chaining refers to packets that span multiple data buffers. However, a single descriptor cannot span multiple packets. The DMA skips to the data buffer of next packet when EOP is detected.

The 25GMAC supports the ring structure for the DMA descriptor. For details about descriptor structure and how the DMA accesses the descriptors, see “Descriptors” on page 1795.

The XGMAC-AXI always uses DMA controller. In DMA, the efficiency of packet transfer is optimized by the following tasks in parallel:

- Multiple Descriptor Fetch
- Data Transfer
- Descriptor write-back.

All these DMA phases work independently in a pipelined fashion such that the descriptor fetch of a new packet can be performed along with data transfer of the previous packet and the descriptor write-back of the last to previous packet. In this way, packet transfers throughput is improved with minimal gaps between them.

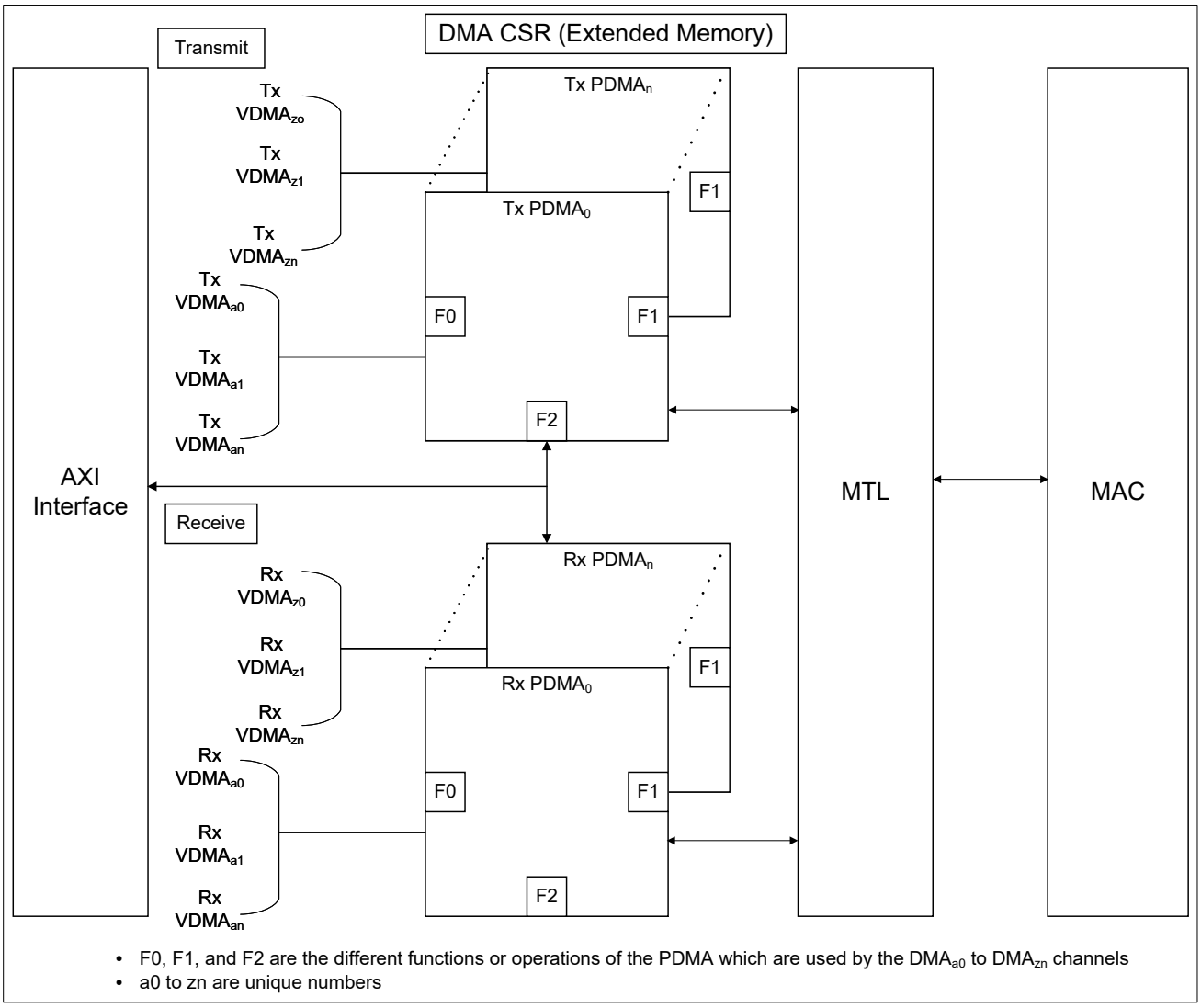
Note In case of Rx DMA, the Data transfer Phase and Descriptor write-back phase cannot be made totally independent, since the order of the data transfer and descriptor write-back has to be strictly followed for maintaining synchronization between the software processing of descriptors and the packet transfers. This ensures that the software processes the closed descriptors only after the corresponding packet buffer is transferred and written into system memory.

The 25GMAC DMA supports scalable architecture by decoupling the logical device from the physical DMA (PDMA). The virtual DMAs are software abstractions of the PDMA. The VDMAs are mapped onto the PDMAs for packet transmission and reception. So, the 25GMAC DMA engine

- Scales up to 128 VDMAs.
- Segregates the register space access to the CSR by the individual DMA engines and the hypervisor.
- Maps directly to the traffic class resulting in better resource utilization
- Supports external SPRAM memory as the CSR registers
- Supports programmable and efficient descriptor cache for the increased number of DMAs
- Supports better resource utilization by mapping the traffic class to the PDMAs.

Figure 2-14 shows the basic block diagram of the DMA architecture in DWC_25gmac. As shown in Figure 2-14, the mapping of the VDMAs to the PDMAs is through programming of the TD2TCMP field of the VDMA_CH(#i)_Tx_Control register and RD2TCMP field of the VDMA_CH(#i)_RX_Control register. The register space for the per DMA channel is in the external SPRAM memory.

Figure 2-14 Basic Block Diagram



2.4.1 Functions of the PDMA

The following are the functions of the PDMA:

- Fetch the descriptor from system memory and write the descriptor data into the cache Memory
- Read the Tx descriptor data from the cache memory and process the Tx descriptor
- Transfer the Tx packet data to the MTL queue
- Read the Rx descriptor data from the cache memory and process the Rx descriptor
- Transfer the Rx packet data from MTL queue and write the data to the system Memory
- Write-back or close the Tx descriptor
- Write-back or close the Rx descriptor

2.4.2 Functions of the VDMA

The VDMA is mapped to a PDMA. The VDMA utilizes the functions of the PDMA to transmit or receive packets. For more details, see Functions of the PDMA.

All the Tx VDMAs that are mapped to the same traffic class, arbitrate using the round robin method. Similarly, all the Rx VDMAs that are mapped to the same traffic class, arbitrate using the round robin method. This is to gain access to the memory controller to read the CSR. The DMA performs the required functions using the PDMA.

The number of PDMA is determined by the parameter associated with AXI Manager interface. The outstanding read requests (ORR) and outstanding write requests (OWR) fields are part of the PDMAs and are present in the common space register. Unique AXI IDs of the various AXI read/write transactions are directly linked with the number of PDMAs.

- Number of Rx PDMAs is equal to the number of Rx Queues selected in the configuration.
- Number of Tx PDMAs is greater than or equal to the number of traffic channels selected in the configuration.
- The sbd_*_dma_ch_no_o signal indicates the VDMA ID.

The debug status of each PDMA indicates the active VDMA number and its functional state, which is available in the DMA_Debug_Status(#i) register.

2.4.3 Application Bus Burst Access

When configured through Bit 0 of the `DMA_SysBus_Mode` register, the DMA attempts to execute the fixed-length burst transfers on the AXI manager interface. The PBL field of Transmit Control and Receive Control registers of respective DMA channel indicates and limits the maximum burst length. The Rx and Tx descriptors are always accessed in the maximum possible (limited by PBL or 16 * 8/bus width) burst length for 16 bytes to be read. For more details about `DMA_SysBus_Mode` register, see the "Register Descriptions" chapter.

The TxDMA initiates a data transfer only when sufficient space is available in the MTL Tx Queue to accommodate the minimum of the following:

- Bytes corresponding to the configured burst (PBL * bus_width/8)
- Remaining bytes in Tx Buffer without EOP
- Number of bytes till EOP

The DMA indicates the start address and the number of transfers required to the AXI manager interface. When the AXI Interface is configured for fixed-length burst, it transfers the data using the best combination of INCR4, INCR8, or INCR16 or higher if enabled and SINGLE transactions. Otherwise (no fixed-length burst), it transfers the data using any INCRx transactions.

The RxDMA initiates a data transfer in the following conditions:

- Sufficient data is available in MTL Rx Queue to accommodate the configured burst
- EOP (when it is less than the configured burst length) is detected in the Rx Queue (possible only in store and forward mode)

The DMA indicates the start address and the number of transfers required to the AXI manager interface. If EOP is reached before the fixed-burst ends on the AXI interface, placeholder transfers are performed in-order to complete the fixed-burst. Otherwise (Bit 0 of `DMA_SysBus_Mode` register is reset), the DMA transfers the data using INCRx transactions.

When the AXI interface is configured for address-aligned beats, both DMA engines ensure that the first burst transfer initiated by the AXI is less than or equal to the size of the configured PBL. Therefore, all subsequent beats start at an address that is aligned to the configured PBL. The DMA can only align the address for beats up to 256 size because the AXI interface does not support more than BLEN 256.

2.4.4 Application Data Buffer Alignment

The Tx and Rx data buffers do not have any restrictions on start address alignment. For example, in systems with 128-bit memory, the start address for buffers can be aligned to any of the 16 bytes. However, the DMA always initiates write transfers with address aligned to the bus width and placeholder data (old data) in the invalid byte lanes. This typically happens during the transfer of the beginning or end of an Ethernet packet. The software driver must discard the placeholder bytes based on the start address of the buffer and size of the packet.

Table 2-2 Application Data Buffer Alignment Examples

Examples	
Buffer Read	If the Tx buffer address is 32'h00000FF2 (for 128-bit data bus), and 32 bytes need to be transferred, the DMA reads three full words from address 32'h00000FF0, but when transferring data to the MTL Tx queue, the additional bytes (the first two bytes) are dropped or ignored. Similarly, the last 14 bytes of the last transfer are also ignored. The DMA always ensures that it transfers a full 32-bit data to the MTL Tx queue, unless it is the end of packet.
Buffer Write	If the Rx buffer address is 32'h0000FF2 (for 64-bit data bus) and 16 bytes of a received packet need to be transferred, the DMA writes 3 full words from address 32'h00000FF0. However, the first 2 bytes of the first transfer and the last 6 bytes of the third transfer have placeholder data. The DMA considers the offset address only if it is the first Rx buffer of the packet. The DMA ignores the offset address and performs full word writes for the middle and the last Rx buffer of the packet.

2.4.5 Buffer Size Calculations

The DMA does not provide Buffer size in the Rx descriptors. The DMA updates only the status fields (RDES) of the descriptors. The driver has to perform the size calculations.

The TxDMA transfers the exact number of bytes (indicated by buffer size field of TDES2) towards the MAC. If a descriptor is marked as first (FD bit of TDES3 is 1), the DMA marks the first transfer from the buffer as SOP. If a descriptor is marked as last (LD bit of TDES3), the DMA marks the last transfer from that data buffer as EOP to the MTL.

The RxDMA transfers data to a buffer until the buffer is full or the end of packet is received from the MTL. When the FD bit of a descriptor is set, the amount of valid data in a buffer is accurately indicated by its buffer size field minus the data buffer pointer offset. The offset is zero when the data buffer pointer is aligned to the data bus width. If a descriptor is marked as last, the buffer may not be full (as indicated by the buffer size in Bits [14:1] of Receive Control Register). To compute the amount of valid data in this final buffer, the driver must read the packet length (PL bits of RDES3[13:0]) and subtract the sum of the buffer sizes of the preceding buffers in this packet. The RxDMA always transfers the start of next packet with a new descriptor.

Note	Even when the start address of a Rx buffer is not aligned to the data width of system bus, the system must allocate a Rx buffer of a size aligned to the system bus width. For example, if the system allocates a Rx buffer of 1,024 bytes (1 KB) starting from address 0x1000, the software can program the buffer start address in the Rx descriptor to have a 0x1002 offset. The RxDMA writes the packet to this buffer with placeholder data in the first two locations (0x1000 and 0x1001). The actual packet is written from location 0x1002. Therefore, the actual useful space in this buffer is 1,022 bytes, even though the buffer size is programmed as 1,024 bytes, because of the start address offset.

2.4.6 DMA Descriptor Fetch Operation

In DMA, the descriptor cache storage (up to 128 descriptors) is available for each DMA channel.

The maximum number of available descriptor in the cache for each RxDMA and each TxDMA depends on the write burst requests (`DWCXG_MAX_WRBREQ`) and read burst request (`DWCXG_MAX_RDBREQ`) respectively, that you specified during configuration.

DMA attempts to fetch multiple descriptors in a burst. The maximum burst length during descriptor fetches for a DMA Channel, is dependent on

- Programmed Burst Length in the `TxPBL` field of the `VDMA_CH(#i)_Tx_Control` register.
- Space available in the Descriptor Cache for that DMA Channel.
- Number of descriptors created by the Host (Offset of the Tail Pointer with respect to the last fetched descriptor).
- The Address Alignment requirement with respect to current descriptor address in case bit[12] AAL of `DMA_SysBus_Mode` is set to 1.

25GMAC selects minimum of these, to calculate the burst length for descriptor fetches and fill-up the cache. After the current descriptor burst fetch is completed, the TxDMA/RxDMA engines do not initiate further descriptor fetches until the space available in the descriptor cache falls below the threshold size programmed in the respective `DMA_Tx_EDMA_Control` or `DMA_Rx_EDMA_Control` register.

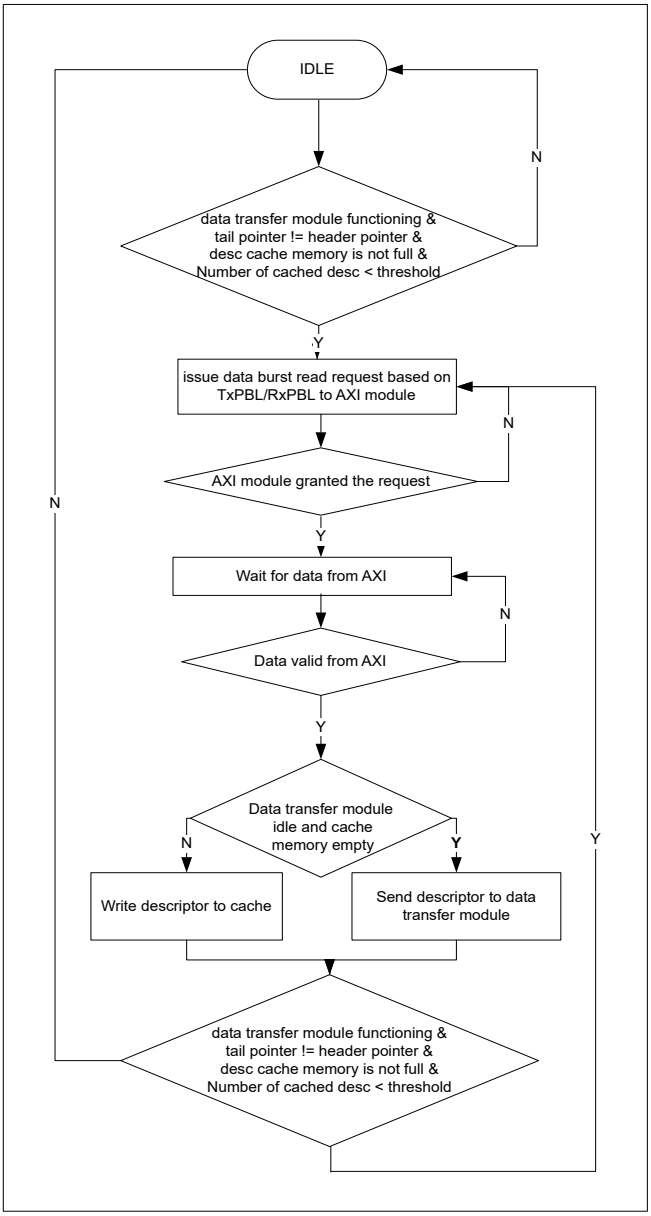
The DMA engine also checks the OWN bit (xDES3[31]) of the descriptor to confirm whether the descriptor contents are valid and can be processed by the DMA. In case any of the descriptor has its OWN bit = 0, then, that descriptor and the rest of the descriptors in that burst are dropped and not written into the cache. The DMA tries to fetch the descriptors again in a burst of length decided as per the conditions, from that point onwards.

The descriptors in the cache are read and given to the Data Transfer engine. Once the Data Transfer engine accepts the descriptor, the next descriptor is read from the cache and thus free up the cache space. The Buffer Unavailable (TBU/RBU) interrupt is generated when all the descriptors created by the host (as indicated by the Tail Pointer) are fetched. It is asserted even if some descriptors are still available in the cache for processing. This advance interrupt provides the opportunity for the software to create more descriptors in advance before the cache gets emptied out completely.

The xBU interrupt is also generated when the DMA engine finds that the OWN bit = 0 for two successive fetches of the same descriptor. It then goes into SUSPEND mode and waits for the software to wake it up by updating/writing into the Tail Pointer register.

Figure 2-15 shows the Tx and Rx Descriptor fetching process. Because of the area consumption, descriptor fetch module does not support outstanding request.

Figure 2-15 Descriptor Fetching Flow Chart



2.4.7 DMA Tx Data Transfer Operation

The data transfer operation by TxDMA is as follows:

1. The Descriptor fetch engine reads a valid descriptor (OWN=1) either from the system memory or the descriptor pre-fetch cache and gives it to the Data transfer engine.
2. It processes the descriptor control bits, buffer sizes and calculates the size of data transfer to be requested as per the settings of the registers (TxPBL field). It checks if the space is available in the corresponding TxQ before it issues the request for data transfer.

3. After the request is accepted by the AXI manager (for scheduling on the bus), this engine calculates the next data transfer size and issues another request. The second request can be for one of the following:
    a. Next burst of data transfer from the same buffer in case the first request is only for part of the buffer
    b. Burst of data transfer from Buffer 2 in case the first request completed the transfer of Buffer 1
    c. Burst of data transfer from Buffer 1 of next descriptor in case the first request completed Buffer 2 (or Buffer 2 was empty)
    d. Start of new packet transfer from Buffer of next descriptor in case the first request completes a packet transfer.
4. At any given time, up to two requests can be outstanding for data transfer completion.
5. A request is considered to be completed once the requested data is fetched and written to the corresponding TxQ in MTL. After this, the engine can issue the next request as given in step 3.
6. After the valid buffers of a descriptor is fetched, the descriptor is freed-up and pushed to the Descriptor write-back engine for closure and the next descriptor from the pre-fetched cache is accepted for processing (Step 2).

All these operations are implemented in a pipe-lined fashion so as to avoid any bottlenecks and overheads due to descriptor reads or writes. Moreover, the engine can request for two packet transfers in sequence (for packets of size smaller than the programmed PBL) without completing the fetch of the first one. This allows the hiding of the system latencies of a read command behind the data transfer of the previous command and thus improve the throughput.

2.4.7.1 DMA Process Flow With Outstanding Request Enabled

Because of the Outstanding Requests support, Tx data transferring functions are divided into two major parts: the "Tx Request Flow" and "Tx Data Flow". The former issues requests to AXI module. The latter transfers data from AXI to TxFIFO.

Figure 2-16 shows the Tx side data transfer flow.

Data reading requests are issued when

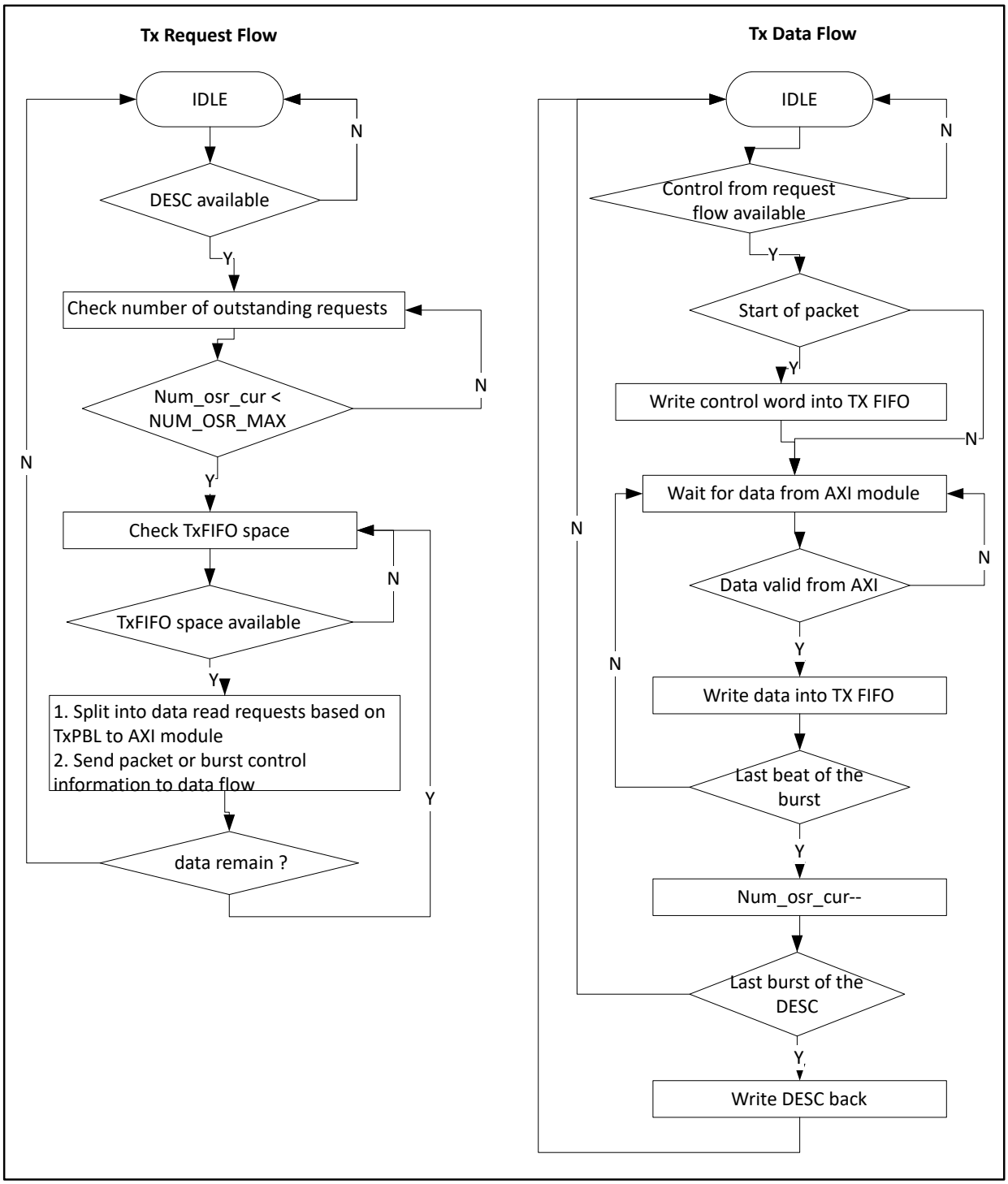
1. There is packet need to be send
2. Not exceed the maximum number of outstanding requests;
3. TxFIFO has space available;

Data is written into TxFIFO when:

1. Available from AXI module
2. Corresponding control information ready

Each time a burst of data transfer is complete, Tx Data Flow releases 1 OSR to Tx Request Flow, so it can continue on further requests.

Figure 2-16 Tx Data Transfer Flow Chart



2.4.8 DMA Rx Data Transfer Operation

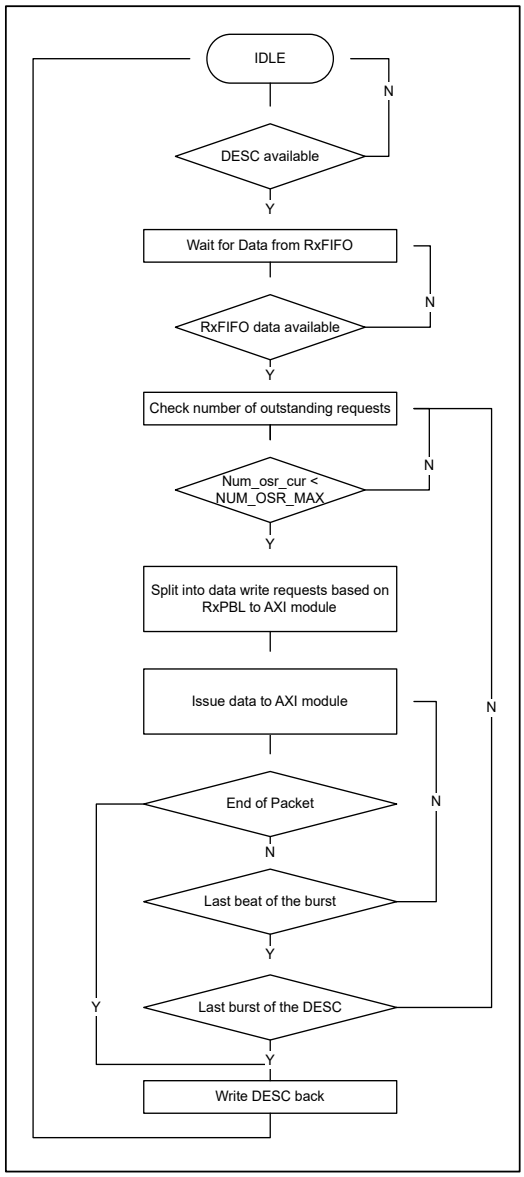
The RxDMA data transfer flow is as follows:

Figure 2-17 shows the Rx side data transfer flow.

1. The Descriptor fetch engine reads a valid descriptor (OWN=1) either from the system memory or the descriptor pre-fetch cache and gives it to the Data transfer engine.
2. The data transfer engine indicates its readiness to transfer data to the MTL RxQ read Controller along with the supported burst-size length as calculated from the buffer sizes and as per the register settings (RxPBL).
3. When the RxQ Read controller selects an RxQ (in case of multiple RxQ configuration) and triggers the Rx DMA to start the data transfer.
4. The Rx DMA engine issues the request which is then accepted and driven by the AXI manager. This requested data is then transferred to the buffer in system memory through the AXI write channel.
5. When the requested data transfer is internally completed (driven on the AXI bus), the data transfer engine jumps back to Step 2 in case the packet transfer is not over (no EOP transferred) in step 4.
6. If both the Rx buffers, pointed in the descriptor are full, it pushes the descriptor to the “Descriptor write-back engine” along with the intermediate status and then accept the next descriptor from the pre-fetch cache and go to Step 2.
7. If the EOP is transferred during the last burst transfer, the engine pushes the descriptor to be closed along with the final packet status read from the RxQ after the EOP and then jump to step 2.

The descriptor pre-fetch and the descriptor closure activities are implemented in a pipe-lined fashion so as to avoid any bottlenecks and overheads due to descriptor reads. However, in step 2, the engine does not initiate another data transfer request unless the previous descriptor write transfer is indicated to be complete as explained in the next section.

Figure 2-17 Rx Data Transfer Flow Chart



Note An additional function, not shown in the flowchart, increments the count when one memory write request is granted, and decrements the count when AXI memory write response arrives. The value of this counter is used to compare the maximum number of outstanding request to control the memory write transaction on AXI bus.

2.4.9 Rx DMA Data Transfer Flow

1. All the VDMAs that are mapped to a traffic class (TC) participate in the round robin arbitration, to select a specific VDMA.
2. The selected DMAs of various traffic class raise a request to the memory controller to read the CSR data of the corresponding VDMA.

3. The selected DMA(#i) puts a request on to the AXI to fetch the descriptors and writes them into its descriptor cache.
4. When a packet is received in MTL queue directed to a specific VDMA(#i), the MTL checks the
   - ❑ Availability of descriptors in the VDMA(#i) cache
   - ❑ PBL

   Only when both checks are successful, the MTL permits the VDMA(#i) to participate in the MTL Rx arbitration.
5. The packets from the Rx queue that win the arbitration are routed to the VDMA(#i).
6. Based on the request from the MTL, VDMA(#i) utilizes the available PDMA(#p) to accept the data.
7. Application must ensure that the programmed Rx buffer size must be less than or equal to the minimum threshold programmed for the Rx queue.

2.4.10 Rx DMA Data Flow

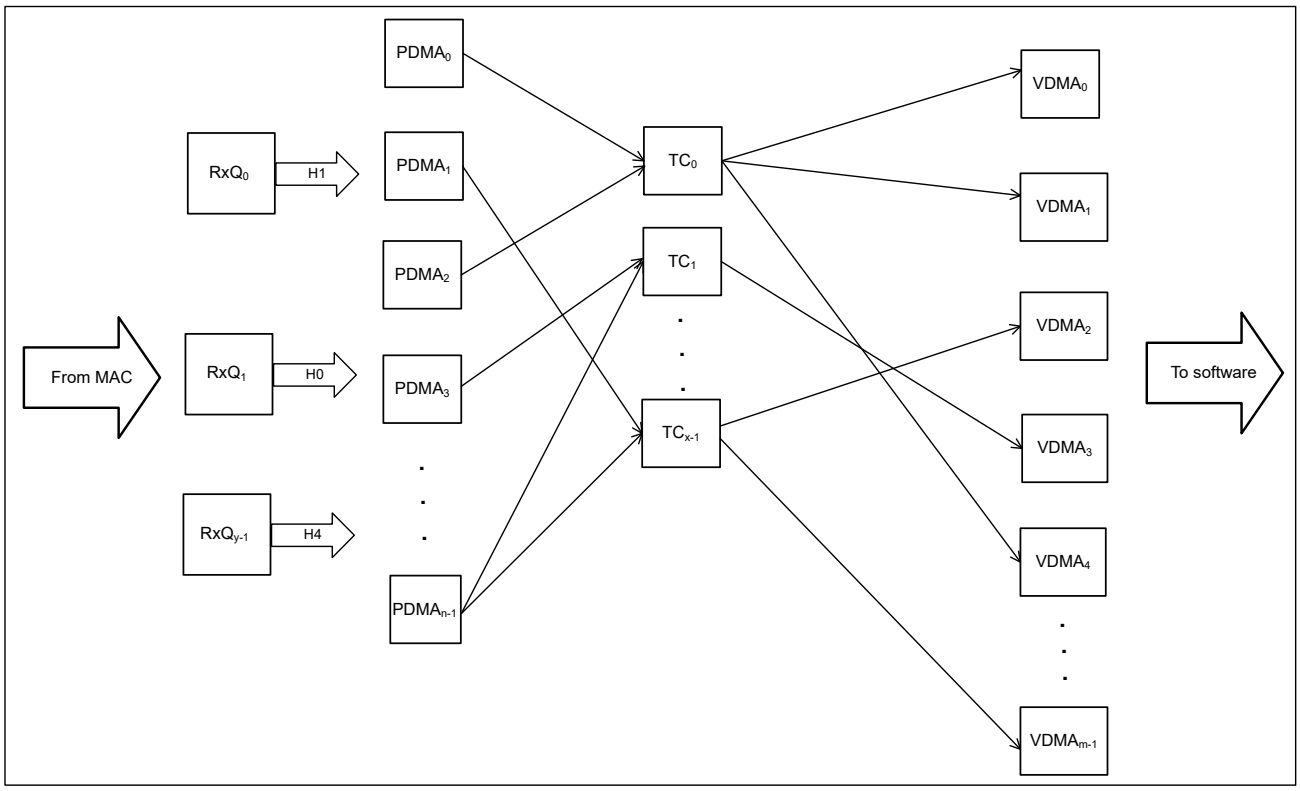
The Rx DMA data flow is as follows:

1. All the VDMAs mapped to a traffic class participate in the round robin arbitration to select a winner from the group of VDMAs.
2. The VDMAs that are selected as winners for their respective traffic class, raise a request to the memory controller to read the CSR data of the corresponding VDMA.
3. The selected DMA(#i) puts a request on the AXI to fetch the descriptors and writes them into its descriptor cache.
4. When the MTL queue receives a packet directed to VDMA(#i), the availability of descriptors in VDMA(#i) cache and its PBL is checked before allowing it to participate in the MTL Rx arbitration.
5. The packets from the queue that win the arbitration are routed to the VDMA(#i).
6. Based on the request from MTL, VDMA(#i) utilizes the available PDM(#p) to accept the data.
7. Application must ensure that the programmed Rx buffer size must be less than or equal to the minimum threshold programmed for any Rx queue.

The PDMA is locked to a VDMA for the entire packet transmission so that the segments are transmitted in order to the transmit queue. To ensure that the other VDMAs mapped to the same traffic class and PDMA are not blocked for the entire duration of the segmentation, the software must map additional PDMAs to the traffic class that is mapped to multiple VDMAs.

For example, in Figure 2-18, RxQ1 and RxQy-1 have packets intended for VDMA0 and VDMA4, respectively. As these 2 VDMAs are mapped to TC0, they can use physical resources of either PDMA0 or PDMA2. If RxQ0 receives packet for VDMA1, which is also mapped to TC0, and there is no PDMA available for data transfer, RxQ0 does not participate in arbitration until one of the PDMAs gets unblocked after completing the data transfer from other RxQ.

Figure 2-18 Rx PDMA to VDMA Flow



With such a type of mapping, each DMA(#i) can be mapped to multiple PDMAs which not only results in better utilization of Tx and Rx PDMA when they are unused by any of its mapped VDMAs but also provides increased bandwidth for higher priority VDMAs.

2.4.11 Mapping of VDMAs to PDMA

Mapping of VDMAs to PDMA occurs in 2 stages:

1. The Tx VDMAs and Rx VDMAs are mapped to a traffic class based on the values programmed in the TD2TCMP field of the VDMA_CH(#i)_TX_Control register and RD2TCMP field of the VDMA_CH(#i)_RX_Control registers during configuration. Application can map multiple VDMAs to the same traffic class.
2. Traffic classes are mapped to PDMAs based on the values programmed in the TP2TCMP field of the PDMA_CH(#i)_TxExtCfg register and RP2TCMP field of the PDMA_CH(#i)_RxExtCfg registers during configuration.

2.4.12 Tx DMA Data Flow

When a VDMA is mapped to a traffic class that is mapped to two or more PDMAs, the packets from the VDMA can be routed to the Transmit queue of any of the PDMA to which the traffic class is mapped.

Using the figure as an example, the packet from VDMA2, VDMA3, and VDMA4 can utilize the hardware resources only from PDMA0. This is because, the 3 VDMAs are mapped to TC1 and TC1 is mapped only to PDMA0.

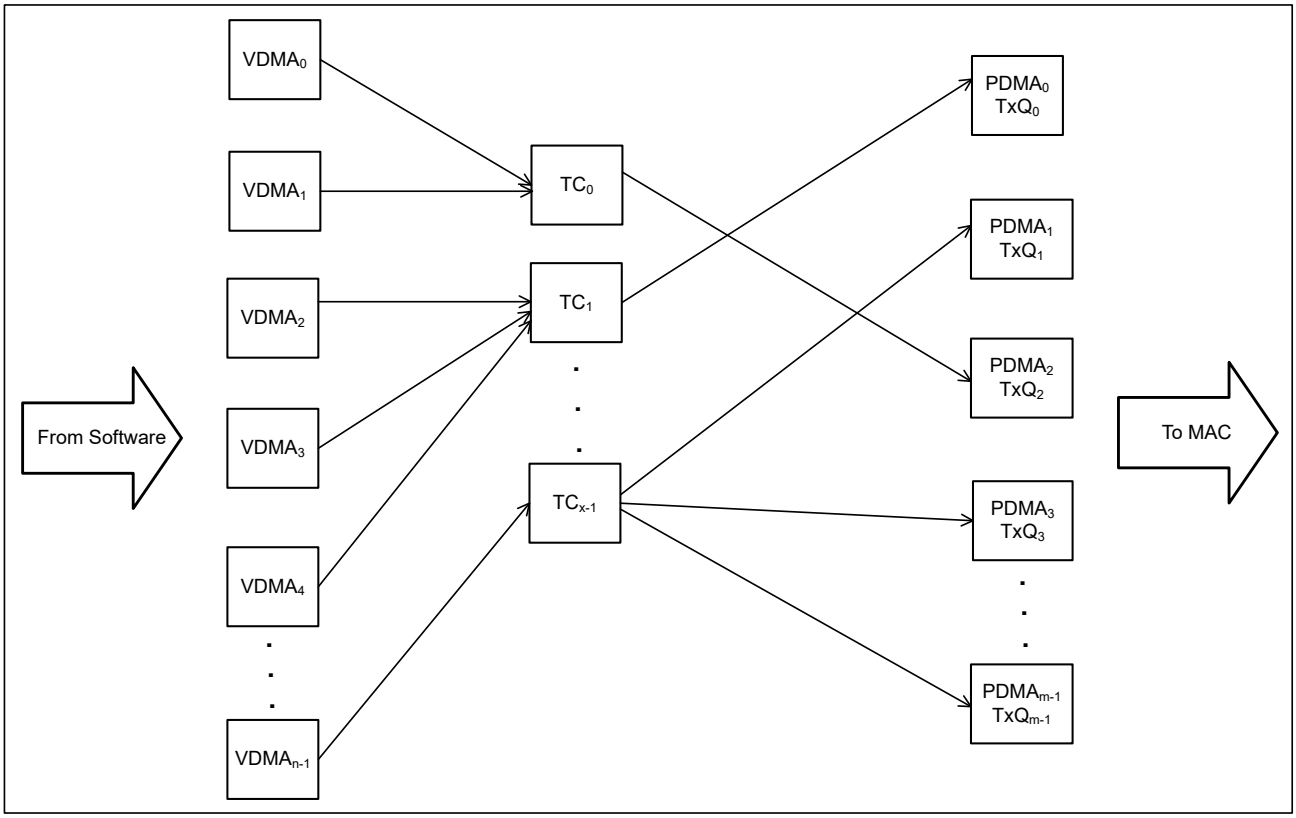
When the MTL scheduler schedules the packet transmission, it is possible that the packets from the same VDMA is not transmitted in the same order in which they were fetched from the system memory. If the application requires that the transmission of packets is in the same order, the VDMA must be mapped to a Traffic class that is mapped to only one PDMA.

If a transmit queue is disabled, the PDMA corresponding to the transmit queue does not participate in the arbitration to load VDMA-CSR details.

Tx DMA Data Flow:

1. All the VDMAs mapped to a traffic class participate in the round robin arbitration to select a winner from the group of VDMAs.
2. The VDMAs that are selected as winners for their respective traffic class, raise a request to the memory controller to read the CSR data of the corresponding VDMA.
3. The memory controller further performs strict priority arbitration and forwards the CSR data read request of each VDMA.
4. The VDMAs and their CSR data are allowed to utilize the virtual function of the PDMA to read the descriptor from cache, transmit the packet and write back the descriptor.

Figure 2-19 Tx VDMA to PDMA Flow



2.4.13 External SPRAM Memory for CSR Space of Per Channel Registers

Because of the decoupling of the software abstraction of DMA from the physical DMA, increased number of DMAs are supported. Software abstraction of DMA involves descriptors, interrupts, and CSR. Retaining the register space of per DMA channel hugely impacts the area. So, to minimize the area, the CSR space of

per VDMA channel is moved to an external 128-bit SPRAM. The memory sizes for a few configurations of Tx and Rx DMA channels are

Number of DMAs (Tx and Rx Each)	Memory Size in Bytes
8	1280
16	2560
20	3200
32	5120
96	15360
128	20480

2.4.14 Configurable Tx and Rx Cache Sizes

The total Tx descriptor cache size depends on the number of Tx VDMAs. Similarly, the total Rx descriptor cache size depends on the number of Rx VDMAs. The total Tx descriptor cache size and the total Rx descriptor cache size is configurable based on the following equation.

DWCXG_PCHTDC_SZ= Specifies Per Tx channel descriptor cache size in number of normal descriptors. Minimum value is 16 and maximum value is DWCXG_MAX_RDBREQ*2, when DWCXG_MAX_RDBREQ > 8

DWCXG_PCHRDC_SZ= Specifies Per Rx channel descriptor cache size in number of normal descriptors. Minimum value is 16 and maximum value is DWCXG_MAX_WRBREQ*2, when DWCXG_MAX_WRBREQ > 8

The possible values of DWCXG_PCHTDC_SZ and DWCXG_PCHRDC_SZ are 16, 32, 64, and 128.

Total descriptor cache size (in bytes) DWCXG_DCACHE_MEM_SIZE = (DWCXG_NUM_DMA_TX_CH * 16 * PCHTDC_SZ) + (DWCXG_NUM_DMA_RX_CH * 16* PCHRDC_SZ)

Use the TXDCSZ field of the VDMA_CH(#i)_TxDescCtrl register and RXDCSZ field of VDMA_CH(#i)_RxDescCtrl register to program the per channel descriptor cache size.

2.4.15 DMA Descriptor Write-Back Operation

The DMA descriptor write-back operation is triggered when the descriptor is passed on to it by the Data Transfer Engine after the corresponding buffer contents are transferred. This engine closes the descriptor with OWN bit = 0 to indicate to the software that the descriptor is freed-up.

For TxDMA, the Descriptor write-back operation clears TDES3 while the other TDES0-TDES2 are not modified. This write-back request is issued independently of the Buffer data transfer requests (pointed by the next descriptor) by the TxDMA engine.

For RxDMA, the Descriptor is closed with the status fields also updated in RDES0-RDES3 fields. To ensure that the buffer transfer to system memory is complete before the descriptor is closed, these transfer operations are performed in sequence with the same AXI-ID. In other words, the RxDMA engine starts performing the next descriptor's buffer transfer only after the current descriptor is written back and closed. This sequencing is done to ensure that the software does not process the descriptor and read the buffer contents before the buffer data transfers are completed by the hardware.

The transfer complete Interrupts TI (bit[0]) in case of TxDMA and RI (bit[6]) in the case of RxDMA is generated when the descriptor corresponding to the last segment of the packet data is closed and also has the Interrupt On Completion bit set. The interrupt is generated only after the AXI response to the descriptor write transfer is received from the AXI subordinate.

If the interrupt generation is not enabled for that packet, the descriptor write operation is deemed to be completed once the descriptor write request is accepted by the AXI subordinate (without waiting for the response from the subordinate). This reduces the latency of descriptor write completion and improves the throughput.

2.4.16 DMA Start/Stop Operation

In addition to software-based control, 25GMAC supports sideband signals also to control per-channel DMA start and stop. The final state of DMA start/stop depends on both sideband signals as well as software.

The software control to start/stop the Tx/RxDMA, is by programing the bit[0] (ST/SR field) of `VDMA_CH(#i)_Tx_Control` or `VDMA_CH(#i)_Rx_Control` register respectively. The sideband signals per-channel are combined with programmable start and stop control bits of the corresponding DMA channel to control the start/stop of the DMA.

When the DMA receives stop command by programming bit[0] (ST/SR field) of `VDMA_CH(#i)_Tx_Control`/ `VDMA_CH(#i)_Rx_Control` register, the DMA is stopped after completing the current packet transfer (if any). However, there can be descriptors in the Memory Cache which are already pre-fetched. The packets corresponding to such descriptors are not transferred. When the DMA is started again by the software by setting the bit[0] (ST/SR field) of the `VDMA_CH(#i)_Tx_Control` or the `VDMA_CH(#i)_Rx_Control` register, the DMA continues processing the descriptor where it was stopped. Table 2-3 shows the function of bit[0] (ST/SR field) of the `VDMA_CH(#i)_Tx_Control/VDMA_CH(#i)_Rx_Control` register.

If the software driver does not process the descriptors residing in the descriptor cache memory, it must program the `VDMA_CH0_TxDesc_List_HAddress` / `VDMA_CH0_TxDesc_List_LAddress` or `DMA_CH0_RxDesc_List_HAddress` /`DMA_CH0_RxDesc_List_LAddress` register. In such cases the DMA flushes the pre-fetched descriptors and process the new list.

Table 2-3 Function of Bit[0], ST/SR Field of DMA Channel Transmit/Receive Control Register

Hardware ST/SR	Software ST/SR	Existing ST/SR	New ST/SR	Comment
0	0	0	0	Default/Reset value
0 to 1	0	0	1	Hardware starts DMA
0	0 to 1	0	1	Software starts DMA
1	1	X	1	Hardware or software starts DMA
1 to 0	1, 0	1	0	Hardware stops DMA
1, 0	1 to 0	1	0	Software Stops DMA

Software based DMA start/stop and sideband based DMA start/stop are mutually exclusive. 25GMAC does not support using both of these simultaneously.

2.4.17 DMA Channel Number Sideband Signals

The DMA channel number sideband signals provide the DMA channel number of all transactions (descriptor/data read/write) on the Manager bus. For the AXI interface, the DMA channel number sideband signals contain both Read/Write Address channel and Write Data channel outputs.

DMA channel number sideband specification is useful for virtualization; for instance, each DMA is assigned to a specific OS/CPU and the system bus must segregate traffic from various DMA channels.

- AXI Write Channel ID:
  - Transmit Descriptor Write-Back: ID = 0
  - Receive Descriptor (and Write-Back): ID = Receive DMA Channel Number + 1
- AXI Read Channel ID:
  - Transmit/Receive Descriptor Fetch: ID = 0
  - Transmit Descriptor (and Data fetch): ID = Transmit DMA Channel Number + 1

For the Tx/Rx Descriptor Fetch and Tx Descriptor Write-Back, all DMA channels share ID0. Software might not be able to implement hardware virtualization. Therefore, the DUT supports unique DMA channel number as sideband. The external logic must translate the sideband number to the required AXI-ID (for virtualization) before connecting to AXI cross-connect. This ensures data/response is given back to DWC_25gmac AXI in the same sequence as requested by DWC_25gmac, even for shared AXI-ID (#0) transactions.

In the AXI Manager block, the gmr_dma_ch_no and gmw_dma_ch_no, DMA channel number sideband signals are available along with the GMR/GMW command channel.

2.4.18 Memory Cache Size Requirements

The DMA uses Cache memory for the following purposes:

- Tx Descriptor Pre-fetch
- Rx Descriptor Pre-fetch
- TSO Header Data Storage

  (Can be separate memory as per the configuration when DWCXG_DIFF_CACHE_MEM_EN = 1)

Each Tx/RxDMA Channel can cache multiple descriptors based on the following parameters:

- DWCXG_MAX_WRBREQ:

  When DWCXG_MAX_WRBREQ <= 8, the per Rx descriptor cache storage is limited to 16 descriptors. Otherwise the descriptor storage is equal to (DWCXG_MAX_WRBREQ * 2).

- DWCXG_MAX_RDBREQ:

  When DWCXG_MAX_RDBREQ <= 8, the per Tx descriptor cache storage is limited to 16 descriptors. Otherwise the descriptor storage equals to (DWCXG_MAX_RDBREQ * 2).

Each descriptor size is 16bytes. Therefore, the memory size in bytes required for descriptor cache is (DWCXG_NUM_DMA_TX_CH) * 16 * (Number of Tx Descriptor size per channel) + (DWCXG_NUM_DMA_RX_CH) * 16 * (Number of Rx Descriptor size per channel)

Each TxDMA/RxDMA channel can cache up to 128 descriptors and size of each Descriptor is 16 bytes. Therefore, the maximum memory size required for descriptor cache is up to:

(DWCXG_NUM_DMA_TX_CH + DWCXG_NUM_DMA_RX_CH) * 128 * 16 bytes

This memory is a single-port RAM whose width is same as the configured data-width (128-bit). This cache can be shared with the TSO header memory when DWCXG_DIFF_CACHE_MEM_EN = 0, and the memory size increases by (DWCXG_NUM_DMA_TSO_CH * DWCXG_TSO_MEM_SIZE) bytes.

2.4.19 Memory Cache Access Arbitration

When the descriptor cache is shared with the TSO header memory, the following accesses can take place:

1. Descriptor Memory Cache Write (Descriptor Read from System Memory)
2. Descriptor Memory Cache Read (Prefetched Descriptor Read)
3. TSO Memory Cache Write (TSO Header Read from System Memory)
4. TSO Header Memory Cache Read (Memory Cache Read)

The memory access requests (1) and (2) are issued by all TxDMA and RxDMA Channels.

The memory access requests (2) and (3) are issued by all TxDMA Channel with TSO enabled.

When you have a common memory for descriptor cache and TSO, the memory arbitration policy (arbitration occurs every clock cycle) is as follows:

- The memory access requests(1) and (3) are mutually exclusive requests as the source of the data is System memory and only one request can be serviced in any clock cycle.
- The memory access requests (1) and (3) have higher priority over memory access requests (2) and (4) so that transactions on AXI bus is not held or blocked.
- The arbitration between memory access requests (2) and (4) is controlled by Bit[5], TMRP field of the DMA_Mode register. When this bit is 0, memory access request (2) has the higher priority over (4). When this bit is 1, then memory access request (4) has the priority over (3).
- The memory access request (2) can arrive from both Tx and RxDMAs. The arbitration between Tx and RxDMA for Descriptor Memory Read is controlled by Bit[4], TDRP field of the DMA_Mode register. When this bit is 0 the RxDMA requests have priority over TxDMA requests. When this bit is 1, TxDMA requests have priority over RxDMA requests. If the requests arrive from multiple TxDMAs or multiple RxDMAs, the higher DMA Channel gets the priority over lower DMA Channels.
- The request (4) can arrive from multiple TxDMAs. The higher DMA channel has priority over lower DMA Channels.

When the descriptor cache is housed in a separate memory, separate arbiters are present. This arbitration occurs every clock cycle. One arbitrates between (1) and (2) while the other is responsible for arbitration between (3) and (4). The rules for each arbiter remains the same.

2.4.20 DMA Error Handling

For any data transfer initiated by a PDMA channel, if the Subordinate responds with an error response, the PDMA stops all operations and updates the error bits and the Fatal Bus Error bit in the Status Register of corresponding VDMA channel. The application must issue a Software reset to 25GMAC to recover. In the XGMAC-AXI configuration, the PDMA receives the error response through rresp_m_i (shown in Figure 2-9) or bresp_m_i signal.

Note

- The reinitializing of descriptor list and restarting of only impacted VDMA related recovery mechanism is not supported, due to dynamic nature of PDMA to VDMA connections. This recovery mechanism must not be used, as the behaviour is not predictable. This is expected to impact the Virtualization related use cases, as the Software reset based recovery mechanism requires the entire 25GMAC (except the AXI interface block) to be reconfigured.
- The same is applicable for Descriptor Definition Error, the reinitializing of descriptor list and restarting of only impacted VDMA related recovery mechanism is not supported. This recovery mechanism must not be used, as the behaviour is not predictable.

2.4.20.1 TxDMA Bus Error Handling Flow

The AXI Bus Error can happen while, fetching the Tx descriptors, reading Tx packet data from system memory, or writing back the Descriptor to the System memory. The 25GMAC performs following operations:

- Stops fetching further descriptors for impacted VDMA and ignores descriptors yet to be fetched as part of any ongoing burst.
- Any ongoing packet transfer for impacted VDMA is terminated with control tag, which is an indication to MAC transmitter to automatically insert CRC error for that packet.
- Any outstanding AXI read requests for impacted VDMA are completed, read data is dropped.
- Any descriptor in the DCACHE is written back with Error Status (bit[27] of TDES3).
- The impacted VDMA is stopped.

After the impacted Tx VDMA channel is stopped by the 25GMAC due to Bus Error, the software driver is expected to follow the steps as described in "Handling Bus Errors and Recovery" on page 1851.

Note

Due to limitation in the design, the Tx VDMA may generate the Tx Process Stopped interrupt immediately after error is detected, that is, without waiting for corresponding pending transfers to complete. It is recommended that Software stops all the Tx/Rx VDMAs and waits for Tx/Rx VDMAs and AXI interface to be idle (DMA_Debug_Status0, DMA_Debug_Status1, and DMA_Debug_Status2 registers are read as all-zeros) prior to issuing Software Reset for recovery. This is required to avoid AXI protocol violation due to any unfinished transfers.

2.4.20.2 RxDMA Bus Error Handling Flow

The AXI Bus Error can happen while, fetching the Rx descriptors, writing Rx packet data to system memory, or writing back the Descriptor to the System memory. The 25GMAC performs following operations:

- Stops fetching further descriptors for impacted VDMA and ignores descriptors yet to be fetched as part of any ongoing burst.
- Any ongoing packet transfer for impacted VDMA is terminated on internal DMA-MTL interface by issuing flush command.
- Any outstanding AXI write requests for impacted VDMA are completed by performing placeholder writes.

- Any descriptor in the DCACHE is written back with Error Status (bit[15] =1 and bit[19:16] = 4'h1000 of RDES3).
- The impacted VDMA is stopped.

After the impacted Rx VDMA channel is stopped by the 25GMAC due to Bus Error, the software driver is expected to follow the steps as described in "Handling Bus Errors and Recovery" on page 1851.

The software driver can optionally program the bit[31], RPF field of the `VDMA_CH(#i)_Rx_Control` registers to prevent any Head of the Line (HOL) blocking of the other VDMA Channels which are not stopped. When this bit is 1 and corresponding VDMA is stopped (either by software or 25GMAC) the Rx packets routed to that Rx VDMA channel are automatically flushed in the Rx Queue. Later, when the VDMA is started by the software driver, the new packets received in the 25GMAC are routed to the Rx VDMA Channel. For more information see "Receive Packet Flush" on page 118.

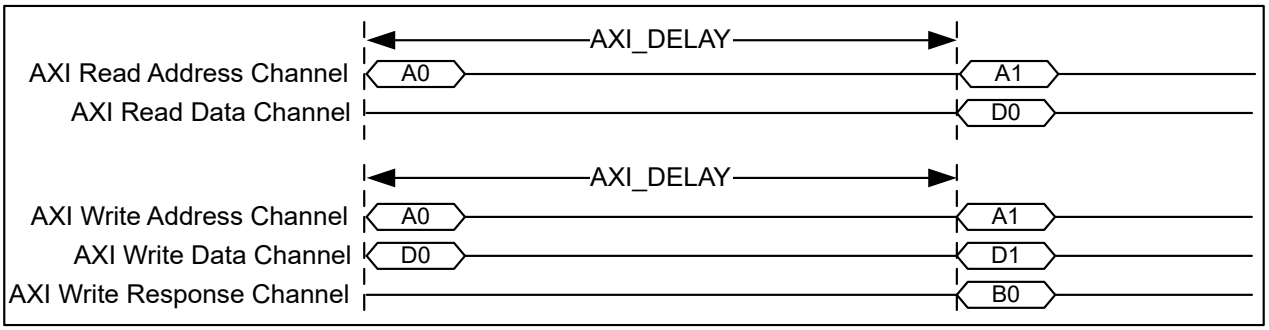
2.4.21 DMA Throughput Tuning

To support up to 10Gbps line rate with a single Tx or RxDMA channel, pipelined processing of more than one frame in a DMA channel, is supported. Increasing the number of outstanding reads and writes masks the memory access latencies due to the pipelining effect and thus improves the effective throughput. The DMA now supports up to 10 Gbps receive and transmit throughput with a single Rx or Tx channel respectively (when the max AXI interconnect latency is 375 application clocks).

In scenarios where the AXI bus has a large delay between request and response, AXI throughput decreases significantly.

For example, in Figure 2-20, if AXI_DELAY is 750ns, and AXI burst length is 3 (4 transfer per burst), and AXI width is 128 bit, the peak throughput does not exceed 682 Mbps.

Figure 2-20 AXI Delay Timing Diagram



If the duration of AXI_DELAY DMA is not waiting in IDLE state, the throughput increases dramatically. That means DMA need to continue on new memory read or write without waiting for response for the current request.

For memory read, corresponding to DMA Tx side data transferring, this method is referred as "Outstanding Requests" or OSR for short.

In a fully posted memory write system, such as PCI and PCIe, the response delay is not an issue for the throughput. However, in 25GMAC, the AXI memory write response serves the following purpose:

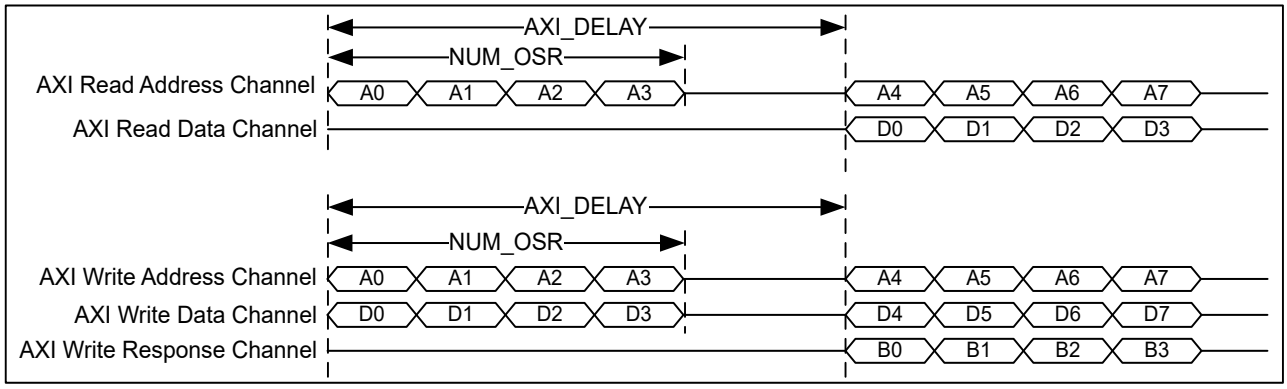
1. Check memory write status; if error occurred during AXI memory write, a Fatal Bus Error is generated
2. It is used to synchronize conventional interrupt (level interrupt) and memory writing. The interrupt corresponding to a descriptor with IOC bit set can only be generated after that descriptor is written into memory for certain. In Posted Write system, this synchronizing is usually done by Message Signaled Interrupt. This implies that a memory-write after the descriptor-write with same attributes, and system ensures the order of these two memory writes.
3. Memory writing transaction shaping: As AXI does not support Posted Write, response must be generated by memory write subordinate. Only a “Pseudo Posted Write” can be designed for AXI memory write.

   AXI memory subordinate usually support a limited number of these pending response, so there is a number in the IP to limit the number of pending response. This behavior is similar to AXI memory read Outstanding Request.

The number of maximum memory read request or write request without response is referred as “NUM_OSR”.

In Figure 2-21, if AXI_DELAY is 750ns, AXI burst length is 3 (4 transfer per burst), AXI width is 128bit, and NUM_OSR is 4, the peak throughput is 2728Mbps. Compared to 689Mbps, the throughput increases 4 times.

Figure 2-21 Timing Diagram for Outstanding Requests (NUM_OSR=4)



2.4.21.1 Parameters

Following features are designed in the DMA to improve throughput:

1. Outstanding Requests (OSR) support on memory read and write in AXI and DMA modules;
2. Descriptor cache;
3. Configurable Burst Length;

These features are carefully designed, and with hardware or software configurable parameters associated. Those parameters include:

1. Hardware Configurable:
   - DWCXG_MAX_RDBREQ: The maximum number of data reading requests can be issued from each DMA data transmission module, without waiting for response.
   - DWCXG_MAX_WRBREQ: The maximum number of data writing requests can be issued from each DMA data reception module, without waiting for response.
   - DWCXG_AXI_GM_MAX_RD_REQUESTS: The maximum number of AXI data reading requests can be issued from AXI module to system AXI bus, without waiting for response.
   - DWCXG_AXI_GM_MAX_WR_REQUESTS: The maximum number of AXI data writing requests can be issued from AXI module to system AXI bus, without waiting for response.
   - DWCXG_TXFIFO_SIZE: MTL TX FIFO size.
   - DWCXG_RXFIFO_SIZE: MTL RX FIFO size.
2. Software Configurable:
   - RD_OSR_LMT field of the DMA_SysBus_Mode register: Software configured maximum number of AXI data reading requests can be issued from AXI module to system AXI bus, it is less than or equal to DWCXG_AXI_GM_MAX_RD_REQUESTS.
   - WR_OSR_LMT field of the DMA_SysBus_Mode register: Software configured maximum number of AXI data writing requests can be issued from AXI module to system AXI bus, it is less than or equal to DWCXG_AXI_GM_MAX_WR_REQUESTS.
   - BLEN fields of the DMA_SysBus_Mode register: When this bit and the UBL bit are 0, the AXI manager can select a burst length of n on the AXI interface. When UBL is 1, this bit has no effect as the AXI manager can always select a burst length up to n on the AXI interface.
   - UBL field of the DMA_SysBus_Mode: For AXI "FIXED" burst type. When this bit is 1, the AXI manager can perform burst transfers that are equal to or less than the maximum allowed burst length programmed in Bits[7:4] for XGMAC-AXI configurations. When this bit is set to 0, the AXI manager performs one of the following burst transfers:
     - Burst transfers of fixed burst lengths as indicated by the BLEN256, BLEN128, BLEN64, BLEN32, BLEN16, BLEN8, or BLEN4 field
     - Burst transfers of length 1

     When this bit is set and no BLEN bit is set, the AXI manager performs bursts transfers of any length up to 16.
   - TxPBL field of the DMA_CH#_Tx_Control register: These bits indicate the maximum number of beats to be transferred in one DMA data transfer. This is the maximum value that is used in a single block Read or Write. The DMA always attempts to burst as specified in PBL each time it starts a burst transfer on the application bus. You can program PBL with any of the following values: 1, 2, 4, 8, 16, or 32. Any other value results in undefined behavior.

     To transfer more than 32 beats, perform the following steps:

     i. Set the PBLx8 mode in the DMA_CH0_Control register.

     ii. Set the PBL.

Note

1. This PBL is for the internal DMA engine. The AXI Interface block can split this PBL requests from DMA engine into multiple AXI requests based on the programmed values in DMA_SysBus_Mode register.
2. The maximum limit is equal to half of the Tx Queue depth (Queue Depth = Queue Size / Datawidth).

- `RxPBL` field of the `DMA_CH#_Rx_Control` register: These bits indicate the maximum number of beats to be transferred in one DMA data transfer. This is the maximum value that is used in a single block Read or Write. The DMA always attempts to burst as specified in PBL each time it starts a burst transfer on the application bus. You can program PBL with any of the following values: 1, 2, 4, 8, 16, or 32. Any other value results in undefined behavior

  To transfer more than 32 beats, perform the following steps:

  a. Set the PBLx8 mode in `DMA_CH0_Control` register.

  b. Set the PBL.

Note

1. This PBL is for the internal DMA engine. The AXI Interface block can split this PBL requests from DMA engine into multiple AXI requests based on the programmed values in DMA_SysBus_Mode register.
2. The maximum limit is equal to half of the Tx Queue depth (Queue Depth = Queue Size / Datawidth).

- `TQS` field of the `MTL_TxQ#_Operation_Mode` register: This field indicates the size of the allocated Transmit queues in blocks of 256 bytes. The sixteenth bit is the starting bit of this field. The width of this field depends on the Tx memory size selected in your configuration. For example, if the memory size is 2048, the width of this field is 3 bits:

  LOG2(2048/256) = LOG2(8) = 3 bits

  A value of 0 indicates 256 bytes. When this queue is enabled (TXQEN=10), a minimum value of 7 (size = 2048) is recommended so that the queue can store more than a complete max-sized Ethernet packet. This field is reserved and RO in single queue configuration, and its reset value corresponds to the transmit FIFO size.

- `RQS` field of the `MTL_RxQ#_Operation_Mode` register: This field indicates the size of the allocated Receive queues in blocks of 256 bytes. The sixteenth bit is the starting bit of this field. The width of this field depends on the Rx memory size selected in your configuration. For example, if the memory size is 2048, the width of this field is 3 bits:

  LOG2(2048/256) = LOG2(8) = 3 bits

  A value of 0 indicates 256 bytes. This field is reserved and RO in single queue configuration, and its reset value corresponds to the receive FIFO size. When this queue is used, a minimum value of 7 (size = 2048) is recommended so that the queue can store more than a complete max-sized Ethernet packet.

- TEDM field of the DMA_Tx_EDMA_Control register: This field controls the sequence of descriptor and data transfers in the DMA engine for all TxDMA channels as follows:
  - 00: Default mode of DMA in which all transfers are independently triggered and can occur in parallel. This gives you the best performance while the other modes must be used for debug purpose only.
  - 01: The value of TDPS field is ignored and only one Descriptor is fetched at a time from the System Memory. The next descriptor is fetched only after the current descriptor is accepted by Data Transfer Engine. The Data transfer, Descriptor close and the next descriptor fetch can still happen in parallel.
  - 10: The value of the TDPS field is ignored and only one Descriptor is fetched at a time from the System Memory. The next descriptor is fetched only after the data transfer corresponding to current descriptor is completed. The Descriptor close and the next descriptor fetch can still happen in parallel.
  - 11: All operations are sequential. Descriptor is fetched, data transfer is completed, descriptor is closed and then only the next descriptor is fetched. The value of TDPS is ignored in this mode.
- REDM field of the DMA_Rx_EDMA_Control register: This field controls the sequence of descriptor and data transfers in the DMA engine for all RxDMA channels as follows:
  - 00: Default mode of DMA in which all transfers are independently triggered and can occur in parallel. This gives you the best performance while the other/following modes must be used for debug purpose only.
  - 01: The value of RDPS field is ignored and only 1 Descriptor is fetched at a time from the System Memory. The next descriptor is fetched only after the current descriptor is accepted by Data Transfer Engine. The Data transfer, Descriptor close and the next descriptor fetch can still happen in parallel.
  - 10: The value of RDPS field is ignored and only 1 Descriptor is fetched at a time from the System Memory. The next descriptor is fetched only after the data transfer corresponding to current descriptor is completed. The Descriptor close and the next descriptor fetch can still happen in parallel.
  - 11: All operations are sequential. Descriptor is fetched, data transfer is completed, descriptor is closed and then only the next descriptor is fetched. The value of RDPS is ignored in this mode.
- TDPS field of the DMA_Tx_EDMA_Control register: This field controls the threshold in the Descriptor cache after which the DMA starts pre-fetching the TxDMA descriptors. The DMA engine for all TxDMA channels initiates requests for the descriptor fetches as soon as the number of descriptors in the cache memory for that DMA channel, falls below or equal to the programmed threshold (each descriptor is 16 bytes).
  - 00: Threshold of 0 descriptors. Descriptor pre-fetch is triggered only when cache is empty.
  - 01: Threshold is 4 descriptors. Descriptor pre-fetch is triggered when 4 descriptors are left in the cache.
  - 10: Threshold is 8 descriptors.
  - 11: Threshold is 12 descriptors.
  - The transmit descriptor prefetch thresholds are scaled by a factor of 2 and 4 when Maximum Outstanding Read Burst Requests for each TxDMA channel selected is 32 and 64 respectively.

- RDPS field of the DMA_Rx_EDMA_Control register: This field controls the threshold in the Descriptor cache after which the DMA starts pre-fetching the RxDMA descriptors. The DMA engine for all RxDMA channels initiates requests for the descriptor fetches as soon as the number of descriptors in the cache memory for that DMA channel, falls below or equal to the programmed threshold (each descriptor is 16 bytes)
  - 00: Threshold of 0 descriptors. Descriptor pre-fetch is triggered only when cache is empty.
  - 01: Threshold is 4 descriptors. Descriptor pre-fetch is triggered when 4 descriptors are left in the cache.
  - 10: Threshold is 8 descriptors.
  - 11: Threshold is 12 descriptors.
  - The Rx Descriptor prefetch thresholds are scaled by a factor of 2 and 4 when Maximum Outstanding Write Burst Requests for each RxDMA Channel selected is 32 and 64 respectively.

The DSL field of the DMA_CH#_Control register specifies the Word, Dword, or Lword number (depending on 64-bit, or 128-bit bus) to skip between two descriptors. The address skipping starts from the end of the current descriptor to the start of the next descriptor. When the DSL value is equal to zero, the DMA takes the descriptor list as contiguous.

By proper tuning of these parameters, line rate with a single channel is achievable. IP users can decide the proper values of those parameters based on their own applications to achieve better area efficiency or performance efficiency or balanced. Following parameters are the most important argument of throughput:

1. AXI_DELAY: The parameter of AXI bus measured delay between requesting to response. This is the parameter that post most impact on the throughput.
2. AXI_SIZE: The “Burst Size” of AXI bus. In the following sections, this parameter is calculated in bytes.
3. AXI_CYC: The clock cycle of AXI bus.
4. AXI_BLEN: The Burst Length of AXI bus. Reference AXI specification for calculation of this value.
5. NUM_OSR: The maximum number of outstanding requests. The value of this parameter is dependent on DWCXG_MAX_RD/WRBREQ and DWCXG_AXI_GM_MAX_RD/WR_REQUESTS, and WR_OSR_LMT or WR_OSR_LMT field of the DMA_SysBus_Mode register. Assuming AXI_BLEN equals to PBL, the final value of this must be

   MIN(DWCXG_MAX_RD/DWCXG_MAX_WRBREQ, DWCXG_AXI_GM_MAX_RD/WR_REQUESTS/Number of AXI IDs, RD_OSR_LMT/WR_OSR_LMT/ Number of AXI IDs)
6. PBL: Programmed value of TxPBL or RxPBL for transmission and reception separately.
7. BLEN: Burst length on AXI bus equals to MIN(AXI_BLEN, PBL). As DMA splits packet transfers to small requests to AXI module based on PBL, and AXI further splits requests from DMA to requests on AXI bus. So this is the eventual burst length of requests on AXI bus.
8. PKT_LEN: The minimum length of packets for which line rate or other target throughput is desired by IP users in applications.
9. BW_MAX: The theoretically maximum throughput on AXI bus under certain conditions.

10. BW_ACT: The actual bandwidth on AXI bus, usually BW"_" ACT=BW"_" MAX
11. BW_LR: The bandwidth required on AXI bus to math Ethernet Line Rate. Usually this equals to "Line Rate = 10Gbps* Ethernet efficiency". Consider IPG, the maximum Ethernet efficiency = (PKT_LEN)/((PKT_LEN + IPG + PREAMBLE));

Following sections demonstrate how to calibrate these parameters to get best throughput with limited hardware resource, and adapt this IP into system level AXI interface. You must decide based on your system-level configurations.

For example in storage applications, length of packets usually is huge, jumbo frames (packet length up to 9k bytes) or super jumbo frames (or jabber frame, up to 64k bytes). In these applications, 64-byte packets is not a major concern. Optimizing the throughput for bigger packets achieves better area efficiency.

Another example is in computing applications, in which many packets with very few bytes are on the Ethernet interface, usually 1 cache line. Then calibrating the design to get best performance for 64 bytes packets is necessary. The side effect is the rapid increase in the hardware resource .

2.4.21.2 Transmit Side Tuning

Basic Concepts

Ideally, the maximum number of outstanding requests required to hit the throughput requirement must be compliant with the following equation:

$$BW_MAX = min\left(\frac{\frac{min\left(NUM_OSR, \frac{AXI_DELAY}{BLEN \times AXI_SIZE}\right)}{ceil\left(\frac{PKT_LEN}{BLEN \times AXI_SIZE}\right)} \times PKT_LEN}{AXI_DELAY}, AXI_CYC \times AXI_SIZE\right)$$

Following sections provide some guidelines on how to tune these parameters and the ones in the design that are mapped to these parameters.

Basic AXI configuration

If line rate throughput is required for DMA, the AXI bandwidth AXI"_" CYC×AXI"_" SIZE = BW_LR must always be true.

AXI_DELAY must be the minimum possible. This makes that increment of AXI"_" CYC×AXI"_" SIZE does not improve the throughput. So, DMA uses the following methods to improve throughput which comes with additional hardware resource costs.

The burst length and number of outstanding requests must match the values in the following guidelines:

1. BLEN: In the equation, if,

   Condition 1: NUM_OSR <= (AXI_DELAY/(BLEN * AXI_SIZE)

   Condition 2: PKT_LEN >= LEN * AXI_SIZE

   DMA throughput increases when BLEN is increased. This means that, if bytes of one request increases, the performance increases. Ideally if (PKT_ LEN)/(ceil((PKT_ LEN)/(BLEN * AXI_ SIZE)))>= (BW_ MAX×AXI_ DELAY), the maximum throughput can be meet by increasing only the BLEN.

For example, in a system with AXI_DELAY = 200ns, AXI_SIZE = 16bytes, PKT_LEN >= 250bytes, AXI_CYC >= 78.125MHz, set `BLEN` to 16 to meet the line rate. Increasing BLEN is also the most cost effective way to boost throughput. But this method has the following limitations:

a. This increase is not linear. When (n+1)×(BLEN * AXI_ SIZE) >= PKT_ LEN > n × (BLEN×AXI_ SIZE), ceil ((PKT_ LEN)/(BLEN * AXI_ SIZE)) is always (n+1). Increasing of `BLEN` does not improve throughput.

b. When the minimum PKT_ LEN<BLEN * AXI_ SIZE, increasing BLEN does not improve throughput.

Principle for setting this parameter is make sure one small packet can fit into one request. Consider packets with 2×(BLEN * AXI_ SIZE) >= PKT_ LEN > BLEN * AXI_ SIZE. This value must be doubled as (minimum PKT_ LEN)/(AXI_ SIZE) * 2, so small packets can always fit in one request.

As descriptor fetching is in burst, without outstanding requests, this parameter must be set to a value no less than NUM_ OSR+descriptor skip length, to make sure there are enough descriptor for data transfer.

Guidelines:

The BLEN must be set to max ((minimum PKT_ LEN)/(AXI_ SIZE)×2, NUM_ OSR + descriptor skip length). And this must be rounded to closest integer power of 2 and applied to parameters `DWCXG_AXI_BL`, `DMA_SysBus_Mode.BLENn`, `DMA_CH#_Tx_Control.TxPBL`.

2. NUM_OSR:

These methods, such as increasing AXI bandwidth, increasing AXI burst length, have their limits on improving throughput. But in system level high AXI delay is unavoidable. So the "outstanding request" method is designed in the DMA.

In the equation, if:

Condition1: ceil ((minimum PKT_LEN)/(BLEN * AXI_SIZE))=1

Condition2: NUM_OSR <= (AXI_ DELAY)/(BLEN * AXI_ SIZE)

Condition3: AXI_CYC * AXI_SIZE >= BW_LR

the equation can be simplified as NUM_ OSR=(BW_ MAX * AXI_ DELAY)/(minimum PKT_ LEN).

Example:

System with AXI_DELAY = 750ns, AXI_SIZE = 16bytes, PKT_LEN >=64bytes, AXI_CYC >= 78.125MHz, set NUM_OSR to 16 to meet the line rate.

The side effects of this method is that it consumes more hardware resource. An additional outstanding request increases 600~700 gates and 256 bits descriptor cache memory increment. Consider trade off between performance and cost when making decisions.

Guidelines to get best performance with limited hardware resource:

Principle for setting this parameter is NUM_ OSR >= (BW_ MAX × AXI_ DELAY)/(minimum PKT_ LEN). This value must be applied to the `DWCXG_MAX_RDBREQ`, and `DWCXG_AXI_GM_PER_ID_MAX_RD_REQUESTS` parameters.

`DWCXG_AXI_GM_MAX_RD_REQUESTS` and `DMA_SysBus_Mode.RD_OSR_LMT` must be set to greater than NUM_OSR. These requests may be occupied by descriptor fetching, and AXI module may split requests from DMA to smaller burst, the value (BW_ MAX * AXI_ DELAY)/(minimum PKT_ LEN) * 2 is preferred.

As each transmitted packet is associated with at least one descriptor writing back, so parameters DWCXG_AXI_GM_MAX_WR_REQUESTS and DMA_SysBus_Mode.WR_OSR_LMT must also be set to no less than NUM_OSR value.

3. Other Parameters:

   DWCXG_TXFIFO_SIZE: This parameter must be set to BW_ MAX *AXI_ DELAY * number of TX Channels, to make sure MTL_TxQ#_Operation_Mode.TQS for each queue can be set to BW_ MAX * AXI_DELAY.

   TQS field of the MTL_TxQ#_Operation_Mode register: This parameter must be set to BW_ MAX * AXI_ DELAY, so each Tx queue can pre-allocate TxFIFO for each channel before it issue a request.

   UBL field of the DMA_SysBus_Mode register: Set this to 1'b1 to avoid unnecessary burst split in AXI, if system level allows this behavior.

   TEDM field of the DMA_Tx_EDMA_Control register: Set this to 2'b00 to make sure data transfer and descriptor fetch are parallelized.

   TDPS field of the DMA_Tx_EDMA_Control register: Set this to 2'b11 to ensure data transfer and descriptor fetch are parallelized. This also ensures enough descriptors are cached locally without waiting for completion of descriptor fetching, before the new data transfer.

2.4.21.3 Receiving Side Tuning

1. BLEN

   For some packets, DMA does not get packet length before issuing AXI memory write requests. In this case, DMA always sets BLEN according to descriptor buffer size and RxPBL field of the VDMA_CH(#i)_Tx_Control and fill the invalid beats, which do not have valid packet data, with placeholder data. This is required by AXI system. Program the VDMA_CH(#i)_Tx_Control.RxPBL field with smaller value.

   There is no equation applied to this situation. Calculating the NUM_OSR with enough budget is the preferred way.

2. As Rx side data transfer and descriptor write-back share the same ID, DWCXG_AXI_GM_MAX_WR_REQUESTS and DMA_SysBus_Mode. WR_OSR_LMT must always be greater than 2 * DWCXG_MAX_WRBREQ.

Note DWCXG_AXI_GM_MAX_WR_REQUESTS and WR_OSR_LMT must always be greater than 2xDWCXG_MAX_WRBREQ

3. Other Parameters

   REDM field of the DMA_Tx_EDMA_Control register: This parameter must be set to 2'b00 to make sure data transferring and descriptor fetching are parallelized.

   RDPS field of the DMA_Tx_EDMA_Control register: This parameter must be set to 2'b11 to ensure data transfer and descriptor fetch are parallelized. This also ensures enough descriptors are cached locally without waiting for descriptor fetching to be complete before new data transfer.

2.4.21.4 Parameter and Suggested Value

Following table shows the suggested values for the parameters.

Parameters	Suggested Value
DWCXG_MAX_RDBREQ	$\frac{BW_MAX \times AXI_DELAY}{minimum\ PKT_LEN}$
DWCXG_AXI_GM_MAX_RD_REQUESTS	$\frac{BW_MAX \times AXI_DELAY}{minimum\ PKT_LEN} \times 2$
DWCXG_AXI_BL	$max(\frac{minimum\ PKT_LEN}{AXI\ SIZE} \times 2, \frac{BW_MAX \times AXI_DELAY}{minimum\ PKT\ LEN} + descriptor\ skip\ length).$
DWCXG_TXFIFO_SIZE	$BW_MAX \times AXI_DELAY \times number\ of\ TX\ Channels$
DMA_SysBus_Mode.RD_OSR_LMT	$\frac{BW_MAX \times AXI_DELAY}{minimum\ PKT_LEN} \times 2$
DMA_SysBus_Mode.BLENn	$max(\frac{minimum\ PKT_LEN}{AXI\ SIZE} \times 2, \frac{BW_MAX \times AXI_DELAY}{minimum\ PKT\ LEN} + descriptor\ skip\ length).$
DMA_SysBus_Mode.UNDEF	1'b1
DMA_CH#_Tx_Control.TxPBL	$max(\frac{minimum\ PKT_LEN}{AXI\ SIZE} \times 2, \frac{BW_MAX \times AXI_DELAY}{minimum\ PKT\ LEN} + descriptor\ skip\ length).$
MTL_TxQ#_Operation_Mode.TQS	$BW_MAX \times AXI_DELAY$
DMA_Tx_EDMA_Control.TEDM	2'b00
DMA_Tx_EDMA_Control.TDPS	2'b11
DMA_CH#_Control.DSL	descriptor skip length
DWCXG_MAX_WRBREQ	$\frac{BW_MAX \times AXI_DELAY}{minimum\ PKT_LEN}$
DWCXG_AXI_GM_MAX_WR_REQUESTS	$\frac{BW_MAX \times AXI_DELAY}{minimum\ PKT_LEN} \times 2$
DWCXG_AXI_GM_PER_ID_MAX_WR_REQUESTS	$\frac{BW_MAX \times AXI_DELAY}{minimum\ PKT_LEN} \times 2$
DWCXG_RXFIFO_SIZE	Refer to Data Book for suggestion
DMA_SysBus_Mode.WR_OSR_LMT	$\frac{BW_MAX \times AXI_DELAY}{minimum\ PKT_LEN} \times 2$
DMA_CH#_Rx_Control.RxPBL	$max(\frac{minimum\ PKT_LEN}{AXI\ SIZE} \times 2, \frac{BW_MAX \times AXI_DELAY}{minimum\ PKT\ LEN} + descriptor\ skip\ length).$
MTL_RxQ#_Operation_Mode.RQS	See Register Chapter
DMA_Rx_EDMA_Control.REDM	2'b00
DMA_Rx_EDMA_Control.RDPS	2'b11

2.5 MAC Transaction Layer (MTL)

The MAC Transaction Layer provides FIFO memory to buffer and regulate the frames between the application system memory and the 25GMAC IP. The native interface enables data transfer between the host and the MAC transaction layer through a simple handshake protocol.

The MTL block also provides a reliable synchronization mechanism for data transfer between the application and 25GMAC clock domains. The MTL has an asynchronous FIFO on the transmit and receive paths, which are 68 bits or 133 bits wide in 64-bit and 128-bit mode respectively.

The MTL block communicates with the application through the Application Transmit Interface (ATI), Application Receive Interface (ARI), and 25GMAC Control Interface (MCI). Default and optional I/O signals are discussed in *Signal Descriptions* chapter.

2.5.1 Transmit Path

The application module drives all transactions related to the Transmit path through ATI.

The application pushes the Ethernet packets into the corresponding queue. The packet is then popped out and transferred to the MAC when the queue threshold is reached (threshold mode) or complete packet is in the queue (store-and-forward mode).

The Tx queue has a default size of 4K bytes. The queue-fill level is indicated to the application so that it can initiate a data fetch in required bursts from the application or system memory. The application indicates the SOP and EOP as packet delimiters through ATI interface. For each packet pushed into the Tx Queue, a control words is required before the SOP data is transferred on the ATI. This control word denotes the operations to be performed on that packet. In configurations with multiple queues, the application must also indicate the queue number for which the transaction is being addressed.

2.5.1.1 Transmit Control Word

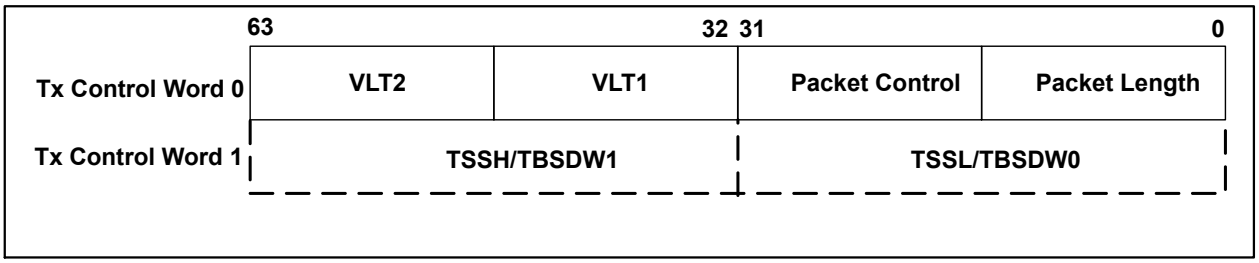
The following control information related to packet transmission is provided as a part of the Control Word through ATI interface:

- Packet Length
- CRC Pad Control
- Source Address Insertion Control (if Enable SA and VLAN Insertion on Tx options is selected)
- VLAN Insertion and Replacement Control and VLAN Tag (if Enable SA and VLAN Insertion on Tx options is selected)
- TCP/IP Checksum Insertion Control (if Enable Transmit TCP/IP Checksum Offload is selected)
- One-step Timestamping Control Correction (if One-step Timestamping correction is selected)
- Transmit Timestamp Enable (if IEEE 1588 Timestamping is selected in your configuration)
- When Time-Based Scheduling (TBS) feature is selected, packet launch time and GSN (AV_EST packet)

Transmit Control Word Format

The control word format for 64-bit data width configuration is shown in Figure 2-22. The Tx Control Word1 is present only when the Enable One-Step Timestamp Feature option or Peer-to-Peer PTP Transparent Clock (P2P TC) Message Support Feature option is selected in your configuration and both `OSTC` and `OSTA` bits are set. The Control Word number is indicated through the `ati_ctrl_type_i` signal of the ATI interface.

Figure 2-22 Tx Control Word Structure for 64-Bit Data Width Configurations



The control word format for 128-bit data width configurations is shown in Figure 2-23.

Figure 2-23 Tx Control Word Structure for 128-Bit Data Width Configurations

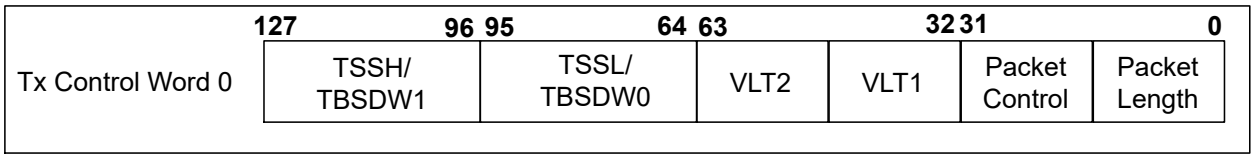


Table 2-4 provides description of Packet Length (Bits[15:0]) and Packet Control Fields (Bits[31:16]).

Table 2-4 Packet Length and Packet Control Fields of the ATI Control Word

Field	Field Name	Description
31	VXN	VXN packet. When set, this bit indicates this is a VXN packet. Available when DMA is enabled.
30	SVTV	Second VLAN Tag Valid When set, this bit indicates that the VLAN Tag2 to be inserted or replaced is available in the VLT2[15:0] field of TX Control Word. This field is available only when the Enable SA and VLAN Insertion on Tx option and the Enable Double VLAN Processing option are selected while configuring the controller. Reserved otherwise.
29	VTV	VLAN Tag Valid When set, this bit indicates that the VLAN Tag1 to be inserted or replaced is available in the VLT1[15:0] field of TX Control Word. This field is available only when the Enable SA and VLAN Insertion on Tx option is selected while configuring the controller. Reserved otherwise.
28:27	VTIR	VLAN Tag Insert or Replace When set, these bits request the MAC to perform VLAN tagging or untagging before transmitting the packets. The application must set the appropriate CRC Pad Control value when this control is enabled. The following list describes the values of these bits: ■ 2'b00: Do not add a VLAN tag. ■ 2'b01: Remove the VLAN tag from the packets before transmission. This option must be used only with the VLAN packets.

Table 2-4 Packet Length and Packet Control Fields of the ATI Control Word (continued)

Field	Field Name	Description
		■ 2'b10: Insert a VLAN tag. ■ 2'b11: Replace the VLAN tag. This option must be used only with the VLAN packets. For insert or replace operations, the tag value is taken from VLT1 field of the TX control word or VLT field of the MAC_VLAN_Incl register, based on the VLTI bit in the MAC_VLAN_Incl register. These bits are valid when the Enable SA and VLAN Insertion on Tx option is selected.
26:24	SAIC	SA Insertion Control These bits request the MAC to insert or replace the Source Address field in the Ethernet packet with the value given in the MAC Address register. The application must set the appropriate CRC Pad Control value when this control is enabled. Bit 26 specifies the MAC Address Register (1 or 0) value that is used for Source Address insertion or replacement. The following list describes the values of these bits: ■ 2'b00: Do not modify the source address ■ 2'b01: Include or insert the source address. For reliable transmission, the application must provide frames without source addresses. ■ 2'b10: Replace the source address. For reliable transmission, the application must provide frames with source addresses. ■ 2'b11: Reserved These bits are valid when the Enable SA and VLAN Insertion on Tx option is selected.
23:22	IVTIR	Inner VLAN Tag Insert or Replace These bits control inner VLAN tagging or untagging before transmitting the packets. The outer VLAN tag must be present for this operation. The application must set the appropriate CRC Pad Control value when this control is enabled. The following list describes the values of these bits: ■ 2'b00: Do not add inner VLAN tag. ■ 2'b01: Remove inner VLAN tag from packets before transmission. This option must be used only with stacked VLAN packets. ■ 2'b10: Insert inner VLAN tag ■ 2'b11: Replace inner VLAN tag. This option must be used only with stacked VLAN packets. The tag value for insertion or replacement is taken from the VLT2 field of the TX control word or the VLT field of the MAC_VLAN_Incl register, based on the VLTI bit in the MAC_VLAN_Incl register. These bits are valid when the Enable Double VLAN Processing option and the Enable SA and VLAN Insertion on Tx option are selected during controller configuration.

Table 2-4 Packet Length and Packet Control Fields of the ATI Control Word (continued)

Field	Field Name	Description
21:20	CIC/OSTA	When both Enable One step timestamp feature and Enable One step timestamp for PTP over UDP/IP feature are not selected, this field is interpreted as CIC. When Enable One step timestamp feature is selected without Enable One step timestamp for PTP over UDP/IP feature; when the OSTC bit is set, bit 20 is interpreted as OSTA and bit 21 is reserved. When the OSTC bit is reset, bits [21:20] is interpreted as CIC. When both Enable One step timestamp feature and Enable One step timestamp for PTP over UDP/IP feature are selected, this field is interpreted as CIC irrespective of OSTC bit setting. When the OSTC bit is set, bit 15 is interpreted as OSTPA. One-Step Timestamp/Packet ID Available (OSTPA) When this bit is set, it indicates the reference Timestamp is available in the TTSH and TTSL fields of TX control word. Checksum Insertion Control (CIC) These bits control the checksum calculation and insertion. The application must set the appropriate CRC Pad Control value when this control is enabled. The following list describes the bit encoding: ■ 2'b00: Checksum Insertion Disabled. ■ 2'b01: Only IP header checksum calculation and insertion are enabled ■ 2'b10: IP header checksum and payload checksum calculation and insertion are enabled but pseudo-header checksum is not calculated in hardware ■ 2'b11: IP Header checksum and payload checksum calculation and insertion are enabled and pseudo-header checksum is calculated in hardware This field is valid when the Enable Transmit TCP/IP Checksum Offload option or Peer-to-Peer PTP Transparent Clock (P2P TC) Message Support feature is selected.
19	OSTC	One-Step Timestamp Correction When this bit is set, the MAC performs the one-step timestamp correction with reference to the timestamp values provided in TTSH and TTSL fields of TX control word. This bit is valid when the Enable One-Step Timestamp Feature option is selected. When the OSTC and OSTPA bits are reset, the control word contains TBSDW0 and TBSDW1.
18	TTSE	Transmit Timestamp Enable When set, this bit enables the IEEE1588 hardware time stamping for the Transmit frame referenced by the descriptor. This bit is valid when the Enable IEEE 1588 Timestamp Support option is selected.
17:16	CPC	CRC Pad Control This field controls the CRC and Pad insertion for Tx frame.

Table 2-4 Packet Length and Packet Control Fields of the ATI Control Word (continued)

Field	Field Name	Description
		The following list describes the values of these bits: ■ 2'b00: CRC and Pad Insertion The MAC appends the cyclic redundancy check (CRC) at the end of the transmitted packet of length greater than or equal to 60 bytes. The MAC automatically adds padding and CRC to a frame shorter than 60 bytes. ■ 2'b01: CRC Insertion (Disable Pad Insertion) The MAC appends the CRC at the end of the transmitted packet but it does not append padding. The application must ensure that the padding bytes are present in the packet being transferred from the Transmit Buffer, that is, the packet being transferred from the Transmit Buffer is of length greater than or equal to 60 bytes. ■ 2'b10: Disable CRC Insertion The MAC does not append the CRC at the end of the transmitted packet. The application must ensure that the padding and CRC bytes are present in the packet being transferred from the Transmit Buffer. ■ 2'b11: CRC Replacement The MAC replaces the last four bytes of the transmitted packet with recalculated CRC bytes. The application must ensure that the padding and CRC bytes are present in the packet being transferred from the Transmit Buffer. When checksum insertion, SA-VLAN insertion, one-step timestamp, or Checksum Insertion Control features are enabled, the software must provide appropriate CRC Pad Control.
Packet Length		
15	OSTPA	One-Step Timestamp /Packet ID Available This bit is valid when the “Enable One-Step Timestamp Feature” option is selected. When OSTC bit and this bit are set, it indicates that the Ingress Timestamp is available in the TTSH and TTSL fields of Tx control word. When OSTC bit is reset and this bit is set, TTSL[9:0] contains the packet ID. Table 4-23 on page 218 indicates content of TTSH and TTSL fields of ATI Tx Control Word for various combinations of OSTC, TTSE, and OSTPA bits. When the OSTPA and OSTC bits are reset, the control word contains TBSDW0 and TBSDW1.
14:0	PL	Transmit Packet Length This field is equal to the length of the packet to be transmitted in bytes. This field must have correct packet length when the DWRR or WFQ algorithm is selected for Traffic scheduler (“Data Center Bridging” on page 324). This field is not used in the XGMAC-MTL configuration.

Table 2-5 provides description of VLAN Tag1 and VLAN Tag2 fields of ATI Control Word.

Table 2-5 VLAN Tag1 and VLAN Tag2 Fields of ATI Control Word

Field	Field Name	Description
31:16	VLT2	VLAN Tag 2 This field contains the inner VLAN Tag to be inserted or replaced in the transmitted packet. These bits are valid when the following are true: ■ The Enable SA and VLAN Insertion on Tx and Enable Double VLAN Processing options are selected ■ The VLTI bit is set in the MAC_VLAN_Incl register ■ The IVTIR field in Tx Control Word indicates Tag insertion or replacement.
15:0	VLT1	VLAN Tag 1 This field contains the outer VLAN Tag to be inserted or replaced in the transmitted packet. These bits are valid when the following are true: ■ The Enable SA and VLAN Insertion on Tx option is selected. ■ The VLTI bit is set in the MAC_VLAN_Incl register. ■ The VTIR field in Tx Control Word indicates Tag insertion or replacement.

Table 2-6 shows the description of Timestamp Low field of ATI Control Word.

Table 2-6 Timestamp Low Field of ATI Control Word

Field	Field Name	Description
31:0	TTSL/ TBSDW0	Transmit Packet Timestamp Low/Packet ID/TBS Dword0 TTSL: Transmit Packet Timestamp Low: When OSTC & OSTPA bits are set, this field contains the lower 32-bits of timestamp (Nanosecond field) when one-step timestamp correction is enabled. VDMAID: When TTSL[9:0] contains a valid Packet ID, the TTSL[(x+10):10] contains the information of the TX VDMA which transferred the packet from the host memory to the MAC for which the timestamp (captured in TS Status FIFO) is associated with. Range Variable[x]: DWCXG_TX_CHW Packet ID: When OSTC is reset and OSTPA is set, TTSL[9:0] contains the packet ID. This field is valid when you Enable One-Step Timestamp. TBSDW0: TBS Dword0: When OSTC bit is reset and you Enable Time Based Scheduling, this field contains the following information: Bit 31: LTV Launch Time Valid indicates the Launch Time (LT) and GSN fields present in this field are valid. Bit 30:12: Reserved

Table 2-6 Timestamp Low Field of ATI Control Word (continued)

Field	Field Name	Description
		Bit 11:8: GSN When LTV bit is set and you Enable Enhancements to Scheduling Traffic, this field indicates the GCL slot number associated with the packet. Bit 7:0: Launch Time (LT) in seconds When LTV bit is set, this field indicates the Launch Time (in seconds) associated with the packet.

Table 2-7 provides description of Timestamp High field of ATI Control Word.

Table 2-7 Timestamp High Field of ATI Control Word

Field	Field Name	Description
31:0	TTSH/ TBSDW1	Transmit Packet Timestamp High/TBS Dword1 TTSH: This field contains the higher 32-bits of timestamp (32-bit Seconds field) when you enable one-step timestamp correction. This field is validated by the OSTA field in Tx Control Word. This field is valid when you enable One-Step Timestamp Feature. TBSDW1: When both OSTC and OSTPA bits are reset and you enable Time Based Scheduling, the bit description is as follows: 31:8: Launch Time (LT) in nanosecond When LTV bit of the TBS Dword0 is set, this field indicates the Launch Time, in nanosecond, associated with the packet. Bit 7:0: Reserved

2.5.1.2 Operation Modes

The following two modes of operation trigger reading of the data towards the MAC:

- Threshold mode: In Threshold (or cut-through) mode, as soon as the number of bytes in the FIFO cross the configured threshold level (or when the end of packet is written before the threshold is crossed), the data is ready to be popped out and forwarded to the MAC. The threshold level is configured by using the TTC bits of Transmit Operation Mode register corresponding to an MTL queue.
- Store-and-forward mode: In store-and-forward mode, the MTL pops the packet towards the MAC only when one or more of the following conditions are true:
  - A complete packet is stored in the FIFO
  - The Tx FIFO becomes almost full
  - The ATI watermark becomes low

  The watermark becomes low when the requested queue does not have space to accommodate the requested burst length on the ATI. Therefore, the MTL never stops in the store-and-forward mode even if the Ethernet packet length is bigger than the Tx FIFO size.

The application can flush complete content of the Tx Queue by setting Bit 0 (FTQ) of Transmit Operation Mode register corresponding to an MTL queue. This bit is self-clearing and initializes the Queue pointers to the default state. If the FTQ bit is set during a packet transfer from the MTL to the MAC, the MTL stops further transfer because the queue is considered to be empty. Therefore, an underflow event occurs at the MAC transmitter.

For information about the initialization and transmit operations for the MTL Layer, see "Initialization Flow" on page 98 through "Transmit Path Timing" on page 102. The timing diagrams are provided in "Transmit Path Timing" on page 102.

In the half-duplex mode of operation, a collision event can occur on the MAC line interface when a packet is transferred from MTL to the MAC.

The MAC indicates a retry attempt to the MTL by sending the status even before transmitting the EOP. On receiving the status, the MTL retransmits the packet, by popping it from the queue. After more than 96 bytes are sent to the MAC, the Queue Controller frees up the space and makes it available to the application or the DMA to push more data. This implies that the retransmission is not possible after this threshold or when the MAC indicates a late-collision event. When a packet transmission is terminated because of under flow and a collision event follows immediately (initiating a retry), retry has higher priority than terminate.

2.5.1.3 Initialization Flow

Upon reset, the MTL is ready to manage the flow of data to and from the application/ DMA (if present), and the MAC.

For configurations with a single Transmit queue, there are no initialization requirements for enabling the MTL. For configurations with multiple Transmit queues, initialize the queue size for each of the queues by programming the TQS field and enable the queue by programming the TXQEN field of the MTL_TxQ(#i)_Operation_Mode register corresponding to the transmit queue. Also, initialize the MAC block as described in "Initializing the MAC Registers" on page 1841.

2.5.1.4 Transmit Operation

The general sequence of events for a Transmit operation is as follows:

1. When a packet is transmitted, the packet control word and the packet data are received on the ATI interface.
2. The received data is pushed into the corresponding queue. This process continues until the EOP is transferred.
3. When the threshold level is crossed or a full packet of data is received into the queue. The transmit queue and the read controller read the packet data and drive it to the MAC.
4. The transmit queue controller continues to transfer data from the queue until a complete packet is transferred to the MAC. When the packet transfer is complete, the MTL waits for the next packet.
5. In XGMAC-MTL configuration, the application can transfer the next packet as soon as MTL is ready to accept data, which is indicated by asserting the `ati_ready_o` signal.

2.5.1.5 Data Flow Between the Application and the MTL

This section describes the interaction process between the MTL and the DMA in the XGMAC-AXI configuration.

Transmit Queue Flush Operation

The 25GMAC provides a control to the software to flush the Tx Queue in the MTL layer through Bit 0 (`FTQ`) of Operation Mode Register of corresponding queue. The Flush operation is immediate. The MTL clears the Tx Queue and the corresponding pointers to the initial state even if it is in the middle of transferring a packet to the MAC. The data already accepted by the MAC transmitter is not flushed. It is scheduled for transmission and results in underflow because Tx Queue does not complete the transfer of rest of the packet. As in all underflow conditions, a runt packet can be transmitted and observed on the line.

The MTL layer also stops accepting any data from the application or DMA during the Flush operation. If it is in the middle of packet transfer to the MAC, it waits for the status from the MAC, before completing the flush operation. Once the flush operation is complete, the MTL starts accepting new packets from the application or the DMA. All data presented to the MTL after a Flush operation is discarded unless it starts with a control word and an SOP marker.

2.5.1.6 Transmit Status Word

At the end of transfer of the Ethernet packet to the MAC and after the MAC completes the transmission of the packet, the MTL provides the Transmit status validity by asserting `ati_txstatus_val_o` on respective queues. For configurations with a single queue, `ati_txstatus_o` indicates the status of the queue when `ati_txstatus_val_o` is asserted. The application can read the status by asserting the `ati_txstatus_ack_i` signal. For configurations with multiple Tx queues, the respective `ati_txstatus_val_o` signal indicates the availability of status on individual queues and application must read the status on `ati_txstatus_o[17:0]` by driving the Tx queue number on `ati_txsqnum_i` and by asserting `ati_txstatus_ack_i`.

The detailed description of the Transmit Status is the provided in Table 2-8. If IEEE 1588 Timestamp feature is enabled, the MTL returns 64-bit timestamp captured for the packet, along with the Transmit status of ATI on `ati_txstatus_o[81:18]`. A valid timestamp is indicated by an active high on `ati_txstatus_o[17]`. If the Disable Transmit Status in MTL option is selected while configuring the controller, the Transmit Status is not provided to the application. When this option is selected, the application can push as many packets as possible into the Tx queues because the application is not

required to read the status words from the Transmit Status FIFO. If this option is selected, you can program Bit 1 (FTS) of MTL_Operation_Mode register to 0 to disable the dependency of the application to read the status words from the Status FIFO.

Table 2-8 Transmit Status Word

Field	Name	Description
81:50	TTSH	Transmit Packet Timestamp High The MTL updates this field with the most significant 32 bits of the timestamp captured for corresponding Transmit packet. This field is valid only if the TTSS bit is set. This field is present when the Enable IEEE 1588 Timestamp Support option is selected.
49:18	TTSL	Transmit Packet Timestamp Low The MTL updates this field with the least significant 32 bits of the timestamp captured for corresponding Transmit packet. This field is valid only if the TTSS bit is set. This field is present when the Enable IEEE 1588 Timestamp Support option is selected.
17	TTSS	Tx Timestamp Status This status bit indicates that a timestamp has been captured for the corresponding Transmit packet. When this bit is set, TTSH and TTSL have timestamp values that were captured for the Transmit packet. This bit is valid only when the Enable IEEE 1588 Timestamp Support option is selected. Otherwise, this bit is reserved.
16	Rsvd	Reserved
15	ES	Error Summary This bit indicates the logical OR of the following bits: ■ Bit[0]: IP Header Error ■ Bit[14]: Jabber Timeout ■ Bit[13]: Packet Flush ■ Bit[12]: Payload Checksum Error ■ Bit[11]: Remote Fault ■ Bit[10]: Local Fault ■ Bit[3]: TBS drop ■ Bit[2]: Underflow Error
14	JT	Jabber Timeout This bit indicates that the MAC transmitter has experienced a jabber time-out error.

Table 2-8 Transmit Status Word (continued)

Field	Name	Description
		This bit is set when: ■ JD bit of the MAC_Tx_Configuration register is set and transmit packet received from application is of size greater than 16383 bytes ■ JD bit and JE bit of MAC_Tx_Configuration register is reset and set respectively and transmit packet received from application is of size greater than 10240 bytes ■ JD bit and JE bit of MAC_Tx_Configuration register are reset and transmit packet received from application is of size greater than 2048 bytes
13	FF	Packet Flushed This bit indicates that the MTL flushed the packet because of a software flush command issued by the application.
12	PCE	Payload Checksum Error This bit indicates that the Checksum Offload engine had a failure and did not insert any checksum into the encapsulated TCP, UDP, or ICMP payload. This failure can be either because of insufficient bytes, as indicated by the Payload Length field of the IP Header or the MTL starting to forward the packet to the MAC transmitter in Store-and-Forward mode without the checksum having been calculated yet. This second error condition only occurs when the Transmit Queue depth is less than the length of the IP packet being transmitted to avoid deadlock, the MTL starts forwarding the packet when the FIFO is full, even in the store-and-forward mode. When the Transmit Checksum Offload engine is not selected, this bit is reserved.
11	LOC	Loss of Carrier This bit indicates that loss of carrier occurred while transmitting the packet; that is, the gmii_crs_i signal was inactive for one or more transmission clock periods during packet transmission. This bit is valid only for packets transmitted without collision in the half-duplex mode.
10	NC	No Carrier This bit indicates that the carrier signal from the PHY is not present at the end of the preamble transmission. This bit is valid only when the MAC operates in the half-duplex mode.
9	LC	Late Collisions When set, this bit indicates that the packet transmission was terminated because of a collision after the collision window (64 bytes including Preamble in MII mode). This bit is not valid if the Underflow error is set.
8	EC	Excessive Collisions When set, this bit indicates that the transmission is terminated after 16 successive collisions while attempting to transmit the current packet. If register is set, this bit is set after the first collision. The transmission of the packet is terminated.
7:4	CC	Collision Count

Table 2-8 Transmit Status Word (continued)

Field	Name	Description
		This 4 bit counter indicates the number of collisions that occurred before the packet was transmitted. The count is not valid when the Excessive Collisions bit is set.
3	ED	Excessive Deferral This bit is set when Deferral Check Bit is set in the MAC_Extended_Configuration register. When set, this bit indicates that either the transmission has ended because of excessive deferral of over 24,288 bit times or Jumbo packet is enabled).
2	UF	Underflow Error This bit indicates that the MAC terminated the packet because the data arrived late from the system memory. The underflow error can occur because of the application filled the MTL Tx Queue slower than the MAC transmit rate. The MTL sets the underflow bit corresponding to a queue in the MTL_Interrupt_Status register.
1	Deferred	When set, this bit indicates that the MAC defers before transmission because of the presence of carrier.
0	IHE	IP Header Error When IP Header Error is set, this bit indicates that the Checksum Offload engine detected an IP header error. When the Transmit Checksum Offload engine is not selected, this bit is reserved. If Tx COE detects an IP header error, it still inserts an IPv4 header checksum if the Ethernet Type field indicates an IPv4 payload.

Note When the Enable IEEE 1588 Timestamp Support option is selected, the width of ati_txstatus_o[] is 82. Otherwise, the width of the ati_txstatus_o is 18.

When the Enable IEEE 1588 Timestamp Support option is selected, the upper 64-bits indicate the timestamp captured for the packet if the option is enabled through ATI Tx Control Word 0 (Bit 19).

2.5.1.7 Transmit Path Timing

This section describes the timing specifications for the MAC Transaction layer (MTL) interface signals in the Transmit path.

Application Transmit Interface

Figure 2-24 illustrates the timing specifications for the Application Transmit Interface (ATI) signals. The handshake mechanism involved in packet transmission is as follows:

1. Before starting a burst transfer of data to ATI, the application must check whether the MTL Tx queue has space to accommodate the burst. The length of the proposed burst transfer is put on `ati_qnum_pbl_i` on the corresponding queue.
2. In response, the MTL asserts or de-asserts `ati_txwatermark_o` on the corresponding queue one clock later, indicating whether it can accommodate the burst data transfer or not.

3. At any point during the transfer, the MTL can alert the application to stop the transfer by de-asserting the `ati_rdy_o` signal. The `ati_rdy_o` signal is de-asserted only when either of the following happens:
   - The queue is almost full
   - The Tx queue is being flushed or checksum is being written if the Enable Transmit TCP/IP Checksum Offload option is selected

     The Tx queue-full event does not occur if the application ensures that enough space is available before the start of data transfer, as described in steps 1 and 2.
4. The application starts a packet transfer by asserting the `ati_ctrl_val_i` and `ati_ctrl_type_i` signals (for writing the control word) followed by the `ati_val_i` and `ati_sop_i` signals. The MTL accepts the control word or data whenever `ati_rdy_o` and `ati_val_i` are asserted.

   In configurations with multiple Tx queues, the `ati_qnum_i` signal must be associated with the `ati_ctrl_val_i` and `ati_val_i` signals to indicate the Tx queue to which the control word or data is addressed.
5. When the application wants to transfer the last bytes of a packet, `ati_eop_i` must be asserted along with the byte enables (`ati_be_i`), indicating the valid number of data bytes on the bus. All data transfers except the EOP transfer must have the byte-enables as all-ones.
6. If `ati_rdy_o` is asserted, the application can transfer the next packet.

Figure 2-24 Application Transmit Interface (ATI) Timing

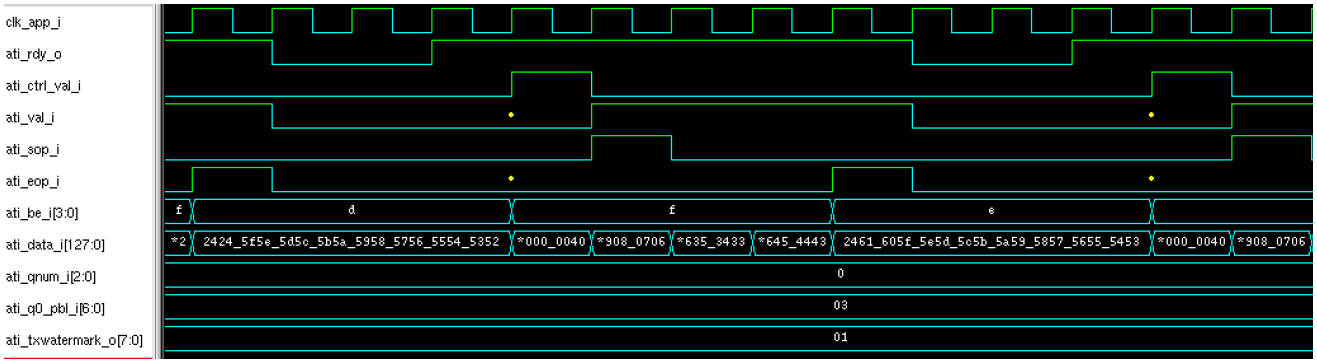


Figure 2-25 ATI Interface Timing with ati_ctrl_val_i, 64-bit with OST Enabled

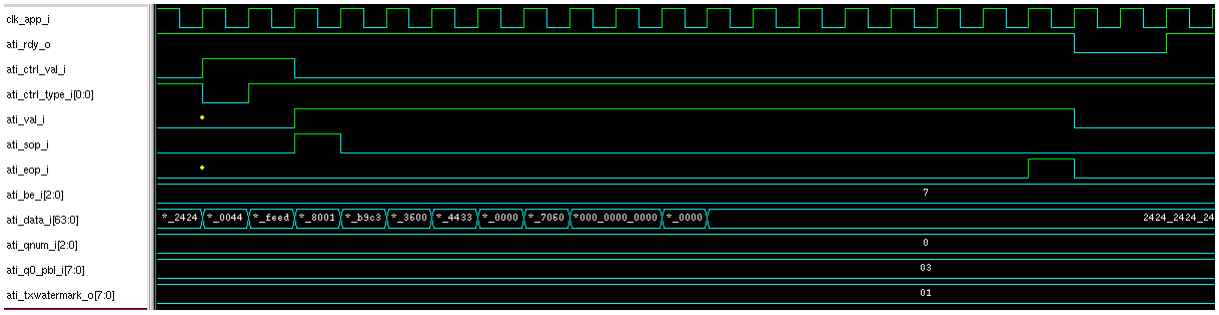
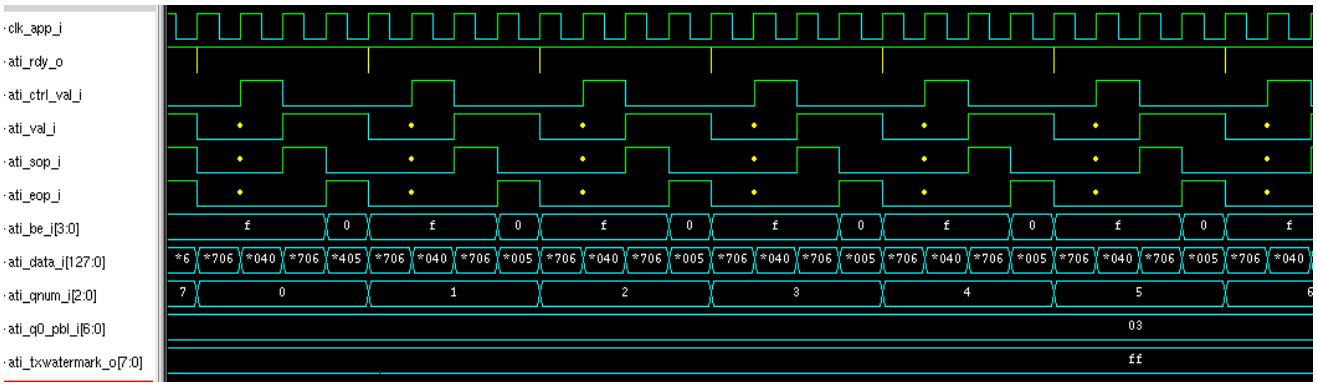


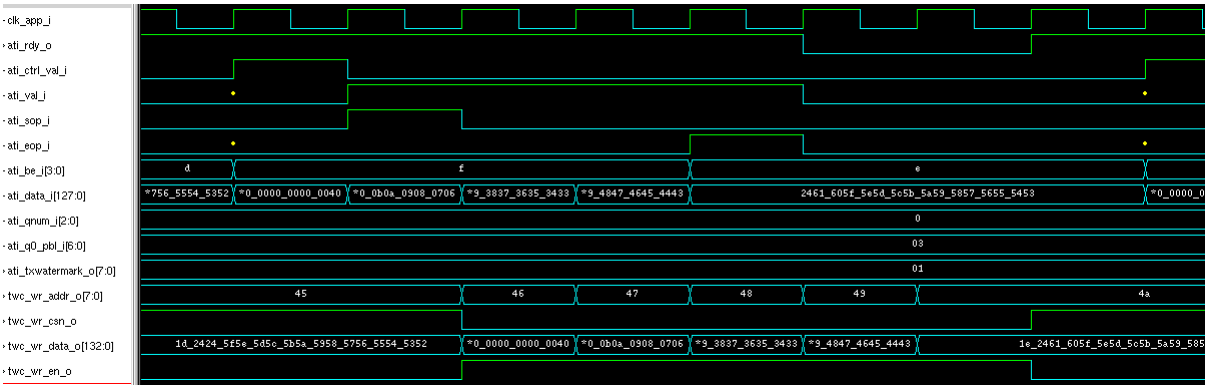
Figure 2-26 ATI Interface Timing, 128-bit SNF Mode Multiple Queue



Transmit Queue Write Interface

The MTL Tx Queue controller transfers the data accepted through ATI to the external memory. Timing specifications for the memory interface signals (`twc_wr_*` signals) are illustrated in Figure 2-27. This figure also shows the timing relationship of the memory interface signals with respect to the ATI signals.

Figure 2-27 Transmit Queue Write Interface Timing



The external memory used for storing packet is synchronous 2-port RAM.

Transmit Queue Read Interface

The Tx Queue Read controller reads the data from the Tx queue and transfers it to the MAC on the MAC Transmit Interface (MTI). The timing relationship of the MTI signals with the Transmit Queue Read Interface signals is shown in Figure 2-28.

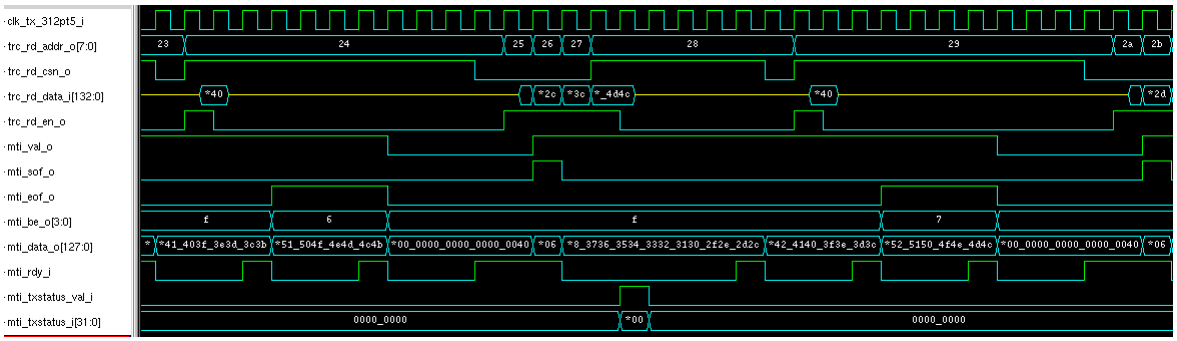
The sequence of events during the transfer of a packet from Tx Queue to the MTI is as follows:

1. When the Tx queue has data (not shown in the diagram), the Read Controller starts the read transfers by asserting the `trc_rd_addr_o` and `trc_rd_csn_o` signals. The assertion of `trc_rd_en_o` is delayed by 1 clock cycle as per synchronous RAM timing requirements. The first location of the packet has Packet Control and Packet Length. These are required to assert some of the sideband signals (such as `mti_crc_pad_ctrl_i`) of the MAC.

> **Note** In configurations with multiple Tx queues, the arbitration scheme selects the non-empty Tx queue for transmission. Before Tx queue arbitration, the first location from each Tx queue is prefetched (containing the packet control and packet length). The packet length information is required for Tx queue arbitration (WFQ and DWRR arbitration schemes).

2. The data is read on the `trc_rd_data_i` bus in the next clock cycle. Because the data is directly transferred to the `mti_data_i` bus to the MAC (MTI), `mti_val_o` is also asserted. The `mti_sop_o` signal is also asserted along with `mti_val_o` in Figure 2-28 because the data corresponds to start of packet.
3. The Tx Queue Read Controller can pre-fetch data from three locations so that it can sustain continuous transfers to the MTI interface without any delay.
4. If the MAC accepts the data (active mti_rdy_o), more data is read from the queue. If the MAC de-asserts `mti_rdy_i`, the read operations are suspended in the next clock cycle and `trc_rd_csn_o` is de-asserted.

Figure 2-28 Transmit Queue Read Interface Timing



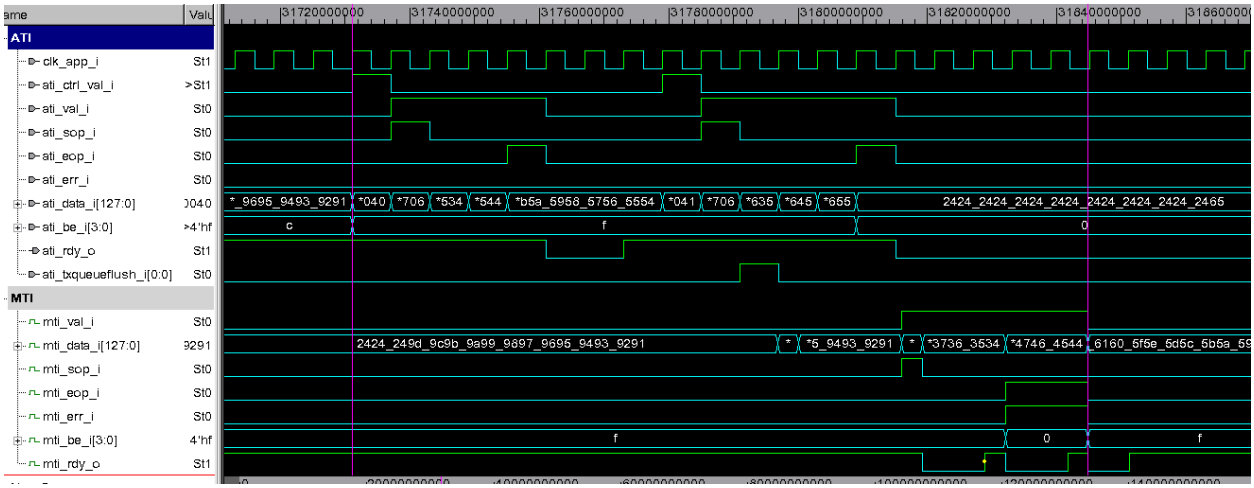
Transmit Queue Flush Operation

The MTL allows a Tx Queue to be flushed at any moment. When the Tx Queue Flush command is issued, the 25GMAC flushes out all the packets in the corresponding Tx Queue (including the currently ongoing packet transfer on the ATI interface).

The Flush command can be issued either by setting 1 on bit[0] of `MTL_TxQ(#i)_Operation_Mode` register or driving `ati_txqueueflush_i[n]` signal for one clock period. The second option is available only in XGMAC-MTL configuration.

Figure 2-29 shows the Tx Queue Flush operation using `ati_txqueueflush_i[n]` as an example. However the flow is same when the Tx Queue is flushed using register bit.

Figure 2-29 Transmit Queue Flush Timing



1. The application pushes two packets into the Tx Queue on the ATI Interface.
2. While transferring the second packet the application issued a flush command by asserting the `ati_txqueueflush_i[n]` signal.
3. There are two packets in the Tx Queue when the Flush command is issued.
4. The first packet transmission started at the MTI Interface, however it got truncated by forcing the CRC error for that packet by asserting `mti_err_i` and `mti_eof_i`.
5. The second packet internally got flushed out by the MTL.

So, the Tx Queue flush command ensures that all the previous packets residing in the Tx Queue gets flushed out. The new packets that arrive after the flush packets are sent out. The new packets can be scheduled for transmission at any time after the flush command is issued.

In case the flush is issued by using a register bit (bit[0] of `MTL_TxQ(#i)_Operation_Mode`), the application can issue the next flush command only when this register bit reads as 0 (when the flush is completed by MTL internally).

Application Transmit Interface with Transmit Status Enabled

The following figures show the timing specifications for the Application Transmit Interface (ATI) signals with Transmit Status Enabled. The handshake mechanism involved in packet transmission is as follows:

1. Before starting a burst transfer of data to ATI, the application must check whether the MTL Tx queue has space to accommodate the burst. The length of the proposed burst transfer is put on `ati_qnum_pbl_i` on the corresponding queue.
2. In response, the MTL asserts or de-asserts `ati_txwatermark_o` on the corresponding queue one clock later, indicating whether it can accommodate the burst data transfer or not.
3. At any point during the transfer, the MTL can alert the application to stop the transfer by de-asserting the `ati_rdy_o` signal. The `ati_rdy_o` signal is de-asserted only when either of the following occurs:
   - The queue is almost full
   - The Tx queue is being flushed or checksum is being written if the Enable Transmit TCP/IP Checksum Offload option is selected

The Tx queue-full event does not occur if the application ensures that enough space is available before the start of data transfer, as described in Steps 1 and 2

4. The application starts a packet transfer by asserting the `ati_ctrl_val_i` and `ati_ctrl_type_i` signals (for writing the control word) followed by the `ati_val_i` and `ati_sop_i` signals. The MTL accepts the control word or data whenever `ati_rdy_o` and `ati_val_i` are asserted. In configurations with multiple Tx queues, the `ati_qnum_i` signal must be associated with the `ati_ctrl_val_i` and `ati_val_i` signals to indicate the Tx queue to which the control word or data is addressed
5. When the application wants to transfer the last bytes of a packet, `ati_eop_i` must be asserted along with the byte enables (`ati_be_i`), indicating the valid number of data bytes on the bus. All data transfers except the EOP transfer must have the byte-enables as all-ones
6. If `ati_rdy_o` is asserted, the application can transfer the next packet
7. In configurations with a single queue, the MTL gives the Transmit status of the packet to the application through `ati_txstatus_o` by asserting `ati_txstatus_val_o`. The application accepts the status by asserting `ati_txstatus_ack_i`. In configurations with multiple queues, the MTL asserts the `ati_txstatus_val_o` to indicate the status availability on the corresponding queue. The application must read the status on `ati_txstatus_o` by driving Tx queue number on `ati_txsqnum_i` and asserting `ati_txstatus_ack_i`.
8. The application need not wait for steps 6 and 7 to start the transfer of next packet on the ATI. It can go to step 2 by driving the new PBL on `ati_q#_pbl_i` to check for the watermark.

Figure 2-30 Application Transmit Interface (ATI) Timing with Transmit Status Enabled

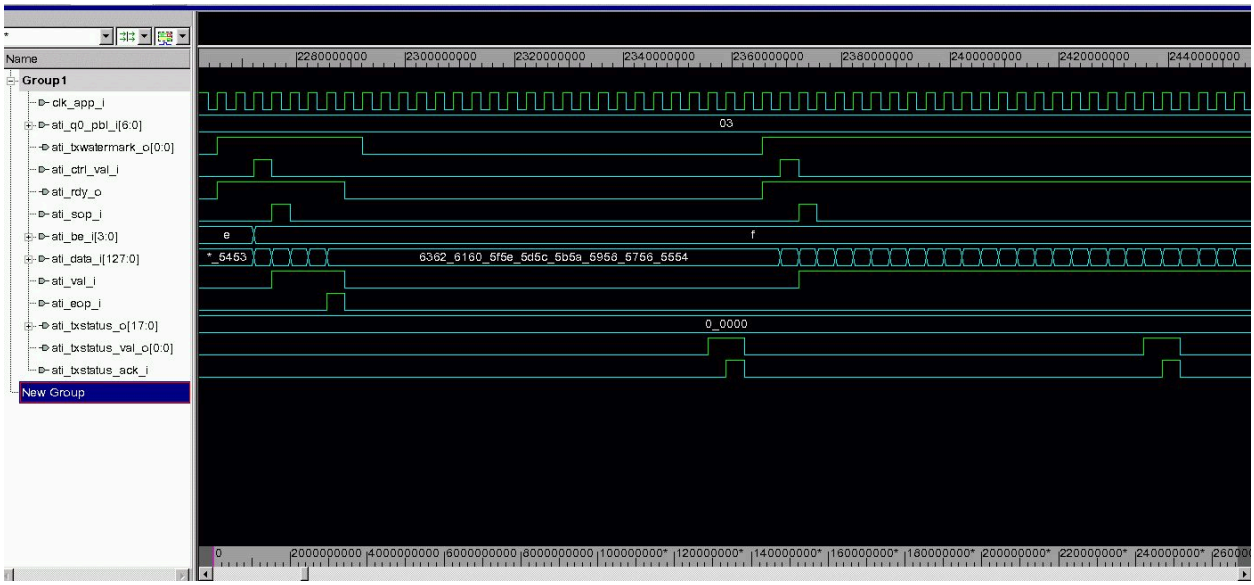


Figure 2-31 ATI Interface Timing with IEEE 1588 Timestamp Feature Selected and Transmit Status Enabled

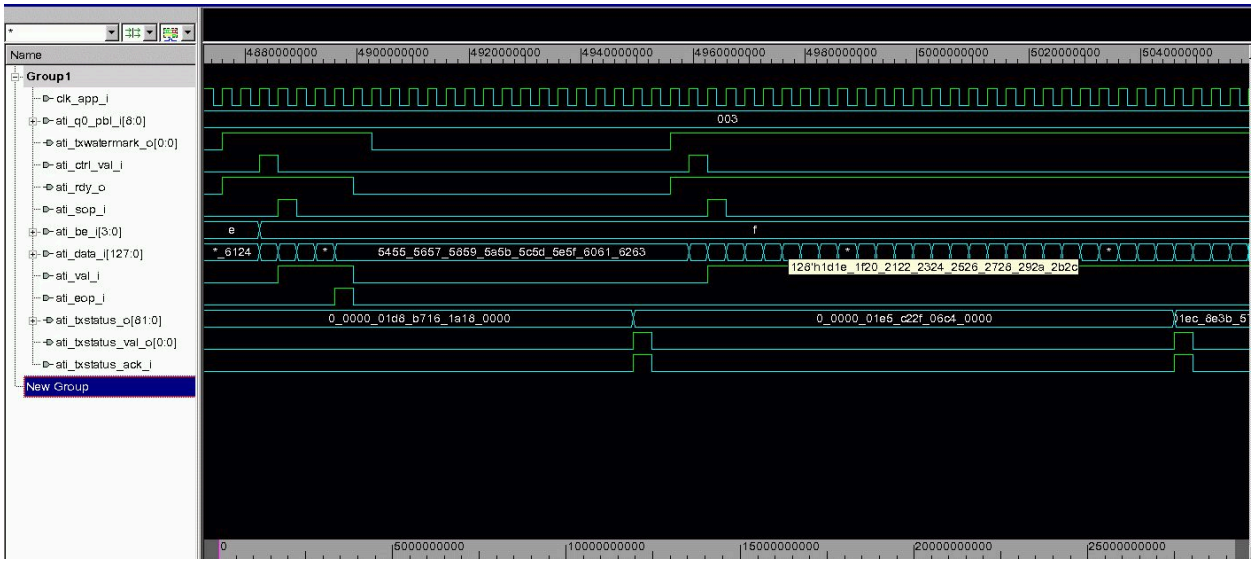
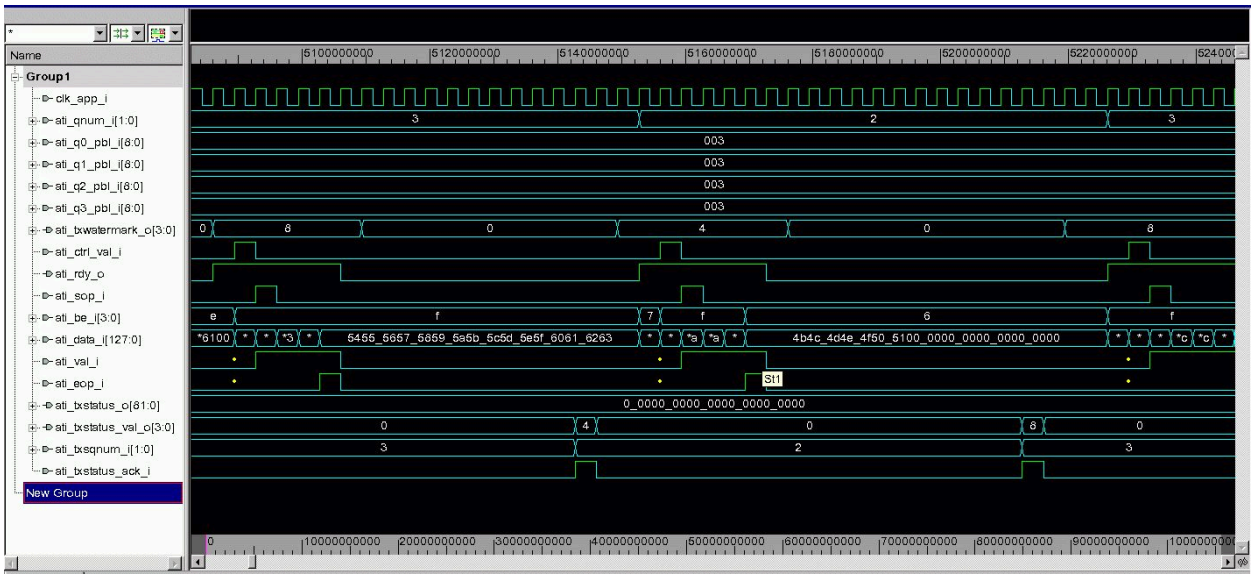


Figure 2-32 ATI Interface Timing with Multi Queue and Transmit Status Enabled



2.5.2 Receive Path

The Rx module receives the packets given by the MAC and pushes them into the Rx Queue. The status (fill level, through PBL and watermark) of the queue is indicated to the application or DMA when it crosses the configured Receive threshold (Bits[1:0] of `MTL_RxQ(#i)_Operation_Mode` register), in threshold mode or when a complete packet is received in store and forward mode. The DMA can use the fill level of the queue to initiate pre-configured burst transfers towards the AXI interface.

Sections "Receive Operation" on page 109 through "Receive Status Word" on page 111 describe receive operations for the MTL Layer. Timing diagrams are provided in "Receive Path Timing Diagrams" on page 113.

2.5.2.1 Receive Operation

During Receive operation, the MTL is a subordinate for the MAC. The general sequence of events is as follows:

1. When the MAC receives a packet, it indicates the availability of receive data along with byte enables to MTL.
2. The MAC indicates the SOP and EOP delimiters.
3. The MTL accepts the data and pushes it into corresponding Rx queue.
4. After the EOP is transferred, the MAC drives the status word which is also pushed into the corresponding Rx queue by the MTL.

> **Note** In Threshold (cut-through) mode, the status words are stored after the packet EOP. In store-and-forward mode, the location for the maximum status words are reserved before writing the SOP. The status is written to reserved locations (before SOP in the Rx queue) after writing the EOP.

5. If IEEE 1588 timestamp feature is enabled and the 64-bit timestamp is available along with the packet status, it is pushed into the Rx queue as a part of the status word. In 64-bit or 128-bit mode, one additional location is taken.
6. The MTL takes the data out of the queue and sends it to the application or DMA depending on the mode:
   - Threshold mode: In the (default) Threshold mode, the MTL reads the data and indicates its availability to the application or DMA when one of the following happens:
     i. data bytes equal to the threshold amount are written to the Rx queue (Bits[1:0] of `MTL_RxQ(#i)_Operation_Mode` register)
     ii. a full packet of data is received into the queue

     When the application or DMA initiates the transfer, the MTL continues to transfer the data from the queue until a complete packet is transferred. When EOP transfer is complete, the MTL pops out the status word and sends it to the application or DMA controller. If 64-bit timestamp is read is available for that packet, then it is given out next and is marked by the assertion of the ari_timestamp_val_o signal on the ARI.

     In this mode, some error packets are not dropped because the error status is received at the end of packet by which time the start of that packet has already been read out of the queue.
   - Store-and-forward mode: In the store-and-forward mode (`MTL_RxQ(#i)_Operation_Mode` register, Bit 5 is set), the initial Rx queue locations are reserved for the status words before writing the SOP. A packet is read out only after it is completely written into the Rx queue.

     In this mode
     - The status (including timestamp if present) is driven out before the SOP of the packet is transferred on the ARI.
     - All error packets are dropped (if configured through Bit 4 of the Receive Operation Mode register of a queue) such that only valid packets are read and forwarded to the application.

> **Note** In configurations with multiple Rx queues, before sending the packet data to the application or DMA on ARI interface, the MTL layer uses the Weighted Strict Priority (WSP)/Strict Priority scheme to perform the arbitration among the queues that have enough data. The selected queue number is indicated through ari_qnum_o[]. All other ARI interface signals are associated with respect to the ari_qnum_o[].

2.5.2.2 Receive Operation Multi-Packet Handling

In Threshold mode, the status is available immediately after the data. In store-and-forward mode, the data is available after the status. The MTL is capable of storing any number of packets into the queue as long as it is not full.

If the MAC receives a packet when the corresponding Rx queue is full, the MTL ignores that packet and the `ari_fifo_ovf_o` output pulse is generated on the corresponding Rx queue. In addition, the MTL increments the overflow counter in the Overflow Counter Register of corresponding queue.

2.5.2.3 Error Handling

If the MTL Rx queue is full before it receives the EOP data from the MAC,

1. An overflow is declared
2. The whole packet (including the status word) is dropped
3. The overflow counter of the corresponding queue is incremented.

This is true even if Bit 4 (FEF) of Receive Operation Mode register of corresponding MTL queue is set.

In threshold mode, if the start address of such a packet has already been transferred to the Read Controller, the rest of the packet is dropped and a placeholder EOP is written to the queue along with the status word with overflow status. The status indicates a partial packet because of overflow. In such packets, the Packet Length field is invalid. If the MTL Receive Queue is configured to operate in the store-and-forward mode and the length of the received packet is more than the queue size, overflow occurs and all such packets are dropped.

The MTL Rx Control logic can filter error and undersized packets, if enabled by using the FEF and FUF bits of Receive Operation Mode register of corresponding MTL queue. If the start address of such a packet has already been transferred to the Rx Queue Read Controller, that packet is not filtered. The start address of the packet is transferred to the Read Controller after the packet crosses the receive threshold set by Bits[1:0] of Receive Operation Mode register of corresponding MTL queue.

If the MTL Receive Queue is configured to operate in the store-and-forward mode, all error packets can be filtered and dropped. For the application or DMA to flush the error packet being read from the queue, it must assert the `ari_packetflush_i` signal. The MTL then stops transferring data to the application (DMA). It internally reads the rest of the packet and drops it. The MTL then starts the transfer of next packet (if available.)

2.5.2.4 Receive Status Word

For every packet, the MTL provides the receive status (`ari_data_o[DW-1:0]`) to the application. This is indicated by an active `ari_rxstatus_val_o` signal. The Receive Status format on ARI is provided in Table 2-9. The fields mentioned in the table have the same description as given for the RxDMA Descriptor status in "Receive Normal Descriptor (Write-Back Format)" on page 1813 and "Receive Context Descriptor (Write-Back Format)" on page 1825. The modified/updated fields are shown in bold.

- When ARI width is 64 bits, the bottom two rows are given on `ari_data_o` bus, followed by the middle two rows when present.
- When ARI width is 128 bits, the bottom four rows are given on `ari_data_o` as the status.

In both configurations (64-bit or 128-bit), the top two rows are given on `ari_data_o` after the last status and qualified with active `ari_timestamp_val_o`.

Table 2-9 ARI Receive Status Format

<table>
<tr><th>31</th><th></th><th></th><th></th><th></th><th></th><th></th><th></th><th>0</th></tr>
<tr><td colspan="9">Timestamp Low[31:0]
(available & valid only when TSA =1 & TSD=0)</td></tr>
<tr><td colspan="9">Timestamp High[31:0](available & valid only when TSA = 1 & TSD=0)</td></tr>
<tr><td colspan="9">Reserved[31:0] Available whenL2 Type indicates that it is either VLAN or DVLAN</td></tr>
<tr><td colspan="5">IVLANTag[15:0]
■ Available in 64-bit or valid in 128-bit only when L2 Type indicates that it is of DVLAN packet</td><td colspan="4">OVLAN Tag[15:0]
■ Available in 64-bit or valid in 128-bit only when L2 Type indicates that it is of VLAN or DVLAN packet</td></tr>
<tr><td colspan="5">MAC Filter Status[31:16]</td><td>VF</td><td>RPNG</td><td colspan="2">Reserved[13:0] or {ITS, Reserved[12:0]}</td></tr>
<tr><td>TDS</td><td>Rsvd</td><td>PTP_TYPE [29:26]</td><td>TSA</td><td>Packet Status [24:16]</td><td>ES</td><td>Rsvd</td><td colspan="2">Packet Length[13:0]</td></tr>
</table>

The Status Word format in the Receive path is similar to RDES3, RDES2, and RDES0 (described in "Receive Normal Descriptor (Write-Back Format)" on page 1813) except that the Status Word RDES3[31:25] has the following changes:

Table 2-10 Status Word Format for Receive Path - Changes from RDES3

RDES3 Bit	Name	Description
31	TSD	Timestamp Dropped When set, it indicates that a timestamp was captured for this packet but was dropped due to the lack of space in Rx FIFO. The host must consider the RTSL and RTSH as valid only when TSD=0 and TSA=1
30	Rsvd	Reserved

Table 2-10 Status Word Format for Receive Path - Changes from RDES3 (continued)

RDES3 Bit	Name	Description
29:26	PTP_TYPE (PMT)	PTP Message Type: These bits are encoded to give the type of the PTP message in the received packet: ■ 0000: No PTP message received ■ 0001: SYNC (all clock types) ■ 0010: Follow-up (all clock types) ■ 0011: Delay_Req (all clock types) ■ 0100: Delay_Resp (all clock types) ■ 0101: Pdelay_Req (in peer-to-peer transparent clock) ■ 0110: Pdelay_Resp (in peer-to-peer transparent clock) ■ 0111: Pdelay_Resp_Follow_Up (in peer-to-peer transparent clock) ■ 1000: Announce ■ 1001: Management ■ 1010: Signaling ■ 1011-1110: Reserved ■ 1111: PTP packet with Reserved message type
25	TSA	Timestamp Available When set, this bit indicates that the timestamp is available after the Rx Status word.

64-Bit Data Width Configurations

For 64-bit data width configurations, the Status Word format is given in Table 2-11.

Table 2-11 Status Word For 64-Bit Configurations

Status Words	Description
{RDES2,RDES3}	The first status word that is driven out on ari_data_o[] for a packet.
{Reserved,RDES0}	This status word is driven on the ARI interface only when the following conditions are met: ■ LT field indicates a tagged packet ■ Providing VLAN tag in the Rx status is enabled in the MAC receiver. Otherwise, this status word contains all zeros.
{Timestamp High, Timestamp Low}	When Bit 25 of the RDES3 is set to 1 this field has the 64-bits of the timestamp status. This status word is driven on the ARI interface only when Bit 25 (TSA) of the RDES3 word is set to 1 and Bit 31 (TSD) is cleared. Otherwise, this status word is not present for the packet.

128-Bit Data Width Configurations

For 128-bit data width configurations, the Status Word format is given in Table 2-12.

Table 2-12 Status Word For 128-Bit Configurations

Status Words	Description
{RDES0, RDES2, RDES3}	The first status word that is driven out on ari_data_o[] for a packet.
{Timestamp High, Timestamp Low}	When Bit 25 of the RDES3 is set to 1, this field has the 64-bits of the timestamp status. Upper 64 bits are reserved. This status word is driven on the ARI interface only when TSA (Bit 25) = 1 and TSD (Bit 31) = 0. Otherwise, this status word is not present for the packet.

2.5.2.5 Receive Path Timing Diagrams

Timing specifications for the MAC Transaction layer (MTL) interface signals in the receive path are illustrated in detail in the following sections. The timing diagrams are organized to show the sequence of transactions that occur when a received packet is transferred from the MAC to the application.

2.5.2.6 Receive FIFO Write Interface

A packet received by the MAC is transferred to the MTL over the MRI interface. The transferred data is accepted and written to the Rx Queue by the write control logic.

1. The MAC starts transferring the received packet to the MTL by asserting `mri_val_o` and `mri_sop_o` along with the corresponding 128-bit data on `mri_data_o`.

   This is directly transferred to the Rx queue in the next clock cycle. This can be seen by the assertion of `rwc_csn_o` and `rwc_wr_en_o` along with the corresponding address and data. The MSB 5 bits of `rwc_wr_data` correspond to the value on `mri_eop_o` and `mri_be_o` signals accepted by the MTL.

   In store-and-forward mode, MTL reserves locations (0x0121, 0x0122 in Figure 2-33) for the status word before writing the first packet data (in location 0x0123 in Figure 2-33).

2. The MAC completes the packet transfer by the asserting the mri_eop_o signal along with the valid data. The Receive packet status on `mri_rxstatus_o` is also valid with `mri_eop_o`.

3. After writing the EOP data, the Rx Queue Write logic transfers the received status to the external RAM in the next clock cycles in locations 0x0121 and 0x0122.

   In Threshold mode, the status words are written after the frame data and therefore, no locations are reserved before writing the SOF data.

Figure 2-33 Receive Queue Write Interface Timing in 128-bit Configuration

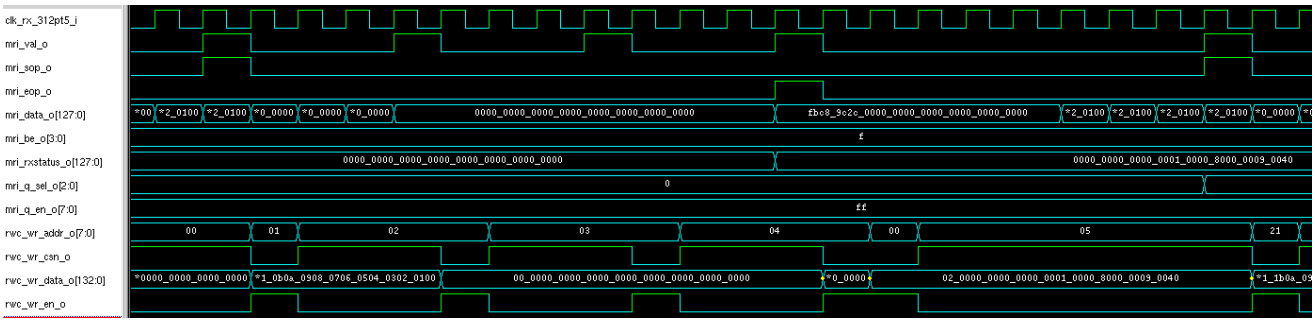


Figure 2-36 and Figure 2-37 show timing for the XGMII receive interface to MTL RXFIFO interface in 128-bit and 64-bit XGMII and GMII configurations. The PHY can safely switch off the RX clock after the RX frame and corresponding status is stored in the MTL Rx FIFO. The sequence of events is as follows:

1. The MAC receives frames on the XGMII.
2. The MAC stores the frame in the MTL RXFIFO.
3. The MAC stores the receive status in the MTL Rx FIFO after the EOF is stored.

In GMII mode, the 64-bit and 128-bit data bus configurations take 14 clock cycles of RX clock. In XGMII mode, the 64-bit and 128-bit data bus configurations takes less than the IEEE specified 128 number of clock cycles of the RX clock:

Figure 2-34 Receive Interface, XGMII to RX FIFO Timing (64-bit Data Bus, XGMII in DDR)

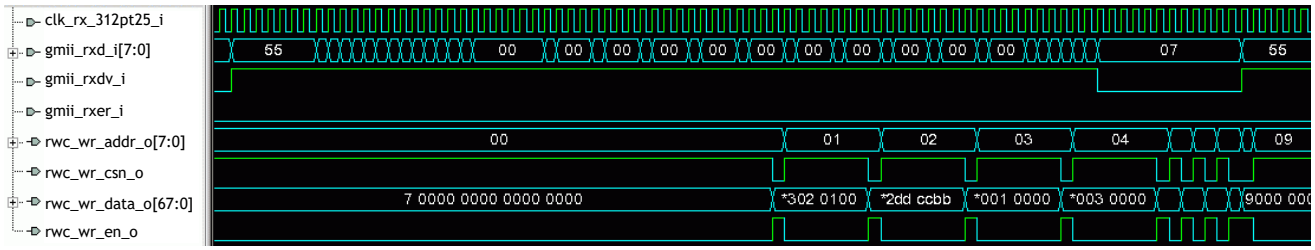


Figure 2-35 Receive Interface, XGMII to RX FIFO Timing (128-bit Data Bus, XGMII in DDR)

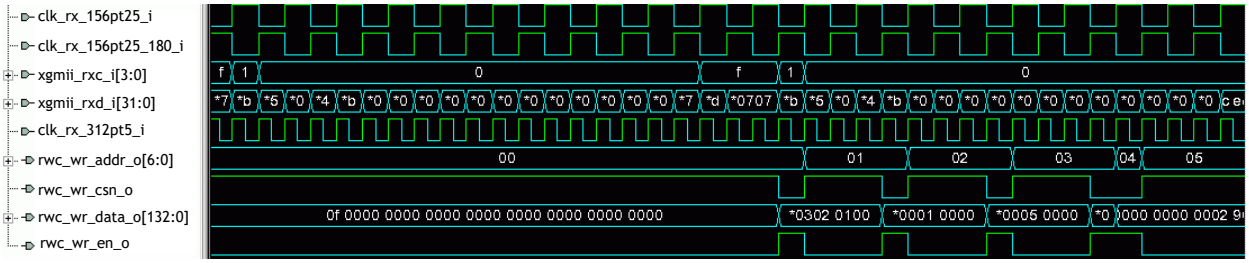


Figure 2-36 Receive Interface, GMII to RX FIFO Timing (64-bit Data Bus, GMII)

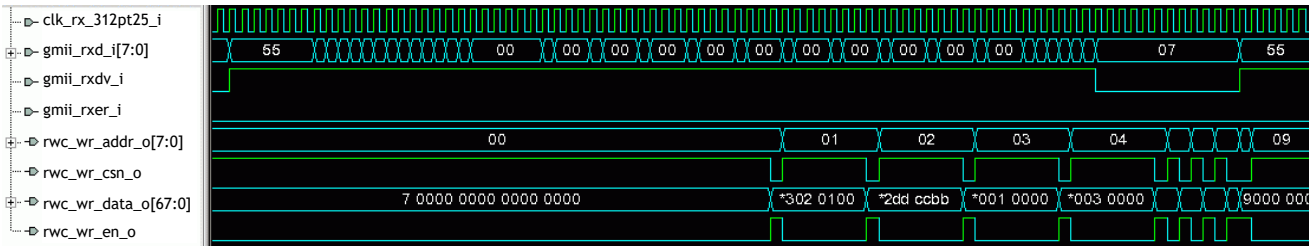
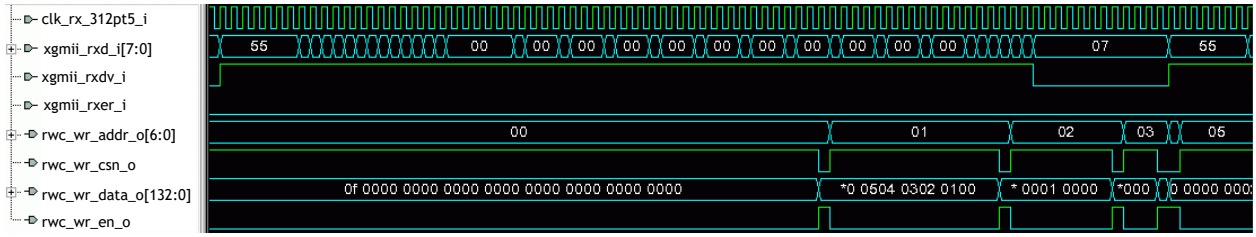


Figure 2-37 Receive Interface, GMII to RX FIFO Timing (128-bit Data Bus, GMII)



2.5.2.7 Application Receive Interface (ARI)

The data received from the MAC and stored in the Rx Queue is transferred to the Application Receive Interface with the following sequence of events, as shown in Figure 2-44:

1. The application can check whether the Rx Queue has the required amount of data (or the complete packet) to support the required burst length on `ari_qnum_pbl_i`.
2. The MTL updates `ari_watermark_o` on the corresponding queue in the next clock cycle which indicates that the MTL has enough data to sustain the burst transfer of length given on `ari_pbl_i` (in step 1).
3. In configurations with multiple Rx queues, the MTL Read controller performs the arbitration among all non-empty queues for which the application is ready to accept data (as indicated by the `ari_ready_i` signal) and selects one queue. The selected queue number is indicated through `ari_qnum_o` signal. The MTL indicates the availability of data by asserting the `ari_val_o` (in Threshold mode) or `ari_rxstatus_val_o` (in store-and-forward mode) signal along with the data. The ARI transfers the data or status in every clock cycle as long as data or status is available in the Rx queue.
4. The application accepts the data from the MTL by asserting `ari_ack_i`. The MTL updates the data in the next clock cycle if it is continuing the burst transfer. The application can pause the transfer by de-asserting `ari_ack_i`, as shown in Figure 2-44.
5. The MTL transfers the remaining data by asserting `ari_val_o` and the corresponding data. When the last few bytes of packet (less bytes than the data bus width) are being transferred, `ari_eop_o` is also asserted along with `ari_be_o`, indicating a valid number of bytes in that data phase.
6. In Threshold mode, the Receive packet status (indicated with the assertion of `ari_rxstatus_val_o` or `ari_timestamp_val_o`) is transferred after the EOP data as shown in Figure 2-40 on page 116. Packet transfer is complete only after the `ari_ack_i` acknowledgement for status.

Figure 2-38 ARI Interface Timing 128-bit – SNF Mode Multi Queue

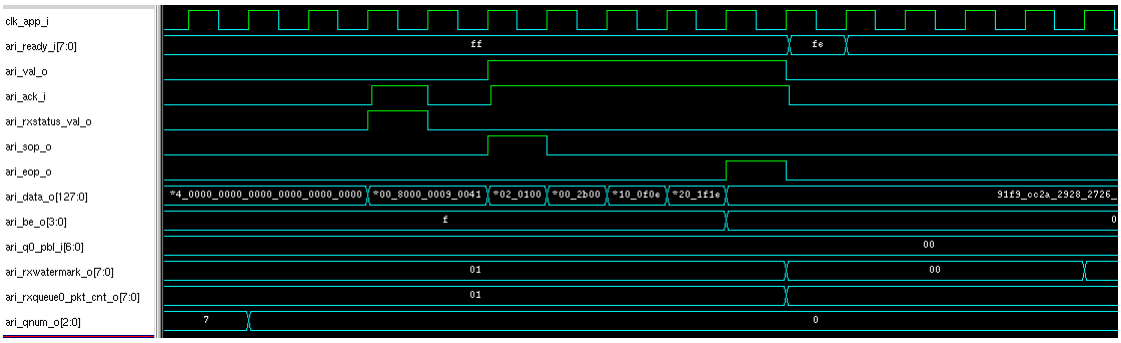


Figure 2-39 Application Receive Interface (ARI) Timing – SNF Mode

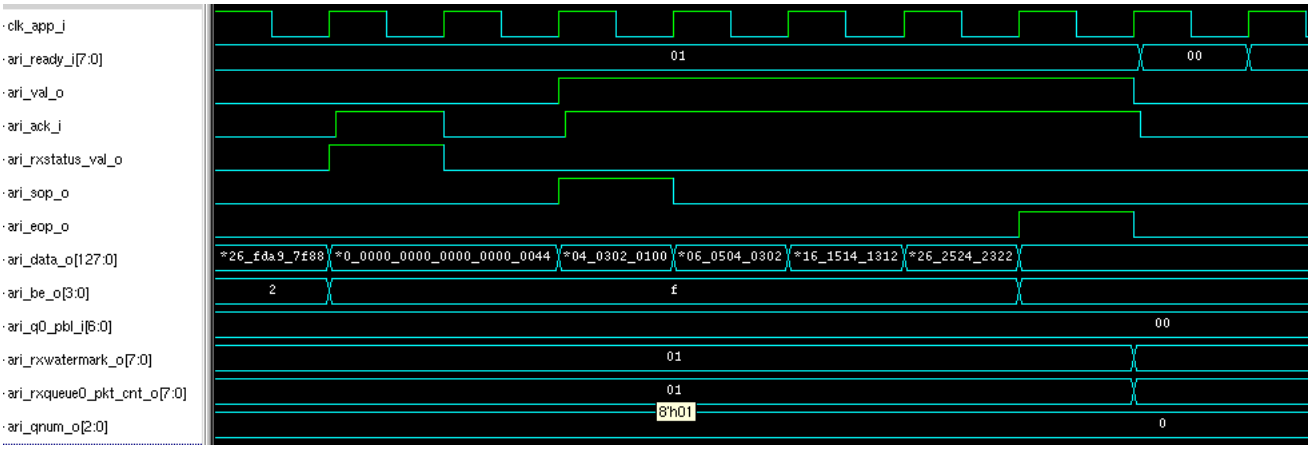


Figure 2-40 Application Receive Interface (ARI) Timing

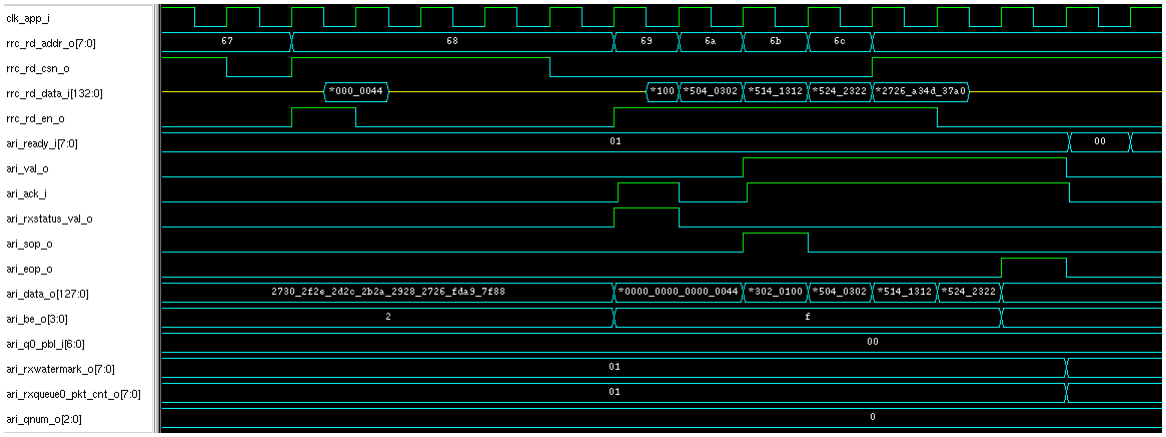
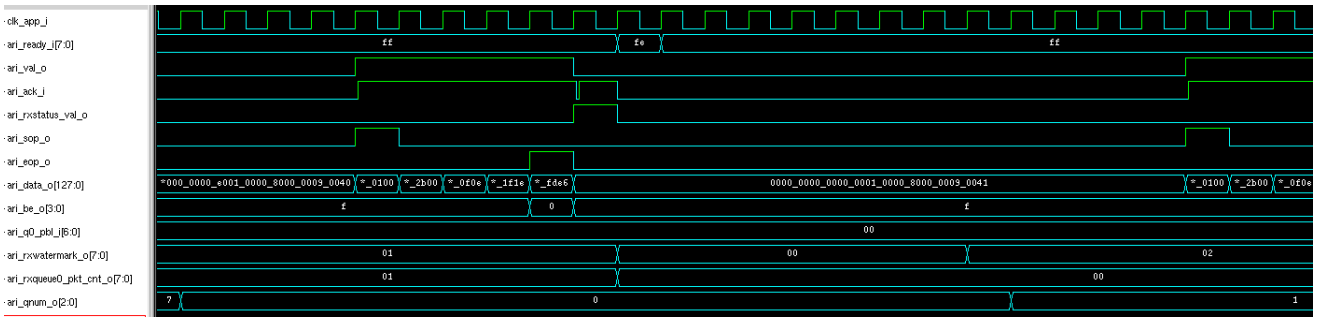


Figure 2-41 ARI Interface Timing 128-bit – Threshold Mode Multiple Queue



2.5.2.8 ARI with Timestamp Feature in Threshold Mode

Figure 2-43 shows the timing of the timestamp transfer when the MAC is configured for a 64-bit data bus. The 64-bit timestamp is given in one cycle after the Receive Status transfer.

Figure 2-42 Application Receive Interface (ARI) Timing – Threshold Mode

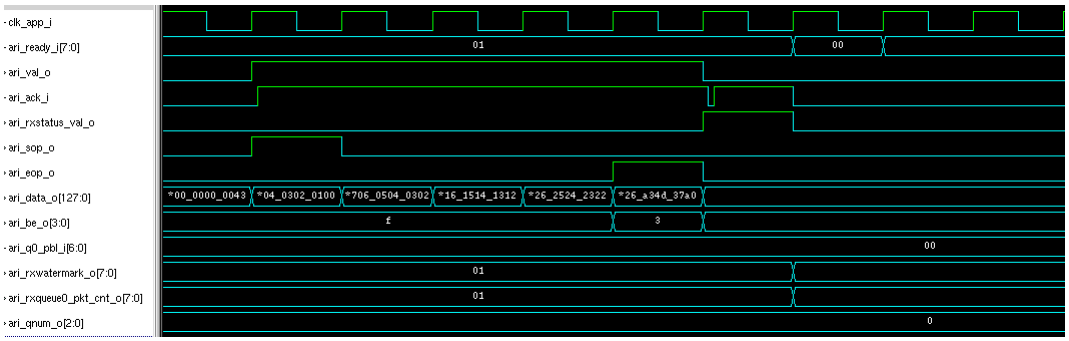
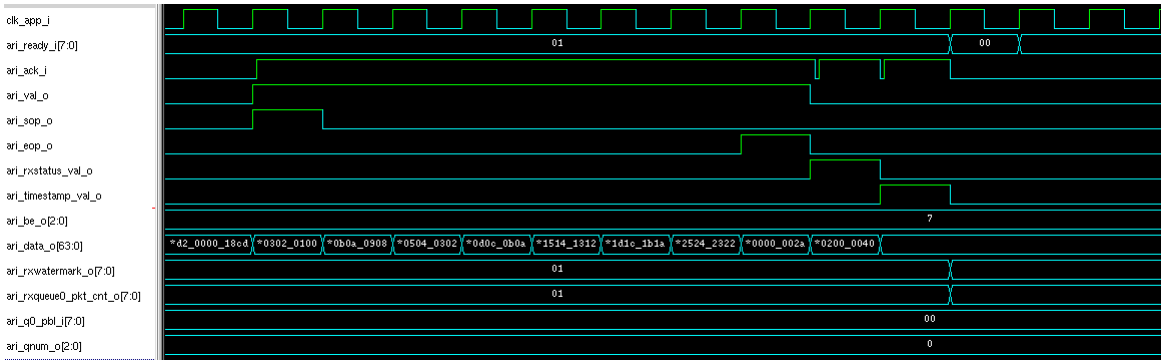
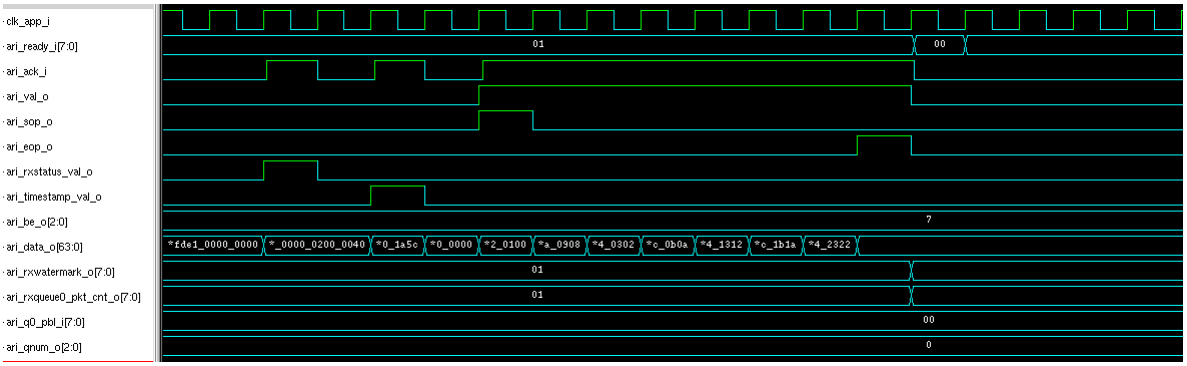


Figure 2-43 ARI Timing for 64-Bit Bus, IEEE 1588 Timestamp Feature Enabled – Threshold Mode



2.5.2.9 ARI with Timestamp Feature in Store-and-Forward Mode

Figure 2-44 ARI Timing for 64-bit Bus, IEEE 1588 Timestamp Feature Enabled – SNF Mode



In store-and-forward (SNF) mode, the MTL starts the packet transfer with the status by asserting the `ari_rxstatus_val_o` and `ari_data_o` signals containing the status. After the application accepts the status words by asserting `ari_ack_i`, the MTL asserts `ari_timestamp_val_o` to indicate the availability of the timestamp on `ari_data_o`. After the application accepts the timestamp (by asserting `ari_ack_i`), the MTL starts driving the packet data by asserting the `ari_val_o`, `ari_sop_o` and `ari_data_o`.

2.5.2.10 Receive Queue Read Interface

The queue read timing diagrams for the synchronous 2-port RAM along with the timing relationship with signals on the ARI is shown in Figure 2-40 on page 116.

The MTL Rx Queue Read Controller pre-fetches the data (rrc_rd_* signals) from the Rx queue when it finds that the Rx queue is not empty. The pre-fetching corresponds to maximum of three locations for synchronous RAM interface. This pre-fetching is done to sustain the burst data transfers without any delay from the ARI interface. The (pre-)fetched data is registered and given out on the ARI with respective ari_* signals. When the application acknowledges the data with ari_ack_i, further data is read from the queue.

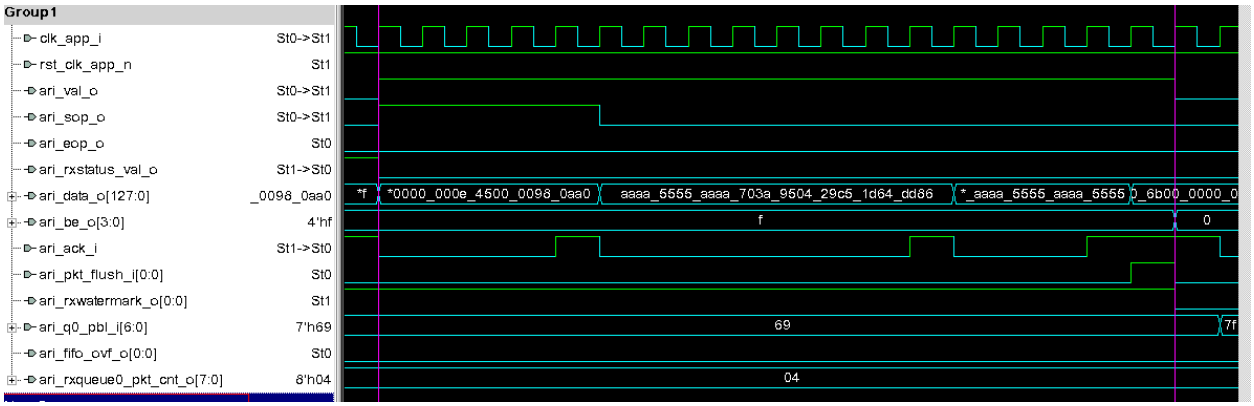
2.5.2.11 Receive Packet Flush

When the application decides to drop the packet (at the top of the Rx Queue) during the start, middle, or end of a packet transfer (`ari_val_o` is asserted), it can assert `ari_pktflush_i[n]` for one clock on the corresponding Rx queue. The MTL response is shown in Figure 2-45.

The MTL de-asserts `ari_val_o`, `ari_rxstatus_val_o`, and `ari_timestamp_val_o` when it finds an active pulse on `ari_packetflush_i` for the currently active queue number. It continues the Read transfer from the Rx queue but does not transfer it to ARI. It flushes the entire packet, including the status. The MTL is now ready to transfer the next packet.

If the `ari_packetflush_i` is asserted for currently inactive queue (there is no activity on the ARI interface for the queue to which `ari_packetflush_i` is issued), the MTL internally flushes the topmost packet in that queue.

Figure 2-45 MTL Response Timing



2.6 The MAC Layer

The MAC layer complies fully with the IEEE 802.3-2018/industry standard, implementing the XGMII/GMII/MII/RGMII/RMII full duplex interface for communication with the physical coding sub-layer. The MAC layer communicates with the application side through the MAC Transmit Interface (MTI), MAC Receive Interface (MRI) and the MAC Control Interface (MCI).

2.6.1 PHY Interface Selection

The XGMAC-CORE supports the following PHY interfaces. You can select from the following supported modes, by programing the `MAC_Tx_Configuration` register as shown in Table 2-13 on page 120.

Note The GMII/MII/RGMII/RMII are optional interfaces and supported only when the corresponding configuration parameter is selected. When optional interface is selected the mac_speed_o[2:0]reflects the programming speed and can be used by the external logic to select the PHY interfaces and clock frequencies.

Table 2-13 PHY Interfaces Supported by 25GMAC

Speed	Interface	Data Width	Interface Clock Frequency	Programming MAC Tx Configuration mac_speed_o[2:0]
10Gbps	XGMII	32-bit	312.5 MHz (SDR) 156.25 MHz (DDW)	3'b000
5Gbps	XGMII	32-bit	156.25 MHz (SDR) 78.125 MHz (DDW)	3'b101
2.5Gbps	XGMII	32-bit	78.125 MHz (SDR) 39.0625 MHz (DDW)	3'b110
1Gbps	GMII	8-bit	125 MHz	3'b011
2.5Gbps	GMII	8-bit	312.5 MHz	3'b010
100Mbps	MII	4-bit	25 MHz	3'b100
10Mbps	MII	4-bit	2.5 MHz	3'b111

Note

- RMII/RGMII PHY Interfaces can only be selected when Additional GMII Port is selected during configuration (DWCXG_ADD_GMII_PORT =1).
- RGMII interface is selected only when input signal, phy_intf_sel_i[1:0] is 2'b01 upon power-on reset or software reset. RGMII speed modes (2.5G/1G/100M/10M) is based on SS[2:0] values of 3’b010, 3'b011, 3'b100, and 3'b111 respectively.
- RMII interface is selected only when input signal, phy_intf_sel_i[1:0] is 2’b10 upon power-on reset or software reset. RMII speed modes (100M/10M) is based on ss[2:0] values 3’b100 and 3’b111.

2.6.2 Transmission

Transmission is initiated when the MTL or application pushes data towards 25GMAC with the SOF signal (`mti_sop_i`) asserted. When the SOF signal is detected, 25GMAC accepts the data and begins transmitting to the XGMII/GMII/MII/RGMII/RMII. The time required to transmit the frame data to the XGMII/GMII/MII/RGMII/RMII after the application initiates transmission is variable, depending on such delay factors as inter-frame gap (IFG) delay, pause transmission requests from the 25GMAC receiver, and the time required to transmit preamble and SFD and any back-off delays for half-duplex mode. Until 25GMAC can accommodate the data from the application in the internal buffer, 25GMAC accepts data received from the MTL by asserting the `mti_rdy_o` signal.

After the end-of-frame (`mti_eop_i`) is transferred to the MAC, the application can wait until the controller completes normal transmission, or start transferring the next frame to the controller, then receive the transmission status.

In the half-duplex mode, if a normal collision occurs during transmission, the MAC sends the Transmit Status to MTL, with the retry bit set to 1. The MAC sends retry request, until one of the following condition is true:

- Packet is successfully transmitted
- Maximum number of retry requests are completed

When maximum number of retry requests are completed, the MAC terminates the packet transmission with Excessive Collision Transmit Status. The MAC accepts and drops all further data until the next SOP is received. The MTL block must retransmit the same packet from SOP, when it observes a Retry request (in the Status) from the MAC. If any one of the following occurs, the MAC terminates the packet transmission:

- No carrier (half-duplex mode)
- Loss of carrier (half-duplex mode)
- Excessive deferral (half-duplex mode)
- Late collisions (half-duplex mode)
- Jabber

The MAC accepts and drops all further data until the next SOP is received.

If the MTL cannot provide the data continuously during the transmission, 25GMAC issues an underflow status. During the normal transfer of a frame from MTL, if the 25GMAC receives an SOF without receiving an EOF for the previous frame, the SOF is ignored and the new frame is considered a continuation of the previous frame.

The following sections describe the functional blocks in the transmit path:

2.6.2.1 Transmit Bus Interface Module (TBU)

The Transmit Bus Interface module (TBU) interfaces the transmit path to the external application through a native interface. It accepts data in a 64- or 128-bit-wide bus. This interface runs on the clk_tx_312pt5_i clock signal. This module also outputs the 32-bit Transmit status to the application at the end of normal transmission, and performs endian conversion of the data bus by swapping the byte lanes and corresponding byte enables.

Additionally, the TBU adds a VLAN tag and/or SA bytes, replaces the VLAN tag, or deletes VLAN tag from the input frame data. The 25GMAC provides for VLAN tag inclusion/deletion or replacement and source address inclusion or replacement. Both the VLAN tagging and untagging feature, and the source address inclusion or replacement feature can be controlled through the sideband signals and CSR.

2.6.2.1.1 VLAN Tag Modification

VLAN Tag Inclusion

The TBU module adds the VLAN tag to the transmitted packet. It uses the VLT bits of the VLAN Tag Inclusion/Replacement (MAC_VLAN_Incl) register to add the VLAN tag value. The VLAN tag can be included on a per-frame basis using the `mti_vlan_ctrl_i` signal in the MTI interface. When the VLAN tag inclusion is enabled for the transmit packet, the application must drive the appropriate value on the `mti_crc_pad_ctrl_i` signal. `MAC_VLAN_Incl` register also provides options for inclusion, deletion, or replacement of VLAN tag. The application can program MAC_VLAN_Incl register to include the VLAN tag for all the transmit frames.

VLAN Tag Replacement

The TBU module replaces the transmitted packet's existing VLAN tag with the contents of `MAC_VLAN_Incl` register. The VLAN tag can be replaced on a per-frame basis using the MTI control signal `mti_vlan_ctrl_i`. When the VLAN tag replacement is enabled for the transmit packet, the application must drive the appropriate value on the `mti_crc_pad_ctrl_i` signal.

VLAN Tag Deletion

The TBU module also deletes the VLAN tag for each transmitted frame, based on MTI control signal. When the VLAN tag deletion is enabled for the transmit packet, the application must drive the appropriate value on the `mti_crc_pad_ctrl_i` signal. `MAC_VLAN_Incl` register also provides options for VLAN tag inclusion/deletion or replacement. The application can program `MAC_VLAN_Incl` register to remove the VLAN tag for all transmit frames.

Source Address Modification

SA Inclusion

The TBU module adds the source address to the packet transferred from the application, using the contents of `MAC_Address0_High` and `MAC_Address0_Low` registers to add the SA to the SA field. When the SA inclusion is enabled for the transmit packet, the application must drive the appropriate value on the `mti_crc_pad_ctrl_i` signal. If the frame transferred to 25GMAC has FCS, it is replaced with an accurate FCS value. The source address inclusion request can be triggered by the MTI sideband control signal (`mti_sa_ctrl_i`) on a per-frame basis or by the source address inclusion/replacement control bits in the `MAC_Tx_Configuration` register.

SA Replacement

The TBU module replaces the source address of the transmitted packet with the contents of `MAC_Address0_High` and `MAC_Address0_Low` registers to the SA field. The source address replacement request is serviced in the same way as an SA inclusion.

2.6.2.2 Transmit Frame Controller Module (TFC)

The Transmit Frame Controller (TFC) consists of a two-stage register, which holds the data and byte enables received from the TBU module. This register regulates data flow to TPE by providing a buffer between the application and the TPE. The TFC also requires this buffer for adding CRC and pad bytes to the frame data input the TBU module sends to the TPE (Transmit Protocol Engine).

The TBU module performs the following types of CRC and Pad insertion related operations:

CRC and Pad Insertion

The MAC appends a cyclic redundancy check (CRC) to the end of the transmitted packet of length greater than or equal to 60 bytes. When the number of bytes received from the application is less than 60 (DA + SA + LT + Data < 60), the MAC automatically appends zeros to make the packet length exactly 60 bytes and 4 bytes of CRC.

CRC Insertion (Disable Pad Insertion)

The MAC appends cyclic redundancy check (CRC) to the end of the transmitted packet, but does not append padding. The application must ensure that the padding bytes are present in the packet being transferred, that is, that the packet being transferred is of length greater than or equal to 60 bytes.

Disable CRC Insertion

The MAC does not append a cyclic redundancy check (CRC) to the end of the transmitted packet. The application must ensure that the padding and CRC bytes are present in the packet being transferred.

CRC Replacement

The MAC replaces the last four bytes of the transmitted packet with recalculated CRC bytes. The application must ensure that the padding and CRC bytes are present in the packet being transferred.

The application must select the correct type of CRC and Pad insertion operation based on the transmit packet modifications expected/enabled in the MAC.

The TFC handles the 25GMAC transmit pause operation because of reception of PAUSE frames. The TFC gives higher priority to transmit a PAUSE frame request from FTX block than the request received from TBU block.

2.6.2.3 Transmit Protocol Engine Module (TPE)

The Transmit Protocol Engine module (TPE) consists of a transmit state machine that controls Ethernet frame transmission. In XGMII and GMII/MII mode, the transmit state machine of this module contains the following features to conform to IEEE 802.3/802.3z specifications.

- Generates preamble and SFD
- Generates jam pattern after normal collision, in the half-duplex mode.
- Supports both, predefined and programmable jabber timeout
- Support flow control for the half duplex mode (back pressure)
- Generates transmit packet status
- Contains timestamp snapshot logic for IEEE 1588 support

When the TPC module requests the TPE module for a new packet transmission, the Transmit State Machine sends out the preamble and SFD, followed by the data received. The preamble is defined as 7 bytes of 8'b10101010 pattern and the SFD is defined as 1 byte of 8b'10101011 pattern. The collision window is defined as 1 slot time (512-bit times for 10/100 Mbps Ethernet). The jam pattern generation is applicable only to half-duplex mode, not to full duplex mode. In full-duplex mode, the Transmit State Machine ignores the `gmii_col_i` signal from the PHY.

In MII mode, if a collision occurs any time from the beginning of the packet to the end of the CRC field, the Transmit State Machine sends a 32-bit jam pattern of 32'h55555555 on MII to inform all other stations that a collision has occurred. If the collision is seen during the preamble transmission phase, the Transmit State Machine completes the transmission of preamble and SFD, and then sends the jam pattern. If the collision

occurs after the collision window and before the end of the FCS field the Transmit State Machine sends a 32-bit jam pattern and sets the late collision bit in the transmit packet status.

Note At the MII interface, the collision signal (gmii_col_i) is asynchronous and is checked by the transmitter after it is double-synchronized to clk_tx domain. This additional latency, delays the recognition of collision or late-collision event. When the output of gmii_col_i synchronizer is asserted after the packet is transmitted, it is not recognized as collision even if the COL signal is high at the MII interface before the end of transmission. Similarly, an assertion of COL signal at the MII input at the last byte of normal collision window might be identified as late collision because of the synchronizer delay.

The TPE module maintains a jabber timer to stop the transmission of Ethernet packets if the TFC module transfers more than 2,048 (default) bytes. The timeout is changed to 10,240 bytes when the Jumbo packet is enabled. The Transmit State Machine uses the deferral mechanism for flow control (back pressure) in the half-duplex mode. When the application requests to stop receiving packets, the Transmit State Machine sends a JAM pattern of (8'h55) 32 bytes when it senses a reception of a packet, provided the Transmit flow control is enabled. This results in a collision and the remote station backs off. The application requests the flow control through a sideband signal, `mti_flowctrl_i` or by setting the FCB bit of the Flow Control Register of the corresponding MTL queue. If the application requests a packet to be transmitted, it is scheduled and transmitted even when the backpressure is activated. If the backpressure continues to be active for a long time (and more than 16 consecutive collision events occur), the remote stations terminate their transmissions because of excessive collisions.

If IEEE 1588 timestamp is enabled for the Transmit packet, this block takes a snapshot of the system time when the SFD is put onto the Transmit GMII or MII bus. The system time source is either an external input or generated internally based on the configuration selected.

Note The collision input is an asynchronous signal. It must be asserted at least for 32-bit times (8 clocks for slower speeds).

2.6.2.3.1 XGMII Mode

The TPE module's transmit state machine performs the following functions to conform to Clause 46 of IEEE 802.3-2018:

- Generates SCC, preamble, and SFD
- Generates transmit frame status
- Blocks frame data from the TFC when the STX blocks transmission due to an IFG delay
- Based on the link fault status from the FSM in the 25GMAC receiver, terminates frame transmission and transmits the remote fault messages/idles during the IFG.

When a new frame transmission from the TFC is requested, the transmit state machine sends the SCC, preamble, and SFD, followed by the received data. The SCC is defined as 1 byte of the 8'b11011111 pattern, the preamble is defined as 7 bytes of the 8'b10101010 pattern, and the SFD is defined as 1 byte of the 8b'10101011 pattern. The Transmit Control signal (`TXC[3:0]`) is generated synchronously with the `TXD[31:0]` from this block. This includes the control character during frame transmission and fault messaging.

GMII/MII Mode

The TPE module's transmit state machine performs the following functions to conform to Clause 35 of IEEE 802.3-2018:

- Generates preamble and SFD
- Generates transmit frame status
- Blocks frame data from the TFC when the STX blocks transmission due to IFG delay
- Converts the 32-bit data from the TFC to 8-bit or 4-bit data (depending on GMII or MII mode) that can be sent on the GMII data bus (TXD[7:0]).

When a new frame transmission from the TFC is requested, the transmit state machine sends the preamble and SFD, followed by the received data. The preamble is defined as 7 bytes of the 8'b10101010 pattern and the SFD is defined as 1 byte of the 8b'10101011 pattern. The TPE state machine transmits a single byte every clock over `TXD[7:0]`. The transmit control signals TX_EN and TX_ER are generated synchronously with the `TXD[7:0]` from this block. If an underflow occurs (no valid data available for transmission even though end-of-frame data is not received), the state machine ends the frame transmission on GMII by asserting the `TXER` signal on the last byte.

2.6.2.4 Transmit Scheduler Module (STX)

The Transmit scheduler module (STX) schedules frame transmission on the GMII or MII interface. It provides an enable signal to the TPE module after satisfying the IPG and back-off delays. The STX module performs the following functions:

- Maintains the inter-packet-gap (IPG) between two transmitted packets

  When operating in XGMII mode, this module maintains an average inter-frame gap (as controlled by the DDIC bit of `MAC_Tx_configuration` register) between two transmitted frames using the Deficit Idle Counter (DIC) algorithm. When operating in GMII mode, this module ensures that there is a minimum inter-frame gap between two transmit frames (controlled by the IFG bits of `MAC_Tx_Configuration` register).

  In the half-duplex mode, when IPG is configured for 96-bit times, the STX module follows the rule of deference specified in the IEEE802.3-2018, Section 4.2.3.2.1. The module resets its IPG counter if a carrier is detected during the first two-thirds (64-bit times for all IPG values) of the IPG interval. If the carrier is detected during the final one-third of the IPG interval, the STX module continues the IPG count and enables the transmitter after the IPG interval.

- Implements the Truncated Binary Exponential Back-off algorithm

  The STX module implements the Truncated Binary Exponential Back-off algorithm when it operates in the half-duplex mode.

- In XGMII mode, this module can reduce the data transmission rate to SONET/SDH rates, using a rate adaptation algorithm that stretches the IFG. The rate adaptation can be enabled through `MAC_Tx_Configuration` register.
- This module also ensures that the transmitter defers transmission (of data frames) when a PAUSE frame with non-zero pause quanta is received.

After satisfying IFG requirements, the STX enables the TPE to transmit the frame.

In XGMII mode, the STX module maintains an idle period according to the DIC Algorithm and performs IFG stretching for rate adaptation if the SONET/SDH rate match bit in `MAC_Tx_Configuration` register is enabled. If frames from the TFC arrive at the TPE module sooner than the calculated IFG time, the TPE waits for the enable signal from the STX before starting the transmission on the XGMII. The STX starts its IFG counter after the EOF, followed by the IDLE, is transmitted on the line. At the end of the IFG count (when the count is zero) the module issues an enable signal to the TPE.

In GMII mode, the STX module maintains an idle period of the configured inter-frame gap (`IFG` bits of `MAC_Tx_Configuration` register) between any two transmitted frames. If frames from the TFC arrive at the TPE module sooner than the configured IFG time, the TPE module waits for the Enable signal from the STX module before starting transmission on the GMII. The STX module starts its IFG counter as soon as the carrier signal of the GMII goes inactive. At the end of the programmed IFG value, the module issues an Enable signal to the TPE.

2.6.2.4.1 DIC Algorithm

Frames are transmitted on four XGMII lanes. Irrespective of the lane on which a frame termination (EOF) occurs, the new frame must always start from Lane 0. This requires adjustment to keep the Start character always aligned on Lane 0 and to maintain the average inter-packet gap (IPG) at 12. To adjust these alignments, a 2-bit Deficit Idle Counter (DIC) maintains the number of IDLE characters inserted or deleted, incrementing for every IDLE character deleted and decrementing for every IDLE character inserted. The Deficit Idle count varies between 0 (minimum) and 3 (maximum).

The algorithm that must be implemented to achieve a 12-byte average IPG is stated in Table 2-14.

Table 2-14 12-Byte IPG Averaging Algorithm

	Current DIC = 0		Current DIC = 1		Current DIC = 2		Current DIC = 3	
EOF Lane	New IPG	New DIC	New IPG	New DIC	New IPG	New DIC	New IPG	New DIC
Txc[3]==1	12	0	12	1	12	2	12	3
Txc[0]==1	11	1	11	2	11	3	15	0
Txc[1]==1	10	2	10	3	14	0	14	1
Txc[2]==1	9	3	13	0	13	1	15	2

The DIC architecture follows Table 2-14 while calculating the new IPG based on the lane in which the frame ends and the last DIC value.

Note DIC algorithm is applicable when back-to-back packets are received in MAC(TPE). In multiple Tx Queue configurations, due to inherent MTL scheduler delays, IPG might be more than the predicted value, based on DIC algorithm.

2.6.2.4.2 Rate Adaptation Algorithm

The rate adaptation algorithm dynamically adapts the XGMII data rate of 10 Gbps to any data rate based on the IFG Stretch Ratio value programmed in the `MAC_Tx_Configuration` register. When the stretch ratio is 13, the XGMII data rate is paced down to 9.2857 Gbps. After 64B/66B encoding (9.576 Gbps rate), this data is just below the SONET/SDH payload data rate (9.58464 Gbps) required in the WIS layer. This feature enables the 25GMAC to support 10GBASE-W applications.

Adaptation to SONET rates occurs when the Stretch Mode, ISM field in `MAC_Tx_Configuration` register is set to 1. The rate is achieved by adaptively changing the IFG to add additional IDLE characters based on the length of the last transmitted packet.

The stretch ratio is the number of bytes in a frame that require one octet of additional IFG extension. A free-running counter (the Stretch counter) counts the transmitted bytes to be considered for IFG extension. During transmission, it compares this value with the programmed stretch ratio. If the stretch count equals the stretch ratio, the IFG is extended by one octet and stored in the Stretch Size variable register. The stretch counter is reset to zero and starts counting until an EOF. If, after an EOF, the stretch count is less than the stretch size and the next frame transfer request from the application is waiting, IFG extension is performed for the previous stretch size value, and the stretch count is deferred to the next frame transmission. Conversely, if no frame transmission request is pending at the end of IFG extension, the stretch count is reset to zero.

Even when rate adaptation is employed, the DIC algorithm must be used for Lane 0 alignment. The DIC algorithm is applied based on the lane in which the IPG extension stops.

Link Fault Signaling

The link fault detection and signaling feature of 25GMAC is fully compliant to the Link Fault Signaling FSM described in Section 46.3.4.3 of IEEE 802.3-2018. 25GMAC receiver performs link fault detection and indicates the link status to 25GMAC transmitter. When the 25GMAC receiver detects a local fault during frame reception, 25GMAC terminates the frame transmission with an error control character and instead sends Remote Fault messages continuously on the XGMII lanes. Table 2-15 shows the remote fault message sequence:

Table 2-15 Remote Fault Message Sequence

Lane 0	Lane 1	Lane 2	Lane 3	Description
Sequence	0x00	0x00	0x00	Reserved
Sequence	0x00	0x00	0x01	Local Fault
Sequence	0x00	0x00	0x02	Remote Fault
Sequence	0x00	0x00	0x03	Reserved

When it detects Remote Fault messages on the receive side, 25GMAC starts sending Idle control characters continuously on the transmit XGMII lines. If a remote fault is detected during a normal frame transmission, the frame is terminated by sending an Error control character. No data is accepted from the application until the link status is OK.

Note

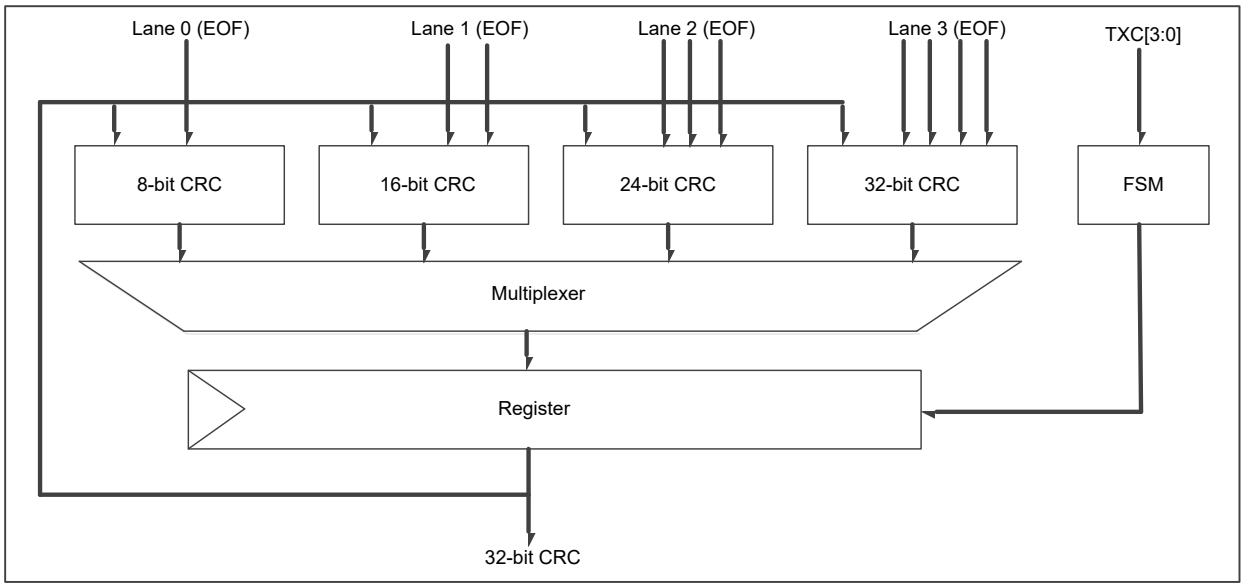
Detection of local or remote fault events is validated with the reception of 4 fault_sequences containing the same fault value, with each pair of fault sequences separated by less than 128 columns and no intervening fault_sequences of a different fault value.

25GMAC only supports sequence order set (0x9c) for the remote and local fault detection on the XGMII. If XGMII receives signal order set (0x5c), it is ignored. Section 4 of IEEE 802.3-2018 specification mentions this as RESERVED for INCITS T11 Fiber Channel use

2.6.2.4.3 Transmit CRC Generator Module (CTX)

The Transmit CRC Generator module interfaces with the TFC module to calculate CRC for insertion into the FCS field of the Ethernet frame. Because the data is sent on the four, 32-bit XGMII lanes, a 32-bit CRC is always calculated at the 312.5-MHz clock speed until the last 32 bits of the frame are transmitted. Depending on the lane in which the EOF occurs, the corresponding CRC is registered and sent on the XGMII lines. For example, if the frame ends on Lane 1, a 16-bit parallel CRC calculation is performed and transmitted. Figure 2-46 explains how CRC is transmitted.

Figure 2-46 CRC Calculation and Transmission



The CTX calculates the 32-bit CRC for the Ethernet frame's FCS field. The encoding is defined by the following generating polynomial.

G (x) = x32 + x26 + x23 + x22 + x16 + x12 + x11 + x10 + x8 + x7 + x5 + x4 + x2 + x + 1

This module gets the 32-bit Ethernet frame from the TFC module which includes (DA + SA + LT + Data + PAD) qualified with a Data Valid signal. The TFC module issues the Start command to reset the CRC before starting the CRC calculation for the new frame.

2.6.2.5 Transmit Flow Control Module (FTX)

The Transmit Flow Control module (FTX) generates PAUSE frames and drives them to the TFC module. The TFC receives PAUSE frames from the FTX, appends the calculated CRC, and sends the frames to the TPE.

The application initiates PAUSE frame generation by requesting the FTX module to send a PAUSE frame in one of the following ways:

- Setting the Flow Control Busy (FCB) bit in the Flow Control register
- Asserting the `mti_flowctrl_i` signal in response to an MTL Receive FIFO Full condition or due to the assertion of the `sbd_flowctrl_i` sideband signal in the 25GMAC Subsystem.

If the application requests flow control by setting the FCB field of Flow Control register, the FTX generates and transmits a PAUSE frame to the TFC. The generated frame contains the Pause Time value programmed in MAC_Q(#i)_Tx_Flow_Ctrl register. To extend or end the pause prior to the time specified in the previously transmitted PAUSE frame, the application must request another PAUSE frame transmission after programming the appropriate value in the Pause Time register.

If the application requests flow control by asserting the mti_flowctrl_i signal, the FTX generates and transmits a PAUSE frame to the TFC. The generated frame contains the Pause Time value programmed in CSR flow control register. After scheduling a PAUSE frame, the FTX tracks the pause quanta (64 byte times, or the time taken to transmit 64 bytes) elapsed after the first PAUSE frame transmission. If the remaining pause quanta value equals the configured threshold (the PLT bits of Flow Control register) pause quanta times, a second PAUSE frame is transmitted to the TFC. The process is repeated as long as the mti_flowctrl_i signal remains asserted.

If the mti_flowctrl_i signal goes inactive prior to the sampling time, the FTX transmits a PAUSE frame with 0 pause time to indicate the other end (a switch, for example) that the Receive buffer is ready to receive new data frames. Zero-quanta PAUSE frame generation is controlled using the Flow Control register's DZQP field (bit[7] of Flow Control register).

2.6.2.6 Transmit XGMII Interface FIFO (TIF)

The XGMII transmit interface can be configured to operate in the following modes:

- Single Data Rate (SDR)

  In this mode, 32-bit XGMII transmit data and 4-bit lane control data is transmitted at every 312.5-MHz rising edge.

- Dual Data Rate (DDR)

  In this mode, 32-bit transmit data and 4-bit lane control data is transmitted at every 156.25-MHz rising and falling edge.

- Dual Data Width (DDW)

  In this mode, 64-bit transmit data and 8-bit lane control data is transmitted at every 156.25-MHz rising edge.

Note Internal 32-bit data is converted to 64-bit data at the XGMII interface in DDW mode using a free-running register FIFO. As the IFG between frames is already determined in the register settings (See "Transmit Scheduler Module (STX)" on page 125), the transmitted Ethernet frames can start in either lane 0 or lane 4.

Because the transmitter operates at 312.5 MHz, an asynchronous FIFO is used to synchronize the transmit frame data to a 156.25-MHz clock for DDR and DDW. The asynchronous FIFO ensures that the transmit data is synchronous to the 156.25-MHz clock, irrespective of the skew variations and phase relationship between the two clock domains.

Note This asynchronous register FIFO only has depth to accommodate skews between the two clocks. To avoid underflows or overflows, make sure the two clocks are frequency-synchronized.

This block is bypassed when the 25GMAC is configured for a separate GMII interface and operating in GMII mode. If the 25GMAC is configured for a multiplexed GMII/XGMII interface (XGMII in DDR and DDW mode) and operating in GMII mode, the 156.25-MHz clock port (that is, the asynchronous FIFO read clock) must be driven with the 312.5-MHz clock (for 2.5G mode) or the 125-MHz clock (for 1G mode) synchronous to `clk_tx_312pt5_i`. The 180-degree out-of-phase 156.25-MHz clock port, `clk_tx_156pt25_180_i`, need not be driven while the 25GMAC is configured for a multiplexed GMII/XGMII interface (XGMII in DDR mode) and operating in GMII mode.

2.6.2.7 MAC Transmit Interface Protocol

The MAC Transmit Interface (MTI) connects the application (the MTL in the 25GMAC subsystem) with 25GMAC to provide Ethernet data for transmission. In the XGMAC-CORE configuration, this is the external interface to the system for transmit data.

The application initiates Ethernet frame transmission by writing the first data of the frame (`mti_sop_i`) to the 25GMAC. Each valid data transfer is indicated by an active `mti_val_i` signal. The application can push new data in as long as the 25GMAC accepts it. The XGMAC-CORE acknowledges each data transmission from the application by asserting `mti_rdy_o`. Thus, the application can assume a successful data transfer if the `mti_rdy_o` signal is asserted.

The application indicates the last data of the frame by asserting the End-of-Frame (`mti_eop_i`) signal along with the Last Data and Byte Enable (`mti_be_i`) signals. The number of valid byte lanes for the last data transfer is decoded with the `mti_be_i` signal. At the end of normal Ethernet frame transmission, the XGMAC-CORE outputs the transmit status (`mti_txstatus_o`) to the application, as indicated by an active `mti_txstatus_val_o` signal. To provide a maximum XGMII throughput, XGMAC-CORE transmission is decoupled from transmit status generation. The XGMAC-CORE can thus accept a new frame-transmission request before issuing the transmit status for the previous request.

Table 2-16 mti_be_i Function for 128-Bit Bus

mti_be_i	Valid Byte Lanes (Little Endian)	Valid Byte Lanes (Big Endian)
0000	mti_data_i[7:0]: Byte lane 0	mti_data_i[127:120]: Byte lane 15
0001	mti_data_i[15:0]: Byte lane 0–1	mti_data_i[127:112]: Byte lanes 15–14
0010	mti_data_i[23:0]: Byte lanes 0–2	mti_data_i[127:104]: Byte lanes 15–13
…		
1110	mti_data_i[119:0]: Byte lanes 0–14	mti_data_i[127:8]: Byte lanes 15–1
1111	mti_data_i[127:0]: Byte lanes 0–15	mti_data_i[127:0]: Byte lanes 15–0

Table 2-17 mti_be_i Function for 64-Bit Bus

x000	mti_data_i[7:0]: Byte lane 0	mti_data_i[63:56]: Byte lane 7
x001	mti_data_i[15:0]: Byte lanes 0–1	mti_data_i[63:48]: Byte lanes 7–6
x010	mti_data_i[23:0]: Byte lanes 0–2	mti_data_i[63:40]: Byte lanes 7–5
…		

Table 2-17 mti_be_i Function for 64-Bit Bus (continued)

x000	mti_data_i[7:0]: Byte lane 0	mti_data_i[63:56]: Byte lane 7
x110	mti_data_i[55:0]: Byte lanes 0–6	mti_data_i[63:8]: Byte lanes 7–1
x111	mti_data_i[63:0]: Byte lanes 0–7	mti_data_i[63:0]: Byte lanes 7–0

If the frame transmission is not successful (due to underflow, local faults, remote faults, and so forth), 25GMAC asserts the transmit status before the EOF is received. The application must act appropriately in response to the status. The 25GMAC drops all further data input until the next SOF.

Table 2-18 Transmit Status at 25GMAC Interface

Bit	Description
31	Packet Retry Requested When set, this bit indicates that the MAC transmitter requested retransmission of the entire packet from the application. This bit has a higher priority than the Bit 0 (Packet Terminated). The MAC expects a retransmission of the packet when this bit is set, irrespective of the value of Packet Terminated bit.
30	Reserved
29	VLAN Frame When set, this bit indicates to the application that the transmitted frame is a VLAN-tagged frame (Type field equals 16'h8100).
28:27	Address Type This field indicates the type of destination address transmitted. ■ 2'b00: Unicast ■ 2'b01: Multicast ■ 2'b10: Reserved ■ 2'b11: Broadcast
26:13	Transmit Byte Count This 14-bit counter indicates the number of bytes transmitted.
12:9	Collision Count This 4-bit counter indicates the number of collisions that occurred before the packet was transmitted. The count is not valid when the Excessive Collisions bit is set.
8	Deferred When 1, this bit indicates that the MAC defers before transmission because of the presence of carrier.
7	Underflow When set, this bit indicates that the 25GMAC terminated the transmission due to data underflow. The Underflow error indicates during the transmission that no more data is available for transmission.

Table 2-18 Transmit Status at 25GMAC Interface (continued)

Bit	Description
6:4	Half-Duplex Mode Error Summary ■ 001: No Carrier This value indicates that the carrier signal from the PHY is not present at the end of preamble transmission. This bit is valid only when the MAC operates in the half-duplex mode. ■ 010: Loss of Carrier This value indicates that the loss of carrier occurred while transmitting the packet, that is, the gmii_crs_i signal is inactive for one or more transmission clock periods during packet transmission. This bit is valid only for packets transmitted without collision in the half-duplex mode. ■ 011: Excessive Deferral This bit is set when Deferral Check bit is set in the MAC_Extended_Configuration register. When set, this bit indicates that the transmission has ended because of excessive deferral of over 24,288 bit times (if Jumbo packet is enabled). ■ 100: Late Collision When set, this bit indicates that the packet transmission was terminated because a collision occurred after the collision window (64 bytes including Preamble in MII mode). This bit is not valid if the Underflow error is set. ■ 101: Excessive Collisions When set, this bit indicates that the transmission is terminated after 16 successive collisions while attempting to transmit the current packet. If Disable Retry Bit in the MAC_Extended_Configuration register is set, this bit is set after the first collision. The transmission of the packet is terminated. ■ 000: Reserved
3	Remote Fault When set, this bit indicates that the 25GMAC has terminated frame transmission and is transmitting Idle messages on the XGMII lines because it detected a Remote Fault message on receive. This bit is reset when no errors are detected on the receive lines.
2	Local Fault When set, this bit indicates that the 25GMAC has terminated frame transmission because it has detected a local fault on the receive and is therefore transmitting remote fault messages on the XGMII lines. This bit is reset when there are no errors on the receive lines.
1	Jabber Time-out When set, this bit indicates the 25GMAC transmitter experienced a jabber time-out. This bit is only set when the Configuration register's JB bit is enabled.
0	Frame Terminated When 1, this bit indicates that the current frame transmission is terminated, because of one or more of the following conditions: ■ Data under-run ■ Local fault ■ Remote fault

Table 2-18 Transmit Status at 25GMAC Interface (continued)

Bit	Description
	■ Jabber time-out ■ No carrier ■ Loss of carrier ■ Excessive Deferral ■ Late Collision ■ Retry count exceeds the attempt limit When reset, this bit indicates that the current frame was successfully transmitted on the XGMII.

2.6.2.8 MAC Layer Transmit Interface (MTI) Timing

Figure 2-47 through Figure 2-52 show the MAC layer Transmit Interface signal timing. The frame transmission handshake mechanism is as follows:

1. The application can initiate a frame transfer by asserting `mti_val_i` along with valid data on `mti_data_i`. The `mti_sop_i` signal is asserted when the start of frame is being transferred.
2. The 25GMAC accepts the data as long as `mti_rdy_o` is active. The application must update the data (or assert `mti_val_i`) on observing an active `mti_rdy_o` signal to transfer data in bursts. After the `mti_val_i` is asserted with the start of packet word, the MAC layer expects `mti_data_i` to be available before `mti_rdy_o` is asserted. This requires that the `mti_val_i` is continuously asserted until end of packet word is accepted.
3. If the MAC layer does not receive a local or remote fault, it starts transmitting the frame on the XGMII (`xgmii_txd` and `xgmii_txc`).
4. At any point during the transfer, the MAC layer can alert the application to stop the transfer by de-asserting `mti_rdy_o`.
5. To transfer the last bytes of a frame, the application must assert `mti_eop_i` along with the byte enables (`mti_be_i`) indicating the valid number of data bytes on the bus. All data transfers except the EOF transfer must have byte enables that are all ones.
6. After transferring the EOF, the application can wait until the MAC layer gives the status of that particular frame transmission or starts pushing in the next frame. The MAC layer accepts the next frame data even if the transmission status of the last frame is not given. The MAC layer de-asserts the `mti_rdy_o` signal if its internal register pipeline is full with the data of next frame.
7. The MTI asserts `mti_txstatus_val_o` (see Figure 2-47 through Figure 2-52) to return the transmit status of the frame to the application through `mti_txstatus_o`. The application must accept it immediately. In Figure 2-47, `mti_txstatus_val_o` is asserted for the previous frame.
8. The application can now start transmitting the next frame (or continue the transmission when the MAC layer asserts the `mti_rdy_o` signal).

Figure 2-47 Transmit Interface Timing (128-bit Databus, XGMII in SDR)

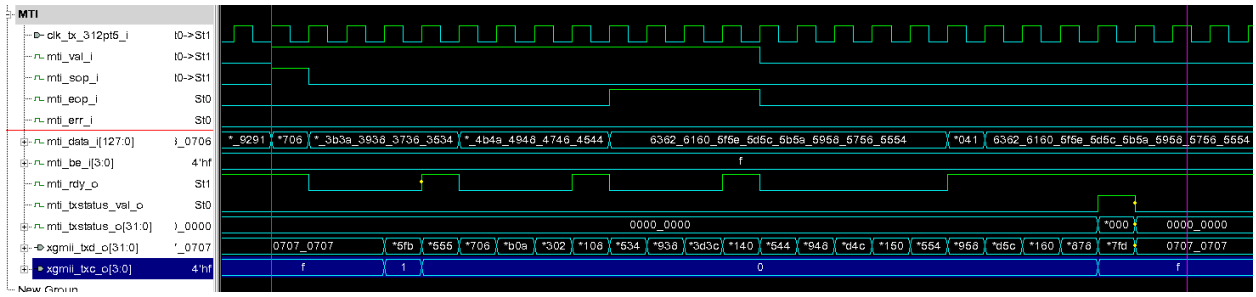


Figure 2-48 Transmit Interface Timing (128-bit Databus, XGMII in DDW)

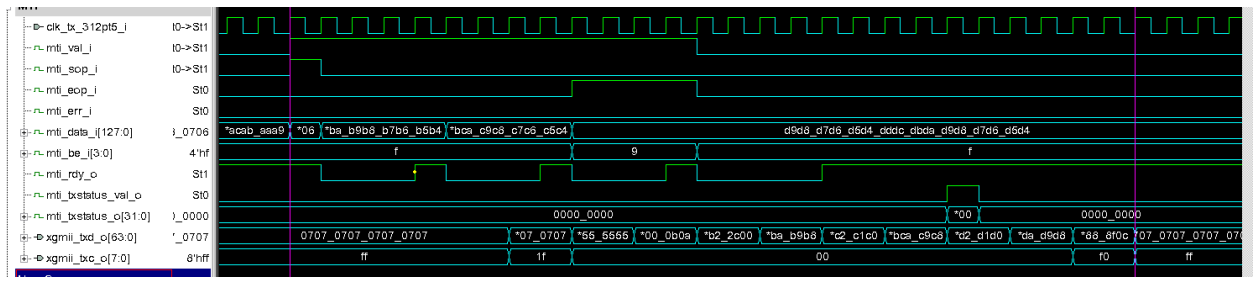
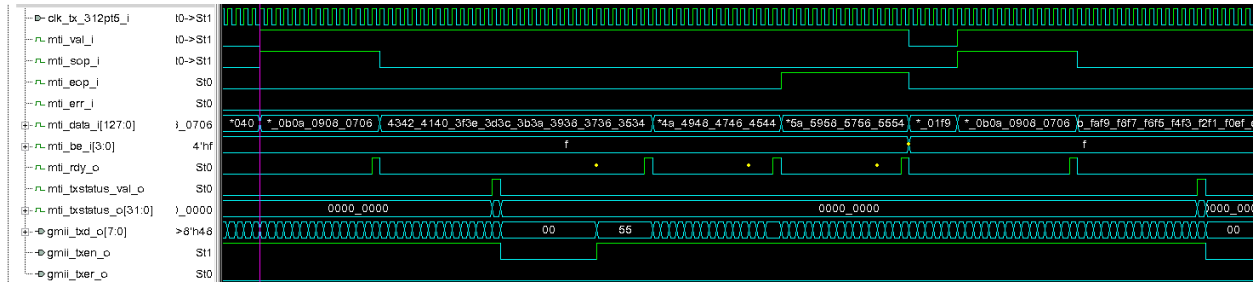


Figure 2-49 Transmit Interface Timing (128-bit Databus, GMII/MII)



Note In MII mode, only gmii_txd_o[3:0] is used for transmission.

Figure 2-50 Transmit Interface Timing (64-bit Databus, XGMII in SDR)

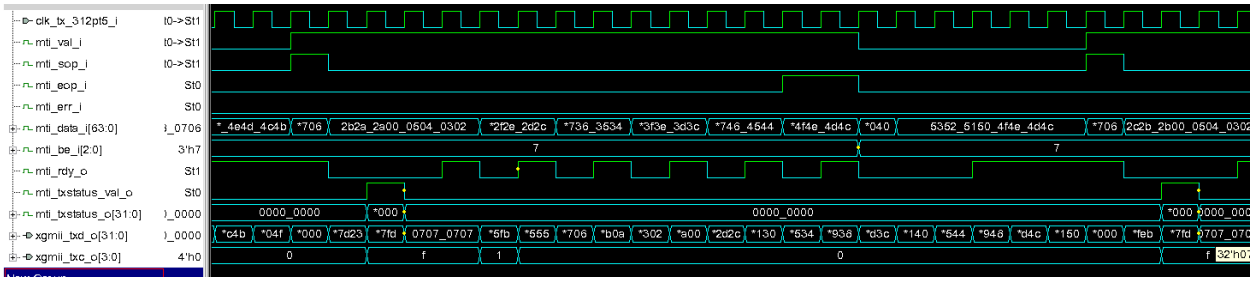


Figure 2-51 Transmit Interface Timing (64-bit Databus, XGMII in DDW)

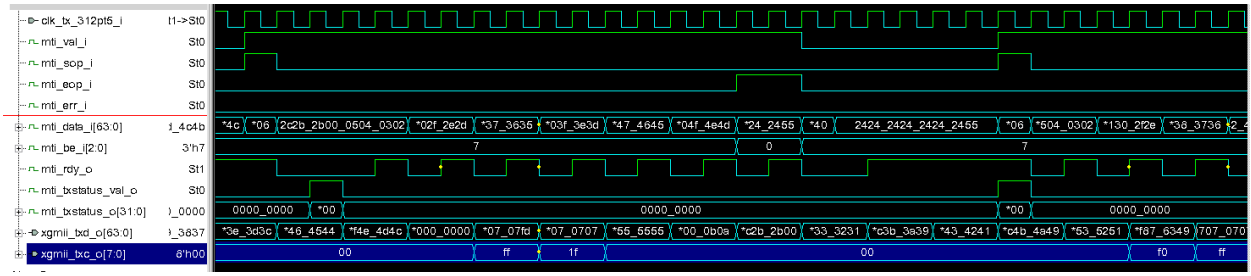
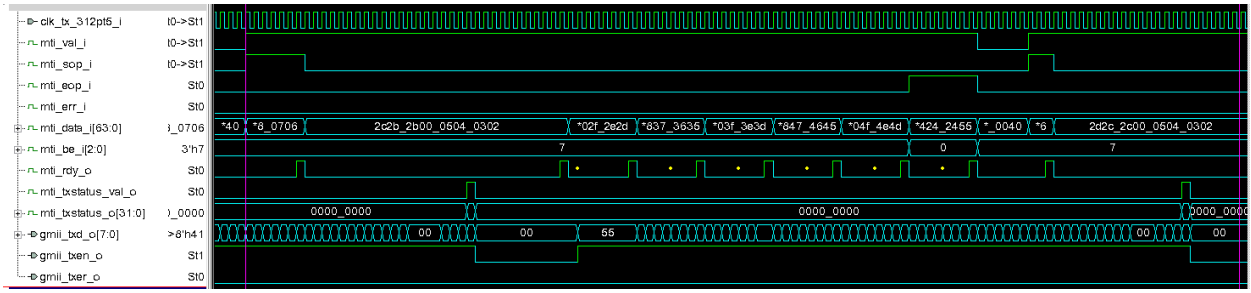


Figure 2-52 Transmit Interface Timing (64-bit Databus, GMII/MII)



Note In MII mode, only gmii_txd_o[3:0] is used for transmission.

2.6.2.9 Collision During Transmission in Half-Duplex Mode

In the half-duplex mode, the MAC accepts the data from the application on MTI and starts the transmission. When a collision is detected, the MAC de-asserts the `mti_rdy_o` signal to stop accepting more data from the application. After transmitting the JAM pattern (4 bytes of 0x55), the MAC sends the transmission status (`mti_txstatus_o`) to the application by asserting `mti_txstatus_val_o`. The MAC requests the application to retry the packet transfer by setting Bit 31 of `mti_txstatus_o` signal.

2.6.3 Reception

When MAC detects an SFD on the XGMII/GMII, a receive operation is initiated. The controller strips the preamble and SFD before processing the frame.

In XGMII mode, the MAC receiver can be configured to support frames with variable preambles, based on the setting of the `DWCXG_VARIABLE_PREAMBLE` parameter.

When MAC is not programmed to support frames with variable preambles (`VPRE` field of `MAC_Extended_Configuration` register is not set) the frame is dropped in the following scenarios:

- The Start control character occurs in any lane other than Lane 0.
- The SFD and preamble are less than 8.
- The SFD is not received on Lane 3 after the start control character.

For preemptive packets, if the VPRE filed of the `MAC_Extended_Configuration` register is set, 25GMAC supports packets with the following formats:

- /S/ + Preamble ((4*n) - 2) + SMDS (lane 3)
- /S/ + Preamble ((4*n) - 3) + SMDS (lane2) + FC (lane3) for n = 2, 3, 4 and so on.

Note

- TPRE field of the MAC_Extended_Configuration register is not supported for preemptive packets.
- In configurations with FPE, for packet reception in XGMII mode, the minimum gap between the last valid byte of previous and first valid byte after SFD of current packet is 4 cycles (IPG columns + Preamble columns)

However, when operating in the 2.5G speed in XGMII mode, if the TPRE field of the MAC_Extended_Configuration register is set, packets with SCC and 5 or 6 preamble bytes are accepted.

When operating in GMII mode, the MAC receiver can receive a frame with a variable preamble and does not drop the frame if preamble bytes are missing.

The header fields are checked for filtering and the FCS field is used to verify the CRC of the frame. The received frame is stored in a shallow buffer until the address filtering is performed. 25GMAC drops the frame if it fails the address filter.

The following are the functional blocks in the 25GMAC receive path:

- "Receive XGMII Interface FIFO (RIF)" on page 136
- "Receive Protocol Engine Module (RPE)" on page 136
- "Receive CRC Module (CRX)" on page 139
- "Receive Frame Controller Module (RFC)" on page 139
- "Receive Flow Control Module (FRX)" on page 140
- "Receive Bus Interface Unit Module (RBU)" on page 140
- "Address Filtering Module (AFM)" on page 140

2.6.3.1 Receive XGMII Interface FIFO (RIF)

The XGMII interface can be configured to operate in SDR, DDR, or DDW modes, as explained in "Transmit XGMII Interface FIFO (TIF)" on page 129. In the DDW or DDR mode, a small asynchronous register FIFO is used to transfer received data that is synchronous to the 156.25-MHz clock to the internal 312.5-MHz clock domain of the 25GMAC receiver. This FIFO ensures that the skew variations and phase relationship between the two clock domains do not affect the data transfer between clock domains.

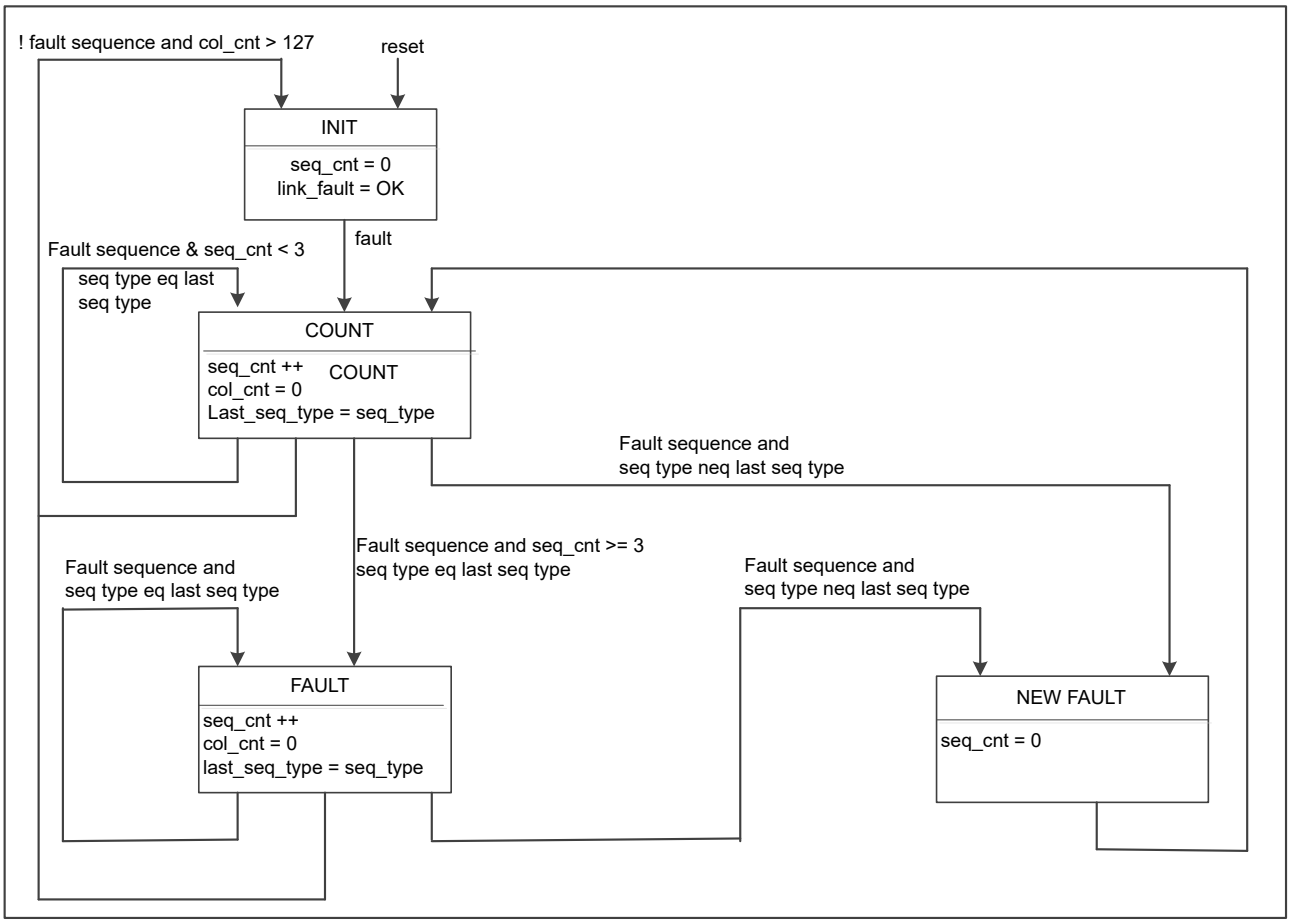
This block is bypassed when the 25GMAC is configured for a separate GMII interface or a multiplexed GMII/XGMII interface (XGMII in the DDR or DDW mode) and operating in GMII mode.

2.6.3.2 Receive Protocol Engine Module (RPE)

The RPE consists of the Receive state machines (one for XGMII and another for GMII), which strip the preamble and SFD. At any time only one FSM is enabled, depending on the configuration programmed by software through CSR (the SS field of MAC_TX_Configuration register).

XGMII Mode

Figure 2-53 Link Fault Signaling



In XGMII mode, the RPE's XGMII receive state machine seeks the Start control character on Lane 0, then seeks the SFD field in the XGMII lanes. Until the SFD field is detected, the state machine drops receive preambles. Once the SFD is detected on Lane 3, the state machine begins sending the Ethernet frame's data to the RFC, beginning with the first byte after the SFD (that is, the destination address). The RPE FSM stops sending data to the RFC when any control character other than the Error control character is received in any lane.

The RPE module detects the local and remote fault messages received on the XGMII lines, implements the Link fault signaling protocol, and sends requests to the 25GMAC transmitter to send Remote Fault or Idle messages. It also detects invalid control characters (except the Terminate control character) received on any lane during frame reception and ensures that the frame checksum error status is set. The state diagram in Figure 2-53 shows how the 25GMAC monitors the Link Fault message from the PHY or remote devices to determine the link fault status. The link fault status from the RPE determines whether the MAC transmitter transmits a Normal Frame message or a Fault message.

GMII/MII Mode

In GMII mode, when the RX_DV signal of the GMII/XGMII becomes active, the receive state machine of the RPE seeks the SFD field from the receive modifier logic. Until the SFD field is detected, the state machine drops receive preambles. When the SFD is detected, the state machine sends the Ethernet frame data to the RFC module, beginning with the first byte following the SFD (destination address).

Auto Pad/CRC Stripping

The RPE module's receive state machine decodes the Length/Type field of the Ethernet frame being received. If the Length/Type field is less than 64 bytes and the MAC is programmed with the Auto Pad Stripping or CRC Stripping options, the state machine sends the frame's data up to the count specified in the Length/Type field, then drops bytes (including the FCS field). If the received Length/Type field is greater than or equal to 64 bytes, the state machine performs no pad stripping, but does strip the FCS. The state machine sends the RFC the appropriate byte enables, along with the last word transferred after receiving the Terminate control character. The state machine enters the status phase, where it sends the received frame status. If the Length/Type field is greater than 600 (hexadecimal), the RPE module sends all received Ethernet frame data to the RFC module, irrespective of the value on the programmed auto-pad or CRC stripping option.

Watchdog Timer

The 25GMAC has an internal watchdog timer, which is programmed to truncate the receive frame. This timer is enabled by default; therefore, frames bigger than 2,048 bytes (10,240 bytes if Jumbo Frame is enabled) are cut off after 2,048 bytes (10,240 bytes) are received at the `RPE`. This feature can be disabled by programming the watchdog timer disable in the `MAC_Rx_Configuration` register. However, even if the watchdog timer is disabled, larger-than-16383 bytes frames are cut off and a watchdog time-out status is given. There is a programmable watchdog timer option provided, where cut-off limit can be specified in steps of 1KB from 2KB to 16383 bytes.

Loopback Support

The 25GMAC supports the loopback of transmitted frames onto its receiver in XGMII mode. By default, the 25GMAC loopback function is disabled, but this feature can be enabled by programming the Loopback bit in the `MAC_Rx_Configuration` register.

Enable the loopback only in the full-duplex mode. In the half-duplex mode, the carrier sense signal (csr) or collision (col) signal inputs are sampled which might result in packet drop.

The transmit and receive clocks are asynchronous, so a free-running, asynchronous FIFO is used to loop transmit data back onto the receive path. The asynchronous FIFO is 36 bits wide, to accommodate `xgmii_txd_o` and `xgmii_txc_o` signals. Because the transmit and receive XGMII clocks can have a ± 100-ppm frequency tolerance, a 5-deep FIFO is required to counter the 2-clock-cycle variation (for a 16,384-byte Jumbo frame) between the transmit and receive clocks.

In GMII mode, the same FIFO is used for loopback (depth of 5 and 36-bit wide) although only 10 bits of each FIFO word are used. The write and read pointers are re-initialized to have an offset of 2 at the start of each frame read from the FIFO. This helps avoid over- or underflow during the transfer of a frame, and ensures that the over- or underflow occurs only during the IFG between the frames.

A FIFO depth of 5 is sufficient to prevent data corruption for frame sizes up to 9,022 bytes with a difference of 200 ppm between the GMII transmit and receive clock frequencies. Therefore, larger frames must not be looped back in GMII mode, as they may become corrupted in this loopback FIFO. However, if the transmit clock is connected to the receive clock (using the `loopback_mode_o` signal), Jumbo frames of up to 16,383 bytes can be supported for loopback in GMII mode.

Note

- The 25GMAC uses the XGMII receive clock in the receive path during loopback operation.
- The internal loopback is a debug mode and has the following mandatory requirements:
  - The minimum IPG between two Tx packets must be at least 12 bytes (96-bit times) including the Terminate character, irrespective of the Tx/Rx clock ppm difference or jitter (IPG field of the `MAC_Tx_Configuration` register programmed to 0).
  - The DIC algorithm must be disabled (DDIC bit of `MAC_Tx_Configuration` register programmed to 1).

Frame Status Generation

At the end of every received frame, the RPE module generates a received frame status and sends it to the RFC module. Control, missed frame, and filter fail statuses are added to the receive status in the RFC module.

2.6.3.3 Receive CRC Module (CRX)

The Receive CRC (CRX) interfaces to the RPE to check for CRC errors in the frame being received. The hash-based filtering of received frames uses the upper 6 bits of the destination address's CRC. The upper 4 bits of VLAN tag CRC are also used for VLAN-based hash filtering of VLAN frames.

This module calculates the 32-bit CRC for the received frame, which includes the Destination Address field through the FCS field. The following polynomial is used to calculate the CRC.

G (x) = x32 + x26 + x23 + x22 + x16 + x12 + x11 + x10 + x8 + x7 + x5 + x4 + x2 + x + 1

The module takes its data from the RPE (that is, DA + SA + LT + Data + PAD + FCS). The RPE also sends a control signal that indicates the validity of the data. The CRC calculation architecture is same as that of the Transmit CRC given in "Transmit CRC Generator Module (CTX)" on page 128. Irrespective of whether Auto Pad or CRC stripping is invoked, the CRX module receives the entire frame to compute the CRC check for a received frame.

2.6.3.4 Receive Frame Controller Module (RFC)

The Receive Frame Controller (RFC) receives Ethernet frame data and status from the RPE. The RFC consists of a five-deep FIFO, to delay the data until the filter status is received from the Address Filter block. 25GMAC supports the following address filtering options:

- Perfect match or hash-based destination address filtering for all frames
- Perfect source address filtering for all frames
- VLAN tag hash-based frame filtering for VLAN frames
- VLAN-based perfect filtering of VLAN frames
- Filtering of PAUSE control frames based on the pass control frame setting in the MAC_Packet_Filter register

If the RA bit of the MAC_Packet_Filter register is set, the RFC does not wait for the address filter result and transfers all received frames to the RBU. At the end of data transfer, the RFC sends the received frame status, which includes the Frame Filter bits (SA Filter Fail, DA Filter Fail, VLAN Filter Fail) and the status from the RFC module. These bits are generated based on the AFM filter-fail signals. The RFC does not drop frames in this mode.

If the RA bit is reset, the RFC module waits up to five clocks (or until its small buffer gets full) for the packet filtering result from the AFM module. If it detects a filter-fail from AFB or a runt packet is received (of size <16 bytes), the RFC drops the frame and does not transfer it to the application.

If the filter response from the AFM is delayed more than 5 clocks, the RFC proceeds with frame transfer to the RBU. It still accepts the delayed response from AFM on a DA or SA filter failure and VLAN filter failure, but does not drop the rest of the frame. The Rx status word indicates the filter-fail status. If there is no response from the AFM until the end-of-frame is transmitted, the Filter Fail status in the Rx status word is updated accordingly.

2.6.3.5 Receive Flow Control Module (FRX)

The Receive Flow Controller (FRX) detects the receiving PAUSE frame and passes the pause time received with the frame to the 25GMAC transmitter. PAUSE frame detection can be enabled or disabled with the Receive Flow Control Enable (RFE) bit of the `MAC_Rx_Flow_Ctrl`.

Once receive flow control is enabled, the FRX begins monitoring the received frame destination address for any match with the control frame's multicast address (48'h0180C2000001). If the FRX detects a match, it indicates to the RFC module that the destination address of the received frame matches the reserved control frame destination address. The RFC then determines whether to transfer the received control frame to the application, based on the Pass Control Frames (PCF) bit setting of the `MAC_Packet_Filter` register.

The FRX also decodes the Type, Opcode, and Pause Timer fields of the control frame being received. At the end of a received frame, the FRX gets the received frame status from the RPE. If the byte count of the status indicates 64 bytes and there is no CRC error, the FRX requests the MAC transmitter to pause data frame transmission for the duration of the decoded pause time value, multiplied by the slot time (64 byte times). Meanwhile, if another PAUSE frame is detected with a zero pause time value, the FRX transfers the pause time value to the transmitter, which exits the pause state immediately. If the received control frame matches neither the Type field (16'h8808), the opcode (16'h00001), nor the byte length (64 bytes), or if there is a CRC error, the FRX does not generate a pause request to the transmitter.

For a PAUSE frame with a unicast destination address, filtering in the FRX module depends on whether the DA matched the contents of MAC Address0 register and the Unicast Detect bit of the `MAC_Rx_Flow_Ctrl` register is set (detecting a PAUSE frame even with a unicast destination address). The `PCF` field (`MAC_Packet_Filter register`) controls the filtering for control frames in addition to the Address filter.

The RFC can also detect the MAC control frames that have opcode fields other than the PAUSE frame Opcode field, which can be called control frames. The status is updated as a Control frame received.

2.6.3.6 Receive Bus Interface Unit Module (RBU)

The Receive Bus Interface Unit (RBU) converts the 32-bit data received from the RFC into a 64/128-bit FIFO protocol on the application side. The RBU interfaces with the application through the MAC receive interface (MRI). The RBU can be configured through coreConsultant or programmed with software to deliver the frame in big- or little-endian format.

2.6.3.7 Address Filtering Module (AFM)

The Address Filtering module (AFM) checks the destination and/or source address on all received frames and reports the address filtering status to the RFC. It also performs perfect VLAN and VLAN-based hashing for VLAN frames.

Address filtering is based on different parameters programmed by the application. These parameters are input to the AFM as control signals, and the AFM reports the address filtering status based on the combination of these inputs. The AFM does not itself filter the receive frames; rather, it reports the address filtering status (whether or not to drop the frame) to the RFC. The AFM also reports whether the receiving frame is a multicast or broadcast frame. The detailed operation of the packet filter in various is described in "Packet Filtering" on page 169.

2.6.3.8 MAC Receive Interface Protocol

The MRI connects the application to 25GMAC receive data path with a simple FIFO-protocol interface. The RBU initiates an Ethernet frame transfer by asserting the `mri_sop_o` signal, along with the data (`mri_data_o`). All valid data transfers are indicated by an active-high `mri_val_o` signal. The RBU transfers the last data of the frame by driving the `mri_eop_o` signal along with the data. The number of byte lanes having valid data in the last transfer (EOF) is indicated by the mri_be_o[3:0] signal. The value of `mri_be_o` is the same as for `mti_be_i`, as given in Table 2-16 and Table 2-17. The 96-bit receive status (`mri_rxstatus_o`) is also valid along with the EOF data transfer.

25GMAC assumes that the application accepts its output data in one clock cycle. Hence it does not require acknowledgement from the application before performing the next data transfer.

The Receive Status format on `mri_rxstatus_o[95:0]` is given in Table 2-19. The fields mentioned in the table have the same description as given for the RxDMA Descriptor status in Sections "Receive Normal Descriptor (Write-Back Format)" on page 1813 and "Receive Context Descriptor (Write-Back Format)" on page 1825. The modified/updated fields are shown in bold.

Table 2-19 Receive Status Format on mri_rxstatus_o[95:0]

<table>
<tr><th>31</th><th></th><th></th><th></th><th></th><th></th><th></th><th></th><th></th><th>0</th></tr>
<tr><td colspan="5">IVLAN Tag[15:0]
(valid only when L2 packet type indicates that it is DVLAN)</td><td colspan="5">OVLAN Tag[15:0]
(valid only when L2 packet type indicates that it is VLAN or DVLAN)</td></tr>
<tr><td colspan="5">MAC Filter Status[31:16]</td><td>VF or OTS</td><td>RPNG</td><td colspan="3">Reserved[13:0] or {ITS, Reserved[12:0]}</td></tr>
<tr><td>OWN[31], CTXT[30]</td><td>PTP Type [29:26]</td><td>Rsvd [25]</td><td colspan="2">ET [24:16]</td><td>ES</td><td>RSVD</td><td colspan="3">PL[13:0]</td></tr>
</table>

2.6.3.9 MAC Receive Interface (MRI) Timing

Figure 2-54 through Figure 2-59 illustrate timing for the MAC Receive Interface signal and the XGMII receive interface in 128- and 64-bit XGMII and GMII configurations. The sequence of events is as follows:

1. The MAC receives frames on the XGMII/GMII.
2. The MAC stores the frame until, if so configured, it filters the frame. When the filter passes the frame, the MAC puts the data onto `mri_data_o` and indicates its validity by asserting `mri_val_o` signal. If the data corresponds to the start of a frame, the MAC also asserts `mri_sop_o` signal.
3. The application must accept the data as soon as the MAC presents it, because the MAC may output further data (if it is available) in the next clock cycle.

4. The MAC indicates transfer of the end-of-frame data by asserting `mri_eop_o`. The `mri_be_o` signal may also have a non-all-ones value during the EOF, showing how many lanes on the data bus have valid data.
5. The MAC updates the receive status bits on `mri_rxstatus_o` during EOF transfer.

Figure 2-54 Receive Interface Timing (128-bit Data Bus, XGMII in SDR)

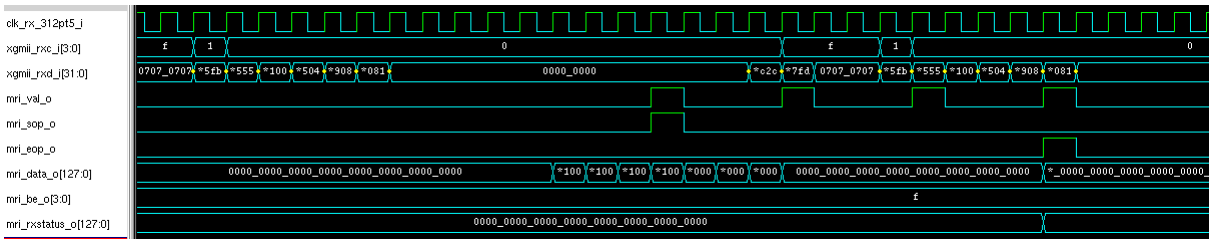


Figure 2-55 Receive Interface Timing (128-bit Data Bus, XGMII in DDW)

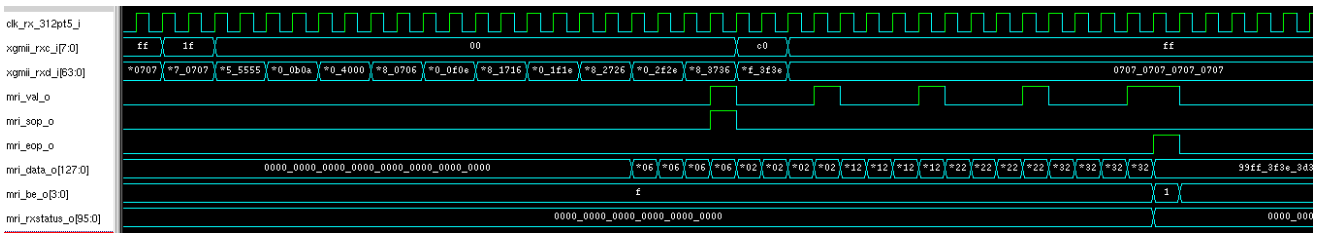
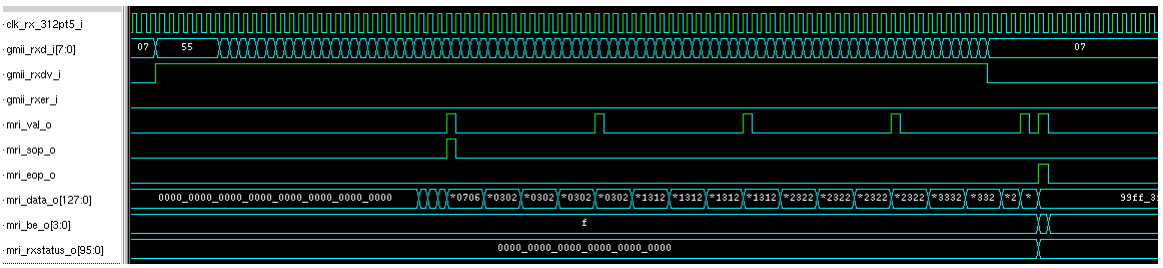


Figure 2-56 Receive Interface Timing (128-bit Data Bus, GMII/MII)



Note In MII mode, only gmii_rxd_i[3:0] is used for transmission

Figure 2-57 Receive Interface Timing (64-bit Data Bus, XGMII in SDR)

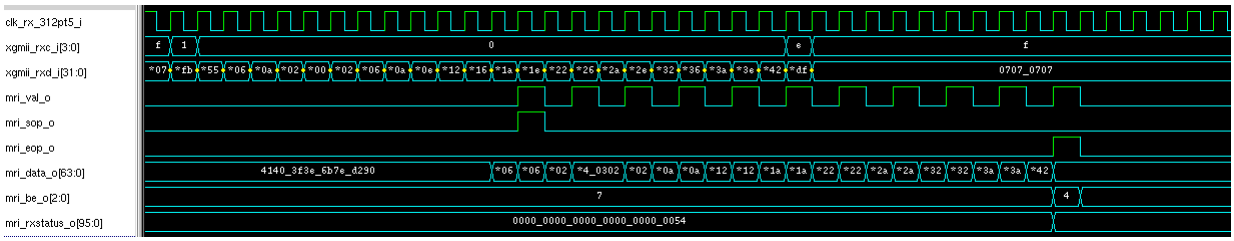


Figure 2-58 Receive Interface Timing (64-bit Data Bus, XGMII in DDW)

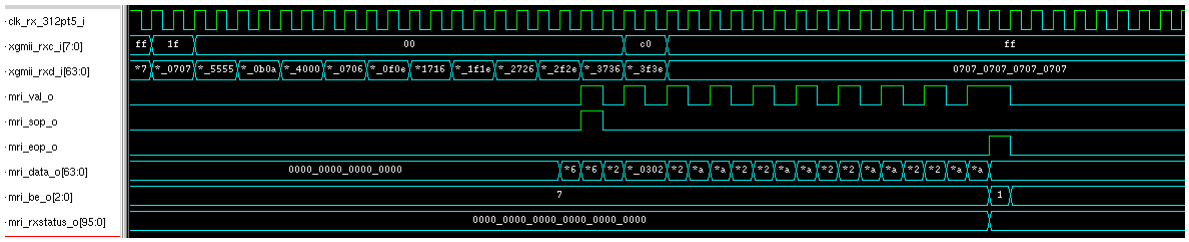
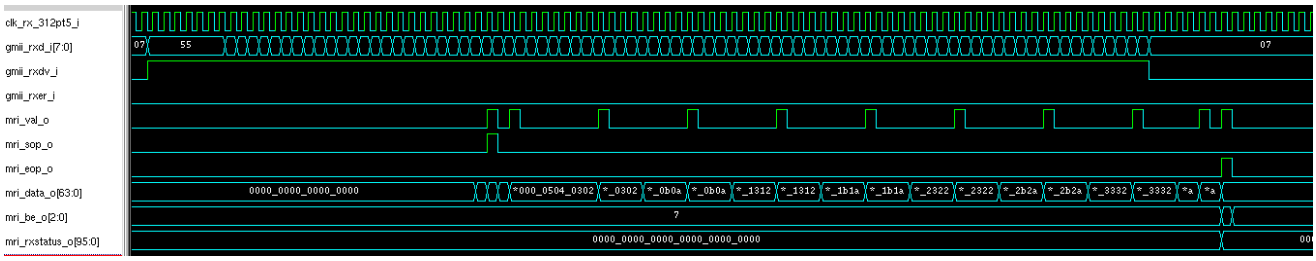


Figure 2-59 Receive Interface Timing (64-bit Data Bus, GMII/MII)



Note In MII mode, only gmii_rxd_i[3:0] is used for reception.

2.7 Interrupt Hierarchy in the MAC

Interrupts can be generated as a result of various events in the 25GMAC. These events are captured in status registers and interrupt enables are provided for each source of interrupt such that the interrupt signal (sbd_intr_o) is asserted for an event only when the corresponding interrupt enable is set.

In the XGMAC-MTL configuration, the sbd_intr_o output is the interrupt signal, which is a level signal (asserted until the interrupt source is read and cleared). In MTL, the interrupts are mainly related to exception events in the TxQ or RxQ in the Transmit and Receive paths respectively. The interrupt status are captured and organized in a hierarchical manner in order to identify the root cause quickly.

The `MTL_Interrupt_Status` register identifies the top level modules that can cause the interrupt to be asserted.

- Bits[15:0] identify 16 Queues (Tx or Rx). They are read-only bits and the application must read the corresponding `MTL_Q(#i)_Interrupt_Status` register to identify the exact cause and set the corresponding bits to 1'b1 to clear the interrupt event. The assertion of sbd_intr_o due to these events is enabled by the corresponding enable bits in `MTL_Q(#i)_Interrupt_Enable` register.
- Bit[16] is made high in case the event is present in the MAC. The application must in turn read the `MAC_Interrupt_Status` register to identify the cause.

Bit[17] is related to indirect access completion events to the TxFIFO and RxFIFO memory in Debug Mode. For more details, see "Memory Access in Debug Mode" on page 248.

The `sbd_sfty_ue_intr_o` interrupt is generated when unrecoverable errors are detected and these errors cannot be masked. Appropriate status is indicated in `DMA_AM_Sfty_ECC_Intr_Sts/DMA_AS_Sfty_ECC_Intr_Sts`, MA_AM_Sfty_DPP_Intr_Sts/`DMA_AS_Sfty_DPP_Intr_Sts`, `DMA_Safety_Interrupt_Status/MTL_Safety_Interrupt_Status` and `DMA_DPP_Interrupt_Status/MAC_DPP_FSM_Interrupt_Status` registers.

The `sbd_sfty_ce_intr_o` interrupt is generated when correctable errors are detected. 25GMAC sets the respective interrupt enable bits (in the DMA_AM_Sfty_Intr_En/`DMA_AS_Sfty_Intr_En` and `DMA_ECC_Interrupt_Enable/MTL_ECC_Interrupt_Enable` register). To find the root cause of the error, read the `DMA_AM_Sfty_ECC_Intr_Sts/DMA_AS_Sfty_ECC_Intr_Sts`, `DMA_Safety_Interrupt_Status/MTL_Safety_Interrupt_Status` and `DMA_ECC_Interrupt_Status/MTL_ECC_Interrupt_Status` registers.

The `MAC_Interrupt_Status` register bits[3:0] identify the modules that can cause an interrupt from 25GMAC. Each event can be prevented from asserting an interrupt on `sbd_intr_o` by programming the `MAC_Interrupt_Enable` register.

Note The Link Status Change interrupt is enabled by default. Set the LSIE field (bit[0]) of the MAC_Interrupt_Enable register to 1 to disable the generation of the Link Status Change interrupt.

Some of the interrupt events are enabled (such as, MMC and MDIO) and controlled by the respective Interrupt Enable register inside the module (MMC, MDIO, and so on). Figure 2-61 on page 147 shows the interrupt hierarchy.

In summary, the 25GMAC can assert an interrupt on the following events:

- Any change in XGMII link fault status (25GMAC receiver)
- Any event in the SMA (MDIO) function
  - Hot-plugging or removal of PHY port0 to port3 (SMA)
  - Alive status change in any of the MMD devices of PHY port0 to port3 (SMA)
  - Link status change in any of the MMD devices of PHY port0 to port3 (SMA)
  - Completion of a continuous write command (SMA)
  - Completion of a single read/write command (SMA)
- Any RMON counter reaching a value equal to the threshold value programmed in the MMC Control register (MMC Rx/Tx)
- MAC receiving an expected magic packet or remote wake-up packet in the power-down mode
- MAC receiver or transmitter enters or exits the Low-Power IDLE mode when Energy Efficient feature is enabled
- Any event on the optional Timestamp functions
- Any packet error event in the MAC Transmitter or Receiver
- Any event on the optional GPIO module

Note	By default, the MAC interrupt status bits are cleared when the register that contains the source of the interrupt is read. If RCWE bit of the MAC_CSR_SW_Ctrl register is programmed to 1, the MAC interrupt status bits are cleared when the bit that contains the source of the interrupt is explicitly written to 1.

Figure 2-60 25GMAC Interrupt Hierarchy

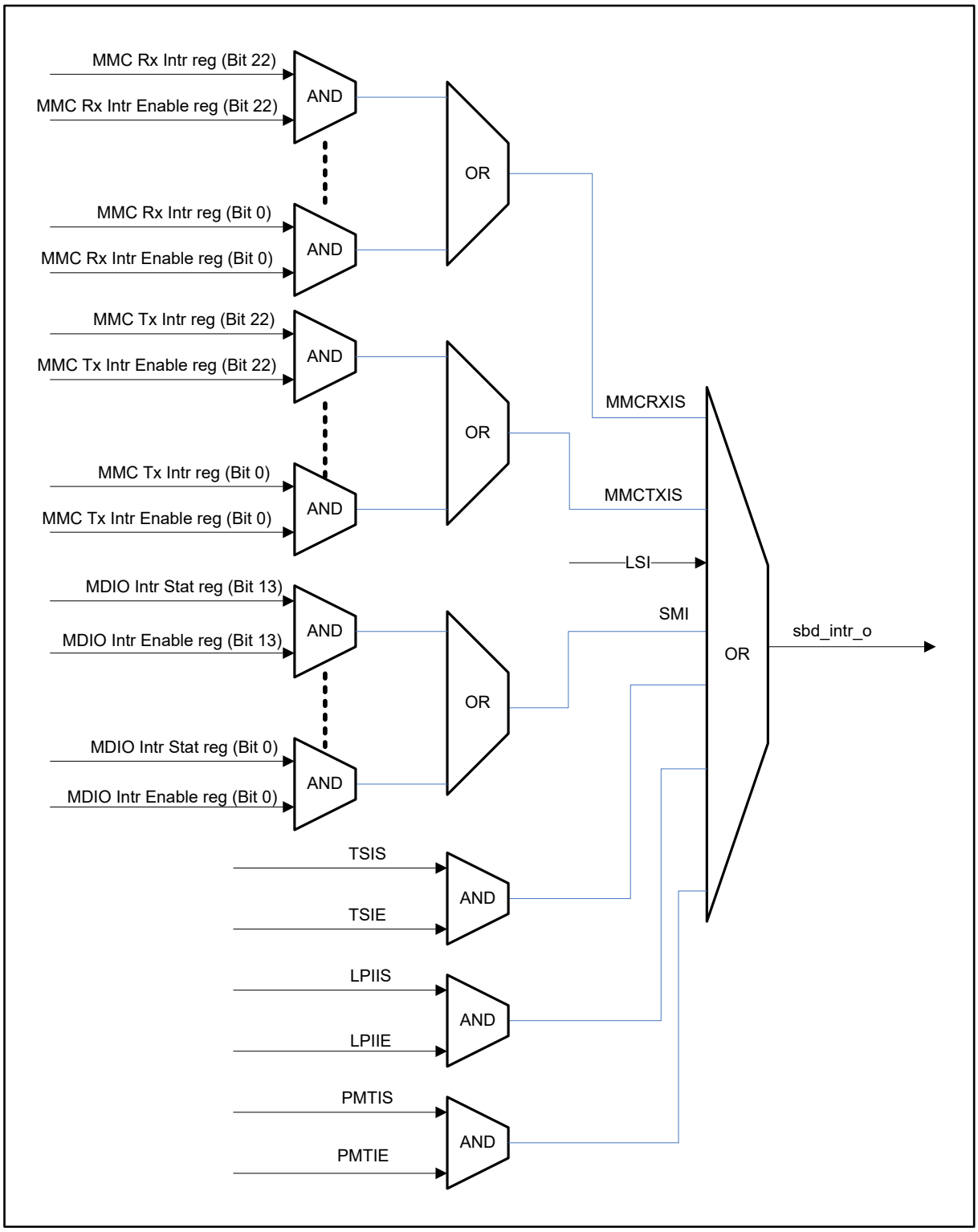


Figure 2-61 25GMAC Interrupt Hierarchy With Correctable Errors

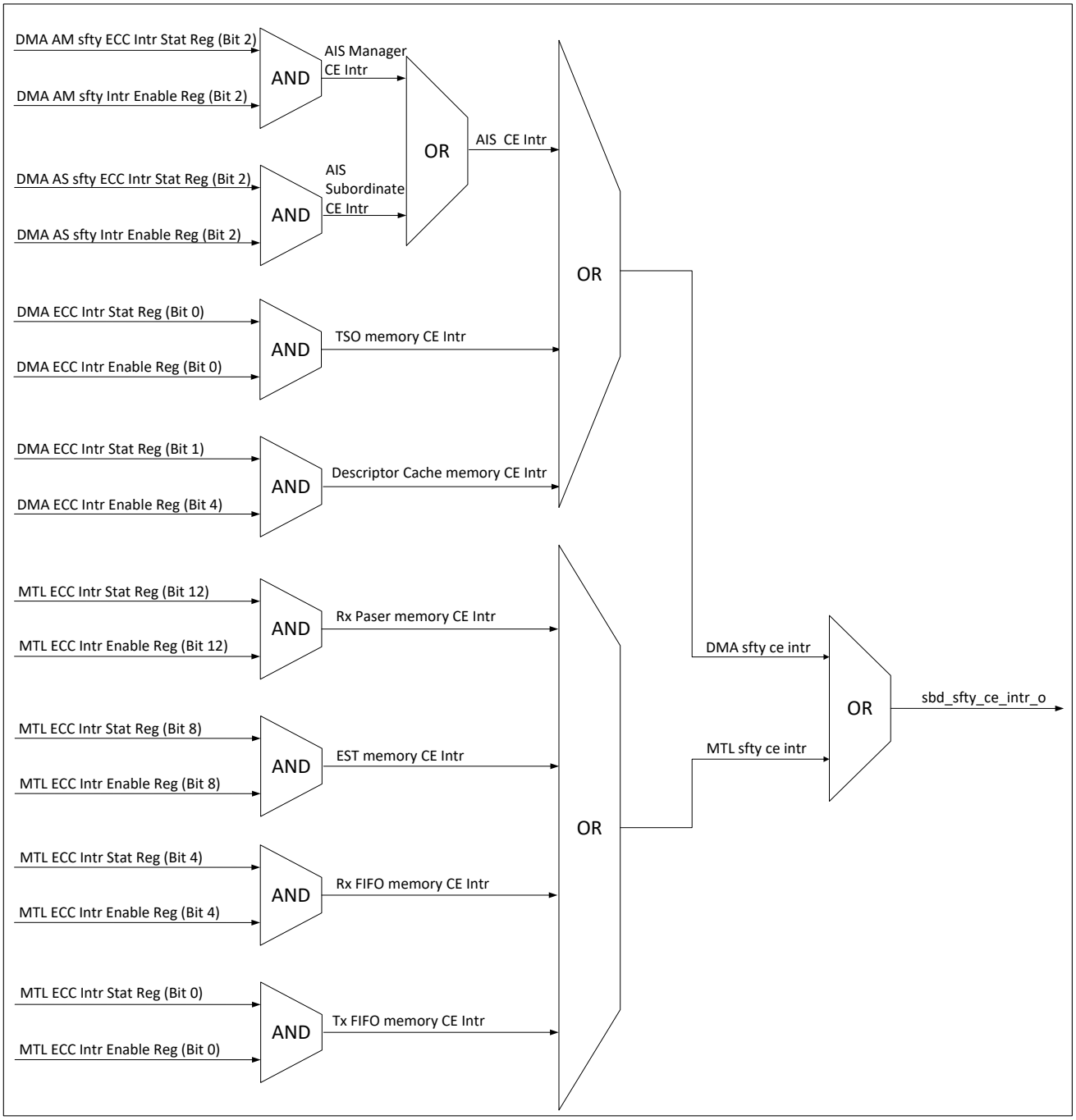
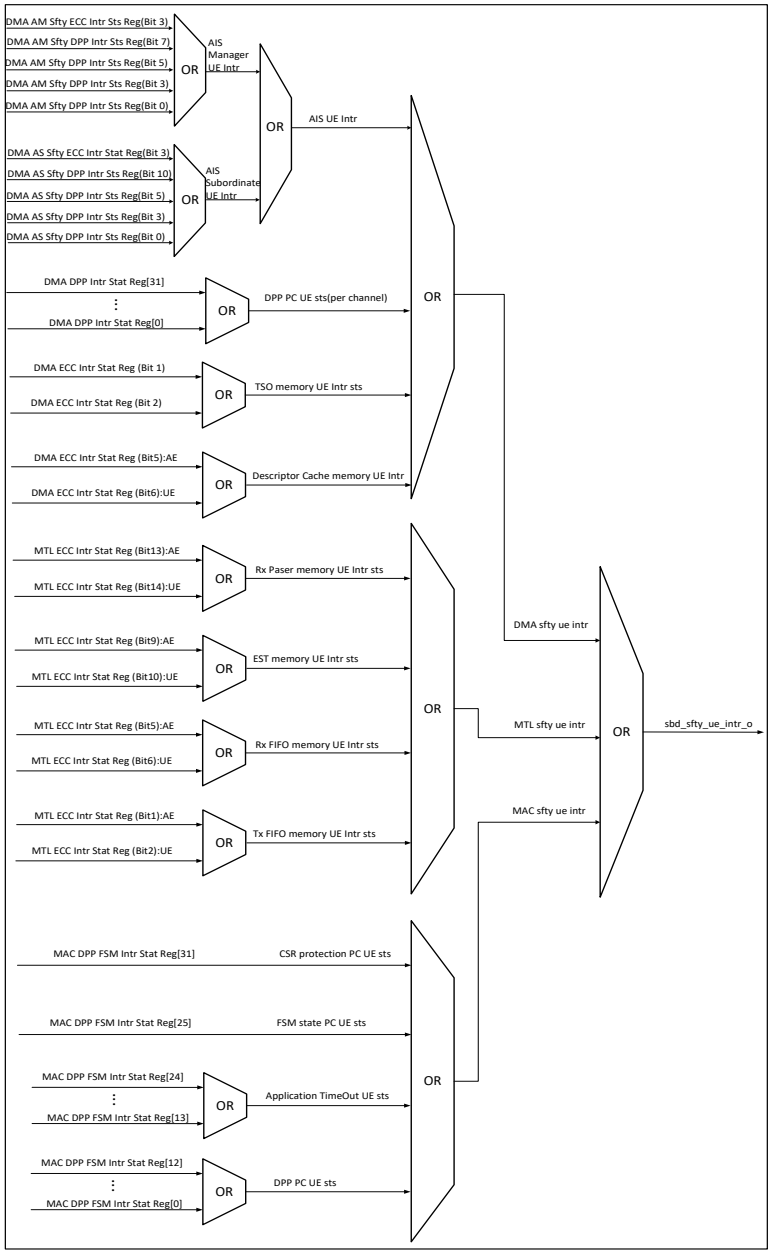


Figure 2-62 25GMAC Interrupt Hierarchy With Unrecoverable Errors



In the XGMAC-AXI configuration, the interrupt status and corresponding enable registers are organized in an hierarchical manner so that it is easier for software to traverse and identify the source of interrupt event quickly. When sbd_intr_o is asserted, the `DMA_Interrupt_Status` register is first level that indicates the major blocks for the interrupt event source. This register is read-only, and it contains bits corresponding to each DMA channel (Tx and Rx pair), the MTL, and the MAC. The software application must then read one (or more) of the following registers corresponding to the bits that are set:

- `VDMA_CH(#i)_Status` register
- `MTL_Interrupt_Status` register
- `MAC_Interrupt_Status` register

The sbd_intr_o interrupt is a level signal and gets de-asserted only when all the enabled interrupt events are cleared in their respective status registers and correspondingly all the bits in the DMA_Interrupt_Status register are cleared.

The `VDMA_CH(#i)_Status` register captures all the interrupt events of that TxDMA and RxDMA channel pair. The `DMA_CH(#i)_Interrupt_Enable` register contains the corresponding enable bits for each of the interrupt event. There are two groups of interrupts in the DMA channel namely: Normal and Abnormal interrupts. They are indicated by Bits[15:14] of `VDMA_CH(#i)_Status` register respectively. The normal group is for events that happen during the normal transfer of packets (TI, RI, TBU) while the abnormal interrupt events are for error events. Interrupt events are cleared by writing 1'b1 to the corresponding bit position. When all the enabled interrupt events are cleared (including the NIS and AIS), the interrupt source from the DMA Channel is cleared and the corresponding bit in `DMA_Interrupt_Status` register is also cleared.

Interrupts are not queued. If the same interrupt event occurs again before the driver responds to the previous one, no additional interrupts are generated. For example, Receive Interrupt Bit[6] of `VDMA_CH(#i)_Status` register indicates that one or more packets were transferred to the application buffer. The driver must scan all descriptors, from the last recorded position to the first one, owned by the DMA to determine how many packets are received.

An interrupt is generated only once for multiple events. The driver must scan the `DMA_Interrupt_Status` register for the cause of the interrupt and clear the source in the respective Status register. The `sbd_intr_o` is cleared only when all the bits of `DMA_Interrupt_Status` register are cleared.

2.7.1 Periodic Scheduling of Transmit and Receive Interrupt

It is not preferable to generate interrupts for every packet transferred by DMA (RI and TI) for system throughput performance reasons. The 25GMAC gives the flexibility to schedule the interrupt at regular intervals using two methods:

1. Set Interrupt on Completion bit in Transmit descriptor (TDES2[31]) once for every required number of packets to be transmitted.
2. Similarly, set the IOC (RDES3[30] in Table 13-15 on page 1813) bit only at some specific intervals of Receive descriptors. This way, whenever a received packet transfer to system memory is complete and any of the descriptors used for that packet transfer has the IOC bit set, only then the RI event is generated.

In addition, an interrupt timer (`DMA_CH(#i)_Rx_Interrupt_Watchdog_Timer`) is given for flexible control and periodic scheduling of Receive Interrupt as described:

1. Rx Interrupt Watchdog Timer based on number of application clock cycles: When the RWT field of `DMA_CH(#i)_Rx_Interrupt_Watchdog_Timer` is programmed with a non-zero value, interrupt timer gets activated as soon as the RxDMA completes a transfer of a received packet to system memory without asserting the Receive Interrupt because the corresponding interrupt of completion IOC bit (RDES3[30]) is not set. When this timer runs out as per the programmed value, RI bit is set and the interrupt is asserted if the corresponding RIE is enabled in `DMA_CH(#i)_Interrupt_Enable` register. The 2-bit RWTU field in the same register selects the granularity of RWT (8-bit) field. The granularity can be programmed to 256, 512, 1024, or 2048 clock cycles, giving a maximum range of 522240 (= 255 * 2048) application clock cycles.

2. Rx Interrupt Watchdog Timer based on number of transferred bytes: When RBCT field is programmed with a non-zero value, the interrupt timer accumulates the packet size in bytes (extracted from the received packet status) beginning with the packet for which the RxDMA does not assert the Receive Interrupt because the corresponding interrupt of completion IOC bit (RDES3[30]) is not set. When the accumulated bytes equals or exceeds the programmed value, RI bit is set and the interrupt is asserted if the corresponding RIE is enabled in `DMA_CH(#i)_Interrupt_Enable` register. The active range of the RBCT field in `DMA_CH(#i)_Rx_Interrupt_Watchdog_Timer` register is 1 to 1023. The granularity is 1KB, which gives a maximum threshold of 1023KB for the interrupt timer.

Both the interrupt timers are stopped and cleared, if the RI is set for a packet transfer whose descriptor's IOC was set or due to expiry of any one of the interrupt timer. The timer is reactivated automatically after the next packet transfer is complete for which the RI event is not generated.

Note When DSPW bit in the DMA_Mode register is set, the generation of the Rx interrupt based on IOC in the Rx Descriptor is not reliable. Therefore, if the DSPW bit set, IOC bit must not be enabled in the Rx Descriptor.

Along with the options seen earlier to generate and regulate the occurrence of Receive Interrupt (RI in `VDMA_CH(#i)_Status` register), an option is added to generate RI event when a RX packet is received & transferred by RX DMA after a long gap/interval of time. This interval depends on the programming of the ITW field in `DMA_CH(#i)_Rx_Interrupt_Watchdog_Timer` register.

When this mode is enabled by writing non-zero value in ITW field of `DMA_CH(#i)_Rx_Interrupt_Watchdog_Timer` register, this timer is (re-)started when:

1. RIE bit is 1 and RI status bit gets cleared/deasserted
2. RI Status bit is 0 and RIE gets set/asserted

This timer is stopped and cleared if a new packet is received before its expiry, and no further action is taken due to this timer. If this timer expires before the completion of transfer of next RX packet to host memory, then the RI event is logged and the interrupts asserted irrespective of the setting of the IOC bit. The other timers specified in `DMA_CH(#i)_Rx_Interrupt_Watchdog_Timer` continue to operate as specified and is started only after a packet is transferred without a corresponding RI interrupt event.

2.7.2 Per channel Transfer Complete Interrupt

The Transmit Transfer complete interrupt (TI) and Receive Transfer complete interrupt (RI) are reflected in `VDMA_CH(#i)_Status` register. The TI bit is set whenever the TxDMA channel closes the a descriptor in which the IOC (Interrupt On Completion - TDES2[30]) bit is set. Similarly, the RI bit is set whenever the RxDMA channel closes the descriptor with LD bit set and in any of the descriptors used for transferring that packet, IOC (Interrupt Enable on completion - RDES3[30]) bit is set.

The common sbd_intr_o output signal is asserted for the Transfer complete interrupts only when the corresponding interrupts are enabled in `DMA_CH(#i)_Interrupt_Enable` register.

The XGMAC-AXI also supports the following per Channel Transfer Complete interrupt signals.

- `sbd_perch_tx_intr_o` (Transmit Per Channel Interrupts)
- `sbd_perch_rx_intr_o` (Receive Per Channel Interrupts)

The behavior of the RI/TI/ `sbd_perch_tx_intr_o/sbd_perch_rx_intr_o` change depending on the settings of `INTM` field bit[13:12] in `DMA_Mode` register.

Table 2-20 explains the behavior of Transfer Complete Interrupt.

Table 2-20 DWC_25gmac Transfer Complete Interrupt Behavior

Sl No	Interrupt Mode	Behavior of the sbd_perch_tx_intr_o[] and sbd_perch_rx_intr_o[]	Behavior of TI or RI and sbd_intr_o
1.	INTM=0	DMA asserts a pulse on these output signals when it detects a transfer complete event on the corresponding Tx/Rx, for which IOC = 1 in the descriptor, irrespective of the corresponding interrupt status.	When the DMA detects a transfer complete event, it writes TI or RI = 1, as applicable. When the application writes TI or RI = 1, DMA clears these bits by writing 0. DMA asserts the sbd_intr_o signal when you enable the corresponding interrupts in the `DMA_CH#i)_Interrupt_Enable` register. DMA writes NIS = 1 for Tx interrupt or Rx interrupt events.
2.	INTM=1	These signals reflect the value of the corresponding TI or RI bits in the `VDMA_CH(#i)_Status` register when you enable the corresponding interrupt. These are level signals and are cleared by the application by writing RI or TI = 1. DMA does not assert this signal when you do not enable the corresponding interrupt.	DMA does not assert the sbd_intr_o signal and writes NIS = 0 for Tx or Rx interrupt events.
3.	INTM=2	DMA queues the Tx or Rx interrupts. These signals reflect the value of the corresponding TI or RI bits in the `VDMA_CH(#i)_Status` register when you enable the corresponding interrupt. These are level signals and are cleared by the application by writing RI or TI = 1. If the DMA detects another Tx or Rx interrupt event before clearing the TI/RI bits for the previous event, DMA writes RI or TI = 1 as applicable.	If the DMA detects another Tx or Rx interrupt event, before the application services the previous event, DMA automatically writes TI or RI = 1, again. However, the sbd_intr_o signal is not generated based on TI/RI. When the DMA detects a transfer complete event, it writes TI or RI = 1, as applicable. When the application writes TI or RI = 1, DMA clears these bits by writing 0. The DMA does not assert the NIS status bit for the Rx or Tx interrupt events.

3

Memory Organization for Tx and Rx Queue

This chapter describes the memory organization for transmit and receive queue supported by 25GMAC.

This chapter contains the following sections:

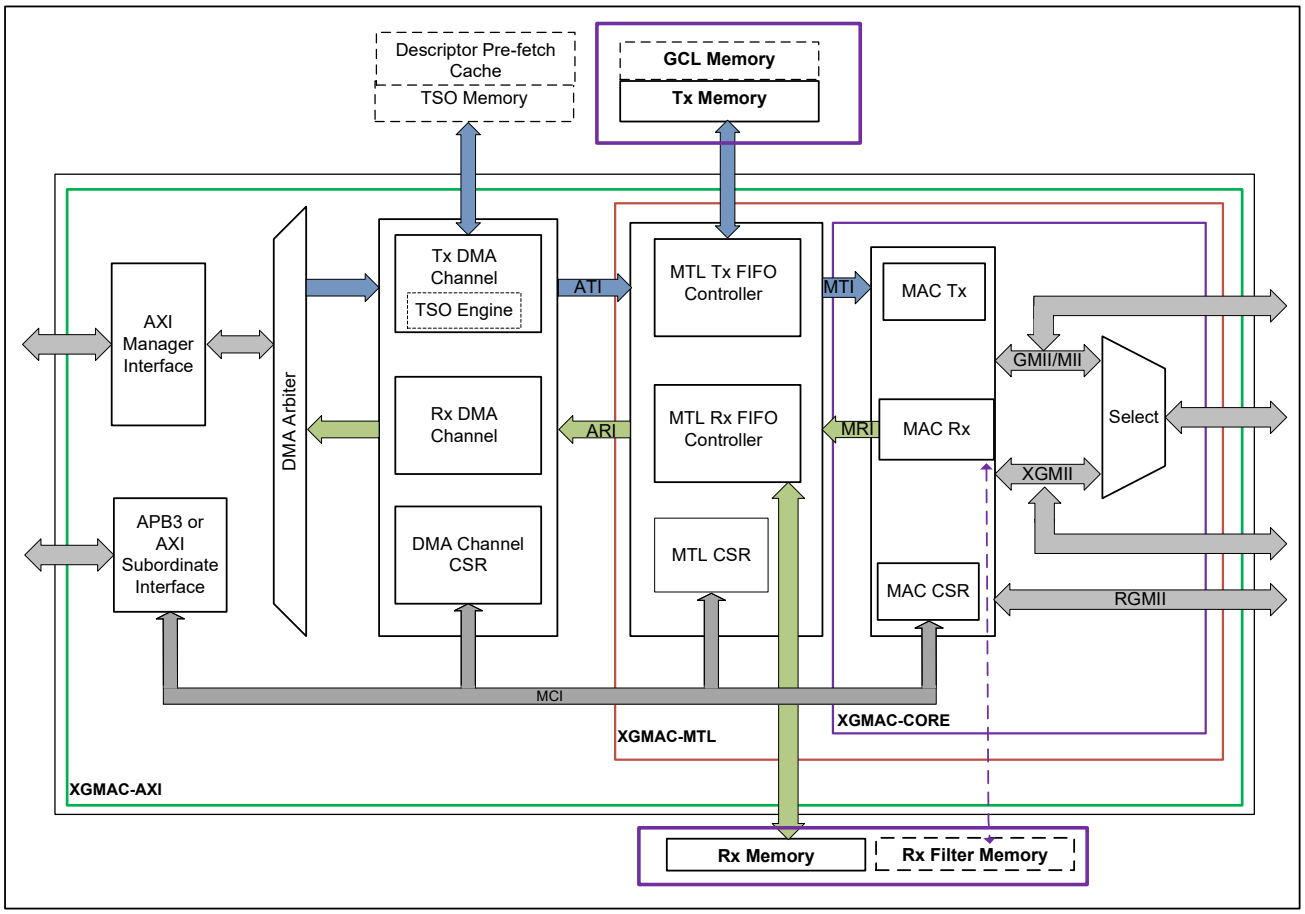
- "Introduction" on page 154
- "Asynchronous 2-Port RAM Feature" on page 155
- "Synchronous 2-Port RAM Feature" on page 157
- "Synchronous Single-Port RAM Feature" on page 159

3.1 Introduction

The transmit FIFO (Tx FIFO) buffers the data transferred from the application to the 25GMAC. Similarly, the receive FIFO (Rx FIFO) stores the packets received from the line until they can be transferred to the application.

When multiple queues are selected, all Tx queue share the Tx FIFO memory and all Rx queue share the Rx FIFO memory. The application can program the size of the FIFO memory allocated to each Tx or Rx queue.

Figure 3-1 25GMAC Memory Interfaces



3.2 Asynchronous 2-Port RAM Feature

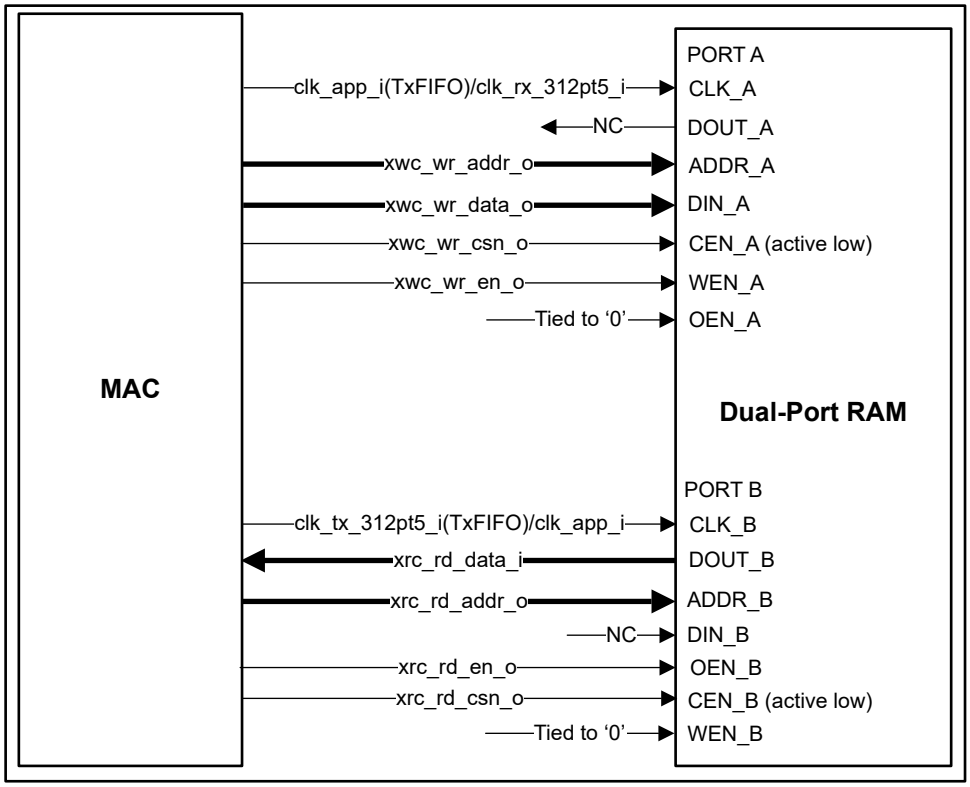
DWC_25gmac provides a generic memory interface for easy interconnection to any vendor-specific, 2-port RAM for implementing the Tx FIFO and Rx FIFO. This section describes how 25GMAC can be interconnected with different types of 2-port RAMs.

The two memory interfaces in 25GMAC provided for connectivity with Transmit and Receive 2-port RAMs are identical. Hence, the connections with the Tx FIFO and Rx FIFO RAMs are illustrated in the following sections in a single diagram for each type of RAM. The “x” variable in the signal names can be replaced with a “t” for the Tx FIFO interface and an “r” for the Rx FIFO interface.

3.2.1 Low-Power, True Dual-Port SRAM Connection

The dual-port SRAM shown in Figure 3-2 consists of two ports, Port A and Port B, which can be used for both read and write operations. 25GMAC uses this DPRAM as an asynchronous FIFO for transferring data from one clock domain to another clock domain. Thus, a port (Port A, for example) is used to write the frame data and the other port is used to read the frame data, with the writes and reads happening in different clock domains. (Consequently, the Read data (DOUT_A) from Port A and write data (DIN_B) to Port B are left unconnected in Figure 3-2.)

Figure 3-2 Low Power, Synchronous Dual-Port RAM Connection



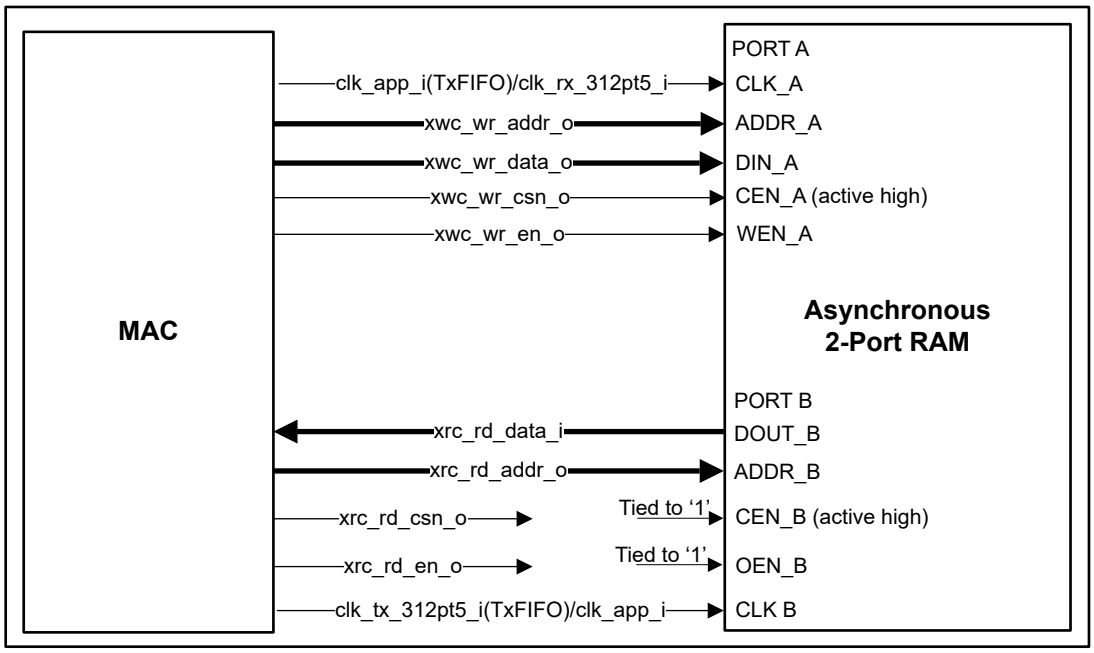
The DPRAM chip enable (CEN) and write enable (WEN) are connected to chip select (*_csn_o) and write enable (*_wr_en_o) signals from the DWC_25gmac. Based on the active states of CEN and WEN, 25GMAC memory interface signals are directly connected or have inverted polarity. Connecting the chip select signal ensures that the 2-port DPRAM I/O drivers are turned off when read/write operation is not scheduled and thereby reduces power consumption.

If the memory module (such as Xilinx FPGA Block Select RAM) has a single enable for the read port (No Output Enable), `xrc_rd_csn_o` must be connected to the enable, with polarity taken into consideration.

3.2.2 High-Speed, Asynchronous Clocked 2-Port RAM Connection

In applications where power consumption is not a factor and memory access times are too critical, the chip enable and read enable signals can always be tied to active high inputs. In such applications, the generic memory interface signals can be connected to 2-port RAM as shown in Figure 3-3.

Figure 3-3 High-Speed, Asynchronous 2-Port SRAM Connection



3.3 Synchronous 2-Port RAM Feature

Figure 3-4 shows the high level architecture for supporting Synchronous 2-port RAM interface. Both the ports of the RAM operate with same (single) clock, which is the application clock.

3.3.1 Data Flow

On the transmit side, the data is read from the Synchronous 2-port RAM memory and then transferred from application clock domain to XGMII TX clock domain using a small asynchronous register FIFO at the MTI interface between the MTL module and MAC blocks. Similarly the data received from the MAC Receiver in XGMII RX clock domain, is transferred to application clock domain using a register FIFO at the MRI interface.

Synchronous 2-port RAM provides area reduction as compared to the Asynchronous 2-port RAM, however some logic area increase is expected in MTL in the Synchronous 2-port RAM configuration due to the addition of asynchronous register FIFOs.

3.3.2 Application Clock Frequency

On the transmit side (ATI interface), the application bandwidth must be at the least, 1.33 times faster than the actual MAC bandwidth. This is to benefit the control word pre-fetch/terminate operation that can happen in the background.

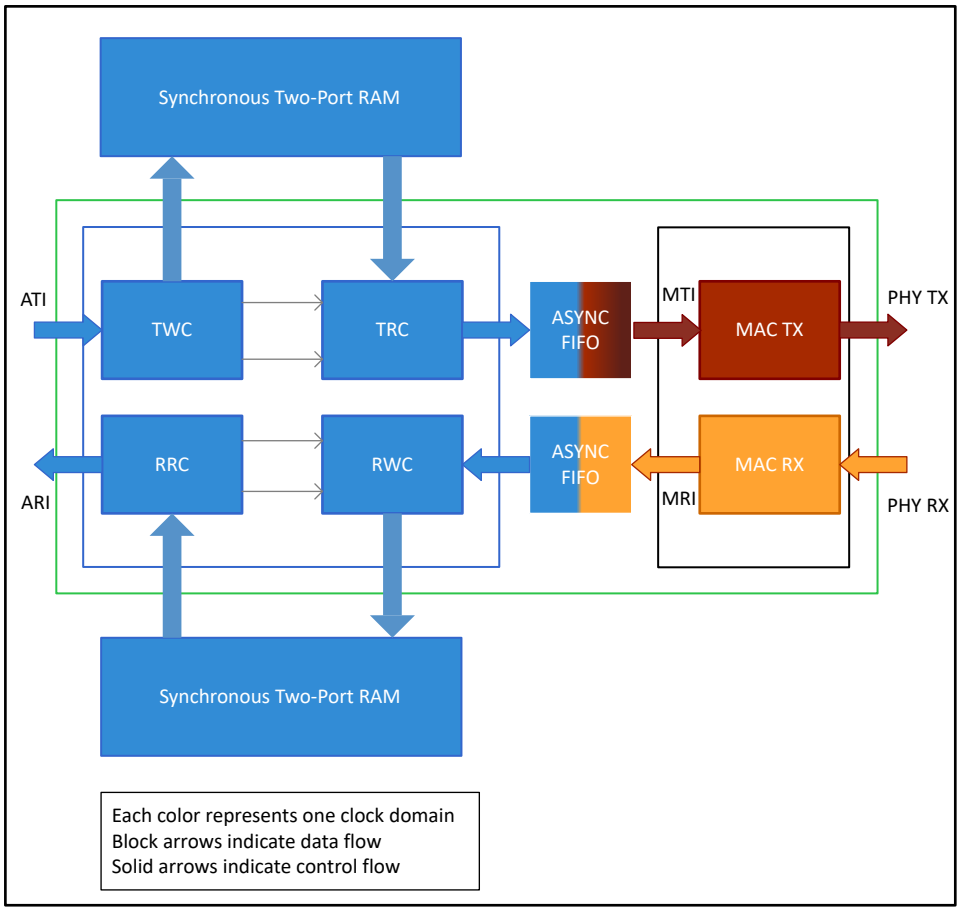
Thus, the minimum application clock frequency in 10G operating mode (with MAC clock frequency at 312.5 MHz) must be:

- 208 MHz, when data width (`DWCXG_DATAWIDTH`) = 64
- 104 MHz, when data width (`DWCXG_DATAWIDTH`) = 128

In 5G/2.5G/1G/100M/10M speed modes, the application clock frequency can be reduced accordingly.

If the application clock frequency falls below this rate, there can be underflow in packet transmissions even when the TxQueue is in store-and-forward mode.

Figure 3-4 Architecture of Synchronous 2-Port RAM

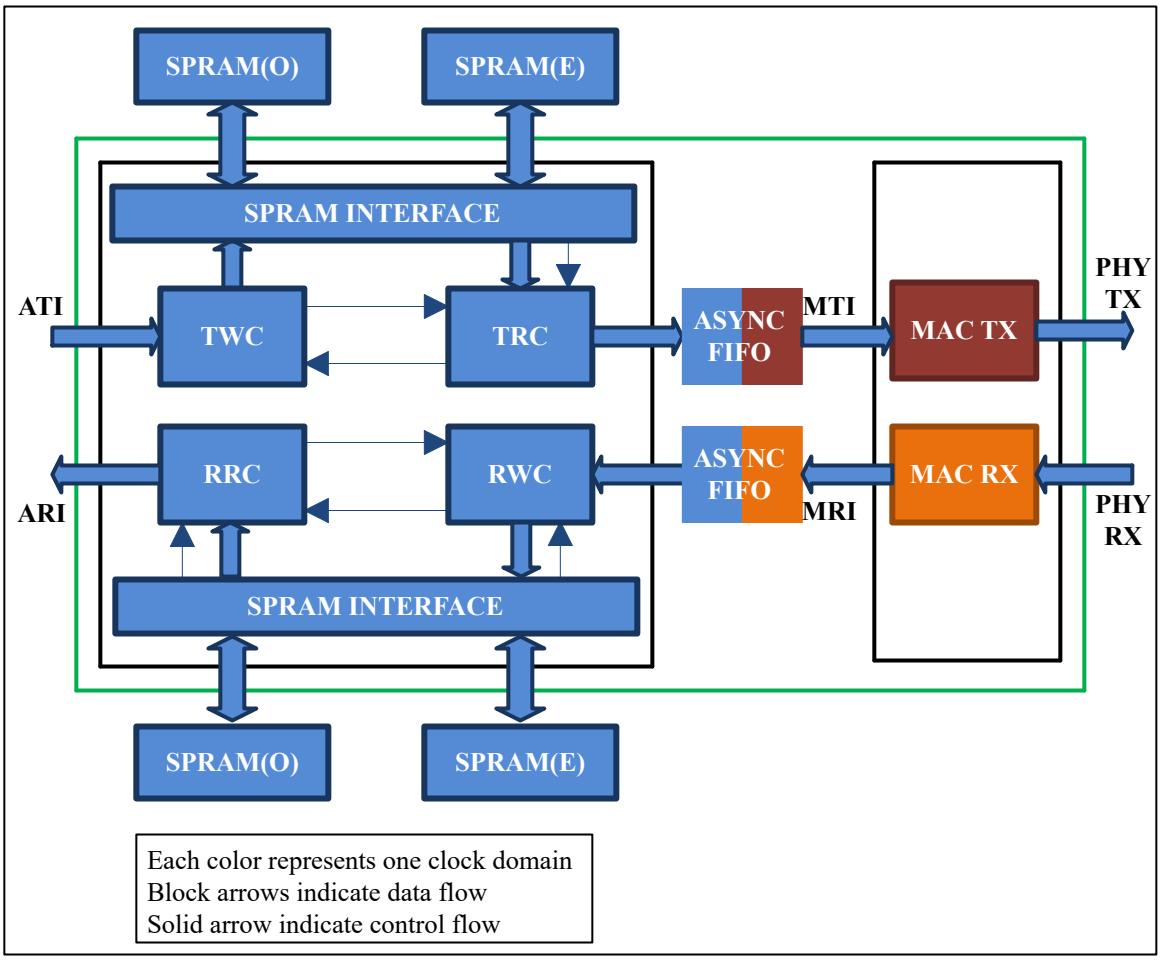


3.4 Synchronous Single-Port RAM Feature

Figure 3-5 shows the high level block diagram of single port RAM interface.

In single-port RAM, transmit and receive FIFO memory is split into two single-port RAM blocks (odd and even) and operates with the application clock. Each single-port RAM block has the same width as 2-port RAM but has half the depth of the total FIFO memory selected in the configuration. Therefore, the total memory space is retained as configured.

Figure 3-5 Architecture of Synchronous Single-Port RAM



3.4.1 Data Flow

On the Transmit side, Data Flow remains same as that of Synchronous 2-port RAM. Data is read from the single-port RAM memory and transferred from application clock domain to XGMII TX clock domain using a small asynchronous register FIFO at the MTI interface between the MTL module and MAC blocks. Similarly, the data received from the MAC Receiver in XGMII RX clock domain, is transferred to application domain using an asynchronous register FIFO at the MRI interface.

The data to the FIFO is stored alternatively in the odd and even single-port RAM blocks based on the odd and even addresses of the FIFO pointer respectively. As the memory is used as a FIFO, data is normally written and read in a sequential manner (incrementing address). Therefore, writes and reads alternate between the two odd and even single-port RAMs. This architecture enables simultaneous reads and writes to the FIFO (in 1 clock cycle) so that the data transfer throughput similar to 2-port RAM memory is achieved. This also allows continuous burst transfers to the host bus and thus utilize the bus bandwidth effectively.

However, as the read and write operations are triggered by independent engines, there can be a contention between them for accessing a single-port RAM block in the same clock. The arbiter block that is between the Read and Write Engines handles the contention:

3.4.1.1 Transmit FIFO

During contention, the write operation is given priority over read operation, with one cycle back pressure onto ATI. The read operation is delayed by a clock, with the assumption that the write operation switches to the other memory bank in the next clock (due to sequential write).

The MAC transmitter does not need continuous data; for example, in 64-bit configurations and operating in 10Gbps mode, the MAC transmitter needs 64-bit word every 2 Tx clock cycles. The depth of the asynchronous FIFO on the MAC and MTL interface ensures that there is no underrun, by prefetching data from the memory.

3.4.1.2 Receive FIFO

Read operation is given priority over write, at first contention. The write operation is delayed by a clock cycle in such cases assuming that the read operation switches to the other memory in the next clock (due to sequential reads). If the read operation does not switch to another memory bank in the next clock, write is given priority.

The MAC receiver does not provide continuous data (For example, in 64-bit configurations and operating in 10Gbps mode, the MAC receiver provides 64-bit word every 2 Rx clock cycles) except at the beginning and towards end of the Rx packet. The depth of asynchronous FIFO on the MAC and MTL interface is selected to avoid the overrun.

3.4.2 Application Clock Frequency Requirements for Memory Transfers

Due to the requirement of simultaneous read and write transfers from the FIFO memories, the application clock frequency should be such that the data transfer bandwidth of memory must be at least twice the bandwidth as that of the line. Also, additional bandwidth is required to allow delays due to contention between read and writes to a single memory block (odd or even).

The probability of contention increases with multiple Queues (on the Tx Side) specifically in XGMAC-AXI and XGMAC-MTL configurations because of the

- Possibility of receiving and writing interleaved data of different queues, all of which are to the same (odd or even) memory clocks in consecutive clock cycles.
- Non-sequential read and write operations of control words for each packet by scheduler in Tx side.
- Non-sequential write operations of status words for each packet in Rx side.

Note

- The values mentioned in Table 3-1 and Table 3-2 are pessimistic and estimated with a worst case scenario assumptions. The same application clock is used for Tx and Rx queues. So the application clock must be greater of the applicable frequency from Table 3-1 and Table 3-2.
- To handle valid frames of size less than 64 bytes, received in the line, increase the application clock frequency. This is because, the overhead due to status is considerably large on the Rx side.
- The computations in Table 3-1 and Table 3-2 assume 12 bytes of IPG and 8 bytes of PREAMBLE and SFD. To handle reduced IPG and PREAMBLE, increase the application clock frequency, to transfer the status in the available gaps between the packets on the application interface (MRI).

3.4.2.1 Transmit Side Clock Frequency Requirement

The minimum application clock frequency estimate to avoid underflow in the Transmit path of the shallow Asynchronous FIFO in Figure 3-5 on page 159, is shown in Table 3-1.

Table 3-1 Application Clock Frequency Estimate for Various Tx Queues and Data Width

Transmitter Operating Speeds		Data Width in Bits	Minimum Application Clock Frequency
XGMII	10Gbps	64	416.67 MHz
		128	312.5 MHz
	5Gbps	64	208.34 MHz
		128	156.25 MHz
	2.5Gbps	64	104.17 MHz
		128	78.125 MHz
GMII	1Gbps	64	62.5 MHz
		128	31.25 MHz
	2.5Gbps	64	156.25 MHz
		128	78.125 MHz
MII	100Mbps	64	6.25 MHz
		128	3.125 MHz
	10Mbps	64	625 KHz
		128	312.5 KHz

3.4.2.2 Receive Side Clock Frequency Requirement

On the Rx side, the contention occurs when the subordinate memory does not accept data continuously; in the worst case, contention can occur for every memory access, if the subordinate memory introduces one busy cycle after every transfer. However, status word writes steal cycles during the IPG (Inter-Packet Gap) of the received packets.

The number of status words to be written depends on the configuration. For example, IEEE 1588 requires 64-bit timestamps, VLAN stripping requires 32-bit VLAN tags and so on. The depth or area of the register Asynchronous FIFO must be larger, to avoid overflows.

Table 3-2 shows the minimum application clock frequency estimates to avoid overflows in the Receive path of the shallow Asynchronous FIFO at various operating/line rates.

Table 3-2 Minimum Application Clock Frequency in Various Speeds

Receiver Operating Speeds		Data Width in Bits	Minimum Application Clock Frequency
XGMII	10Gbps	64	124 MHz
		128	108 MHz
	5Gbps	64	62 MHz
		128	54 MHz
	2.5Gbps	64	31 MHz
		128	27 MHz
GMII	1Gbps	64	17 MHz
		128	14 MHz
	2.5Gbps	64	43 MHz
		128	35 MHz
MII	100Mbps	64	2 MHz
		128	1.5 MHz
	10Mbps	64	200 KHz
		128	150 KHz

4

Optional Features

This chapter provides information about the optional features of DWC_25gmac.

This chapter contains the following sections:

- "Source Address or VLAN Insertion, Replacement, or Deletion" on page 164
- "Double VLAN Processing" on page 167
- "Packet Filtering" on page 169
- "Flexible Receive Parser" on page 190
- "IEEE 1588 Timestamps" on page 204
- "Media Clock Generation and Recovery" on page 229
- "One-Step Timestamp" on page 236
- "PTP Offload Function" on page 242
- "IPv4 ARP Offload" on page 247
- "Memory Access in Debug Mode" on page 248
- "Flow Control" on page 252
- "Checksum Offload Engine" on page 259
- "TCP Segmentation Offload" on page 267
- "Receive Side Scaling Function" on page 274
- "QoS Function" on page 277
- "Management Counters" on page 280
- "ITU-T G999.1 Function Support" on page 286
- "External Lookup Interface" on page 287
- "VxLAN/NVGRE Support" on page 290

4.1 Source Address or VLAN Insertion, Replacement, or Deletion

The 25GMAC supports the following functions for Transmit packets:

- "Source Address Insertion or Replacement" on page 164
- "VLAN Insertion, Replacement, or Deletion" on page 165

You can enable the Source Address (SA) and VLAN insertion by selecting the Enable SA and VLAN Insertion on Tx option.

4.1.1 Source Address Insertion or Replacement

The application can use the SA insertion or replacement feature to instruct the MAC to do the following for Tx packets:

- Insert the content of the MAC Address Registers in the SA field
- Replace the content of the SA field with the content of the MAC Address registers

When SA insertion is enabled, the application must ensure that the packets sent to the MAC do not have the SA field. The MAC does not check whether the SA field is present in the Transmit packet and it inserts the content of MAC Address registers in the SA field. Similarly, when SA replacement is enabled, the application must ensure that the SA field is present in the packets sent to the MAC. The MAC replaces the six bytes following the Destination Address field in the Transmit packet with the content of the MAC Address registers.

You can enable the SA insertion or replacement feature for all Transmit packets or selective packets:

- Enabling SA insertion or replacement for all packets

  To enable this feature for all packets, program the SARC field of the MAC_Tx_Configuration register.

- Enabling SA insertion or replacement for selective packets

  To enable this feature for selective packets, use the following method depending on the configuration:

  - XGMAC-MTL or XGMAC-CORE:

    Use the SA Insertion Control input signals ati_sa_ctrl_i [2:0] and mti_sa_ctrl_i [2:0], respectively. When Bit 2 of ati_sa_ctrl_i or mti_sa_ctrl_i signal is high, it indicates insertion or replacement by the MAC_Address1 registers. When Bit 2 of ati_sa_ctrl_i or mti_sa_ctrl_i signal is low, it indicates insertion or replacement by the MAC_Address0 registers.

  - XGMAC-AXI:

    Program the SA Insertion Control field (Bits[25:23] of TDES3) in the first Transmit descriptor of the packet. When Bit 25 of TDES3 is set, the SA Insertion Control field indicates insertion or replacement by MAC_Address1 registers. When Bit 25 of TDES3 is reset, it indicates insertion or replacement by MAC_Address0 registers.

If MAC_Address1 registers are not enabled, the MAC_Address0 registers are used for insertion or replacement irrespective of the value of the most-significant bit of the SA Insertion Control field.

4.1.2 VLAN Insertion, Replacement, or Deletion

The software can use the VLAN insertion, replacement, or deletion feature to instruct the MAC to do the following for Tx packets:

- Delete the VLAN Type and VLAN Tag fields
- Insert or replace the VLAN Type and VLAN Tag fields

  Insertion or replacement is done based on the setting of `VLTI` bit in the `MAC_VLAN_Incl` register as described in Table 4-1.

Table 4-1 VLAN Insertion or Replacement Based on VLTI Bit

Condition	Description
VLTI bit is set	The MAC inserts or replaces the following: ■ VLAN Type field (C-VLAN or S-VLAN as indicated by the CSVL bit of the MAC_VLAN_Incl register) ■ VLAN Tag field with ❑ Content of the mti_vlan_tag_i signal in the XGMAC-CORE configuration ❑ VLT 1 field of the ATI control word in the XGMAC-MTL configuration ❑ Content of the VT field of Transmit context descriptor of the packet, in the XGMAC-AXI configuration
VLTI bit is reset	The MAC inserts or replaces the following: ■ VLAN Type field (C-VLAN or S-VLAN as indicated by the CSVL bit of the MAC_VLAN_Incl register) ■ VLAN Tag field with the VLT field of the MAC_VLAN_Incl register

When VLAN replacement or deletion is enabled, the MAC checks if the VLAN Type field (0x8100 or 0x88a8) is present after the DA and SA fields in the Transmit packet. The replace or delete operation does not occur if the VLAN Type field is not detected in two bytes following the DA and SA fields. However, when VLAN insertion is enabled, the MAC does not check the presence of VLAN Type field in the Transmit packet and just inserts the VLAN Type and VLAN Tag fields.

You can enable the VLAN insertion, replacement, or deletion feature for all Tx packets or selective packets:

- To enable this feature for all packets, program the VLC field of the `MAC_VLAN_Incl` register.
- To enable this feature for selective packets,
  - XGMAC-CORE configuration: Use the `mti_vlan_ctrl_i` input.
  - XGMAC-MTL configuration: Program the `VTIR` field of ATI control word.
  - XGMAC-AXI configuration: Program the `VTIR` field of TDES2 Normal Descriptor (see Table 13-4 on page 1801).

4.1.3 Queue/Channel Based VLAN Insertion on Tx

25GMAC supports channel/queue based VLAN tag insertion on all transmitted packets. Queue/Channel specific VLAN tag registers are accessed using indirect addressing through the `MAC_VLAN_Incl` register. VLAN type and tag value can be independently programmed for each queue/channel.

4.1.3.1 Enabling Queue/Channel Based VLAN Tag Insertion on Tx

To enable this feature while configuring the controller in the coreConsultant GUI:

Select the Enable Queue/Channel based VLAN tag insertion on Tx option under the General Features section during the Specify Configuration activity in coreConsultant.

For details about this option, see General Features Parameters in the *Parameter Descriptions* chapter.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

When you enable this feature and `CBTI` field set in the `MAC_VLAN_Incl` register, VLAN tag is inserted on every packet that is transmitted from a queue/channel, with the programmed tag value taken from queue/channel specific `MAC_VLAN_tag` register

4.1.3.2 Programming Guidelines for Queue/Channel Based VLAN Tag Insertion

“Programming Sequence for Queue/Channel Based VLAN Inclusion Register” on page 1862.

4.2 Double VLAN Processing

The 25GMAC supports the double VLAN tagging feature in which the MAC can process up to two VLAN tags (inner and outer). When you enable this feature, the 25GMAC supports the following:

- Insertion, replacement, or deletion of up to two VLAN tags in the Transmit path
- Packet filtering and stripping based on any one of the two VLAN Tags in the Receive path. Stripping and providing up to two VLAN Tags in the Receive path as a part of the Receive status

Enable this feature by selecting the Enable Double VLAN Processing option.

4.2.1 Transmit Path

Table 4-2 describes the features supported by the MAC on the Transmit side.

Table 4-2 Double VLAN Processing Features in Tx Path

Feature	Description
Support for C-VLAN and S-VLAN Tag types	The inner or outer VLAN tag can be of C-VLAN and S-VLAN type. The VLAN type is specified through the CSVL bit of MAC_VLAN_Incl and registers. The 25GMAC supports processing of any sequence of outer and inner VLAN tags. Note: The 25GMAC does not support the C-VLAN S-VLAN sequence. The MAC does not check whether the packet provided by the application has a valid sequence of the VLAN Tag types or the insertion or replacement operation results in invalid sequence of VLAN Tag type. Therefore, the application must provide correct sequence of VLAN Tag types and program the MAC in such a way that it results in correct sequence of VLAN Tag types in the transmitted packet. The application must ensure the following: ■ The inner tag must not be S-VLAN when outer C-VLAN Tag insertion is enabled. ■ The outer tag must not be C-VLAN when inner S-VLAN Tag insertion is enabled. ■ The inner tag must not be S-VLAN when outer tag must be replaced with C-VLAN. ■ The outer tag must not be C-VLAN when inner tag must be replaced with S-VLAN.
VLAN Tag deletion	You can enable the VLAN tag deletion for outer or inner tag through VLC field in the MAC_VLAN_Incl and register, respectively. When VLAN deletion is enabled, the MAC deletes the tag present at the corresponding position. When a packet has only one tag, it is considered as the outer tag. If inner tag deletion is enabled and the packet has only one tag, the MAC does not delete the tag.
VLAN Tag Insertion or Replacement	You can enable the VLAN tag insertion or replacement for outer or inner tag through VLC field in the MAC_VLAN_Incl and register, respectively. When VLAN tag insertion or replacement is enabled, the VLTI bit in the MAC_VLAN_Incl or MAC_Inner_VLAN_Incl register is used to determine whether the VLAN tag must be taken from the register or the Control Word.

4.2.2 Receive Path

Table 4-3 describes the features supported by the MAC on the Receive side and the corresponding bits in the `MAC_VLAN_Tag` register.

Table 4-3 Double VLAN Processing in Rx Path

Feature	Description
Outer or inner VLAN tag-based filtering	The MAC can filter packets based on the outer or inner VLAN tag through the ERIVLT bit.
C-VLAN or S-VLAN tag-based filtering	The MAC can filter packets based on the C-VLAN or S-VLAN type based on the ERSVLM bit.
Outer and Inner VLAN Tag stripping	The MAC can strip the outer and inner VLAN Tags from received frame based on the EVLS and EIVLS bits.
16-bit outer and inner VLAN Tag and Type in Rx status	The MAC can provide the 16-bit outer and inner VLAN Tag and Type in the Rx status based on the EVLRXS and EIVLRXS bits, respectively.
Disabling or skipping checking of outer VLAN Tag type	The MAC can disable or skip checking of outer VLAN Tag type to match C-VLAN or S-VLAN based on the DOVLTC bit.

4.3 Packet Filtering

The 25GMAC supports the following types of filtering for Rx packets:

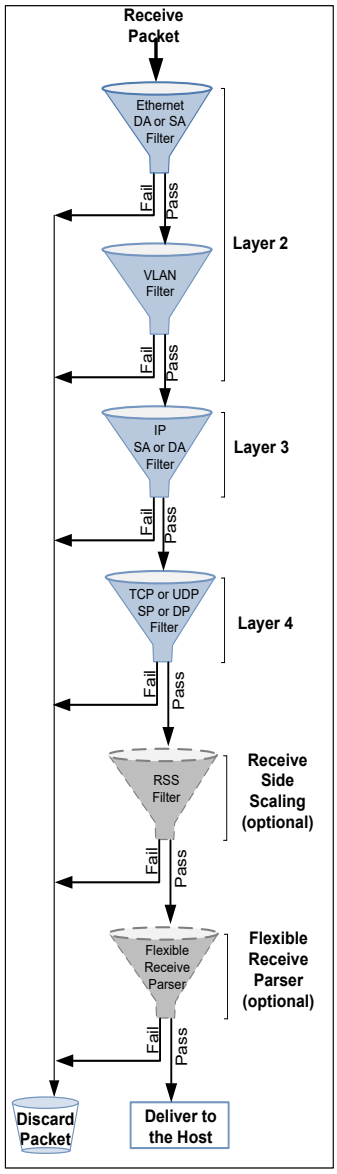
- "Source Address or Destination Address Filtering" on page 170
  The Address Filtering Module (AFM) checks the source address and destination address fields of each incoming packet.
- "VLAN Filtering" on page 173
  The 25GMAC supports the VLAN tag-based and VLAN hash filtering. The VLAN hash filtering is an optional feature which you can select through coreConsultant.

  When you do not enable the `DWCXG_ERVFE` parameter, the VLAN tag-based perfect filter is available in the configuration.
- "Extended VLAN Filtering" on page 175
  The 25GMAC supports multiple VLAN tag-based and VLAN hash filtering. Extended RX VLAN Filtering and Routing is an optional feature which you can select through the coreConsultant.

  When you enable the `DWCXG_ERVFE` parameter in the configuration, the VLAN tag-based perfect filter is available.
- "Layer 3 and Layer 4 Match Filtering" on page 187
  Layer 3 filtering refers to source address and destination address filtering. Layer 4 filtering refers to source port and destination port filtering. Layer 3 and Layer 4 filtering is an optional feature which you can select through coreConsultant.

Figure 4-1 shows the filtering sequence for Rx packets.

Figure 4-1 Packet Filtering Sequence



4.3.1 Source Address or Destination Address Filtering

The Address Filtering Module of the MAC checks the source address (SA) and destination address (DA) fields of each incoming packet.

The destination address hash filtering uses internal hash table when you select hash table size less than or equal to 256. When you specify the hash table size greater than 256, 25GMAC uses the external, MAC RXF memory based DA hash table for filtering and routing, through the `MAC_Indir_Access_Ctrl` and `MAC_Indir_Access_Data` registers. For more details see "Programming Guidelines for External DA Hash Table-Based Filtering and Routing" on page 1863.

4.3.1.1 Unicast Destination Address Filtering

The MAC supports up to 32 MAC addresses for unicast perfect filtering. If perfect filtering is selected (HUC field of the MAC_Packet_Filter register is reset), the MAC compares all 48 bits of received unicast address with the programmed MAC address for any match. The default MacAddr0 is always enabled.

The MacAddr1 to MacAddr31 addresses are selected with an individual enable bit. For MacAddr1 to MacAddr31 addresses, you can mask each byte during comparison with corresponding received DA byte by setting the corresponding Mask Byte Control bit in the register. This enables group address filtering for the DA.

In hash filtering mode (when HUC field is 1), the MAC performs imperfect filtering for unicast addresses using a hash table. 25GMAC uses internal hash table-based filtering, when the hash table size is less than or equal to 256. When the hash table size is greater than 256, 25GMAC uses external, RXF memory-based DA has table for filtering and routing.

For internal hash-table based filtering, the MAC uses the upper 6 bits (based on hash table size) CRC of the received destination address to index the content of the hash table. For example, for 64-bit sized hash table, a value of 6'b000000 selects Bit 0 of internal hash table register, and a value of 6'b111111 selects Bit 63 of internal hash table register (MAC_Hash_Table_Reg). If the corresponding bit (indicated by the 6-bit CRC) is set to 1, the unicast packet is considered to have passed the Hash filter; otherwise, the packet is considered to have failed the hash filter. A similar operation occurs when a 128-bit or 256-bit hash table is selected.

Note	A sample C routine that generates the 6-bit hash for an input DA is provided in the resources directory of your workspace.

4.3.1.2 Multicast Destination Address Filtering

To program the MAC to pass all multicast packets, set the PM bit in the MAC_Packet_Filter register. If the PM bit is reset, the MAC performs the filtering for multicast addresses based on the HMC field of the MAC_Packet_Filter register.

In perfect filtering mode, the multicast address is compared with the programmed MAC Destination Address registers (1-31). Group address filtering is also supported.

In hash filtering mode (when HMC field is 1), the MAC performs imperfect filtering for multicast addresses using hash table. 25GMAC uses internal hash-table based filtering, when the hash table size is less than or equal to 256. When the hash table size is greater than 256, 25GMAC uses external, MAC RXF memory-based DA hash table for filtering and routing.

In hash filtering mode, the MAC performs imperfect internal hash-table based filtering or external hash-table based filtering and routing, as described in "Unicast Destination Address Filtering" on page 171.

4.3.1.3 Hash or Perfect Address Filtering

To configure the DA filter to pass a packet when its DA matches either the Hash filter or the Perfect filter, set the HPF bit and the corresponding HUC or HMC bits in the MAC_Packet_Filter register. This is applicable to both unicast and multicast packets. If the HPF bit is reset, only one of the filters (Hash or Perfect) is applied to received packet.

4.3.1.4 Broadcast Address Filtering

The MAC does not filter any broadcast packets by default. To program the MAC to reject all broadcast packets, set the DBF field in the MAC_Packet_Filter register.

4.3.1.5 Unicast Source Address Filtering

The MAC can perform perfect filtering based on the source address field of received packets. By default, the MAC compares the SA field with the values programmed in the SA registers. You can configure the MAC Address registers [1-31] to use SA instead of DA for comparison by setting Bit 30 of the corresponding register.

The MAC also supports group filtering with SA. You can filter a group of addresses by masking one or more bytes of the address. The MAC drops the packets that fail the SA filter if the SAF bit is set in the MAC_Packet_Filter register. Otherwise, the result of the SA filter is given as a status bit in the Receive Status word. (see Table 2-19 on page 141). When the SAF bit is set, the SA filter and DA filter result is ANDed to decide whether the packet needs to be forwarded. This means that the packet is dropped if either filter fails. The packet is forwarded to the application only if the packet passes both filters in-order.

4.3.1.6 Inverse Filtering

For DA and SA filtering, you can invert the filter-match result at the final output by setting the DAIF and SAIF fields of the MAC_Packet_Filter register. The DAIF field is applicable for both unicast and multicast DA packets. The result of the unicast or multicast destination address filter is inverted in this mode. Similarly, when the SAIF field is set, the result of unicast SA filter is reversed.

Table 4-4 and Table 4-5 summarize the DA and SA filtering based on the type of packets received.

When the RA bit of the MAC_Packet_Filter register is set, all packets are forwarded to the system along with the correct result of the address filtering in the Rx Status.

Table 4-4 Destination Address Filtering

Packet Type	PR	HPF	HUC	DAIF	HMC	PM	DBF	DA Filter Operation
Broadcast	1	X	X	X	X	X	X	Pass
	0	X	X	X	X	X	0	Pass
	0	X	X	X	X	X	1	Fail
Unicast	1	X	X	X	X	X	X	Pass all packets
	0	X	0	0	X	X	X	Pass on Perfect/Group filter match
	0	X	0	1	X	X	X	Fail on Perfect/Group filter match
	0	0	1	0	X	X	X	Pass on Hash filter match
	0	0	1	1	X	X	X	Fail on Hash filter match
	0	1	1	0	X	X	X	Pass on Hash or Perfect/Group filter match

Table 4-4 Destination Address Filtering (continued)

Packet Type	PR	HPF	HUC	DAIF	HMC	PM	DBF	DA Filter Operation
	0	1	1	1	X	X	X	Fail on Hash or Perfect/Group filter match
Multicast	1	X	X	X	X	X	X	Pass all packets
	X	X	X	X	X	1	X	Pass all packets
	0	X	X	0	0	0	X	Pass on Perfect/Group filter match and drop Pause packets if PCF = 0x
	0	0	X	0	1	0	X	Pass on Hash filter match and drop Pause packets if PCF = 0x
	0	1	X	0	1	0	X	Pass on Hash or Perfect/Group filter match and drop Pause packets if PCF = 0x
	0	X	X	1	0	0	X	Fail on Perfect/Group filter match and drop Pause packets if PCF = 0x
	0	0	X	1	1	0	X	Fail on Hash filter match and drop Pause packets if PCF = 0x
	0	1	X	1	1	0	X	Fail on Hash or Perfect/Group filter match and drop Pause packets if PCF = 0x

Table 4-5 Source Address Filtering

Packet Type	PR	SAIF	SAF	SA Filter Operation
Unicast	1	X	X	Pass all packets.
	0	0	0	Pass status on Perfect or Group filter match but do not drop packets that fail
	0	1	0	Fail status on Perfect or Group filter match but do not drop packet
	0	0	1	Pass on Perfect or Group filter match and drop packets that fail
	0	1	1	Fail on Perfect or Group filter match and drop packets that fail

4.3.2 VLAN Filtering

The 25GMAC supports the following types of VLAN filtering:

- "VLAN Tag Perfect Filtering" on page 174
- "VLAN Tag Hash Filtering" on page 174

4.3.2.1 VLAN Tag Perfect Filtering

In VLAN tag perfect filtering, the MAC compares the VLAN tag of received packet and provides the VLAN packet status to the application. Based on the programmed mode, the MAC compares the lower 12 bits or all 16 bits of received VLAN tag to determine the perfect match.

If VLAN tag perfect filtering is enabled, the MAC forwards the VLAN-tagged packets along with VLAN tag match status and drops the VLAN packets that do not match. You can also enable the inverse matching for VLAN packets by setting the VTIM bit of the MAC_VLAN_Tag register. In addition, you can enable matching of S-VLAN tagged packets along with the default C-VLAN tagged packets by setting the ESVL bit of the MAC_VLAN_Tag register. The VLAN packet status bit (Bit 15 RDES2) indicates the VLAN tag match status for the matched packets.

Note The source or destination address (if enabled) has precedence over the VLAN tag filters. This means that a packet that fails the source or destination address filter is dropped irrespective of the VLAN tag filter results. By default, the VLAN tag-based perfect filter is available in all configurations.

4.3.2.2 VLAN Tag Hash Filtering

The MAC provides VLAN tag hash filtering with a 16-bit Hash table. You can enable this feature by selecting the Enable Address Filter VLAN Hash Table option in coreConsultant. The MAC performs the VLAN hash matching based on the `VTHM` field of the `MAC_VLAN_Tag` register. If the `VTHM` field is 1, the most significant 4 bits of CRC-32 of VLAN tag are used to index the content of the VLAN Hash Table register. A value of 1 in the VLAN Hash Table register, corresponding to the index, indicates that the VLAN tag of the packet matched and the packet must be forwarded. A value of 0 indicates that VLAN-tagged packet must be dropped.

Note The 16 or 12 bits of VLAN Tag are considered for CRC-32 computation based on ETV bit in MAC_VLAN_Tag register.

When ETV bit is reset, most significant four bits of CRC-32 of VLAN Tag are inverted and used to index the content of MAC_VLAN_Hash_Table register in the MAC.

When ETV bit is set, most significant four bits of CRC-32 of VLAN Tag are directly used to index the content of MAC_VLAN_Hash_Table register in the MAC.

The MAC also supports the inverse matching for VLAN packets. In the inverse matching mode, when the VLAN tag of a packet matches the perfect or hash filter, the packet must be dropped. If the VLAN perfect and VLAN hash match are enabled, a packet is considered as matched if either the VLAN hash or the VLAN perfect filter matches. When inverse match is set, a packet is forwarded only when both perfect and hash filters indicate mismatch.

Table 4-6 shows the various possibilities for VLAN matching and the final VLAN match status. When the `RA` field of the `MAC_Packet_Filter` register is set, all packets are received and the VLAN match status is indicated in the VF bit of the Receive Normal Descriptor (write-back format). When the RA bit is not set and the `VTFE` field is 1 in the `MAC_Packet_Filter` register, the packet is dropped if the final VLAN match status is Fail. In Table 4-6, value X implies that this column can have any value. For more details, see “Receive Normal Descriptor (Write-Back Format)” on page 1813.

When VLAN VID is programmed to 0 in the VL field of the `MAC_VLAN_Tag` register, all VLAN-tagged packets are considered as perfect matched but the status of the VLAN hash match depends on the `VTHM` and `VTIM` fields of the `MAC_VLAN_Tag` register.

Table 4-6 VLAN Match Status

VID	VLAN Perfect Filter Match Result	VTHM Bit	VLAN Hash Filter Match Result	VTIM Bit	Final VLAN Match Status
VID = 0	Pass	0	X[a]	X	Pass
	Pass	1	X	0	Pass
	Pass	1	Fail	1	Pass
	Pass	1	Pass	1	Fail
VID != 0	Pass	X	X	0	Pass
	Fail	0	X	0	Fail
	Fail	1	Fail	0	Fail
	Fail	1	Pass	0	Pass
	Fail	0	X	1	Pass
	Pass	X	X	1	Fail
	Fail	1	Pass	1	Fail
	Fail	1	Fail	1	Pass

a. In this table, X represents any value.

4.3.3 Extended VLAN Filtering

Based on the values of the DWCXG_ERVFE and DWCXG_NRVF parameters in the configuration, IP uses either internal Extended Rx VLAN Tag filters or external Rx VLAN Tag lookup.

When you select 4 to 32 Rx VLAN filters, IP enables Extended Rx VLAN filtering and routing. When you select more than 32 Rx VLAN filters, IP uses external Rx VLAN Tag Lookup-based filtering & routing.

Perfect filtering is done for the VLAN Tag Filters. For each VLAN Tag Filter, the VLAN Tag ID is compared. The combined result of all the enabled VLAN Tag Filters determines the overall VLAN Tag Filter result. The filter result is passed to the application as part of the status bits.

When the VLAN Tag Filter gives a pass result, the frame is considered to have passed the VLAN Tag Filters. If the frame fails all the filters, the frame is considered to have failed the VLAN filter.

In addition to Perfect Filtering, Hash Filtering can also be enabled. In Hash Filtering, the overall VLAN Filter result is determined by the Hash Filter result and the perfect filter result. The overall filter result determines if the frame is dropped or forwarded to the application based on the programming on the VTFE and RA bits of the MAC_Packet_Filter Register.

4.3.3.1 Comparison Modes of Extended Receive VLAN Filtering

For each VLAN Tag filter, the application has the following comparison modes:

- Program the MAC to compare an outer VLAN Tag or an inner VLAN Tag with the programmed VID.
- Choose the width of the VID field, 12 or 16 bits, for the comparison.
- Enable or disable type check for each filter. When type check is enabled, the application can choose if the VID comparison must be only for the SVLAN or CVLAN types frames. For example, when a filter is enable for 16 bit comparison, type is SVLAN, and outer VLAN tag, any single or double VLAN tagged frames with outer SVLAN tags are compared with this filter, and result is obtained.

4.3.3.2 Filtering

When Extended RX VLAN Filtering and Routing feature is enabled, the application can enable both Perfect and Hash Filtering. The overall VLAN Filter Result is based on the perfect filter result and the Hash Filter result (if enabled). The filter result is passed to the application as part of the status bits.

Perfect Filtering is done based on the `MAC_VLAN_Tag_Filter` registers. For each VLAN Tag Filter, the MAC compares the relevant VLAN Tag ID and gives a result. If any one of the VLAN Tag Filters gives a match then the frame is considered to have passed the VLAN Tag Filters. If the frame mismatches all the filters, then the frame is considered to have failed the VLAN filter. This behavior is applicable only when the Inverse Filtering is not enabled in `MAC_VLAN_Tag_Ctrl` Register.

If inverse Filtering is enabled and the frame has mis-matched all the relevant filters then it is considered to have passed the VLAN filter. If the frame matches any one of the relevant filters then it is considered as a fail. If none of the enabled filters can perform a comparison or if none of the filters are enabled, then the frame is bypassed to the application.

The overall filter result and the programming on the `VTFE` and `RA` fields of the `MAC_Packet_Filter` register determine if the frame is dropped or forwarded to the application.

- RA == 1:

  Frame is always forwarded; irrespective of the filter result.

- RA == 0 && VTFE == 1:

  Only if the VLAN Tag Filter result is a pass, MAC forwards the frame.

- RA == 0 && VTFE == 0:

  Frame is always forwarded; irrespective of the filter result.

If the frame is forwarded to the application, the relevant filter result is indicated through the Status bits.

4.3.3.3 Filter Status

MAC indicates the VLAN Filter Status through one bit in the status - VF in RDES2. When Extended RX VLAN Filtering and Routing is enabled, two status bits are used to indicate the comparison result of VLAN tags. The Outer VLAN Tag Filter Pass and Inner VLAN Tag Filter Pass bits are defined in the following positions. The status indicated through these bits is dependent on the programming:

In RDES2

- Bit 15 - Outer VLAN tag Filter Status
- Bit 13 - Inner VLAN Tag Filter Status

In ARI status: MAC_Packet_Filter Status

- Bit 15 - outer VLAN tag Filter Status
- Bit 13 - Inner VLAN Tag Filter Status

MRI Status:

- Bit 47 - outer VLAN tag Filter Status
- Bit 45 - Inner VLAN Tag Filter Status

Outer VLAN Tag Filter Status (`OTS`)

- This bit is valid for both Single and Double VLAN Tagged frames.
- When this field is
  - 1:
    - In perfect Filtering, with inverse filtering disabled, indicates that the Outer VLAN Tag of the frame has matched one of the VLAN Tag Filters.
    - In Inverse Filtering, indicates that the VLAN Tag of the frame has passed all the relevant VLAN Tag Filters.
  - 0:

    Indicates one of the following:
    - The Outer VLAN Tag of the frame has either failed the relevant Outer VLAN Tag Filters or bypassed them.
    - None of the filters are enabled for Outer VLAN Tag Comparison
    - In Inverse Filtering, indicates that it has failed at least one of the filters or bypassed all the filters programmed for Outer VLAN Tag Comparison

Inner VLAN Tag Filter Status (`ITS`)

- In perfect Filtering, without inverse filtering enabled, if this bit is set, it indicates that the frame's Inner VLAN Tag has matched one of the VLAN Tag Filters.
- If this bit is reset, it indicates that the frame's Inner VLAN Tag has either failed the relevant Inner VLAN Tag Filters or bypassed them.
- If none of the filters are enabled for Inner VLAN Tag Comparison, then this bit is reset.
- If Inverse Filtering is enabled and this bit is set, then the frame's VLAN Tag has passed all the relevant VLAN Tag Filters. If it is reset, then it has failed at least one of or bypassed all the Filters programmed for Inner VLAN Tag Comparison.
- This bit is valid for only Double VLAN Tagged frames, when Double VLAN Processing is enabled.

The application must look at the status bits and the programming to determine if the Frame has passed or failed the VLAN Filter.

Table 4-7 and Table 4-8 show the possible Filter combinations and the corresponding filter results. These tables explain the scenarios when Double VLAN Processing and Hash VLAN filter are enabled in the design.

Legend for the Table

- VTIM: VLAN Tag Inverse Match Enable, bit 17 in `VLAN_Tag_Ctrl` register.
- HFO: Hash Filter enabled for Outer VLAN Tag Comparison, bit 25 and bit 27 in `VLAN_Tag_Ctrl` register.
- HFI: Hash Filter enabled for Inner VLAN Tag Comparison, bit 25 and bit 27 in `VLAN_Tag_Ctrl` register.
- PFO: Perfect Filter comparison enabled for Outer VLAN Tag. Any of the `MAC_VLAN_Tag_Filter` registers is enabled (bit 16 == 1) and programmed for Outer VLAN Tag comparison (bit 20 == 0).
- PFI: Perfect Filter comparison enabled for Inner VLAN Tag. Any of the `MAC_VLAN_Tag_Filter` registers is enabled (bit 16 == 1) and programmed for Inner VLAN Tag comparison (bit 20 == 1).
- OTS: Outer VLAN Tag Filter Status
- ITS: Inner VLAN Tag Filter Status

Table 4-7 shows the possible values of status bits (OTS and ITS) when at least one Perfect filter is enabled.

Table 4-7 OTS and ITS Bit Values with At Least 1 Perfect Filter Enabled

VTIM	HFO	HFI	PFO	PFI	OTS	ITS
0	0	0	0	1	0	1/0
0	0	0	1	0	1/0	0
0	0	0	1	1	1/0	1/0
0	1	0	1	1	1/0	1/0
0	1	0	1	0	1/0	0
0	1	0	0	1	1/0	1/0
0	0	1	1	1	1/0	1/0
0	0	1	1	0	1/0	1/0
0	0	1	0	1	0	1/0
1	0	0	0	1	0	1/0
1	0	0	1	0	1/0	0
1	0	0	1	1	1/0	1/0
1	1	0	1	1	1/0	1/0
1	1	0	1	0	1/0	0
1	1	0	0	1	1/0	1/0
1	0	1	1	1	1/0	1/0
1	0	1	1	0	1/0	1/0

Table 4-7 OTS and ITS Bit Values with At Least 1 Perfect Filter Enabled (continued)

VTIM	HFO	HFI	PFO	PFI	OTS	ITS
1	0	1	0	1	0	1/0

Table 4-8 shows the possible values of status bits (OTS and ITS) when none of the perfect filters are enabled and only the VLAN Hash Filter is enabled.

Table 4-8 OTS and ITS Bit Values With Only VLAN Hash Filter Enabled

VTIM	HFO	HFI	OTS	ITS
0	0	0	0	0
0	1	0	1/0	0
0	0	1	0	1/0
1	0	0	0	0
1	1	0	1/0	0
1	0	1	0	1/0

With no perfect filters enabled, any VLAN packet is considered to have bypassed the perfect filter. If Hash Filter is enabled for one of the Tags, then the respective Status bit depends on the Filter's result. The Status bits are set to 0 if VLAN Hash Filter is also not enabled.

If the value of `ITS/OTS` is shown as 1/0; it indicates that the final result is dependent on the enabled relevant filter's result.

Example 1: The second row of Table 4-7 indicates that at least one Perfect Filter is enabled for Outer VLAN tag comparison and none of the filters are enabled for Inner VLAN tag comparison. Inverse VLAN Filtering is not enabled. The bit OTS is given as 1/0. If the received frame passes at the least one of the enabled Outer VLAN Tag filters then the bit is set to 1. If the frame doesn't pass any of the enabled Outer VLAN Tag filters, then the bit is set to 0.

Example 2: Last Row of Table 4-7 indicates that Inverse Filtering is enabled. Hash Filter and at the least one perfect filter is enabled for Inner VLAN Tag comparison. If the received frame's Inner VLAN tag mismatches with both the Hash Filter and all the enabled Perfect filters, the frame has the ITS == 1; otherwise, ITS == 0. OTS == 0 as no comparison is performed.

Limitations:

The status bits in the descriptor for Extended RX VLAN Filtering and Routing feature have certain restrictions based on other features that you have enabled. In some configurations, the entire VLAN Filter result is provided as VF instead of two separate bits: OTS and ITS. The following points describe the scenarios where a single VLAN Filter status is provided.

- When you select Enable External Lookup Interface, Enable Receive Side Scaling, and Enable Header-Payload Split
  - If either the OOS bit of the MAC_RSS_Control Register or the ELEN bit of the MAC_Rx_Configuration register is 1, then the ITS bit is not provided.
- When you select Only the Enable External Lookup Interface
  - If the ELEN field of the MAC_Rx_Configuration register is 1, ITS field is not provided. The bit 15 of RDES2 is defined as VF Status rather than OTS bit.
  - If the ELEN field of the MAC_Rx_Configuration register is not 1; the VLAN Filter result is provided as OTS and ITS in RDES2.
- When you select only Enable Receive Side Scaling, and Enable Header-Payload Split
  - If the OOS field of the MAC_RSS_Control register is 1, the ITS bit is not provided. The bit 15 of RDES2 is defined as VF Status rather than OTS bit.
  - If the OOS field of the MAC_RSS_Control register is not 1, the VLAN Filter result is provided as OTS and ITS in RDES2.
- When you do not select any of, Enable External Lookup Interface, Enable Receive Side Scaling, and Enable Header-Payload Split
  - The bits OTS and ITS both are valid.
  - The IVT & OVT fields of the RDES3 holds the VLAN Tag values if programmed accordingly.
- When you select Enable Support for Virtualized Network
  - The status bits OTS and ITS are valid and give the VLAN Tag Match for the Outer and Inner VLAN Tags of the Outer Header of a tunnel frame.
  - For a tunneled frame, the L2 and L3 Status fields of the RDES2 correspond to inner packet's fields. But the ITS status bit indicates the VLAN Filter status of the inner VLAN Tag of the Outer Packet.
  - For a tunneled frame, the VLAN Tags are not provided in the RDES0. So, VLAN Tag Stripping cannot be enabled for Tunnel frames. Also, only Filtering based on DWCXG_ERVFE is applicable; not routing.
  - For non-tunnel frames, both the Status bits are still valid as mentioned for Tunnel frames. The tags can also be provided in the Status.

When the Status bit 15 of RDES2 is defined as VF instead of OTS; the behavior is as follows.

- When Inverse Filtering is not enabled
  - The Status Bit is set if any of the VLAN Tag Filter passes.
  - If the frame's VLAN tags do not match any of the relevant VLAN Filters, then the VF bit is set to 0. In this case, if VTFE bit of the Packet Filter register is set, then the frame is not forwarded to the application.
  - If both the VLAN tags bypass the relevant filters, then the VF bit is 0. In this case, if VTFE bit of the Packet Filter register is set, the frame is forwarded to the application.
- When Inverse Filtering is enabled
  - VF bit is set only when both the Tags (if applicable) have mismatched the relevant enabled filters.
  - If one of the tags has mismatched the relevant filters and the other tag has bypassed it, the VF bit is set to 1.
  - If either of the tags matches any of the relevant filters, then the VF bit is set to 0.
  - If both the tags have bypassed the relevant filters, the frame is forwarded to the application with the status bit VF set to 0.

The VLAN Tag stripping is dependent on the individual tags match or mismatch. For example, if the MAC has "Strip on Tag Match" enabled for the Outer VLAN Tag; even though the Over All Filter Result as indicated by the Bit VF is a fail (set to 0); this tag is stripped.

4.3.3.4 Stripping

Each of the VLAN Tags has individual control over stripping. The programming options of Always strip, never strip, strip on pass and strip on fail are available. Inner or Outer VLAN Tag Stripping is based on the individual tags' pass or fail results. If a tag is bypassed by all the relevant filters then stripping is not applicable for it.

- If strip on Pass is enabled for the outer VLAN Tag, then the stripping occurs only if the Outer VLAN tag has passed the relevant Filters. The Outer VLAN Tag Filter Result bit is set.
- If strip on Fail is enabled for the outer VLAN Tag, then stripping occurs only if the Outer VLAN Tag has failed relevant filters. The Outer VLAN Tag Filter Result Bit is reset.
- If the Outer VLAN tag of the received frame is bypassed by the entire filter (no comparison has been made), then the tag is not stripped, though the Status Bit is still 0.
- As multiple filters are enabled, it is possible that the received VLAN frame could have matched any one or more of the filters. The VLAN Tag's value is not always deterministic from the filter status bits.
- So if the application strips the VLAN Tag based on the filter result, it might lose the VID. So if you have enabled stripping for any of the tags, put the tag in the status. For this, the application must enable the respective "VLAN Tag in Status" bit 24 or bit 31 in the MAC_VLAN_Tag_Ctrl Register.

4.3.4 Rx VLAN Tag Lookup-Based Filtering

25GMAC enables the external Rx VLAN Tag Lookup-based filtering and routing when you specify

- DWCXG_ERVFE = 1 && DWCXG_NRVF = 4096

The external Rx VLAN Tag lookup-based filtering and routing uses external 12-bit receive VLAN tag lookup-based packet filtering, routing and duplication. Based on the configuration, 25GMAC implements separate 4k lookup tables for both outer and inner receive VLAN tag in the external RXF memory. These lookup tables are accessible through `MAC_Indir_Access_Ctrl` and `MAC_Indir_Access_Data` indirect access register. For more details, see the *Register Descriptions* chapter.

You can configure the packet filtering, receive-channel routing and duplication information in respective inner/outer VLAN tag lookup entry. For more details regarding external VLAN tag lookup entry structure, see the `MAC_EXT_ROVTL_IndReg` and `MAC_EXT_RIVTL_IndReg` indirect register details in the *Register Descriptions* chapter.

Perfect filtering is performed using external Rx VLAN tag lookups. The receive 12-bit outer/inner VLAN Tag ID is used as outer/inner lookup index to select its entry. The filter result is sent to the application as part of the status bits, as follows:

- AF = 1, the frame is considered to have passed the VLAN Tag Filters.
- AF = 0, the frame is considered to have failed or bypass the VLAN filter based on UNL field. For more detail refer comparison mode section.
  - UNL = 1, bypass the VLAN filter or use the next VLAN lookup-based configuration.
  - UNL = 0, the frame is considered to have failed the VLAN filter.

When you enable Hash filtering, the overall VLAN filter result is determined by the results of both Hash filter and perfect filter. The overall filter result determines if the frame is dropped or forwarded to the application based on the programming of the `VTFE` and `RA` fields of the `MAC_Packet_Filter` Register.

4.3.4.1 Comparison Modes of Extended Receive VLAN Filtering

Program the `EROVTL` and `ERIVTL` fields of `MAC_VLAN_Tag_Ctrl` register to enable, disable, or bypass the outer and inner VLAN tag lookup with or without VLAN type check. For more details, see the *Register Descriptions* chapter.

Based on the configured VLAN tag type, outer VLAN tag is used for outer lookup indexing and inner VLAN tag is used for inner lookup indexing. Non-zero values of the `EROVTL` and `ERIVTL` fields determine the VLAN tag type to be considered for lookup filtering. If VLAN tag type is not configured, VLAN tag is ignored for filtering.

The result of the VLAN perfect filtering depends on the results of the Outer/Inner VLAN tag lookup, as shown in Table 4-9:

Table 4-9 VLAN Perfect Filtering Results

Outer VLAN tag lookup result	Inner VLAN tag lookup result (Double VLAN is configured, enabled and received packet has double VLAN)	VLAN Perfect Filtering result[a]
Bypass	Bypass	Bypass
	Fail	Fail
	Pass	Pass
Fail	X	Fail

Table 4-9 VLAN Perfect Filtering Results (continued)

Outer VLAN tag lookup result	Inner VLAN tag lookup result (Double VLAN is configured, enabled and received packet has double VLAN)	VLAN Perfect Filtering result[a]
Pass	X	Pass

a. Policing counter can override VLAN Perfect filtering result with fail, if it is enabled and condition has met to fail

Outer VLAN tag lookup decides the result, as shown Table 4-10:

Table 4-10 VLAN Tag Lookup Result

EROVTL	Is Rx Outer VLAN Type According to EROVTL ?	AF Field of Outer Lookup Entry	UNL Field of Outer Lookup Entry	Outer VLAN Tag Lookup Result
0	X	X	X	Bypass
Non-Zero	No	X	X	Bypass
	Yes	0	0	Fail
		1	X	Pass
		0	1	Bypass

If Double VLAN is configured, enabled and received packet has double VLAN, then Inner VLAN tag lookup decides result, as shown in Table 4-11:

Table 4-11 Inner VLAN Tag Lookup Result

ERIVTL Register Field	Is Rx Inner VLAN type according to ERIVTL ?	AF Field of Inner Lookup Entry	UNL Field of Inner Lookup Entry	Inner VLAN Tag Lookup Result
0	X	X	X	Bypass
Non-Zero	No	X	X	Bypass
	Yes	0	0	Fail
		1	X	Pass
		0	1	Bypass

4.3.4.2 Filtering

When you enable external Rx VLAN lookup-based filtering, routing, and duplication in the configuration, the application can enable both Perfect and Hash Filtering. The overall VLAN Filter result is based on the result of both perfect filter result and the Hash Filter result, if enabled. The filter result is passed to the application as part of the status bits.

Perfect Filtering is done based on configured Rx VLAN tag lookups. For each Rx VLAN tag lookup, the MAC uses relevant 12-bit VLAN Tag ID to select lookup entry for a result. Based on AF and UNL fields of entry, frame is considered to have pass/fail/bypass filtering based on Table 4-10 and Table 4-11. This behavior is applicable only when the Inverse Filtering is not enabled in the MAC_VLAN_Tag_Ctrl register.

If inverse filtering is enabled and the frame has passed the filtering then, it is considered to fail the filtering and vice-versa. Inverse filtering is applicable on combine filter result (Perfect and imperfect VLAN filtering). And it is not applicable if combined result is bypass.

The overall filter result and the programming of the VTFE and RA fields of the MAC_Packet_Filter Register determine if the frame is dropped or forwarded to the application.

- RA = 1: Frame is always forwarded, irrespective of the filter result.
- RA = 0 && VTFE = 1: Only if the VLAN tag filter result is a pass, MAC forwards the frame.
- RA = 1 && VTFE = 0: The frame is forwarded irrespective of the filter result. If the frame is forwarded to the application, then the relevant filter result is indicated through the Status bits.

Note When the source or destination address filtering is enabled, it has precedence over the VLAN tag filters. This implies that a packet that fails the source or destination address filter is dropped irrespective of the VLAN tag filter results.

4.3.4.3 Filter Status

The VLAN packet status bit (Bit 15 in RDES2, Bit 15 of MAC status data word of ARI status, Bit 47 in MRI status) indicates the Rx VLAN tag match status for the packets.

Following table shows the various possibilities of final VLAN filtering result status based on VLAN perfect and VLAN hash filter result. When the RA bit of the MAC_Packet_Filter register is set, all packets are received and the VLAN match status is indicated in the VF bit of the Receive Normal Descriptor (write-back format).

When the RA bit is not set and the VTFE bit is set in the MAC_Packet_Filter register, the packet is dropped if the final VLAN filtering result is Fail. In Table 4-12, value X implies that this column can have any value. But the final result and status depends on VLAN hash match result as well as VTHM and VTIM bits in the MAC_VLAN_Tag register. For more details, see “Receive Normal Descriptor (Write-Back Format)” on page 1813.

Table 4-12 VLAN Hash Match Result

VLAN Perfect Filter Result	VTHM Field	VLAN Hash Filter Match Result	VTIM Field	Final VLAN Filtering Result (Final VLAN Status)	Availability of External Rx VLAN Tag Lookup Based DMA Selection
Bypass	0	X	X	Bypass	No
	1	Result not available[a]	X	Bypass	No
		Fail	0	Fail	No
			1	Pass	No
		Pass	0	Pass	No

Table 4-12 VLAN Hash Match Result (continued)

VLAN Perfect Filter Result	VTHM Field	VLAN Hash Filter Match Result	VTIM Field	Final VLAN Filtering Result (Final VLAN Status)	Availability of External Rx VLAN Tag Lookup Based DMA Selection
			1	Fail	No
Pass	X	X	0	Pass	Yes
			1	Fail	No
Fail	X	X	0	Fail	No
			1	Pass	Yes

a. Result not available for VLAN hash implies that, considering its register programming and actual received packet, VLAN hash filtering is not qualified to use

Note To get used lookup index, enable VLAN stripping. Application can use stripped VLAN tag to get it. For more details, “Receive Normal Descriptor (Write-Back Format)” on page 1813.

4.3.4.4 Stripping

Each of the VLAN Tags has individual control over stripping. The programming options of Always strip, never strip, strip on pass and strip on fail are available. When programming option is always strip, it always stripes. When the programming option is never strip, it never stripes. When programming options are strip on pass or strip on fail, it uses respective inner/outer tag filtering result. Such results are based on the respective VLAN tag lookup and hash result considering inverse programming. If result is bypass, stripping is not applicable when programmed option is strip on pass or strip on fail. Table 4-13 describes outer VLAN tag filtering result for outer VLAN tag stripping

Table 4-13 Outer VLAN Tag Filtering Result for Stripping

Outer VLAN Tag Lookup Result[a]	VLAN Hash is Enabled for Outer Tag and VTHM bit is set	VLAN Hash Filter Match Result	VTIM Bit	Outer VLAN tag Filtering Result for Stripping
Bypass	No	X	X	Bypass
	Yes	Result not available[b]	X	Bypass
		Fail	0	Fail
			1	Pass
		Pass	0	Pass
			1	Fail
Pass	X	X	0	Pass

Table 4-13 Outer VLAN Tag Filtering Result for Stripping (continued)

Outer VLAN Tag Lookup Result[a]	VLAN Hash is Enabled for Outer Tag and VTHM bit is set	VLAN Hash Filter Match Result	VTIM Bit	Outer VLAN tag Filtering Result for Stripping
			1	Fail
Fail	X	X	0	Fail
			1	Pass

a. Policing counter can override outer VLAN tag lookup result with fail, if it is enabled and condition for fail is met. For more details see, “Policing Counters” on page 409.
b. Result not available for VLAN hash implies, considering its register programming and actual received packet, VLAN hash filtering is not qualified to use.

Table 4-14 describes inner VLAN tag filtering result for inner VLAN tag stripping. Applicable only when Double VLAN is configured, enabled and received packet has double VLAN.

Table 4-14 Inner VLAN Tag Filtering Result for Stripping

Inner VLAN Tag Lookup Result[a]	VLAN hash is enabled for Inner Tag and VTHM Bit is Set	VLAN Hash Filter Match Result	VTIM Bit	Inner VLAN Tag Filtering Result for Stripping
Bypass	No	X	X	Bypass
	Yes	Result not available[b]	X	Bypass
		Fail	0	Fail
			1	Pass
		Pass	0	Pass
			1	Fail
Pass	X	X	0	Pass
			1	Fail
Fail	X	X	0	Fail
			1	Pass

a. Policing counter can override inner VLAN tag lookup result with fail, if it is enabled and condition for fail is met. For more details see, “Policing Counters” on page 409.
b. Result not available for VLAN hash implies, considering its register programming and actual received packet, VLAN hash filtering is not qualified to use.

4.3.4.5 Enabling Extended Receive VLAN Filtering

To enable Extended Receive VLAN filtering while configuring the controller in the coreConsultant GUI:

Select the Extended Rx VLAN Filter Enable option under the Filtering section during the Specify Configuration activity in coreConsultant.

For details, see the “Filtering Parameters” in the *Parameter Descriptions* chapter.

Note	See the “Enabled” field of the parameter descriptions to understand the dependencies.

The number of VLAN perfect filters available depends on the value of the Number of VLAN Tag Filters (`DWCXG_NRVF` parameter). Using this feature,

- The application can enable a minimum of 4 and a maximum of 32 VLAN Tag filters.
- Every VLAN tag filter can be controlled by programming, to check the type and tag value of the received VLAN frame.
- The MAC can route the received VLAN traffic, based on the VLAN filter result and the enabled DMA channel number.

4.3.4.6 Programming Guidelines for Extended VLAN Filtering and Routing on Receive

For details about programming guidelines, see

- “Setting up Extended VLAN Filtering” on page 1865
- “Programming Guidelines for External DA Hash Table-Based Filtering and Routing” on page 1863
- “Setting up External Rx VLAN Tag Lookup-Based Filtering and Routing” on page 1866

4.3.5 Layer 3 and Layer 4 Match Filtering

The 25GMAC supports Layer 3 and Layer 4 based packet filtering. The Layer 3 filtering refers to the IP Source or Destination Address filtering in the IPv4 or IPv6 packets whereas Layer 4 filtering refers to the Source or Destination Port number filtering in TCP or UDP.

You can enable this feature by selecting the Enable Layer 3 and Layer 4 Packet Filter option in coreConsultant. The Layer 3 and Layer 4 packet filtering feature automatically enables the IPC Full Checksum Offload Engine on the Receive side. For Layer 3 or Layer 4 filtering operation, you must set the IPC bit of the `MAC_Rx_Configuration` register to enable the Rx Checksum Offload Engine.

When Layer 3 and Layer 4 filtering is enabled, the packets are filtered in the following way:

- Matched Packets: The MAC forwards the packets that match all enabled fields to the application along with the status. The MAC gives the matched field status only if the IPC bit of the `MAC_Rx_Configuration` register is set and one of the following conditions is true:
  - All enabled Layer 3 and Layer 4 fields match
  - At least one of the enabled field matches and other fields are bypassed or disabled

  When multiple Layer 3 and Layer 4 filters are enabled, any filter match is considered as a match. If more than one filter matches, the MAC provides the status of the lowest filter where Filter 0 is the lowest filter and Filter 7 is the highest filter. For example, if Filter 0 and Filter 1 match, the MAC gives the status corresponding to filter 0.

Note	The source or destination address and VLAN tag filters (if enabled) have precedence over Layer 3 and Layer 4 filter. This means that a packet which fails the source or destination address or VLAN tag filter is dropped irrespective of the Layer 3 and Layer 4 filter results.

- Unmatched Packets: The MAC drops the packets that do not match any of the enabled fields. You can use the inverse match feature to block or drop a packet with specific TCP or UDP over IP fields and forward all other packets.
  - XGMAC-CORE configuration:

    When a packet is dropped, a partial packet with appropriate terminate status is received on the MRI in the XGMAC-CORE configuration. This is because there is no buffer in the MAC receiver for storing the data until the TCP/IP fields are received and matched.
  - XGMAC-AXI configuration:

    The terminated or partial packets are dropped in the MTL Rx FIFO. If the Rx FIFO operates in the Threshold (cut-through) mode and the threshold is programmed to a small value such that packet transfer to the application starts before the failed Layer 3 and Layer 4 filter results are available, the application receives a partial packet with appropriate terminate status.
- Non-TCP or UDP IP Packets: By default, all non-TCP or UDP IP packets are bypassed from the Layer 3 and Layer 4 filters. You can optionally program the MAC to drop all non-TCP or UDP over IP packets.

By default, the Layer 3 & Layer 4 Match filter operate independently and the results of the L3 and L4 filter are not associated. However, when L5TEN (Bit 17 of `MAC_L3_L4_Control#(i)` register) is set, then the match results of both are combined (logical AND) to provide a single result.

In other words, this L3-L4 merged filter result is a PASS only when both the L3 and L4 filter gives a match result thus providing a 5-tuple filter for TCP/UDP packets.

This mode is valid only when either of (L3SA Match, L3DA Match) and either of (L4SP Match, L4DP Match) are enabled.

4.3.5.1 Layer 3 and Layer 4 Filters Register Set

The MAC implements a set of indirect registers for Layer 3 and Layer 4 based packet filtering. The indirect register set contains a control register namely, `MAC_L3_L4_Control0` register, to control the packet filtering. In addition, there are five address registers to program the Layer 3 and Layer 4 fields to be matched, such as:

- MAC_Layer4_Address0
- MAC_Layer3_Addr0_Reg0
- MAC_Layer3_Addr1_Reg0
- MAC_Layer3_Addr2_Reg0
- MAC_Layer3_Addr3_Reg0

You can configure the MAC to have up to 32 such independent set of registers. You can configure the first 7 filters with a granularity of 1. There after, the granularity is 2^3, 2^4, or 2^5.

4.3.5.2 Layer 3 Filtering

The 25GMAC supports perfect matching or inverse matching for IP Source Address and Destination Address. In addition, you can match the complete IP address or mask the lower bits matching, that is, compare all bits of the address except the specified lower mask bits.

For IPv6 packets filtering, you can use the 4 `MAC_Layer3_Addr[0:3]_Reg`(#i) registers to contain the 128-bit IP Source Address or IP Destination Address. The IP Source Address or Destination Address must be programmed in the order defined in the IPv6 specification, that is, the first byte of the IP Source Address or Destination Address in the received packet is in the higher byte of the register and the subsequent registers follow the same order.

For IPv4 packet filtering, you can use the MAC_Layer3_Addr0_Reg#(i) and MAC_Layer3_Addr1_Reg#(i) registers to contain the 32-bit IP Source Address and IP Destination Address, respectively. The remaining two registers are reserved. The IP Source Address or Destination Address must be programmed in the order defined in the IPv4 specification, that is, the first byte of IP Source Address and Destination Address in the received packet in the higher byte of the respective register.

4.3.5.3 Layer 4 Filtering

The 25GMAC supports perfect matching or inverse matching for TCP or UDP Source and Destination Port numbers. However, you can program only one type (TCP or UDP) at a time. The MAC_Layer4_Address(#i) register contains the 16-bit Source and Destination Port numbers of TCP or UDP, that is, the lower 16 bits for Source Port number and higher 16 bits for Destination Port number.

The TCP or UDP Source and Destination Port numbers must be programmed in the order defined in the TCP or UDP specification, that is, the first byte of TCP or UDP Source and Destination Port number in the received packet is in the higher byte of the register.

4.3.6 Hash Based Layer 3 and Layer 4 Filtering

25GMAC also supports an indirect Layer 3 and Layer 4 hash-based packet filtering. This function is described in detail in “Receive Side Scaling Function” on page 274.

When both Layer 3 and Layer 4 perfect match filtering and RSS Hash-based filtering are enabled and performed on the received packet, the packet is forwarded when any of the following conditions is met

1. When Layer 3-4 filter gives a match.

   For more details, see “Layer 3 and Layer 4 Match Filtering” on page 187.

2. Received packet is compatible for RSS hashing, where required Tuples are enabled. Also, the RSS hash lookup result indicates that the packet must be forwarded.

   For more details, see “Receive Side Scaling Function” on page 274

The received packet is dropped only when both the enabled filters indicate that the packet is to be dropped.

4.4 Flexible Receive Parser

4.4.1 Overview of Flexible Receive Parser

The 25GMAC controller has in-built programmable parser to parse the Ethernet packet based on a software controlled/programmable rule-set. This helps to support filtering and packet steering decision (DMA channel selection) of the received packet, based on any header field of existing protocols or custom/future protocols.

The parser uses a software programmed lookup table that contains instructions and filtering and routing decisions to be taken by the controller on the received packets.

- The maximum length of the packet data that the parser can operate on, is configurable.
  - 256B: 256 Bytes for all packets
  - 128B: 128 Bytes for all packets
  - 64B: 64 Bytes for all packets
- Support for up to 256 instructions, each processing 32-bit frame data with a mask
  - Match and Inverse Match support per instruction
  - Packet data compared with programmed data and mask bits
- Frame filtering based on any of the packet fields
  - Drop/accept packet or bypass packet filtering.
- Determine the DMA channel for packet parsing
- Support for out-of-order packet parsing
  - Decision tree can be out of order; decision tree need not be based on the same order as the packet data arrival.
- Flexible receive parser always operates in receive clock domain
- Flexible receive parser supports stream gate filtering, a subset of IEEE 802.1 Qci specification; stream filter classification and stream gate instance. Stream gate filtering is supported only in
  - Full duplex mode
  - 100 Mbps or higher link speed

4.4.2 Description of Flexible Receive Parser

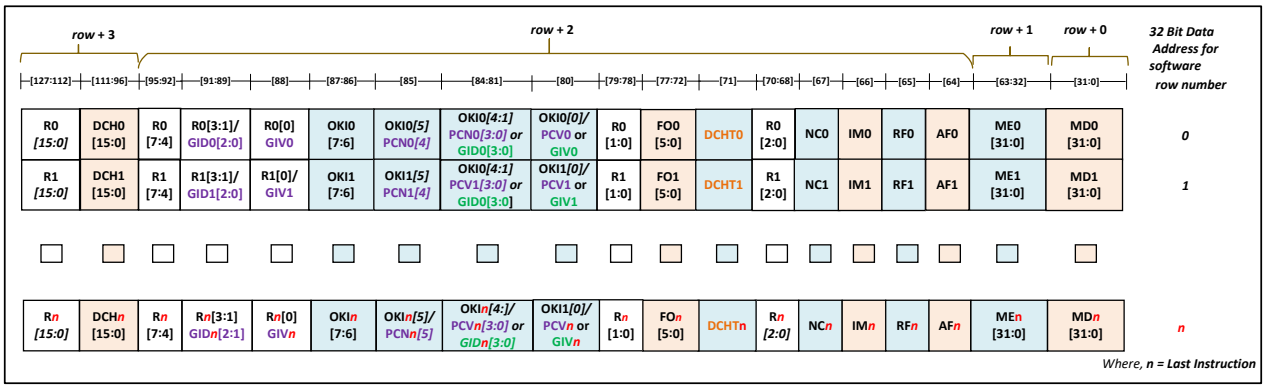
This section describes the block diagram and the details of the flexible receive parser.

4.4.2.1 Flexible Receive Parser Instruction Table (IT) Format

Before enabling the Flexible Receive Parser, the software must build the 128-bit wide instruction table for the parser. Each instruction is programmed as an entry in the instruction memory.

Figure 4-2 shows the graphical representation of the instruction table.

Figure 4-2 Flexible Receive Parser Instruction Table



The format of each entry in the instruction table is shown in Table 4-15.

Table 4-15 Flexible Receive Parser Instruction Entry Format

Field	Field Name	Description
31:0	MD	Match Data The 4-byte data. This data is used for comparing with incoming packet data starts at the frame offset as defined in [79:72] of the entry. The comparison is done only on those bits whose corresponding mask bits are setto1 (ME bits of [63:32]).
62:32	ME	Match Enable When the ME bit is set to 1, the corresponding packet data bit is used for comparing. Otherwise, corresponding data bit is don't care.
64	AF	Accept Frame
65	RF	Reject Frame
66	IM	Inverse Match
67	NIC	Next Instruction Control match = MD & ME Bits == (Packet data [frame offset* 8*4]) & ME bits If IM = 0; ENTRY_MATCH = match If IM = 1; ENTRY_MATCH = !match
		Continued... ■ When ENTRY_MATCH == 1 ❑ if AF == 1 and RF == 0, (PASS result) Accept and route the frame to DMA channel given by the DCH field (Bits[111:96]) of this entry ❑ else if AF == 0 and RF = 1, reject or drop the frame ❑ else if AF == 1 and RF = 1, (Routing BYPASS result)

Table 4-15 Flexible Receive Parser Instruction Entry Format (continued)

Field	Field Name	Description
		Accept the frame, but ignore DCH but route the frame to DMA Channel number given by MAC ❑ else if AF == 0 and RF == 0, (intermediate result) ● If NIC == 1 Continue to next sequential entry in the Instruction Table ● else NIC = 0 Jump to next entry given by OKI field (Bits[87:80]) in this entry ■ When ENTRY_MATCH == 0 (intermediate result) ❑ if NIC == 1 Jump to next entry given by OKI field (Bits[87:80]) in this entry ❑ else NIC == 0 Continue to next sequential entry in the instruction table
71:68	Reserved	
77:72	FO	Frame Offset [77:72] indicates the frame offset in terms of 4 bytes Frame offset in the packet data to be compared for match, in terms of 4 bytes. ■ 0 (value): 0 (frame offset in bytes) ■ 1: 4 ■ 2: 8 ■ ... ■ 63: 252 Note: The width of this field depends on the number of configured parsable bytes ■ 6 bits wide: 256 parsable bytes ■ 5 bits wide: 128 parsable bytes (bit[77] is reserved) ■ 4 bits wide: 64 parsable bytes (bit [77:76] is reserved)
79:78	Reserved	
88:80	OKI/PCV, PCN / GIV,GID	OK Index (OKI) This field gives the next entry in the instruction table, to be processed, OKI field is valid only when, ■ ENTRY_MATCH = 0 and NIC = 1 ■ ENTRY_MATCH = 1 and NIC = 0, AF = 0, RF = 0. Note: The OKI width depends on the configured FRP entries (Instructions); ■ 9 bits: 512 instructions. ■ 8 bits: 256 instructions; bit [88] is reserved.

Table 4-15 Flexible Receive Parser Instruction Entry Format (continued)

Field	Field Name	Description
		■ 7 bits: 128 instructions; bits [88:87] are reserved. ■ 6 bits: 64 instruction; bits [88:86] are reserved. When you enable the Policing Counter feature, interpret this field as follows: x = 80 + (log2(DWCXG_NUM_PC) ■ Bit [80]: PCV (Policing Counter Valid) ■ Bits [x: 81]: PCN (Policing Counter Number) ■ Bits [88: x+1]: Reserved For more information about PCN, PCV, see “Programming Guidelines for the Policing Counters” on page 1874. When you do not enable Policing Counter feature, but enable Stream-Gate filtering feature, and when AF = 1 and NIC = 0, interpret this field as follows: ■ Bits [80]: GIV ■ Bit [84:81]: GID ■ Bits [88: 85]: Reserved
88	GIV	Gate-ID Valid When this field is 1, ENTRY_MATCH = 1, and AF = 1, indicates that the packet is further checked against the Stream Gate Filter, given by GID. For more details about stream-gate filtering, see “Programming the Stream-Gate Filtering Registers” on page 1868. **Note:** Only when both Stream-Gate Filtering and Policing Counter features are enabled, interpret this field as GIV. Otherwise, this field is reserved.
91:89	GID	Gate-ID This field represents the Stream-Gate filter number to be applied on the packet, when GIV = 1. **Note:** Only when both Stream-Gate Filtering and Policing Counter features are enabled, interpret this field as GID. Otherwise, this field is reserved.
95:92	Reserved	
111:96	DCH	DMA Channel Number Indicates the DMA Channel Number when ENTRY_MATCH==1 and AF == 1. The encoding is as follows: ■ bit[0]: DMA channel number 0 ■ bit[1]: DMA channel number 1 ■ bit[2]: DMA channel number 2 ■ ... ■ bit[15]: DMA channel number 15

Table 4-15 Flexible Receive Parser Instruction Entry Format (continued)

Field	Field Name	Description
		Note: The DMA Channel number is encoded bitwise. 25GMAC IP supports multicast/broadcast across DMA channels. For more information see, “Multicast and Broadcast Support” on page 200 The field width depends on configured DMA Channels. 16 Channels: 16 bits 15 Channels: 15 bits and so on. Remaining bits are reserved.
127:112	Reserved	These bits are not implemented. This field is only for software to view.

Note

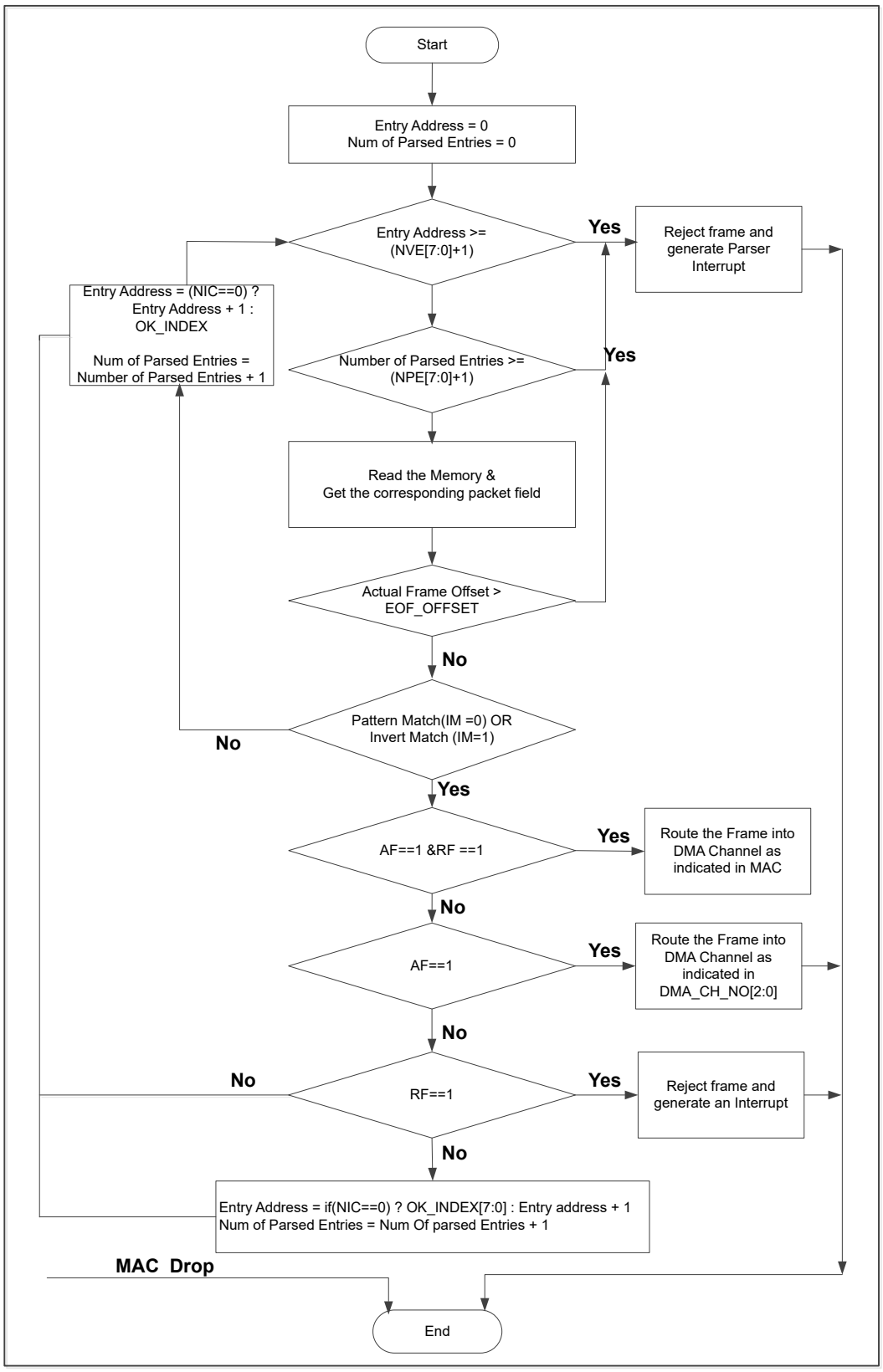
- Instruction Table Width: 112 bits
- Instruction Table Depth: Configurable - 256, 128, or 64.
- Byte Ordering: The first byte received on the line is compared with MD[7:0] of the instruction table.
- Entry and instruction are used synonymously.
- All the reserved bits are read as zeros.

Note

Error Correction Code (ECC) is used only for security purpose. During programming, ECC is calculated for the whole instruction data (except MSB, reserved bits) as well its address. After programming, 25GMAC recomputes the ECC and compares with existing ECC bits. If it finds a mismatch, it generates safety interrupt. For more details, see “Description of ECC Protection for Memories” on page 429

The parser begins parsing from the 0^{th} entry, for each received packet. The subsequent parsing entry location (next entry or OKI) depends on the parser result of the current entry. Figure 4-3 shows the flexible receive parser flow.

Figure 4-3 Flexible Receive Parser Flow Chart



4.4.2.2 Software Access to the Instruction Table

The software can write/read the flexible receive parser instruction table using the following register sets through indirect addressing:

- MTL_Indirect_Acc_Control_Status
- MTL_Indirect_Acc_data

Note The indirect registers are shared by multiple indirect accessible blocks. Set the ACCSEL field of the MTL_Indirect_Acc_Control_Status register to access the Instruction Table.

It is recommended not to update valid entries in the instruction table when frame reception and frame parsing is in progress. This can cause unintended results.

For more details, see the *Register Descriptions* chapter.

Note To program multiple parsing scheme, program them across instruction table and select either of it to run it first by programming the following at instruction 0:

- Set all bits to 0 except OKI. Set OKI to the first instruction location of the parsing scheme which you want to run first.

It is possible to reprogram the unused instructions when parsing is in progress, to build a new instruction list (or sequence).

4.4.2.3 Number of Valid Entries in Instruction Table (NVE)

This is the number of valid entries in the Instruction Table starting from the 0th instruction. The default value is set to number of supported instructions minus 1. Software can override the default value by writing into the NVE field of the MTL_RXP_Control_Status register (Number of Valid Entries). Possible values can be from 0 to number of supported instructions minus 1; 0 implies 1st instruction, 1 implies 2nd instruction, and so on.

While parsing, if the instruction address is greater than NVE, an error is flagged in the NVEOVIS field of MTL_RXP_Interrupt_Control_Status register. Also, 25GMAC generates an interrupt status, MTLPIS field of the MTL_Interrupt_Status register.

Note NVE Overflow error status is reported as consolidated error status in the receive descriptor (write-back format). For more information, see “Receive Normal Descriptor (Write-Back Format)” on page 1813

4.4.2.4 Number of Parsable Entries (NPE)

Indicates the number of entries/instructions that can be parsed on an incoming packet. The default value is set to number of supported instructions minus 1. Software can override the default value by writing into MTL_RXP_Control_Status registers field (Number of Parsable Entries). Possible values can be from 0 to number of supported instructions minus 1; 0 implies 1st parsable instruction and so on.

During parsing, if the number of parsed entries is more than NPE+1, an error is flagged in the NPIOVIS field of MTL_RXP_Interrupt_Control_Status register. Also, 25GMAC generates an interrupt status in the MTLPIS field of MTL_ Interrupt_Status register.

Note

NVE indicates the number of valid entries starting from the 0th entry of the instruction table built by the software, whereas NPE indicates the worst case number of parsable entries considering the various possible paths that can be taken.

NVE Overflow error status is reported as consolidated error status in the receive descriptor (write-back format). For more information, see "Receive Normal Descriptor (Write-Back Format)" on page 1813

4.4.2.5 Frame Offset

Indicates the frame offset, in terms of 4 bytes; used to compare respective receive frame offset data with the instruction's match data.

In an instruction, an error is flagged in the `MTL_RXP_Interrupt_Control_Status` register, FOOVIS (Frame Offset Overflow Interrupt Status) when Frame Offset is more than packet's EOF size or supported number of parsable bytes.

Also, 25GMAC generates an interrupt status in the MTLPIS field of `MTL_Interrupt_Status` register.

Note

- If the frame ends after the (frame_offset *4) bytes but before ((frame_offset *4)+3) bytes, the Receive parser declares Frame Offset Error if comparison is enabled (using MASK bits) for the non-received bytes; from (frame_offset *4) to ((frame_offset *4)+3).
- Frame Offset Overflow error status is reported as a consolidated error status in the receive descriptor (write-back format). For more details, see "Receive Normal Descriptor (Write-Back Format)" on page 1813

4.4.2.6 VLAN Strip Handling

The MAC supports stripping the outer as well as inner VLAN tags. When MAC is programmed to strip the VLAN tags, the Flexible Receive parser considers the stripped VLAN tags as part of the incoming packet.

4.4.2.7 PAD Strip/CRC Strip Handling

The pad strip and CRC strip are controlled by the software and applicable to all the received packets. When these features are enabled, software must build the Rx parser instructions accordingly.

Note

Enable PAD and CRC stripping in real use case.

4.4.2.8 Runt Packet Handling

The Rx parser is designed for minimum packet size to be of 64B. If the runt packet is received without an additional IPG (at least 64B minus Runt Packet size), the ongoing parsing of preceding packet could be impacted.

4.4.2.9 Out of Order Processing

The Rx parser flow can be out of order within the bytes of a received frame. The decision tree need not be based on the order in which packet arrives. This implies that the next entry and next frame offset (FO[]) can be less than the current instruction number and current Frame Offset.

For example, you can look for matching/mismatching IP header and then look for Ethernet Header to decide on Filtering/Channel Selection. Also, it can skip from higher address entry to lower address entry in the table.

4.4.2.10 Ethernet Line Speed Dependency

Flexible receive parser is implemented in the receive (Rx) clock domain.

Example: For 10G Line Speed and 312.5MHz receive (Rx) clock, assuming worst case example of back-to-back 64 bytes frame reception.

The processor has 67.2ns (including 64B of next frame, 12B of IPG, 8B of Preamble time of next to next frame) to process the current frame.

Number of parser cycles available at 312.5MHz: 67.2ns/3.2ns = ~21 cycles.

Flexible receive parser stops the current parsing when it detects SOF of the next of next packet. To account for the implementation latency, actual parsing cycles are: 21 - 3 = 18.

So, with one Parallel Instruction Processor Engine, the parser can process up to 18 instructions. The number of Parallel Instruction Processor Engines (PIPEs) can be increased, to have a greater instruction processing capacity. At higher (receive) clock speeds of the parser, fewer PIPEs can be accommodated.

Use the DWCXG_FRP_PIPE parameter to set the number of Parallel Instruction Processor Engines (PIPEs).

Table 4-16 Instruction Parsing Capacity with Different Parallel Instruction Processor Engines (PIPEs)

Line Speed	Flexible Receive Parser Clock (Rx Clock)	No of PIPEs	Worst-Case Available Time for Processing	Available Clock Cycle	Parsable Instructions
10G	312.5MHz	6	67.2ns	~18	18 * 6 =~ 108
10G	312.5MHz	5	67.2ns	~18	18 * 5 =~ 90
10G	312.5MHz	4	67.2ns	~18	18* 4 =~ 72

Increase of PIPEs can impact area and timing. Supporting 5 or 6 PIPEs at 312.5 MHz requires higher drive strength, and/or LVT gates.

The number of PIPE engines to be activated, is controlled through the programming of ANP field of MTL_RXP_Control_Status register. This feature allows the user to select smaller number of PIPEs at higher operating speed, if the logic path timing is not met with all PIPEs activated together with the target clock frequency.

For example, say the number of PIPEs included in the design during RTL generation DWCXG_FRP_PIPE = 6. Considering that this logic path has large number of combinational logic levels, it is possible that the timing is not met for this path for say (25Gb/s mode). But if timing is met with 3 PIPEs @ 781.25Mhz, then user can program this register field to 3 for safe operation.

When the link speed changes, it is assumed that software does not program the number of parsable entries/instructions beyond the intended limit as per the calculations in Table 4-16. If programmed incorrectly, parser does not parse the packet further and flags a parsing error and is treated as NPE[] overflow.

Note There are no constraints on the number of JUMPs in the Instruction Table. The architecture does not have additional latency to execute the JUMPs. Table 4-16 shows the worst-case number of Instructions that Rx Parser can parse, assuming receive clock of 312.5 MHz.

4.4.2.11 Frame Accept

In case of an Entry Match (considering the Inverse match) and Frame Accept bit is set and RF is not set, the frame is accepted and routed to the corresponding instruction's DMA channel number.

4.4.2.12 Frame Reject

In case of an Entry Match (considering the Inverse match) and Frame Accept bit is not set and Frame Reject bit is set, the frame is dropped.

An interrupt is generated when the frame is dropped due to AF = 0 and RF = 1; PDRFIE field of the MTL_RXP_Interrupt_Control_Status register must be set to 1.

25GMAC also generates an interrupt status, MTLPIS bit of the MTL_Interrupt_Status register.

4.4.2.13 Flexible Receive Parser Bypass Routing

In case of an Entry Match (considering the Inverse match) and AF and RF bits are set, accept the frame, but bypass the DMA channel routing selected by the flexible receive parser. In such cases DMA channel number provided by MAC/MTL, based on one of the filtering and routing mechanisms is selected.

4.4.2.14 DMA Channel Selection

The MAC/MTL also determines the DMA Channel number based on incoming packet fields.

When the parser is enabled (FRPE field is set to 1 in the MTL_Operation_Mode register) the parser overrides the DMA selection criteria of the MAC. If the static Rx Queue to DMA Channel mapping is selected, it has higher priority over Rx parser routing information. The Rx parser and MAC_Packet_Filter results are considered for packet filtering, but not the routing information.

When the parser is disabled (FRPE field is set to 0 in the MTL_Operation_Mode register) the MAC/MTL determines the DMA channel number.

Frames that pass the FRP filtering, are routed based on the Instruction Table. All the other frames that meet any of the following criteria, are treated as residual frames.

- Parsing errors such as NVE, NPE, FOO, and so on.
- Incomplete parsing
- Any other frames that could not be dropped despite the drop result from FRP. For more details, see "Receive Packet Drop Indication" on page 202.

Residual frames are routed to

- Channel 0, for express traffic
- Channel number configured in the FPE_RCH field of MTL_RXP_Control_Status register, for preemption traffic.

Note DMA channel number in the instruction has one-hot encoding. So, the frame is routed to the channel whose respective bit is set.

4.4.2.15 Receive Queue Selection

The Rx Queue is determined by the existing MAC functionality, irrespective of the enabling/disabling of the Rx Parser. This is because the Rx parser is mainly designed for selecting the DMA channel and Rx queuing is decided by the MAC to enable the proper functioning of the Pause/PFC feature.

Note When there are multiple queues with multiple DMA Channels enabled, a specific DMA channel might receive packets out-of-order. This is because, queue selection is based on MAC criteria (VLAN Priority and other criteria) and RxDMA channel selection is based on Rx parser. If in-order packet reception is necessary, you must program Single Queue configuration.

4.4.2.16 MAC (L2) Packet Filtering/Drop/Error Handling

The MAC does the packet filtering based on the received packet fields. MAC filtering/L2 filtering can also be enabled with parser. But MAC filtering has higher priority over flexible receiver parser.

Flexible receiver parser 's result has impact on RA (Receive All) and L2 Filtering (such as SA, DA, VLAN).

Table 4-17 RXP Impact on RA and L2 Filtering

RA (Receive All)	L2 Filter Pass (MAC)	Impact on Flexible Receive Parser (FRP) Result
No	No (L2 Drop)	Bypass FRP result.
No	Yes (L2 Accept)	FRP result is used.
Yes	No (L2 Drop)	Bypass FRP result.
Yes	Yes (L2 Accept)	If FRP result is Accept, use FRP Result. If FRP result is Fail/Drop, bypass the FRP result and route the frame to residual channel based on the traffic. For more details, see “DMA Channel Selection” on page 199.

Note When you enable Flexible Receive Parser, use only L2 filtering and routing (DA, SA, and VLAN).

4.4.2.17 Multicast and Broadcast Support

25GMAC supports multicast and broadcast only when DDS (DA duplication select) is enabled and the packet is routed to the highest queue number.

When Flexible Receive Parser is enabled, the packet routing is decided by the DMA Channel Numbers (DCH[]) in the Instruction Table. In other words, such multicast packets are transferred to all the DMA channels for which the DCH bits are set.

Note When a frame does not fulfill the requirement for broadcast/multicast, Flexible Receive Parser routes the frame to the to the lowest selected DMA Channel Number in the instruction table.

4.4.2.18 Statistical Counters

When Flexible Receive Parser is enabled, following counters are implemented in the indirect IFR register block:

1. `MTL_RXP_Drop_Cnt`

   Incremented when Flexible Receive Parser drops a packet due to AF = 0 and RF = 1.

2. `DMA_RXP_Error_Cnt`

   Incremented when a RXP encounters error. The error scenarios are

   - Current Instruction Number > NVE[]
   - Number Parsed Instructions > NPE[] + 1
   - Current Instruction's Frame offset > EOF frame offset or last parsable frame offset.
   - Parsing is not complete even after receiving the next to next packet SOF.

3. `MTL_RXP_Bypass_cnt`

   This is incremented when Rx Parser bypasses a packet due to AF = 1 and RF = 1.

4. `DMA_CH(#i)_RXP_Accept_cnt (i=0;i<DWCXG_NUM_DMA_RX_CH)`

   This is per DMA Channel register. It is incremented when a Rx Parser accept a packet due to AF = 1 and RF = 0.

   For more details, see "Software Access to Indirect FRP Register (IFR) Block" on page 201.

4.4.2.19 Software Access to Indirect FRP Register (IFR) Block

The software can write/read the flexible receive parser's Indirect Registers using the following register sets through indirect addressing:

- `MTL_Indirect_Acc_Control_Status`
- `MTL_Indirect_Acc_data`

Note

All indirect Registers of flexible receive parser are 32-bit wide.

The indirect registers are shared by multiple indirect accessible blocks. Set the ACCSEL field of the MTL_Indirect_Acc_Control_Status register to access the indirect Register block of flexible receive parser.

For more information, see the *Register Descriptions* chapter.

4.4.2.20 Receive Cut-Through Functionality

In the cut-through mode, the Rx controller attempts to transfer the received packets to DMA just after the cut-through threshold amount of packet data is received.

However due to the possible delay in the result of Flexible Receive parser, worst case latency can be more than the programmed cut-through threshold. As the packet is held in the FIFO until the result from the Flexible Receive Parser is determined, the cut-through functionality is re-defined. This is because, the Rx controller attempts to transfer the packet to DMA after at least RTC number of bytes are received and Flexible Receive Parser results are available.

4.4.2.21 Receive Packet Drop Indication

In most cases, the packet is dropped internally in the Rx Queue. However, back-to-back packets can arrive for the same queue and Rx parser result for the first packet is not available before the second packet arrives. In such cases, the packet cannot be flushed internally and therefore, it is still forwarded to the DMA or to the Application Interface and the status is indicated accordingly.

For more details, see FRPSM and FRPSL bits of RDES2 in "Receive Normal Descriptor (Write-Back Format)" on page 1813.

4.4.2.22 Uncorrectable ECC Error Handling

When the Rx parser detects an uncorrectable ECC error while checking the instruction table, software gets the interrupt for unrecoverable error. In such a case, software must re-program the instruction table.

For more details, see "Error Correction Code (ECC) Protection for Memories" on page 429.

In such a case, FRPE bit is reset and parsing is suspended for all subsequent packets.

4.4.3 Enabling the Flexible Receive Parser

To enable the Flexible Receive Parser, set FRPE bit to 1, in the `MTL_Operation_Mode` register.

It is not required to disable the L2 filtering when flexible receive parser is enabled, though L2 filtering result has an effect on flexible receiver parser result.

After setting FRPE bit to 1, optionally, software can wait until RXPI field of `MTL_RXP_Control_Status` register becomes 1 (active).

For details about these options, see "MAC (L2) Packet Filtering/Drop/Error Handling" on page 200.

To enable Flexible Receive Parser while configuring the controller in the coreConsultant GUI:

1. Set the Number of DMA channels on Receive Side to a value greater than 1, in the Multiple Queues Support.
2. Select Enable Flexible Receive Parser, in the Filtering features.

Note See the "Enabled" field of the parameter descriptions to understand the dependencies.

4.4.4 Registers Related to the Flexible Receive Parser

- `MTL_Operation_Mode`
- `MTL_Interrupt_Status`
- `MTL_RXP_Control_Status`
- `MTL_RXP_Interrupt_Control_Status`
- `MTL_Indirect_Acc_Control_Status`
- `MTL_Indirect_Acc_Data`
- `MTL_RXP_Drop_Cnt` (Indirect Access Register)
- `MTL_RXP_Error_Cnt` (Indirect Access Register)
- `DMA_CH_RXP_Accept_Cnt` (Indirect Access Register)

Indirect Access Registers are implemented in the Indirect FRP (IFR) register block. For more details, see "Software Access to Indirect FRP Register (IFR) Block" on page 201.

For more details, see the *Register Descriptions* chapter.

4.4.5 Programming Guidelines for Flexible Receive Parser

For details of programming guidelines for Flexible Receive Parser, see "Setting Up the Flexible Receive Parser" on page 1868.

4.5 IEEE 1588 Timestamps

The IEEE 1588 defines a Precision Time Protocol (PTP) which enables precise synchronization of clocks in measurement and control systems implemented with technologies such as network communication, local computing, and distributed objects. The PTP applies to systems communicating by local area networks supporting multicast messaging, including (but not limited to) Ethernet. This protocol enables heterogeneous systems that include clocks of varying inherent precision, resolution, and stability to synchronize. The protocol supports system-wide synchronization accuracy in the sub-microsecond range with minimal network and local clock computing resources.

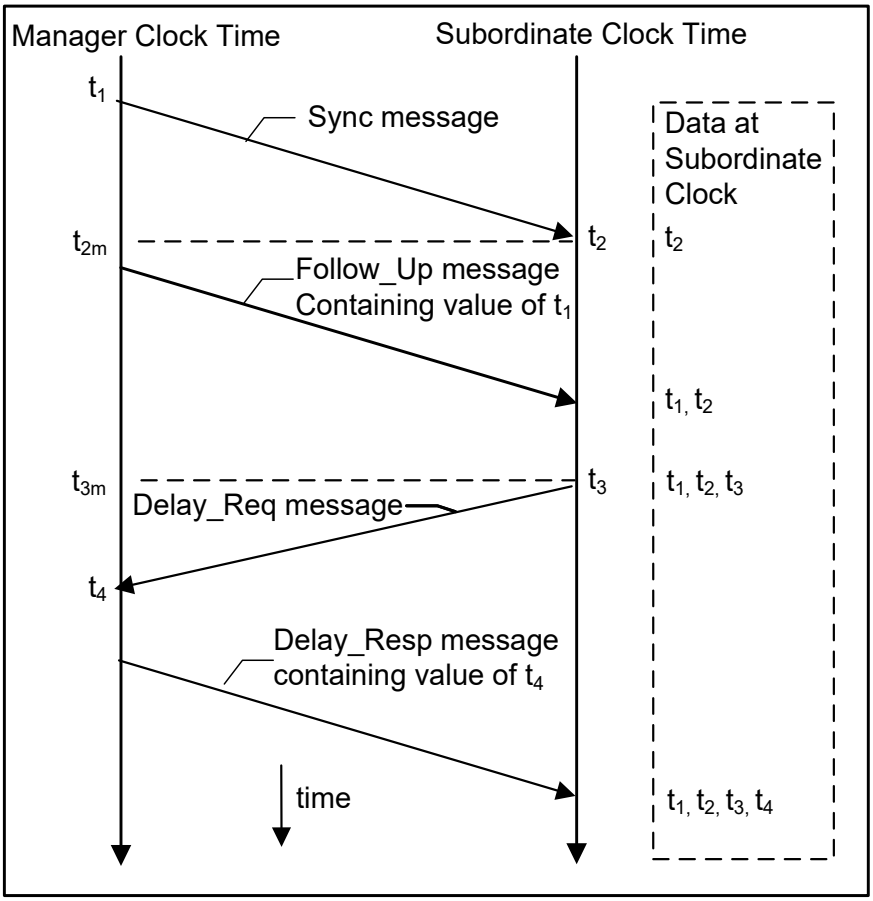
The 25GMAC supports the IEEE 1588-2002 (version 1) and IEEE 1588-2008 (version 2). The IEEE 1588-2002 supports PTP transported over UDP/IP. The IEEE 1588-2008 supports PTP transported over Ethernet. The 25GMAC provides programmable support for both standards. It supports the following features:

- Supports both timestamp formats
- Provides an option to take snapshot of all packets or only PTP type packets
- Provides an option to take snapshot of only event messages
- Provides an option to take the snapshot based on the clock type: ordinary, boundary, end-to-end transparent, and peer-to-peer transparent
- Provides an option to select the node to be a manager or subordinate for ordinary and boundary clock
- Identifies the PTP message type, version, and PTP payload in packets sent directly over Ethernet and sends the status
- Provides an option to measure sub-second time in digital or binary format

4.5.1 Delay Request-Response Mechanism

The system or network is classified into the manager and subordinate nodes for distributing the timing and clock information. Figure 4-4 shows the process that PTP uses for synchronizing a subordinate node to a manager node by exchanging PTP messages.

Figure 4-4 Networked Time Synchronization



As shown in Figure 4-4, the PTP uses the following process:

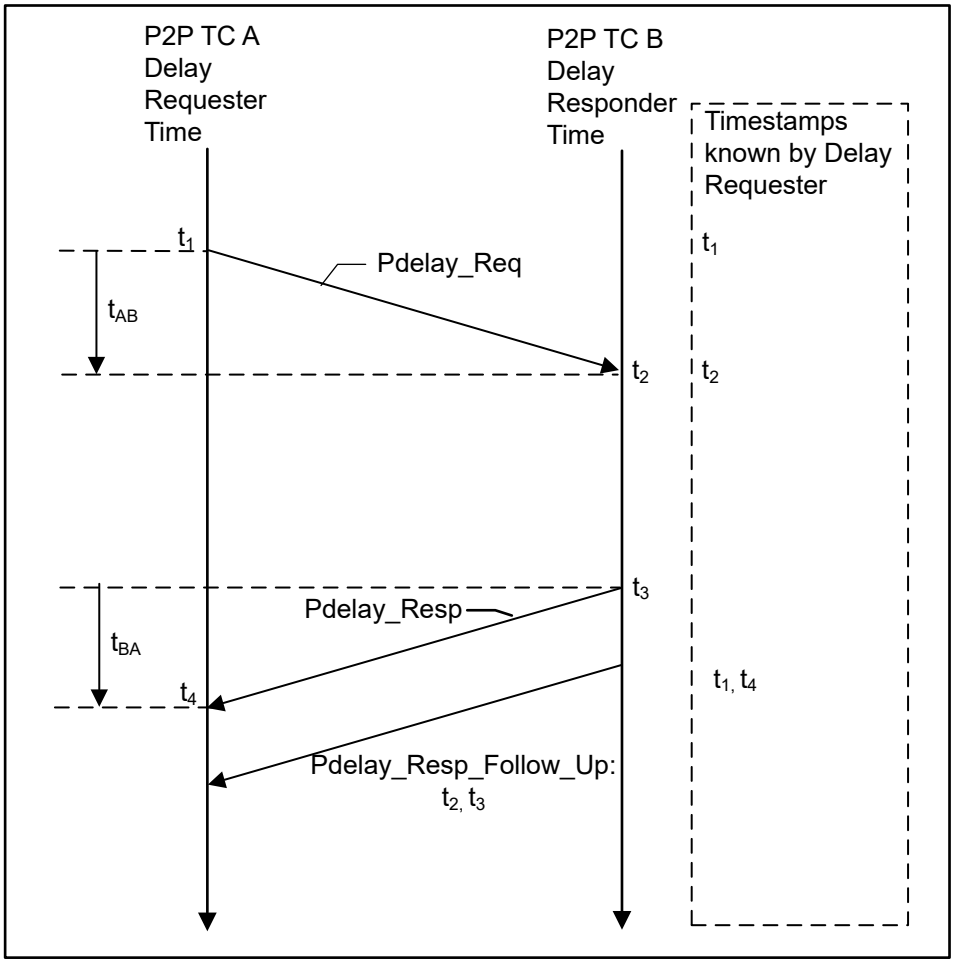
1. The manager broadcasts the PTP Sync messages to all its nodes. The Sync message contains the reference time information of the manager. This message leaves the system of the manager at t1. This time must be captured for Ethernet ports at GMII or XGMII.
2. The subordinate receives the Sync message and also captures the exact time, t2, using its timing reference.
3. The manager sends a Follow_up message to the subordinate, which contains t1 information for later use.
4. The subordinate sends a Delay_Req message to the manager and notes the exact time, t3, at which this packet leaves the GMII or XGMII interface.
5. The manager receives the message, capturing the exact time t4, at which the message enters its system.
6. The manager sends the t4 information to the subordinate in the Delay_Resp message.
7. The subordinate uses the four values of t1, t2, t3, and t4 to synchronize its local timing reference to the timing reference of the manager.

Most of the PTP implementation is done in the software that is above the UDP layer. However, the hardware support is required to capture the exact time when specific PTP packets enter or leave the Ethernet port at the GMII or XGMII interface. This timing information must be captured and returned to the software for proper implementation of PTP with high accuracy.

4.5.2 Peer-to-Peer PTP Transparent Clock (P2P TC) Message Support

The IEEE 1588-2008 supports peer-to-peer PTP (Pdelay) message in addition to the SYNC, Delay Request, Follow-up, and Delay Response messages. Figure 4-5 shows the method to calculate the propagation delay in clocks supporting peer-to-peer path correction.

Figure 4-5 Propagation Delay Calculation in Clocks Supporting Peer-to-Peer Path Correction



As shown in Figure 4-5, the propagation delay is calculated in the following way:

1. Port 1 issues a Pdelay_Req message and generates a timestamp (t1) for the Pdelay_Req message.
2. Port 2 receives the Pdelay_Req message and generates a timestamp (t2) for this message.
3. Port 2 returns a Pdelay_Resp message and generates a timestamp (t3) for this message.

To minimize errors because of any frequency offset between the two ports, Port 2 returns the Pdelay_Resp message as quickly as possible after the receipt of the Pdelay_Req message. Port 2 returns any one of the following:

- Difference between the timestamps t2 and t3 in the Pdelay_Resp message
- Difference between the timestamps t2 and t3 in the Pdelay_Resp_Follow_Up message
- Timestamps t2 and t3 in the Pdelay_Resp and Pdelay_Resp_Follow_Up messages, respectively

4. Port 1 generates a timestamp (t4) on receiving the Pdelay_Resp message.
5. Port 1 uses all four timestamps to compute the mean link delay.

4.5.3 Clock Types

The 25GMAC supports the following clock types defined in the IEEE 1588-2008:

- "Ordinary Clock" on page 207
- "Boundary Clock" on page 208
- "End-to-End Transparent Clock" on page 208
- "Peer-to-Peer Transparent Clock" on page 208

4.5.3.1 Ordinary Clock

The ordinary clock in a domain supports a single copy of the protocol. The ordinary clock has a single PTP state and a single physical port. In typical industrial automation applications, an ordinary clock is associated with an application device such as a sensor or an actuator. In telecom applications, the ordinary clock can be associated with a timing demarcation device.

The ordinary clock can be a principal manager or a subordinate clock. It supports the following features:

- Sends and receives PTP messages. The timestamp snapshot can be controlled as described in MAC_Timestamp_Control in the *Register Descriptions* chapter.
- Maintains the data sets such as timestamp values.

Table 4-18 shows the messages for which you can take the timestamp snapshot on the receive side for manager and subordinate nodes.

Table 4-18 Ordinary Clock: PTP Messages for Snapshot

Manager	Subordinate
Delay_Req	SYNC

For an ordinary clock, you can take the snapshot of either of the following PTP message types: version 1 or version 2. You cannot take the snapshots for both PTP message types. You can take the snapshot by setting the TSVER2ENA bit and selecting the snapshot mode in MAC_Timestamp_Control.

4.5.3.2 Boundary Clock

The boundary clock typically has several physical ports communicating with the network. The messages related to synchronization, manager-subordinate hierarchy, and signaling terminate in the protocol engine of the boundary clock and such messages are not forwarded. The PTP message type status given by the MAC helps you to identify the type of message and take appropriate action.

The boundary clock is similar to the ordinary clock except for the following features:

- The clock data sets are common to all ports of the boundary clock.
- The local clock is common to all ports of the boundary clock.

4.5.3.3 End-to-End Transparent Clock

The end-to-end transparent clock supports the end-to-end delay measurement mechanism between the subordinate clocks and the manager clock. The end-to-end transparent clock forwards all messages like normal bridge, router, or repeater. The residence time of a PTP packet is the time taken by the PTP packet from the Ingress port to the Egress port.

The residence time of a SYNC packet inside the end-to-end transparent clock is updated in the correction field of the associated Follow_Up PTP packet before it is transmitted. Similarly, the residence time of a Delay_Req packet, inside the end-to-end transparent clock, is updated in the correction field of the associated Delay_Resp PTP packet before it is transmitted. Therefore, the snapshot needs to be taken at both Ingress and Egress ports only for the messages mentioned in Table 4-19. You can take the snapshot by setting the SNAPTYPSEL bits to 10 in the MAC_Timestamp_Control register.

Table 4-19 End to End Transparent Clock: PTP Messages for Snapshot

PTP Messages
SYNC
Delay_Req

4.5.3.4 Peer-to-Peer Transparent Clock

The peer-to-peer transparent clock differs from the end-to-end transparent clock in the way it corrects and handles the PTP timing messages. In all other aspects, it is identical to the end-to-end transparent clock.

In the peer-to-peer transparent clock, the computation of the link delay is based on an exchange of Pdelay_Req, Pdelay_Resp, and Pdelay_Resp_Follow_Up messages with the link peer. The residence time of the Pdelay_Req and the associated Pdelay_Resp packets is added and inserted into the correction field of the associated Pdely_Resp_Followup packet. Therefore, support for taking snapshot for the event messages related to Pdelay is added as shown in Table 4-20.

Table 4-20 Peer-to-Peer Transparent Clock: PTP Messages for Snapshot

PTP Messages
SYNC
Pdelay_Req
Pdelay_Resp

You can take the snapshot by setting the SNAPTYPESEL bit to 11 in the MAC_Timestamp_Control register.

4.5.4 Reference Timing Source

To get a snapshot of the time, the MAC requires a reference time in 64-bit format as defined in the IEEE 1588-2002 (80-bit format as defined in the IEEE 1588-2008). The 25GMAC provides the following two options for using the reference timing source in a node:

- External Timestamp Input

  This option takes an external 64-bit timing reference and its clock as input. The clock input is used to synchronize the timing reference to the MAC clock domain. The 64-bit timing reference is split into the following two 32-bit signals:

  - Upper 32-bits providing the time in seconds
  - Lower 32-bits providing the time in nanoseconds

  This timing reference is used to timestamp the packets.

- Internal Reference Time (80-bit)

  This option takes only the reference clock input and uses it to internally generate the Reference time (also called the system time) and capture timestamps. The timestamp has the following fields:

  - UInteger48 secondsField

    The seconds field is the integer portion of the timestamp in units of seconds. It is 48-bits wide. For example, 2.000000001 seconds are represented as secondsField = 0x0000_0000_0002.

  - UInteger32 nanosecondsField

    The nanoseconds field is the fractional portion of the timestamp in units of nanoseconds. For example, 2.000000001 seconds are represented as nanoseconds = 0x0000_0001.

    The nanoseconds field supports the following two modes:

    - Digital rollover mode: In this mode, the maximum value in the nanoseconds field is 0x3B9A_C9FF, that is, (10e9-1) nanoseconds.
    - Binary rollover mode: In this mode, the nanoseconds field rolls over and increments the seconds field after value 0x7FFF_FFFF. Accuracy is ~0.466 ns per bit.

      You can set these modes through TSCTRLSSR bit in the MAC_Timestamp_Control register.

When you select the timestamp feature, the timestamp maintained in the MAC is 64-bit wide. The overflow to upper 16-bits of the seconds register happens once in 130 years. You can read the values of the upper 16-bits of the seconds field from the CSR register. This feature is optional. You can use this feature by selecting the Add IEEE 1588 Higher Word Register option while configuring the controller.

4.5.5 System Time Register Module

The system time generator module is an optional module. It is not available if external time updating is enabled. The 64-bit time is maintained in this module and updated using the input reference clock (clk_ptp_ref_i). This time is the source for taking snapshots (timestamps) of Ethernet packets being transmitted or received at the GMII interface.

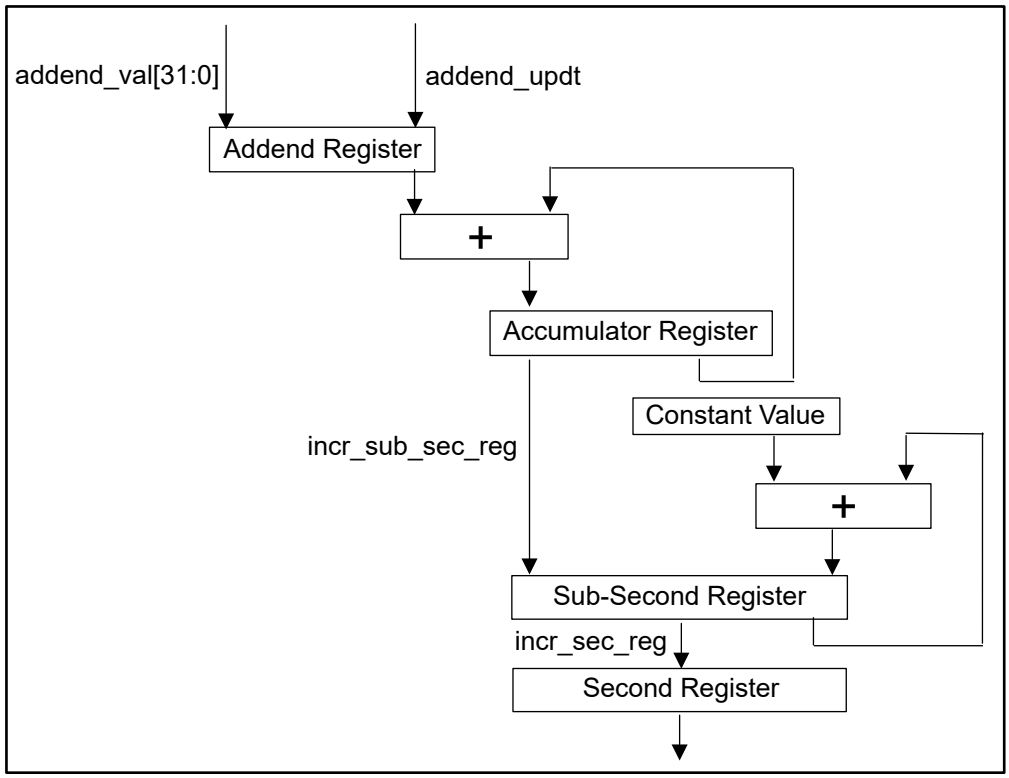
The system time counter can be initialized or corrected using the coarse correction method. In this method, the initial value or the offset value is written to the Timestamp Update register. For initialization, the system time counter is written with the value in the Timestamp Update register. For system time correction, the offset value is added to or subtracted from the system time.

In the fine correction method, the frequency drift of a Register Bus Interface clock (clk_ptp_ref_i) with respect to the manager clock (as defined in IEEE 1588) is corrected over a period of time instead of in one clock, as in coarse correction. This helps maintain linear time and does not introduce drastic changes (or a large jitter) in the reference time between PTP Sync message intervals. In this method, an accumulator sums up the contents of the Addend register, as shown in Figure 4-6. The arithmetic carry that the accumulator generates is used as a pulse to increment the system time counter. The accumulator and the addend are 32-bit registers. The accumulator acts as a high-precision frequency multiplier or divider.

Note	You must connect a PTP clock with a frequency higher than the frequency required for the specified accuracy.

This algorithm is shown in Figure 4-6.

Figure 4-6 System Time Update Using Fine Method



The System Time Update logic requires a 50-MHz clock frequency to achieve 20-ns accuracy. The frequency division is the ratio of the reference clock frequency to the required clock frequency. For example, if the reference clock (clk_ptp_ref_i) is 66 MHz, this ratio is calculated as 66 MHz / 50 MHz = 1.32. Therefore, the default addend value to be set in the register is 2^{32} / 1.32, 0xC1F07C1F.

If the reference clock drifts lower, for example, to 65 MHz, the ratio is 65 / 50, or 1.3 and the value to set in the addend register is 2^{32} / 1.30, or 0xC4EC4EC4. If the clock drifts higher, for example, to 67 MHz, the addend register must be set to 0xBF0B7672. When the clock drift is nil, the default addend value of 0xC1F07C1F (2^{32} / 1.32) must be programmed.

In Figure 4-6, the constant value used to accumulate the sub-second register is decimal 43, which achieves an accuracy of 20 ns in the system time (in other words, it is incremented in 20 ns steps). When External Time Update is enabled, the optional System Time module is not available. Two different methods are used to update the System Time register depending on the configuration.

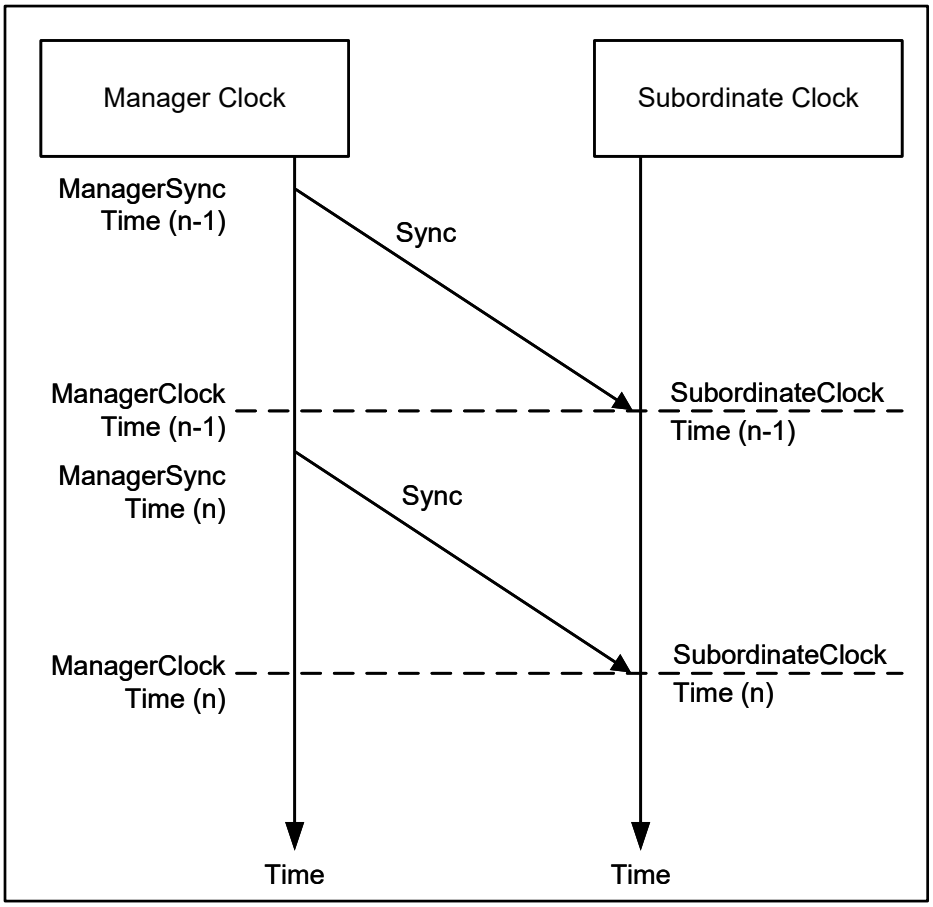
The software must calculate the drift in frequency based on the Sync messages and accordingly update the Addend register.

Initially, the Register Bus Interface clock is set with FreqCompensationValue0 in the Addend register. This value is as follows:

$FreqCompensationValue_0 = 2^{32}$ / FreqDivisionRatio

If MasterToSlaveDelay is initially assumed to be the same for consecutive Sync messages, the algorithm given in this section must be applied. After a few Sync cycles, frequency lock occurs. The subordinate clock can then determine a precise MasterToSlaveDelay value and synchronize again with the manager, using the new value.

Figure 4-7 Sync Message Timing



The algorithm is as follows:

- At time $MasterSyncTime_n$ the manager sends the subordinate clock a Sync message. The subordinate receives this message when its local clock is $SlaveClockTime_n$ and computes$MasterClockTime_n$ as

  $MasterClockTime_n = MasterSyncTime_n + MasterToSlaveDelay_n$

- The manager clock count for current Sync cycle, $MasterClockCount_n$ is

  $MasterClockCount_n = MasterClockTime_n – MasterClockTime_{n-1}$ (assuming that MasterToSlaveDelay is the same for Sync cycles n and n – 1)

- The subordinate clock count for current Sync cycle, $SlaveClockCount_n$ is

  $SlaveClockCount_n = SlaveClockTime_n – SlaveClockTime_{n-1}$

- The difference between manager and subordinate clock counts for current Sync cycle, $ClockDiffCount_n$ is

  $ClockDiffCount_n = MasterClockTime_n – SlaveClockTime_n$

- The frequency-scaling factor for subordinate clock, $FreqScaleFactor_n$ is

  $FreqScaleFactor_n = (MasterClockCount_n + ClockDiffCount_n)/ SlaveClockCount_n$

- The frequency compensation value for Addend register, $FreqCompensationValue_n$ is

  $FreqCompensationValue_n = FreqScaleFactor_n * FreqCompensationValue_{n-1}$

In theory, this algorithm achieves lock in one Sync cycle. However, it might take several cycles, because of changing network propagation delays and operating conditions. This algorithm is self-correcting. If the Register Bus Interface clock is initially set to an incorrect value from the manager, the algorithm corrects it at the cost of more Sync cycles.

4.5.6 Transmit Path Functions

The MAC captures a timestamp when the Start Packet Delimiter (SFD) of a packet is sent on the GMII or XGMII interface. The packets, for which you want to capture timestamps, can be controlled on per-packet basis. Each Transmit packet can be marked to indicate whether a timestamp must be captured for it.

The MAC does not process the transmitted packets to identify the PTP packets. You must specify the packets for which you want to capture timestamps. You can do this as follows:

- XGMAC-CORE Configurations

  You can specify the packets through input signals. To return the captured timestamps to the application, the MAC uses a separate 64-bit bus. The MAC gives the timestamp on this 64-bit bus along with Tx status on separate bus, validated by a common signal.

- XGMAC-MTL Configurations

  You can specify the packets through input signals. The 64-bit timestamp information is written to the MAC_TxTimestamp_Status_Seconds and MAC_TxTimestamp_Status_Nanoseconds register. The MAC generates "Tx Timestamp Captured" interrupt when 64-bit timestamp information is written to these registers.

  When Tx status forwarding to application is enabled, the captured timestamps is provided to the application on ati_txstatus_o[81:18], validated by ati_txstatus_val_o signal.

- XGMAC-AXI Configurations

  You can specify the packets by using the control bits in the Transmit descriptor. The 64-bit timestamp information is written to the MAC_TxTimestamp_Status_Seconds and MAC_TxTimestamp_Status_Nanoseconds register. The MAC generates "Tx Timestamp Captured" interrupt when 64-bit timestamp information is written to these registers.

When a timestamp is taken for a Transmit packet, it is given back to the host through either of the following schemes:

1. Tx Timestamp Status FIFO Streaming Interface, (see Figure 4-8 on page 214)
2. MAC_Tx_Timestamp_Status_* registers

In both preceding schemes, the PacketID field is extended to additionally provide the ID or information of the TX VDMA which transferred the packet from the host memory to the MAC for which this timestamp is associated with.

The application should provide the VDMA ID in the Transmit Context Descriptor (bits[15:10] of CTXT.TDES0). This field is validated by PIDV bit (bit[25] of CTXT.TDES3), the same bit used for validating Packet ID field (bits[9:0] of CTXT.TDES0).

Note The width of `tx_ts_data_o` signal is 74+LOG2(DWCXG_NUM_DMA_TX_CH), that is, bits[63:0] is 64-bit Timestamp, bits[73:64] is 10-bit Packet ID and bits[74+LOG2(DWCXG_NUM_DMA_TX_CH)-1:74] is VDMA ID.

4.5.6.1 Tx Timestamp Status FIFO with Packet Identifier

The MAC provides an option to store more than one (2, 4, 8, or 16) captured timestamps with the associated packet identifier and VDMA identifier provided by the application. These can be accessed through MAC_TxTimestamp_Status_PktID, MAC_TxTimestamp_Status_Seconds and MAC_TxTimestamp_Status_Nanoseconds registers. The packet identifier helps the application to correlate the timestamp with the packet.

Table 4-21 Details of the Operating Modes

TTSE	OSTC	OSTPA	ATI Control Word 1 (TTSL & TTSH)	Timestamp Captured & Given in TS StatusFIFO	Comments
0	0	X	Not Valid	No	Default non-PTP packet
0	1	0	Not Valid	No	1-step PTP packet to insert Origin TS only (for example, SYNC, DelayResp. PdelayResp)
0	1	1	Ingress TS	No	1-step PTP packet to insert Origin TS and/or update Correction
1	0	0	Not Valid	Yes (PID = 0, VDMAID=0)	PTP 2-step without PID (for example, any PTP Event Message)

Table 4-21 Details of the Operating Modes (continued)

TTSE	OSTC	OSTPA	ATI Control Word 1 (TTSL & TTSH)	Timestamp Captured & Given in TS StatusFIFO	Comments
1	1	0	Not Valid	Yes (PID = 0, VDMAID=0)	1-step PTP packet to insert Origin TS (Example: SYNC, DelayResp. PdelayResp)
1	0	1	VDMAID, Packet ID	Yes (valid PID and VDMAID)	PTP 2-step with PID (Example: Any PTP Event Message)
1	1	1	Ingress TS	Yes (PID = 0, VDMAID=0)	1-step PTP packet to insert Origin TS, update Correction & also give Origin TS it to software (DelayReq, PDelayReq)

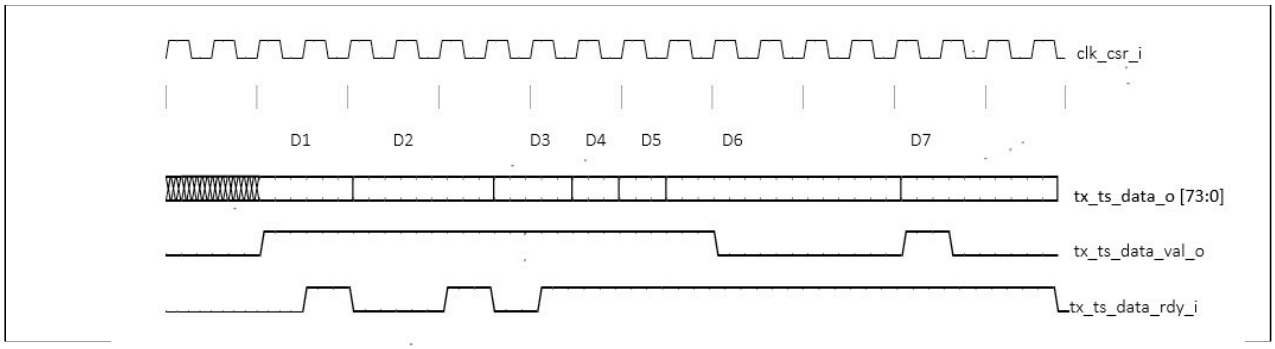
4.5.6.2 Transmit Timestamp Status FIFO Streaming Interface

When bit[29] of the `MAC_Timestamp_Control` register is

- 1: The output of the Tx timestamp FIFO is available at the I/O ports.
- 0: The output of the Tx timestamp FIFO can be read through status registers.

The packet ID, timestamp seconds and sub second values can be read through the `tx_ts_data_o` signal. The valid timestamp value in the tx_ts_data_o signal is sampled when the `tx_ts_data_val_o` and the `tx_ts_data_rdy_i` are high.

Figure 4-8 Transmit Timestamp FIFO Streaming Interface Timing Diagram



The application can assert the `tx_ts_data_rdy_i` signal in n clock cycles. Data is sampled in the n+1th clock cycle if the `tx_ts_data_val_o` is high.

4.5.7 Receive Path Functions

The MAC can be programmed to capture the timestamp of all packets received on the GMII or XGMII interface or to process packets to identify the valid PTP messages. You can control the snapshot of the time to be sent to the application by using the following options of the `MAC_Timestamp_Control` register.

- Enable snapshot for all packets
- Enable snapshot for IEEE 1588 version 1 or version 2 timestamp
- Enable snapshot for PTP packets transmitted directly over Ethernet or UDP-IP-Ethernet
- Enable timestamp snapshot for the received packet for IPv4 or IPv6
- Enable timestamp snapshot only for EVENT messages (SYNC, DELAY_REQ, PDELAY_REQ, or PDELAY_RESP)
- Enable the node to be a manager or subordinate and select the snapshot type

  This feature controls the type of messages for which snapshots are taken.

Note

- 25GMAC also supports the PTP messages over VLAN packets.
- When the packet size is less than 64 Bytes on the Rx side, 25GMAC does not generate timestamp for such packets, and passes it to the application.

Table 4-22 Timestamp Snapshot Dependency on Register Bits

SNAPTYPSEL	TSMSTRENA	TSEVNTENA	PTP Messages
00	x	0	SYNC, Follow_Up, Delay_Req, Delay_Resp
00	0	1	SYNC
00	1	1	Delay_Req
01	x	0	SYNC, Follow_Up, Delay_Req, Delay_Resp, Pdelay_Req,Pdelay_Resp, Pdelay_Resp_Follow_Up
01	0	1	SYNC, Pdelay_Req, Pdelay_Resp
01	1	1	Delay_Req, Pdelay_Req, Pdelay_Resp
10	x	x	SYNC, Delay_Req
11	x	x	Pdelay_Req, Pdelay_Resp

The timestamp is provided as follows:

- XGMAC-CORE Configurations

  The MAC provides the timestamp and the corresponding status, along with EOP, on a 64-bit bus (mri_timestamp_o) on MAC Receive Interface (MRI). An additional signal (mri_timestamp_val_o) validates the presence of timestamp for the receive packet.

- XGMAC-MTL Configurations

The MTL provides the timestamp on the data bus (ari_data_o[N - 1:0]) after the EOP data and receive status has been transferred. Bit 25 in the first receive status word indicate the availability of timestamp. The MTL provides the additional status related to the PTP packet in bits [29:26] of Receive Status Word.

- XGMAC-AXI Configurations

  The DMA returns the timestamp to the software inside the context Receive Descriptor. The timestamp message status is written in RDES3 and the snapshot of the timestamp is written in RDES0 and RDES1 fields of context descriptor. The RDES0 field holds the 32 least significant bits of the timestamp.

Note

The timestamp on RX packet is captured and provided in Rx Status only for valid Ethernet packets without any of the following errors:

- Giant packet error
- Length error
- CRC error
- Invalid code error
- Watchdog timeout error

4.5.8 Timestamp Correction

According to the IEEE 1588 standard, a timestamp must be captured when the message timestamp point (leading edge of the first bit of the octet immediately following the Start Frame Delimiter octet) crosses the boundary between the node and the network. Because the reference timing source (the PTP clock clk_ptp_ref_i) is different from the MAC Tx/Rx clock, correction needs to be performed to the captured timestamp for latency due to synchronization. Also, correction due to the latency between the internal snapshot point (XGMII/GMII) and the recommended capture point (the boundary between the node and the network) needs to be performed.

4.5.8.1 Ingress Correction

In the receive side, the timestamp captured at the internal snapshot point is corrected for latency and synchronization by adding the correction value (Ingress Correction Value) programmed in the MAC_Timestamp_Ingress_Corr_Nanosecond. The value that needs to be programmed in the ingress correction register is calculated as described as follows.

The timestamp correction due to synchronization is compensated by adding INGRESS_SYNC_CORR to the synchronized timestamp value, which is as follows:

```
INGRESS_SYNC_CORR = (1*PTP_CLK_PER + 4*RX_CLK_PER)
```

The latency correction between the message timestamp point and the XGMII receive interface is done by subtracting the latency value (INGRESS_LATENCY) with the captured timestamp.

Thus Ingress correction is as follows:

```
Ingress Correction = INGRESS_SYNC_CORR - INGRESS_LATENCY - (3*RX_CLK_PER)
```

(3*RC_CLK_PER) is due to the internal latency between the XGMII receive interface and the internal timestamp snapshot point. Ingress correction is performed by programming the TSIC field in the MAC_Timestamp_Ingress_Corr_Nanosecond register. The ingress correction can be positive/negative and has an unit of nanoseconds. A negative value is represented in complement form as described as follows.

When the TSCTRLSSR =1 in the MAC_Timestamp_Control register, this has an accuracy of 1 ns.

If the correction is positive, it is represented by setting bit 31 to 0 and bits 30:0 containing <ingress_correction_value> represented in binary. The value must not exceed 0x 3B9A_C9FF (1 second or 10^9 nanoseconds). If the correction is negative, it is represented by setting bit 31 to 1 and bits 30:0 containing 10^9 – <ingress_correction_value> represented in binary.

For example, if the required correction value is -5 ns, the programmed value must be 0xBB9A_C9FB.

When the TSCTRLSSR = 0 in the MAC_Timestamp_Control register, this has an accuracy of ~0.466 ns. If the correction is positive, it is represented by setting bit 31 to 0 and bits 30:0 containing <ingress_correction_value> represented in binary. The maximum value is 0x7FFF_FFFF. If the correction is negative, it is represented by setting bit 31 to 1 and bits 30:0 containing 2^{31} – <ingress_correction_value> represented in binary.

4.5.8.2 Egress Correction

In the transmit side, the timestamp captured at the internal snapshot point is corrected for latency and synchronization by adding the correction value (Egress Correction Value) programmed in the MAC_Timestamp_Egress_Asym_Corr register. The value that needs to be programmed in the egress correction register is calculated as described as follows.

The timestamp correction due to synchronization is compensated by adding EGRESS_SYNC_CORR to the synchronized timestamp value, which is as follows:

```
EGRESS_SYNC_CORR = (1*PTP_CLK_PER + 4*TX_CLK_PER)
```

The latency correction between the recommended capture point and the internal timestamp snapshot point is done by adding the latency value (EGRESS_LATENCY) with the captured timestamp.

Thus, Egress correction is as follows:

```
Egress Correction = EGRESS_SYNC_CORR + EGRESS_LATENCY
```

Egress correction is performed by programming the TSEC field in the MAC_Timestamp_Egress_Corr_Nanosecond register. The egress correction is always positive and has an unit of nanoseconds.

When TSCTRLSSR bit in MAC_Timestamp_Control register is set, this has an accuracy of 1 ns.

The value must not exceed 0x 3B9A_C9FF. When TSCTRLSSR bit in MAC_Timestamp_Control register is reset, this has an accuracy of ~0.466 ns. The maximum value is 0x7FFF_FFFF.

4.5.8.3 Sub-Nanosecond Accuracy

When Enable IEEE 1588 Sub-Nanoseconds Timestamp Support is selected, then ingress and egress correction can have sub-nanoseconds accuracy. The sub-nanosecond part of the ingress and egress correction is programmed in the TSICSNS and TSECSNS field of the MAC_Timestamp_Ingress_Corr_Subnanosecond and MAC_Timestamp_Egress_Corr_Subnanosecond registers, respectively.

In this case, the correction has a unit of nanosecond, multiplied by 2^8. The least significant 8 bits of the value must be programmed in the sub-nanoseconds register.

For example, if the required egress correction is 3.6 ns and the TSCTRLSSR = 1, the TSEC must be 0x3 and the TSECSNS must be 0x99 (3.6 * 256). Similarly, if the required ingress correction is -3.6 ns and the TSCTRLSSR =1, the TSIC must be 0xBB9A_C9FC and the TSICSNS must be 0x66 ((10^9 – 3.6) * 256).

4.5.8.4 Maximum PTP Clock Frequency

The maximum PTP clock frequency is limited by the maximum resolution of the reference time (1 ns resulting in 1 GHz) and the timing constraints achievable for logic operating on the PTP clock. In addition, the resolution or granularity of the reference time source determines the accuracy of the synchronization. Therefore, a higher PTP clock frequency gives better system performance.

4.5.8.5 Minimum PTP Clock Frequency

The minimum PTP clock frequency depends on the time required between two consecutive SFD bytes and the time taken for synchronizing the time to the GMII or MII clock domain. This relationship is given in the following equation:

```
3 * PTP clock period + 4 * GMII/MII clock period <= Minimum gap between two
 SFDs
```

The GMII or MII clock frequency is fixed by IEEE specification. Therefore, the minimum PTP clock frequency required for proper operation depends on the operating mode and operating speed of the MAC as shown in Table 4-23.

- The CSR clock frequency must not be less than the following speed modes of the MAC transmitter clock frequency.
  - 1/5th for XGMII
  - 1/20th for GMII
  - 1/40th for MII

  For example, if Tx Timestamp is captured for back-to-back 64 bytes size packets, which is transferred to CSR clock domain for providing in status registers. In XGMII speed mode, two consecutive timestamps is captured within 19 Tx clock cycles. This can be successfully transferred to CSR clock domain only if transfer occurs within 19 Tx clock cycles. This requires CSR clock frequency to be not less than 1/6th of Tx clock frequency (1/5th to accommodate any phase differences or additional random delay in CDC path).
- Use a clock that has constant frequency, preferably a divided application clock.
- When IEEE 1588 timestamp feature is enabled with internal timestamp, use a PTP clock frequency which is greater than 5 MHz. This is because the 8-bit MAC_Sub_Second_Increment register limits the minimum PTP frequency that can be used to ~4 MHz.

Table 4-23 Minimum PTP Clock Frequency Example

Mode	Minimum Gap Between Two SFDs	Minimum PTP Frequency with External Timestamp Input	Minimum PTP Frequency with Internal Timestamp
10 Mbps full duplex	168 MII clocks (128 clocks for a 64-byte packet + 24 clocks of min IFG + 16 clocks of preamble)	~45 KHz	5 MHz

Table 4-23 Minimum PTP Clock Frequency Example (continued)

Mode	Minimum Gap Between Two SFDs	Minimum PTP Frequency with External Timestamp Input	Minimum PTP Frequency with Internal Timestamp
100 Mbps full duplex	168 MII clocks (128 clocks for a 64-byte packet + 24 clocks of min IFG + 16 clocks of preamble)	~0.5 MHz	5 MHz
1G Mbps full duplex	84 GMII clocks (64 clocks for a 64-byte packet + 12 clocks of min IFG + 8 clocks of preamble)	~4.7 MHz	5 MHz
2.5G Full duplex (GMII)	84 GMII clocks (64 clocks for a 64-byte packet + 12 clocks of min IFG + 8 clocks of preamble)	12 MHz	12 MHz
2.5G Full duplex (XGMII)	21 XGMII clocks (64 clocks for a 64-byte packet + 12 clocks of min IFG + 8 clocks of preamble)	14 MHz	14 MHz
5G full duplex	21 XGMII clocks (16 clocks for a 64-byte packet + 3 clocks of min IFG + 2 clocks of preamble)	28 MHz	28 MHz
10G full duplex	21 XGMII clocks (16 clocks for a 64-byte packet + 3 clocks of min IFG + 2 clocks of preamble)	56 MHz	56 MHz

4.5.9 PTP Processing and Control

Table 4-24 shows the common message header for the PTP messages. This format is taken from the IEEE 1588-2008.

Table 4-24 Message Format Defined in IEEE 1588-2008

Bits 7	6	5	4	3	2	1	0	Octets	Offset
transportSpecific				messageType				1	0
Reserved				versionPTP				1	1
messageLength								2	2
domainNumber								1	4
Reserved								1	5
flagField								2	6
correctionField								8	8
Reserved								4	16
sourcePortIdentity								10	20
sequenceId								2	30
controlField (*)								1	32
logMessageInterval								1	33

(*) – controlField is used in version 1. In version 2, messageType field is used for detecting different message types.

There are some fields in the Ethernet payload that you can use to detect the PTP packet type and control the snapshot to be taken. These fields are different for the following PTP packets:

- "PTP Packets Over IPv4" on page 220
- "PTP Frames Over IPv6" on page 222
- "PTP Packets Over Ethernet" on page 223

4.5.9.1 PTP Packets Over IPv4

Table 4-25 shows the fields that are matched to control snapshot for the PTP packets sent over UDP over IPv4 for IEEE 1588 version 1 and 2. The octet positions for the tagged packets are offset by 4. This is based on the IEEE 1588-2008, Annex D and the message format defined in Table 4-25.

Table 4-25 IPv4-UDP PTP Packet Fields Required for Control and Status

Field Matched	Octet Position	Matched Value	Description
MAC Packet Type	12, 13	0x0800	IPv4 datagram

Table 4-25 IPv4-UDP PTP Packet Fields Required for Control and Status (continued)

Field Matched	Octet Position	Matched Value	Description
IP version and Header Length	14	0x45	IP version is IPv4
Layer 4 Protocol	23	0x11	UDP
IP Multicast Address (IEEE 1588 version 1)	30, 31, 32, 33	0xE0, 0x00, 0x01, 0x81 (or 0x82 or 0x83 or 0x84)	Multicast IPv4 addresses allowed: ■ 224.0.1.129 ■ 224.0.1.130 ■ 224.0.1.131 ■ 224.0.1.132
IP Multicast Address (IEEE 1588 version 2)	30, 31, 32, 33	0xE0, 0x00, 0x01, 0x81 (Hex) 0xE0, 0x00, 0x00, 0x6B (Hex)	■ PTP Primary multicast address: 224.0.1.129 ■ PTP Pdelay multicast address: 224.0.0.107
UDP Destination Port	36, 37	0x013F, 0x0140	■ 0x013F: PTP event message[a] ■ 0x0140: PTP general messages
PTP Control Field (IEEE 1588 version 1)	74	0x00, 0x01, 0x02, 0x03, 0x04	■ 0x00: SYNC ■ 0x01: Delay_Req ■ 0x02: Follow_Up ■ 0x03: Delay_Resp ■ 0x04: Management
PTP Message Type Field (IEEE 1588 version 2)	42 (nibble)	0x0, 0x1, 0x2, 0x3, 0x8, 0x9, 0xB, 0xC, 0xD	■ 0x0: SYNC ■ 0x1: Delay_Req ■ 0x2: Pdelay_Req ■ 0x3: Pdelay_Resp ■ 0x8: Follow_Up ■ 0x9: Delay_Resp ■ 0xA: Pdelay_Resp_Follow_Up ■ 0xB: Announce ■ 0xC: Signaling ■ 0xD: Management

Table 4-25 IPv4-UDP PTP Packet Fields Required for Control and Status (continued)

Field Matched	Octet Position	Matched Value	Description
PTP Version	43 (nibble)	0x1 or 0x2	■ 0x1: Supports PTP version 1 ■ 0x2: Supports PTP version 2

a. PTP event messages are SYNC, Delay_Req (IEEE 1588 version 1 and 2) or Pdelay_Req, Pdelay_Resp (IEEE 1588 version 2 only)

4.5.9.2 PTP Frames Over IPv6

Table 4-26 provides information about the fields that are matched to control the snapshots for the PTP packets sent over UDP over IPv6 for IEEE 1588 version 1 and 2. The octet positions for the tagged packets are offset by 4. This is based on the IEEE 1588-2008, Annex D and the message format defined in Table 4-24.

Table 4-26 IPv6-UDP PTP Packet Fields Required for Control and Status

Field Matched	Octet Position	Matched Value	Description
MAC Packet Type	12, 13	0x86DD	IP datagram
IP Version	14 (Bits [7:4])	0x6	IP version is IPv6
Layer 4 Protocol	20[a] (*)	0x11	UDP
PTP Multicast Address	38 – 53	FF0x:0:0:0:0:0:0:181 (Hex) FF02:0:0:0:0:0:0:6B (Hex)	■ PTP Primary multicast address: FF0x:0:0:0:0:0:0:0:181 (Hex) ■ PTP Pdelay multicast address: FF02:0:0:0:0:0:0:0:6B (Hex)
UDP Destination Port	56, 57[a]	0x013F, 0x140	■ 0x013F: PTP event message ■ 0x0140: PTP general messages
PTP Control Field (IEEE 1588 version 1)	94[a]	0x00, 0x01, 0x02, 0x03, or 0x04	■ 0x00: SYNC ■ 0x01: Delay_Req ■ 0x02: Follow_Up ■ 0x03: Delay_Resp ■ 0x04: Management (version1)
PTP Message Type Field (IEEE 1588 version 2)	62[a] (nibble)	0x0, 0x1, 0x2, 0x3, 0x8, 0x9, 0xB, 0xC, or 0xD	■ 0x0: SYNC ■ 0x1: Delay_Req ■ 0x2: Pdelay_Req ■ 0x3: Pdelay_Resp

Table 4-26 IPv6-UDP PTP Packet Fields Required for Control and Status (continued)

Field Matched	Octet Position	Matched Value	Description
			■ 0x8: Follow_Up ■ 0x9: Delay_Resp ■ 0xA: Pdelay_Resp_Follow_Up ■ 0xB: Announce ■ 0xC: Signaling ■ 0xD: Management
PTP Version	63 (nibble)	0x1 or 0x2	■ 0x1: Supports PTP version 1 ■ 0x2: Supports PTP version 2

a. The Extension Header is not defined for PTP packets.

4.5.9.3 PTP Packets Over Ethernet

Table 4-27 provides information about the fields that are matched to control the snapshots for the PTP packets sent over Ethernet for IEEE 1588 version 1 and 2. The octet positions for the tagged packets are offset by 4. This is based on the IEEE 1588-2008, Annex D and the message format defined in Table 4-24.

Table 4-27 Ethernet PTP Packet Fields Required for Control And Status

Field Matched	Octet Position	Matched Value	Description
MAC Destination Multicast Address[a]	0–5	01-1B-19-00-00-00 01-80-C2-00-00-0E	All PTP messages can use any of the following multicast addresses[b]: ■ 01-1B-19-00-00-00 ■ 01-80-C2-00-00-0E[c]
MAC Packet Type	12, 13	0x88F7	PTP Ethernet packet
PTP Control Field (IEEE 1588 version 1)	46	0x00, 0x01, 0x02, 0x03, or 0x04	■ 0x00: SYNC ■ 0x01: Delay_Req ■ 0x02: Follow_Up ■ 0x03: Delay_Resp ■ 0x04: Management
PTP Message Type Field (IEEE 1588 version 2)	14 (nibble)	0x0, 0x1, 0x2, 0x3, 0x8, 0x9, 0xB, 0xC, or 0xD	■ 0x0: SYNC ■ 0x1: Delay_Req ■ 0x2: Pdelay_Req ■ 0x3: Pdelay_Resp

Table 4-27 Ethernet PTP Packet Fields Required for Control And Status (continued)

Field Matched	Octet Position	Matched Value	Description
			■ 0x8: Follow_Up ■ 0x9: Delay_Resp ■ 0xA: Pdelay_Resp_Follow_Up ■ 0xB: Announce ■ 0xC: Signaling ■ 0xD: Management
PTP Version	15 (nibble)	0x1 or 0x2	■ 0x1: Supports PTP version 1 ■ 0x2: Supports PTP version 2

a. The unicast address match of destination addresses (DA), programmed in MAC address 0 to 31, is used if the TSENMACADDR bit of the MAC_Timestamp_Control register is set.
b. IEEE 1588-2008, Annex F
c. The MAC does not consider the PTP version 1 messages with Peer delay multicast address (01-80-C2-00-00-0E) as valid PTP messages.

4.5.10 Improving the Timestamping Accuracy Using the Pulse and Latency Input from PCS

Ingress and Egress latencies from Generic Reconciliation Sublayer (gRS) are used for the adjustment of the captured timestamp only when the EPCSL = 1, in the `MAC_Timestamp_Control` register. The MAC adjusts the captured ingress and egress timestamp based on the per packet latencies provided by the gRS.

The MAC receives the Ingress and Egress latencies observed in the gRS sublayer in units of bits through the following IO ports as timestamp correction values. These latency values are converted to time, in nanoseconds, based on the operating speed.

- xpcs_rx_mtp_i
- xpcs_rx_num_unit_chg_i[15:0]
- xpcs_tx_mtp_i
- xpcs_tx_num_unit_chg_i[15:0]

The Ingress latency value is received on the `xpcs_rx_num_unit_chg_i` bus validated by the `xpcs_rx_mtp_i` signal. The Egress latency value is received on the `xpcs_tx_num_unit_chg_i` bus validated by the `xpcs_tx_mtp_i` signal.

4.5.10.1 Transmit Path

In the transmit path, when the `xpcs_tx_mtp_i` signal receives the pulse during a packet transmission, the Egress latency available on the `xpcs_tx_num_unit_chg_i` bus is added to the already captured timestamp value.

For 2-step timestamping, the maximum delay from the point of SFD/SMD column on XGMII interface to the reception of Egress latency pulse for a 64-byte packet is

- 16 clock cycles on the XGMII.
- 64 clock cycles on the GMII.
- 128 clock cycles on the MII.

The latency value is discarded if the pulse validating the `xpcs_tx_num_unit_chg_i` bus is received after the packet transmission in the MAC.

For 1-step timestamping, requirements on the maximum delay from the point of SFD/SMD column on XGMII interface to the reception of Egress latency pulse is stringent as shown in the table:

Table 4-28 Maximum Delay for One-Step Timestamping

Type of Packet	Maximum Delay		
	XGMII	GMII	MII
PTP over Ethernet	3 Tx clock cycles	12 Tx clock cycles	24 Tx clock cycles
PTP over UDP/IPv4	7 Tx clock cycles	28 Tx clock cycles	56 Tx clock cycles
PTP over UDP/IPv6	13 Tx clock cycles	52 Tx clock cycles	104 Tx clock cycles

4.5.10.2 Receive Path

In the receive path, when the `xpcs_rx_mtp_i` signal receives the pulse during a packet reception, the Ingress latency available on the `xpcs_rx_num_unit_chg_i` bus is subtracted from the already captured timestamp value. The MAC adjusts the captured Ingress timestamp with the received latency, only when the Ingress latency pulse is received 5 Rx clock cycles after the SFD/SMD column on XGMII interface for the previous packet.

Note

- When the EPCSL = 1 in the MAC_Timestamp_Control register, the values programmed in the `MAC_Timestamp_Ingress_Corr_Nanosecond`, `MAC_Timestamp_Ingress_Corr_Subnanosecond`, `MAC_Timestamp_Egress_Corr_Nanosecond`, and `MAC_Timestamp_Egress_Corr_Subnanosecond` registers must not include the fixed latency part, between the message timestamp point and the point at which the timestamp snapshot is taken.
- When latency pulse of a receiving packet is not asserted, IP considers the latency value of the previous packet.
- When you provide multiple latency values in a single packet, only the latest latency value is considered for calculation.
- Timestamp capturing and corrections apply to both preemption and express type packets.

4.5.11 Additional/Auxiliary Features

The 25GMAC supports the following auxiliary features:

- "Fixed Pulse-Per-Second Output" on page 226
- "Flexible Pulse-Per-Second Output" on page 226
- "Auxiliary Snapshots with External Events" on page 227

4.5.11.1 Fixed Pulse-Per-Second Output

The 25GMAC supports the pulse-per-second (PPS) output that is given to indicate 1 second interval (default). You can change the frequency of the PPS output by setting the PPSCTRL0 field in the MAC_PPS_Control register.

4.5.11.2 Flexible Pulse-Per-Second Output

The 25GMAC also provides the flexibility to program the start or stop time, width, and interval of the pulse generated on the ptp_pps_o output. You can enable this feature by selecting the Enable Flexible Pulse-Per-Second Output option in coreConsultant. You can configure up to four separate PPS outputs by using the Number of Pulse-Per-Second Outputs option in coreConsultant.

The 25GMAC provides the following features with the flexible PPS output option:

- Supports programming the start or stop time in terms of system time.
- Supports programming the start point of the single pulse and start and stop points of the pulse train in terms of 64-bit system time. The Target Time registers are used to program the start and stop time.
- Supports programming the stop time in advance, that is, you can program the stop time before the actual start time has elapsed.
- Supports programming the width between the rising edge and corresponding falling edge of PPS signal output in terms of number of units of sub-second increment value programmed in the MAC_Sub_Second_Increment register. You can program the width of pulse from 1 to 2^{32}-1 units of sub-second increment value.
- Supports programming the interval, between the rising edges of PPS signal, in terms of number of units of sub-second increment value. You can program the interval between pulses from 1 to 2^{32}-1 units of sub-second increment value.
- Provides the option to terminate the programmed PPS start or stop request.
- Indicates error if the start or stop time being programmed has already elapsed.

Note The PTP Reference clock mentioned in the following sections is the clock at which the system time gets updated. When the TSCFUPDT bit of the MAC_Timestamp_Control register is set to 0, this clock is similar to the clk_ptp_ref_i clock. In the Fine Correction mode, this is the clock tick at which the system time gets updated (using incr_sub_sec_reg shown in Figure 4-6 on page 210).

PPS Start or Stop Time

You can initially program the start time in the Target Time registers. If you have enabled additional flexible PPS outputs, you need to program the following registers corresponding to an enabled flexible PPS output:

- MAC_PPS1_Target_Time_Seconds and MAC_PPS1_Target_Time_Nanoseconds registers for second Flexible PPS output
- MAC_PPS2_Target_Time_Seconds and MAC_PPS2_Target_Time_Nanoseconds registers for third Flexible PPS output
- MAC_PPS3_Target_Time_Seconds and MAC_PPS3_Target_Time_Nanoseconds registers for fourth Flexible PPS output

If required, you can again program the start or stop time but you can do it only after the earlier programmed value is synchronized to the PTP clock domain. Bit 31 of MAC_PPS(#i)_Target_Time_Nanoseconds register indicates that the synchronization is complete. This enables you to program the start or stop time in advance even before the earlier stop or start time has elapsed.

To ensure proper PPS signal output, you must program advanced system time for the start or stop time. If the application programs a start or stop time that has already elapsed, the MAC sets an error status bit indicating the programming error. If enabled, the MAC also sets the Target Time Reached interrupt event. The application can terminate the start or stop request only if the corresponding start or stop time has not elapsed. If the time has elapsed, the terminate command has no effect.

PPS Width and Interval

The PPS width and interval are programmed in terms of granularity of system time, that is, number of the units of sub-second increment value. For example, to have a PPS pulse width of 40 ns and interval of 100 ns with PTP reference clock of 50 MHz, you must program the width and interval to values 2 and 5, respectively. You can achieve smaller granularity by using a faster PTP reference clock.

Before giving the command to trigger a pulse or pulse train on the PPS output, you must program or update the interval and width of the PPS signal output.

4.5.11.3 Auxiliary Snapshots with External Events

The auxiliary snapshot feature allows you to store a snapshot of the system time based on an external event. The event is considered to be the rising edge of the ptp_aux_ts_trig_i sideband signal. This feature is independent of whether the system time is generated internally or given as input (on ptp_timestamp_i[63:0] bus). You can use this feature by selecting the Add IEEE 1588 Auxiliary Snapshot option in coreConsultant.

You can configure up to four auxiliary snapshot inputs and also specify the depth (4, 8, or 16) of a single common auxiliary snapshot FIFO. The snapshots taken for any input are stored in a common FIFO. The application can read the MAC_Timestamp_Status register to know the timestamp of which input is available for reading at the top of this FIFO.

The MAC stores these snapshots in a FIFO. Only 64-bits of the timestamp are stored in the FIFO. You can read the upper 16-bits of seconds from the MAC_System_Time_Higher_Word_Seconds register when it is present. When a snapshot is stored, the MAC indicates this to the application with an interrupt. The value of the snapshot is read through a FIFO register access. If the FIFO becomes full and an external trigger to take the snapshot is asserted, a snapshot trigger-missed status (ATSSTM) is set in the

MAC_Timestamp_Status register. This indicates that the latest auxiliary snapshot of the timestamp is not stored in the FIFO. The latest snapshot is not written to the FIFO when it is full.

When a application reads the 64-bit timestamp from the FIFO, the space becomes available to store the next snapshot. You can clear a FIFO by setting the ATSFC bit in the MAC_TxTimestamp_Status_Nanoseconds register. When multiple snapshots are present in the FIFO, the count is indicated in Bits[29:25] of the MAC_Timestamp_Status register.

4.6 Media Clock Generation and Recovery

To reduce the cabling required for media applications in an AVB network, the media clock is sent over the same link that is used for data transfer. The media clock synchronization can be done using the IEEE 1588 PTP protocol and PTP manager clock is used as a reference clock source.

Media clock recovery is used to generate synchronous media clocks, for example audio, at all ends of the network. In the network, only one node can be a manager/generator of the media clock. The 25GMAC supports recovering of up to 4 media clocks based on PTP protocol clock and Presentation Time of the received audio packet.

4.6.1 Overview of Media Clock Generation and Recovery

25GMAC supports dedicated hardware for media clock recovery. This hardware generates a reference clock (for example, 6KHz or 8KHz for AVB class A stream), which is multiplied to get the required media clock (for example, audio MCLK of 12.88 MHz).

The media clock generation and recovery process is mainly hardware based and involves very little software.

Note	The PPS and MCGR modes are mutually exclusive, that is, all the instances selected in the configuration must operate either in PPS mode or in MCGR mode. Mixed operation mode, where a few instances operate in PPS mode and others in MCGR mode, is not supported. Therefore, program all the instances selected in configuration to the same mode, irrespective of whether they are used or not.

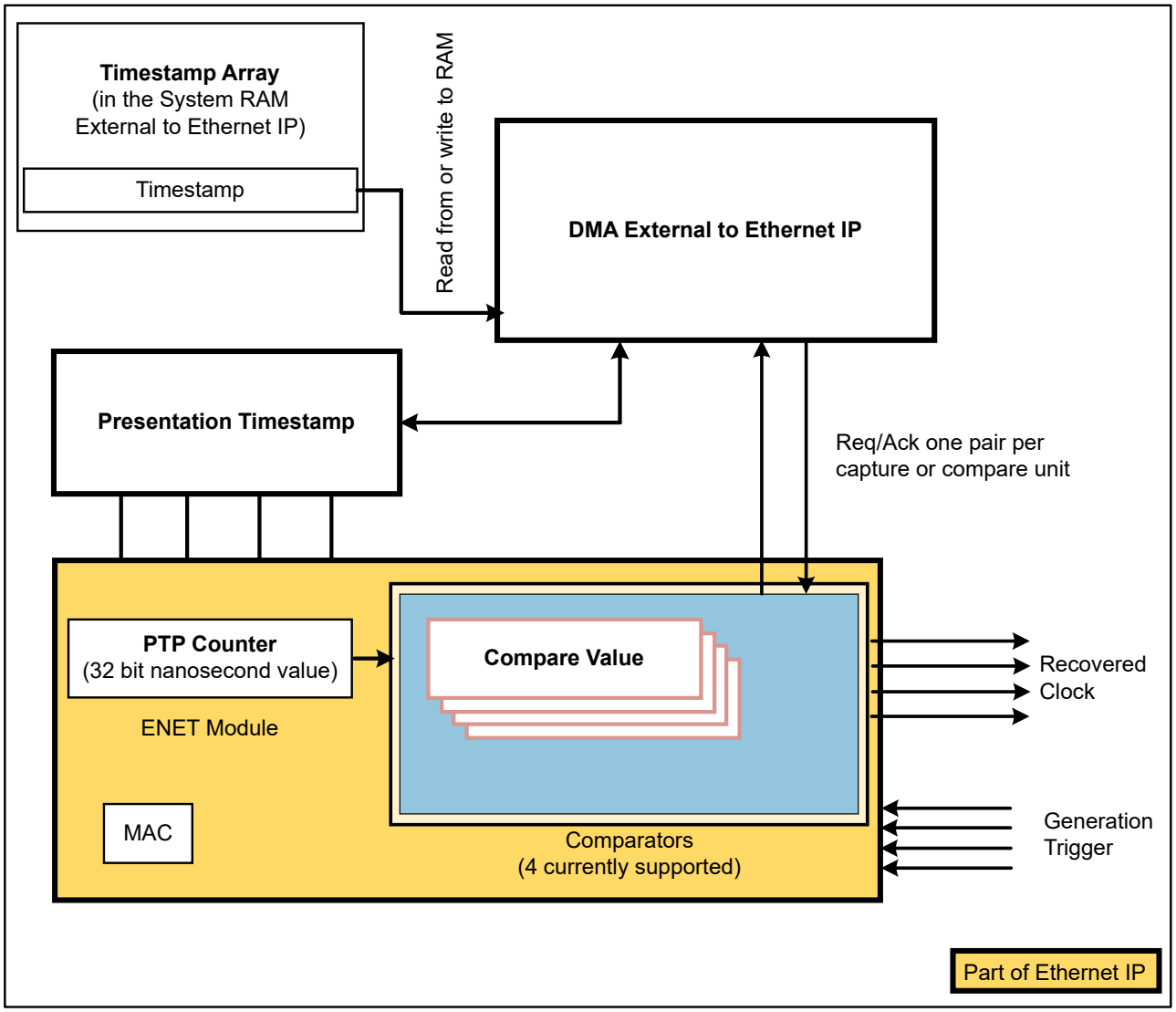
4.6.2 Description of Media Clock Generation and Recovery

The media clock generation and recovery support involves a dedicated external hardware and a thin layer of software.

The media clock recovery process involves host software extracting the presentation time from the 1722 frames and programming them into a CSR register, which is read by 25GMAC to generate reference clock. When the timer expires, the previous value is either toggled or a pulse is sent. The output at the recovered clock pin is generally a wave of 750Hz ranging up to 8KHz depending on the application that is synced with, at the other end of the network.

As shown in Figure 4-9, the timestamp array and the DMA module are external to 25GMAC.

Figure 4-9 Block Diagram of Media Clock Generation and Recovery



For the operation of the media clock generation and recovery, 25GMAC supports,

- A 32-bit Presentation Time nanosecond counter that wraps at the full 32-bit value (and not after one second) to match up with the 1722 presentation time format.
- A mechanism to handshake with external application (such as DMA) to program the presentation time into the CSR register. The hardware compares the programmed timestamp value with the Presentation time and generates a wave based on the programmable settings.

4.6.2.1 Media Clock Generation

The 25GMAC controller follows these steps to generate the media clock:

1. Samples the generated presentation time based on the trigger input.
2. Captures the sampled value into a CSR register.
3. Generates a request to the application to read the presentation time from the CSR register.

The application inserts the presentation time read from the CSR into the 1722 transmit packets, to recover the clock at the destination.

4.6.2.2 Media Clock Recovery

The 25GMAC supports a dedicated hardware for media clock recovery. This hardware generates a reference clock (e.g. 6 or 8 kHz for AVB class A stream) which is then multiplied to get the desired media clock (for example, audio MCLK of 12.288 MHz) by a hardware clock multiplier external to the 25GMAC. The media clock generation or recovery involves a very thin layer of software at the host that

1. Extracts the presentation time from the 1722 packets.
2. Programs the presentation time into a CSR register.
3. Toggles/pulses an external pin when the programmed time is reached.

The output at the recovered clock pin is typically a wave. The frequency of the wave depends on the application which is synced with, at the other end of Enabling Media Generation and Recovery the network.

4.6.3 Presentation Time Counter

The presentation time counter follows the 1722 presentation time format. In this format, time is represented as a 32bit counter in nanoseconds that does a binary rollover on reaching the maximum value of 32'hFFFF_FFFF. This counter is another view of the PTP System time (1588 timer).

In the PTP system time,

- PTP[63:32] represents time in seconds.
- PTP[31:0] represents time in nanoseconds.

For a specific PTP system time there exists a corresponding Presentation time, which is a 32bit value in nanoseconds.

The Presentation time computed is referred as Current Presentation Time (CPT). The CPT can be viewed as derived from the 64bit PTP System time. The CPT is computed as follows:

```
PTPNS[63:0] = (PTP[63:32] * 32'd1,000,000,000) + PTP[31:0]
```

where, PTPNS is the PTP system time converted into a 64bit nanosecond value.

Current Presentation Time[31:0] = PTPNS[31:0]

Media clock recovery is supported only when both of the following conditions are met:

- System time is internally generated in 25GMAC
- CPT is computed in the same way as the Internal PTP System Time generation.

Because both CPT and System time values are in nanoseconds, and must be synchronous, the increment cycle of both the timers are same. The time update occurs the same instance, but the updated value can be different. The software must compute a separate 32bit update-value in nanoseconds for the CPT update.

Note

- The value of CPT can be obtained by reading the `MAC_Presn_Time_ns` register.
- 25GMAC supports both fine-grain and course-grain correction modes.
- For the PTP time, use only the digital rollover mode. This is because, CPT uses the same increment value as PTP, and CPT requires the time to be in nanoseconds.

4.6.4 Comparators

The comparators handshake with the application to program the timestamps into the MAC_PPS(#i)_Target_Time_Seconds register when you program the following for recovery mode.

- The PTGE field of MAC_Timestamp_Control register
- Presentation time control bits of the particular instance

The timestamps received from the application are referred as Target Presentation Time (TPT).

As shown in the Figure 4-9, 25GMAC supports four comparators to simultaneously manage multiple media clocks that might be required. For both, generation and recovery, program the MCGREN(#i) field of MAC_PPS_Control register

The comparators handshake with the application to

- Program the timestamps into the `MAC_PPS(#i)_Target_Time_Seconds` register when PTGE field of `MAC_Timestamp_Control` register is set.
- Set the Presentation Time Control bits of the particular instance for recovery mode.

The timestamps received from the application are referred as Target Presentation Time (TPT).

The MCGR and PPS modes are mutually exclusive. When MCGREN#i field of `MAC_PPS_Control` register is set to 1, the corresponding PPS instance#i operates in MCGR mode. Otherwise, by default, the PPS instance#i operates in PPS mode.

The comparator module sends up to two requests to the application for TPT write, when PTGE field of `MAC_Timestamp_Control` register is set and the Presentation time control bits of the particular instance is set for MCGR mode. Subsequent requests are generated each time a Presentation Time match occurs. (CPT transitions from a value less than TPT, to a value greater than or equal to CPT). The first request is asserted when a specific comparator instance is set to MCGR mode with a non-zero Presentation Time Control and an additional request is made when the first data is received into the Comparator. This allows the application to write the next TPT value while 25GMAC processes the previous TPT value for a match.

The TPT read from the `MAC_PPS(#i)_Target_Time_Second` register is considered as a future time. A toggle/pulse is generated in the next cycle when a match is detected. When a match is detected, the mcgr_dma_req_o#i request for that comparator is asserted (to obtain next TPT) until the corresponding application acknowledgement is set.

The presentation control bits (PPSCMD#i field of `MAC_PPS_Control` register) determine the shape of the generated waveform. Also, these bits can be programmed to either toggle or generate a high/low pulse for one PTP clock cycle, upon match.

Clock Recovery: A match is when the free-running CPT value matches the received TPT value. A output signal ptp_pps_o is asserted (toggle, low pulse, or high pulse) based on the presentation control value programmed.

Clock Generation: Based on the presentation control value programmed in `MAC_PPS_Control` register, the particular comparator captures the presentation time and programs it into `MAC_PPS(#i)_Target_Time_Second` register. A request is raised to the application to read the captured timestamp. Until the acknowledgement is received from the application, acknowledging that the read operation is complete, no new timestamps are captured.

4.6.5 Media Clock Generation and Recovery Flow

Figure 4-10 shows the media clock generation and recovery flow.

For both media clock generation and recovery, set the PTGE field of `MAC_Timestamp_Control register` for CPT generation and set the MCGREN#i field of `MAC_PPS_Control` register for the corresponding instance to be enabled in the MCGR mode.

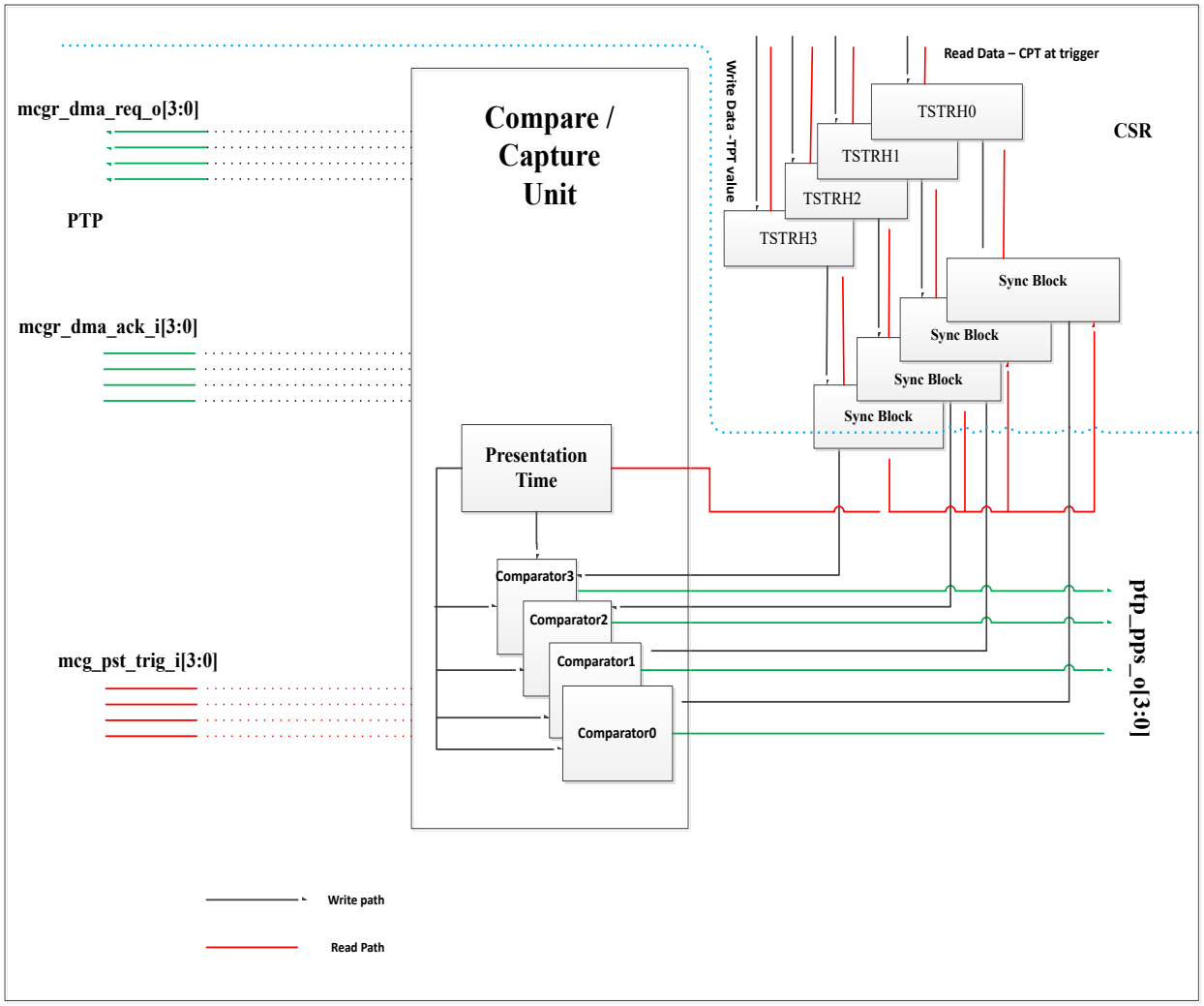
For Media Clock Generation, the generated presentation time is sampled at positive, negative or at both the edges of trigger inputs (mcg_pst_trig[i]) based on the mode programmed in `MAC_PPS_Control` register. The sampled value is captured in `MAC_PPS(#i)_Target_Time_Seconds` register. After the `MAC_PPS(#i)_Target_Time_Seconds` register is updated with the presentation time, 25GMAC asserts mcgr_dma_req_o#i to request the application to read the presentation time. Application acknowledges that it has read the captured timestamp by asserting the corresponding mcgr_dma_ack_i[#i].

For Media Clock Recovery, PPS instance asserts the mcgr_dma_req_o#i register to indicate the application to program the `MAC_PPS(#i)_Target _Time_Seconds` with TPT value and acknowledge the corresponding request using `mcgr_dma_ack_i`. When the application programs the `MAC_PPS(#i)_Target_Time_Seconds` with the TPT value, the particular comparator compares the programmed TPT value with the CPT and generates the waveform on `ptp_pps_o[3:0]` based on the presentation time control value programmed in `MAC_PPS_Control` register.

Note

- The consecutive triggers to sample the presentation time should always be asserted after a few PTP clock cycles to allow for synchronization delays. (This is not an issue when the input trigger maximum frequency is 8KHz and the PTP clock is running at least at 1MHz)
- DMA acknowledgement can be both posted as well as non-posted, in the Media Clock Recovery Mode.

Figure 4-10 Media Clock Generation and Recovery Flow



4.6.6 Enabling Media Generation and Recovery

When you enable the Flexible Pulse Per Second Output feature, by selecting DWCXG_FLEXI_PPS_OUT_EN, the media generation and recovery feature is also enabled. The number of comparators and the clock outputs are determined by value of the DWCXG_PPS_OUT_NUM parameter.

For details about these options, see IEEE 1588 Timestamp Parameters in *"Parameter Descriptions"* chapter.

Note See the "Enabled" field of the parameter descriptions to understand the dependencies.

4.6.7 Signals Related to Media Generation and Recovery

- `mcg_pst_trig_i`
- `mcg_dma_req_o`
- `mcgr_dma_ack_i`

For a description of signals related to Media Generation and Recovery, see the “Signal Descriptions” chapter.

4.6.8 Registers Related to Media Clock Generation and Recovery

- MAC_PPS_Control
- MAC_PPS_Target_Time_Seconds
- MAC_Presn_Time_ns
- MAC_Presn_Time_updt
- MAC_Timestamp_Control

For more details, see the *“Register Descriptions”* chapter.

4.6.9 Programming Guidelines for Media Clock Generation and Recovery

For more details about programming guidelines for media clock generation and recovery, see “Programming Guidelines for Media Clock Generation and Recovery” on page 1888.

4.7 One-Step Timestamp

The 25GMAC supports the one-step timestamp feature. When this feature is enabled, the MAC identifies the offset in the packet and inserts the timestamp received from the application or the timestamp taken for the packet being transmitted at that offset.

You can enable the one-step timestamp feature by selecting the Enable One-Step Timestamp Feature option. You can enable the one-step timestamp feature for a packet as follows:

- XGMAC-AXI configuration: Set the OSTC bit in the transmit descriptor
- XGMAC-MTL configuration: Set the OSTC bit in the transmit control word
- XGMAC-CORE configuration: Assert the mti_ost_en_i signal

For updating the correction field in certain PTP packets, the ingress timestamp is to be given in the TSSL and TSSH fields. If the Add IEEE 1588 Higher Word Register option is selected, the timestamp is 80-bit and it also contains the content of higher 16 bits of Higher Timestamp Word Register.

Note One-step timestamps require the sub-second time is programmed to operate in digital format (TSCTRLSSR bit in MAC_Timestamp_Control Register).

4.7.1 MAC Transmit PTP Mode

Depending upon the type of message and its mode, the MAC updates the following fields of Transmit PTP packets:

- correctionField in the PTP header of messages
- originTimestamp in SYNC, Delay_Req, and Pdelay_Req messages

Note For residence/turnaround time correction, it is assumed that the differences between egress and ingress timestamp might be a few seconds. So, the XGMAC-CORE supports a difference up to 8 seconds. Do not use this feature if your implementations requires a residence/turnaround time greater than 8 seconds.

Table 4-29 shows how the PTP mode is selected for a message based on the settings of SNAPTYPSEL, TSMSTRENA, and TSEVNTENA bits of the MAC_Timestamp_Control register. The MAC updates the corresponding fields (originTimestamp, correction field) in the transmit PTP packets received from the application.

Table 4-29 MAC Transmit PTP Mode

Programming			Mode	Per Packet Control[a]			Messages Processed on Tx
SNAPTYPSEL	TSMSTRENA	TSEVNTENA		TTSE[b]	OSTC[c]	TTS[d]	
2'b00	X	0	End-to-end transparent	0	1	Ingress TS	Sync (correction field for residence time and Ingress Asym cor)
							Delay_Req (correction field for residence time and Egress Asym Cor)

Table 4-29 MAC Transmit PTP Mode (continued)

Programming			Mode	Per Packet Control[a]			Messages Processed on Tx
SNAPTYPSEL	TSMSTRENA	TSEVNTENA		TTSE[b]	OSTC[c]	TTS[d]	
2'b00	0	1	Ordinary or Boundary Subordinate	1	1	X	Delay_Req (originTimestamp field) Delay_Req (correction field for Egress Asym cor)
2'b00	1	1	Ordinary or Boundary Manager	0	1	X	Sync (originTimestamp field) Sync (correction field for sub-nanosecond cor)
2'b01	X	0	End-to-End Transparent with support for peer delay mechanism	0	1	Ingress TS	Sync (correction field for residence time and Ingress Asym cor)
						Ingress TS	Pdelay_Req (correction field for residence time and Egress Asym Cor)
						Ingress TS	Pdelay_Resp (correction field for residence time and Ingress Asym Cor)
2'b01	0	1	Ordinary or Boundary Subordinate with support for peer delay mechanism or Peer to Peer Transparent	0	1	Ingress TS	Sync (correction field for residence time and Ingress Asym cor) (applicable only for Peer to Peer transparent clock operation)
				1	1	X	Delay_Req (originTimestamp field) Delay_Req (correction field for Egress Asym cor)
				1	1	X	Pdelay_Req (originTimestamp field) Pdelay_Req (correction field for Egress Asym Cor)
				0	1	Ingress TS for Pdelay_Req	Pdelay_Resp (correction field for turnaround time and Ingress Asym Cor)

Table 4-29 MAC Transmit PTP Mode (continued)

Programming				Per Packet Control[a]			
SNAPTYPSEL	TSMSTRENA	TSEVNTENA	Mode	TTSE[b]	OSTC[c]	TTS[d]	Messages Processed on Tx
2'b01	1	1	Ordinary or Boundary Manager with support for peer delay mechanism	0	1	X	Sync (originTimestamp field) Sync (correction field for sub-nanosecond cor)
				1	1	X	Pdelay_Req (originTimestamp field) Pdelay_Req (correction field for Egress Asym Cor)
				0	1	Ingress TS for Pdelay_Req	Pdelay_Resp (correction field for turnaround time and Ingress Asym Cor)
2'b10	X	X	End-to-End Transparent	0	1	Ingress TS	Sync (correction field for residence time and Ingress Asym cor)
						Ingress TS	Delay_Req (correction field for residence time and Egress Asym Cor)
2'b11	X	X	Peer-to-Peer Transparent	0	1	Ingress TS	Sync (correction field for residence time and Ingress Asym cor)

a. The per packet control values provided here are the recommended settings used by devices in typical PTP operation, for the programmed mode.
b. TTSE represents TTSE bit of transmit descriptor in XGMAC-AXI configuration, TTSE bit in TX control word in XGMAC-MTL configuration, mti_ena_timestamp_i signal in XGMAC-CORE configuration. The TTSE function is independent of the OST function and the programmed operation mode for OST. The MAC captures and returns the timestamp when the TTSE bit is set.
c. OSTC represents OSTC bit of transmit descriptor in XGMAC-AXI configuration, OSTC bit in TX control word in XGMAC-MTL configuration, mti_ost_en_i signal in XGMAC-CORE configuration. When OSTC is set to 0, OST operation is not performed; that is PTP packet is not modified.
d. TTS represents the timestamp value provided in the TTSH, TTSL fields of transmit descriptor in XGMAC-AXI configuration, timestamp provided in the TTSH, TTSL fields in the TX control word in XGMAC-MTL configuration, mti_ost_i signal in XGMAC-CORE configuration.

Note

1. Residence time/turnaround time is calculated as the difference between the captured timestamp (egress timestamp) and the ingress timestamp
2. When sub-nanosecond feature is enabled, residence time calculation includes sub-nanosecond accuracy.
3. Clocks supporting peer delay mechanism doesn't use delay request/response, but it is included in OST for flexibility

4.7.2 One-Step Time Stamping for PTP Over UDP

One step timestamp for PTP over UDP/IP is a configurable feature available in configurations with Enable IEEE 1588 Timestamp Support option selected. When you select this feature, 25GMAC supports the one step timestamp operation for PTP messages sent over UDP/IPv4 and UDP/IPv6. Because origin timestamp and/or correction field (based on the PTP message type and the programmed PTP mode) in the PTP message are updated as part of the one step timestamp operation, it also requires update to the checksum field of the UDP header encapsulating the PTP message payload.

In 25GMAC, checksum is updated for a packet, in the MTL before the packet is sent to the MAC where the PTP message is updated for one step timestamp operation. PTP requires the timestamp to be captured while the packet (SFD of the packet) is on the line and one step timestamp requires the origin timestamp and/or correction field to be updated in the PTP header while the packet is being transmitted on the line. Because the PTP message is present in the UDP payload, it is not possible to update the checksum field in the UDP header for modifications made to the PTP message, because it is already transmitted by then. Instead, the following options are supported that are in line with specification mentioned in Annex D & E of IEEE 1588-2008:

- When PTP message is sent over UDP/IPv4, the UDP checksum may be set to 0. The application must set the UDP checksum to 0 and set CIC to 0 or 1 while providing the packet to the MAC and the MAC does not update checksum for changes in origin timestamp or correction field.
- When PTP message is sent over UDP/IPv6, a transmitting node extends the UDP payload of all PTP messages by 2 octets beyond the end of the PTP message. The contents of the UDP checksum field or the final 2 octets of the UDP payload might be modified to ensure that the UDP checksum remains intact after modification of PTP fields. The application must add the two additional bytes at the end of the PTP message while providing the packet to the MAC and the MAC updates these two bytes to keep the checksum correct.

A programmable option (CSC field in MAC_Timestamp_Control register) supports UDP with non-zero checksum for IPv4. When set, the two bytes at the end of the PTP message are updated to keep the UDP checksum correct, similar to PTP over UDP/IPv6. So, the application must add two pad bytes at the end of the PTP message sent to the 25GMAC and 25GMAC updates these two bytes to keep the checksum correct.

Updating the pad bytes to keep the checksum correct requires the old value of correction field and/or origin timestamp in the packet. To get better latency and area, the application can set the origin timestamp field to zero, for PTP message in particular modes in which 25GMAC overwrites the origin timestamp. For example, 25GMAC operating in Ordinary/Boundary manager mode overwrites the origin timestamp field of the sync message, when OSTC is set. Because the sync message is originated by the port operating in Ordinary/Boundary manager mode, it is possible for the application generating the message to set the origin timestamp field to zero.

Transmit checksum engine is automatically enabled when this feature is selected. 25GMAC uses the transmit checksum engine to identify the start of PTP message in the UDP/IP packet. Checksum engine has the capability to detect some kind of errors in the header (L3) and payload (L4). Also checksum is not computed for some type of packets, which is indicated by "No" in Table 4-40. For one step operation the packet is free of those errors and have "Yes/Not applicable" for the corresponding packet type in Table 4-40. If an error is reported by the checksum engine or there is a "No" in Table 4-40, for PTP messages sent over UDP/IP, no update is performed on the PTP message.

For more details about the errors detected by transmit checksum offload engine, see “Transmit Checksum Offload Engine” on page 259.

Note

- The packet is sent with specific IP address and UDP port numbers and does not have IPv4 options/IPv6 extension headers. Applications must take care of these requirements before giving the packet to the 25GMAC. 25GMAC does not validate them. As PTP is identified by specific value in the UDP destination port and as 25GMAC does not check these fields, it assumes an IP packet with OSTC set as a PTP packet. Therefore it is the responsibility of the application to set the OSTC bit only for IP packets which contains PTP messages.
- For PTP over UPD/IPv6 and PTP over UDP/IPv4 with CSC set, the 25GMAC does not verify the presence of the two pad bytes, instead it overwrites the last two bytes of the UDP payload.
- When Enable One step timestamp for PTP over UDP/IP feature is selected, Enable Transmit TCP/IP Checksum Offload feature is automatically enabled and this is enabled only for Queue 0 in multiple transmit queue configurations. If the application requires OST support for PTP messages sent over UDP/IP for packets sent from queues other than 0, transmit checksum engine must be manually enabled for those transmit queues. If checksum engine is not enabled, no updates are done for PTP messages sent over UDP/IP even though OSTC is set.
- When Enable One step timestamp for PTP over UDP/IP feature is selected and OSTC is set, the Transmit Checksum Offload Engine processes the UDP/IP packet for identifying the PTP Message Header start offset irrespective of value in CIC field. The CIC field needs to be appropriately set based on whether Transmit Checksum Offload Engine is expected to perform checksum insertion for that packet.
- One step timestamp operation is not supported for tunneled packets

When you select Enable One step timestamp for PTP over UDP/IP feature and OSTC is set, the Transmit Checksum Offload Engine processes the UDP/IP packet for identifying the PTP Message Header start offset irrespective of value in CIC field. The CIC field needs to be appropriately set based on whether Transmit Checksum Offload Engine is expected to perform checksum insertion for that packet.

1. TTSE represents
   - TTSE bit of transmit descriptor in the XGMAC-AXI configuration
   - TTSE bit in TX control word in the XGMAC-MTL configuration
   - mti_ena_timestamp_i signal in the XGMAC-CORE configuration
2. OSTC represents
   - OSTC bit of transmit descriptor in XGMAC-AXI configuration
   - OSTC bit in TX control word in XGMAC-MTL configuration
   - mti_ost_en_i signal in XGMAC-CORE configuration
3. TTS represents the timestamp value provided in the
   - TTSH and TTSL fields of transmit descriptor in XGMAC-AXI configuration
   - TTSH and TTSL fields in the TX control word in XGMAC-MTL configuration
   - mti_ost_i signal in XGMAC-CORE configuration
4. The per packet control values specified are the recommended settings used by devices in a typical PTP operation, for the programmed mode.

5. The TTSE function is independent of the OST function and the programmed operation mode for OST. The MAC captures and returns the timestamp when the TTSE bit is set
6. Residence time/turnaround time is calculated as the difference between the captured timestamp (egress timestamp) and the ingress timestamp.
7. When sub-nanosecond feature is enabled, residence time calculation includes sub-nanosecond accuracy.
8. Clocks supporting peer delay mechanism doesn't use delay request/response, but it is included in OST for flexibility.

4.8 PTP Offload Function

This is an optional feature that enables the automatic generation of specific PTP packets when the XGMAC-CORE works as a specific node in the PTP network. These packets might be generated periodically or triggered by the host software. In other modes, this feature can parse the incoming PTP packets on the receiver, and automatically generate and respond to the required PTP packets. This helps in offloading certain functions of a PTP node with better accuracy and lower latency of responses.

Based on the programmed mode, the 25GMAC generates PTP Ethernet messages periodically or from the application, or based on reception of a particular PTP message. The following table indicates the PTP message generation criteria.

Table 4-30 PTP Message Generation Criteria

Programming					
SNAPTYPSEL	**TSMSTRENA**	**TSEVNTENA**	**Mode**	**Criteria for Generation of PTP Messages**	**PTP Message Type Generated**
2'b00	0	1	Ordinary or Boundary Subordinate	SYNC message reception	Delay_Req
2'b00	1	1	Ordinary or Boundary Manager	Periodic or on trigger from application	SYNC
				Delay_Req message reception	Delay_Resp
2'b01	0	1	Transparent Subordinate	Periodic or on trigger from application	Pdelay_Req
				Pdelay_Req message reception	Pdelay_Resp
				SYNC message reception	Delay_Req
2'b01	1	1	Transparent Manager	Periodic or on trigger from application	Pdelay_Req
				Pdelay_Req message reception	Pdelay_Resp
				Periodic or on trigger from application	SYNC
				Delay_Req message reception	Delay_Resp
2'b11	X	X	Peer-to-Peer Transparent	Periodic or on trigger from application	Pdelay_Req

Table 4-30 PTP Message Generation Criteria (continued)

Programming					
SNAPTYPSEL	TSMSTRENA	TSEVNTENA	Mode	Criteria for Generation of PTP Messages	PTP Message Type Generated
				Pdelay_Req message reception	Pdelay_Resp
All other programming combinations are invalid for PTP Offload feature.					

Note Clocks supporting peer delay mechanism must not generate delay request/delay response messages, as per IEEE 1588. This is supported for flexibility, with a programmable control bit (DRRDIS). You can use the DRRDIS bit to control the response generation for delay request/delay response message, that is, for example, in transparent subordinate mode, delay request is generated in response to received sync only when the bit is reset.

For example, when the 25GMAC is set as an ordinary clock or boundary Register Bus Interface clock in the PTP network, it can respond to the reception of SYNC messages with an automatic generation and transmission of the corresponding Delay_Req message. Similarly, various other modes of operation is explained in Table 4-30.

The 25GMAC supports the multicast communication model for the generation of SYNC and Pdelay_Req PTP messages, for instance, the Destination Address field of the generated PTP over Ethernet packet is the defined special multicast addresses (0x011B19000000 for all except peer delay mechanism messages and 0x0180C200000E for peer delay mechanism messages).

When the MAC responds to received SYNC, Delay_Req and Pdelay_Req PTP messages with special multicast destination address, it also uses the corresponding special multicast address in the DA field of the automatically generated Delay_Req, Delay_Resp, and Pdelay_Resp PTP messages, respectively.

When the MAC responds to received SYNC, Delay_Req and Pdelay_Req PTP messages with unicast destination address, it takes the SA field of the received packets and makes them as the DA field of the automatically generated Delay_Req, Delay_Resp, and Pdelay_Resp PTP messages, respectively.

At the same time, all the received PTP messages are forwarded to the application along with Rx status, indicating whether the response was generated by 25GMAC (PTONR bit).

When the 25GMAC automatically generates a PdelayReq or responds with a Delay_Req, the egress timestamp of these two PTP messages are provided in the Tx TS status (Tx Timestamp Status Register and interrupt generated).

In addition to messageType and version, PTP fields match for basic PTP over Ethernet message detection, the following additional fields are matched to qualify the received PTP message type:

1. The domainNumber field is checked for a match against the value programmed in the CSR.
2. The twoStepFlag in flagField field is checked for one-step indication (1'b0).
3. The transportSpecific field is checked for Default PTP over Ethernet (4'h0) or 802.1AS mode (4'h1) when enabled.

4.8.1 PTP Packet Generation

This section explains the format and content of the automatically generated PTP packets by the 25GMAC when this mode is enabled. The template of the Common PTP Message Header is provided, as well as the detailed description of the fields of specific PTP packets generated.

Table 4-31 Common PTP Message Header Fields

Bits								Octets	Offset
7	6	5	4	3	2	1	0		
transportSpecific				messageType				1‘	0
reserved				versionPTP				1	1
messageLength								2	2
domainNumber								1	4
reserved								1	5
flagField								2	6
correctionField								8	8
reserved								4	16
sourcePortIdentity								10	20
sequenceId								2	30
sequenceId								2	30
controlField								1	32
logMessageInterval								1	33

messageType

The following encoded values are used for PTP message types:

- SYNC – 4‘h0
- Delay_Req – 4‘h1
- Pdelay_Req – 4‘h2
- Pdelay_Resp – 4‘h3
- Delay_Resp – 4‘h9

transportSpecific

The following transport protocol encoding are used:

- Default PTP over Ethernet – 4‘h0
- 802.1AS mode – 4‘h1

versionPTP

It is always set to 2 as PTP version 2 is supported.

domainNumber

This contains the value from the MAC_PTO_Control Register.

flagField

- alternateMasterFlag (Octet 0 bit 0) – 1b0 for SYNC and Delay_Resp
- twoStepFlag (Octet 0 bit 1) – 1‘b0 for SYNC and Pdelay_Resp
- unicastFlag (Octet 0 bit 2) – 1‘b0 for Multicast Address, 1'b1 for Unicast Address

correctionField

For Pdelay_Resp PTP message, the difference between its egress timestamp and corresponding Pdelay_Req packet ingress timestamp are added to the correctionField.

sourcePortIdentity

This filed takes the value programmed in the MAC_Source_Port_Identityx registers.

sequenceId

Pdelay_Resp and Delay_Resp use the same sequenceId field from received Pdelay_Req and Delay_Req PTP messages. For SYNC/Delay_Req, Pdelay_Req, a separate sequenceId counter is maintained. These sequenceId counters gets incremented by 1 every time the corresponding message is generated and transmitted.

controlField

The following encoded values are used for controlField:

- SYNC – 8‘h00
- Delay_Req – 8‘h01
- Pdelay_Req – 8‘h05
- Pdelay_Resp – 8‘h05
- Delay_Resp – 8‘h03

logMessageInterval

- SYNC – This contains logSyncInterval from the corresponding MAC_Log_Message_Interval register.
- Delay_Resp – This contains the DRSYNCR value taken from the MAC_Log_Message_Interval register for a multicast PTP message and 8’h7F for unicast PTP message.
- Delay_Req, Pdelay_Req and Pdelay_Resp – 8‘h7F

where logSyncInterval = log2 (Mean Value of Interval in seconds)

The 25GMAC supports values of -15 to 15 for logSyncInterval fields, which translates to a range from 32.768µS (2-15) to 2^{15} sec. For a given value of log sync interval(N), the time interval between two SYNC packets is given by the following:

- $2^{(30+N)}$ nanoSec, when N is negative (-1 to -15)
- 2^{N} Sec, when N is positive (0 to 15)

For example

- When logSyncInterval is programmed to 1, the interval is 2^1; therefore, the SYNC message is sent once every 2 seconds.
- When logSyncInterval is programmed to -1, the interval is 2^{-1} = 0.536 sec; therefore, the SYNC message is sent once every 536 milliseconds. The value is 0.536 seconds, because 2^{-30} = 1 ns.
- When logSyncInterval is programmed to -5, the interval is 2^{-5} = 33.55ms; therefore, the SYNC message is sent once every 33.55 milliseconds.

Note The MAC uses the PTP system time to generate the intervals for periodic packet transmission. For negative values of log message interval programmed, the generated period may deviate from the value given by the equation 2(30+N), due to the non-binary nature of the nanoseconds field of the system time.

4.8.2 PTP Message Specific Fields

messageLength

There is no suffix supported, so this field contains the length of the PTP message that includes 34 byte PTP common header and the body specific to the message type.

For SYNC and Delay_Req packets, this field contains 44, whereas for Delay_Resp, Pdelay_Req and Pdelay_Resp, it contains 54.

originTimestamp

This field is the captured egress timestamp for SYNC, Delay_Req, and Pdelay_Req PTP messages.

receiveTimestamp

For Delay_Resp PTP message, this is the ingress timestamp of the corresponding received Delay_Req PTP message.

requestingPortIdentity

For Delay_Resp and Pdelay_Resp PTP messages, this is the sourcePortIdentity field taken from the corresponding received Delay_Req and Pdelay_Req PTP messages.

requestReceiptTimestamp

For the Pdelay_Resp PTP message, this field is set to 0.

4.9 IPv4 ARP Offload

The 25GMAC supports the Address Recognition Protocol (ARP) Offload for IPv4 packets. You can enable this feature by selecting the Enable IPv4 ARP Offload option. When you enable this feature, the 25GMAC generates the ARP reply packets for broadcast ARP request packets. The ARP packet for IPv4 is L2 layer packet with Length/Type of 0x0806.

The ARP offloading process is as follows:

1. The 25GMAC receiver gets an ARP request if the Target Protocol Address of request matches the IPv4 address programmed in the MAC_ARP_Address register.
2. The 25GMAC generates an ARP reply packet.
3. The 25GMAC copies the Sender Hardware Address field in the ARP request to the following fields:
   - DA field of the Ethernet packet header
   - Target Hardware Address field of the ARP reply packet
4. The 25GMAC copies the Sender Protocol Address field in the ARP request to the Target Protocol Address field in the ARP reply packet.
5. The 25GMAC places its MAC address in the following fields:
   - SA field of the Ethernet packet header
   - Sender Hardware Address field of the ARP reply packet
6. The 25GMAC copies the Target Protocol Address field in the ARP request to the Sender Protocol Address field in the ARP reply packet.
7. The 25GMAC sets the opcode field in ARP reply packet to 2, indicating ARP reply.
8. The 25GMAC re-calculates the CRC and performs padding for generated ARP reply packet.

The ARP request packet is forwarded to the application with the RPNG status bit set to 0 when the Receiver schedules the ARP reply to the 25GMAC transmitter. The 25GMAC processes only one ARP request at a time as it cannot store the fields of multiple ARP requests for the reply. Hence, when 25GMAC receives an ARP request while the ARP reply to an earlier ARP request is not fully transmitted, the 25GMAC does not generate the ARP reply for the new ARP request. The 25GMAC forwards the new ARP request packet to the application with Reply Packet Not Generated (RPNG, Bit 14 of RDES2 in Normal Descriptor) status bit set.

In power-down mode, if the 25GMAC receives an ARP request for which it cannot generate the reply due to an already scheduled reply, the 25GMAC drops the new ARP request without any reply. For more details about the power-down mode, see “Power-Down Mode Support” on page 365.

Note When the received ARP request is less than64 bytes packet length, 25GMAC does not send ARP response. It is treated as a normal packet and forwarded to the application based on the 25GMAC filter settings.

4.10 Memory Access in Debug Mode

The 25GMAC supports the debug access to the FIFO memory. You can enable this feature by selecting the Enable Debug Memory Access option. When you enable this feature, you must assign the entire selected Tx memory to Tx Queue 0 and Rx memory to Rx Queue 0. In addition, the offload features such as Checksum Offload Engine and IEEE timestamp must not be enabled with this feature.

When this feature is enabled, the MAC insert the pad (if required) and computed CRC in the Transmit packet. The FIFOs operate in the store-and-forward mode.

Note

- The debug access mode is not designed for dynamic switching from normal operation mode to debug access mode. The software/application must ensure that the Transmit and Receive paths are idle (All the debug status registers in CSR space are read as all-zeros). The software/application must wait for idle state or use mechanism such as Transmit Queue flush or soft reset to reach the idle state faster, prior to switching to debug access mode.
- While switching from debug access mode to the normal mode of operation, DMA must be reconfigured.
- In both the memory debug access modes, that is, Debug Mode Access and Subordinate Mode Access for applicable memories, the respective whole memory is accessed as a single memory; there is no per queue or hardware or software owned lists based logical partitioning. For example, the whole of Transmit Queue memory must be allocated to Transmit Queue 0 by programming TQS field of MTL_TxQ0_Operation_Mode register. Similarly, the whole of Receive Queue memory must be allocated to Receive Queue 0 by programming RQS field of MTL_RxQ0_Operation_Mode register. You cannot program per queue or hardware or software owned list sizes for other memories such as TSO Memory, EST memory, and Receive Parser Memory. In debug access mode, the MAC internally switches to accessing the respective entire memory as a single memory.

4.10.1 Subordinate Mode Access

You can use this mode to send or receive a packet using the Register interface. The host controller can push a complete packet into the Tx FIFO and then schedule it for transmission. Similarly, it can read the packet received and stored in the Rx FIFO through the registers.

The following section describes the sequence of programming to transmit or receive a packet in this mode.

4.10.1.1 Transmitting Packets through a Register Bus Interface

To write a packet to the Tx memory through the register bus interface, complete the steps given in Table 4-32.

Table 4-32 Steps to Write a Packet to Tx Memory Through a Register Bus Interface

Register Name	Step Number and Description
MTL_Debug_Control	1. Select the FIFO for subordinate access by using the DBGMOD, FDBGEN, and FIFOSEL fields.
MTL_FIFO_Debug_Data	2. Write the control word in the FDBGDATA field.
MTL_Debug_Control	3. Complete the following: a. Write the Control Word encoding to the PKTSTATE field.

Table 4-32 Steps to Write a Packet to Tx Memory Through a Register Bus Interface (continued)

Register Name	Step Number and Description
	b. Set the FIFOSEL field to Tx FIFO. c. Set the FIFOWREN bit. Note: The Control Word must contain the valid packet size.
MTL_Debug_Status	4. Read the FIFOBUSY bit and wait till this bit is low. When this bit is low, read the LOCR field (if required).
MTL_FIFO_Debug_Data	5. Write the SOP data in the FDBGDATA field.
MTL_Debug_Control	6. Complete the following steps: a. Write the SOP encoding in the PKTSTATE field. b. Set the FIFOSEL field to Tx FIFO. c. Set the FIFOWREN bit.
MTL_Debug_Status	7. Read the FIFOBUSY bit and wait till this bit is low. When this bit is low, read the LOCR field (if required).
MTL_FIFO_Debug_Data	8. Write subsequent data words in the FDBGDATA field.
MTL_Debug_Control	9. Complete the following steps: a. Write the packet data encoding in the PKTSTATE field. b. Set the FIFOSEL field to Tx FIFO. c. Set the FIFOWREN bit.
MTL_Debug_Status	10. Read the FIFOBUSY bit and wait till this bit is low. When this bit is low, read the LOCR field (if required).
MTL_Debug_Control and MTL_Debug_Status	11. Repeat, Step 8, Step 9, and Step 10 and till the last word of the packet data.
MTL_FIFO_Debug_Data	12. Write the last data word in the FDBGDATA field.
MTL_Debug_Control	13. Complete the following steps: a. Write the EOP encoding in the PKTSTATE field. b. Set the FIFOSEL field to Tx FIFO. c. Set the FIFOWREN bit. d. Set the number of valid bytes in the BYTEEN field.
MTL_Debug_Status	14. Read the FIFOBUSY bit and wait till this bit is low. When this bit is low, read the LOCR field (if required). The packet is transmitted after the EOP is written.

4.10.1.2 Receiving Packets through a Register Bus Interface

To read a packet from the Rx memory through a register bus interface, complete the steps given in Table 4-33.

Table 4-33 Steps to Read a Packet from Rx Memory Through a Register Bus Interface

Register Name	Step Number and Description
MTL_Debug_Control	1. Select the FIFO for subordinate access by programming the PKTIE, DBGMOD, and FDBGEN fields.
MTL_Debug_Status	2. Wait for the PKTI interrupt.
MTL_Debug_Control	3. Complete the following: a. Set the FIFOSEL field to Rx FIFO. b. Set the FIFORDEN bit.
MTL_Debug_Status	4. Read the FIFOBUSY bit and wait till this bit is low.
MTL_Debug_Control	5. Read the PKTSTATE and BYTEEN fields.
MTL_FIFO_Debug_Data	6. Read the data in the FDBGDATA field.
MTL_Debug_Status	7. Read the LOCR field (if required).
MTL_Debug_Status, MTL_Debug_Control, MTL_FIFO_Debug_Data	8. Repeat Step 3 through Step 7 till the complete packet, along with the Rx status, is received.

4.10.2 Debug Mode Access

You can use this mode for to test the FIFO and TSO (if present) memories by performing writes operations in a location and read back the same. You can switch the mode when the complete word is read or written in one location. The following sections describe how to write to or read from FIFO memory in debug mode.

4.10.2.1 Writing to FIFO Memory in Debug Mode

To write to FIFO memory in the debug mode, complete the steps given in Table 4-34.

Table 4-34 Steps to Write to FIFO memory in Debug Mode

Register Name	Step Number and Description
MTL_Debug_Control	1. Select the FIFO for debug access by using the DBGMOD and FDBGEN fields. Set RSTALL to clear all the pointers to the start location (0x0). If you want to clear the pointers of a specific memory, select using FIFOSEL and set RSTSEL.
MTL_FIFO_Debug_Data	2. Write the data in the FDBGDATA field.
MTL_Debug_Control	3. Complete the following: a. Set the FIFOSEL field to Tx FIFO, Rx FIFO, or TSO FIFO. b. Set the FIFOWREN bit.

Table 4-34 Steps to Write to FIFO memory in Debug Mode (continued)

Register Name	Step Number and Description
MTL_Debug_Status	4. Read the FIFOBUSY bit and wait till this bit is low. When this bit is low, read the LOCR field (if required).
MTL_Debug_Control, MTL_Debug_Status	5. Repeat Step 2, Step 3, and Step 4 twice to write one complete data word in 64-bit mode. Repeat these steps four times in 128-bit mode. The LOCR field is incremented to the next location when a complete data word is written. You can repeat the Step 2 to Step 5 for writing in to the subsequent locations.

4.10.2.2 Reading from FIFO Memory in Debug Mode

To read from FIFO memory in the debug mode, complete the steps given in Table 4-35.

Table 4-35 Steps to Read from FIFO Memory in Debug Mode

Register Name	Step Number and Description
MTL_Debug_Control	1. Select the FIFO for debug access by using the DBGMOD and FDBGEN fields. Set RSTALL to clear all the pointers to the start location (0x0). If you want to clear the pointers of a specific memory, select using FIFOSEL and set RSTSEL. 2. Complete the following: a. Set the FIFOSEL field to Tx FIFO, Rx FIFO, or TSO FIFO. b. Set the FIFORDEN bit.
MTL_Debug_Status	3. Read the FIFOBUSY bit and wait till this bit is low. When this bit is low, read the LOCR field (if required).
MTL_FIFO_Debug_Data	4. Read the data in the FDBGDATA field.
MTL_Debug_Status	5. Read the LOCR field (if required).
MTL_Debug_Control, MTL_Debug_Status, MTL_FIFO_Debug_Data	6. Repeat Step 2 and Step 5 twice to read one complete data word in 64-bit mode. Repeat these steps four times in 128-bit mode. The LOCR field is incremented to the next location when a complete data word is read. You can repeat the Step 2 to Step 5 for reading from the subsequent locations.

4.11 Flow Control

This section describes the flow control for Transmit and Receive paths.

4.11.1 Transmit Flow Control

The Transmit Flow Control involves transmitting pause packets in full-duplex mode and back-pressure in half-duplex mode, to control the flow of packets from the remote end.

The pause packets are transmitted as per the values programmed in the MAC_Q0_Tx_Flow_Ctrl register.

You can enable the transmit flow control independently for each priority by using the priority flow control (PFC) packets. Program the register bits as follows:

- TFE = 1 in the MAC_Q(#i)_Tx_Flow_Ctrl register.
- PFCE = 1 in the MAC_Rx_Flow_Ctrl register.

4.11.1.1 Flow Control in Full-Duplex Mode

The 25GMAC supports full-duplex flow control operations using

- IEEE pause packets
- Priority flow control (PFC) packets

The PFC packets are used only when you select the Enable Data Priority Flow Control option in coreConsultant or specify the DWCXG_PFC_EN parameter while configuring the 25GMAC controller.

The PFCE bit of the MAC_Rx_Flow_Ctrl register determines whether PFC packets or IEEE pause control packets are used for the flow control.

- PFCE = 1

  MAC sends the PFC packets.

  Each MAC_Q(#i)_Tx_Flow_Ctrl register controls the PFC packet generation and content for the corresponding RxQ(#i) flow control trigger and priorities of packets mapped to that queue.

  Each bit of the multi-bit sbd_flow_ctrl_i signal is associated with the corresponding MAC_Q(#i)_Tx_Flow_Ctrl register.

- PFCE = 0

  MAC sends pause packets as per the programming of the MAC_Q0_Tx_Flow_Ctrl register. The other registers, MAC_Q[1:7]_Tx_Flow_Ctrl are ignored.

  Assertion of any bit of the multi-bit sbd_flow_ctrl_i triggers the flow control. However, only when all the bits of the sbd_flow_ctrl_i signal are de-asserted, the flow control is deactivated.

Pause Packet Structure

Table 4-36 describes the fields of a pause packet.

Table 4-36 Pause Packet Fields

Octets	Field	Pause Packet
0 - 5	DA	Special multicast address = 0180-c200-0001
6 - 11	SA	MAC_Adddress0 register

Table 4-36 Pause Packet Fields (continued)

Octets	Field	Pause Packet
12 - 13	Type	Control type = 0x8808
14 - 15	Opcode	0x0001
16 - 17	PT/PEV	PT field of MAC_Q0_Tx_Flow_Ctrl register
18 - 19	PT0	Reserved
20 - 21	PT1	Reserved
...	...	Reserved
32 - 33	PT7	Reserved
34 - 59	...	Reserved
60 - 63	FCS	CRC

Pause Packet Control

When the FCB = 1, in the MAC_Q0_Tx_Flow_Ctrl register, the MAC generates and transmits a single pause packet. If the FCB bit is set again after the Pause packet transmission is complete, the MAC sends another Pause packet irrespective of whether the pause time is complete or not. To extend the pause or terminate the pause prior to the time specified in the previously-transmitted Pause packet, the application must program the Pause Time register with appropriate value and then again set the FCB bit.

Similarly, when the mti_flowctrl_i signal (any bit in a multi-bit bus) is asserted, the MAC generates and transmits a single pause packet. If the mti_flowctrl_i signal remains asserted at a configurable number of slot times before the pause time runs out, the MAC transmits a second pause packet. This process is repeated as long as the mti_flowctrl_i signal remains active.

If the mti_flowctrl_i signal goes inactive (all bits in a multi-bit bus) prior to the sampling time and the DZPQ bit in the MAC_Q(#i)_Tx_Flow_Ctrl register is 0, the MAC transmits a pause packet with zero pause time to indicate to the remote end that the Receive buffer is ready to receive new data packets.

Priority Flow Control (PFC) Packets

For PFC packets, you can specify the priority, pause time, and other controls for a queue in the MAC_Q(#i)_Tx_Flow_Ctrl register. The MAC supports independent flow control for each Rx queue and the priorities of packets in that queue is taken from the corresponding PSRQ fields of the MAC_RxQ_Ctrl2 and MAC_RxQ_Ctrl3 registers. Each Rx queue has one trigger input. When the trigger is activated,

1. The priority value programmed in the RxQ PSRQ(#i) field is copied to the PEV field.
2. The value in the PT field of the corresponding MAC_Q(#i)_Tx_Flow_Ctrl is copied to the PT(#i) field of the PFC packet.
3. If multiple priorities are programmed in the same Rx queue, multiple priorities are assigned to the PFC packet with same pause time values.

For example, when mti_flow_ctrl_i[4] is high and PSRQ4 = 0x05 (Priority 0 and Priority 2),

1. The value in the PEV field is set to 0x05
2. The value in the PT field of the MAC_Q4_Tx_Flow_Ctrl register is copied to the PT0 and PT2 fields of the PFC packet.

When the mti_flow_ctrl_i[4] becomes low and the DZPQ field of MAC_Q4_Tx_Flow_Ctrl register = 1, a PFC packet with PEV = 0x05 and PT0 and PT2 = 0 (Zero quanta PFC) is transmitted.

Table 4-37 PFC Packet Fields

Octets	Field	PFC Packet
0 - 5	DA	Special multicast address = 0180-c200-0001
6 - 11	SA	MAC_Adddress0 register
12 - 13	Type	Control type = 0x8808
14 - 15	Opcode	0x0101
16 - 17	PT/PEV	Priority Enable Vector indicating which of the following PT# fields are valid
18 - 19	PT0	PT field of MAC_Q0_Tx_Flow_Ctrl register
20 - 21	PT1	PT field of MAC_Q1_Tx_Flow_Ctrl register
...	...	
32 - 33	PT7	PT field of MAC_Q7_Tx_Flow_Ctrl register
34 - 59	...	
60 - 63	FCS	CRC

In case of simultaneous multiple triggers, the MAC sends one consolidated PFC packet. This PFC packet has the PEV set to the combined (ORed) values of the corresponding PSRQ[i] fields and the respective PT[i] fields are taken from the appropriate MAC_Q(#i)_Tx_Flow_Ctrl registers. This reduces the PFC traffic on the transmitter and improves data throughput.

Table 4-38 describes the flow control in the Tx path based on the setting of the following bits:

- EHFC bit of the MTL_RxQ0_Operation_Mode register
- TFE bit of the MAC_Q0_Tx_Flow_Ctrl register
- HD bit of the MAC_Extended_Configuration register

Table 4-38 Flow Control in Tx Path

EHFC	TFE	HD	Description
x	0	x	The MAC transmitter does not perform the flow control or back-pressure operation.
0	1	1	The MAC transmitter performs back-pressure when bit [0] of the MAC_Q0_Tx_Flow_Ctrl register is 1.

Table 4-38 Flow Control in Tx Path (continued)

EHFC	TFE	HD	Description
			The MAC transmitter performs back-pressure when the sideband signal, sbd_flowctrl_i is 1.
1	1	1	The MAC transmitter performs back-pressure when bit [0] of the MAC_Q0_Tx_Flow_Ctrl register is 1. In addition, the MAC Tx performs back-pressure when Rx Queue level crosses the threshold set by RFA field of MTL_RxQ#_Flow_Control register. The MAC transmitter performs back-pressure when the sideband signal sbd_flowctrl_i is 1.
0	1	0	The MAC transmitter sends the pause packet when bit [0] of the MAC_Q0_Tx_Flow_Ctrl register is 1. The MAC transmitter sends the pause packet when the sideband signal sbd_flowctrl_i is 1.
1	1	0	The MAC transmitter sends the pause packet when bit [0] of the MAC_Q0_Tx_Flow_Ctrl register is 1. In addition, the MAC Tx sends a Pause packet when Rx Queue level crosses the threshold set by Bits[10:8] of MTL_RxQ0_Operation_Mode register. The MAC transmitter sends the pause packet when the sideband signal sbd_flowctrl_i is 1.

4.11.1.2 Flow Control in Half-Duplex Mode

In the half-duplex mode, the MAC uses the deferral mechanism for the flow control (back-pressure). When the application requests to stop receiving packets, the MAC sends a JAM pattern of 32 bytes when it senses a packet reception, provided the transmit flow control is enabled. This results in a collision and the remote station backs off. If the application requests a packet to be transmitted, it is scheduled and transmitted even when the back-pressure is activated. If the back-pressure is active for a long time (more than 16 consecutive collision events occur), the remote stations terminate the transmission because of excessive collisions.

4.11.1.3 Triggering Transmit Flow Control

The transmit flow control implies

- Transmitting pause packets in full-duplex mode
- Initiating back pressure in half-duplex mode

to control the flow of packets from the remote end.

The application can request the MAC to send a pause packet or initiate back-pressure by using either of the following methods:

- Software Trigger
- Hardware Trigger

Triggering Flow Control Through Software

The application appropriately programs the FCB bit in the corresponding MAC_Q(#i)_Tx_Flow_Ctrl register.

Triggering Flow Control Through Hardware

The flow control operation of the MAC is enabled when the EHFC bit of corresponding MTL_RxQ(#i)_Operation_Mode register is set.

The hardware flow control types:

- Rx Queue Threshold Based Flow Control:
  - The flow control signal to the MAC is asserted when the fill level of the Rx queue crosses the threshold configured in RFA field of the MTL_RxQ(#i)_Flow_Control register.
  - The flow control signal is de-asserted when the fill-level of the queue falls below the threshold configured in the RFD field of the MTL_RxQ(#i)_Flow_Control.

  The hardware flow control generated based on the Rx Queue threshold crossing condition is applicable only when the Rx queue size is 4,096 bytes or more.

Note The RFS sets the threshold at which the flow control is triggered and the MAC schedules to send a PAUSE frame to be sent by the local transmitter. The remote transmitter can continue to send packets until it receives the PAUSE packet. So the RXQ needs space to take in this data even after the flow-control is triggered, to avoid overflow and loss of packets. The max-space required that is calculated theoretically can be 2 max-sized packets + a little more, assuming that the read from the RXFIFO does not occur during this time. Theoretically, this makes RFA=4 or more, which implies that RxQ must have at the least 4 KB size. Practically, the system works with RFA=2 (Full – 2KB, with very little probability of overflow). If you have FIFO size of less than 4K, with RFA=2, the flow control is triggered with just 1000B in the RxQ which is even less than max packet size. The RFA field MTL_RxQ(#i)_Operation_Mode register is used for activating the Flow Control. These bits control the threshold (fill-level of Rx queue) at which the flow control is activated. For Example, if your RXQ size is 2K and you set the RFA to 1K; as soon as you receive 1K, the Flow Control is activated. If you send a packet of 1500 bytes (which is allowed as per spec), this triggers flow-control even before you receive a complete packet, which reduces the throughput considerably. If your RXQ size is 4K, you can set the RFA to 2K, so that you are able to receive the whole packet.

- Sideband Signal Based Flow Control Signal:

  The XGMAC-AXI and XGMAC-MTL configurations support an optional and separate sideband flow control signal `sbd_flowctrl_i`.

  You can select this signal by selecting the Add Rx Flow Control Enable as Primary Input option while configuring the controller in coreConsultant.

  You can use the sbd_flowctrl_i signal to enable the hardware flow control for any Rx queue size.

  For configurations with 4,096 bytes or larger Rx queues, if sideband flow control signal is also enabled, the flow control signal based on the Rx Queue (`hw_flowctrl`) and the sideband signal (`sbd_flowctrl_i`) are logically ORed and provided to the MAC. Therefore, flow control is initiated when either of these signals is asserted. The flow control is disabled when both these signals are de-asserted.

4.11.2 Receive Flow Control

In the receive path, the flow control is functional only in full-duplex mode. If the pause packet is received in half-duplex mode, the packet is considered as a normal control packet.

In configurations with Enable Priority Flow Control option selected, you can enable the Pause flow control by setting the RFE bit in the `MAC_Rx_Flow_Ctrl` register.

Table 4-39 describes the flow control in the Rx path based on the RFE bit of the MAC_Rx_Flow_Ctrl register

Table 4-39 Flow Control in Rx Path

RFE	Description
0	The MAC receiver does not detect the received pause packets.
1	The MAC receiver detects or processes the pause packets and responds to such packets by stopping the MAC transmitter.

In configurations with multiple queues, you can enable the PFC packet detection by setting the PFCE bit in the MAC_Rx_Flow_Ctrl register. If PFC packet detection is enabled, the MTL Tx queues corresponding to the received priority is blocked when the PFC packet is received. If PFC packet detection is not enabled in configurations with multiple queues and RFE bit is enabled, the MAC transmitter is blocked when the IEEE pause packet is received.

The following list describes the Rx flow control:

1. The MAC checks the destination address of the received pause packet for either of the following:
   - Multicast destination address: The DA matches the unique multicast address specified for the control packet (48'h0180C2000001).
   - Unicast destination address: The DA matches the content of the MAC_Address0 register and the UP bit of the MAC_Rx_Flow_Ctrl register is set.

     If the UP bit is set and the MAC processes pause packets with unicast destination address in addition to the unique multicast address.
2. The MAC decodes the following fields of the received packet:
   - Type field: This field is checked for 16'h8808.
   - Opcode field: This field is checked for 16'h0001 (Pause packet) or 16'h0101 (PFC packet).
   - Pause Time field: The pause time is captured to determine the time for which transmitter needs to be blocked.

     The Pause Time Vector field: For the PFC packet, the pause time vector field is captured to determine the time for which the MTL Tx queue corresponding to the received priority needs to be blocked.
   - Priority Enable Vector field: This field is valid only for PFC packets. It is captured to determine the MTL Tx queue to be paused for the received priority.
3. If the byte count of the status indicates 64 bytes and there is no CRC error, the MAC transmitter performs the following:
   - For IEEE pause packets:

     Pauses the transmission of data packet for a duration of

     decoded Pause Time value * slot time (64 byte times).

- For PFC packets:
  - XGMAC-CORE

    The MAC blocks the Tx queues corresponding to the priority. The MAC receiver starts the pause timer corresponding to the active priorities indicated in the PEV field, with the respective PT[n] values. The mti_disable_prty_o signal bits is driven high for the respective pause times. Use the mti_disable_prty_o signal to block your respective Tx queues associated with the priorities (mti_disable_prty_o[0] corresponds to priority 0 and so on).
  - XGMAC-MTL, XGMAC-AXI

    When there are multiple Tx Queues, the queues that are paused are decided by a two-step process. In the Tx side, each queue is mapped to traffic classes as per the programming of the Q2TCMAP field of the respective MTL_TxQ(#i)_Operation_Mode register. The priorities associated with a traffic class is decided by the respective PSTC[n] field of the MTL_TC_Prty_Map register.

    If you enable PFC for a specific priority, all the Traffic Classes that have the priority associated with the PFC, are paused by the scheduler. As a result, all the Tx queues mapped to those traffic classes are paused.

    All bits of the mti_disable_prty_o signal are asserted when the MAC transmitter is in LPI mode. This is to ensure the ETS scheduler start over again after exiting the LPI mode.

4. The MAC transfers the received control packet to the application based on the setting of the PCF field in the MAC_Packet_Filter register.
5. If subsequent pause or PFC packets are received before the earlier Pause Time expires, the MAC updates the Pause Timer with new value.
6. In the case of VXLAn/NVGRE Tunneled Packet this feature is supported only for the Outer Packet as this is a L3 Link Protocol.

4.12 Checksum Offload Engine

Communication protocols such as TCP/UDP implement checksum fields, which are useful to determine the integrity of the data transmitted over a network. Because the most widespread use of Ethernet is to encapsulate TCP/UDP over IP datagrams, the 25GMAC Subsystem has an optional Checksum Offload Engine to support checksum calculation and insertion in the transmit path, and error detection in the receive path.

4.12.1 Transmit Checksum Offload Engine

The checksum offload engine supports two types of checksum calculation and insertion.

- "IP Header Checksum Engine" on page 259
- "TCP/UDP/ICMP Checksum Engine" on page 260

The checksum engine can be controlled for each packet by the CIC bits:

- Transmit control word0, Bits [21:20] in XGMAC-MTL configuration
- TDES3 Bits[17:16] in XGMAC-AXI configuration

Note

- The checksum for TCP, UDP, or ICMP is calculated over a complete packet, and then inserted into its corresponding header field. Because of this requirement, when this function is enabled in a packet, the corresponding TxQ automatically operates in the store-and-forward mode even if the 25GMAC is configured for Threshold (cut-through) mode.
- You must make sure that the TxQ is deep enough to store a complete packet before that packet is transferred to the MAC transmitter. The reason being that when space is not available to accept the programmed burst length of data, then the MTL TxQ starts reading out the packet to avoid dead-lock. Therefore, you must enable the checksum insertion only in the packets that are less than the following number of bytes in size:

  TXQ SIZE in bytes – ((PBL + 5)*(DATAWIDTH/8))

  where TxQ size is given by the TQS field in MTL_TxQ(#i)_Operation_Mode register and the DATAWIDTH (64-bit, 128-bit) is selected during RTL configuration.

  This calculation is also applicable for the normal and jumbo packets. The PBL is the TxPBL field in the VDMA_CH(#i)_Tx_Control register in XGMAC-AXI configurations or ati_q[n]_pbl_i in XGMAC-MTL.

Note

See IETF specifications RFC 791, RFC 793, RFC768, RFC 792, RFC 2460, and RFC 4443 for IPv4, TCP, UDP, ICMP, IPv6, and ICMPv6 packet header specifications, respectively.

4.12.1.1 IP Header Checksum Engine

In IPv4 datagrams, the integrity of the header fields is indicated by the 16-bit Header Checksum field (the eleventh and twelfth bytes of the IPv4 datagram). The COE detects an IPv4 datagram when the Type field of Ethernet packet has the value 0x0800 and the Version field of IP datagram has the value 0x4. The checksum field of the input packet is ignored during calculation and replaced with the calculated value.

IPv6 headers do not have a checksum field. Therefore, the COE does not modify the IPv6 header fields.

The IP Header Checksum engine considers the packet having an error for the following conditions:

- For IPv4 datagrams:
  - The received Ethernet type is 0x0800, but the Version field of IP header is not equal to 0x4.
  - The IPv4 Header Length field indicates a value less than 0x5 (20 bytes).
  - The total packet length is less than the value given in the IPv4 Header Length field.
- For IPv6 datagrams:
  - The Ethernet type is 0x86dd but the IP header Version field is not equal to 0x6.
  - The packet ends before the IPv6 header (40 bytes) or extension header (as given in the corresponding Header Length field in an extension header) is completely received.

If COE detects an IP header error, it still inserts an IPv4 header checksum if the Ethernet Type field indicates an IPv4 payload.

25GMAC does not report error status of transmitted packets in the XGMAC-AXI and optionally reports the error status in the XGMAC-MTL configuration. So, the IP Header Error and Payload Checksum Error is provided in the MAC_Rx_Tx_Status register; an interrupt is generated when enabled.

The Payload Checksum engine declares an error when it detects that the packet has been forwarded to the MAC Transmitter engine in the store-and-forward mode without the end of packet (EOP) being written to the FIFO, or when the packet ends before the number of bytes indicated by the Payload Length field in the IP Header is received or when the Packet size (as indicated by IP header) is such that it can not accommodate minimum header size of TCP, UDP or ICMP header. When this engine detects these type of errors, it does not modify the TCP, UDP, or ICMP header.

When the packet is longer than the indicated payload length, the COE ignores them as stuff bytes, and no error is reported. When this engine detects this, it still inserts the calculated checksum into the corresponding TCP, UDP, or ICMP header field.

4.12.1.2 TCP/UDP/ICMP Checksum Engine

The TCP/UDP/ICMP Checksum Engine processes the IPv4 or IPv6 header (including extension headers) and determines whether the encapsulated payload is TCP, UDP, or ICMP. The checksum is calculated for the TCP, UDP, or ICMP payload and inserted into its corresponding field in the header. The Tx COE can work in the following two modes:

- The TCP, UDP, or ICMPv6 pseudo-header is not included in the checksum calculation and is assumed to be present in the Checksum field of the input packet. This engine includes the Checksum field in the checksum calculation, and then replaces the Checksum field with the final calculated checksum.
- The engine ignores the Checksum field, includes the TCP, UDP, or ICMPv6 pseudo-header data into the checksum calculation, and overwrites the checksum field with the final calculated value.

Note For ICMP-over-IPv4 packets, the Checksum field in the ICMP packet must always be 16'h0000 in both modes, because pseudo-headers are not defined for such packets. If it does not equal 16'h0000, an incorrect checksum may be inserted into the packet.

The Payload Checksum engine declares an error when it detects that the packet has been forwarded to the MAC Transmitter engine in the store-and-forward mode without the end of packet (EOP) being written to the FIFO, or when the packet ends before the number of bytes indicated by the Payload Length field in the IP Header is received or when the Packet size (as indicated by IP header) is such that it can not accommodate minimum header size of TCP, UDP or ICMP header. When the packet is longer than the indicated payload length, the COE ignores them as stuff bytes, and no error is reported. When this engine detects the first type of error, it does not modify the TCP, UDP, or ICMP header. For the second error type, it still inserts the calculated checksum into the corresponding header field.

Table 4-40 describes the functions supported by Transmit Checksum Offload engine based on the packet type. When the MAC does not insert the checksum, it is indicated as "No" in the table.

Note

Checksum insertion for IPv4 or IPv6 packets that are greater than the frame size constraint specified in Transmit Checksum Offload Engine must not be enabled because it might result in incorrect checksum insertion or unexpected behavior.

If the total packet size from the ATI interface or DMA engine exceeds the limit of 14'h3FFF, the checksum is not inserted for that packet.

Table 4-40 Transmit Checksum Offload Engine Functions for Different Packet Types

Packet Type	Hardware IP Header Checksum Insertion	Hardware TCP/UDP Checksum Insertion
Non-IPv4 or IPv6 packet	No	No
IPv4 packet is greater than 1,522 bytes (2,000 bytes when IEEE 802.3ad Support for 2K Packets is enabled in MAC) but less than or equal to the frame size constraint specified in Transmit Checksum Offload Engine.	Yes	Yes
IPv6 packet is greater than 1,522 bytes (2,000 bytes when IEEE 802.3ad Support for 2K Packets is enabled in MAC) but less than or equal to the frame size constraint specified in Transmit Checksum Offload Engine.	Not applicable	Yes
IPv4 with TCP, UDP, or ICMP	Yes	Yes
IPv4 packet has IP options (IP header is longer than 20 bytes)	Yes	Yes
Packet is an IPv4 fragment	Yes	No
IPv6 packet with the following next header fields in main or extension headers: ■ Hop-by-hop options (in IPv6 main header) ■ Hop-by-hop options (in IPv6 extension header) ■ Destinations options ■ Routing (with segment left 0) ■ Routing (with segment left > 0)	■ Not applicable ■ Not applicable ■ Not applicable ■ Not applicable ■ Not applicable ■ Not applicable	■ Yes ■ No ■ Yes ■ No ■ No ■ Yes

Table 4-40 Transmit Checksum Offload Engine Functions for Different Packet Types (continued)

Packet Type	Hardware IP Header Checksum Insertion	Hardware TCP/UDP Checksum Insertion
■ TCP, UDP, or ICMP ■ Authentication ■ Any other next header field in main or extension headers	■ Not applicable ■ Not applicable	■ No ■ No
IPv4 packet has TCP header with Options fields	Yes	Yes
IPv4 Tunnels: ■ IPv4 packet in an IPv4 tunnel ■ IPv6 packet in an IPv4 tunnel	■ Yes (Both IPv4 tunnel and packet header) ■ Yes (IPv4 tunnel header)	■ See "Checksum Engine for Tunneled Packets" on page 263 ■ See "Checksum Engine for Tunneled Packets" on page 263
IPv6 Tunnels: ■ IPv4 packet in an IPv6 tunnel ■ IPv6 packet in an IPv6 tunnel	■ Yes (IPv4packet header) ■ Not applicable	■ See "Checksum Engine for Tunneled Packets" on page 263 ■ See "Checksum Engine for Tunneled Packets" on page 263
IPv4 packet has 802.3ac tag (with C-VLAN Tag or S-VLAN Tag when enabled).	Yes	Yes
IPv6 packet has 802.3ac tag (with C-VLAN Tag or S-VLAN Tag when enabled).	Not applicable	Yes
IPv4 frames with security features (such as encapsulated security payload)	Yes	No
IPv6 frames with security features (such as encapsulated security payload)	Not applicable	No

4.12.1.3 Checksum Engine for Tunneled Packets

Checksum insertion is enabled for tunneled packets when VNE bit of MAC_Tx_Configuration register is set and the VNP=3'b011 in the Tx Descriptor of the corresponding packet. The checksum engine calculates and inserts TCP, UDP, or ICMP payload checksum of the packet as per the rules mentioned in Table 4-40. For VXLAN tunneled packet, the UDP checksum of the outer header is not modified and assumed to be set to 0 by the application.

Note that when VNP is not set to 3'b011 for tunneled packets, the checksum engine performs checksum insertion only in the outer headers. In case VNP=3'b011 for non-tunneled packets, the checksum engine may incorrectly consider UDP packets as tunneled packets in VXLAN mode and bypasses the calculation of UDP header checksum but modify certain fields inside the payload assuming it to be the inner header. Similarly in NVGRE mode, IP packets with protocol field = GRE are considered as tunneled packet and the engine might modify payload bytes assuming it to be inner header checksum.

In case of tunneled packets, if the Outer Ethernet and Outer IP Header has the following discrepancies, the Packet is left untouched:

1. The Ethernet packet Type is neither IPv4 nor IPv6.
2. There is a IPv4 version mismatch between Ethernet Type and IP Header.
3. The IPv4 packet with Options header or IPv4 packet is a fragmented packet.
4. The IPv6 packet with the Extension Headers.
5. The IPv4/IPv6 packet with non-UDP protocol field in VxLAN mode or non-GRE protocol field in NVGRE mode.
6. The packet prematurely ends before the complete IPv4/IPv6 Header is processed.

Also, for the tunneled packets, checksum rules in section "IP Header Checksum Engine" on page 259, and section "TCP/UDP/ICMP Checksum Engine" on page 260, are applicable for the Inner Ethernet/IP and Inner TCP/UDP/ICMP.

4.12.2 Receive Checksum Offload Engine

You can enable the Receive Checksum Offload Engine (Rx COE) by selecting the Enable Receive TCP/IP Checksum Check option and setting the IPC bit of the MAC_Rx_Configuration register. When this option is selected, both IPv4 and IPv6 packet in the received Ethernet packets are detected and processed for data integrity. The MAC receiver identifies IPv4 or IPv6 packets by checking for value 0x0800 or 0x86DD, respectively, in the Type field of the received Ethernet packet. This identification is applicable to single VLAN-tagged packets. It is also applicable to double VLAN-tagged packets when the Enable Double VLAN Processing option is selected and the EDVLP bit of the MAC_VLAN_Tag register is set.

The Rx COE calculates the IPv4 header checksums and checks that they match the received IPv4 header checksums. The result of this operation (pass or fail) is given to the RFC module for insertion into the receive status word. The IP Header Error bit is set for any mismatch between the indicated payload type (Ethernet Type field) and the IP header version, or when the received packet does not have enough bytes, as indicated by the Length field of the IPv4 header (or when fewer than 20 bytes are available in an IPv4 header), when DIS_TCP_EF bit is set in the MTL_RxQ(#i)_Operation_Mode register.

This engine also identifies a TCP, UDP, or ICMP payload in the received IP datagrams (IPv4 or IPv6) and calculates the checksum of such payloads properly, as defined in the TCP, UDP, or ICMP specifications. This engine includes the TCP, UDP, or ICMPv6 pseudo-header bytes for checksum calculation and checks whether the received checksum field matches the calculated value. The result of this operation is given as

a Payload Checksum Error bit in the receive status word. This status bit is also set if the length of the TCP, UDP, or ICMP payload does not match the expected payload length given in the IP header.

Table 4-41 describes the functions supported by the Rx COE based on the packet type. When the payload of an IP packet is not processed (indicated as "No" in the table), the information (whether the checksum engine is bypassed or not) is given in the receive status.

Note The MAC does not append payload checksum bytes to the received Ethernet packets.

Table 4-41 Receive Checksum Offload Engine Functions for Different Packet Types

Packet Type	25GMAC IP Header Checksum Checking	25GMAC TCP/UDP/ICMP Checksum Checking
Non-IPv4 or IPv6	No	No
IPv4 packet is greater than 1,522 bytes (2,000 bytes when IEEE 802.3ad Support for 2K Packets is enabled in the MAC)	Yes	Yes
IPv6 packet is greater than 1,522 bytes (2,000 bytes when IEEE 802.3ad Support for 2K Packets is enabled in the MAC)	Not applicable	Yes
IPv4 with TCP, UDP, or ICMP	Yes	Yes
IPv4 header's protocol field contains a protocol other than TCP, UDP, or ICMP	Yes	No
IPv4 packet has IP options (IP header is longer than 20 bytes)	Yes	Yes
Packet is an IPv4 fragment	Yes	No
Authentication	Not applicable	Yes
IPv6 packet with the following next header fields in main or extension headers: ■ Hop-by-hop options (in IPv6 main header) ■ Hop-by-hop options (in IPv6 extension header) ■ Destinations options ■ Routing (with segment left 0) ■ Routing (with segment left > 0) ■ TCP, UDP, or ICMP ■ Any other next header field in main or extension headers	■ Not applicable ■ Not applicable ■ Not applicable ■ Not applicable ■ Not applicable ■ Not applicable ■ Not applicable	■ Yes ■ No ■ Yes ■ Yes ■ No ■ Yes ■ No
IPv4 packet has TCP header with Options fields	Yes	Yes

Table 4-41 Receive Checksum Offload Engine Functions for Different Packet Types (continued)

Packet Type	25GMAC IP Header Checksum Checking	25GMAC TCP/UDP/ICMP Checksum Checking
IPv4 Tunnels: ■ IPv4 packet in an IPv4 tunnel ■ IPv6 packet in an IPv4 tunnel	■ Yes (Both IPv4 tunnel and packet header) ■ Yes (IPv4 tunnel header)	■ See "Checksum Offload for Tunneled packets" on page 265 ■ See "Checksum Offload for Tunneled packets" on page 265
IPv6 Tunnels: ■ IPv4 packet in an IPv6 tunnel ■ IPv6 packet in an IPv6 tunnel	■ Yes (IPv4 packet header) ■ Not applicable	■ See "Checksum Offload for Tunneled packets" on page 265 ■ See "Checksum Offload for Tunneled packets" on page 265
IPv4 packet has 802.3ac tag (with C-VLAN Tag or S-VLAN Tag when enabled).	Yes	Yes
IPv6 packet has 802.3ac tag (with C-VLAN Tag or S-VLAN Tag when enabled).	Not applicable	Yes
IPv4 frames with security features (such as encapsulated security payload)	Yes	No
IPv6 frames with security features (such as encapsulated security payload)	Not applicable	No

Note Table 4-41 applies for header-payload split, Receive Side Scaling function, and L3/L4 filtering as well. However IPv6 packets with routing extension headers of type segment left > 0, are processed for split header, Receive Side Scaling function, and L3/L4 filtering.

4.12.2.1 Checksum Offload for Tunneled packets

The 25GMAC can be programmed to support tunneled packets by setting VNE bit in MAC_Tx_Configuration register and the tunneling mode (VXLAN or NVGRE) is decided by the setting of VNM bit in the same register. In this mode, the RxCOE when enabled, performs the checksum verification for received tunneled packets:

Checksum offload for Outer packet:

For VxLAN Tunneled packets:

- If outer L3 type is IPv4, header checksum is verified and Error is flagged in case of failure.
- If Outer L3 type is IPv4 or IPv6 and received UDP checksum is non-zero, the received UDP checksum is verified. If the checksum is incorrect, it is flagged as payload checksum error and the inner header is ignored for further processing.
- If Outer L3 type is IPv4 and received UDP checksum is all zeros, the received UDP checksum is considered to be valid and the inner header of the tunneled packet is verified for checksum errors.
- If Outer L3 type is IPv6 and received UDP checksum is all zeros, verification of UDP checksum is based on the control bit VUCC in the MAC_Packet_Filter register. When VUCC = 0, such packets are considered as invalid tunneled packets and marked with Outer Payload Checksum error. When VUCC=1, the 25GMAC considers it as valid and continues to process the inner headers.
- For NVGRE tunneled packets, if outer L3 type is IPv4, header checksum is verified and Error is flagged in case of failure.

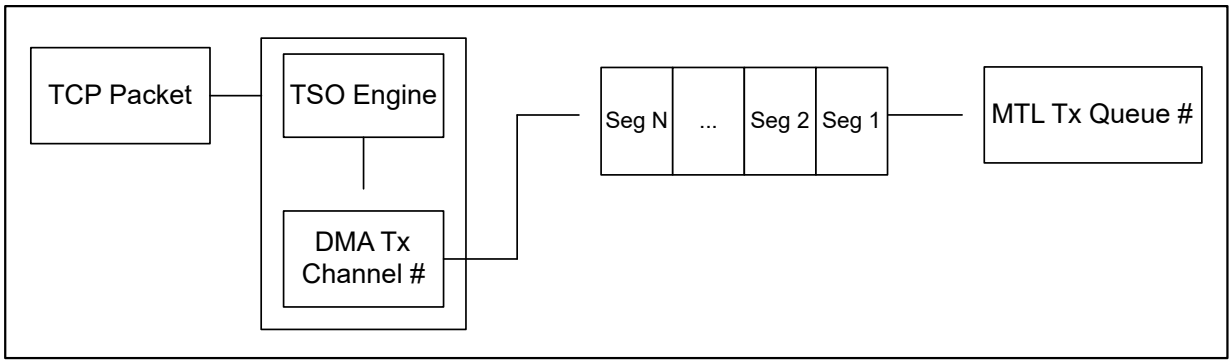
Checksum Offload for Inner Packets:

- For tunneled packets, the COE checks and detects the IPv4 header checksum errors and the TCP, UDP, or ICMP checksum errors encapsulated in IPv4 or IPv6 datagrams payload of received packets as shown in Table 4-41.

4.13 TCP Segmentation Offload

The 25GMAC supports the TCP segmentation offload (TSO) feature in which the DMA splits a large TCP packet into multiple smaller packets and passes these packets to the MTL as shown in Figure 4-11.

Figure 4-11 TCP Segmentation Offload Overview



4.13.1 Description of the TSO Feature

To enable this feature select the Enable TCP Segmentation Offloading for TCP/IP Packets option and program the TSE bit of the corresponding VDMA_CH(#i)_Tx_Control register.

To enable segmentation for a packet, the application must set the TSE bit of TDES3 of the first normal descriptor.

Note The TCP Segmentation Offload (TSO) feature is not supported when TxPBL programmed in the VDMA_CH(#i)_Tx_Control register is less than 4 and PBLx8 is not set.

The application must program the length of the TCP packet payload in TDES3[17:0] and the length of TCP header in TDES3[22:19]. The maximum length of TCP packet payload that can be segmented is 256 KB.

The header of the packet, including the Ethernet header, L3 header and L4 header must be provided in Buffer1 of the first normal descriptor of the TSO packet. Only buffer 1 of the first normal descriptor of a packet enabled for TSO is taken as the buffer containing the header. The TCP payload can begin from buffer 2 of the first normal descriptor and continue to buffer1 and buffer 2 of the second normal descriptor and subsequent descriptors. The maximum header length supported for TSO feature is 1023 bytes.

The TCP payload may span across multiple buffers and multiple descriptors. The size of buffers containing the TCP payload must add up to be equal to the TCP payload length provided in TDES3[17:0] of the first normal descriptor.

The MAC always calculates and appends CRC and inserts Padding (if required) for all packets segmented by the DMA. If the TSE bit of TDES3 is enabled, the CRC PAD Control (CPC) field of TDES3 is reserved. To determine the size of a TCP packet after segmentation, the DMA uses the Maximum Segment Size (MSS) provided by the application through context descriptor. The DMA segments only those packets which have payload size greater than MSS.

For packets whose TCP payload length is less than or equal to the MSS value, the 25GMAC recalculates the IP header checksum and TCP checksum when the TSE bit in the descriptor and the TSE bit in the VDMA_CH(#i)_Tx_Control register are set.

The application must provide the MSS by either programming the MSS value in the VDMA_CH(#i)_Control register or by providing a context descriptor. The DMA uses the last programmed value of MSS or the last MSS value provided through context (whichever is provided later).

The header length plus the MSS size (which is equal to the size of each TCP segment) must not exceed 16383 bytes, otherwise, the MAC transmitter truncates the packet after 16383 bytes causing a CRC error.

The header length plus MSS size plus programmed PBL value in the VDMA_CH(#i)_Tx_Control register must be lesser than the Tx Queue size programmed in TQS field of MTL_TxQ(#i)_Operation_mode register.

The MSS+header must be less than or equal to half the programmed Tx Queue size. The value programmed in the MSS field must be more than the configured Datawidth in bytes. It is recommended to use a MSS value of 64 bytes or more.

The DMA also supports segmentation of VLAN Tagged TCP/IP frames, so if the TCP packet has a VLAN Tag, the same tag is used for all the segments irrespective of the VLAN tag type provided (C-VLAN or S-VLAN). The VLAN Tag insert/replace control bits are used for all segments. If the Double VLAN Feature is selected, the DMA passes both the tags for all segments irrespective of the VLAN tag types provided (C-VLAN or S-VLAN). The VLAN Tag Insert/Replace control bits for both tags is applicable for all segments.

If Double VLAN feature is not selected, the application must not set the TSE bit in TDES3 for a TCP/IP packet with two tags. The DMA behavior in this scenario is unpredictable. If the TSE bit is set in TDES3 for the packet and TCP header length provided is less than 5 (meaning an invalid TCP header because it is less than 20 bytes), the DMA does not perform segmentation. Instead, the DMA transmits the entire packet as a single packet. In this scenario, the CRC pad control bits are forced by DMA to 2'b00 (MAC does CRC and padding), and checksum insertion control bits are forced to 2'b11 (hardware does the checksum calculation for both header and payload).

4.13.2 Tunnel Packet Handling

In the case of Tunneled packets (as indicated by VNP = 3’b011 in the Transmit Descriptor) then the Header of each segmented packet includes Outer L2 + Outer L3 + Outer UDP (in case of VXLAN mode) + Tunnel Header + Inner L2 + Inner L3 + Inner TCP. The segmentation is done on the Inner TCP Payload. The TSO engine modifies the required fields in both Outer IP header (payload length) as well as the Inner IP + TCP header during segmentation.

4.13.3 DMA Operation with TSO Feature

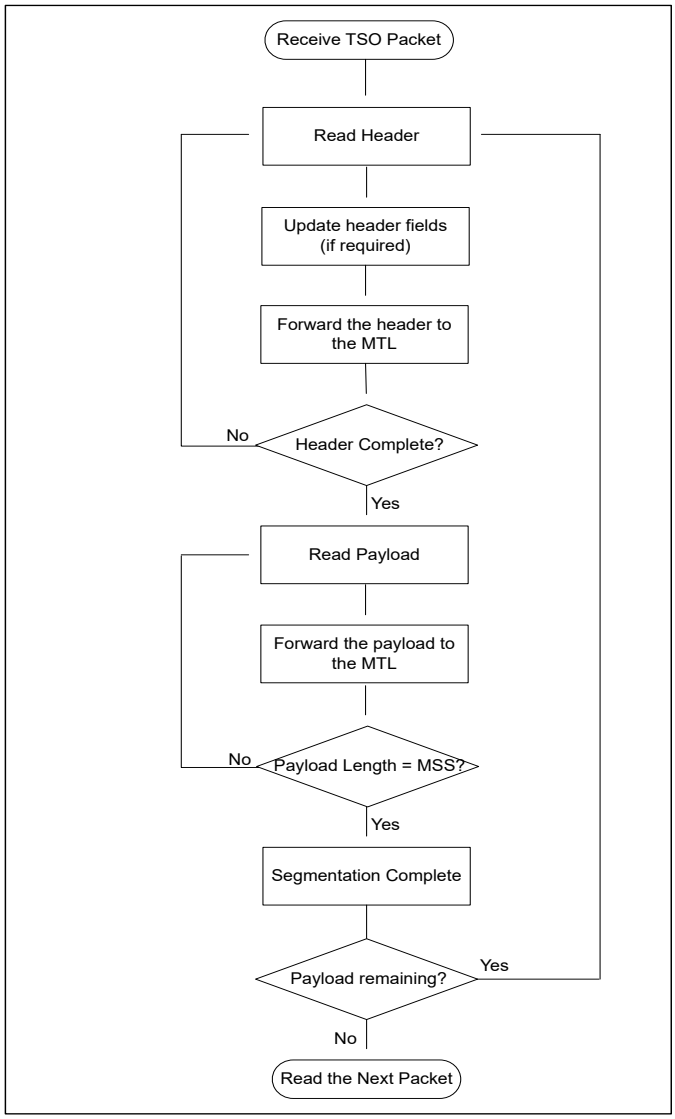
Figure 4-12 shows the TSO flow.

When the TSO feature is enabled, the TxDMA operation is as follows:

1. The application sets up the Transmit descriptor (TDES0-TDES3) and sets the Own bit (TDES3[31]) after setting up the corresponding data buffers with Ethernet Packet data.
2. The application increases the offset value of the Descriptor Tail pointer of the DMA Tx channel.

3. While in the Run state, the DMA fetches the next available descriptor and does one of the following:
   - If the descriptor is a context descriptor and the context is between the last descriptor of a packet and the first descriptor of the subsequent packet, the DMA stores the context values.
   - If the descriptor is a context descriptor and the context is between the first and last descriptors of a packet, the DMA closes the context descriptor indicating a Context Descriptor Error (TDES3[23]) and fetches the next descriptor.
   - If the descriptor is a normal descriptor, the DMA checks for the TSE bit in the first descriptor (that contains the start of packet). If TSE bit is not set, the DMA continues with the default mode of operation or OSF operation (if enabled).

Figure 4-12 TCP Segmentation Offload Flow



4. The DMA calculates the number of segments from the TCP payload length(TDES3[17:0]) and the MSS value.

5. The DMA goes through channel arbitration to fetch the data buffers. The DMA fetches the header and payload separately.
6. For the first segment, the DMA fetches the header from the system memory and stores it in the TSO memory (if present and when the length of header is not greater than the TSO memory size). Set TMWD (TDESC2[30]) bit to disable the external memory writing even if previous conditions are true.

   If the current segmented packet is not the first segment, the DMA fetches the header from the local TSO memory if available. Otherwise, it fetches the header buffer in system memory again, as done for the first segment. For NDMA, in such cases (header not available is TSO memory), the DMA does not close the first descriptor containing the header buffer until the header for last segment is fetched. TSO first descriptor (with header buffer) is always closed after last segment.
7. The required fields in the header bytes are modified/updated as per the segmentation requirements and written into the corresponding MTL Tx queue.
8. The DMA then takes the payload buffer pointer, fetches the MSS number of payload bytes from the system memory and directly pushes it into the MTL TxQ. In case the buffers in the descriptor do not have enough data for the MSS payload (except for the last segment), the DMA closes this descriptor, if this is not the first descriptor.
9. The DMA jumps to step 3 and repeats the process until the last segment is written into the TxQ.
10. The DMA closes the last descriptor and also the first descriptor (containing the header buffer when it is not stored in TSO memory. TSO first descriptor (with header buffer) is always closed after the last segment and then moves on to the next packet transfer.

The DMA repeats all these steps if more descriptors are available. When no descriptor is available, the DMA enters the suspend state.

4.13.4 TCP/IP Header Fields

While segmenting a TCP packet, the DMA automatically updates the TCP/IP header fields. Table 4-42 describes how the TCP and IP headers are updated.

Table 4-42 TSO: TCP and IP Header Fields

Packet Sequence	TCP Header	IP Header
First packet	1. The sequence number is not updated. The value provided in the header is used. 2. If set, the FIN and PSH flags are cleared. 3. The TCP checksum is calculated.	■ IPv4 Header ❑ Total Length = MSS + TCP Header Length + IP Header Length ❑ Identification field is not modified. It is sent as per the header provided by the software. ■ IPv4 Header Checksum is recalculated. ■ IPv6 Header Payload Length = MSS + TCP Header Length + IP Extension Header Length

Table 4-42 TSO: TCP and IP Header Fields (continued)

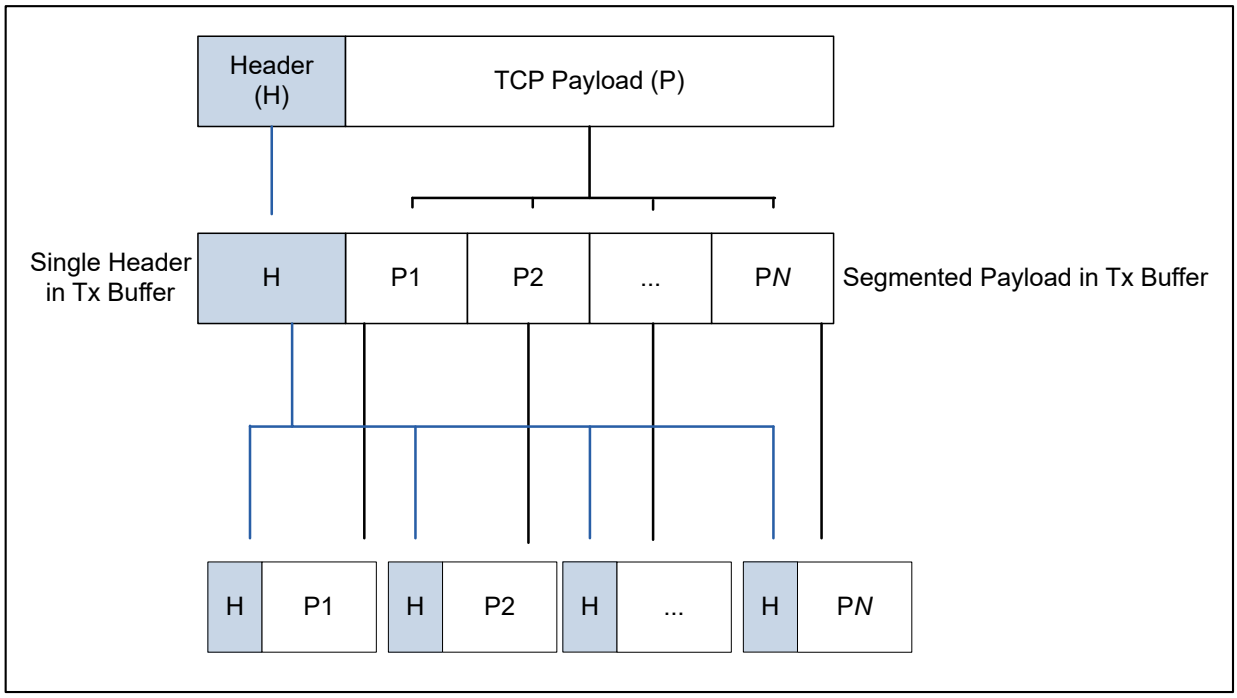
Packet Sequence	TCP Header	IP Header
Subsequent packets	1. The sequence number is updated. The MSS value is added to the sequence number value of previous segment. 2. If set, the FIN and PSH flags are cleared. 3. The TCP checksum is calculated.	■ IPv4 Header ❑ Total Length = MSS + TCP Header Length + IP Header Length ❑ Identification field = Previous Identification Field + 1 ❑ IPv4 Header Checksum is recalculated ■ IPv6 Header Payload Length = MSS + TCP Header Length + IP Extension Header Length
Last packet	1. The sequence number is updated. The MSS Value is added to the previous sequence number value. 2. If FIN and PSH flags were set in original header, these flags are set. 3. The TCP checksum is calculated.	■ IPv4 Header ❑ Total Length = Remaining Payload + TCP Header Length + IP Header Length ❑ Identification Field = Previous Identification Field + 1 ❑ IPv4 header Checksum is recalculated ■ IPv6 Header Payload Length = Remaining Payload Length + TCP Header Length + IP Extension Header Length

4.13.5 Header and Payload Fields of Segmented Packets

After segmentation, the split packets use the same header as the parent TCP packet for header fields other than the ones described in Table 4-42. Figure 4-13 shows how same header is used for the header fields of segmented packets.

The application must create the header in Buffer 1 of the first descriptor of the packet to be segmented and provide the header length in TDES2 of the first descriptor (FD = 1). When the FD bit is set, the DMA reads the header from the header buffer to which the TDES0 is pointing. Buffer 2 of the first descriptor can be used for payload and TDES0 and TDES1 of subsequent descriptors. For subsequent descriptors (FD = 0), the address to which the TDES0 and TDES1 are pointing is treated as payload buffer address of the same packet.

Figure 4-13 Header and Payload Fields of Segmented Packets



4.13.6 Context Descriptor Sequence

You can use the context descriptor to provide the MSS value for segmentation. The application must provide the context descriptor before the normal descriptor to be used for the corresponding TCP packet. If the application wants to provide a new MSS, it must create the context descriptor in the descriptor list before the first normal descriptor of the packet to be segmented with the new MSS value. The MSS value in the context descriptor is valid only if the TCMSSV bit of TDES3 in context descriptor is set and the OSTC bit is reset.

When the application provides a context descriptor with a valid MSS value, the DMA internally stores the MSS value and uses this MSS value for all subsequent packets that have the TSO Enabled through the TSE bit of TDES3 normal descriptor.

If the application places a context descriptor in the middle of a packet (between the first and last descriptors of a packet), the DMA does the following:

1. The DMA ignores the context and closes the descriptor.
2. The DMA indicates the error in descriptor status
3. The DMA generates an interrupt if the CDEE bit is set in the Interrupt_Enable register of the corresponding DMA channel.

The application can read the interrupt status through CDE bit of the Status register corresponding to a DMA channel.

4.13.7 External TSO Memory Interface

The TSO Memory Interface (TMI) is used to read and write the header into an external single port RAM (Single-Port RAM) interface. You can enable this interface by selecting the Enable Separate Memory for TCP/IP Headers option.

When the TSO memory is not enabled, the DMA reads the header buffer from the system memory through the AXI interface for each segmented packet. To reduce the buffer reads, you can use the TSO memory in which the DMA can store the header data. The DMA reads the header buffer from the system memory for the first segment and writes it into the TSO memory. For the subsequent segments, the DMA reads the header from the TSO memory. It does not initiate any transactions on the AXI interface.

When you enable the TMI, the read and write data width is same as the configured data width and the memory is synchronous to the application clock (aclk_i). The size of the TSO memory allocated for each TSO channel varies from 64 bytes to a maximum of 1 KB. The total size of the TMI is equal to the number of TSO channels multiplied by the size allocated for each TSO channel. You can select the memory size by using the Per-Channel TSO Memory Size in Bytes option. The memory address width is given by the following:

```
M = log2 (DWCXG_TSO_MEM_SIZE) /DATAWIDTH
```

4.13.7.1 TSO Memory Synchronous Timing

shows the synchronous timing for the TSO memory.

Figure 4-14 TSO Memory Synchronous Timing

4.14 Receive Side Scaling Function

The MAC receiver parses the received packet with the Receive Checksum Engine function and extracts the following four tuples of the received packet:

- Layer 3 IPv4/IPv6 Source Address, Destination Address fields
- Layer 4 fields like TCP/UDP/ICMP Source Port and Destination Ports of the packet

The 25GMAC controller extracts the Layer 4 tuple fields only when it is a supported payload. For a description of the supported payloads, see "Receive Checksum Offload Engine" on page 263.

These fields and the corresponding packet type are then given to the RSS engine, which generates the 32-bit Hash (Toeplitz hash function). This algorithm takes in the key programmed in the `MAC_RSS_Address` register and the 2-Tuple or 4-Tuple fields that are enabled by the corresponding bits in the `MAC_RSS_Control` register, considering mask tuple mask bits in case of external RSS lookup table. The masking indirect registers are accessible through MAC_RSS_* indirect access register. For more detail, see the *Register Descriptions* chapter.

The 32-bit Hash value generated by the RSS engine is attached along with the packet as status.

The LS bits of the 32-bit Hash value are also used as an address to the RSS Lookup Table to read the DMA channel number to which this packet is to be assigned. The packet is then routed to the corresponding DMA channel read from this table. In case the lookup table indicates the packet is to be dropped, the MAC receiver cuts the rest of the frame and gives an error status.

If the RSS function is disabled (Bit[0] of the `RSS_Control` register) or if the received packet does not have any Tuple (non-IP packet), the engine bypasses the RSS hash calculation and also lookup to the re-direction table. The received packet is routed to the DMA channel number decided through the Ethernet DA-based lookup engine.

The software must set-up the Redirection table and program the hash key during initialization before enabling the MAC Receiver. The re-direction table can be altered during active receive operation, but the DMA channel routing takes proper effect on the next receiving packet after some synchronization delays. The change does not take effect on the packets which are already received and present in the MTL Queues.

If the HASH key or lookup table data is modified during active receiver operation, inconsistencies may occur in the RSS hash calculation for packets which use the partial/incorrect Hash key.

RSS Operation:

1. Received packet type is checked, selected tuple fields (with mask bits in case of external RSS lookup) are directed to Configurable Hash Function block (CHF).
2. A few clock cycles later, the CHF generates the hash value.
3. The LS bits of the calculated hash value (depends on the Redirection Table Size) is used as the address to get the DMA channel number to which this packet is intended. The lookup table can also indicate whether the packet is to be dropped by the MAC receiver. For the selected lookup entry,
   - If DMCHB = 0 && DMCHLE = 0: DMCH field holds binary number of DMA receive channel.
   - If DMCHLE = 1: Selects the receive channel list number for packet duplication.

   The MTL RxQ Controller redirects this packet to the DMA channel along with the calculated 32-bit RSS hash value.

4. The DMA adds the hash value indication to the Receive Descriptor (RDES1).
5. Software uses the hash value in the Receive Descriptor to select which queue the packet must be redirected in the stack.

4.14.1 RSS Function when VXLAN/NVGRE Mode is Enabled

The RSS function works differently in the case when VXLAN/NVGRE mode is enabled by setting VNM bit in `MAC_Tx_Configuration` register. In such cases, the tunneled packets are expected to be received and the Hash based RSS as explained is not useful for the tunneled packet. Hence in this mode, the RSS Hash function operates in a different way to decide the DMA channel to which the received packet must be routed.

Instead of the L3/L4 Tuple fields, VNI/TNI of the tunnel header, FlowID (in case of NVGRE), Ethernet DA of the Inner Header and VLAN Tag fields of the header are (considering mask bits in case of External RSS lookup) input to the Hash Engine. The masking indirect registers are accessible through MAC_RSS_* indirect access register. For more details, see the *Register Descriptions* chapter.

The LS bits of the calculated hash is used as the address of the Redirection lookup table. The target DMA channel information or whether the packet is to be dropped is decided based on the lookup result and the packet is forwarded accordingly along with the 32-bit hash value.

The Routing of the Tunneled Packets in the 25GMAC does not consider inner packet L3-L4 fields. In case a non-tunneled packet is received in this mode, the RSS function is bypassed.

For VxLAN packets, the inputs of the RSS hash function are given in the following order:

- VNID[23:0] followed by L2 DA[47:0] of the Inner packet and VLAN tag[15:0] (if present) of Inner packet.

For NVGRE packets, the inputs of the RSS hash function are given in the following order:

- VSID[23:0] followed by flowID[7:0], L2 DA[47:0] of Inner packet and VLAN tag[15:0] (if present) of Inner packet.

Note

For VxLAN/NVGRE type tunneled packets, if the DMA channel value obtained from the RSS function is greater than the number of Receive DMA channels present in the configuration, the corresponding packet is routed to default receive DMA channel (Receive Channel 0). Destination MAC address based DMA channel number is not used.

The DA/SA and VLAN filtering have precedence over the RSS filtering. Therefore, packets that fail either DA/SA or VLAN filter are dropped, irrespective of the RSS filter results.

4.14.2 External RSS Lookup-Based Filtering, Routing, and Packet Duplication

Based on the RSS lookup table size that you specified in the configuration using the DWCXG_RSS_TABLE_SIZE parameter, 25GMAC uses either internal or external RSS lookup. If the selected size is greater than 256, 25GMAC uses the external RSS Lookup-based filtering and routing.

This feature supports external Receive-Side-Scaling (RSS) Lookup-based packet filtering, routing, and duplication. 25GMAC uses the external RXF memory for lookup, which is accessible through MAC_RSS_* indirect access register.

You can configure packet filtering, receive-channel routing, and duplication information in the RSS lookup entry. For more details about the RSS_Lookup_Data register, see the *Register Descriptions* chapter.

When both Layer 3 and Layer 4 perfect match filtering and external RSS lookup-based hash filtering are enabled and performed on the received packet, the packet is forwarded when any of the following conditions are met.

- Layer 3-4 filter result is a pass.
- Received packet is compatible for RSS hashing (required Tuples are enabled) and the RSS hash lookup result indicates that the packet can be forwarded.

The received packet is dropped only when both the enabled filters indicate that the packet is to be dropped.

4.15 OoS Function

The Out of Sequence (OoS) packet detection function is activated when the RSS function and Split Header Payload features are enabled and OOSE bit in MAC_RSS_Control register is set. The purpose of this function is to indicate to the software whether the current packet being transferred by the RxDMA to the software is of the same TCP connection and is in sequence with the previous packet transferred by the same RxDMA. This information along with the RSS Hash key and “Split header and payload” transfers by the RxDMA helps in implementing the Receive Packet coalescing (RPC) function. The software driver is in a position to combine multiple TCP packets into a single packet and push to the upper stack.

The OoS module has two parts: the OoS Parser and RSS hash/sequence number comparator.

The OOS support is not applicable or evoked when the MAC is programmed to support VXLAN/NVGRE tunneled packets. However, the same RSS algorithm is used just for DMA Channel selection for tunneled packets.

4.15.1 OoS Parser

The first part resides in the MAC RX block where the TCP/IP parser function is active. This function identifies the TCP/IP 4-Tuple fields and also extracts the sequence number field. It also calculates the expected sequence number of the next packet in the same connection and forwards both the sequence number values to the RxDMA.

In addition, the OoS parser also checks the other fields of the TCP packet to determine whether this packet is eligible for coalescing or not. If it is not eligible, then it indicates the same to the RxDMA which puts this status in the Rx Descriptor (Bit NCP of RDES3). This status bit informs the software driver that this packet must be forwarded to the stack immediately and it must not wait for more packets in that connection for coalescing.

The eligibility for coalescing depends on the following.

Fragmented frame (IP total packet length must be received packet length)

- Non-TCP packet
- IP options are present
- IP ECN CE is set
- TCP segment has no data
- CWR (Congestion Window Reduced) flag is set
- ECE (ECN Echo) flag is set
- SYN flag is set
- FIN flag is set
- PSH flag is set when PSHE (Bit[9]) of MAC_RSS_Control register is set
- URG flag is set
- RST flag is set
- ACK flag is not set
- Non-TCP Timestamp option when TCP Options fields are present

4.15.2 Hash/Sequence Number Comparator

The second part of the OoS function resides in each RxDMA. This stores the RSS hash of the previous packet and the expected sequence number of the next packet of the same connection. When the current packet status is received (RSS Hash, current sequence number), it compares the status against the stored values. If the sequence number matches the expected one and the RSS values are the same as that of the previous packet, the current packet is considered to be in sequence and of the same connection. Hence, the RxDMA sets the ISP bit of the RDES3 status. In all other conditions, the ISP is set to "0". This function is summarized as given in Table 4-43.

Table 4-43 Summary of the Hash/Sequence Number Comparator Function

RSS HASH	SEQ_NUM	NCP Condition of Current Packet	NCP Status of Previous Packet	Descriptor Status Bits	
				ISP	NCP
Match	Match	0	0	1	0
Match	Match	0	1	0	0
Match	Match	1	0	1	1
Match	Match	1	1	0	1
No match	x	0	x	0	0
No match	x	1	x	0	1
x	No match	0	x	0	0
x	No match	1	x	0	1

The RxDMA transfers the "Header" bytes (Start of packet to end-of TCP/UDP header) into the header buffer and the rest of the payload into the Payload buffers of the Rx Descriptor. When the Rx Descriptor is closed with the EOP, the DMA also gives the following status:

- RSS Hash (RDES1)
- L3-L4 packet Type (RDES3)
- In-Sequence (ISP) and No-coalescing (NCP) status (RDES3)

The software can use the status information to determine whether the packet must be coagulated with the previous packet.

4.15.3 OoS Interrupt

When the OOS function is enabled, the Rx DMA determines if any of the TCP/IP packets transferred to host memory is in-sequence with respect to the previous packet and sets the ISP bit in Receive Descriptor write-back status. This function is updated so that an interrupt is generated when the current TCP/IP packet is an NCP (Non-Coalescing) packet or is out-of-sequence with the previous one.

The events mentioned earlier sets the interrupt status bit OSI (Bit[5]) of `VDMA_CH(#i)_Status` register. This bit sets the RI bit when corresponding OSIE bit of `VDMA_CH(#i)_Interrupt_Enable` register is set, which generates interrupt on the sbd_intr_o and sbd_perch_rx_intr_o output pins when RIE bit of `VDMA_CH(#i)_Interrupt_Enable` is programmed to 1.

This function is summarized as given in Table 4-44.

Table 4-44 Summary of the Out of Sequence Function

NCP Condition of Current packet	ISP Status of Previous packet	ISP Condition of current packet	Out of Sequence Status (OSI)
1	X	X	1
0	X	1	0
0	1	0	1
0	0	0	0

4.16 Management Counters

The MAC Management Counters extend the register address space of CSR. The MMC maintains a set of registers for statistics on the received and transmitted frames. These include a control register for controlling the behavior of the MMC counters, two 32-bit registers containing interrupts generated (receive and transmit), and two 32-bit registers containing masks for the Receive and Transmit Interrupt registers. The application can access these registers through the MAC Control Interface (MCI). Though each register is 32-bit wide, they can also be 64-bit wide. Write data is qualified with corresponding mci_be_i signals. Therefore, non-32 bit accesses are allowed only when the address is word-aligned.

The organization of these registers is shown in the *Register Descriptions* chapter. MMC registers are accessed using transactions in the same way that CSR addresses are accessed. This section describes the statistics counters and lists the address for each. Counters, which are 64-bits wide, are divided into two 32-bit registers for access through the MCI. If the MMC module is configured for only 32-bit counters, the registers for the upper 32 bits are reserved with the exception of following four octet counters: txoctetcount_gb, txoctetcount_g, rxoctetcount_gb, and rxoctetcount_g. These octet counters are always 64-bit irrespective of whether the MMC module is configured for 32-bit or 64-bit counters mode.

The mac management counters gathers statistics on encapsulated IPv4, IPv6, TCP, UDP, or ICMP payloads in the received Ethernet packets. This gathering is only enabled when you select the Receive Checksum Offload Engine in coreConsultant

The receive MMC counters are updated only for frames passed by the Destination Address Filter (AFM) block. Statistics of frames that are dropped by the AFM are not updated unless they are runt frames of less than 6 bytes (that is, DA bytes that are not fully received).

4.16.1 Address Assignments

The following explanations pertain to terminology used in Table 4-45 on page 281.

- Transmitted frames are considered good frames when none of the following errors is observed:
  - Frame Underflow error
  - Local/remote fault detected.
  - Jabber time-out
  - No carrier or loss of carrier
  - Late collision
  - Excessive Deferral
  - Excessive Collision
- Received frames are considered good when none of the following errors is observed:
  - CRC error
  - Length error
  - Runt frames (with or without CRC errors)
  - Out-of-range frames
  - Watchdog time-out

- The maximum transmit frame size depends on the frame type, as follows:
  - Untagged frame maxsize = 1,518
  - VLAN Frame maxsize = 1,522
  - Jumbo Frame maxsize = 9,018
  - JumboVLAN Frame maxsize = 9,022
- The maximum receive frame size depends on the frame type and control bits (JE, S2KP, GPSLCE and EDVLP) as follows:

Table 4-45 Maximum Frame Size Dependency

JE	S2KP	GPSLCE	EDVLP	Untagged framemaxsize in bytes	Single VLAN framemaxsize in bytes	Double VLAN framemaxsize in bytes
1	X	X	1	9018	9022	9026
0	1	X	X	2000	2000	2000
0	0	1	1	GPSL	GPSL+4	GPSL+8
0	0	0	1	1518	1522	1526
1	X	X	0	9018	9022	9022
0	0	1	0	GPSL	GPSL+4	GPSL+4
0	0	0	0	1518	1522	1522

Preambles of frames are not counted in the octet counters.

4.16.2 Rx Channel Mapping with Rx Frame

This section shows the Rx Channel Mapping with Rx Frame/Priority/IPC MMC category list.

Table 4-46 RxMMC Category list

RxMMC Counter Label	RxMMC Counter Parameter	RxMMC counter mapped to Rx Channel?
Rx Frame Counters		
MMC Rx Frame Counter for Good and Bad Frames	DWCXG_MMC_RXFRMGB_CNT_EN	Yes
MMC Rx Frame size Counter for all Good Frames	DWCXG_MMC_RXOCTG_CNT_EN	Yes

Table 4-46 RxMMC Category list (continued)

RxMMC Counter Label	RxMMC Counter Parameter	RxMMC counter mapped to Rx Channel?
MMC Rx Frame size Counter for Good and Bad Frames	DWCXG_MMC_RXOCTGB_CNT_EN	Yes
MMC Rx Frame Counter for Good Broadcast Frames	DWCXG_MMC_RXBCASTG_CNT_EN	Yes
MMC Rx Frame Counter for Good Multicast Frames	DWCXG_MMC_RXMCASTG_CNT_EN	Yes
MMC Rx Frame Counter for CRC Errors	DWCXG_MMC_RXCRCERR_CNT_EN	Yes
MMC Rx Frame Counter for Runt Frames	DWCXG_MMC_RXRUNTERR_CNT_EN	Yes
MMC Rx Frame Counter for Jabber Error Frames	DWCXG_MMC_RXJABERR_CNT_EN	Yes
MMC Rx Frame Counter for Undersize Frames	DWCXG_MMC_RXUNDERSZG_CNT_EN	Yes
MMC Rx Frame Counter for Giant Frames	DWCXG_MMC_RXOVERSZG_CNT_EN	Yes
MMC Rx Frame Counter for Various Frame Sizes	DWCXG_MMC_RXADDR_RNG_CNT_EN	Yes
MMC Rx Frame Counter for Unicast Frames	DWCXG_MMC_RXUCASTG_CNT_EN	Yes
MMC Rx Frame Counter for Length Errors	DWCXG_MMC_RXLENERR_CNT_EN	Yes
MMC Rx Frame Counter for Out of Range Errors	DWCXG_MMC_RXOUTOFRNG_TYP_CNT_EN	Yes
MMC Rx Frame Counter for Pause Frames	DWCXG_MMC_RXPAUSEFRM_CNT_EN	Yes
MMC Rx Frame Counter for Overflow Errors	DWCXG_MMC_RXFIFOOVFL_CNT_EN	No
MMC Rx Frame Counter for VLAN Frames	DWCXG_MMC_RXVLANFRGB_CNT_EN	Yes
MMC Rx Frame Counter for Watchdog Timeout Errors	DWCXG_MMC_RXWDGERR_CNT_EN	Yes
MMC Rx Frame Counter for Discarded frames due to filter fail (MAC)	DWCXG_MMC_RXFRMDISGB_CNT_EN	No
Enable MMC Rx Frame Counter for Packets with Alignment Error	DWCXG_MMC_RXALGNERR_CNT_EN	No

Table 4-46 RxMMC Category list (continued)

RxMMC Counter Label	RxMMC Counter Parameter	RxMMC counter mapped to Rx Channel?
MMC Rx Frame size Counter for Discarded frames due to filter fail (MAC)	DWCXG_MMC_RXOCTDISGB_CNT_EN	No
MMC LPI Rx Micro Second Timer	DWCXG_MMC_RXLPI_USEC_EN	No
MMC LPI Rx Entry Transition Counter	DWCXG_MMC_RXLPI_TRANS_CNTR_EN	No
Rx Priority Counters		
Enable Per Priority Rx Frame Counter	DWCXG_MMC_RXFRMGB_PR_CNT_EN	Yes
Enable Per Priority Rx Bad Frame Counter	DWCXG_MMC_RXFRMB_PR_CNT_EN	Yes
Enable Per Priority Rx PFC Frame Counter	DWCXG_MMC_RXFRM_PFC_GB_CNT_EN	Yes
Enable Per Priority Rx Frame size Counter	DWCXG_MMC_RXOCTGB_PR_CNT_EN	Yes
Enable Per Priority Rx Discard Counter	DWCXG_MMC_RXFRMDISGB_PR_CNT_EN	No
Rx IPC Counters		
Enable MMC Counter for Good IPv4 Packets	DWCXG_MMC_RX_IPV4_GD_PKTS_CNT_EN	Yes
Enable MMC Counter for IPv4 Packets with Header Error	DWCXG_MMC_RX_IPV4_HDRERR_PKTS_CNT_EN	Yes
Enable MMC Counter for IPv4 Packets without TCP, UDP, and ICMP	DWCXG_MMC_RX_IPV4_NOPAY_PKTS_CNT_EN	Yes
Enable MMC Counter for Fragmented IPv4 Packets	DWCXG_MMC_RX_IPV4_FRAG_PKTS_CNT_EN	Yes
Enable MMC Counter for IPv4 UDP Packets with Checksum Disabled	DWCXG_MMC_RX_IPV4_UDSBL_PKTS_CNT_EN	Yes
Enable MMC Counter for Good IPv6 Packets	DWCXG_MMC_RX_IPV6_GD_PKTS_CNT_EN	Yes
Enable MMC Counter for IPv6 Packets with Header Error	DWCXG_MMC_RX_IPV6_HDRERR_PKTS_CNT_EN	Yes
Enable MMC Counter for IPv6 Packets without TCP, UDP, and ICMP	DWCXG_MMC_RX_IPV6_NOPAY_PKTS_CNT_EN	Yes
Enable MMC Counter for UDP Packets without Error	DWCXG_MMC_RX_UDP_GD_PKTS_CNT_EN	Yes

Table 4-46 RxMMC Category list (continued)

RxMMC Counter Label	RxMMC Counter Parameter	RxMMC counter mapped to Rx Channel?
Enable MMC Counter for UDP Packets with Error	DWCXG_MMC_RX_UDP_ERR_PKTS_CNT_EN	Yes
Enable MMC Counter for TCP Packets without Error	DWCXG_MMC_RX_TCP_GD_PKTS_CNT_EN	Yes
Enable MMC Counter for TCP Packets with Error	DWCXG_MMC_RX_TCP_ERR_PKTS_CNT_EN	Yes
Enable MMC Counter for ICMP Packets without Error	DWCXG_MMC_RX_ICMP_GD_PKTS_CNT_EN	Yes
Enable MMC Counter for ICMP Packets with Error	DWCXG_MMC_RX_ICMP_ERR_PKTS_CNT_EN	Yes
Enable MMC Octet Counter for IPv4 Packets without Checksum Error	DWCXG_MMC_RX_IPV4_GD_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for IPv4 Packets with Header Checksum Error	DWCXG_MMC_RX_IPV4_HDRERR_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for IPv4 Packets without UDP, TCP, and ICMP	DWCXG_MMC_RX_IPV4_NOPAY_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for Fragmented IPv4 Packets	DWCXG_MMC_RX_IPV4_FRAG_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for IPv4 UDP Packets with Checksum Disabled	DWCXG_MMC_RX_IPV4_UDSBL_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for IPv6 Packets without Checksum Error	DWCXG_MMC_RX_IPV6_GD_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for IPv6 Packets with Header Error	DWCXG_MMC_RX_IPV6_HDRERR_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for IPv6 Packets without TCP, UDP, and ICMP	DWCXG_MMC_RX_IPV6_NOPAY_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for UDP Packets without Checksum Error	DWCXG_MMC_RX_UDP_GD_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for UDP Packets with Checksum Error	DWCXG_MMC_RX_UDP_ERR_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for TCP Packets without Checksum Error	DWCXG_MMC_RX_TCP_GD_OCTET_CNT_EN	Yes

Table 4-46 RxMMC Category list (continued)

RxMMC Counter Label	RxMMC Counter Parameter	RxMMC counter mapped to Rx Channel?
Enable MMC Octet Counter for TCP Packets with Checksum Error	DWCXG_MMC_RX_TCP_ERR_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for ICMP Packets without Checksum Error	DWCXG_MMC_RX_ICMP_GD_OCTET_CNT_EN	Yes
Enable MMC Octet Counter for ICMP Packets with Checksum Error	DWCXG_MMC_RX_ICMP_ERR_OCTET_CNT_EN	Yes

4.17 ITU-T G999.1 Function Support

The 25GMAC can support a limited feature set of the ITU-T G999.1 specifications related to Ethernet encapsulated packets. When you enable this feature, the 25GMAC supports the following:

- Generation of DFC (Data Flow Control) pause packets in the transmit side
- Parsing and routing of received data units to RxQueues based on the stream ID bits in the TCI (Tag Control Identifier) field of the header bytes.

Note The MAC supports only the Ethernet encapsulated data units.

To enable this feature, program the G9991EN field in the `MAC_Tx_Configuration` register

4.17.1 Transmit Path

When G9991EN bit is set, the 25GMAC can generate pause data units whenever there is a change in the `xgmac_dfc_i` signal. The `xgmac_dfc_i` is a 64-bit signal, where each bit corresponds to a stream ID. The first 64 bits in the DFC field of the transmitted pause unit is updated with the value on the `xgmac_dfc_i` signal.

After the G999.1 mode is enabled (both the hardware and software controlled) PAUSE and PFC flow control are not supported. The rest of the MAC transmitter functions (Insertion or replacement of SA, VLAN tags, PAD, CRC fields, maintaining IPG as per specifications, and so on) remain the same as in non-G999.1 mode.

Note When the xgmac_dfc_i signal changes, a Pause unit is scheduled to be transmitted after the current packet transmission (if any). In case the xgmac_dfc_i values change after the initial packet is scheduled or while the Pause packet is being transmitted, the new value (or part of) may get inserted in the transmitted packet provided those bytes of the DC field is yet to be scheduled for transmission. If the MAC s not able to transmit the new value of the DFC field in the current PAUSE packet, it schedules another pause packet for transmission with the latest DFC value.

4.17.2 Receive Path

In G999.1 mode, 25GMAC considers the received packet with a value of 0x8100 (CVLAN type) after the SA field, as G999.1 data unit. The 2 bytes after the 0x8100 are considered as the TCI field. In this mode, the 25GMAC routes the received G999.1 data unit to one of the receive queue based on the stream ID (sid1 and sid0) in the TCI field. The stream ID to queue mapping is programmed in the PSRQn field of the `MAC_RxQ_Ctrl2` and `MAC_RxQ_Ctrl3` registers. All other packets are considered as untagged (non-G999.1) packets and are forwarded accordingly.

Note In G999.1 mode, received packet with value 0x88A8 (SVLAN type) after the SA field is considered as untagged packet, for the purpose of routing, even if SVLAN processing is enabled.

The receive status word is updated the same way as is in case of non-G999.1 mode, based on the operating mode of the 25GMAC.

The VLAN-based filters and VLAN tag stripping feature must be disabled in this mode. The DA and SA-based filtering can be still used in this mode.

4.18 External Lookup Interface

The 25GMAC supports an interface for External Lookup engine that can be used for filtering and routing the received packets to a specific RxDMA channel. In this interface, the 25GMAC

- For non-tunneled packets, provides the DA and VLAN tags of the received packet as outputs for use by the external lookup engine and receives the DMA channel number and status information.
- For tunneled packets it additionally provides the VNID/TNID, DA of Inner packet and VLAN tags of the Inner packet of the received packet as outputs for use by the external lookup engine and receives the DMA channel number and status information.
- Routes the packet based on the DMA channel number provided by the external lookup interface
- Routes the packet to more than one DMA channel (known as packet duplication) if more than one DMA channel is selected by the external lookup interface, provided the packet is routed to the highest receive queue number (by programming of the controller).

To enable this feature, set the ELEN field of the `MAC_Rx_Configuration` register.

Note

- External Lookup based filtering and routing is not supported for preemptive packets.
- Do not enable the following features through programming, when external lookup is enabled:
  - RSS
  - OOS
  - Split header

4.18.1 Receive DMA Routing and Status

When the external lookup interface feature is selected in the configuration, I/Os are available for providing the DA, outer and inner VLAN tags as outputs, and getting the result of the external lookup as input. These signals are described in “External Lookup Interface Signal” in the *Signal Descriptions* chapter.

When VxLAN feature is selected, additional I/Os are available for providing VNID/TNID, Inner packet's DA and Inner packet's VLAN tags for Tunneled packets. The outputs are driven even if this feature is not enabled (ELEN=0), but the inputs from the Lookup engine is accepted and processed only when the feature is enabled.

The DA field along with its valid signal is provided as soon as the DA field is received. The outer VLAN tag and its valid signal are provided if the frame is a VLAN tagged frame. The inner VLAN tag and its valid signal are provided if the frame is a double tagged frame and double VLAN processing is enabled.

The 25GMAC provides a lookup ready signal (xgmac_elkup_rdy_o), which is asserted when the last available field (such as, inner VLAN tag in case of double tagged packet or when the MAC detects an untagged packet) is provided. The xgmac_elkup_rdy_o signal is de-asserted when one of the following occurs:

- The result is provided by the external lookup interface logic (assertion of xgmac_dch_val_i)
- Packet canceled by the 25GMAC (for example, filter failure)
- Timeout (EOP bytes of current packet is being transferred out or the start of next packet is detected before the result of the current packet is provided by the external lookup engine).

When tunneling support is enabled, (VNE of MAC_Tx_Configuration register) and tunneled packet is received, the xgmac_elkup_rdy_o is not asserted till VNI & Inner Packet L2 Headers are available.

The 25GMAC accepts the lookup result only when the ready signal is asserted. In case the lookup result is not received, the packet is routed to the RxDMA Channel 0.

Note It is possible that the ready signal is not provided for a packet when the packet is canceled internally before the last valid field is provided to the external lookup interface.

The DMA channel number input (`xgmac_dch_i` signal) contains one bit for each DMA channel, for instance, bit 0 for DMA channel 0, and so on. The packet is routed to DMA channels, based on this input. One or more bits can be set simultaneously. For more details, see "Packet Duplication" on page 288.

In most cases, the packet is routed to one DMA channel selected by the external lookup interface by setting only one bit in the DMA channel number input.

Along with the DMA channel number, the external lookup engine can provide additional information (on input) as part of the lookup result. If it is provided (`xgmac_elrd_val_i` is asserted), this data is inserted in either IVT or OVT field of RDES0. The `xgmac_iovt_sel_i` controls this selection as shown in Table 4-47.

Table 4-47 Rx Status Word and Selection Control

xgmac_elrd_val_i	xgmac_iovt_sel_i	Field in Rx Status Word
1	0	OVT
1	1	IVT

The value of these input control signals are also given as IOS and ELD status bits of RDES2 of the corresponding packet. If the lookup result is not provided, the packet is routed to DMA channel 0, with the ELD and IOS fields in the receive status set to 0.

If the lookup result is provided with DMA channel number set to all zeroes, the packet is dropped.

Note When the external lookup interface is enabled, it overrides all means of DMA channel selection in MAC (for example, DA-based, L3/L4 header based and RSS-based DMA channel selection are not applicable and the DMA channel selection settings in the corresponding registers/tables are ignored).

4.18.2 Packet Duplication

The 25GMAC can forward a packet to more than one DMA channel, provided if the packet is present in the highest receive queue (by configuring the controller), and more than one bit is set in the input DMA channel number from the External lookup engine. A packet is normally routed to particular receive queue based on the packet type and the programming in the `MAC_RxQ_Ctrl1`, `MAC_RxQ_Ctrl2`, and `MAC_RxQ_Ctrl3` registers. Only the packets that are routed to the highest RxQ number are eligible for packet duplication. So, if packet duplication for multicast packets is required, program the `MCBCQ` field of `MAC_RxQ_Ctrl1` register to point to the highest RxQ present in your configuration.

The packet duplication is done based on transferring the same packet to all the required RxDMA sequentially by the Read Controller of that RxQ. This packet is removed from the queue only after the packet is transferred to all selected DMA channels. Hence, use this feature for low-bandwidth traffic or ensure there is sufficient space available in this queue to avoid overflow. In case any of the required RxDMA is busy with transferring a packet from another queue, this queue is selected only after that packet transfer is over, resulting in additional delays due to head-of line blocking.

Note	When more than one bit is set for the DMA channel number for a packet which is not written to the highest receive queue, the packet is routed to only one DMA channel as indicated by the lowest bit set in xgmac_dch_i signal.

4.19 VxLAN/NVGRE Support

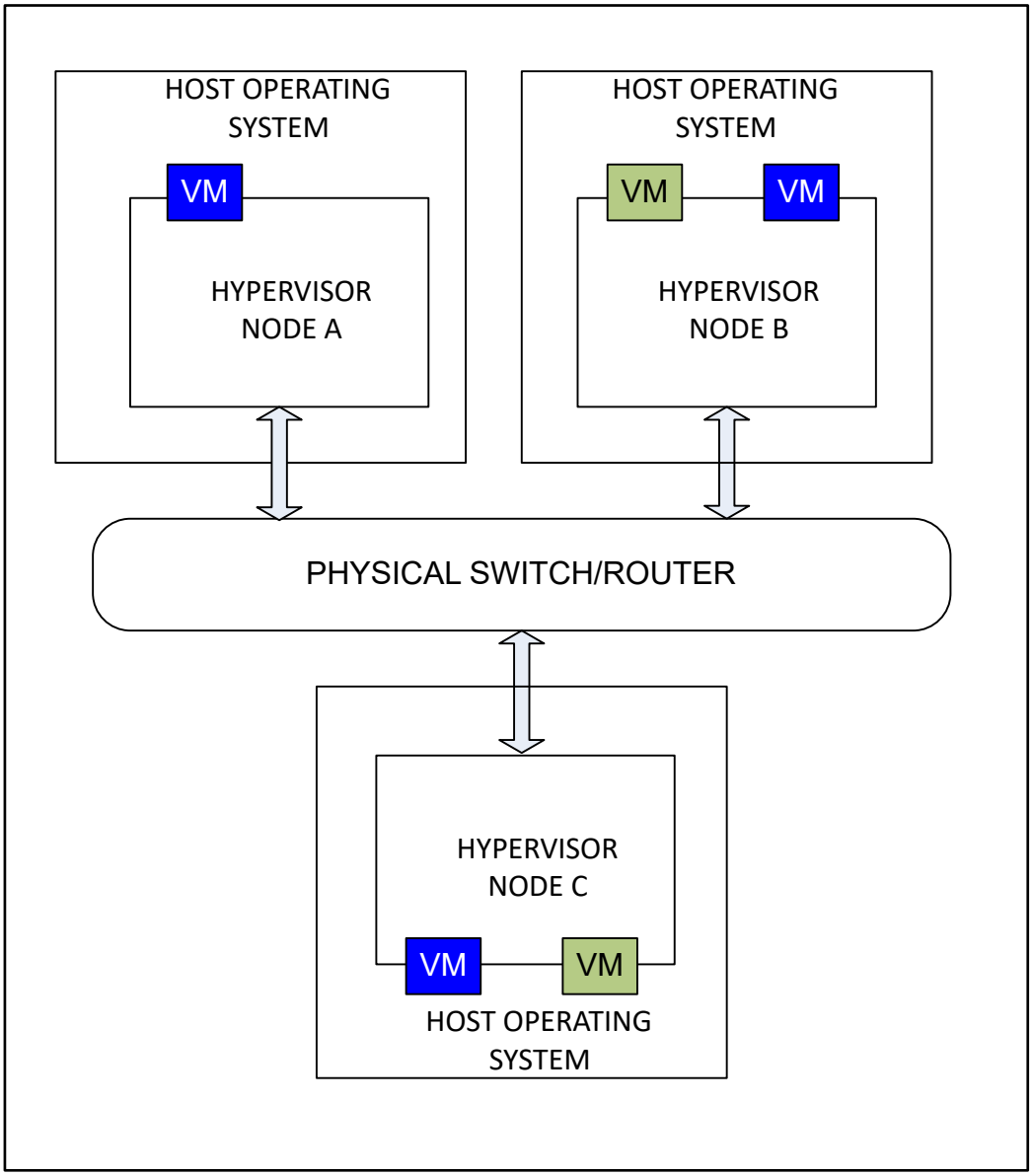
4.19.1 Overview

Both VxLAN and NVGRE are overlay encapsulation protocols to support L2 layer virtualization over L3 network on a logical group of machines called Virtual Machines. Some of the features of the VxLAN/NVGRE is summarized:

- Enables L2 Virtualization over L3 Network
- Support over 16million Virtual Network Segments
- Makes use of encapsulation technique (Tunneling) for routing L2 packet over L3 network
- Addresses the following limitations of traditional L2 Virtualization using VLAN:
  - Number of active paths due to Spanning Tree protocol
  - 4096 limitation of VLAN
  - IP address isolation among VMs across Virtual Network segments
- The L2/L3 addresses of the (MAC/IP addresses) are local to a Virtual Network segment. So, these addresses are unique only within a virtual network segment.

Figure 4-15 shows the overview of VxLAN/NVGRE support explaining the VM, HV and Virtual Segment functions:

Figure 4-15 Overview of Network Virtualization

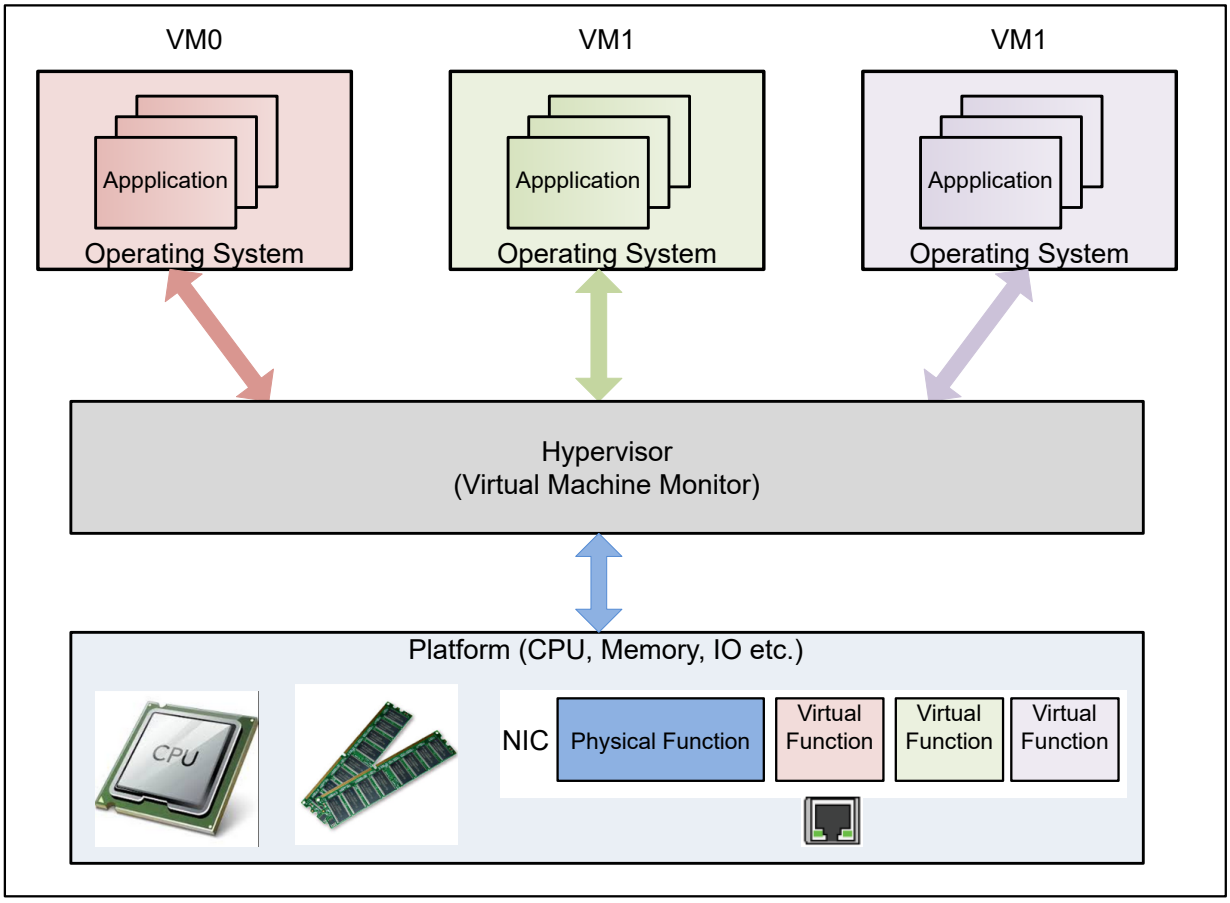


The packet format of the VxLAN and NVGRE is summarized in "VxLAN / NVGRE Packet Format" on page 1927.

The VxLAN/NVGRE enables the Server Machine virtualization in data centric environment. The set of Machines residing on multiple locations over L2/L3 network can be logically grouped to form a Virtual Network Segment and one can have 16 million such Virtual Network segments. Each Logical Network is identified by a unique ID called VNI/VSID of the VxLAN/NVGRE packet field. For more details, see "VxLAN / NVGRE Packet Format" on page 1927. The Virtual Machines (VMs) can switch the Network segment by software configuration through network management.

Figure 4-16 shows an example of I/O virtualization with three Guest OS sharing a single platform:

Figure 4-16 Example of I/O Virtualization



The heart of the Server virtualization resides in the Software layer called Hypervisor. The hypervisor functionality is as follows:

4.19.1.1 Overview of the Hypervisor Functionality

The VM (Virtual Machines) generates the source/original packet (Ethernet Packet + L3/L4 Payload). On the transmit side, this packet has to go through the software layer called Hypervisor. The hypervisor is responsible for building the Outer Packet Headers (Outer Ethernet + L3/L4 Headers). Similarly on the Receiver side the Hypervisor Software is responsible for striping the Outer headers (Outer Ethernet + L3/L4 Headers) before its forwarding to the VM (Virtual Machines).

The Hypervisor is controlled by the Management software about information like which VMs belongs to which VNI/VSID, IP Multicast address to be used for a particular Virtual Network Segments and so on. Also, VMs are unaware of the existence of the Hypervisor software and VMs works as if Hypervisor software is not present. Also, note that VMs can communicate only to another VM belonging to same Virtual Network segment and it cannot communicate to other virtual Network segment and to non-VMs.

Hypervisor creates the database (lookup tables) required for creating the outer header for encapsulating the packets given by VMs for transmission. Similarly, the HV uses the lookup database and VNID/TNID of the received packet and inner L2 headers to route the packets received from the port to the appropriate VM.

This database is built during the initial ARP Request/Response generated from the VMs prior to normal communication, Hypervisor is able to learn and is able to encapsulate the outer Header (with the information required to transport on the Local L2 and public L3 Network) without the knowledge of the VMs.

4.19.1.2 Features implemented in 25GMAC

The support for this overlay encapsulation protocols is added when "Enable support for VxLAN/NVGRE packets" is selected during configuration. This feature is available only in XGMAC-AXI configurations. The main features for this function are as follows:

- Support for either VxLAN based or NVGRE based processing of tunneled packets
- User programmable UDP Destination Port ID field in VxLAN and Protocol Type field in NVGRE Header used in parser
- In the Transmit path, the XGMAC-AXI supports:
  - Per packet control bit in TX Descriptor which can be used to mark or tag a VxLAN/NVGRE packet
  - Considers the tunneling header while performing the TCP/IP Segmentation function
  - Calculation and insertion of outer and inner IPv4 header checksum and payload checksums in the inner TCP/UDP/ICMP header
- In the Receive path, the following functions are supported:
  - Rx Checksum Engine can identify tunneled and non tunneled packets and perform checksum verification accordingly
  - Perform the header payload splitting at the inner header boundary
  - In this mode, the tunneled packets can be directed to a specific DMA channel based on the specific fields of the outer & inner headers based on a HASH function or based on an external lookup engine
  - VLAN stripping, packet filtering is based on outer L2, L3 and L4 header fields

4.19.2 VxLAN/NVGRE Controllability

The support for processing VxLAN/NVGRE tunneled packets in both Transmit and Receive direction can be controlled by software as follows:

- When bit[24] of `MAC_Tx_Configuration` register (VNE) is set to 1, 25GMAC recognizes and processes the Tunneled Packets. When this bit is set to 0, 25GMAC reverts to default mode in which it processes only the outer header.
- Bit[25] of `MAC_Tx_Configuration` register (VNM) selects the tunneling mode. When VNE is set and this bit is 0, 25GMAC processes VxLAN tunneled packets. When VNM and VNE are set to 1, it processes NVGRE tunneled packets.
- Tunneled packets are identified by a specific/unique value of the outer header's UDP Destination port ID (for VxLAN) or the Protocol Type field in outer IP header (in case of NVGRE). This unique value can be programmed by software in the `MAC_Tunnel_Identifier` register.

For more details on the processing of tunneled packets, see:

- "RSS Function when VXLAN/NVGRE Mode is Enabled" on page 275
- "Transmit Checksum Offload Engine" on page 259
- "Receive Checksum Offload Engine" on page 263
- "TCP Segmentation Offload" on page 267

5

PHY Interfaces

This chapter provides information about the PHY interfaces of 25GMAC.

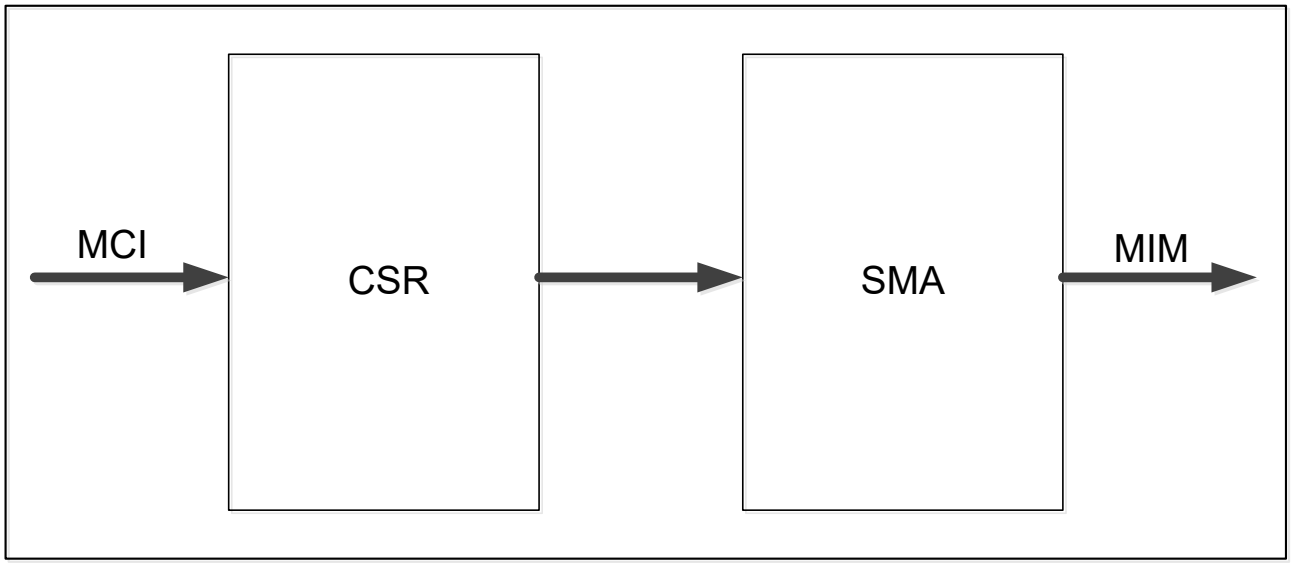
This chapter contains the following sections:

- "Station Management Agent Interface (MDIO)" on page 296
- "Reduced Gigabit Media Independent Interface (RGMII) to PHY " on page 303
- "Reduced Media Independent Interface (RMII)" on page 310
- "Multiple PHY Interfaces" on page 315

5.1 Station Management Agent Interface (MDIO)

The Station Management Agent (SMA) supports the application in accessing any PHY register through a two-wire Station Management Interface (SMI). To increase the number of MDIO Managed Devices (MMD) per port, the MDIO interface (IEEE 802.3-2018, Clause 22) is extended to support more device registers while still retaining logical compatibility with earlier MDIO interfaces. This extended MDIO interface is specified in the IEEE 802.3-2018, Clause 45.

Figure 5-1 Station Management Agent Interface Block



Through the station management agent, the application sends control data to the PHY (MIM write cycle) and receives status information from the PHY (MIM read cycle).

The software accesses the SMA specific registers in the CSR block for triggering the write or read transactions. The SMA block converts these commands from the software to serial transactions in Clause 45 frame format or Clause 22 frame format as instructed.

5.1.1 MDIO Interface Format

Using the extended MDIO interface, the application can address one of the 32 PHYs, any one of the 32 devices within the PHY, and one of the many registers within the given device and send control data or receive status information. Only one register in one PHY device can be addressed at a time. For more details on frame protocol see Clause 45 of IEEE 802.3-2018.

Table 5-1 shows the MDIO frame format:

Table 5-1 MDIO Clause 45 Frame Format

Frame	Description
IDLE	The MDIO line is three-state; there is no clock on sma_mdc_o
PREAMBLE	32 bits of continuous ones
START	Start of frame is 2'00

Table 5-1 MDIO Clause 45 Frame Format (continued)

Frame	Description
OPCODE	■ 2'b00: Address ■ 2'b01: Write ■ 2'b10: Post-read increment address ■ 2'b11: Read.
PHY ADDR	5-bit address select for one of 32 PHYs
DEV ADDR	5-bit address select for one of 32 devices
TA	Turnaround ■ 2'bZ0: Read and post-read increment address ■ 2'b10: Write and address MDIO accesses Where Z is the tri-state level
DATA/ADDRESS	16-bit value: For an address cycle (OPCODE = 2'b00), this frame contains the address of the register to be accessed on the next cycle. For the data cycle of a write frame, this field contains the data to be written to the register. For read or post-read increment address frames, this field contains the contents of the register read from the PHY. ■ In address and data write cycles, the 25GMAC drives the MDIO line during the transfer of these 16 bits. ■ In read and post-read increment address cycles, the PHY drives the MDIO line during the transfer of these 16 bits.

The frame format for Clause 22 frames is also supported and shown in Table 5-2. The control register (MDIO Clause 22 Port) indicates which ports are Clause 22 and which are Clause 45. The Clause 22 frame format is shown in Table 5-2. For Clause 22 frames, only the lower five bits of the RA field (MDIO Single Command Address Register) are used as the frame's REG ADDR field.

Table 5-2 MDIO Clause 22 Frame Format

Frame	Description
IDLE	The MDIO line is three-state; there is no clock on xgmii_mdc_o
PREAMBLE	32 bits of continuous ones
START	Start-of-frame is 2'01
OPCODE	■ 2'b01 for Write ■ 2'b10 for Read.
PHY ADDR	5-bit address select for one of 32 PHYs

Table 5-2 MDIO Clause 22 Frame Format (continued)

Frame	Description
REG ADDR	5-bit register address to select the register within each MMD
TA	Turnaround ■ 2'bZ0: Read ■ 2'b10: Write Where Z is the tri-state level
DATA	Any 16-bit value ■ In a write operation, the 25GMAC drives MDIO ■ In a read operation, the PHY drives MDIO

In addition to normal single-read and single-write operations, the SMA also supports continuous write, continuous scan, and post-read increment address. These operations are described the following sections.

Note The SMA module supports accesses to four PHY ports (addresses 0–3) only in the Continuous Scan and Continuous Write modes of operation (See "MDIO Continuous Write Operation" on page 300 and "MDIO Continuous Scan Operation" on page 301).

5.1.2 MDIO Operating Clock Frequency

For MIM accesses, the maximum operating frequency of the MDC (`sma_mdc_o` signal) is 2.5 MHz, as specified in IEEE 802.3-2018.

In the XGMAC-CORE configuration, the sma_mdc_o clock is derived from the application clock, clk_csr_i, using a divider-counter. The divide factor depends on the clock range setting (CR field and CRS field) in the MDIO Single Command Control/Data register.

You can select the clock divide factor as mentioned in Table 5-3, to meet the IEEE specifications. However, if your system supports higher clock frequencies on the MIM interface, you can select a different divider.

Table 5-3 MDC Clock Generation With CRS = 0

CRS Field of MDIO Single Command Control/Data	CR[2:0] Field of MDIO Single Command Control/Data	clk_csr_i Frequency (MHz)	Generated MDC Clock Frequency
0	3'b000	100-150	clk_csr_i/62
0	3'b001	150-250	clk_csr_i/102
0	3'b010	250-300	clk_csr_i/122
0	3'b011	300-450	clk_csr_i/142
0	3'b100	350-400	clk_csr_i/162

Table 5-3 MDC Clock Generation With CRS = 0 (continued)

CRS Field of MDIO Single Command Control/Data	CR[2:0] Field of MDIO Single Command Control/Data	clk_csr_i Frequency (MHz)	Generated MDC Clock Frequency
0	3'b101	400-500	clk_csr_i/202
0	3'b110 - 3'b111	Reserved	Reserved

If you require higher MDC clock frequency, set the CRS field of the MDIO Single Command Control/Data register. Table 5-4 shows the MDC clock generation based on CR field when CRS is set to 1.

Table 5-4 MDC Clock Generation With CRS = 1

CRS Field of MDIO Single Command Control/Data	CR[2:0] Field of MDIO Single Command Control/Data	clk_csr_i Frequency (MHz)	Generated MDC Clock Frequency
1	3'b000	-	clk_csr_i/4
1	3'b001	-	clk_csr_i/6
1	3'b010	-	clk_csr_i/8
1	3'b011	-	clk_csr_i/10
1	3'b100	-	clk_csr_i/12
1	3'b101	-	clk_csr_i/14
1	3'b110	-	clk_csr_i/16
1	3'b111	-	clk_csr_i/18

5.1.3 MDIO Preamble Suppression

25GMAC supports 32-bit preamble generation on all Read or Write operation. Some PHYs support reduction in the preamble content which can operate with only 1-bit preamble. When PSE (Preamble Suppression Enable, bit[30] of MDIO Single Command Control/Data Register) is set, preamble suppression mode is enabled and 25GMAC drives only 1-bit preamble for all MDIO operations.

5.1.4 MDIO Single Read/Write Operations

The software can perform a read or write operation to a specific register of any device of any port by programming SMA MDIO Single Command Address register and MDIO Single Command Control/Data register. If the PHY port is specified (in MDIO clause 22 port) as conforming to IEEE 802.3-2018, Clause 45, the SMA automatically initiates the transfer, as the clause specifies. Otherwise, the SMA initiates a read/write transfer in the frame format specified in Clause 22. The SMA assumes all ports conform to Clause 45 by default.

The Single Read/Write command usually has the highest priority among the three possible operations in the SMA (single read/write, continuous write and continuous scan) followed by the continuous write operation. On receiving a single read/write command, the SMA suspends continuous write/scan operation at the end of the current write/read cycle (access to a register is over) and initiates this command.

The sequence for initiating this operation (programming guidelines) is as follows:

1. Program the MDIO Single Command Address register with the appropriate PHY port number, device ID and register address.
2. Program the MDIO Single Command Control/Data register to initiate the required operation. If you are not performing 32-bit accesses to the CSR, make sure you write the Busy bit after writing the other bits, because setting the Busy bit triggers the command.
3. The SMA automatically performs the address cycle (if not disabled by SADDR bit in MDIO Single Command Control/Data register) followed by the read/write/post-increment read data transfer cycle. The Busy bit remains high during this period.
4. Wait for the Busy bit to clear (for polling) or for the Single Command Completion interrupt to be set.
5. On receiving the interrupt (or when Busy bit is cleared), the given single-command operation is completed and the data bits in MDIO Single Command Control/Data register have valid data read from the required PHY Device register.
6. The SMA is ready for another single command.

5.1.5 MDIO Continuous Write Operation

This command helps reduce the software's load when it must perform write operations to the same register in multiple devices or to multiple PHY ports. The software can give a single command and the SMA initiates writes to the target registers sequentially.

Continuous write operation is available for ports 0 to 31, and this command only accesses the devices enabled in the corresponding MDIO Port Device In-Use register. Thus, these registers must be configured properly for correct and fast completion of this command.

The programming sequence is as follows:

1. Program the MDIO Continuous Write Data register with the required data value.
2. Configure the required operation in the MDIO Continuous Write Address register by programming the CPRT and CADDR bits.
   - If CPRT is low, CADDR is taken as the common device address in all ports to which register write operations are performed.
   - If CPRT is set high, CADDR is taken as the port number whose devices are all accessed in the register write operation. CADDR must have values of 0–3 in this mode: other PHY port numbers are invalid.
3. Configure the register address in the CREGADDR bits and set the Continuous Write Busy bit. This step and Step 2 can be done in a single 32-bit write access to the register.
4. The SMA schedules the continuous write operation to all the ports or devices. It performs the two-transfer cycle if the PHY port is configured as a Clause 45 port or a single cycle if it is a Clause 22 port.

5. Wait until the Continuous Write Busy bit is cleared (for polling) or the Continuous Write Completion interrupt is set.
6. When the SMA asserts the interrupt (or the Busy bit is cleared), the given command is complete and the SMA is ready for another continuous write command to a different port, device, or register.

5.1.6 MDIO Continuous Scan Operation

This command helps reduce the load on the software for checking the status of the devices on each port. It also helps in automatic connectivity detection (hot-plug-in) of PHY devices (on a XENPAK module). This command usually has the lowest priority and can be enabled to always run in the background. This command performs read accesses to two specific registers (MDIO Single Command Address register and MDIO Port Connect/Disconnect Status register) of each device on all ports to check their link and device statuses and to update the corresponding status registers in the SMA (MDIO Port Nx4P0 Link and MDIO Port Nx4P0 Alive status registers).

Continuous scan operation only applies to ports and this command only accesses the devices enabled in the corresponding MDIO Port Device In Use register. These registers must be configured properly for correct and fast completion of a scan cycle.

The programming sequence is as follows:

1. Configure the MDIO Port Nx4P0 Device In Use registers to enable the devices present in the PHY port. If a particular port does not have any PHY connected to it (that is, its current status is disconnected), enable only one of the devices (for example, PMDPMA) to check for connectivity.
2. Enable the continuous scan to the required ports by setting the bits in the MDIO Continuous Scan Port Enable register. The SMA scans the ports in sequence.
3. Enable the required interrupts in the MDIO Interrupt Enable register (bits 11:0).
4. The SMA scans all the ports and updates the corresponding status registers.
5. The MDIO Port Connect-Disconnect Status register is updated as soon as the read access to any device register to that port is completed. The corresponding interrupt is asserted if any bit's value changes.
6. The MDIO Port Nx4P0 Link and MDIO Port Nx4P0 Alive status registers are updated at the end of each scan cycle and if any bits in the corresponding ports change, the corresponding interrupt is asserted.
7. A new sequence is started automatically from Port 0.
8. The scan continues as long as any MDIO Continuous Scan Port Enable register bit is set.

5.1.7 Trailing Clocks and Back-to-Back Transactions

The 25GMAC drives MDC clock on the gmii_mdc_o port for duration of the MDIO frame. There is no clock driven during the idle period. The trailing clocks feature can be used if the PHY needs the MDC clock to be active for some cycles after the MDIO frame. The NTC field in `MAC_MDIO_Address` register allows programming of trailing clocks from 0 to 7.

The 25GMAC supports Back to Back transactions which allows start of next MDIO frame even before the trailing clocks are completed for previous MDIO frame. This feature can be enabled by setting BTB bit in `MAC_MDIO_Address` register when trailing clocks feature is also enabled. When Back to Back transactions is enabled, the GMII Busy is cleared immediately after MDIO frame completion. This allows the software to issue the next command, which is executed by 25GMAC while trailing clocks are still on for

the previous MDIO frame. When Back to Back transactions is not enabled, the GMII Busy is cleared after the trailing clocks are completed for MDIO frame.

5.1.8 Enabling Station Management Agent

To enable this feature while configuring the controller in the coreConsultant GUI:

Select the Enable Station Management (MDIO Interface) option under the PHY Interface section during the Specify Configuration activity in coreConsultant.

For details about this option, see PHY Interface Parameters in the "*Parameter Descriptions*" chapter.

Note See the "Enabled" field of the parameter descriptions to understand the dependencies.

5.2 Reduced Gigabit Media Independent Interface (RGMII) to PHY

The Reduced Gigabit Media Independent Interface (RGMII) module is implemented in 25GMAC to reduce the pin count as mentioned in the RGMII specification.

The Reduced Gigabit Media Independent Interface (RGMII) specification reduces the pin count of the interconnection between the MAC and the PHY for GMII and MII interfaces. To achieve this, the data path and control signals are reduced and multiplexed together with both edges of the Transmit and Receive clocks. For Gigabit operation, the clocks operate at 125 MHz; for 10/100 operation, the clock rates are 2.5 MHz/25 MHz.

The RGMII module is instantiated between the GMII of the MAC and the PHY to translate the control and data signals between the GMII and RGMII protocols.

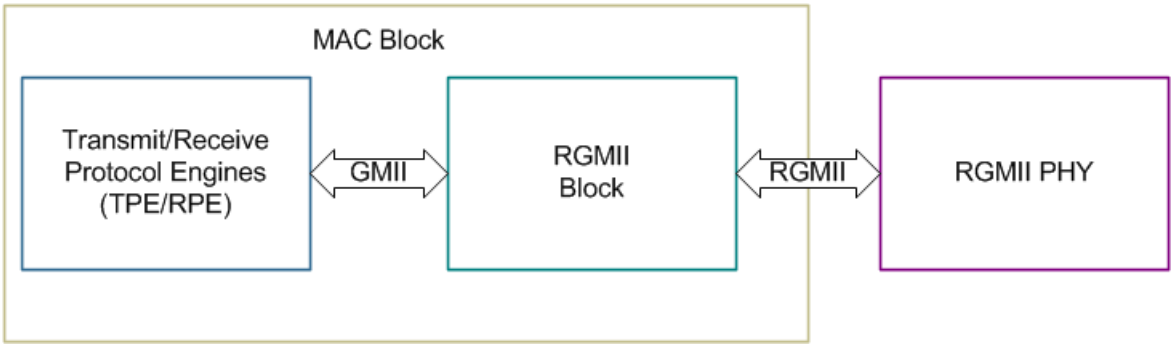
The RGMII block has the following characteristics:

- Supports 10 Mbps, 100 Mbps, and 1000 Mbps operation rates
- Requires no additional clock because both edges of the incoming clocks are used. To simplify back-end implementation of the MAC, there are separate clock inputs (180 degrees out-of-phase with the associated transmit/receive clocks) for logic that uses the falling edges. For guidelines for clock connections when the MAC operates with an RGMII, see *Synopsys Ethernet 25GMAC User Guide*.
- Extracts the in-band (link speed, link status, and duplex mode) status signals from the PHY and provides them to the MAC for link detection

5.2.1 Functional Description of RGMII PHY Interface

Figure 5-2 shows the position of the RGMII block relative to the MAC and RGMII PHY. The RGMII block is placed between the GMII and the PHY to translate the GMII signals to RGMII signals.

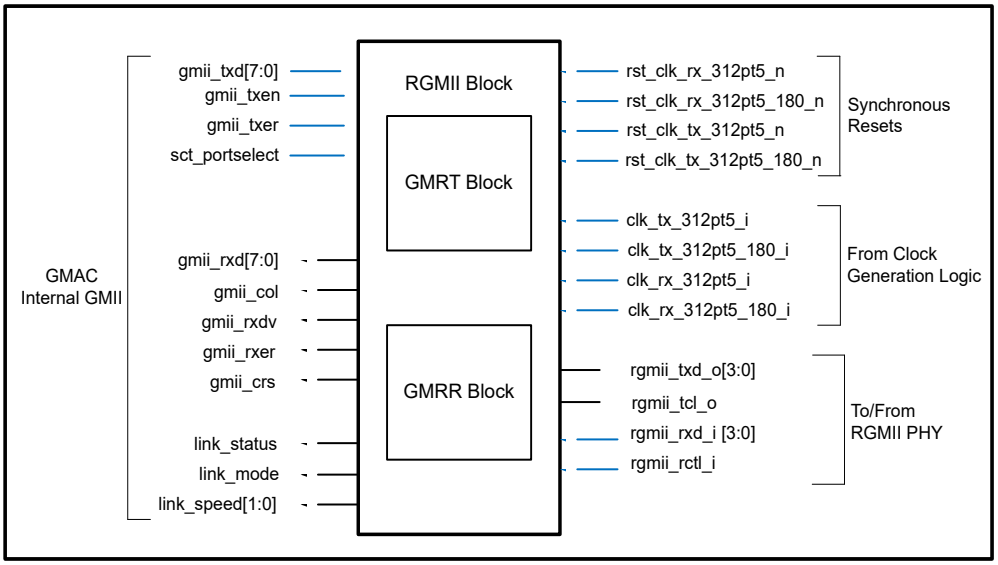
Figure 5-2 RGMII Block Diagram



The following list describes the RGMII components shown in Figure 5-3:

- GMII-RGMII Transmit (GMRT) Block: This block translates all GMII transmit signals to RGMII signals. The GMRT registers all GMII input signals at the rising edge of clk_tx_312pt5_i and generates all RGMII transmit signals at the rising edge of either clk_tx_312pt5_i and clk_tx_312pt5_180_i.
- GMII-RGMII Receive (GMRR) Block: This block translates all RGMII receive signals to GMII receive signals. The GMRR registers all RGMII input signals at the rising edge of clk_rx_312pt5_i and clk_rx_312pt5_180_i and generates all GMII receive signals at the rising edge of clk_rx_312pt5_i.

Figure 5-3 RGMII Block Pinout Diagram



5.2.2 RGMII Clocks

The RGMII block uses the transmit and receive clocks, and two additional clocks (clk_tx_312pt5_180_i and clk_rx_312pt5_180_i) from the clock generation logic to reduce the pin count of the interconnection between the MAC and the PHY for GMII and MII interfaces.

The RGMII operation uses both edges of the RGMII transmit and receive clocks (clk_tx_312pt5_i and clk_rx_312pt5_i), which run at 125 MHz for gigabit operation or 2.5 MHz/25 MHz for 10/100 operation. These clocks come from a separate PLL logic block. To simplify implementation of the MAC, the RGMII block provides two additional inputs to drive the logic that uses the falling edges of clk_tx_312pt5_i and clk_rx_312pt5_i:

- clk_tx_312pt5_180_i must be 180 degrees out-of-phase with respect to clk_tx_312pt5_i.
- clk_rx_312pt5_180_i must be 180 degrees out-of-phase with respect to clk_rx_312pt5_i.

Separate synchronous reset inputs are present to synchronously reset the logic driven by each of the four RGMII clocks.

5.2.3 Signal Conversion

The RGMII block performs the following conversions:

- “Transmit Data Conversion” on page 305
- “Transmit Control Signal Conversion” on page 305
- “Receive Data Conversion” on page 306
- “Receive Control Signal Conversion” on page 306
- “Receive Status Signal Conversion” on page 306

5.2.3.1 Transmit Data Conversion

As defined in the RGMII specification, multiplexing of data and control in Gigabit mode is accomplished by using both edges of the Transmit clock. The RGMII block accepts the 8-bit GMII transmit data from the MAC GMII (gmii_txd[7:0]) on the rising edge of clk_tx_312pt5_i. For Gigabit operation, the RGMII block then converts gmii_txd[7:0] to two 4-bit nibbles and transmits the data on the RGMII transmit data port (rgmii_txd_o[3:0]) as follows:

- At the rising edge of clk_tx_312pt5_i, rgmii_txd_o[3:0] contains gmii_txd[3:0]
- At the falling edge of clk_tx_312pt5_i (rising edge of clk_tx_312pt5_180_i), rgmii_txd_o[3:0] contains gmii_txd[7:4]

For 10/100 mode, the Transmit data to the PHY is 4 bits. Therefore, for 10/100 mode, the RGMII block drives the gmii_txd[3:0] data to the PHY on rgmii_txd_o[3:0] at the rising edge of clk_tx_312pt5_i.

5.2.3.2 Transmit Control Signal Conversion

The RGMII block accepts the GMII transmit control signals (gmii_txen and gmii_txer) from the MAC GMII on the rising edge of clk_tx_312pt5_i. The RGMII block then multiplexes gmii_txen and gmii_txer onto the rgmii_tctl_o output (as required by the RGMII specification) as follows:

- At the rising edge of clk_tx_312pt5_i, rgmii_tctl_o is gmii_txen
- At the falling edge of clk_tx_312pt5_i (rising edge of clk_tx_312pt5_180_i), rgmii_tctl_o is the logical XOR of gmii_txen and gmii_txer.

Table 5-5 shows the mapping of the PLS_DATA.request from the MAC to the corresponding signaling on the GMII and RGMII control and data signals.

Table 5-5 Encoding of gmii_txen, gmii_txer, rgmii_tctl_o, and rgmii_txd_o Signals

gmii_txen	gmii_txer	gmii_txd[a]	rgmii_tctl_o[b]	Description	PLS_DATA.request
0	0	0x00–0xFF	0,0	Normal inter-packet	TRANSMIT_COMPLETE
0	1	0x00	0,1	Reserved	ZERO, ONE (8 bits)
0	1	0x01	0,1	LPI pattern	TRANSMIT_LPI in EEE mode
0	1	0x02–0x0E0,1	0,1	Reserved	
0	1	0x0F	0,1	Carrier extend	EXTEND (8 bits)
0	1	0x10–0x1E	0,1	Reserved	
0	1	0x1F	0,1	Carrier extend error	EXTEND_ERROR (8 bits)
0	1	0x20–0xFF	0,1	Reserved	
1	0	0x00–0xFF	1,1	Normal data transmission	ZERO, ONE (8 bits)

Table 5-5 Encoding of gmii_txen, gmii_txer, rgmii_tctl_o, and rgmii_txd_o Signals (continued)

gmii_txen	gmii_txer	gmii_txd[a]	rgmii_tctl_o[b]	Description	PLS_DATA.request
1	1	0x00–0xFF	1,0	Transmit error propagation	None applicable

a. If the RGMII interface is configured to transmit the configuration during the IPG, rgmii_txd[3:0] reflects the Duplex Mode, Port Select, Speed (encoded as 00 for 10 Mbps, 01 for 100 Mbps, and 10 for 1000 Mbps), and Link Up/Down bits of the MAC_Tx_Configuration register during this period.
b. Values at rising, falling edges of clk_tx_312pt5_i

5.2.3.3 Receive Data Conversion

For 1000 Mbps mode, the RGMII block registers the 4-bit data on the rgmii_rxd_i[3:0] signal at both edges of the Receive clock (clk_rx_312pt5_i) and transfers the resulting 8-bit data to the 25GMAC GMII on the rising edge of clk_rx_312pt5_i clock, as follows:

- gmii_rxd[3:0] is the value received on rgmii_rxd_i[3:0] at the rising edge of clk_rx_312pt5_i
- gmii_rxd[7:4] is the value received on rgmii_rxd_i[3:0] at the falling edge of clk_rx_312pt5_i (rising edge of clk_rx_312pt5_180_i)

For 10/100 mode, the RGMII block registers the 4-bit data on rgmii_rxd_i[3:0] only at the rising edge of clk_rx_312pt5_i and transfers the data to the 25GMAC GMII on the rising edge of clk_rx_312pt5_i, as follows:

- gmii_rxd[3:0] is the value received on rgmii_rxd_i[3:0] at the rising edge of clk_rx_312pt5_i
- gmii_rxd[7:4] is 0x0

5.2.3.4 Receive Control Signal Conversion

The RGMII block samples the rgmii_rctl_i signal from the PHY at both edges of the clk_rx_i clock and drives the resulting GMII receive control signals to the 25GMAC GMII as follows:

- gmii_rxdv is the value received on rgmii_rctl_i at the rising edge of clk_rx_312pt5_i.
- gmii_rxer is the logical XOR of the following two signals
  - The value received on rgmii_rctl_i at the rising edge of clk_rx_312pt5_i (gmii_rxdv).
  - The value received on rgmii_rctl_i at the falling edge of clk_rx_312pt5_i (rising edge of clk_rx_312pt5_180_i).

5.2.3.5 Receive Status Signal Conversion

To generate the GMII link status signals, the RGMII block decodes the rgmii_rxd_i[3:0] value sampled at the rising edge of clk_rx_312pt5_i when the value of rgmii_rctl_i is 0 for both the rising and falling edges of clk_rx_312pt5_i (normal inter-packet value). The following are the decoded status signals:

- link_status is the value of rgmii_rxd_i[0]
- link_speed is the value of rgmii_rxd_i[2:1]
- link_mode is the value of rgmii_rxd_i[3]

Table 5-6 shows the mapping of the PLS_DATA.indicate from the 25GMAC to the corresponding signaling on the GMII and RGMII control and data signals. In addition, the RGMII block asserts gmii_crs to the 25GMAC GMII when any of the following conditions is true:

- The gmii_rxdv is true
- The gmii_rxdv is false, gmii_rxer is true, and the value of gmii_rxd is 0xFF
- A carrier extend, carrier extend error, or false carrier occurs in Gigabit mode (see Table 5-6).
- A false carrier occurs in 10/100 mode.
- The RGMII is transmitting data, carrier extend, or carrier extend error on the RGMII transmit outputs.

The RGMII block asserts gmii_col to the MAC GMII when both of the following conditions are true:

- The RGMII is transmitting data, carrier extend, or carrier extend error on the RGMII transmit outputs
- The RGMII is asserting either gmii_crs or gmii_rxdv signal

Table 5-6 Decoding of rgmii_rctl_i and rgmii_rxd_i

rgmii_rctl_i	gmii_rxd[7:0]([3:0] in 10/100 mode)[a]	gmii_rxdv	gmii_rxer	Description	PLS_DATA.indicate or PHY_status
0,0	XXXXXXX0 or XXXXXXX1	0	0	Normal inter-packet	Indicates link status: ■ 0: down ■ 1: up
0,0	XXXXX00X or XXXXX01X or XXXXX10X or XXXXX11X	0	0	Normal inter-packet	Indicates Receive clock frequency: ■ 00: 2.5 MHz ■ 01: 25 MHz ■ 10: 125 MHz ■ 11: Reserved
0,0	XXXX1XXX or XXXX0XXX	0	0	Normal inter-packet	Indicates duplex status: ■ 0: half-duplex ■ 1: full-duplex
0,1	0x01 (or 0x1 in 10/100 mode)	0	1	Low Power Idle (LPI)	Remote MAC in LPI in the Energy Efficient Ethernet (EEE)mode
0,1	0x0E (or 0xE in 10/100 mode)	0	1	False carrier indication	False carrier present
0,1	0x0F	0	1	Carrier extend	EXTEND (8 bits)
0,1	0x1F	0	1	Carrier extend error	ZERO, ONE (8 bits)
0,1	0xFF (or 0xF in 10/100 mode)	0	1	Carrier sense	PLS_Carrier.Indicate

Table 5-6 Decoding of rgmii_rctl_i and rgmii_rxd_i (continued)

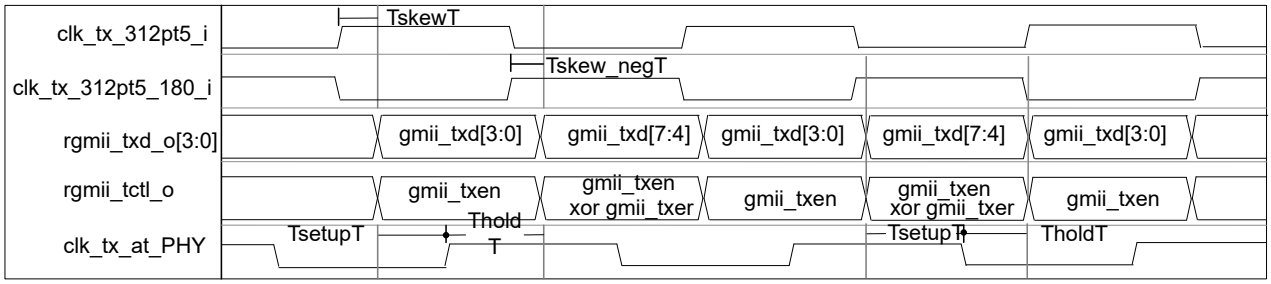
rgmii_rctl_i	gmii_rxd[7:0]([3:0] in 10/100 mode)[a]	gmii_rxdv	gmii_rxer	Description	PLS_DATA.indicate or PHY_status
1,1	0x00–0xFF	1	0	Normal data reception	ZERO, ONE (8 bits)
1,0	0x00–0xFF	1	1	Data reception error	ZERO, ONE (8 bits)

a. Values at rising, falling edges of clk_rx_312pt5_i

5.2.4 RGMII Transmit Timing

Figure 5-4 shows the timing for RGMII transmission. The RGMII specification defines a transmit skew (TskewT) of ± 0.5 ns. As shown in Figure 5-4, the trace delay on the PC board for clk_tx_i going to a receiving PHY must satisfy the setup and hold time requirements of the RGMII Transmit signals at the receiving PHY.

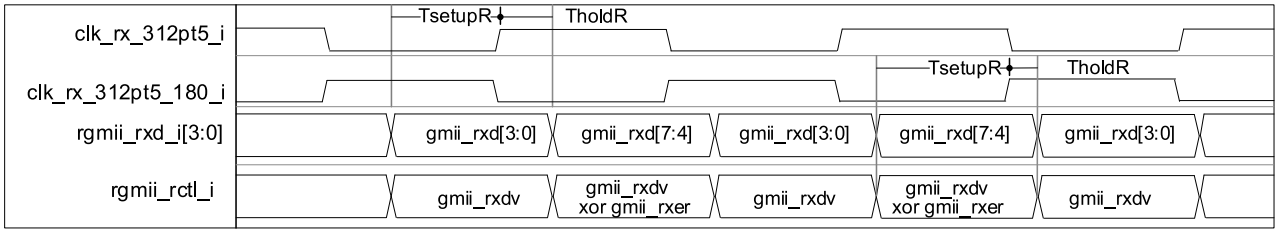
Figure 5-4 RGMII Transmit Timing



5.2.5 RGMII Receive Timing

Figure 5-5 shows the receive timing for RGMII. The TsetupR and TholdR values are required to be a minimum of 1 ns.

Figure 5-5 RGMII Receive Timing



5.2.6 Enabling the RGMII PHY Interface

To enable this interface while configuring the controller in the coreConsultant GUI:

Select the Enable RGMII Support option in the PHY Interface section during the Specify Configuration activity in coreConsultant.

For details about this option, see PHY Interface Parameters in the *Parameter Descriptions* chapter.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

5.3 Reduced Media Independent Interface (RMII)

5.3.1 Introduction to Reduced Media Independent Interface

The Reduced Media Independent Interface (RMII) specification reduces the pin count between Ethernet PHYs and Switch ASICs (only in 10/100 mode). According to the IEEE 802.3u, an MII contains 16 pins for data and control. In devices incorporating multiple MAC or PHY interfaces (such as switches), the number of pins adds significant cost with increase in port count. The RMII specification addresses this problem by reducing the pin count to 7 for each port — a 62.5% decrease in pin count

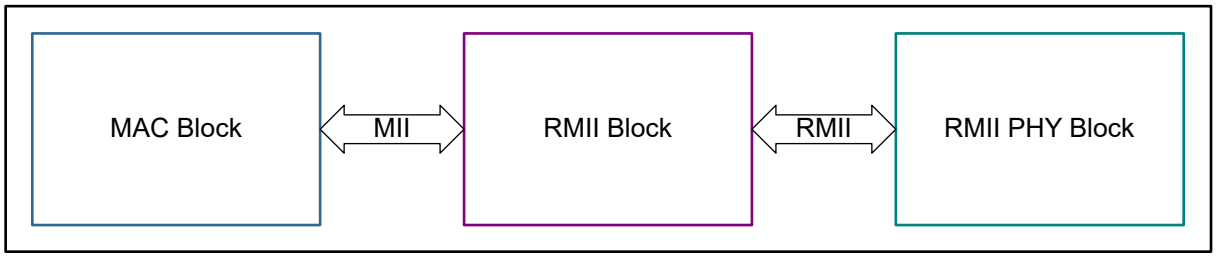
The RMII module is instantiated between the MAC and the PHY. This helps in translating the MII of the MAC into the RMII. The RMII block has the following characteristics:

- Supports 10 Mbps and 100 Mbps operating rates. It does not support the 1000 Mbps operation
- Provides independent 2-bits wide Transmit and Receive paths by sourcing two clock references externally.

5.3.2 Description of Reduced Media Independent Interface

Figure 5-6 shows the position of the RMII block relative to the 25GMAC and RMII PHY. The RMII block is placed in front of the 25GMAC to translate the MII signals to RMII signals.

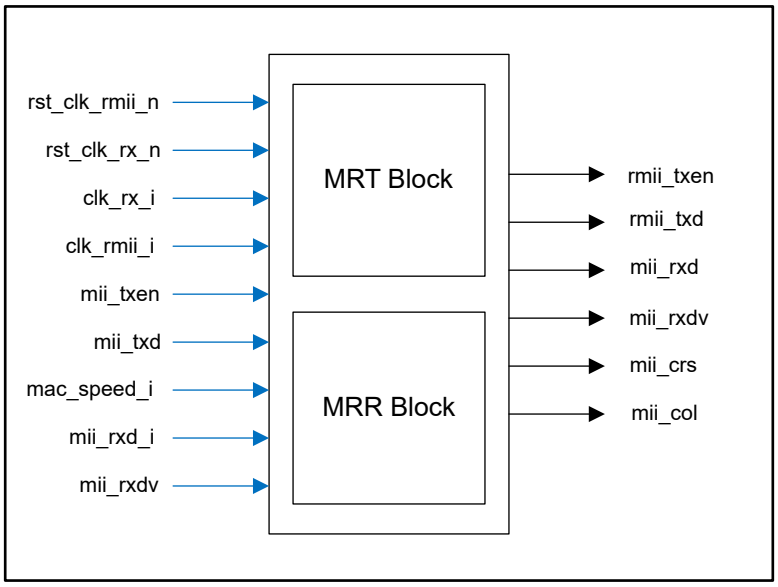
Figure 5-6 RMII Block Diagram



The following list describes the components of RMII shown in Figure 5-7:

- MII-RMII Transmit (MRT) Block: This block translates all MII transmit signals to RMII transmit signals. All RMII signals are synchronous to clk_rmii_i.
- MII-RMII Receive (MRR) Block: This block translates all RMII receive signals to MII receive signals. All MII signals are synchronous to clk_rx_i. You must ensure that the same clock is connected to both clk_tx_i and clk_rx_i clock input ports in RMII mode. This is required because clock-MUXing logic is avoided inside the MAC.

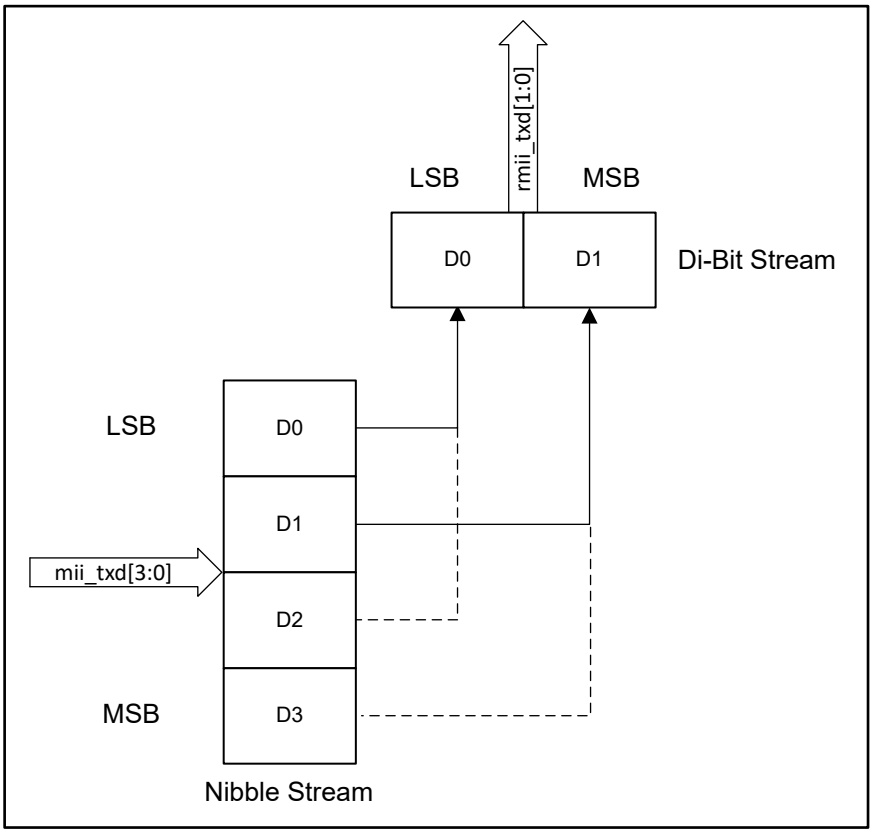
Figure 5-7 RMII Pinout



5.3.2.1 Transmit Bit Ordering

Each nibble from the MII interface must be transmitted on the RMII interface di-bit at a time with the order of di-bit transmission shown in Figure 5-8. The lower order bits (D1 and D0) are transmitted first followed by higher order bits (D2 and D3).

Figure 5-8 Transmit Bit Ordering



5.3.2.2 RMII Transmit Timing Diagrams

RMII transmit timing diagrams show the various MII transmissions.

Figure 5-9 through Figure 5-12 show MII-to-RMII transaction timing. Figure 5-9 shows the start of MII transmission and the following RMII transmission in 100 Mbps mode.

Figure 5-9 Start of MII and RMII Transmission in 100 Mbps Mode

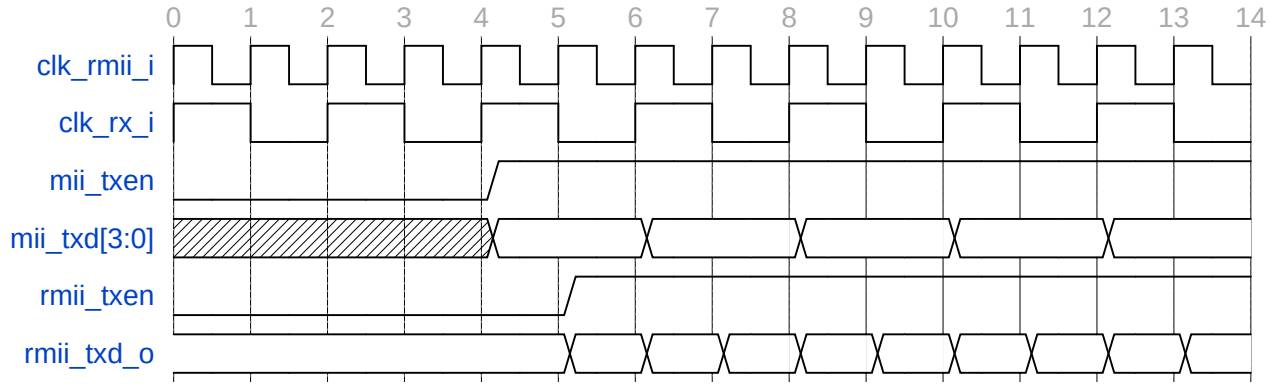


Figure 5-10 shows the end of packet transmission for MII and RMII in 100 Mbps mode.

Figure 5-10 End of MII and RMII Transmission in 100 Mbps Mode

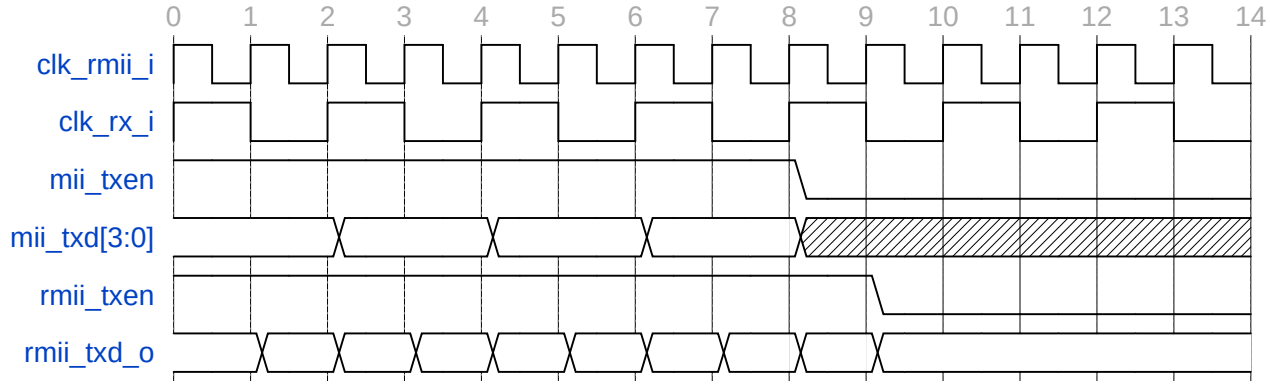


Figure 5-11 shows the start of MII transmission and the following RMII transmission in 10-Mbps mode.

Figure 5-11 Start of MII and RMII Transmission in 10 Mbps Mode

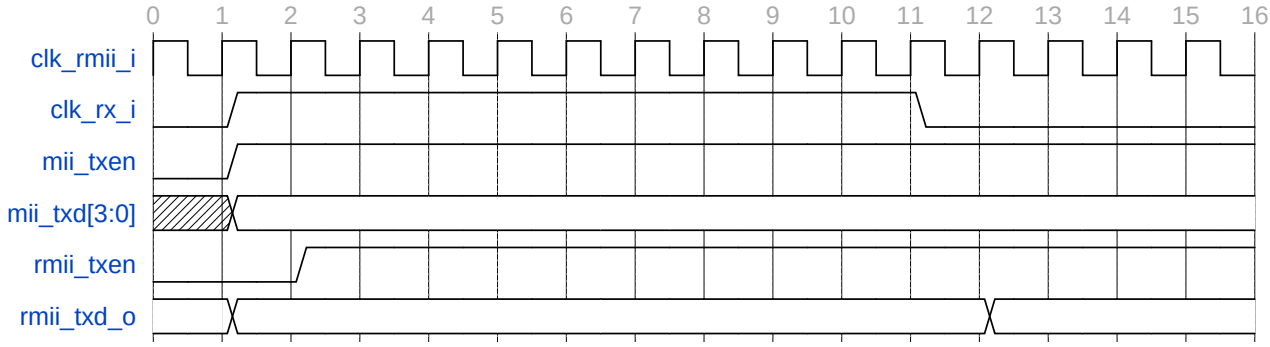
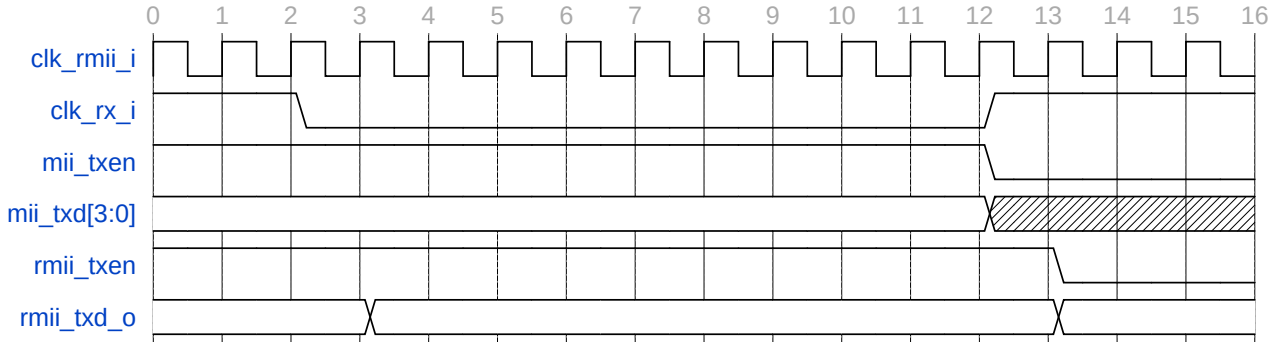


Figure 5-12 shows the end of MII transmission and RMII transmission in 10-Mbps mode.

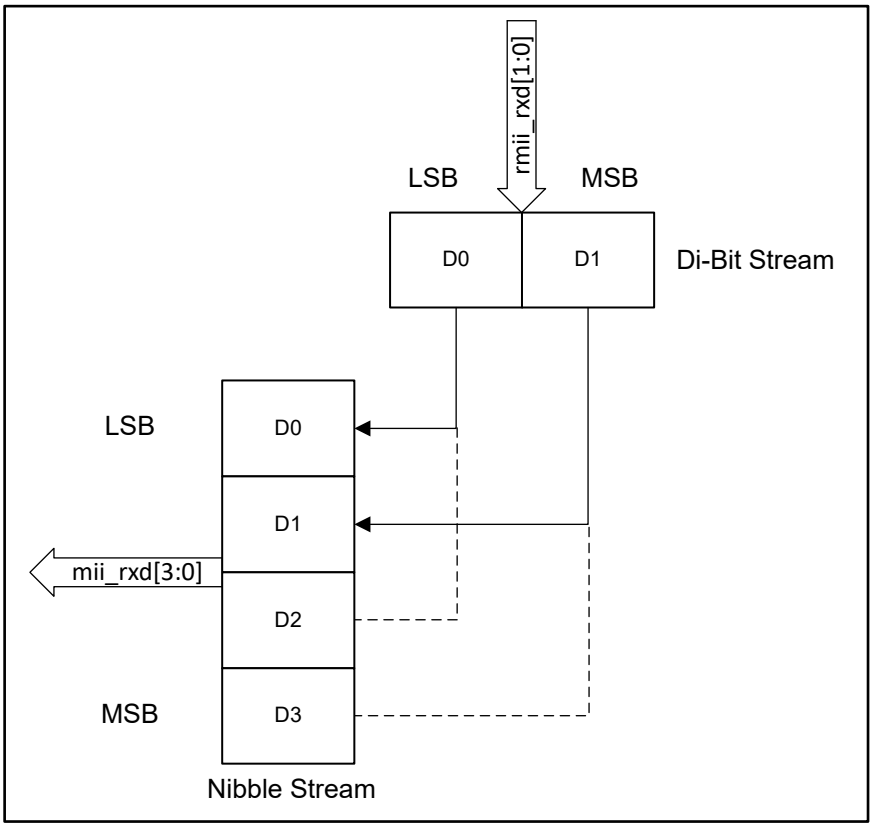
Figure 5-12 End of MII and RMII Transmission in 10 Mpbs Mode



5.3.2.3 Receive Bit Ordering

Each nibble is transmitted to the MII interface from the di-bit received from the RMII interface in the nibble transmission order shown in Figure 5-13. The lower order bits (D0 and D1) are received first, followed by the higher order bits (D2 and D3).

Figure 5-13 Receive Bit Ordering



5.3.2.4 Enabling Reduced Media Independent Interface

To enable this interface while configuring the controller in the coreConsultant GUI:

Select the Enable RMII option under the PHY Interface section during the Specify Configuration activity in coreConsultant.

For details about this option, see PHY Interface Parameters the *Parameter Descriptions* chapter.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

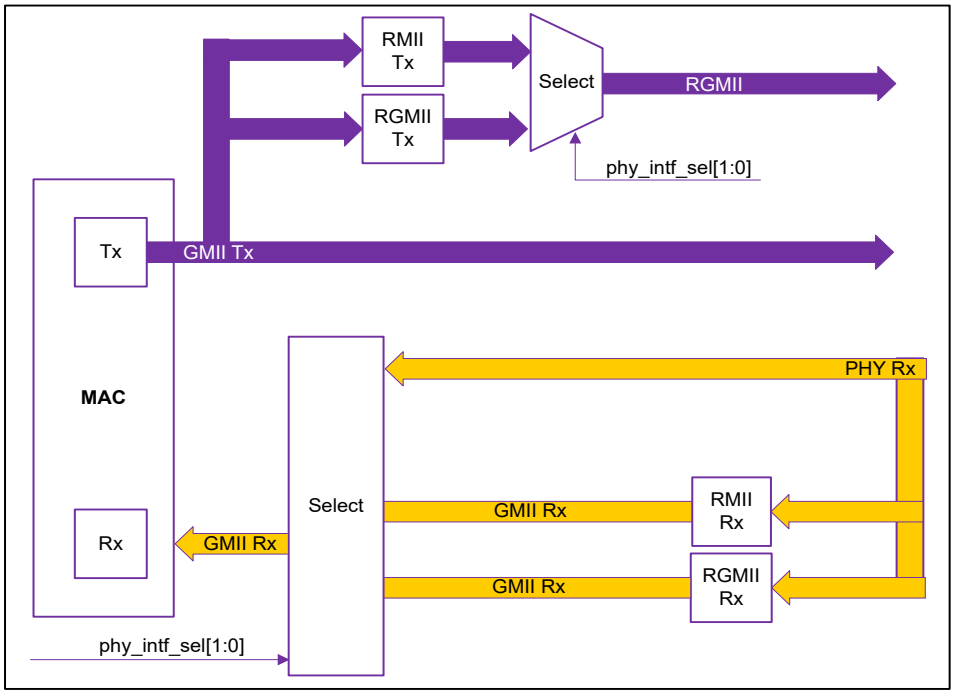
5.4 Multiple PHY Interfaces

25GMAC supports multiple PHY interfaces by using a multiplexing log.

Figure 5-14 shows the multiplexing logic implemented to support multiple programmable PHY interfaces.

Note	When the MAC is configured for a single PHY interface, all MUX logic is removed and the corresponding PHY interface is connected directly to the ports.

Figure 5-14 Multiplexing Logic for Multiple Programmable PHY Interfaces



6

Multiple Channels and Queues Support

This chapter describes how the 25GMAC controller supports multiple queues and channels.

This chapter contains the following sections:

- "Block Diagram" on page 318
- "DMA" on page 319
- "Multiple Queues Support in the MAC" on page 321
- "Multiple Queues Support in the MAC Transaction Layer" on page 322
- "Tx Side Arbitration Between DMA and MTL" on page 333
- "Rx Side Arbitration Between MTL and DMA" on page 334
- "Rx Queue to DMA Mapping" on page 335
- "Broadcast/Multicast Packet Duplication" on page 339
- "Selection of Tag Priorities Assigned to Tx and Rx Queues" on page 341
- "Distribution of Ingress Packets from MAC to Rx Queues" on page 342

6.1 Block Diagram

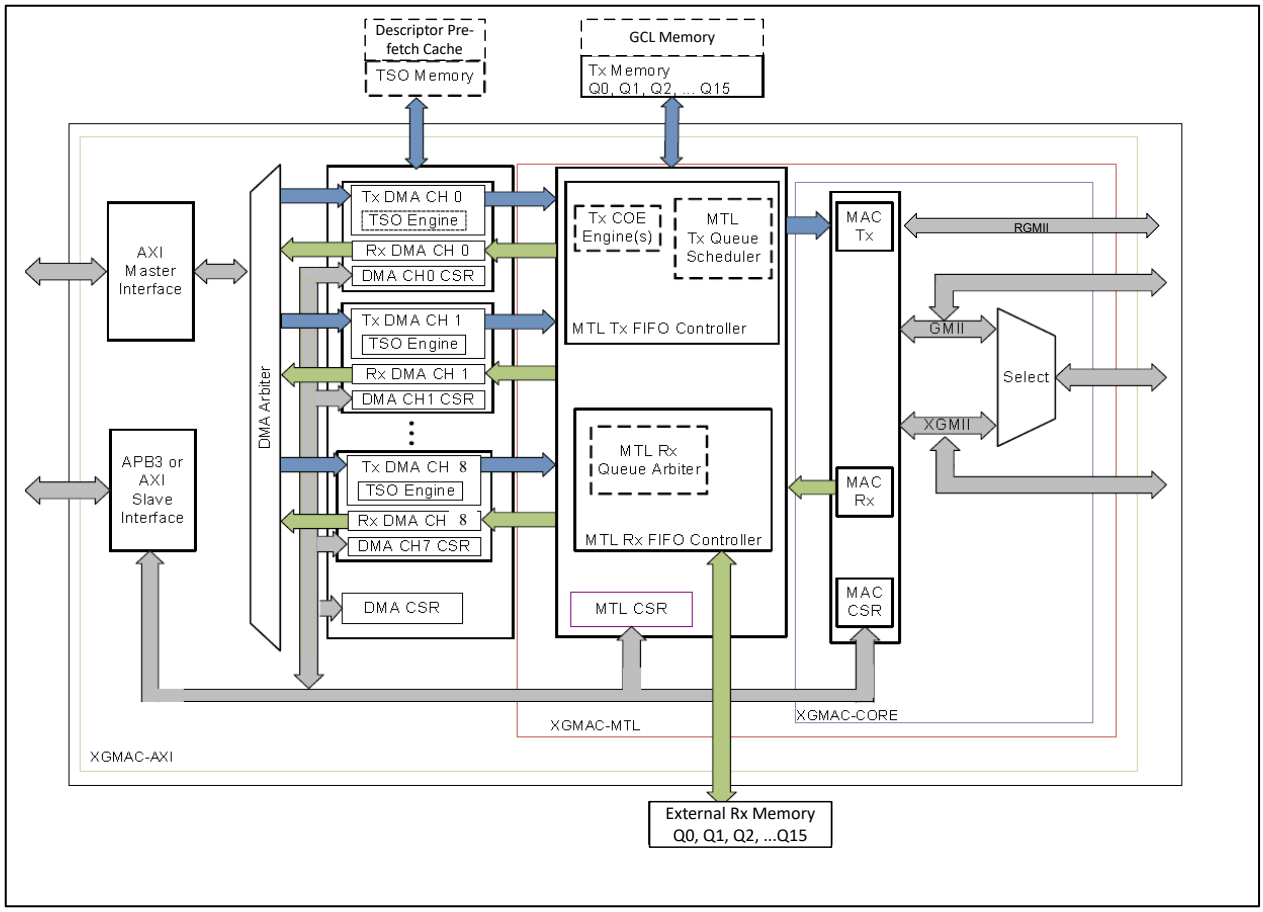
The 25GMAC supports up to 16 queues and channels on Tx and Rx paths. Figure 6-1 shows the architecture of 25GMAC with multiple queues and channels.

Note

In multiple channel/queue configurations, for

- Half-duplex operations, enable only the Q0/CH0 on Tx.
- Full-duplex operations, for single queue or channel, enable any queue/channel.

Figure 6-1 25GMAC with Multiple Channels and Queues



6.2 DMA

This section describes how multiple queues are supported in the Transmit and Receive paths in the XGMAC-AXI configurations.

6.2.1 Transmit Path

The 25GMAC supports up to 16 Tx channels in XGMAC-AXI configurations. You can select the number of Tx channels by using the Number of DMA Channels on Transmit Side option. The fixed priority scheme is used in the Transmit path to arbitrate between the multiple TxDMA channels to access the common AXI bus. In this scheme, the channel with highest priority always wins the arbitration when it requests the bus. Table 6-1 provides information about the priority levels of TxDMA channels.

Table 6-1 Fixed Priority Scheme for TxDMA Channels

Priority Level	Channel
0 (low)	Channel 0
1	Channel 1
2	Channel 2
3	Channel 3
...	...
15 (high)	Channel 15

The fixed priority scheme does not necessarily starve the lower priority channels for the AXI bus. The AXI4 protocol capabilities and the TxDMA characteristics are as follows:

- Read command requests from a TxDMA for a burst of data transfers is driven on a separate AXI channel and gets executed in 1 clock cycle. The actual data transfer starts on a separate AXI channel only after the request is accepted and takes multiple clock cycles to complete, with relatively large read access latencies.
- AXI manager is free to drive out subsequent requests even before the previous data transfers are completed.
- The TxDMA requests for another data transfer only after the data transfer for its previous request is completed and transferred to the MTL TxQ and space is available in TxQ to take in the next request.
- The 25GMAC can be programmed to control the maximum number of outstanding requests, Requested Burst sizes by DMA, and the burst sizes of each data transfer on AXI.

Considering these, 25GMAC can be programmed such that the requests of all the TxDMA are placed and executed on the AXI bus before the data transfer of the first TxDMA is completed and it raises the next request to the arbiter.

For example, in a 128-bit AXI data bus, set the following:

1. TxDMA PBL = 64 (1024 bytes)
2. Number of Outstanding AXI Read requests = 32

3. Maximum Burst Length on AXI (BLEN) = 32 (512 bytes)
4. Number of active TxDMA = 8

Assuming that all the TxDMA place a request simultaneously and get serviced by the fixed priority algorithm, all the requests are placed on the AXI bus in a window of 32 clocks (2 clocks per AXI request, 2 AXI burst requests per TxDMA request). The data transfer of the first request takes 32 clock cycles and the read-access latencies (safely assume tens of cycles). So the TxDMA arbiter is already IDLE and has serviced all the TxDMA requests before this data transfer completes.

Effectively, all the TxDMA channels get equal access to AXI bus and the corresponding TxQs are filled up at the same rate. The data gets read out from the TxQs as per the Traffic Class scheduler in the MTL. So at a steady state (assuming that AXI bandwidth available to fetch data from system memory is always more than the 25GMAC operating line rate), the bandwidth for each channel is controlled by the priorities and settings of the Traffic Scheduler.

6.2.1.1 WRR Arbitration Support for AXI Data-Reads on TxDMA

The Weighted Round Robin arbitration is supported for Transmit DMA channel read access to AXI Read Data Channel. The weights corresponding to each Transmit DMA Channel is programmed in the indirect registers using the DMA_CH_Ind_Control and DMA_CH_Ind_Data registers. The default weights in the indirect register correspond to Round Robin arbitration.

6.2.2 Receive Path

The 25GMAC supports up to 16 Rx channels in the XGMAC-AXI configurations. You can select the number of Rx channels by using the Number of DMA Channels on Receive Side option. In the Rx direction, the MTL Rx Controller selects the RxDMA for which it is transferring or reading the data from the Rx FIFO memory. This scheduling is based on the programming done in the respective MTL_RxQ[x]_Control register.

Each RxDMA indicates when it is ready to transfer data and the size of the burst-length (number of beats) that is has to transfer. The scheduler checks whether sufficient data (of requested burst length) is available to be transferred to these DMAs and then selects the RxDMA that gets serviced using the programmed priorities.

6.2.3 Priority Scheme for TxDMA and RxDMA

The DMA arbiter performs the fixed priority arbitration between the Tx and Rx paths of DMA channels for accessing descriptors and data buffers. Descriptor accesses of a Tx or Rx channel is given higher priority when requests from both are active together.

6.3 Multiple Queues Support in the MAC

You can enable or disable a Tx Queue by programming the TXQEN field MTL_TxQ(#i)_Operation_Mode register of corresponding queue. When you select multiple Tx queues without selecting the DCB/AV feature, the queues are scheduled for generic queuing using WRR or WSP algorithms.

When you select the Enable Data Center Bridging feature, DCB is enabled for all selected Tx queues and scheduling is based on WRR, WSP, DWRR, or WFQ algorithms.

To enable a Tx queue for AV, select the Enable Audio Video Bridging option and then the Enable Support for AV in Tx Queue # option. The queuing is based on the CBS or SP algorithms.

You can enable or disable a Rx Queue by programming the RXQ#EN field of corresponding queue in the MAC_RxQ_Ctrl0 register. The following list explains how Rx queues are enabled based on the selected features:

1. Received tagged packets can be routed to any of the Rx Queue from Q0 to Q7.
2. Each queue has a corresponding field (PSRQ fields in the MAC_RxQ_Ctrl2 or MAC_RxQ_Ctrl3 registers) by which you can indicate the packets of specific VLAN priority values that must be routed/pushed into that queue.
3. Ensure that each priority value can be routed to a specific queue within the set of AV/DCB enabled queue. In other words. bits corresponding to the same priority value must not be set in multiple PSRQx fields within the AV/DCB set enabled queues, but you can route multiple priorities into the same queue.
4. Other types of packets such as un-tagged packets, multicast packets, and so on, are routed to specific Rx queues as programmed in the MAC_RxQ_Ctrl1 register. Because the MAC supports only 4 types of other packets, a maximum of 16 RxQ is possible, as there are only 8 values for VLAN tag priority.
5. When DCB feature is enabled, the value programmed in the PSRQ fields of the respective RxQ is used to populate the Priority Enable Vector field of the PFC frame that is generated and transmitted when the flow control is triggered in that Rx Queue. Therefore, only Rx Queue0 to Rx Queue7 can be used for PFC based flow control.
6. When PFC-based flow control is disabled (or DCB feature is not selected), then a PAUSE frame is generated when the flow control is triggered in any of the Rx Queues.
7. When you select the Enable Audio Video Bridging option, AV is enabled for all the selected Rx queues. The queuing is done based on the VLAN Tag priority for AV packets. The VLAN Tag priority must match the PSRQ field of the MAC_RxQ_Ctrl2 and MAC_RxQ_Ctrl3 registers. The AV control packets (tagged or untagged) are routed based on the AVCPQ field of the MAC_RxQ_Ctrl1 register. The PTP over Ethernet packets are routed based on the AVPTPQ field of MAC_RxQ_Ctrl1 register.

6.4 Multiple Queues Support in the MAC Transaction Layer

The 25GMAC supports up to 16 queues in the Tx direction and up to 16 queues in the Rx direction. The number of Tx queues and Rx queues can be configured independently.

In the XGMAC-AXI configurations, the number of Tx queues is equal to the number of DMA channels but the number of Rx queues can be different than the number of RxDMA channels.

You can select the number of Tx queues in the Number of Transmit Queues option. You can control a Tx queue through `MTL_TxQ(#i)_Operation_Mode` register. You can select the number of Rx queues by using the Number of Receive Queues option. You can control an Rx queue through `MTL_RxQ(#i)_Operation_Mode` register.

6.4.1 Queue Memory

All MTL Tx queues share one Tx memory and all Rx queues share one Rx memory with programmable individual queue sizes in blocks of 256 bytes. The application can program the Tx queue size in the `TQS` field of `MTL_TxQ(#i)_Operation_Mode` register and the Rx queue size in the `RQS` field of the `MTL_RxQ(#i)_Operation_Mode` register.

The 25GMAC supports the total Tx memory size of up to 256 KB and total Rx memory size of up to 256 KB. The total memory size can be selected in powers of 2, that is, 4 KB, 8 KB, 16 KB, 32 KB, 64 KB, 128 KB, and 256 KB. In configurations with a single queue, the maximum Tx memory size is 32 KB and the maximum Rx memory size is 64 KB.

6.4.2 Traffic Classes and Transmit Queues

The 25GMAC controller supports independent configuration of transmit queues and traffic classes (TC), where you can select up to 8 traffic classes and up to 16 transmit queues. You can program the bandwidth required for each traffic class by programming the `MTL_TCn_Quantum_Weight` register corresponding to a traffic class.

A queue must be mapped to a traffic class by programming the `Q2TCMAP` field in the `MTL_Txn_Operation_Mode` register for a particular queue. If the number of configured transmit queues is more than traffic classes, then one or more queues can be mapped to a traffic class.

Note	The number of transmit queues is always greater than or equal to the number of traffic classes.

The scheduling between traffic classes is determined by the scheduling algorithm implemented in the transmit scheduler.

Note	All Queues within a traffic class are selected in a round robin fashion (when packets are available) when the traffic class is selected by the scheduler for packet transmission. This is true for any of the scheduling algorithms.

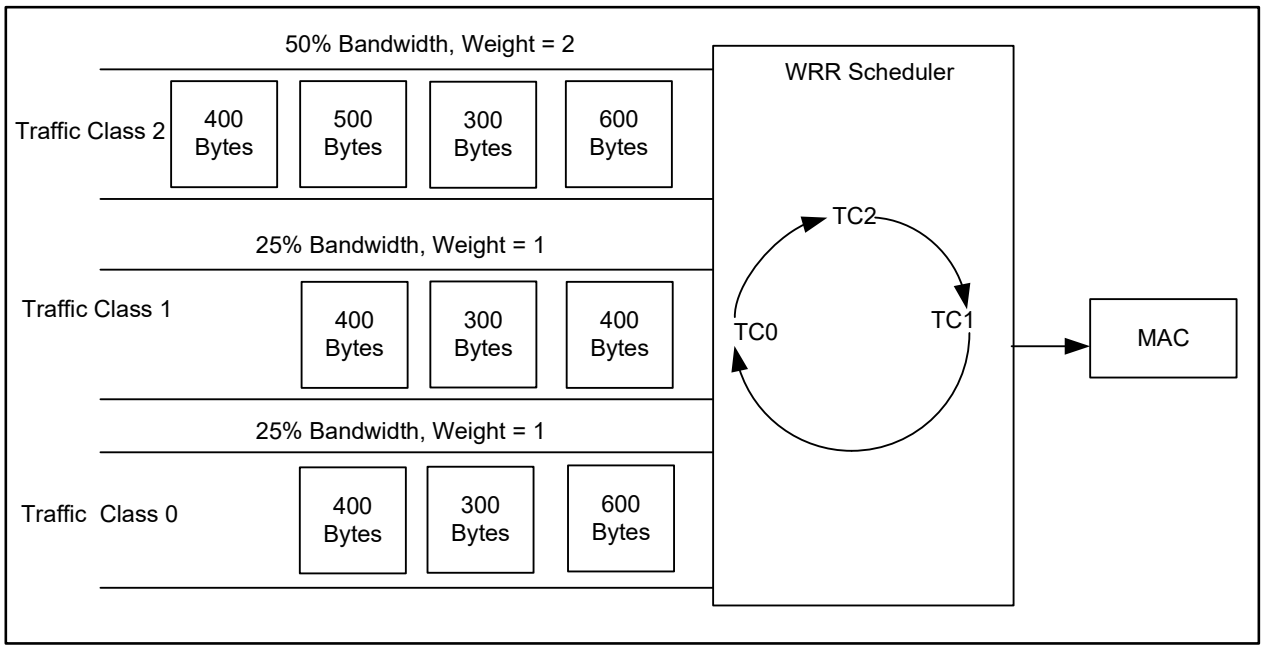
When Data Center Bridging is not selected, the following scheduling algorithms are supported:

- “Weighted Round Robin” on page 323
- “Strict Priority” on page 324

6.4.2.1 Weighted Round Robin

In Weighted Round Robin (WRR) algorithm, each traffic class is assigned a weight based on the percentage of configured bandwidth. All traffic classes are serviced in the round-robin order according to the programmed weights. This algorithm is less complex and requires less hardware as compared to other algorithms. Large packets get more bandwidth in this algorithm.

Figure 6-2 Weighted Round Robin Example



In Figure 6-2, Traffic Class 2, Traffic Class1 and Traffic Class 0 are configured for 50%, 25%, and 25% bandwidth, respectively. The corresponding weights are 2, 1, and 1. Before the WRR scheduler is started, Traffic Class 2 has four packets; Traffic Class 1 has three packets; and Traffic Class 0 has three packets. The packets are transmitted in the following order:

Table 6-2 Packet Transmission Order

Packet Sequence	Traffic Class Number
Packet 1	Traffic Class 2
Packet 2	Traffic Class 2
Packet 3	Traffic Class 1
Packet 4	Traffic Class 0
Packet 5	Traffic Class 2
Packet 6	Traffic Class 2
Packet 7	Traffic Class 1
Packet 8	Traffic Class 0

Table 6-2 Packet Transmission Order (continued)

Packet Sequence	Traffic Class Number
Packet 9	Traffic Class 1
Packet 10	Traffic Class 0

6.4.2.2 Strict Priority

In the strict priority algorithm, Traffic Class 0 has the lowest priority and the priority increases with the Traffic Class number. For example, in a 4-TC configuration, TC3 has the highest priority, followed by TC2, TC1, and finally TC0.

The highest priority Traffic Class is serviced first. If there is no packet in the highest priority Traffic Class, then the next priority Traffic Class is serviced.

6.4.3 Data Center Bridging

The 25GMAC supports the Data Center Bridging (DCB) feature. You can enable this feature by selecting the Enable Data Center Bridging option while configuring the controller. Priority-based flow control is supported when DCB is enabled. In the transmit side, up to eight traffic classes are supported, which can be mapped to one of the eight priorities. You can map one or more priorities to a traffic class by programming the `MTL_TC_Prty_Map0` and `MTL_TC_Prty_Map1` register. The following additional scheduling algorithms are supported when the DCB feature is enabled:

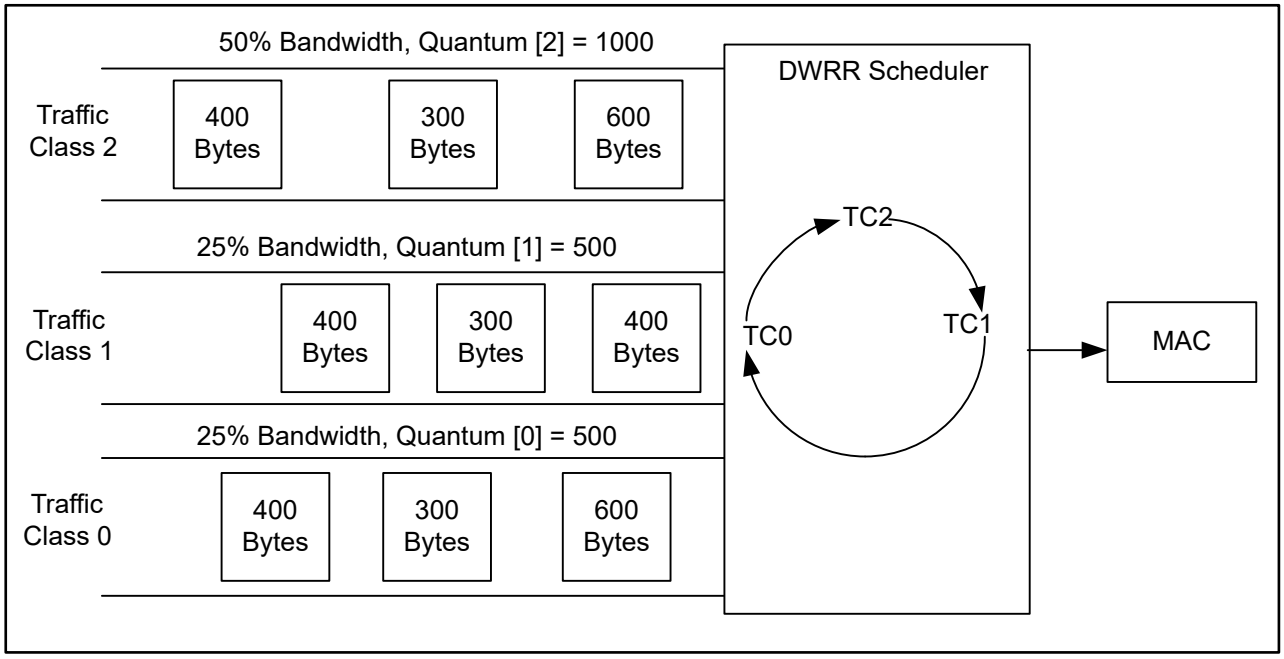
- "Deficit Weighted Round Robin" on page 324
- "Weighted Fair Queuing" on page 326

The application must put packets with correct priorities in the respective programmed queue on the Transmit side. The controller uses the programmed priorities for blocking the transmit queue when a PFC packet is received. If multiple priorities are assigned to a traffic class, PFC is enabled, and one or more priorities assigned to a traffic class are paused, then all queues in the traffic class are paused. Similarly the same priority can be assigned to more than one Traffic Class. In this case, all the Traffic Classes with that particular priority are paused when that priority is paused.

6.4.3.1 Deficit Weighted Round Robin

In Deficit Weighted Round Robin (DWRR) algorithm, each Traffic Class is assigned a weight based on the percentage of configured bandwidth. A deficit counter holds the credit for transmission. The quantum value, proportional to the bandwidth, is added to the deficit counter every time a Traffic Class is scanned. The packet in the Traffic Class is selected for transmission only if the credit in the deficit counter is greater than or equal to the size of the packet available at the head of the Traffic Class. A selected Traffic Class is serviced till the outstanding credit is lesser than the packet size at the head of the Traffic Class during a particular Traffic Class scan cycle. After this, the next lower priority Traffic Class is serviced. When the packets in a Traffic Class are over, the credit becomes zero for that Traffic Class scan cycle. This algorithm is less complex, and it provides more fairness in scheduling as compared to other algorithms.

Figure 6-3 Deficit Weighted Round Robin Example



In Figure 6-3, Traffic Class 2, Traffic Class 1, and Traffic Class 0 are configured for 50%, 25%, and 25% bandwidth, respectively. The corresponding quantum values are 1000, 500, and 500. Before the DWRR scheduler is started, each Traffic Class has three packets. Table 6-3 shows the Traffic Class scan cycle number and the packet size at the head of Traffic Classes, deficit counter value before and after packet in selected Traffic Class is transmitted, and Traffic Class scheduled for transmission.

Table 6-3 Deficit Weighted Round Robin

Traffic Class Scan Cycle Number	Packet Size at the Head of Traffic Class			Deficit Counter Value Before the Packet is Transmitted in Selected Traffic Class			Deficit Counter Value after the Packet is Transmitted in Selected Traffic Class			Traffic Class Scheduled for Transmission
	Q2	Q1	Q0	Q2	Q1	Q0	Q2	Q1	Q0	
1	600	400	600	1000	500	500	400	500	500	Q2
	300	400	600	400	500	500	100	500	500	Q2
	400	400	600	100	500	500	100	100	500	Q1
	400	300	600	100	100	500	100	100	500	None
2	400	300	600	1100	600	1000	700	600	1000	Q2
	N/A	300	600	0	600	1000	0	300	1000	Q1
	N/A	400	600	0	300	1000	0	300	400	Q0
	N/A	400	300	0	300	400	0	300	100	Q0

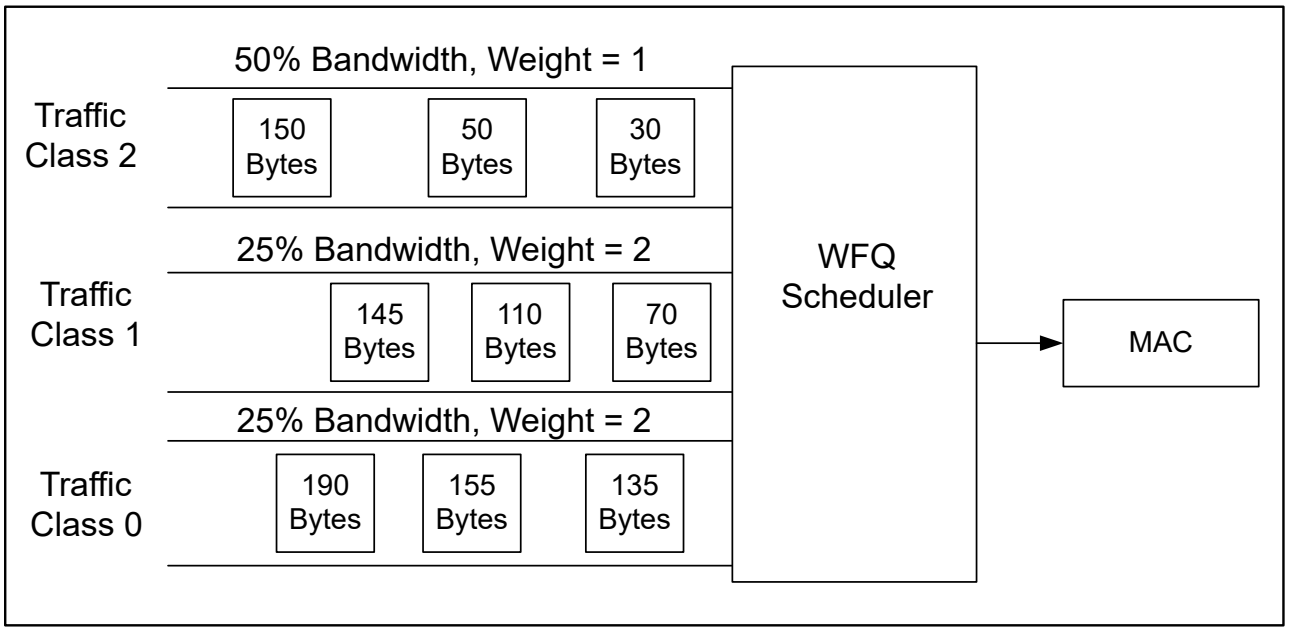
Table 6-3 Deficit Weighted Round Robin (continued)

Traffic Class Scan Cycle Number	Packet Size at the Head of Traffic Class			Deficit Counter Value Before the Packet is Transmitted in Selected Traffic Class			Deficit Counter Value after the Packet is Transmitted in Selected Traffic Class			Traffic Class Scheduled for Transmission
	N/A	400	400	0	300	100	0	300	100	None
3	N/A	400	400	0	800	600	0	400	600	Q1
	N/A	N/A	400	0	N/A	600	0	0	200	Q0
	N/A	N/A	N/A	0	N/A	N/A	0	0	0	None

6.4.3.2 Weighted Fair Queuing

In Weighted Fair Queuing (WFQ) algorithm, each Traffic Class is assigned a weight based on the percentage of configured bandwidth. The algorithm computes the finish time of packets at the head of the Traffic Classes based on the packet size and weights for the Traffic Class. The packets with earliest finish time is scheduled for transmission first. The algorithm is more complex and less scalable as compared to other algorithms.

Figure 6-4 Weighted Fair Queuing Example



In Figure 6-4, Traffic Class 2, Traffic Class 1, and Traffic Class 0 are configured for 50%, 25%, and 25% bandwidth, respectively. The corresponding weights are 1, 2, and 2. Before the WFQ scheduler starts, each Traffic Class has 3 packets. Table 6-4 shows the packet size at the head of the Traffic Classes, computed finish time (packet size * weight) and Traffic Class scheduled for transmission.

Table 6-4 Weighted Fair Queuing

Packet Size at the Head of Traffic Class			Computed Finish Time in Clock Cycles (Packet Size * Weights)			Traffic Class Scheduled for Transmission
TC2	TC1	TC0	TC2	TC1	TC0	
30	70	135	30	140	270	TC2
50	70	135	50	140	270	TC2
150	70	135	150	140	270	TC1
150	110	135	150	220	270	TC2
N/A	110	135	N/A	220	270	TC1
N/A	145	135	N/A	290	270	TC0
N/A	145	155	N/A	290	310	TC1
N/A		155	N/A	N/A	310	TC0
N/A		190	N/A	N/A	380	TC0

6.4.4 Audio Video Bridging

6.4.4.1 Overview of Audio Video Feature

25GMAC supports the AV data transfer in all speed modes. The AV feature enables transmission of time-sensitive traffic over bridged local area networks (LANs). The following standards define various aspects of the AV feature implementation:

- IEEE 802.1Qav-2009:

  Allows the bridges to provide time-sensitive and loss-sensitive real-time audio video data transmission (AV traffic). It specifies the priority regeneration and controlled bandwidth queue draining algorithms that are used in bridges and AV traffic sources.

- IEEE 802.1Qat-2009:

  Allows the network resources to be reserved for specific traffic streams traversing a bridged local area network.

- IEEE 802.1AS-2011:

  Specifies the protocol and procedures used to ensure that the synchronization requirements are met for time-sensitive applications such as audio and video across bridged and virtual-bridged LANs consisting of LAN media where the transmission delays are fixed and symmetrical. For example, IEEE 802.3-2018 full-duplex links include the maintenance of synchronized time during normal operation followed by addition, removal, or failure of network components and network reconfiguration.

Note In the XGMAC-CORE configurations, the audio-video feature is limited to identifying the AV Type payloads in the Ethernet packet and indicating it on the MAC Receive Interface (MRI) through mri_q_sel_o signal along with the SOP.

For more information, see “Receive Path Functions” on page 215.

6.4.4.2 Transmit Path Functions

The Transmit path of Queue 0 supports the strict priority algorithm, and it is used for best-effort traffic. For a queue, the strict priority algorithm determines that a packet is available for transmission if the queue contains one or more packets. When the threshold mode for MTL Tx FIFO is enabled, the strict priority algorithm determines that a packet is available for transmission if the queue contains a partial packet of size equal to the programmed threshold limit.

The Transmit paths of additional queues support traffic management by using the credit-based shaper algorithm. For a queue, the credit-based shaper algorithm determines that a queue is available for transmission if the following conditions are true:

- The queue contains one or more packets.
- The credit for the queue is positive as per the algorithm.

You can disable the credit-based shaper algorithm for all queues or lower-priority queues. For example, you can either disable the credit-based shaper algorithm for Queue 1 and Queue 2 or only for Queue 1. You must not disable the credit-based shaper algorithm for Queue 2 and enable it for Queue 1. When you disable the credit-based shaper algorithm for a queue, the channel uses the default strict priority algorithm.

Each Transmit DMA has a separate descriptor chain for fetching the transmit data. The Transmit channel that gets the access to the system bus depends on the DMA arbiter.

The Transmit path has a shared FIFO (MTL layer) for each queue, as shown in Figure 6-1. The data fetched by the DMA is put in the respective part of the FIFO. The MTL Tx Queue Scheduler controls which part of the FIFO data is transmitted by the MAC. If the credit-based shaper algorithm is enabled for a queue, the corresponding queue is selected for transmission if the following conditions are true:

- If the packet is available in the channel and has a positive or zero credit.
- If the higher priority queue has no packet waiting in the FIFO.

If the credit-based shaper algorithm is disabled for all queues, the packet to be transmitted from a queue is selected based on the priority scheme described in Figure 6-2.

6.4.4.3 Receive Path Functions

When you select the AV feature, the Receive path of Queue 0 is enabled by default. All traffic is received on this channel. You can enable the Receive paths of additional queues. By enabling the Receive paths of multiple channels, you can demultiplex the received data to send the packets into separate receive channels.

To differentiate between the AV and non-AV traffic, the MAC provides a status that indicates if it is an AV packet and its corresponding VLAN Priority tag value. This status is updated in the Extended Status field of the Receive descriptor as explained in "Receive Descriptor" on page 1811. All received packets with the EtherType field of 0x22F0 are detected as AV packets. The AV packets can be of the following two types:

- AV data packets: The AV data packets are always tagged. The tagged AV data packets are received based on the programmed priority value. To specify the channel to which an AV packet with a given priority must be sent, program Bits[15:8] in the Transmit Flow Control Register of the corresponding queue.
- AV control packets: The AV control packets can be either tagged or untagged. The untagged AV control packets are received on Queue 0 by default. To receive these packets on any other queue, you can program Bits[2:0] of the `MAC_RxQ_Ctrl1` register. Similar to the AV data packets, the tagged AV control packets are received based on the programmed priority value.

In addition to the AV packets, you can receive the untagged PTP packets on any queue. By default, the PTP packets (tagged or untagged) are received on Queue 0. To receive these packets on any other queue, you need to program Bits[6:4] of the `MAC_RxQ_Ctrl1` register.

6.4.4.4 Credit-Based Shaper Algorithm

The MTL Queue Scheduler uses the credit-based shaper algorithm to arbitrate the AV traffic in all queues and the legacy Ethernet traffic in Queue 0. You can program the additional queues to use the credit-based shaper algorithm.

The following sections provide information about how you can implement the credit-based shaper algorithm:

- "Credit Value" on page 329
- "idleSlopeCredit and sendSlopeCredit Values" on page 329
- "Bandwidth Status" on page 330

6.4.4.5 Credit Value

The credit value is accumulated every transmit clock cycle. The credit to be added or subtracted per cycle can be fractional based on the required idleSlope and sendSlope values. The credit for a TC builds up only when the packet is available but it cannot be transmitted because the MAC is sending a packet from another TC. The 25GMAC supports a mode (CC field of ETS control register) in which the credit can build up in advance for a TC which has no outstanding packets. This enables sending a burst of high priority traffic in a TC as soon as the data is available. When the CC field is reset, the credit accumulator in the credit-based shaper algorithm is set to zero if there is positive credit, and there is no packet to transmit in a TC. The credit does not accumulate when there is no packet waiting in a queue and other queues are transmitting. When set, the credit accumulator in the credit-based shaper algorithm is not reset to zero if there is positive credit and no packet to transmit in a queue. The credit accumulates even when there is no packet waiting in a queue and other queues are transmitting.

6.4.4.6 idleSlopeCredit and sendSlopeCredit Values

The software must program the idleSlopeCredit and sendSlopeCredit values. The programmed values must be the credit accumulated or drained per clock cycle scaled by 1024, such as, 320 * 1024 = 327680 and 480 * 1024 = 491520. In addition, the software must program the hiCredit and loCredit values, scaled by 1024, to adjust for scaling of the idleSlopeCredit and sendSlopeCredit values.

Programming example for idleSlope, sendSlope, hiCredit, and loCredit

Consider a case where 3 TCs are in use (TC0, TC1, and TC2) and of which:

- TC2 [AV enabled] has 45% bandwidth allocated
- TC1 [AV enabled] has 35% bandwidth allocated and the remaining bandwidth is assigned to TC0
- TC0 [Best Effort] gets the left over bandwidth of 20%

Consider 1500 bits for hiCredit and 800 bits for loCredit programming

TC2 Programming

idleSlope: (45/100) * 32 * 1024 = 14746 (45 is the bandwidth, 32 since MAC uses a 32 bit interface, and 1024 is the scaling factor)

sendSlope: ((100-45)/100) * 32 * 1024 = 18022 (100- bandwidth must be used to compute sendSlope)

hiCredit: ((100-45)/100) * 1500 * 1024 = 844800

loCredit: ((100-45)/100) * -800 * 1024 = -446600

TC1 Programming

idleSlope: (35/100) * 32 * 1024 = 11469

sendSlope: ((100-35)/100) * 32 * 1024 = 21299

hiCredit: ((100-35)/100) * 1500 * 1024 = 998400

loCredit: ((100-35)/100) * -800 * 1024 = -532480

The 2's complement value must be programmed for loCredit field because it is always a negative value.

6.4.4.7 Bandwidth Status

The hardware maintains the status of the actual bandwidth consumed by each higher priority queue in the CBS status registers. This enables the software to estimate the average bandwidth consumed by numerically higher traffic classes as compared to the reserved bandwidth.

The CBS status register gives the average number of bits transmitted during the previous programmed slot interval (1, 2, 4, 8, or 16 slots of 125 µs) in a queue. The status register is updated even if the credit-based shaper algorithm is not enabled for a queue. The number of slots over which the average bits transmitted per slot are computed is programmed in Bits[6:4] of the CBS control register of the respective queue. For example, if you have programmed two slots, the average bits are computed over slot numbers 0–1, 2–3, 4–5, and so on.

The value programmed in the idleSlopeCredit register of a queue is proportional to the bandwidth reserved for the queue. The software can allocate any bandwidth that is not used by the higher priority queue to the reserved bandwidth of the lower priority queue.

A lower priority queue, which is using the credit-based shaper algorithm, cannot use the unused reserved bandwidth of any higher priority queue that is using the credit-based shaper algorithm. However, a lower priority queue, which is using the strict-priority algorithm, can use the unused reserved bandwidth of any higher priority queue that uses the credit-based shaper algorithm. For example, Queue 1 and Queue 2 use the credit-based shaper algorithm (with reserved bandwidth of 50% and 25%, respectively) and Queue 0 uses the strict-priority algorithm. If Queue 1 uses only 40% of reserved bandwidth, the remaining 10% is used by Queue 0. Queue 2 cannot exceed the reserved bandwidth of 25%.

6.4.4.8 Enabling Audio Video Bridging

To enable this feature while configuring the controller in the coreConsultant GUI:

1. Select the Enable Audio Video Bridging option under the Multiple Queues Support section during the Specify Configuration activity in coreConsultant.
2. Select the Enable Support for AV in Tx Queue x options (x = 1 to 7) under the Multiple Queues Support section during the Specify Configuration activity in coreConsultant.

For details about these option, see the *Parameter Descriptions* chapter.

See the “Enabled” field of the parameter descriptions to understand the dependencies.

6.4.4.9 Slot Number Function With Audio Video Bridging

Introduction to Slot Number Function

When the AV feature is enabled, you can use the slot number function to schedule the data fetching from the system memory by the DMA. This feature is useful when the source AV data needs to be transmitted at specific intervals.

You can program the slot number at which the DMA must fetch the data from system memory in the TDES3. This 4-bit field allows the application to schedule data up to 16 slots of 125 each. This field is applicable only for the AV channels.

Description of Slot Number Function

When the DMA fetches a Tx descriptor, it compares the slot number of the Tx descriptor with the internally generated reference slot interval. The slot interval is a counter that is updated every 125 micro seconds of the IEEE 1588 system time. In addition, the slot interval counter is initialized to zero when the seconds field of the system time is incremented, that is, the sub-second counter rolls over. The DMA fetches the data only if it matches the current slot or the next slot. The DMA remains in the descriptor fetch state till there is a match.

To enable the DMA to fetch the data only if it matches the current slot or the next two slots, you can program Bit 1 of the Slot Function Control and Status register of the corresponding DMA channel.

If the slot number in the descriptor is less than the reference slot number, the DMA takes it as a future slot.

You can enable the check for slot number by setting Bit 0 of the Slot Function Control and Status register of corresponding DMA channel. When this check is not enabled, the packets are fetched immediately after the descriptor is read. In addition, Bits [19:16] indicate the value of the reference slot number in DMA.

6.4.4.10 Programming Guidelines for Initializing the DMA

“Initializing the DMA Registers” on page 1843.

6.4.4.11 Programming Guidelines for Enabling Slot Number Checking

“Enabling Slot Number Checking” on page 1880.

6.4.4.12 Programming Guidelines for Enabling Average Bits Per Slot Reporting

“Enabling Average Bits Per Slot Reporting” on page 1880.

6.4.4.13 Programming Guidelines for Disabling Flow Control in AV Queues

The 25GMAC supports programming of same priorities for AV and DCB enabled Rx queues, when both coexist. For example, priority 1 can be programmed in PSRQ[n] field corresponding to Rx Queue enabled for AV in `MAC_RxQ_Ctrl2` or `MAC_RxQ_Ctrl3` register and same priority 1 can be programmed in PSRQ[n] field corresponding to Rx Queue enabled for DCB in `MAC_RxQ_Ctrl2` or `MAC_RxQ_Ctrl3` register, at the same time. When AV and DCB operation coexists, it requires that the Flow Control is disabled for AV enabled queues.

See "Disabling Flow Control for AV Enabled Queues" on page 1880

6.5 Tx Side Arbitration Between DMA and MTL

As the number of TxDMA channels is always equal to the number of Tx Queues in the MTL, they are always mapped directly. For instance, each TxDMA pushes data into its respective Tx Queue assigned to it. The main reason for this strategy is to optimize and increase the efficiency of the DMA data transfer, as well as to simplify the Tx checksum engines in the MTL.

The data inside each Tx Queue is stored in packets. Hence, if two DMAs are allowed to transfer data into the same queue, when a TxDMA starts a packet transfer, the other DMA cannot transfer data unless the previous packet is completely pushed-in. This means that the second DMA remains idle until the first packet is transferred. Therefore, each DMA is always connected directly to its corresponding Tx Queue.

6.6 Rx Side Arbitration Between MTL and DMA

After completing the current packet processing, the DMA Channel controller fetches the next descriptor. Once the descriptor fetching is complete, it evaluates the amount of data to be transferred to the Rx Buffer based on the programmed PBL and Rx Buffer Length, and accordingly requests the MTL to transfer the data.

After servicing the current request, the MTL Rx arbiter selects a Rx Queue based on the arbitration scheme (RAA field of the MTL_Operation_Mode register) and the weights programmed in the `MTL_RxQn_Control` Register. Arbitration is done among Queues that has the packet to be transferred to a DMA, which is ready to transfer data. For example, if the DMA is in the process of fetching or closing a descriptor, it is not in the "ready" state. It is ready to transfer data only after it reads in a valid descriptor with non-empty buffer locations. Once the Rx Queue is selected, a complete frame/`PBL` (programmable burst length) amount of data (based on the `RXQ_FRM_ARBIT` field of the `MTL_RxQn_Control` register) is read out from that Queue and routed to the RxDMA Channel based on the Rx Channel selection criteria defined in the section "Rx Queue to DMA Mapping" on page 335.

6.7 Rx Queue to DMA Mapping

The packets in the Rx Queue can be routed to any one of the multiple DMA channels by programming the following registers:

- MTL_RxQ_DMA_Map0 Register (for queues 0, 1, 2 and 3)
- MTL_RxQ_DMA_Map1 Register (for queues 4, 5, 6 and 7)
- MTL_RxQ_DMA_Map2 Register (for queues 8, 9, 10 and 11)

Two types of Rx Queue to DMA mapping is possible through programming.

6.7.1 Static Mapping

In this mode, all the packets of a Rx Queue is connected to a specific DMA channel. For example, all the packets from Rx Queue 0 can be routed to a DMA channel by programming Q0MDMACH (bit[3:0]) and Q0DDMACH (bit[7] = 0) of the MTL_RxQ_DMA_Map0 Register.

Similarly, packets from other Rx Queues can be routed to any DMA channel by programming register fields corresponding to each Queue.

6.7.2 Dynamic (per packet) Mapping

In this mode, the destination DMA channel of a packet being read from a Rx Queue is not constant but decided independently for each packet. For example, if the Q1DDMACH bit of the MTL_RxQ_DMA_Map0 register is set, the static mapping is disabled for Rx Queue 1 and the value in Q1MDMACH is ignored. The destination DMA channel is decided by the controller receiver for each packet, depending on the following, in the decreasing order of priority

6.7.2.1 L3-L4 filter Based DMA Selection

When the Enable Layer 3 and Layer 4 Packet Filter feature is present and enabled in your configuration, the TCP/UDP and IP header fields of the received packet are matched against the corresponding values programmed and enabled for comparison in the MAC_L3_L4_Control register. If the match is successful, the DMA channel number programmed in the DMCHN field of the MAC_L3_L4_Control register is selected as the destination DMA channel number only if DMCHEN = 1 and DMCHLE = 0 of the same register. However, if DMCHEN = 1 and DMCHLE = 1, the DMCH field contains the Receive channel list number for packet duplication.

If none of the L3-L4 Registers give a comparison match, the next category as given in"RSS-Based DMA Selection", makes the DMA channel selection.

6.7.2.2 RSS-Based DMA Selection

When the Enable Receive Side Scaling feature is present and enabled in your configuration, the RSS hash engine used the L3/L4 fields of the received packets to search in the RSS Lookup table. The DMA channel number programmed in the corresponding location is the destination DMA.

Based on the RSS lookup table size that you select using the DWCXG_RSS_TABLE_SIZE parameter, 25GMAC uses either internal or external RSS lookup.

External RSS lookup selects the DMA channel number based on the DMCHB, DMCHLE, and DMCH fields of the selected lookup entry, whose result is pass. If

- DMCHB
  - 0: Routing is based on RSS lookup
  - 1: RSS does not select or specify a DMA number. DMA selection is left to others; VLAN or DA/SA filtering.
- DMCHLE
  - 0: DMCH field contains the binary value of the DMA receive channel number.
  - 1: Selects the Receive channel list number for packet duplication.

The queuing is decided by the VLAN priority of tagged frames or other Layer 2 "Type" field factors as the "queuing" decision is taken before the RSS result is available.

The routing of a received packet into a specific queue is independent of "Receive Side Scaling Function" on page 274. When the RSS function gives the RSS hash results, DMA channel number associated with this packet, and other OoS status bits after an unspecified latency delay (depends on the length of the TCP/IP header), these are stored and tagged along with the packet being written to the RxQueue.

Note If the RxQ is programmed to operate in cut-through mode, then the threshold to start the reading of the packet is triggered only after the DMA channel number is received from the MAC receiver and if the programmed threshold is already crossed.

When this packet is available at the top-of-the RxQueue, the controller first reads the tag and determines the DMA Channel Number and then forwards the RSS hash and other status along with the packet content to that DMA. The RxDMA transfers the packet content to the host memory buffers and updates the Rx Descriptor with the RSS hash and other status.

6.7.2.3 Extended VLAN Based DMA Selection

When you enable the Extended RX VLAN Filtering and Routing feature in your configuration, the MAC uses the VLAN Tags of the received frame to determine the destination DMA Channel number.

If none of the VLAN filters result in a comparison match, the next category, "Ethernet DA-Based DMA Selection" on page 337, makes the DMA Channel selection.

Routing is applicable only if the entire VLAN Filter has passed. Routing is only based on Perfect Filter Result. Each perfect filter has a DMA Channel Enable and a DMA Channel Number field which can be programmed. Only if the DMA Channel Enable bit is set, routing is applicable for that filter.

The frame is routed to the DMA Channel of the smallest Matching Filter, if enabled. If the DMA Channel number of the filter is not enabled, the frame gets routed to Channel 0. For example, if the VLAN tag of frame matches filters 7, 3, and 1, the MAC checks if the DMA Channel Number of Filter 1 is enabled through programming. If yes, the frame is routed to the programmed value; otherwise it is routed to the DA filter based DMA channel or to channel 0. When the inverse filter is enabled; it is routed to the DMA channel number of the least mismatched filter, if enabled.

If Hash Filter is also enabled, it is used to determine the Filter result only. For non-tunneled packets, routing depends on the enabled Perfect Filters. If none of the perfect filters are enabled or if all of them are bypassed, the VLAN Filter based routing does not take place. If all the perfect filters give a fail result and the Hash Filter has passed, the VLAN Filter result is a pass but routing is based on DA Based Addressing or to Channel 0. Similar behavior is seen when inverse Filtering is enabled as well.

If VXLAN/NVGRE support is enabled, for tunnel frames, filtering is applicable but not routing.

The overall VLAN Filter is produced and it decides if the frame can be dropped or forwarded. If the frame gets a fail result from the VLAN Filter, then it is dropped if VTFE = 1 & RA = 0.

RDES0 cannot be used to give the outer/& inner VLAN Tags of the outer header. If stripping is enabled, the matching tag value is lost as it cannot be passed through the status. The application needs to be cautious before enabling stripping.

6.7.2.4 External Rx VLAN Tag Lookup-Based DMA Selection

When you enable External Rx VLAN Tag Lookup-based Filtering and Routing feature in your configuration, the MAC uses the VLAN Tags of the received frame to determine the destination DMA Channel number.

If Rx VLAN Lookup is unable to provide a DMA selection, the next category, Ethernet DA-Based DMA Selection, performs the DMA Channel selection.

Routing is applicable only if the final VLAN filtering result is Pass and VLAN perfect filter (External VLAN Lookup based filtering) result is either Pass or Fail. Only the VLAN perfect filter has the capability of DMA selection. The external RxVLAN lookup selects the DMA channel number based on the RCHLE and RCH fields of the selected lookup entry, as follows:

- RCHLE is 0: RCH field contains the binary value of the DMA receive channel number.
- RCHLE is 1: Uses the Receive channel list number for packet duplication.

If VLAN Hash Filter is also enabled, it is used to determine the Filter result only. For non-tunneled VLAN frames, routing depends on the VLAN perfect filter. If the VLAN perfect filter-based DMA selection is not available, routing is based on DA Based Addressing or it is routed to Channel 0.

6.7.2.5 Ethernet DA-Based DMA Selection

The DA address of the received packet is compared against the programmed DA values in MAC Address Registers. If the address matches any of the programmed values, the corresponding DCS field determines the destination DMA channel number.

When the DA based address comparison is unable to determine the DMA selection, the next category, external DA hash table-based DMA Selection determines the DMA Channel selection, if enabled and configured.

The external DA hash table selects the DMA channel number based on the `RCHLE` and `RCH` fields of the selected hash table entry. If the `RCHLE` field is 0, the `RCH` field contains the destination channel number as a binary value. Otherwise, the `RCH` field contains the receive channel list number for packet duplication.

If none of the previous operations are able to make a successful match/decision, the packet is routed to DMA Channel 0, by default. A summary of the filtering result and the DMA channel selection in this mode is shown in Table 6-5.

Table 6-5 L3L4 RSS Filter Results

L2/VLAN Filter Match[a]	MAC_Packet_ Filter [IPFE]	L3/L4 Filter Match[b]	RSS Match[c]	Action on Packet[d]	DMCHEN[e]	DMA Channel Assigned
1	X	Fail	Pass	Forward	X	RSS-DMCH[f]

Table 6-5 L3L4 RSS Filter Results (continued)

L2/VLAN Filter Match[a]	MAC_Packet_ Filter [IPFE]	L3/L4 Filter Match[b]	RSS Match[c]	Action on Packet[d]	DMCHEN[e]	DMA Channel Assigned
1	0	Fail	Bypass	Forward	X	MACAddr-DCS[g]/ VLAN-DMACHN
1	X	Fail	Drop	Drop	X	MACAddr-DCS
1	X	Pass	X	Forward	1	L3L4-DMCHN[h]
1	X	Pass	Pass	Forward	0	RSS-DMCH
1	X	Pass	Drop/Bypass	Forward	0	MACAddr-DCS
1	X	Bypass	Pass	Forward	X	RSS-DMCH
1	X	Bypass	Bypass	Forward	X	MACAddr-DCS
1	X	Bypass	Drop	Drop	X	MACAddr-DCS
1	1	Fail	Bypass	Drop	X	MACAddr-DCS
0	X	Pass	X	Drop	1	L3L4-DMCHN
0	X	Pass	Pass	Drop	0	RSS-DMCH
0	X	Bypass/Fail	Pass	Drop	X	RSS-DMCH
0	X	X	Bypass/Drop	Drop	X	0

a. L2/VLAN Filter match means that combined result of the Ethernet DA, SA, DA Hash, VLAN perfect & hash match filters. When all of the enabled filters give a match, then this is taken as “1”, otherwise “0”
b. L3-L4 Filter match has 3 values: Fail: All enabled L3L4 filter failed Pass: Any of enabled L3L4 filter gave a match Bypass: No L3L4 filter enabled for received packet type
c. RSS Match has the following three values: Bypass: RSS not enabled for received packet or L3/L4 tuples not enabled. Pass: RSS bit PD = 0 after RSS lookup Drop: RSS bit PD = 1 after RSS Lookup
d. Action on Packet: DMA channel number is indicated even if packet is “Drop” because of packet may reach the application because of MAC_packet_Filter[RA] = 1
e. DMCHEN: Field of the MAC_L3_L4_Controlx register that gave a match
f. RSS-DMCH: Field read from the RSS_Lookup_Data
g. MACAddr-DCS: Field of the MAC_Addressx_High register that gave a DA match
h. L3L4-DMCHN: Field (DMCHN) of the MAC_L3_L4_Controlx register that gave a match

Note

You can also assign multiple queues to one DMA channel. In this case the order of the packets that gets to the DMA channel from different Queues are affected and does not follow the Scheduling scheme programmed in the RAA field of the MTL_Operation_Mode register. For more details, see “Tx Side Arbitration Between DMA and MTL” on page 333.

For VxLAN/NVGRE type tunneled packets, Receive DMA channel selection is always based on either RSS function or External Lookup Interface (that is, MAC address based Receive DMA channel selection is not considered).

See “RSS Function when VXLAN/NVGRE Mode is Enabled” on page 275 for RSS based Receive DMA channel selection for tunneled packets.

6.8 Broadcast/Multicast Packet Duplication

Broadcast/Multicast Packet Duplication supports sending and receiving Broadcast/Multicast packets to multiple DMA channels.

In a virtual system that supports multiple guest OS (for instance, each OS owning a DMA), where each DMA acts as an independent Ethernet Node, there can be a requirement to route the received multicast/broadcast packets to all/multiple DMA channels. This is implemented by the `MTL_RX_FIFO_read_controller` register reading the packet from the same queue multiple times and route the same packet to the specified multiple DMA channels.

To support multiple DMA channels through software control, program the Duplication Selection of the DA (`DDS` field of the `MAC_Extended_Configuration` register).

When `DDS` is 0, the `DCS` value holds the binary number of the destination channel (unique destination with no duplication).

When DDS is set to 1, the XDCS field in the MAC_DChSel_IndReg(#i) Register is taken as the destination Rx DMA for the packet matching the associated DA/SA filter register. The XDCS field is specified as one-hot format where each bit when set, represents the RxDMA channel to which the packet should be transferred. In other words, when multiple bits are set, the packet is duplicated and distributed to all the corresponding RxMA channels.

6.8.1 Receive Channel-List Based Packet Duplication

Use the `DWCXG_RCHLIST_CNT` parameter to specify the number of receive channel list.

Based on the number of receive channel list, 25GMAC implements indirect register based receive-channel list array, which is accessible through the `MTL_RXP_Indirect_Acc_*` registers.

Each receive-channel list (array entry) can be used for packet duplication. The receive channel-list has per-channel (DMA) selection bit for the packet to be duplicated for the channel. Packet duplication is supported only for packets which are routed to the highest Rx queue.

Many receive filters use the channel list array for packet duplication. Such a filter selects the receive channel list number to determine multiple DMA selection for packet duplication. If the filter selects receive channel list number for a packet which is not routed to highest queue, it does not route the packet to multiple channels. Instead, it routes the packet to LSB selected channel-number from the list.

For more detail about channel list and its array structure see the `MTL_RCHLIST_IndReg` register in the *Register Descriptions* chapter.

6.8.2 Programmable Unicast Packet Duplication

Program the UDC field, bit[31] of the MAC_RxQ_Ctrl4 register to enable the unicast packet duplication.

The unicast packet duplication occurs only when the following conditions are met:

1. Highest receive queue is selected and enabled in the configuration
2. DA, SA, and VLAN filters pass
3. More than 1 bit is set in the DMA channel number that is specified in the XDCS field of the MAC_DchSel_IndReg(#i) register that matches the unicast designation address of the received packet.

25GMAC performs packet duplication by default when the received packets have multicast or broadcast destination address. Table 6-6 shows various scenarios when duplication occurs and when it does not.

Table 6-6 Unicast Packet Duplication

UDC	Unicast Packet Received	Queue Selection	One-Hot DMA Channel	Comments
x	no	other queue methods	single or multi-bit selected	duplication only when multi-bit is selected. No duplication when single bit is selected.
0	yes			
1				no duplication
1		highest queue	multi-bit selected	duplication

6.9 Selection of Tag Priorities Assigned to Tx and Rx Queues

The VLAN Tag priorities can be assigned to Traffic Classes by programming the corresponding PSTC# field in following registers:

- MTL_TC_Prty_Map0 (for Traffic Class 0 to 3)
- MTL_TC_Prty_Map1 (for Traffic Class 4 to 7)

The bit corresponding to the VLAN Tag priority can be set in the PSTC# field for assigning the priority to the Traffic Class. For example, to assign VLAN Tag priority of 3 to Traffic Class 0, set bit [3] in the PSTC0 field.

Multiple VLAN Tag priority can be assigned to a single Traffic Class. The Tx packet association to a particular Tx Queue is governed by the application. The MAC does not check the priority of a packet in a particular Tx Queue. The settings of the MTL_TC_Prty_Map(#i) registers are only used for Pause Flow Control (PFC). The settings in the PSTC# field determine the Traffic Class (and corresponding Tx Queues) blocked for transmission when a PFC packet is received with the corresponding VLAN Tag priorities enabled.

In the Receive direction, the VLAN Tag priorities can be assigned to Rx Queues by programming the PSRQ# field in the corresponding MAC_RxQ_Ctrl2or MAC_RxQ_Ctrl3 registers. The bit corresponding to the VLAN Tag priority can be set in the PSRQ field for assigning that priority to the corresponding Rx Queue. For example, to assign VLAN Tag priority of 3 to Rx Queue 0, set bit [3] in the PSRQ field of the MAC_RxQ_Ctrl2 register. More than one Rx Queue cannot be assigned the same VLAN Tag priority. However, more than one VLAN Tag priorities can be assigned to the same Rx Queue. A maximum of 8 priority queues are supported as there are only 8 priorities available (3-bit field in the VLAN Tag). A programmable PRQSO field of the MAC_RxQ_Ctrl5 register provides the flexibility to select the start offset of the priority queue.

Note

- When PRQSO is programmed to a non-zero value, the highest valid PSRQ(#i) field is only up to the one corresponding to highest Receive Queue selected in the configuration. Otherwise the received packets corresponding to programmed VLAN Tag priorities in the unmapped PSRQ(#i) field are routed to Receive Queue 0. For example, when 8 Receive Queues are selected in the configuration and PRQSO is programmed to 4, the highest PSRQ(#i) that can be programmed is PSRQ3
- The following requirements of PRQSO=0 apply even when it is programmed to a non-zero value:
  - The Express packets priority queue and Preempt packets special queues must not be mapped to the same Receive Queue
  - The Preempt packets priority queue and Express packets special queues must not be mapped to the same Receive Queue

The settings in the PSRQ field are used for VLAN Tagged Rx packet routing to Rx Queues as well as for PFC based Tx flow control. The received VLAN Tagged Rx packet is routed to the Rx Queue that has the VLAN Tag priority match. In PFC based Tx flow control, PSRQ field corresponding to a particular Rx Queue is used for enabling VLAN Tag priorities in the PFC packet transmitted when the corresponding Rx Queue threshold levels are reached.

6.10 Distribution of Ingress Packets from MAC to Rx Queues

The 25GMAC controller provides multiple Rx queues to support classification and distribution of ingress packets. The MAC receiver pushes the ingress packets to the Rx queues based on packet type (tagged, untagged, multicast, PTP, AVB, DCB, and so on), tag priorities, MAC packet filter results, preemptable or express packet, and so on. The distribution is controlled by the settings of the `MAC_RxQ_Ctrl0` through `MAC_RxQ_Ctrl5` and `MAC_Packet_Filter` register fields.

Based on the packet filter settings, the MAC receiver decides to either drop or forward the Ingress packet. For more details, see "Packet Filtering" on page 169. However, the results of Layer 3 and Layer 4 filtering, mentioned in "Layer 3 and Layer 4 Match Filtering" on page 187 are not considered for the distribution of ingress packets into the Rx queues. Only the results of DA/SA/VLAN filters mentioned in "Source Address or Destination Address Filtering" on page 170 and "VLAN Filtering" on page 173 are applicable.

Filtering of the broadcast packets is exclusively controlled by the `DBF` field, bit[5] of the `MAC_Packet_Filter` register. The filtering of multicast packets is controlled by the `PM` field, bit[4] of the `MAC_Packet_Filter` register, in addition to the DA/SA match.

By default, packets that fail any of the DA/SA filters, are dropped by the MAC Receiver. However, packets that fail the DA/SA/VLAN filter can still be forwarded to the application when the `RA` field of the `MAC_Packet_Filter` register is 1. Even when `RA` field is 0, VLAN filter fail packets can be forwarded if the `VTFE` field of the `MAC_Packet_Filter` register is 0.

The ingress packets go through the following operations in the decreasing order of priority.

1. RA = 0: Ingress packets are dropped if any one of the following condition is true.
   - DA/SA filter fail
   - `VLAN` filter fail and `VTFE` field of the `MAC_Packet_Filter` register is 1
2. RA = 1: Ingress packets that fail the DA/SA filter are also forwarded to the Rx queue as mentioned in Table 6-7.

Table 6-7 Routing DA/SA Failed Packets to Rx Queue

Packet Type	TSN Type	Destination Rx Queue Determined By
Broadcast/Multicast	Express	if MFFQE = 1, MFFQ; else if VFFQE = 1, VFFQ; (only for tagged packets) else RxQ0
Unicast	Express	if UFFQE = 1, UFFQ; else if VFFQE = 1, VFFQ; (only for tagged packets) else RxQ0
All	Preemptable	RQ field

3. RA = 1 and `VTFE` = 0: Ingress tagged packets that pass the DA/SA filter but fail the VLAN filter, are forwarded to the Rx queue as mentioned in Table 6-8.

Table 6-8 Routing DA/SA Pass But VLAN Filter Fail Packets to Rx Queue

TSN Packet Type	Destination Rx Queue Determined By
Express	if VFFQE = 1, VFFQ; else PSRQ
Preemptable	RQ field

4. Ingress packets that pass the DA/SA/VLAN filters are forwarded to the Rx queue
   - Table 6-9 shows the decreasing order of priority when `OMCBCQ` field of the `MAC_RxQ_Ctrl1` register is 0.

Table 6-9 Priority Routing of Filter Pass Packets When OMCBCQ = 0

Packet Type	TSN Type	Tagged	Destination Rx Queue Determined By
Broadcast/Multicast	Express	Yes	if MCBCQEN = 1, MCBCQ; else PSRQ
		No	if MCBCQEN = 1, MCBCQ; else UPQ
	Preemptable	Yes	if MCBCQEN = 1, PMCBCQ; else PSRQ
		No	if MCBCQEN = 1, PMCBCQ; else RQ
Unicast AVTP Control	Express	No	AVCPQ
		Yes	if TACPQE = 1, AVCPQ; else PSRQ
Unicast PTP over Ethernet	Express	No	PTPQ
		Yes	PTPQ or PSRQ based on TPQC
Unicast DCB/LLDP Control	Express	No	DCBCPQ
Remaining Unicast	Express	No	UPQ
		Yes	PSRQ
	Preemptable	No	RQ
		Yes	PSRQ

   - Table 6-10 shows decreasing order of priority when `OMCBCQ` field of the `MAC_RxQ_Ctrl1` register is 1.

Table 6-10 Priority Routing of Filter Pass Packets When OMCBCQ = 1

Packet Type	TSN Type	Tagged	Destination Rx Queue Determined By
All AVTP Control	Express	No	AVCPQ
		Yes	if TACPQE = 1, AVCPQ; else PSRQ

Packet Type	TSN Type	Tagged	Destination Rx Queue Determined By
All PTP over Ethernet	Express	No	PTPQ
		Yes	PTPQ or PSRQ based on TPQC
Remaining Broadcast/Multicast	Express	Yes	if MCBCQEN = 1, MCBCQ; else PSRQ
		No	if MCBCQEN = 1, MCBCQ; else UPQ
	Preemptable	No	if MCBCQEN = 1, PMCBCQ; else PSRQ
		Yes	if MCBCQEN = 1, PMCBCQ; else RQ
Unicast DCB/LLDP Control	Express	No	DCBCPQ
Remaining Unicast	Express	No	UPQ
		Yes	PSRQ
	Preemptable	No	RQ
		Yes	PSRQ

Note

- If the RxQ pointed by the various fields in Table 6-7 through Table 6-10 is not enabled in the RxQ_Ctrl0 register as per the packet type, the packet is written to
  - RxQ0 for Express packets
  - RQ for Preemptable packets

  If RxQ0/RQ is not enabled, the packet is dropped.
- If the AV8021ASMENfield of the MAC_Timestamp_Control register is 1, MAC receiver checks only for untagged PTP packet. Tagged packet with PTP payload are considered as Remaining (Unicast/Broadcast/Multicast) tagged packets in Table 6-7 through Table 6-10.

7

Power Management and Energy Efficient Ethernet

This chapter describes the power management features of 25GMAC.

This chapter contains the following sections:

- "Overview of Power Management " on page 346
- "Remote Wake-Up Mode" on page 347
- "Magic Packet Detection Mode" on page 350
- "PMT Interrupt " on page 352
- "Design Partition for Low-Power Support" on page 353
- "Energy Efficient Ethernet" on page 360

7.1 Overview of Power Management

25GMAC supports the power management functionality through the power management block which supports the reception and detection of magic packets and remote wake-up (or wake-on-LAN) frames. 25GMAC also supports the IEEE 1801 standard, Unified Power Format (UPF), for power management.

The Power Management (PMT) block is available in the receive path of 25GMAC. The PMT block does not perform the clock gate function, but generates interrupts for remote wake-up frames and magic packets received by the MAC. You can implement the transmit and application clocks gating (outside the 25GMAC controller) with the signals output from the PMT block during the power-down mode.

You can enable the PMT block function in your controller by selecting the Enable Power Management option in coreConsultant. You can select any one or both of the following power management techniques.

- Remote Wakeup
- Magic Packet Detection

Both the modes support VLAN tagged packets, Destination Address (DA) filtering with perfect unicast MAC 0 address match, and switch off the major blocks during the power-down mode

When you enable the power-down mode, the MAC drops all received frames and does not forward these frames to the application. The MAC comes out of the power-down mode only when a magic packet or a remote wake-up frame is received and the corresponding detection is enabled. To generate power management events, the application can program the MAC_PMT_Control_Status register.

7.1.1 PMT Registers

You can access the following PMT registers in similar manner as you access the 25GMAC CSR registers. For mapping information, see MAC Registers Map in Register Descriptions chapter.

- MAC_PMT_Control_Status
- MAC_RWK_Packet_Filter

Note The features defined in the bits PWRDWN and RWKPFE are functional only when power gating is enabled. When power gating is not enabled, these features are non-functional.

7.2 Remote Wake-Up Mode

This section describes the Remote Wakeup Packet based power saving mode

The remote wake-up packet feature is implemented based on the *Device Class Power Management Reference Specification* and various implementation-specific white papers.

The MAC implements a filter lookup table (programmed through `MAC_RWK_Packet_Filter` register) in which CRC, offset, and byte mask of the pattern embedded in remote wakeup packet and the filter operation commands are programmed.

The pattern embedded in the remote wakeup packet is located at any offset after the Destination address and Source address fields. In addition to the CRC match for the pattern, the MAC receiver also checks the following, to detect the received packet as a valid remote wakeup packet:

- The packet must be addressed to it (Destination Address of the received packet must perfect match the MAC_Address0_High and `MAC_Address0_Low` registers) or with multicast/broadcast address
- The packet must not have length error, FCS error, dribble bit error, GMII error, and collision
- The packet must not be runt (length including Ethernet header and FCS is at least 64 bytes)

When a valid remote wakeup packet is received, the MAC receiver sets the `RWKPRCVD` bit in `MAC_PMT_Control_Status` register and triggers the interrupt on `pmt_intr_o` output port. The `PMTIS` bit in `MAC_Interrupt_Status` register is set when power-gating is not enabled in low-power mode.

7.2.1 Remote Wake-Up Packet Filters

The PMT block supports the following four programmable filters that enable you to define different receive frame patterns:

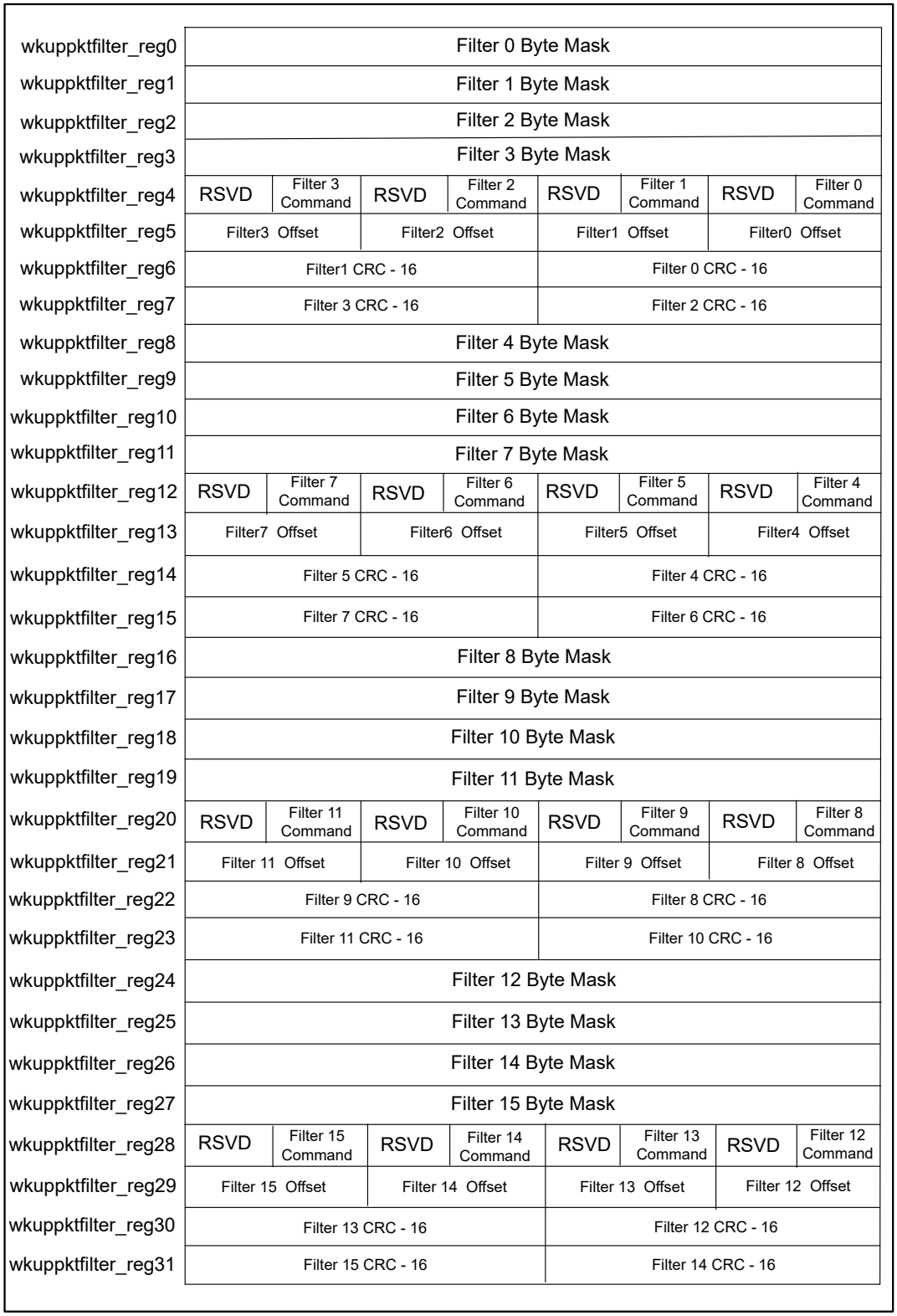
- The Filter Offset determines the offset from which the frame is examined.
- The Filter Byte Mask determines the bytes of the frame that must be examined.
- The Filter Command controls the filter operation.
- The Filter CRC-16 contains the CRC_16 value calculated from the pattern.

The MAC identifies a frame as remote wake-up frame if the following conditions are true:

- The destination address (DA) matches the value in `MAC_Address0` for unicast packets or the incoming packet is a multicast packet (including broadcast).
- The Filter CRC-16 matches the CRC of the incoming pattern. The remote wake-up CRC block determines the CRC value that is compared with Filter CRC-16.

The remote wake-up frame is checked only for length error, FCS error, and XGMII error to ensure that it is not a runt frame. The remote wake-up frame detection is updated in the PMT Control and Status register for every remote wake-up frame received. A PMT interrupt to the application triggers a read operation to the PMT Control and Status register to determine reception of a remote wake-up frame. Figure 7-1 describes the Remote Wake-Up Frame Filter register.

Figure 7-1 Remote Wake-Up Frame Filter Register



7.2.2 VLAN Frame Support

The VLAN frame and a remote wake-up frame have the same wake-up pattern but at a different offset (4 bytes difference). In a VLAN frame, a four byte VLAN header is inserted between the Source Address and Frame Type field.

The VLAN header includes the following:

- type ID assigned to bytes 12:13.
- VLAN identifier, VID, assigned to bytes 14:15.

The PMT block reads the VLAN type in the type ID field of offset 12 in bytes 12:13. If the VLAN type is detected, the PMT block increments the offset (by 4 bytes) to value 16. The PMT block handles the VLAN packet identification and offset increment functionality.

7.2.3 Enabling PMT through Remote Wake-Up Mode

To enable the remote wake-up mode feature:

1. From the coreConsultant GUI, select the Enable Power Management option under the Low Power Management section during the Specify Configuration activity in coreConsultant.
2. To enable Remote-Wakeup mode, select the Enable Remote Wake-Up option under the Low Power Management section during the Specify Configuration activity in coreConsultant.

   For details about these options, see Low Power Management Parameters in the *"Parameter Descriptions"* chapter.

Note See the "Enabled" field of the parameter descriptions to understand the dependencies.

When you enable the low-power mode in the PMT block, the MAC drops all received packets and does not forward any packet to the MTL Rx Queue or the application. The MAC comes out of the low-power mode when a remote wake-up packet is received and the corresponding detection is enabled.

7.3 Magic Packet Detection Mode

This section describes the power saving through magic packet detection.

Note

- The magic packet feature is based on the Magic Packet technology white paper from Advanced Micro Device (AMD).
- The watchdog timeout limit for a magic packet is 2,048 bytes irrespective of the value programmed in WD bit of `MAC_Configuration` register and PWE bit in `MAC_Watchdog_Timeout` register.
- The value programmed in DCRCC bit of `MAC_Ext_Configuration` register is applicable to a magic packet only when Enable IPv4 ARP Offload is selected in the configuration.

In the magic packet based power saving is a mode, the reception of valid magic packet by MAC receiver triggers an exit from low-power mode. The MAC enters power saving mode when PWRDWN bit of `MAC_PMT_Control_Status` register is programmed to 1. Exit from the magic packet based low-power mode is enabled by setting the MGKPKTEN bit of `MAC_PMT_Control_Status` register to 1.

The magic packet is based on a method that uses Advanced Micro Device's Magic Packet technology to power up the sleeping device on the network.

The magic packet contains a unique pattern at any offset after the Destination address, Source address, and Length/Type fields. In addition to the unique pattern matching, the MAC receiver also checks for the following, to detect the received packet as a valid magic packet:

- The packet must be addressed to it (Destination Address of the received packet must perfect match the MAC_Address0_High and `MAC_Address0_Low` registers) or with multicast/broadcast address
- The packet must not have length error, FCS error, dribble bit error, GMII error, and collision
- The packet must not be runt (length including Ethernet header and FCS is at least 64 bytes)

The MAC receives a specific packet of information, called a magic packet, addressed to the node on the network. The MAC checks the following types of packets to determine whether these packets meet the wake-up requirements:

- Unicast: The packets that are addressed to the MAC. For unicast packets, the Destination Address (DA) matches the value in MAC_Address0 register.
- Multicast: The packets with multicast address (including broadcast).

The packets that pass the address filtering (unicast or multicast) are checked to determine whether they meet the remote magic packet data format of 6 bytes of all ones followed by a MAC address (Unicast MAC Address that matches the value in the MAC_Address0 register) appearing 16 times.

The application enables the magic packet detection by writing 1 to bit 1 (MGKPKTEN) of the MAC_PMT_Control_Status register. The PMT block constantly monitors each frame addressed to the node for a specific magic packet pattern. Each frame received is checked for a 48'hFF_FF_FF_FF_FF_FF pattern following the destination and source address fields, and the Length or Type fields. The PMT block then checks the frame for 16 repetitions of the MAC address without any breaks or interruptions. In case of a break in the 16 repetitions of the address, the PMT block again scans the 48'hFF_FF_FF_FF_FF_FF pattern in the incoming frame. The 16 repetitions can be anywhere in the frame, but must be preceded by the synchronization stream (48'hFF_FF_FF_FF_FF_FF). The device can also accept a multicast frame, as long as the 16 duplications of the MAC address are detected. If the number of repetitions of 8'hFF are more than 6,

the PMT block checks for 16 repetitions of the MAC address without any breaks or interruptions, after the last 6 repetitions of 8'hFF.

If the MAC address of a node is 48'h00_11_22_33_44_55, then the PMT block scans the address for the following data sequence:

Destination Address Source Address Length/Type……. FF FF FF FF FF FF

00 11 22 33 44 55 00 11 22 33 44 55 00 11 22 33 44 55 00 11 22 33 44 55

00 11 22 33 44 55 00 11 22 33 44 55 00 11 22 33 44 55 00 11 22 33 44 55

00 11 22 33 44 55 00 11 22 33 44 55 00 11 22 33 44 55 00 11 22 33 44 55

00 11 22 33 44 55 00 11 22 33 44 55 00 11 22 33 44 55 00 11 22 33 44 55

…CRC

The magic packet is checked only for length error, FCS error, and XGMII error to ensure that it is not a runt frame.

The magic packet detection is updated in the `MAC_PMT_Control_Status` register, for the received magic packet. A PMT interrupt to the application triggers a read to the `MAC_PMT_Control_Status` register to determine if a magic packet frame is received.

7.3.1 Enabling PMT through Magic Packet Mode

To enable the Magic Packet Mode feature:

1. From the coreConsultant GUI, select the Enable Power Management option under the Low Power Management section during the Specify Configuration activity in coreConsultant.
2. To enable Magic Packet Detection, select the Enable Magic Packet Detection option under the Low Power Management section during the Specify Configuration activity in coreConsultant.

   For details about these options, see Low Power Management Parameters in the *"Parameter Descriptions"* chapter.

Note See the "Enabled" field of the parameter descriptions to understand the dependencies.

When you enable the low-power mode in the PMT block, the MAC drops all received packets and does not forward any packet to the MTL Rx Queue or the application. The MAC comes out of the low-power mode when a magic packet is received and the corresponding detection is enabled.

7.4 PMT Interrupt

In addition to the mci_intr_o or the sbd_intr_o signals, 25GMAC asserts the PMT interrupt signal, pmt_intr_o signal when a valid remote wake-up packet is received. The pmt_intr_o signal is synchronous to the Rx clock domain. This signal is provided so that you can stop the application clock when the MAC is in the power-down mode. As the pmt_intr_o signal is generated in the PHY Rx clock domain, it is not cleared immediately when the MAC_PMT_Control_Status register is read. This is because the resultant clear signal has to cross to the PHY Rx clock domain, and then clear the interrupt source. This delay is at least 4 clock cycles of the Rx clock and can be significant when the 25GMAC operates in the 10 Mbps mode. When the application clears the PWRDWN bit in Remote Wake-Up Packet Detection register, the MAC comes out of the power-down mode, but this event does not generate the PMT interrupt. For a description of signals related to PMT, see Power Management Interface Signals in the *Signal Descriptions* chapter.

When you enable interrupt by programming the PMTIE field in the MAC_Interrupt_Enable register to 1 and CSR clock is not gated off in low-power mode:

- In the XGMAC-CORE and XGMAC-MTL configurations,

  25GMAC triggers an interrupt to the application on the mci_intr_o output port.
- In the XGMAC-AXI configuration,

  25GMAC triggers an interrupt on the sbd_intr_o output port.

7.5 Design Partition for Low-Power Support

The design is physically partitioned into power switchable and power always-on domains to support power gating (compliant with Unified Power Format, UPF Standard) in Magic Packet and Remote Wakeup Packet based power saving modes.

Power gating is an effective technique for reducing the leakage power. In power gating, the circuit blocks that are not in use are temporarily turned off to reduce the leakage power of the chip. This temporary shutdown time is known as the low-power mode or inactive mode. The circuit blocks switch to the active mode when required for operation again. These two modes are switched at the appropriate time and in the suitable manner to maximize power performance.

The hardware description languages (HDL) do not provide a mechanism for describing the power connections at the RTL level. Therefore, verification and back-end implementation of a power-gated circuit at the RTL level is challenging. The IEEE 1801 standard, also known as the Unified Power Format (UPF), provides a consistent format that you can use to specify the power-aware design information that cannot be specified in the HDL code. In addition, you can use the UPF when you do not want to directly specify power-aware design information within the HDL logic.

The UPF defines a language format and simulation semantics for defining a power domain and a set of power supplies to the power domain. A power domain is a group of elements in the design that share a common set of power supply needs. The UPF file specifies the following:

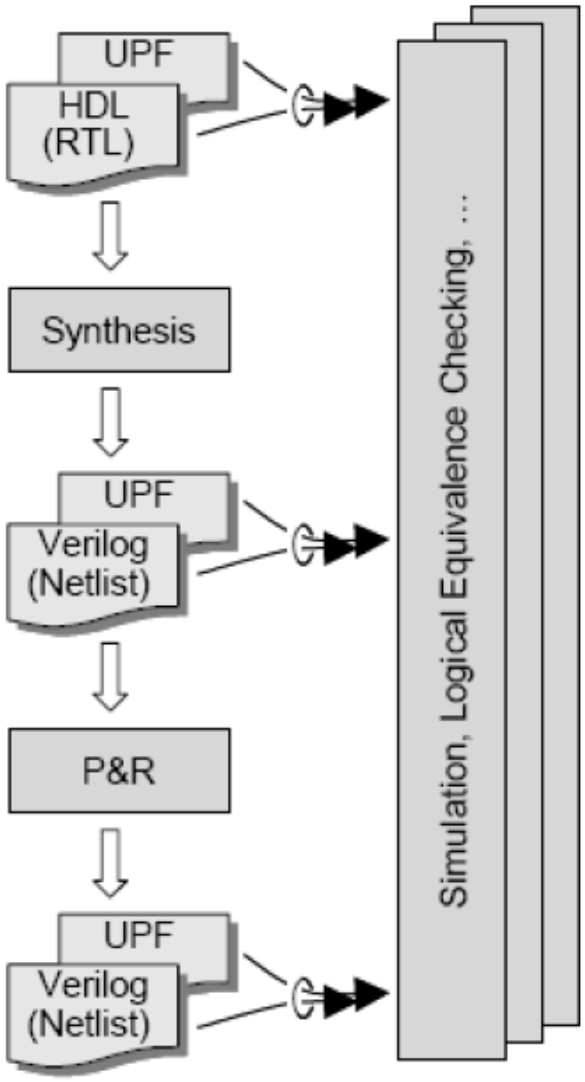
- How to create a supply network to supply power to each design element?

  The supply network consists of supply ports, switches, and supply nets. A supply net is a conductor that carries supply voltage or ground throughout a given power domain. By controlling the operating voltages of each supply net and whether the supply nets (and their connected design elements) are turned on or off, the supply network provides power at the level that is required by the functional areas of the chip to complete the computational task in a timely manner.
- How individual supply nets behave with other supply nets?
- How the logic functionality is extended to support dynamic power switching to design elements?

Note	UPF is enabled in 25GMAC when Enable Power Management configuration option is selected during the controller configuration in coreConsultant.

Figure 7-2 shows the UPF implementation.

Figure 7-2 UPF Implementation



7.5.1 Advantages of UPF

UPF has the following advantages:

- UPF is independent of the RTL code and can be used to add power-related functionality to the RTL without modifying it.
- UPF enables easy mapping from UPF to the logic design. Anything created in a UPF specification is created within a specific scope of the logic design. This facilitates writing the UPF code and debugging and analyzing the low-power design specification.
- UPF defines a support package in SystemVerilog and VHDL to facilitate verification. This package defines routines for querying the current state of a supply net or port and setting the state of a supply port. This allows the testbench to mimic the off-chip power supplies that are connected to the supply pads of a chip.

The ability to define a UPF supply net to an HDL logic value conversion is provided when supply net is connected to a logic port in the design. This facilitates integrating the supply network defined in UPF to power-aware models in the design without requiring re-writing of model.

- UPF provides a consistent way to specify power implementation intent throughout the design process including synthesis, physical implementation, and verification. This helps in better checking, interoperability, and productivity with mixed EDA flows.

  The UPF file is the input to several tools such as simulation, synthesis, formal verification, and place-and-route tools. Synthesis tools can read the RTL or UPF design input files and produce a netlist.

7.5.2 UPF Flow Block Diagram

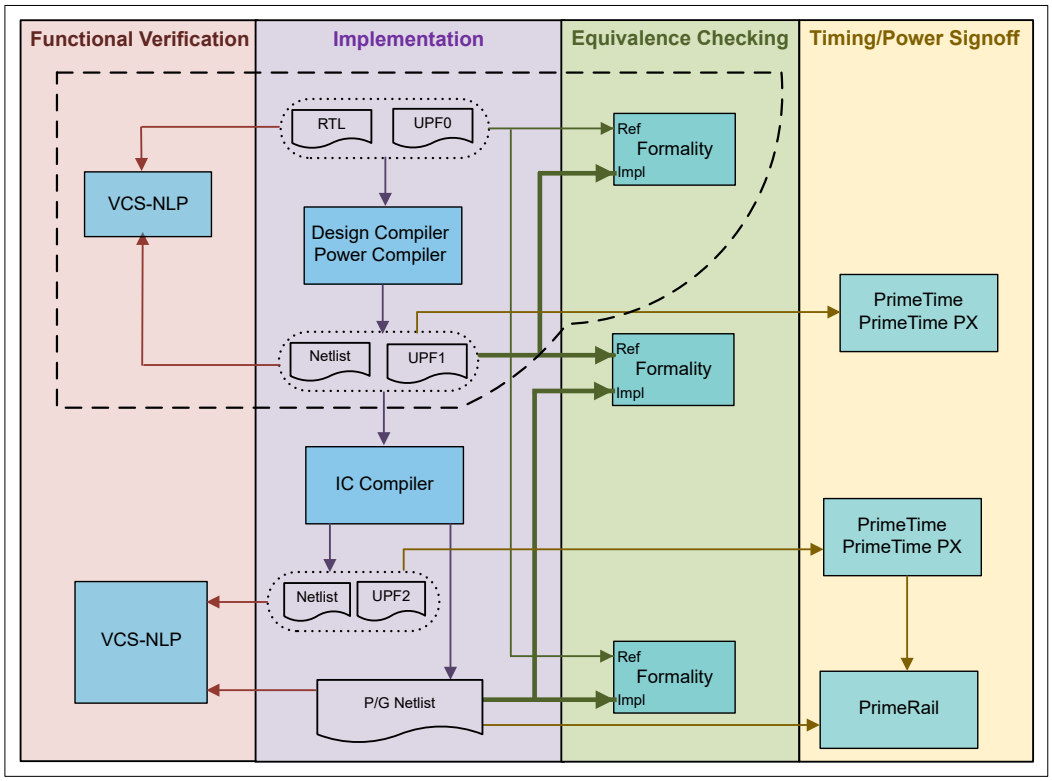
The PMT block of the 25GMAC provides the power management functionality. The PMT block enables a system to go into the low-power or sleep mode and then wake up on receiving a specific Ethernet packet. In the sleep mode, only the Always On Modules require power. Therefore, you can power off the rest of the logic of the system.

To enable UPF support, you must enable the Enable Power Management configuration option.

You can use the UPF support described in this chapter to isolate the Always On logic into a separate voltage island in the implementation flow.

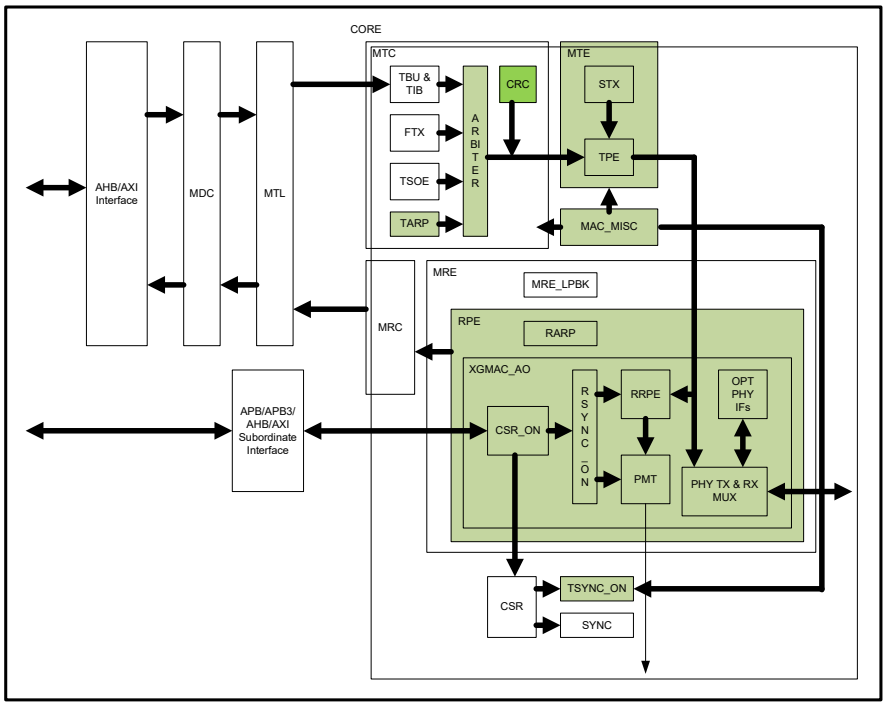
Figure 7-3 shows the block diagram of the 25GMAC with UPF support.

Figure 7-3 UPF Flow Block Diagram



When you specify the DWCXG_ARP_EN parameter, the Always-On blocks differ. The difference is shown in Figure 7-4, which shows the block diagram of the 25GMAC with UPF support when the ARP offload feature is enabled.

Figure 7-4 UPF Flow Block Diagram With IPv4 ARP Offload Enabled



7.5.3 Address Filtering with UPF

The PMT block processes all packets that are passed by the Address Filtering module. You can control the address filtering by programming the MAC_Packet_Filter register. When you configure the 25GMAC for the UPF support, only one MAC Address (MAC_Address0) is used for filtering because, the rest of the Address Filter block is in the power-off domain.

In UPF configurations, the PMT block processes any packet whose destination address (DA) matches the value in the MAC_Address0 register. However, for magic packet and remote wake-up frame detection, the PMT block accepts all multicast packets (including broadcast).

7.5.4 ARP Response Generation With UPF

The ARP block processes any packet whose Destination Address matches the value in the MAC_Address0 register. It generates an ARP response if a valid ARP request packet is received without any intervention from the application. The received ARP request is not forwarded to the application.

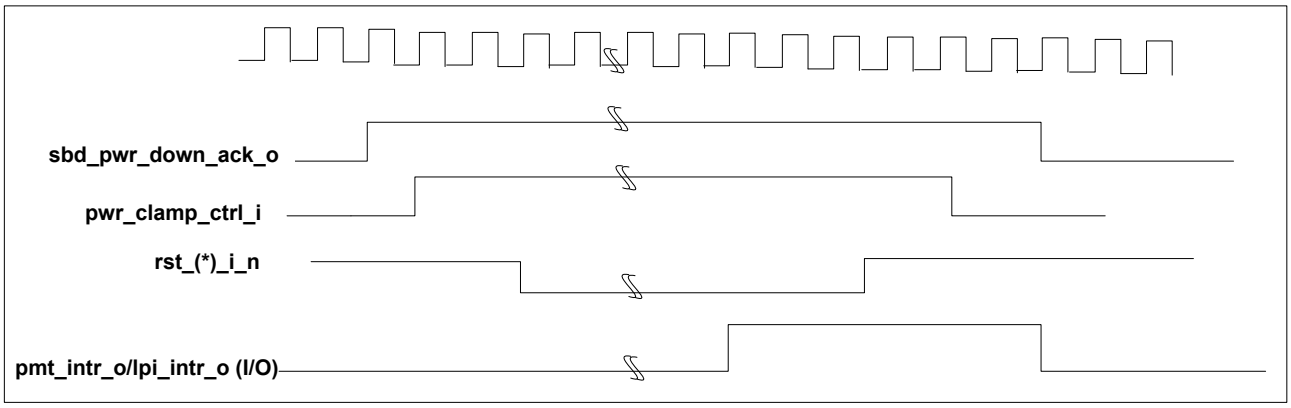
If the MAC is processing one ARP request and receives a new ARP request, then it does not generate an ARP response for the new ARP request. Instead, it just drops the packet. The ARP Response for the current Request is transmitted.

In UPF mode, the disable CRC check programming is not valid. The received packet is considered a valid ARP request only if the CRC check passes.

7.5.5 Power-Down and Power-Up Sequence

Figure 7-5 shows the power-down and power-up sequence.

Figure 7-5 Power-Down and Power-Up Sequence



7.5.5.1 Power-Down Sequence

The following list describes the power-down sequence:

1. The software must perform the following tasks:
   a. Disable the Transmit DMA (if applicable) by clearing the `ST` bit of the `DMA_CH0_Tx_Control` register.
   b. Wait for any previous frame transmissions to complete. You can check this by reading Bits [18:16] of `MAC_Debug` register and Bit[4] of `MTL_TxQ(#i)_Debug` register of all MTL Tx Queues.
   c. Disable the MAC transmitter and MAC receiver by clearing Bit [0] (TE) of `MAC_Tx_Configuration` register and Bit [0] (RE) in `MAC_Rx_Configuration` register.
   d. In the
      - XGMAC-MTL configuration:

        Wait until the application empties all frames from the Rx FIFO.
      - XGMAC-AXI configuration:

        Wait until the Receive DMA empties all frames from the Rx FIFO.

      You can check this by reading Bits [29:16] of `MTL_RxQ(#i)_Debug` register of all Rx Queues. If these bits are 0, it indicates that the Rx FIFO is empty.
   e. Configure the magic packet (bit 2) and/or remote wake-up (bit 1) detection in the `MAC_PMT_Control_Status` register (offset 0x00C0).
   f. If `DWCXG_ARP_EN` = 1 during Power Down, program the `ARPEN` field [bit 31] of the `MAC_Rx_Configuration` Register.
   g. Enable the MAC Receiver by setting Bit 0 (RE), and then set Bit 0 (PWRDWN) in the `MAC_PMT_Control_Status` register to initiate the power-down sequence in MAC.

      When only clock-gating is used (ARP offloading is not used), use the `sbd_pwr_down_ack_o` signal to gate-off the transmit clock, application clock, and CSR clock.

2. The MAC asserts the `sbd_pwr_down_ack_o` signal after all FIFOs are empty and the power down is set in the PMT block.

   This indicates that the always-on block is actively programmed and the power management controller can start switching off the power.

3. The Power Management Controller performs the following tasks:

   a. Assert the power clamp control (`pwr_clamp_ctrl_i`) to clamp the resets to the always-on blocks.

   b. Assert the power isolation control signal (`pwr_isolate_i`) to clamp the voltages of the isolation cells.

   c. Assert the `pwr_down_ctrl_i` signal to shutdown the power to the blocks in the power-OFF hierarchy.

Note

- When Energy Efficient Ethernet feature is enabled and the MAC Transmitter is in the LPI mode when it is put into the power-down mode, the GMII or MII interface are clamped to assert the LPI pattern. If the MAC Transmitter is not in the LPI mode when it is put into the power-down mode, the GMII or MII interface are clamped to all-zero.
- After the sbd_pwr_down_ack_o signal is asserted by MAC, the pwr_clamp_ctrl_i can be asserted after 1 CSR clock cycle. Wait for clamp circuit delay (which depends on the cell library) to assert pwr_isolate_i signal. Wait for isolation cell delay (which depends on the cell library) to assert pwr_down_ctrl_i signal.

7.5.5.2 Power-Up Sequence

The MAC wakes up on receiving the magic packet or remote wake-up frame. The following list describes the power-up sequence:

1. The MAC asserts `pmt_intr_o` signal.
2. The Power Management Controller must perform the following tasks:

   a. De-assert the `pwr_down_ctrl_i` signal to enable power to the blocks in the power-OFF hierarchy.

   b. De-assert the `pwr_isolate_i` isolation cell control signal.

   c. Assert resets to ensure that all registers in the blocks that were shutdown are reset. To properly initialize all asynchronous logic, the reset must be active for at least 4-clocks duration of the slowest clock in 25GMAC.

   d. De-assert the reset signals.

   e. De-assert the `pwr_clamp_ctrl_i` signal.

Note

- If the Energy Efficient Ethernet feature is enabled and the GMII or MII interface is clamped to the LPI pattern when the MAC is in the power-down mode, the GMII or MII transmitter starts driving all-zero; that is, the MAC transmitter comes out of the LPI mode when the pwr_clamp_ctrl_i signal is de-asserted.
- In the XGMAC-AXI configuration, the application reset is synchronized to all clock domains. The application must wait for the de-assertion of resets before it de-asserts the pwr_clamp_ctrl_i signal.

  In the XGMAC-CORE configuration the application must wait for the de-assertion of reset input (rst_clk_rx_n) before it de-asserts the pwr_clamp_ctrl_i signal. In the XGMAC-MTL configuration, the application must wait for de-assertion of reset inputs (rst_clk_rx_n and rst_clk_app_n) before it de-asserts the pwr_clamp_ctrl_i signal.
- After getting the wakeup interrupt, pwr_down_ctrl_i signal can be de-asserted after 1 CSR clock cycle. Wait for stable power applied to de-assert pwr_isolate_i signal. Wait for isolation cell delay (which depends on the cell library) to de-assert the reset signal. Ensure at least 3 application clock cycles between de-assertion of pwr_down_ctrl_i and reset signals. After the reset signal is de-asserted, wait for 25 (~6 cycles for all resets to be asserted + ~7 cycles for reset stretching + ~6 cycles for all resets to be de-asserted; 25 to be on the safer side) clock cycles of slowest clock to de-assert the pwr_clamp_ctrl_i signal.

3. The software must perform the following tasks:
   a. De-assert the pmt_intr_o by reading the PMT Control and Status register.
   b. Perform a write operation (with reset values) to the MAC_PMT_Control_Status and the MAC_RWK_Packet_Filter registers (if present) so that the corresponding values in the always-on block gets synchronized. Otherwise, the values of these registers are different.
   c. Perform write operations to the MAC_Tx_Configuration, MAC_Rx_Configuration and MAC_Address0 registers to synchronize the values in the CSR module and the respective bits in the always-on block. Otherwise, the MAC receiver is ON even though the Receive Enable bit is set to 0.

After completing these steps, the software must initialize all registers, enable the transmitter, and program the DMA (in DMA configurations) to resume the normal operation.

7.6 Energy Efficient Ethernet

Energy Efficient Ethernet (EEE) is an optional operational mode that enables the IEEE 802.3 Media Access Control (MAC) sub-layer along with a family of Physical layers to operate in the Low-Power Idle (LPI) mode. The EEE operational mode supports the IEEE 802.3-2018 MAC operation at 100 Mbps, 1000 Mbps,10 Gbps, 5 Gbps, and 2.5 Gbps.

The LPI mode allows power saving by switching off the parts of the communication device functionality when there is no data to be transmitted and received. The systems on the both sides of the link can disable some functionality and save power during low link-utilization period. The MAC controls whether the system must enter or exit the LPI mode and communicates this to the PHY.

The EEE specifies the capabilities negotiation methods that the link partners can use to determine whether EEE is supported and then select the best set of parameters common to both devices.

7.6.1 LPI Transmit Path Functions

In the transmit path, the software must set the LPITXEN bit of the MAC_LPI_Control_Status register to indicate to the MAC to stop the transmission and initiate the LPI protocol. The MAC completes the transmission in progress and generates its transmission status. The MAC starts transmitting the LPI pattern instead of the IDLE pattern if the link status has been UP continuously for a period specified in the LPI LS TIMER field of MAC_LPI_Timers_Control register. The PHY Link Status bit of the LPI Control and Status Register indicates the link status of the PHY.

To make the PHY enter the LPI state, the MAC performs the tasks described in Table 7-1.

Table 7-1 Transmit Path Functions For the GMII and XGMII Interfaces

GMII/MII Interface	XGMII Interface
1. De-asserts TX_EN. 2. Asserts TX_ER. 3. Sets TXD[7:0] to 0x01 in GMII and TXD[3:0] in MII. 4. Updates the LPI entry status in the MAC_LPI_Control_Status register and generates the sbd_intr_o interrupt if the LPIIE bit of the MAC_Interrupt_Enable register is set to 1.	1. Drives XGMII_TXC[7:0] or [3:0] to 4'b1111. XGMII_TXC[7:0] is for the DDW mode; XGMII_TXC[3:0] is for the DDR/SDR mode. 2. Drives XGMII_TXD[63:0] or [31:0] to 32'h06060606. XGMII_TXD[63:0] is for the DDW mode; XGMII_TXD[31:0] is for the DDR/SDR mode. 3. Updates the LPI entry status in the MAC_LPI_Control_Status register and generates the sbd_intr_o interrupt if the LPIIE bit of the MAC_Interrupt_Enable register is set to 1.
Note: The MAC maintains the same state for the specified signals while it remains in the LPI state.	

To bring the PHY out of the LPI state, that is, when the software resets the LPITXEN bit, the MAC performs the following tasks:

1. Stops transmitting the LPI pattern and starts transmitting the IDLE pattern.
2. Starts the LPI TW TIMER.

The MAC cannot start the normal frame transmission until the wake-up time specified for the PHY expires. The auto-negotiated wake-up interval is programmed in the TWT field (bits [15:0]) of `MAC_LPI_Timers_Control` register.

3. Updates the LPI exit status of the `MAC_LPI_Control_Status` register and generates the `sbd_intr_o` interrupt if the LPIIE bit of the `MAC_Interrupt_Enable` register is set to 1.

Note In the DDR or DDW mode, when the MAC is coming out of the LPI mode, it internally generates reset for the XGMII transmit interface. This takes care of differences in the inter-clock arrival latency. In addition, it aligns the internal read or write pointers offsets after the Tx clock gating is removed.

Figure 7-6 shows the behavior of the TX_EN, TX_ER, and TXD[7:0] signals during the LPI mode transitions.

Figure 7-6 LPI Transitions for GMII/MII (Transmit)

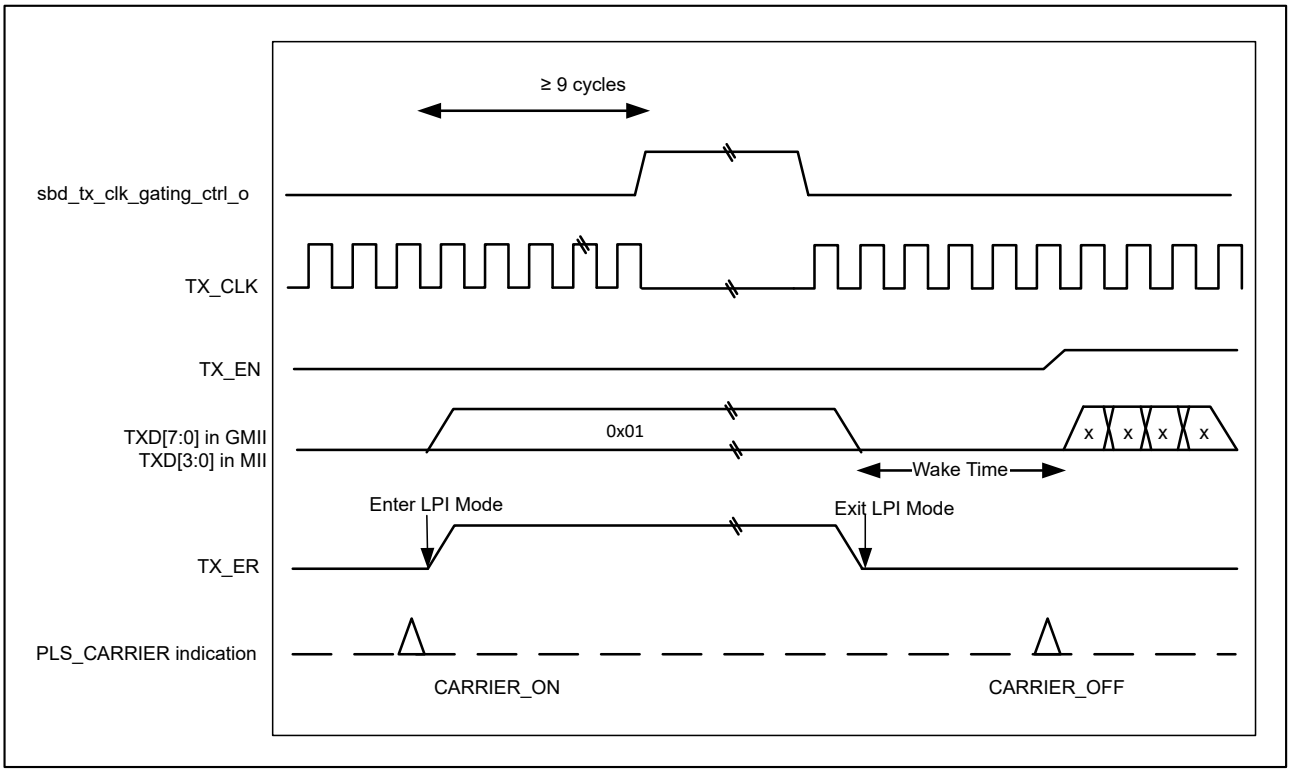
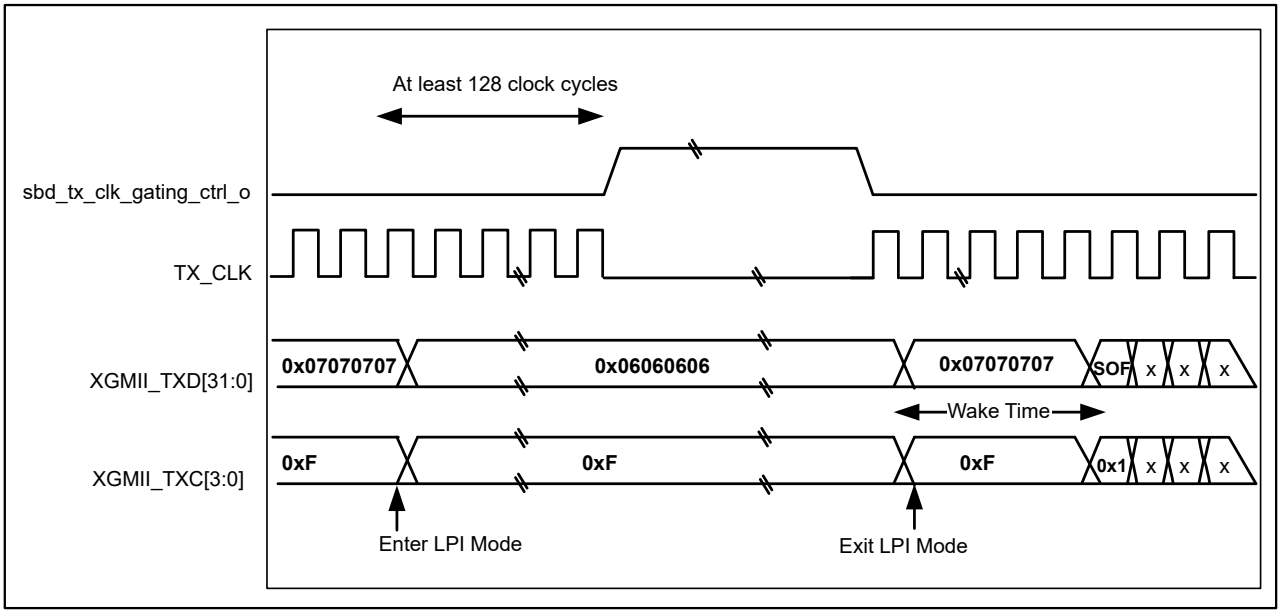


Figure 7-7 shows the behavior of the XGMII_TXD[31:0] and XGMII_TXC[3:0] signals during the LPI mode transitions.

Note When the MAC drives the LPI pattern, it does not internally generate the PLS_CARRIER indication (required by the Energy Efficient Ethernet standard) to defer normal transmission. The internal logic of the MAC ensures that it does not transmit normal frame when it is driving the LPI pattern and also during the subsequent LPI TW time.

Figure 7-7 LPI Transitions for XGMII (Transmit)



Note

- The 25GMAC does not stop the TX_CLK clock. You can stop this clock (as shown in Figure 7-6 and Figure 7-7) if your PHY supports it and the XGMAC-CORE sets the sbd_tx_clk_gating_ctrl_o signal to 1.
- The assertion of this clock-gating signal can be controlled by programming Bit[21] of MAC_LPI_Control_Status register.
- In the XGMAC-CORE configuration, when the TX clock gating is done during the LPI mode, you cannot use the LPITXA bit of the MAC_LPI_Control_Status register as the XGMAC-CORE cannot detect exit condition with the TX clock.
- If the MAC is in TX LPI mode and the TX clock is stopped, the application must not write to CSR registers that are synchronized to TX clock domain.

7.6.1.1 Automated Entry/Exit of LPI mode in Transmit Path

The MAC transmitter can be programmed to enter and exit LPI IDLE mode automatically based on whether it is IDLE for a specific period of time or has a packet to transfer. These modes are enabled and controlled by MAC_LPI_Control_Status register.

When LPITXA (Bit[19]) and LPITXEN (Bit[16]) of MAC_LPI_Control_Status register are set, the MAC transmitter enters LPI IDLE state when the MAC transmit path (including the MTL layers and DMA layers) are idle. The MAC transmitter exits the LPI IDLE state and clear the LPITXEN bit as soon as any of functions in the TX path (DMA, MTL or MAC) becomes non-idle due to initiation of a packet transfer.

In addition, when Bit[20] (LPIATE) is also set, the MAC transmitter enters LPI IDLE state only if the Transmit path remains in idle state (no activity) for the time period indicated by the value in MAC_LPI_Auto_Entry_Timer. In this mode also, the MAC transmitter exits the LPI IDLE state as soon as any of the functions becomes non-idle. However, the LPITXEN bit is not cleared but remains active so that re-entry to LPI IDLE state is possible without any software intervention when the MAC becomes idle again.

When both LPIATE and LPITXA bits are clear, the entry and exit of LPI IDLE state is directly controlled by the software by programming the LPITXEN bit.

7.6.2 LPI Receive Path Functions

When the PHY receives the signals from the link partner to enter the LPI state, the PHY and MAC perform the tasks mentioned in Table 7-2.

Table 7-2 Receive Path Functions for the GMII and XGMII Interfaces

GMII/MII Interface	XGMII Interface
1. The PHY asserts RX_ER. 2. The PHY sets RXD[7:0] to 0x01 in GMII and RXD[3:0] in MII. 3. The PHY de-asserts RX_DV. 4. The MAC updates the RLPIEN bit of the MAC_LPI_Control_Status register and generates the sbd_intr_o interrupt if the LPIIE bit of the MAC_Interrupt_Enable register is set to 1.	1. The PHY drives XGMII_RXC[7:0] or [3:0] to 4'b1111. XGMII_RXC[7:0] is for the DDW mode; XGMII_RXC[3:0] is for the DDR/SDR mode. 2. The PHY drives XGMII_RXD[63:0] or [31:0] to 32'h06060606. XGMII_RXD[63:0] is for the DDW mode; XGMII_RXD[31:0] is for the DDR/SDR mode. 3. The MAC updates the RLPIEN bit of the MAC_LPI_Control_Status register and generates the sbd_intr_o interrupt if the LPIIE bit of the MAC_Interrupt_Enable register is set to 1.
Note: The PHY maintains the specified signals while it remains in the LPI state.	

When the PHY receives the signals from the link partner to exit the LPI state, the PHY and MAC perform the following tasks:

1. The PHY de-asserts RX_ER.
2. The PHY sets the RXD[7:0] in GMII/RXD[3:0] in MII to 8'h00 for GMII or XGMII_RXD[31:0] to 32'h07070707 for XGMII.
3. The PHY returns to a normal inter-frame state.
4. The MAC updates the RLPIEX bit of the MAC_LPI_Control_Status register and generates the sbd_intr_o interrupt if the LPIIE bit of the MAC_Interrupt_Enable register is set to 1. The MAC also generates the sbd_intr_o interrupt on the additional sideband signal synchronized to Rx clock (lpi_intr_o).

Note In the DDR or DDW mode, when the MAC is coming out of the Rx LPI mode, it internally generates reset for the XGMII transmit interface. This takes care of differences in the inter-clock arrival latency. In addition, it aligns the internal read or write pointers offsets after the Rx clock gating is removed.

Figure 7-8 shows the behavior of the RX_ER, RX_DV, and RXD[7:0] signals during the LPI transitions.

Figure 7-8 LPI Transitions for GMII/MII (Receive)

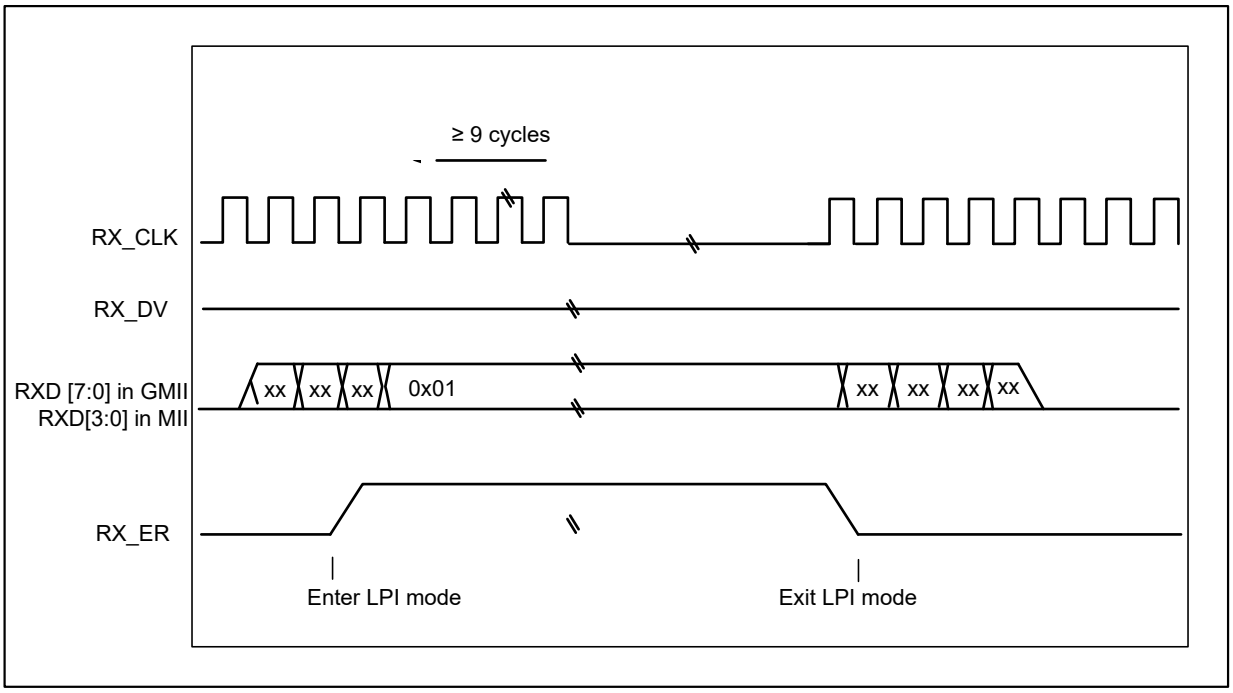
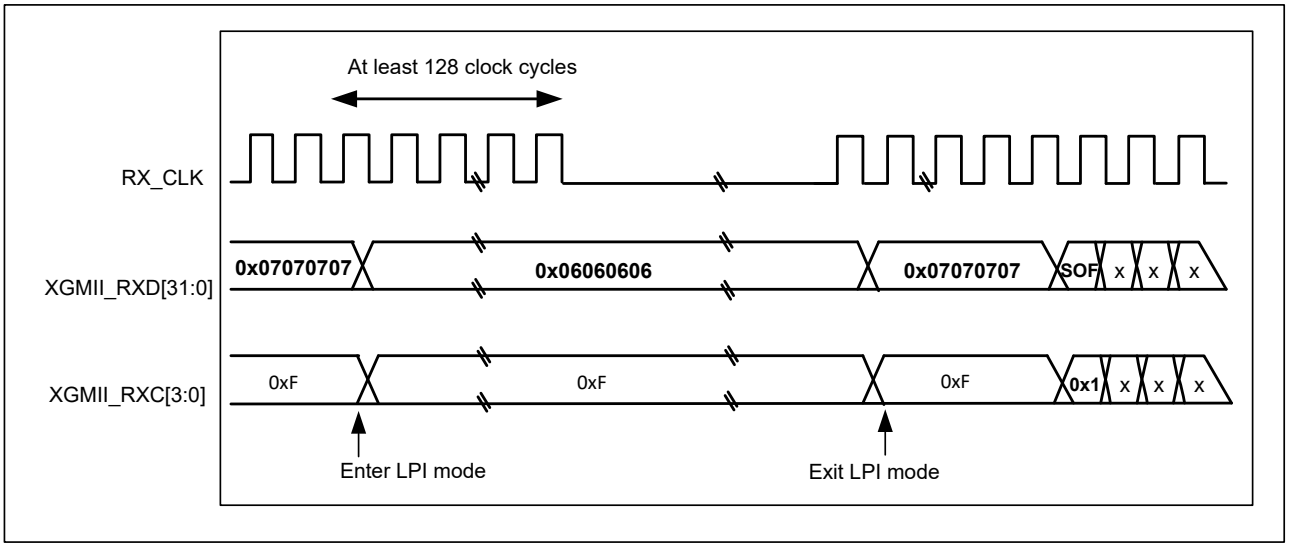


Figure 7-9 shows the behavior of XGMII_RXD[31:0] and XGMII_RXC[3:0] signals during the LPI mode transitions.

Figure 7-9 LPI Transitions for XGMII (Receive)



Note

- If the RX_CLK_stoppable bit (in PHY register written through MDIO) is asserted when the PHY is indicating LPI to the MAC, then the PHY may halt the RX_CLK at any time more than 128 clock cycles (for XGMII) and 9 clock cycles (for GMII) after the start of the LPI state as shown in Figure 7-8.
- If the MAC is in RX LPI mode and the RX clock is stopped by PHY, the application must not write to CSR registers that are synchronized to RX clock domain.

7.6.3 LPI Timers

The transmitter maintains the following timers that are loaded with the respective values from MAC_LPI_Timers_Control and MAC_LPI_Auto_Entry_Timer registers:

- LPI LS TIMER

  The LPI LS TIMER counts, in milliseconds, the time expired since the link status is UP. This timer is cleared every time the link goes down and is incremented when the link is UP.

  The XGMAC-CORE does not drive the LPI pattern unless the terminal count, programmed by the software, is reached. This ensures a minimum time during which no LPI pattern is driven after a link is established with the remote station. This period is defined as 1 second in the IEEE standard 802.3-az, version 2.0. The LPI LS TIMER is 10-bit wide. Therefore, the software can program up to 1023 milliseconds.

- LPI TW TIMER

  The LPI TW TIMER counts, in microseconds, the time expired since the de-assertion of Transmit LPI. The MAC can resume the normal transmit operation after the Transmit TW time. The software programs the Transmit TW time based on auto-negotiation.

  The MAC supports the LPI TW TIMER in units of microsecond. The LPI TW TIMER is 16-bit wide. Therefore, the software can program up to 65535 us.

- LPI AUTO ENTRY TIMER

  This timer counts in steps of eight microseconds, the time for which the MAC transmit path has to remain in idle state (no activity), before the MAC Transmitter enters the LPI IDLE state and starts transmitting the LPI pattern. This timer is enabled when LPIATE field of the `MAC_LPI_Control_Status` register is set.

7.6.4 Power-Down Mode Support

The LPI mode (low-power mode) and power-down mode features are implemented independently in 25GMAC. However, the MAC can go to the power-down mode when it is in the LPI mode. Similarly, the MAC can go to the LPI mode when it is in the power-down mode.

This section describes the power-down and power-up sequence for the following scenarios:

- "Power-Down Support when MAC in Receive LPI Mode" on page 366
- "Receive LPI Support when MAC is in Power-Down Mode" on page 367

Note

- When the 25GMAC is in the low-power and power-down mode, the link partner must come out of the low-power mode before it sends any packets to wake up 25GMAC.
- When the 25GMAC is in the power-down mode, it cannot enter the TX LPI mode as the bit 16 (LPI Enable) of the LPI Control and Status Register is in power-off domain.
- When the 25GMAC is in TX LPI mode, the MAC can enter the power-down mode if the Receiver is not in the Rx LPI state. If the Receiver is in Rx LPI state, the power-down support is as explained in "Power-Down Support when MAC in Receive LPI Mode" on page 366.

7.6.4.1 Power-Down Support when MAC in Receive LPI Mode

When the MAC is in the Rx LPI mode but not in the power-down mode, you cannot perform any write operation on the MAC_PMT_Control_Status. If the MAC enters the power-down mode when it is already in the Rx LPI mode, it waits for the Rx Exit Interrupt to update the PMT Control and Status Register. This is because the Rx clocks may not be available during Rx LPI Mode.

When PHY receives the LPI pattern, it can disable the receive clock after 128 cycles and go into the LPI mode on the receive side. The application can program the bit 16 (LPITXEN) of MAC_LPI_Control_Status to make the PHY enter the LPI mode on the transmit side. The MAC starts sending the LPI pattern after completing the current frame transmission. When the transmit side is in the LPI mode, the transmit clock can be gated after sbd_tx_clk_gating_ctrl_o is asserted. To save more power, the application can make the 25GMAC to go to the power-down mode. However, to make the 25GMAC to go to the power-down mode, the application must have programmed the PMT Control and Status Register.

The following sections describe the power-down and power-up sequence when the MAC is in the Rx LPI mode:

- "Power-Down Sequence" on page 366
- "Power-Up Sequence" on page 367

7.6.4.2 Power-Down Sequence

Note The power-down support (with UPF flow) with LPI is not supported in this release.

The following list describes the power-down sequence:

1. The software must set the bit 0 (PWRDWN) of the MAC_PMT_Control_Status.
2. The Power Management Controller must perform the following tasks:
   a. Wait for the assertion of the sbd_pwr_down_ack_o signal.
   b. Assert the pwr_clamp_ctrl_i signal to clamp the voltages of the isolation cells.
   c. Assert the pwr_down_ctrl_i signal to shutdown the power to the blocks in the power-OFF hierarchy.

7.6.4.3 Power-Up Sequence

The PHY comes out of the LPI mode by enabling the receive clock and sending the normal IDLE pattern instead of the LPI pattern. The following list describes the power-up sequence:

1. The MAC asserts the `lpi_intr_o` signal.

   At this stage, the Power Management controller can do either of the following:

   - Wait for the remote wake-up frame to be detected.
   - Initiate the 25GMAC to come out of power-down mode.

2. The MAC assert the `pmt_intr_o` signal on receiving the remote wake-up packet.
3. The Power Management Controller must perform the following tasks:
   a. De-assert the `pwr_down_ctrl_i` signal to enable power to the blocks in the power-OFF hierarchy.
   b. Assert resets to ensure that all registers, in the blocks that were shutdown, are reset.
   c. De-assert the resets.
   d. De-assert the `pwr_clamp_ctrl_i` signal.
4. The software must perform the following tasks:
   a. De-assert the pmt_intr_o by reading the `MAC_PMT_Control_Status` register.
   b. Perform a write operation (with reset values) to the `MAC_PMT_Control_Status` register and the `MAC_RWK_Packet_Filter` register (if present) so that the corresponding values in the always-on block gets synchronized. Otherwise, the values of these registers are different.
   c. Perform write operations to the `MAC_Rx_Configuration`, `MAC_Address0_High` and `MAC_Address0_Low` registers to synchronize the values in the CSR module and the respective bits in the always-on block. Otherwise, the MAC receiver is ON even though the Receiver Enable bit is set to 0.

After completing these steps, the software must initialize all the registers and enable the transmitter.

In the XGMAC-AXI configuration, program the `ST` field of the `VDMA_CH(#i)_TX_Control` register and `SR` field of the `DMA_CH(#i)_RX_Control` register, to resume the normal operation.

7.6.4.4 Receive LPI Support when MAC is in Power-Down Mode

In the power-down mode, a part of the receive block (including the Rx LPI) is in the always ON state. The PHY is operational to receive the magic packet or remote wake-up frames. If the PHY and Link Partner support LPI, then the PHY might assert the LPI pattern and can remove the Rx clocks.

For more details about the steps performed by PHY to assert the LPI pattern, see "LPI Receive Path Functions" on page 363.

When the MAC is already in the power-down mode and it receives the Rx LPI patterns, it does not generate the Rx LPI entry Interrupt. This is because the CSR module is in the power-off domain. However, the MAC generates the Rx LPI Exit Interrupt. The power-up sequence is similar to as described in "Power-Up Sequence" on page 367.

7.6.5 Sideband Control to Change Speed

You can change the speed and the Tx LPI state (in EEE mode) using sideband control signals without modifying the registers. Additional sideband signals are supported when you select Add Sideband Signals for Speed Change and EEE option in the coreConsultant GUI. To enable this mode, program the SBDIOEN field (Bit[8]) of the MAC_Extended_Configuration register to 1. In this mode, the sideband input signals override the corresponding values set in the CSR registers. When the registers are read, it reflects the values of the input signals.

Table 7-3 shows the additional signals supported for speed change control:

Table 7-3 Additional Signals Support for Speed Change Control

Signal	Control
sbd_speed_sel_i	Replaces Bits[31:29] of MAC_Tx_Configuration register
xgmac_tx_active_o	Indicates that MAC Transmitter is active
mtl_tx_active_o	Indicates that MTL TX path is actively transferring data
xgmac_rx_active_o	Indicates that MAC Receiver is active
mtl_rx_active_o	Indicates that MTL RX path is actively transferring data

7.6.5.1 Steps for Changing Speed

Follow this sequence to change the speed in this mode.

1. Stop transmitting packet data on ATI interface (ati_val_i = 0) and assert sbd_dis_receive_i to low (to disable Receiver).
2. Wait for xgmac_tx_active_o and mtl_tx_active_o to become low to ensure that the data is transferred completely; assert the sbd_dis_transmit_i signal to disable the MAC transmitter.
3. Wait for the xgmac_rx_active_o and mtl_rx_active_o signal to become low to ensure that the data is transferred completely.
4. Change the speed by changing the sbd_speed_sel_i signal to the value.
5. De-assert the sbd_dis_transmit_i, sbd_dis_receive_i to low to resume operation in new speed.

7.6.5.2 Triggering Transmit LPI in EEE Mode

The following side band signals are supported to control the triggering of Transmit LPI in EEE mode.

- sbd_lpitxen_i

  To replace LPITXEN in MAC_LPI_Control_Status register

  This signal must be used only when either (LPITXA=1 and LPIATE=1) or (LPITXA=0 and LPIATE=0) in MAC_LPI_Control_Status register.

  When LPITXA=0 and LPIATE=0, and this signal is asserted, it instructs the MAC Transmitter to enter the LPI state. When this signal is low, the MAC Transmitter exits the LPI state and resumes normal transmission. Assert this signal only when xgmac_tx_active_o and mtl_tx_active_o are 0.

When LPITXA=1 and LPIATE=1, and this signal is asserted, it enables the MAC Transmitter in EEE mode. The MAC enters the LPI state when the TX path is IDLE for the duration programmed in LPIET Timer and exits the LPI state when the MAC Tx path is in non-IDLE state (when packet data is input on ATI).

- sbd_lpi_link_sts_i

  To replace PLS in `MAC_LPI_Control_Status` register.

8

Time-Sensitive Networking

Time sensitive network (TSN) is a set of standards developed by the IEEE 802.1 work group. The standards define mechanisms for the time-sensitive transmission of data over Ethernet.

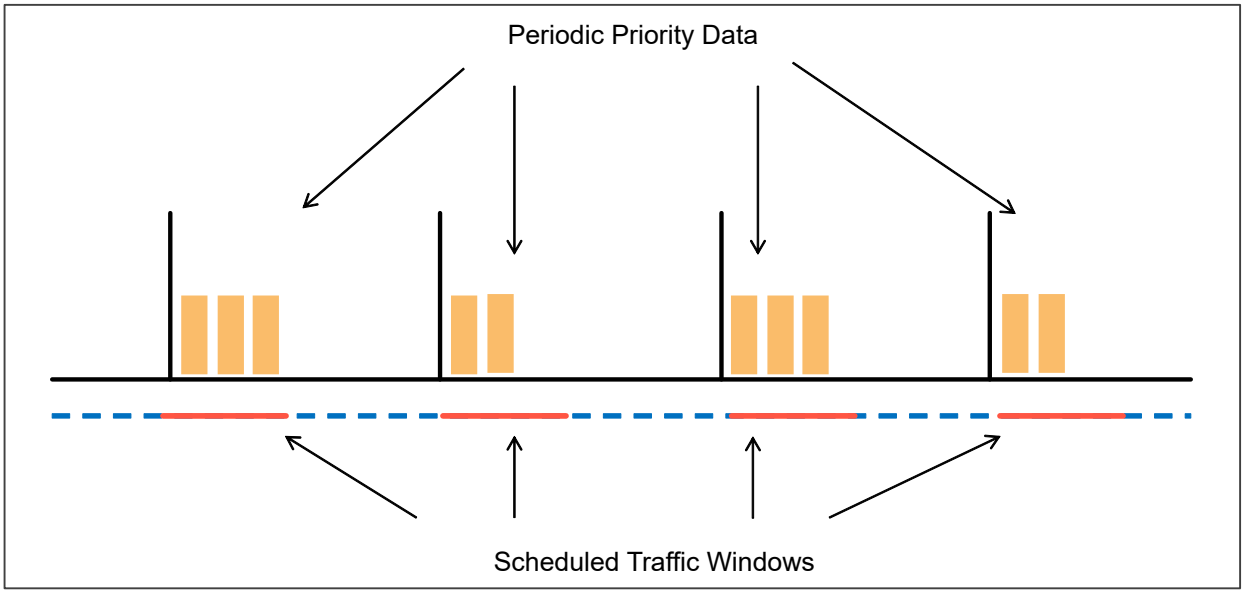
This chapter contains the following sections:

- "Enhancements to Scheduled Traffic (EST)" on page 372
- " Frame Preemption (FPE)" on page 385
- "Time-Based Scheduling" on page 399
- "Stream-Gate Filtering" on page 406
- "Policing Counters" on page 409
- "Asynchronous Traffic Shaper (ATS)" on page 413

8.1 Enhancements to Scheduled Traffic (EST)

The IEEE 802.1Qbv-2015 defines the schedule for each of the queues on every egress port which makes the implementation aware of traffic arrival schedule. This information can be used to block the lower priority traffic from transmission in this time window/slot. This ensures that scheduled traffic is forwarded from sender to receiver through all the network nodes with a deterministic delay.

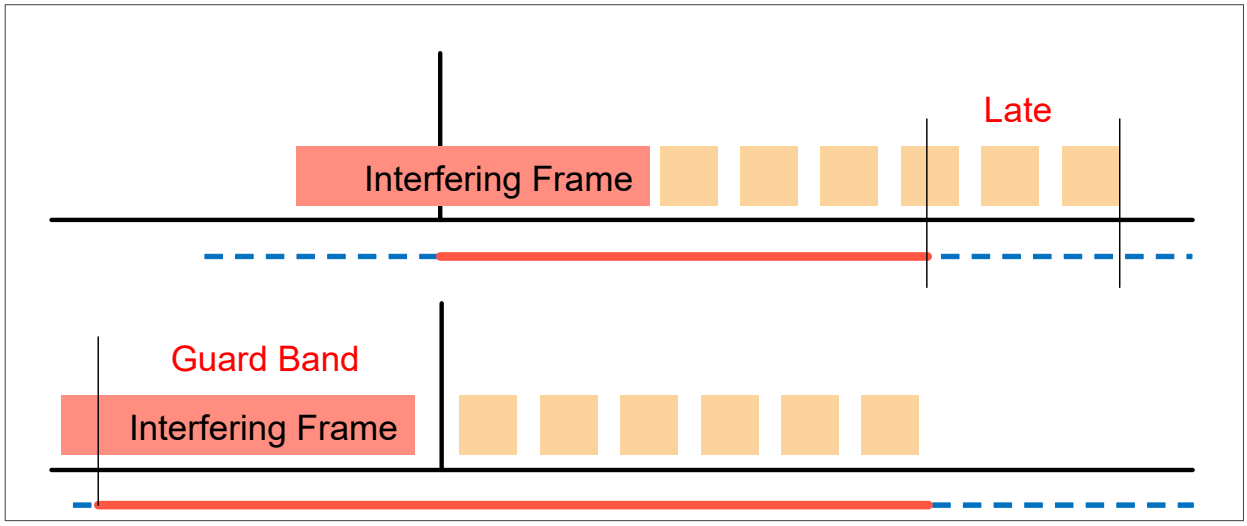
Figure 8-1 Time Aware Shaper Implementation for Traffic Scheduling



One of the important requirements to achieve a low latency is to ensure there are no interfering frames during the scheduled windows that are reserved for high priority traffic. The use of scheduled traffic imposes limitations while starting a transmission.

As shown in Figure 8-2, if an interfering frame begins transmission just before the start of a reserved time period, it can extend transmission into the reserved window, and potentially interfere with higher priority traffic. Therefore, a guard band whose size is equal to the largest possible interfering frame is required before the window starts.

Figure 8-2 Implementing a Guard Band to Avoid Delays Due to Interfering Frames



A larger guard band equates to a less efficient use of network bandwidth. However, with the implementation of IEEE802.1Qbu (frame preemption), this issue is addressed. Frame preemption breaks the interfering frame into smaller fragments. Therefore, the guard band needs to be only as large as the largest possible interfering fragment instead of the largest possible interfering frame.

During the guard band, only the frames that can complete the transmission of the entire frame before the next gate close event are permitted. This ensures that the high priority traffic can always start at the beginning of the window reserved for it.

The software must program the CTR, BTR, and Time Intervals of the GCL rows such that the difference between the CTR and the sum of the BTR and time intervals of fully executed GCL rows must be greater than 8 PTP clock period. Alternatively, the CTR must be equal to the sum of the BTR and Time Intervals of the fully executed GCL rows.

8.1.1 Frequently Used Terms in EST

Following are some of the frequently used terms in the EST support and their definitions.

- Gate Control List

  The list in the hardware memory that holds the gate controls and the associated time intervals.

- Gate Controls

  For a given schedule (row in gate control list) there is a Gate Open (O) and Gate Close (C) state associated with each TC. The set of O or C values, whose width is same as configured TCs is called the Gate Controls. Example: CCOCOOCO means TC7=C, TC6=C, TC5=O and so on.

- Time Interval

  Time interval (in nano seconds) is a 16, 20, or 24-bit configurable field in the Gate Control list that indicates the time for which the associated gate controls are valid.

- Base Time Register

  Each Gate Control List is associated with a 64-bit Base Time Register that holds the start time (in PTP format) for the list.

8.1.2 Updates to the Transmit Scheduling to Support EST

To support EST, following updates are required to the transmit scheduling:

- Implementation of Gate Control List (GCL)
- Enforcing gate controls in the scheduler
- Accounting for Gate Open (O) duty cycle in the computation of idleSlope (CBS)

8.1.2.1 Implementation of Gate Control List (GCL)

Figure 8-3 shows block diagram from the IEEE 802.1Qbv spec that illustrates how the gate control list governs the gate close (C) and open (O) events based on the schedule provided for each event.

GCL has the following two parts:

- Time Interval

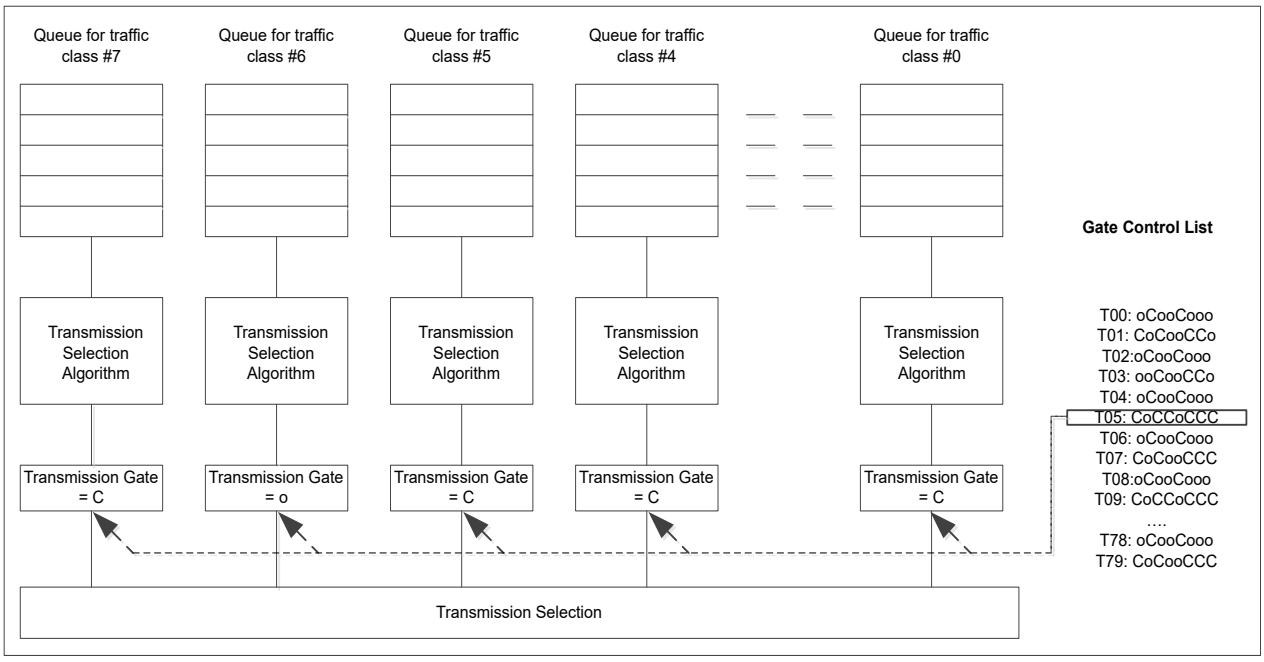
  Defines the time in nanoseconds for which the gate controls are valid and must be applied before reading the next gate controls from the list. Configurable width of 16, 20 or 24 to represent a maximum of 64us, 1ms, or 16ms schedule interval respectively. Supports left shift of the time interval up to 7 bits to be able to apply a multiplication factor from 1 to 128ns (in steps of 2^n). The maximum value (post shifting) of this field must be 999,999,999 ns.

  Time Interval (TI) value must be large enough to account for the implementation delays. Any value less than that cannot be accurately implemented. To account for the implementation constraints, the minimum time interval for CGL entry must be equal to the time required to transmit fragment_size + IPG/EIPG + preamble overheads.

- Gate Control

  Defines the Open (O) represented by logic 1 or Close (C) represented by logic 0 state for the gate of each TC.

Figure 8-3 Block Diagram from IEEE 802.1 Qbv Specification - GCL Governing Gate Close and Open Events



The implementation of GCL consists of the following two Gate Control Lists:

- HWOL - Hardware Owned List which is a list for hardware access.
- SWOL - Software Owned List which is a list for software access.

The access to these lists is mutually exclusive. The ownership to the list is set by hardware in the SWOL field of MTL_EST_Status register. Following table provides the implementation details of GCL.

Table 8-1 External Memory Used for Holding the Two Gate-Control Lists

Gate Control (up to 8 bits)	Time Interval (ns) 16, 20, or 24 bits	
OOCCCCCC	T0	HWOL or SWOL
OOOOCCCC	T1	
CCCCOOCC	T2	
CCCCCCOO	T3	
I	I	
I	I	
OCOCOCOO		
OOOOCCCC	T last	
OOOOCCCC	T0	SWOL or HWOL

Table 8-1 External Memory Used for Holding the Two Gate-Control Lists (continued)

Gate Control (up to 8 bits)	Time Interval (ns) 16, 20, or 24 bits	
OOOOCCCC	T1	
OOCCCCCC	T2	
OOOOCCCC	T3	
I	I	
I	I	
OCOCCCCC		
OOCCCCCC	T last	

8.1.2.2 Registers Related to Gate Control List

Following are the set of 4 registers (one for each GCL) related to GCL. These registers are implemented through Indirect Addressing using the GCL_Control and GCL_Data registers.

1. 64-bit Base Time Register (BTR)
2. 40-bit Cycle Time (CTR)
3. m-bit Time Extension (TER)
4. n-bit [Gate Control] List Length (LLR) (n depends of the configured GCL depth)

8.1.2.2.1 Base Time Register (BTR)

Base Time Registers is a 64-bit register that holds the start time to execute the GCL. The format of the BTR is same as the PTP format (upper 32-bits holds time in seconds and lower 32 bits hold time in nano seconds). Once the execution of a given list begins, the implementation can update the value in BTR to indicate the next list execution begin time.

8.1.2.2.2 Cycle Time Register (CTR)

Cycle Time Register is a 40-bit register that holds the time at which the execution of the GCL must be repeated. The Cycle Time Register consists of an 8-bit value in seconds, and a 32-bit value in nanoseconds (similar to the PTP time format with truncated seconds register).

For a given Gate Control List the start time is "Base Time" + N * "Cycle Time" where N is an integer value representing the iteration number starting with 0 for first iteration. If the Gate Control List execution takes longer than the Cycle time, then the list is truncated at the Cycle Time and the subsequent loop begins at Cycle Time.

8.1.2.2.3 Time Extension Register (TER)

Time Extension Register (TER) is a m-bit (where m = Configured Time Interval width + 7) register that holds the amount of time (in nano seconds) the current Gate Control List can be extended before switching to the new Gate Control List. This avoids smaller fragments of the current list being executed before switching to a new list.

8.1.2.2.4 List Length Register (LLR)

List Length Register (LLR) is an n-bit register (when n is 7, 8, 9, 10, or 11 for a GCL configured depth of 64, 128, 256, 512, or 1024 respectively) that holds the integer value of the length of the GCL (that is the number of valid rows in each GCL). The processing of the GCL stops after the number of rows read equals to the LLR value.

8.1.3 Transmission Gating Implementation

A Bridge or an end station can be enhanced to allow transmission from each TC that is yet to be scheduled relative to a known timescale. To achieve this, a transmission gate is associated with each TC; the state of the transmission gate determines if the queued frames can be selected for transmission.

For a TC, the transmission gate can be in one of the following two states:

1. OPEN

   Queued frames are selected for transmission, in accordance with the definition of the transmission selection algorithm associated with the TC.

2. CLOSED

   Queued frames are not selected for transmission.

A frame on a traffic class queue is not available for transmission if the transmission gate is in the closed state or if there is insufficient time available to transmit the entirety of that frame before the next gate-close event associated with that queue.

The implementation has visibility to the current schedule of gate controls and the immediate next schedule of the gate controls. So the maximum Gate Open period does not exceed the sum of the two Time Intervals. This is because, a frame is selected for transmission only if the gate is currently Open and the duration of gate open interval is greater than the time taken to transfer the entire frame.

The implementation must know the frame size before the transmission, so that the transmission overruns can be avoided and only the frames that can complete are scheduled at all times.

The implementation adequately compensates for the implementation delays in the data transfer from the buffer to the line by offsetting the current time with all the relevant delays (provided by the CTOV field of MTL_EST_Control register). This ensures that the schedule provided is always accurately implemented at the line.

Note

- A 20 Byte overhead is added to the packet size to account for the IFG and Preamble overheads on the line. Therefore for a 128 Byte frame to be transmitted, the Gate Open window must at the least be able to accommodate148 Bytes.
- In 100 Mbps mode of operation, the rounding down error is about 0.4%. For example, the Gate open duration must be at least 1024 Bytes for transmitting 1000 Byte Frames at a speed of 100 Mbps (20 Bytes for line overheads, and 4 Bytes for rounding down error)

8.1.4 Idle Slope Computation Updates

When EST is enabled, credit is accumulated only when the gate is open; therefore, the effective data rate of the idleSlope must be increased to reflect the duty cycle for the transmission gate associated with the queue.

The idleSlope is computed based on the GateOpenTime and OperCycleTime values. Program the idleSlope registers (implemented one per CBS enabled TC) based on the following equation. The existing MTL register has sufficient bit width to accommodate the new values for idleSlope.

```
idleSlope = (operIdleSlope(N) * OperCycle/GateOpenTime)
```

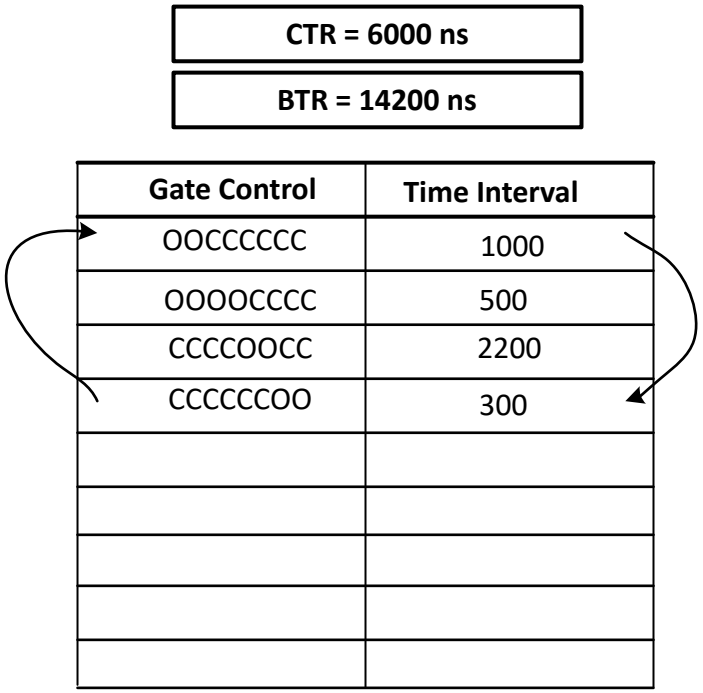
8.1.5 Operational Details of GCL

Set the switch to software List, the SSWL field of the MTL_EST_Control Register, so that hardware can access the programmed gate control list. The first set of gate controls are applied when the current time is equal to the value in the Base Time Register (BTR) and is held until the programmed "Time Interval" value.

To avoid transmission overruns, one additional gate control event is always read ahead from the list. This enables the GCL to determine the next gate close events (if any) for the TCs that are open.

The scheduling based on the Gate Open state and Time Interval of only the current and subsequent schedule. An internal accumulator is used to add the time intervals when gate controls are applied. BTR + Accumulator holds the time at which the next set of gate controls are to be applied.

Figure 8-4 GCL and Associated Registers - BaseTime and CycleTime



The GCL is read progressively from the first row adhering to the schedule. The read operations continue until the List Length (from LLR Register) is reached and the execution of the list restarts at BTR + CTR time. At this point the BTR is incremented by value in CTR to mark the beginning of a new cycle. In the absence of any gate controls, all the gates are deemed to be in Open state, during the execution of the list.

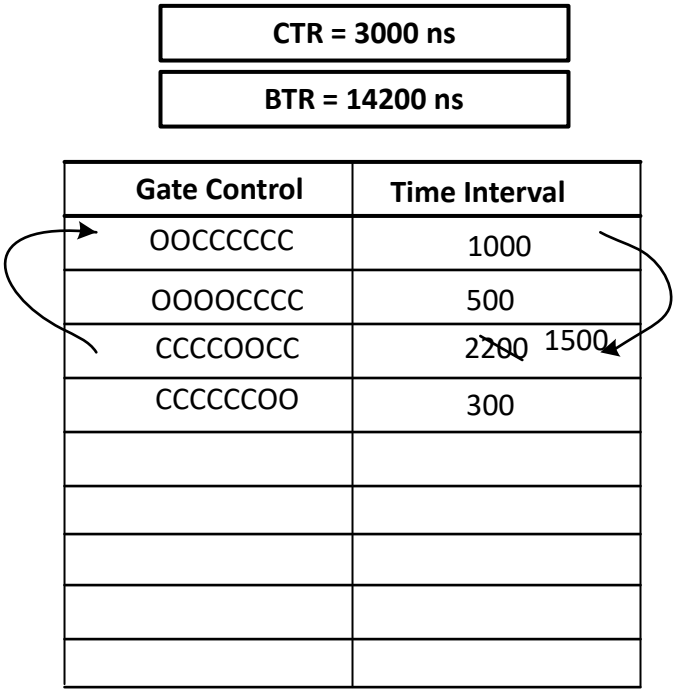
In cases where the execution time of the list is greater than the CycleTime, the list is truncated and the next iteration starts when the current time equals BTR + CTR.

Table 8-2 GCL and Associated Registers - BTR and CTR

Current Time	Gate Control Applied	Accumulator Value	BTR (with updates)
14200	OOCCCCCC	1000	14200
15200	OOOOCCCC	1500	14200
15700	CCCCOOCC	3700	14200
17900	CCCCCCOO	4000	14200
18200	OOOOOOOO	0	20200
20200	OOCCCCCC	1000	20200
21200	OOOOCCCC	1500	20200

In the example in Table 8-2, the execution starts at 14200 and the first set of Gate Controls "OOCCCCCC" are applied immediately. As the time interval is 1000ns the next set of gate controls are applied at 14200 (BTR) + 1000 (Accumulator) = 15200 ns as shown in Figure 8-5. As there are no gate controls available after the execution of the last gate control and before the next iteration of the loop, the gates are deemed to be in open state during that time period as depicted at time 18200 in Table 8-2.

Figure 8-5 GCL and Associated Registers - BaseTime and CycleTime List Execution Time > CycleTime



As the list execution takes longer than the allocated CycleTime, the list is truncated and the list is started from the BTR+CTR as shown in Table 8-3.

Table 8-3 GCL and Associated Registers - BTR and CTR, Execution Time > Cycle Time

Current Time	Gate Control Applied	Accumulator Value	BTR (with updates)
14200	OOCCCCCC	1000	14200
15200	OOOOCCCC	1500	14200
15700	CCCCOOCC	3700	14200
17200	OOCCCCCC	1000	17200
18200	OOOOCCCC	1500	17200
18700	CCCCOOCC	3700	17200

While applying the third set of gate controls, the BTR + CycleTime (17200) < BTR + Accumulator (17900), so the list is truncated and execution switches to a new iteration at 17200.

8.1.6 Installing a New GCL

When a new software programmed GCL is available and must be executed at the new BTR value, the switch to the new GCL can happen in one of the following ways.

- New Base Time aligned with current schedule
- New Base Time unaligned with current schedule

8.1.6.1 New Base Time Aligned with Current Schedule

If the choice of cycle time for the new gating cycle is unchanged from the cycle time for the current gating cycle, and if the BTR chosen for the new gating cycle (new BTR) is an integer multiple of the current cycle time (+ current BTR), then the new gating cycle exactly lines up with the old gating cycle, that is, the cycle start times for the new gating cycle is same as they would have been for the old configuration. This could be considered to be the ideal case and allows the new gating cycle to be installed and executed with no timing issues. The implementation completes the execution of an iteration of the current list and switches to the new list at the beginning of the BaseTime listed in the new list.

If (New BaseTime > = Current Time) ConfigChangeTime = New BaseTimeElse If (New BaseTime < Current Time)

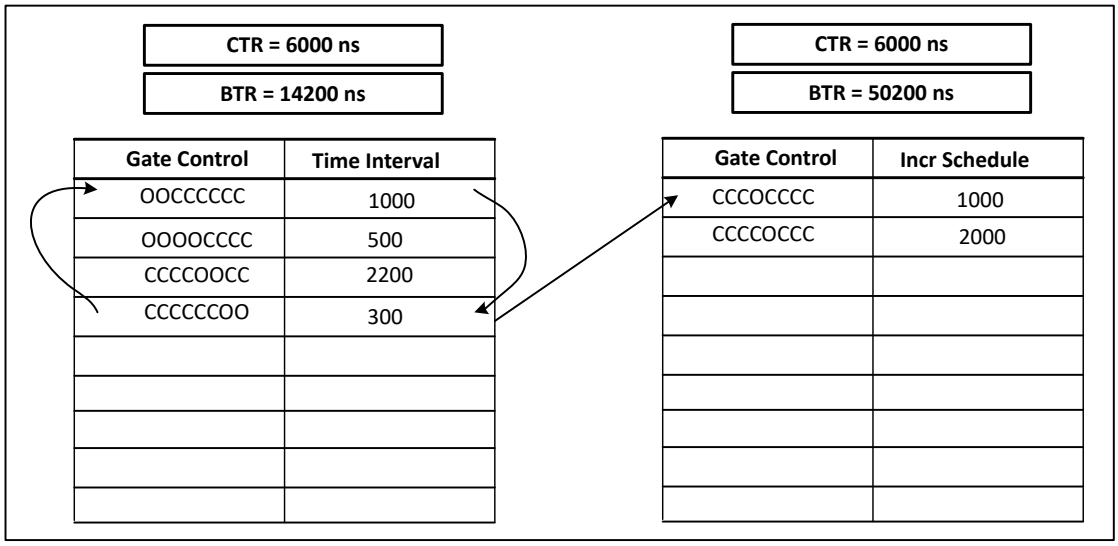
1. Set the BTRError
2. ConfigChangeTime = (New BaseTime + N* New CycleTime)

   where N is the smallest integer for which the relation ConfigChangeTime >= CurrentTime and (N =< 512) is TRUE.

   When N > 512, the hardware cannot auto recover and the loop count value in BTRError reporting is set to 11111111 requiring the software to reprogram the New Base Time.

   Figure 8-6 illustrates the installation of the new GCL along with the timelines.

Figure 8-6 Switching to a New Configuration that is Aligned with the Existing Configuration



In the example in Figure 8-6, after the sixth iteration of the first GCL, the BTR values of the old and new GCL are equal. At that point the new GCL is processed as a natural extension to the existing GCL.

Table 8-4 GCL and Associated Registers - BTR and CTR

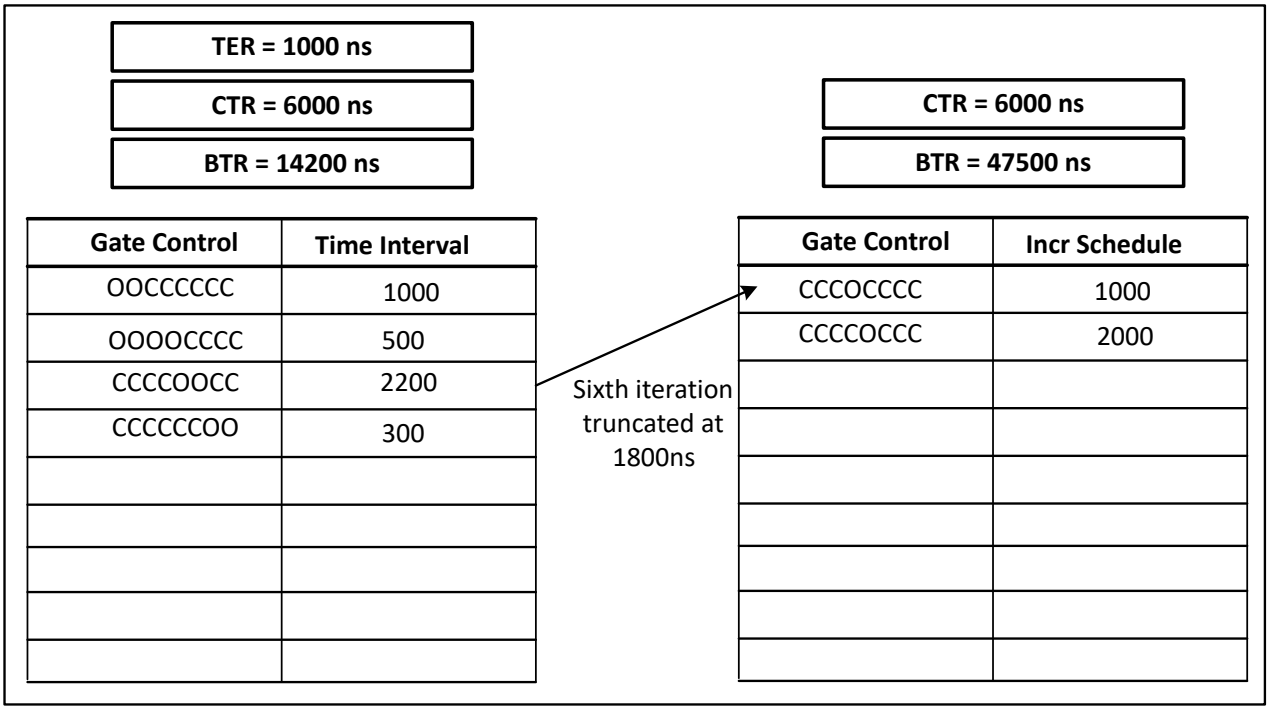
Current Time	Gate Control Applied	Accumulator Value	BTR (with updates)
44200	OOCCCCCC	1000	44200
45200	OOOOCCCC	1500	44200
45700	CCCCOOCC	3700	44200
47900	CCCCCCOO	4000	44200
48200	OOOOOOOO	0	50200
50200	CCCOCCCC	1000	50200
51200	CCCCOCCC	3000	50200
53200	OOOOOOOO	0	56200

8.1.6.2 New Base Time Unaligned with Current Schedule

If new CycleTime differs from current CycleTime or New BaseTime is in the future and is not an integer multiple of current CycleTime, then the old and new cycles do not line up; when new BaseTime is reached (when the new configuration is installed and starts to execute), the last old cycle is normally truncated to start the first new cycle. This could be undesirable if it results in a very short last old cycle; arguably it would be better to simply extend the penultimate old cycle by that small amount, rather than starting a very short cycle. The Cycle Time Extension Register (related to the current config list) allows this extension of the last old cycle to be done in a defined way; if the last complete old cycle ends normally in less than

current Cycle Time Extension (TER) ns before the new base time, then the last complete cycle before new BaseTime is reached is extended so that it ends at new BaseTime.

Figure 8-7 Switching to New List Truncating the Current List



At the end of the fifth iteration the Current time + Cycle time extension (TER) < New BTR so the sixth iteration of current configuration is started. During the sixth iteration of the current list when the new BTR value is smaller than the next schedule in the current list, it switches to the new list.

Table 8-5 Extending to New List By Truncating the Current List

Current Time	Gate Control Applied	Accumulator Value	BTR (with updates)
44200	OOCCCCCC	1000	44200
45200	OOOOCCCC	1500	44200
45700	CCCCOOCC	3700	44200
47500	CCCOCCCC	4000	44200
48500	CCCCOCCC	0	50200
50500	OOOOOOOO	1000	50200

In Figure 8-8, the current configuration list is extended instead of starting a new iteration as the extension time of 800ns is less than the allowed cycle extension time (TER) of 1000ns.

Figure 8-8 Switching to New List By Extending the Current List

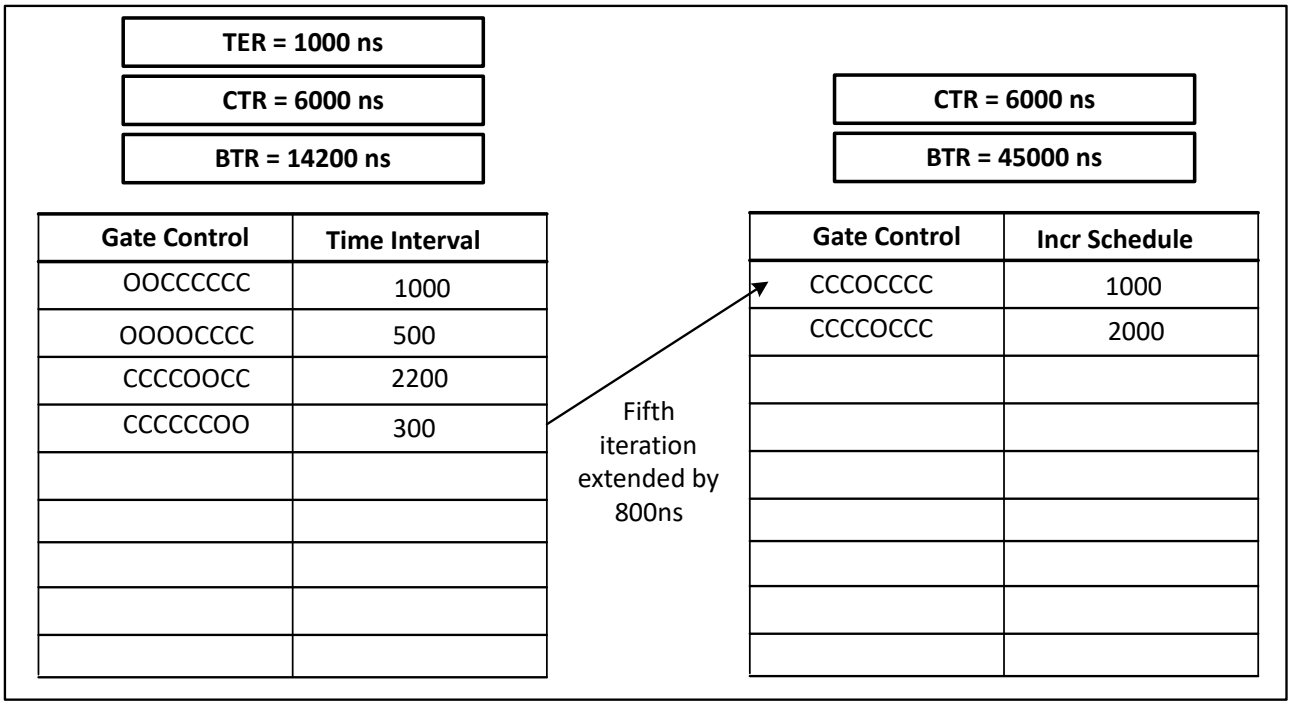


Table 8-6 Switching to New List By Extending the Current List

Current Time	Gate Control Applied	Accumulator Value	BTR (with updates)
38200	OOCCCCCC	1000	38200
39200	OOOOCCCC	1500	38200
39700	CCCCOOCC	3700	38200
41900	CCCCCCOO	4000	38200
42200	OOOOOOOO	0	44200
45000	CCCOCCCC	1000	45000
46000	CCCCOCCC	3000	45000
48000	OOOOOOOO	0	51000

8.1.7 Impact of Transmit Scheduling Algorithms on EST

When EST is used in isolation, the Gate Control List manages the final open/close state of the Transmit Class along with the checks carried out by the Transmission Selection Algorithm in MTL. As the Gate Controls operate on a predefined repetitive schedule, it is recommended to use Strict Priority or Credit Based Shaper (CBS) scheduling algorithms.

Other algorithms such as the Weighted Round Robin (WRR), Deficit Weighted Round Robin (DWRR), and Weighted Fair Queuing (WFQ) implement masking of the transmit channels based on the current winning transmit channel. The algorithm is applied only among the group of transmit channels that open simultaneously. To ensure transmit channels whose gates are "Open" get priority, these algorithms are modified to treat "gate open" transmit channels and "gate closed" transmit channels as separate groups giving priority to gate open transmit channels.

For example, consider 4 transmit channels (TC3, TC2, TC1, TC0) with weights 4:3:2:1; TC3 & TC2 are in Open state in one slot, while TC1 & TC0 are in Open state in another slot. In this case, the scheduler works as follows:

1. In the first slot, the TC3 & TC2 are serviced in the ratio of 4:3 for the duration the slot is open.
2. In the second slot, the transmit channels TC1 & TC0 are serviced in the ratio of 2:1.
3. Fresh arbitration is started every time a slot is opened.

In other words, the traffic does not get distributed in the intended ratio of 4:3:2:1; but as two groups with different ratios and only for the duration of the slot when the gates are open continuously.

When multiple queues map to one TC, DWC_25gmac has a first-stage scheduler (Round Robin) for the queue to TC selection. When EST is enabled, 25GMAC uses an enhanced Round Robin. Based on the open window size of EST, the scheduler masks the round robin for queues whose frames do not fit in the window size. This ensures that the scheduler picks the correct queue in the Round Robin, considering the frame sizes too.

8.1.8 Error Reported in EST

Following are the errors reported in EST

- BTR Programming Error

  This error indicates that the value of the New BTR is less than the current time. 25GMAC attempts to recover from this error by recursively adding the New Cycle Time to the New BTR such that

  For more details, see the BTRL field of the MTL_EST_Status register in the *"Register Description"* chapter.

- Constant Gate Control Error

  This error occurs when the list length (LLR) is 1 and the programmed Time Interval (TI) value after the optional Left Shifting is more than or equal to the Cycle Time (CTR).

  This implies that the Gates are either always Closed or always Open based on the Gate Control values. As this is not supported, it is reported as an error.

8.1.9 Interrupts in EST for Stream-Gate Filtering

Program the MTL_SGF_Interrupt_Enable register appropriately, to enable interrupts. The field position in this register to enable the interrupts correspond to the field positions in the MTL_SGF_Status register. When interrupt generation is enabled, the following events trigger the generation of an interrupt.

8.1.10 Programming Guidelines for EST

For details about programming guidelines for EST, see "Programming Guidelines for EST" on page 1883.

8.2 Frame Preemption (FPE)

8.2.1 Overview of the Frame Preemption

The IEEE 802.1Qbv-2015 defines the schedule for each of the queues on every egress port which makes the implementation aware of traffic arrival schedule. This information can be used to block the lower priority traffic from transmission in this time window/slot. This ensures that scheduled traffic is forwarded from sender to receiver through all the network nodes with a deterministic delay.

One of the important requirements to achieve a low latency is to ensure there are no interfering frames during the scheduled windows that are reserved for high priority traffic. The use of scheduled traffic imposes limitations while starting a transmission.

If an interfering frame begins transmission just before the start of a reserved time period, it can extend transmission into the reserved window, and potentially interfere with higher priority traffic. Therefore, a guard band whose size is equal to the largest possible interfering frame is required before the window starts.

A larger guard band equates to a less efficient use of network bandwidth. However, with the implementation of IEEE802.1Qbu (frame preemption), this issue is addressed. Frame preemption breaks the interfering frame into smaller fragments. Therefore, the guard band needs to be only as large as the largest possible interfering fragment instead of the largest possible interfering frame.

During the guard band, only the frames that can complete the transmission of the entire frame before the next gate close event are permitted. This ensures that the high priority traffic can always start at the beginning of the window reserved for it.

8.2.2 Description of the Frame Preemption

You can select the Frame Preemption feature (FPE) feature during RTL configuration of the controller. FPE feature can be selected independently with or without selecting the EST feature.

Preemption allows one or more higher priority (express) frames to interrupt the transmission of a lower priority (preemptable) frame; the preemptable frame transmission is resumed and completed after the express frame transmission is complete. To support frame preemption, the following two abstractions of the MAC are used:

- A preemptable MAC, called pMAC, which carries the preemptable traffic.
- An express MAC, called eMAC, which carries the express traffic.

In the implementation, only parts of the MAC that holds the state during preemption is replicated and represented as pMAC and eMAC. The MAC uses the following two ways to puts on hold, the transmission of the preemptable traffic, in the presence of express traffic:

- The MTL scheduler interrupts the preemptable traffic that is currently being transmitted.

  When the preemption capability is active, the MAC interrupts the transmission and reception of preemptable frames. A preempted fragment can be followed by zero or more express frames, before the continuation fragments. The preemptable frame can be fragmented any number of times, however, the minimum final and non-final fragment length criterion must be is met.

  However, interleaving of more than 1 preemptable packet is not permitted. This implies that if a preemptable packet is fragmented by an express packet, another preemptable packet cannot be transferred until all the remaining fragments of the first preempted packet are transferred.

- The MTL scheduler prevents starting the transmission of preemptable traffic.

  When the preemption capability is inactive, the pMAC entity is disabled and only express traffic is transmitted or received.

8.2.3 Enabling the Frame Preemption

Enable the frame preemption feature by setting EFPE field of the MAC_FPE_CTRL_STS register.

8.2.4 GCL Modification to Support FPE

In the EST-only configuration, the GCL entry has up to 24 bits of Time Interval and up to 8 high order bits representing the Gate Open/Close state requirements as shown in Figure 8-9.

Figure 8-9 Gate Control List When FPE is Disabled

Gate Control (up to 8 bits)	Time Interval(ns) 16, 20 , or 24 bits
OOCCCCCC	T0
OOOOCCCC	T1
CCCCOOCC	T2
CCCCCCOO	T3
|	|
|	|
OCOCOCOO	T126
OOOOCCCC	T127

EST only supports SetGate operation, which implies that the gates are set to either open or close at a given time interval.

However, when both Frame Preemption (FPE) and EST are enabled, the GCL also supports Set-and-Hold-MAC and Set-and-Release-MAC operations, in addition to the SetGate operation.

To enable the support of hold and release operations, the format of the GCL is slightly changed while configuring and enabling the FPE. The first traffic class (TC0) is always programmed to carry preemption traffic and therefore it is always Open. The bit corresponding to TC0 is renamed as Release/Hold MAC control.

The TX Queues whose packets are preemptable are indicated by setting the PEC field of the MTL_FPE_CTRL_STS register. The GCL field of the corresponding Queue is ignored and considered as always "Open". So, even if the software writes a "0" ("C"), it is ignored for such queues.

Figure 8-10 Gate Control List When FPE is Enabled

Gate Control (up to 7 bits)	Release/Hold Indication	Time Interval(ns) 16, 20, or 24 bits
CCCOOOO	0	T0
CCCCOOO	0	T1
OCCCOOO	1	T2
COCCOOO	1	T3
CCOCOOO	1	T4
CCCOOOO	0	T5
CCCCOOO	0	T6

When the Release/Hold bit transitions from a '0' to '1', a Set-and-Hold-MAC operation is performed. This marks the cease of the preemptable traffic. This is achieved by sending a Hold request to MTL "ha" ns in advance (where ha is the time interval mentioned in the Hold Advance (HADV) field of the MTL_FPE_Advance register). Set the LBHT field (bit[7]) of the MTL_FPE_CTRL_STS register to 1, to start Hold Operation from second row (when the Hold-Release field is 1) after new GCL is installed, irrespective of the value of Hold-Release field in the first row of GCL; that is, 0 to 1 transition on the Hold-Release field is not necessary.

When the Release/Hold bit transitions from a '1' to '0' a Set-and-Release-MAC operation is performed. This marks the resuming of the preemptable traffic. This is achieved by sending a Release request to MTL "ra" ns in advance (where ra is the time interval mentioned in the Release Advance (RADV) field of the MTL_FPE_Advance register). The preemptable traffic is blocked for the time duration the Release/Hold bit is set to a '1' in the Gate Control List.

Note The value of HADV must be equal to the time required to transmit minimum_fragment_size + IPG/EIPG + Preamble, in nanosecond.

8.2.5 Impact of Preemption on CBS

In CBS, the definition of "Transmit" is as follows:

- TRUE for the duration of frame transmission from the queue;
- FALSE when frame transmission from the queue is complete.

When CBS algorithm is used along with frame preemption, the value of "Transmit" is TRUE only while the frame is being transmitted by the MAC. If the frame transmission is delayed or interrupted (for instance, a preemptable frame transmission is interrupted to allow the transmission of an express frame from a different queue, or the start of express frame is delayed because a preemptable frame is being transmitted) the value of "Transmit" is FALSE until transmission of the frame commences or is resumed.

Also, the value of "Transmit" is FALSE during the transmission of any overhead that is a consequence of frame preemption. For example, additional frame overhead (mCRC, Fragment Count) that is added to the preemptable frame.

At any given time, if there are no frames in the queue, and the value of Transmit is FALSE, and credit is positive value, the credit value is set to zero if here is no preemptable frame from the queue for which transmission is in progress but has been interrupted.

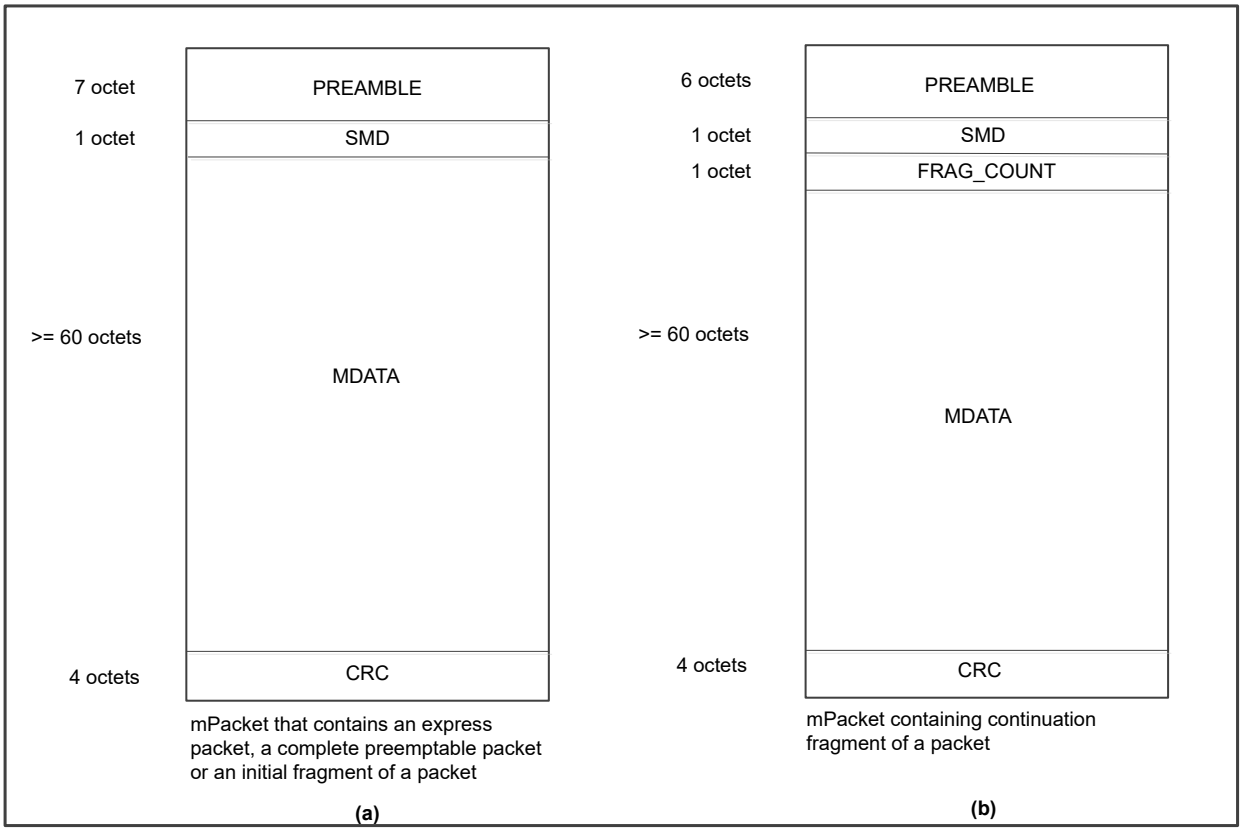
8.2.6 mPacket Format

When the preemption capability is active, MAC sends mPackets to the PHY. An mPacket can be one of the following:

1. A express packet
2. A preemptable packet
3. An initial fragment of a preemptable packet
4. A continuation fragment of a preemptable packet

Figure 8-11 shows the format of the mPacket. It contains an express packet, a complete preemptable packet or the initial fragment of a preemptable packet. Figure 8-11(b) shows the format of an mPacket containing a continuation fragment of a packet.

Figure 8-11 mPacket Formats



Preamble

The preamble in the mPacket shown in Figure 8-11 (a) contains seven octets. The preamble in the mPacket shown in Figure 8-11 (b) contains six octets. Each octet contains the value of 0x55 (transmitted in order from left to right 10101010).

Start mPacket Delimiter (SMD)

The value of the SMD indicates whether the mPacket contains an express packet, the start of a preemptable packet (initial fragment or complete packet), or any of continuation fragments of a preemptable packet. Table 8-7 shows the valid SMD values.

Table 8-7 Possible SMD Values of mPacket

mPacket Type	Notation	Frame Count	Value
verify packet	SMD-V	-	0x07
respond packet	SMD-R	-	0x19
express packet	SMD-E	-	0xD5
preemptable packet start	SMD-S0	0	0xE6
	SMD-S1	1	0x4C
	SMD-S2	2	0x7F
	SMD-S3	3	0xB3
continuation fragment	SMD-C0	0	0x61
	SMD-C1	1	0x52
	SMD-C2	2	0x9E
	SMD-C3	3	0x2A

frag_count

A frag_count is a modulo-4 counter that increments for each continuation fragment of the preemptable packet. The frag_count protects against mPacket reassembly errors by enabling detection of the loss of up to 3 packet fragments.

The frag_count field is present only in mPackets with SMD-C notation (continuation fragment). The frag_count is zero in the first continuation fragment of each preemptable packet.

Table 8-8 Possible frag_count Values

frag_count	Value
0	0xE6
1	0x4C
2	0x7F

Table 8-8 Possible frag_count Values (continued)

frag_count	Value
3	0xB3

mData

The contents of the packet from the MAC, starting with the first byte after the SFD to the last byte before the FCS are sent in the mData fields of one or more mPackets for that frame. The minimum size of the mData field is 60 bytes.

CRC

The CRC field contains a cyclic redundancy check (CRC) and has an indication of the final mPacket of a frame. In the final mPacket of a frame, the CRC field contains the last 4 octets of the MAC frame (the FCS field).

For other mPackets, the CRC field contains an mCRC value. The mCRC is calculated on the octets of the packet from the first octet of the frame (the octet following the SFD of preemption frames) to the last octet of the packet transmitted in that mPacket by performing an XOR of the calculated 32 bit CRC value of the fragment and a value of 0x0000FFFF.

Summary of Packet Formats

- Express Packet

  7 bytes of PREAMBLE, SMD-E, Data, and CRC

- Complete preemptable Packet

  7 bytes of PREAMBLE, <Current preemptable packet SMD>, Data, CRC

- Initial Fragment (non-final) of preemptable Packet

  7 bytes of PREAMBLE, <Current preemptable packet SMD>, Data, mCRC

- Continuation fragments (non-final) of preemptable packet

  6 bytes of PREAMBLE, <Current preemptable continuation fragment SMD>, <Current preemptable continuation fragment FC>, Data, mCRC

- Final fragment of preemptable packet

  6 bytes of PREAMBLE, <Current preemptable continuation fragment SMD>, <Current preemptable continuation fragment FC>, Data, CRC

Note	When Frame Preemption is enabled, the MAC receiver communicates the exceptions related to received preempted fragments (incorrect fragment sequencing, missed fragment, and so on) to MTL layer by reporting it as CRC error. Software must not set the DCRCC (Disable CRC Checking for Received Packets) bit of MAC_Ext_Configuration register to 1; otherwise the MTL layer might forward the unintended preempted fragments to the application.

Table 8-9 Current and Previous SMD Values for Preemptable Packet

Previous Preemptable Packet SMD	Current Preemptable Packet SMD
SMD-S0	SMD-S1
SMD-S1	SMD-S2
SMD-S2	SMD-S3
SMD-S3	SMD-S0

Table 8-10 Current and Previous SMD Values for Preemptable Fragment

Previous Preemptable fragment SMD	Previous Preemptable fragment FC	Current Preemptable continuation fragment SMD	Current Preemptable continuation fragment FC
SMD-S0	NA	SMD-C0	FC0
SMD-S1	NA	SMD-C1	FC0
SMD-S2	NA	SMD-C2	FC0
SMD-S3	NA	SMD-C3	FC0
SMD-C0	FC0	SMD-C0	FC1
SMD-C0	FC1	SMD-C0	FC2
SMD-C0	FC2	SMD-C0	FC3
SMD-C0	FC3	SMD-C0	FC0
SMD-C1	FC0	SMD-C1	FC1
SMD-C1	FC1	SMD-C1	FC2
SMD-C1	FC2	SMD-C1	FC3
SMD-C1	FC3	SMD-C1	FC0
SMD-C2	FC0	SMD-C2	FC1
SMD-C2	FC1	SMD-C2	FC2
SMD-C2	FC2	SMD-C2	FC3
SMD-C2	FC3	SMD-C2	FC0
SMD-C3	FC0	SMD-C3	FC1
SMD-C3	FC1	SMD-C3	FC2

Previous Preemptable fragment SMD	Previous Preemptable fragment FC	Current Preemptable continuation fragment SMD	Current Preemptable continuation fragment FC
SMD-C3	FC2	SMD-C3	FC3
SMD-C3	FC3	SMD-C3	FC0

8.2.7 Transmit Preemption

8.2.7.1 MTL Tx Preemption

To support preemption MAC must have more than 1 TX queue with at least 1 Queue designated as Express Queue. (and 1 queue designated as preemption queue). When FPE is enabled (setting EFPE=1 in MAC_FPE_CTRL_STS register), the MTL preempts a preemptable frame, when a “hold” request is asserted (EST/Qbv configured and enabled) express frames are available for transmission, that is, frame is present in MTL FIFO and is qualified for arbitration, after ensuring that the minimum mPacket mData field size is met. Therefore, preemption occurs only if at least 60 bytes of the preemptable frame have been transmitted and at least 64 bytes (including the frame CRC) remain to be transmitted.

The earliest starting position of preemption is controlled by the AFSZ field of the MTL_ FPE_CTRL_STS Register. Preemption does not occur until at least 64 * (1+ AFSZ) - 4 bytes of the preemptable frame have been sent.

When preemption occurs, all the preemptable queues are blocked and only the express queues are allowed to arbitrate (if more than one express queue has traffic) and transmit.

Continuation fragment of the preempted frame is the first frame to be transmitted after “release” request is asserted (EST/Qbv configured and enabled) and all the express traffic transmission completes.

Note All the PTP packets must be transmitted as express packets.

MTL communicates the following frame-type information to the MAC using a 2-bit preemption control signal on the MTI interface qualified by SoF and EoF.

- Express Frame
- Preemption Frame (Full or Fragment)
- Continuation Fragment (non-Final or Final)

Table 8-11 Preemption Control Values on MTI Interface for Various Frame Types

Qualifier	Preemption Control Value	Frame Type
SoF	00	Start Express
SoF	01	Start Preemption
SoF	10	Continuation Fragment
SoF	11	Reserved
EoF	00	End of Frame

Table 8-11 Preemption Control Values on MTI Interface for Various Frame Types (continued)

Qualifier	Preemption Control Value	Frame Type
EoF	01	End of Fragment

If the mti_sof_o and mti_eof_o are sent in the same cycle on the MTI interface, the mti_preemp_ctrl (Preemption Control Value) for EoF is taken as "00", End of Frame.

MTL must wait for the previous fragment status to be received before resuming the continuation fragments of a preempted frame. Fragment status is described in detail in the Tx Fragment Status section

8.2.7.2 MAC Tx Preemption

MAC supports preemption by implementing the functionality needed to generate the mPackets as described in “mPacket Format” on page 388.

Based on the Preemption Control value received on the MTI interface (qualified with SoF and EoF), MAC determines the frame type (shown in Table 8-7) and generates mPackets accordingly.

When the preemption capability is active, MAC replaces the SFD of a preemption packet with an SMD-S value. A 2-bit rolling frame count is encoded in the SMD-S value.

The SMD-E value is the same as the SFD value so the SFD of an express packet does not need to be replaced.

8.2.7.3 Tx Fragment Status

MAC sends Tx fragment status to indicate successful transmission of fragmented mPackets. To indicate a fragment status, bit-31 of the mti_txstatus_o field is set and all other values in the status field is set to ‘0’.

In case of a transmission error (underflow, jabber, and so on) the frame status is sent (and not a fragment status) with an error indication along with all other relevant status fields. The bit-31 of mti_txstatus_o is set to ‘0’ to indicate a Frame Status. In case of receiving an error status for a transmitted fragment, the MTL drops the remaining fragments and does not send any more continuation fragments.

8.2.8 Receive Preemption

8.2.8.1 MAC Receive Preemption

When FPE is enabled, the MAC Receiver pases the incoming packets and differentiates between Express packets and preemptable packets. An SMD containing an SMD-E indicates express packet, and SMD containing an SMD-S indicates the first mPacket of a preemptable packet.

If an mPacket containing an SMD-S is received when MAC has not completed receiving the previous preempted packet, MAC sets a CRC Error status for the previously received partial packet.

When an SMD-S is detected, MAC records the frame count indicated by the SMD and then begins sending data on the MRI interface.

The MAC checks the last four bytes of the mPacket . If the last four bytes of the mPacket do not match CRC, that indicates the end of the packet with or without a CRC error as per the CRC check result. If the last four octets of the mPacket match, that indicates that the packet was preempted.

An SMD containing an SMD-C indicates an mPacket that continues the data for a preempted packet. Upon receiving an SMD value of SMD-C, MAC checks the following:

1. A preempted packet is in progress
2. The frame count indicated by the SMD matches the frame count of the packet in progress
3. The frag_count value indicates the next fragment count.

If any of these checks fail, the mPacket is discarded and MAC sets a CRC Error status for the partially received packet.

If all the checks pass, the next fragment count is incremented modulo 4.

When a packet is preempted, the MAC saves the state of the partially received packet (filter check status, timestamp, length fields and so on) and processes the received Express packets until the continuation fragment is received.

The MAC Receiver sends a "placeholder status" (all zeros in mri_rxstatus_o) for all the mPacket fragments successfully received and sends the Rx status with the final fragment. If an error is detected during any of the fragments the Rx status is sent and the fragment is marked as final fragment. All subsequent continuation fragments received for this packet are dropped in the MAC.

The MAC communicates the following frame type information to the MTL using a 2-bit preemption control signal on the MRI interface qualified by mri_sof_o and mri_eof_o.

1. Express Frame
2. Preemption Frame (Full or Fragment)
3. Continuation Fragment (non-Final or Final)

Table 8-12 Preemption Control Values on MRI Interface for Various Frame Types

Qualifier	Preemption Control Value	Frame Type
SoF	00	Start Express
SoF	01	Start Preemption
SoF	10	Continuation Fragment
SoF	11	Reserved
EoF	00	End of Frame
EoF	01	End of Fragment

If the mri_sof_o and mri_eof_o are sent in the same cycle on the MRI interface, the mri_preemp_ctrl (Preemption Control Value) for EoF is taken as '0' indicating End of Frame.

Note

Minimum Preemptive fragment size supported on Rx is 48 bytes

The preemption fragments of minimum 48 bytes can be received with minimum Inter Packet Gap (IPG) of

- 24 bytes in the XGMII speed mode (6 IDLE columns including TERM character)
- 18 bytes in the GMII speed mode

If there are no exceptions in the preemption fragments of size between 48 to 64 bytes, the minimum IPG supported is

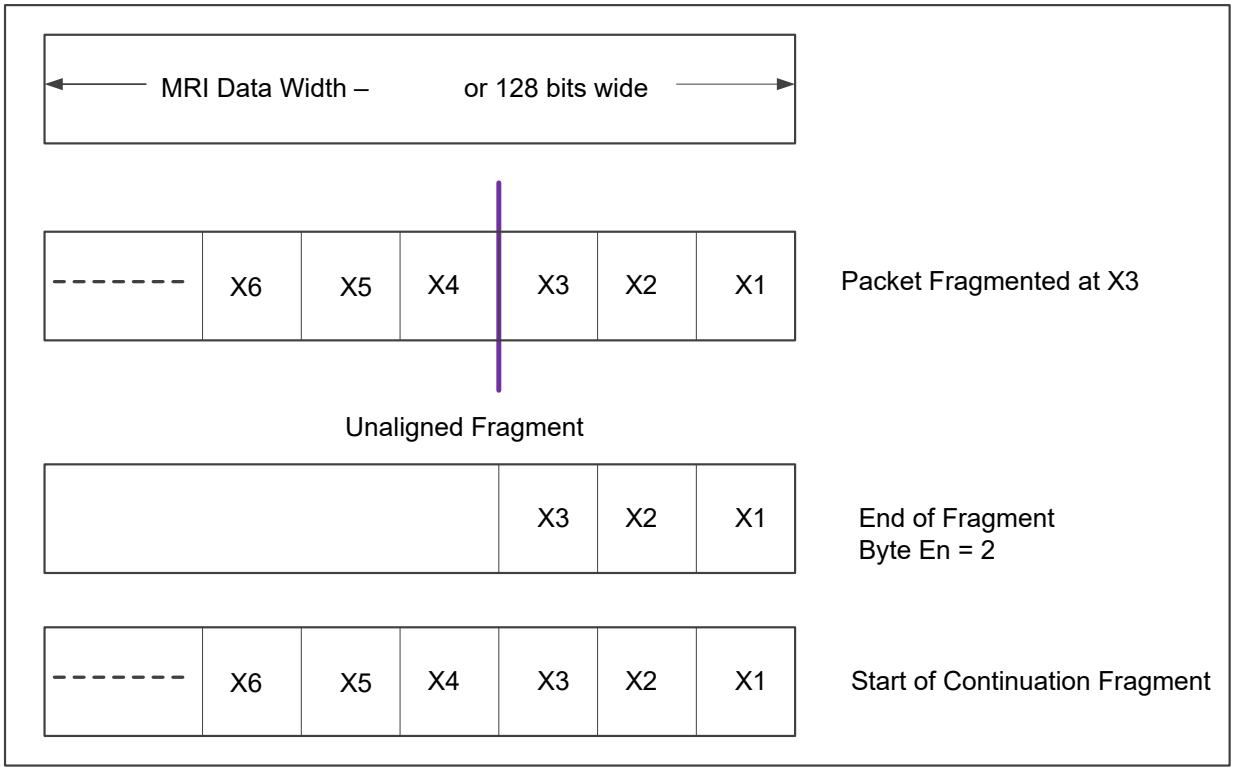
- 20 bytes in the XGMII speed mode (5 IDLE columns including TERM character)
- 17 bytes in the GMII speed mode

8.2.8.2 Data Alignment

When a received frame cannot be fragmented on any byte boundary, MAC retains the unaligned bytes of data in the previous fragment and resends them with the next fragment as shown in Figure 8-12.

MTL can choose to ignore the partial data (based on Byte Enable value) that comes with End of Fragment and instead use the "data width aligned" data received in the first beat of the next continuation fragment.

Figure 8-12 Data Alignment Feature of MAC



8.2.8.3 MTL Receive Preemption

The MTL RX must have at least 2 Receive Queues to support FPE function as the preemptable packets and the express packets must be routed to separate RX Queues. The destination RX Queue of a received packet is controlled by MAC_RxQ_Ctrl(#i) registers. These must be carefully programmed such that preemptable traffic and express traffic are not routed to the same RX queue. As the Queue mapping for tagged packets is based on VLAN user-priority field, this means that priority of preemptable and express packets are exclusive to each other. In other words, packets of a certain priority (traffic class) are either express or preemptable but cannot be both.

If MCBCQEN of MAC_RxQ_Ctrl1 register is set, preemptive Multicast/Broadcast packets are routed to PMCBCQ as programmed in MAC_RxQ_Ctrl4. Therefore, ensure that PMCBCQ is not same as any of the queues programmed for express queues.

Preemption traffic routing is carried out as shown in Table 8-13.

Table 8-13 Preemption Control Values on MRI Interface for Various Frame Types

Preemption Packet Type	Queue Routing
Multicast and Broadcast Preemptive Frames passing the SA/DA and VLAN filters with MCBCQEN field of MAC_RxQ_Ctrl1 register is set	PMCBCQ
AV Tagged Packets passing the SA/DA and VLAN filters	PSRQ[b]
DCB/Generic Tagged Packets passing the SA/DA and VLAN filters	PSRQ
Tagged Packets failing the SA/DA and VLAN filters without setting the Receive All (RA) or setting VTFE	Dropped
All other packets	RQ

a. Only if the queue is enabled for AV feature
b. Only if the queue is enabled for DCB/Generic feature
c. Only if the queue is enabled for AV feature

In addition to the PSRQ (Priority Selected in Rx Queue) based Queue routing and PMCBCQ (preemptive multicast/broadcast queue), a programmable “Residue Queue” (RQ) is provided to route all other preemptive packets received (Untagged, SA/ DA or VLAN Filter Fails forwarded due to RA (receive all) being set or VTFE being reset). In case of filter fail, Queue 0 is used as the default queue. So Q0 must not be used by the preemptive traffic routing. Therefore, RQ cannot be programmed as 0.

8.2.8.4 MTL Receive Arbitration

On the ARI Interface, data is fetched from the MTL Rx FIFO based on the arbitration selected in the MTL_Operation_Mode register. Frame based arbitration can be used only when all the MTL Preemption Queues are operating in Store and Forward mode. Otherwise there is loss of bandwidth on the ARI interface because all the fragments of a preemptable packet are not available. Therefore, express packets received between the fragments are blocked until all the fragments are received and transferred; this defeats the purpose of express traffic.

When operating in either Threshold (cut through) or Store and forward modes of operation, PBL based arbitration is recommended over Frame based arbitration. In case of PBL based arbitration, the watermark check is always performed and the arbitration/transfer of data in terms of chunks of "PBL" size of data which is almost similar to the concepts of "fragments". Therefore the express queue packets get blocked for less time as well as the ARI interface transfers the data without loss of efficiency.

8.2.9 Received Preemption Frames Support in Offload Engines

When Frame Preemption is enabled, all the packets processed by the Offload Engines (ARP, PTO) and PMT, PTP, PAUSE, and PFC packets must be received as express/normal packets. These offload engine logic and functions do not recognize preemption frames.

8.2.10 Verify and Respond mPackets

When FPE function is present, the MAC can receive and detect the Verify and Respond mPackets, even when FPE is not enabled by software. When MAC detects valid Verify/Respond mPackets, it notifies the software by setting the RVER and RRSP fields of MAC_FPE_CTRL_STS register respectively. Optionally an interrupt can be generated. As such packets have empty (all-zero) data payload, they are dropped inside the MAC and not forwarded to the MRI.

Software can set the SVER and SRSP fields of MAC_FPE_CTRL_STS register to request MAC to transmit Verify and Respond mPackets respectively. Upon successful transmission of these frames, MAC clears the SVER/SRSP bits and sets the TVER & TRSP fields of MAC_FPE_CTRL_STS register. Optionally an interrupt can be generated when these events occur.

When Frame Preemption is enabled,

- For the preemption frames received with CRC error indication, ignore the following status fields
  - Packet length
  - Dribble error indication
  - Runt frame indication
- The preemption frames received with CRC error and with sizes ranging from 64 to 67 bytes are treated as runt packets. So, the corresponding MMC Rx Frame Counter for Runt Frames is incremented.
- Do not disable CRC checking on receive. This is required because, internally, FPE error fragments are identified using CRC checks. For more details, see the DCRCC field in the MAC_Ext_Configuration Register in the *Register Descriptions* chapter.
- Forward Undersized Good Packets (FUP) field of MTL_RxQ_Operation_Mode register must not be set. A new programmable mode bit (ARV) to auto-generate a Respond packet up on receiving a Verify packet is added.

8.2.11 Frame Preemption and MMC Counter and Interrupt Registers

The following MMC counters and associated Interrupt registers are instantiated/present in the MAC when Frame Preemption feature and any one of the existing MMC counters are selected during RTL configuration.

Table 8-14 MMC Counters and Associated Interrupt Registers

Frame Assembly Error Counter	Description	Associated MMC Counter
Frame Assembly Error Counter	A 32-bit counter that provides the number of MAC frames with reassembly errors on the Receiver, due to mismatch in the Fragment Count value.	MMC_Rx_Packet_Assembly_Err_Cntr
Frame SMD Error Counter	A 32-bit counter that provides the number of received MAC frames rejected due to arriving with an SMD-C when there was no preceding preempted frame	MMC_Rx_Packet_SMD_Err_Cntr
Frame Assembly OK Counter	A 32-bit counter that provides the number of MAC frames that were successfully reassembled	MMC_Rx_Packet_Assembly_Ok_Cntr
MAC Rx Fragment Counter	A 32-bit counter that provides the number of additional mPackets received due to preemption	MMC_Rx_FPE_Fragment_Cntr
MAC Tx Fragment Counter	A 32-bit counter that provides the number of additional mPackets transmitted due to preemption	MMC_Tx_FPE_Fragment_Cntr
Hold Request Counter	A 32-bit counter that maintains the count of number of times a hold request is given to MAC	MMC_Tx_Hold_Req_Cntr

8.2.11.1 Additional Registers Associated With MMC Interrupts

Following are the additional registers associated with MMC interrupts for the MMC error counters:

- MMC_FPE_Tx_Interrupt
- MMC_FPE_Tx_Interrupt_Mask
- MMC_Rx_Interrupt
- MMC_FPE_Rx_Interrupt_Mask

8.3 Time-Based Scheduling

Time-based scheduling feature is suitable for traffic whose periodicity and rate are predictable. This is an optional feature. To improve the quality-of-service of such traffic,

- The transmit DMA fetches the packet from the host memory for transmission at designated time. This helps the software to setup the Transmit descriptors in advance even before packet is ready/available. It reduces the overhead on the software and avoids constant monitoring of the time and preparing descriptors just in time when the packet is targeted to be transmitted.
- The MAC transmits the packet only at the designated/pre-determined time even if the packets are fetched in advance. This helps in maintaining a constant transmission rate that can be consumed by the receiver station; therefore avoiding congestion and excessive buffering in the network.

8.3.1 Description of Time-Based Scheduling

The time-based scheduling feature supports fetching and launching an Ethernet packet at (or after) a pre-determined time. The time-based scheduling is supported only in the following modes/configurations:

- Full duplex mode
- Link speed is 100Mbps or higher
- MTL and higher configurations
- Configurations in which time-stamping is enabled.

- Do not enable time-based scheduling (or enhanced descriptors) for the channel for which TSO or transmit timestamp or one-step timestamping control correction feature is enabled.
- When fetch time is enabled, overrides the audio-video slot function.
- In EST enabled configurations, the ESTM mode bit setting impacts the time-based scheduling feature only when the DUT processes the GCL list.
- When an unaligned new GCL list (with a different CTR) is installed, ensure that there is no traffic during the installation of the new list. Traffic during the switching of the lists might result in unpredictable behavior regarding Fetch, Launch, and Launch expiry as the CTR and BTR values are updated while the frame is being processed.

8.3.2 Definitions Used

Following are the definitions specific to the time-based scheduling feature:

- Launch Time

  The time beyond which MTL can schedule the packet for transmission.

- Fetch Time

  The time beyond which the TxDMA can schedule a packet-fetch from the host memory.

- Expiry Time

  The time beyond which the packet is dropped by MTL.

8.3.2.1 Launch Time

In the MTL configuration, you can specify the launch time as a Control Word for each packet.

In the AXI configuration, the launch time is valid only for the specific frame that is scheduled for transmission. The launch time is reset after the transmission of one frame. You can specify the launch time in the Transmit Enhanced Descriptor. For more details, see "Enhanced Descriptor for Time-Based Scheduling" on page 1829.

Following are the two formats of the launch time.

- Normal/Absolute Format
- EST/Offset Format

Normal/Absolute Format

In this format, the Launch time is an absolute time value at which the packet is launched for transmission. This format is supported when you enable the Time-Based Scheduling feature using the `DWCXG_TBS` parameter.

When you enable both Time-Based scheduling (DWCXG_TBS) and Enhancements to Scheduled Traffic, EST (DWCXG_AV_EST) features, the launch time is interpreted in the Normal/Absolute format if the ESTM bit of the MTL_TBS_CTRL register is set to 0.

The launch time is a 32-bit value, where most-significant 8-bits represent the time in seconds and the rest 24 bits represent the time in 256 ns. The launch time is compared against the IEEE 1588 based System/PTP Time (bits[39:8]) and rolls over after 256 seconds.

The maximum value of the lower 32-bits of System Time is 999,999,999 decimal (0x3B9AC9FF) and it wraps to 0 when reaching this value (representing a full second). Therefore, the maximum value of the lower 24 bits of the Launch Time (after multiplying by 256) must be 0x3B9AC9.

As the maximum value of Launch time is 256 seconds,

- Launch Time is greater than current System Time when its value is between System Time[39:8] and System Time[39:8] + 128 sec
- Launch Time is less than current System Time when its value is between "System Time[39:8] + 128sec" and "System Time[39:8] + 256 sec", because this is a modulo 256 computation.

EST/Offset Format

In this format, the Launch time is a offset value relative to the time indicated by the Base Time Register (BTR) of the GCL list provided in the GCL Slot Number (GSN). The value in the BTR is always updated to the start time of the current loop of the GCL.

This format is used when you enable both Time-Based scheduling (DWCXG_TBS) and Enhancements to Scheduled Traffic, EST (DWCXG_AV_EST) features. For each packet, the GSN value and the Launch Time value are specified in the Transmit Context Descriptor in the AXI configuration, or as Control Word in the MTL configurations.

The launch time offset is a 32-bit value; the upper 8-bits represent the time in seconds and the rest 24 bits represent the time in 256 ns, which is added to the BTR value corresponding to GCL Slot Number. The value of the Launch Time Offset must be smaller than the value of the Cycle Time (specified in the CTR register that is implemented when you enable EST).

If the CTR is greater than or equal to 1s, the maximum value of the lower-24 bits of Launch Time offset must be 999,999,999 decimal (0x3B9AC9FF).

In this format, the launch time is a 64-bit value, which is interpreted as,

Launch Time = Launch GSN BTR[63:0] + (Launch Time Offset[31:0] << 8), which is compared with the System Time [63:0].

GSN BTR is the Base Time value at which the Launch GSN loop started execution.

GCL Slot Number (GSN)

When you enable Time-Based scheduling (DWCXG_TBS) and Enhancements to Scheduled Traffic, EST (DWCXG_AV_EST) features, a modulo 16 count of the GCL loop count is implemented and known as GSN. The count is incremented for every new GCL loop; installation of new GCL list does not impact the count. The current GCL slot number can be obtained by reading the CGSN field of the MTL_EST_Status register.

The maximum value of GSN is 15. Therefore GSN values between Current GSN (CGSN field of MTL_EST_Status register) and CGSN+8 represent current or future slots; all other GSN values are interpreted as elapsed slots or past slots. So, for the correct interpretation of time, GSN value must be between CGSN and CGSN+8.

8.3.2.2 Launch Expiry Time

Normal/Absolute Mode

In the normal mode, when the LEOV (Launch Expiry Offset Valid bit of MTL_TBS_CTRL register) is set, the Launch Expiry Offset (LEOS field of MTL_TBS_CTRL register) determines the maximum amount of time a frame is eligible for launch, starting from the time the frame becomes eligible for launch.

The Launch Expiry Offset is a 24-bit value defined in 256ns units, with a maximum possible value of 999,999,999 ns (0x3B9AC9FF).

Launch Expiry Time = (Launch Time[39:8] + LEOS[32:8]) * 256ns

The packet with a specific Launch time is considered eligible for transmission when the Launch time is less than the System Time and (if LEOV is set) the System Time is less than the Launch Expiry time.

When the System time is greater than or equal to the Launch Expiry time, the frame is categorized as expired and is dropped from the MTL FIFO.

Note

1. For correct interpretation and meaningful operation, the Fetch, Launch, and Launch Expiry time must never be set to a value larger than Current System Time + 128 seconds; such a value is interpreted as time that has already elapsed.
2. In full duplex mode, the frames dropped from the MTL FIFO have Error Summary (Bit 15) and TBS Drop (bit 3) of Transmit Status set.

EST/Offset Mode

In the EST mode of operation, when the LEOV field of the MTL_TBS_CTRL register is set, the launch expiry GSN offset (LEGOS field of the MTL_TBS_CTRL register) and Launch Expiry Offset (LEOS field of MTL_TBS_CTRL register) determine the maximum amount of time a frame remains eligible for launch, starting from the time the frame becomes eligible for launch.

Launch Expiry Offset = (LEGOS: LEOS)

LEGOS holds the GSN offset (multiples of CTR time) and LEOS holds maximum value of CTR (sub CTR values) in ns.

Launch Expiry offset is a 24-bit value defined in the units of 256 ns, with a maximum possible value of the smaller of 999,999,999ns or CTR-1 ns.

The Launch Expiry GSN is computed as follows:

Launch Expiry GSN = (Launch GSN + LEGOS + CCMA)

Where,

CCMA is the CTR Carry due to Modulo addition. This value is 1 if ((Launch Time Offset + LEOS) << 8) is equal to or greater than CTR. When CCMA = 1,

Launch Expiry Offset = (Launch Time Offset+ LEOS) - CTR

When CCMA = 0,

Launch Expiry Offset = (Launch Time Offset + LEOS).

Launch Expiry Time = Launch Expiry GSN BTR[63:0] + Launch Expiry Time Offset.

When LEOV is not set, Launch Expiry Time is not checked.

When LEOV is set, and

- the system time is greater than the Launch Expiry time, the frame is dropped from MTL FIFO. The frame is considered as expired.
- the launch time is smaller than the System Time and Launch Expiry Time is greater than system time, the frame is considered eligible for launch.

Note

1. Max value of LEGOS is 7. This implies that when LEOV is set, the frame has a maximum life time of < 8 GCL loop iterations after it becomes eligible for Launch.
2. The slot number of the First GCL List executed each time after EST is enabled, is zero.
3. All computation in TBS EST mode is done with a granularity of 256 ns.

8.3.2.3 Fetch Time

In configurations with DMA, the Fetch Time can also be indicated for each packet by the software. This is done by setting FTOV field of `DMA_TBS_CTRLn` (where n = 0, 1, 2, 3) register.

If the FTOV filed is not set, the Fetch Time Offset is not valid and the DMA fetches packets without any time constraints.

The fetch time accounts for all possible delays in the DMA fetch operation and ensures that the frame is present in the MTL FIFO before the launch time.

Note

1. If (DWCXG_NUM_DMA_TX_CH <= 4)

   DMA_TBS_CTRLn (where n = 0, 1, 2, 3) are mapped with DMA Transmit Channel based on the channel number
2. If (DWCXG_NUM_DMA_TX_CH > 4)

   The application must program the respective channel's TFSEL bit of the VDMA_CH(#i)_TX_Control register to select the specific DMA_TBS_CTRLn register to use.

Normal/Absolute Mode

In the Normal mode fetch time derived/calculated by reducing the time specified in the Fetch Time Offset (FTOS) field of DMA_TBS_CTRL register from the given Launch time.

Normal mode of operation, the Fetch Launch Time is computed as:

Fetch Time[39:8] = (Launch Time[39:8] - FTOS[31:8])

The Fetch time is 32-bits and is compared against System Time[39:8] to determine the Eligible for fetching the frame:

- The Fetch time is defined as "greater" than System Time if the Fetch Time is in the range of "System Time[39:8]" and "System Time[39:8] + 128 seconds". The frame is considered as not-eligible for fetch.
- The Fetch time is defined as "smaller" than the System Time if the Fetch time is in the range of "System Time[39:8] + 128 sec" and "System Time[39:8] + 256 seconds". The frame is considered as eligible for fetch.

This is a modulo 256 computation.

EST/Offset Mode

In the EST/Offset mode, the Fetch GSN Offset (FGOS field of MTL_TBS_CTRL register) provides the Slot Number offset to be deducted from the Launch GSN. In this case the FTOS value must be the smaller of 999,999,999ns or CTR-1 ns.

If (Launch Time Offset >= FTOS):

Fetch Time Offset = ((Launch Time Offset - FTOS) * 256ns)

CBFS(CTR Borrow for Fetch Subtraction) = 0

If (Launch Time Offset < FTOS)

Fetch Time Offset = CTR + ((Launch Time Offset - FTOS) * 256ns)

CBFS (CTR Borrow for Fetch Subtraction) = 1

The Fetch GSN is computed as follows:

Fetch GSN = Launch GSN - FGOS - CBFS

Fetch Time = Fetch GSN BTR[63:0] + Fetch Time Offset

The frame is marked eligible for DMA fetch when the fetch time is smaller than the System Time.

- Maximum value of FGOS is 7. This implies that when FTOV is set, the frame can be fetched at a maximum of < 8 GCL loop iterations before it becomes eligible for Launch.
- An interrupt is generated when a frame is dropped due to Launch time expiry. For more details, see the *Register Descriptions* chapter.

DMA Operations Sequence

Time-Based Scheduling requires the use of Enhanced Descriptors. For more details, see "Enhanced Descriptor for Time-Based Scheduling" on page 1829.

Following is the sequence of operation when FTOV = 1.

1. Fetch the first Transmit Enhanced Normal Descriptor. (Check the FD in the Normal Descriptor).
2. If LTV is set and Fetch is enabled, compute the Fetch Time based on the Launch Time. Wait for the System Time to be greater than Fetch Time.
3. Read the Frame (Data) from the host memory and transfer to MTL FIFO
4. Close the Enhanced Normal Descriptor
5. Fetch the next Normal Descriptor (if the previous Descriptor was not the Last)
6. Repeat steps 4 to 6, until the last descriptor of the frame (LD is set)

After the Last Descriptor of a frame, program the next Transmit Enhanced Normal Descriptor with a new Launch Time and with LTV bit set. Otherwise, the subsequent frames are processed without any time restrictions.

Control Word (MTL Configurations)

Time-based scheduling ensures that a packet is allowed to arbitrate for transmission only if the System Time is greater than the Launch Time.

In MTL configuration,

- Launch Time and GSN, use the same field that is used to hold the 64-bit timestamp value for OSTC (One Step Timestamping Correction). If the LTV (Bit 31, TBSDW0) bit is set and OSTC and OSTPA bits are not set (bit 19 and 15 of first control word), the Time Stamp field is interpreted as Launch Time. For details of Launch Time Definition see TBSDW0 in Table 2-6 on page 96 and TBSDW1 in Table 2-7 on page 97.
- The last two Control D-words is considered to the Time Stamp for OSTC, when bit-19 of Control-Word0 (OSTC) is set.

When the MTL TxFIFO Read Controller reads the first control word, both bit 19 and bit 15 are reset, it fetches the next control word. When the next control word's bit 31 (TBSDW0) is set, the frame has a valid Launch Time (as per the description of TBSDW0 and TBSDW1). If bit-31 is set, it reads the subsequent control words to get the Launch Time. For more details, see Table 2-6 on page 96 and TBSDW1 in Table 2-7 on page 97.

Impact on Transmission Selection Algorithm

Time-Based Scheduler does not directly influence the Transmission Selection Algorithm but has an indirect influence as it determines if a queue is eligible to participate in the scheduling.

If a frame of a specific queue has a valid Launch Time, the Time Based Scheduler ensures that the frame participates in Transmit Scheduling (TRC Scheduler) only after the Launch time has elapsed.

The gating done by Time-Base scheduler is only for the Scheduling (picking frame for Transmission). It might not alter any other characteristics and behavior of the frame. For example, the Frame waiting for the Launch time to lapse continues to gain credits in CBS. The CBS credits are not lost, as the frame is marked as available but not ready. Strict Priority or CBS are recommended Transmit Scheduling algorithms for the TBS implementation because, scheduling for TBS enabled queues are more predictable.

8.3.3 Enabling the Time-Based Scheduling Feature

To enable this feature while configuring the controller in the coreConsultant GUI:

1. Select the Time Based Scheduler feature during the Specify Configuration activity in coreConsultant.
2. Select Enable Time Based Scheduling option.

For details about this option, see Time Based Scheduling Parameters in the *Parameter Descriptions* chapter.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

8.3.4 Programming Guidelines for Time-Based Scheduling

For details about programming guidelines, see “Setting up the Time-Based Scheduling Function” on page 1886.

8.4 Stream-Gate Filtering

The flexible receive parser supports a subset of IEEE 802.1 Qci specification, stream-gate filtering. The stream-gate filtering support is based on stream-gate instance and stream filter instance. Policing or metering functions of the flow meter instance must be implemented by the application, as show in Figure 8-13 on page 406.

- **Stream Filter**

  Identifies the Stream ID and maps to one of the 8 Gate IDs; Only 8 Gate IDs are supported.

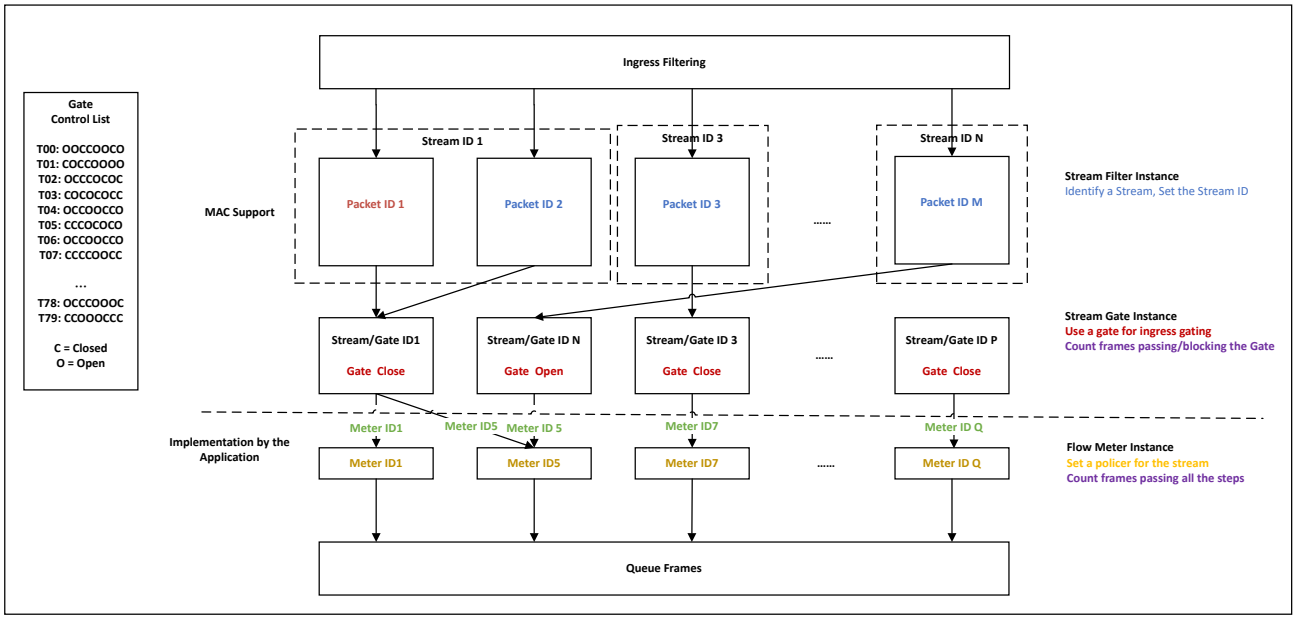
- **Stream Gate Instance**

  All Gate IDs share the same GCL.

Based on the configuration, the flexible receive parser can parse the first 64, 128, or 256 bytes of a frame. When you enable stream-gate filtering in the configuration, in addition to filtering and DMA channel routing, the parser performs stream-gate filtering. The result of the stream-gate filtering is based on the Gate ID of the received frames and the GCL programming.

For details of the Gate ID and Gate ID valid fields, see "Flexible Receive Parser Instruction Table (IT) Format" on page 190.

Figure 8-13 Block Diagram of Stream-Gate Filtering



When the Flexible receive parser result is available, the parser uses the Gate ID (GID field) of the Instruction Table and the Gate Open/Close status of the stream-gate filtering, to determine if the result is pass or fail. When the stream-gate filtering fails, the failed traffic is routed in one of the following ways:

- DMA channel, channel 0 for express traffic
- Channel number configured in the FPE_RCH field of MTL_RXP_Control_Status register, for preemption traffic.

The details of the fail/drop is indicated in the FRPSM field of the RDES2 Normal Descriptor (Write-Back Format). For details of the Flexible Receive Parser Receive Status, see Table 13-20 on page 1825 .

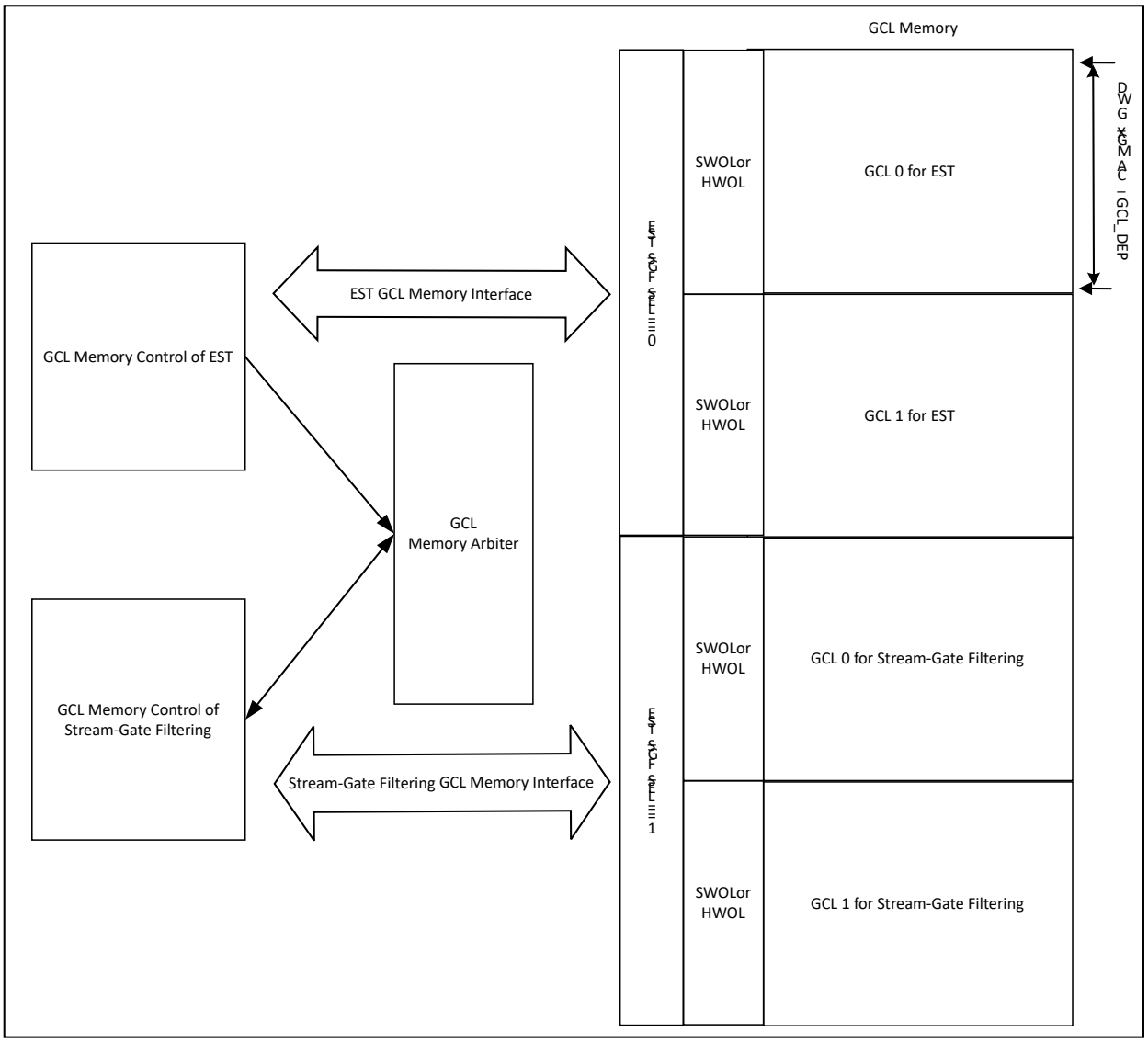
When the stream-gate filtering passes, the frames are forwarded by the flexible receive parser.

If you enable the receive management counter in your configuration, the stream-gate filtering tracks the pass/fail packets in the `SGF_Pass_PktCnt` and `SGF_Fail_PktCnt` packets counters respectively. For more details, see the *"Register Description"* chapter.

8.4.1 Stream-Gate Filtering with EST

To support stream-gate filtering with EST, flexible receive parser implements one external GCL memory that can be used by both EST and stream-gate filtering. A memory arbiter is used to avoid EST and stream-gate filtering accessing the same GCL memory simultaneously, as shown in Figure 8-14.

Figure 8-14 Memory Implementation for Stream-Gate Filtering with EST



8.4.2 Stream-Gate Filtering with Frame Preemption

When you enable stream-gate filtering for preemption frames, the filtering is performed on the preemption frames. For fragmented frames, stream-gate filtering is performed only on the first fragment. Continuation fragments are not considered for stream-gate filtering.

8.4.3 Enabling Stream-Gate Filtering

You can enable stream-gate filtering only after enabling Flexible Receive Parser. For details of enabling Flexible Receive Parser, see “Enabling the Flexible Receive Parser” on page 202.

To enable Stream-Gate filtering while configuring the controller in the coreConsultant GUI:

- Select Enable Stream-Gate Filtering (SGF), in the Time Sensitive Networking features.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

8.5 Policing Counters

8.5.1 Overview of Policing Counters

Policing counters feature counts the filter events and gathers the diagnostic information of the received packets. This feature can monitor events associated with filters such as DA/SA, VLAN, L3_L4, flexible receive parser, and stream-gate filtering. This feature maintains a set of registers to gather the filter events information that include, register for controlling the behavior of the 32-bit policing counters and policing counter status registers for providing status of the filter events. When a counter value crosses the programmed threshold, all the subsequent packets which asserts the filter events associated to the counter are dropped as a corrective measure. Interrupts are generated based on the programmed threshold and the selected time window.

8.5.2 Description of Policing Counters

You can use 16 or 32 policing counters, each of 32 bits, to track the filter event.

The policing counters can track the following filter events, only for perfect filtering:

- DA/SA perfect filter pass
- DA/SA perfect filter fail
- VLAN perfect filter pass
- VLAN perfect filter fail
- L3_L4 perfect filter pass
- L3_L4 perfect filter fail
- Flexible receive parser, accept frame
- Flexible receive parser, reject frame
- stream-gate filter pass

Note Do not associate policing counter with inverse filtering of DA, VLAN, and L3_L4, as this can cause unpredictable behavior.

You can program the filter events to be associated with any of these policing counters. A policing counter can be associated with more than one filter event. Each policing counter has a programmable time window and threshold.

A time window is provided as a sideband input, using the `sbd_pc_tim_wdw_i` input signal. The rising edge of the signal represents the start of the time window, and the falling edge represents the end of the time window. You can program the counters to select any one of the 4 sideband time windows. You can program the policing counter to increment

- Irrespective of the selected time window (free running mode)
- Within the selected time window (time-window mode)

The programmable threshold is controlled by the `MAC_PCCtrl_IndReg` register. When the count reaches the programmed threshold, an interrupt is generated. You can program the register to restart the counters at the end of each time window.

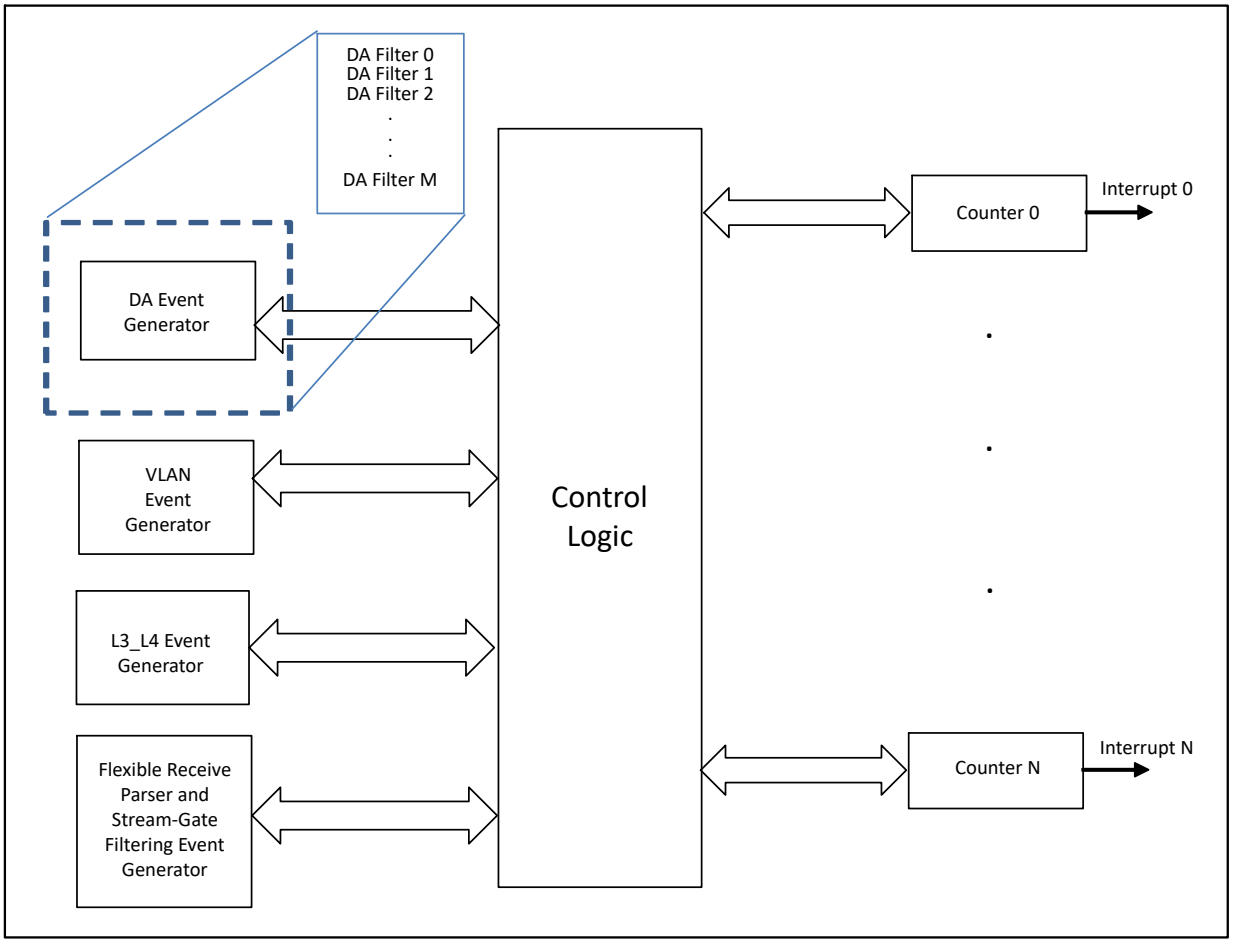
For more details, see “Programming Guidelines for the Policing Counters” on page 1874.

8.5.2.1 Event Generators

The policing counter captures the filter events in the status registers, to indicate that an event is generated. Each filter event has an enable and a counter selection programming as shown in Table 8-15. When multiple filter events are associated with the same counter, the events are merged and updated in the corresponding counter.

Setting/resetting of the IPFE, VTFE, and SAF fields of the MAC_Packet_Filter register has no impact on the policing counter.

Figure 8-15 Policing Counter Flow Diagram



8.5.2.2 Policing Counter Interrupts

Policing counters support 2 types of interrupts:

- Time window based interrupt

  When you enable time window based interrupt, the interrupt is triggered at the end of a time window. Each policing counter has a time window based interrupt status bit, which is either write-1 to clear or read-on-clear based on the value programmed in the RCWE field of the MAC_CSR_SW_Ctrl register.

- Threshold based interrupt

  When the counter reaches the threshold that you programmed, an interrupt is asserted. Each policing counter has a threshold-based interrupt status bit, which is either write-1 to clear or read-on-clear based on the value programmed in the RCWE field of the `MAC_CSR_SW_Ctrl` register.

The combined status of the policing counter interrupts is indicated in the PCIS field of the `MAC_Interrupt_Status` register.

8.5.2.3 Policing Counter Selection and Filter Status

Table 8-15 shows the registers to be programmed to enable policing counters for various filters and where the filter event status is indicated.

The filter event status is set when at the least one corresponding filter event is triggered. Based on the value of the RCWE field of the MAC_CSR_SW_Ctrl register, the status bits of the various policing counter status registers can be cleared either by writing 1 or by clear-on-read.

Table 8-15 Policing Counter Selection and Filter Status Indication

Filter Type	Register for Enabling/Selection of Policing Counter	Indicating the Filter Event Status in the Policing Counter Status Register
SA/DA pass	MAC_DPCSel_IndReg	MAC_PCStatus_DA_IndReg
VLAN pass	MAC_VPCSel_IndReg	MAC_PCStatus_VLAN_IndReg
L3_L4 pass	MAC_LPCSel_IndReg	MAC_PCStatus_L3L4_IndReg
Flexible Receive Parser and Stream-gate filtering	See, bit position 91:80 in the Flexible Receive Parser Instruction Table, Table 4-15 on page 191.	See, Flexible Receive Parser Receive Status, Table 13-20 on page 1825
Filter fail	MAC_FPCSel_IndReg0	FFAIL field of the MAC_PCStatus_Filter_Event_IndReg

8.5.2.4 Policing the Bandwidth

The policing counters mapped to various filter pass events also provide bandwidth estimates of number of packets and the associated traffic type that are forwarded to the application. When a filter pass event's policing counter value exceeds the programmed threshold value, and the FFAT field of the MAC_PCCtrl_IndReg is set, the packets are treated as filter fail, and processed accordingly, until the associated counter is cleared. Therefore, the bandwidth policing is in terms of number of packets and not in terms of packet sizes.

When a packet is considered filter fail, based on the FFAT field of the MAC_PCCtrl_IndReg,

- A filter fail event is not asserted. This is because, there is no actual filter failure, but only a threshold exceed.
- Policing counters associated with the filter pass events are incremented and the status for the corresponding filter is also updated.

- If the filter corresponding to a filter pass event is not enabled to drop the packets using IPFE/VTFE and SAF fields of the MAC_Packet_Filter register, the packets are not dropped. Instead, the packets are forwarded to the application with a fail status.
- Multicast pass-all packets are forwarded to the application.

8.5.2.5 Registers Related to Policing Counters

- MAC_PCntr_IndReg
- MAC_PCStatus_Filter_Event_IndReg
- MAC_PCStatus_L3L4_IndReg
- MAC_VPCSel_IndReg
- MAC_DPCSel_IndReg
- MAC_LPCSel_IndReg
- MAC_PCCtrl_IndReg
- MAC_FPCSel_IndReg0
- MAC_PCTH_Intr_Enable
- MAC_PCTH_Intr_Status

8.5.3 Enabling Policing Counters

To enable policing counters while configuring the controller in the coreConsultant GUI:

Select Enable Policing Counters (PC) under the Time Sensitive Networking section during the Specify Configuration activity in coreConsultant.

For details about this option, see Time Sensitive Networking Parameters in the *Parameter Descriptions* chapter.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

8.5.4 Programming Guidelines for Policing Counters

“Programming Guidelines for the Policing Counters” on page 1874.

8.6 Asynchronous Traffic Shaper (ATS)

When the DWCXG_ATS parameter is selected, DWC_25gmac supports Asynchronous Traffic Shaper (ATS) end-station scheduler state machines and transmission selection algorithm at the egress-port/transmit-side.

ATS can be enabled per traffic class by programming 2'b11 to the TSA field of the `MTL_TC(#i)_ETS_Control` register. Every queue mapped to ATS enabled TC becomes an ATS enabled queue.

Each ATS enabled transmit queue has a separate ATS scheduler state machine instance, which assigns eligibility time to the frames of all emitted streams at the egress port, for which the ATS transmission selection algorithm is executed.

Both ATS scheduler state machine and transmission selection algorithm use the same IEEE 1588 reference system clock/time.

Note

The TBS and ATS features can be simultaneously selected in the configuration. However, enabling TBS and ATS features at same time for the same Traffic Class (TC) is not supported.

The TBS has higher precedence, that is, if TBS is enabled for a certain DMA with the Enhanced Descriptor indicating a valid launch time, the design does not capture frame arrival time internally even if the EPAT field is programmed to 0. With such programming, the DMA is considered as a non-ATS DMA.

8.6.1 Fetching Frame Arrival Time

When the EPAT field of `MTL_TxQ(#i)_Operation_Mode[31]` is selected, ATS uses application given frame arrival time via enhanced-transmit-normal-descriptor; otherwise, it uses internally captured IEEE 1588 reference system time (PTP reference time) when frame is available on DMA (ATI interface).

For DMA configuration and above, when the EPAT field of `MTL_TxQ(#i)_Operation_Mode[31]` is selected, ATS feature needs Enhanced Descriptors to be enabled for all the DMA channels that intend to use the ATS feature (by setting the EDSE bit of `DMA_CH(#i)_TX_Control` register).

For a given DMA channel, which is enabled for enhanced descriptor, and when there is no context descriptor, and if the first descriptor has LTV field set to 0 & FATV field is set to 1, then in ETDES4 & ETDES5 word of that descriptor, application can program frame arrival time for ATS. For more details, see the corresponding descriptor section and Figure 13-11 on page 1829.

Such application programmed frame arrival time is forwarded to MTL via transmit control words. If a context descriptor is received with valid OSTC values immediately before receiving a first normal descriptor with LTV/FATV bit set, then the LTV/FATV are ignored.

The maximum value that can be programmed to FAT field is:

- 6 bits seconds part of ptp_time
- 30 bits of nanoseconds ptp_time [29:0] in case of digital rollover mode
- ptp_time [30:1] in case of binary rollover mode

In both modes, bits [29:0] of FAT are used for nanoseconds and bits [35:30] are used for seconds. The nanoseconds part in binary rollover mode has a granularity of 0.465ns.

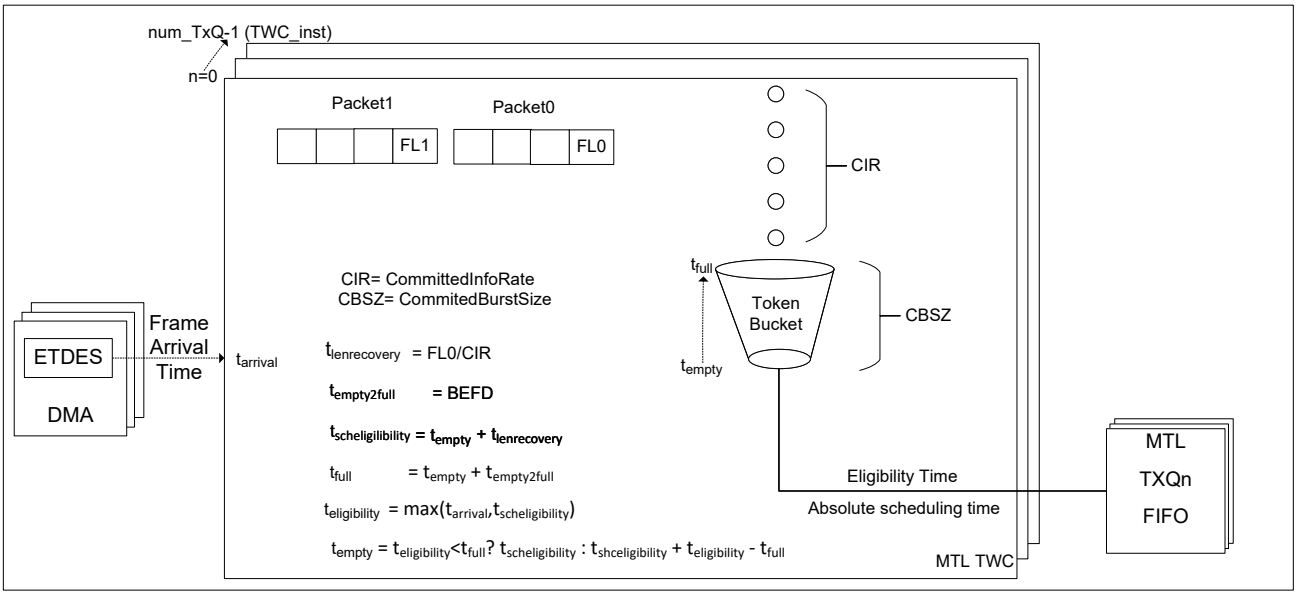
Therefore, not using bit [0] of nanoseconds part provides a granularity of ~1ns. This is the same granularity as in digital rollover mode.

Note The timestamping feature selection is required for ATS.

8.6.2 Eligibility Time (ET Time) Computation

For ATS enabled transmit queue, every packet invokes ET Time computation logic.

Figure 8-16



ET Time Computation logic has FAT Time and Frame length available along with the frame availability.

CIR – Committed Information Rate should be programmed in `MTL_TxQ(#i)_Committed_info_rate` with the required value, with granularity of 1% of the selected speed mode. CIRs of all the ATS enabled queues should not exceed the data rate. Signal `mac_speed_o` is taken as reference to indicate the operation speed.

Encoding of `mac_speed_o`:

- 3'b000 -10G
- 3'b001 -Reserved
- 3'b010 -2.5G
- 3'b011 -1G
- 3'b100 -100M
- 3'b101 -5G
- 3'b110 -2.5G
- 3'b111 -10M

To calculate `len_recovery_duration` with one 1 % of granularity, the nearest power of 2 is multiplied based on the selected speed mode and conversion to ns. Refer below table for details,

Speed select (mac_speed_o)	Speed Value	Multiplier value (decimal)	Powers of 2
3'b000	10G	10	3
3'b001	25G	4	2
3'b010	2.5G	40	5
3'b011	1G	100	7
3'b100	100M	1k	10
3'b101	5G	20	4
3'b110	2.5G	40	5
3'b111	10M	10k	13

For example, if the speed selected is 100Mbps, 1 programmed to register `MTL_TxQ(#i)_Committed_info_rate` means 1Mbps (1% of programmed CIR value).

The Bucket Empty to Full Duration (BEFD) field should be programmed in `MTL_TxQ(#i)_Bucket_Empty_2_Full` with the required value.

For the first packet, Bucket Empty time takes the current system in seconds and nanoseconds based on digital and binary rollover mode for the selected ATS TC. After every frame's eligibility time computation, a new time in future gets assigned as Bucket Empty time.

The maximum value that can be programmed to BEFD field of `MTL_TxQ(#i)_Bucket_Empty_2_Full` is:

- 32'h3B9A_C9FF for digital rollover mode
- 32'h7FFF_FFFF for binary rollover mode

BEFD should also be programed based on digital or binary rollover mode. Programing of BEFD in binary rollover mode has to be programed with 1ns resolution.

For eligibility time per packet greater than current system time:

The eligibility time of 36 bits gives bigger resolution to nano-sec part.

In digital rollover mode, [37:32], [29:0] bits of current system time if compared with eligibility time and [37:32], [30:1] in case of Binary rollover mode.

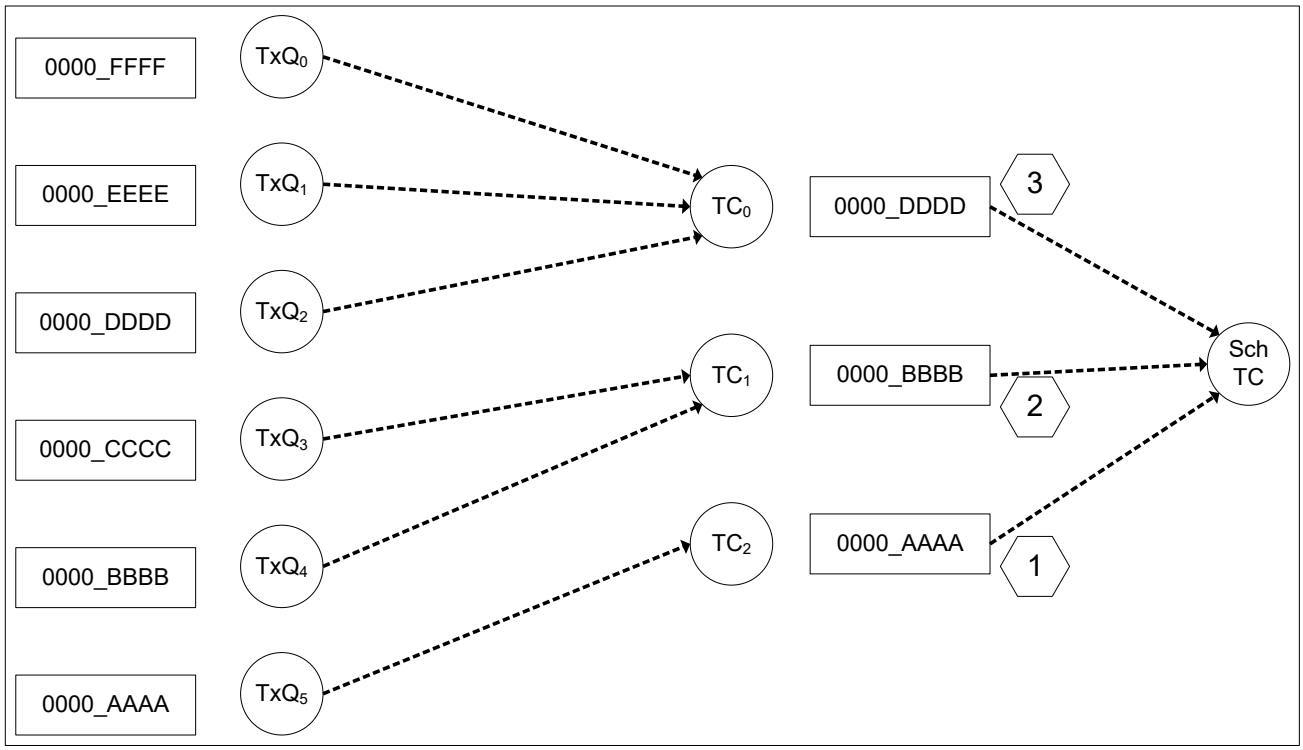
Based on every frame's FAT Time, Frame Length and current Bucket Empty time, a new eligibility time is calculated and assigned. This ET time is used for comparisons between ATS enabled TCs to decide which TC gets to schedule the frame first in terms of ascending ET time.

8.6.3 Transmission Selection Algorithm

At the read side, ATS transmission selection algorithm determines the frame eligible for transmission based on the ascending order of eligibility time assigned to each frame. A frame is eligible for transmission if the assigned eligibility is less than or equal to the current time (system reference time).

The maximum value of eligibility time is 64 s with a sliding window of plus-minus (+/-) 32 s.

Figure 8-17 ATS Packets Scheduling Example



For an ATS enabled TC, all the transmit queues that are mapped to that TC will fetch eligibility time after the packet information is available in MTL Tx FIFO.

In the example shown in Figure 8-17, all TCs are ATS enabled. The packets are mapped to the queues in the following manner:

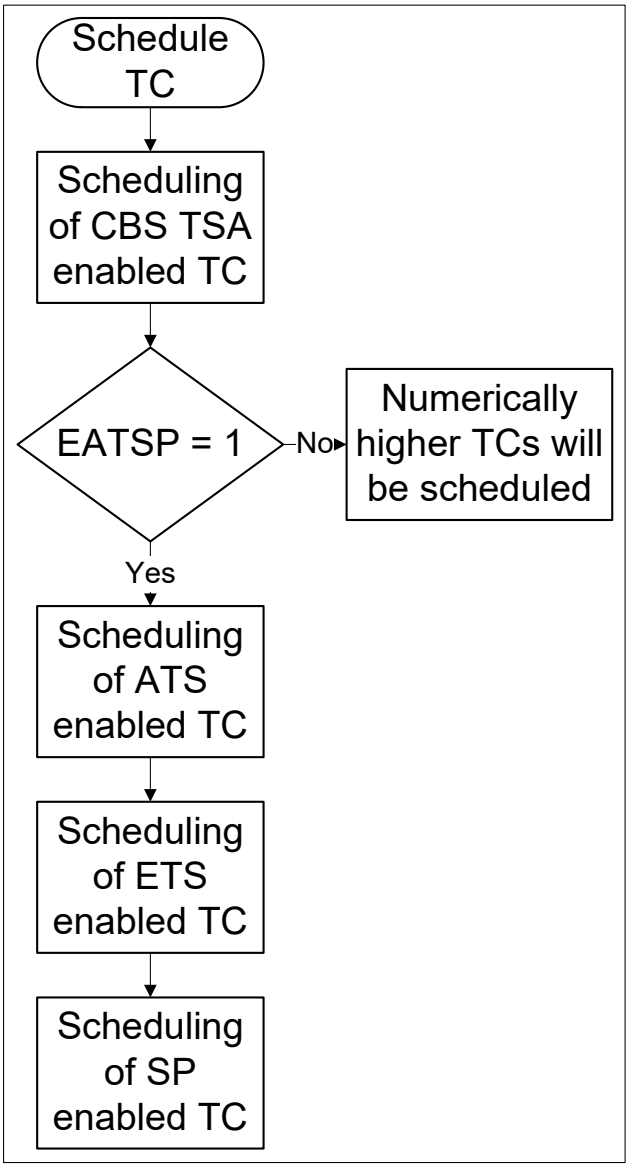
- TxQ_0, TxQ_1, TxQ_2 are mapped to TC_0
- TxQ_3 and TxQ_4 mapped to TC_1
- TxQ_5 mapped to TC_2

For every TC, queue ready for ATS based transmission is chosen from the least eligibility times that are available for that particular TC mapped TxQs, provided the ET times are also lesser than or equal to the current time.

For the example shown in Figure 8-17 on page 416, TxQ_2, TxQ_4, and TxQ_5 are eligible for scheduling from TC_0, TC_1, and TC_2 respectively. And between all three TCs, the lowest ET time TC, which is TC_2 will get scheduled first, followed by TC_1 and TC_0.

Priority is determined based on the programming of TSA field of TCs. The highest TC will have the highest priority. The priority for ATS can be enhanced to be placed ahead of ETS. Also, the SP and EATSP fields of `MTL_Operation_Mode[3]` should be set to 1. Priority for other scheduling algorithms do not change and are determined based on the programming of the TSA field of TCs.

Figure 8-18 ATS Scheduling based on priority bit



Note

1. Preemptable TCs can also select ATS algorithm for transmission. The selection can be done in the TSA field of `MTL_TC(#i)_ETS_Control[1:0]`.

   In case of preemptable TCs and express TCs; express TC will be scheduled.

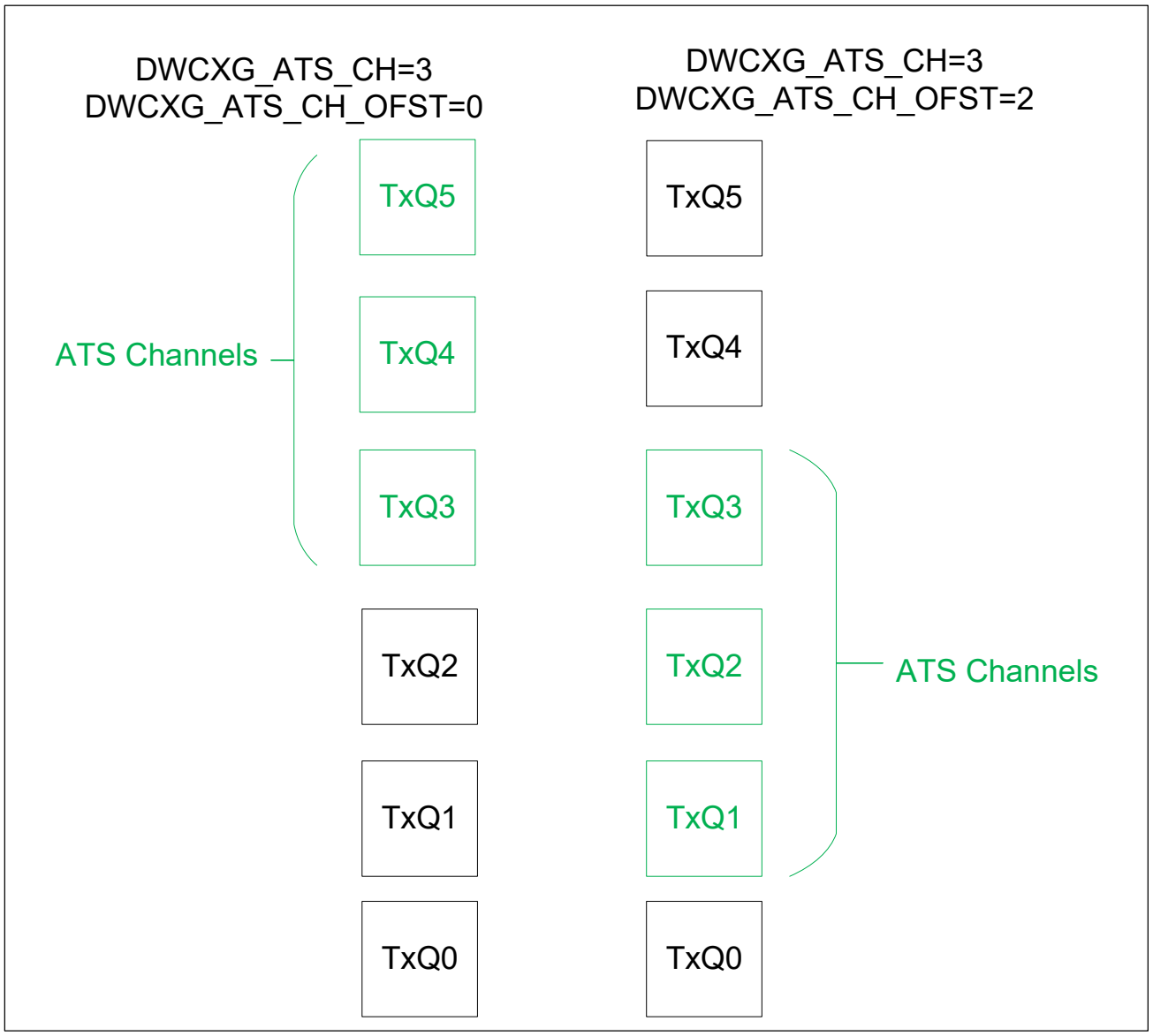
2. When TxQs mapped to the same TC have a valid and equal Eligibility Time, the highest queue among that mapped TC is selected as the candidate scheduled if that TC wins the arbitration.

   Similarly, if more than one TCs have valid and equal Eligibility Time, then the highest TC participate in the arbitration for packet scheduling among these TCs.

Note

The ATS scheduler instance is available only for those TxQs which are selected by `DWCXG_ATS_CH` and `DWCXG_ATS_CH_OFST` parameters.

Figure 8-19 Channel and Channel Offset correlation



8.6.4 DSTS Support with ATS feature

A CSR control bit (PTP_SEL) of the `MTL_Operation_Mode` register is added to select between local or external ptp timestamp for the computation of ATS related logic.

9

Automotive Safety Features

This chapter describes the automotive safety features supported by DWC_25gmac.

These features are available when you specify the `DWCXG_ASP_ALL` parameter in the configuration and you have the corresponding license.

This chapter contains the following sections:

- "AXI Interface Safety Protection" on page 420
- "Error Correction Code (ECC) Protection for Memories" on page 429
- "On-Chip Data Path Parity Protection" on page 438
- "FSM One-Hot and Timeout Protection" on page 451
- "Application/CSR Interface Timeout Protection" on page 454
- "Control Register Protection" on page 456

9.1 AXI Interface Safety Protection

The AXI interface safety protection supports error correction code (ECC) and parity protection for various channels in the AXI manager and AXI subordinate interfaces.

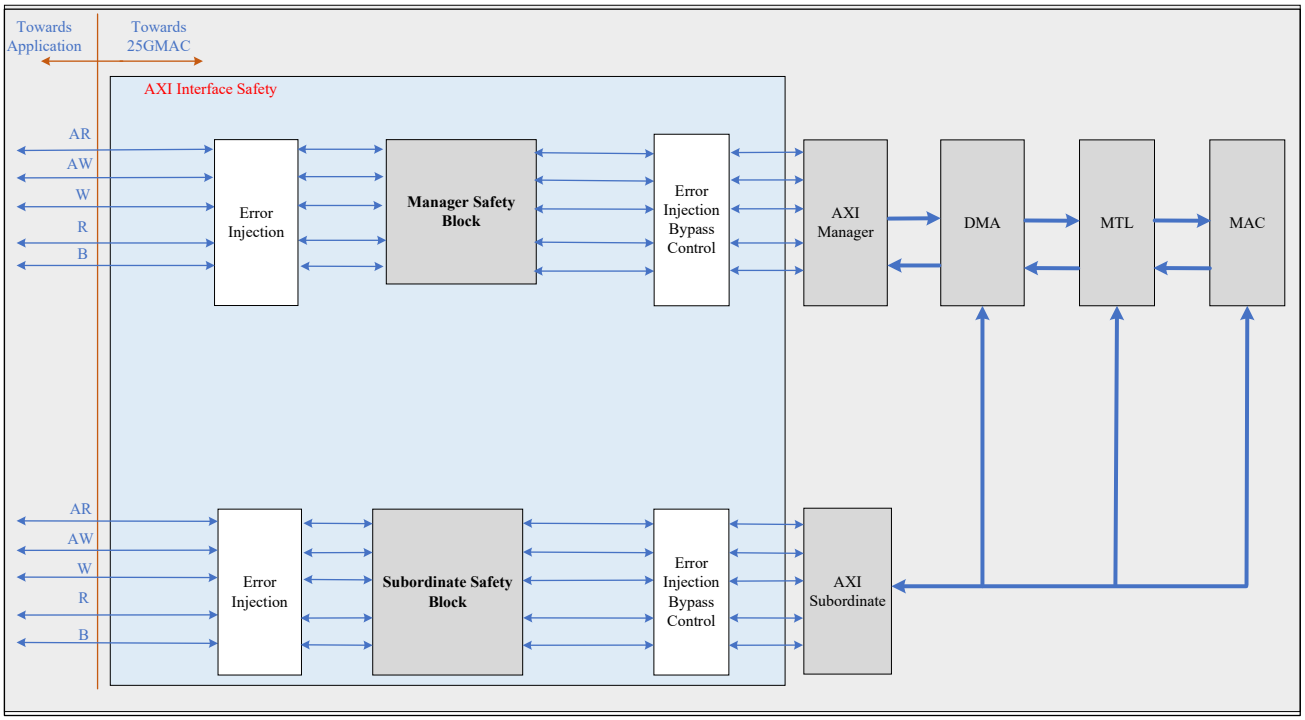
9.1.1 Overview

When you set the DWCXG_ASP_ALL parameter in the configuration and the corresponding automotive license is available, the AXI interface safety protection feature is enabled.

To disable this feature, set the AMSD field of the DMA_AM_Sfty_Ctrl register and ASSD field of the DMA_AS_Sfty_Ctrl register for the AXI Manager and Subordinate interfaces respectively. For the subordinate interface, you can select either AXI subordinate or AXI4-Lite subordinate.

Figure 9-1 shows the top-level architecture of AXI interface safety protection feature.

Figure 9-1 Top-Level Architecture of AXI Interface Safety Protection



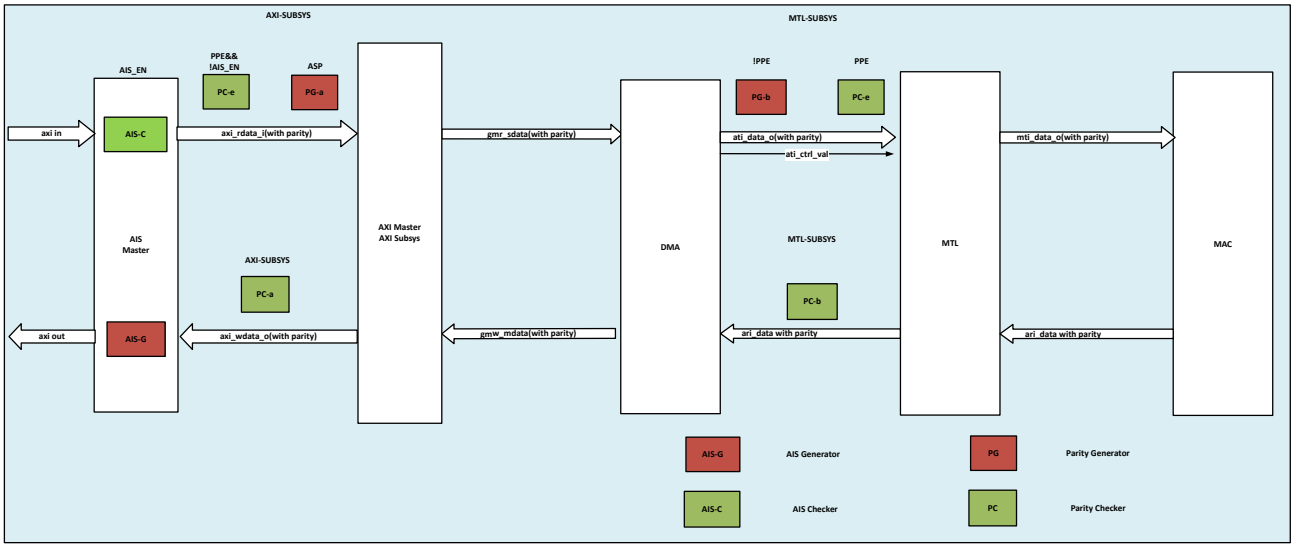
The ECC logic uses Hamming code based SECDED (Single Error Correction and Double Error Detection) engine of the DesignWare Building Block (DWbb BCM46). The Parity logic supports ARC compatibility mode.

9.1.2 Description of AXI Interface Safety Protection

9.1.2.1 AXI Manager Interface Safety Protection

The AXI Manager Interface Safety block protects the AXI Manager interface with ECC and FSM one-hot protection features. Figure 9-2 shows the block diagram of AXI Manager interface safety for Tx/Rx path. The AXI Manager Interface Safety Block uses the AIS-G (AXI Interface Safety) Generators and AIS-C (AXI Interface Safety) Checkers.

Figure 9-2 Block Diagram of AXI Manager Interface Safety for Transmit/Receive Path



The following generators and checkers are present for ECC and Parity computation:

- Read Address Channel Parity Generator
- Write Address Channel Parity Generator
- Read Data Channel ECC and Parity Checker
- Write Data Channel ECC and Parity Generator
- Write Response Channel Parity Checker

9.1.2.1.1 AXI Manager Interface Read/Write Address Channel Safety Protection

The AXI Manager Interface Read/Write Address Channel signals are grouped into the following two payloads for the computation of Parity:

- Read/Write Address Channel Address (ARADDR/AWADDR)
- Read/Write Address Channel Control Signals (ARCTRL/AWCTRL)

The Read Address Channel Payload 0 and Payload 1 used for computation of Parity is packed as follows:

- Read Address Channel Payload 0 = ARADDR
- Read Address Channel Payload 1 = {ARQOS, ARPROT, ARCACHE, ARLOCK, ARBURST, ARSIZE, ARLEN, ARID}

Similarly, the Write Address Channel Payload 0 and Payload 1 used for computation of Parity is packed as follows:

- Write Address Channel Payload 0 = AWADDR
- Write Address Channel Payload 1 = {AWQOS, AWPROT, AWCACHE, AWLOCK, AWBURST, AWSIZE, AWLEN, AWID}

The computed Parity signals are packed and driven on the ARUSER/AWUSER as follows:

- ARUSER = {Read Address Channel Payload 1 Parity, Read Address Channel Payload 0 Parity}
- AWUSER = {Write Address Channel Payload 1 Parity, Write Address Channel Payload 0 Parity}

The parity signal for ARVALID/AWVALID/ARREADY/AWREADY is driven on the non-standard sideband signal.

9.1.2.1.2 AXI Manager Interface Write Data channel Safety Protection

The AXI Manager Interface Write Data channel signals are grouped into the following two payloads for the computation of ECC and Parity:

- Write Data channel Data (WDATA)
- Write Data channel Control Signals (WCTRL)

The Write Data channel Payload 0 and Payload 1 used for computation of ECC and Parity is packed as follows:

- Write Data channel Payload 0 = WDATA
- Write Data channel Payload 1 = {WLAST, WSTRB, WID}

The computed ECC and parity signals are packed and driven on the WUSER as follows:

- WUSER = {Write Data channel Payload 1 Parity, Write Data channel Payload 0 ECC}

The parity signal for WVALID/WREADY is driven on the non-standard sideband signal.

9.1.2.1.3 AXI Manager Interface Read Data Channel Safety Protection

The AXI Manager Interface Read Data Channel signals are grouped into the following two payloads for the validation of ECC and Parity:

- Read Data Channel Data (RDATA)
- Read Data Channel Control Signals (RCTRL)

The Read Data Channel Payload 0 and Payload 1 used for validation of ECC and Parity is packed as follows:

- Read Data Channel Payload 0 = RDATA
- Read Data Channel Payload 1 = {RLAST, RRESP, RID}

The computed ECC and Parity signals are packed and driven on the RUSER as follows:

- RUSER = {Read Data channel Payload 1 Parity, Read Data channel Payload 0 ECC}

The parity signal for RVALID/RREADY is driven on the non-standard sideband signal.

9.1.2.1.4 AXI Manager Interface Write Response Channel Safety Protection

The AXI Manager Interface Write Response Channel signals are grouped into the following payload for the validation of Parity:

- Write Response Channel Control Signals (BCTRL)

The Write Response Channel Payload 0 used for validation of Parity is packed as follows:

- Write Response Channel Payload 0 = {BRESP, BID}

The computed Parity signals are packed and driven on the BUSER as follows:

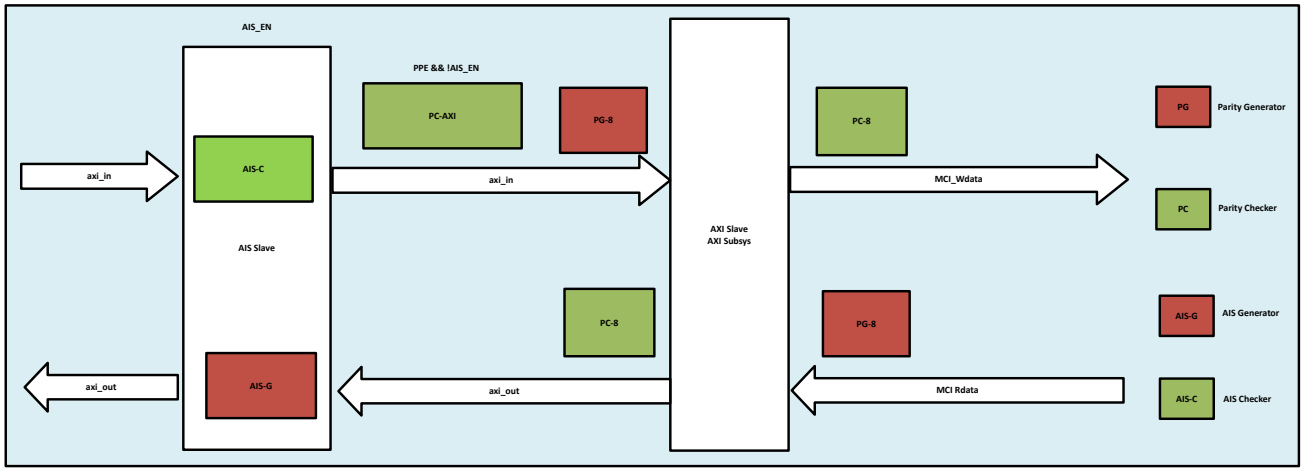
- BUSER = {Write Response Channel Payload 0 Parity}

The parity signal for BVALID/BREADY is driven on the non-standard sideband signal.

9.1.2.2 AXI Subordinate Interface Safety Protection

Figure 9-2 shows the block diagram of AXI subordinate interface safety for Tx/Rx path. The AXI Subordinate Interface Safety block is used to protect the AXI Subordinate interface with ECC and FSM one-hot protection features. Figure 9-2 shows the AIS-G (AXI Interface Safety Generators) and AIS-C (AXI Interface Safety) checkers.

Figure 9-3 Block Diagram of AXI Subordinate Interface Safety for Transmit/Receive Path



The following generators and checkers are present for ECC and Parity computation:

- Read Address Channel Parity Checker
- Write Address Channel Parity Checker
- Write Data Channel ECC and Parity Checker
- Read Data Channel ECC and Parity Generator
- Write Response Channel Parity Generator

9.1.2.2.1 AXI Subordinate Interface Read/Write Address Channel Safety Protection

The AXI Subordinate Interface AR/AW Channel signals are grouped into the following two payloads (similar to the AXI Manager Interface Safety Protection) for the checking of Parity:

- Read/Write Address Channel Address (ARADDR/AWADDR)
- Read/Write Address Channel Control Signals (ARCTRL/AWCTRL)

The Read Address Channel Payload 0 and Payload 1 used for computation of Parity is packed as follows:

- Read Address Channel Payload 0 = ARADDR
- Read Address Channel Payload 1 = {ARLOCK, ARBURST, ARSIZE, ARLEN, ARID}

Similarly, the Write Address Channel Payload 0 and Payload 1 used for computation of Parity is packed as follows:

- Write Address Channel Payload 0 = AWADDR
- Write Address Channel Payload 1 = {AWLOCK, AWBURST, AWSIZE, AWLEN, AWID}

The computed Parity signals are packed and driven on the ARUSER/AWUSER as follows:

- ARUSER = {Read Address Channel Payload 1 Parity, Read Address Channel Payload 0 Parity}
- AWUSER = {Write Address Channel Payload 1 Parity, Write Address Channel Payload 0 Parity}

The parity signal for ARVALID/AWVALID/ARREADY/AWREADY is driven on the non-standard sideband signal.

9.1.2.2.2 AXI Subordinate Interface Write Data channel Safety Protection

The AXI Subordinate Interface Write Data channel signals are grouped into the following two payloads (similar to the AXI Manager Interface Safety Protection) for the checking of ECC and Parity:

- Write Data Channel Data (WDATA)
- Write Data Channel Control Signals (WCTRL)

The Write Data channel Payload 0 and Payload 1 used for computation of ECC and Parity is packed as follows:

- Write Data channel Payload 0 = WDATA
- Write Data channel Payload 1 = {WSTRB, WID}

The computed ECC and Parity signals are packed and driven on the WUSER as follows:

- WUSER = {Write Data channel Payload 1 Parity, Write Data channel Payload 0 ECC}

The parity signal for WVALID/WREADY is driven on the non-standard sideband signal.

9.1.2.2.3 AXI Subordinate Interface Read Data Channel Safety Protection

The AXI Subordinate Interface Read Data channel signals are grouped into the following two payloads for the computation of ECC and Parity:

- Read Data Channel Data (RDATA)
- Read Data Channel Control Signals (RCTRL)

The RUSER, Read Data channel Payload 0 and Read Data channel Payload 1 used for driving/computation of ECC and Parity are packed similar to that of AXI Manager Interface Safety Protection.

9.1.2.2.4 AXI Subordinate Interface Write Response Channel Safety Protection

The AXI Subordinate Interface Write Response Channel signals are grouped into the following payload for the computation of Parity:

- Write Response Channel Control Signals (BCTRL)

The BUSER, Write Response Channel Payload 0 used for driving/computation of Parity are packed similar to that of AXI Manager Interface Safety Protection.

9.1.2.2.5 AXI Subordinate Interface xVALID and xREADY Safety Protection

The AXI Interface handshake signals xVALID and xREADY are protected with the parity signal. The xVALID Parity generator computes the parity for the xVALID signal and drives the same along with the xUSER signal.

- AXI Subordinate Interface: RVALID/BVALID

The xREADY Parity checker validates the parity of the xREADY signal. The xREADY signal parity is received through the respective non-standard sideband signal.

- AXI Subordinate Interface: ARREADY/AWREADY/WREADY

The xREADY Parity checker validates the parity of the xREADY signal. The xREADY signal parity is received through the respective non-standard sideband signal.

- AXI Subordinate Interface: RREADY/BREADY

The xVALID Parity checker validates the parity of the xVALID signal. The xVALID signal parity is received through the respective xUSER signal.

- AXI Subordinate Interface: ARVALID/AWVALID/WVALID

Note

- If the xREADY Parity check fails, the respective payload is discarded.
- If the xVALID Parity check fails, the respective payload is discarded

9.1.2.3 ECC Generator and Checker

The ECC Generator and Checker uses Hamming code based SECDED (Single Error Correction and Double Error Detection) engine of DesignWare Building Block (DWbb BCM46). Table 9-1 indicates the ECC check-bits width for data width selected in the configuration:

Table 9-1 ECC Payload Width Versus ECC Check-Bits Width

Data Width	ECC Check-Bits Width
32 bits	7
64 bits	8
128 bits	9

9.1.2.4 Parity Generator and Checker

The Parity Generator and Checker supports the following modes:

- Normal mode

  In this mode, control signals are packed as payload and 1 bit of parity per payload byte.

- ARC compatibility mode

Based on the value of the `DWCXG_ASP_PPE_ODD` parameter that you specify during configuration, odd or even parity type is selected. For more details, see the *"Parameter Descriptions"* chapter.

9.1.2.4.1 ARC Compatibility Parity Mode

The ARC Compatibility Parity mode is supported to align the Parity Generation implementation with ARC processor Parity implementation. The parity mode provides the interoperability between the ARC processor and 25GMAC.

Table 9-2 shows the details of parity computation in the case of ARC Compatibility Parity Mode:

Table 9-2 Parity Generator and Checker - ARC Compatibility Parity Mode

Sr. No.	AXI Channel Payload	ARC Compatibility Parity Mode
1	ARADDR/AWADDR	The Parity Generator computes and Checker validates the ARADDR/AWADDR parity per 8-bits.Note: If the payload is not multiple of 8, then 0s are appended to make multiple of 8-bits.
2	AR/AW Channel Control (ARCTRL/AWCTRL)	The Parity Generator computes and Checker validates the Parity for AR/AW Channel Control signals as follows: [0] - Parity bit for ARID/AWID [1] - Parity bit for ARLEN/AWLEN [2] - Parity Bit for {ARLOCK, ARBURST, ARSIZE}/{AWLOCK, AWBURST, AWSIZE} Parity Bit 3 and 4 valid exists only for AXI Manager. [3] - Parity Bit for {ARPROT, ARCACHE}/{AWPROT, AWCACHE} [4] - Parity Bit for ARQOS/AWQOS
3	Write Data channel Control (WCTRL)	The Parity Generator computes the Parity for W Channel Control signals as follows: For AXI subordinate, parity is generated as follows: [0] - Parity Bit for WID [1] - Parity Bit for WSTRB For AXI Manager, parity is generated as follows: [0] - Parity Bit for WSTRB [1] - Parity Bit for WLAST
4	Read Data channel Control (RCTRL)	The Parity Generator computes and Checker validates the Parity for Read Data channel Control signals as follows: [0] - Parity Bit for RID [1] - Parity Bit for RRESP and RLAST
5	Write Response Channel Control (BCTRL)	The Parity Generator computes and Checker validates the Parity for Write Response Channel Control signals as follows: [0] - Parity Bit for BID [1] - Parity Bit for BRESP

9.1.2.5 Register Slice

The Register Slice of Depth 1 is used to break the timing path between the ECC or the parity computation logic and the external AXI interconnect/AXI Manager/AXI Subordinate Interface logic, to meet the higher frequency requirements.

9.1.2.6 Safety Interrupts

The AXI Manager and Subordinate Interface Safety correctable (ECC errors only) and uncorrectable interrupts are generated on the existing safety interrupt pins namely `sbd_sfty_ce_intr_o` and `sbd_sfty_ue_intr_o` respectively.

9.1.2.7 ECC Error Counter

The ECC Checker supports following ECC Error Counters:

- Correctable Error Counter: counts the occurrence of 1-bit errors
- Uncorrectable Error Counter: counts the occurrence of 2-bit errors

9.1.2.8 ECC Granularity

The ECC generation is a combinational logic. As the payload size increases for ECC computation, it becomes difficult to meet the higher timing requirements. The ECC granularity feature helps to improve the timing characteristics of the ECC generator logic. The ECC generator supports the following combination of ECC granularities for various payloads which are multiple 32-bits. In Table 9-3, * indicates that, if the payload width is less than the granularity, 64-bits in this case, ECC is generated with full-width granularity.

Table 9-3 ECC Payload and ECC Check-bits With Various ECC Granularity (Multiple 32-bits)

Payload Width	Full-Width Granularity	32-Bit Width Granularity	64-Bit Width Granularity
32	7 (1 granule)	7 (1 granule)	7 * (1 granule)
64	8 (1 granule)	2 * 7 (2 granule)	8 * (1 granule)
128	9 (1 granule)	4 * 7 (4 granule)	2 * 8 (2 granule)

For example, if the payload width is 64-bits and ECC granularity is 32-bits. 7-bit ECC check-bits is computed for 2 chunks of 32-bit payload. In the ECC monitor,

- If the uncorrectable error is found in one chunk of ECC check-bits, uncorrectable error interrupt is set (irrespective of correctable or uncorrectable error in the other chunk).
- Correctable error interrupt is not set, even if the other chunk has the correctable error. And so, the respective correctable error counter is not incremented.
- Only the uncorrectable error counter is incremented.

This is applicable for any configuration that has multiple ECC chunks and the ECC granularity is less than the width of the payload.

9.1.2.9 Error Injection in Debug Mode

9.1.2.9.1 ECC Error Injection in Debug Mode

The 25GMAC supports ECC error injection in debug mode for AXI Manager RDATA and AXI Subordinate WDATA signals. The indirect registers `DMA_Sfty_Dbg_EIC`, `DMA_AM_Sfty_*_Dbg_*` and `DMA_AS_Sfty_*_Dbg_*` are used for error injection in debug mode.

9.1.2.9.2 Parity Error Injection in Debug Mode

The 25GMAC supports parity error injection for AXI Manager and AXI Subordinate input control signals. The indirect registers, `DMA_Sfty_Dbg_EIC`, `DMA_AM_Sfty_*_Dbg_*` and `DMA_AS_Sfty_*_Dbg_*` are used for error injection in debug mode.

Note

- In case of an uncorrectable error on AXI manager or AXI subordinate, application must issue a hard reset as a recovery mechanism.

9.2 Error Correction Code (ECC) Protection for Memories

9.2.1 Overview

The Error Correction Code (ECC) block can correct 1-bit error and detect 2-bit error. Enable/disable the error correction feature as follows:

- For MAC RXF blocks, use the RXFED bits of the `MTL_ECC_Control` register.
- For MTL blocks, use the M*ED bits of the `MTL_ECC_Control` register
- For the DMA blocks, TSOED/DESCED bit of `MTL_ECC_control` register

The ECC protection is supported for,

- Memory data and memory address
- Error statistics and interrupts to monitor the error behavior of memory blocks
- Uncorrectable errors and software recovery mechanism
- Debug mode to inject errors

9.2.2 Description of ECC Protection for Memories

The Error Correction Code (ECC) block can correct 1-bit error and detect 2-bit error.

At the write interface, ECC check-bits are generated by computing ECC on the contents of the data bus and respective address is appended with the data that is written to the memory.

At the read interface, ECC check-bits are recomputed on the content of the read data and the respective address is compared with the received check-bits in the memory.

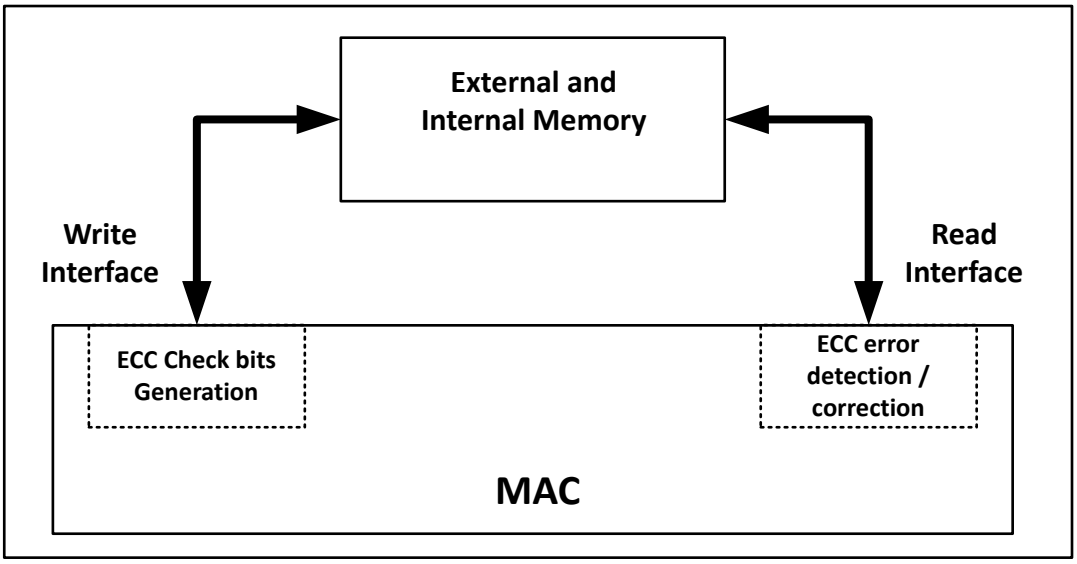
For Rx parser internal instruction table:

- Write interface implies, software writes to the instruction table.
- Read interface implies, software reads the instruction table.
- 25GMAC additionally performs periodic re-computation of the ECC check-bits (from 0th instruction up to NVE) based on the content of the instruction data and respective address, which is compared with the existing check-bits of the instruction.

  To program the interval of the periodic check, use the `CTMR` field of the `MAC_FSM_ACT_timer` register. For more details, see the *Register Descriptions* chapter.

Figure 9-4 shows the block diagram of ECC protection for memories

Figure 9-4 Block Diagram of ECC Protection for Memories



The number of ECC-checkbits generated for each data block depends on the size of the data bus and address bus.

ECC generator is added to all the memory interfaces in 25GMAC. These are:

- 2-port RAM/single-port RAM memories used for the TX Queues and RX Queues in the MTL.
- Single-port RAM memory used for storing TCP/UDP headers in the TSO offload engines.
- Single-port RAM memory used for the Gate Control List in the 802.1Qbv EST function.
- Single-port RAM memory used for the descriptor cache when the configuration uses a different memory with TSO memory.
- Single-port RAM memory used for MAC Receive Filter Lookups
- Internal memory (register array) used for the Instruction Table of rules for the Flexible RX Parser.

Table 9-4 describes the memory bus widths, ECC code size and the generated checkbits size.

Table 9-4 Memory Bus Width, ECC Code Size, and Checkbits Size

Memory Block	Configured Data Width	Additional Control Bits	Total Data Width (without ECC)	Address Width (Depends on Memory Size)	ECC Code Reference Range	Required Check bits	Total Data Width (with ECC)
MTL Tx Memory (2 memories in case of single-port RAM and 1 memory in case of 2-port RAM configuration	64	4	68 bits	4 to 15	Total Data width + Address width <= 120bits	8 bits	76 bits

Table 9-4 Memory Bus Width, ECC Code Size, and Checkbits Size (continued)

Memory Block	Configured Data Width	Additional Control Bits	Total Data Width (without ECC)	Address Width (Depends on Memory Size)	ECC Code Reference Range	Required Check bits	Total Data Width (with ECC)
	128	5	133 bits	4 to 15	Total Data width + Address width <= 247bits	9 bits	142 bits
MTL Rx Memory (2 memories in case of single-port RAM and 1 memory in case of 2-port RAM configuration	64	4	68 bits	3 to 16	Total Data width + Address width <= 120bits	8 bits	76 bits
	128	5	133 bits	3 to 16	Total Data width + Address width <= 247bits	9 bits	142 bits
MTL GCL Memory	DWCXG__NUM_TC (1 to 8) + DWCXG_EST_WID (16,20, 24)		DWCXG_NUM_TC (1 to 8) + DWCXG_EST_WID (16, 20,24)	6 to 10	Total Data width + Address width <= 26 bits Total Data width + Address width <= 57 bits	5 bits or 7 bits	DWCXG_NUM_TC (1 to 8)+ DWCXG_EST_WID (16, 20,24) + (6 or 7 bits)
Internal Register array based Instruction Table of MTL Rx Parser	112	0	112 bits	6 to 8	112 (data width) +8 (Address Width) <= 120bits	8 bits	120 bits
DMA TSO Memory/Descriptor cache memory	64		64 bits	2 to 11	Data width + Address width <= 120 bits	8 bits	72
	128		128 bits	2 to 11	Data width + Address width <= 247 bits	9 bits	137

Table 9-4 Memory Bus Width, ECC Code Size, and Checkbits Size (continued)

Memory Block	Configured Data Width	Additional Control Bits	Total Data Width (without ECC)	Address Width (Depends on Memory Size)	ECC Code Reference Range	Required Check bits	Total Data Width (with ECC)
MAC RxF memory	DWCXG_RXF_MEM_DW	0	DWCXG_RXF_MEM_DW	DWCXG_RXF_MEM_AW	DWCXG_RXF_MEM_DW+ DWCXG_RXF_MEM_AW	DWCXG_RXF_MEM_EW	DWCXG_RXF_MEM_DW+ DWCXG_RXF_MEM_EW

9.2.2.1 Handling the Address Mismatch

ECC is calculated for both memory data and memory address. However, only data is written in the memory along with the ECC. (address is excluded).

On the read interface, the ECC is checked. When an ECC correctable error is detected over the range of the address bit position, it is treated as an uncorrectable error. This is because data might have been read from a different location.

9.2.2.2 Diagnostic Support for the Error Management

Following statistics are provided for monitoring the error behavior on each of the memory blocks.

- Error status provided to management
  - A separate status for correctable, uncorrectable, and address mismatch is specified in the `MTL_ECC_Interrupt_Status` register.
  - Memory locations at which correctable and uncorrectable errors are detected is specified in the `MTL_ECC_Err_Addr_Status` register. In addition, a control bit (MEEAO field of MTL_ECC_Control register) decides, whether the first errored memory address is reported or the latest memory address that has error, is reported.
  - `MTL_ECC_Err_Cntr_Status` register has separate counters to count the number of correctable and uncorrectable errors
- Interrupts provided to management
  - Separate interrupts are generated for correctable and unrecoverable errors.
  - The `sbd_sfty_ue_intr_o` interrupt is generated when unrecoverable errors are detected and these errors cannot be masked.
  - The `sbd_sfty_ce_intr_o` interrupt is generated when correctable errors are detected. 25GMAC sets the respective interrupt enable bits (in the DMA/`MTL_ECC_Interrupt_enable` register).
  - To find the root cause of the error, read the DMA/`MTL_Safety_Interrupt_Status` and DMA/`MTL_ECC_Interrupt_Status` registers.
  - To clear the interrupt, write 1 in the respective interrupt status bit in DMA/`MTL_ECC_Interrupt_Status` registers.

For more details, see the *Register Descriptions* chapter.

Note

- The status/counters/interrupts are generated per memory.
- MTL Tx and Rx memory is logically divided into multiple queues. But, ECC diagnostics (status, counter, or interrupts) are common to all the queues.

9.2.2.3 Handling Uncorrectable Errors

Recovery from an uncorrectable error is based on the memory block and the operating mode of 25GMAC.

- MTL Tx memory

  If an uncorrectable error is detected, the packet is terminated with a forcible EOF. A CRC error (`mti_ecc_err`) is introduced by MAC for the transmitted packet, and the remaining packets in the FIFO are flushed with the assertion of mti_error and mti_eof signals. Flush status (Bit 13) is indicated to the application.

  When an uncorrectable error is detected the application must reset 25GMAC to ensure proper functionality.

  Note The MTL FIFO flush is issued for 25GMAC to recover with limited support.

  25GMAC generates an interrupt to indicate to the application that a correctable, uncorrectable or address mismatch error has occurred and the status is indicated in `MTL_ECC_Interrupt_Status` register.

- MTL Rx memory

  When an uncorrectable error is detected, reset 25GMAC IP to ensure proper functionality.

  25GMAC generates an interrupt to indicate to the application that correctable, uncorrectable or address mismatch errors are detected and the status is indicated in `MTL_ECC_Interrupt_Status` register.

  On detection of uncorrectable error in the receive packet data or status read from MTL Rx memory, 25GMAC drops the packet if possible; otherwise it forwards the packet with safety error status with error summary to be set.

  Note

  - XGMAC-MTL configuration: Issue hard reset. Soft reset is not supported in this configuration.
  - XGMAC-AXI configuration: Issue soft reset by programming the SWR field of the DMA_Mode register.

- MTL EST memory

When an uncorrectable error is detected in the MTL EST memory, 25GMAC follows these sequence of actions for recovery

a. 25GMAC disables the EST feature by resetting the EEST bit of MTL_EST_Control register. It sets all the gates to open state, generates an interrupt, and sets the status in the MTL_ECC_Interrupt_status register. This allows the software to reprogram the correct value in the EST memory.

b. The location of the uncorrected error in the EST memory is available in the EUEAS/ECEAS field of the MTL_ECC_Err_Addr_Status register.

25GMAC generates an interrupt to indicate to the application, if any correctable, uncorrectable and address mismatch errors are detected and the appropriate status is indicated in MTL_ECC_Interrupt_Status register.

Software must reprogram and restart EST after an uncorrectable error is reported in the EST memory. Issuing a reset is not mandatory in this case; however, other factors might cause a need to reset.

- Internal Register Based Instruction Table of MTL Rx Parser Memory

When an uncorrectable error is detected in the Instruction Table (IT) of MTL Rx Parser, the parser halts the parsing of all subsequent packets by disabling the parser.

- Halts the parsing of all subsequent packets by disabling the parser.
- Drops the currently parsed and already buffered packets if possible; otherwise routes the packets to the residual channel with Error Type as safety error and FRP safety error status. After the software completes the necessary steps against the detected ECC error, Rx Parser can re-enable the FRPE field of the MTL_Operation_Mode register, to continue parsing.

25GMAC generates an interrupt to indicate to the application that correctable, uncorrectable, or address mismatch errors are detected and the status is indicated in the MTL_ECC_Interrupt_Status register.

Same error could be detected and reported by the periodic flexible receive parser memory checks performed by 25GMAC. To avoid recursive reporting of injected ECC UE error, perform a valid write to the corrupted memory location.

- DMA TSO Memory

When an uncorrectable error is detected in the DMA TSO memory for any channel, ati_err signal is asserted which eventually inserts CRC error for the respective packet; 25GMAC closes the Transmit descriptor immediately.

When an uncorrectable error is detected in the Descriptor Cache memory for any channel, an interrupt is also generated to indicate to the application if any correctable, uncorrectable and address mismatch errors are detected. An appropriate status is indicated in DMA_ECC_Interrupt_Status register.

- XGMAC-MTL configuration: Issue a hard reset; soft reset is not supported in this configuration.
- XGMAC-AXI configuration: Issue soft reset by programming the SWR field of the DMA_Mode register. Otherwise, CSR gets corrupted.

- MAC RXF Memory

  When an uncorrectable error is detected, reset the 25GMAC IP to ensure proper functionality. 25GMAC generates an interrupt to indicate to the application that correctable, uncorrectable or address mismatch errors are detected and the status is indicated in the `MTL_ECC_Interrupt_Status` register.

9.2.2.4 ECC Error Injection Capabilities

25GMAC supports error injection capabilities for each memory as a static configuration. You can program the type of error and the position where the errors are injected in the data word/address/ECC bits.

Before injecting error, follow these steps:

- Program the Error Injection Mode. Use the `EIM` field of the `MTL_DPP_ECC_EIC` register.
- Program the Bit Location of error injection. Use the `BLEI` field of the `MTL_DPP_ECC_EIC` register.

Memories	Selection	Field to Program	Register
Tx, Rx, TSO, and Descriptor Cache	ECC Inject Error Enable	EIEE	MTL_Debug_Control
	ECC Inject Address Error Enable	EIAEE	
	ECC Inject Error Control; for example, insert 1-bit or 2-bit error	EIEC	
GCL	ECC Inject Error Enable	GCLEIEE	MTL_GCL_Control
	ECC Inject Address Error Enable	GCLEIAEE	
	ECC Inject Error Control; for example, insert 1-bit or 2-bit error	GCLEIEC	
MTL Rx Parser instruction table	ECC Inject Error Enable	RXPEIEE	MTL_Indirect_Acc_Control_Status
	ECC Inject Error Control; for example, insert 1-bit or 2-bit error either in data, check-bit or address	RXPEIEC	
MAC RXF memory DA Hash table, Rx VLAN Lookups	ECC Inject Error Control; for example, no-error, insert 1-bit or 2-bit error, either in data, check-bit, or address	MIAEIEC	MAC_Indir_Access_Ctrl
MAC RXF memory RSS Lookup	ECC Inject Error Control; for example, no-error, insert 1-bit or 2-bit error, either in data, check-bit, or address	RLEIEC	MAC_RSS_Address

ECC address error injection supports only 1-bit error injection. DUT does not detect multiple bit address error. However, in the ECC 2-bit error injection mode,

- 1-bit error is always inserted at the 0^{th} bit.
- Other 1-bit error is inserted at the location indicated by the BLEI field of `MTL_DPP_ECC_EIC` register. If BLEI is configured to 0, 25GMAC inserts only 1-bit error, at the 0^{th} bit.

While using ECC error injection in the debug mode,

- Ensure that there is no traffic in 25GMAC
- When multiple CSR writes are required for writing single data word into the memory, the application must ensure that all the CSR writes corresponding to one memory write maintains the same value for the error injection control word.

Note When operating in the flexible receive parser debug ECC error injection mode, the read must be performed only once to the memory location, to check the safety interrupt generation.

This is because, when a read access is performed (with STARTBUSY=1), the full 128-bits (96-bits) word are read from the flexible parser memory. It returns the 32-bits corresponding to the ADDR field of the MTL_Indirect_Acc_Control_Status register.

9.2.3 Enabling ECC Protection for Memories

Set any one of the following parameters to 1, to enable ECC protection for memories feature. After setting the parameter, set the appropriate bits of the `MTL_ECC_Control` register to enable ECC on specific features as per your configuration.

- DWCXG_ASP_ECC
- DWCXG_ASP_ALL

To enable the ECC protection for MAC Rx filter memory, program the RXFED field of the `MTL_ECC_Control` register. For more details, see the `MTL_ECC_Control` register description in the *Register Description* chapter.

For details about these options, see Automotive Safety Features Parameters in *Parameter Description* chapter.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

9.2.4 Signals Related to ECC Protection for Memories

- `tmi_wdata_ecc_o`
- `tmi_rdata_ecc_i`
- `dmi_wdata_ecc_o`
- `dmi_rdata_ecc_i`
- `est_wdata_ecc_o`
- `est_rdata_ecc_o`
- `tx_odd_wdata_ecc_o`
- `tx_odd_rdata_ecc_o`
- `tx_even_wdata_ecc_o`
- `tx_even_rdata_ecc_o`
- `rx_odd_wdata_ecc_o`
- `rx_odd_rdata_ecc_o`

- rx_even_wdata_ecc_o
- rx_even_rdata_ecc_o
- twc_wr_data_ecc_o
- trc_rd_data_ecc_o
- rwc_wr_data_ecc_o
- rrc_rd_data_ecc_o
- sbd_sfty_ue_intr_o
- sbd_sfty_ce_intr_o
- rxf_wdata_ecc_o
- rxf_rada_ecc_i

For a description of signals related to ECC Protection for Memories, see the *Signal Descriptions* chapter.

9.2.5 Registers Related to ECC Protection for Memories

- MTL_ECC_Control
- MTL_ECC_Interrupt_enable
- MTL_ECC_Interrupt_Status
- MTL_ECC_Err_Addr_Status
- MTL_ECC_Err_Sts_Rctl
- DMA_ECC_Interrupt_enable
- DMA_ECC_Interrupt_Status
- MTL_EST_Control
- MTL_GCL_Control
- MTL_Indirect_Acc_Control_Status
- MAC_Indir_Access_Ctrl
- MAC_RSS_Address
- MTL_Safety_Interrupt_Status
- DMA_Safety_Interrupt_Status

For more details, see the *Register Descriptions* chapter.

9.2.6 Programming Guidelines for ECC Protection for Memories

For more details about programming guidelines for ECC protection for memories, see “Programming Guidelines for ECC Protection for Memories” on page 1890.

9.3 On-Chip Data Path Parity Protection

9.3.1 Overview

This feature provides parity protection for Transmit and Receive data paths, where feasible. For every byte of data, one bit parity is generated.

At the transmit side, the data from the host memory is protected with parity until it is transmitted to TPE of the MAC layer.

At the receive side, data from the MAC RPE block is protected with parity until it is sent to the host.

Note In case of invalid bytes during EoF, parity is generated and checked for the invalid bytes, if any.

9.3.2 Transmit Datapath Parity Protection

Figure 9-5 describes the locations of parity generators and checkers in the transmit data path:

Following are the locations of the parity generators and checkers in the transmit data path:

- Parity is generated on the input data port, PG-a/b in Figure 9-5. If "parity on external ports" (DWCXG_ASP_PPE) is selected, the input parity provided by the application is propagated; otherwise parity is generated internally. For more details, see the *Signal Descriptions* chapter.

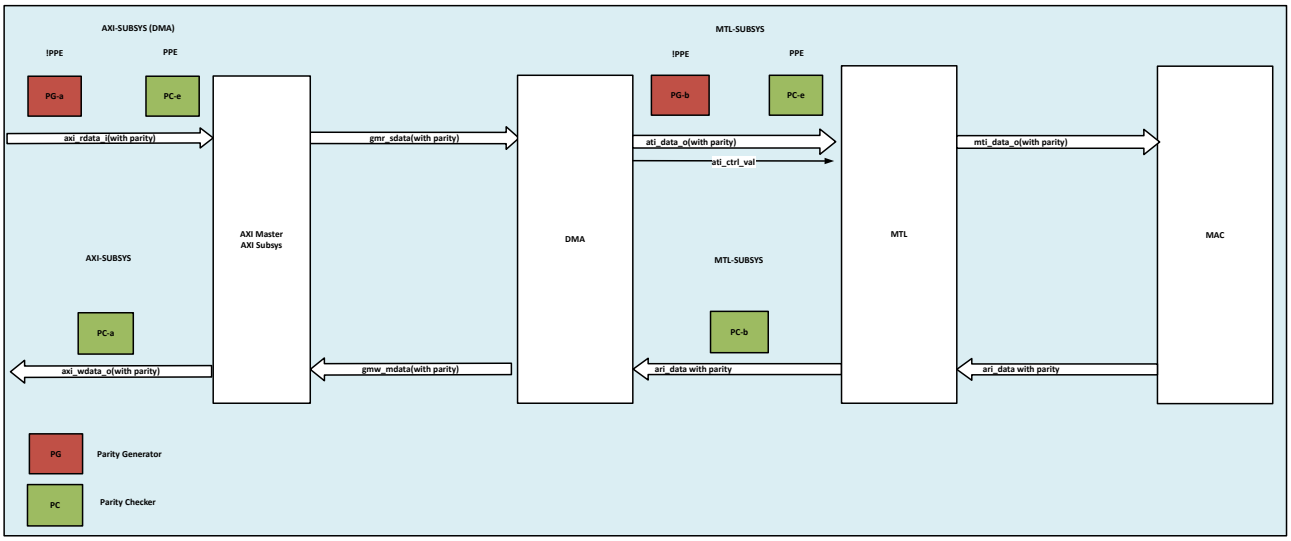
  Note Parity generator PG-a/b exists only if “parity on external ports" is not selected (DWCXG_ASP_PPE). Here a/b/c indicates the various configurations. For example, AXI/MTL.

- A parity check is performed on the input parity data received from the application interface, PC-e in Figure 9-5, and the respective error status is specified in the ATPES field of `MAC_DPP_FSM_Interrupt_status` register. This check identifies if the fault is internal or external to 25GMAC. The PC-e parity checker is present only if "parity on external ports" (`DWCXG_ASP_PPE`) is selected during configuration. Also, PC-e is present at different interfaces, AXI/ATI based on the configuration, AXI/MTL.

When uncorrectable ECC error is encountered, 25GMAC drops the packet if possible. The ongoing packet is terminated with CRC corruption and flushed; the `mti_error` and `mti_eof` signals are asserted.

In the XGMAC-AXI configuration, when uncorrectable DPP error is detected in the descriptor data, the packet is terminated on ATI and `ati_error` and `ati_eof` signals are asserted.

Figure 9-5 Block Diagram of Transmit/Receive Data Path Parity Protection



9.3.3 Receive Data Path Parity Protection

On the receive side, parity check is performed on the output data of AXI manager interface or ARI interface (PC-a/b, based on the configuration) as show in Figure 9-5 and the respective parity error status is reported in the ARPES bit of `MAC_DPP_FSM_Interrupt_Status` register

Note

- Parity generator PG-a/b exists only if "parity on external ports" is not selected (DWCXG_ASP_PPE). Here a/b indicates the various configurations. For example, AXI/MTL.
- PC-e does not check parity during reset.
- On detection of parity (uncorrectable) error on
  - ARI data, 25GMAC marks the packet with Error Type as safety error.
  - AXI manager write data, 25GMAC sets the wdata_m_se_o signal to indicate that respective write data-word has uncorrectable error.

9.3.4 DMA Data Path Parity Protection

The Transmit DMA fetches packet data and descriptor from the host memory through the AXI/Native Manager interface. In transmit DMA, the descriptors are consumed. When TSO feature is selected, the Transmit DMA modifies some of the header fields of the packet for segmentation. So, the header data from TSO controller/memory and the control data from the TDT module are multiplexed with the actual data from host memory before sending on the ATI interface, to MTL. When descriptor cache is selected, the Transmit/Receive DMA prefetch descriptor from Cache memory could be the same or different from TSO memory. Therefore, the descriptor from cache and descriptor from System Memory are sent through the TDF/RDF to TDT/RDT. In addition, TSO/descriptor cache memory have a debug access mode.

When TSO feature is absent/inactive, the fetched packet is transferred by the TxDMA to the MTL TxFIFO by aligning/concatenating (as necessary) the data fetched in different burst transfers. The TxDMA also generates and pushes the "control words" for each packet in the same data path before the start of a packet.

There are three data path for the Receive DMA:

- Write data/descriptor back to AXI manager
- TD/RD write-back mode through TDW/RDW
- Receive DMA data write to AXI manager through RDT from ARI data

RDT has byte oriented manipulation such as shifting/multiplex operations with ARI data.

Following are the locations of parity generators and checkers in the DMA data path

- The parity is generated on the control data from the DMA TDT module (PG2-ctrlw shown in Figure 9-6). PG2-ctrlw is a per-channel block. One channel with the corresponding parity is selected in TDT.
- If TCP segmentation (`DWCXG_TSO_EN`) feature is selected during the configuration, the parity is generated on the un-modified header from TSO external memory through
  - Memory Arbiter (PG1-hdr in Figure 9-6) and terminated at TDT TSO block (before the TSO header replacement, PC1-hdr in Figure 9-6.
  - The modified header data provided by the DMA TSO controller (PG2-tso in Figure 9-6).

  PC1-hdr and PG2-tso are per-channel blocks. One channel with the corresponding parity is selected in TDT.
- If TCP segmentation (`DWCXG_TSO_EN`) feature is selected during the configuration, parity check is performed on the feedback header data (PC-hdr in Figure 9-6) which is originally generated from the TDT AXI read data after TSO segmentation and wrote into TSO memory. As the TSO memory has debug access mode (`DWCXG_CACHE_MEM_DBG_ACCESS`), one parity generator is performed before the PC-hdr (PG-dbg in Figure 9-6). The respective parity error is reported in TPES bit of `MAC_DPP_FSM_Interrupt_status` register.
- As the read descriptor data is consumed in both TDT and RDT modules, the read TX/RX descriptor data is from host memory through AXI manager or external Descriptor Cache memory. Parity generator must be put at the memory read interface after ECC check (Figure 9-6).

  Read Descriptor Parity check is performed on this data (PC1-td/rd in Figure 9-6). PC1-td/rd are per-channel block. So, we have `DWCXG_TX_CH+DWCXG_RX_CH` error status. The respective parity error is reported in `TDPES/RDPES` field of `DMA_DPP_FSM_Interrupt_status` register.
- As the descriptor cache memory write path must be protected, one parity checker is put at the write interface before ECC generator (PC-desc in Figure 9-6). Because the Descriptor Cache memory has a debug access mode, when different memory configurations are used, one parity generator such as TSO memory, must be performed before PC-desc. The respective parity error is reported in DCPES bit of `MAC_DPP_FSM_Interrupt_status` register.
- Parity is generated on the modified Tx/Rx write-back status which is generated from DMA TDW/RDW module (PG-tdw/rdw in Figure 9-6).
- On detection of ECC uncorrectable or parity error in the descriptor data path, 25GMAC closes that descriptor without any process and provides descriptor error along with Error type as safety error.

  Such detected safety error is propagated through descriptor-cache/TSO memory with additional memory bit, which is also protected by ECC.

Note

If RxDMA gets a descriptor

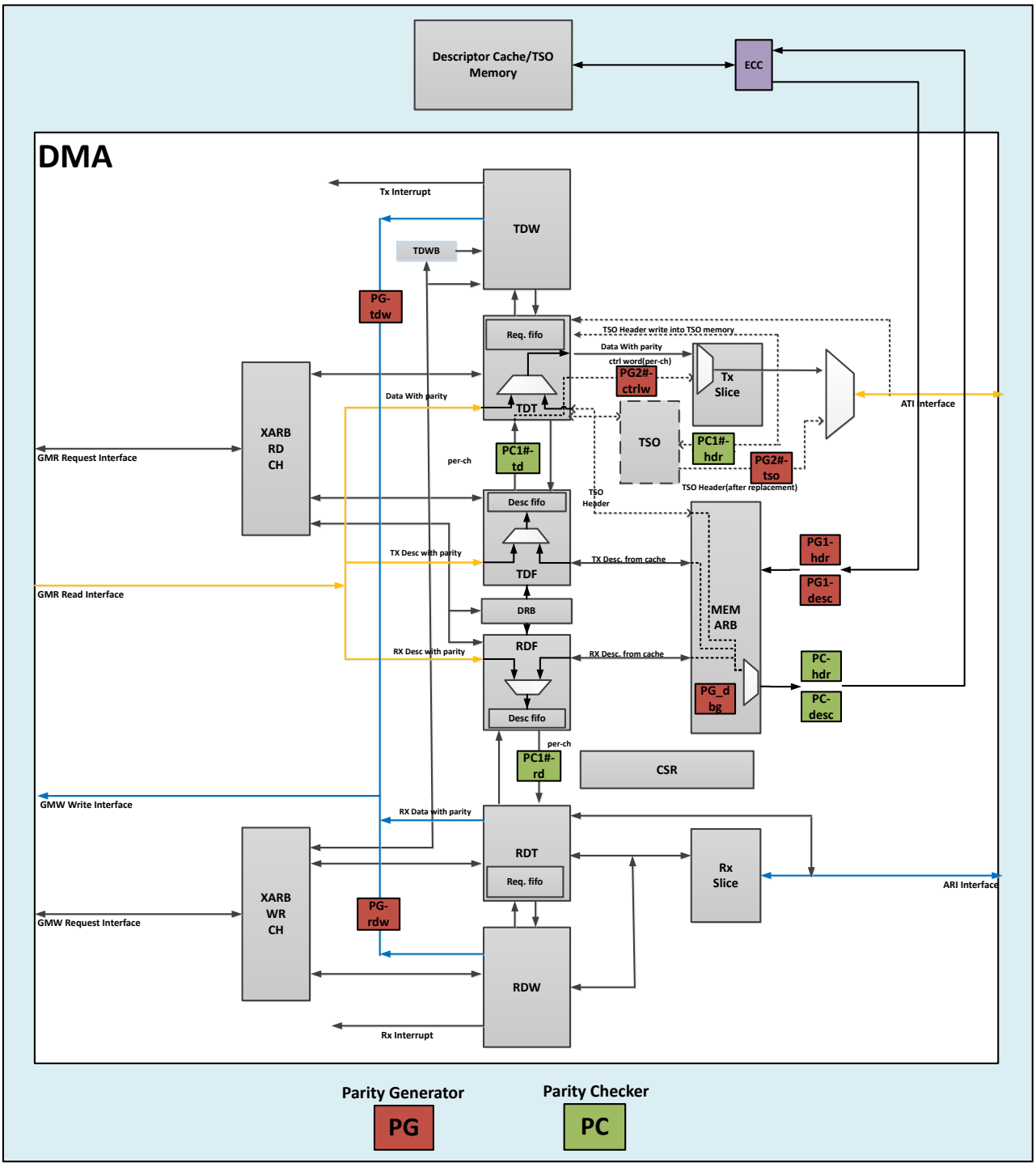
- with OWN bit = 0, and
- has safety error,

DMA closes the descriptor with safety error and goes to SUSPEND state. This is because, OWN bit can be corrupted because of safety error.

Note

Based on the value of the DWCXG_DIFF_CACHE_MEM_EN parameter, Descriptor Cache Memory and TSO Memory can be same or different. If they are same and use one memory arbitration block, the PG1-desc and PG1-hdr are one block; similarly, PC-desc and PC-hdr are one block. Otherwise, they are in separate blocks.

Figure 9-6 DMA Data Path Parity Protection

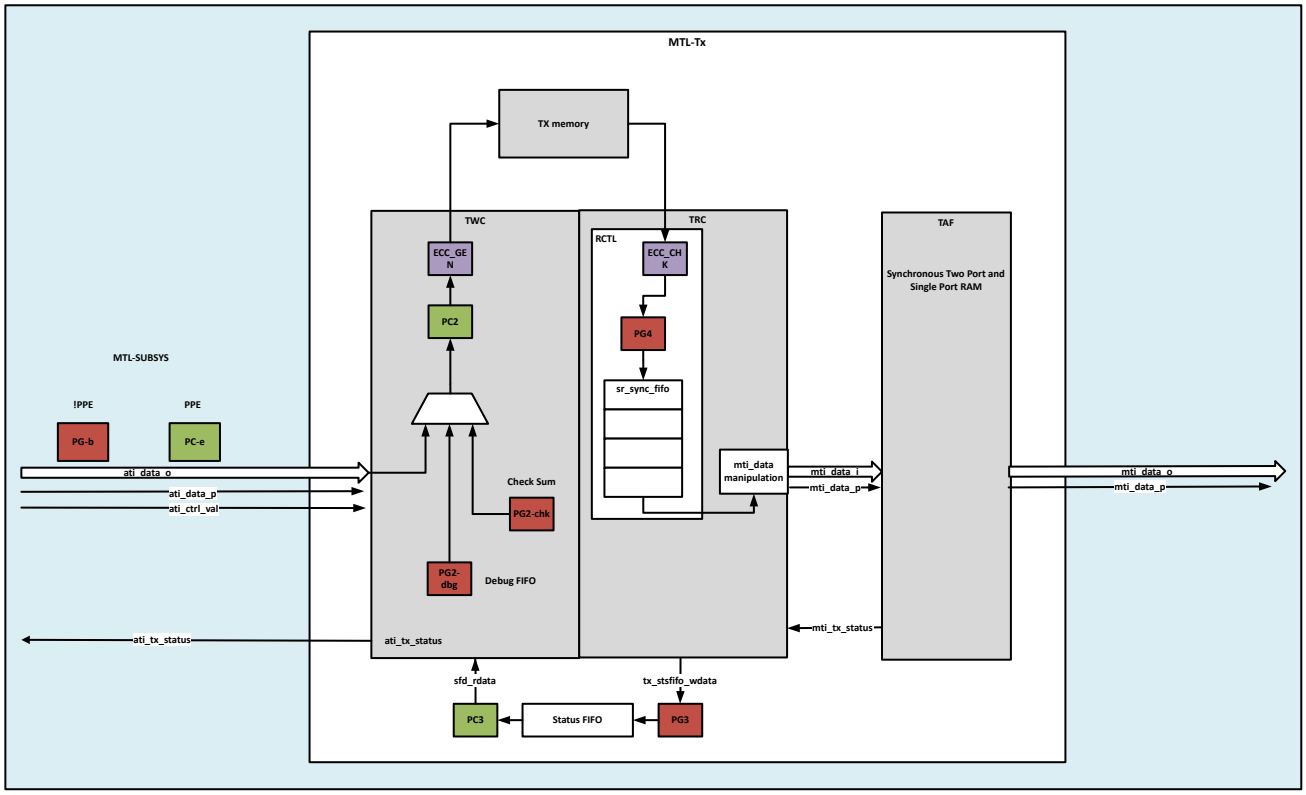


9.3.5 MTL-Tx Data Path Parity Protection

At the MTL-Tx, the packet data is stored/written into the TXFIFO memory. When TX Checksum Offload Engine is active, the calculated checksums/control signals are also generated and written into the TXFIFO memory. Therefore, the input data from the ATI interface along with the internally generated checksum control word are multiplexed before it is written into the MTL TX FIFO. Tx status which is received from MAC is written into a FIFO.

At the MTL-Tx read side, the packet data is read out from the MTL read-side. After ECC check in MTL read-side, packet data write into the TAF, finally transmitted into MAC.

Figure 9-7 MTL-Tx Data Path Parity Protection



Following are the locations of parity generators and checkers in MTL transmit data path:

- If checksum offloading feature (`DWCXG_TX_COE`) feature is selected during the configuration, parity is generated on the checksum data generated in MTL (PG2-chk in Figure 9-7) see the *Signal Descriptions* chapter.
- Parity check is performed at the transmit write controller (twc module) in MTL (PC2 in Figure 9-7) before writing the data to TX FIFO and the parity generation is terminated at this checker because, the ECC protects the external memory paths. The respective parity error status is reported in `MTPES` field of `MAC_DPP_FSM_Interrupt_status` register.
- If transmit Status Enable (`DWCXG_TXSTSE`) is active in MTL, parity is generated on the received status from MAC (PG3 in Figure 9-7).

- If transmit Status Enable (DWCXG_TXSTSE) is active in MTL, parity check is performed on the Tx Status which is transferred on ATI interface (PC3 in Figure 9-7). The respective parity error status is reported in MTSPES bit of `MAC_DPP_FSM_Interrupt_Status` register.
- Parity is generated at the transmit read controller (trc module) in MTL (PG4 in Figure 9-7) after reading the data from TX FIFO, to protect the data path from TX-FIFO to MAC Layer. The corresponding parity checker is terminated at TBU block in MAC layer.

Note	When a parity error (PC2 in Figure 9-7) is detected at transmit ATI interface, the ati_err signal is asserted which inserts CRC error for the respective packet.

9.3.6 MTL-Rx Data Path Parity Protection

At the MTL-Rx, the packet data and status from the MTL Rx FIFO are sent to DMA where it is multiplexed with the descriptor write-back status from the RDW module before transferring to the host.

Following are the locations of parity generators and checkers in receive data path:

- Parity checker is performed on the MTL Rx FIFO write side before the ECC generator (PC6 in Figure 9-8) which can protect the data path from RPE output through RFC/RBU/RAF to Rx-FIFO write side (RWC module). The respective parity error status is reported in MRWCPES bit of `MAC_DPP_FSM_Interrupt_status` register.
- Parity is generated on the MTL Rx FIFO read data (PG7 in Figure 9-8) after ECC correction.
- Parity checker is performed on the MTL Rx FIFO read control out data if the ARI data bus is lost in Parity checker within the un-MTL-Subsys configuration (PC7 in Figure 9-8). The respective parity error status is reported in MRRCPES bit of `MAC_DPP_FSM_Interrupt_status` register.

When a parity (uncorrectable) error is detected on the MTL Rx data path, if the error is detected

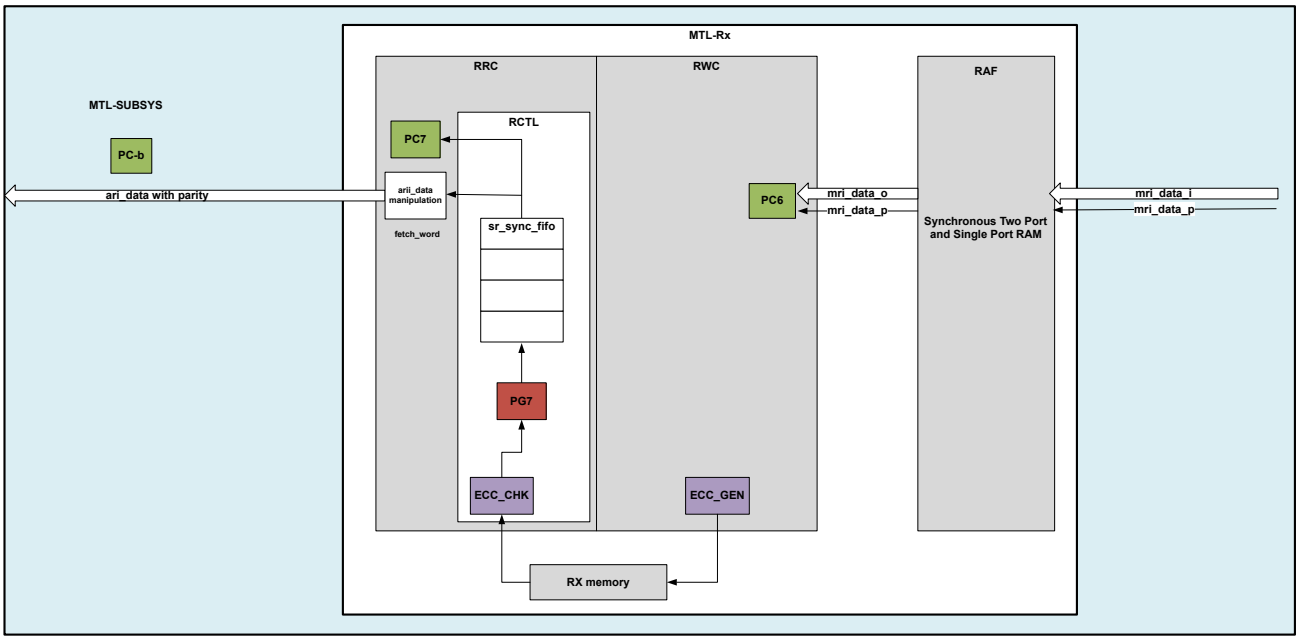
1. Before the Rx FIFO write, 25GMAC drops the packet if possible. Otherwise forwards the packet with Error Type as safety error.
2. After the Rx FIFO read, 25GMAC sets the ari_sftyerr_o signal.

   For more details, see *Signal Descriptions* chapter.

In XGMAC-AXI configurations, after asserting the ari_sftyerr_o signal, 25GMAC marks the packet with Error Type as safety error.

Note	Parity checker PC7 exists only if "MTL Subsystem" is not selected (DWCXG_MTL_SUBSYS). Otherwise PC7 is not present and PC-c does the parity check on ARI data bus.

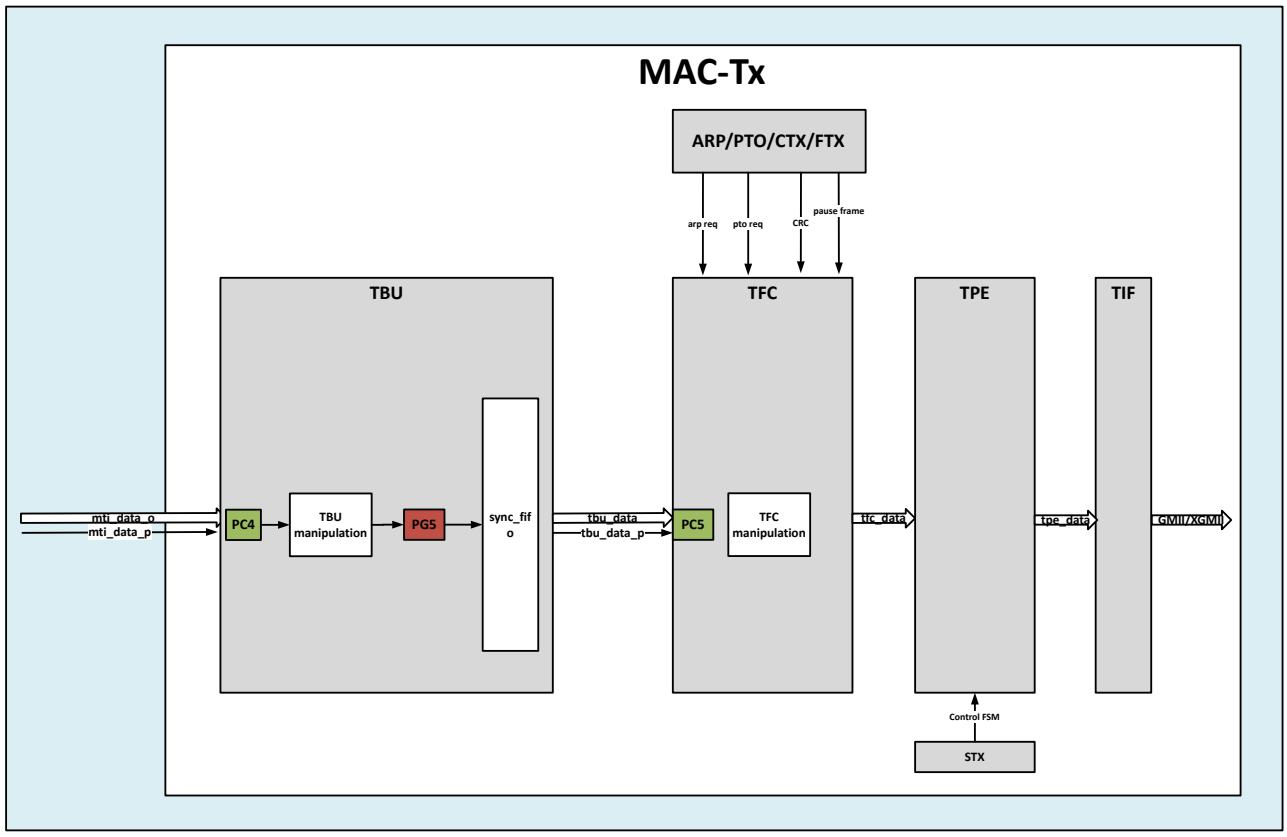
Figure 9-8 MTL-Rx Data Path Parity Protection



9.3.7 MAC-Tx Data Path Parity Protection

At the MAC layer, packet data go through the TBU (switch endian, VLAN tag/SA offload) and TFC (CRC padding/ARP, PTO, TBU, FPE, Pause request arbitration), then complete transmission within the preamble/SCC/SFC in TPE.

Figure 9-9 MAC-Tx Data Path Parity Protection



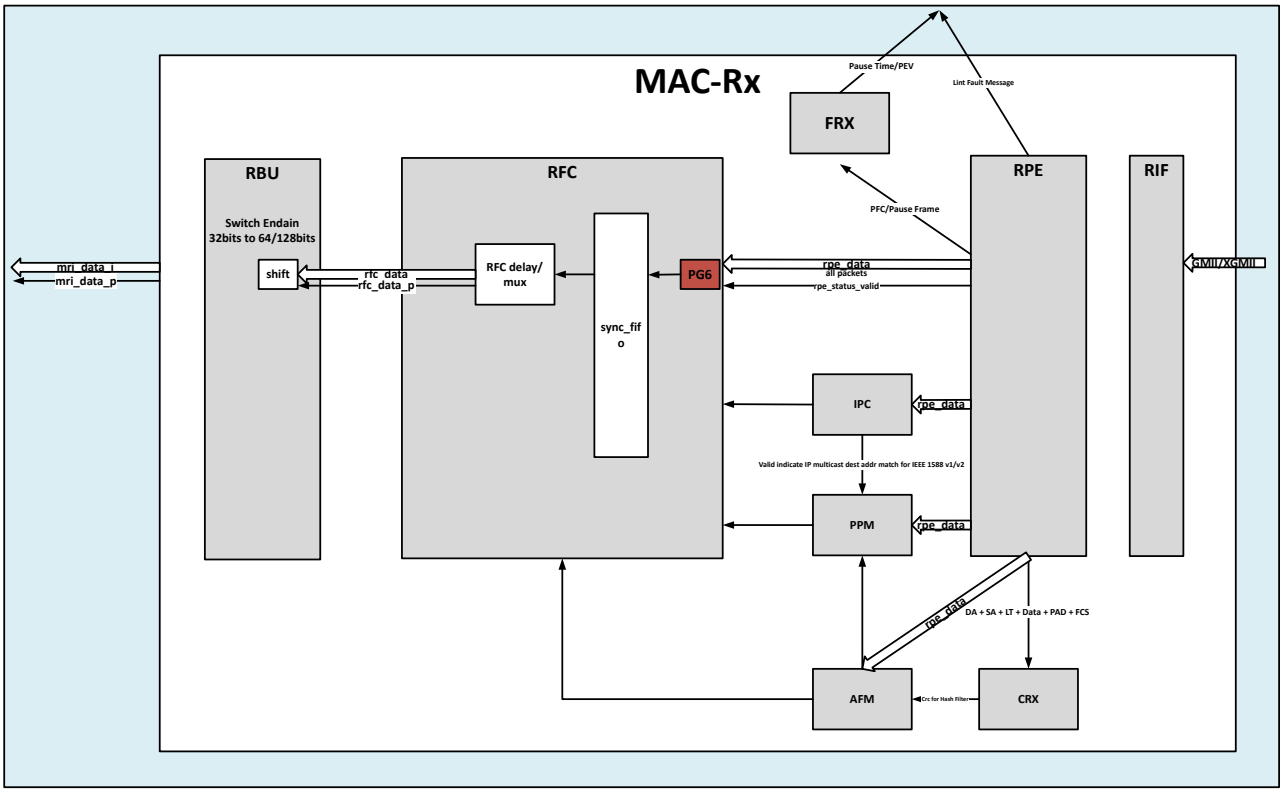
Following are the locations of parity generators and checkers in MAC transmit data path:

- Parity check is performed at the Transmit Bus Unit (TBU module) in the MAC (PC4 in Figure 9-9) before the TBU manipulation. PG4 and PC4 protect the data path from Tx-FIFO read side to MAC layer. The respective parity error status is reported in the MTBUPES bit of `MAC_DPP_FSM_Interrupt_status` register.
- Parity Generator is performed at the Transmit But Unit (TBU module) in MAC (PG5 in Figure 9-9) before the TBU sync-fifo after the TBU manipulation. Therefore, the corresponding Parity Checker is performed at the Transmit Frame Control (TFC module) in MAC (PC5 in Figure 9-9) before the TFC manipulation (MAC generated packet offload such as PTO/ARP/FPE/CTRL Frame and CRC/Padding insertion). PC5 is terminated at this point because there are bit-wise operations after TFC manipulation in Transmit Protocol Engine (TPE module). The respective parity error status is reported in MTFCPES bit of `MAC_DPP_FSM_Interrupt_status` register.

9.3.8 MAC-Rx Data Path Parity Protection

At the MAC receive path, the packet data and status from the MAC RPE are sent to MTL-Rx. Figure 9-10 shows the locations of parity generators and checkers in the receive data path.

Figure 9-10 MAC-Rx Data Path Parity Protection



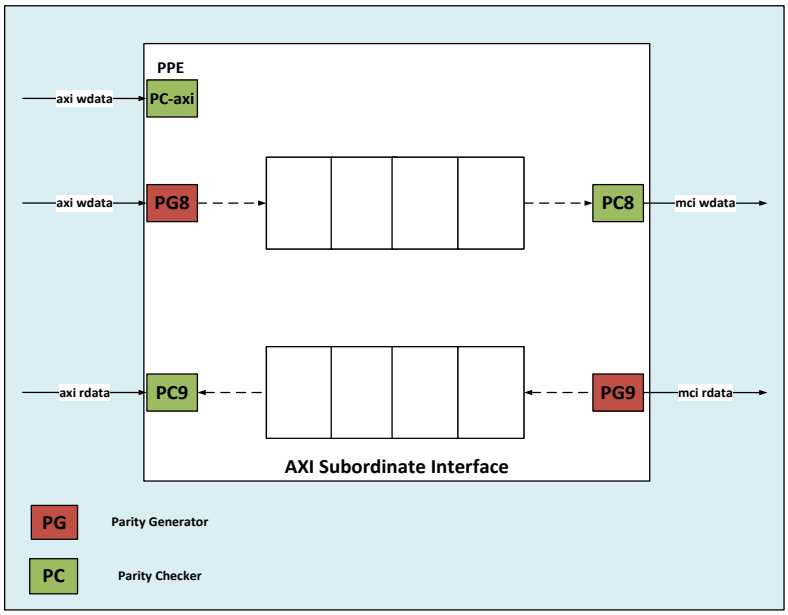
On the MAC receive data path, parity is generated on the Receive Frame Control (RFC module) input data (PG6 in Figure 9-10) after Receive Protocol Engine drops the Preamble/SFD, before packet data write into RFC sync-FIFO. The corresponding Parity Checker is performed at MTL Rx-FIFO write side because the Receive Bus Unit (RBU module) and RFC module does simple byte-oriented manipulations such as data shifting/multiplexing.

When a parity (uncorrectable) error at is detected at the MTL Rx FIFO write checker, 25GMAC drops the packet if possible; otherwise the packet is forwarded with Error Type as safety error.

9.3.9 AXI Subordinate Data Path Parity Protection

The AXI subordinate interfaces with the application and provides the 32bit, 64-bit or 128-bit CSR access and also handles the Narrow burst and FIXED or INCR burst. As the data is not modified in this block, the parity generators and checkers are provided at the AXI and CSR interface. If "parity on external ports" feature (DWCXG_ASP_PPE) is selected during the configuration, input/output parity ports are driven by/to the application. For more details, see the *Signal Descriptions* chapter.

Figure 9-11 AXI Subordinate Data Path Parity Protection



Following are the locations of parity generators and checkers in the AXI salve module:

- Parity is generated on the AXI subordinate write data (wdata_s_i, at PG8 in Figure 9-11). PG8 is always present, irrespective of the selection of "parity on external ports" feature (DWCXG_ASP_PPE).

  When "parity on external ports" feature is selected, either externally provided parity or the internal generated parity can be selected by setting the EPSI field of MTL_DPP_Control register.
- Parity is generated on the CSR read data (mci_rdata_o, PG9 in Figure 9-11).
- When "parity on external ports" feature is selected, (DWCXG_ASP_PPE is enabled) and if the parity check on the external secondary ports is enabled (by setting the EPSI bit of `MTL_DPP_Control` register), parity check is performed on the AXI salve write data (`wdata_s_p_i`, PC-axi in Figure 9-11). The respective parity error status is reported in the CWPES field of the `MAC_DPP_FSM_Interrupt_Status` register.
- Parity check is performed on the CSR write data (`mci_wdata_i`, PC8 in Figure 9-11. The respective parity error status is reported in the CWPES field of the `MAC_DPP_FSM_Interrupt_Status` register.

  If a parity error detected on the CSR write data, the write to the respective address location is stopped and the address for which parity error is detected is specified in the `MAC_AXI_SLV_DPE_Addr_Status` register.

  For the AXI or AXI4-Lite subordinate data path, when parity (uncorrectable) error is detected in the CSR write/read path, RTL closes the respective CSR write/read transaction with SLVERR error status.

  For the APB3, APB4, or MCI subordinate data path, when parity (uncorrectable) error is detected in the CSR write/read path, RTL closes the respective CSR write/read transaction by asserting the response signal.

For more details, see the *Signal Descriptions* chapter.

- Parity check is performed on the AXI Subordinate read data (rdata_s_o, PC9 in Figure 9-11) and the respective parity error status is reported in the ASRPES field of MAC_DPP_FSM_Interrupt_Status register.

9.3.10 Parity Error Injection

Each parity checker supports a control to inject parity error. When enabled, one programed (EIM/BLEI field of the MTL_DPP_ECC_EIC register) bit parity/data of the first valid data at the generators are flipped. Application must set the appropriate bit in the MTL_DPP_Control/DMA_DPP_Control register; after the respective parity/data bit is flipped the hardware clears the same.

After the checker inserts error in the parity/data input, the checker must detect the mismatch and report the error status in the MAC_DPP_FSM_Interrupt_Status/DMA_DPP_Interrupt_Status register.

When the application injects error in the data path or memories, 25GMAC does not associate the corresponding uncorrectable error either with transmit/receive packet or descriptor.

9.3.11 Interrupt for Parity Error

25GMAC generates an interrupt, sbd_sfty_ue_intr_o to indicate to the application if any parity error is detected. Appropriate status is indicated in the DMA_Safety_Interrupt_Status or MTL_Safety_Interrupt_Status, and the MAC_DPP_FSM_Interrupt_Status or DMA_DPP_Interrupt_Status registers. All the parity errors reported by the different checkers are logged in the MAC_DPP_FSM_Interrupt_Status or DMA_DPP_Interrupt_Status register.

9.3.12 Recovering from a Data Path Parity Error

If a Data Path Parity error is reported, it is recommended to soft-reset the IP. This is because, data path parity errors are non-recoverable and the functionality of the IP is not guaranteed after a safety non-recoverable interrupt is reported.

9.3.13 Enabling On-Chip Data Path Parity Protection

Set the DWCXG_ASP_ALL parameter to 1 during the configuration to enable this feature. When you enable this feature, by default, parity generation/detection feature is also enabled. You can program the OPE field of the MTL_DPP_Control register to select odd or even parity generation/detection.

For details about these options, see Automotive Safety Features Parameters in the *Parameter Descriptions* chapter.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

9.3.14 Signals Related to On-Chip Data Path Parity Protection

- ati_data_p_i
- ari_data_p_o
- mci_wdata_p_i
- mci_rdata_p_o
- sbd_sfty_ue_intr_o

- mci_wdata_p_i
- pwdata_p_i
- prdata_p_o
- wdata_m_p_o
- rdata_m_p_i
- wdata_s_p_i
- rdata_s_p_o

For a description of signals related to On-Chip Data Path Protection, see the *Signal Descriptions* chapter.

9.3.15 Registers Related to On-Chip Data Path Parity Protection

- MAC_DPP_FSM_Interrupt_Status
- MAC_AXI_SLV_DPE_Addr_Status
- MTL_DPP_Control
- DMA_DPP_Interrupt_Status
- DMA_DPP_Control

For more details, see the *Register Descriptions* chapter.

9.3.16 Programming Guidelines for On-Chip Data Path Parity Protection

For details about programming guidelines for on-chip data path parity protection, see "Programming Guidelines for On-Chip Datapath Parity Protection" on page 1892.

9.4 FSM One-Hot and Timeout Protection

9.4.1 Overview

The FSM one-hot protection feature supports FSM one-hot and time out protection.

9.4.2 FSM One-Hot Protection

Set the DWCXG_ASP_ALL parameter to 1 during the configuration to enable this feature.

25GMAC implements one-hot encoding scheme for holding the state register. FSM state is monitored for every clock, after the reset is de-asserted. When a bit flips due to transient errors or permanent faults, the erroneous FSM is set to its default state and an unrecoverable error is indicated.

When you enable the FSM one-hot protection by setting the PRTYEN field of the MAC_FSM_Control register, the FSM state error is indicated by setting the FSMPES field of the MAC_DPP_FSM_Interrupt_Status register. Also, the Safety Interrupt (sbd_sfty_ue_intr_o) is asserted when an FSM error is detected.

25GMAC also supports Error Injection mode for the FSM parity error check. Program the Error Injection enable, bits [23:16] of the MAC_FSM_Control register, for the respective clock domain.

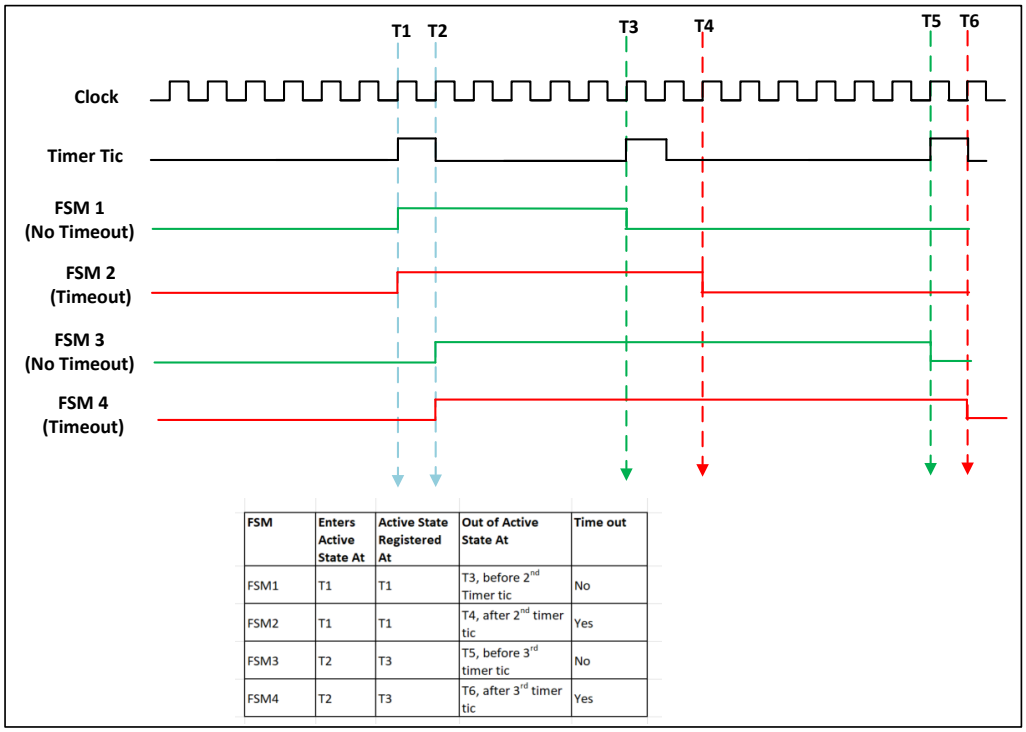
Note The FSM Parity error is classified as non-recoverable error and software intervention (IP reset) is required to recover from the error state.

9.4.3 FSM Timeout Protection

The FSM Timeout is a mechanism to ensure that all FSMs in 25GMAC can complete a transaction and return to a known completion state (IDLE or any other state that indicates completion of a transaction/transfer) within the programmed timeout value.

Program the TMR field of MAC_FSM_ACT_Timer register with a value that indicates the number of CSR cycles required to generate a 1 microsecond tic. This microsecond tic is internally used to generate the programmed timeout duration. Two timeout tics (normal mode timeout and large mode timeouts) are generated to provide flexibility to choose one per clock domain, by using the bits [31:24] of MAC_FSM_Control Register. Supported values for Timeout duration are 1us, 1.024ms, 16.384ms, 65.5536ms, 262.144ms, 1048.576ms (~1sec), 4194.304ms (~4s), 8.388s, 16.777s, and 33.554s which is based on the programmable bits NTMRMD, LTMRMD fields of MAC_FSM_ACT_Timer register. Interface timeouts are based on the normal mode tic generation; only the FSM timeouts depend on either normal or large tick selection.

Figure 9-12 FSM Timeout Timing Diagram



FSM	Enters Active State At	Active State Registered At	Out of Active State At	Time out
FSM1	T1	T1	T3, before 2nd Timer tic	No
FSM2	T1	T1	T4, after 2nd timer tic	Yes
FSM3	T2	T3	T5, before 3rd timer tic	No
FSM4	T2	T3	T6, after 3rd timer tic	Yes

In Figure 9-12, the timer ticks are generated based on the programmable timeout value. The timer ticks are synchronized and made available in all the clock domains that have the FSMs.

At every timer tick, all the FSMs are monitored for active state (non-IDLE state, which indicates the FSM is actively processing transactions or hand-shakes) and an FSM active flag bit is set. The active flag bit is implemented for every FSM that is monitored for timeout. When the FSM reaches an IDLE or transaction completion state, the active flag bit of that FSM is reset. At the subsequent timer tick, all the FSMs' active flag bits are monitored and any flag that is set indicates a timeout for that FSM. This process of setting the flag and checking at subsequent timer tick is repeated at every tick.

Timeout Error Injection per clock domain is enabled by programming [15:8] fields of `MAC_FSM_Control` register. When Timeout error injection enable is set for a clock domain, the FSMs in that clock domain automatically timeout and generate an interrupt even without traffic. As error injection is a debug mode, TMR value need not indicate 1us, but can be programmed to a smaller value at which debug mode ticks are needed. FSM Timeout and Parity Error Injection modes can be used for testing at key-on/key-off.

The FSM timeout status is specified in the [15:8] field of `MAC_DPP_FSM_Interrupt_Status` register as per the respective clock domains. One FSM timeout status bit is made available per clock domain. The safety interrupt (`sbd_sfty_ue_intr_o`) is asserted when the timeout error is set for any of the clock domains. 25GMAC does not attempt to recover from a FSM timeout condition and relies on the application to take the corrective action (resetting 25GMAC).

Note

- Based on the time at which the FSM enters the active state relative to the timer ticks the timeout period could be any value between the programmed timeout and 2x programmed timeout.
- FSM related to PPS (when configured) will not be protected by timeouts as the delays involved in this FSM state transitions can be huge.

9.4.4 Recovering From FSM Parity/Timeout Errors

If a FSM parity/timeout error is reported, it is recommended to soft-reset the IP. This is because, data path FSM parity/timeout errors are non-recoverable and the functionality of the IP is not guaranteed after a safety non-recoverable interrupt is reported.

9.4.5 Enabling FSM Parity and Timeout Protection

Set the `DWCXG_ASP_ALL` parameter to 1 during the configuration, to enable one-hot FSM protection feature. To enable the FSM timeout feature, program the `TMOUTEN` field of the `MAC_FSM_Control` register. For details about these options, see Automotive Safety Features Parameters in the *Parameter Descriptions* chapter.

Note See the “Enabled” field of the parameter descriptions to understand the dependencies.

9.4.6 Signals Related to FSM Parity and Timeout Protection

- `sbd_stfy_ue_intr_o`

For a description of signals related to FSM Parity and Timeout Protection, see the *Signal Descriptions* chapter.

9.4.7 Registers Related to FSM Parity and Timeout Protection

- `MAC_FSM_Control`
- `MAC_FSM_ACT_Timer`
- `MAC_DPP_FSM_Interrupt_Status`

For more details, see the *Register Descriptions* chapter.

9.4.8 Programming Guidelines for FSM Parity and Timeout Protection

For details about programming guidelines for FSM protection, see “Programming Guidelines” on page 1839.

9.5 Application/CSR Interface Timeout Protection

9.5.1 Overview

This feature provides timeout protection to the application/CSR Interface.

9.5.2 Description

All the interfaces which has the handshake mechanism in 25GMAC are monitored for potential hangs due to the external agent (Manager/Subordinate/Interconnect/Application client) not responding to the requests/transfers initiated by 25GMAC. After the request is initiated the response arrival interval is monitored. If the response does not arrive within a programmed time (TMR field of `MAC_FSM_ACT_Timer`) the timeout is triggered, using similar trigger generation as explained in the Section FSM protection.

The hardware does not attempt to recover from Application/CSR Interface hangs and relies on software to take appropriate corrective action.

The Application Interface timeout status is provided in the [23:16] field of `MAC_DPP_FSM_Interrupt_Status` register. The Safety interrupt (`sbd_sfty_ue_intr_o`) is asserted when application timeout status is set.

9.5.3 ATI and ARI Interface

The timeout is triggered under the following scenarios:

- 25GMAC requests for the Transmit packet status transfer by asserting the `ati_txstatus_val_o` signal and the application does not respond with assertion of `ati_txstatus_ack_i`, within the programmed time.
- 25GMAC requests for a data/status of receive packet by asserting `ari_rx_status_val_o` and the application does not respond with assertion of `ari_ack_i` within the programmed time

This feature is available in the XGMAC-MTL configuration.

The Timeout status for the ATI and ARI interface is set in the ATITES and ARITES fields of `MAC_DPP_FSM_Interrupt_Status` register.

9.5.4 AXI Manager Interface

The timeout is triggered under the following scenarios:

- The 25GMAC initiates Write or Read ADDRESS transfer (by asserting AWVALID or ARVALID) and corresponding ready (AWREADY or ARREADY) is not asserted within the programmed time.
- The 25GMAC initiates Write DATA transfer (by asserting WVALID) and corresponding ready (WREADY) is not asserted within a programmed time.

This feature is available in XGMAC-AXI configuration.

The Timeout status for the AXI manager interface is set in the MSTTES field of `MAC_DPP_FSM_Interrupt_Status` register.

9.5.5 AXI Subordinate Interface

The timeout is triggered under the following scenarios:

- The 25GMAC responds to a Read request (by asserting RVALID) and external Manager does not accept the response data, by asserting (RREADY) within a programmed time.
- The 25GMAC responds to a Write request (by asserting BVALID) and external Manager does not accept the response by asserting (BREADY) within a programmed time.

This feature is available in AXI-SUBORDINATE configuration

The Timeout status for the AXI subordinate interface is set in "SLVTES" field of `MAC_DPP_FSM_Interrupt_Status` register.

Note

- Protection is not needed for other Register Bus interfaces (MCI/APB/AHB) because potential hangs can be only when 25GMAC does not respond to the requests. In such cases the 25GMAC internal protection logic (FSM timeout) detects such defects.
- Based on the time at which the transfer is initiated relative to the timer ticks, the timeout period could be any value between the programmed timeout and 2x times the programmed timeout.
- When an unrecoverable safety interrupt is issued and the read of the Interrupt status register returns all zeros, it implies that the CSR read is not functional. As soft reset of 25GMAC is not possible without the CSR access, and therefore, a hard reset of 25GMAC is recommended.

9.5.6 Enabling Application/CSR Interface Timeout Protection

Based on your configuration, this feature is available as follows:

- ATI and ARI Interface timeout protection is supported in XGMAC-MTL configuration.
- AXI Manager Interface timeout protection is available in XGMAC-AXI configuration.
- AXI Subordinate Interface timeout protection is available in XGMAC-AXI subordinate configuration.

9.5.7 Signals Related to Application/CSR Interface Timeout Protection

Following is the main signal related to Application/CSR interface timeout protection feature:

- `sbd_sfty_ue_intr_o`

For a description of signals related to Application/CSR Interface Timeout Protection, see the *Signal Descriptions* chapter.

9.5.8 Programming Guidelines for Application/CSR Interface Timeout Protection

See “Programming Guidelines for FSM and Timeout” on page 1893 for details.

For a description of signals related to Application/CSR Interface Timeout Protection, see the *Signal Descriptions* chapter.

9.6 Control Register Protection

All the critical control registers in each of the sub-modules (MAC, MTL, DMA) are parity protected. As the read operations from the application are not predictable, a mechanism to periodically (by a timer tick) read the registers and validate the parity, is defined. Parity checking is done when application read or safety read occurs.

When an application read/write and safety read occur in the same clock cycle, priority is given to application read.

9.6.1 Parity Generation

On detection of a valid write to a protected control register, a 4-bit parity is calculated based on the write data to the register. After reset, the byte-wise parity of all the protected control registers is auto-populated based on the reset values of the registers.

Initially after reset, CSR Parity registers are filled with parity associated with reset value of each registers. Byte Enable is supported for parity generation and parity is updated only for those bytes that are enabled.

Mask bits are assigned for each bit of safety protected CSR registers according to which parity is calculated. That is, all the reserved bits and hardware writable bits are excluded from the parity computation by using the mask bits.

9.6.2 Parity Checking

Periodically, an internal safety read is initiated on the register interface and the address generated by this module covers only the protected safety registers. For each of these safety read operations (along with any application initiated reads for protected registers) parity on the read data is computed and compared with stored parity value. On detecting a mismatch, an interrupt is generated by setting CSR Parity Error status bit (CPI) of `MAC_DPP_FSM_Interrupt_Status` register. The periodicity of the safety register checks is programmable; set the CTMR field of `MAC_FSM_ACT_timer` register.

When a parity mismatch is detected, CADD fields in `MAC_SCSR_Parity_Status` register logs the offset address of the register where the parity mismatch occurred.

9.6.3 Error Injection Capability

Enable the Error injection capability by setting the CEIN bit in the `MAC_SCSR_Control` register. Depending on the Error injection mode (EIM field of `MTL_DPP_ECC_EIC` register), error can be injected in the read out CSR data or CSR parity bits, calculated during parity generation. The location of error injection is determined by the value of the BLEI field of the `MTL_DPP_ECC_EIC` register. An application read to the error injected register, results in a parity error being reported; that is, CPI bit of the `MAC_DPP_FSM_Interrupt_status` register is set.

Due to the propagation delays in reporting safety errors, there might be up to 100 CSR clock cycles delay in reporting safety errors after the error occurs.

10

Parameter Descriptions

This chapter details all the configuration parameters. **You can use the coreConsultant GUI configuration reports to determine the complete configuration state of the controller.** Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

The parameter descriptions in this chapter include the **Enabled:** attribute which indicates the values required to be set on other parameters before you can change the value of this parameter.

These tables define all of the configuration options for this component.

- "Application Interface Parameters" on page 458
- "General Features Parameters" on page 464
- "General Features / CDC Synchronization Parameters" on page 466
- "Buffer Management Parameters" on page 468
- "Filtering Parameters" on page 470
- "PHY Interface Parameters" on page 473
- "Multiple Queues Support Parameters" on page 475
- "TCP/IP Offloading Features Parameters" on page 479
- "IEEE 1588 Timestamping Parameters" on page 483
- "Time Sensitive Networking Parameters" on page 486
- "RMON Counters Parameters" on page 489
- "RMON Counters / Transmit Counters Parameters" on page 491
- "RMON Counters / Receive Counters Parameters" on page 494
- "RMON Counters / Receive IPC Counters Parameters" on page 498
- "Automotive Safety Features Parameters" on page 503
- "Low Power Management Parameters" on page 506

10.1 Application Interface Parameters

Table 10-1 Application Interface Parameters

Label	Description
Top-Level Configuration	
Application Interface Configuration	Configures the top-level application interface. **Values:** ■ XGMAC-CORE (0) ■ XGMAC-MTL (1) ■ XGMAC-AXI (2) **Default Value:** XGMAC-AXI **Enabled:** Always **Parameter Name:** DWCXG
External CSR Memory Address Width	Indicates Address width of External CSR Memory. **Values:** -2147483648, ..., 2147483647 **Default Value:** (DWCXG_TTL_REG_ROWS<=256) ? 8 : (DWCXG_TTL_REG_ROWS<=512) ? 9 : (DWCXG_TTL_REG_ROWS<=1024) ? 10 : (DWCXG_TTL_REG_ROWS<=2048) ? 11 : 12 **Enabled:** SNPS_RSVDPARAM_50 **Parameter Name:** DWCXG_DCSRAW
External CSR Memory Data Width	Indicates External CSR Memory Datawidth. **Values:** -2147483648, ..., 2147483647 **Default Value:** 128 **Enabled:** SNPS_RSVDPARAM_50 **Parameter Name:** DWCXG_DCSRDW
Application Data Path	
Data Width	Configures the width of the data bus on the system interface. **Values:** 64, 128 **Default Value:** 128 **Enabled:** Always **Parameter Name:** DWCXG_DATAWIDTH

Table 10-1 Application Interface Parameters (continued)

Label	Description
Endian Mode	Configures the system interface to be either little-endian or big-endian mode or to have an input pin (sbd_endianness_i) to configure both modes. The sbd_endianness_i input pin is added when you select BOTH_OPTIONS. **Values:** ■ LITTLE_ENDIAN (0) ■ BIG_ENDIAN (1) ■ BOTH_OPTIONS (2) **Default Value:** LITTLE_ENDIAN **Enabled:** Always **Parameter Name:** DWCXG_ENDIAN_NESS
CSR Port	
CSR Interface	Enables the CSR port to have the MCI, APB3, APB4, AXI4-Lite, or AXI interface as the subordinate interface. **Values:** ■ MCI (0) ■ APB3 (1) ■ AXI (2) ■ AXI4-Lite (3) ■ APB4 (4) **Default Value:** (DWCXG==2) ? 2 : 1 **Enabled:** Always **Parameter Name:** DWCXG_CSR_PORT
Use Different Clock for CSR	Uses a separate clock port instead of the default application clock for the CSR interface. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG!=0 **Parameter Name:** DWCXG_CSR_SLV_CLK
CSR Port Data Width	Configures the width of the data bus on the AXI subordinate port. **Values:** 32, 64, 128 **Default Value:** (DWCXG_CSR_PORT==2) ? DWCXG_DATAWIDTH : 32 **Enabled:** DWCXG_CSR_PORT==2 **Parameter Name:** DWCXG_CSR_DATAWIDTH

Table 10-1 Application Interface Parameters (continued)

Label	Description
AXI Interface Options	
Address Width	Configures the width of the address bus on the AXI Manager interface. **Values:** 32, 40, 48 **Default Value:** 32 **Enabled:** DWCXG>1 **Parameter Name:** DWCXG_ADDRWIDTH
Maximum Outstanding Read Burst Requests for each Transmit DMA Channel	Indicates the maximum number of outstanding Read burst requests for each Transmit DMA channel. **Values:** 2, 4, 8, 16, 32, 64 **Default Value:** 2 **Enabled:** (DWCXG == 2) || SNPS_RSVDPARAM_50 **Parameter Name:** DWCXG_MAX_RDBREQ
Maximum Outstanding Write Burst Requests for each Receive DMA Channel	Indicates the maximum number of outstanding Write burst requests for each Rx DMA channel. **Values:** 2, 4, 8, 16, 32, 64 **Default Value:** 2 **Enabled:** (DWCXG == 2) || SNPS_RSVDPARAM_50 **Parameter Name:** DWCXG_MAX_WRBREQ
Enable Unique AXI ID for Descriptor Access	Enables the logic for assigning Unique AXI IDs for Descriptor Access based on Tx/Rx channel number **Values:** 0, 1 **Default Value:** [<functionof> (SNPS_RSVDPARAM_50 ? 1 : 0)] **Enabled:** DWCXG == 2 **Parameter Name:** DWCXG_UAXIID
AXI Manager Bus ID Width	Sets the AXI Manager bus ID width. **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 **Default Value:** [<functionof> (SNPS_RSVDPARAM_50 ? (DWCXG_NUM_CSR_PDMA>15?5+(DWCXG_UAXIID?2:0):4+(DWCXG_UAXIID?2:0)) : DWCXG_EDMA ? (DWCXG_NUM_DMA_CSR_CH>15 ? 5+(DWCXG_UAXIID ? 2 : 0) : 4+(DWCXG_UAXIID ? 2 : 0)) : (DWCXG_NUM_DMA_CSR_CH>8 ? 5 : 4))] **Enabled:** DWCXG == 2 **Parameter Name:** DWCXG_AXI_GM_AXI_ID_WIDTH

Table 10-1 Application Interface Parameters (continued)

Label	Description
AXI Subordinate Bus ID Width	Sets the AXI subordinate bus ID width. **Values:** 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 **Default Value:** DWCXG_AXI4_LITE_SLAVE) ? 2 : (DWCXG_AXI_GM_AXI_ID_WIDTH < 8) ? 8 : (DWCXG_AXI_GM_AXI_ID_WIDTH+1 **Enabled:** DWCXG_CSR_PORT==2 **Parameter Name:** DWCXG_GS_ID
AXI Maximum Outstanding Read Requests	Indicates the maximum number of outstanding Read requests on the AXI interface. **Values:** 8, 16, 32, 64, 128, 256 **Default Value:** SNPS_RSVDPARAM_50 ? (DWCXG_NUM_CSR_PDMA<4 ? 8 : DWCXG_NUM_CSR_PDMA<8 ? 16 :DWCXG_NUM_CSR_PDMA<16 ? 32 : 64) : DWCXG_EDMA ? (DWCXG_NUM_DMA_CSR_CH<4 ? 8 : DWCXG_NUM_DMA_CSR_CH<8 ? 16 : DWCXG_NUM_DMA_CSR_CH<16 ? 32 : 64) : (DWCXG_NUM_DMA_CSR_CH<=4 ? 8 : DWCXG_NUM_DMA_CSR_CH<=8 ? 16 : DWCXG_NUM_DMA_CSR_CH<=16 ? 32 : 64) **Enabled:** DWCXG == 2 **Parameter Name:** DWCXG_AXI_GM_MAX_RD_REQUESTS
AXI Maximum Outstanding Write Requests	Indicates the maximum number of outstanding Write requests on the AXI interface. **Values:** 8, 16, 32, 64, 128, 256 **Default Value:** SNPS_RSVDPARAM_50 ? (DWCXG_NUM_CSR_PDMA<4 ? 8 : DWCXG_NUM_CSR_PDMA<8 ? 16 : DWCXG_NUM_CSR_PDMA<16 ? 32 : 64) : DWCXG_EDMA ? (DWCXG_NUM_DMA_CSR_CH<4 ? 8 : DWCXG_NUM_DMA_CSR_CH<8 ? 16 : DWCXG_NUM_DMA_CSR_CH<16 ? 32 : 64) : (DWCXG_NUM_DMA_CSR_CH<=4 ? 8 : DWCXG_NUM_DMA_CSR_CH<=8 ? 16 : DWCXG_NUM_DMA_CSR_CH<=16 ? 32 : 64) **Enabled:** DWCXG == 2 **Parameter Name:** DWCXG_AXI_GM_MAX_WR_REQUESTS
Sideband Signals	
Add DMA Start and Stop as Primary inputs	Enables DMA Starting and Stopping through Side-Band input signals. **Values:** 0, 1 **Default Value:** 0 **Enabled:** ((!DWCXG_CORE)&&(!DWCXG_MTL_SUBSYS)&&DWCXG_EDMA) **Parameter Name:** DWCXG_DMA_STRTSTP
Add Rx Flow Control Enable as Primary Input	Enables side-band Rx flow control input from the application. **Values:** 0, 1 **Default Value:** DWCXG_PFC_EN&&DWCXG_NUM_RXQ==1 **Enabled:** DWCXG!=0 **Parameter Name:** DWCXG_SBD_FC_EN

Table 10-1 Application Interface Parameters (continued)

Label	Description
Add MAC Tx/Rx Disable as Primary Input	Enables the top-level inputs for MAC Tx/Rx disable. **Values:** 0, 1 **Default Value:** 0 **Enabled:** (DWCXG==0)||(DWCXG==1) **Parameter Name:** DWCXG_ADD_TXRX_DIS_IO
Add General Purpose IO	Adds the side-band general purpose IO signals. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_GPIO_EN
GP Input Signal Width	Sets the width of the general purpose input bus. **Values:** 0, 4, 8, 12, 16 **Default Value:** DWCXG_GPIO_EN ? 4 : 0 **Enabled:** DWCXG_GPIO_EN **Parameter Name:** DWCXG_GIW
GP Output Signal Width	Sets the width of the general purpose output bus. **Values:** 0, 4, 8, 12, 16 **Default Value:** DWCXG_GPIO_EN ? 4 : 0 **Enabled:** DWCXG_GPIO_EN **Parameter Name:** DWCXG_GOW
Add Speed Mode/EEE State/MAC_Address0 Register Controls as Primary Input	External IO control interface for changing the speed mode/EEE state/MAC_Address0 register. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_ADD_SBD_IO
Per Channel Transmit Descriptor Cache Size	Specifies Per Tx channel descriptor cache size in number of normal descriptors **Values:** 16, 32, 64, 128 **Default Value:** [<functionof> (DWCXG_MAX_RDBREQ<=8 ? 16 : DWCXG_MAX_RDBREQ*2)] **Enabled:** (DWCXG == 2) || SNPS_RSVDPARAM_50 **Parameter Name:** DWCXG_PCHTDC_SZ

Table 10-1 Application Interface Parameters (continued)

Label	Description
Per Channel Receive Descriptor Cache Size	Specifies Per Rx channel descriptor cache size in number of normal descriptors **Values:** 16, 32, 64, 128 **Default Value:** [<functionof> (DWCXG_MAX_WRBREQ<=8 ? 16 : DWCXG_MAX_WRBREQ*2)] **Enabled:** (DWCXG == 2) || SNPS_RSVDPARAM_50 **Parameter Name:** DWCXG_PCHRDC_SZ
Descriptor Cache Memory Size in Bytes	Specifies the size of the separate memory of pre-fetched descriptors. **Values:** 512, ..., 524288 **Default Value:** [<functionof> (DWCXG_NUM_DMA_TX_CH* DWCXG_PCHTDC_SZ * 16 + DWCXG_NUM_DMA_RX_CH* DWCXG_PCHRDC_SZ * 16)] **Enabled:** DWCXG_CACHE_MEM_EN&&DWCXG_EDMA&&DWCXG_TSO_MEM_EN&&DWCXG_DIFF_CACHE_MEM_EN **Parameter Name:** DWCXG_DCACHE_MEM_SIZE
TSO (and Descriptor Cache) Memory Size in Bytes	Specifies the size of the separate memory and/or pre-fetched descriptors. **Values:** 0, ..., 1064960 **Default Value:** [<functionof> DWCXG_EDMA&&DWCXG_TSO_MEM_EN ? (DWCXG_DIFF_CACHE_MEM_EN ? DWCXG_TOT_TSO_MSIZE : (DWCXG_DCACHE_MEM_SIZE + DWCXG_TOT_TSO_MSIZE)) : DWCXG_EDMA ? DWCXG_DCACHE_MEM_SIZE : DWCXG_TSO_MEM_EN ? DWCXG_TOT_TSO_MSIZE : 0] **Enabled:** DWCXG_TSO_MEM_EN||DWCXG_EDMA **Parameter Name:** DWCXG_CACHE_MEM_SIZE

10.2 General Features Parameters

Table 10-2 General Features Parameters

Label	Description
Reset Mode	
Enable Asynchronous or Synchronous Resets for All Flops	Specifies the asynchronous or synchronous reset for all flip-flops. **Values:** ■ Synchronous Reset (0) ■ Asynchronous Reset (1) **Default Value:** Asynchronous Reset **Enabled:** Always **Parameter Name:** DWCXG_RESET_MODE
Duplex Mode	
Enables Full Duplex Only Mode	Enables Full Duplex Only Mode. When GMII is selected and this feature is deselected in configuration, Half-Duplex mode can be enabled through programming when operating in 10/100M speed modes. **Values:** 0, 1 **Default Value:** 1 **Enabled:** DWCXG_XGMII_GMII==1 **Parameter Name:** DWCXG_FDUPLX_ONLY
USER DEFINED VERSION	
User-Defined Controller Version	Specifies the user-defined version# inserted in the MAC_Version register (for example, 8'h10 for version 1.0). **Values:** 0x10, ..., 0xff **Default Value:** 0x10 **Enabled:** Always **Parameter Name:** DWCXG_USER_VER
Double VLAN	
Enable Double VLAN Processing	Enables the logic to process up to two VLAN tags. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_DOUBLE_VLAN_EN

Table 10-2 General Features Parameters (continued)

Label	Description
SA-VLAN Insertion	
Enable Queue/Channel based VLAN tag insertion on Tx	Enables Queue/Channel based VLAN tag insertion. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_NUM_TXQ>1 **Parameter Name:** DWCXG_CBTI_EN
Enable SA and VLAN Insertion on Tx	Enables the logic for the following features for Transmit packets: ■ Source address insertion or replacement ■ VLAN insertion, replacement, or deletion **Values:** 0, 1 **Default Value:** DWCXG_CBTI_EN **Enabled:** !DWCXG_CBTI_EN **Parameter Name:** DWCXG_SA_VLAN_INS_CTRL_EN
Transmit Status	
Enable Transmit Status Interface in XGMAC-MTL	Enables transmit status forwarding in the MTL. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG==1 **Parameter Name:** DWCXG_TXSTSE
ITU-T G.999.1 Function	
Enable ITU-T G.999.1 Function	Enables the support for processing Ethernet Encapsulated ITU-T G.999.1 packets. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_G9991_EN

10.3 General Features / CDC Synchronization Parameters

Table 10-3 General Features / CDC Synchronization Parameters

Label	Description
CDC Synchronization Configuration	
Number of Stages in CDC Synchronizer	Configures the number of stages in the CDC synchronizer **Values:** ■ TWO_STAGE (2) ■ THREE_STAGE (3) ■ FOUR_STAGE (4) **Default Value:** TWO_STAGE **Enabled:** Always **Parameter Name:** DWCXG_F_SYNC_TYPE
CDC Synchronization Selection	
Synchronize CSR MAC Address to Tx/Rx Clock Domain	Enables the synchronization of MAC address registers to the Transmit or Receive clock domain. Otherwise, the CSR MAC Address Register value that is written with the application clock is directly used in the Transmit or Receive clock domain. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_SYNC_MACADDR
Synchronize Pause Time to Tx Clock Domain	Enables the synchronization of the Pause Timer register to the Transmit clock domain. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_SYNC_PAUSETIME
Synchronize VLAN Tag Register to Rx/Tx Clock Domain	Enables the synchronization of the VLAN tag register to the Receive and Transmit clock domain. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_SYNC_VLANTAG
Synchronize VLAN Hash Table to Rx Clock Domain	Enables the synchronization of the VLAN Hash Table register to the Receive clock domain. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_VLAN_HASH_EN **Parameter Name:** DWCXG_SYNC_VLAN_HASHTABLE

Table 10-3 General Features / CDC Synchronization Parameters (continued)

Label	Description
Synchronize Hash Table to Rx Clock Domain	Enables the synchronization of the Hash Table register to the Receive clock domain. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_DA_HASH_EN&&(DWCXG_HASH_TABLE>0)&&(DWCXG_HASH_TABLE<=256) **Parameter Name:** DWCXG_SYNC_HASHTABLE
Synchronize PMT Remote Wake-Up Packet Filter Registers to Rx Clock Domain	Enables the synchronization of the PMT Remote Wake-Up packet filter registers to the Receive clock domain. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_PMT_RWK_EN==1 **Parameter Name:** DWCXG_SYNC_PMT_RWK_FILT
Synchronize ARP Protocol Address Register to Rx and Tx Clock Domains	Enables the synchronization of the ARP Protocol Address Register to the receive and transmit clock domain. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_ARP_EN **Parameter Name:** DWCXG_SYNC_ARP_PROT_ADDR

10.4 Buffer Management Parameters

Table 10-4 Buffer Management Parameters

Label	Description
Receive FIFO Depth	
MTL Receive FIFO Size in bytes	Specifies the size of the MTL Receive FIFO. **Values:** 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144 **Default Value:** 4096 **Enabled:** DWCXG != 0 **Parameter Name:** DWCXG_RXFIFO_SIZE
MTL Rx FIFO Address Bus Width	Specifies the address bus width for the selected Receive FIFO depth. **Values:** 3, ..., 16 **Default Value:** [<functionof> (log(DWCXG_RXFIFO_SIZE)/log(2)) - ((DWCXG_DATAWIDTH/64)+2)] **Enabled:** DWCXG>0 **Parameter Name:** DWCXG_RX_FIFO_PTR_WIDTH
MTL Rx FIFO Data Width	Specifies the data bus width to MTL Receive FIFO DPRAM including overhead bits. **Values:** 68, ..., 263 **Default Value:** [<functionof> (DWCXG_DATAWIDTH/64)+3+DWCXG_DATAWIDTH] **Enabled:** DWCXG != 0 **Parameter Name:** DWCXG_RXFIFO_DATA_WIDTH
Transmit FIFO Depth	
MTL Transmit FIFO Size in bytes	Specifies the size of the MTL Transmit FIFO. **Values:** 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144 **Default Value:** 4096 **Enabled:** DWCXG != 0 **Parameter Name:** DWCXG_TXFIFO_SIZE
Maximum Number of Packets in the Transmit FIFO	Specifies the maximum number of packets that can be pushed into the Transmit FIFO at a time without any transmit status being read out on the ATI interface. **Values:** 2, 4, 8 **Default Value:** 2 **Enabled:** DWCXG == 1 && DWCXG_TXSTSE **Parameter Name:** DWCXG_MAX_FRAME_IN_TXFIFO

Table 10-4 Buffer Management Parameters (continued)

Label	Description
MTL Transmit FIFO Address Bus Width	Specifies the address bus width for selected Transmit FIFO depth. **Values:** 4, ..., 15 **Default Value:** [<functionof> (log(DWCXG_TXFIFO_SIZE)/log(2)) - ((DWCXG_DATAWIDTH/64)+2)] **Enabled:** DWCXG != 0 **Parameter Name:** DWCXG_TX_FIFO_PTR_WIDTH
MTL Transmit FIFO Data Width	Specifies the data bus width to MTL Transmit FIFO DPRAM including overhead bits. **Values:** 68, ..., 263 **Default Value:** [<functionof> (DWCXG_DATAWIDTH/64)+3+DWCXG_DATAWIDTH] **Enabled:** DWCXG != 0 **Parameter Name:** DWCXG_TXFIFO_DATA_WIDTH
Memory Type	
MTL Tx and Rx Memory Type	Select RAM interfaces: ASYNC DPRAM, SYNC DPRAM, and SPRAM. **Values:** ■ ASYNC_DPRAM (0) ■ SYNC_DPRAM (1) ■ SPRAM (2) **Default Value:** ASYNC_DPRAM **Enabled:** DWCXG!=0 **Parameter Name:** DWCXG_MTL_MEMTYP
Debug Mode Enable	
Enable Debug Memory Access	Enables read, write, and debug access to the Transmit, Receive, and TSO memories through the CSR interface. **Values:** 0, 1 **Default Value:** DWCXG_ASP_ECC **Enabled:** (DWCXG>0)&&!DWCXG_ASP_ECC **Parameter Name:** DWCXG_DBG_FIFO_ACCESS

10.5 Filtering Parameters

Table 10-5 Filtering Parameters

Label	Description
Additional MAC Address Registers	
Enable Additional 1-63 MAC Address Registers	Enables MAC address registers 1-63 for the address filter block. **Values:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 39, 47, 63 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_ADD_MAC_ADDR_REG
DA Hash Filter	
Enable Destination Address Hash Table Filter	Enables the DA Hash Table register and related filter logic. **Values:** 0, 1 **Default Value:** 0 **Enabled:** !(SNPS_RSVDPARAM_49||SNPS_RSVDPARAM_38) **Parameter Name:** DWCXG_DA_HASH_EN
Destination Address Hash Table Size	Specifies the size (width for internal table or number of entries for external table) of the Destination Address Hash Table for its filtering and routing logic. Up to 256 size selection, IP implements internal register-based table. For more than 256 size selection, IP implements external RXF memory-based table. **Values:** 64, 128, 256, 512, 1024, 2048, 4096 **Default Value:** 64 **Enabled:** DWCXG_DA_HASH_EN **Parameter Name:** DWCXG_HASH_TABLE
VLAN Filter	
Enable Address Filter VLAN Hash Table	Enables the VLAN Hash Table register and related filter logic. **Values:** 0, 1 **Default Value:** 0 **Enabled:** !SNPS_RSVDPARAM_49 **Parameter Name:** DWCXG_VLAN_HASH_EN
Extended Rx VLAN Filter Enable	Enables the VLAN tags based Filtering and Routing logic. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_ERVFE

Table 10-5 Filtering Parameters (continued)

Label	Description
Number of VLAN Tag Filters	Specifies the number of VLAN tag based filters. When 4096 option is selected, the VLAN Tag filters are implemented in External Receive Filter (RXF) Memory. **Values:** 1, 4, 8, 16, 24, 32, 40, 48, 64, 4096 **Default Value:** (DWCXG_ERVFE==1) ? 4 : 1 **Enabled:** Always **Parameter Name:** DWCXG_NRVF
IP Filter	
Enable Layer 3 and Layer 4 Packet Filter	Enables Layer 3 and Layer 4 based frame filtering for Receive packets. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_L3_L4_FILTER_EN
Number of Layer 3 and Layer 4 Packet Filters	Specifies the number of additional Layer 3 and Layer 4 packet filters. **Values:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 16, 24, 32, 40, 48, 64 **Default Value:** (DWCXG_L3_L4_FILTER_EN==1) ? 1 : 0 **Enabled:** DWCXG_L3_L4_FILTER_EN==1 **Parameter Name:** DWCXG_L3_L4_FILTER_NUM
Flexible Receive Parser	
Enable Flexible Receive Parser	Selects the Flexible Receive Parser logic to filter or forward received packets to specific Receive DMA channel. **Values:** 0, 1 **Default Value:** 0 **Enabled:** (DWCXG>1)&&(DWCXG_NUM_DMA_RX_CH>1) **Parameter Name:** DWCXG_FRP_EN
Maximum FRP Parsable Bytes from Start of Packet	Decides the maximum number of bytes from the Start of Packet that is parsed by the Parser. **Values:** 64, 128, 256 **Default Value:** 256 **Enabled:** DWCXG_FRP_EN **Parameter Name:** DWCXG_FRP_PB
Maximum Entries of Parser Instruction Table	Specifies the maximum number of Parser Instructions. **Values:** 64, 128, 256, 512 **Default Value:** 256 **Enabled:** DWCXG_FRP_EN **Parameter Name:** DWCXG_FRP_ENTRIES

Table 10-5 Filtering Parameters (continued)

Label	Description
Number of Parallel Instruction Processor Engines	Specifies the number of Parallel Instruction Processor Engines to implement inside FRP processor. **Values:** 2, 3, 4, 5, 6 **Default Value:** 6 **Enabled:** DWCXG_FRP_EN **Parameter Name:** DWCXG_FRP_PIPE
Receive Filter Memory	
Enable MAC Receive Filter (RXF) Memory	Enable External Memory for MAC Receive Filters. This is automatically selected when any or all of following features are selected: DA Hash Table Size greater than 256, Number of VLAN Tag Filters equal to 4096, RSS Lookup Depth greater than 256. **Values:** 0, 1 **Default Value:** DWCXG_EXT_DAH_EN||DWCXG_EXT_RVTL_EN||DWCXG_EXT_RSSL_EN **Enabled:** DWCXG_EXT_DAH_EN||DWCXG_EXT_RVTL_EN||DWCXG_EXT_RSSL_EN **Parameter Name:** DWCXG_RXF_MEM_EN
MAC RXF Memory address width	Specifies the address width for MAC RXF memory **Values:** 0, ..., 14 **Default Value:** (DWCXG_RXF_MEM_ENT>0) ? [<functionof> ceil(log(DWCXG_RXF_MEM_ENT)/log(2))] : 0 **Enabled:** DWCXG_RXF_MEM_EN **Parameter Name:** DWCXG_RXF_MEM_AW
MAC RXF Memory data width	Specifies the data width for MAC RXF Memory. Note: It must be less than or equal to 32bits. **Values:** 0, ..., 10 **Default Value:** [<functionof> max(DWCXG_RXF_DAH_DW, DWCXG_RXF_RVTL_DW, DWCXG_RXF_RSSL_DW)] **Enabled:** DWCXG_RXF_MEM_EN **Parameter Name:** DWCXG_RXF_MEM_DW
MAC RXF Memory ECC width	Specifies the Error-Correcting Code width for MAC RXF Memory. **Values:** 0, ..., 9 **Default Value:** (DWCXG_RXF_MEM_EDW<=26) ? 6 : ((DWCXG_RXF_MEM_EDW>26) & (DWCXG_RXF_MEM_EDW<=57)) ? 7 : ((DWCXG_RXF_MEM_EDW>57) & (DWCXG_RXF_MEM_EDW<=120)) ? 8 : ((DWCXG_RXF_MEM_EDW>120) & (DWCXG_RXF_MEM_EDW<=247)) ? 9 : 0 **Enabled:** DWCXG_RXF_MEM_EN&&DWCXG_ASP_ECC **Parameter Name:** DWCXG_RXF_MEM_EW

10.6 PHY Interface Parameters

Table 10-6 PHY Interface Parameters

Label	Description
Line Interface	
PHY Interface Configuration	Select PHY interfaces: XGMII or XGMII + GMII. XGMII is used for 10G and USXGMII 2.5G/5G, GMII is used for 1G/2.5G (over-clocked 1G) and GMII (4-bit) for 10M/100M. **Values:** ■ XGMII-ONLY (0) ■ XGMII-GMII (1) **Default Value:** XGMII-ONLY **Enabled:** !SNPS_RSVDPARAM_49 **Parameter Name:** DWCXG_XGMII_GMII
Add GMII Port	This option allows separate GMII Interface. When this feature is not selected, GMII/MII is shared with XGMII interface (lower 8-bit/4-bit of TXD/RXD for Data and lower 2-bit of TXC/RXC for Controls). **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_XGMII_GMII == 1 **Parameter Name:** DWCXG_ADD_GMII_PORT
Enable RGMII	This option allows RGMII support on separate PHY interface (Shared with RMII, if selected). The interface is Dual clock-edged and operates in 10M/100M/1G/2.5G (overclocked 1G). **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_ADD_GMII_PORT **Parameter Name:** DWCXG_RGMII_EN
Enable RMII	This option allows RMII support on separate PHY interface (Shared with RGMII, if selected). The interface operates in 10M/100M. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_ADD_GMII_PORT **Parameter Name:** DWCXG_RMII_EN

Table 10-6 PHY Interface Parameters (continued)

Label	Description
XGMII Interface Configuration	Specifies the XGMII interface type (XGMII-DDR selection is deprecated). The XGMII-SDR and XGMII-DDW interface is 32-bit and 64-bit, respectively. **Values:** ■ XGMII-SDR (0) ■ XGMII-DDR (1) ■ XGMII-DDW (2) **Default Value:** XGMII-SDR **Enabled:** Always **Parameter Name:** DWCXG_XGMII
Add Handshake/Backpressure Function in XGMII Interface	This option adds the handshake/backpressure function in the XGMII Interface. This Handshake interface is not valid when operating in GMII/RGMII/RMII PHY interface mode. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_XGMII==0 **Parameter Name:** DWCXG_XGMII_SDR_BCKPRES
VARIABLE PREAMBLE	
Receive Frames with Variable Preamble	This option allows frames with variable preamble to be received. Otherwise, they are dropped. **Values:** 0, 1 **Default Value:** 0 **Enabled:** !SNPS_RSVDPARAM_49&&!(DWCXG!=0&&DWCXG_DATAWIDTH_64) **Parameter Name:** DWCXG_VARIABLE_PREAMBLE
SMA	
Enable Station Management Block (MDIO Interface)	Enables the Station Management Agent block for communicating with or configuring the PHY through MDIO interface. **Values:** 0, 1 **Default Value:** 1 **Enabled:** Always **Parameter Name:** DWCXG_SMA_EN

10.7 Multiple Queues Support Parameters

Table 10-7 Multiple Queues Support Parameters

Label	Description
Number of Traffic Classes	Specifies the number of traffic classes. **Values:** 1, 2, 3, 4, 5, 6, 7, 8 **Default Value:** ((DWCXG_DCB_EN||DWCXG_AV_ENABLE)&&DWCXG>0) ? 2 : 1 **Enabled:** DWCXG>0 **Parameter Name:** DWCXG_NUM_TC
Number of Transmit Queues	Specifies the number of transmit queues. **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 **Default Value:** DWCXG>0 ? DWCXG_NUM_TC : ((DWCXG_DCB_EN||DWCXG_AV_ENABLE) ? 2 : 1) **Enabled:** Always **Parameter Name:** DWCXG_NUM_TXQ
Number of Receive Queues	Specifies the number of receive queues. **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 **Default Value:** DWCXG_DCB_EN ? 2 : 1 **Enabled:** Always **Parameter Name:** DWCXG_NUM_RXQ
Number of DMA channels on Receive side	Specifies the number of DMA channels on the Receive side. **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 20, 24, 28, 32, 40, 48, 56, 64, 80, 96, 112, 128 **Default Value:** 1 **Enabled:** DWCXG>1 **Parameter Name:** DWCXG_NUM_DMA_RX_CH
Number of DMA channels on Transmit side.	Specifies the number of DMA channels on the Transmit side. This is automatically set equal to Number of Transmit Queues selected. **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 20, 24, 28, 32, 40, 48, 56, 64, 80, 96, 112, 128 **Default Value:** (DWCXG>1) ? DWCXG_NUM_TXQ : 1 **Enabled:** Always **Parameter Name:** DWCXG_NUM_DMA_TX_CH

Table 10-7 Multiple Queues Support Parameters (continued)

Label	Description
Select Number of Entries in Receive Channel List for Packet Duplication	Specifies the Number of Entries in Receive Channel List for Packet Duplication. The selection of this feature automatically selects packet duplication capability in the highest numbered Receive Queue. **Values:** 0, 8, 16, 32, 64 **Default Value:** ((DWCXG_NUM_DMA_RX_CH>1)&&SNPS_RSVDPARAM_50)?8:0 **Enabled:** (DWCXG>1)&&(DWCXG_NUM_DMA_RX_CH>1) **Parameter Name:** DWCXG_RCHLIST_CNT
Enable Data Center Bridging Schedulers	Enables the Scheduler Algorithms Specified in Data Centre Bridging Standard. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_DCB_EN
Enable Priority Flow Control	Enables the Priority Flow Control feature. **Values:** 0, 1 **Default Value:** DWCXG_DCB_EN **Enabled:** Always **Parameter Name:** DWCXG_PFC_EN
Enables Audio Video Bridging Feature.	Adds logic for supporting the Audio Video functionality and provides the option to select additional channels for queueing the time-sensitive traffic. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_AV_ENABLE
Enable Support for AV in TC1	Enables AV Support in TC1. **Values:** 0, 1 **Default Value:** DWCXG_AV_ENABLE&&DWCXG_NUM_TC==2 **Enabled:** DWCXG_AV_ENABLE&&(DWCXG_NUM_TC>2||DWCXG_NUM_TC==2) **Parameter Name:** DWCXG_TC1_AV_EN

Table 10-7 Multiple Queues Support Parameters (continued)

Label	Description
Enable Support for AV in TC2	Enables AV Support in TC2. **Values:** 0, 1 **Default Value:** DWCXG_AV_ENABLE&&((DWCXG_NUM_TC>2&&(DWCXG_TC1_AV_EN))||DWCXG_NUM_TC==3) **Enabled:** DWCXG_AV_ENABLE&&(!DWCXG_TC1_AV_EN&&(DWCXG_NUM_TC>3||DWCXG_NUM_TC==3)) **Parameter Name:** DWCXG_TC2_AV_EN
Enable Support for AV in TC3	Enables AV Support in TC3. **Values:** 0, 1 **Default Value:** DWCXG_AV_ENABLE&&((DWCXG_NUM_TC>3&&(DWCXG_TC2_AV_EN))||DWCXG_NUM_TC==4) **Enabled:** DWCXG_AV_ENABLE&&(!DWCXG_TC2_AV_EN&&(DWCXG_NUM_TC>4||DWCXG_NUM_TC==4)) **Parameter Name:** DWCXG_TC3_AV_EN
Enable Support for AV in TC4	Enables AV Support in TC4. **Values:** 0, 1 **Default Value:** DWCXG_AV_ENABLE&&((DWCXG_NUM_TC>4&&(DWCXG_TC3_AV_EN))||DWCXG_NUM_TC==5) **Enabled:** DWCXG_AV_ENABLE&&(!DWCXG_TC3_AV_EN&&(DWCXG_NUM_TC>5||DWCXG_NUM_TC==5)) **Parameter Name:** DWCXG_TC4_AV_EN
Enable Support for AV in TC5	Enables AV Support in TC5. **Values:** 0, 1 **Default Value:** DWCXG_AV_ENABLE&&((DWCXG_NUM_TC>5&&(DWCXG_TC4_AV_EN))||DWCXG_NUM_TC==6) **Enabled:** DWCXG_AV_ENABLE&&(!DWCXG_TC4_AV_EN&&(DWCXG_NUM_TC>6||DWCXG_NUM_TC==6)) **Parameter Name:** DWCXG_TC5_AV_EN

Table 10-7 Multiple Queues Support Parameters (continued)

Label	Description
Enable Support for AV in TC6	Enables AV Support in TC6. **Values:** 0, 1 **Default Value:** DWCXG_AV_ENABLE&&((DWCXG_NUM_TC>6&&(DWCXG_TC5_AV_EN))||DWCXG_NUM_TC==7) **Enabled:** DWCXG_AV_ENABLE&&(!DWCXG_TC5_AV_EN&&(DWCXG_NUM_TC>7||DWCXG_NUM_TC==7)) **Parameter Name:** DWCXG_TC6_AV_EN
Enable Support for AV in TC7	Enables AV Support in TC7. **Values:** 0, 1 **Default Value:** DWCXG_AV_ENABLE&&((DWCXG_NUM_TC>7&&(DWCXG_TC6_AV_EN))||DWCXG_NUM_TC==8) **Enabled:** DWCXG_AV_ENABLE&&(!DWCXG_TC6_AV_EN&&DWCXG_NUM_TC==8) **Parameter Name:** DWCXG_TC7_AV_EN
Enable External Lookup Interface	Enables the interface with an external lookup engine, which provides the RxDMA channel selection for the received packet. **Values:** 0, 1 **Default Value:** 0 **Enabled:** (DWCXG>1)&&(DWCXG_NUM_DMA_RX_CH>1)&&(DWCXG_NUM_DMA_RX_CH<=16) **Parameter Name:** DWCXG_EXT_LKUP_EN

10.8 TCP/IP Offloading Features Parameters

Table 10-8 TCP/IP Offloading Features Parameters

Label	Description
TCP Segmentation Offload	
Enable TCP Segmentation Offloading for TCP/IP Packets	Enables TCP Segmentation Offload (TSO) on the transmit side. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG == 2 **Parameter Name:** DWCXG_TSO_EN
Number of Transmit DMA Channels Enabled for TSO	Specifies the number of DMA channels enabled for TSO. **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 **Default Value:** 1 **Enabled:** DWCXG_TSO_EN&&DWCXG_NUM_TXQ>1 **Parameter Name:** DWCXG_NUM_DMA_TSO_CH
Enable Memory for TCP/IP Headers	Enables the use of a separate memory for TCP/IP headers used in the TCP Segmentation Offloading. Otherwise, during segmentation, the DMA reads the header data multiple times from the system memory. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_TSO_EN **Parameter Name:** DWCXG_TSO_MEM_EN
Per-Channel TSO Memory Size in Bytes	Specifies the size of the external memory (in bytes) for each channel that is used for storing the TCP/IP headers in TCP Segmentation Offloading. **Values:** 64, 128, 256, 512, 1024 **Default Value:** 256 **Enabled:** DWCXG_TSO_MEM_EN **Parameter Name:** DWCXG_TSO_MEM_SIZE
Descriptor Cache & TSO Memory Datawidth	Specifies the data bus width of the TSO Memory. **Values:** 64, ..., 129 **Default Value:** DWCXG_ASP_ALL&&SNPS_RSVDPARAM_33 ? [<functionof> (DWCXG_DATAWIDTH + 1)] : DWCXG_DATAWIDTH **Enabled:** Always **Parameter Name:** DWCXG_TSOMEM_DW

Table 10-8 TCP/IP Offloading Features Parameters (continued)

Label	Description
TSO (and Descriptor Cache) Memory Address Width	Address bus width of the separate memory for TCP/IP headers or pre-fetched descriptors. **Values:** 0, ..., 18 **Default Value:** [<functionof> (DWCXG_CACHE_MEM_SIZE_POW2 == 0 ? 0 : log(DWCXG_CACHE_MEM_SIZE_POW2)/log(2) - DWCXG_BEW)] **Enabled:** DWCXG_TSO_MEM_EN||DWCXG_EDMA **Parameter Name:** DWCXG_CACHE_MEM_ADDR_WIDTH
Cache Memory Width for Descriptors	Specifies the data bus width of the Descriptor Cache memory. **Values:** 64, ..., 129 **Default Value:** DWCXG_ASP_ALL&&SNPS_RSVDPARAM_33 ? [<functionof> (DWCXG_DATAWIDTH + 1)] : DWCXG_DATAWIDTH **Enabled:** Always **Parameter Name:** DWCXG_DCACHE_DW
DMA Descriptor Cache Memory	
Enable Separate Descriptor Cache Memory	Enables different memory for Descriptor Cache **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_EDMA&&DWCXG_TSO_MEM_EN **Parameter Name:** DWCXG_DIFF_CACHE_MEM_EN
Descriptor Cache Memory Address Width	Address bus width of the pre-fetched descriptors memory. **Values:** 0, ..., 17 **Default Value:** [<functionof> (DWCXG_DCACHE_MEM_SIZE_POW2 == 0 ? 0 : log(DWCXG_DCACHE_MEM_SIZE_POW2)/log(2) - DWCXG_BEW)] **Enabled:** DWCXG_CACHE_MEM_EN&&DWCXG_EDMA&&((DWCXG_TSO_MEM_EN&&DWCXG_DIFF_CACHE_MEM_EN)||!DWCXG_TSO_MEM_EN) **Parameter Name:** DWCXG_DCACHE_MEM_ADDR_WIDTH

Table 10-8 TCP/IP Offloading Features Parameters (continued)

Label	Description
Checksum Offload Engine	
Enable Receive TCP/IP Checksum Check	Enables the Checksum Offload Engine (COE) in the Receive path. The COE checks and detects the IPv4 header checksum errors and the TCP, UDP, or ICMP checksum errors encapsulated in IPv4 or IPv6 datagrams payload of Received packets. **Values:** 0, 1 **Default Value:** DWCXG_VXN_ENA||DWCXG_TIME_STAMPING||DWCXG_L3_L4_FILTER_EN||DWCXG_SPLIT_HDR_EN||DWCXG_RSS_EN||DWCXG_OOS_EN **Enabled:** !DWCXG_VXN_ENA&&!DWCXG_TIME_STAMPING&&!DWCXG_L3_L4_FILTER_EN&&!DWCXG_SPLIT_HDR_EN&&!DWCXG_RSS_EN&&!DWCXG_OOS_EN **Parameter Name:** DWCXG_RX_COE
Enable Transmit TCP/IP Checksum Offload	Enables the Checksum Offload Engine in the transmit path. In this mode, the DWC_25gmac can calculate and insert the checksums for TCP, UDP, or ICMP segments encapsulated in IPv4 or IPv6 datagrams. The DWC_25gmac can also insert IPv4 header checksums in the transmitted frames. **Values:** 0, 1 **Default Value:** DWCXG>0&&(DWCXG_TSO_EN||DWCXG_VXN_ENA||DWCXG_POU_OST_EN) **Enabled:** DWCXG>0&&!DWCXG_TSO_EN&&!DWCXG_VXN_ENA&&!DWCXG_POU_OST_EN **Parameter Name:** DWCXG_TX_COE
Number of Tx Queues Enabled with Tx COE	Specifies the number of Tx Queues that support TxCOE. **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 **Default Value:** DWCXG_NUM_DMA_TSO_CH **Enabled:** DWCXG_TX_COE&&DWCXG_NUM_TXQ>1 **Parameter Name:** DWCXG_NUM_TX_COE
IPv4 ARP Offload	
Enable IPv4 ARP Offload	Enables the IPv4 Address Recognition Protocol Offload feature. In this mode, the DWC_25gmac can process the ARP request in the receive path and generate a corresponding ARP response packet in the transmit path. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_ARP_EN

Table 10-8 TCP/IP Offloading Features Parameters (continued)

Label	Description
Header-Payload Split	
Enable Header-Payload Split Feature	Enables the logic to split the packet header (Layer 2, Layer 3, and Layer 4) in a different buffer and the payload in a different buffer. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG == 2 **Parameter Name:** DWCXG_SPLIT_HDR_EN
RSS	
Enable Receive Side Scaling	Enables the RSS function, DMA channel, and packet drop indication in the controller. **Values:** 0, 1 **Default Value:** DWCXG_VXN_ENA&&DWCXG_NUM_DMA_RX_CH>1 **Enabled:** DWCXG_AXI_SUBSYS&&!DWCXG_VXN_ENA **Parameter Name:** DWCXG_RSS_EN
RSS Lookup Table Size	Specifies the size of the Lookup table. The width of the table depends on the number of Rx DMA channels. When the option selected is greater than 256, the Lookup table shall be implemented in External Receive Filter Memory (RXF). **Values:** 0, 16, 32, 64, 128, 256, 512, 1024, 2048 **Default Value:** (DWCXG_RSS_EN==1) ? 64 : 0 **Enabled:** DWCXG_RSS_EN **Parameter Name:** DWCXG_RSS_TABLE_SIZE
Enable Support for Virtualized Network	Enables the support for L2 Virtualized network over L3 network. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_EDMA&&DWCXG_DATAWIDTH==128 **Parameter Name:** DWCXG_VXN_ENA

10.9 IEEE 1588 Timestamping Parameters

Table 10-9 IEEE 1588 Timestamping Parameters

Label	Description
IEEE 1588 Timestamping	
Enable IEEE 1588 Timestamp Support	Adds logic for supporting IEEE1588 timestamping protocol. Snoops the Receive packets and captures Timestamp for packets marked as PTP packets. The Timestamp is also captured for Transmit packets marked as PTP packets. **Values:** 0, 1 **Default Value:** DWCXG_AV_ENABLE==1||DWCXG_SGF==1 **Enabled:** DWCXG_AV_ENABLE==0&&DWCXG_SGF==0 **Parameter Name:** DWCXG_TIME_STAMPING
IEEE 1588 System Time Source	Specifies the source of 64-bit reference System Time to take the Timestamps that are provided with the status. **Values:** ■ Internal (0) ■ External (1) ■ Both Options (2) **Default Value:** Internal **Enabled:** DWCXG_TIME_STAMPING==1 **Parameter Name:** DWCXG_SYSTIME_SOURCE
Add IEEE 1588 Higher Word Register	Adds a register to store the most significant 16 bits [47:32] of the seconds value if IEEE 1588 version 2 format of Timestamp is used. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1 **Parameter Name:** DWCXG_ADV_TIME_HIGH_WORD
Enable Interface for Higher Accuracy Timestamping	This feature provides additional input signals that provides the dynamic latency delay of the packet through the PHY as per IEEE 802.3cx. This information is used to adjust the timestamp of ingress and egress packets and make it more accurate. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_TIME_STAMPING **Parameter Name:** DWCXG_ETA

Table 10-9 IEEE 1588 Timestamping Parameters (continued)

Label	Description
Enable Dual Time Source Domain Capability	Enables the Additional Free Running Time Source Compliant to IEEE 802.1AS Standard to enable TSN networks with multiple (g)PTP domains. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_TIME_STAMPING **Parameter Name:** DWCXG_DSTS
Add IEEE 1588 Auxiliary Snapshot	Stores the timestamp snapshots taken with an external trigger into a FIFO. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_TIME_STAMPING==1 **Parameter Name:** DWCXG_ADV_TIME_AUX_SNAP
Number of IEEE 1588 Auxiliary Snapshot Inputs	Specifies the number of IEEE 1588 Auxiliary snapshot inputs. **Values:** 0, 1, 2, 3, 4 **Default Value:** (DWCXG_ADV_TIME_AUX_SNAP==1) ? 1:0 **Enabled:** DWCXG_ADV_TIME_AUX_SNAP==1 **Parameter Name:** DWCXG_AUX_SNAP_IN_NUM
FIFO Depth for IEEE 1588 Auxiliary Snapshots	Specifies the depth of the FIFO to store the IEEE 1588 Auxiliary snapshots. **Values:** 4, 8, 16 **Default Value:** 4 **Enabled:** DWCXG_ADV_TIME_AUX_SNAP==1 **Parameter Name:** DWCXG_AUX_FIFO_DEPTH
Enable Flexible Pulse-Per-Second Output	Enables the feature to program the start or stop time, width, and interval of the pulse generated on the ptp_pps_o output. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_TIME_STAMPING==1&&DWCXG_SYSTIME_SOURCE!=1 **Parameter Name:** DWCXG_FLEXI_PPS_OUT_EN
Number of Pulse-Per-Second Outputs	Specifies the number of Pulse-Per-Second (PPS) Outputs. **Values:** 0, 1, 2, 3, 4 **Default Value:** (DWCXG_FLEXI_PPS_OUT_EN==0) ? 0 : 1 **Enabled:** DWCXG_FLEXI_PPS_OUT_EN **Parameter Name:** DWCXG_PPS_OUT_NUM

Table 10-9 IEEE 1588 Timestamping Parameters (continued)

Label	Description
Enable PTP Timestamp Offload Feature	Enables PTP Offload (SYNC, Delay_Resp, Pdelay_Req, and Pdelay_Resp PTP packet generation) in the Transmit path. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_TIME_STAMPING==1 **Parameter Name:** DWCXG_PTO_EN
Enable One step timestamp for PTP over UDP/IP feature	Enables One step timestamp operation for PTP message sent over UDP/IP **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_TIME_STAMPING==1 **Parameter Name:** DWCXG_POU_OST_EN
Enable One-Step Timestamp Feature	Enables One-Step Timestamp in the Transmit path. **Values:** 0, 1 **Default Value:** DWCXG_PTO_EN||DWCXG_POU_OST_EN||SNPS_RSVDPARAM_45 **Enabled:** DWCXG_TIME_STAMPING&&!DWCXG_PTO_EN&&!DWCXG_POU_OST_EN&&!SNPS_RSVDPARAM_45 **Parameter Name:** DWCXG_OST_EN
Enable IEEE 1588 Sub-Nanoseconds Timestamp Support	Adds sub-nanoseconds resolution to the system timestamp and timestamp correction. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_OST_EN&&DWCXG_SYSTIME_SOURCE!=1 **Parameter Name:** DWCXG_ADV_SUBNSEC_SUPPORT
FIFO Depth for IEEE 1588 Tx Timestamp	Specifies the FIFO depth to store the IEEE 1588 Tx Timestamp for PTP packets. This FIFO is used to store the Transmit Timestamp and the associated packet ID. **Values:** 1, 2, 4, 8, 16 **Default Value:** 1 **Enabled:** (DWCXG_TIME_STAMPING||DWCXG_OST_EN)&&!DWCXG_CORE **Parameter Name:** DWCXG_TTS_FIFO_DEPTH

10.10 Time Sensitive Networking Parameters

Table 10-10 Time Sensitive Networking Parameters

Label	Description
Enhancements to Scheduling Traffic	
Enable Enhancements to Scheduling Traffic (EST)	Supports IEEE 802.1Qbv Enhancements to Scheduling Traffic Using Gate Control List. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_AV_ENABLE&&DWCXG_NUM_TC>1&&(DWCXG_TC1_AV_EN||DWCXG_TC2_AV_EN||DWCXG_TC3_AV_EN||DWCXG_TC4_AV_EN||DWCXG_TC5_AV_EN||DWCXG_TC6_AV_EN||DWCXG_TC7_AV_EN) **Parameter Name:** DWCXG_AV_EST
Stream Gate Filtering	
Enable Stream Gate Filtering (SGF)	Enables Partial IEEE 802.1 Qci Specification, Stream-Gate Filtering Based on Stream-Gate Instance and Stream Filter Instance. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_FRP_EN **Parameter Name:** DWCXG_SGF
Gate Control List	
Depth of one instance of Gate Control List	Depth of one instance of Gate Control List used in single feature (EST or SGF). **Values:** 64, 128, 256, 512, 1024 **Default Value:** 256 **Enabled:** DWCXG_AV_EST||DWCXG_SGF **Parameter Name:** DWCXG_GCL_DEP
Frame Preemption Support	
Enable Frame Preemption Support	IEEE 802.3br Frame Preemption Support for Prioritizing the Express Traffic. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG>0&&DWCXG_NUM_TC>1&&DWCXG_NUM_RXQ>1 **Parameter Name:** DWCXG_FPE

Table 10-10 Time Sensitive Networking Parameters (continued)

Label	Description
Policing Counters	
Enable Policing Counters (PC)	Enables Policing counters (PC) functionality. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_PC
Number of Policing counters (PC)	Indicates number of Policing counters (PC). **Values:** 16, 32 **Default Value:** 16 **Enabled:** DWCXG_PC **Parameter Name:** DWCXG_NUM_PC
Enable Time Based Scheduling	Adds logic for Time Based Scheduling. **Values:** 0, 1 **Default Value:** 0 **Enabled:** (DWCXG>0)&&DWCXG_TIME_STAMPING&&(DWCXG_NUM_TXQ>1) **Parameter Name:** DWCXG_TBS
Select Number of Transmit DMA channels with TBS	Specifies the number of DMA channels enabled for TBS (starting from the highest Transmit channel in descending order). **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 **Default Value:** 1 **Enabled:** DWCXG_TBS&&DWCXG_TX_PDMA>1 **Parameter Name:** DWCXG_NUM_DMA_TBS_CH
Asynchronous Traffic Shaper	Adds logic for IEEE 802.1Qcr Asynchronous Traffic Shaper (ATS) **Values:** 0, 1 **Default Value:** 0 **Enabled:** (DWCXG>0)&&DWCXG_TIME_STAMPING&&(DWCXG_NUM_TXQ>1) **Parameter Name:** DWCXG_ATS
Select Number of Transmit DMA channels with ATS	Specifies the number of DMA channels enabled for ATS (starting from the highest Transmit channel in descending order). **Values:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 **Default Value:** 1 **Enabled:** DWCXG_ATS&&DWCXG_TX_PDMA>1 **Parameter Name:** DWCXG_NUM_DMA_ATS_CH

Table 10-10 Time Sensitive Networking Parameters (continued)

Label	Description
ATS Channel offset	Specifies the ATS channel offset **Values:** 0, ..., ((DWCXG_TX_PDMA)-(DWCXG_NUM_DMA_ATS_CH)) **Default Value:** 0 **Enabled:** DWCXG_ATS&&DWCXG_TX_PDMA>1 **Parameter Name:** DWCXG_DMA_ATS_CH_OFFSET

10.11 RMON Counters Parameters

Table 10-11 RMON Counters Parameters

Label	Description
Enable MAC Management Counter (MMC)	Enables the selection of Link Remote Monitoring (RMON) counters for statistics of ingress and egress Ethernet packets. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_MMC_EN
Enable the Receive Management Counters for TCP/UDP/ICMP/IP Packets	Enables the Management counters for monitoring and collection statistics of ingress Ethernet packets with TCP/UDP/ICMP/IP payloads. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_MMC_EN&&DWCXG_RX_COE **Parameter Name:** DWCXG_MMC_IPC_EN
Enable Rx MMC Counters based on Destination RxDMA Channel	This allows the option to update the Rx MAC/IPC Management Counters only for ingress packets that is destined to be serviced or transferred by selected RxDMA channels. **Values:** 0, 1 **Default Value:** 0 **Enabled:** (DWCXG>1)&&(DWCXG_NUM_DMA_RX_CH>1)&&(DWCXG_MMC_RX_EN_ATLEAST_ONE||DWCXG_MMC_PRIORITY_EN||DWCXG_MMC_IPC_RX_EN_ATLEAST_ONE) **Parameter Name:** DWCXG_RMMC_RCH_MAP
64-bit Counter Enable	
Enable 64-bit MMC Counter	Enable 64-bit wide counter for frame counters and error counters. **Values:** 0, 1 **Default Value:** 0 **Enabled:** (DWCXG_MMC_EN&&(DWCXG_MMC_RX_EN_ATLEAST_ONE||DWCXG_MMC_TX_EN_ATLEAST_ONE)) **Parameter Name:** DWCXG_EN_64BIT_CNT

Table 10-11 RMON Counters Parameters (continued)

Label	Description
Per Priority	
Enable Management (RMON) counters for Each Priority Group.	Enables Management (RMON) Counters for each VLAN Priority group. **Values:** 0, 1 **Default Value:** 0 **Enabled:** (DWCXG_MMC_EN_TX_ALL&&DWCXG_MMC_EN_RX_ALL)&&(DWCXG_PFC_EN||DWCXG_NUM_RXQ>1) **Parameter Name:** DWCXG_MMC_PRIORITY_EN

10.12 RMON Counters / Transmit Counters Parameters

Table 10-12 RMON Counters / Transmit Counters Parameters

Label	Description
Transmit Counters	
Select All MMC Tx Counters	Selects all RMON Counters for Transmit management. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_MMC_EN **Parameter Name:** DWCXG_MMC_EN_TX_ALL
Select Individual Transmit Counters	
MMC Tx Frame Counter for Various Frame Sizes	Enables the frame counters for all the transmitted frames of different sizes (64, 65-127, 128-255, 256-511, 512-1023, 1024-maxsize). **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXADDR_RNG_CNT_EN
MMC Tx Frame size Counter for all Good and Bad Frames	Enables the frame size counter for all the good and bad transmitted frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXOCTGB_CNT_EN
MMC Tx Frame Counter for Good and Bad Frames	Enables frame counter for good and bad transmitted frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXFRMGB_CNT_EN
MMC Tx Frame Counter for Good Broadcast Frames	Enables frame counter for good transmitted broadcast frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXBCASTG_CNT_EN
MMC Tx Frame Counter for Good Multicast Frames	Enables frame counter for good transmitted multicast frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXMCASTG_CNT_EN

Table 10-12 RMON Counters / Transmit Counters Parameters (continued)

Label	Description
MMC Tx Frame Counter for Good and Bad Unicast Frames	Enables frame counter for good and bad transmitted unicast frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXUCASTGB_CNT_EN
MMC Tx Frame Counter for Good and Bad Multicast Frames	Enables frame counter for good or bad transmitted multicast frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXMCASTGB_CNT_EN
MMC Tx Frame Counter for Good and Bad Broadcast Frames	Enables frame counter for good or bad transmitted broadcast frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXBCASTGB_CNT_EN
MMC Tx Frame Counter for Underflow Errors	Enables frame counter for frames transmitted with Underflow Error bit set. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXUNDRFLW_CNT_EN
MMC Tx Frame size Counter for all Good Frames	Enables the frame size counter for good transmitted frames only. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXOCTG_CNT_EN
MMC Tx Frame Counter for Good Frames	Enables the frame counter for good transmitted frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXFRMG_CNT_EN

Table 10-12 RMON Counters / Transmit Counters Parameters (continued)

Label	Description
MMC Tx Frame Counter for Pause Frames	Enables the frame counter for good transmitted PAUSE frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXPAUSE_CNT_EN
MMC Tx Frame Counter for VLAN Frames	Enables the frame counter for transmitted VLAN-tagged frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXVLAN_CNT_EN
MMC LPI Tx Micro Second Timer	Enables the LPI Transmit Micro Second Timer **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL&&DWCXG_EEE_EN **Enabled:** DWCXG_MMC_EN&&DWCXG_EEE_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXLPI_USEC_EN
MMC LPI Tx Entry Transition Counter	Enables the LPI Transmit Entry Transition Counter. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_TX_ALL&&DWCXG_EEE_EN **Enabled:** DWCXG_MMC_EN&&DWCXG_EEE_EN&&!DWCXG_MMC_EN_TX_ALL **Parameter Name:** DWCXG_MMC_TXLPI_TRANS_CNTR_EN

10.13 RMON Counters / Receive Counters Parameters

Table 10-13 RMON Counters / Receive Counters Parameters

Label	Description
Receive Counters	
Select All MMC Rx Counters	Selects all RMON counters for Receive management. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_MMC_EN **Parameter Name:** DWCXG_MMC_EN_RX_ALL
Select Individual Receive Counters	
MMC Rx Frame Counter for Good and Bad Frames	Enables a counter for received good or bad frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXFRMGB_CNT_EN
MMC Rx Frame size Counter for all Good Frames	Enables the frame size counter for received good frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXOCTG_CNT_EN
MMC Rx Frame size Counter for Good and Bad Frames	Enables the frame size counter for all received frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXOCTGB_CNT_EN
MMC Rx Frame Counter for Good Broadcast Frames	Enables a counter for good broadcast frames received. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXBCASTG_CNT_EN
MMC Rx Frame Counter for Good Multicast Frames	Enables a counter for good multicast frames received. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXMCASTG_CNT_EN

Table 10-13 RMON Counters / Receive Counters Parameters (continued)

Label	Description
MMC Rx Frame Counter for CRC Errors	Enables a counter for frames received with a CRC error. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXCRCERR_CNT_EN
MMC Rx Frame Counter for Runt Frames	Enables a counter for received runt (less-than-64-byte, with CRC error) frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXRUNTERR_CNT_EN
MMC Rx Frame Counter for Jabber Error Frames	Enables a counter for frames received with a Jabber error. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXJABERR_CNT_EN
MMC Rx Frame Counter for Undersize Frames	Enables a counter for received undersize (less than 64-byte, no error) frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXUNDERSZG_CNT_EN
MMC Rx Frame Counter for Giant Frames	Enables a counter for received oversize (size > 1,518 or 1,522 bytes) frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXOVERSZG_CNT_EN
MMC Rx Frame Counter for Various Frame Sizes	Enables the frame counters for all received frames of different sizes (64, 65-127, 128-255, 256-511, 512-1023, 1024-maxsize). **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXADDR_RNG_CNT_EN

Table 10-13 RMON Counters / Receive Counters Parameters (continued)

Label	Description
MMC Rx Frame Counter for Unicast Frames	Enables a counter for unicast frames received. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXUCASTG_CNT_EN
MMC Rx Frame Counter for Length Errors	Enables a counter for frames received with a length error. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXLENERR_CNT_EN
MMC Rx Frame Counter for Out of Range Errors	Enables out-of-range (Length field between 1,500 and 1,536) frame counter. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXOUTOFRNG_TYP_CNT_EN
MMC Rx Frame Counter for Pause Frames	Enables a counter for received PAUSE frames. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXPAUSEFRM_CNT_EN
MMC Rx Frame Counter for Overflow Errors	Enables a counter to count the number of times Rx MTL FIFO has overflowed. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL&&!DWCXG_CORE **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL&&!DWCXG_CORE **Parameter Name:** DWCXG_MMC_RXFIFOOVFL_CNT_EN
MMC Rx Frame Counter for VLAN Frames	Enables counter for VLAN good or bad frames received. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXVLANFRGB_CNT_EN

Table 10-13 RMON Counters / Receive Counters Parameters (continued)

Label	Description
MMC Rx Frame Counter for Watchdog Timeout Errors	Enables a counter for frames with watchdog time-out errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXWDGERR_CNT_EN
MMC Rx Frame Counter for Discarded frames due to filter fail	Enables a counter for frames which are discarded due to filter failure. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXFRMDISGB_CNT_EN
Enable MMC Rx Frame Counter for Packets with Alignment Error	Enables counter for packets received with Alignment Error (dribble) in the 10/100 Mbps mode. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL&&DWCXG_MII_SUPPORT **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL&&DWCXG_MII_SUPPORT **Parameter Name:** DWCXG_MMC_RXALGNERR_CNT_EN
MMC Rx Frame size Counter for Discarded frames due to filter fail	Enables a Frame size counter for frames which are discarded due to filter failure. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL **Enabled:** DWCXG_MMC_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXOCTDISGB_CNT_EN
MMC LPI Rx Micro Second Timer	Enables the LPI Rx Micro Second Timer **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL&&DWCXG_EEE_EN **Enabled:** DWCXG_MMC_EN&&DWCXG_EEE_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXLPI_USEC_EN
MMC LPI Rx Entry Transition Counter	Enables the LPI Rx Entry Transition Counter. **Values:** 0, 1 **Default Value:** DWCXG_MMC_EN_RX_ALL&&DWCXG_EEE_EN **Enabled:** DWCXG_MMC_EN&&DWCXG_EEE_EN&&!DWCXG_MMC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RXLPI_TRANS_CNTR_EN

10.14 RMON Counters / Receive IPC Counters Parameters

Table 10-14 RMON Counters / Receive IPC Counters Parameters

Label	Description
Receive IPC Counters	
Enable All MMC IPC Rx Counters	Enables all Rx IPC counters. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_MMC_IPC_EN **Parameter Name:** DWCXG_MMC_IPC_EN_RX_ALL
Select Individual Receive IPC Counters	
Enable MMC Counter for Good IPv4 Packets	Enables MMC Counter for IPv4 packets with no checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_GD_PKTS_CNT_EN
Enable MMC Counter for IPv4 Packets with Header Error	Enables MMC Counter for IPv4 packets with checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_HDRERR_PKTS_CNT_EN
Enable MMC Counter for IPv4 Packets without TCP, UDP, and ICMP	Enables MMC Counter for IPv4 packets which do not have TCP, UDP, or ICMP. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_NOPAY_PKTS_CNT_EN
Enable MMC Counter for Fragmented IPv4 Packets	Enables Rx frame counter for all IPv4 packets received with fragments. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_FRAG_PKTS_CNT_EN
Enable MMC Counter for IPv4 UDP Packets with Checksum Disabled	Enables MMC Counter for IPv4 UDP packets with checksum disabled. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_UDSBL_PKTS_CNT_EN

Table 10-14 RMON Counters / Receive IPC Counters Parameters (continued)

Label	Description
Enable MMC Counter for Good IPv6 Packets	Enables MMC Counter for IPv6 packets that do not have checksum-related errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV6_GD_PKTS_CNT_EN
Enable MMC Counter for IPv6 Packets with Header Error	Enables MMC Counter for IPv6 packets received with header error. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV6_HDRERR_PKTS_CNT_EN
Enable MMC Counter for IPv6 Packets without TCP, UDP, and ICMP	Enables MMC Counter for IPv6 packets not having TCP, UDP, or ICMP. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV6_NOPAY_PKTS_CNT_EN
Enable MMC Counter for UDP Packets without Error	Enables MMC Counter for packets containing UDP with no errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_UDP_GD_PKTS_CNT_EN
Enable MMC Counter for UDP Packets with Error	Enables MMC Counter for packets containing UDP with errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_UDP_ERR_PKTS_CNT_EN
Enable MMC Counter for TCP Packets without Error	Enables MMC Counter for packets containing TCP with no errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_TCP_GD_PKTS_CNT_EN

Table 10-14 RMON Counters / Receive IPC Counters Parameters (continued)

Label	Description
Enable MMC Counter for TCP Packets with Error	Enables MMC Counter for packets containing TCP with errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_TCP_ERR_PKTS_CNT_EN
Enable MMC Counter for ICMP Packets without Error	Enables MMC Counter for packets containing ICMP with no errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_ICMP_GD_PKTS_CNT_EN
Enable MMC Counter for ICMP Packets with Error	Enables MMC Counter for packets containing ICMP with errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_ICMP_ERR_PKTS_CNT_EN
Enable MMC Octet Counter for IPv4 Packets without Checksum Error	Enables MMC Counter counting IPv4 datagrams bytes when there are no checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_GD_OCTET_CNT_EN
Enable MMC Octet Counter for IPv4 Packets with Header Checksum Error	Enables an Rx octet counter for all IPv4 datagrams received with a header error. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_HDRERR_OCTET_CNT_EN
Enable MMC Octet Counter for IPv4 Packets without UDP, TCP, and ICMP	Enables MMC Counter counting bytes of IPv4 datagrams with no UDP, TCP, or ICMP. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_NOPAY_OCTET_CNT_EN

Table 10-14 RMON Counters / Receive IPC Counters Parameters (continued)

Label	Description
Enable MMC Octet Counter for Fragmented IPv4 Packets	Enables an Rx octet counter for all IPv4 datagrams received with fragments. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_FRAG_OCTET_CNT_EN
Enable MMC Octet Counter for IPv4 UDP Packets with Checksum Disabled	Enables MMC Counter counting bytes in IPv4 UDP datagram which has checksum disabled. This includes only UDP bytes and not the IPv4 header. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV4_UDSBL_OCTET_CNT_EN
Enable MMC Octet Counter for IPv6 Packets without Checksum Error	Enables MMC Counter counting bytes in IPv6 datagram having no checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV6_GD_OCTET_CNT_EN
Enable MMC Octet Counter for IPv6 Packets with Header Error	Enables MMC Counter counting bytes in IPv6 datagram having header errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV6_HDRERR_OCTET_CNT_EN
Enable MMC Octet Counter for IPv6 Packets without TCP, UDP, and ICMP	Enables MMC Counter counting bytes in IPv6 datagram not having TCP, UDP, or ICMP. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_IPV6_NOPAY_OCTET_CNT_EN
Enable MMC Octet Counter for UDP Packets without Checksum Error	Enables MMC Counter counting bytes in UDP which do not have checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_UDP_GD_OCTET_CNT_EN

Table 10-14 RMON Counters / Receive IPC Counters Parameters (continued)

Label	Description
Enable MMC Octet Counter for UDP Packets with Checksum Error	Enables MMC Counter counting bytes in UDP which have checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_UDP_ERR_OCTET_CNT_EN
Enable MMC Octet Counter for TCP Packets without Checksum Error	Enables MMC Counter counting bytes in TCP which do not have checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_TCP_GD_OCTET_CNT_EN
Enable MMC Octet Counter for TCP Packets with Checksum Error	Enables MMC Counter counting bytes in TCP which have checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_TCP_ERR_OCTET_CNT_EN
Enable MMC Octet Counter for ICMP Packets without Checksum Error	Enables MMC Counter counting bytes in ICMP which do not have checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_ICMP_GD_OCTET_CNT_EN
Enable MMC Octet Counter for ICMP Packets with Checksum Error	Enables MMC Counter counting bytes in ICMP which have checksum errors. **Values:** 0, 1 **Default Value:** DWCXG_MMC_IPC_EN_RX_ALL **Enabled:** DWCXG_MMC_IPC_EN&&!DWCXG_MMC_IPC_EN_RX_ALL **Parameter Name:** DWCXG_MMC_RX_ICMP_ERR_OCTET_CNT_EN

10.15 Automotive Safety Features Parameters

Table 10-15 Automotive Safety Features Parameters

Label	Description
Enable All Automotive Safety Features	Selects all the Automotive Safety Features such as, SECDED Protection for External Memories, On-chip Data Path Parity Protection, FSM State Parity and FSM Timeout Protection, Application and CSR Interface Timeout Protection. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG>0 **Parameter Name:** DWCXG_ASP_ALL
Enable SECDED ECC protection for external memories	Adds Error Correcting code(SECDED) for external memory interface **Values:** 0, 1 **Default Value:** DWCXG_ASP_ALL **Enabled:** !DWCXG_ASP_ALL **Parameter Name:** DWCXG_ASP_ECC
ECC Code Width for Tx FIFO	ECC code width for Transmit FIFO memory. This along with MTL FIFO data width determine the data width of the MTL Transmit FIFO when SECDED ECC protection for external memories feature is selected. **Values:** -2147483648, ..., 2147483647 **Default Value:** DWCXG_ASP_ECC==1 ? (DWCXG_DATAWIDTH==32 ? ((DWCXG_TX_FIFO_PTR_WIDTH+35) <= 57 ? 7 : (DWCXG_TX_FIFO_PTR_WIDTH+35) <= 120 ? 8 : 9) : DWCXG_DATAWIDTH==64 ? ((DWCXG_TX_FIFO_PTR_WIDTH+68) <= 120 ? 8 : 9) : (DWCXG_TX_FIFO_PTR_WIDTH+133) <= 247 ? 9 : 10) : 0 **Enabled:** DWCXG>0 **Parameter Name:** DWCXG_TX_ECW
ECC Code Width for Rx FIFO	ECC code width for Receive FIFO memory. This along with MTL FIFO data width determine the data width of the MTL Receive FIFO when SECDED ECC protection for external memories feature is selected. **Values:** -2147483648, ..., 2147483647 **Default Value:** DWCXG_ASP_ECC==1 ? (DWCXG_DATAWIDTH==32 ? ((DWCXG_RX_FIFO_PTR_WIDTH+35) <= 57 ? 7 : (DWCXG_RX_FIFO_PTR_WIDTH+35) <= 120 ? 8 : 9) : DWCXG_DATAWIDTH==64 ? ((DWCXG_RX_FIFO_PTR_WIDTH+68) <= 120 ? 8 : 9) : (DWCXG_RX_FIFO_PTR_WIDTH+133) <= 247 ? 9 : 10) : 0 **Enabled:** DWCXG>0 **Parameter Name:** DWCXG_RX_ECW

Table 10-15 Automotive Safety Features Parameters (continued)

Label	Description
ECC Code Width for GCL Memory	Width of the ECC Code Word Computed for the Entries in the GCL Memory of EST and SGF Features. **Values:** -2147483648, ..., 2147483647 **Default Value:** DWCXG_AV_EST&&DWCXG_SGF&&DWCXG_ASP_ECC) ? ((DWCXG_EST_ECW > DWCXG_SGF_ECW) ? DWCXG_EST_ECW : DWCXG_SGF_ECW) : ((DWCXG_AV_EST&&DWCXG_ASP_ECC)? DWCXG_EST_ECW : ((DWCXG_SGF&&DWCXG_ASP_ECC) ? DWCXG_SGF_ECW : 0) **Enabled:** DWCXG_AV_EST||DWCXG_SGF **Parameter Name:** DWCXG_GCL_ECW
ECC Code Width for Descriptor Cache Memory	Width of the ECC Code Word Computed for the Descriptors in Descriptor Cache Memory. **Values:** -2147483648, ..., 2147483647 **Default Value:** (DWCXG_DCACHE_MEM_EN&&DWCXG_ASP_ECC)==1 ? ((((DWCXG_DCACHE_DW+DWCXG_DCACHE_MEM_ADDR_WIDTH) <= 57) ? 7 : ((DWCXG_DCACHE_DW+DWCXG_DCACHE_MEM_ADDR_WIDTH) <= 120) ? 8 : 9)) : 0 **Enabled:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Parameter Name:** DWCXG_DCACHE_ECW
ECC Code Width for External CSR Memory	Width of the ECC Code Word Computed for the data in External CSR Memory. **Values:** -2147483648, ..., 2147483647 **Default Value:** (SNPS_RSVDPARAM_50&&DWCXG_ASP_ECC)==1 ? 9 : 0 **Enabled:** SNPS_RSVDPARAM_50 **Parameter Name:** DWCXG_HDMA_ECW
ECC Code Width for Descriptor Cache and TSO Memory	Width of the ECC Code Word Computed for the Descriptors and Header Entries in the Descriptor Cache and TSO Memory. **Values:** -2147483648, ..., 2147483647 **Default Value:** (DWCXG_CACHE_MEM_EN&&DWCXG_ASP_ECC)==1 ? ((((DWCXG_TSOMEM_DW+DWCXG_CACHE_MEM_ADDR_WIDTH) <= 57) ? 7 : ((DWCXG_TSOMEM_DW+DWCXG_CACHE_MEM_ADDR_WIDTH) <= 120) ? 8 : 9)) : 0 **Enabled:** DWCXG_TSO_MEM_EN||DWCXG_EDMA **Parameter Name:** DWCXG_CACHE_ECW

Table 10-15 Automotive Safety Features Parameters (continued)

Label	Description
Adds ECC or Parity Ports on Application interface	Selects the Mode (Parity for Data/Control Signals or ECC for Data Signals) for the ECC or Parity Ports on Application Interface. **Values:** ■ NO_PARITY_ECC (0) ■ DATA_PARITY_ONLY (1) ■ CTRL_PARITY_DATA_ECC (2) **Default Value:** NO_PARITY_ECC **Enabled:** DWCXG_ASP_ALL **Parameter Name:** DWCXG_ASP_PPE
Enable Odd Parity ports on Application interface	Selects the Odd Parity ports on external application interface enabling Interface Odd Parity Checking. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_ASP_PPE!=0 **Parameter Name:** DWCXG_ASP_PPE_ODD
Enable AXI Interface Parity/ECC Mode	Selects the Normal Parity/ECC Mode (One Parity Bit Per 8-bit Data) or ARC Mode Parity/ECC Computation. **Values:** 0, 1 **Default Value:** DWCXG_ASP_PPE==2 **Enabled:** DWCXG_ASP_PPE==2 **Parameter Name:** DWCXG_AISPM
Select AXI Interface ECC Granularity	Selects the Granularity for ECC Computation, Separate ECC Word for Each Group of 32-bit, 64-bit or Full Data Word. **Values:** ■ ECCDW (0) ■ ECC32 (1) ■ ECC64 (2) **Default Value:** DWCXG_AISPM ? 1 : 0 **Enabled:** DWCXG_ASP_PPE==2&&!DWCXG_AISPM **Parameter Name:** DWCXG_AISEG

10.16 Low Power Management Parameters

Table 10-16 Low Power Management Parameters

Label	Description
Enable Power Management (PMT) Block	Selects the Power Management feature using Magic Packet or Wake-up frames. **Values:** 0, 1 **Default Value:** 0 **Enabled:** Always **Parameter Name:** DWCXG_PMT_EN
Enable Detection of Magic Packet Frame	Selects the AMD Magic Packet frame detection. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_PMT_EN **Parameter Name:** DWCXG_PMT_MGK_EN
Enable Detection of Remote Wake-Up Frame	Selects the Power Management block Remote Wake-up frame detection. **Values:** 0, 1 **Default Value:** 0 **Enabled:** DWCXG_PMT_EN **Parameter Name:** DWCXG_PMT_RWK_EN
Number of Remote Wake-Up Packet Filters	Selects the number of remote wake-up packet filters. **Values:** 0, 4, 8, 16 **Default Value:** DWCXG_PMT_RWK_EN ? 4 : 0 **Enabled:** DWCXG_PMT_RWK_EN **Parameter Name:** DWCXG_NUM_PMT_RWK_FILT
Enable Energy Efficient Ethernet (EEE)	Adds Logic for Supporting the Energy Efficient Ethernet (EEE) functionality. **Values:** 0, 1 **Default Value:** 0 **Enabled:** SNPS_RSVDPARAM_49!=1 **Parameter Name:** DWCXG_EEE_EN

11

Register Descriptions

This chapter details all possible registers in the controller. They are arranged hierarchically into maps and blocks (banks). For configurable IP titles, your actual configuration might not contain all of these registers.

Attention: For configurable IP titles, do not use this document to determine the exact attributes of your register map. It is for reference purposes only.

When you configure the controller in coreConsultant, you must access the register attributes for your actual configuration at `workspace/report/ComponentRegisters.html` or `workspace/report/ComponentRegisters.xml` after you have completed the report creation activity. That report comes from the exact same source as this chapter but removes all the registers that are not in your actual configuration. This does not apply to non-configurable IP titles. In addition, all parameter expressions are evaluated to actual values. Therefore, the Offset and Memory Access values might change depending on your actual configuration.

Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

Exists Expressions

These expressions indicate the combination of configuration parameters required for a register, field, or block to exist in the memory map. The expression is only valid in the local context and does not indicate the conditions for existence of the parent. For example, the expression for a bit field in a register assumes that the register exists and does not include the conditions for existence of the register.

Offset

The term *Offset* is synonymous with *Address.*

Memory Access Attributes

The Memory Access attribute is defined as `<ReadBehavior>/<WriteBehavior>` which are defined in the following table.

Table 11-1 Possible Read and Write Behaviors

Read (or Write) Behavior	Description
RC	A read clears this register field.
RS	A read sets this register field.
RM	A read modifies the contents of this register field in a manner not described by any of the above.
Wo	You can only write once to this register field.
W1C	A write of 1 clears this register field.
W1S	A write of 1 sets this register field.
W1T	A write of 1 toggles this register field.
W0C	A write of 0 clears this register field.
W0S	A write of 0 sets this register field.
W0T	A write of 0 toggles this register field.
WC	Any write clears this register field.
WS	Any write sets this register field.
WM	A write modifies this register field in a manner not described by any of the above.
no Read Behavior attribute	You cannot read this register. It is Write-Only.
no Write Behavior attribute	You cannot write to this register. It is Read-Only.

Table 11-2 Memory Access Examples

Memory Access	Description
R	Read-only register field.
W	Write-only register field.
R/W	Read/write register field.
R/W1C	You can read this register field. Writing 1 clears it.
RC/W1C	Reading this register field clears it. Writing 1 clears it.
R/Wo	You can read this register field. You can only write to it once.

Special Optional Attributes

Some register fields might use the following optional attributes.

Table 11-3 Optional Attributes

Attribute	Description
Volatile	As defined by the IP-XACT specification. If true, indicates in the case of a write followed by read, or in the case of two consecutive reads, there is no guarantee as to what is returned by the read on the second transaction or that this return value is consistent with the write or read of the first transaction. The element implies there is some additional mechanism by which this field can acquire new values other than by reads/writes/resets and other access methods known to IP-XACT. For example, when the controller updates the register field contents.
Testable	As defined by the IP-XACT specification. Possible values are unconstrained, untestable, readOnly, writeAsRead, restore. Untestable means that this field is untestable by a simple automated register test. For example, the read-write access of the register is controlled by a pin or another register. readOnly means that you should not write to this register; only read from it. This might apply for a register that modifies the contents of another register.
Reset Mask	As defined by the IP-XACT specification. Indicates that this register field has an unknown reset value. For example, the reset value is set by another register or an input pin; or the register is implemented using RAM.
* Varies	Indicates that the memory access (or reset) attribute (read, write behavior) is not fixed. For example, the read-write access of the register is controlled by a pin or another register. Or when the access depends on some configuration parameter; in this case the post-configuration report in coreConsultant gives the actual access value.

Register definitions for each component memory map.

Table 11-4 Registers for the DWC_xgmac_map Memory Map

Register	Offset	Description
MAC Protocol-Core Registers. DWCXG_CORE Exists: Always		
“MAC_Tx_Configuration” on page 544	0x0	The MAC Transmit Configuration register establishes the operating mode of the MAC transmitter.
“MAC_Rx_Configuration” on page 550	0x4	The MAC Receive Configuration register establishes the operating mode of the MAC receiver.
“MAC_Packet_Filter” on page 555	0x8	The MAC Packet Filter register contains the filter controls for receiving packets. Some of the controls...
“MAC_WD_JB_Timeout” on page 561	0xc	The Watchdog and Jabber Timeout register controls the watchdog timeout limit for the received packets...
“MAC_Hash_Table_Reg0” on page 564	0x10	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
“MAC_Hash_Table_Reg1” on page 565	0x14	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_Hash_Table_Reg2" on page 566	0x18	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg3" on page 567	0x1c	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg4" on page 568	0x20	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg5" on page 569	0x24	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg6" on page 570	0x28	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg7" on page 571	0x2c	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg8" on page 572	0x30	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg9" on page 573	0x34	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg10" on page 574	0x38	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg11" on page 575	0x3c	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg12" on page 576	0x40	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg13" on page 577	0x44	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg14" on page 578	0x48	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_Hash_Table_Reg15" on page 579	0x4c	You can specify the width of the hash table by using the Hash Table Size option in coreConsultant. The...
"MAC_DAH_TBL_SIZE" on page 580	0x4c	This register is used to configure external DA Hash table size based on the selected CRC bits to...
"MAC_VLAN_Tag" on page 581	0x50	The VLAN Tag register identifies the IEEE 802.1Q VLAN type packets. <This register hold control...
"MAC_VLAN_Tag_Ctrl" on page 585	0x50	MAC_VLAN_Tag_Ctrl register is a re-defined version of MAC_VLAN_Tag register. This register holds...
"MAC_VLAN_Tag_Data" on page 591	0x54	This register holds the read/write data for Indirect Access of the Per MAC_VLAN_Tag_Filter registers....

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_VLAN_Tag_Filter(#i)(for i = 0; i <= DWCXG_NRVF-1)" on page 593	0x54	This register contains per VLAN Tag filter control information.
"MAC_VLAN_Hash_Table" on page 596	0x58	When the ERSVLM bit is set of MAC_VLAN_Tag_Ctrl, the 16-bit VLAN Hash Table register is used for...
"MAC_VLAN_Incl" on page 597	0x60	The VLAN Tag Inclusion or Replacement register contains the VLAN tag for insertion or replacement...
"MAC_VLAN_Incl(#i)(for i = 0; i <= DWCXG_NUM_TXQ-1)" on page 601	0x60	The per Tx Queue VLAN Tag Inclusion register contains the VLAN tag for insertion in the Transmit...
"MAC_Inner_VLAN_Incl" on page 603	0x64	The Inner VLAN Tag Inclusion or Replacement register contains the inner VLAN tag to be inserted...
"MAC_RVLAN_LKP_SIZE" on page 606	0x68	This register is used to configure external receive outer/inner VLAN tag lookup size based on the...
"MAC_Rx_Eth_Type_Match" on page 608	0x6c	The ET field of this register contains the value that needs to be compared with the Ethernet type...
"MAC_Q0_Tx_Flow_Ctrl" on page 609	0x70	The MAC_Q0_Tx_Flow_Ctrl register controls the generation of the Control (Pause Command) packets...
"MAC_Q1_Tx_Flow_Ctrl" on page 612	0x74	The MAC_Q1_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority...
"MAC_Q2_Tx_Flow_Ctrl" on page 615	0x78	The MAC_Q2_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority...
"MAC_Q3_Tx_Flow_Ctrl" on page 618	0x7c	The MAC_Q3_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority...
"MAC_Q4_Tx_Flow_Ctrl" on page 621	0x80	The MAC_Q4_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority...
"MAC_Q5_Tx_Flow_Ctrl" on page 624	0x84	The MAC_Q5_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority...
"MAC_Q6_Tx_Flow_Ctrl" on page 627	0x88	The MAC_Q6_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority...
"MAC_Q7_Tx_Flow_Ctrl" on page 630	0x8c	The MAC_Q7_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority...
"MAC_Rx_Flow_Ctrl" on page 633	0x90	The Receive Flow Control register controls the pausing of MAC Transmit based on the received Pause...
"MAC_RxQ_Ctrl4" on page 635	0x94	The Receive Queue Control 4 register controls the routing of - Packets which fail the Destination...
"MAC_RxQ_Ctrl5" on page 639	0x98	The Receive Queue Control 5 register allows selection of start offset of the Receive queue from...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_RxQ_Ctrl0" on page 640	0xa0	The Receive Queue Control 0 register activates the queue management in the MAC Receiver. This register...
"MAC_RxQ_Ctrl1" on page 643	0xa4	This register allows the selection of the Receive queues to which the received untagged or special...
"MAC_RxQ_Ctrl2" on page 647	0xa8	This register controls the routing of tagged packets based on the USP (user Priority) field of the...
"MAC_RxQ_Ctrl3" on page 649	0xac	This register controls the routing of tagged packets based on the USP (user Priority) field of the...
"MAC_Interrupt_Status" on page 651	0xb0	The Interrupt Status register. The register contains status of the events which generates interrupt...
"MAC_Interrupt_Enable" on page 659	0xb4	The Interrupt Enable register contains the masks for generating the interrupts.
"MAC_Rx_Tx_Status" on page 661	0xb8	The Receive Transmit Status register contains the Receive and Transmit Error status.
"MAC_Tx_Clk_Slot_Interval" on page 665	0xbc	This register indicates the Number of MAC Transmit Clocks that constitutes a slot interval.
"MAC_PMT_Control_Status" on page 666	0xc0	The PMT Control and Status register is present only when you select the PMT module in...
"MAC_RWK_Packet_Filter" on page 670	0xc4	The Remote Wakeup Filter registers are implemented as 8, 16, or 32 indirect access registers (wkuppktfilter_reg#i)...
"RWK_Filter(#i)_Byte_Mask(for i = 0; i <= DWCXG_NUM_PMT_RWK_FILT-1)" on page 672	0xc4	Filter i Byte Mask. The filter i byte mask register defines the bytes of the packet that are examined...
"RWK_Filter(#i)_(#i+3)_Command(for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1)" on page 673	0xc4	This register controls the filter operation.
"RWK_Filter(#i)_(#i+3)_Offset(for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1)" on page 677	0xc4	This register defines the offset (within the packet) from which the filter examines the packet
"RWK_Filter(#i)_(#i+1)_CRC(for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/2)-1)" on page 679	0xc4	Filter i CRC-16. This register contains the CRC-16 to be matched.
"MAC_LPI_Control_Status" on page 680	0xd0	The LPI Control and Status register controls the LPI functions and provides the LPI interrupt status....
"MAC_LPI_Timers_Control" on page 685	0xd4	The LPI Timers Control register controls the timeout values in the LPI states. It specifies the...
"MAC_LPI_Auto_Entry_Timer" on page 686	0xd8	This register is used to store the LPI Auto Entry Timer Value in Micro-Seconds. This register is...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_1US_Tic_Counter" on page 687	0xdc	This register controls the generation of the Reference time (1micro-second tic) for all the LPI...
"MAC_Tunnel_Identifier" on page 688	0xe0	This register provides the programmable value to used as the identifier for VxLAN and NVGRE tunneled...
"MAC_Version" on page 689	0x110	The version register identifies the version of DWC_25gmac. This register contains two bytes: one...
"MAC_Debug" on page 690	0x114	The Debug register provides the debug status of various MAC blocks.
"MAC_HW_Feature0" on page 692	0x11c	This register indicates the presence of the first set of optional features or functions of DWC_25gmac....
"MAC_HW_Feature1" on page 697	0x120	This register indicates the presence of second set of the optional features or functions of DWC_25gmac....
"MAC_HW_Feature2" on page 701	0x124	This register indicates the presence of the third set of optional features or functions of DWC_25gmac....
"MAC_HW_Feature3" on page 705	0x128	This register indicates the presence of the fourth set of optional features or functions of DWC_25gmac....
"MAC_HW_Feature4" on page 711	0x12c	This register indicates the presence of the fifth set of optional features or functions of DWC_25gmac....
"MAC_Extended_Configuration" on page 713	0x140	The MAC Extended Configuration register establishes the operating mode of the MAC transmitter.
"MAC_Ext_Cfg1" on page 719	0x144	This register contains Split mode control field and offset field for Header-Payload Split...
"MAC_DPP_FSM_Interrupt_Status" on page 721	0x150	This register contains the status of Automotive Safety related Data Path Parity Errors, Interface...
"MAC_AXI_SLV_DPE_Addr_Status" on page 733	0x154	This register gives the address of the Register Write transaction in which parity error...
"MAC_FSM_Control" on page 734	0x158	This register is used to control the FSM State parity and timeout error injection in Debug...
"MAC_FSM_ACT_Timer" on page 741	0x15c	This register is used to select the FSM and Interface Timeout values.
"SNPS_SCS_REG1" on page 744	0x160	Synopsys Reserved Register
"MAC_SCSR_Control" on page 745	0x164	This register is used to control safety protected CSR parity generation.
"MAC_SCSR_Parity_Status" on page 747	0x168	This register indicates the offset address of the register where parity_error occurred.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_SCSR_Parity_Err_Count" on page 748	0x16c	This register indicates error count associated with parity check.
"MAC_Flex_Hdr_Cfg" on page 749	0x194	This register contains flexible header enable, length and start position for Flexible Header...
"MAC_Flex_Hdr_Low" on page 752	0x198	This register contains programmable Lower Static Flexible Header for Mac-gen pause/PFC/hardware...
"MAC_Flex_Hdr_High" on page 753	0x19c	This register contains programmable Upper Static Flexible Header for Mac-gen (pause/PFC)/hardware...
"MDIO_Single_Command_Address" on page 754	0x200	This register contains the addresses of the ports, devices, and registers accessed by the SMA during...
"MDIO_Single_Command_Control_Data" on page 755	0x204	This register controls single read/write operations, and contains the data bits read on a single-read...
"MDIO_Continuous_Write_Address" on page 758	0x208	You can program this register to start continuous write operation.
"MDIO_Continuous_Write_Data" on page 760	0x20c	This register contains the data to be written during continuous write operation. Do not change the...
"MDIO_Continuous_Scan_Port_Enable" on page 761	0x210	This register controls the PHY ports and corresponding devices (enabled by the corresponding Device...
"MDIO_Interrupt_Status" on page 768	0x214	This register gives the source of the interrupt raised by the SMA module on the sbd_intr_o output...
"MDIO_Interrupt_Enable" on page 773	0x218	This register controls the enabling of interrupt sources in the MDIO Interrupt Status...
"MDIO_Port_Connect_Disconnect_Status" on page 777	0x21c	This register gives the hot plug-in status for Ports 0-31. This register is updated at the end of...
"MDIO_Clause_22_Port" on page 783	0x220	This register configures the SMA to access the PHY ports using either the Clause 22 or Clause 45...
"MDIO_Port_Nx4_Indirect_Control" on page 788	0x224	This register contain fields that control the Port range select for accessing register sets.
"MDIO_PortNx4P0_Device_In_Use" on page 789	0x230	This register gives the status of each device on Port Nx4 Plus0 (Here N is as per the PRS field...
"MDIO_PortNx4P0_Link_Status" on page 792	0x234	This register gives the link status of the devices in PHY Port Nx4Plus0 ((Here N is as per the PRS...
"MDIO_PortNx4P0_Alive_Status" on page 795	0x238	This register gives the device status in PHY Port Nx4Plus0 ((Here N is as per PRS field of "MDIO...
"MDIO_PortNx4P1_Device_In_Use" on page 798	0x240	This register gives the status of each device on Port Nx4 Plus1 (Here N is as per the PRS field...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MDIO_PortNx4P1_Link_Status" on page 801	0x244	This register gives the link status of the devices in PHY Port Nx4Plus1 ((Here N is as per PRS field...
"MDIO_PortNx4P1_Alive_Status" on page 804	0x248	This register gives the device status in PHY Port Nx4Plus1 ((Here N is as per PRS field of MDIO...
"MDIO_PortNx4P2_Device_In_Use" on page 807	0x250	This register gives the status of each device on Port Nx4 Plus2 (Here N is as per the PRS field...
"MDIO_PortNx4P2_Link_Status" on page 810	0x254	This register gives the link status of the devices in PHY Port Nx4Plus2 ((Here N is as per the PRS...
"MDIO_PortNx4P2_Alive_Status" on page 813	0x258	This register gives the device status in PHY Port Nx4Plus2 ((Here N is as per the PRS field of MDIO...
"MDIO_PortNx4P3_Device_In_Use" on page 816	0x260	This register gives the status of each device on Port Nx4 Plus3 (Here N is as per the PRS field...
"MDIO_PortNx4P3_Link_Status" on page 819	0x264	This register gives the link status of the devices in PHY Port Nx4Plus3 ((Here N is as per the PRS...
"MDIO_PortNx4P3_Alive_Status" on page 822	0x268	This register gives the device status in PHY Port Nx4Plus3 ((Here N is as per the PRS field of MDIO...
"MAC_GPIO_Control" on page 825	0x278	The GPIO Control register controls the GPIO.
"MAC_GPIO_Status" on page 827	0x27c	The General Purpose IO register provides the control to drive the following: up to 16 bits of output...
"MAC_FPE_CTRL_STS" on page 829	0x280	This register controls the operation of Frame Preemption.
"MAC_CSR_SW_Ctrl" on page 833	0x290	This register contains software programmable controls for changing the CSR access response and status...
"MAC_Address0_High" on page 835	0x300	The MAC_Address0_High register holds the upper 16 bits of the 0th 6-byte MAC address of the station....
"MAC_Address0_Low" on page 837	0x304	The MAC_Address0_Low register holds the lower 32 bits of the 0th 6-byte MAC address of the...
"MAC_Address(#i)_High(for i = 1; i <= DWCXG_ADD_MAC_ADDR_REG)" on page 838	(0x0008*i) +0x0300	The MAC_Address1_High register holds the upper 16 bits of the 1th 6-byte MAC address of the station....
"MAC_Indir_Access_Ctrl" on page 840	0x700	This register provides the Indirect Access control and status for MAC_(#MSEL)_IndReg(#AOFF)...
"MAC_Indir_Access_Data" on page 844	0x704	This register holds the read/write data for Indirect Access of MAC_(#MSEL)_IndReg(#AOFF)...
"MAC_DChSel_IndReg(#i)(for i = 0; i <= DWCXG_ADD_MAC_ADDR_REG+1)" on page 845	0x704	This register contains DCS (XDCS) one hot encoded value of receive channel selection for...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_PCCtrl_IndReg#(i)(for i = 0; i <= @DWCXG_NUM_PC-1)" on page 846	0x704	This register contains control information for policing counters associated with MAC_(#MSEL)_(#AOFF)...
"MAC_PCntr_IndReg#(i)(for i = 0; i <= @DWCXG_NUM_PC-1)" on page 848	0x704	This register contains count value of policing counters.
"MAC_DPCSel_IndReg(#i)(for i = 0; i <= max((DWCXG_ADD_MAC_ADDR_REG+1),DWCXG_NUM_PC))" on page 849	0x704	This register contains policing counter number corresponding to each DA or SA filter event.
"MAC_VPCSel_IndReg(#i)(for i = 0; i <= DWCXG_NRVF_PC-1)" on page 851	0x704	This register contains policing counter number corresponding to each VLAN filter event.
"MAC_LPCSel_IndReg(#i)(for i = 0; i <= DWCXG_L3_L4_FILTER_NUM_PC+1)" on page 853	0x704	This register contains policing counter number corresponding to each L3L4 filter event.
"MAC_EXT_DAH_IndReg(#i)_(#j)(for j = 0; j <= EXT_DAH_ENT_DCNT-1)" on page 854	0x704	The MAC_EXT_DAH_IndReg0_0 registers, which are accessible through MAC_Indir_Access_* register. Each...
"MAC_EXT_ROVTL_IndReg(#i)_(#j)(for j = 0; j <= EXT_RVTL_ENT_DCNT-1)" on page 856	0x704	The MAC_EXT_ROVTL_IndReg0_0 registers, which are accessible through MAC_Indir_Access_* register....
"MAC_EXT_RIVTL_IndReg(#i)_(#j)(for j = 0; j <= EXT_RVTL_ENT_DCNT-1)" on page 859	0x704	The MAC_EXT_RIVTL_IndReg0_0 registers, which are accessible through MAC_Indir_Access_* register....
"MAC_FPCSel_IndReg0" on page 862	0x704	This register contains policing counter number for events other than the Filter Pass events which...
"MAC_PCStatus_DA_IndReg#(i)(for i = 0; i <= DWCXG_ADD_MAC_ADDR_REG_PC_DCNT-1)" on page 863	0x704	This register contains pass status for up to 32 DA/SA filter where each bit corresponds to specific...
"MAC_PCStatus_VLAN_IndReg#(i)(for i = 0; i <= DWCXG_NRVF_PC_DCNT-1)" on page 864	0x704	This register contains pass status for up to 32 VLAN filter where each bit corresponds to specific...
"MAC_PCStatus_L3L4_IndReg#(i)(for i = 0; i <= DWCXG_L3_L4_FILTER_NUM_PC_DCNT-1)" on page 865	0x704	This register contains pass status for up to 32 L3_L4 filter where each bit corresponds to specific...
"MAC_PCStatus_Filter_Event_IndReg" on page 866	0x704	This register contains status for the events monitored using policing counters apart from Filter...
"MAC_L2TLSel_IndReg#(i)(for i = 0; i <= @SNPS_RSVDPARAM_28-1)" on page 867	0x704	This set of registers forms the entries of the L2 2-Tuple Lookup Table for DA/SA & VLAN tuple match...
"MAC_PCTH_Intr_Enable" on page 870	0x708	This register contains threshold based interrupt enable for corresponding policing counters.
"MAC_PCTH_Intr_Status" on page 871	0x70c	This register contains threshold based interrupts for corresponding policing counters.
"MAC_PCTW_Intr_Enable" on page 872	0x730	This register contains enable bits for Time based interrupt.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_PCTW_Intr_Status" on page 873	0x734	This register contains Time window based interrupt status.
"MMC_Control" on page 874	0x800	The MMC Control register establishes the operating mode of the management counters.
"MMC_Rx_Interrupt" on page 877	0x804	The MMC Receive Interrupt Status register maintains the interrupts generated when receive statistic...
"MMC_Tx_Interrupt" on page 884	0x808	The MMC Transmit Interrupt Status register maintains the interrupts generated when the transmit...
"MMC_Receive_Interrupt_Enable" on page 892	0x80c	The MMC Receive Interrupt Enable register maintains the enables for the interrupts generated when...
"MMC_Transmit_Interrupt_Enable" on page 899	0x810	The MMC Transmit Interrupt Enable register maintains the enables for the interrupts generated when...
"Tx_Octet_Count_Good_Bad_Low" on page 907	0x814	This register provides lower 32 bits of transmitted octet count, excluding preamble, in good and...
"Tx_Octet_Count_Good_Bad_High" on page 908	0x818	This register provides upper 32 bits of transmitted octet count, excluding preamble, in good and...
"Tx_Packet_Count_Good_Bad_Low" on page 909	0x81c	This register provides lower 32 bits of transmitted good and bad packet count.
"Tx_Packet_Count_Good_Bad_High" on page 910	0x820	This register provides upper 32 bits of transmitted good and bad packet count.
"Tx_Broadcast_Packets_Good_Low" on page 911	0x824	This register provides lower 32 bits of transmitted good broadcast packets count.
"Tx_Broadcast_Packets_Good_High" on page 912	0x828	This register provides upper 32 bits of transmitted good broadcast packets count.
"Tx_Multicast_Packets_Good_Low" on page 913	0x82c	This register provides lower 32 bits of transmitted good multicast packets count.
"Tx_Multicast_Packets_Good_High" on page 914	0x830	This register provides upper 32 bits of transmitted good multicast packets count.
"Tx_64Octets_Packets_Good_Bad_Low" on page 915	0x834	This register provides lower 32 bits of transmitted 64 octet size good and bad packets count.
"Tx_64Octets_Packets_Good_Bad_High" on page 916	0x838	This register provides upper 32 bits of transmitted 64 octet size good and bad packets count.
"Tx_65To127Octets_Packets_Good_Bad_Low" on page 917	0x83c	This register provides lower 32 bits of transmitted 65-to-127 octet size good and bad packets...
"Tx_65To127Octets_Packets_Good_Bad_High" on page 918	0x840	This register provides upper 32 bits of transmitted 65-to-127 octet size good and bad packets...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"Tx_128To255Octets_Packets_Good_Bad_Low" on page 919	0x844	This register provides lower 32 bits of transmitted 128-to-255 octet size good and bad packets...
"Tx_128To255Octets_Packets_Good_Bad_High" on page 920	0x848	This register provides upper 32 bits of transmitted 128-to-255 octet size good and bad packets...
"Tx_256To511Octets_Packets_Good_Bad_Low" on page 921	0x84c	This register provides lower 32 bits of transmitted 256-to-511 octet size good and bad packets...
"Tx_256To511Octets_Packets_Good_Bad_High" on page 922	0x850	This register provides upper 32 bits of transmitted 256-to-511 octet size good and bad packets...
"Tx_512To1023Octets_Packets_Good_Bad_Low" on page 923	0x854	This register provides lower 32 bits of transmitted 512-to-1023 octet size good and bad packets...
"Tx_512To1023Octets_Packets_Good_Bad_High" on page 924	0x858	This register provides upper 32 bits of transmitted 512-to-1023 octet size good and bad packets...
"Tx_1024ToMaxOctets_Packets_Good_Bad_Low" on page 925	0x85c	This register provides lower 32 bits of transmitted 1024-tomaxsize octet size good and bad packets...
"Tx_1024ToMaxOctets_Packets_Good_Bad_High" on page 926	0x860	This register provides upper 32 bits of transmitted 1024-tomaxsize octet size good and bad packets...
"Tx_Unicast_Packets_Good_Bad_Low" on page 927	0x864	This register provides lower 32 bits of transmitted good bad unicast packets count.
"Tx_Unicast_Packets_Good_Bad_High" on page 928	0x868	This register provides upper 32 bits of transmitted good bad unicast packets count.
"Tx_Multicast_Packets_Good_Bad_Low" on page 929	0x86c	This register provides lower 32 bits of transmitted good bad multicast packets count.
"Tx_Multicast_Packets_Good_Bad_High" on page 930	0x870	This register provides upper 32 bits of transmitted good bad multicast packets count.
"Tx_Broadcast_Packets_Good_Bad_Low" on page 931	0x874	This register provides lower 32 bits of transmitted good bad broadcast packets count.
"Tx_Broadcast_Packets_Good_Bad_High" on page 932	0x878	This register provides upper 32 bits of transmitted good bad broadcast packets count.
"Tx_Underflow_Error_Packets_Low" on page 933	0x87c	This register provides lower 32 bits of transmitted packets aborted due to packet underflow error...
"Tx_Underflow_Error_Packets_High" on page 934	0x880	This register provides upper 32 bits of transmitted packets aborted due to packet underflow error...
"Tx_Octet_Count_Good_Low" on page 935	0x884	This register provides lower 32 bits of transmitted good octets count.
"Tx_Octet_Count_Good_High" on page 936	0x888	This register provides upper 32 bits of transmitted good octets count.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"Tx_Packet_Count_Good_Low" on page 937	0x88c	This register provides lower 32 bits of transmitted good packets count.
"Tx_Packet_Count_Good_High" on page 938	0x890	This register provides upper 32 bits of transmitted good packets count.
"Tx_Pause_Packets_Low" on page 939	0x894	This register provides lower 32 bits of transmitted Pause packets count.
"Tx_Pause_Packets_High" on page 940	0x898	This register provides upper 32 bits of transmitted Pause packets count.
"Tx_VLAN_Packets_Good_Low" on page 941	0x89c	This register provides lower 32 bits of transmitted good VLAN packets count.
"Tx_VLAN_Packets_Good_High" on page 942	0x8a0	This register provides upper 32 bits of transmitted good VLAN packets count.
"Tx_LPI_USEC_Cntr" on page 943	0x8a4	This register provides duration in microsecond for which MAC was in Tx LPI.
"Tx_LPI_Tran_Cntr" on page 944	0x8a8	This register provides number of times MAC transitioned to Tx LPI state.
"Priority_Interrupt_Status" on page 945	0x8cc	This register provides the Per Priority MMC Interrupt Status information.
"Tx_Per_Priority_Status" on page 946	0x8d0	This register indicates which Tx Per Priority Counters of the selected priority (PRMMCSEL field...
"Tx_Per_Priority_Pkt_Good_Bad" on page 948	0x8d4	This register provides transmitted per priority good bad packets count.
"Tx_Per_Priority_PFC_Pkt_Good_Bad" on page 949	0x8d8	This register provides transmitted per priority good bad PFC packets count.
"Tx_Per_Priority_GPFC_Pkt_Good_Bad" on page 950	0x8dc	This register provides transmitted per priority good bad generated PFC packets count.
"Tx_Per_Priority_Octet_Good_Bad_Low" on page 951	0x8e0	This register provides lower 32 bits of transmitted per priority good bad octets count.
"Tx_Per_Priority_Octet_Good_Bad_High" on page 952	0x8e4	This register provides upper 32 bits of transmitted per priority good bad octets count.
"Rx_Packet_Count_Good_Bad_Low" on page 953	0x900	This register provides lower 32 bits of received good and bad packets count.
"Rx_Packet_Count_Good_Bad_High" on page 954	0x904	This register provides upper 32 bits of received good and bad packets count.
"Rx_Octet_Count_Good_Bad_Low" on page 955	0x908	This register provides lower 32 bits of received octet count, excluding preamble, in good and bad...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"Rx_Octet_Count_Good_Bad_High" on page 956	0x90c	This register provides upper 32 bits of received octet count, excluding preamble, in good and bad...
"Rx_Octet_Count_Good_Low" on page 957	0x910	This register provides lower 32 bits of received good octets count.
"Rx_Octet_Count_Good_High" on page 958	0x914	This register provides upper 32 bits of received good octets count.
"Rx_Broadcast_Packets_Good_Low" on page 959	0x918	This register provides lower 32 bits of received good broadcast packets count.
"Rx_Broadcast_Packets_Good_High" on page 960	0x91c	This register provides upper 32 bits of received good broadcast packets count.
"Rx_Multicast_Packets_Good_Low" on page 961	0x920	This register provides lower 32 bits of received good multicast packets count.
"Rx_Multicast_Packets_Good_High" on page 962	0x924	This register provides upper 32 bits of received good multicast packets count.
"Rx_CRC_Error_Packets_Low" on page 963	0x928	This register provides lower 32 bits of received packets with CRC error count.
"Rx_CRC_Error_Packets_High" on page 964	0x92c	This register provides upper 32 bits of received packets with CRC error count.
"Rx_Runt_Error_Packets" on page 965	0x930	This register provides received packets with Runt error count.
"Rx_Jabber_Error_Packets" on page 966	0x934	This register provides received packets with Jabber error count.
"Rx_Undersize_Packets_Good" on page 967	0x938	This register provides count of received good undersize packets.
"Rx_Oversize_Packets_Good" on page 968	0x93c	This register provides count of received good oversize packets.
"Rx_64Octets_Packets_Good_Bad_Low" on page 969	0x940	This register provides lower 32 bits of received 64 octet size good and bad packets count.
"Rx_64Octets_Packets_Good_Bad_High" on page 970	0x944	This register provides upper 32 bits of received 64 octet size good and bad packets count.
"Rx_65To127Octets_Packets_Good_Bad_Low" on page 971	0x948	This register provides lower 32 bits of received 65-to-127 octet size good and bad packets...
"Rx_65To127Octets_Packets_Good_Bad_High" on page 972	0x94c	This register provides upper 32 bits of received 65-to-127 octet size good and bad packets...
"Rx_128To255Octets_Packets_Good_Bad_Low" on page 973	0x950	This register provides lower 32 bits of received 128-to-255 octet size good and bad packets...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"Rx_128To255Octets_Packets_Good_Bad_High" on page 974	0x954	This register provides upper 32 bits of received 128-to-255 octet size good and bad packets...
"Rx_256To511Octets_Packets_Good_Bad_Low" on page 975	0x958	This register provides lower 32 bits of received 256-to-511 octet size good and bad packets...
"Rx_256To511Octets_Packets_Good_Bad_High" on page 976	0x95c	This register provides upper 32 bits of received 256-to-511 octet size good and bad packets...
"Rx_512To1023Octets_Packets_Good_Bad_Low" on page 977	0x960	This register provides lower 32 bits of received 512-to-1023 octet size good and bad packets...
"Rx_512To1023Octets_Packets_Good_Bad_High" on page 978	0x964	This register provides upper 32 bits of received 512-to-1023 octet size good and bad packets...
"Rx_1024ToMaxOctets_Packets_Good_Bad_Low" on page 979	0x968	This register provides lower 32 bits of received 1024-tomaxsize octet size good and bad packets...
"Rx_1024ToMaxOctets_Packets_Good_Bad_High" on page 980	0x96c	This register provides upper 32 bits of received 1024-tomaxsize octet size good and bad packets...
"Rx_Unicast_Packets_Good_Low" on page 981	0x970	This register provides lower 32 bits of received good unicast packets count.
"Rx_Unicast_Packets_Good_High" on page 982	0x974	This register provides upper 32 bits of received good unicast packets count.
"Rx_Length_Error_Packets_Low" on page 983	0x978	This register provides lower 32 bits of received packets with Length error count.
"Rx_Length_Error_Packets_High" on page 984	0x97c	This register provides upper 32 bits of received packets with Length error count.
"Rx_OutofRange_Packets_Low" on page 985	0x980	This register provides lower 32 bits of received packets with out of range length field count.
"Rx_OutofRange_Packets_High" on page 986	0x984	This register provides upper 32 bits of received packets with out of range length field count.
"Rx_Pause_Packets_Low" on page 987	0x988	This register provides lower 32 bits of received Pause packets count.
"Rx_Pause_Packets_High" on page 988	0x98c	This register provides upper 32 bits of received Pause packets count.
"Rx_FIFOOverflow_Packets_Low" on page 989	0x990	This register provides lower 32 bits of received packets count that had Rx Queue overflow.
"Rx_FIFOOverflow_Packets_High" on page 990	0x994	This register provides upper 32 bits of received packets count that had Rx Queue overflow.
"Rx_VLAN_Packets_Good_Bad_Low" on page 991	0x998	This register provides lower 32 bits of received good bad VLAN packets count.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"Rx_VLAN_Packets_Good_Bad_High" on page 992	0x99c	This register provides upper 32 bits of received good bad VLAN packets count.
"Rx_Watchdog_Error_Packets" on page 993	0x9a0	This register has a count of packets that are received with watchdog error.
"Rx_LPI_USEC_Cntr" on page 994	0x9a4	This register provides duration in microsecond for which MAC was in Rx LPI.
"Rx_LPI_Tran_Cntr" on page 995	0x9a8	This register provides number of times MAC transitioned to Rx LPI state.
"Rx_Discard_Packets_Good_Bad_Low" on page 996	0x9ac	This register provides lower 32 bits of received good bad packets count that were discarded due...
"Rx_Discard_Packets_Good_Bad_High" on page 997	0x9b0	This register provides upper 32 bits of received good bad packets count that were discarded due...
"Rx_Discard_Octets_Good_Bad_Low" on page 998	0x9b4	This register provides lower 32 bits of received good bad octets count that were discarded due to...
"Rx_Discard_Octets_Good_Bad_High" on page 999	0x9b8	This register provides upper 32 bits of received good bad octets count that were discarded due to...
"Rx_Alignment_Error_Packets" on page 1000	0x9bc	This register provides the number of packets received by DWC_25gmac with alignment (dribble) error....
"RMMC_RCHM_SKP_CNTR_LOW" on page 1001	0x9c8	This register provides lower 32 bits of skip counts of the Rx Channel Mapped Rx Frame/Priority MMC....
"RMMC_RCHM_SKP_CNTR_HIGH" on page 1002	0x9cc	This register provides upper 32 bits of skip counts of the Rx Channel Mapped Rx Frame/Priority MMC....
"Rx_Per_Priority_Status" on page 1003	0x9d0	This register indicates which Rx Per Priority Counters of the selected priority (PRMMCSEL field...
"Rx_Per_Priority_Pkt_Good_Bad" on page 1005	0x9d4	This register provides received per priority good bad packets count.
"Rx_Per_Priority_Pkt_Bad" on page 1006	0x9d8	This register provides received per priority bad packets count.
"Rx_Per_Priority_PFC_Pkt_Good_Bad" on page 1007	0x9dc	This register provides received per priority good bad PFC packets count.
"Rx_Per_Priority_Octet_Good_Bad_Low" on page 1008	0x9e0	This register provides lower 32 bits of received per priority good bad octets count.
"Rx_Per_Priority_Octet_Good_Bad_High" on page 1009	0x9e4	This register provides upper 32 bits of received per priority good bad octets count.
"Rx_Per_Priority_Discard_Good_Bad" on page 1010	0x9e8	This register provides received per priority discarded good bad packets count.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"SGF_Pass_PktCnt" on page 1011	0x9f0	This register provides the number of Stream-Gate Filter (SGF) Passed Packets
"SGF_Fail_PktCnt" on page 1012	0x9f4	This register provides the number of SGF Failed Packets
"MMC_FPE_Tx_Interrupt" on page 1013	0xa00	This register maintains the interrupts generated from all FPE related Transmit statistics counters....
"MMC_FPE_Tx_Interrupt_Mask" on page 1015	0xa04	This register maintains the masks for interrupts generated from all FPE related Transmit statistics...
"MMC_Tx_FPE_Fragment_Cntr" on page 1017	0xa08	This register provides the number of additional mPackets transmitted due to preemption.
"MMC_Tx_Hold_Req_Cntr" on page 1018	0xa0c	This register provides the count of number of times a hold request is given to MAC
"MMC_Tx_Gate_Orun_Cntr_Low" on page 1019	0xa10	This register provides the lower 32-bits of counter that counts the number of times packets overrun...
"MMC_Tx_Gate_Orun_Cntr_High" on page 1020	0xa14	This register provides the upper 32-bits of counter that counts the number of times packets overrun...
"MMC_FPE_Rx_Interrupt" on page 1021	0xa20	This register maintains the interrupts generated from all FPE related Receive statistics counters....
"MMC_FPE_Rx_Interrupt_Mask" on page 1024	0xa24	This register maintains the masks for interrupts generated from all FPE related Receive statistics...
"MMC_Rx_Packet_Assembly_Err_Cntr" on page 1026	0xa28	This register provides the number of MAC frames with reassembly errors on the Receiver, due to mismatch...
"MMC_Rx_Packet_SMD_Err_Cntr" on page 1027	0xa2c	This register provides the number of received MAC frames rejected due to unknown SMD value and MAC...
"MMC_Rx_Packet_Assembly_OK_Cntr" on page 1028	0xa30	This register provides the number of MAC frames that were successfully reassembled and delivered...
"MMC_Rx_FPE_Fragment_Cntr" on page 1029	0xa34	This register provides the number of additional mPackets received due to preemption.
"Tx_Single_Collision_Good_Packets" on page 1030	0xa40	This register provides the number of successfully transmitted packets by DWC_25gmac after a single...
"Tx_Multiple_Collision_Good_Packets" on page 1031	0xa44	This register provides the number of successfully transmitted packets by DWC_25gmac after multiple...
"Tx_Deferred_Packets" on page 1032	0xa48	This register provides the number of successfully transmitted by DWC_25gmac after a deferral in...
"Tx_Late_Collision_Packets" on page 1033	0xa4c	This register provides the number of packets aborted by DWC_25gmac because of late collision...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"Tx_Excessive_Collision_Packets" on page 1034	0xa50	This register provides the number of packets aborted by DWC_25gmac because of excessive (16) collision...
"Tx_Carrier_Error_Packets" on page 1035	0xa54	This register provides the number of packets aborted by DWC_25gmac because of carrier sense error...
"Tx_Excessive_Deferral_Error" on page 1036	0xa58	This register provides the number of packets aborted by DWC_25gmac because of excessive deferral...
"MMC_IPC_Rx_Interrupt_Mask" on page 1037	0xa5c	This register maintains the mask for the interrupt generated from the receive IPC statistic counters....
"MMC_IPC_Rx_Interrupt" on page 1048	0xa60	This register maintains the interrupt that the receive IPC statistic counters generate. The MMC...
"RxIPv4_Good_Packets_Low" on page 1059	0xa64	This register provides lower 32 bits of good IPv4 datagrams received by DWC_25gmac with the TCP,...
"RxIPv4_Good_Packets_High" on page 1060	0xa68	This register provides upper 32 bits of good IPv4 datagrams received by DWC_25gmac with the TCP,...
"RxIPv4_Header_Error_Packets_Low" on page 1061	0xa6c	RxIPv4 Header Error Packets This register provides lower 32 bits of IPv4 datagrams received by...
"RxIPv4_Header_Error_Packets_High" on page 1062	0xa70	RxIPv4 Header Error Packets This register provides upper 32 bits of IPv4 datagrams received by...
"RxIPv4_No_Payload_Packets_Low" on page 1063	0xa74	This register provides lower 32 bits of IPv4 datagram packets received by DWC_25gmac that did not...
"RxIPv4_No_Payload_Packets_High" on page 1064	0xa78	This register provides upper 32 bits of IPv4 datagram packets received by DWC_25gmac that did not...
"RxIPv4_Fragmented_Packets_Low" on page 1065	0xa7c	This register provides lower 32 bits of good IPv4 datagrams received by DWC_25gmac with...
"RxIPv4_Fragmented_Packets_High" on page 1066	0xa80	This register provides upper 32 bits of good IPv4 datagrams received by DWC_25gmac with...
"RxIPv4_UDP_Checksum_Disabled_Packets_Low" on page 1067	0xa84	This register provides lower 32 bits of good IPv4 datagrams received by DWC_25gmac that had a UDP...
"RxIPv4_UDP_Checksum_Disabled_Packets_High" on page 1068	0xa88	This register provides upper 32 bits of good IPv4 datagrams received by DWC_25gmac that had a UDP...
"RxIPv6_Good_Packets_Low" on page 1069	0xa8c	This register provides lower 32 bits of good IPv6 datagrams received by DWC_25gmac with the TCP,...
"RxIPv6_Good_Packets_High" on page 1070	0xa90	This register provides upper 32 bits of good IPv6 datagrams received by DWC_25gmac with the TCP,...
"RxIPv6_Header_Error_Packets_Low" on page 1071	0xa94	This register provides lower 32 bits of IPv6 datagrams received by DWC_25gmac with header (length...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"RxIPv6_Header_Error_Packets_High" on page 1072	0xa98	This register provides upper 32 bits of IPv6 datagrams received by DWC_25gmac with header (length...
"RxIPv6_No_Payload_Packets_Low" on page 1073	0xa9c	This register provides lower 32 bits of IPv6 datagram packets received by DWC_25gmac that did not...
"RxIPv6_No_Payload_Packets_High" on page 1074	0xaa0	This register provides upper 32 bits of IPv6 datagram packets received by DWC_25gmac that did not...
"RxUDP_Good_Packets_Low" on page 1075	0xaa4	This register provides lower 32 bits of good IP datagrams received by DWC_25gmac with a good UDP...
"RxUDP_Good_Packets_High" on page 1076	0xaa8	This register provides upper 32 bits of good IP datagrams received by DWC_25gmac with a good UDP...
"RxUDP_Error_Packets_Low" on page 1077	0xaac	This register provides lower 32 bits of good IP datagrams received by DWC_25gmac whose UDP payload...
"RxUDP_Error_Packets_High" on page 1078	0xab0	This register provides upper 32 bits of good IP datagrams received by DWC_25gmac whose UDP payload...
"RxTCP_Good_Packets_Low" on page 1079	0xab4	This register provides lower 32 bits of good IP datagrams received by DWC_25gmac with a good TCP...
"RxTCP_Good_Packets_High" on page 1080	0xab8	This register provides upper 32 bits of good IP datagrams received by DWC_25gmac with a good TCP...
"RxTCP_Error_Packets_Low" on page 1081	0xabc	This register provides lower 32 bits of good IP datagrams received by DWC_25gmac whose TCP payload...
"RxTCP_Error_Packets_High" on page 1082	0xac0	This register provides upper 32 bits of good IP datagrams received by DWC_25gmac whose TCP payload...
"RxICMP_Good_Packets_Low" on page 1083	0xac4	This register provides lower 32 bits of good IP datagrams received by DWC_25gmac with a good ICMP...
"RxICMP_Good_Packets_High" on page 1084	0xac8	This register provides upper 32 bits of good IP datagrams received by DWC_25gmac with a good ICMP...
"RxICMP_Error_Packets_Low" on page 1085	0xacc	This register provides lower 32 bits of good IP datagrams received by DWC_25gmac whose ICMP payload...
"RxICMP_Error_Packets_High" on page 1086	0xad0	This register provides upper 32 bits of good IP datagrams received by DWC_25gmac whose ICMP payload...
"RxIPv4_Good_Octets_Low" on page 1087	0xad4	This register provides lower 32 bits of bytes received by DWC_25gmac in good IPv4 datagrams encapsulating...
"RxIPv4_Good_Octets_High" on page 1088	0xad8	This register provides upper 32 bits of bytes received by DWC_25gmac in good IPv4 datagrams encapsulating...
"RxIPv4_Header_Error_Octets_Low" on page 1089	0xadc	This register provides lower 32 bits of bytes received by DWC_25gmac in IPv4 datagrams with header...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"RxIPv4_Header_Error_Octets_High" on page 1090	0xae0	This register provides upper 32 bits of bytes received by DWC_25gmac in IPv4 datagrams with header...
"RxIPv4_No_Payload_Octets_Low" on page 1091	0xae4	This register provides lower 32 bits of bytes received by DWC_25gmac in IPv4 datagrams that did...
"RxIPv4_No_Payload_Octets_High" on page 1092	0xae8	This register provides upper 32 bits of bytes received by DWC_25gmac in IPv4 datagrams that did...
"RxIPv4_Fragmented_Octets_Low" on page 1093	0xaec	This register provides lower 32 bits of bytes received by DWC_25gmac in fragmented IPv4 datagrams....
"RxIPv4_Fragmented_Octets_High" on page 1094	0xaf0	This register provides upper 32 bits of bytes received by DWC_25gmac in fragmented IPv4 datagrams....
"RxIPv4_UDP_Checksum_Disable_Octets_Low" on page 1095	0xaf4	This register provides lower 32 bits of bytes received by DWC_25gmac in a UDP segment that had the...
"RxIPv4_UDP_Checksum_Disable_Octets_High" on page 1096	0xaf8	This register provides upper 32 bits of bytes received by DWC_25gmac in a UDP segment that had the...
"RxIPv6_Good_Octets_Low" on page 1097	0xafc	This register provides lower 32 bits of bytes received by DWC_25gmac in good IPv6 datagrams encapsulating...
"RxIPv6_Good_Octets_High" on page 1098	0xb00	This register provides upper 32 bits of bytes received by DWC_25gmac in good IPv6 datagrams encapsulating...
"RxIPv6_Header_Error_Octets_Low" on page 1099	0xb04	This register provides lower 32 bits of bytes received by DWC_25gmac in IPv6 datagrams with header...
"RxIPv6_Header_Error_Octets_High" on page 1100	0xb08	This register provides upper 32 bits of bytes received by DWC_25gmac in IPv6 datagrams with header...
"RxIPv6_No_Payload_Octets_Low" on page 1101	0xb0c	This register provides lower 32 bits of bytes received by DWC_25gmac in IPv6 datagrams that did...
"RxIPv6_No_Payload_Octets_High" on page 1102	0xb10	This register provides upper 32 bits of bytes received by DWC_25gmac in IPv6 datagrams that did...
"RxUDP_Good_Octets_Low" on page 1103	0xb14	This register provides lower 32 bits of bytes received by DWC_25gmac in a good UDP segment. This...
"RxUDP_Good_Octets_High" on page 1104	0xb18	This register provides upper 32 bits of bytes received by DWC_25gmac in a good UDP segment. This...
"RxUDP_Error_Octets_Low" on page 1105	0xb1c	This register provides lower 32 bits of bytes received by DWC_25gmac in a UDP segment that had checksum...
"RxUDP_Error_Octets_High" on page 1106	0xb20	This register provides upper 32 bits of bytes received by DWC_25gmac in a UDP segment that had checksum...
"RxTCP_Good_Octets_Low" on page 1107	0xb24	This register provides lower 32 bits of bytes received by DWC_25gmac in a good TCP segment. This...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"RxTCP_Good_Octets_High" on page 1108	0xb28	This register provides upper 32 bits of bytes received by DWC_25gmac in a good TCP segment. This...
"RxTCP_Error_Octets_Low" on page 1109	0xb2c	This register provides lower 32 bits of bytes received by DWC_25gmac in a TCP segment that had checksum...
"RxTCP_Error_Octets_High" on page 1110	0xb30	This register provides upper 32 bits of bytes received by DWC_25gmac in a TCP segment that had checksum...
"RxICMP_Good_Octets_Low" on page 1111	0xb34	This register provides lower 32 bits of bytes received by DWC_25gmac in a good ICMP segment. This...
"RxICMP_Good_Octets_High" on page 1112	0xb38	This register provides upper 32 bits of bytes received by DWC_25gmac in a good ICMP segment. This...
"RxICMP_Error_Octets_Low" on page 1113	0xb3c	This register provides lower 32 bits of bytes received by DWC_25gmac in a ICMP segment that had...
"RxICMP_Error_Octets_High" on page 1114	0xb40	This register provides upper 32 bits of bytes received by DWC_25gmac in a ICMP segment that had...
"RMMC_IPC_RCHM_SKP_CNTR_LOW" on page 1115	0xb44	This register provides lower 32 bits of skip counts of the Rx Channel Mapped Rx IPC MMC. Basically...
"RMMC_IPC_RCHM_SKP_CNTR_HIGH" on page 1116	0xb48	This register provides upper 32 bits of skip counts of the Rx Channel Mapped Rx IPC MMC. Basically...
"MAC_L3_L4_Address_Control" on page 1117	0xc00	This register has the indirect address and control for accessing the Layer 3 and Layer 4 Filter...
"MAC_L3_L4_Data" on page 1119	0xc04	This register has the indirect data and control for accessing the Layer 3 and Layer 4 Filter...
"MAC_L3_L4_Control#(i)(for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)" on page 1120	0xc04	The Layer 3 and Layer 4 Control register controls the operations of filter 0 of Layer 3 and Layer...
"MAC_Layer4_Address#(i)(for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)" on page 1126	0xc04	Layer 4 Address 0 register has the Layer 4 (TCP, UDP) Source and Destination Port fields checked...
"MAC_Layer3_Addr0_Reg#(i)(for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)" on page 1127	0xc04	For IPv4 packets, the Layer 3 Address 0 Register 0 register contains the 32-bit IP Source Address...
"MAC_Layer3_Addr1_Reg#(i)(for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)" on page 1128	0xc04	For IPv4 packets, the Layer 3 Address 1 Register 0 register contains the 32-bit IP Destination Address...
"MAC_Layer3_Addr2_Reg#(i)(for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)" on page 1129	0xc04	The Layer 3 Address 2 Register 0 register is reserved for IPv4 packets. For IPv6 packets, it contains...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_Layer3_Addr3_Reg#(i)(for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)" on page 1130	0xc04	The Layer 3 Address 3 Register 0 register is reserved for IPv4 packets. For IPv6 packets, it contains...
"MAC_ARP_Address" on page 1131	0xc10	The ARP Address register contains the IPv4 Destination Address of the MAC. This register is available...
"MAC_RSS_Control" on page 1132	0xc80	This register controls the RSS Hash function. This register is present when Enable support for Virtualized...
"MAC_RSS_Address" on page 1135	0xc88	This register contains the address control bits to indirectly access the RSS Hash Key or the RSS...
"MAC_RSS_Data" on page 1138	0xc8c	This register has the data to be written into or read from the RSS Lookup Table or the RSS Hash...
"RSS_Lookup_Data#(i)(for i = 0; i <= DWCXG_RSS_TABLE_SIZE-1)" on page 1139	0xc8c	The format of each location in the RSS Lookup Table is given in the following table. The size of...
"RSS_Hash_Key(#i)(for i = 0; i <= 9)" on page 1141	0xc8c	The RSS Hash Key to be used in the RSS Hash calculation is 40 bytes. This is organized as 10 registers...
"MAC_RSS_IPV4UDP_4TMBIT#(i)(for i = 0; i <= 2)" on page 1142	0xc8c	This is an indirect register. Each 32-bit data word of the register is accessible through MAC_RSS_*...
"MAC_RSS_IPV6UDP_4TMBIT#(i)(for i = 0; i <= 8)" on page 1143	0xc8c	This is an indirect register. Each 32-bit data word of the register is accessible through MAC_RSS_*...
"MAC_RSS_IPV4TCP_4TMBIT#(i)(for i = 0; i <= 2)" on page 1144	0xc8c	This is an indirect register, accessible through MAC_RSS_* register. It contains mask bits for RSS...
"MAC_RSS_IPV6TCP_4TMBIT#(i)(for i = 0; i <= 8)" on page 1145	0xc8c	This is an indirect register. Each 32-bit data word of the register is accessible through MAC_RSS_*...
"MAC_RSS_IPV4UP_2TMBIT#(i)(for i = 0; i <= 1)" on page 1146	0xc8c	This is an indirect register, accessible through MAC_RSS_* register. It contains mask bits for RSS...
"MAC_RSS_IPV6UP_2TMBIT#(i)(for i = 0; i <= 7)" on page 1147	0xc8c	This is an indirect register, accessible through MAC_RSS_* register. It contains ${MSB_mask}:${LSB_mask}...
"MAC_RSS_TUNNEL_TMBIT#(i)(for i = 0; i <= 2)" on page 1148	0xc8c	This is an indirect register. Each 32-bit data word of the register is accessible through MAC_RSS_*...
"MAC_Timestamp_Control" on page 1149	0xd00	This register controls the operation of the System Time generator and processing of PTP packets...
"MAC_Sub_Second_Increment" on page 1156	0xd04	The Sub-Second Increment register is present only when the IEEE 1588 timestamp feature is selected...
"MAC_System_Time_Seconds" on page 1158	0xd08	The System Time-Seconds register, along with System Time-Nanoseconds register, indicates the current...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_System_Time_Nanoseconds" on page 1159	0xd0c	The System Time-Nanoseconds register, along with System Time-Seconds register, indicates the current...
"MAC_System_Time_Seconds_Update" on page 1160	0xd10	The System Time-Seconds Update register, along with the System Time-Nanoseconds Update register,...
"MAC_System_Time_Nanoseconds_Update" on page 1161	0xd14	The System Time-Nanoseconds Update register, along with the System Time-Seconds Update register,...
"MAC_Timestamp_Addend" on page 1162	0xd18	The Timestamp Addend register is present only when the IEEE 1588 Timestamp feature is selected without...
"MAC_System_Time_Higher_Word_Seconds" on page 1163	0xd1c	This register contains the most-significant 16-bits of timestamp seconds value.The System Time -...
"MAC_Timestamp_Status" on page 1164	0xd20	The Timestamp Status registeris present only when the IEEE 1588 Timestamp feature is selected. All...
"MAC_Rx_Domain_Time_Incr" on page 1170	0xd24	This register indicates the increment value of the PTP time maintained in Receive clock domain that...
"MAC_Tx_Domain_Time_Incr" on page 1172	0xd28	This register indicates the increment value of the PTP time maintained in Transmit clock domain...
"MAC_Rx_PCH_CRC_Mismatch_Counter" on page 1174	0xd2c	The PCH CRC Mismatch Counter for Debugging purpose.
"MAC_Tx_Timestamp_Status_Nanoseconds" on page 1175	0xd30	The MAC_TxTimestamp_Status_Nanoseconds register, along with MAC_TxTimestamp_Status_Seconds, gives...
"MAC_Tx_Timestamp_Status_Seconds" on page 1176	0xd34	The register contains the higher 32 bits of the timestamp (in seconds) captured when a PTP packet...
"MAC_Tx_Timestamp_Status_PktID" on page 1177	0xd38	This register contains the PTP packet ID corresponding to the Tx Timestamp Status in MAC_TxTimestamp_Status_Nanoseconds...
"MAC_Auxiliary_Control" on page 1179	0xd40	The Auxiliary Timestamp Control register controls the Auxiliary Timestamp snapshot.
"MAC_Auxiliary_Timestamp_Nanoseconds" on page 1181	0xd48	The Auxiliary Timestamp - Nanoseconds register, along with MAC_Auxiliary_Timestamp_Seconds, gives...
"MAC_Auxiliary_Timestamp_Seconds" on page 1182	0xd4c	The Auxiliary Timestamp - Seconds register contains the lower 32 bits of the Seconds field of the...
"MAC_Timestamp_Ingress_Asym_Corr" on page 1183	0xd50	The MAC Timestamp Ingress Asymmetry Correction register contains the Ingress Asymmetry Correction...
"MAC_Timestamp_Egress_Asym_Corr" on page 1184	0xd54	The MAC Timestamp Egress Asymmetry Correction register contains the Egress Asymmetry Correction...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_Timestamp_Ingress_Corr_Nanosecond" on page 1185	0xd58	The MAC Timestamp Ingress Correction Nanosecond register contains the correction value to be used...
"MAC_Timestamp_Ingress_Corr_Subnanosecond" on page 1186	0xd5c	The MAC Timestamp Ingress Correction Sub-Nanosecond register contains the sub-nanosecond part of...
"MAC_Timestamp_Egress_Corr_Nanosecond" on page 1187	0xd60	The MAC Egress Timestamp Correction Nanosecond register contains the correction value to be used...
"MAC_Timestamp_Egress_Corr_Subnanosecond" on page 1188	0xd64	The MAC Timestamp Correction Egress Sub-nanosecond register contains the sub-nanosecond part of...
"MAC_PPS_Control" on page 1189	0xd70	The PPS Control register is present only when the Timestamp feature is selected and External Timestamp...
"MAC_PPS_Extnd_Ctrl" on page 1196	0xd74	PPS Extended Control register. This register contains the additional controls for the PPS...
"MAC_PPS0_Target_Time_Seconds" on page 1197	0xd80	The PPS0 Target Time Seconds register, along with PPS0 Target Time Nanoseconds register, is used...
"MAC_PPS0_Target_Time_Nanoseconds" on page 1198	0xd84	The PPS0 Target Time Nanoseconds register is present only when more than one Flexible PPS output...
"MAC_PPS0_Interval" on page 1200	0xd88	The PPS0 Interval register contains the number of units of sub-second increment value between the...
"MAC_PPS0_Width" on page 1201	0xd8c	The PPS0 Width register contains the number of units of sub-second increment value between the rising...
"MAC_PPS1_Target_Time_Seconds" on page 1202	0xd90	The PPS1 Target Time Seconds register, along with PPS1 Target Time Nanoseconds register, is used...
"MAC_PPS1_Target_Time_Nanoseconds" on page 1203	0xd94	The PPS1 Target Time Nanoseconds register, along with PPS1 Target Time Seconds register, is used...
"MAC_PPS1_Interval" on page 1205	0xd98	The PPS1 Interval register contains the number of units of sub-second increment value between the...
"MAC_PPS1_Width" on page 1206	0xd9c	The PPS1 Width register contains the number of units of sub-second increment value between the rising...
"MAC_PPS2_Target_Time_Seconds" on page 1207	0xda0	The PPS2 Target Time Seconds register, along with PPS2 Target Time Nanoseconds register, is used...
"MAC_PPS2_Target_Time_Nanoseconds" on page 1208	0xda4	This register is present only when more than two Flexible PPS output is selected while configuring...
"MAC_PPS2_Interval" on page 1209	0xda8	The PPS2 Interval register contains the number of units of sub-second increment value between the...
"MAC_PPS2_Width" on page 1210	0xdac	The PPS2 Width register contains the number of units of sub-second increment value between the rising...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_PPS3_Target_Time_Seconds" on page 1211	0xdb0	The PPS3 Target Time Seconds register, along with PPS3 Target Time Nanoseconds register, is used...
"MAC_PPS3_Target_Time_Nanoseconds" on page 1212	0xdb4	This register is present only when more than three Flexible PPS output is selected while configuring...
"MAC_PPS3_Interval" on page 1213	0xdb8	The PPS3 Interval register contains the number of units of sub-second increment value between the...
"MAC_PPS3_Width" on page 1214	0xdbc	The PPS3 Width register contains the number of units of sub-second increment value between the rising...
"MAC_PTO_Control" on page 1215	0xdc0	This register controls the PTP Offload Engine operation. This register is available only when the...
"MAC_Source_Port_Identity0" on page 1218	0xdc4	This register along with MAC_Source_Port_Identity1 and MAC_Source_Port_Identity2 contains the 80-bit...
"MAC_Source_Port_Identity1" on page 1219	0xdc8	This register along with MAC_Source_Port_Identity0 and MAC_Source_Port_Identity2 contains the 80-bit...
"MAC_Source_Port_Identity2" on page 1220	0xdcc	This register along with MAC_Source_Port_Identity0 and MAC_Source_Port_Identity1 contains the 80-bit...
"MAC_Log_Message_Interval" on page 1221	0xdd0	This register contains the periodic intervals for automatic PTP packet generation. This register...
"MAC_Presn_Time_ns" on page 1223	0xde0	This register contains the 32-bit binary rollover equivalent time of the PTP System Time in...
"MAC_Presn_Time_Updt" on page 1224	0xde4	This field holds the 32-bit value of MAC 1722 Presentation Time in ns, that should be added to the...
"MAC_Local_Timestamp_Control" on page 1225	0xe00	This register controls the operation of the Local System Time generator and processing of PTP packets...
"MAC_Local_Sub_Second_Increment" on page 1228	0xe04	The Local Sub-Second Increment register is present only when the Dual Time Source Domain Capability...
"MAC_Local_System_Time_Seconds" on page 1230	0xe08	The Local System Time-Seconds register, along with Local System Time-Nanoseconds register, indicates...
"MAC_Local_System_Time_Nanoseconds" on page 1231	0xe0c	The Local System Time-Nanoseconds register, along with Local System Time-Seconds register, indicates...
"MAC_Local_System_Time_Seconds_Update" on page 1232	0xe10	The Local System Time-Seconds Update register, along with the Local System Time-Nanoseconds Update...
"MAC_Local_System_Time_Nanoseconds_Update" on page 1233	0xe14	The Local System Time-Nanoseconds Update register, along with the Local System Time-Seconds Update...
"MAC_Local_System_Time_Higher_Word_Seconds" on page 1235	0xe1c	This register contains the most-significant 16-bits of local timestamp seconds value.The Local System...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MAC_Dual_TS_Src_Ctrl" on page 1236	0xe24	MAC Dual Timestamp Source Control. This register contains the per timestamp function clock selection...
"MAC_Local_Presn_Time_ns" on page 1239	0xe30	This register contains the 32-bit binary rollover equivalent time of the PTP Local System Time in...
"MAC_Local_Presn_Time_Updt" on page 1240	0xe34	This field holds the 32-bit value of MAC Local 1722 Presentation Time in ns, that should be added...
Mac Transaction Layer (MTL) Common/General Registers. DWCXG_MTL Exists: (DWCXG>0)		
"MTL_Operation_Mode" on page 1241	0x1000	The Operation Mode register establishes the Transmit and Receive operating modes and commands.
"MTL_Debug_Control" on page 1244	0x1008	The MTL Debug Control register controls the operation mode of debug access to the TXFIFO, RXFIFO...
"MTL_Debug_Status" on page 1249	0x100c	The Debug Status register contains the state of the transactions to FIFO memory in Debug or Subordinate...
"MTL_FIFO_Debug_Data" on page 1251	0x1010	The FIFO Debug Data register contains the data to be written to or read from the FIFOs. Note: ...
"MTL_Interrupt_Status" on page 1252	0x1020	The software driver (application) reads this register during interrupt service routine or polling...
"MTL_RxQ_DMA_Map0" on page 1259	0x1030	The Receive Queue and DMA Channel Mapping 0 register controls the static or dynamic mapping of Receive...
"MTL_RxQ_DMA_Map1" on page 1263	0x1034	The Receive Queue and DMA Channel Mapping 1 register controls the static or dynamic mapping of Receive...
"MTL_RxQ_DMA_Map2" on page 1267	0x1038	The Receive Queue and DMA Channel Mapping 2 register controls the static or dynamic mapping of Receive...
"MTL_RxQ_DMA_Map3" on page 1271	0x103c	The Receive Queue and DMA Channel Mapping 3 register controls the static or dynamic mapping of Receive...
"MTL_TC_Prty_Map0" on page 1275	0x1040	This register contains the priority values assigned to traffic classes 0 to 3.
"MTL_TC_Prty_Map1" on page 1276	0x1044	This register contains the priority values assigned to traffic classes 4 to 7.
"MTL_TBS_CTRL" on page 1277	0x1048	This register controls the operation of Time Based Scheduling.
"MTL_TBS_STATS" on page 1279	0x104c	This register provides the One Hot encoded Queue numbers that have the frame drop related...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MTL_EST_Control" on page 1280	0x1050	This register controls the operation of Time Aware Shaper conforming to Enhancements to Scheduled...
"MTL_EST_Overhead" on page 1284	0x1054	This register indicates the value of Overhead in bytes to be considered for EST Scheduling.
"MTL_EST_Status" on page 1286	0x1058	This register reports the events related to Enhancements to Scheduled Transmission (IEEE...
"MTL_EST_Sch_Error" on page 1290	0x1060	This register provides the Traffic Class numbers that have the GCL scheduling related error.
"MTL_EST_Frm_Size_Error" on page 1291	0x1064	This register provides the Traffic Class numbers that have the Frame Size related scheduling...
"MTL_EST_Frm_Size_Capture" on page 1292	0x1068	This register captures the Frame Size and Traffic Class Number of the first occurrence of the Frame...
"MTL_EST_Intr_Enable" on page 1294	0x1070	This register implements the Interrupt Enable bits for the various events in the EST scheduler....
"MTL_GCL_Control" on page 1296	0x1080	This register provides the control for reading/writing to the Gate Control lists.
"MTL_GCL_Data" on page 1300	0x1084	This register holds the read data or write data in case of reads and writes respectively.
"MTL_FPE_CTRL_STS" on page 1301	0x1090	This register controls the operation of, and provides status for Frame Preemption (IEEE...
"MTL_FPE_Advance" on page 1304	0x1094	This register holds the Hold and Release Advance time.
"MTL_RXP_Control_Status" on page 1305	0x10a0	This is the Control and Status register for Flexible Receive Parser.
"MTL_RXP_Interrupt_Control_Status" on page 1309	0x10a4	This is Interrupt Control and Status register for Flexible Receive Parser.
"MTL_Rx_SPkt_Ctrl" on page 1313	0x10a8	This register controls the processing of the Header fields of the IEEE1722 special packets.
"MTL_Indirect_Acc_Control_Status" on page 1314	0x10b0	This is Control register to access the MTL indirect registers/tables.
"MTL_Indirect_Acc_Data" on page 1319	0x10b4	This is the data register to Access MTL indirect registers/tables.
"MTL_RXP_Drop_Cnt" on page 1320	0x10b4	This register provides the number of packets dropped by the Flexible Receive Parser. This is an...
"MTL_RXP_Error_Cnt" on page 1321	0x10b4	This register provides Error Counts of the Flexible Receive Parser. This is an indirect register...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MTL_RXP_Bypass_Cnt" on page 1322	0x10b4	The MTL_RXP_Bypass_Cnt registers provides the count of the number of frames bypassed by Rx Parser....
"MTL_FRP_ACNTR(#i)(for i = 0; i <= DWCXG_FRP_NUM_AC-1)" on page 1323	0x10b4	The MTL_FRP_ACNTR(#i) registers are indirect and provide the count of the number of frames accepted...
"MTL_FRP_ACNTR_CNFG(#i)(for i = 0; i <= DWCXG_FRP_NUM_AC-1)" on page 1324	0x10b4	The MTL_FRP_ACNTR_CNFG(#i) registers are indirect and allows application to configure corresponding...
"MTL_RCHLIST_IndReg(#i)_(#j)(for j = 0; j <= DWCXG_RCHLIST_DCNT-1)" on page 1325	0x10b4	The MTL_RCHLIST_IndReg0_0 registers, which are accessible through MTL_RXP_Indirect_Acc_* register....
"MTL_RMMC_RCH_MAP_IndReg(#i)(for i = 0; i <= DWCXG_RCHLIST_DCNT-1)" on page 1327	0x10b4	The MTL_RMMC_RCH_MAP_IndReg(#i) registers, which are accessible through MTL_Indirect_Acc_* register....
"MTL_ECC_Control" on page 1329	0x10c0	The MTL_ECC_Control register establishes the operating mode of ECC related to MAC and MTL...
"MTL_Safety_Interrupt_Status" on page 1332	0x10c4	The MTL_Safety_Interrupt_Status registers indicates the occurrence of various safety related...
"MTL_ECC_Interrupt_Enable" on page 1333	0x10c8	The MTL_ECC_Interrupt_Enable register provides enable bits for the ECC interrupts.
"MTL_ECC_Interrupt_Status" on page 1336	0x10cc	The MTL_ECC_Interrupt_Status register provides MAC/MTL ECC Interrupt Status.
"MTL_ECC_Err_Sts_Rctl" on page 1343	0x10d0	The MTL_ECC_Err_Sts_Rctl register establishes the control for ECC Error status capture.
"MTL_ECC_Err_Addr_Status" on page 1345	0x10d4	The MTL_ECC_Err_Addr_Status register provides the memory addresses for the correctable and uncorrectable...
"MTL_ECC_Err_Cntr_Status" on page 1346	0x10d8	The MTL_ECC_Err_Cntr_Status register provides Error count for Correctable and uncorrectable ECC...
"MTL_DPP_Control" on page 1347	0x10e0	The MTL_DPP_Control establishes the operating mode of Data Parity protection and error...
"MTL_DPP_ECC_EIC" on page 1355	0x10e4	The MTL_DPP_ECC_EIC establishes the operating mode of ECC/DPP error injection.
"MTL_SGF_Control" on page 1357	0x10e8	This register controls the operation of Stream-Gate Filtering.
"MTL_SGF_Extended_Control" on page 1360	0x10ec	This register controls the extended operation of Stream-Gate Filtering.
"MTL_SGF_Status" on page 1362	0x10f0	This register provides Status related to Stream-Gate Filtering.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"MTL_SGF_Interrupt_Enable" on page 1365	0x10f4	This register implements the Interrupt Enable bits for the various events that generate an interrupt....
MTL Traffic Class/Queue (#i) Registers. DWCXG_MTL_TCQ0 Exists: (DWCXG>0&&(DWCXG_NUM_RXQ>0||DWCXG_NUM_TXQ>0))		
"MTL_TxQ0_Operation_Mode" on page 1367	0x1100	The Queue 0 Transmit Operation Mode register establishes the Transmit queue operating modes and...
"MTL_TxQ0_Underflow" on page 1371	0x1104	The Queue 0 Underflow Counter register contains the number of packets aborted because of Transmit...
"MTL_TxQ0_Debug" on page 1373	0x1108	The Queue 0 Transmit Debug register gives the debug status of various blocks related to the Transmit...
"MTL_TC0_ETS_Control" on page 1375	0x1110	This register configures the transmission selection algorithm, for Traffic Class 0.This register...
"MTL_TC0_ETS_Status" on page 1377	0x1114	This register contains the information of average bits transmitted over a slot interval of 10 million...
"MTL_TC0_Quantum_Weight" on page 1378	0x1118	The TC1 Quantum or Weights register controls the average traffic transmitted in Traffic Class 0....
"MTL_TxQ(#i)_Bucket_Empty_2_Full(for i = 0; i <= min(7,DWCXG_NUM_TXQ-1))" on page 1380	(0x0080*i) +0x1128	The MTL Queue 0 ATS bucket empty 2 full register controls the Asynchronous Traffic shaper scheduler...
"MTL_TxQ(#i)_Committed_Info_Rate(for i = 0; i <= min(7,DWCXG_NUM_TXQ-1))" on page 1381	(0x0080*i) +0x112C	The MTL Queue 0 ATS committed info rate register controls the Asynchronous Traffic Shaper scheduler...
"MTL_RxQ0_Operation_Mode" on page 1382	0x1140	The Queue 0 Receive Operation Mode register establishes the Receive queue 0 operating modes and...
"MTL_RxQ0_Missed_Pkt_Overflow_Cnt" on page 1386	0x1144	The Queue 0 Overflow Counter register contains the counter for packets missed because of Receive...
"MTL_RxQ0_Debug" on page 1388	0x1148	The Queue 0 Receive Debug register gives the debug status of various blocks related to the Receive...
"MTL_RxQ0_Control" on page 1390	0x114c	The Queue 0 Receive Control register controls the receive arbitration and passing of received packets...
"MTL_RxQ0_Flow_Control" on page 1391	0x1150	This register controls the activation and de-activation threshold in the queue for triggering the...
"MTL_Q0_Interrupt_Enable" on page 1393	0x1170	This register contains the interrupt enable bits for Traffic class/Queue 0.
"MTL_Q0_Interrupt_Status" on page 1395	0x1174	This register contains the interrupt status bits for Traffic class/Queue 0.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
MTL Traffic Class/Queue (#i) Registers. DWCXG_MTL_TCQ__i_ (for i = 1; i <= max(min(7,DWCXG_NUM_RXQ-1),min(7,DWCXG_NUM_TXQ-1))) Exists: (DWCXG>0&&(DWCXG_NUM_RXQ>1||DWCXG_NUM_TXQ>1))		
"MTL_TxQ1_Operation_Mode" on page 1397	(0x0080*i) +0x1100	The Queue 1 Transmit Operation Mode register establishes the Transmit queue operating modes and...
"MTL_TxQ1_Underflow" on page 1401	(0x0080*i) +0x1104	The Queue 1 Underflow Counter register contains the number of packets aborted because of Transmit...
"MTL_TxQ1_Debug" on page 1403	(0x0080*i) +0x1108	The Queue 1 Transmit Debug register gives the debug status of various blocks related to the Transmit...
"MTL_TC1_ETS_Control" on page 1405	(0x0080*i) +0x1110	This register configures the transmission selection algorithm, for Traffic Class 1.This register...
"MTL_TC1_ETS_Status" on page 1407	(0x0080*i) +0x1114	This register contains the information of average bits transmitted over a slot interval of 10 million...
"MTL_TC1_Quantum_Weight" on page 1408	(0x0080*i) +0x1118	The TC1 Quantum or Weights register controls the average traffic transmitted in Traffic Class 1....
"MTL_TC1_SendSlopeCredit" on page 1410	(0x0080*i) +0x111c	MTL registers for traffic classes and queues 1 registers.
"MTL_TC1_HiCredit" on page 1411	(0x0080*i) +0x1120	This register contains the hiCredit value required for the credit-based shaper algorithm for...
"MTL_TC1_LoCredit" on page 1412	(0x0080*i) +0x1124	This register contains the loCredit value required for the credit-based shaper algorithm for...
"MTL_RxQ1_Operation_Mode" on page 1413	(0x0080*i) +0x1140	The Queue 1 Receive Operation Mode register establishes the Receive queue 1 operating modes and...
"MTL_RxQ1_Missed_Pkt_Overflow_Cnt" on page 1416	(0x0080*i) +0x1144	The Queue 1 Overflow Counter register contains the counter for packets missed because of Receive...
"MTL_RxQ1_Debug" on page 1418	(0x0080*i) +0x1148	The Queue 1 Receive Debug register gives the debug status of various blocks related to the Receive...
"MTL_RxQ1_Control" on page 1420	(0x0080*i) +0x114c	The Queue 1 Receive Control register controls the receive arbitration and passing of received packets...
"MTL_RxQ1_Flow_Control" on page 1421	(0x0080*i) +0x1150	This register controls the activation and de-activation threshold in the queue for triggering the...
"MTL_Q1_Interrupt_Enable" on page 1423	(0x0080*i) +0x1170	This register contains the interrupt enable bits for Traffic class/Queue 1.
"MTL_Q1_Interrupt_Status" on page 1425	(0x0080*i) +0x1174	This register contains the interrupt status bits for Traffic class/Queue 1.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
MTL Queue (#i) Registers. DWCXG_MTL_Q__i_ (for i = 8; i <= max(DWCXG_NUM_RXQ-1,DWCXG_NUM_TXQ-1)) Exists: (DWCXG>0&&(DWCXG_NUM_RXQ>8||DWCXG_NUM_TXQ>8))		
"MTL_TxQ8_Operation_Mode" on page 1427	(0x0080*j) +0x1100	The Queue 8 Transmit Operation Mode register establishes the Transmit queue operating modes and...
"MTL_TxQ8_Underflow" on page 1431	(0x0080*j) +0x1104	The Queue 8 Underflow Counter register contains the counter for packets aborted because of Transmit...
"MTL_TxQ8_Debug" on page 1433	(0x0080*j) +0x1108	The Transmit Debug register gives the debug status of various blocks related to this Transmit Queue...
"MTL_TxQ(#j)_Bucket_Empty_2_Full(for j = 8; j <= DWCXG_NUM_TXQ-1)" on page 1435	(0x0080*j) +0x1128	The MTL Queue 8 ATS bucket empty 2 full register controls the Asynchronous Traffic shaper scheduler...
"MTL_TxQ(#j)_Committed_Info_Rate(for j = 8; j <= DWCXG_NUM_TXQ-1)" on page 1436	(0x0080*j) +0x112C	The MTL Queue 8 ATS committed info rate register controls the Asynchronous Traffic Shaper scheduler...
"MTL_RxQ8_Operation_Mode" on page 1437	(0x0080*j) +0x1140	The Queue 8 Receive Operation Mode register establishes the Receive queue operating modes and...
"MTL_RxQ8_Missed_Pkt_Overflow_Cnt" on page 1440	(0x0080*j) +0x1144	The Queue 8 Overflow Counter register contains the counter for packets missed because of Receive...
"MTL_RxQ8_Debug" on page 1442	(0x0080*j) +0x1148	The Queue 8 Receive Debug register gives the debug status of various blocks related to the Receive...
"MTL_RxQ8_Control" on page 1444	(0x0080*j) +0x114c	The Queue 8 Receive Control register controls the receive arbitration and passing of received packets...
"MTL_RxQ8_Flow_Control" on page 1446	(0x0080*j) +0x1150	This register controls the activation and de-activation threshold in the queue for PAUSE/PFC flow...
"MTL_Q8_Interrupt_Enable" on page 1448	(0x0080*j) +0x1170	This register contains the interrupt enable bits for Traffic class/Queue 8.
"MTL_Q8_Interrupt_Status" on page 1450	(0x0080*j) +0x1174	This register contains the interrupt status bits for Traffic class/Queue 8.
DMA Common/General Registers. DWCXG_DMA Exists: (DWCXG>1)		
"DMA_Mode" on page 1452	0x3000	The Bus Mode register establishes the bus operating modes for the DMA.
"DMA_SysBus_Mode" on page 1456	0x3004	The System Bus mode register controls the behavior of the AXI manager. It mainly controls burst...
"DMA_Interrupt_Status" on page 1460	0x3008	The application reads this Interrupt Status register during interrupt service routine or polling...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"AXI_Tx_AR_ACE_Control" on page 1464	0x3010	This register is used to control the AXI4 Cache Coherency Signals for read transactions by all the...
"AXI_Rx_AW_ACE_Control" on page 1467	0x3018	This register is used to control the Cache Coherency Signals of the AXI4-ACE Lite on the Receive...
"AXI_TxRx_AWAR_ACE_Control" on page 1470	0x301c	This register is used to control the AXI4 Cache Coherency Signals for Descriptor write transactions...
"DMA_Debug_Status0" on page 1472	0x3020	The Debug Status 0 register gives the Receive and Transmit process status for DMA Channel 0 - Channel...
"DMA_Debug_Status1" on page 1473	0x3024	The Debug Status1 register gives the per Transmit DMA active status.
"DMA_Debug_Status3" on page 1474	0x302c	The Debug Status3 register gives the per Receive DMA active status.
"DMA_Tx_EDMA_Control" on page 1475	0x3040	The DMA_Tx_EDMA_Control register controls the sequence of all TxDMA operations with the DMA...
"DMA_Rx_EDMA_Control" on page 1477	0x3044	The DMA_Rx_EDMA_Control register controls the sequence of all RxDMA operations with the DMA...
"AXI_LPI_Entry_Interval" on page 1479	0x3050	This register is used to control the AXI LPI entry interval.
"DMA_TBS_CTRL0" on page 1480	0x3054	This register is used to control the TBS0 attributes.
"DMA_TBS_CTRL1" on page 1482	0x3058	This register is used to control the TBS1 attributes.
"DMA_TBS_CTRL2" on page 1484	0x305c	This register is used to control the TBS2 attributes.
"DMA_TBS_CTRL3" on page 1486	0x3060	This register is used to control the TBS3 attributes.
"DMA_Safety_Interrupt_Status" on page 1488	0x3064	This register indicates summary (whether error occurred in DMA/MTL/MAC and correctable/uncorrectable)...
"DMA_ECC_Interrupt_enable" on page 1492	0x3068	This register is used to enable the Automotive Safety related TSO memory ECC error interrupt.
"DMA_ECC_Interrupt_Status" on page 1494	0x306c	This register indicates the Automotive Safety related TSO memory ECC error interrupt status.
"DMA_DPP_Control" on page 1498	0x3070	The DMA_DPP_Control establishes the operating mode of Data Parity protection and error injection...
"DMA_DPP_Interrupt_Status" on page 1501	0x3074	This register contains the status of Automotive Safety related Data Path Parity Errors per channel...
"DMA_CH_Ind_Ctrl" on page 1504	0x3080	This register is used to program Indirect Access DMA configuration registers

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"DMA_CH_Ind_Data" on page 1507	0x3084	This register holds the read/write data for Indirect Access of the registers indicated by DMA_(#MSEL)_(#AOFF)....
"PDMA_CH(#i)_TxExtCfg(for i = 0; i <= DWCXG_NUM_TXQ-1)" on page 1508	0x3084	This register controls the parameters (like PBL, ORRQ, etc) that corresponds to this PDMA/Hardware...
"PDMA_CH(#i)_TxDbgSts(for i = 0; i <= DWCXG_NUM_TXQ-1)" on page 1511	0x3084	This register provides the active TX DMA numbers which is being processed by the different functions...
"VDMA_CH(#i)_TxDescCtrl(for i = 0; i <= DWCXG_NUM_DMA_TX_CH-1)" on page 1513	0x3084	This register contains the per channel programming field for Tx Descriptor Cache size.
"PDMA_CH(#i)_RxExtCfg(for i = 0; i <= DWCXG_NUM_RXQ-1)" on page 1515	0x3084	This register controls the parameters (like PBL, ORRQ, etc) that corresponds to this PDMA/Hardware...
"PDMA_CH(#i)_RxDbgSts(for i = 0; i <= DWCXG_NUM_RXQ-1)" on page 1518	0x3084	This register provides the active RX DMA numbers which is being processed by the different functions...
"VDMA_CH(#i)_RxDescCtrl(for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)" on page 1520	0x3084	This register contains the per channel programming field for Rx Descriptor Cache size.
"VDMA_CH(#i)_RxFrameBuf_BaseAddr_Hi(for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)" on page 1522	0x3084	This register stores the MS bits of base address of the FrameBuffer[n], whose lower address bits...
"VDMA_CH(#i)_RxFrameBuf0_BaseAddr_Lo(for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)" on page 1523	0x3084	This register stores the LS 32bits of base address of the FrameBuffer0, whose upper address bits...
"VDMA_CH(#i)_RxFrameBuf1_BaseAddr_Lo(for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)" on page 1524	0x3084	This register stores the LS 32bits of base address of the FrameBuffer1, whose upper address bits...
"VDMA_CH(#i)_RxFrameBuf2_BaseAddr_Lo(for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)" on page 1525	0x3084	This register stores the LS 32bits of base address of the FrameBuffer2, whose upper address bits...
"VDMA_CH(#i)_RxFrameBuf3_BaseAddr_Lo(for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)" on page 1526	0x3084	This register stores the LS 32bits of base address of the FrameBuffer3, whose upper address bits...
"PDMA_CH(#i)_RxPktInfoBuf_BaseAddr_Hi(for i = 0; i <= DWCXG_NUM_RXQ-1)" on page 1527	0x3084	This register contains the MS/Upper bits of the Start address of the PacketInfo Buffer associated...
"PDMA_CH(#i)_RxPktInfoBuf_BaseAddr_Lo(for i = 0; i <= DWCXG_NUM_RXQ-1)" on page 1528	0x3084	This register contains the Start address of the PacketInfo Buffer associated with the PDMA. All...
"PDMA_CH(#i)_RxPktInfoBuf_CurrOffset(for i = 0; i <= DWCXG_NUM_RXQ-1)" on page 1529	0x3084	This register indicates the offset from the start of the Packet Info Buffer to which the last Packet...
"DMA_Sfty_Ind_Ctrl" on page 1531	0x3088	This a control register to access DMA Safety related indirect registers.

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"DMA_Sfty_Ind_Data" on page 1535	0x308c	This field contains data to read/write from DMA Safety indirect registers.
"DMA_Sfty_Dbg_EIC" on page 1536	0x308c	This register contains control signals for Debug Error Injection.
"DMA_AM_Sfty_Ctrl" on page 1539	0x308c	This register contains control signal for AXI Manager interface safety.
"DMA_AM_Sfty_Intr_En" on page 1540	0x308c	This register contains the interrupt enables for AXI Manager Interface Safety.
"DMA_AM_Sfty_ECC_Intr_Sts" on page 1541	0x308c	This register contains the ECC interrupt status for the AXI Manager Interface Safety.
"DMA_AM_Sfty_DPP_Intr_Sts" on page 1542	0x308c	This register contains the DPP interrupt status for AXI Manager Interface safety.
"DMA_AM_Sfty_RDC_ECC_Err_Cntr_Sts" on page 1544	0x308c	This register contains the ECC Error counters for read data of AXI Manager Interface safety.
"DMA_AM_Sfty_RDC_Dbg_Data" on page 1545	0x308c	This register contains the data to be driven on AXI Manager read data in Debug Error Injection...
"DMA_AM_Sfty_RDC_Dbg_Ctrl" on page 1546	0x308c	This register contains the data to be driven on AXI Manager Read Data channel control signals in...
"DMA_AM_Sfty_WRC_Dbg_Ctrl" on page 1548	0x308c	This register contains the data to be driven on AXI Manager write response channel control signals...
"DMA_AS_Sfty_Ctrl" on page 1549	0x308c	This register contains control signal for AXI Subordinate interface safety module.
"DMA_AS_Sfty_Intr_En" on page 1550	0x308c	This register contains the interrupt enables for AXI Subordinate Interface Safety.
"DMA_AS_Sfty_ECC_Intr_Sts" on page 1551	0x308c	This register contains the ECC interrupt status for the AXI Subordinate Interface Safety.
"DMA_AS_Sfty_DPP_Intr_Sts" on page 1552	0x308c	This register contains the DPP interrupt status for AXI Subordinate Interface safety.
"DMA_AS_Sfty_WDC_ECC_Err_Cntr_Sts" on page 1555	0x308c	This register contains the ECC Error counters for write data of AXI Subordinate Interface...
"DMA_AS_Sfty_WDC_Dbg_Data" on page 1556	0x308c	This register contains the data to be driven on AXI Subordinate write data in Debug Error Injection...
"DMA_AS_Sfty_WDC_Dbg_Ctrl" on page 1557	0x308c	This register contains the data to be driven on AXI Subordinate Write Data channel control signals...
"DMA_AS_Sfty_RAC_Dbg_Addr" on page 1558	0x308c	This register contains the data to be driven on address bus of AXI Subordinate read address channel...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"DMA_AS_Sfty_RAC_Dbg_Ctrl" on page 1559	0x308c	This register contains the data to be driven on control signals of AXI Subordinate read address...
"DMA_AS_Sfty_WAC_Dbg_Addr" on page 1561	0x308c	This register contains the data to be driven on address bus of AXI Subordinate write address channel...
"DMA_AS_Sfty_WAC_Dbg_Ctrl" on page 1562	0x308c	This register contains the data to be driven on control signals of AXI Subordinate write address...
"DMA_Interrupt_Status16_47" on page 1564	0x3090	The application reads this Interrupt Status register during interrupt service routine or polling...
"DMA_Interrupt_Status48_79" on page 1566	0x3094	The application reads this Interrupt Status register during interrupt service routine or polling...
"DMA_Interrupt_Status80_111" on page 1567	0x3098	The application reads this Interrupt Status register during interrupt service routine or polling...
"DMA_Interrupt_Status112_127" on page 1569	0x309c	The application reads this Interrupt Status register during interrupt service routine or polling...
DMA Channel (#i) Registers. DWCXG_DMA_CH__i_ (for i = 0; i <= DWCXG_NUM_DMA_CSR_CH-1) Exists: (DWCXG>1&&(DWCXG_NUM_DMA_RX_CH>0||DWCXG_NUM_DMA_TX_CH>0))		
"VDMA_CH0_Control" on page 1571	(0x0080*i) +0x3100	The VDMA Channel0 Control register specifies the MSS value for segmentation, length to skip between...
"VDMA_CH0_Tx_Control" on page 1573	(0x0080*i) +0x3104	The VDMA Channel0 Transmit Control register controls the Tx features such as PBL, TCP segmentation,...
"VDMA_CH0_Rx_Control" on page 1577	(0x0080*i) +0x3108	The VDMA Channel0 Receive Control register controls the Rx features such as PBL, buffer size, and...
"VDMA_CH0_Slot_Function_Control_Status" on page 1581	(0x0080*i) +0x310c	This register contains the control bits for slot function and its status for Transmit path.
"VDMA_CH0_TxDesc_List_HAddress" on page 1583	(0x0080*i) +0x3110	The Channel0 Tx Descriptor List HAddress register has the higher 8 or 16 bits of the start address...
"VDMA_CH0_TxDesc_List_LAddress" on page 1584	(0x0080*i) +0x3114	The Channel0 Tx Descriptor List LAddress register has the lower 32 bits of the start address of...
"VDMA_CH0_RxDesc_List_HAddress" on page 1586	(0x0080*i) +0x3118	The Channel0 Rx Descriptor List HAddress register has the higher 8 or 16 bits of the start address...
"VDMA_CH0_RxDesc_List_LAddress" on page 1587	(0x0080*i) +0x311c	The Channel0 Rx Descriptor List LAddress register has the lower 32 bits of the start address of...
"VDMA_CH0_TxDesc_Tail_LPointer" on page 1589	(0x0080*i) +0x3124	The Channel0 Tx Descriptor Tail LPointer register has the 32 bits end address of the Transmit descriptor...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"VDMA_CH0_RxDesc_Tail_LPointer" on page 1590	(0x0080*i) +0x312c	The Channel0 Rx Descriptor Tail LPointer register has the 32 bits end address of the Receive descriptor...
"VDMA_CH0_Tx_Control2" on page 1591	(0x0080*i) +0x3130	The Tx_Control2 register contains programmable control to Outstanding read requests per DMA and...
"VDMA_CH0_Rx_Control2" on page 1593	(0x0080*i) +0x3134	The Channel0 Rx Control2 register contains programmable control to Outstanding Write requests and...
"VDMA_CH0_Interrupt_Enable" on page 1595	(0x0080*i) +0x3138	The Channel0 Interrupt Enable register enables the interrupts reported by the Status register.
"VDMA_CH0_Rx_Interrupt_Watchdog_Timer" on page 1599	(0x0080*i) +0x313c	The Receive Interrupt Watchdog Timer register indicates the watchdog/Idle timeout for Receive Interrupt...
"VDMA_CH0_Current_App_TxDesc_H" on page 1602	(0x0080*i) +0x3140	The Channel0 Current Application Transmit Descriptor High register has the higher 32 bits of the...
"VDMA_CH0_Current_App_TxDesc_L" on page 1603	(0x0080*i) +0x3144	The Channel0 Current Application Transmit Descriptor High register has the lower 32 bits of the...
"VDMA_CH0_Current_App_RxDesc_H" on page 1604	(0x0080*i) +0x3148	The Channel0 Current Application Receive Descriptor High register has the higher 32 bits of the...
"VDMA_CH0_Current_App_RxDesc_L" on page 1605	(0x0080*i) +0x314c	The Channel0 Current Application Receive Descriptor Low register has the lower 32 bits of the current...
"VDMA_CH0_Current_App_TxBuffer_H" on page 1606	(0x0080*i) +0x3150	The Channel0 Current Application Transmit Buffer Address High register has the higher 8 or 16 bits...
"VDMA_CH0_Current_App_TxBuffer_L" on page 1607	(0x0080*i) +0x3154	The Channel0 Current Application Transmit Buffer Address Low register has the lower 32 bits of the...
"VDMA_CH0_Current_App_RxBuffer_H" on page 1608	(0x0080*i) +0x3158	The Channel0 Current Application Receive Buffer Address High register has the higher 8 or 16 bits...
"VDMA_CH0_Current_App_RxBuffer_L" on page 1609	(0x0080*i) +0x315c	The Channel0 Current Application Receive Buffer Address Low register has the lower 32 bits of the...
"VDMA_CH0_Status" on page 1610	(0x0080*i) +0x3160	The software driver (application) reads the Status register during interrupt service routine or...
"VDMA_CH0_Debug_Status" on page 1616	(0x0080*i) +0x3164	DMA Channe0 Debug Status register
"VDMA_CH0_Desc_Mem_Cache_Fill_Level" on page 1620	(0x0080*i) +0x3168	The VDMA Channel0 Descriptor Cache Fill Level Status register.
"VDMA_CH0_Miss_Packet_Cnt" on page 1622	(0x0080*i) +0x316c	This register has the number of the packets that are dropped by the DMA either due to Bus Error...
"VDMA_CH0_Tx_Data_Xfer_Ring_Offset" on page 1623	(0x0080*i) +0x3170	This register indicates the Offset of the Tx Descriptor List, for which the DMA engine is fetching...

Table 11-4 Registers for the DWC_xgmac_map Memory Map (continued)

Register	Offset	Description
"VDMA_CH0_Rx_Data_Xfer_Ring_Offset" on page 1624	(0x0080*i) +0x3174	The VDMA Channel0 Receive Data Transfer Ring Offset register indicates the Ring Offset of the Rx...
"VDMA_CH0_Tx_Desc_Write_Ring_Offset" on page 1625	(0x0080*i) +0x3178	This register indicates the Offset of the Tx Descriptor List, for which the DMA engine is closing...
"VDMA_CH0_Rx_Desc_Write_Ring_Offset" on page 1627	(0x0080*i) +0x317c	This register indicates the Offset of the Rx Descriptor List, for which the DMA engine is closing...

11.1 DWCXG_CORE Registers

11.1.1 MAC_Tx_Configuration

- **Description:** The MAC Transmit Configuration register establishes the operating mode of the MAC transmitter.
- **Size:** 32 bits
- **Offset:** 0x0
- **Exists:** Always

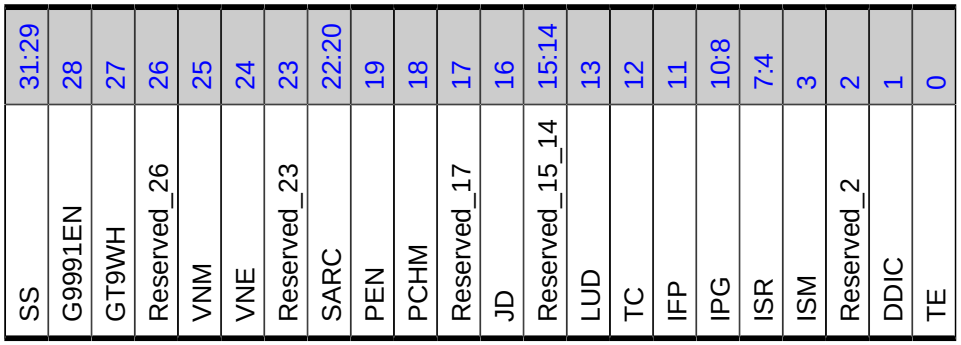


Table 11-5 Fields for Register: MAC_Tx_Configuration

Bits	Name	Memory Access	Description
31:29	SS	R/W	Speed Selection. Software programs these bits to instruct the MAC to operate in one of the following possible speeds: ■ 3'b000 10G XGMII ■ 3'b001 25G XGMII ■ 3'b010 2.5G GMII ■ 3'b011 1G GMII ■ 3'b100 100M MII ■ 3'b101 5G XGMII ■ 3'b110 2.5G XGMII ■ 3'b111 10M MII These bits must be programmed only once; after a hardware reset and before the transmitter and receiver are enabled (by bit 0 of this register and bit 0 of the MAC_Rx_Configuration register). **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
28	G9991EN	R/W	G999_1 Mode enable. When set, the MAC Transmitter and Receiver operates in the G999_1 mode and able to process Ethernet encapsulated G999_1 packets. **Value After Reset:** 0x0 **Exists:** DWCXG_G9991_EN

Table 11-5 Fields for Register: MAC_Tx_Configuration (continued)

Bits	Name	Memory Access	Description
27	GT9WH	R/W	G9991 Without Ethernet Header encapsulation enable. When set, MAC Transmitter and Receiver processes the G9991 DATA and DFC packets in Native format (without Ethernet encapsulation). This bit is functional only when G9991EN is set. When this bit is clear and bit G9991EN =1, the MAC processes the Ethernet Encapsulated G999.1 packets. **Value After Reset:** 0x0 **Exists:** DWCXG_G9991_EN
26	Reserved_26	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
25	VNM	R/W	VxLAN-NVGRE Mode. When this is set to 1 and VNE=1, DWC_25gmac processes NVGRE protocol based tunneled packets. When this bit is 0 and VNE=1, DWC_25gmac processes VxLAN protocol based tunneled packets. **Value After Reset:** 0x0 **Exists:** DWCXG_VXN_ENA
24	VNE	R/W	VxLAN-NVGRE Mode. When this bit is set to 1 and VNM=1, DWC_25gmac processes NVGRE protocol based tunneled packets. When this bit is set to 1 and VNM=0, DWC_25gmac processes VxLAN protocol based tunneled packets. **Value After Reset:** 0x0 **Exists:** DWCXG_VXN_ENA
23	Reserved_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
22:20	SARC	R/W	Source Address Insertion or Replacement Control. This field controls the source address insertion or replacement for all transmitted packets. Bit 22 specifies which MAC Address register (0 or 1) is used for source address insertion or replacement based on the values of Bits[21:20]: **2'b0x:** The SA insertion-replacement is controlled by one of the following: ■ mti_sa_ctrl_i input signal in XGMAC-CORE configuration ■ SAIC field (Bits[26:24]) of Transmit Control Word in XGMAC-MTL configuration ■ SAIC field (Bits[25:23]) of TDES3 Normal Descriptor in XGMAC-AXI configuration

Table 11-5 Fields for Register: MAC_Tx_Configuration (continued)

Bits	Name	Memory Access	Description
			2'b10: ■ If Bit 22 is set to 0, the MAC inserts the content of the MAC Address 0 registers (MAC_Address0_High and MAC_Address0_Low) in the SA field of all transmitted packets. ■ If Bit 22 is set to 1 and the Enable MAC Address Register 1 option is selected while configuring the controller, the MAC inserts the content of the MAC Address 1 registers (MAC_Address1_High and MAC_Address1_Low) in the SA field of all transmitted packets. **2'b11:** ■ If Bit 22 is set to 0, the MAC replaces the content of the MAC Address 0 registers (MAC_Address0_High and MAC_Address0_Low) in the SA field of all transmitted packets. ■ If Bit 22 is set to 1 and the MAC Address Register 1 is enabled, the MAC replaces the content of the MAC Address 1 registers (MAC_Address1_High and MAC_Address1_Low) in the SA field of all transmitted packets. **Note:** ■ Changes to this field take effect only on the start of a packet. If you write to this register field when a packet is being transmitted, only the subsequent packet can use the updated value, that is, the current packet does not use the updated value. **Value After Reset:** 0x0 **Exists:** DWCXG_SA_VLAN_INS_CTRL_EN
19	PEN	R/W	PCH Enable. This (PEN) bit is reserved for this release. Do not set this bit for proper operation **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_45
18	PCHM	R/W	PCHM Mode. This bit (PCHM) is reserved for this release. Do not set this bit for proper operation **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_45
17	Reserved_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-5 Fields for Register: MAC_Tx_Configuration (continued)

Bits	Name	Memory Access	Description
16	JD	R/W	Jabber Disable. When this bit is set, the DWC_25gmac disables the jabber timer on the transmitter. Transmission of up to 16383-byte packets is supported. When this bit is reset, DWC_25gmac cuts off the transmitter if the application sends more than 2048 bytes of data (10240 bytes if JE (in MAC_Rx_Configuration register is set high) during transmission. **Value After Reset:** 0x0 **Exists:** Always
15:14	Reserved_15_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13	LUD	R/W	Link Up or Down. This bit indicates whether the link is up or down during transmission of configuration in the RGMII interface. ■ 1'b0: Link down ■ 1'b1: Link up **Value After Reset:** 0x0 **Exists:** DWCXG_RGMII_EN
12	TC	R/W	Transmit Configuration in RGMII. When set, this bit enables the transmission of duplex mode, link speed, and link up or down information to the PHY in the RGMII, port. When this bit is reset, no such information is driven to the PHY. ■ 1'b0: Disable Transmit Configuration in RGMII ■ 1'b1: Enable Transmit Configuration in RGMII **Value After Reset:** 0x0 **Exists:** DWCXG_RGMII_EN
11	IFP	R/W	IPG Control When this bit is set, the minimum IPG is increased in steps of 32 bits from the default 96 bits, depending on the programming of the IPG field. In the XGMII mode, this bit has higher priority than the ISM bit. When this bit is reset, the minimum IFG is decreased in steps of 8 bits from the default 96 bits, depending on the programming of the IPG field. **Value After Reset:** 0x0 **Exists:** Always
10:8	IPG	R/W	Inter-Packet Gap

Table 11-5 Fields for Register: MAC_Tx_Configuration (continued)

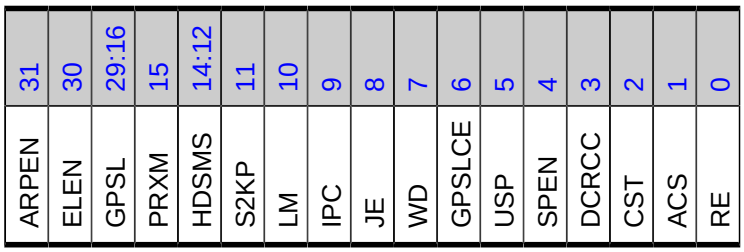
Bits	Name	Memory Access	Description
			These bits control the minimum IPG between packets during transmission. When IFP is cleared, the minimum IPG between transmitted packets is reduced in both GMII and XGMII mode. The corresponding IPG values are given as follows. ■ 3'b000: 96 bit times ■ 3'b001: 88 bit times ■ 3'b010: 80 bit times ■ 3'b011: 72 bit times ■ 3'b100: 64 bit times ■ 3'b101: 3'b111: Reserved In the half-duplex mode, the minimum IPG can be configured only for 64-bit times (IPG = 100). Lower values are not considered. When a JAM pattern is transmitted because of backpressure activation, the MAC does not consider the minimum IPG. When IFP is set, the minimum IPG is increased in steps of 32 bits as follows. In 10G (XGMII) mode, when IFP is set, it overrides the functions of DDIC and ISM bits. The EIPG bits (in MAC_Extended_Configuration register) along with IPG bits gives a 10-bit value to control the required IPG as follows: ■ {EIPG,IPG}: Minimum Inter-Packet Gap ■ 10'h000: 96 bit times ■ 10'h001: 128 bit times ■ 10'h002: 160 bit times ■ 10'h003: 192 bit times ■ 10'h004: 224 bit times ■ ... ■ 10'h3FF: 32832 bit times **Value After Reset:** 0x0 **Exists:** Always
7:4	ISR	R/W	IFG Stretch Ratio. This value controls the number of bytes in a transmitted packet for which one octet of IDLE is added to the IFG. To operate DWC_25gmac for 10GBASE-W (WIS layer), the value must be 13 (decimal). Note that 0 is not a valid value and must not be written. Before enabling ISM (IFG Stretch Mode) bit, this ISR number must be programmed to a non-zero value. **Value After Reset:** 0x0 **Exists:** Always

Table 11-5 Fields for Register: MAC_Tx_Configuration (continued)

Bits	Name	Memory Access	Description
3	ISM	R/W	IFG Stretch Mode. When this bit is set, DWC_25gmac enforces extended inter-packet spacing (one extra octet of IDLE for every ISR (IFG Stretch Ratio) number of bytes) to support less-than-10-Gbps data rates. When this bit is reset, inter-packet gap extension is not performed. **Value After Reset:** 0x0 **Exists:** Always
2	Reserved_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	DDIC	R/W	Disable DIC Algorithm. When this bit is set, the Deficit Idle Count (DIC) algorithm is disabled and DWC_25gmac always has a minimum IFG of 12 bytes between transmitted packets. When reset to zero, the DIC is activated and the minimum IFG varies between 9 and 15 bytes. **Value After Reset:** 0x0 **Exists:** Always
0	TE	R/W	Transmitter Enable. When this bit is set, the Tx state machine of the MAC is enabled for transmission on the GMII or XGMII interface. When this bit is reset, the MAC Tx state machine is disabled after it completes the transmission of the current packet. The Tx state machine does not transmit any more packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.2 MAC_Rx_Configuration

- **Description:** The MAC Receive Configuration register establishes the operating mode of the MAC receiver.
- **Size:** 32 bits
- **Offset:** 0x4
- **Exists:** Always



31	30	29:16	15	14:12	11	10	9	8	7	6	5	4	3	2	1	0
ARPEN	ELEN	GPSL	PRXM	HDSMS	S2KP	LM	IPC	JE	WD	GPSLCE	USP	SPEN	DCRCC	CST	ACS	RE

Table 11-6 Fields for Register: MAC_Rx_Configuration

Bits	Name	Memory Access	Description
31	ARPEN	R/W	ARP enable. When this bit is set, ARP offload is enabled. **Value After Reset:** 0x0 **Exists:** DWCXG_ARP_EN
30	ELEN	R/W	External lookup enabled. When this bit is set, the external lookup logic is enabled and the DMA channel selection is only based on the information provided by the external Lookup interface. **Value After Reset:** 0x0 **Exists:** DWCXG_EXT_LKUP_EN
29:16	GPSL	R/W	Giant Packet Size Limit. If the received packet size is greater than the value programmed in this field in units of bytes, the MAC declares the received packet as Giant packet. The value programmed in this field must be greater than or equal to 1518 bytes. Any other programmed value is considered as 1518 bytes. For VLAN tagged packets, the MAC adds 4 bytes to the programmed value. The value in this field is applicable when the GPSLCE bit is set in MAC_Rx_Configuration register. **Value After Reset:** 0x0 **Exists:** Always
15	PRXM	R/W	PCH Rx Mode. This (PEN) bit is reserved for this release. Do not set this bit for proper operation **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_45

Table 11-6 Fields for Register: MAC_Rx_Configuration (continued)

Bits	Name	Memory Access	Description
14:12	HDSMS	R/W	Maximum Size for Splitting the Header Data. These bits indicate the maximum header size allowed for splitting the header data in the received packet: ■ 3'b000: 64 bytes ■ 3'b001: 128 bytes ■ 3'b010: 256 bytes ■ 3'b011: 512 bytes ■ 3'b100: 1023 bytes ■ 3'b101-3'b111: Reserved **Value After Reset:** 0x0 **Exists:** DWCXG_SPLIT_HDR_EN
11	S2KP	R/W	IEEE 802.3as Support for 2K Packets. When this bit is set, the MAC considers all packets with up to 2000 bytes length as normal packets. When the JE bit is not set, the MAC considers all the received packets of size more than 2K bytes as Giant packets. When this bit is reset and the JE bit is not set, the MAC considers all the received packets of size more than 1518 bytes (1522 bytes for tagged) as giant packets. **Note:** When the JE bit is set, setting this bit has no effect on the giant packet status. **Value After Reset:** 0x0 **Exists:** Always
10	LM	R/W	Loopback Mode. When this bit is set, the MAC operates in the loopback mode at GMII or XGMII. The (X)GMII Rx clock input (clk_rx_312pt5_i) is required for the loopback to work properly. This is because the Tx clock is not internally looped back. **Value After Reset:** 0x0 **Exists:** Always

Table 11-6 Fields for Register: MAC_Rx_Configuration (continued)

Bits	Name	Memory Access	Description
9	IPC	R/W	Checksum Offload. When set, this bit enables the IPv4 header checksum checking and IPv4 or IPv6 TCP, UDP, or ICMP payload checksum checking. When this bit is reset, the COE function in the receiver is disabled. The Layer 3 and Layer 4 Packet Filter, Enable Header-Payload Split, Enable Receive Side Scaling and Enable Support for Virtualized Network features automatically selects the IPC Full Checksum Offload Engine on the Receive side. This bit must be set when any of these features are enabled. **Value After Reset:** 0x0 **Exists:** DWCXG_RX_COE
8	JE	R/W	Jumbo Packet Enable. When this bit is set, the MAC allows jumbo packets of 9018 bytes (9022 bytes for VLAN tagged packets) without reporting a giant packet error in the Rx packet status. **Value After Reset:** 0x0 **Exists:** Always
7	WD	R/W	Watchdog Disable. When this bit is set, the MAC disables the watchdog timer on the receiver. The MAC can receive packets of up to 16383 bytes. When this bit is reset, the MAC does not allow more than 2048 bytes (10240 if JE is set high) of the packet being received. The MAC cuts off any bytes received after 2048 bytes. **Value After Reset:** 0x0 **Exists:** Always
6	GPSLCE	R/W	Giant Packet Size Limit Control Enable. When this bit is set, the MAC considers the value in GPSL field in MAC_Rx_Configuration register to declare a received packet as Giant packet. This field must be programmed to more than 1518 bytes. Otherwise, the MAC considers 1518 bytes as giant packet limit. When this bit is reset, the MAC considers a received packet as Giant packet when its size is greater than 1518 bytes (1522 bytes for tagged packet). The watchdog timeout limit, Jumbo Packet Enable and 2K Packet Enable have higher precedence over this bit, that is the MAC considers a received packet as Giant packet when its size is greater than 9018 bytes (9022 bytes for tagged packet) with Jumbo Packet Enabled and greater than 2000 bytes with 2K Packet Enabled. The watchdog timeout, if

Table 11-6 Fields for Register: MAC_Rx_Configuration (continued)

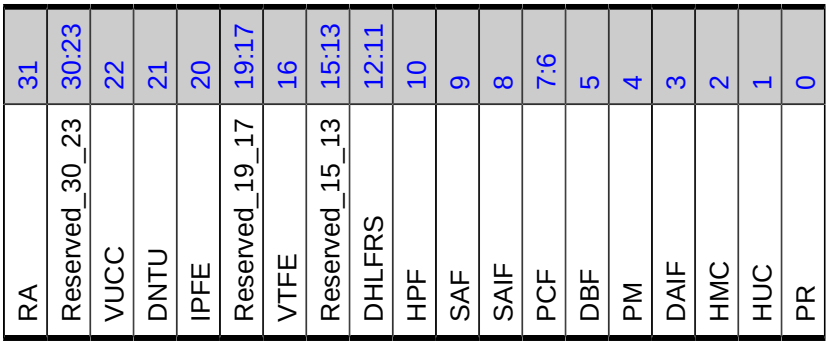
Bits	Name	Memory Access	Description
			enabled, terminates the received packet when watchdog limit is reached. Therefore, the programmed giant packet limit must be less than the watchdog limit to get the giant packet status. **Value After Reset:** 0x0 **Exists:** Always
5	USP	R/W	Unicast Slow Protocol Packet Detect. When this bit is set, the MAC detects the Slow Protocol packets with unicast address of the station specified in the MAC_Address0_High and MAC_Address0_Low registers. The MAC also detects the Slow Protocol packets with the Slow Protocols multicast address (01-80-C2-00-00-02). When this bit is reset, the MAC detects only Slow Protocol packets with the Slow Protocol multicast address specified in the IEEE 802.3-2018, Section 5. **Value After Reset:** 0x0 **Exists:** Always
4	SPEN	R/W	Slow Protocol Detection Enable. When this bit is set, MAC processes the Slow Protocol packets (Ether Type 0x8809) and provides the Rx status. The MAC discards the Slow Protocol packets with invalid sub-types. When this bit is reset, the MAC forwards all error-free Slow Protocol packets to the application. The MAC considers such packets as normal Type packets. **Value After Reset:** 0x0 **Exists:** Always
3	DCRCC	R/W	Disable CRC Checking for Received Packets. When this bit is set, the MAC receiver does not check the CRC field in the received packets. When this bit is reset, the MAC receiver always checks the CRC field in the received packets. **Value After Reset:** 0x0 **Exists:** Always

Table 11-6 Fields for Register: MAC_Rx_Configuration (continued)

Bits	Name	Memory Access	Description
2	CST	R/W	CRC stripping for Type packets. When this bit is set, the last four bytes (FCS) of all packets of Ether type (field greater than 1536) are stripped and dropped before forwarding the packet to the application. When this bit is reset,CRC field of Ethernet packet is not stripped. This function is not valid when the IP Checksum Engine (Type 1) is enabled in the MAC receiver. This function is valid when Type 2 Checksum Offload Engine is enabled. **Value After Reset:** 0x0 **Exists:** Always
1	ACS	R/W	Automatic Pad or CRC Stripping. When this bit is set, the MAC strips the Pad or FCS field on the incoming packets, only if the value of the length field is less than 1536 bytes. All received packets with length field greater than or equal to 1536 bytes are passed to the application without stripping the Pad or FCS field. When this bit is reset, the MAC passes all the incoming packets to the application, without any modification. **Value After Reset:** 0x0 **Exists:** Always
0	RE	R/W	Receiver Enable. When this bit is set, the Rx state machine of the MAC is enabled for receiving packets from the GMII or XGMII interface. When this bit is reset, the MAC Rx state machine is disabled after it completes the reception of the current packet. The Rx state machine does not receive any more packets from the GMII or XGMII interface. **Value After Reset:** 0x0 **Exists:** Always

11.1.3 MAC_Packet_Filter

- **Description:** The MAC Packet Filter register contains the filter controls for receiving packets. Some of the controls from this register go to the address check block of the MAC which performs the first level of address filtering. The second level of filtering is performed on the incoming packet based on other controls such as Pass Bad Packets and Pass Control Packets.
- **Size:** 32 bits
- **Offset:** 0x8
- **Exists:** Always



31	30:23	22	21	20	19:17	16	15:13	12:11	10	9	8	7:6	5	4	3	2	1	0
RA	Reserved_30_23	VUCC	DNTU	IPFE	Reserved_19_17	VTFE	Reserved_15_13	DHLFRS	HPF	SAF	SAIF	PCF	DBF	PM	DAIF	HMC	HUC	PR

Table 11-7 Fields for Register: MAC_Packet_Filter

Bits	Name	Memory Access	Description
31	RA	R/W	Receive All. When this bit is set, the MAC Receiver module passes all the received packets to the application, irrespective of whether they pass the address filter or not. The result of the SA or DA filtering is updated (pass or fail) in the corresponding bit in the Rx Status Word. When this bit is reset, the Receiver module passes only those packets to the application that pass the SA or DA address filter. **Value After Reset:** 0x0 **Exists:** Always
30:23	Reserved_30_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-7 Fields for Register: MAC_Packet_Filter (continued)

Bits	Name	Memory Access	Description
22	VUCC	R/W	VxLAN UDP/IPv6 Checksum Control. This bit provides the control to process received VxLAN tunneled packet with IPv6 outer header and having UDP Checksum field with all-zeros value. This is provided because the UDP/IPv6 and VxLAN specifications have inherent contradiction for the checksum field. UDP/IPv6 requires a non-zero checksum while VxLAN allows all-zero or no checksum for UDP. When this bit is set to 1 (and VNE=1, VNM=0), DWC_25gmac accepts the received VxLAN packets with all-zero UDP checksum field and process the inner headers. When this bit is 0 (and VNE=1, VNM=0), it declares such UDP/IPv6 packets with all-zero checksum fields as having Outer Payload (L4) Checksum error. **Value After Reset:** 0x0 **Exists:** DWCXG_VXN_ENA
21	DNTU	R/W	Drop Non-TCP/UDP over IP Packets. When this bit is set, the MAC drops the non-TCP or UDP over IP packets. The MAC forward only those packets that are processed by the Layer 4 filter. When this bit is reset, the MAC forwards all non-TCP or UDP over IP packets. When support for L2 Virtualized network over L3 Network is enabled, ■ If VNE=1 and VNM=0, all non VxLAN type packets and all non-TCP/UDP type normal packets are dropped. ■ If VNE=1 and VNM=1, all non NVGRE type packets and all non-TCP/UDP type normal packets are dropped. ■ If VNE=0, all non-TCP/UDP type normal packets are dropped. **Note:** When this bit is set in NVGRE mode (when VNE=1, VNM=1), NVGRE packets (which are also non-TCP/UDP) are not dropped. **Value After Reset:** 0x0 **Exists:** DWCXG_L3_L4_FILTER_EN

Table 11-7 Fields for Register: MAC_Packet_Filter (continued)

Bits	Name	Memory Access	Description
20	IPFE	R/W	Layer 3 and Layer 4 Filter Enable. When this bit is set, the MAC drops packets that do not match the enabled Layer 3 and Layer 4 filters. If Layer 3 or Layer 4 filters are not enabled for matching, this bit does not have any effect. When this bit is reset, the MAC forwards all packets irrespective of the match status of the Layer 3 and Layer 4 fields. **Value After Reset:** 0x0 **Exists:** DWCXG_L3_L4_FILTER_EN
19:17	Reserved_19_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	VTFE	R/W	VLAN Tag Filter Enable. When this bit is set, the MAC drops the VLAN tagged packets that do not match the VLAN Tag. When this bit is reset, the MAC forwards all packets irrespective of the match status of the VLAN Tag. **Value After Reset:** 0x0 **Exists:** Always
15:13	Reserved_15_13	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
12:11	DHLFRS	R/W	DA Hash Index or L3/L4 Filter Number in Receive Status. This field selects either DA hash index, L3/L4 filter number or both information to provide in receive descriptor. Values: 0x0: Use both backward compatible DA hash index and L3/L4 filter number fields in receive status. For its proper usage, program DA hash table and number of L3/L4 filters in such way that these index fields can be accommodated respective index/number. 0x1: Use combined DA hash table index and L3/L4 filter number fields to provide only DA hash table index. 0x2: Use combined DA hash table index and L3/L4 filter number fields to provide only L3/L4 filter number. 0x3: Reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_L3_L4_FILTER_EN||DWCXG_EXT_DAH_EN)

Table 11-7 Fields for Register: MAC_Packet_Filter (continued)

Bits	Name	Memory Access	Description
10	HPF	R/W	Hash or Perfect Filter. When this bit is set, the address filter passes a packet if it matches either the perfect filtering or hash filtering as set by the HMC or HUC bit. When this bit is reset and the HUC or HMC bit is set, the packet is passed only if it matches the Hash filter. **Value After Reset:** 0x0 **Exists:** DWCXG_DA_HASH_EN
9	SAF	R/W	Source Address Filter Enable. When this bit is set, the MAC compares the SA field of the received packets with the values programmed in the enabled MAC_Address registers. If the comparison fails, the MAC drops the packet. When this bit is reset, the MAC forwards the received packet to the application with updated SAF bit of the Rx Status depending on the SA address comparison. **Note:** According to the IEEE specification, Bit 47 of the SA is reserved. However, in DWC_25gmac, the MAC compares all 48 bits. The software driver must consider this while programming the MAC address registers for SA. **Value After Reset:** 0x0 **Exists:** (DWCXG_ADD_MAC_ADDR_REG>0)
8	SAIF	R/W	SA Inverse Filtering. When this bit is set, the Address Check block operates in the inverse filtering mode for SA address comparison. If the SA of a packet matches the values programmed in the MAC_Address registers, it is marked as failing the SA Address filter. When this bit is reset, if the SA of a packet does not match the values programmed in the MAC_Address registers, it is marked as failing the SA Address filter. **Value After Reset:** 0x0 **Exists:** (DWCXG_ADD_MAC_ADDR_REG>0)
7:6	PCF	R/W	Pass Control Packets. These bits control the forwarding of all control packets (including unicast and multicast Pause packets). ■ 00: The MAC filters all control packets from reaching the application. ■ 01: The MAC forwards all control packets except Pause packets to the application even if they fail the Address filter.

Table 11-7 Fields for Register: MAC_Packet_Filter (continued)

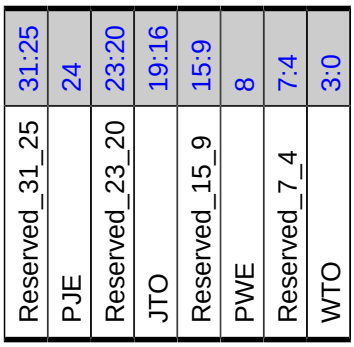
Bits	Name	Memory Access	Description
			■ 10: The MAC forwards all control packets to the application even if they fail the Address filter. ■ 11: The MAC forwards the control packets that pass the Address filter. When G9991 Ethernet encapsulated mode is enabled, this is true for DFC control packets (when both OPCODE and TIME field matches it is DFC Pause packet and when Type field matches it is DFC generic control packet). When G9991 native mode is enabled, these bits control the forwarding of DFC PAUSE packets (when both OPCODE and TIME field matches it is DFC Pause packet and the DFC generic control packet definition is not valid in this case as the Type field is not present) as follows: ■ 00/01: The MAC filters all DFC Pause packets from reaching the application. ■ 10/11: The MAC forwards all the DFC Pause packets to application. **Value After Reset:** 0x0 **Exists:** Always
5	DBF	R/W	Disable Broadcast Packets. When this bit is set, the AFM module blocks all incoming broadcast packets. In addition, it overrides all other filter settings. When this bit is reset, the AFM module passes all the received broadcast packets. **Value After Reset:** 0x0 **Exists:** Always
4	PM	R/W	Pass All Multicast. When this bit is set, it indicates that all the received packets with a multicast destination address (first bit in the destination address field is 1) are passed. When this bit is reset, filtering of multicast packet is done depending on HMC bit. **Value After Reset:** 0x0 **Exists:** Always
3	DAIF	R/W	DA Inverse Filtering. When this bit is set, the Address Check block operates in inverse filtering mode for the DA address comparison for both unicast and multicast packets. When this bit is reset, normal filtering of packets is performed. **Value After Reset:** 0x0 **Exists:** Always

Table 11-7 Fields for Register: MAC_Packet_Filter (continued)

Bits	Name	Memory Access	Description
2	HMC	R/W	Hash Multicast. When this bit is set, the MAC performs the destination address filtering of received multicast packets according to the hash table. When this bit is reset, the MAC performs the perfect destination address filtering for multicast packets, that is, it compares the DA field with the values programmed in MAC_Address registers. **Value After Reset:** 0x0 **Exists:** DWCXG_DA_HASH_EN
1	HUC	R/W	Hash Unicast. When this bit is set, the MAC performs the destination address filtering of unicast packets according to the hash table. When this bit is reset, the MAC performs a perfect destination address filtering for unicast packets, that is, it compares the DA field with the values programmed in MAC_Address registers. **Value After Reset:** 0x0 **Exists:** DWCXG_DA_HASH_EN
0	PR	R/W	Promiscuous Mode. When this bit is set, the Address Filtering module passes all incoming packets irrespective of the destination or source address. The MAC clears the SA or DA Filter Fail status bits of the Rx Status Word when PR is set. **Value After Reset:** 0x0 **Exists:** Always

11.1.4 MAC_WD_JB_Timeout

- **Description:** The Watchdog and Jabber Timeout register controls the watchdog timeout limit for the received packets and jabber timeout limit for transmitted packets.
- **Size:** 32 bits
- **Offset:** 0xc
- **Exists:** Always



31:25	24	23:20	19:16	15:9	8	7:4	3:0
Reserved_31_25	PJE	Reserved_23_20	JTO	Reserved_15_9	PWE	Reserved_7_4	WTO

Table 11-8 Fields for Register: MAC_WD_JB_Timeout

Bits	Name	Memory Access	Description
31:25	Reserved_31_25	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
24	PJE	R/W	Programmable Jabber Enable. When this bit is set and the JD bit of the MAC_Tx_Configuration register is reset, the JTO field is used as jabber timeout limit for a transmitted packet. When this bit is cleared, the jabber timeout limit for a transmitted packet is controlled by setting of JD and JE bits in MAC_Tx_Configuration and MAC_Rx_Configuration registers, respectively. **Value After Reset:** 0x0 **Exists:** Always
23:20	Reserved_23_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-8 Fields for Register: MAC_WD_JB_Timeout (continued)

Bits	Name	Memory Access	Description
19:16	JTO	R/W	Jabber Timeout. When the PJE bit is set and the JD bit of the MAC_Tx_Configuration register is reset, this field is used as jabber timeout limit for a transmitted packet. If the length of a transmitted packet exceeds the value of this field, such a packet is terminated and declared as an error packet. Encoding is as follows: ■ 4'h0: 2 KB ■ 4'h1: 3 KB ■ 4'h2: 4 KB ■ 4'h3: 5 KB ■ ... ■ 4'hC: 14 KB ■ 4'hD: 15 KB ■ 4'hE: 16383 Bytes ■ 4'hF: Reserved **Value After Reset:** 0x0 **Exists:** Always
15:9	Reserved_15_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	PWE	R/W	Programmable Watchdog Enable. When this bit is set and the WD bit of the MAC_Rx_Configuration register is reset, the WTO field is used as watchdog timeout limit for a received packet. When this bit is cleared, the watchdog timeout for a received packet is controlled by setting of WD and JE bits in MAC_Rx_Configuration register. **Value After Reset:** 0x0 **Exists:** Always
7:4	Reserved_7_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-8 Fields for Register: MAC_WD_JB_Timeout (continued)

Bits	Name	Memory Access	Description
3:0	WTO	R/W	Watchdog Timeout. When the PWE bit is set and the WD bit of the MAC_Rx_Configuration register is reset, this field is used as watchdog timeout limit for a received packet. If the length of a received packet exceeds the value of this field, such a packet is terminated and declared as an error packet. Encoding is as follows: ■ 4'h0: 2 KB ■ 4'h1: 3 KB ■ 4'h2: 4 KB ■ 4'h3: 5 KB ■ ... ■ 4'hC: 14 KB ■ 4'hD: 15 KB ■ 4'hE: 16383 Bytes ■ 4'hF: Reserved **Value After Reset:** 0x0 **Exists:** Always

11.1.5 MAC_Hash_Table_Reg0

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg0 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x10
- **Exists:**
  ((DWCXG_DA_HASH_EN&&(DWCXG_HASH_TABLE>0)&&(DWCXG_HASH_TABLE<=256)))

Table 11-9 Fields for Register: MAC_Hash_Table_Reg0

Bits	Name	Memory Access	Description
31:0	HT31T0	R/W	0th 32 bits of Hash Table. This field contains the 0th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.6 MAC_Hash_Table_Reg1

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg1 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x14
- **Exists:**
  ((DWCXG_DA_HASH_EN&&(DWCXG_HASH_TABLE>0)&&(DWCXG_HASH_TABLE<=256)))

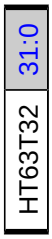


Table 11-10 Fields for Register: MAC_Hash_Table_Reg1

Bits	Name	Memory Access	Description
31:0	HT63T32	R/W	1th 32 bits of Hash Table. This field contains the 1th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.7 MAC_Hash_Table_Reg2

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg2 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x18
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>64)

Table 11-11 Fields for Register: MAC_Hash_Table_Reg2

Bits	Name	Memory Access	Description
31:0	HT95T64	R/W	2th 32 bits of Hash Table. This field contains the 2th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.8 MAC_Hash_Table_Reg3

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg3 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x1c
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>64)

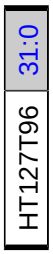


Table 11-12 Fields for Register: MAC_Hash_Table_Reg3

Bits	Name	Memory Access	Description
31:0	HT127T96	R/W	3th 32 bits of Hash Table. This field contains the 3th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.9 MAC_Hash_Table_Reg4

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg4 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x20
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>128)

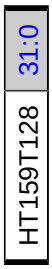


Table 11-13 Fields for Register: MAC_Hash_Table_Reg4

Bits	Name	Memory Access	Description
31:0	HT159T128	R/W	4th 32 bits of Hash Table. This field contains the 4th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.10 MAC_Hash_Table_Reg5

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg5 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x24
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>128)

Table 11-14 Fields for Register: MAC_Hash_Table_Reg5

Bits	Name	Memory Access	Description
31:0	HT191T160	R/W	5th 32 bits of Hash Table. This field contains the 5th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.11 MAC_Hash_Table_Reg6

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg6 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x28
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>128)

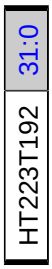


Table 11-15 Fields for Register: MAC_Hash_Table_Reg6

Bits	Name	Memory Access	Description
31:0	HT223T192	R/W	6th 32 bits of Hash Table. This field contains the 6th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.12 MAC_Hash_Table_Reg7

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg7 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x2c
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>128)

Table 11-16 Fields for Register: MAC_Hash_Table_Reg7

Bits	Name	Memory Access	Description
31:0	HT255T224	R/W	7th 32 bits of Hash Table. This field contains the 7th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.13 MAC_Hash_Table_Reg8

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg8 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x30
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>256)

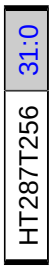


Table 11-17 Fields for Register: MAC_Hash_Table_Reg8

Bits	Name	Memory Access	Description
31:0	HT287T256	R/W	8th 32 bits of Hash Table. This field contains the 8th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.14 MAC_Hash_Table_Reg9

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg9 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x34
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>256)

Table 11-18 Fields for Register: MAC_Hash_Table_Reg9

Bits	Name	Memory Access	Description
31:0	HT319T288	R/W	9th 32 bits of Hash Table. This field contains the 9th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.15 MAC_Hash_Table_Reg10

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg10 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x38
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>256)

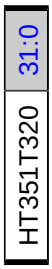


Table 11-19 Fields for Register: MAC_Hash_Table_Reg10

Bits	Name	Memory Access	Description
31:0	HT351T320	R/W	10th 32 bits of Hash Table. This field contains the 10th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.16 MAC_Hash_Table_Reg11

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg11 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x3c
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>256)

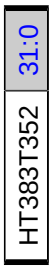


Table 11-20 Fields for Register: MAC_Hash_Table_Reg11

Bits	Name	Memory Access	Description
31:0	HT383T352	R/W	11th 32 bits of Hash Table. This field contains the 11th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.17 MAC_Hash_Table_Reg12

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg12 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x40
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>256)

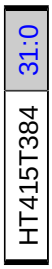


Table 11-21 Fields for Register: MAC_Hash_Table_Reg12

Bits	Name	Memory Access	Description
31:0	HT415T384	R/W	12th 32 bits of Hash Table. This field contains the 12th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.18 MAC_Hash_Table_Reg13

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg13 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x44
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>256)

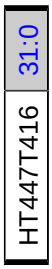


Table 11-22 Fields for Register: MAC_Hash_Table_Reg13

Bits	Name	Memory Access	Description
31:0	HT447T416	R/W	13th 32 bits of Hash Table. This field contains the 13th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.19 MAC_Hash_Table_Reg14

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg14 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x48
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>256)

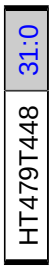


Table 11-23 Fields for Register: MAC_Hash_Table_Reg14

Bits	Name	Memory Access	Description
31:0	HT479T448	R/W	14th 32 bits of Hash Table. This field contains the 14th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.20 MAC_Hash_Table_Reg15

- **Description:** You can specify the width of the hash table by using the Hash Table Size option in coreConsultant.
  The 64-bit, 128-bit, or 256-bit hash table is used for group address filtering. For hash filtering, the content of the destination address in the incoming packet is passed through the CRC logic and the upper six (seven or eight in 128- or 256-bit Hash) bits of the CRC are used to index the content of the Hash table. The most significant bits determines the register to be used (Hash Table Register X), and the least significant five bits determine the bit within the register. For example, a hash value of 7b'1100000 (in 128-bit Hash) selects Bit 0 of the Hash Table Register 3 and a value of 8b'10111111 (in 256-bit Hash) selects Bit 31 of the Hash Table Register 5.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the DA (See IEEE 802.3-2018, Section 3.2.8 for the steps to calculate CRC32).
  - Perform bit-wise reversal for the value obtained in Step 1.
  - Take the upper 6 (or 7 or 8) bits from the value obtained in Step 2.

  If the corresponding bit value of the MAC_Hash_Table_Reg15 register is 1'b1, the packet is accepted. Otherwise, it is rejected. If the PM bit is set in MAC_Packet_Filter, all multicast packets are accepted regardless of the multicast hash values.
  If the Hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the Hash Table Register X registers are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x4c
- **Exists:** DWCXG_INT_DAH_EN&&(DWCXG_HASH_TABLE>256)

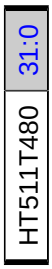


Table 11-24 Fields for Register: MAC_Hash_Table_Reg15

Bits	Name	Memory Access	Description
31:0	HT511T480	R/W	15th 32 bits of Hash Table. This field contains the 15th 32 Bits of the Hash table. **Value After Reset:** 0x0 **Exists:** Always

11.1.21 MAC_DAH_TBL_SIZE

- **Description:** This register is used to configure external DA Hash table size based on the selected CRC bits to be used as DA hash table indexing.
- **Size:** 32 bits
- **Offset:** 0x4c
- **Exists:** DWCXG_DA_HASH_EN&&(DWCXG_HASH_TABLE>256)

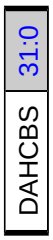


Table 11-25 Fields for Register: MAC_DAH_TBL_SIZE

Bits	Name	Memory Access	Description
31:0	DAHCBS	R/W	DA Hash CRC Bits Selection This field selects which bits of internally generated DA Hash 32-bit wide CRC to be used for external DA Hash-table indexing. Set respective bit to 1'b1, which is required to include as hash table index considering maximum supported hash table size. Based on selected bits (bit which is set to 1), its concatenation from LSB to MSB is used as hash table index. **Value After Reset:** 0x0 **Exists:** Always

11.1.22 MAC_VLAN_Tag

- **Description:** The VLAN Tag register identifies the IEEE 802.1Q VLAN type packets. <This register hold control fields for normal VLAN perfect match and hash filtering, only when Extended Internal Rx VLAN Filters and External Rx VLAN Tag Lookup based perfect filter features are not selected.
- **Size:** 32 bits
- **Offset:** 0x50
- **Exists:** !DWCXG_ERVFE

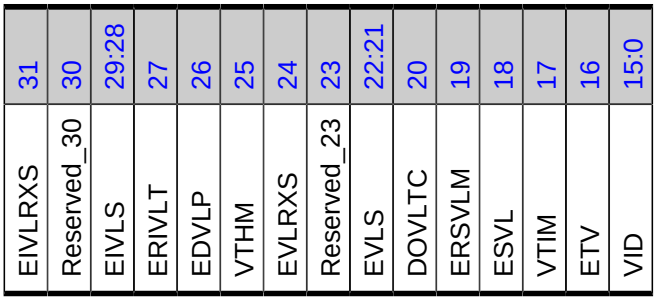


Table 11-26 Fields for Register: MAC_VLAN_Tag

Bits	Name	Memory Access	Description
31	EIVLRXS	R/W	Enable Inner VLAN Tag in Rx Status. When this bit is set, the MAC provides the inner VLAN Tag in the Rx status. When this bit is reset, the MAC does not provide the inner VLAN Tag in Rx status. **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN
30	Reserved_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29:28	EIVLS	R/W	Enable Inner VLAN Tag Stripping on Receive. This field indicates the stripping operation on inner VLAN Tag in the received packet: ■ 2'b00: Do not strip ■ 2'b01: Strip if VLAN filter passes ■ 2'b10: Strip if VLAN filter fails ■ 2'b11: Always strip **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN

Table 11-26 Fields for Register: MAC_VLAN_Tag (continued)

Bits	Name	Memory Access	Description
27	ERIVLT	R/W	Enable Inner VLAN Tag for Perfect Filtering. When this bit and the EDVLP field are set, the MAC receiver enables operation on the inner VLAN Tag (if present). When this bit is reset, the MAC receiver enables operation on the outer VLAN Tag (if present). The ERSVLM bit determines which VLAN type is enabled for filtering or matching. **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN
26	EDVLP	R/W	Enable Double VLAN Processing. When this bit is set, the MAC enables processing of up to two VLAN Tags on Tx and Rx (if present). When this bit is reset, the MAC enables processing of up to one VLAN Tag on Tx and Rx (if present). **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN
25	VTHM	R/W	VLAN Tag Hash Table Match Enable. When this bit is set, the most significant four bits of CRC of VLAN Tag are used to index the content of the MAC_VLAN_Hash_Table register. A value of 1 in the VLAN Hash Table register, corresponding to the index, indicates that the packet matched the VLAN hash table. When the ETV bit is set, the CRC of the 12-bit VLAN Identifier (VID) is used for comparison. When the ETV bit is reset, the CRC of the 16-bit VLAN tag is used for comparison. When this bit is reset, the VLAN Hash Match operation is not performed. **Value After Reset:** 0x0 **Exists:** DWCXG_VLAN_HASH_EN
24	EVLRXS	R/W	Enable VLAN Tag in Rx status. When this bit is set, MAC provides the outer VLAN Tag in the Rx status. When this bit is reset, the MAC does not provide the outer VLAN Tag in Rx status. **Value After Reset:** 0x0 **Exists:** Always
23	Reserved_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-26 Fields for Register: MAC_VLAN_Tag (continued)

Bits	Name	Memory Access	Description
22:21	EVLS	R/W	Enable VLAN Tag Stripping on Receive. This field indicates the stripping operation on the outer VLAN Tag in the received packet: ■ 2'b00: Do not strip ■ 2'b01: Strip if VLAN filter passes ■ 2'b10: Strip if VLAN filter fails ■ 2'b11: Always strip **Value After Reset:** 0x0 **Exists:** Always
20	DOVLTC	R/W	Disable VLAN Type Check. When this bit is set, the MAC does not check whether the VLAN Tag specified by the ERIVLT bit is of type S-VLAN or C-VLAN. When this bit is reset, the MAC filters or matches the VLAN Tag specified by the ERIVLT bit only when VLAN Tag type is similar to the one specified by the ERSVLM bit. **Value After Reset:** 0x0 **Exists:** Always
19	ERSVLM	R/W	Enable Receive S-VLAN Match. When this bit is set, the MAC receiver enables filtering or matching for S-VLAN (Type = 0x88A8) packets. When this bit is reset, the MAC receiver enables filtering or matching for C-VLAN (Type = 0x8100) packets. The ERIVLT bit determines the VLAN tag position considered for filtering or matching. **Value After Reset:** 0x0 **Exists:** Always
18	ESVL	R/W	Enable S-VLAN. When this bit is set, the MAC transmitter and receiver consider the S-VLAN packets (Type = 0x88A8) as valid VLAN tagged packets. **Value After Reset:** 0x0 **Exists:** Always
17	VTIM	R/W	VLAN Tag Inverse Match Enable. When this bit is set, this bit enables the VLAN Tag inverse matching. The packets without matching VLAN Tag are marked as matched. When reset, this bit enables the VLAN Tag perfect matching. The packets with matched VLAN Tag are marked as matched. **Value After Reset:** 0x0 **Exists:** Always

Table 11-26 Fields for Register: MAC_VLAN_Tag (continued)

Bits	Name	Memory Access	Description
16	ETV	R/W	Enable 12-Bit VLAN Tag Comparison. When this bit is set, a 12-bit VLAN identifier is used for comparing and filtering instead of the complete 16-bit VLAN tag. Bits[11:0] of VLAN tag are compared with the corresponding field in the received VLAN-tagged packet. Similarly, when enabled, only 12 bits of the VLAN tag in the received packet are used for hash-based VLAN filtering. When this bit is reset, all 16 bits of the 15th and 16th bytes of the received VLAN packet are used for comparison and VLAN hash filtering. **Value After Reset:** 0x0 **Exists:** Always
15:0	VID	R/W	VLAN Tag Identifier for Receive Packets. This field contains the IEEE 802.1Q VLAN tag to identify the VLAN packets. This VLAN tag identifier is compared to the 15th and 16th bytes of the packets being received for VLAN packets. The following list describes the bits of this field: ■ Bits[15:13]: User Priority ■ Bit 12: Canonical Format Indicator (CFI) or Drop Eligible Indicator (DEI) ■ Bits[11:0]: VLAN Identifier (VID) field of VLAN tag When the ETV bit is set, only the VID is used for comparison. If this field ([11:0] if ETV is set) is all zeros, the MAC does not check the 15th and 16th bytes for VLAN tag comparison and declares all packets with Type field value of 0x8100 or 0x88a8 as VLAN packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.23 MAC_VLAN_Tag_Ctrl

- **Description:** MAC_VLAN_Tag_Ctrl register is a re-defined version of MAC_VLAN_Tag register. This register holds the control and addressing fields required for indirect accessing of the MAC_VLAN_Tag_Filter registers, when Extended Internal Rx VLAN Filters are enabled.
  And also it holds the control fields for MAC VLAN Hash Filter and perfect filtering, when either of Extended Internal Rx VLAN Filters or External Rx VLAN Tag Lookup based perfect filter feature is selected.
- **Size:** 32 bits
- **Offset:** 0x50
- **Exists:** DWCXG_ERVFE

31	30	29:28	27	26	25	24	23	22:21	20	19	18	17	16	15:14	13:12	11:8	7:2	1	0
EIVLRXS	Reserved_30	EIVLS	ERIVLT	EDVLP	VTHM	EVLRXS	Reserved_23	EVLS	DOVLTC	ERSVLM	ESVL	VTIM	ETV	ERIVTL	EROVTL	Reserved_11_8	OFS	CT	OB

Table 11-27 Fields for Register: MAC_VLAN_Tag_Ctrl

Bits	Name	Memory Access	Description
31	EIVLRXS	R/W	Enable Inner VLAN Tag in Rx Status. When this bit is set, the MAC provides the inner VLAN Tag in the Rx status. When this bit is reset, the MAC does not provide the inner VLAN Tag in Rx status. **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN
30	Reserved_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29:28	EIVLS	R/W	Enable Inner VLAN Tag Stripping on Receive. This field indicates the stripping operation on inner VLAN Tag in the received packet: ■ 2'b00: Do not strip ■ 2'b01: Strip if VLAN filter passes ■ 2'b10: Strip if VLAN filter fails ■ 2'b11: Always strip **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN

Table 11-27 Fields for Register: MAC_VLAN_Tag_Ctrl (continued)

Bits	Name	Memory Access	Description
27	ERIVLT	R/W	Enable Inner VLAN Tag for VLAN hash filtering. When this bit and the EDVLP field are set, the MAC receiver enables hash operation on the inner VLAN Tag (if present). When this bit is reset, the MAC receiver enables hash operation on the outer VLAN Tag (if present). The ERSVLM bit determines which VLAN type is enabled for filtering or matching. **Value After Reset:** 0x0 **Exists:** DWCXG_VLAN_HASH_EN&&DWCXG_DOUBLE_VLAN_EN
26	EDVLP	R/W	Enable Double VLAN Processing. When this bit is set, the MAC enables processing of up to two VLAN Tags on Tx and Rx (if present). When this bit is reset, the MAC enables processing of up to one VLAN Tag on Tx and Rx (if present). **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN
25	VTHM	R/W	VLAN Tag Hash Table Match Enable. When this bit is set, the most significant four bits of CRC of VLAN Tag are used to index the content of the MAC_VLAN_Hash_Table register. A value of 1 in the VLAN Hash Table register, corresponding to the index, indicates that the packet matched the VLAN hash table. When the ETV bit is set, the CRC of the 12-bit VLAN Identifier (VID) is used for comparison. When the ETV bit is reset, the CRC of the 16-bit VLAN tag is used for comparison. When this bit is reset, the VLAN Hash Match operation is not performed. **Value After Reset:** 0x0 **Exists:** DWCXG_VLAN_HASH_EN
24	EVLRXS	R/W	Enable VLAN Tag in Rx status. When this bit is set, MAC provides the outer VLAN Tag in the Rx status. When this bit is reset, the MAC does not provide the outer VLAN Tag in Rx status. **Value After Reset:** 0x0 **Exists:** Always
23	Reserved_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-27 Fields for Register: MAC_VLAN_Tag_Ctrl (continued)

Bits	Name	Memory Access	Description
22:21	EVLS	R/W	Enable VLAN Tag Stripping on Receive. This field indicates the stripping operation on the outer VLAN Tag in received packet: ■ 2'b00: Do not strip ■ 2'b01: Strip if VLAN filter passes ■ 2'b10: Strip if VLAN filter fails ■ 2'b11: Always strip **Value After Reset:** 0x0 **Exists:** Always
20	DOVLTC	R/W	Disable VLAN Type Check for VLAN hash filtering. When this bit is set, the MAC does not check whether the VLAN Tag specified by the ERIVLT bit is of type S-VLAN or C-VLAN. When this bit is reset, the MAC filters or matches the VLAN Tag specified by the ERIVLT bit only when VLAN Tag type is similar to the one specified by the ERSVLM bit. **Value After Reset:** 0x0 **Exists:** DWCXG_VLAN_HASH_EN
19	ERSVLM	R/W	Enable Receive S-VLAN Match for VLAN hash filtering. When this bit is set, the MAC receiver enables filtering or matching for S-VLAN (Type = 0x88A8) packets. When this bit is reset, the MAC receiver enables filtering or matching for C-VLAN (Type = 0x8100) packets. The ERIVLT bit determines the VLAN tag position considered for filtering or matching. **Value After Reset:** 0x0 **Exists:** DWCXG_VLAN_HASH_EN
18	ESVL	R/W	Enable S-VLAN. When this bit is set, the MAC transmitter and receiver consider the S-VLAN packets (Type = 0x88A8) as valid VLAN tagged packets. **Value After Reset:** 0x0 **Exists:** Always

Table 11-27 Fields for Register: MAC_VLAN_Tag_Ctrl (continued)

Bits	Name	Memory Access	Description
17	VTIM	R/W	VLAN Tag Inverse Match/Result Enable. When this bit is set, it enables the VLAN Tag inverse matching for non-Lookup based filtering. A match is marked when Packets have non-matching VLAN tags.For lookup-based filtering, it inverses actual result. When reset, it enables the VLAN Tag perfect matching for non-Lookup based filtering. A match is marked when packets have matching VLAN tags.For lookup-based filtering, it uses actual result. **Value After Reset:** 0x0 **Exists:** Always
16	ETV	R/W	Enable 12-Bit VLAN Tag Comparison for VLAN hash filtering. When this bit is set, a 12-bit VLAN identifier is used for VLAN hash filtering instead of the complete 16-bit VLAN tag. When this bit is reset, all 16 bits of the 15th and 16th bytes of the received VLAN packet are used for VLAN hash filtering. **Value After Reset:** 0x0 **Exists:** DWCXG_VLAN_HASH_EN
15:14	ERIVTL	R/W	Enable external Receive Inner VLAN Tag Lookup based perfect filtering and routing. When EDVLP field is set to 1, this field configures external receive Inner VLAN Tag lookup based perfect filtering and routing. Non-zero value enables usage of Inner VLAN Tag Lookup with different VLAN Type. When both Outer and Inner VLAN Tag lookup are enabled, respective lookup's UNL entry field (Use Next Lookup) is used to select usage of Outer and/or Inner lookup based on the receive VLAN Tag. **Note:** This external lookup always does 12bit VLAN Tag Comparison **Values:** ■ 0x0 (BYPS_LKP): Bypass Inner VLAN Tag Lookup ■ 0x1 (EN_CTYP): Enable Inner VLAN Tag Lookup with C-VLAN type (0x8100) ■ 0x2 (EN_STYP): When EDVLP field is set to 1, enable Inner VLAN Tag Lookup with S-VLAN type (0x88A8). Otherwise it uses C-VLAN type. ■ 0x3 (EN_BOTH): When EDVLP field is set to 1, enable Inner VLAN Tag Lookup with both C & S VLAN type. Otherwise it uses C-VLAN type. **Value After Reset:** 0x0 **Exists:** (DWCXG_EXT_RVTL_EN&&DWCXG_DOUBLE_VLAN_EN)

Table 11-27 Fields for Register: MAC_VLAN_Tag_Ctrl (continued)

Bits	Name	Memory Access	Description
13:12	EROVTL	R/W	Enable external Receive Outer VLAN Tag Lookup based perfect filtering and routing. This field configures external receive Outer VLAN Tag lookup based perfect filtering and routing. Non-zero value enables usage of Outer VLAN Tag Lookup with different VLAN Type. When both Outer an Inner VLAN Tag lookup are enabled, respective lookup's UNL entry field (Use Next Lookup) is used to select usage of Outer and/or Inner lookup based on the receive VLAN Tag. **Note:** This external lookup always does 12bit VLAN Tag Comparison **Values:** ■ 0x0 (BYPS_LKP): Bypass Outer VLAN Tag Lookup ■ 0x1 (EN_CTYP): Enable Outer VLAN Tag Lookup with C-VLAN type (0x8100) ■ 0x2 (EN_STYP): When EDVLP field is set to 1, enable Outer VLAN Tag Lookup with S-VLAN type (0x88A8). Otherwise it uses C-VLAN type. ■ 0x3 (EN_BOTH): When EDVLP field is set to 1, enable Outer VLAN Tag Lookup with both C & S VLAN type. Otherwise it uses C-VLAN type. **Value After Reset:** 0x0 **Exists:** DWCXG_ERVFE&&(DWCXG_NRVF==4096)
11:8	Reserved_11_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7:2	OFS	R/W	Offset. This field holds the address offset of the MAC VLAN Tag Filter register which the application is trying to access. The width of the field depends on the number of MAC VLAN Tag registers enabled. **Value After Reset:** 0x0 **Exists:** DWCXG_ERVFE&&(DWCXG_NRVF>1)&&(DWCXG_NRVF<4096)

Table 11-27 Fields for Register: MAC_VLAN_Tag_Ctrl (continued)

Bits	Name	Memory Access	Description
1	CT	R/W	Command Type. This bit indicates if the current register access is a read or a write. When set, it indicate a read operation. When reset, it indicates a write operation. **Value After Reset:** 0x0 **Exists:** DWCXG_ERVFE&&(DWCXG_NRVF>1)&&(DWCXG_NRVF<4096)
0	OB	R/W	Operation Busy. This bit is set along with a read or write command for initiating the indirect access to per VLAN Tag Filter register. The next indirect register access can be initiated only after this bit is reset. During a write operation, the bit is reset only after the data has been written into the Per VLAN Tag register. During a read operation, the data must be read from the MAC_VLAN_Tag_Data register only after this bit is reset. **Value After Reset:** 0x0 **Exists:** DWCXG_ERVFE&&(DWCXG_NRVF>1)&&(DWCXG_NRVF<4096) **Testable:** untestable

11.1.24 MAC_VLAN_Tag_Data

- **Description:** This register holds the read/write data for Indirect Access of the Per MAC_VLAN_Tag_Filter registers. During the read access, this field contains valid read data only after the OB bit is reset. During the write access, this field must be valid prior to setting the OB bit in the MAC_VLAN_Tag_Ctrl register.
- **Size:** 32 bits
- **Offset:** 0x54
- **Exists:** DWCXG_ERVFE&&(DWCXG_NRVF>1)&&(DWCXG_NRVF<4096)

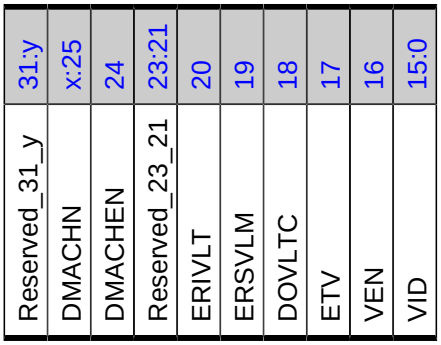


Table 11-28 Fields for Register: MAC_VLAN_Tag_Data

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_BN_RCH_BN_RCHL_W + 25
x:25	DMACHN	R/W	Refer DMACHN field of MAC_VLAN_Tag_Filter(#i) indirect register. **Value After Reset:** 0x0 **Exists:** (DWCXG>1) **Range Variable[x]:** DWCXG_BN_RCH_BN_RCHL_W + 24
24	DMACHEN	R/W	Refer DMACHEN field of MAC_VLAN_Tag_Filter(#i) indirect register. **Value After Reset:** 0x0 **Exists:** (DWCXG>1)
23:21	Reserved_23_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-28 Fields for Register: MAC_VLAN_Tag_Data (continued)

Bits	Name	Memory Access	Description
20	ERIVLT	R/W	Refer ERIVLT field of MAC_VLAN_Tag_Filter(#i) indirect register. **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN
19	ERSVLM	R/W	Refer ERSVLM field of MAC_VLAN_Tag_Filter(#i) indirect register. **Value After Reset:** 0x0 **Exists:** Always
18	DOVLTC	R/W	Refer DOVLTC field of MAC_VLAN_Tag_Filter(#i) indirect register. **Value After Reset:** 0x0 **Exists:** Always
17	ETV	R/W	Refer ETV field of MAC_VLAN_Tag_Filter(#i) indirect register. **Value After Reset:** 0x0 **Exists:** Always
16	VEN	R/W	Refer VEN field of MAC_VLAN_Tag_Filter(#i) indirect register. **Value After Reset:** 0x0 **Exists:** Always
15:0	VID	R/W	Refer VID field of MAC_VLAN_Tag_Filter(#i) indirect register. **Value After Reset:** 0x0 **Exists:** Always

11.1.25 MAC_VLAN_Tag_Filter(#i) (for i = 0; i <= DWCXG_NRVF-1)

- **Description:** This register contains per VLAN Tag filter control information.
- **Size:** 32 bits
- **Offset:** 0x54
- **Exists:** DWCXG_INT_RVTL_EN&&(DWCXG_NRVF> 0)

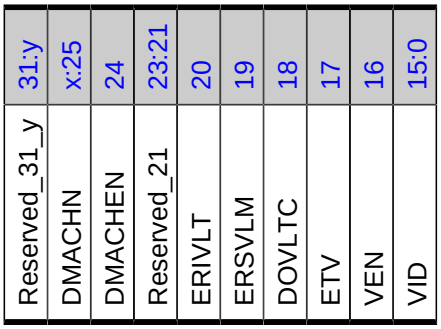


Table 11-29 Fields for Register: MAC_VLAN_Tag_Filter(#i) (for i = 0; i <= DWCXG_NRVF-1)

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_BN_RCH_BN_RCHL_W + 25
x:25	DMACHN	R/W	DMA Channel Number. The DMA Channel number to which the VLAN Tagged Packet is to be routed if it passes this VLAN Tag Filter is programmed in this field. This field is valid only when DMACHEN=1. If the Routing based on the VLAN Tag Filter is not necessary, this field need not be programmed. **Value After Reset:** 0x0 **Exists:** (DWCXG>1) **Range Variable[x]:** DWCXG_BN_RCH_BN_RCHL_W + 24
24	DMACHEN	R/W	DMA Channel Number Enable. If this bit is set, it enables the DMA Channel Number value programmed in the field DMACH. If this is reset, then the Routing does not occur based on the VLAN Filter result. The packet is routed based on the DA Based DMA Channel Routing. **Value After Reset:** 0x0 **Exists:** (DWCXG>1)

Table 11-29 Fields for Register: MAC_VLAN_Tag_Filter(#i) (for i = 0; i <= DWCXG_NRVF-1) (continued)

Bits	Name	Memory Access	Description
23:21	Reserved_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
20	ERIVLT	R/W	Enable Inner VLAN Tag. When this bit and the EDVLP field are set, the MAC receiver enables operation on the inner VLAN Tag (if present). When this bit is reset, the MAC receiver enables operation on the outer VLAN Tag (if present). The ERSVLM bit determines which VLAN type is enabled for filtering or matching. **Value After Reset:** 0x0 **Exists:** DWCXG_DOUBLE_VLAN_EN
19	ERSVLM	R/W	Enable Receive S-VLAN Match. When this bit is set, the MAC receiver enables filtering or matching for S-VLAN (Type = 0x88A8) packets. When this bit is reset, the MAC receiver enables filtering or matching for CVLAN (Type = 0x8100) packets. The ERIVLT bit determines the VLAN tag position considered for filtering or matching. **Value After Reset:** 0x0 **Exists:** Always
18	DOVLTC	R/W	Disable VLAN Type Check. When this bit is set, the MAC does not check whether the VLAN Tag specified by the ERIVLT bit is of type S-VLAN or C-VLAN. When this bit is reset, the MAC filters or matches the VLAN Tag specified by the ERIVLT bit only when VLAN Tag type is similar to the one specified by the ERSVLM bit. **Value After Reset:** 0x0 **Exists:** Always
17	ETV	R/W	Enable 12-Bit VLAN Tag Comparison. When this bit is set, a 12-bit VLAN identifier is used for comparing and filtering instead of the complete 16-bit VLAN tag. Bits[11:0] of VLAN tag are compared with the corresponding field in the received VLAN-tagged packet. Similarly, when enabled, only 12 bits of the VLAN tag in the received packet are used for hash-based VLAN filtering. When this bit is reset, all 16 bits of the 15th and 16th bytes of the received VLAN packet are used for comparison and VLAN hash filtering. **Value After Reset:** 0x0 **Exists:** Always

Table 11-29 Fields for Register: MAC_VLAN_Tag_Filter(#i) (for i = 0; i <= DWCXG_NRVF-1) (continued)

Bits	Name	Memory Access	Description
16	VEN	R/W	VLAN Tag Enable. This bit is used to enable or disable the VLAN Tag. When this bit is set, the MAC compares the VLAN Tag of received packet with the VLAN Tag ID. When this bit is not set, no comparison is performed irrespective of the programming of the other fields. **Value After Reset:** 0x0 **Exists:** Always
15:0	VID	R/W	VLAN Tag ID. This field holds the VLAN Tag value which is used by the MAC for perfect comparison. It is valid when VLAN Tag Enable is set. **Value After Reset:** 0x0 **Exists:** Always

11.1.26 MAC_VLAN_Hash_Table

- **Description:** When the ERSVLM bit is set of MAC_VLAN_Tag_Ctrl, the 16-bit VLAN Hash Table register is used for group address filtering based on the VLAN tag. For hash filtering, the content of the 16-bit VLAN tag or 12-bit VLAN ID (based on the ETV bit of MAC_VLAN_Tag_Ctrl register) in the incoming packet is passed through the CRC logic. The upper four bits of the calculated CRC are used to index the contents of the VLAN Hash table. For example, a hash value of 4b'1000 selects Bit 8 of the VLAN Hash table.
  The hash value of the destination address is calculated in the following way:
  - Calculate the 32-bit CRC for the VLAN tag or ID (For steps to calculate CRC32, see Section 3.2.8 of IEEE 802.3-2018).
  - Perform bit-wise reversal for the value obtained in step 1.
  - Take the upper four bits from the value obtained in step 2.

  If the VLAN hash Table register is configured to be double-synchronized to the (X)GMII clock domain, the synchronization is triggered only when Bits[15:8] (in little-endian mode) or Bits[7:0] (in big-endian mode) of this register are written.
  If double-synchronization is enabled, consecutive writes to this register must be performed after at least four clock cycles in the destination clock domain.
- **Size:** 32 bits
- **Offset:** 0x58
- **Exists:** (DWCXG_VLAN_HASH_EN)

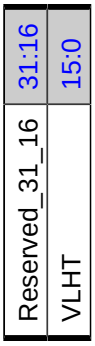
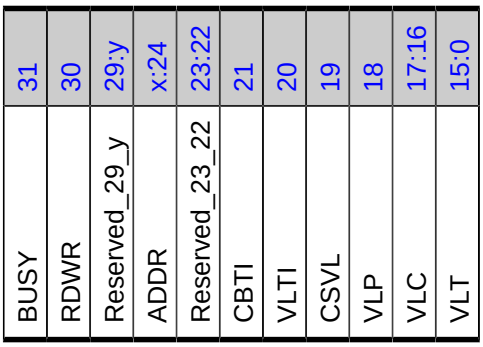


Table 11-30 Fields for Register: MAC_VLAN_Hash_Table

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:0	VLHT	R/W	VLAN Hash Table. This field contains the 16-bit VLAN Hash Table. **Value After Reset:** 0x0 **Exists:** Always

11.1.27 MAC_VLAN_Incl

- **Description:** The VLAN Tag Inclusion or Replacement register contains the VLAN tag for insertion or replacement in the Transmit packets. It also contains the VLAN tag insertion controls.
- **Size:** 32 bits
- **Offset:** 0x60
- **Exists:** (DWCXG_SA_VLAN_INS_CTRL_EN)



31	30	29:y	x:24	23:22	21	20	19	18	17:16	15:0
BUSY	RDWR	Reserved_29_y	ADDR	Reserved_23_22	CBTI	VLTI	CSVL	VLP	VLC	VLT

Table 11-31 Fields for Register: MAC_VLAN_Incl

Bits	Name	Memory Access	Description
31	BUSY	R	BUSY. This bit indicates the status of the read/write operation of indirect access to the queue/channel specific VLAN inclusion register. When this bit is reset, write to this register is complete in write operation or read data is valid in read operation When this bit is set, No further writes are allowed to this register. This does not have any effect when CBTI is reset. **Value After Reset:** 0x0 **Exists:** DWCXG_CBTI_EN **Testable:** untestable
30	RDWR	R/W	Read Write Control. This bit controls the read or write operation for indirectly accessing the queue/channel specific VLAN Inclusion register. When set, indicates write operation and when reset indicates read operation This does not have any effect when CBTI is reset. **Value After Reset:** 0x0 **Exists:** DWCXG_CBTI_EN
29:y	Reserved_29_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_TXQW + 24

Table 11-31 Fields for Register: MAC_VLAN_Incl (continued)

Bits	Name	Memory Access	Description
x:24	ADDR	R/W	Address. This field selects one of the queue/channel specific VLAN Inclusion register for read/write access. This does not have any effect when CBTI is reset. **Value After Reset:** 0x0 **Exists:** DWCXG_CBTI_EN **Range Variable[x]:** DWCXG_TXQW + 23
23:22	Reserved_23_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
21	CBTI	R/W	Channel Based Tag Insertion. When this bit is set, outer VLAN tag is inserted for every packets transmitted by the MAC. The tag value is taken from the queue/channel specific VLAN tag register. This overrides VLTI, VLP, VLC and VLT fields of this register When this bit is set, a write operation to byte 3 of this register initiates the read/write access to the indirect register. When reset, outer VLAN operation is based on the setting of VLTI, VLP, VLC, and VLT fields of this register. **Value After Reset:** 0x0 **Exists:** DWCXG_CBTI_EN
20	VLTI	R/W	VLAN Tag Input. When this bit is set, it indicates that the VLAN tag to be inserted or replaced in Tx packet must be taken from the following: ■ The mti_vlan_tag_i signal in XGMAC-CORE configurations ■ The control word in XGMAC-MTL configurations ■ The Tx descriptor in XGMAC-AXI configurations When this bit is reset, it indicates that the VLAN tag to be inserted or replaced in Tx packet must be taken from the VLT field of this register. **Value After Reset:** 0x0 **Exists:** Always
19	CSVL	R/W	C-VLAN or S-VLAN. When this bit is set, S-VLAN type (0x88A8) is inserted or replaced in the 13th and 14th bytes of transmitted packets. When this bit is reset, C-VLAN type (0x8100) is inserted or replaced in the 13th and 14th bytes of transmitted packets. **Value After Reset:** 0x0 **Exists:** Always

Table 11-31 Fields for Register: MAC_VLAN_Incl (continued)

Bits	Name	Memory Access	Description
18	VLP	R/W	VLAN Priority Control. When this bit is set, the control bits[17:16] are used for VLAN deletion, insertion, or replacement. When this bit is reset, bits[17:16] are ignored and the following is used: ■ The mti_vlan_ctrl_i signal in XGMAC-CORE configurations ■ The control word in XGMAC-MTL configurations ■ The Tx descriptor in XGMAC-AXI configurations **Value After Reset:** 0x0 **Exists:** Always
17:16	VLC	R/W	VLAN Tag Control in Transmit Packets. ■ 2'b00: No VLAN tag deletion, insertion, or replacement ■ 2'b01: VLAN tag deletion The MAC removes the VLAN type (bytes 13 and 14) and VLAN tag (bytes 15 and 16) of all transmitted packets with VLAN tags. ■ 2'b10: VLAN tag insertion The MAC inserts VLT in bytes 15 and 16 of the packet after inserting the Type value (0x8100 or 0x88a8) in bytes 13 and 14. This operation is performed on all transmitted packets, irrespective of whether they already have a VLAN tag. ■ 2'b11: VLAN tag replacement The MAC replaces VLT in bytes 15 and 16 of all VLAN-type transmitted packets (Bytes 13 and 14 are 0x8100 or 0x88a8). **Note:** Changes to this field take effect only on the start of a packet. If you write this register field when a packet is being transmitted, only the subsequent packet can use the updated value, that is, the current packet does not use the updated value. **Value After Reset:** 0x0 **Exists:** Always

Table 11-31 Fields for Register: MAC_VLAN_Incl (continued)

Bits	Name	Memory Access	Description
15:0	VLT	R/W	VLAN Tag for Transmit Packets. This field contains the value of the VLAN tag to be inserted or replaced. The value must only be changed when the transmit lines are inactive or during the initialization phase. Bits[15:13] are the User Priority field, Bit 12 is the CFI/DEI field, and Bits[11:0] are the VID field in the VLAN tag. The following list describes the bits of this field: ■ Bits[15:13]: User Priority ■ Bit 12: Canonical Format Indicator (CFI) or Drop Eligible Indicator (DEI) ■ Bits[11:0]: VLAN Identifier (VID) field of VLAN tag **Value After Reset:** 0x0 **Exists:** Always

11.1.28 MAC_VLAN_Incl(#i) (for i = 0; i <= DWCXG_NUM_TXQ-1)

- **Description:** The per Tx Queue VLAN Tag Inclusion register contains the VLAN tag for insertion in the Transmit packets from respective Tx Queue. It also contains the VLAN tag insertion controls.
- **Size:** 32 bits
- **Offset:** 0x60
- **Exists:** DWCXG_CBTI_EN&&(DWCXG_NUM_TXQ>0)

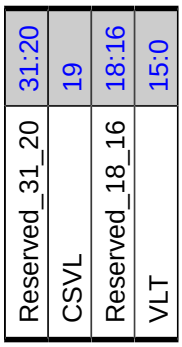


Table 11-32 Fields for Register: MAC_VLAN_Incl(#i) (for i = 0; i <= DWCXG_NUM_TXQ-1)

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19	CSVL	R/W	C-VLAN or S-VLAN. When this bit is set, S-VLAN type (0x88A8) is inserted or replaced in the 13th and 14th bytes of transmitted packets. When this bit is reset, C-VLAN type (0x8100) is inserted or replaced in the 13th and 14th bytes of transmitted packets. **Value After Reset:** 0x0 **Exists:** Always
18:16	Reserved_18_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-32 Fields for Register: MAC_VLAN_Incl(#i) (for i = 0; i <= DWCXG_NUM_TXQ-1) (continued)

Bits	Name	Memory Access	Description
15:0	VLT	R/W	VLAN Tag for Transmit Packets. This field contains the value of the VLAN tag to be inserted or replaced. The value must only be changed when the transmit lines are inactive or during the initialization phase. Bits[15:13] are the User Priority field, Bit 12 is the CFI/DEI field, and Bits[11:0] are the VID field in the VLAN tag. The following list describes the bits of this field: ■ Bits[15:13]: User Priority ■ Bit 12: Canonical Format Indicator (CFI) or Drop Eligible Indicator (DEI) ■ Bits[11:0]: VLAN Identifier (VID) field of VLAN tag **Value After Reset:** 0x0 **Exists:** Always

11.1.29 MAC_Inner_VLAN_Incl

- **Description:** The Inner VLAN Tag Inclusion or Replacement register contains the inner VLAN tag to be inserted or replaced in the Transmit packet. It also contains the inner VLAN tag insertion controls.
- **Size:** 32 bits
- **Offset:** 0x64
- **Exists:** (DWCXG_DOUBLE_VLAN_EN&&DWCXG_SA_VLAN_INS_CTRL_EN)

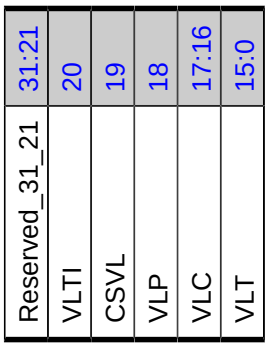


Table 11-33 Fields for Register: MAC_Inner_VLAN_Incl

Bits	Name	Memory Access	Description
31:21	Reserved_31_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
20	VLTI	R/W	VLAN Tag Input. When this bit is set, it indicates that the VLAN tag to be inserted or replaced in Tx packet must be taken from the following: ■ The mti_vlan_tag_i signal in XGMAC-CORE configurations ■ The control word in XGMAC-MTL configurations ■ The Tx descriptor in XGMAC-AXI configurations **Value After Reset:** 0x0 **Exists:** Always
19	CSVL	R/W	C-VLAN or S-VLAN. When this bit is set, S-VLAN type (0x88A8) is inserted or replaced in the 13th and 14th bytes of transmitted packets. When this bit is reset, C-VLAN type (0x8100) is inserted or replaced in the 13th and 14th bytes of transmitted packets. **Value After Reset:** 0x0 **Exists:** Always

Table 11-33 Fields for Register: MAC_Inner_VLAN_Incl (continued)

Bits	Name	Memory Access	Description
18	VLP	R/W	VLAN Priority Control. When this bit is set, the VLC field is used for VLAN deletion, insertion, or replacement. When this bit is reset, the VLC field is ignored and the following is used: ■ The mti_vlan_ctrl_i signal in XGMAC-CORE configurations ■ The control word in XGMAC-MTL configurations ■ The Tx descriptor in XGMAC-AXI configurations **Value After Reset:** 0x0 **Exists:** Always
17:16	VLC	R/W	VLAN Tag Control in Transmit Packets. ■ 2'b00: No VLAN tag deletion, insertion, or replacement ■ 2'b01: VLAN tag deletion The MAC removes the VLAN type (bytes 17 and 18) and VLAN tag (bytes 19 and 20) of all transmitted packets with VLAN tags. ■ 2'b10: VLAN tag insertion The MAC inserts VLT in bytes 19 and 20 of the packet after inserting the Type value (0x8100 or 0x88a8) in bytes 17 and 18. This operation is performed on all transmitted packets, irrespective of whether they already have a VLAN tag. ■ 2'b11: VLAN tag replacement The MAC replaces VLT in bytes 19 and 20 of all VLAN-type transmitted packets (Bytes 17 and 18 are 0x8100 or 0x88a8). **Note:** Changes to this field take effect only on the start of a packet. If you write this register field when a packet is being transmitted, only the subsequent packet can use the updated value, that is, the current packet does not use the updated value. **Value After Reset:** 0x0 **Exists:** Always

Table 11-33 Fields for Register: MAC_Inner_VLAN_Incl (continued)

Bits	Name	Memory Access	Description
15:0	VLT	R/W	VLAN Tag for Transmit Packets. This field contains the value of the VLAN tag to be inserted or replaced. The value must only be changed when the transmit lines are inactive or during the initialization phase. Bits[15:13] are the User Priority field, Bit 12 is the CFI/DEI field, and Bits[11:0] are the VID field in the VLAN tag. The following list describes the bits of this field: ■ Bits[15:13]: User Priority ■ Bit 12: Canonical Format Indicator (CFI) or Drop Eligible Indicator (DEI) ■ Bits[11:0]: VLAN Identifier (VID) field of VLAN tag **Value After Reset:** 0x0 **Exists:** Always

11.1.30 MAC_RVLAN_LKP_SIZE

- **Description:** This register is used to configure external receive outer/inner VLAN tag lookup size based on the selected VLAN tag bits to be used as lookup indexing.
- **Size:** 32 bits
- **Offset:** 0x68
- **Exists:** DWCXG_ERVFE&&(DWCXG_NRVF==4096)

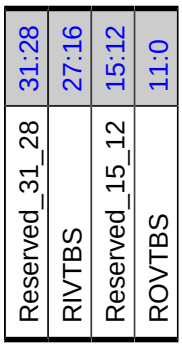


Table 11-34 Fields for Register: MAC_RVLAN_LKP_SIZE

Bits	Name	Memory Access	Description
31:28	Reserved_31_28	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
27:16	RIVTBS	R/W	Receive Inner VLAN Tag Bits Selection This field selects which bits of receive inner VLAN tag to be used for external receive inner VLAN tag lookup indexing. Set respective bit to 1'b1, which is required to include as lookup index considering maximum supported lookup size. Based on selected bits (bit which is set to 1), its concatenation from LSB to MSB is used as lookup index. **Value After Reset:** 0x0 **Exists:** (DWCXG_EXT_RVTL_EN&&DWCXG_DOUBLE_VLAN_EN)
15:12	Reserved_15_12	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-34 Fields for Register: MAC_RVLAN_LKP_SIZE (continued)

Bits	Name	Memory Access	Description
11:0	ROVTBS	R/W	Receive Outer VLAN Tag Bits Selection This field selects which bits of receive outer VLAN tag to be used for external receive outer VLAN tag lookup indexing. Set respective bit to 1'b1, which is required to include as lookup index considering maximum supported lookup size. Based on selected bits (bit which is set to 1), its concatenation from LSB to MSB is used as lookup index. **Value After Reset:** 0x0 **Exists:** DWCXG_EXT_RVTL_EN

11.1.31 MAC_Rx_Eth_Type_Match

- **Description:** The ET field of this register contains the value that needs to be compared with the Ethernet type field of the received packet. The result is indicated in the ET/LT field of receive status word. This field must be programmed with values greater than or equal to 1536. Values less than these are ignored.
- **Size:** 32 bits
- **Offset:** 0x6c
- **Exists:** Always

Table 11-35 Fields for Register: MAC_Rx_Eth_Type_Match

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:0	ET	R/W	Ethernet Type. This field holds the Ethernet Type value used for length/type matching. **Value After Reset:** 0x0 **Exists:** Always

11.1.32 MAC_Q0_Tx_Flow_Ctrl

- **Description:** The MAC_Q0_Tx_Flow_Ctrl register controls the generation of the Control (Pause Command) packets by the Flow control module of the MAC. A Write to a register with the Busy bit(FCB) set to 1 triggers the Flow Control block to generate a Pause packet. The fields of the control packet are selected as specified in the IEEE 802.3-2018 specification, and the Pause Time value from this register is used in the Pause Time field of the control packet. The Busy bit remains set until the control packet is transmitted by the MAC. The application must make sure that the Busy bit is cleared before writing to the register.
- **Size:** 32 bits
- **Offset:** 0x70
- **Exists:** Always

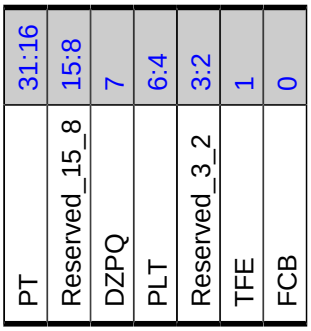


Table 11-36 Fields for Register: MAC_Q0_Tx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:16	PT	R/W	Pause Time. This field holds the value to be used in the Pause Time field in the Tx control packet. If the Pause Time bits are configured to be double-synchronized to the (X)GMII clock domain, consecutive writes to this register must be performed only after at least four clock cycles in the destination clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-36 Fields for Register: MAC_Q0_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
7	DZPQ	R/W	Disable Zero-Quanta Pause. When this bit is set, it disables the automatic generation of the zero-quanta Pause packets on de-assertion of the flow-control signal from the following: ■ The mti_flowctrl_i[0] signal in XGMAC-CORE configurations ■ The FIFO layer or external sideband flow control signal sbd_flowctrl_i[0] in XGMAC-MTL configurations ■ The FIFO layer in XGMAC-AXI configurations When this bit is reset, normal operation with automatic zero-quanta Pause packet generation is enabled. **Value After Reset:** 0x0 **Exists:** Always
6:4	PLT	R/W	Pause Low Threshold. This field configures the threshold of the Pause timer at which the input flow control signal mti_flowctrl_i, sbd_flowctrl_i or from the FIFO layer is checked for automatic retransmission of the Pause packet. The threshold values must be always less than the Pause Time configured in Bits[31:16]. For example, if PT = 100H (256 slot times), and PLT = 001, a second Pause packet is automatically transmitted if the transmit pause frame request signal is asserted at 228 (256 - 28) slot times after the first Pause packet is transmitted. The following list provides the threshold values for different values: ■ 3'b000: Pause Time minus 4 Slot Times (PT - 4 slot times) ■ 3'b001: Pause Time minus 28 Slot Times (PT - 28 slot times) ■ 3'b010: Pause Time minus 36 Slot Times (PT - 36 slot times) ■ 3'b011: Pause Time minus 144 Slot Times (PT - 144 slot times) ■ 3'b100: Pause Time minus 256 Slot Times (PT - 256 slot times) ■ 3'b101 - 3'b111: Reserved The slot time is defined as the time taken to transmit 512 bits (64 bytes) on the GMII or XGMII interface. This (approximate) computation is based on the packet size (64, 1518, 2000, 9018, or 16384) + 2 Pause Packet Size + IPG in Slot Times. **Value After Reset:** 0x0 **Exists:** Always

Table 11-36 Fields for Register: MAC_Q0_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	TFE	R/W	Transmit Flow Control Enable. In the full-duplex mode, when this bit is set, the MAC enables the flow control operation based on the Tx Pause packets. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause packets by itself. In the half-duplex mode, when this bit is set, the MAC enables the backpressure operation. When this bit is reset, the backpressure feature is disabled. **Value After Reset:** 0x0 **Exists:** Always
0	FCB	R/W	Flow Control Busy or Backpressure Activate. This bit initiates a Pause packet in the full-duplex mode and activates the backpressure function in the half-duplex mode if the TFE bit is set. In the full-duplex mode, this bit must be read as 1'b0 before writing to this register. To initiate a Pause packet, the application must set this bit to 1'b1. During Control packet transfer, this bit continues to be set to indicate that a packet transmission is in progress. When Pause packet transmission is complete, the MAC resets this bit to 1'b0. You must not write to this register until this bit is cleared. When this bit is set (and TFE bit is set) in the half-duplex mode, the MAC asserts the backpressure. During backpressure, when the MAC receives a new packet, the transmitter starts sending a JAM pattern resulting in a collision. For the backpressure function, this control bit is logically ORed with the H/W transmit pause frame request signal. When the MAC is configured for the full-duplex mode, the BPA is automatically disabled. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.33 MAC_Q1_Tx_Flow_Ctrl

- **Description:** The MAC_Q1_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority Flow Control) packets by the Flow control module of the MAC. A write to a register with the Busy bit set to 1 triggers the Flow Control block to generate a PFC packet for corresponding priority. The fields of the PFC packet are selected as specified in the IEEE 802.3-2018 specification, and the Pause Time value from this register is used in the Pause Time field for corresponding priority in the PFC packet. The Busy bit remains set until the PFC packet is transmitted by the MAC. The application must make sure that the Busy bit is cleared before writing to the register.
- **Size:** 32 bits
- **Offset:** 0x74
- **Exists:** (DWCXG_PFC_EN)

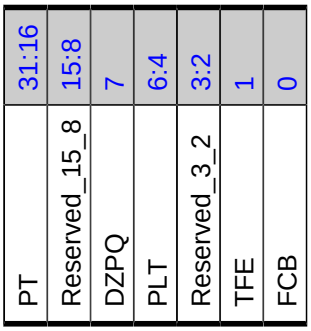


Table 11-37 Fields for Register: MAC_Q1_Tx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:16	PT	R/W	Pause Time. This field holds the value to be used in the Pause Time field in the Tx control packet. If the Pause Time bits are configured to be double-synchronized to the (X)GMII clock domain, consecutive writes to this register must be performed only after at least four clock cycles in the destination clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-37 Fields for Register: MAC_Q1_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
7	DZPQ	R/W	Disable Zero-Quanta Pause. When this bit is set, it disables the automatic generation of the zero-quanta Pause packets on de-assertion of the flow-control signal from the FIFO layer (MTL or external sideband flow control signal sbd_flowctrl_i[1] or mti_flowctrl_i[1]). When this bit is reset, normal operation with automatic zero-quanta Pause packet generation is enabled. **Value After Reset:** 0x0 **Exists:** Always
6:4	PLT	R/W	Pause Low Threshold. This field configures the threshold of the Pause timer at which the input flow control signal mti_flowctrl_i (or sbd_flowctrl_i) is checked for automatic retransmission of the Pause packet. The threshold values must be always less than the Pause Time configured in Bits[31:16]. For example, if PT = 100H (256 slot times), and PLT = 001, a second Pause packet is automatically transmitted if the mti_flowctrl_i signal is asserted at 228 (256 - 28) slot times after the first Pause packet is transmitted. The following list provides the threshold values for different values: ■ 000: Pause Time minus 4 Slot Times (PT - 4 slot times) ■ 001: Pause Time minus 28 Slot Times (PT - 28 slot times) ■ 010: Pause Time minus 36 Slot Times (PT - 36 slot times) ■ 011: Pause Time minus 144 Slot Times (PT - 144 slot times) ■ 100: Pause Time minus 256 Slot Times (PT - 256 slot times) ■ 101-111: Reserved The slot time is defined as the time taken to transmit 512 bits (64 bytes) on the GMII or XGMII interface. This (approximate) computation is based on the packet size (64, 1518, 2000, 9018, or 16384) + 2 Pause Packet Size + IPG in Slot Times. **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-37 Fields for Register: MAC_Q1_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
1	TFE	R/W	Transmit Flow Control Enable. When this bit is set, the MAC enables the flow control operation based on the Tx Pause packets. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause packets by itself. **Value After Reset:** 0x0 **Exists:** Always
0	FCB	R/W	Flow Control Busy. This bit initiates a Pause packet if the TFE bit is set. To initiate a Pause packet, the application must set this bit to 1'b1. During Control packet transfer, this bit continues to be set to indicate that a packet transmission is in progress. When Pause packet transmission is complete, the MAC resets this bit to 1'b0. You must not write to this register until this bit is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.34 MAC_Q2_Tx_Flow_Ctrl

- **Description:** The MAC_Q2_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority Flow Control) packets by the Flow control module of the MAC. A write to a register with the Busy bit set to 1 triggers the Flow Control block to generate a PFC packet for corresponding priority. The fields of the PFC packet are selected as specified in the IEEE 802.3-2018 specification, and the Pause Time value from this register is used in the Pause Time field for corresponding priority in the PFC packet. The Busy bit remains set until the PFC packet is transmitted by the MAC. The application must make sure that the Busy bit is cleared before writing to the register.
- **Size:** 32 bits
- **Offset:** 0x78
- **Exists:** (DWCXG_PFC_EN)

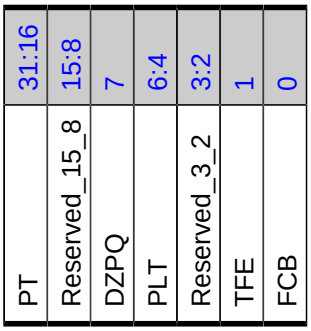


Table 11-38 Fields for Register: MAC_Q2_Tx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:16	PT	R/W	Pause Time. This field holds the value to be used in the Pause Time field in the Tx control packet. If the Pause Time bits are configured to be double-synchronized to the (X)GMII clock domain, consecutive writes to this register must be performed only after at least four clock cycles in the destination clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-38 Fields for Register: MAC_Q2_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
7	DZPQ	R/W	Disable Zero-Quanta Pause. When this bit is set, it disables the automatic generation of the zero-quanta Pause packets on de-assertion of the flow-control signal from the FIFO layer (MTL or external sideband flow control signal sbd_flowctrl_i[2] or mti_flowctrl_i[2]). When this bit is reset, normal operation with automatic zero-quanta Pause packet generation is enabled. **Value After Reset:** 0x0 **Exists:** Always
6:4	PLT	R/W	Pause Low Threshold. This field configures the threshold of the Pause timer at which the input flow control signal mti_flowctrl_i (or sbd_flowctrl_i) is checked for automatic retransmission of the Pause packet. The threshold values must be always less than the Pause Time configured in Bits[31:16]. For example, if PT = 100H (256 slot times), and PLT = 001, a second Pause packet is automatically transmitted if the mti_flowctrl_i signal is asserted at 228 (256 - 28) slot times after the first Pause packet is transmitted. The following list provides the threshold values for different values: ■ 000: Pause Time minus 4 Slot Times (PT - 4 slot times) ■ 001: Pause Time minus 28 Slot Times (PT - 28 slot times) ■ 010: Pause Time minus 36 Slot Times (PT - 36 slot times) ■ 011: Pause Time minus 144 Slot Times (PT - 144 slot times) ■ 100: Pause Time minus 256 Slot Times (PT - 256 slot times) ■ 101-111: Reserved The slot time is defined as the time taken to transmit 512 bits (64 bytes) on the GMII or XGMII interface. This (approximate) computation is based on the packet size (64, 1518, 2000, 9018, or 16384) + 2 Pause Packet Size + IPG in Slot Times. **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-38 Fields for Register: MAC_Q2_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
1	TFE	R/W	Transmit Flow Control Enable. When this bit is set, the MAC enables the flow control operation based on the Tx Pause packets. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause packets by itself. **Value After Reset:** 0x0 **Exists:** Always
0	FCB	R/W	Flow Control Busy. This bit initiates a Pause packet if the TFE bit is set. To initiate a Pause packet, the application must set this bit to 1'b1. During Control packet transfer, this bit continues to be set to indicate that a packet transmission is in progress. When Pause packet transmission is complete, the MAC resets this bit to 1'b0. You must not write to this register until this bit is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.35 MAC_Q3_Tx_Flow_Ctrl

- **Description:** The MAC_Q3_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority Flow Control) packets by the Flow control module of the MAC. A write to a register with the Busy bit set to 1 triggers the Flow Control block to generate a PFC packet for corresponding priority. The fields of the PFC packet are selected as specified in the IEEE 802.3-2018 specification, and the Pause Time value from this register is used in the Pause Time field for corresponding priority in the PFC packet. The Busy bit remains set until the PFC packet is transmitted by the MAC. The application must make sure that the Busy bit is cleared before writing to the register.
- **Size:** 32 bits
- **Offset:** 0x7c
- **Exists:** (DWCXG_PFC_EN)

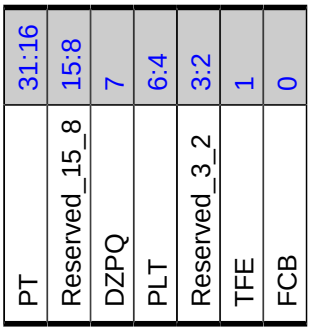


Table 11-39 Fields for Register: MAC_Q3_Tx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:16	PT	R/W	Pause Time. This field holds the value to be used in the Pause Time field in the Tx control packet. If the Pause Time bits are configured to be double-synchronized to the (X)GMII clock domain, consecutive writes to this register must be performed only after at least four clock cycles in the destination clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-39 Fields for Register: MAC_Q3_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
7	DZPQ	R/W	Disable Zero-Quanta Pause. When this bit is set, it disables the automatic generation of the zero-quanta Pause packets on de-assertion of the flow-control signal from the FIFO layer (MTL or external sideband flow control signal sbd_flowctrl_i[3] or mti_flowctrl_i[3]). When this bit is reset, normal operation with automatic zero-quanta Pause packet generation is enabled. **Value After Reset:** 0x0 **Exists:** Always
6:4	PLT	R/W	Pause Low Threshold. This field configures the threshold of the Pause timer at which the input flow control signal mti_flowctrl_i (or sbd_flowctrl_i) is checked for automatic retransmission of the Pause packet. The threshold values must be always less than the Pause Time configured in Bits[31:16]. For example, if PT = 100H (256 slot times), and PLT = 001, a second Pause packet is automatically transmitted if the mti_flowctrl_i signal is asserted at 228 (256 - 28) slot times after the first Pause packet is transmitted. The following list provides the threshold values for different values: ■ 000: Pause Time minus 4 Slot Times (PT - 4 slot times) ■ 001: Pause Time minus 28 Slot Times (PT - 28 slot times) ■ 010: Pause Time minus 36 Slot Times (PT - 36 slot times) ■ 011: Pause Time minus 144 Slot Times (PT - 144 slot times) ■ 100: Pause Time minus 256 Slot Times (PT - 256 slot times) ■ 101-111: Reserved The slot time is defined as the time taken to transmit 512 bits (64 bytes) on the GMII or XGMII interface. This (approximate) computation is based on the packet size (64, 1518, 2000, 9018, or 16384) + 2 Pause Packet Size + IPG in Slot Times. **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-39 Fields for Register: MAC_Q3_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
1	TFE	R/W	Transmit Flow Control Enable. When this bit is set, the MAC enables the flow control operation based on the Tx Pause packets. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause packets by itself. **Value After Reset:** 0x0 **Exists:** Always
0	FCB	R/W	Flow Control Busy. This bit initiates a Pause packet if the TFE bit is set. To initiate a Pause packet, the application must set this bit to 1'b1. During Control packet transfer, this bit continues to be set to indicate that a packet transmission is in progress. When Pause packet transmission is complete, the MAC resets this bit to 1'b0. You must not write to this register until this bit is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.36 MAC_Q4_Tx_Flow_Ctrl

- **Description:** The MAC_Q4_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority Flow Control) packets by the Flow control module of the MAC. A write to a register with the Busy bit set to 1 triggers the Flow Control block to generate a PFC packet for corresponding priority. The fields of the PFC packet are selected as specified in the IEEE 802.3-2018 specification, and the Pause Time value from this register is used in the Pause Time field for corresponding priority in the PFC packet. The Busy bit remains set until the PFC packet is transmitted by the MAC. The application must make sure that the Busy bit is cleared before writing to the register.
- **Size:** 32 bits
- **Offset:** 0x80
- **Exists:** (DWCXG_PFC_EN)

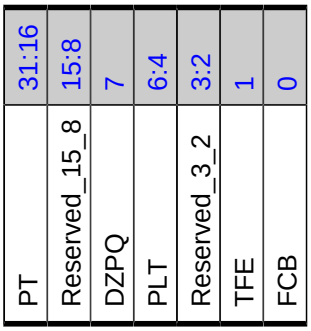


Table 11-40 Fields for Register: MAC_Q4_Tx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:16	PT	R/W	Pause Time. This field holds the value to be used in the Pause Time field in the Tx control packet. If the Pause Time bits are configured to be double-synchronized to the (X)GMII clock domain, consecutive writes to this register must be performed only after at least four clock cycles in the destination clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-40 Fields for Register: MAC_Q4_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
7	DZPQ	R/W	Disable Zero-Quanta Pause. When this bit is set, it disables the automatic generation of the zero-quanta Pause packets on de-assertion of the flow-control signal from the FIFO layer (MTL or external sideband flow control signal sbd_flowctrl_i[4] or mti_flowctrl_i[4]). When this bit is reset, normal operation with automatic zero-quanta Pause packet generation is enabled. **Value After Reset:** 0x0 **Exists:** Always
6:4	PLT	R/W	Pause Low Threshold. This field configures the threshold of the Pause timer at which the input flow control signal mti_flowctrl_i (or sbd_flowctrl_i) is checked for automatic retransmission of the Pause packet. The threshold values must be always less than the Pause Time configured in Bits[31:16]. For example, if PT = 100H (256 slot times), and PLT = 001, a second Pause packet is automatically transmitted if the mti_flowctrl_i signal is asserted at 228 (256 - 28) slot times after the first Pause packet is transmitted. The following list provides the threshold values for different values: ■ 000: Pause Time minus 4 Slot Times (PT - 4 slot times) ■ 001: Pause Time minus 28 Slot Times (PT - 28 slot times) ■ 010: Pause Time minus 36 Slot Times (PT - 36 slot times) ■ 011: Pause Time minus 144 Slot Times (PT - 144 slot times) ■ 100: Pause Time minus 256 Slot Times (PT - 256 slot times) ■ 101-111: Reserved The slot time is defined as the time taken to transmit 512 bits (64 bytes) on the GMII or XGMII interface. This (approximate) computation is based on the packet size (64, 1518, 2000, 9018, or 16384) + 2 Pause Packet Size + IPG in Slot Times. **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-40 Fields for Register: MAC_Q4_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
1	TFE	R/W	Transmit Flow Control Enable. When this bit is set, the MAC enables the flow control operation based on the Tx Pause packets. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause packets by itself. **Value After Reset:** 0x0 **Exists:** Always
0	FCB	R/W	Flow Control Busy. This bit initiates a Pause packet if the TFE bit is set. To initiate a Pause packet, the application must set this bit to 1'b1. During Control packet transfer, this bit continues to be set to indicate that a packet transmission is in progress. When Pause packet transmission is complete, the MAC resets this bit to 1'b0. You must not write to this register until this bit is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.37 MAC_Q5_Tx_Flow_Ctrl

- **Description:** The MAC_Q5_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority Flow Control) packets by the Flow control module of the MAC. A write to a register with the Busy bit set to 1 triggers the Flow Control block to generate a PFC packet for corresponding priority. The fields of the PFC packet are selected as specified in the IEEE 802.3-2018 specification, and the Pause Time value from this register is used in the Pause Time field for corresponding priority in the PFC packet. The Busy bit remains set until the PFC packet is transmitted by the MAC. The application must make sure that the Busy bit is cleared before writing to the register.
- **Size:** 32 bits
- **Offset:** 0x84
- **Exists:** (DWCXG_PFC_EN)

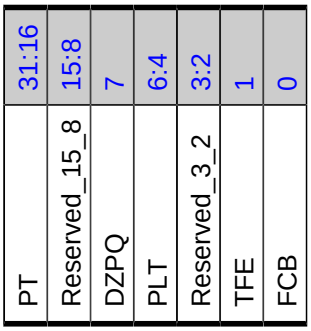


Table 11-41 Fields for Register: MAC_Q5_Tx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:16	PT	R/W	Pause Time. This field holds the value to be used in the Pause Time field in the Tx control packet. If the Pause Time bits are configured to be double-synchronized to the (X)GMII clock domain, consecutive writes to this register must be performed only after at least four clock cycles in the destination clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-41 Fields for Register: MAC_Q5_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
7	DZPQ	R/W	Disable Zero-Quanta Pause. When this bit is set, it disables the automatic generation of the zero-quanta Pause packets on de-assertion of the flow-control signal from the FIFO layer (MTL or external sideband flow control signal sbd_flowctrl_i[5] or mti_flowctrl_i[5]). When this bit is reset, normal operation with automatic zero-quanta Pause packet generation is enabled. **Value After Reset:** 0x0 **Exists:** Always
6:4	PLT	R/W	Pause Low Threshold. This field configures the threshold of the Pause timer at which the input flow control signal mti_flowctrl_i (or sbd_flowctrl_i) is checked for automatic retransmission of the Pause packet. The threshold values must be always less than the Pause Time configured in Bits[31:16]. For example, if PT = 100H (256 slot times), and PLT = 001, a second Pause packet is automatically transmitted if the mti_flowctrl_i signal is asserted at 228 (256 - 28) slot times after the first Pause packet is transmitted. The following list provides the threshold values for different values: ■ 000: Pause Time minus 4 Slot Times (PT - 4 slot times) ■ 001: Pause Time minus 28 Slot Times (PT - 28 slot times) ■ 010: Pause Time minus 36 Slot Times (PT - 36 slot times) ■ 011: Pause Time minus 144 Slot Times (PT - 144 slot times) ■ 100: Pause Time minus 256 Slot Times (PT - 256 slot times) ■ 101-111: Reserved The slot time is defined as the time taken to transmit 512 bits (64 bytes) on the GMII or XGMII interface. This (approximate) computation is based on the packet size (64, 1518, 2000, 9018, or 16384) + 2 Pause Packet Size + IPG in Slot Times. **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-41 Fields for Register: MAC_Q5_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
1	TFE	R/W	Transmit Flow Control Enable. When this bit is set, the MAC enables the flow control operation based on the Tx Pause packets. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause packets by itself. **Value After Reset:** 0x0 **Exists:** Always
0	FCB	R/W	Flow Control Busy. This bit initiates a Pause packet if the TFE bit is set. To initiate a Pause packet, the application must set this bit to 1'b1. During Control packet transfer, this bit continues to be set to indicate that a packet transmission is in progress. When Pause packet transmission is complete, the MAC resets this bit to 1'b0. You must not write to this register until this bit is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.38 MAC_Q6_Tx_Flow_Ctrl

- **Description:** The MAC_Q6_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority Flow Control) packets by the Flow control module of the MAC. A write to a register with the Busy bit set to 1 triggers the Flow Control block to generate a PFC packet for corresponding priority. The fields of the PFC packet are selected as specified in the IEEE 802.3-2018 specification, and the Pause Time value from this register is used in the Pause Time field for corresponding priority in the PFC packet. The Busy bit remains set until the PFC packet is transmitted by the MAC. The application must make sure that the Busy bit is cleared before writing to the register.
- **Size:** 32 bits
- **Offset:** 0x88
- **Exists:** (DWCXG_PFC_EN)

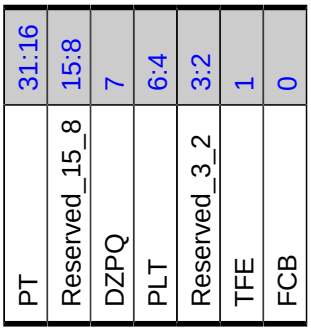


Table 11-42 Fields for Register: MAC_Q6_Tx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:16	PT	R/W	Pause Time. This field holds the value to be used in the Pause Time field in the Tx control packet. If the Pause Time bits are configured to be double-synchronized to the (X)GMII clock domain, consecutive writes to this register must be performed only after at least four clock cycles in the destination clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-42 Fields for Register: MAC_Q6_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
7	DZPQ	R/W	Disable Zero-Quanta Pause. When this bit is set, it disables the automatic generation of the zero-quanta Pause packets on de-assertion of the flow-control signal from the FIFO layer (MTL or external sideband flow control signal sbd_flowctrl_i[6] or mti_flowctrl_i[6]). When this bit is reset, normal operation with automatic zero-quanta Pause packet generation is enabled. **Value After Reset:** 0x0 **Exists:** Always
6:4	PLT	R/W	Pause Low Threshold. This field configures the threshold of the Pause timer at which the input flow control signal mti_flowctrl_i (or sbd_flowctrl_i) is checked for automatic retransmission of the Pause packet. The threshold values must be always less than the Pause Time configured in Bits[31:16]. For example, if PT = 100H (256 slot times), and PLT = 001, a second Pause packet is automatically transmitted if the mti_flowctrl_i signal is asserted at 228 (256 - 28) slot times after the first Pause packet is transmitted. The following list provides the threshold values for different values: ■ 000: Pause Time minus 4 Slot Times (PT - 4 slot times) ■ 001: Pause Time minus 28 Slot Times (PT - 28 slot times) ■ 010: Pause Time minus 36 Slot Times (PT - 36 slot times) ■ 011: Pause Time minus 144 Slot Times (PT - 144 slot times) ■ 100: Pause Time minus 256 Slot Times (PT - 256 slot times) ■ 101-111: Reserved The slot time is defined as the time taken to transmit 512 bits (64 bytes) on the GMII or XGMII interface. This (approximate) computation is based on the packet size (64, 1518, 2000, 9018, or 16384) + 2 Pause Packet Size + IPG in Slot Times. **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-42 Fields for Register: MAC_Q6_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
1	TFE	R/W	Transmit Flow Control Enable. When this bit is set, the MAC enables the flow control operation based on the Tx Pause packets. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause packets by itself. **Value After Reset:** 0x0 **Exists:** Always
0	FCB	R/W	Flow Control Busy. This bit initiates a Pause packet if the TFE bit is set. To initiate a Pause packet, the application must set this bit to 1'b1. During Control packet transfer, this bit continues to be set to indicate that a packet transmission is in progress. When Pause packet transmission is complete, the MAC resets this bit to 1'b0. You must not write to this register until this bit is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.39 MAC_Q7_Tx_Flow_Ctrl

- **Description:** The MAC_Q7_Tx_Flow_Ctrl register is per priority and controls the generation of the PFC (Priority Flow Control) packets by the Flow control module of the MAC. A write to a register with the Busy bit set to 1 triggers the Flow Control block to generate a PFC packet for corresponding priority. The fields of the PFC packet are selected as specified in the IEEE 802.3-2018 specification, and the Pause Time value from this register is used in the Pause Time field for corresponding priority in the PFC packet. The Busy bit remains set until the PFC packet is transmitted by the MAC. The application must make sure that the Busy bit is cleared before writing to the register.
- **Size:** 32 bits
- **Offset:** 0x8c
- **Exists:** (DWCXG_PFC_EN)

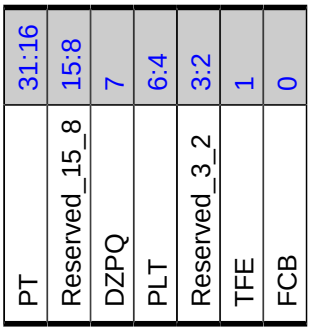


Table 11-43 Fields for Register: MAC_Q7_Tx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:16	PT	R/W	Pause Time. This field holds the value to be used in the Pause Time field in the Tx control packet. If the Pause Time bits are configured to be double-synchronized to the (X)GMII clock domain, consecutive writes to this register must be performed only after at least four clock cycles in the destination clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-43 Fields for Register: MAC_Q7_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
7	DZPQ	R/W	Disable Zero-Quanta Pause. When this bit is set, it disables the automatic generation of the zero-quanta Pause packets on de-assertion of the flow-control signal from the FIFO layer (MTL or external sideband flow control signal sbd_flowctrl_i[7] or mti_flowctrl_i[7]). When this bit is reset, normal operation with automatic zero-quanta Pause packet generation is enabled. **Value After Reset:** 0x0 **Exists:** Always
6:4	PLT	R/W	Pause Low Threshold. This field configures the threshold of the Pause timer at which the input flow control signal mti_flowctrl_i (or sbd_flowctrl_i) is checked for automatic retransmission of the Pause packet. The threshold values must be always less than the Pause Time configured in Bits[31:16]. For example, if PT = 100H (256 slot times), and PLT = 001, a second Pause packet is automatically transmitted if the mti_flowctrl_i signal is asserted at 228 (256 - 28) slot times after the first Pause packet is transmitted. The following list provides the threshold values for different values: ■ 000: Pause Time minus 4 Slot Times (PT - 4 slot times) ■ 001: Pause Time minus 28 Slot Times (PT - 28 slot times) ■ 010: Pause Time minus 36 Slot Times (PT - 36 slot times) ■ 011: Pause Time minus 144 Slot Times (PT - 144 slot times) ■ 100: Pause Time minus 256 Slot Times (PT - 256 slot times) ■ 101-111: Reserved The slot time is defined as the time taken to transmit 512 bits (64 bytes) on the GMII or XGMII interface. This (approximate) computation is based on the packet size (64, 1518, 2000, 9018, or 16384) + 2 Pause Packet Size + IPG in Slot Times. **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-43 Fields for Register: MAC_Q7_Tx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
1	TFE	R/W	Transmit Flow Control Enable. When this bit is set, the MAC enables the flow control operation based on the Tx Pause packets. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause packets by itself. **Value After Reset:** 0x0 **Exists:** Always
0	FCB	R/W	Flow Control Busy. This bit initiates a Pause packet if the TFE bit is set. To initiate a Pause packet, the application must set this bit to 1'b1. During Control packet transfer, this bit continues to be set to indicate that a packet transmission is in progress. When Pause packet transmission is complete, the MAC resets this bit to 1'b0. You must not write to this register until this bit is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.40 MAC_Rx_Flow_Ctrl

- **Description:** The Receive Flow Control register controls the pausing of MAC Transmit based on the received Pause packet.
- **Size:** 32 bits
- **Offset:** 0x90
- **Exists:** Always

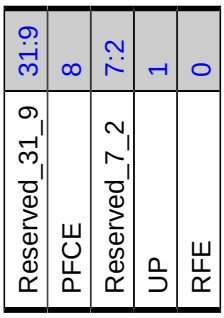


Table 11-44 Fields for Register: MAC_Rx_Flow_Ctrl

Bits	Name	Memory Access	Description
31:9	Reserved_31_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	PFCE	R/W	Priority Based Flow Control Enable. ■ 1: Enables generation and reception of priority-based flow control (PFC) packets. ■ 0: Enables generation and reception of IEEE 802.3-2018 Pause control packets. **Value After Reset:** 0x0 **Exists:** DWCXG_PFC_EN
7:2	Reserved_7_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-44 Fields for Register: MAC_Rx_Flow_Ctrl (continued)

Bits	Name	Memory Access	Description
1	UP	R/W	Unicast Pause Packet Detect. A pause packet is processed when it has the unique multicast address specified in the IEEE 802.3-2018. When this bit is set, the MAC can also detect Pause packets with unicast address of the station. This unicast address must be as specified in MAC_Address0_High and MAC_Address0_Low. When this bit is reset, the MAC only detects Pause packets with unique multicast address. **Note:** The MAC does not process a Pause packet if the multicast address is different from the unique multicast address. This is also applicable to the received PFC packet when the Priority Flow Control (PFC) is enabled. The unique multicast address (0x01_80_C2_00_00_01) is as specified in IEEE 802.1Qbb-2011. **Value After Reset:** 0x0 **Exists:** Always
0	RFE	R/W	Receive Flow Control Enable. ■ 1: When MAC operates in full-duplex mode, the MAC decodes the received Pause packet and disables its transmitter for a specified (Pause) time. ■ 0: When MAC operates in half-duplex mode, the decode function of the Pause packet is disabled. When PFC is enabled: Flow control is enabled for PFC packets. The MAC decodes the received PFC packet and disables the corresponding Traffic Class, with matching priorities, for a duration of received Pause time. **Value After Reset:** 0x0 **Exists:** Always

11.1.41 MAC_RxQ_Ctrl4

- **Description:** The Receive Queue Control 4 register controls the routing of
  - Packets which fail the Destination or Source Address filter and VLAN Filter. Packets refer to Express packets only when FPE is configured
  - Preemptive Multicast/Broadcast packets when MCBCQEN of MAC_RxQ_Ctrl1 is enabled to a separate Rx Queue
- **Size:** 32 bits
- **Offset:** 0x94
- **Exists:** (DWCXG_NUM_RXQ>1)||DWCXG_DDUP

31	x:24	23:y	x:17	16	15:y	x:9	8	7:y	x:1	0
UDC	PMCBCQ	Reserved_23_y	VFFQ	VFFQE	Reserved_15_y	MFFQ	MFFQE	Reserved_7_y	UFFQ	UFFQE

Table 11-45 Fields for Register: MAC_RxQ_Ctrl4

Bits	Name	Memory Access	Description
31	UDC	R/W	Unicast Packet Duplication Control. When this bit is set along with DDS bit of the MAC_Extended_Configuration register, the MAC routes the received packet with Unicast Destination Address to the highest Receive Queue selected in configuration, provided that more than one bit are set in the DMA Channel number specified in XDCS field of MAC_DChSel_IndReg(#i) register that matched the Unicast Destination Address of the received packet. Note: When the above condition is satisfied for unicast packet but the highest Receive Queue selected in configuration is not enabled or when DA/SA/VLAN filter fails and corresponding filter fail Receive Queue is enabled, it will be routed to Receive Queue determined by other queuing methods. When this bit is reset, the MAC routes the received packet with Unicast Destination Address based other queuing methods, the DMA Channel number selected is binary value from DCS field of MAC_Address(#i)_High register (if DDS bit is reset) or lowest bit set in one-hot value from XDCS field of MAC_DChSel_IndReg(#i) indirect register (if DDS is set)

Table 11-45 Fields for Register: MAC_RxQ_Ctrl4 (continued)

Bits	Name	Memory Access	Description
			that matched the Unicast Destination Address of the received packet. **Value After Reset:** 0x0 **Exists:** (DWCXG>1)&&(((DWCXG_NUM_DMA_RX_CH>1)&&!SNPS_RSVDPARAM_50)||((DWCXG_RCHLIST_CNT>0)&&SNPS_RSVDPARAM_50))
x:24	PMCBCQ	R/W	Preemption Multicast/Broadcast Queue. This field holds the Rx queue number to which the preemptive Multicast/Broadcast packets passing the Destination or Source Address filter or the VLAN tag filter must be routed. This field is valid only when the MCBCQEN bit of MAC_RxQ_Ctrl1 register is set. **Value After Reset:** 0x0 **Exists:** DWCXG_FPE **Range Variable[x]:** DWCXG_RXQW + 23
23:y	Reserved_23_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_RXQW + 17
x:17	VFFQ	R/W	VLAN Tag Filter Fail Packets Queue. This field holds the Rx queue number to which the tagged packets failing the Destination or Source Address filter (and UFFQE/MFFQE not enabled) or failing the VLAN tag filter must be routed. This field is valid only when the VFFQE bit is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1) **Range Variable[x]:** DWCXG_RXQW + 16

Table 11-45 Fields for Register: MAC_RxQ_Ctrl4 (continued)

Bits	Name	Memory Access	Description
16	VFFQE	R/W	VLAN Tag Filter Fail Packets Queuing Enable. When this bit is set, the tagged packets which fail the Destination or Source address filter or fail the VLAN tag filter, are routed to the Rx Queue Number programmed in the VFFQ. When this bit is reset, the tagged packets which fail the Destination or Source address filter or fail the VLAN tag filter are routed based on other routing options. This bit is valid only when the RA bit of the MAC_Packet_Filter register is set. values: ■ 0: VLAN tag Filter Fail Packets Queuing is disabled. ■ 1: VLAN tag Filter Fail Packets Queuing is enabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1)
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_RXQW + 9
x:9	MFFQ	R/W	Multicast Address Filter Fail Packets Queue. This field holds the Rx queue number to which Multicast Packets failing the Destination or Source Address filter must be routed. This field is valid only when MFFQE the bit is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1) **Range Variable[x]:** DWCXG_RXQW + 8

Table 11-45 Fields for Register: MAC_RxQ_Ctrl4 (continued)

Bits	Name	Memory Access	Description
8	MFFQE	R/W	Multicast Filter Fail Packets Queuing Enable. When this bit is set, the Multicast packets which fail the Destination or Source address filter are routed to the Rx Queue Number programmed in the MFFQ. When this bit is reset, the Multicast packets which fail the Destination or Source address filter are routed based on other routing options. This bit is valid only when the RA bit of the MAC_Packet_Filter register is set. values: ■ 0: Multicast Address Filter Fail Packets Queuing is disabled. ■ 1: Multicast Address Filter Fail Packets Queuing is enabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1)
7:y	Reserved_7_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_RXQW + 1
x:1	UFFQ	R/W	Unicast Address Filter Fail Packets Queue. This field holds the Rx queue number to which Unicast Packets failing the Destination or Source Address filter must be routed. This field is valid only when UFFQE the bit is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1) **Range Variable[x]:** DWCXG_RXQW
0	UFFQE	R/W	Unicast Filter Fail Packets Queuing Enable. When this bit is set, the Unicast packets which fail the Destination or Source address filter are routed to the Rx Queue Number programmed in the UFFQ. When this bit is reset, the Unicast packets which fail the Destination or Source address filter are routed based on other routing options. This bit is valid only when the RA bit of the MAC_Packet_Filter register is set. values: ■ 0: Unicast Address Filter Fail Packets Queuing is disabled. ■ 1: Unicast Address Filter Fail Packets Queuing is enabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1)

11.1.42 MAC_RxQ_Ctrl5

- **Description:** The Receive Queue Control 5 register allows selection of start offset of the Receive queue from which received packets are routed based on VLAN Tag Priority field.
- **Size:** 32 bits
- **Offset:** 0x98
- **Exists:** (DWCXG_NUM_RXQ>1)

Table 11-46 Fields for Register: MAC_RxQ_Ctrl5

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3:0	PRQSO	R/W	Priority Receive Queue Start Offset. This field holds the Receive queue number from which received packets are routed based on the VLAN Tag Priority field. For example, value of 0 indicates PSRQ0, PSRQ1, .. PSRQ7 fields in MAC_RxQ_Ctrl2 and MAC_RxQ_Ctrl3 registers are mapped to Receive Queue 0, Receive Queue 1, .. Receive Queue 7 respectively. Similarly, value of 3 indicates PSRQ0, PSRQ1, .. PSRQ7 fields in MAC_RxQ_Ctrl2 and MAC_RxQ_Ctrl3 registers are mapped to Receive Queue 3, Receive Queue 4, .. Receive Queue 10 respectively. The Receive queue number programmed in this field must not be more than number of Receive queues selected in configuration. **Note:** When this field is programmed to a value such that Receive queues selected in configuration are lesser than the sum of programmed value and 7, all the PSRQx fields cannot be mapped to the Receive queues. For example, if number of Receive queues selected in configuration is 8 and this field is programmed to 5, only the PSRQ0, PSRQ1 and PSRQ2 can be mapped to Receive Queue 5, Receive Queue 6 and Receive Queue 7 respectively. **Value After Reset:** 0x0 **Exists:** Always

11.1.43 MAC_RxQ_Ctrl0

- **Description:** The Receive Queue Control 0 register activates the queue management in the MAC Receiver. This register is present only when you select multiple queues in the Receive path.
- **Size:** 32 bits
- **Offset:** 0xa0
- **Exists:** (DWCXG_NUM_RXQ>1)

31:30	29:28	27:26	25:24	23:22	21:20	19:18	17:16	15:14	13:12	11:10	9:8	7:6	5:4	3:2	1:0
RXQ15EN	RXQ14EN	RXQ13EN	RXQ12EN	RXQ11EN	RXQ10EN	RXQ9EN	RXQ8EN	RXQ7EN	RXQ6EN	RXQ5EN	RXQ4EN	RXQ3EN	RXQ2EN	RXQ1EN	RXQ0EN

Table 11-47 Fields for Register: MAC_RxQ_Ctrl0

Bits	Name	Memory Access	Description
31:30	RXQ15EN	R/W	Receive Queue 15 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>15)
29:28	RXQ14EN	R/W	Receive Queue 14 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>14)
27:26	RXQ13EN	R/W	Receive Queue 13 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>13)
25:24	RXQ12EN	R/W	Receive Queue 12 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>12)
23:22	RXQ11EN	R/W	Receive Queue 11 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>11)

Table 11-47 Fields for Register: MAC_RxQ_Ctrl0 (continued)

Bits	Name	Memory Access	Description
21:20	RXQ10EN	R/W	Receive Queue 10 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>10)
19:18	RXQ9EN	R/W	Receive Queue 9 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>9)
17:16	RXQ8EN	R/W	Receive Queue 8 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>8)
15:14	RXQ7EN	R/W	Receive Queue 7 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>7)
13:12	RXQ6EN	R/W	Receive Queue 6 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>6)
11:10	RXQ5EN	R/W	Receive Queue 5 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>5)
9:8	RXQ4EN	R/W	Receive Queue 4 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>4)
7:6	RXQ3EN	R/W	Receive Queue 3 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>3)

Table 11-47 Fields for Register: MAC_RxQ_Ctrl0 (continued)

Bits	Name	Memory Access	Description
5:4	RXQ2EN	R/W	Receive Queue 2 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>2)
3:2	RXQ1EN	R/W	Receive Queue 1 Enable. This field is similar to the RXQ0EN field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1)
1:0	RXQ0EN	R/W	Receive Queue 0 Enable. This field indicates whether Rx Queue 0 is enabled. ■ 2'b00: Not enabled ■ 2'b01: Queue 0 enabled for Audio Video Bridging ■ 2'b10: Queue 0 enabled for Data Center Bridging/ Generic ■ 2'b11: Reserved **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>0)

11.1.44 MAC_RxQ_Ctrl1

- **Description:** This register allows the selection of the Receive queues to which the received untagged or special packets are routed or written. This register is present only when you select multiple queues in the Receive path.
  Note:
  - Program the queue number in the fields within the range of Rx Queues that you have selected in your controller.
  - PFC-based flow control is limited only to Rx Queues 0 to 7.
- **Size:** 32 bits
- **Offset:** 0xa4
- **Exists:** (DWCXG_NUM_RXQ>1)

31:28	27:24	23	x:21	20	19:16	15	14:12	11:8	7:4	3:0
AVCPQ	PTPQ	TACPQE	TPQC	OMCBCQ	DCBCPQ	MCBCQEN	Reserved_14_12	MCBCQ	RQ	UPQ

Table 11-48 Fields for Register: MAC_RxQ_Ctrl1

Bits	Name	Memory Access	Description
31:28	AVCPQ	R/W	AV Control Packets Queue. This field specifies the Rx queue on which the received AV Control packets (tagged and untagged) are routed. ■ 4'b0000: Rx Queue 0 ■ 4'b0001: Rx Queue 1 ■ ... ■ 4'b1011: Rx Queue 11 ■ 4'b1100 - 4'b1111: Reserved **Value After Reset:** 0x0 **Exists:** DWCXG_AV_ENABLE

Table 11-48 Fields for Register: MAC_RxQ_Ctrl1 (continued)

Bits	Name	Memory Access	Description
27:24	PTPQ	R/W	PTP Packets Queue. This field specifies the Rx queue on which the untagged PTP packets sent over the Ethernet payload (not over IPv4 or IPv6) are routed. ■ 4'b0000: Rx Queue 0 ■ 4'b0001: Rx Queue 1 ■ ... ■ 4'b1011: Rx Queue 11 ■ 4'b1100 - 4'b1111: Reserved When the AV8021ASMEN bit of MAC_Timestamp_Control register is reset, even though VLAN tagged PTP over Ethernet packets are processed, it is routed based on the VLAN Tag priority. **Value After Reset:** 0x0 **Exists:** DWCXG_TIME_STAMPING
23	TACPQE	R/W	Tagged AV Control Packets Queuing Enable. When set, the MAC routes the received Tagged AV Control packets to the Rx queue specified by AVCPQ field. If Tagged AV Control packets received are expected to be preempted then do not set the TACPQE bit to enable a separate queue. When reset, the MAC routes the received Tagged AV Control packets based on the tag priority matching the PSRQ fields in MAC_RxQ_Ctrl2 and MAC_RxQ_Ctrl3 registers. **Value After Reset:** 0x0 **Exists:** DWCXG_AV_ENABLE
x:21	TPQC	R/W	Tagged PTP over Ethernet Packets Queuing Control. This field controls the routing of the VLAN Tagged PTP over Ethernet packets. If Audio Video Bridging Feature is selected in the configuration: The following programmable options are allowed. ■ 2'b00: VLAN Tagged PTP over Ethernet packets are routed as generic VLAN Tagged packet (based on the PSRQ field for only the non-AV enabled Rx Queues). ■ 2'b01: VLAN Tagged PTP over Ethernet packets are routed to Rx Queue specified by PTPQ field (That Rx Queue can be enabled for AV or non-AV traffic). ■ 2'b10: VLAN Tagged PTP over Ethernet packets are routed to only AV enabled Rx Queues based on the PSRQ field. ■ 2'b11: Reserved

Table 11-48 Fields for Register: MAC_RxQ_Ctrl1 (continued)

Bits	Name	Memory Access	Description
			If Audio Video Bridging Feature is not selected in the configuration: The following programmable options are allowed. ■ 1'b0: VLAN Tagged PTP over Ethernet packets are routed as generic VLAN Tagged packet (based on the PSRQ field for the Data Center Bridging/Generic enabled Rx Queues). ■ 1'b1: VLAN Tagged PTP over Ethernet packets are routed to Rx Queues specified by PTPQ field. **Value After Reset:** 0x0 **Exists:** DWCXG_TIME_STAMPING **Range Variable[x]:** DWCXG_AV_ENABLE&&DWCXG_TIME_STAMPING?2:! DWCXG_AV_ENABLE&&DWCXG_TIME_STAMPING?1:0 + 20
20	OMCBCQ	R/W	Programmable control for Over-riding MCBCQ Priority. When set to 1, this field enables over-riding the MCBC Queue priority. When set to 0, this field disables over-riding the MCBC Queue priority. **Value After Reset:** 0x0 **Exists:** Always
19:16	DCBCPQ	R/W	DCB Control Packets Queue. This field specifies the Rx queue on which the received DCB control packets are routed. ■ 4'b0000: Rx Queue 0 ■ 4'b0001: Rx Queue 1 ■ ... ■ 4'b1011: Rx Queue 11 ■ 4'b1100 - 4'b1111: Reserved The DCB data packets are routed based on the setting of the PSRQ field corresponding to its USP (User Priority) field in the MAC_RxQ_Ctrl2/3 registers. **Value After Reset:** 0x0 **Exists:** DWCXG_DCB_EN
15	MCBCQEN	R/W	Multicast or Broadcast Queue Enable. This bit specifies that Multicast or Broadcast packets routing to the Rx Queue is enabled and the Multicast or Broadcast packets must be routed to Rx Queue specified in MCBCQ field. **Value After Reset:** 0x0 **Exists:** Always

Table 11-48 Fields for Register: MAC_RxQ_Ctrl1 (continued)

Bits	Name	Memory Access	Description
14:12	Reserved_14_12	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
11:8	MCBCQ	R/W	Multicast or Broadcast Queue. This field specifies the Rx queue on which the Multicast or Broadcast packets are routed. ■ 4'b0000: Rx Queue 0 ■ 4'b0001: Rx Queue 1 ■ ... ■ 4'b1011: Rx Queue 11 ■ 4'b1100 - 4'b1111: Reserved **Value After Reset:** 0x0 **Exists:** Always
7:4	RQ	R/W	Frame Preemption Residue Queue This field holds the Rx queue number to which the residual preemption frames must be forwarded. Preemption frames that are tagged and pass the SA/DA/VLAN filtering are routed based on the PSRQ field. All other frames are treated as residual frames and are routed to the queue number mentioned in this field. Queue-0 is used as a default queue for express frames, so this field cannot be programmed to a value 0. **Value After Reset:** 0x0 **Exists:** DWCXG_FPE
3:0	UPQ	R/W	Untagged Packet Queue. This field specifies the Rx queue on which the received untagged packets are routed. ■ 4'b0000: Rx Queue 0 ■ 4'b0001: Rx Queue 1 ■ ... ■ 4'b1011: Rx Queue 11 ■ 4'b1100 - 4'b1111: Reserved **Value After Reset:** 0x0 **Exists:** Always

11.1.45 MAC_RxQ_Ctrl2

- **Description:** This register controls the routing of tagged packets based on the USP (user Priority) field of the received packets to the Rx Queues 0 to 3. This register is present when multiple Rx Queues are selected while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xa8
- **Exists:** (DWCXG_NUM_RXQ>1)

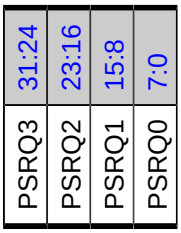


Table 11-49 Fields for Register: MAC_RxQ_Ctrl2

Bits	Name	Memory Access	Description
31:24	PSRQ3	R/W	Priorities Selected in the Receive Queue 3. This field decides the priorities assigned to Rx Queue 3. All packets with priorities that match the values set in this field are routed to Rx Queue 3. For example, if PSRQ3[6, 3] are set, packets with USP field equal to 3 or 6 are routed to Rx Queue 3. The software must ensure that the content of this field is mutually exclusive to the PSRQ fields for other queues, that is, the same priority is not mapped to multiple Rx queues. When the Data Center Bridging feature is selected: This field also determines the priorities to be included in the PFC packet sent to remote station when Rx Queue 3 crosses the flow control threshold settings. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>3)

Table 11-49 Fields for Register: MAC_RxQ_Ctrl2 (continued)

Bits	Name	Memory Access	Description
23:16	PSRQ2	R/W	Priorities Selected in the Receive Queue 2. This field decides the priorities assigned to Rx Queue 2. All packets with priorities that match the values set in this field are routed to Rx Queue 2. For example, if PSRQ2[1, 0] are set, packets with USP field equal to 1 or 0 are routed to Rx Queue 2. The software must ensure that the content of this field is mutually exclusive to the PSRQ fields for other queues, that is, the same priority is not mapped to multiple Rx queues. When the Data Center Bridging feature is selected: This field also determines the priorities to be included in the PFC packet sent to remote station when Rx Queue 2 crosses the flow control threshold settings. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>2)
15:8	PSRQ1	R/W	Priorities Selected in the Receive Queue 1. This field decides the priorities assigned to Rx Queue 1. All packets with priorities that match the values set in this field are routed to Rx Queue 1. For example, if PSRQ1[4] is set, packets with USP field equal to 4 are routed to Rx Queue 1. The software must ensure that the content of this field is mutually exclusive to the PSRQ fields for other queues, that is, the same priority is not mapped to multiple Rx queues. When the Data Center Bridging feature is selected: This field also determines the priorities to be included in the PFC packet sent to remote station when Rx Queue 1 crosses the flow control threshold settings. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1)
7:0	PSRQ0	R/W	Priorities Selected in the Receive Queue 0. This field decides the priorities assigned to Rx Queue 0. All packets with priorities that match the values set in this field are routed to Rx Queue 0. For example, if PSRQ0[5] is set, packets with USP field equal to 5 are routed to Rx Queue 0. The software must ensure that the content of this field is mutually exclusive to the PSRQ fields for other queues, that is, the same priority is not mapped to multiple Rx queues. When the Data Center Bridging feature is selected: This field also determines the priorities to be included in the PFC packet sent to remote station when Rx Queue 0 crosses the flow control threshold settings. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>0)

11.1.46 MAC_RxQ_Ctrl3

- **Description:** This register controls the routing of tagged packets based on the USP (user Priority) field of the received packets to the Rx Queues 4 to 7. This register is present when the "Enable Data Center Bridging" option is selected and with more than four Rx queues, while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xac
- **Exists:** (DWCXG_NUM_RXQ>4)

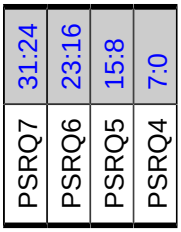


Table 11-50 Fields for Register: MAC_RxQ_Ctrl3

Bits	Name	Memory Access	Description
31:24	PSRQ7	R/W	Priorities Selected in the Receive Queue 7. This field decides the priorities assigned to Rx Queue 7. All packets with priorities that match the values set in this field are routed to Rx Queue 7. For example, if PSRQ7[7, 4] are set, packets with USP field equal to 7 or 4 are routed to Rx Queue 7. The software must ensure that the content of this field is mutually exclusive to the PSRQ fields for other queues, that is, the same priority is not mapped to multiple Rx queues. When the Data Center Bridging feature is selected: This field also determines the priorities to be included in the PFC packet sent to remote station when Rx Queue 7 crosses the flow control threshold settings. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>7)

Table 11-50 Fields for Register: MAC_RxQ_Ctrl3 (continued)

Bits	Name	Memory Access	Description
23:16	PSRQ6	R/W	Priorities Selected in the Receive Queue 6. This field decides the priorities assigned to Rx Queue 6. All packets with priorities that match the values set in this field are routed to Rx Queue 6. For example, if PSRQ6[5] are set, packets with USP field equal to 5 are routed to Rx Queue 6. The software must ensure that the content of this field is mutually exclusive to the PSRQ fields for other queues, that is, the same priority is not mapped to multiple Rx queues. When the Data Center Bridging feature is selected: This field also determines the priorities to be included in the PFC packet sent to remote station when Rx Queue 6 crosses the flow control threshold settings. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>6)
15:8	PSRQ5	R/W	Priorities Selected in the Receive Queue 5. This field decides the priorities assigned to Rx Queue 5. All packets with priorities that match the values set in this field are routed to Rx Queue 5. For example, if PSRQ5[6] is set, packets with USP field equal to 6 are routed to Rx Queue 5. The software must ensure that the content of this field is mutually exclusive to the PSRQ fields for other queues, that is, the same priority is not mapped to multiple Rx queues. When the Data Center Bridging feature is selected: This field also determines the priorities to be included in the PFC packet sent to remote station when Rx Queue 5 crosses the flow control threshold settings. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>5)
7:0	PSRQ4	R/W	Priorities Selected in the Receive Queue 4. This field decides the priorities assigned to Rx Queue 4. All packets with priorities that match the values set in this field are routed to Rx Queue 4. For example, if PSRQ4[7:4] is set, packets with USP field equal to 7, 6, 5, or 4 are routed to Rx Queue 4. The software must ensure that the content of this field is mutually exclusive to the PSRQ fields for other queues, that is, the same priority is not mapped to multiple Rx queues. When the Data Center Bridging feature is selected: This field also determines the priorities to be included in the PFC packet sent to remote station when Rx Queue 4 crosses the flow control threshold settings. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>4)

11.1.47 MAC_Interrupt_Status

- **Description:** The Interrupt Status register.
  The register contains status of the events which generates interrupt when corresponding interrupt is enabled in MAC_Interrupt_Enable register
- **Size:** 32 bits
- **Offset:** 0xb0
- **Exists:** Always

31:26	25:24	23	22:19	18	17	16	15	14	13	12	11	10	9	8:6	5	4	3:2	1	0
Reserved_31_26	LS	PCIS	Reserved_22_19	MFRIS	MFTIS	FPEIS	GPIIS	RXESIS	TXESIS	TSIS	MMCRXIPIS	MMCTXIS	MMCRXIS	Reserved_8_6	LPIIS	PMTIS	Reserved_3_2	SMI	LSI

Table 11-51 Fields for Register: MAC_Interrupt_Status

Bits	Name	Memory Access	Description
31:26	Reserved_31_26	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-51 Fields for Register: MAC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
25:24	LS	R	Link Status. When current Mode is not RGMII : These bits indicate the current link status received on the XGMII receive interface. ■ 2'b00: Link OK ■ 2'b01: Reserved ■ 2'b10: Local link fault ■ 2'b11: Remote link fault The bits [27:26] are reserved When current mode is RGMII: Bit[27]: ■ 0: Link Down ■ 1: Link Up Bit[26:25]: ■ 2'b00: 10Mbps ■ 2'b01: 100Mbps ■ 2'b10: 1Gbps ■ 2'b11: Reserved Bit[24]: ■ 0: Half Duplex ■ 1: Full Duplex **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
23	PCIS	R	Policing Counter Interrupt Status This bit is set high when an interrupt is generated in the MAC_PCTH_Intr_Status or MAC_PCTW_Intr_Status Register. This bit is cleared when all bits in MAC_PCTH_Intr_Status or MAC_PCTW_Intr_Status register are cleared. **Values:** ■ 0x0 (INACTIVE): MAC_PCTH_Intr_Status or MAC_PCTW_Intr_Status is not active ■ 0x1 (ACTIVE): MAC_PCTH_Intr_Status or MAC_PCTW_Intr_Status is active **Value After Reset:** 0x0 **Exists:** DWCXG_PC **Testable:** untestable

Table 11-51 Fields for Register: MAC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
22:19	Reserved_22_19	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
18	MFRIS	R	MMC FPE Receive Interrupt Status This bit is set high when an interrupt is generated in the MMC FPE Receive Interrupt Register. This bit is cleared when all bits in this interrupt register are cleared. This bit is valid only when you select the Enable MAC Management Counters (MMC) option along with FPE support. **Values:** ■ 0x0 (INACTIVE): MMC FPE Receive Interrupt status not active ■ 0x1 (ACTIVE): MMC FPE Receive Interrupt status active **Value After Reset:** 0x0 **Exists:** DWCXG_FPE&&DWCXG_MMC_FPE_EN **Testable:** untestable
17	MFTIS	R	MMC FPE Transmit Interrupt Status This bit is set high when an interrupt is generated in the MMC FPE Transmit Interrupt Register. This bit is cleared when all bits in this interrupt register are cleared.This bit is valid only when you select the Enable MAC Management Counters (MMC) option along with FPE support. **Values:** ■ 0x0 (INACTIVE): MMC FPE Transmit Interrupt status not active ■ 0x1 (ACTIVE): MMC FPE Transmit Interrupt status active **Value After Reset:** 0x0 **Exists:** (DWCXG_FPE||DWCXG_AV_EST)&&DWCXG_MMC_FPE_EN **Testable:** untestable

Table 11-51 Fields for Register: MAC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
16	FPEIS	R	Frame Preemption Interrupt Status This bit indicates an interrupt event during the operation of Frame Preemption (Bits[19:16] of MAC_FPE_CTRL_STS register is set). To reset this bit, the application must clear the event in MAC_FPE_CTRL_STS that has caused the interrupt. **Values:** ■ 0x0 (INACTIVE): Frame Preemption Interrupt status not active ■ 0x1 (ACTIVE): Frame Preemption Interrupt status active **Value After Reset:** 0x0 **Exists:** DWCXG_FPE **Testable:** untestable
15	GPIIS	R	GPI Interrupt Status. When the GPIO feature is enabled:This bit is set when an active event (LL or LH) occurs on the GPIS field of the MAC_GPIO_Status register and the corresponding GPIE bit is enabled in the MAC_GPIO_Control register. This bit is cleared on reading lane 0 (GPIS) of the MAC_GPIO_Status register. Access restriction applies. This bit is cleared when this bit is read (or this bit is written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14	RXESIS	R	Receive Error Status Interrupt. This bit indicates the error status of received packets. This bit is set when the Watchdog Timeout (WDT) event occurs during reception. Access restriction applies. This bit is cleared when the corresponding interrupt source bit is read (or corresponding interrupt source bit is written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set) in the MAC_Rx_Tx_Status register. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-51 Fields for Register: MAC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
13	TXESIS	R	Transmit Error Status Interrupt. ■ This bit indicates the error status of transmitted packets. This bit is set when any of the following bits is set in the MAC_Rx_Tx_Status during transmission. ■ Excessive Collision (EXCOL) ■ Late Collision (LCOL) ■ Excessive Deferral (EXDEF) ■ Loss of Carrier (LCARR) ■ No Carrier (NCARR) ■ Jabber Timeout (TJT) Access restriction applies. This bit is cleared when the corresponding interrupt source bit is read (or corresponding interrupt source bit is written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set) in the MAC_Rx_Tx_Status register. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
12	TSIS	R	Timestamp Interrupt Status. If the Timestamp feature is enabled: This bit is set when any of the following conditions is true: ■ The system time value is equal to or exceeds the value specified in the Target Time High and Low registers. ■ There is an overflow in the Seconds register. ■ The Target Time Error occurred, that is, programmed target time already elapsed. ■ When a timestamp of a transmitted PTP packet is available in the MAC_TxTimestamp_Status_Seconds register. When the Auxiliary Snapshot feature is enabled: This bit is set when the auxiliary snapshot trigger is asserted. Access restriction applies. This bit is cleared when the corresponding interrupt source bit is read (or corresponding interrupt source bit is written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set) in the MAC_Timestamp_Status register. **Value After Reset:** 0x0 **Exists:** ((DWCXG_TIME_STAMPING&&(DWCXG_SYSTIME_SOURCE!=1||!DWCXG_CORE))||DWCXG_ADV_TIME_AUX_SNAP||DWCXG_PTO_EN) **Testable:** untestable

Table 11-51 Fields for Register: MAC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
11	MMCRXIPIS	R	MMC Receive Checksum Offload Interrupt Status This bit is set high when an interrupt is generated in the MMC Receive Checksum Offload Interrupt Register. This bit is cleared when all bits in this interrupt register are cleared. This bit is valid only when you select the Enable MAC Management Counters (MMC) and Enable Receive TCP/IP Checksum Check options. **Values:** ■ 0x0 (INACTIVE): MMC Receive Checksum Offload Interrupt status not active ■ 0x1 (ACTIVE): MMC Receive Checksum Offload Interrupt status active **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_IPC_RX_PKTS_ATLEAST_ONE||DWCXG_MMC_IPC_RX_DGRAM_BCNT_ATLEAST_ONE||DWCXG_MMC_IPC_RX_PAYLD_BCNT_ATLEAST_ONE **Testable:** untestable
10	MMCTXIS	R	MMC Transmit Interrupt Status. This bit is set high when an interrupt is generated in the MMC Transmit Interrupt register. This bit is cleared when all bits in this interrupt register are cleared. This bit is valid only when you select the Enable DWC_25gmac Management Counter (MMC) option. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
9	MMCRXIS	R	MMC Receive Interrupt Status. This bit is set high when an interrupt is generated in the MMC Receive Interrupt register. This bit is cleared when all bits in this interrupt register are cleared. This bit is valid only when you select the Enable DWC_25gmac Management Counter (MMC) option. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
8:6	Reserved_8_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-51 Fields for Register: MAC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
5	LPIIS	R	LPI Interrupt Status. When the Energy Efficient Ethernet feature is enabled: This bit is set for any LPI state entry or exit in the MAC Transmitter or Receiver. In all other modes: This bit is reserved. **Note:** On the transmit side the Exit Interrupt is generate only when the XGMII/GMII interface comes out of the LPI and TW timer expires. Access restriction applies. This bit is cleared when the corresponding interrupt source bit of MAC_LPI_Control_Status register is read (or corresponding interrupt source bit of MAC_LPI_Control_Status register is written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** DWCXG_EEE_EN **Testable:** untestable
4	PMTIS	R	PMT Interrupt Status. This bit is set when a Magic packet or Wake-on-LAN packet is received in the power-down mode (RWKPRCVD and MGKPRCVD bits in MAC_PMT_Control_Status register). This bit is cleared when Bits[6:5] are cleared because of a Read operation to the MAC_PMT_Control_Status register. This bit is valid only when you select the Enable Power Management option. Access restriction applies. This bit is cleared when Bits[6:5] of MAC_PMT_Control_Status register is read (or corresponding interrupt source bit of MAC_PMT_Control_Status register is written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** DWCXG_PMT_EN **Testable:** untestable
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-51 Fields for Register: MAC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
1	SMI	R	SMA Interrupt. This bit is set when any of the bits in the MDIO Interrupt Status register is set. This bit is the logical OR of all the bits in that register and is cleared only when all the bits in the MDIO Interrupt register are cleared **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	LSI	R	Link Status change Interrupt. This bit is set when the LS (Link Status) bits change their value. It is cleared when this register is read. This interrupt is generated in RGMII mode when the link status change is detected in the RGMII Rx Interface. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.48 MAC_Interrupt_Enable

- **Description:** The Interrupt Enable register contains the masks for generating the interrupts.
- **Size:** 32 bits
- **Offset:** 0xb4
- **Exists:** Always

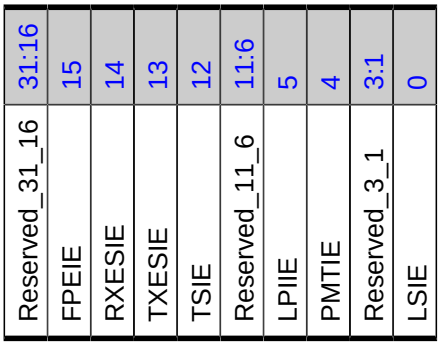


Table 11-52 Fields for Register: MAC_Interrupt_Enable

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15	FPEIE	R/W	Frame Preemption Interrupt Enable When this bit is set, it enables the assertion of the interrupt when FPEIS field is set in the MAC_Interrupt_Status register. **Values:** ■ 0x0 (DISABLE): Frame Preemption Interrupt is disabled ■ 0x1 (ENABLE): Frame Preemption Interrupt is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_FPE
14	RXESIE	R/W	Receive Error Status Interrupt Enable. When this bit is set, it enables the assertion of the interrupt signal because of the setting of the RXESIS bit in the MAC_Interrupt_Status register. **Value After Reset:** 0x0 **Exists:** Always
13	TXESIE	R/W	Transmit Error Status Interrupt Enable. When this bit is set, it enables the assertion of the interrupt signal because of the setting of the TXESIS bit in the MAC_Interrupt_Status register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-52 Fields for Register: MAC_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
12	TSIE	R/W	Timestamp Interrupt Enable. When this bit is set, it enables the assertion of the interrupt signal because of the setting of TSIS bit in MAC_Interrupt_Status register. **Value After Reset:** 0x0 **Exists:** ((DWCXG_TIME_STAMPING&&(DWCXG_SYSTIME_SOURCE!=1||!DWCXG_CORE))||DWCXG_ADV_TIME_AUX_SNAP||DWCXG_PTO_EN)
11:6	Reserved_11_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5	LPIIE	R/W	LPI Interrupt Enable. When this bit is set, it enables the assertion of the interrupt signal because of the setting of LPIIS bit in MAC_Interrupt_Status register. **Value After Reset:** 0x0 **Exists:** DWCXG_EEE_EN
4	PMTIE	R/W	PMT Interrupt Enable. When this bit is set, it enables the assertion of the interrupt signal because of the setting of PMTIS bit in MAC_Interrupt_Status register. **Value After Reset:** 0x0 **Exists:** DWCXG_PMT_EN
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	LSIE	R/W	Link Status Change Interrupt Enable. When this bit is set, it enables the assertion of the interrupt signal because of the setting of LSI bit in MAC_Interrupt_Status register. **Value After Reset:** 0x1 **Exists:** Always

11.1.49 MAC_Rx_Tx_Status

- **Description:** The Receive Transmit Status register contains the Receive and Transmit Error status.
- **Size:** 32 bits
- **Offset:** 0xb8
- **Exists:** Always

31:14	13	12	11:9	8	7:6	5	4	3	2	1	0
Reserved_31_14	PCE	IHE	Reserved_11_9	RWT	Reserved_7_6	EXCOL	LCOL	EXDEF	LCARR	NCARR	TJT

Table 11-53 Fields for Register: MAC_Rx_Tx_Status

Bits	Name	Memory Access	Description
31:14	Reserved_31_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13	PCE	R	Payload Checksum Error. This bit indicates that the Checksum Offload engine detected IP payload error and did not insert any checksum into the encapsulated TCP, UDP, or ICMP payload. This failure can be either because of insufficient bytes, as indicated by the Payload Length field of the IP Header or the MTL starting to forward the packet to the MAC transmitter in Store-and-Forward mode without the checksum having been calculated yet. This second error condition only occurs when the Transmit Queue depth is less than the length of the IP packet being transmitted to avoid deadlock, the MTL starts forwarding the packet when the FIFO is full, even in the store-and-forward mode. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** DWCXG_TX_COE

Table 11-53 Fields for Register: MAC_Rx_Tx_Status (continued)

Bits	Name	Memory Access	Description
12	IHE	R	IP Header Error. This bit indicates that the Checksum Offload engine detected IP header error. This failure can be either because of received Ethernet type is 0x0800 but Version field of IPv4 header is not equal to 0x4, IPv4 Header Length field indicates a value less than 0x5 (20 bytes), total packet length is less than the value given in the IPv4 Header Length field, received Ethernet type is 0x86dd but IPv6 header Version field is not equal to 0x6, packet ends before the IPv6 header (40 bytes) or extension header (as given in the corresponding Header Length field in an extension header) is completely received. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** DWCXG_TX_COE
11:9	Reserved_11_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	RWT	R	Receive Watchdog Timeout. This bit is set when a packet with length greater than 2048 bytes is received (10240 bytes when Jumbo Packet mode is enabled) and the WD bit is reset in the MAC_Rx_Configuration register or when a packet with length greater than 16383 bytes is received and the WD bit is set in the MAC_Rx_Configuration register. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-53 Fields for Register: MAC_Rx_Tx_Status (continued)

Bits	Name	Memory Access	Description
5	EXCOL	R	Excessive Collisions When the FTS bit is reset in the MTL_Operation_Mode register, this bit indicates that the transmission aborted after 16 successive collisions while attempting to transmit the current packet. If the DR bit is set in the MAC_Configuration register, this bit is set after the first collision and the packet transmission is aborted. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): No collision ■ 0x1 (ACTIVE): Excessive collision is sensed **Value After Reset:** 0x0 **Exists:** !DWCXG_FDUPLX_ONLY&&!DWCXG_CORE
4	LCOL	R	Late Collision When the FTS bit is reset in the MTL_Operation_Mode register, this bit indicates that the packet transmission aborted because a collision occurred after the collision window (64 bytes including Preamble in MII mode). This bit is not valid if the Underflow error occurs. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): No collision ■ 0x1 (ACTIVE): Late collision is sensed **Value After Reset:** 0x0 **Exists:** !DWCXG_FDUPLX_ONLY&&!DWCXG_CORE
3	EXDEF	R	Excessive Deferral When the FTS bit is reset in the MTL_Operation_Mode register and the DC bit is set in the MAC_Extended_Configuration register, this bit indicates that the transmission ended because of excessive deferral of over 24,288 bit times (when Jumbo packet is enabled). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): No Excessive deferral ■ 0x1 (ACTIVE): Excessive deferral **Value After Reset:** 0x0 **Exists:** !DWCXG_FDUPLX_ONLY&&!DWCXG_CORE

Table 11-53 Fields for Register: MAC_Rx_Tx_Status (continued)

Bits	Name	Memory Access	Description
2	LCARR	R	Loss of Carrier When the FTS bit is reset in the MTL_Operation_Mode register, this bit indicates that the loss of carrier occurred during packet transmission, that is, the Phy Carrier interface signal was inactive for one or more transmission clock periods during packet transmission. This bit is valid only for packets transmitted without collision. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Carrier is present ■ 0x1 (ACTIVE): Loss of carrier **Value After Reset:** 0x0 **Exists:** !DWCXG_FDUPLX_ONLY&&!DWCXG_CORE
1	NCARR	R	No Carrier When the FTS bit is reset in the MTL_Operation_Mode register, this bit indicates that the carrier signal from the PHY is not present at the end of preamble transmission. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Carrier is present ■ 0x1 (ACTIVE): No carrier **Value After Reset:** 0x0 **Exists:** !DWCXG_FDUPLX_ONLY&&!DWCXG_CORE
0	TJT	R	Transmit Jabber Timeout. This bit indicates that the Transmit Jabber Timer expired, which happens when the packet size exceeds 2048 bytes (10240 bytes when the Jumbo packet is enabled) and the JD bit is reset in the MAC_Tx_Configuration register. This bit is set when the packet size exceeds 16383 bytes and the JD bit is set in the MAC_Tx_Configuration register. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always

11.1.50 MAC_Tx_Clk_Slot_Interval

- **Description:** This register indicates the Number of MAC Transmit Clocks that constitutes a slot interval.
- **Size:** 32 bits
- **Offset:** 0xbc
- **Exists:** (!DWCXG_CORE)&&(DWCXG_NUM_TC>1)&&(!DWCXG_TX_AV_ENABLE)

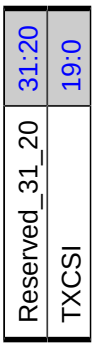
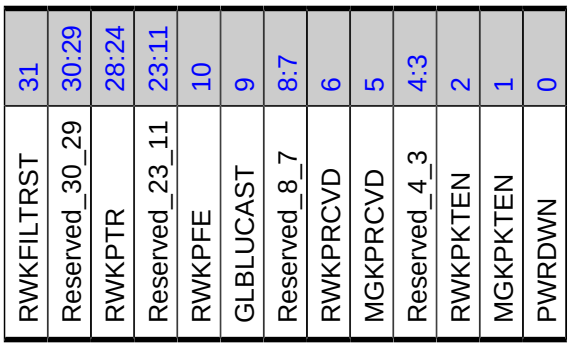


Table 11-54 Fields for Register: MAC_Tx_Clk_Slot_Interval

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19:0	TXCSI	R/W	MAC Transmit Clock Slot Interval. This field indicates the number of MAC Transmit Clocks that constitutes a slot interval. The value programmed is slot interval divided by MAC Transmit Clock period. For example, the slot interval of 125us is used for ABS field of MTL_TC(#i)_ETS_Status register, so the values to be programmed are as below: Speed Mode : Value ■ 10M : 312 ■ 100M : 3125 ■ 1G : 15625 ■ 2.5G GMII : 39063 ■ 2.5G XGMII : 9766 ■ 5G XGMII : 19531 ■ 10G : 39063 ■ 25G XGMII : 97656 When this field is all-zeros, the fixed value corresponding to 125us slot interval based on operating speed is used. **Value After Reset:** 0x0 **Exists:** Always

11.1.51 MAC_PMT_Control_Status

- **Description:** The PMT Control and Status register is present only when you select the PMT module in coreConsultant.
- **Size:** 32 bits
- **Offset:** 0xc0
- **Exists:** (DWCXG_PMT_EN)



31	30:29	28:24	23:11	10	9	8:7	6	5	4:3	2	1	0
RWKFILTRST	Reserved_30_29	RWKPTR	Reserved_23_11	RWKPFE	GLBLUCAST	Reserved_8_7	RWKPRCVD	MGKPRCVD	Reserved_4_3	RWKPKTEN	MGKPKTEN	PWRDWN

Table 11-55 Fields for Register: MAC_PMT_Control_Status

Bits	Name	Memory Access	Description
31	RWKFILTRST	R/W	Remote Wake-Up Packet Filter Register Pointer Reset. When this bit is set, the remote wake-up packet filter register pointer (RWKPTR) is reset to 0. It is automatically cleared after 1 clock cycle. **Value After Reset:** 0x0 **Exists:** DWCXG_PMT_RWK_EN **Testable:** untestable
30:29	Reserved_30_29	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
28:24	RWKPTR	R	Remote Wake-up FIFO Pointer. This field gives the current value of the Remote Wake-up Packet Filter register pointer. When the value of this pointer reaches the maximum for the selected number of filters (which equals 2 times the number of RWK filters -1), the contents of the Remote Wake-up Packet Filter register are transferred to the Rx clock (clk_rx_312pt5_i) domain when a Write occurs to that register. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-55 Fields for Register: MAC_PMT_Control_Status (continued)

Bits	Name	Memory Access	Description
23:11	Reserved_23_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
10	RWKPFE	R/W	Remote Wake-up Packet Forwarding Enable. When this bit is set along with RWKPKTEN, the MAC receiver drops all the received packets until it receives the expected Wake-up packet. All packets after that event including the received wake-up packet are forwarded to application. This bit is then self-cleared on receiving the wake-up packet. The application can also clear this bit before the expected wake-up packet is received. In such cases, the MAC reverts to the default behavior where packets received are forwarded to the application. This bit must only be set when RWKPKTEN is set high and PWRDWN bit is set low. The setting of this bit has no effect when PWRDWN bit is set high. **Note:** If Magic Packet Enable and Wake-Up Packet Enable are both set along with setting of this bit and Magic Packet is received prior to wake-up packet, this bit is self-cleared on receiving Magic Packet, the received Magic packet is dropped, and all packets that are received after the Magic Packet are forwarded to application. **Value After Reset:** 0x0 **Exists:** DWCXG_PMT_RWK_EN
9	GLBLUCAST	R/W	Global Unicast. When this bit is set, any unicast packet filtered by the MAC (DA matches MAC_Address0 register) address recognition is detected as a remote wake-up packet. **Value After Reset:** 0x0 **Exists:** DWCXG_PMT_RWK_EN
8:7	Reserved_8_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-55 Fields for Register: MAC_PMT_Control_Status (continued)

Bits	Name	Memory Access	Description
6	RWKPRCVD	R	Remote Wake-Up Packet Received. When this bit is set, it indicates that the power management event is generated because of the reception of a remote wake-up packet. This bit is cleared when this register is read. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always
5	MGKPRCVD	R	Magic Packet Received. When this bit is set, it indicates that the power management event is generated because of the reception of a magic packet. This bit is cleared when this register is read. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always
4:3	Reserved_4_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	RWKPKTEN	R/W	Remote Wake-Up Packet Enable. When this bit is set, a power management event is generated when the MAC receives a remote wake-up packet. **Value After Reset:** 0x0 **Exists:** DWCXG_PMT_RWK_EN
1	MGKPKTEN	R/W	Magic Packet Enable. When this bit is set, a power management event is generated when the MAC receives a magic packet. **Value After Reset:** 0x0 **Exists:** DWCXG_PMT_MGK_EN

Table 11-55 Fields for Register: MAC_PMT_Control_Status (continued)

Bits	Name	Memory Access	Description
0	PWRDWN	R/W	Power Down. When this bit is set, the MAC receiver drops all the received packets until it receives the expected magic packet or remote wake-up packet. This bit is then self-cleared and the power-down mode is disabled. The software can clear this bit before the expected magic packet or remote wake-up packet is received. The packets received by the MAC after this bit is cleared are forwarded to the application. This bit must only be set when the Magic Packet Enable, Global Unicast, or Remote Wake-Up Packet Enable bit is set high. **Note:** You can gate-off the CSR clock during the power-down mode. However, when the CSR clock is gated-off, you cannot perform any read or write operations on this register. Therefore, the Software cannot clear this bit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.52 MAC_RWK_Packet_Filter

- **Description:** The Remote Wakeup Filter registers are implemented as 8, 16, or 32 indirect access registers (wkuppktfilter_reg#i) based on whether 4, 8, or 16 Remote Wakeup Filters are selected in the configuration and accessed by application through MAC_RWK_Packet_Filter register. When the Remote Wakeup Filters are to be programmed, the entire set of wkuppktfilter_reg registers must be written. The wkuppktfilter_reg register is programmed by sequentially writing the eight, sixteen or thirty-two register values in MAC_RWK_Packet_Filter register for wkuppktfilter_reg0, wkuppktfilter_reg1, ..., wkuppktfilter_reg31 respectively. The wkuppktfilter_reg register is read in a similar way. The MAC updates the wkuppktfilter_reg register current pointer value in RWKPTR field of MAC_PMT_Control_Status register.
  The Remote Wakeup Filters are arranged in blocks of 4 filters each and each such block have eight 32-bit wide registers, that is, wkuppktfilter_reg0-7, wkuppktfilter_reg8-15, wkuppktfilter_reg16-23 and wkuppktfilter_reg24-31. The fields of Remote Wakeup Filter are described as follows:
  Filter i Byte Mask: The filter i byte mask register defines the bytes of the packet that are examined by filter i (0, 1, 2, 3, .., 15) to determine whether or not a packet is a wake-up packet.
  - The MSB (31st bit) must be zero.
  - Bit j[30:0] is the byte mask.
  - If Bit j (byte number) of the byte mask is set, the CRC block processes the Filter i Offset + j of the incoming packet; otherwise Filter i Offset + j is ignored.

  Filter i Command: The 4-bit filter i command controls the filter i operation.
  - Bit 3 specifies the address type, defining the destination address type of the pattern. When the bit is set, the pattern applies to only multicast packets; when the bit is reset, the pattern applies only to unicast packet.
  - Bit 2 (Inverse Mode), when set, reverses the logic of the CRC16 hash function signal, to reject a packet with matching CRC-16 value. Bit 2, along with Bit 1, allows a MAC to reject a subset of remote wake-up packets by creating filter logic such as "Pattern 1 AND NOT Pattern 2".
  - Bit 1 (And_Previous) implements the Boolean logic. When set, the result of the current entry is logically ANDed with the result of the previous filter. This AND logic allows a filter pattern longer than 32 bytes by splitting the mask among two, three, or four filters. This depends on the number of filters that have the And_Previous bit set. The details are,
    - The And_Previous bit setting is applicable within a set of 4 filters.
    - Setting of And_Previous bit of filter that is not enabled has no effect, that is setting And_Previous bit of lowest number filter in the set of 4 filters has no effect. For example, setting of And_Previous bit of Filter 0 has no effect.
    - If And_Previous bit is set for filter to form AND chained filter, the AND chain breaks at the point any filter is not enabled. For example: If Filter 2 And_Previous bit is set (bit 1 in Filter 2 command is set) but Filter 1 is not enabled (bit 0 in Filter 1 command is reset), then only Filter 2 result is considered. If Filter 2 And_Previous bit is set (bit 1 in Filter 2 command is set), Filter 3 And_Previous bit is set (bit 1 in Filter 3 command is set), but Filter 1 is not enabled (bit 0 in Filter 1 command is reset), then only Filter 2 result ANDed with Filter 3 result is considered. If Filter 2 And_Previous bit is set (bit 1 in Filter 2 command is set), Filter 3 And_Previous bit is set (bit 1 in Filter 3 command is set), but Filter 2 is not enabled (bit 0 in Filter 2 command is reset), then since setting of Filter 2 And_Previous bit has no effect only Filter 1 result ORed with Filter 3 result is considered.
    - If filters chained by And_Previous bit setting have complementary programming, then a frame might never pass the AND chained filter. For example, if Filter 2 And_Previous bit is set (bit 1 in Filter 2 command is set), Filter 1 Address_Type bit is set (bit 3 in Filter 1

command is set) indicating multicast detection and Filter 2 Address_Type bit is reset (bit 3 in Filter 2 command is reset) indicating unicast detection or vice versa, a remote wakeup frame does not pass the AND chained filter as a remote wakeup frame cannot be of both unicast and multicast address type.

- Bit 0 is the enable for filter i. If Bit 0 is not set, filter i is disabled.

Filter i Offset: The filter i offset register defines the offset (within the packet) from which the filter i examines the packets.

- This 8-bit pattern-offset is the offset for the filter i first byte to be examined.
- The minimum allowed offset is 12, which refers to the 13th byte of the packet.
- The offset value 0 refers to the first byte of the packet.

Filter i CRC-16: The filter i CRC-16 register contains the CRC-16 value calculated from the pattern and the byte mask programmed in the Remote Wakeup filter register.

- The 16-bit CRC calculation uses the following polynomial: G(x) = x^16 + x^15 + x^2 + 1
- Each mask, used in the hash function calculation, is compared with a 16-bit value associated with that mask. Each filter has the following:
  - 32-bit Mask: Each bit in this mask corresponds to one byte in the detected packet. If the bit is 1, the corresponding byte is taken into the CRC-16 calculation.
  - 8-bit Offset Pointer: Specifies the byte to start the CRC-16 computation. The pointer and the mask are used together to locate the bytes to be used in the CRC-16 calculations.

- **Size:** 32 bits
- **Offset:** 0xc4
- **Exists:** (DWCXG_PMT_RWK_EN)

Table 11-56 Fields for Register: MAC_RWK_Packet_Filter

Bits	Name	Memory Access	Description
31:0	WKUPFRMFTR	R/W	Wake-up Packet Filter This field contains the various controls of RWK Packet filter. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.53 RWK_Filter(#i)_Byte_Mask (for i = 0; i <= DWCXG_NUM_PMT_RWK_FILT-1)

- **Description:** Filter i Byte Mask. The filter i byte mask register defines the bytes of the packet that are examined by filter i to determine whether or not a packet is a wake-up packet. The MSB (31st bit) must be zero. Bit j[30:0] is the byte mask. If Bit j (byte number) of the byte mask is set, the CRC block processes the byte pointed by Filter i Offset + j of the incoming packet; otherwise Filter i Offset + j is ignored.
  You can access these registers indirectly through the MAC_RWK_Packet_Filter register.
- **Size:** 32 bits
- **Offset:** 0xc4
- **Exists:** DWCXG_PMT_RWK_EN&&(DWCXG_NUM_PMT_RWK_FILT > 0)

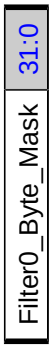


Table 11-57 Fields for Register: RWK_Filter(#i)_Byte_Mask (for i = 0; i <= DWCXG_NUM_PMT_RWK_FILT-1)

Bits	Name	Memory Access	Description
31:0	Filter0_Byte_Mask	R/W	Byte Mask Each bit in this mask corresponds to one byte in the detected packet. If the bit is 1', the corresponding byte is taken into the CRC16 calculation. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.54 RWK_Filter(#i)_(#i+3)_Command (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1)

- **Description:** This register controls the filter operation.
- **Size:** 32 bits
- **Offset:** 0xc4
- **Exists:** DWCXG_PMT_RWK_EN&&(DWCXG_NUM_PMT_RWK_FILT > 0)

31:28	27:24	23:20	19:16	15:12	11:8	7:4	3:0
RSVD3	Filter3_Command	RSVD2	Filter2_Command	RSVD1	Filter1_Command	RSVD0	Filter0_Command

Table 11-58 Fields for Register: RWK_Filter(#i)_(#i+3)_Command (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1)

Bits	Name	Memory Access	Description
31:28	RSVD3	R	Reserved **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
27:24	Filter3_Command	R/W	Command3 The 4-bit filter command controls the filter operation. ■ Bit 3 specifies the address type, defining the destination address type of the pattern. When the bit is set, the pattern applies to only multicast packets; when the bit is reset, the pattern applies only to unicast packet. ■ Bit 2 (Inverse Mode), when set, reverses the logic of the CRC16 hash function signal, to reject a packet with matching CRC_16 value. ■ Bit 2, along with Bit 1, allows a MAC to reject a subset of remote wake-up packets by creating filter logic such as "Pattern 1 AND NOT Pattern 2". ■ Bit 1 (And_Previous) implements the Boolean logic. When set, the result of the current entry is logically ANDed with the result of the previous filter. This AND logic allows a filter pattern longer than 32 bytes by splitting the mask among two, three, or four filters. This depends on the number of filters that have the And_Previous bit set. ■ Bit 0 is the enable for filter. If Bit 0 is not set, filter is disabled. **Value After Reset:** 0x0

Table 11-58 Fields for Register: RWK_Filter(#i)_(#i+3)_Command (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1) (continued)

Bits	Name	Memory Access	Description
			Exists: Always **Testable:** untestable
23:20	RSVD2	R	Reserved **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
19:16	Filter2_Command	R/W	Command2 The 4-bit filter command controls the filter operation. ■ Bit 3 specifies the address type, defining the destination address type of the pattern. When the bit is set, the pattern applies to only multicast packets; when the bit is reset, the pattern applies only to unicast packet. ■ Bit 2 (Inverse Mode), when set, reverses the logic of the CRC16 hash function signal, to reject a packet with matching CRC_16 value. ■ Bit 2, along with Bit 1, allows a MAC to reject a subset of remote wake-up packets by creating filter logic such as "Pattern 1 AND NOT Pattern 2". ■ Bit 1 (And_Previous) implements the Boolean logic. When set, the result of the current entry is logically ANDed with the result of the previous filter. This AND logic allows a filter pattern longer than 32 bytes by splitting the mask among two, three, or four filters. This depends on the number of filters that have the And_Previous bit set. ■ Bit 0 is the enable for filter. If Bit 0 is not set, filter is disabled. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15:12	RSVD1	R	Reserved **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
11:8	Filter1_Command	R/W	Command1 The 4-bit filter command controls the filter operation. ■ Bit 3 specifies the address type, defining the destination address type of the pattern. When the bit is set, the pattern applies to only multicast packets; when the bit is reset, the pattern applies only to unicast packet.

Table 11-58 Fields for Register: RWK_Filter(#i)_(#i+3)_Command (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1) (continued)

Bits	Name	Memory Access	Description
			■ Bit 2 (Inverse Mode), when set, reverses the logic of the CRC16 hash function signal, to reject a packet with matching CRC_16 value. ■ Bit 2, along with Bit 1, allows a MAC to reject a subset of remote wake-up packets by creating filter logic such as "Pattern 1 AND NOT Pattern 2". ■ Bit 1 (And_Previous) implements the Boolean logic. When set, the result of the current entry is logically ANDed with the result of the previous filter. This AND logic allows a filter pattern longer than 32 bytes by splitting the mask among two, three, or four filters. This depends on the number of filters that have the And_Previous bit set. ■ Bit 0 is the enable for filter. If Bit 0 is not set, filter is disabled. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
7:4	RSVD0	R	Reserved **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
3:0	Filter0_Command	R/W	Command 0 The 4-bit filter command controls the filter operation. ■ Bit 3 specifies the address type, defining the destination address type of the pattern. When the bit is set, the pattern applies to only multicast packets; when the bit is reset, the pattern applies only to unicast packet. ■ Bit 2 (Inverse Mode), when set, reverses the logic of the CRC16 hash function signal, to reject a packet with matching CRC_16 value. ■ Bit 2, along with Bit 1, allows a MAC to reject a subset of remote wake-up packets by creating filter logic such as "Pattern 1 AND NOT Pattern 2". ■ Bit 1 (And_Previous) implements the Boolean logic. When set, the result of the current entry is logically ANDed with the result of the previous filter. This AND logic allows a filter pattern longer than 32 bytes by splitting the mask among two, three, or four filters. This depends on the number of filters that have the And_Previous bit set. ■ Bit 0 is the enable for filter. If Bit 0 is not set, filter is disabled. **Value After Reset:** 0x0

Table 11-58 Fields for Register: RWK_Filter(#i)_(#i+3)_Command (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1) (continued)

Bits	Name	Memory Access	Description
			Exists: Always **Testable:** untestable

11.1.55 RWK_Filter(#i)_(#i+3)_Offset (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1)

- **Description:** This register defines the offset (within the packet) from which the filter examines the packet
- **Size:** 32 bits
- **Offset:** 0xc4
- **Exists:** DWCXG_PMT_RWK_EN&&(DWCXG_NUM_PMT_RWK_FILT > 0)

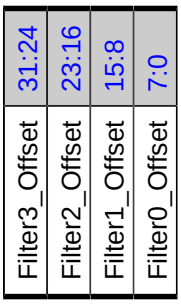


Table 11-59 Fields for Register: RWK_Filter(#i)_(#i+3)_Offset (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1)

Bits	Name	Memory Access	Description
31:24	Filter3_Offset	R/W	Offset3 This filter offset defines the offset (within the packet) from which the filter examines the packets. ■ This 8-bit pattern-offset is the offset for the filter first byte to be examined. ■ The minimum allowed offset is 12, which refers to the 13th byte of the packet. ■ The offset value 0 refers to the first byte of the packet. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
23:16	Filter2_Offset	R/W	Offset2 This filter offset defines the offset (within the packet) from which the filter examines the packets. ■ This 8-bit pattern-offset is the offset for the filter first byte to be examined. ■ The minimum allowed offset is 12, which refers to the 13th byte of the packet. ■ The offset value 0 refers to the first byte of the packet. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-59 Fields for Register: RWK_Filter(#i)_(#i+3)_Offset (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/4)-1) (continued)

Bits	Name	Memory Access	Description
15:8	Filter1_Offset	R/W	Offset1 This filter offset defines the offset (within the packet) from which the filter examines the packets. ■ This 8-bit pattern-offset is the offset for the filter first byte to be examined. ■ The minimum allowed offset is 12, which refers to the 13th byte of the packet. ■ The offset value 0 refers to the first byte of the packet. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
7:0	Filter0_Offset	R/W	Offset0 This filter offset defines the offset (within the packet) from which the filter examines the packets. ■ This 8-bit pattern-offset is the offset for the filter first byte to be examined. ■ The minimum allowed offset is 12, which refers to the 13th byte of the packet. ■ The offset value 0 refers to the first byte of the packet. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.56 RWK_Filter(#i)_(#i+1)_CRC (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/2)-1)

- **Description:** Filter i CRC-16.
  This register contains the CRC-16 to be matched.
- **Size:** 32 bits
- **Offset:** 0xc4
- **Exists:** DWCXG_PMT_RWK_EN&&(DWCXG_NUM_PMT_RWK_FILT > 0)

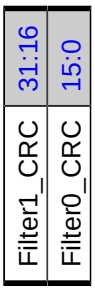
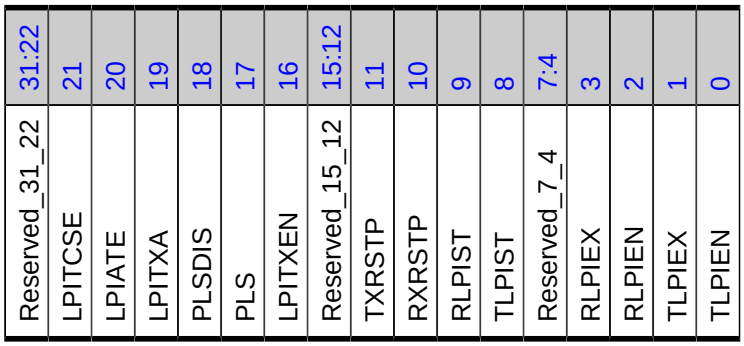


Table 11-60 Fields for Register: RWK_Filter(#i)_(#i+1)_CRC (for i = 0; i <= (DWCXG_NUM_PMT_RWK_FILT/2)-1)

Bits	Name	Memory Access	Description
31:16	Filter1_CRC	R/W	CRC1 This filter CRC-16 contains the CRC_16 value of the pattern. ■ The 16-bit CRC calculation uses the following polynomial: G(x) = x^16 + x^15 + x^2 + 1 **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15:0	Filter0_CRC	R/W	CRC0 This filter CRC-16 contains the CRC_16 value of the pattern. ■ The 16-bit CRC calculation uses the following polynomial: G(x) = x^16 + x^15 + x^2 + 1 **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.57 MAC_LPI_Control_Status

- **Description:** The LPI Control and Status register controls the LPI functions and provides the LPI interrupt status. The status bits are cleared when this register is read. This register is present only when you select the Energy Efficient Ethernet feature while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xd0
- **Exists:** (DWCXG_EEE_EN)



31:22	21	20	19	18	17	16	15:12	11	10	9	8	7:4	3	2	1	0
Reserved_31_22	LPITCSE	LPIATE	LPITXA	PLSDIS	PLS	LPITXEN	Reserved_15_12	TXRSTP	RXRSTP	RLPIST	TLPIST	Reserved_7_4	RLPIEX	RLPIEN	TLPIEX	TLPIEN

Table 11-61 Fields for Register: MAC_LPI_Control_Status

Bits	Name	Memory Access	Description
31:22	Reserved_31_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
21	LPITCSE	R/W	LPI Tx Clock Stop Enable. When this bit is 0, the output transmit clock gating signal(sbd_tx_clk_gating_ctrl_o) is not asserted. When this bit is 1, the output transmit clock gating signal(sbd_tx_clk_gating_ctrl_o) is asserted during Transmit LPI mode which can be used to Gate the Transmit Clock. **Value After Reset:** 0x0 **Exists:** Always

Table 11-61 Fields for Register: MAC_LPI_Control_Status (continued)

Bits	Name	Memory Access	Description
20	LPIATE	R/W	LPI Auto Timer Enable. This bit controls the automatic entry of the MAC Transmitter into and exit out of the LPI state. When LPIATE, LPITXA, and LPITXEN bits are set, the MAC Transmitter enters LPI state only when the complete MAC Tx data path is IDLE for a period indicated by the MAC_LPI_Auto_Entry_Timer register. After entering LPI state, if the data path becomes non-IDLE (due to a new packet being accepted for transmission), the Transmitter exits LPI state but does not clear LPITXEN bit. This enables the re-entry into LPI state when it is IDLE again. When LPIATE is 0, the LPI Auto timer is disabled and MAC Transmitter enters LPI state based on the settings of LPITXA and LPITXEN bit descriptions. **Value After Reset:** 0x0 **Exists:** Always
19	LPITXA	R/W	LPI Tx Automate. This bit controls the behavior of the MAC when it is entering or coming out of the LPI mode on the Transmit side. This bit is not functional in the XGMAC-CORE configurations as the Tx clock gating is done during the LPI mode. If the LPITXA and LPITXEN bits are set to 1, the MAC enters the LPI mode only after all outstanding packets (in the controller) and pending packets (in the application interface) have been transmitted. The MAC comes out of the LPI mode when the application sends any packet for transmission or the application issues a Tx FIFO Flush command. In addition, the MAC automatically clears the LPITXEN bit when it exits the LPI state. If Tx FIFO Flush is set in the FTQ bit of MTL_TxQ0_Operation_Mode register, when the MAC is in the LPI mode, it exits the LPI mode. When this bit is 0, the LPITXEN bit directly controls the behavior of the MAC when it enters or comes out of the LPI mode. **Value After Reset:** 0x0 **Exists:** Always

Table 11-61 Fields for Register: MAC_LPI_Control_Status (continued)

Bits	Name	Memory Access	Description
18	PLSDIS	R/W	PHY Link Status Disable. This bit disables the link status received on the XGMII receive paths, which are used for activating the LPI LS TIMER. When set to 1, the MAC ignores the link status from the XGMII interface and uses the link status from bit 17 (PLS). When set to 0, the MAC uses the link status from the XGMII in the 10G mode. In the GMII or 1G mode, the controller does not use this bit. The controller depends on bit 17 (PLS) for activating the LPI LS TIMER. **Value After Reset:** 0x0 **Exists:** Always
17	PLS	R/W	PHY Link Status. This bit indicates the link status of the PHY. The MAC Transmitter asserts the LPI pattern only when the link status is up (OKAY) at least for the time indicated by the LPI LS TIMER. When this bit is set, the link is considered to be okay (UP) and when this bit is reset, the link is considered to be down. **Value After Reset:** 0x0 **Exists:** Always
16	LPITXEN	R/W	LPI Transmit Enable. When this bit is set, it instructs the MAC Transmitter to enter the LPI state. When this bit is reset, it instructs the MAC to exit the LPI state and resume normal transmission. This bit is cleared when the LPITXA bit is set and the MAC exits the LPI state because of the arrival of a new packet for transmission. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15:12	Reserved_15_12	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
11	TXRSTP	R	Tx Reset in Progress. This bit indicates that the transmit EEE specific reset is in progress. **Value After Reset:** 0x0 **Exists:** !(DWCXG_XGMII == 0)

Table 11-61 Fields for Register: MAC_LPI_Control_Status (continued)

Bits	Name	Memory Access	Description
10	RXRSTP	R	Rx Reset in progress. This bit indicates that the receive EEE specific reset is in progress. **Value After Reset:** 0x0 **Exists:** !(DWCXG_XGMII == 0)
9	RLPIST	R	Receive LPI State. When this bit is set, it indicates that the MAC is receiving the LPI pattern on the GMII or XGMII interface. **Value After Reset:** 0x0 **Exists:** Always
8	TLPIST	R	Transmit LPI State. When this bit is set, it indicates that the MAC is transmitting the LPI pattern on the GMII or XGMII interface. **Value After Reset:** 0x0 **Exists:** Always
7:4	Reserved_7_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	RLPIEX	R	Receive LPI Exit. When this bit is set, it indicates that the MAC Receiver has stopped receiving the LPI pattern on the GMII or XGMII interface, exited the LPI state, and resumed the normal reception. Access restriction applies. This bit is cleared by a read into this register (or this bit is written to 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). **Note:** This bit might not be set if the MAC stops receiving the LPI pattern for a very short duration, such as, less than three clock cycles of CSR clock. **Value After Reset:** 0x0 **Exists:** Always

Table 11-61 Fields for Register: MAC_LPI_Control_Status (continued)

Bits	Name	Memory Access	Description
2	RLPIEN	R	Receive LPI Entry. When this bit is set, it indicates that the MAC Receiver has received an LPI pattern and entered the LPI state. Access restriction applies. This bit is cleared by a read into this register (or this bit is written to 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). **Note:** This bit might not be set if the MAC receives the LPI pattern for a very short duration, such as, less than three clock cycles of CSR clock. **Value After Reset:** 0x0 **Exists:** Always
1	TLPIEX	R	Transmit LPI Exit. When this bit is set, it indicates that the MAC transmitter exited the LPI state after the application cleared the LPITXEN bit and the LPI TW Timer has expired. Access restriction applies. This bit is cleared by a read into this register (or this bit is written to 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always
0	TLPIEN	R	Transmit LPI Entry. When this bit is set, it indicates that the MAC Transmitter has entered the LPI state because of the setting of the LPITXEN bit. Access restriction applies. This bit is cleared by a read into this register (or this bit is written to 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always

11.1.58 MAC_LPI_Timers_Control

- **Description:** The LPI Timers Control register controls the timeout values in the LPI states. It specifies the time for which the MAC transmits the LPI pattern and also the time for which the MAC waits before resuming the normal transmission. This register is present only when you select the Energy Efficient Ethernet feature while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xd4
- **Exists:** (DWCXG_EEE_EN)

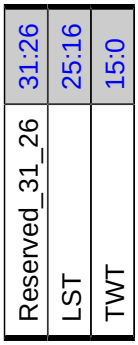


Table 11-62 Fields for Register: MAC_LPI_Timers_Control

Bits	Name	Memory Access	Description
31:26	Reserved_31_26	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
25:16	LST	R/W	LPI LS TIMER. This field specifies the minimum time (in milliseconds) for which the link status from the PHY must be up (OKAY) before the LPI pattern can be transmitted to the PHY. The MAC does not transmit the LPI pattern even when the LPITXEN bit is set unless the LPI LS Timer reaches the programmed terminal count. The default value of the LPI LS Timer is 1000 (1 sec) as defined in the IEEE standard. **Value After Reset:** 0x3e8 **Exists:** Always
15:0	TWT	R/W	LPI TW TIMER. This field specifies the minimum time (in microseconds) for which the MAC waits after it stops transmitting the LPI pattern to the PHY and before it resumes the normal transmission. The TLPIEX status bit is set after the expiry of this timer. **Value After Reset:** 0x0 **Exists:** Always

11.1.59 MAC_LPI_Auto_Entry_Timer

- **Description:** This register is used to store the LPI Auto Entry Timer Value in Micro-Seconds. This register is present only when you select the Energy Efficient Ethernet feature while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xd8
- **Exists:** (DWCXG_EEE_EN)

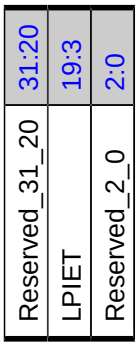


Table 11-63 Fields for Register: MAC_LPI_Auto_Entry_Timer

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19:3	LPIET	R/W	LPI Entry Timer. This field is programmed with a value in micro-seconds. When LPIATE, LPITXA, and LPITXEN bits of the MAC_LPI_Control_Status are set, DWC_25gmac automatically enters Tx LPI mode after transmitter is IDLE for LPIET amount of time. Bits[2:0] are read-only such that the granularity of this timer is 8 micro-seconds. **Value After Reset:** 0x0 **Exists:** Always
2:0	Reserved_2_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.60 MAC_1US_Tic_Counter

- **Description:** This register controls the generation of the Reference time (1micro-second tic) for all the LPI timers. This timer has to be programmed by the software initially. This register is present only when you select the Energy Efficient Ethernet feature while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xdc
- **Exists:** (DWCXG_EEE_EN)

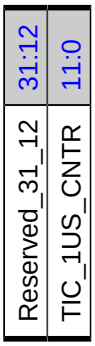


Table 11-64 Fields for Register: MAC_1US_Tic_Counter

Bits	Name	Memory Access	Description
31:12	Reserved_31_12	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
11:0	TIC_1US_CNTR	R/W	1US TIC Counter. This counter must be programmed with the number of clock cycles of CSR clock that constitutes a period of 1us. (Subtract 1 from the value before programming). For example if the CSR clock frequency is 100MHz, then this field needs to be programmed to value 100 - 1 = 99 (which is 0x63). This is required to generate the 1US events which are used to update some of the EEE related timers and counters. **Value After Reset:** 0x63 **Exists:** Always

11.1.61 MAC_Tunnel_Identifier

- **Description:** This register provides the programmable value to used as the identifier for VxLAN and NVGRE tunneled packets. This register is present only when you select the "Enable support for VxLAN/NVGRE packets" while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xe0
- **Exists:** (DWCXG_VXN_ENA)

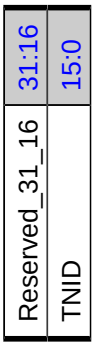


Table 11-65 Fields for Register: MAC_Tunnel_Identifier

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:0	TNID	R/W	Tunnel Identifier. Value of this field is used to compare and match with specific fields of incoming packets to identify VxLAN or NVGRE tunneled packets. UDP/IP packets received with programmed value as the UDP Destination Port field are identified as VXLAN packets. Similarly, IP packets having the programmed value as the Protocol Type field are identified as NVGRE packet. **Value After Reset:** 0x12b5 **Exists:** Always

11.1.62 MAC_Version

- **Description:** The version register identifies the version of DWC_25gmac. This register contains two bytes: one that Synopsys uses to identify the controller release number, and the other is user defined (that set while configuring the controller.)
- **Size:** 32 bits
- **Offset:** 0x110
- **Exists:** Always

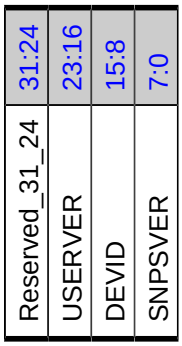


Table 11-66 Fields for Register: MAC_Version

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:16	USERVER	R	User-defined Version (configured with coreConsultant) **Value After Reset:** DWCXG_USER_VER **Exists:** Always
15:8	DEVID	R	Indicates the Device family **Value After Reset:** 0x55 **Exists:** Always
7:0	SNPSVER	R	Synopsys-defined Version (2.0) **Value After Reset:** 0x42 **Exists:** Always

11.1.63 MAC_Debug

- **Description:** The Debug register provides the debug status of various MAC blocks.
- **Size:** 32 bits
- **Offset:** 0x114
- **Exists:** Always

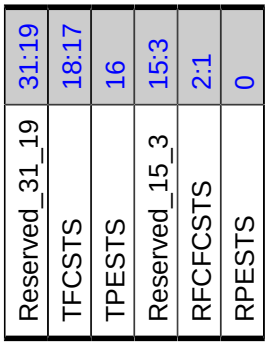


Table 11-67 Fields for Register: MAC_Debug

Bits	Name	Memory Access	Description
31:19	Reserved_31_19	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
18:17	TFCSTS	R	MAC Transmit Packet Controller Status. This field indicates the state of the MAC Transmit Packet Controller module: ■ 2'b00: Idle state ■ 2'b01: Waiting for one of the following: ❑ Status of the previous packet or ❑ IPG period to be over ■ 2'b10: Generating and transmitting a Pause control packet ■ 2'b11: Transferring input packet for transmission **Value After Reset:** 0x0 **Exists:** Always
16	TPESTS	R	MAC GMII or XGMII Transmit Protocol Engine Status. When this bit is set, it indicates that the MAC GMII or XGMII transmit protocol engine is actively transmitting data, and it is not in the Idle state. **Value After Reset:** 0x0 **Exists:** Always
15:3	Reserved_15_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-67 Fields for Register: MAC_Debug (continued)

Bits	Name	Memory Access	Description
2:1	RFCFCSTS	R	MAC Receive Packet Controller FIFO Status. This field indicates the active state of the small FIFO Read and Write controllers of the MAC Receive Packet Controller module. When bit 2 is set, small FIFO Read controller is active. When bit 1 is set, small FIFO Write controller is active. **Value After Reset:** 0x0 **Exists:** Always
0	RPESTS	R	MAC, GMII, or XGMII Receive Protocol Engine Status. When this bit is set, it indicates that the MAC, GMII, or XGMII receive protocol engine is actively receiving data, and it is not in the Idle state. **Value After Reset:** 0x0 **Exists:** Always

11.1.64 MAC_HW_Feature0

- **Description:** This register indicates the presence of the first set of optional features or functions of DWC_25gmac. The software driver can use this register to dynamically enable or disable the programs related to the optional blocks.
- **Size:** 32 bits
- **Offset:** 0x11c
- **Exists:** Always

31	30	29	28	27	26:25	24:23	22:17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EDMA	EDIFFC	VXN	Reserved_28	SAVLANINS	TSTSSEL	PHYSEL	ADDMACADRSEL	RXCOESEL	Reserved_15	TXCOESEL	EEESEL	TSSEL	AVSEL	RAVSEL	ARPOFFSEL	MMCSEL	MGKSEL	RWKSEL	SMASEL	VLHASH	HDSEL	RMIISEL	GMIISEL	RGMIISEL

Table 11-68 Fields for Register: MAC_HW_Feature0

Bits	Name	Memory Access	Description
31	EDMA	R	Enhanced DMA. This bit is always set to 1 when AXI-DMA interface is present in DWC_25gmac. Indicates that only the "Enhanced DMA" option is supported. This bit is retained for backward-compatibility. **Value After Reset:** DWCXG_EDMA **Exists:** Always
30	EDIFFC	R	Different Descriptor Cache. When set to 1, then DMA mode Separate Memory is selected for the Descriptor Cache. **Value After Reset:** DWCXG_DIFF_CACHE_MEM_EN **Exists:** Always
29	VXN	R	VxLAN/NVGRE Support. When set to 1 then the support for VxLAN/NVGRE is selected. **Value After Reset:** DWCXG_VXN_ENA **Exists:** Always
28	Reserved_28	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-68 Fields for Register: MAC_HW_Feature0 (continued)

Bits	Name	Memory Access	Description
27	SAVLANINS	R	Source Address or VLAN Insertion Enable. This bit is set to 1 when the Enable SA and VLAN Insertion on Tx option is selected. **Value After Reset:** DWCXG_SA_VLAN_INS_CTRL_EN **Exists:** Always
26:25	TSSTSSEL	R	Timestamp System Time Source. This bit indicates the source of the Timestamp system time: ■ 01: Internal ■ 10: External ■ 11: Both ■ 00: Reserved **Value After Reset:** (DWCXG_TIME_STAMPING)?(DWCXG_SYSTIME_SOURCE+1):0 **Exists:** Always
24:23	PHYSEL	R	RGMII Interface Select This field reflect the IO value of RGMII Interface signal which when set to 1 indicates RGMII interface is selected. ■ 00: Reserved ■ 01: RGMII interface ■ 10: RMII Interface ■ 11: Reserved **Value After Reset:** 0x0 **Exists:** DWCXG_RGMII_EN
22:17	ADDMACADRSEL	R	MAC Addresses 1-31 Selected. This field gives the number of additional MAC addresses selected using the Enable Additional 1-31 MAC Address Registers option. **Value After Reset:** DWCXG_ADD_MAC_ADDR_REG **Exists:** Always
16	RXCOESEL	R	Receive Checksum Offload Enabled. This bit is set to 1 when the Enable Receive TCP/IP Checksum Check option is selected. **Value After Reset:** DWCXG_RX_COE **Exists:** Always
15	Reserved_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-68 Fields for Register: MAC_HW_Feature0 (continued)

Bits	Name	Memory Access	Description
14	TXCOESEL	R	Transmit Checksum Offload Enabled. This bit is set to 1 when the Enable Transmit TCP/IP Checksum Offload option is selected. **Value After Reset:** DWCXG_TX_COE **Exists:** Always
13	EEESEL	R	Energy Efficient Ethernet Enabled. This bit is set to 1 when the Enable Energy Efficient Ethernet (EEE) option is selected. **Value After Reset:** DWCXG_EEE_EN **Exists:** Always
12	TSSEL	R	IEEE 1588-2008 Timestamp Enabled. This bit is set to 1 when the Enable IEEE 1588 Timestamp Support option is selected. **Value After Reset:** DWCXG_TIME_STAMPING **Exists:** Always
11	AVSEL	R	AV Feature Enabled. This bit is set to 1 when the Enable Audio Video Bridging option is selected. **Value After Reset:** DWCXG_AV_ENABLE **Exists:** Always
10	RAVSEL	R	Rx Side Only AV Feature Enable. This bit is set to 1 when the Enable Audio Video Bridging option on Rx Side Only is selected. **Value After Reset:** (DWCXG_AV_ENABLE&&!DWCXG_TX_AV_ENABLE)?\"1\":\"0\" **Exists:** Always
9	ARPOFFSEL	R	ARP Offload Enabled. This bit is set to 1 when the Enable IPv4 ARP Offload option is selected. **Value After Reset:** DWCXG_ARP_EN **Exists:** Always
8	MMCSEL	R	RMON Module Enable. This bit is set to 1 when the Enable 25GMAC Management Counter (MMC) option is selected. **Value After Reset:** DWCXG_MMC_EN **Exists:** Always

Table 11-68 Fields for Register: MAC_HW_Feature0 (continued)

Bits	Name	Memory Access	Description
7	MGKSEL	R	PMT Magic Packet Enable. This bit is set to 1 when the Enable Magic Packet Detection option is selected. **Value After Reset:** DWCXG_PMT_MGK_EN **Exists:** Always
6	RWKSEL	R	PMT Remote Wake-up Packet Enable. This bit is set to 1 when the Enable Remote Wake-Up Packet Detection option is selected. **Value After Reset:** DWCXG_PMT_RWK_EN **Exists:** Always
5	SMASEL	R	SMA (MDIO) Interface. This bit is set to 1 when the Enable Station Management Block (MDIO Interface) option is selected. **Value After Reset:** DWCXG_SMA_EN **Exists:** Always
4	VLHASH	R	VLAN Hash Filter Selected. This bit is set to 1 when the Enable Address Filter VLAN Hash Table option is selected. **Value After Reset:** DWCXG_VLAN_HASH_EN **Exists:** Always
3	HDSEL	R	Half-duplex Support This bit is set to 1 when the half-duplex mode is selected. **Values:** ■ 0x0 (INACTIVE): No Half-duplex support ■ 0x1 (ACTIVE): Half-duplex support **Value After Reset:** ((!DWCXG_FDUPLX_ONLY)?\"1\":\"0\") **Exists:** Always
2	RMIISEL	R	RMII Support. This bit is set to 1 when the RMII Interface option is selected. **Value After Reset:** DWCXG_RMII_EN **Exists:** Always
1	GMIISEL	R	1000/100/10 Mbps Support. This bit is set to 1 when the GMII Interface option is selected. **Value After Reset:** DWCXG_GMII_SUPPORT **Exists:** Always

Table 11-68 Fields for Register: MAC_HW_Feature0 (continued)

Bits	Name	Memory Access	Description
0	RGMIISEL	R	RGMII Support. This bit is set to 1 when the RGMII Interface option is selected. **Value After Reset:** DWCXG_RGMII_EN **Exists:** Always

11.1.65 MAC_HW_Feature1

- **Description:** This register indicates the presence of second set of the optional features or functions of DWC_25gmac. The software driver can use this register to dynamically enable or disable the programs related to the optional blocks.
- **Size:** 32 bits
- **Offset:** 0x120
- **Exists:** Always

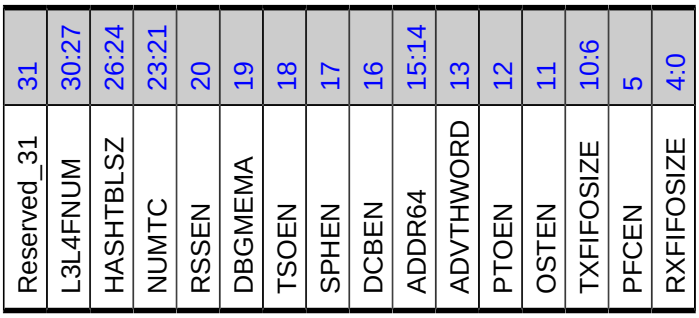


Table 11-69 Fields for Register: MAC_HW_Feature1

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:27	L3L4FNUM	R	Total number of L3 or L4 Filters. This field indicates the total number of L3 or L4 filters: ■ 4'b0000: No L3 or L4 Filter ■ 4'b0001: 1 L3 or L4 Filter ■ 4'b0010: 2 L3 or L4 Filters ■ ... ■ 4'b1000: 8 L3 or L4 ■ 4'b1001: 16 L3 or L4 ■ 4'b1010: 32 L3 or L4 **Value After Reset:** (DWCXG_L3_L4_FILTER_NUM<=8)?DWCXG_L3_L4_FILTER_NUM:(DWCXG_L3_L4_FILTER_NUM<=16)?9: (DWCXG_L3_L4_FILTER_NUM<=24)?10: (DWCXG_L3_L4_FILTER_NUM<=32)?11: (DWCXG_L3_L4_FILTER_NUM<=40)?12: (DWCXG_L3_L4_FILTER_NUM<=48)?13: ((DWCXG_L3_L4_FILTER_NUM<=64)?14:0) **Exists:** Always

Table 11-69 Fields for Register: MAC_HW_Feature1 (continued)

Bits	Name	Memory Access	Description
26:24	HASHTBLSZ	R	Hash Table Size. This field indicates the size of the hash table: ■ 3'b000: No hash table selected ■ 3'b001: 64 ■ 3'b010: 128 ■ 3'b011: 256 ■ 3'b100: 512 ■ 3'b101: 1024 ■ 3'b110: 2048 ■ 3'b111: 4096 **Value After Reset:** DWCXG_DA_HASH_EN?((log(DWCXG_HASH_TABLE)/log(2))-5):0 **Exists:** Always
23:21	NUMTC	R	Number of Traffic Classes. This field indicates the number of traffic classes selected: ■ 3'b000: 1 Traffic Class ■ 3'b001: 2 Traffic Classes ■ 3'b010: 3 Traffic Classes ■ ... ■ 3'b111: 8 Traffic **Value After Reset:** DWCXG_NUM_TC-1 **Exists:** Always
20	RSSEN	R	Internal Register based RSS Feature Enabled **Value After Reset:** DWCXG_RSS_EN **Exists:** Always
19	DBGMEMA	R	Debug Memory Interface Enabled. This bit is set to 1 when the Enable Debug Memory Access option is selected. **Value After Reset:** DWCXG_DBG_FIFO_ACCESS **Exists:** Always
18	TSOEN	R	TCP Segmentation Offload Enable. This bit is set to 1 when the Enable TCP Segmentation Offloading for TCP/IP Packets option is selected. **Value After Reset:** DWCXG_TSO_EN **Exists:** Always

Table 11-69 Fields for Register: MAC_HW_Feature1 (continued)

Bits	Name	Memory Access	Description
17	SPHEN	R	Header-Payload Split Feature Enable. This bit is set to 1 when the Enable Header-Payload Split Structure option is selected. **Value After Reset:** DWCXG_SPLIT_HDR_EN **Exists:** Always
16	DCBEN	R	DCB Feature Enable. This bit is set to 1 when the Enable Data Center Bridging option is selected. **Value After Reset:** DWCXG_DCB_EN **Exists:** Always
15:14	ADDR64	R	Address Width. This field indicates the configured address width: ■ 2'b00: 32 ■ 2'b01: 40 ■ 2'b10: 48 ■ 2'b11: Reserved **Value After Reset:** (DWCXG_ADDRWIDTH==48)?2: ((DWCXG_ADDRWIDTH==40)?1:0) **Exists:** Always
13	ADVTHWORD	R	IEEE 1588 High Word Register Enable. This bit is set to 1 when the Add IEEE 1588 Higher Word Register option is selected. **Value After Reset:** DWCXG_ADV_TIME_HIGH_WORD **Exists:** Always
12	PTOEN	R	PTP Offload Enable. This bit is set to 1 when the Enable PTP Timestamp Offload Feature is selected. **Value After Reset:** DWCXG_PTO_EN **Exists:** Always
11	OSTEN	R	One-Step Timestamping Enable. This bit is set to 1 when the Enable One-Step Timestamp Feature is selected. **Value After Reset:** DWCXG_OST_EN **Exists:** Always

Table 11-69 Fields for Register: MAC_HW_Feature1 (continued)

Bits	Name	Memory Access	Description
10:6	TXFIFOSIZE	R	MTL Transmit FIFO Size. This field contains the configured value of MTL Tx FIFO in bytes expressed as Log to base 2 minus 7, that is, Log2(MTL Transmit FIFO Size in bytes) - 7: ■ 4'b00011: 1024 bytes ■ 4'b00100: 2048 bytes ■ 4'b00101: 4096 bytes ■ 4'b00110: 8192 bytes ■ 4'b00111: 16384 bytes ■ 4'b01000: 32 KB ■ 4'b01001: 64 KB ■ 4'b01010: 128 KB ■ 4'b01011: 256 KB ■ 4'b01100-4'b11111: Reserved **Value After Reset:** DWCXG!=0?((log(DWCXG_TXFIFO_SIZE)/log(2))-7):0 **Exists:** Always
5	PFCEN	R	PFC Enable This bit is set to 1 when the Enable PFC Feature is selected. **Value After Reset:** DWCXG_PFC_EN **Exists:** Always
4:0	RXFIFOSIZE	R	MTL Receive FIFO Size. This field contains the configured value of MTL Rx FIFO in bytes expressed as Log to base 2 minus 7, that is, Log2(MTL Receive FIFO Size in bytes) - 7: ■ 4'b00011: 1024 bytes ■ 4'b00100: 2048 bytes ■ 4'b00101: 4096 bytes ■ 4'b00110: 8192 bytes ■ 4'b00111: 16384 bytes ■ 4'b01000: 32 KB ■ 4'b01001: 64 KB ■ 4'b01010: 128 KB ■ 4'b01011: 256 KB ■ 4'b01100-4'b11111: Reserved **Value After Reset:** DWCXG!=0?((log(DWCXG_RXFIFO_SIZE)/log(2))-7):0 **Exists:** Always

11.1.66 MAC_HW_Feature2

- **Description:** This register indicates the presence of the third set of optional features or functions of DWC_25gmac. The software driver can use this register to dynamically enable or disable the programs related to the optional blocks.
- **Size:** 32 bits
- **Offset:** 0x124
- **Exists:** Always

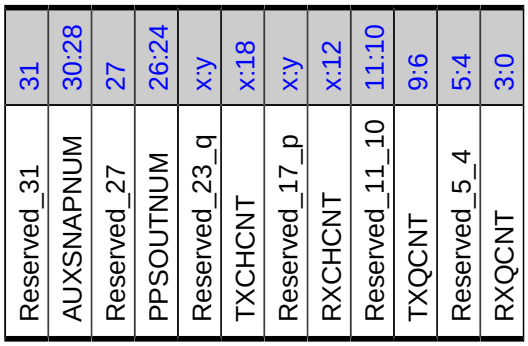


Table 11-70 Fields for Register: MAC_HW_Feature2

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:28	AUXSNAPNUM	R	Number of Auxiliary Snapshot Inputs. This field indicates the number of auxiliary snapshot inputs: ■ 3'b000: No auxiliary input ■ 3'b001: 1 auxiliary input ■ 3'b010: 2 auxiliary inputs ■ 3'b011: 3 auxiliary inputs ■ 3'b100: 4 auxiliary inputs ■ 3'b101 - 3'b111: Reserved **Value After Reset:** DWCXG_AUX_SNAP_IN_NUM **Exists:** Always
27	Reserved_27	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-70 Fields for Register: MAC_HW_Feature2 (continued)

Bits	Name	Memory Access	Description
26:24	PPSOUTNUM	R	Number of PPS Outputs. This field indicates the number of PPS outputs: ■ 3'b000: No PPS output ■ 3'b001: 1 PPS output ■ 3'b010: 2 PPS outputs ■ 3'b011: 3 PPS outputs ■ 3'b100: 4 PPS outputs ■ 3'b101 - 3'b111: Reserved **Value After Reset:** DWCXG_PPS_OUT_NUM **Exists:** Always
x:y	Reserved_23_q	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (SNPS_RSVDPARAM_50?1:2) + (SNPS_RSVDPARAM_50?23:22) - 1 **Range Variable[y]:** (SNPS_RSVDPARAM_50?23:22)
x:18	TXCHCNT	R	Number of DMA Transmit Channels. This field indicates the number of DMA Transmit channels: ■ 5'b00000: 1 DMA Tx Channel ■ 5'b00001: 2 DMA Tx Channels ■ ... ■ 5'b01111: 16 DMA Tx Channels ■ 5'b10000: 20 DMA Tx Channels ■ 5'b10001: 24 DMA Tx Channels ■ 5'b10010: 28 DMA Tx Channels ■ 5'b10011: 32 DMA Tx Channels ■ 5'b10100: 40 DMA Tx Channels ■ 5'b10101: 48 DMA Tx Channels ■ 5'b10110: 56 DMA Tx Channels ■ 5'b10111: 64 DMA Tx Channels ■ 5'b11000: 80 DMA Tx Channels ■ 5'b11001: 96 DMA Tx Channels ■ 5'b11010: 112 DMA Tx Channels ■ 5'b11011: 128 DMA Tx Channels ■ 5'b11100 to 5'b11111: Reserved **Value After Reset:** DWCXG_TXCHCNT **Exists:** Always **Range Variable[x]:** (SNPS_RSVDPARAM_50?5:4) + 17

Table 11-70 Fields for Register: MAC_HW_Feature2 (continued)

Bits	Name	Memory Access	Description
x:y	Reserved_17_p	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (SNPS_RSVDPARAM_50?1:2) + (SNPS_RSVDPARAM_50?17:16) - 1 **Range Variable[y]:** (SNPS_RSVDPARAM_50?17:16)
x:12	RXCHCNT	R	Number of DMA Receive Channels. This field indicates the number of DMA Receive channels: ■ 5'b00000: 1 DMA Rx Channel ■ 5'b00001: 2 DMA Rx Channels ■ ... ■ 5'b01111: 16 DMA Rx Channels ■ 5'b10000: 20 DMA Rx Channels ■ 5'b10001: 24 DMA Rx Channels ■ 5'b10010: 28 DMA Rx Channels ■ 5'b10011: 32 DMA Rx Channels ■ 5'b10100: 40 DMA Rx Channels ■ 5'b10101: 48 DMA Rx Channels ■ 5'b10110: 56 DMA Rx Channels ■ 5'b10111: 64 DMA Rx Channels ■ 5'b11000: 80 DMA Rx Channels ■ 5'b11001: 96 DMA Rx Channels ■ 5'b11010: 112 DMA Rx Channels ■ 5'b11011: 128 DMA Rx Channels ■ 5'b11100 to 5'b11111: Reserved **Value After Reset:** DWCXG_RXCHCNT **Exists:** Always **Range Variable[x]:** (SNPS_RSVDPARAM_50?5:4) + 11
11:10	Reserved_11_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
9:6	TXQCNT	R	Number of MTL Transmit Queues. This field indicates the number of MTL Transmit queues: ■ 4'b0000: 1 MTL Tx Queue ■ 4'b0001: 2 MTL Tx Queues ■ ... ■ 4'b1111: 16 MTL Tx Queues **Value After Reset:** DWCXG_NUM_TXQ-1 **Exists:** Always

Table 11-70 Fields for Register: MAC_HW_Feature2 (continued)

Bits	Name	Memory Access	Description
5:4	Reserved_5_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3:0	RXQCNT	R	Number of MTL Receive Queues. This field indicates the number of MTL Receive queues: ■ 4'b0000: 1 MTL Rx Queue ■ 4'b0001: 2 MTL Rx Queues ■ ... ■ 4'b1011: 12 MTL Rx Queues **Value After Reset:** DWCXG_NUM_RXQ-1 **Exists:** Always

11.1.67 MAC_HW_Feature3

- **Description:** This register indicates the presence of the fourth set of optional features or functions of DWC_25gmac. The software driver can use this register to dynamically enable or disable the programs related to the optional blocks.
- **Size:** 32 bits
- **Offset:** 0x128
- **Exists:** Always

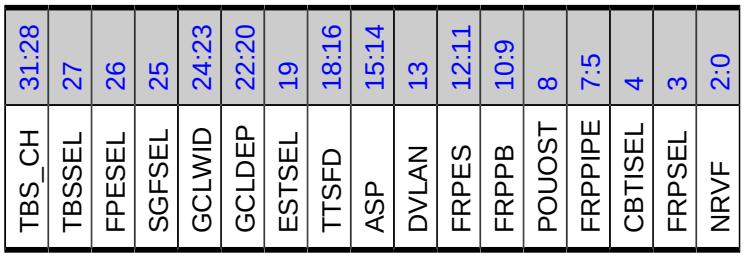


Table 11-71 Fields for Register: MAC_HW_Feature3

Bits	Name	Memory Access	Description
31:28	TBS_CH	R	The number of DMA channels enabled for TBS (starting from the highest Tx Channel, in descending order) This field provides the number of DMA channels enabled for TBS (starting from the highest Tx Channel, in descending order): ■ 0000: 1 DMA Tx Channel enabled for TBS ■ 0001: 2 DMA Tx Channels enabled for TBS ■ 0010: 3 DMA Tx Channels enabled for TBS ■ ... ■ 1111: 16 DMA Tx Channels enabled for TBS Based on the configuration, one or more bits of this field must be treated as Reserved/Read-Only. **Value After Reset:** (DWCXG_TBS)?(DWCXG_NUM_DMA_TBS_CH-1):0 **Exists:** Always
27	TBSSEL	R	Time Based Scheduling Enable This bit is set to 1 when the Time Based Scheduling feature is selected. **Values:** ■ 0x0 (INACTIVE): Time Based Scheduling Enable feature is not selected ■ 0x1 (ACTIVE): Time Based Scheduling Enable feature is selected **Value After Reset:** DWCXG_TBS **Exists:** Always

Table 11-71 Fields for Register: MAC_HW_Feature3 (continued)

Bits	Name	Memory Access	Description
26	FPESEL	R	Frame Preemption Enable This bit is set to 1 when the Enable Frame preemption feature is selected. **Values:** ■ 0x0 (INACTIVE): Frame Preemption Enable feature is not selected ■ 0x1 (ACTIVE): Frame Preemption Enable feature is selected **Value After Reset:** DWCXG_FPE **Exists:** Always
25	SGFSEL	R	Per-Stream Filtering Select This bit is set to 1 when the Per-Stream Filtering feature is selected. **Values:** ■ 0x0 (INACTIVE): Per Stream Filtering feature is not selected ■ 0x1 (ACTIVE): Per Stream Filtering feature is selected **Value After Reset:** DWCXG_SGF **Exists:** Always
24:23	GCLWID	R	Width of the Time Interval field in the Gate Control List This field indicates the width of the Configured Time Interval Field **Values:** ■ 0x0 (NOWIDTH): Width not configured ■ 0x1 (WIDTH16): 16 ■ 0x2 (WIDTH20): 20 ■ 0x3 (WIDTH24): 24 **Value After Reset:** (((DWCXG_AV_EST==0)&&(DWCXG_SGF==0))?\"0\":(DWCXG_GCL_WID==24)?\"3\":(DWCXG_GCL_WID==20)?\"2\":(DWCXG_GCL_WID==16)?\"1\":\"0\") **Exists:** Always

Table 11-71 Fields for Register: MAC_HW_Feature3 (continued)

Bits	Name	Memory Access	Description
22:20	GCLDEP	R	Depth of the Gate Control List This field indicates the depth of Gate Control list expressed as Gate Control List Depth selected x2 when Stream Gate Filtering is enabled and expressed as Log2(GCL Depth selected)-5 when Stream Gate Filtering is not enabled **Values:** ■ 0x0 (NODEPTH): No Depth configured ■ 0x1 (DEPTH64): 64 ■ 0x2 (DEPTH128): 128 ■ 0x3 (DEPTH256): 256 ■ 0x4 (DEPTH512): 512 ■ 0x5 (DEPTH1024): 1024 ■ 0x6 (RSVD): Reserved **Value After Reset:** (((DWCXG_AV_EST==0)&&(DWCXG_SGF==0))?\"0\":(DWCXG_GCL_DEP==1024)?\"5\":(DWCXG_GCL_DEP==512)?\"4\":(DWCXG_GCL_DEP==256)?\"3\":(DWCXG_GCL_DEP==128)?\"2\":(DWCXG_GCL_DEP==64)?\"1\":\"0\") **Exists:** Always
19	ESTSEL	R	Enhancements to Scheduling Traffic Enable This bit is set to 1 when the Enable Enhancements to Scheduling Traffic feature is selected. **Values:** ■ 0x0 (INACTIVE): Enable Enhancements to Scheduling Traffic feature is not selected ■ 0x1 (ACTIVE): Enable Enhancements to Scheduling Traffic feature is selected **Value After Reset:** DWCXG_AV_EST **Exists:** Always

Table 11-71 Fields for Register: MAC_HW_Feature3 (continued)

Bits	Name	Memory Access	Description
18:16	TTSFD	R	Tx Timestamp FIFO Depth This value indicates the depth of the Tx Timestamp FIFO ■ 3'b000: Reserved ■ 3'b001: 1 ■ 3'b010: 2 ■ 3'b011: 4 ■ 3'b100: 8 ■ 3'b101: 16 ■ 3'b110: Reserved ■ 3'b111: Reserved **Value After Reset:** (DWCXG_TIME_STAMPING&&!DWCXG_CORE)?((DWCXG_TTS_FIFO_DEPTH==1)?\"1\":(DWCXG_TTS_FIFO_DEPTH==2)?\"2\":(DWCXG_TTS_FIFO_DEPTH==4)?\"3\":(DWCXG_TTS_FIFO_DEPTH==8)?\"4\":(DWCXG_TTS_FIFO_DEPTH==16)?\"5\":\"0\"):\"0\" **Exists:** Always
15:14	ASP	R	Automotive Safety Package This field along with EASP field of MAC_HW_Feature4 register indicates the encoded value for the different Safety features: {EASP,ASP}: Description ■ 3'b000: No Safety features selected ■ 3'b001: All the Automotive Safety features are selected with ECC and Parity Ports on External Application Interface for Data and Control respectively ■ 3'b010: All the Automotive Safety features are selected without ECC or Parity Ports on External Application Interface ■ 3'b011: All the Automotive Safety features are selected with the Parity Ports on External Application Interface ■ 3'b100: Only ECC feature selected **Value After Reset:** ((DWCXG_ASP_ALL&&DWCXG_ASP_PPE==1)?\"3\":(DWCXG_ASP_ALL&&DWCXG_ASP_PPE==0)?\"2\":(DWCXG_ASP_ALL&&DWCXG_ASP_PPE==2)?\"1\":\"0\") **Exists:** Always
13	DVLAN	R	Double VLAN Processing Enabled This bit is set to 1 when the Enable Double VLAN Processing feature is selected. **Value After Reset:** DWCXG_DOUBLE_VLAN_EN **Exists:** Always

Table 11-71 Fields for Register: MAC_HW_Feature3 (continued)

Bits	Name	Memory Access	Description
12:11	FRPES	R	Supported Flexible Receive Parser Instructions This field indicates the Max Number of Parser Instructions supported by Flexible Receive Parser. **Values:** ■ 0x0 (M_64ENTR): 64 Instructions ■ 0x1 (M_128ENTR): 128 Instructions ■ 0x2 (M_256ENTR): 256 Instructions ■ 0x3 (RSVD): Reserved **Value After Reset:** DWCXG_FRP_EN?((DWCXG_FRP_ENTRIES==512)?\"3\":(DWCXG_FRP_ENTRIES==256)?\"2\":(DWCXG_FRP_ENTRIES==128)?\"1\":\"0\"):\"0\" **Exists:** Always
10:9	FRPPB	R	Supported Flexible Receive Parser Parsable Bytes This field indicates the supported Max Number of bytes of the packet data to be Parsed by Flexible Receive Parser. **Values:** ■ 0x0 (M_64BYTES): 64 Bytes ■ 0x1 (M_128BYTES): 128 Bytes ■ 0x2 (M_256BYTES): 256 Bytes ■ 0x3 (RSVD): Reserved **Value After Reset:** DWCXG_FRP_EN?((DWCXG_FRP_PB==256)?\"2\":(DWCXG_FRP_PB==128)?\"1\":\"0\"):\"0\" **Exists:** Always
8	POUOST	R	One Step for PTP over UDP/IP Feature Enable This bit is set to 1 when the Enable One step timestamp for PTP over UDP/IP feature is selected. **Value After Reset:** DWCXG_POU_OST_EN **Exists:** Always
7:5	FRPPIPE	R	Supported Parallel Instruction Processor Engines (PIPEs) This field indicates the maximum number of Instruction Processors supported by flexible receive parser. Values: ■ 0x0: 1 PIPE ■ 0x1: 2 PIPEs ■ ... ■ 0x15: 16 PIPEs **Value After Reset:** DWCXG_FRP_EN?DWCXG_FRP_PIPE:\"0\" **Exists:** Always

Table 11-71 Fields for Register: MAC_HW_Feature3 (continued)

Bits	Name	Memory Access	Description
4	CBTISEL	R	Queue/Channel based VLAN tag insertion on Tx Enable This bit is set to 1 when the Enable Queue/Channel based VLAN tag insertion on Tx Feature is selected. **Value After Reset:** DWCXG_CBTI_EN **Exists:** Always
3	FRPSEL	R	Supported Flexible Receive Parser. This bit is set to 1 when the Enable Flexible Programmable Receive Parser option is selected. **Values:** ■ 0x0 (INACTIVE): Flexible Receive Parser feature is not selected ■ 0x1 (ACTIVE): Flexible Receive Parser feature is selected **Value After Reset:** (DWCXG_FRP_EN)?\"1\":\"0\" **Exists:** Always
2:0	NRVF	R	Enabled number of Extended VLAN Tag Filters or External VLAN tag lookup size This field indicates selected Number of Extended VLAN Tag Filters or External VLAN tag lookup size: ■ 3'b000: No Extended Rx VLAN Filters ■ 3'b001: 4 Extended Rx VLAN Filters ■ 3'b010: 8 Extended Rx VLAN Filters ■ 3'b011: 16 Extended Rx VLAN Filters ■ 3'b100: 24 Extended Rx VLAN Filters ■ 3'b101: 32 Extended Rx VLAN Filters ■ 3'b110: Reserved ■ 3'b111: External Receive 12bit VLAN Tag Lookup is selected **Value After Reset:** (DWCXG_NRVF==4096)?\"7\":((DWCXG_NRVF>32)&&(DWCXG_NRVF<=64))?\"6\":(DWCXG_NRVF==32)?\"5\":(DWCXG_NRVF==24)?\"4\":(DWCXG_NRVF==16)?\"3\":(DWCXG_NRVF==8)?\"2\":(DWCXG_NRVF==4)?\"1\":\"0\" **Exists:** Always

11.1.68 MAC_HW_Feature4

- **Description:** This register indicates the presence of the fifth set of optional features or functions of DWC_25gmac. The software driver can use this register to dynamically enable or disable the programs related to the optional blocks.
- **Size:** 32 bits
- **Offset:** 0x12c
- **Exists:** Always

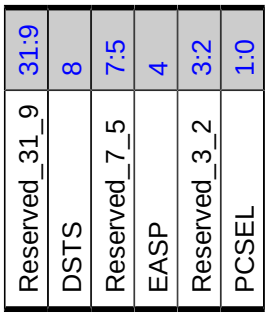


Table 11-72 Fields for Register: MAC_HW_Feature4

Bits	Name	Memory Access	Description
31:9	Reserved_31_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	DSTS	R	Dual Time Source Domain Capability This field indicates that the Dual Time Source Domain Capability is enabled **Value After Reset:** (DWCXG_DSTS)?\"1\":\"0\" **Exists:** Always
7:5	Reserved_7_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	EASP	R	Extended Automotive Safety Package This field along with ASP field of MAC_HW_Feature3 register indicates the encoded value for the different Safety features **Value After Reset:** ((!DWCXG_ASP_ALL)&&DWCXG_ASP_ECC)?\"1\":\"0\" **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-72 Fields for Register: MAC_HW_Feature4 (continued)

Bits	Name	Memory Access	Description
1:0	PCSEL	R	Policing Counters This value indicates the number of policing counters implemented when PC is enabled **Values:** ■ 0x0 (INACTIVE): Policing counter feature is not selected ■ 0x1 (PCNUM_16): 16 Policing counters are selected ■ 0x2 (PCNUM_32): 32 Policing counters are selected **Value After Reset:** (DWCXG_PC)?((DWCXG_NUM_PC==32)?\"2\":(DWCXG_NUM_PC==16)?\"1\":\"0\"):\"0\" **Exists:** Always

11.1.69 MAC_Extended_Configuration

- **Description:** The MAC Extended Configuration register establishes the operating mode of the MAC transmitter.
- **Size:** 32 bits
- **Offset:** 0x140
- **Exists:** Always

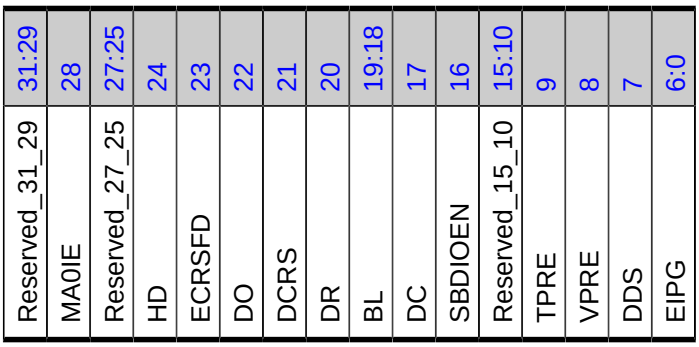


Table 11-73 Fields for Register: MAC_Extended_Configuration

Bits	Name	Memory Access	Description
31:29	Reserved_31_29	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
28	MA0IE	R/W	MAC Address 0 Input Enable. When this bit is set, the following sideband input signal is selected over the corresponding ADDRLO and ADDRHI fields of MAC_Address0_Low and MAC_Address0_High registers, respectively. ■ sbd_macaddr0_i When this bit is reset, the above sideband input signal is ignored. **Value After Reset:** (DWCXG_MA0IE_DFLT) **Exists:** SNPS_RSVDPARAM_15
27:25	Reserved_27_25	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-73 Fields for Register: MAC_Extended_Configuration (continued)

Bits	Name	Memory Access	Description
24	HD	* Varies	Half-Duplex Mode When this bit is set, the MAC operates in the half-duplex mode in which it can transmit and receive simultaneously. This bit is RO with default value of 1'b0 in the full-duplex-only configurations. This bit is enabled only for 10/100Mbps. **Values:** ■ 0x0 (FDUPLX): Full-duplex mode ■ 0x1 (HDUPLX): Half-duplex mode **Value After Reset:** 0x0 **Exists:** Always **Memory Access:** ((DWCXG_FDUPLX_ONLY)?\"read-only\":\"read-write\")
23	ECRSFD	R/W	Enable Carrier Sense before Transmission in Full-Duplex mode When this bit is set, the MAC transmitter checks the CRS signal before packet transmission in the full-duplex mode. The MAC starts the transmission only when the CRS signal is low. When this bit is reset, the MAC transmitter ignores the status of the CRS signal. **Values:** ■ 0x0 (DISABLE): ECRSFD is disabled ■ 0x1 (ENABLE): ECRSFD is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_XGMII_GMII == 1
22	DO	R/W	Disable Receive Own When this bit is set, the MAC disables the reception of packets when the GMII/MII Transmit Data Enable signal (gmii_txen_o) is asserted in the half-duplex mode. When this bit is reset, the MAC receives all packets given by the PHY. This bit is not applicable in the full-duplex mode. **Values:** ■ 0x0 (ENABLE): Enable Receive Own ■ 0x1 (DISABLE): Disable Receive Own **Value After Reset:** 0x0 **Exists:** !(DWCXG_FDUPLX_ONLY)

Table 11-73 Fields for Register: MAC_Extended_Configuration (continued)

Bits	Name	Memory Access	Description
21	DCRS	R/W	Disable Carrier Sense During Transmission When this bit is set, the MAC transmitter ignores the MII CRS signal during packet transmission in the half-duplex mode. As a result, no errors are generated because of Loss of Carrier or No Carrier during transmission. When this bit is reset, the MAC transmitter generates errors because of Carrier Sense. The MAC can even terminate the transmission. **Values:** ■ 0x0 (ENABLE): Enable Carrier Sense During Transmission ■ 0x1 (DISABLE): Disable Carrier Sense During Transmission **Value After Reset:** 0x0 **Exists:** !(DWCXG_FDUPLX_ONLY)
20	DR	R/W	Disable Retry When this bit is set, the MAC attempts only one transmission. When a collision occurs on the GMII or MII interface, the MAC ignores the current packet transmission and reports a Packet Terminate with excessive collision error in the Tx packet status. When this bit is reset, the MAC retries based on the settings of the BL field. This bit is applicable only in the half-duplex mode. **Values:** ■ 0x0 (ENABLE): Enable Retry ■ 0x1 (DISABLE): Disable Retry **Value After Reset:** 0x0 **Exists:** !(DWCXG_FDUPLX_ONLY)
19:18	BL	R/W	Back-Off Limit The back-off limit determines the random integer number (r) of slot time delays (512 bit times for 10/100 Mbps) for which the MAC waits before rescheduling a transmission attempt during retries after a collision. n = retransmission attempt. The random integer r takes the value in the range 0 to 2^k This bit is applicable only in the half-duplex mode. **Values:** ■ 0x0 (MIN_N_10): k = min(n,10) ■ 0x1 (MIN_N_8): k = min(n,8) ■ 0x2 (MIN_N_4): k = min(n,4) ■ 0x3 (MIN_N_1): k = min(n,1) **Value After Reset:** 0x0 **Exists:** !(DWCXG_FDUPLX_ONLY)

Table 11-73 Fields for Register: MAC_Extended_Configuration (continued)

Bits	Name	Memory Access	Description
17	DC	R/W	Deferral Check When this bit is set, the deferral check function is enabled in the MAC. The MAC issues a Packet Terminate status, along with the excessive deferral error bit set in the Tx packet status, when the Tx state machine is deferred for more than 24,288 bit times in 10 or 100 Mbps mode. Deferral begins when the transmitter is ready to transmit, but it is prevented because of an active carrier sense signal (CRS) on MII. The defer time is not cumulative. For example, if the transmitter defers for 10,000 bit times, because the CRS signal is active and the CRS signal becomes inactive, the transmitter transmits and collision occurs. Because of collision, the transmitter needs to back off and then defer again after back off is complete. In such a scenario, the deferral timer is reset to 0, and it is restarted. When this bit is reset, the deferral check function is disabled and the MAC defers until the CRS signal goes inactive. This bit is applicable only in the half-duplex mode. **Values:** ■ 0x0 (DISABLE): Deferral check function is disabled ■ 0x1 (ENABLE): Deferral check function is enabled **Value After Reset:** 0x0 **Exists:** !(DWCXG_FDUPLX_ONLY)
16	SBDIOEN	R/W	Sideband Input and Output Enable. When this bit is set, the following sideband input signals are selected over the corresponding register fields. ■ sbd_lpitxen_i ■ sbd_link_sts_i ■ sbd_speed_sel_i When this bit is reset, the sideband input signals are ignored. **Value After Reset:** 0x0 **Exists:** DWCXG_ADD_SBD_IO
15:10	Reserved_15_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-73 Fields for Register: MAC_Extended_Configuration (continued)

Bits	Name	Memory Access	Description
9	TPRE	R/W	Truncated Preamble Enable. When set, MAC supports processing of packets with 7 bytes preamble on RX side such that SFD is seen on lane2. On reset, packet with truncated preamble are dropped. This field has an impact only if the speed select is set to 3'b110 (XGMII 2.5G mode) TPRE = 1 For express packets, the supported format is ■ /S/ + Preamble (6/5)+ SFD (lane3/lane2) For preemptive packets, truncated preamble is not supported. Therefore, the supported format is ■ /S/ + Preamble (6)+ SFD (lane3) **Value After Reset:** 0x0 **Exists:** Always
8	VPRE	R/W	Variable Preamble Enable. When set, MAC supports processing of packets with variable preamble on RX side. On reset, packet with variable preamble are dropped. VPRE = 1 For express packets, the supported format is ■ /S/ + Preamble (n) + SFD, where n = 0, 1, 2, 3, ... For preemptive packets ■ /S/ + Preamble ((4*n)-2) + SMDS (lane3) ■ /S/ + Preamble ((4*n)-3) + SMDC (lane2) + FC (lane3) format. where n = 1, 2, 3, ... **Value After Reset:** 0x1 **Exists:** DWCXG_VARIABLE_PREAMBLE
7	DDS	R/W	DA Duplication Select. ■ 1: The received packet with Multicast/Broadcast (and Unicast when UDC bit of MAC_RxQ_Ctrl4 register is set) Destination address is routed to multiple Receive DMA Channels. The Receive DMA Channels is identified by XDCS field of MAC_DChSel_IndReg(#i) register corresponding to the MAC Address register that matches the Multicast/Broadcast (and Unicast when UDC bit of MAC_RxQ_Ctrl4 register is set) DA. The XDCS field is interpreted to be a one-hot value, each bit corresponding to the Receive DMA Channel. ■ 0: The received packet is routed to a single Receive DMA Channel. The Receive DMA Channel is identified by the DCS field of MAC_Address_High register corresponding

Table 11-73 Fields for Register: MAC_Extended_Configuration (continued)

Bits	Name	Memory Access	Description
			to the MAC Address register that matches the Destination address in the received packet. The DCS field is interpreted as a binary value. Values: ■ 0x0: DCS field in MAC_Address_High register is interpreted as binary value ■ 0x1: XDCS field in MAC_DChSel_IndReg register represent the target DMA Channels using a one-hot like encoding. **Value After Reset:** 0x0 **Exists:** (DWCXG_DDUP&&!SNPS_RSVDPARAM_50)
6:0	EIPG	R/W	Extended Inter-Packet Gap. When IFP bit in MAC_Tx_configuration register is set, the minimum IPG is increased in steps of 32 bits, depending on the programming of this field and IPG field in MAC_Tx_Configuration register. {EIPG,IPG}: Minimum Inter-Packet Gap ■ 10'h000: 96 bit times ■ 10'h001: 128 bit times ■ 10'h002: 160 bit times ■ 10'h003: 192 bit times ■ 10'h004: 224 bit times ■ ... ■ 10'h3FF: 32832 bit times **Note:** The extended inter-packet gap (IPG) feature must be enabled only in full-duplex mode. There might be undesirable effects on back-pressure function and frame transmission if it is enabled in half-duplex mode. **Value After Reset:** 0x0 **Exists:** Always

11.1.70 MAC_Ext_Cfg1

- **Description:** This register contains Split mode control field and offset field for Header-Payload Split feature.
- **Size:** 32 bits
- **Offset:** 0x144
- **Exists:** (DWCXG_SPLIT_HDR_EN)

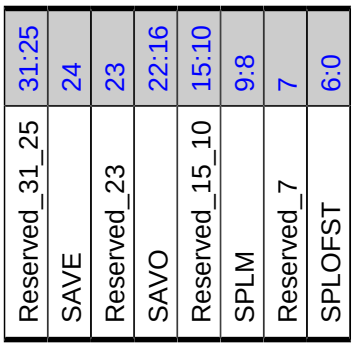


Table 11-74 Fields for Register: MAC_Ext_Cfg1

Bits	Name	Memory Access	Description
31:25	Reserved_31_25	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
24	SAVE	R/W	Split AV Enable ■ When this bit is set to 1, and the received packet is an AV Type packet, the header is split at SAVO bytes from the beginning of Length/Type field of the packet, for L2 Split. ■ When this bit is set to 0, header is split at SPLOFST bytes from the beginning of Length/Type field of the frame, for L2 Split. **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_ENABLE)
23	Reserved_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-74 Fields for Register: MAC_Ext_Cfg1 (continued)

Bits	Name	Memory Access	Description
22:16	SAVO	R/W	Split AV Offset When SAVE bit is set to 1, and the received packet is an AV Type packet, these bits indicate the value of the offset from the beginning of Length/Type field at which header must be split, when appropriate SPLM is selected. The reset value of this field is 2 bytes, indicating a split at L2 header. Value is in terms of bytes. **Value After Reset:** 0x2 **Exists:** (DWCXG_AV_ENABLE)
15:10	Reserved_15_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
9:8	SPLM	R/W	Split Mode These bits control the mode of splitting the incoming Rx packets. They are **Values:** ■ 0x0 (L3L4): Split at L3/L4 header ■ 0x1 (L2OFST): Split at L2 header with an offset. Always Split at SPLOFST bytes from the beginning of Length/Type field of the Frame ■ 0x2 (COMBN): Combination mode: Split similar to SPLM=00 for IP packets that are untagged or tagged and VLAN stripped and Split similar to SPLM=01 for non-IP packets ■ 0x3 (RSVD): Reserved **Value After Reset:** 0x0 **Exists:** Always
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:0	SPLOFST	R/W	Split Offset These bits indicate the value of offset from the beginning of Length/Type field at which header split must take place when the appropriate SPLM is selected. The reset value of this field is 2 bytes indicating a split at L2 header. Value is in terms of bytes. **Value After Reset:** 0x2 **Exists:** Always

11.1.71 MAC_DPP_FSM_Interrupt_Status

- **Description:** This register contains the status of Automotive Safety related Data Path Parity Errors, Interface Timeout Errors, FSM State Parity Errors, and FSM State Timeout Errors.
- **Size:** 32 bits
- **Offset:** 0x150
- **Exists:** (DWCXG_ASP_DPP||DWCXG_ASP_FSM||DWCXG_ASP_CSR)

31	30:26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CPI	Reserved_30_26	FSMPES	ARITES	ATITES	SLVTES	MSTTES	Reserved_20	Reserved_19	Reserved_18	PTES	ATES	CTES	RTES	TTES	ASRPES	CWPES	MRRCPES	MRWCPES	ARPES	MTFCPES	MTBUPES	MTSPES	MTPES	TSOPES	TPES	DPES	ATPES

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status

Bits	Name	Memory Access	Description
31	CPI	R/W	Control Register Parity Check Error When this bit is set, it indicates a parity check error for CSR registers. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30:26	Reserved_30_26	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
25	FSMPES	R/W	FSM State Parity Error Status Indicates that parity error is detected in one of the FSM State registers. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Subordinate Read/Write Timeout Error Status not detected ■ 0x1 (ACTIVE): Subordinate Read/Write Timeout Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL **Testable:** untestable
24	ARITES	R/W	ARI Interface Timeout Error Status When this bit is set, it indicates that an Application timeout has occurred on ARI Interface. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Timeout not detected on ARI Interface ■ 0x1 (ACTIVE): Timeout detected on ARI Interface **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG_AXI_SLAVE) **Testable:** untestable
23	ATITES	R/W	ATI Interface Timeout Error Status When this bit is set, it indicates that an Application timeout has occurred on ATI Interface. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Timeout not detected on ATI Interface ■ 0x1 (ACTIVE): Timeout detected on ATI Interface **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG_AXI_SLAVE) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
22	SLVTES	R/W	Subordinate Read/Write Timeout Error Status When this bit is set, it indicates that an Application/CSR timeout has occurred on the AXI subordinate interface. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Subordinate Read/Write Timeout Error Status not detected ■ 0x1 (ACTIVE): Subordinate Read/Write Timeout Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG_AXI_SLAVE) **Testable:** untestable
21	MSTTES	R/W	Manager Read/Write Timeout Error Status When this bit is set, it indicates that an Application/CSR timeout has occurred on the manager (AXI) interface. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Manager Read/Write Timeout Error Status not detected ■ 0x1 (ACTIVE): Manager Read/Write Timeout Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG_AXI_SUBSYS||DWCXG_MTL_SUBSYS) **Testable:** untestable
20	Reserved_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19	Reserved_19	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
18	Reserved_18	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
17	PTES	R/W	PTP FSM Timeout Error Status When this bit is set, it indicates that one of the PTP FSM timeout has occurred. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): PTP FSM Timeout Error Status not detected ■ 0x1 (ACTIVE): PTP FSM Timeout Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG_AV_EST||DWCXG_FLEXI_PPS_OUT_EN) **Testable:** untestable
16	ATES	R/W	APP FSM Timeout Error Status When this bit is set, it indicates that one of the APP FSM timeout has occurred. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): APP FSM Timeout Error Status not detected ■ 0x1 (ACTIVE): APP FSM Timeout Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(((DWCXG>0||((DWCXG_EEE_EN||DWCXG_SMA_EN)&&!(DWCXG_CORE||DWCXG_CSR_SLV_CLK))))||(((DWCXG_AXI_SUBSYS)&&!DWCXG_CSR_SLV_CLK))) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
15	CTES	R/W	CSR FSM Timeout Error Status When this bit is set, it indicates that one of the CSR FSM timeout has occurred. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): CSR FSM Timeout Error Status not detected ■ 0x1 (ACTIVE): CSR FSM Timeout Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(((((DWCXG_EEE_EN||DWCXG_SMA_EN)&&(DWCXG_CORE||DWCXG_CSR_SLV_CLK))||((DWCXG_AXI_SLAVE)&&DWCXG_CSR_SLV_CLK)))||((DWCXG_AXI_SLAVE)&&DWCXG_CSR_SLV_CLK)) **Testable:** untestable
14	RTES	R/W	Rx FSM Timeout Error Status When this bit is set, it indicates that one of the Rx FSM timeout has occurred. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Rx FSM timeout has not been detected ■ 0x1 (ACTIVE): Rx FSM timeout has been detected **Value After Reset:** 0x0 **Exists:** ((DWCXG_ASP_ALL)) **Testable:** untestable
13	TTES	R/W	Tx FSM Timeout Error Status When this bit is set, it indicates that one of the Tx FSM timeout has occurred. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Tx FSM timeout has not been detected ■ 0x1 (ACTIVE): Tx FSM timeout has been detected **Value After Reset:** 0x0 **Exists:** ((DWCXG_ASP_ALL)) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
12	ASRPES	R/W	AXI Subordinate Read data path Parity checker Error Status When this bit is set, it indicates that parity error is detected at the AXI Subordinate read data interface on "rdata_s_o" or at PC9 checker. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): AXI Subordinate Read data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): AXI Subordinate Read data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)) **Testable:** untestable
11	CWPES	R/W	CSR Write data path Parity checker Error Status When this bit is set, it indicates that parity error is detected at the CSR write data interface on mci_wdata_i (or at PC8 checker). This bit is set to 1 when EPSI bit of MTL_DPP_Control register is set and a parity mismatch is detected on the input subordinate parity ports (or at PC-axi/apb/mci subordinate checker). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): CSR Write data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): CSR Write data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(((DWCXG_ASP_PPE==1)||((DWCXG_ASP_PPE==2)&&(DWCXG_APB_SLAVE||DWCXG_MCI_SLAVE)))||DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
10	MRRCPES	R/W	MTL RRC data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected at the MTL RRC output data parity checker (or at PC7). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MTL RRC data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): MTL RRC data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>1)) **Testable:** untestable
9	MRWCPES	R/W	MTL RWC data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected at the MTL RWC input data parity checker (or at PC6). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MTL RWC data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): MTL RWC data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>0)) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
8	ARPES	R/W	Application Receive interface data path Parity Error Status When this bit is set, it indicates that a parity error is detected at the following checkers based on the system configuration: ■ In MTL configuration (DWCXG = 1), parity checker (PC-b) at ARI interface. ■ In AXI configuration (DWCXG = 2), parity checker (PC-a) at AXI manager interface. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Application Receive interface data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): Application Receive interface data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>0)) **Testable:** untestable
7	MTFCPES	R/W	MAC TFC data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected on the MAC TFC (or at PC5). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MAC TFC data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): MAC TFC data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_DPP&&(DWCXG>=0) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
6	MTBUPES	R/W	MAC TBU data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected on the MAC TBU (or at PC4). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MAC TBU data path Parity Error Status not detected ■ 0x1 (ACTIVE): MAC TBU data path Parity Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_DPP&&(DWCXG>=0) **Testable:** untestable
5	MTSPES	R/W	MTL Tx Status data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected on the MTL Tx Status data on ATI interface (or at PC3). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MTL TX Status data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): MTL TX Status data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>0)&&DWCXG_TXSTSE) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
4	MTPES	R/W	MTL data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected at the MTL transmit write controller parity checker (or at PC2). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MTL data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): MTL data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>0)) **Testable:** untestable
3	TSOPES	R/W	TSO header (before replacement) data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected at the DMA TSO parity checker (or at PC1-hdr). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): DMA TSO Header (before replacement) data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): DMA TSO Header (before replacement) data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&DWCXG_TSO_EN&&DWCXG>1) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
2	TPES	R/W	TSO or Descriptor Cache write data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected at the DMA TSO parity checker (or at PC-hdr). When this bit is set, it indicates that a parity error is detected at the DMA TSO or Descriptor Cache parity checker (or at PC-hdr/PC-desc). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): DMA TSO Header or Descriptor Cache (shared same memory with TSO) write data path Parity checker Error Status not detected ■ 0x1 (ACTIVE): DMA TSO Header or Descriptor Cache (shared same memory with TSO) write data path Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&DWCXG_TSO_MEM_EN&&DWCXG>1) **Testable:** untestable
1	DPES	R/W	Descriptor Cache write data path Parity checker Error Status When this bit is set, it indicates that a parity error is detected at the Descriptor Cache parity checker (or at PC-desc). Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&DWCXG_DCACHE_MEM_EN&&DWCXG>1) **Testable:** untestable

Table 11-75 Fields for Register: MAC_DPP_FSM_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
0	ATPES	R/W	Application Transmit Interface Parity checker Error Status When set, this field indicates that a parity error is detected on the AXI Manager read data parity checker or MTL ATI data parity checker. Following are the locations of the checkers based on the system configuration, ■ In MTL configuration (DWCXG = 1), parity checker (PC-e) at ATI interface. ■ In AXI configuration (DWCXG = 2), parity checker (PC-e) at AXI manager interface. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Application Transmit Interface Parity checker Error Status not detected ■ 0x1 (ACTIVE): Application Transmit Interface Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG_ASP_PPE==1)&&(DWCXG>0)) **Testable:** untestable

11.1.72 MAC_AXI_SLV_DPE_Addr_Status

- **Description:** This register gives the address of the Register Write transaction in which parity error occurred.
- **Size:** 32 bits
- **Offset:** 0x154
- **Exists:** (DWCXG_ASP_DPP&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE))

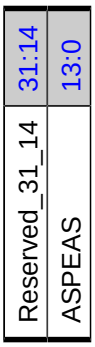


Table 11-76 Fields for Register: MAC_AXI_SLV_DPE_Addr_Status

Bits	Name	Memory Access	Description
31:14	Reserved_31_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:0	ASPEAS	R	AXI Subordinate data path Parity Error Address Status This field holds the CSR address for which parity error is detected on the CSR write data. This field holds the first address for which parity error is detected on the write data which is cleared on read. **Value After Reset:** 0x0 **Exists:** Always

11.1.73 MAC_FSM_Control

- **Description:** This register is used to control the FSM State parity and timeout error injection in Debug mode.
- **Size:** 32 bits
- **Offset:** 0x158
- **Exists:** (DWCXG_ASP_FSM||DWCXG_ASP_CSR||(DWCXG_RXP&&DWCXG_ASP_ECC))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7:2	1	0
Reserved_31	Reserved_30	Reserved_29	PLGRNML	ALGRNML	CLGRNML	RLGRNML	TLGRNML	Reserved_23	Reserved_22	Reserved_21	PPEIN	APEIN	CPEIN	RPEIN	TPEIN	Reserved_15	Reserved_14	Reserved_13	PTEIN	ATEIN	CTEIN	RTEIN	TTEIN	Reserved_7_2	PRTYEN	TMOUTEN

Table 11-77 Fields for Register: MAC_FSM_Control

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30	Reserved_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29	Reserved_29	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
28	PLGRNML	R/W	PTP Large/Normal Mode Select When this bit is set, large mode tic generation is used for PTP domain; otherwise, normal mode tic generation is used. **Values:** ■ 0x0 (DISABLE): normal mode tic generation is used for PTP domain ■ 0x1 (ENABLE): large mode tic generation is used for PTP domain **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG_AV_EST||DWCXG_FLEXI_PPS_OUT_EN||DWCXG_PSF)

Table 11-77 Fields for Register: MAC_FSM_Control (continued)

Bits	Name	Memory Access	Description
27	ALGRNML	R/W	APP Large/Normal Mode Select When this bit is set, large mode tic generation is used for APP domain; otherwise, normal mode tic generation is used. **Values:** ■ 0x0 (DISABLE): normal mode tic generation is used for APP domain ■ 0x1 (ENABLE): large mode tic generation is used for APP domain **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_FSM&&(DWCXG>0||((DWCXG_EEE_EN||DWCXG_SMA_EN)&&!(DWCXG_CORE||DWCXG_CSR_SLV_CLK))))
26	CLGRNML	R/W	CSR Large/Normal Mode Select When this bit is set, large mode tic generation is used for CSR domain; otherwise, normal mode tic generation is used. **Values:** ■ 0x0 (DISABLE): normal mode tic generation is used for CSR domain ■ 0x1 (ENABLE): large mode tic generation is used for CSR domain **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_FSM&&(((DWCXG_EEE_EN||DWCXG_SMA_EN)&&(DWCXG_CORE||DWCXG_CSR_SLV_CLK))||((DWCXG_AXI_SLAVE||DWCXG_ASP_CSR)&&DWCXG_CSR_SLV_CLK)))
25	RLGRNML	R/W	Rx Large/Normal Mode Select When this bit is set, large mode tic generation is used for Rx domain; otherwise, normal mode tic generation is used. **Values:** ■ 0x0 (DISABLE): normal mode tic generation is used for Rx domain ■ 0x1 (ENABLE): large mode tic generation is used for Rx domain **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL

Table 11-77 Fields for Register: MAC_FSM_Control (continued)

Bits	Name	Memory Access	Description
24	TLGRNML	R/W	Tx Large/Normal Mode Select When this bit is set, large mode tic generation is used for Tx domain; otherwise, normal mode tic generation is used. **Values:** ■ 0x0 (DISABLE): normal mode tic generation is used for Tx domain ■ 0x1 (ENABLE): large mode tic generation is used for Tx domain **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL
23	Reserved_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
22	Reserved_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
21	Reserved_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
20	PPEIN	R/W	PTP FSM Parity Error Injection When this bit is set, Error Injection for PTP FSM Parity is enabled. **Values:** ■ 0x0 (DISABLE): PTP FSM Parity Error Injection is disabled ■ 0x1 (ENABLE): PTP FSM Parity Error Injection is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG_AV_EST||DWCXG_FLEXI_PPS_OUT_EN||DWCXG_PSF)

Table 11-77 Fields for Register: MAC_FSM_Control (continued)

Bits	Name	Memory Access	Description
19	APEIN	R/W	APP FSM Parity Error Injection When this bit is set, Error Injection for APP FSM Parity is enabled. **Values:** ■ 0x0 (DISABLE): APP FSM Parity Error Injection is disabled ■ 0x1 (ENABLE): APP FSM Parity Error Injection is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG>0||((DWCXG_EEE_EN||DWCXG_SMA_EN)&&!(DWCXG_CORE||DWCXG_CSR_SLV_CLK)))
18	CPEIN	R/W	CSR FSM Parity Error Injection When this bit is set, Error Injection for CSR Parity is enabled. **Values:** ■ 0x0 (DISABLE): CSR FSM Parity Error Injection is disabled ■ 0x1 (ENABLE): CSR FSM Parity Error Injection is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(((DWCXG_EEE_EN||DWCXG_SMA_EN)&&(DWCXG_CORE||DWCXG_CSR_SLV_CLK))||((DWCXG_AXI_SLAVE||DWCXG_ASP_CSR)&&DWCXG_CSR_SLV_CLK))
17	RPEIN	R/W	Rx FSM Parity Error Injection When this bit is set, Error Injection for Rx FSM Parity is enabled. **Values:** ■ 0x0 (DISABLE): Rx FSM Parity Error Injection is disabled ■ 0x1 (ENABLE): Rx FSM Parity Error Injection is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL
16	TPEIN	R/W	Tx FSM Parity Error Injection When this bit is set, Error Injection for Tx FSM Parity is enabled. **Values:** ■ 0x0 (DISABLE): Tx FSM Parity Error Injection is disabled ■ 0x1 (ENABLE): Tx FSM Parity Error Injection is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL

Table 11-77 Fields for Register: MAC_FSM_Control (continued)

Bits	Name	Memory Access	Description
15	Reserved_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
14	Reserved_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13	Reserved_13	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
12	PTEIN	R/W	PTP FSM Timeout Error Injection When this bit is set, Error Injection for PTP FSM timeout is enabled. **Values:** ■ 0x0 (DISABLE): PTP FSM Timeout Error Injection is disabled ■ 0x1 (ENABLE): PTP FSM Timeout Error Injection is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM&&(DWCXG_AV_EST||DWCXG_FLEXI_PPS_OUT_EN||DWCXG_PSF)
11	ATEIN	R/W	APP FSM Timeout Error Injection When this bit is set, Error Injection for APP FSM timeout is enabled. **Values:** ■ 0x0 (DISABLE): APP FSM Timeout Error Injection is disabled ■ 0x1 (ENABLE): APP FSM Timeout Error Injection is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_FSM&&(DWCXG>0||((DWCXG_EEE_EN||DWCXG_SMA_EN)&&!(DWCXG_CORE||DWCXG_CSR_SLV_CLK))))

Table 11-77 Fields for Register: MAC_FSM_Control (continued)

Bits	Name	Memory Access	Description
10	CTEIN	R/W	CSR FSM Timeout Error Injection When this bit is set, Error Injection for CSR timeout is enabled. **Values:** ■ 0x0 (DISABLE): CSR FSM Timeout Error Injection is disabled ■ 0x1 (ENABLE): CSR FSM Timeout Error Injection is enabled **Value After Reset:** 0x0 **Exists:** (((DWCXG_EEE_EN||DWCXG_SMA_EN)&&(DWCXG_CORE||DWCXG_CSR_SLV_CLK))||((DWCXG_AXI_SLAVE||DWCXG_ASP_CSR)&&DWCXG_CSR_SLV_CLK))&&DWCXG_ASP_FSM
9	RTEIN	R/W	Rx FSM Timeout Error Injection When this bit is set, Error Injection for Rx FSM timeout is enabled. **Values:** ■ 0x0 (DISABLE): Rx FSM Timeout Error Injection is disabled ■ 0x1 (ENABLE): Rx FSM Timeout Error Injection is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL
8	TTEIN	R/W	Tx FSM Timeout Error Injection When this bit is set, Error Injection for Tx FSM timeout is enabled. **Values:** ■ 0x0 (DISABLE): Tx FSM Timeout Error Injection is disabled ■ 0x1 (ENABLE): Tx FSM Timeout Error Injection is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL
7:2	Reserved_7_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-77 Fields for Register: MAC_FSM_Control (continued)

Bits	Name	Memory Access	Description
1	PRTYEN	R/W	FSM Parity Checking Enable When this bit is set, FSM parity feature is enabled. **Values:** ■ 0x0 (DISABLE): FSM Parity feature is disabled ■ 0x1 (ENABLE): FSM Parity feature is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL
0	TMOUTEN	R/W	FSM Timeout Checking Enable When this bit is set, FSM timeout feature is enabled. **Values:** ■ 0x0 (DISABLE): FSM timeout feature is disabled ■ 0x1 (ENABLE): FSM timeout feature is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_FSM||DWCXG_ASP_CSR||(DWCXG_RXP&& DWCXG_ASP_ECC)

11.1.74 MAC_FSM_ACT_Timer

- **Description:** This register is used to select the FSM and Interface Timeout values.
- **Size:** 32 bits
- **Offset:** 0x15c
- **Exists:** (DWCXG_ASP_FSM||DWCXG_ASP_CSR||(DWCXG_RXP&&DWCXG_ASP_ECC))

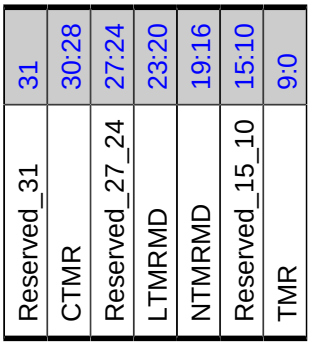


Table 11-78 Fields for Register: MAC_FSM_ACT_Timer

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:28	CTMR	R/W	CSR Parity Or Rx Parser Instruction Table ECC Check Interval Value This register is used for setting timer value for generating timer tic for CSR parity or Rx Parser Instruction Table ECC check. **Values:** ■ 0x0 (M_1MILLI_SEC): 1ms ■ 0x1 (M_1MICRO_SEC_CSR): 1us ■ 0x2 (M_4MILLI_SEC_CSR): 4.096ms (~4ms) ■ 0x3 (M_16MILLI_SEC_CSR): 16.384ms (~16ms) ■ 0x4 (M_64MILLI_SEC_CSR): 65.536ms (~64ms) ■ 0x5 (M_256MILLI_SEC_CSR): 262.144ms (~256ms) ■ 0x6 (M_1SEC_CSR): 1.048sec (~1sec) ■ 0x7 (ALWAYS_EN): Always Enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_CSR||(DWCXG_RXP&&DWCXG_ASP_ECC)
27:24	Reserved_27_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-78 Fields for Register: MAC_FSM_ACT_Timer (continued)

Bits	Name	Memory Access	Description
23:20	LTMRMD	R/W	Large Mode Timeout Value This field provides the mode value to be used for large mode FSM and other interface time outs. The timeout duration based on the mode value is as follows: **Values:** ■ 0x0 (DISABLE): Timer disabled ■ 0x1 (M_1MICRO_SEC): 1us ■ 0x2 (M_4MILLI_SEC): 1.024ms (~4ms) ■ 0x3 (M_16MILLI_SEC): 16.384ms (~16ms) ■ 0x4 (M_64MILLI_SEC): 65.536ms (~64ms) ■ 0x5 (M_256MILLI_SEC): 262.144ms (~256ms) ■ 0x6 (M_1SEC): 1.048sec (~1sec) ■ 0x7 (M_4SEC): 4.194sec (~4sec) ■ 0x8 (M_16SEC): 16.777sec (~16sec) ■ 0x9 (M_32SEC): 33.554sec (~32sec) ■ 0xa (M_64SEC): 67.108sec (~64sec) ■ 0xb (RSVD): Reserved **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_FSM||DWCXG_ASP_CSR)
19:16	NTMRMD	R/W	Normal Mode Timeout Value This field provides the value to be used for normal mode FSM and other interface time outs. The timeout duration based on the mode value is as follows: **Values:** ■ 0x0 (DISABLE): Timer disabled ■ 0x1 (M_1MICRO_SEC): 1us ■ 0x2 (M_4MILLI_SEC): 1.024ms (~4ms) ■ 0x3 (M_16MILLI_SEC): 16.384ms (~16ms) ■ 0x4 (M_64MILLI_SEC): 65.536ms (~64ms) ■ 0x5 (M_256MILLI_SEC): 262.144ms (~256ms) ■ 0x6 (M_1SEC): 1.048sec (~1sec) ■ 0x7 (M_4SEC): 4.194sec (~4sec) ■ 0x8 (M_16SEC): 16.777sec (~16sec) ■ 0x9 (M_32SEC): 33.554sec (~32sec) ■ 0xa (M_64SEC): 67.108sec (~64sec) ■ 0xb (RSVD): Reserved **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_FSM||DWCXG_ASP_CSR)
15:10	Reserved_15_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-78 Fields for Register: MAC_FSM_ACT_Timer (continued)

Bits	Name	Memory Access	Description
9:0	TMR	R/W	CSR Clocks for 1us Tic This field indicates the number of CSR clocks required to generate 1us tic. **Value After Reset:** 0x0 **Exists:** Always

11.1.75 SNPS_SCS_REG1

- **Description:** Synopsys Reserved Register
- **Size:** 32 bits
- **Offset:** 0x160
- **Exists:** DWCXG_ASP_ALL

Table 11-79 Fields for Register: SNPS_SCS_REG1

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** DWCXG<2 **Range Variable[x]:** DWCXG>0?8:16 + DWCXG>0?24:16 - 1 **Range Variable[y]:** DWCXG>0?24:16
x:0	MAC_SCS1	R/W	Synopsys Reserved. All the bits must be set to 0. This field is reserved for Synopsys Internal use, and must always be set to 0 unless instructed by Synopsys. Setting any bit to 1 might cause unexpected behavior in the IP. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG>1?32:DWCXG>0?24:16 - 1

11.1.76 MAC_SCSR_Control

- **Description:** This register is used to control safety protected CSR parity generation.
- **Size:** 32 bits
- **Offset:** 0x164
- **Exists:** ((DWCXG_ASP_ALL))

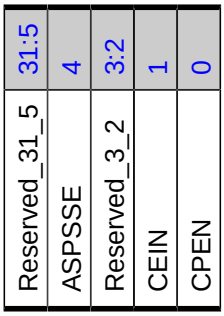


Table 11-80 Fields for Register: MAC_SCSR_Control

Bits	Name	Memory Access	Description
31:5	Reserved_31_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	ASPSSE	R/W	ASP Safety Subordinate Enable. When set, enables the Safety Subordinate interface which is used connect all safety errors from IP to a centralized safety manager block. **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_43
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	CEIN	R/W	CSR Error Injection When this bit is set, CSR error injection is enabled. Error is injected in the subsequent write operation where at the least one of the byte enable matches with byte enable indicated in CEBE. After error injection, this bit is cleared by the design. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-80 Fields for Register: MAC_SCSR_Control (continued)

Bits	Name	Memory Access	Description
0	CPEN	R/W	CSR Parity Enable When this bit is set, CSR parity checking is enabled. **Value After Reset:** 0x0 **Exists:** Always

11.1.77 MAC_SCSR_Parity_Status

- **Description:** This register indicates the offset address of the register where parity_error occurred.
- **Size:** 32 bits
- **Offset:** 0x168
- **Exists:** ((DWCXG_ASP_ALL))

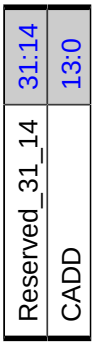


Table 11-81 Fields for Register: MAC_SCSR_Parity_Status

Bits	Name	Memory Access	Description
31:14	Reserved_31_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:0	CADD	R	CSR Address with error This field indicates the offset address of last CSR register for which read data parity mismatched with reference parity. For example, when parity mismatch occurred when MAC_Rx_Configuration register is read, this field will contain value of 16'h0004. **Value After Reset:** 0x0 **Exists:** Always

11.1.78 MAC_SCSR_Parity_Err_Count

- **Description:** This register indicates error count associated with parity check.
- **Size:** 32 bits
- **Offset:** 0x16c
- **Exists:** ((DWCXG_ASP_ALL))

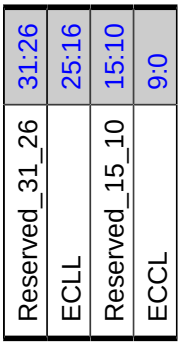


Table 11-82 Fields for Register: MAC_SCSR_Parity_Err_Count

Bits	Name	Memory Access	Description
31:26	Reserved_31_26	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
25:16	ECLL	R	Count of parity errors in the previous safety loop This field indicates the Count of parity errors in previous loop of all safety register parity check. Any application read related errors are also included in the count. Value saturates at all "1"s and is cleared on read. **Value After Reset:** 0x0 **Exists:** Always
15:10	Reserved_15_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
9:0	ECCL	R	Count of parity errors in current safety loop This field indicates the Count of parity errors in the current loop of all safety register parity check. Application read related errors are also included in the count. Value saturates at all "1"s and is cleared on read. **Value After Reset:** 0x0 **Exists:** Always

11.1.79 MAC_Flex_Hdr_Cfg

- **Description:** This register contains flexible header enable, length and start position for Flexible Header Feature
- **Size:** 32 bits
- **Offset:** 0x194
- **Exists:** (([<functionof> ((SNPS_RSVDPARAM_52| |SNPS_RSVDPARAM_53) ? 1 : 0)]))

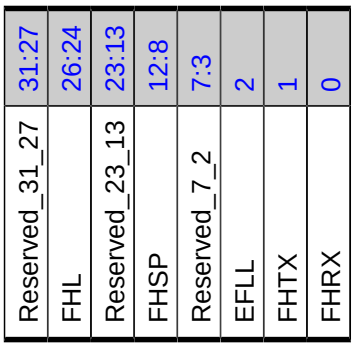


Table 11-83 Fields for Register: MAC_Flex_Hdr_Cfg

Bits	Name	Memory Access	Description
31:27	Reserved_31_27	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
26:24	FHL	R/W	Flexible Header Length When FHRX/FHTX bit is set, all the packets coming on the line in the Rx path and all the packets transmitted from the application on the Tx path must have Flexible Header of the following size, in bytes: ■ 3'b000 - 3'b111: 1 Byte to 8 Bytes **Value After Reset:** 0x3 **Exists:** SNPS_RSVDPARAM_52
23:13	Reserved_23_13	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-83 Fields for Register: MAC_Flex_Hdr_Cfg (continued)

Bits	Name	Memory Access	Description
12:8	FHSP	R/W	Flexible Header Start Position When FHRX/FHTX bit is set, FH is present in Tx/Rx packets at one of the following encoded positions ■ 5'b00000: Before DA field (Byte 0) ■ 5'b00001 - 5'b01011: RSVD ■ 5'b01100: After SA field (Byte 12) ■ 5'b01101: Reserved ■ 5'b01110 - 5'11101: Byte 14 to Byte 29 **Value After Reset:** 0xc **Exists:** SNPS_RSVDPARAM_52
7:3	Reserved_7_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	EFLL	R/W	Insert Flexible Header for Pause/PFC frames ■ When this bit is set along with FHTX, static flexible header of the programmed length is inserted in the MAC-generated pause/PFC frames at the programmed position ■ When this bit is set along with FHRX, in Rx path, pause/PFC frame must have FH programmed in the SFH (static flexible header) field. ■ When this bit is reset, pause/PFC frames do not have flexible header inserted in them, in both Tx and Rx paths. **Value After Reset:** 0x0 **Exists:** Always
1	FHTX	R/W	Flexible Header Enable for Tx path When this is set, all the packets from the Software is expected to have FHL bytes Flexible Header in the Tx path. When set to 0, normal/standard Ethernet packets are expected to be received and transmitted. This bit programming must be not changed when there are packets present in the TX path. **Value After Reset:** 0x0 **Exists:** Always

Table 11-83 Fields for Register: MAC_Flex_Hdr_Cfg (continued)

Bits	Name	Memory Access	Description
0	FHRX	R/W	Flexible Header Enable for Rx path When this is set, all the incoming packets from the Line is expected to have FHL bytes Flexible Header in the Rx path. When set to 0, normal/standard Ethernet packets are expected to be received and transmitted. This bit programming must be not changed when there are packets present in the RX path. **Value After Reset:** 0x0 **Exists:** Always

11.1.80 MAC_Flex_Hdr_Low

- **Description:** This register contains programmable Lower Static Flexible Header for Mac-gen pause/PFC/hardware offload engines generated packets (such as ARP/PTP) Flexible Header Feature
- **Size:** 32 bits
- **Offset:** 0x198
- **Exists:** (SNPS_RSVDPARAM_53| |SNPS_RSVDPARAM_52)

Table 11-84 Fields for Register: MAC_Flex_Hdr_Low

Bits	Name	Memory Access	Description
31:0	SFHL	R/W	Static Flexible Header - Lower When Flexible header feature is enabled in the Tx path, all the MAC generated Tx packets such as ARP/PTP packets and Link Layer packets (such as pause/PFC and verify-response if EFLL is set) have FH inserted from this programmable field, at the programmed FH position. This field must be used for FH length of up to 4 bytes **Value After Reset:** 0x0 **Exists:** Always

11.1.81 MAC_Flex_Hdr_High

- **Description:** This register contains programmable Upper Static Flexible Header for Mac-gen (pause/PFC)/hardware offload engines generated packets (such as ARP/PTP) Flexible Header Feature
- **Size:** 32 bits
- **Offset:** 0x19c
- **Exists:** (SNPS_RSVDPARAM_52)

Table 11-85 Fields for Register: MAC_Flex_Hdr_High

Bits	Name	Memory Access	Description
31:0	SFHU	R/W	Static Flexible Header - Upper When Flexible Header feature is enabled: This field is used for programming FH's upper 4 bytes when FH length is from 5 bytes to 8 bytes for Link Layer/ MAC auto-generated packets **Value After Reset:** 0x0 **Exists:** Always

11.1.82 MDIO_Single_Command_Address

- **Description:** This register contains the addresses of the ports, devices, and registers accessed by the SMA during single read or write transfers.
- **Size:** 32 bits
- **Offset:** 0x200
- **Exists:** (DWCXG_SMA_EN)

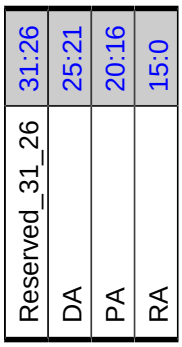


Table 11-86 Fields for Register: MDIO_Single_Command_Address

Bits	Name	Memory Access	Description
31:26	Reserved_31_26	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
25:21	DA	R/W	Device Address Device Address for indirect addressing of device for single read/write. **Value After Reset:** 0x0 **Exists:** Always
20:16	PA	R/W	Port Address Port Address used for indirect single read/write access. **Value After Reset:** 0x0 **Exists:** Always
15:0	RA	R/W	Register Address Register Address for single read/write. When using Clause 22 frames, only the lower 5 bits of the field are used. **Value After Reset:** 0x0 **Exists:** Always

11.1.83 MDIO_Single_Command_Control_Data

- **Description:** This register controls single read/write operations, and contains the data bits read on a single-read operation and the data to be written during single-write operations.
- **Size:** 32 bits
- **Offset:** 0x204
- **Exists:** (DWCXG_SMA_EN)

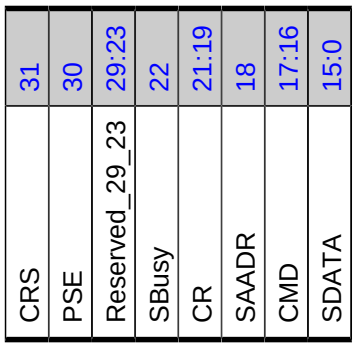


Table 11-87 Fields for Register: MDIO_Single_Command_Control_Data

Bits	Name	Memory Access	Description
31	CRS	R/W	Clock Range Select. When this set overrides the default MDIO clock generation based on CSR clock. When this bit is set MDIO Clock generation using CSR clock such that MDIO frequency is equals to CSR clock divided by 4, 6, 8, 10, 12, 14, 16, 18. Therefore, this mode enables faster clock on MDIO. **Value After Reset:** 0x0 **Exists:** Always
30	PSE	R/W	Preamble Suppression Enable. When this bit is set, the SMA suppresses the 32-bit preamble and transmits MDIO packets with only 1 preamble bit. When this bit is 0, the MDIO packet always has 32 bits of preamble as defined in the IEEE specifications. **Value After Reset:** 0x0 **Exists:** Always
29:23	Reserved_29_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-87 Fields for Register: MDIO_Single_Command_Control_Data (continued)

Bits	Name	Memory Access	Description
22	SBusy	R/W	Busy When you set this bit, it triggers a single read, single write, or post-increment read operation. DWC_25gmac clears this bit when the triggered operation is completed. During the MDIO transfer cycle, this bit remains high to indicate the cycle in progress. When the Busy bit is high, do not modify any bits in SMA registers 1 or 2. When this bit is set high, bits[15:0] of MDIO_Single_Command_Control_Data must be kept valid for MDIO write access or bits[15:0] of MDIO_Single_Command_Control_Data are invalid (for MDIO read access). **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
21:19	CR	R/W	Application Clock Range The Application Clock Range selection determines the relationship between the CSR and MDC clocks. Do not change the value of CR bits unless there is no operation (continuous write or scan) in progress on the SMA interface. When CRS bit is set to 0: ■ 3'b000 ❑ clk_csr_i: 100-150 MHz ❑ MDC clock: clk_csr_i/62 ■ 3'b001 ❑ clk_csr_i: 150-250 MHz ❑ MDC clock: clk_csr_i/102 ■ 3'b010 ❑ clk_csr_i: 250-300 MHz ❑ MDC clock: clk_csr_i/122 ■ 3'b011 ❑ clk_csr_i: 300-350 MHz ❑ MDC clock: clk_csr_i/142 ■ 3'b100 ❑ clk_csr_i: 350-400 MHz ❑ MDC clock: clk_csr_i/162 ■ 3'b101 ❑ clk_csr_i: 400-500 MHz ❑ MDC clock: clk_csr_i/202 ■ 3'b110

Table 11-87 Fields for Register: MDIO_Single_Command_Control_Data (continued)

Bits	Name	Memory Access	Description
			❑ clk_csr_i: Reserved ❑ MDC clock: Reserved ■ 3'b111 ❑ clk_csr_i: Reserved ❑ MDC clock: Reserved When CRS bit is set to 1, MDIO clock is derived based on CSR clock as follows: ■ 3'b000: MDC clock: clk_csr_i/4 ■ 3'b001: MDC clock: clk_csr_i/6 ■ 3'b010: MDC clock: clk_csr_i/8 ■ 3'b011: MDC clock: clk_csr_i/10 ■ 3'b100: MDC clock: clk_csr_i/12 ■ 3'b101: MDC clock: clk_csr_i/14 ■ 3'b110: MDC clock: clk_csr_i/16 ■ 3'b111: MDC clock: clk_csr_i/18 **Value After Reset:** 0x0 **Exists:** Always
18	SAADR	R/W	Skip Address Frame The application sets this bit to prevent the SMA from sending address frames before read, write, or post-increment read frames. **Value After Reset:** 0x0 **Exists:** Always
17:16	CMD	R/W	Control Command SMA commands are written to these bits. The command codes are: ■ 2'b11: SINGLE_READ ■ 2'b01: SINGLE_WRITE ■ 2'b10: POST INCREMENT READ OPERATION ■ 2'b00: Reserved (Results are undefined) **Value After Reset:** 0x0 **Exists:** Always
15:0	SDATA	R/W	Single Write Data For single writes, this field contains data to be written. For single reads, this field contains data that has been read. **Value After Reset:** 0x0 **Exists:** Always

11.1.84 MDIO_Continuous_Write_Address

- **Description:** You can program this register to start continuous write operation.
- **Size:** 32 bits
- **Offset:** 0x208
- **Exists:** (DWCXG_SMA_EN)

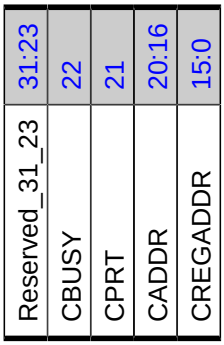


Table 11-88 Fields for Register: MDIO_Continuous_Write_Address

Bits	Name	Memory Access	Description
31:23	Reserved_31_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
22	CBUSY	R/W	Continuous Write Busy When this bit is set, continuous write operation starts. When continuous write operation is complete, this bit is reset. This bit must read as 0 before the next continuous write command is issued. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
21	CPRT	R/W	Continuous Port or Device Address Select When this bit is set, the CADDR is used as the port address and continuous writes are performed on all port devices. When this bit is unset, CADDR is used as the device address and continuous writes are performed on all ports. **Value After Reset:** 0x0 **Exists:** Always

Table 11-88 Fields for Register: MDIO_Continuous_Write_Address (continued)

Bits	Name	Memory Access	Description
20:16	CADDR	R/W	Continuous Port or Device Address Common address to which continuous writes are applied. Used either as a device address or a port address, depending on the CPRT setting. As a port address, the valid range is 0-3 only. **Value After Reset:** 0x0 **Exists:** Always
15:0	CREGADDR	R/W	Continuous Register Address Register address to which continuous writes are performed. For Clause 22 ports, only the five least significant bits are valid. **Value After Reset:** 0x0 **Exists:** Always

11.1.85 MDIO_Continuous_Write_Data

- **Description:** This register contains the data to be written during continuous write operation. Do not change the register's contents until the Continuous Write Busy bit (CBUSY) is cleared.
- **Size:** 32 bits
- **Offset:** 0x20c
- **Exists:** (DWCXG_SMA_EN)

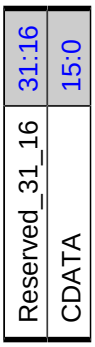


Table 11-89 Fields for Register: MDIO_Continuous_Write_Data

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:0	CDATA	R/W	Continuous Write Data Data to be used in the MDIO frame during a continuous write. Do not modify the register's contents unless the Continuous Write Busy bit (CBUSY) is cleared. **Value After Reset:** 0x0 **Exists:** Always

11.1.86 MDIO_Continuous_Scan_Port_Enable

- **Description:** This register controls the PHY ports and corresponding devices (enabled by the corresponding Device In Use register) to be accessed during the SMA's continuous scan operation. If you change any bit in this register during a continuous scan, it takes effect at the end of the current scan read cycle. In other words, if Bit 2 (Port 2) is enabled while Port 3, 0, or 1 is being scanned, port 2 scanning is scheduled at the end of the Port 1 scan. If any bit is cleared by software, it takes effect at the end of the reading the last register of last enabled device.
- **Size:** 32 bits
- **Offset:** 0x210
- **Exists:** (DWCXG_SMA_EN)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PORT31SCE	PORT30SCE	PORT29SCE	PORT28SCE	PORT27SCE	PORT26SCE	PORT25SCE	PORT24SCE	PORT23SCE	PORT22SCE	PORT21SCE	PORT20SCE	PORT19SCE	PORT18SCE	PORT17SCE	PORT16SCE	PORT15SCE	PORT14SCE	PORT13SCE	PORT12SCE	PORT11SCE	PORT10SCE	PORT9SCE	PORT8SCE	PORT7SCE	PORT6SCE	PORT5SCE	PORT4SCE	PORT3SCE	PORT2SCE	PORT1SCE	PORT0SCE

Table 11-90 Fields for Register: MDIO_Continuous_Scan_Port_Enable

Bits	Name	Memory Access	Description
31	PORT31SCE	R/W	Port 31 Continuous Scan Enable. This bit indicates that Port 31 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30	PORT30SCE	R/W	Port 30 Continuous Scan Enable. This bit indicates that Port 30 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
29	PORT29SCE	R/W	Port 29 Continuous Scan Enable. This bit indicates that Port 29 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-90 Fields for Register: MDIO_Continuous_Scan_Port_Enable (continued)

Bits	Name	Memory Access	Description
28	PORT28SCE	R/W	Port 28 Continuous Scan Enable. This bit indicates that Port 28 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
27	PORT27SCE	R/W	Port 27 Continuous Scan Enable. This bit indicates that Port 27 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
26	PORT26SCE	R/W	Port 26 Continuous Scan Enable. This bit indicates that Port 26 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
25	PORT25SCE	R/W	Port 25 Continuous Scan Enable. This bit indicates that Port 25 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
24	PORT24SCE	R/W	Port 24 Continuous Scan Enable. This bit indicates that Port 24 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-90 Fields for Register: MDIO_Continuous_Scan_Port_Enable (continued)

Bits	Name	Memory Access	Description
23	PORT23SCE	R/W	Port 23 Continuous Scan Enable. This bit indicates that Port 23 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
22	PORT22SCE	R/W	Port 22 Continuous Scan Enable. This bit indicates that Port 22 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
21	PORT21SCE	R/W	Port 21 Continuous Scan Enable. This bit indicates that Port 21 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
20	PORT20SCE	R/W	Port 20 Continuous Scan Enable. This bit indicates that Port 20 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
19	PORT19SCE	R/W	Port 19 Continuous Scan Enable. This bit indicates that Port 19 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-90 Fields for Register: MDIO_Continuous_Scan_Port_Enable (continued)

Bits	Name	Memory Access	Description
18	PORT18SCE	R/W	Port 18 Continuous Scan Enable. This bit indicates that Port 18 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
17	PORT17SCE	R/W	Port 17 Continuous Scan Enable. This bit indicates that Port 17 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
16	PORT16SCE	R/W	Port 16 Continuous Scan Enable. This bit indicates that Port 16 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15	PORT15SCE	R/W	Port 15 Continuous Scan Enable. This bit indicates that Port 15 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14	PORT14SCE	R/W	Port 14 Continuous Scan Enable. This bit indicates that Port 14 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-90 Fields for Register: MDIO_Continuous_Scan_Port_Enable (continued)

Bits	Name	Memory Access	Description
13	PORT13SCE	R/W	Port 13 Continuous Scan Enable. This bit indicates that Port 13 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
12	PORT12SCE	R/W	Port 12 Continuous Scan Enable. This bit indicates that Port 12 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
11	PORT11SCE	R/W	Port 11 Continuous Scan Enable. This bit indicates that Port 11 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
10	PORT10SCE	R/W	Port 10 Continuous Scan Enable. This bit indicates that Port 10 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
9	PORT9SCE	R/W	Port 9 Continuous Scan Enable. This bit indicates that Port 9 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-90 Fields for Register: MDIO_Continuous_Scan_Port_Enable (continued)

Bits	Name	Memory Access	Description
8	PORT8SCE	R/W	Port 8 Continuous Scan Enable. This bit indicates that Port 8 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
7	PORT7SCE	R/W	Port 7 Continuous Scan Enable. This bit indicates that Port 7 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
6	PORT6SCE	R/W	Port 6 Continuous Scan Enable. This bit indicates that Port 6 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
5	PORT5SCE	R/W	Port 5 Continuous Scan Enable. This bit indicates that Port 5 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
4	PORT4SCE	R/W	Port 4 Continuous Scan Enable. This bit indicates that Port 4 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-90 Fields for Register: MDIO_Continuous_Scan_Port_Enable (continued)

Bits	Name	Memory Access	Description
3	PORT3SCE	R/W	Port 3 Continuous Scan Enable. This bit indicates that Port 3 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
2	PORT2SCE	R/W	Port 2 Continuous Scan Enable. This bit indicates that Port 2 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	PORT1SCE	R/W	Port 1 Continuous Scan Enable. This bit indicates that Port 1 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	PORT0SCE	R/W	Port 0 Continuous Scan Enable. This bit indicates that Port 0 is enabled for continuous scan. Software can disable the port for continuous scan by setting this bit to 0. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.87 MDIO_Interrupt_Status

- **Description:** This register gives the source of the interrupt raised by the SMA module on the sbd_intr_o output line. The status bits are cleared when the software reads the corresponding bytes.
- **Size:** 32 bits
- **Offset:** 0x214
- **Exists:** (DWCXG_SMA_EN)

31:14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved_31_14	CWCOMPINT	SNGLCOMPINT	PORTNx4P3ALINT	PORTNx4P2ALINT	PORTNx4P1ALINT	PORTNx4P0ALINT	PORTNx4P3LSINT	PORTNx4P2LSINT	PORTNx4P1LSINT	PORTNx4P0LSINT	PORTNx4P3CONINT	PORTNx4P2CONINT	PORTNx4P1CONINT	PORTNx4P0CONINT

Table 11-91 Fields for Register: MDIO_Interrupt_Status

Bits	Name	Memory Access	Description
31:14	Reserved_31_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13	CWCOMPINT	R	Continuous Write Completion Interrupt. Set when a continuous write completes. **Note:** This bit is valid only when PRS field of MDIO Port Nx4 Indirect Control register is 0. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always
12	SNGLCOMPINT	R	Single Command Completion Interrupt. Set when a single command completes. **Note:** This bit is available only when PRS field of MDIO Port Nx4 Indirect Control register is 0. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always

Table 11-91 Fields for Register: MDIO_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
11	PORTNx4P3ALINT	R	Device Present Status Change Interrupt (Port Nx4 Plus3). When set, indicates that an interrupt occurred due to a change in one or more of the Port Nx4 Plus3 device's Alive status. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-1 (or when byte-1 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always
10	PORTNx4P2ALINT	R	Device Present Status Change Interrupt (Port Nx4 Plus2). When set, indicates that an interrupt occurred due to a change in one or more of the Port Nx4 Plus2 device's Alive status. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-1 (or when byte-1 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always
9	PORTNx4P1ALINT	R	Device Present Status Change Interrupt (Port Nx4 Plus1). When set, indicates that an interrupt occurred due to a change in one or more of the Port Nx4 Plus1 device's Alive status. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-1 (or when byte-1 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always

Table 11-91 Fields for Register: MDIO_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
8	PORTNx4P0ALINT	R	Device Present Status Change Interrupt (Port Nx4 Plus0). When set, indicates that an interrupt occurred due to a change in one or more of the Port Nx4 Plus0 device's Alive status. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-1 (or when byte-1 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always
7	PORTNx4P3LSINT	R	Link Status Change Interrupt (Port Nx4 Plus3). When set, indicates that the link status changed on at least one of the Port Nx4 Plus3 devices. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-0 (or when byte-0 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always
6	PORTNx4P2LSINT	R	Link Status Change Interrupt (Port Nx4 Plus2). When set, indicates that the link status changed on at least one of the Port Nx4 Plus2 devices. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-0 (or when byte-0 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always
5	PORTNx4P1LSINT	R	Link Status Change Interrupt (Port Nx4 Plus1). When set, indicates that the link status changed on at least one of the Port Nx4 Plus1 devices. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-0 (or when byte-0 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always

Table 11-91 Fields for Register: MDIO_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
4	PORTNx4P0LSINT	R	Link Status Change Interrupt (Port Nx4 Plus0). When set, indicates that the link status changed on at least one of the Port Nx4 Plus0 devices. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-0 (or when byte-0 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always
3	PORTNx4P3CONINT	R	Connect/Disconnect Event Interrupt (Port Nx4 Plus3). When set, indicates that Port Nx4 Plus3 caused an interrupt due to a connection or disconnection. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-0 (or when byte-0 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always
2	PORTNx4P2CONINT	R	Connect/Disconnect Event Interrupt (Port Nx4 Plus2). When set, indicates that Port Nx4 Plus2 caused an interrupt due to a connection or disconnection. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-0 (or when byte-0 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always
1	PORTNx4P1CONINT	R	Connect/Disconnect Event Interrupt (Port Nx4 Plus1). When set, indicates that Port Nx4 Plus1 caused an interrupt due to a connection or disconnection. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-0 (or when byte-0 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always

Table 11-91 Fields for Register: MDIO_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
0	PORTNx4P0CONINT	R	Connect/Disconnect Event Interrupt (Port Nx4 Plus0). When set, indicates that Port Nx4 Plus0 caused an interrupt due to a connection or disconnection. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register Access restriction applies. Clears on reading byte-0 (or when byte-0 is written when RCWE bit in MAC_CSR_SW_Ctrl register is set) **Value After Reset:** 0x0 **Exists:** Always

11.1.88 MDIO_Interrupt_Enable

- **Description:** This register controls the enabling of interrupt sources in the MDIO Interrupt Status register.
- **Size:** 32 bits
- **Offset:** 0x218
- **Exists:** (DWCXG_SMA_EN)

31:14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved_31_14	CWCOMPIE	SNGLCOMPIE	PTRNx4P3ALIE	PTRNx4P2ALIE	PTRNx4P1ALIE	PTRNx4P0ALIE	PTRNx4P3LSIE	PTRNx4P2LSIE	PTRNx4P1LSIE	PTRNx4P0LSIE	PTRNx4P3CONIE	PTRNx4P2CONIE	PTRNx4P1CONIE	PTRNx4P0CONIE

Table 11-92 Fields for Register: MDIO_Interrupt_Enable

Bits	Name	Memory Access	Description
31:14	Reserved_31_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13	CWCOMPIE	R/W	Continuous Write Completion Interrupt Enable. When set, this bit enables the Continuous Write Completion interrupts. **Note:** This bit is available only when the PRS field of MDIO Port Nx4 Indirect Control register is 0 **Value After Reset:** 0x0 **Exists:** Always
12	SNGLCOMPIE	R/W	Single Command Completion Interrupt Enable. When set, this bit enables Single Command Completion interrupts. **Note:** This bit is available only when the PRS field of MDIO Port Nx4 Indirect Control register is 0. **Value After Reset:** 0x0 **Exists:** Always

Table 11-92 Fields for Register: MDIO_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
11	PTRNx4P3ALIE	R/W	Device Present Status Change Interrupt (Port Nx4 Plus3) Enable. When set, this bit enables Device Present Status Change interrupts on Port Nx4 Plus3. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
10	PTRNx4P2ALIE	R/W	Device Present Status Change Interrupt (Port Nx4 Plus2) Enable. When set, this bit enables Device Present Status Change interrupts on Port Nx4 Plus2. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
9	PTRNx4P1ALIE	R/W	Device Present Status Change Interrupt (Port Nx4 Plus1) Enable. When set, this bit enables Device Present Status Change interrupts on Port Nx4 Plus1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
8	PTRNx4P0ALIE	R/W	Device Present Status Change Interrupt (Port Nx4 Plus0) Enable. When set, this bit enables Device Present Status Change interrupts on Port Nx4 Plus0. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
7	PTRNx4P3LSIE	R/W	Link Status Change Interrupt (Port Nx4 Plus3) Enable. When set, this bit enables Link Status Change interrupts on Port Nx4 Plus3. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-92 Fields for Register: MDIO_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
6	PTRNx4P2LSIE	R/W	Link Status Change Interrupt (Port Nx4 Plus2) Enable. When set, this bit enables Link Status Change interrupts on Port Nx4 Plus2. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	PTRNx4P1LSIE	R/W	Link Status Change Interrupt (Port Nx4 Plus1) Enable. When set, this bit enables Link Status Change interrupts on Port Nx4 Plus1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	PTRNx4P0LSIE	R/W	Link Status Change Interrupt (Port Nx4 Plus0) Enable. When set, this bit enables Link Status Change interrupts on Port Nx4 Plus0. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	PTRNx4P3CONIE	R/W	Connect/Disconnect Event Interrupt (Port Nx4 Plus3) Enable. When set, this bit enables Connect/Disconnect Event interrupts on Port Nx4 Plus3. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	PTRNx4P2CONIE	R/W	Connect/Disconnect Event Interrupt (Port Nx4 Plus2) Enable. When set, this bit enables Connect/Disconnect Event interrupts on Port Nx4 Plus2. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-92 Fields for Register: MDIO_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
1	PTRNx4P1CONIE	R/W	Connect/Disconnect Event Interrupt (Port Nx4 Plus1) Enable. When set, this bit enables Connect/Disconnect Event interrupts on Port Nx4 Plus1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
0	PTRNx4P0CONIE	R/W	Connect/Disconnect Event Interrupt (Port Nx4 Plus0) Enable. When set, this bit enables Connect/Disconnect Event interrupts on Port Nx4 Plus0. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

11.1.89 MDIO_Port_Connect_Disconnect_Status

- **Description:** This register gives the hot plug-in status for Ports 0-31. This register is updated at the end of any read operation to any register of the corresponding PHY port. It is updated during single-read transfers or during continuous scan operations. During continuous scan operation, a PHY port is only accessed when the corresponding MDIO Continuous Scan Port Enable register bit is set.
- **Size:** 32 bits
- **Offset:** 0x21c
- **Exists:** (DWCXG_SMA_EN)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PORT31CON	PORT30CON	PORT29CON	PORT28CON	PORT27CON	PORT26CON	PORT25CON	PORT24CON	PORT23CON	PORT22CON	PORT21CON	PORT20CON	PORT19CON	PORT18CON	PORT17CON	PORT16CON	PORT15CON	PORT14CON	PORT13CON	PORT12CON	PORT11CON	PORT10CON	PORT9CON	PORT8CON	PORT7CON	PORT6CON	PORT5CON	PORT4CON	PORT3CON	PORT2CON	PORT1CON	PORT0CON

Table 11-93 Fields for Register: MDIO_Port_Connect_Disconnect_Status

Bits	Name	Memory Access	Description
31	PORT31CON	R	Port 31 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 31. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
30	PORT30CON	R	Port 30 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 30. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
29	PORT29CON	R	Port 29 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 29. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
28	PORT28CON	R	Port 28 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 28. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always

Table 11-93 Fields for Register: MDIO_Port_Connect_Disconnect_Status (continued)

Bits	Name	Memory Access	Description
27	PORT27CON	R	Port 27 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 27. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
26	PORT26CON	R	Port 26 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 26. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
25	PORT25CON	R	Port 25 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 25. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
24	PORT24CON	R	Port 24 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 24. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
23	PORT23CON	R	Port 23 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 23. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
22	PORT22CON	R	Port 22 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 22. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always

Table 11-93 Fields for Register: MDIO_Port_Connect_Disconnect_Status (continued)

Bits	Name	Memory Access	Description
21	PORT21CON	R	Port 21 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 21. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
20	PORT20CON	R	Port 20 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 20. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
19	PORT19CON	R	Port 19 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 19. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
18	PORT18CON	R	Port 18 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 18. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
17	PORT17CON	R	Port 17 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 17. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
16	PORT16CON	R	Port 16 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 16. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always

Table 11-93 Fields for Register: MDIO_Port_Connect_Disconnect_Status (continued)

Bits	Name	Memory Access	Description
15	PORT15CON	R	Port 15 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 15. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
14	PORT14CON	R	Port 14 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 14. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
13	PORT13CON	R	Port 13 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 13. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
12	PORT12CON	R	Port 12 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 12. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
11	PORT11CON	R	Port 11 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 11. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
10	PORT10CON	R	Port 10 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 10. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always

Table 11-93 Fields for Register: MDIO_Port_Connect_Disconnect_Status (continued)

Bits	Name	Memory Access	Description
9	PORT9CON	R	Port 9 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 9. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
8	PORT8CON	R	Port 8 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 8. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
7	PORT7CON	R	Port 7 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 7. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
6	PORT6CON	R	Port 6 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 6. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
5	PORT5CON	R	Port 5 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 5. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
4	PORT4CON	R	Port 4 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 4. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always

Table 11-93 Fields for Register: MDIO_Port_Connect_Disconnect_Status (continued)

Bits	Name	Memory Access	Description
3	PORT3CON	R	Port 3 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 3. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
2	PORT2CON	R	Port 2 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 2. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
1	PORT1CON	R	Port 1 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 1. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always
0	PORT0CON	R	Port 0 Connect/Disconnect. The hardware sets this bit when a connection event is detected on Port 0. It is cleared when a disconnection event is detected. **Value After Reset:** 0x0 **Exists:** Always

11.1.90 MDIO_Clause_22_Port

- **Description:** This register configures the SMA to access the PHY ports using either the Clause 22 or Clause 45 packet format. All SMA operations (single or continuous) depend on setting these bits correctly.
- **Size:** 32 bits
- **Offset:** 0x220
- **Exists:** (DWCXG_SMA_EN)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PTR31CL22	PTR30CL22	PTR29CL22	PTR28CL22	PTR27CL22	PTR26CL22	PTR25CL22	PTR24CL22	PTR23CL22	PTR22CL22	PTR21CL22	PTR20CL22	PTR19CL22	PTR18CL22	PTR17CL22	PTR16CL22	PTR15CL22	PTR14CL22	PTR13CL22	PTR12CL22	PTR11CL22	PTR10CL22	PTR9CL22	PTR8CL22	PTR7CL22	PTR6CL22	PTR5CL22	PTR4CL22	PTR3CL22	PTR2CL22	PTR1CL22	PTR0CL22

Table 11-94 Fields for Register: MDIO_Clause_22_Port

Bits	Name	Memory Access	Description
31	PTR31CL22	R/W	Port31CL22 Enable. Indicates that Port 31 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
30	PTR30CL22	R/W	Port30CL22 Enable. Indicates that Port 30 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
29	PTR29CL22	R/W	Port29CL22 Enable. Indicates that Port 29 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
28	PTR28CL22	R/W	Port28CL22 Enable. Indicates that Port 28 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always

Table 11-94 Fields for Register: MDIO_Clause_22_Port (continued)

Bits	Name	Memory Access	Description
27	PTR27CL22	R/W	Port27CL22 Enable. Indicates that Port 27 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
26	PTR26CL22	R/W	Port26CL22 Enable. Indicates that Port 26 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
25	PTR25CL22	R/W	Port25CL22 Enable. Indicates that Port 25 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
24	PTR24CL22	R/W	Port24CL22 Enable. Indicates that Port 24 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
23	PTR23CL22	R/W	Port23CL22 Enable. Indicates that Port 23 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
22	PTR22CL22	R/W	Port22CL22 Enable. Indicates that Port 22 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
21	PTR21CL22	R/W	Port21CL22 Enable. Indicates that Port 21 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always

Table 11-94 Fields for Register: MDIO_Clause_22_Port (continued)

Bits	Name	Memory Access	Description
20	PTR20CL22	R/W	Port20CL22 Enable. Indicates that Port 20 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
19	PTR19CL22	R/W	Port19CL22 Enable. Indicates that Port 19 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
18	PTR18CL22	R/W	Port18CL22 Enable. Indicates that Port 18 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
17	PTR17CL22	R/W	Port17CL22 Enable. Indicates that Port 17 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
16	PTR16CL22	R/W	Port16CL22 Enable. Indicates that Port 16 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
15	PTR15CL22	R/W	Port15CL22 Enable. Indicates that Port 15 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
14	PTR14CL22	R/W	Port14CL22 Enable. Indicates that Port 14 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always

Table 11-94 Fields for Register: MDIO_Clause_22_Port (continued)

Bits	Name	Memory Access	Description
13	PTR13CL22	R/W	Port13CL22 Enable. Indicates that Port 13 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
12	PTR12CL22	R/W	Port12CL22 Enable. Indicates that Port 12 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
11	PTR11CL22	R/W	Port11CL22 Enable. Indicates that Port 11 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
10	PTR10CL22	R/W	Port10CL22 Enable. Indicates that Port 10 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
9	PTR9CL22	R/W	Port9CL22 Enable. Indicates that Port 9 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
8	PTR8CL22	R/W	Port8CL22 Enable. Indicates that Port 8 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
7	PTR7CL22	R/W	Port7CL22 Enable. Indicates that Port 7 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always

Table 11-94 Fields for Register: MDIO_Clause_22_Port (continued)

Bits	Name	Memory Access	Description
6	PTR6CL22	R/W	Port6CL22 Enable. Indicates that Port 6 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
5	PTR5CL22	R/W	Port5CL22 Enable. Indicates that Port 5 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
4	PTR4CL22	R/W	Port4CL22 Enable. Indicates that Port 4 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
3	PTR3CL22	R/W	Port3CL22 Enable. Indicates that Port 3 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
2	PTR2CL22	R/W	Port2CL22 Enable. Indicates that Port 2 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
1	PTR1CL22	R/W	Port1CL22 Enable. Indicates that Port 1 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always
0	PTR0CL22	R/W	Port0CL22 Enable. Indicates that Port 0 is connected to a PHY that only supports Clause 22. When this bit is reset, the PHY supports Clause 45. **Value After Reset:** 0x0 **Exists:** Always

11.1.91 MDIO_Port_Nx4_Indirect_Control

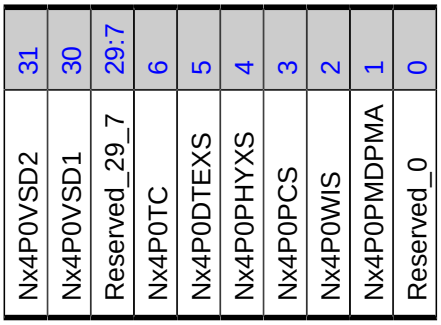
- **Description:** This register contain fields that control the Port range select for accessing register sets.
- **Size:** 32 bits
- **Offset:** 0x224
- **Exists:** (DWCXG_SMA_EN)

Table 11-95 Fields for Register: MDIO_Port_Nx4_Indirect_Control

Bits	Name	Memory Access	Description
31:3	Reserved_31_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2:0	PRS	R/W	Port Range Select. This register field controls the Port range select for accessing all the following register sets ■ MDIO Port Nx4P0/1/2/3 Device In Use register ■ MDIO Port Nx4P0/1/2/3 Link Status register ■ MDIO Port Nx4P0/1/2/3 Alive Status register ■ MDIO Interrupt Status register ■ MDIO Interrupt Enable register ❑ 0: Selects the Port address range of 0 to 3 ❑ 1: Selects the Port Address range of 4 to 7 ❑ 2: Selects the Port Address range of 8 to 11 ❑ 3: Selects the Port Address range of 12 to 15 ❑ 4: Selects the Port Address range of 16 to 19 ❑ 5: Selects the Port Address range of 20 to 23 ❑ 6: Selects the Port Address range of 24 to 27 ❑ 7: Selects the Port Address range of 28 to 31 **Value After Reset:** 0x0 **Exists:** Always

11.1.92 MDIO_PortNx4P0_Device_In_Use

- **Description:** This register gives the status of each device on Port Nx4 Plus0 (Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register). Set these bits to indicate that a device is present and that the SMA must access it during continuous scan/write operations. If any bit is cleared, the corresponding device is not scanned when the Continuous Scan command is issued nor is it written when the Continuous Write command is issued. This register setting does not affect single read or write commands.
- **Size:** 32 bits
- **Offset:** 0x230
- **Exists:** (DWCXG_SMA_EN)



31	30	29:7	6	5	4	3	2	1	0
Nx4P0VSD2	Nx4P0VSD1	Reserved_29_7	Nx4P0TC	Nx4P0DTEXS	Nx4P0PHYXS	Nx4P0PCS	Nx4P0WIS	Nx4P0PMDPMA	Reserved_0

Table 11-96 Fields for Register: MDIO_PortNx4P0_Device_In_Use

Bits	Name	Memory Access	Description
31	Nx4P0VSD2	R/W	VSD2 Device is in Use When Vendor-Specific Device 2 is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
30	Nx4P0VSD1	R/W	VSD1 Device is in Use When Vendor-Specific Device 1 is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-96 Fields for Register: MDIO_PortNx4P0_Device_In_Use (continued)

Bits	Name	Memory Access	Description
6	Nx4P0TC	R/W	TC Device is in Use When TC is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
5	Nx4P0DTEXS	R/W	DTEXS Device is in Use When DTE XS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
4	Nx4P0PHYXS	R/W	PHYXS Device is in Use When PHY XS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
3	Nx4P0PCS	R/W	PCS Device is in Use When PCS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
2	Nx4P0WIS	R/W	WIS Device is in Use When WIS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
1	Nx4P0PMDPMA	R/W	PMA Device is in Use When PMD/PMA is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always

Table 11-96 Fields for Register: MDIO_PortNx4P0_Device_In_Use (continued)

Bits	Name	Memory Access	Description
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.93 MDIO_PortNx4P0_Link_Status

- **Description:** This register gives the link status of the devices in PHY Port Nx4Plus0 ((Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register) at the end of each continuous scan cycle. Up to eight devices are supported, as specified in IEEE 802.3-2018. You can disable the polling of any device by clearing its corresponding MDIO Port Nx4Plus0 Device In Use register bit.
  Note: All the register fields are renamed from P0 to Nx4P0 (Nx4 Plus 0)
- **Size:** 32 bits
- **Offset:** 0x234
- **Exists:** (DWCXG_SMA_EN)

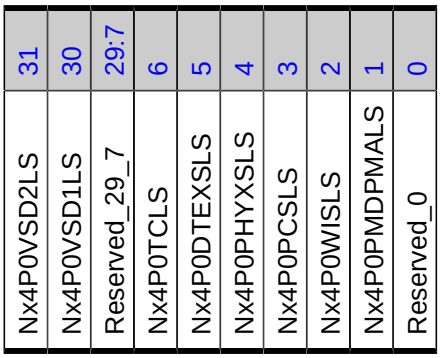


Table 11-97 Fields for Register: MDIO_PortNx4P0_Link_Status

Bits	Name	Memory Access	Description
31	Nx4P0VSD2LS	R	Link Status of VSD2 Device When the Vendor-Specific Device 2 link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
30	Nx4P0VSD1LS	R	Link Status of VCD1 Device When the Vendor-Specific Device 1 link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-97 Fields for Register: MDIO_PortNx4P0_Link_Status (continued)

Bits	Name	Memory Access	Description
6	Nx4P0TCLS	R	Link Status of TC Device When the TC link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	Nx4P0DTEXSLS	R	Link Status of DTEXS Device When the DTE XS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	Nx4P0PHYXSLS	R	Link Status of PHYXS Device When the PHY XS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	Nx4P0PCSLS	R	Link Status of PCS Device When the PCS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	Nx4P0WISLS	R	Link Status of WIS Device When the WIS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
1	Nx4P0PMDPMALS	R	Link Status of PMA Device When the PMD/PMA link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-97 Fields for Register: MDIO_PortNx4P0_Link_Status (continued)

Bits	Name	Memory Access	Description
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.94 MDIO_PortNx4P0_Alive_Status

- **Description:** This register gives the device status in PHY Port Nx4Plus0 ((Here N is as per PRS field of "MDIO Port Nx4 Indirect Control" register) at the end of each continuous scan cycle. Up to eight devices are supported, as specified in IEEE 802.3-2018. You can disable the polling of any device by clearing its corresponding MDIO Port Nx4Plus0 Device In Use register bit.
  Note: All the register fields are renamed from P0 to Nx4P0 (Nx4 Plus 0)
- **Size:** 32 bits
- **Offset:** 0x238
- **Exists:** (DWCXG_SMA_EN)

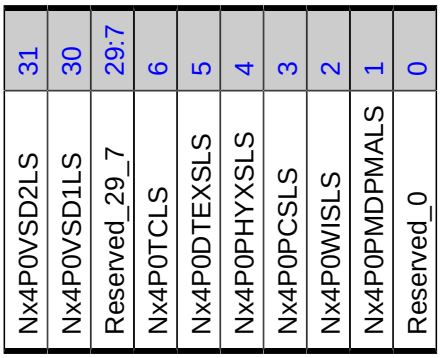


Table 11-98 Fields for Register: MDIO_PortNx4P0_Alive_Status

Bits	Name	Memory Access	Description
31	Nx4P0VSD2LS	R	Alive Status of VSD2 Device When Vendor-Specific Device 2 is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
30	Nx4P0VSD1LS	R	Alive Status of VSD1 Device When Vendor-Specific Device 1 is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-98 Fields for Register: MDIO_PortNx4P0_Alive_Status (continued)

Bits	Name	Memory Access	Description
6	Nx4P0TCLS	R	Alive Status of TC Device When TC is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	Nx4P0DTEXSLS	R	Alive Status of DTEXS Device When DTE XS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	Nx4P0PHYXSLS	R	Alive Status of PHYXS Device When PHY XS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	Nx4P0PCSLS	R	Alive Status of PCS Device When PCS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	Nx4P0WISLS	R	Alive Status of WIS Device When WIS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-98 Fields for Register: MDIO_PortNx4P0_Alive_Status (continued)

Bits	Name	Memory Access	Description
1	Nx4P0PMDPMALS	R	Alive Status of PMA Device When PMD/PMA is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.95 MDIO_PortNx4P1_Device_In_Use

- **Description:** This register gives the status of each device on Port Nx4 Plus1 (Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register). Set these bits to indicate that a device is present and that the SMA must access it during continuous scan/write operations. If any bit is cleared, the corresponding device is not scanned when the Continuous Scan command is issued nor is it written when the Continuous Write command is issued. This register setting does not affect single read or write commands.
  Note: All the register fields are renamed from P1 to Nx4P1 (Nx4 Plus 1)
- **Size:** 32 bits
- **Offset:** 0x240
- **Exists:** (DWCXG_SMA_EN)

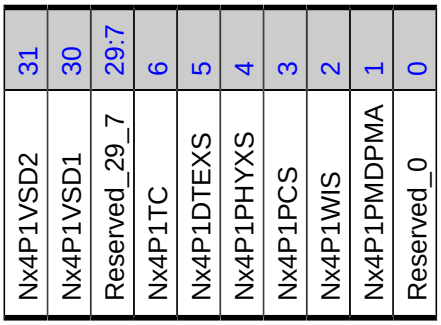


Table 11-99 Fields for Register: MDIO_PortNx4P1_Device_In_Use

Bits	Name	Memory Access	Description
31	Nx4P1VSD2	R/W	VSD2 Device is in Use When Vendor-Specific Device 2 is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
30	Nx4P1VSD1	R/W	VSD1 Device is in Use When Vendor-Specific Device 1 is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-99 Fields for Register: MDIO_PortNx4P1_Device_In_Use (continued)

Bits	Name	Memory Access	Description
6	Nx4P1TC	R/W	TC Device is in Use When TC is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
5	Nx4P1DTEXS	R/W	DTEXS Device is in Use When DTE XS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
4	Nx4P1PHYXS	R/W	PHYXS Device is in Use When PHY XS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
3	Nx4P1PCS	R/W	PCS Device is in Use When PCS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
2	Nx4P1WIS	R/W	WIS Device is in Use When WIS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
1	Nx4P1PMDPMA	R/W	PMA Device is in Use When PMD/PMA is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always

Table 11-99 Fields for Register: MDIO_PortNx4P1_Device_In_Use (continued)

Bits	Name	Memory Access	Description
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.96 MDIO_PortNx4P1_Link_Status

- **Description:** This register gives the link status of the devices in PHY Port Nx4Plus1 ((Here N is as per PRS field of MDIO Port Nx4 Indirect Control register) at the end of each continuous scan cycle. Up to eight devices are supported, as specified in IEEE 802.3-2018. You can disable the polling of any device by clearing its corresponding MDIO Port Nx4Plus1 Device In Use register bit.
  Note: All the register fields are renamed from P1 to Nx4P1 (Nx4 Plus 1)
- **Size:** 32 bits
- **Offset:** 0x244
- **Exists:** (DWCXG_SMA_EN)

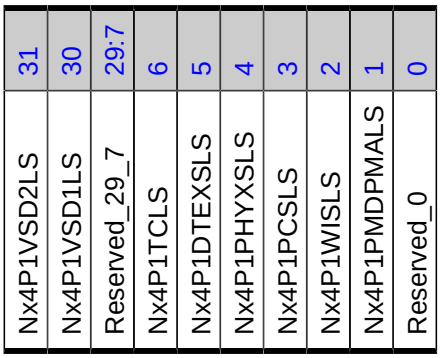


Table 11-100 Fields for Register: MDIO_PortNx4P1_Link_Status

Bits	Name	Memory Access	Description
31	Nx4P1VSD2LS	R	Link Status of VSD2 Device When the Vendor-Specific Device 2 link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
30	Nx4P1VSD1LS	R	Link Status of VSD1 Device When the Vendor-Specific Device 1 link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-100 Fields for Register: MDIO_PortNx4P1_Link_Status (continued)

Bits	Name	Memory Access	Description
6	Nx4P1TCLS	R	Link Status of TC Device When the TC link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	Nx4P1DTEXSLS	R	Link Status of DTEXS Device When the DTE XS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	Nx4P1PHYXSLS	R	Link Status of PHYXS Device When the PHY XS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	Nx4P1PCSLS	R	Link Status of PCS Device When the PCS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	Nx4P1WISLS	R	Link Status of WIS Device When the WIS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
1	Nx4P1PMDPMALS	R	Link Status of PMA Device When the PMD/PMA link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-100 Fields for Register: MDIO_PortNx4P1_Link_Status (continued)

Bits	Name	Memory Access	Description
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.97 MDIO_PortNx4P1_Alive_Status

- **Description:** This register gives the device status in PHY Port Nx4Plus1 ((Here N is as per PRS field of MDIO Port Nx4 Indirect Control register) at the end of each continuous scan cycle. Up to eight devices are supported, as specified in IEEE 802.3-2018. You can disable the polling of any device by clearing its corresponding MDIO Port Nx4Plus1 Device In Use register bit.
  Note: All the register fields are renamed from P1 to Nx4P1 (Nx4 Plus 1)
- **Size:** 32 bits
- **Offset:** 0x248
- **Exists:** (DWCXG_SMA_EN)

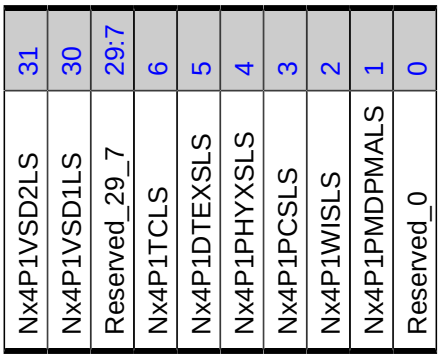


Table 11-101 Fields for Register: MDIO_PortNx4P1_Alive_Status

Bits	Name	Memory Access	Description
31	Nx4P1VSD2LS	R	Alive Status of VSD2 Device When Vendor-Specific Device 2 is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
30	Nx4P1VSD1LS	R	Alive Status of VSD1 Device When Vendor-Specific Device 1 is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-101 Fields for Register: MDIO_PortNx4P1_Alive_Status (continued)

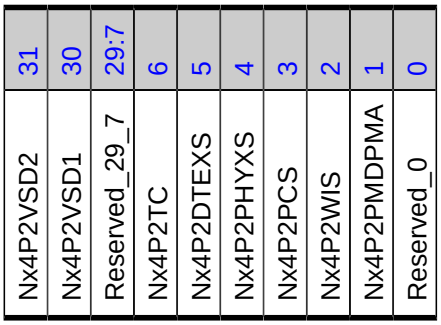
Bits	Name	Memory Access	Description
6	Nx4P1TCLS	R	Alive Status of TC Device When TC is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	Nx4P1DTEXSLS	R	Alive Status of DTEXS Device When DTE XS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	Nx4P1PHYXSLS	R	Alive Status of PHYXS Device When PHY XS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	Nx4P1PCSLS	R	Alive Status of PCS Device When PCS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	Nx4P1WISLS	R	Alive Status of WIS Device When WIS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-101 Fields for Register: MDIO_PortNx4P1_Alive_Status (continued)

Bits	Name	Memory Access	Description
1	Nx4P1PMDPMALS	R	Alive Status of PMA Device When PMD/PMA is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.98 MDIO_PortNx4P2_Device_In_Use

- **Description:** This register gives the status of each device on Port Nx4 Plus2 (Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register). Set these bits to indicate that a device is present and that the SMA must access it during continuous scan/write operations. If any bit is cleared, the corresponding device is not scanned when the Continuous Scan command is issued nor is it written when the Continuous Write command is issued. This register setting does not affect single read or write commands.
  Note: All the register fields are renamed from P2 to Nx4P2 (Nx4 Plus 2)
- **Size:** 32 bits
- **Offset:** 0x250
- **Exists:** (DWCXG_SMA_EN)



31	30	29:7	6	5	4	3	2	1	0
Nx4P2VSD2	Nx4P2VSD1	Reserved_29_7	Nx4P2TC	Nx4P2DTEXS	Nx4P2PHYXS	Nx4P2PCS	Nx4P2WIS	Nx4P2PMDPMA	Reserved_0

Table 11-102 Fields for Register: MDIO_PortNx4P2_Device_In_Use

Bits	Name	Memory Access	Description
31	Nx4P2VSD2	R/W	VSD2 Device is in Use When Vendor-Specific Device 2 is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
30	Nx4P2VSD1	R/W	VSD1 Device is in Use When Vendor-Specific Device 1 is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-102 Fields for Register: MDIO_PortNx4P2_Device_In_Use (continued)

Bits	Name	Memory Access	Description
6	Nx4P2TC	R/W	TC Device is in Use When TC is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
5	Nx4P2DTEXS	R/W	DTEXS Device is in Use When DTE XS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
4	Nx4P2PHYXS	R/W	PHYXS Device is in Use When PHY XS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
3	Nx4P2PCS	R/W	PCS Device is in Use When PCS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
2	Nx4P2WIS	R/W	WIS Device is in Use When WIS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
1	Nx4P2PMDPMA	R/W	PMA Device is in Use When PMD/PMA is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always

Table 11-102 Fields for Register: MDIO_PortNx4P2_Device_In_Use (continued)

Bits	Name	Memory Access	Description
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.99 MDIO_PortNx4P2_Link_Status

- **Description:** This register gives the link status of the devices in PHY Port Nx4Plus2 ((Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register) at the end of each continuous scan cycle. Up to eight devices are supported, as specified in IEEE 802.3-2018. You can disable the polling of any device by clearing its corresponding MDIO Port Nx4Plus1 Device In Use register bit.
  Note: All the register fields are renamed from P2 to Nx4P2 (Nx4 Plus 2)
- **Size:** 32 bits
- **Offset:** 0x254
- **Exists:** (DWCXG_SMA_EN)

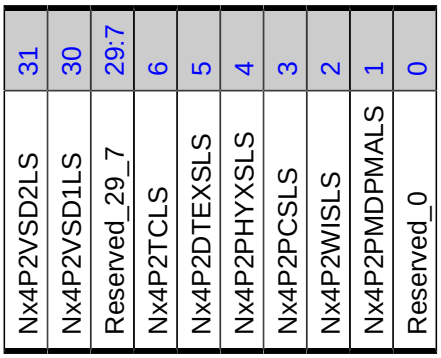


Table 11-103 Fields for Register: MDIO_PortNx4P2_Link_Status

Bits	Name	Memory Access	Description
31	Nx4P2VSD2LS	R	Link Status of VSD2 Device When the Vendor-Specific Device 2 link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
30	Nx4P2VSD1LS	R	Link Status of VSD1 Device When the Vendor-Specific Device 1 link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-103 Fields for Register: MDIO_PortNx4P2_Link_Status (continued)

Bits	Name	Memory Access	Description
6	Nx4P2TCLS	R	Link Status of TC Device When the TC link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	Nx4P2DTEXSLS	R	Link Status of DTEXS Device When the DTE XS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	Nx4P2PHYXSLS	R	Link Status of PHYXS Device When the PHY XS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	Nx4P2PCSLS	R	Link Status of PCS Device When the PCS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	Nx4P2WISLS	R	Link Status of WIS Device When the WIS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
1	Nx4P2PMDPMALS	R	Link Status of PMA Device When the PMD/PMA link is up, this bit is set to 1. Here N is as per PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-103 Fields for Register: MDIO_PortNx4P2_Link_Status (continued)

Bits	Name	Memory Access	Description
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.100 MDIO_PortNx4P2_Alive_Status

- **Description:** This register gives the device status in PHY Port Nx4Plus2 ((Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register) at the end of each continuous scan cycle. Up to eight devices are supported, as specified in IEEE 802.3-2018. You can disable the polling of any device by clearing its corresponding MDIO Port Nx4Plus2 Device In Use register bit.
  Note: All the register fields are renamed from P2 to Nx4P2 (Nx4 Plus 2)
- **Size:** 32 bits
- **Offset:** 0x258
- **Exists:** (DWCXG_SMA_EN)

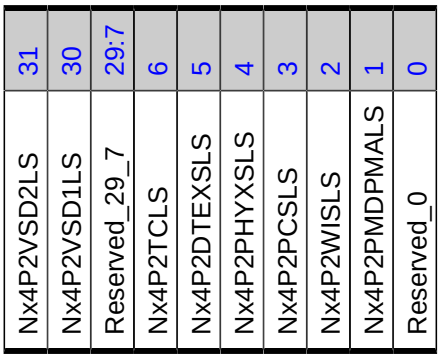


Table 11-104 Fields for Register: MDIO_PortNx4P2_Alive_Status

Bits	Name	Memory Access	Description
31	Nx4P2VSD2LS	R	Alive Status of VSD2 Device When Vendor-Specific Device 2 is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
30	Nx4P2VSD1LS	R	Alive Status of VSD1 Device When Vendor-Specific Device 1 is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-104 Fields for Register: MDIO_PortNx4P2_Alive_Status (continued)

Bits	Name	Memory Access	Description
6	Nx4P2TCLS	R	Alive Status of TC Device When TC is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	Nx4P2DTEXSLS	R	Alive Status of DTEXS Device When DTE XS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	Nx4P2PHYXSLS	R	Alive Status of PHYXS Device When PHY XS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	Nx4P2PCSLS	R	Alive Status of PCS Device When PCS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	Nx4P2WISLS	R	Alive Status of WIS Device When WIS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-104 Fields for Register: MDIO_PortNx4P2_Alive_Status (continued)

Bits	Name	Memory Access	Description
1	Nx4P2PMDPMALS	R	Alive Status of PMA Device When PMD/PMA is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.101 MDIO_PortNx4P3_Device_In_Use

- **Description:** This register gives the status of each device on Port Nx4 Plus3 (Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register). Set these bits to indicate that a device is present and that the SMA must access it during continuous scan/write operations. If any bit is cleared, the corresponding device is not scanned when the Continuous Scan command is issued nor is it written when the Continuous Write command is issued. This register setting does not affect single read or write commands.
  Note: All the register fields are renamed from P3 to Nx4P3 (Nx4 Plus 3)
- **Size:** 32 bits
- **Offset:** 0x260
- **Exists:** (DWCXG_SMA_EN)

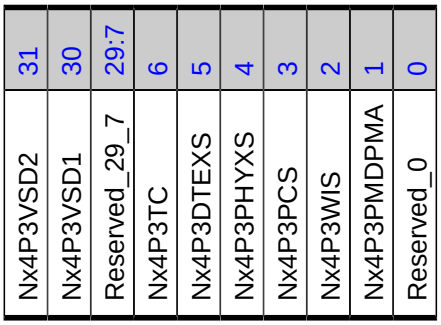


Table 11-105 Fields for Register: MDIO_PortNx4P3_Device_In_Use

Bits	Name	Memory Access	Description
31	Nx4P3VSD2	R/W	VSD2 Device is in Use When Vendor-Specific Device 2 is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
30	Nx4P3VSD1	R/W	VSD1 Device is in Use When Vendor-Specific Device 1 is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-105 Fields for Register: MDIO_PortNx4P3_Device_In_Use (continued)

Bits	Name	Memory Access	Description
6	Nx4P3TC	R/W	TC Device is in Use When TC is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
5	Nx4P3DTEXS	R/W	DTEXS Device is in Use When DTE XS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
4	Nx4P3PHYXS	R/W	PHYXS Device is in Use When PHY XS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
3	Nx4P3PCS	R/W	PCS Device is in Use When PCS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
2	Nx4P3WIS	R/W	WIS Device is in Use When WIS is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always
1	Nx4P3PMDPMA	R/W	PMA Device is in Use When PMD/PMA is present, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x1 **Exists:** Always

Table 11-105 Fields for Register: MDIO_PortNx4P3_Device_In_Use (continued)

Bits	Name	Memory Access	Description
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.102 MDIO_PortNx4P3_Link_Status

- **Description:** This register gives the link status of the devices in PHY Port Nx4Plus3 ((Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register) at the end of each continuous scan cycle. Up to eight devices are supported, as specified in IEEE 802.3-2018. You can disable the polling of any device by clearing its corresponding MDIO Port Nx4Plus3 Device In Use register bit.
  Note: All the register fields are renamed from P3 to Nx4P3 (Nx4 Plus 3)
- **Size:** 32 bits
- **Offset:** 0x264
- **Exists:** (DWCXG_SMA_EN)

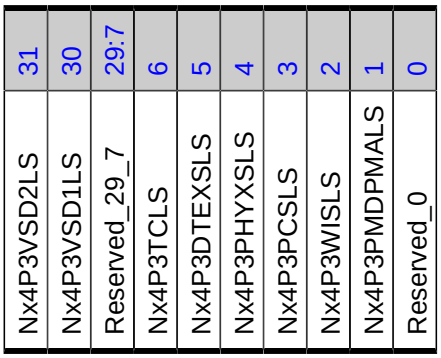


Table 11-106 Fields for Register: MDIO_PortNx4P3_Link_Status

Bits	Name	Memory Access	Description
31	Nx4P3VSD2LS	R	Link Status of VSD2 Device When the Vendor-Specific Device 2 link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
30	Nx4P3VSD1LS	R	Link Status of VSD1 Device When the Vendor-Specific Device 1 link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-106 Fields for Register: MDIO_PortNx4P3_Link_Status (continued)

Bits	Name	Memory Access	Description
6	Nx4P3TCLS	R	Link Status of TC Device When the TC link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	Nx4P3DTEXSLS	R	Link Status of DTEXS Device When the DTE XS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	Nx4P3PHYXSLS	R	Link Status of PHYXS Device When the PHY XS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	Nx4P3PCSLS	R	Link Status of PCS Device When the PCS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	Nx4P3WISLS	R	Link Status of WIS Device When the WIS link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
1	Nx4P3PMDPMALS	R	Link Status of PMA Device When the PMD/PMA link is up, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-106 Fields for Register: MDIO_PortNx4P3_Link_Status (continued)

Bits	Name	Memory Access	Description
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.103 MDIO_PortNx4P3_Alive_Status

- **Description:** This register gives the device status in PHY Port Nx4Plus3 ((Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register) at the end of each continuous scan cycle. Up to eight devices are supported, as specified in IEEE 802.3-2018. You can disable the polling of any device by clearing its corresponding MDIO Port Nx4Plus0 Device In Use register bit.
  Note: All the register fields are renamed from P3 to Nx4P3 (Nx4 Plus 3)
- **Size:** 32 bits
- **Offset:** 0x268
- **Exists:** (DWCXG_SMA_EN)

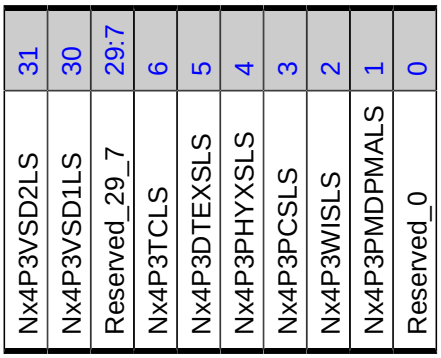


Table 11-107 Fields for Register: MDIO_PortNx4P3_Alive_Status

Bits	Name	Memory Access	Description
31	Nx4P3VSD2LS	R	Alive Status of VSD2 Device When Vendor-Specific Device 2 is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
30	Nx4P3VSD1LS	R	Alive Status of VSD1 Device When Vendor-Specific Device 1 is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
29:7	Reserved_29_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-107 Fields for Register: MDIO_PortNx4P3_Alive_Status (continued)

Bits	Name	Memory Access	Description
6	Nx4P3TCLS	R	Alive Status of TC Device When TC is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
5	Nx4P3DTEXSLS	R	Alive Status of DTEXS Device When DTE XS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
4	Nx4P3PHYXSLS	R	Alive Status of PHYXS Device When PHY XS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
3	Nx4P3PCSLS	R	Alive Status of PCS Device When PCS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
2	Nx4P3WISLS	R	Alive Status of WIS Device When WIS is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always

Table 11-107 Fields for Register: MDIO_PortNx4P3_Alive_Status (continued)

Bits	Name	Memory Access	Description
1	Nx4P3PMDPMALS	R	Alive Status of PMA Device When PMD/PMA is present and functioning properly, this bit is set to 1. Here N is as per the PRS field of MDIO Port Nx4 Indirect Control register **Value After Reset:** 0x0 **Exists:** Always
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.104 MAC_GPIO_Control

- **Description:** The GPIO Control register controls the GPIO.
- **Size:** 32 bits
- **Offset:** 0x278
- **Exists:** (DWCXG_GPIO_EN)

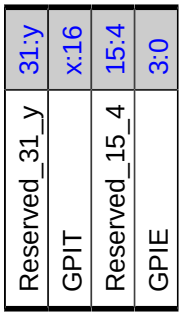


Table 11-108 Fields for Register: MAC_GPIO_Control

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** DWCXG_GIW<16 **Range Variable[y]:** DWCXG_GIW + 16
x:16	GPIT	R/W	GPI Type. When this bit is set, it indicates that the corresponding GPIS bit in MAC_GPIO_Status register is of latched-low (LL) type. When this bit is reset, it indicates that the corresponding GPIS is of latched-high (LH) type. The number of bits available in this field depends on the GP Input Signal Width option. Other bits are not used (reserved and always reset). **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_GIW + 15
15:4	Reserved_15_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-108 Fields for Register: MAC_GPIO_Control (continued)

Bits	Name	Memory Access	Description
3:0	GPIE	R/W	GPI Interrupt Enable. ■ When this bit is set and the programmed event (LL or LH) occurs on the corresponding GPIS bit of MAC_GPIO_Status register, the GPIIS bit of DMA_Interrupt_Status register is set and an interrupt is generated on the mci_intr_o or sbd_intr_o signal. The GPIIS bit is cleared when the application reads the Bits[7:0] of MAC_GPIO_Status register. ■ When this bit is reset, the GPIIS bit is not set when any event occurs on the corresponding GPIS bits. **Value After Reset:** 0x0 **Exists:** Always

11.1.105 MAC_GPIO_Status

- **Description:** The General Purpose IO register provides the control to drive the following: up to 16 bits of output ports (GPO) and status of up to 16 input ports (GPIS).
  The GPIO Control register controls the GPIO.
- **Size:** 32 bits
- **Offset:** 0x27c
- **Exists:** (DWCXG_GPIO_EN)

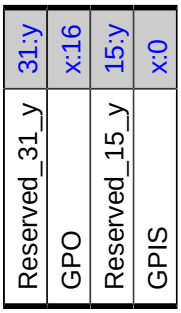


Table 11-109 Fields for Register: MAC_GPIO_Status

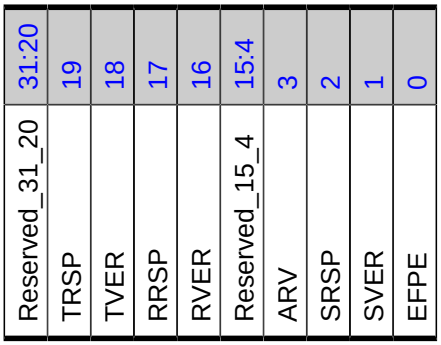
Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** DWCXG_GOW<16 **Testable:** untestable **Range Variable[y]:** DWCXG_GOW + 16
x:16	GPO	R/W	General Purpose Output. When this bit is set, it directly drives the gpo_o output ports. When this bit is reset, it does not directly drive the gpo_o output ports. The number of bits available in this field depends on the GP Input Signal Width option. Other bits are not used (reserved and always reset). **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** DWCXG_GOW + 15
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** DWCXG_GIW<16 **Testable:** untestable **Range Variable[y]:** DWCXG_GIW

Table 11-109 Fields for Register: MAC_GPIO_Status (continued)

Bits	Name	Memory Access	Description
x:0	GPIS	R	General Purpose Input Status. This field gives the status of the signals connected to the gpi_i port. This field is of the following types based on the setting of the corresponding GPIT field of MAC_GPIO_Control register: ■ Latched-low (LL): This field is cleared when the corresponding gpi_i input becomes low. This field remains low until the application reads this field after which this field reflects the current value of gpi_i input. ■ Latched-high (LH): This field is set when the corresponding gpi_i input becomes high. This field remains high until the application reads this field after which this field reflects the current value of gpi_i input. The number of bits available in this field depends on the GP Input Signal Width option. Other bits are not used (reserved and always reset). 0000H **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** DWCXG_GIW - 1

11.1.106 MAC_FPE_CTRL_STS

- **Description:** This register controls the operation of Frame Preemption.
- **Size:** 32 bits
- **Offset:** 0x280
- **Exists:** (DWCXG_FPE)



31:20	19	18	17	16	15:4	3	2	1	0
Reserved_31_20	TRSP	TVER	RRSP	RVER	Reserved_15_4	ARV	SRSP	SVER	EFPE

Table 11-110 Fields for Register: MAC_FPE_CTRL_STS

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19	TRSP	R	Transmitted Respond Frame This field is set when a Respond mPacket is transmitted (triggered by setting SRSP field). An interrupt can be generated for this event if FPEIE bit of MAC_Interrupt_Enable is set. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Not transmitted Respond Frame ■ 0x1 (ACTIVE): transmitted Respond Frame **Value After Reset:** 0x0 **Exists:** Always

Table 11-110 Fields for Register: MAC_FPE_CTRL_STS (continued)

Bits	Name	Memory Access	Description
18	TVER	R	Transmitted Verify Frame This field is set when a Verify mPacket is transmitted (triggered by setting SVER field). An interrupt can be generated for this event if FPEIE bit of MAC_Interrupt_Enable is set. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Not transmitted Verify Frame ■ 0x1 (ACTIVE): transmitted Verify Frame **Value After Reset:** 0x0 **Exists:** Always
17	RRSP	R	Received Respond Frame This field is set when a Respond mPacket is received. An interrupt can be generated for this event if FPEIE bit of MAC_Interrupt_Enable is set. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Not received Respond Frame ■ 0x1 (ACTIVE): Received Respond Frame **Value After Reset:** 0x0 **Exists:** Always
16	RVER	R	Received Verify Frame This field is set when a Verify mPacket is received. An interrupt can be generated for this event if FPEIE bit of MAC_Interrupt_Enable is set. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Not received Verify Frame ■ 0x1 (ACTIVE): Received Verify Frame **Value After Reset:** 0x0 **Exists:** Always
15:4	Reserved_15_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-110 Fields for Register: MAC_FPE_CTRL_STS (continued)

Bits	Name	Memory Access	Description
3	ARV	R/W	Auto-generate Respond mPacket on receiving Verify mPacket When set, function to auto-generate a Respond mPacket on receiving Verify mPacket is enabled. **Values:** ■ 0x0 (DISABLE): Autogenerate Respond mPacket on receiving Verify mPakcet is disabled ■ 0x1 (ENABLE): Autogenerate Respond mPacket on receiving Verify mPakcet is enabled **Value After Reset:** 0x0 **Exists:** Always
2	SRSP	R/W	Send Respond mPacket When set, this bit enabled hardware to send a Respond mPacket. Reset by hardware after sending the Respond mPacket. Access restriction applies. Can be set by software or by hardware when ARV bit is set and a verify mPacket is received. Self-cleared. Writing 0 has no effect. **Values:** ■ 0x0 (DISABLE): Send Respond mPacket is disabled ■ 0x1 (ENABLE): Send Respond mPacket is enabled **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	SVER	R/W	Send Verify mPacket When set, this bit enables hardware to send a verify mPacket. Reset by hardware after sending the Verify mPacket. Access restriction applies. Writing 1 sets the bit. Self-cleared. Writing 0 has no effect. **Values:** ■ 0x0 (DISABLE): Send Verify mPacket is disabled ■ 0x1 (ENABLE): Send Verify mPacket is enabled **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-110 Fields for Register: MAC_FPE_CTRL_STS (continued)

Bits	Name	Memory Access	Description
0	EFPE	R/W	Enable Tx Frame Preemption When set, Frame Preemption Tx functionality is enabled. **Values:** ■ 0x0 (DISABLE): Tx Frame Preemption is disabled ■ 0x1 (ENABLE): Tx Frame Preemption is enabled **Value After Reset:** 0x0 **Exists:** Always

11.1.107 MAC_CSR_SW_Ctrl

- **Description:** This register contains software programmable controls for changing the CSR access response and status bits clearing.
- **Size:** 32 bits
- **Offset:** 0x290
- **Exists:** Always

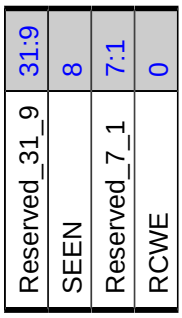


Table 11-111 Fields for Register: MAC_CSR_SW_Ctrl

Bits	Name	Memory Access	Description
31:9	Reserved_31_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	SEEN	R/W	Subordinate Error Response Enable ■ When this bit is set, the MAC responds with Subordinate Error for accesses to reserved registers in CSR space. ■ When this bit is reset, the MAC responds with Okay response to any register accessed from CSR space. **Values:** ■ 0x0 (DISABLE): Subordinate Error response Disabled ■ 0x1 (ENABLE): Subordinate Error response Enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_CSR_PORT>0)
7:1	Reserved_7_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-111 Fields for Register: MAC_CSR_SW_Ctrl (continued)

Bits	Name	Memory Access	Description
0	RCWE	R/W	Register Clear on Write 1 Enable When this bit is set, the access mode of list of register fields provided in Appendix F changes to Clear on Write 1, the application needs to set that respective bit to 1 to clear it. When this bit is reset, the access mode of list of register fields provided in Appendix F remain as Clear on Read. **Values:** ■ 0x0 (DISABLE): Register Clear on Write 1 Disable ■ 0x1 (ENABLE): Register Clear on Write 1 Enable **Value After Reset:** 0x0 **Exists:** Always

11.1.108 MAC_Address0_High

- **Description:** The MAC_Address0_High register holds the upper 16 bits of the 0th 6-byte MAC address of the station. The first DA byte that is received on the (X)GMII interface corresponds to the LS byte (Bits [7:0]) of the MAC_Address0_Low register. For example, if 0x112233445566 is received (0x11 in lane 0 of the first column) on the (X)GMII as the destination address, then the MAC_Address0 register [47:0] is compared with 0x665544332211.
  If the MAC address registers are configured to be double-synchronized to the (X)GMII clock domain, then the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the MAC_Address0_Low register are written, so MAC_Address0_High register must be written prior to MAC_Address0_Low register. For proper synchronization updates, the consecutive writes to MAC_Address0_Low register must be performed after at least four clock cycles of the slowest clock between the source and the destination clock domains.
- **Size:** 32 bits
- **Offset:** 0x300
- **Exists:** Always

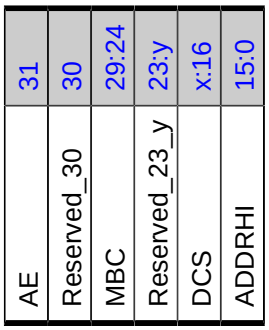


Table 11-112 Fields for Register: MAC_Address0_High

Bits	Name	Memory Access	Description
31	AE	R/W	Address Enable. ■ 1: The address filter module uses the 0th MAC address for perfect filtering. ■ 0: The address filter module ignores the address for filtering. **Value After Reset:** 0x1 **Exists:** Always
30	Reserved_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-112 Fields for Register: MAC_Address0_High (continued)

Bits	Name	Memory Access	Description
29:24	MBC	R/W	Mask Byte Control. These bits are mask control bits for comparing each of the MAC Address bytes. When set high, the MAC does not compare the corresponding byte of received DA or SA with the contents of MAC Address0 registers. Each bit controls the masking of the bytes as follows: ■ Bit 29: MAC Address High [15:8] ■ Bit 28: MAC Address High [7:0] ■ Bit 27: MAC Address Low[31:24] ■ ... ■ Bit 24: MAC Address Low[7:0] You can filter a group of addresses (known as group address filtering) by masking one or more bytes of the address. **Value After Reset:** 0x0 **Exists:** Always
23:y	Reserved_23_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_BN_RCH_BN_RCHL_W + 16
x:16	DCS	R/W	DMA Channel Select. If the DDS bit of MAC_Extended_Configuration register is not set: This field contains the binary representation of the DMA Channel number to which an Rx packet whose DA matches the MAC Address0 content is routed. If the DDS bit of MAC_Extended_Configuration register is set: The XDCS field of MAC_DChSel_IndReg register contains the one-hot representation of one or more DMA Channel numbers to which an Rx packet whose DA matches the MAC Address0 content is routed. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_RX_CH>1) **Range Variable[x]:** DWCXG_BN_RCH_BN_RCHL_W + 15
15:0	ADDRHI	R/W	MAC Address0[47:32]. This field contains the upper 16 bits[47:32] of the 0th 6-byte MAC address. The MAC uses this field of the MAC_Address0_High register for inserting in Destination Address field of the Transmit Flow Control (Pause) Packets. **Value After Reset:** 0xffff **Exists:** Always

11.1.109 MAC_Address0_Low

- **Description:** The MAC_Address0_Low register holds the lower 32 bits of the 0th 6-byte MAC address of the station.
- **Size:** 32 bits
- **Offset:** 0x304
- **Exists:** Always

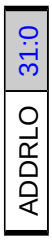


Table 11-113 Fields for Register: MAC_Address0_Low

Bits	Name	Memory Access	Description
31:0	ADDRLO	R/W	MAC Address0[31:0]. This field contains the lower 32 bits of 0th 6-byte MAC address. The MAC uses this field of the MAC_Address0_Low register for inserting in Destination Address field of the Transmit Flow Control (Pause) Packets. **Value After Reset:** 0xffffffff **Exists:** Always

11.1.110 MAC_Address(#i)_High (for i = 1; i <= DWCXG_ADD_MAC_ADDR_REG)

- **Description:** The MAC_Address1_High register holds the upper 16 bits of the 1th 6-byte MAC address of the station. The first DA byte that is received on the (X)GMII interface corresponds to the LS byte (Bits [7:0]) of the MAC_Address1_Low register. For example, if 0x112233445566 is received (0x11 in lane 0 of the first column) on the (X)GMII as the destination address, then the MAC_Address1 register [47:0] is compared with 0x665544332211.
  If the MAC address registers are configured to be double-synchronized to the (X)GMII clock domain, then the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in big-endian mode) of the MAC_Address1_Low register are written, so MAC_Address1_High register must be written prior to MAC_Address1_Low register. For proper synchronization updates, the consecutive writes to MAC_Address1_Low register must be performed after at least four clock cycles of the slowest clock between the source and the destination clock domains.
- **Size:** 32 bits
- **Offset:** (0x0008*i)+0x0300
- **Exists:** (DWCXG_ADD_MAC_ADDR_REG>0)

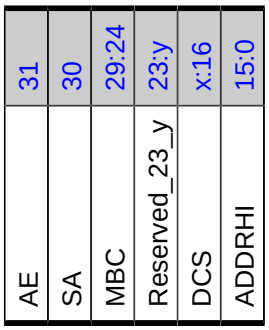


Table 11-114 Fields for Register: MAC_Address(#i)_High (for i = 1; i <= DWCXG_ADD_MAC_ADDR_REG)

Bits	Name	Memory Access	Description
31	AE	R/W	Address Enable. ■ 1: The address filter module uses the 1th MAC address for perfect filtering. ■ 0: The address filter module ignores the address for filtering. **Value After Reset:** 0x0 **Exists:** Always
30	SA	R/W	Source Address. ■ 1: The MAC Address1[47:0] is used to compare with the SA fields of the received packet. ■ 0: The MAC Address1[47:0] is used to compare with the DA fields of the received packet. **Value After Reset:** 0x0 **Exists:** Always

Table 11-114 Fields for Register: MAC_Address(#i)_High (for i = 1; i <= DWCXG_ADD_MAC_ADDR_REG) (continued)

Bits	Name	Memory Access	Description
29:24	MBC	R/W	Mask Byte Control. These bits are mask control bits for comparing each of the MAC Address bytes. When set high, the MAC does not compare the corresponding byte of received DA or SA with the contents of MAC Address1 registers. Each bit controls the masking of the bytes as follows: ■ Bit 29: MAC Address High [15:8] ■ Bit 28: MAC Address High [7:0] ■ Bit 27: MAC Address Low[31:24] ■ ... ■ Bit 24: MAC Address Low[7:0] You can filter a group of addresses (known as group address filtering) by masking one or more bytes of the address. **Value After Reset:** 0x0 **Exists:** Always
23:y	Reserved_23_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_BN_RCH_BN_RCHL_W + 16
x:16	DCS	R/W	DMA Channel Select. If the DDS bit of MAC_Extended_Configuration register is not set: This field contains the binary representation of the DMA Channel number to which an Rx packet whose DA matches the MAC Address1 content is routed. If the DDS bit of MAC_Extended_Configuration register is set: The XDCS field of MAC_DChSel_IndReg register contains the one-hot representation of one or more DMA Channel numbers to which an Rx packet whose DA matches the MAC Address1 content is routed. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_RX_CH>1) **Range Variable[x]:** DWCXG_BN_RCH_BN_RCHL_W + 15
15:0	ADDRHI	R/W	MAC Address1[47:32]. This field contains the upper 16 bits[47:32] of the 1th 6-byte MAC address. **Value After Reset:** 0xffff **Exists:** Always

11.1.111 MAC_Indir_Access_Ctrl

- **Description:** This register provides the Indirect Access control and status for MAC_(#MSEL)_IndReg(#AOFF) registers.
- **Size:** 32 bits
- **Offset:** 0x700
- **Exists:**
  DWCXG_DDUP||DWCXG_PC||DWCXG_EXT_DAH_EN||DWCXG_EXT_ROVTL_EN||DWCXG_EXT_RIVTL_EN

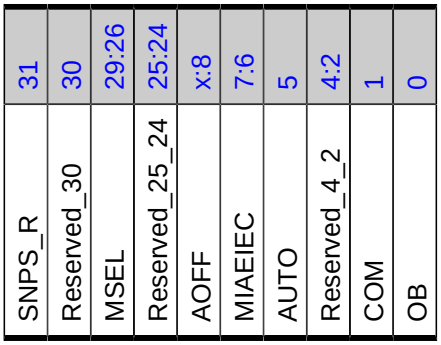


Table 11-115 Fields for Register: MAC_Indir_Access_Ctrl

Bits	Name	Memory Access	Description
31	SNPS_R	R/W	Synopsys Reserved This bit must be set to 0. This field is reserved for Synopsys Internal use, and must always be set to 0 unless instructed by Synopsys. Setting any bit to 1 might cause unexpected behavior in the IP. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30	Reserved_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-115 Fields for Register: MAC_Indir_Access_Ctrl (continued)

Bits	Name	Memory Access	Description
29:26	MSEL	R/W	Mode Select This field is used in indirect access of MAC_(#MSEL)_IndReg(#AOFF) registers. This field must be set along with initiation of read/write to MAC_(#MSEL)_IndReg(#AOFF) registers and must not be changed until the OB is reset. Values: ■ 0x0: Reserved for future use. (DChSel (DMA_Channel_Select)) ■ 0x1: PCCtrl (Policing_Counter_Control) ■ 0x2: PCntr (Policing_Counter) ■ 0x3: DPCSel (DA_Policing_Pounter_Select) ■ 0x4: VPCSel (VLAN_Policing_Counter_Select) ■ 0x5: LPCSel (L3L4_Policing_Counter_Select) ■ 0x6: FPCSel (Additional_Event_Policing_Counter_Select) ■ 0x7: PCStatus (Policing_Counter_Status for DA, VLAN, L3L4 and Additional_Event) ■ 0x8: EXT_DAH (External DA Hashtable) ■ 0x9: EXT_ROVTL (External Rx Outer VLAN Lookup) ■ 0xA: EXT_RIVTL (External Rx Inner VLAN Lookup) ■ 0xB: L2TLSel (L2 2Tuple Lookup) ■ 0xD - 0xF: Reserved for future use. **Value After Reset:** 0x0 **Exists:** Always
25:24	Reserved_25_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-115 Fields for Register: MAC_Indir_Access_Ctrl (continued)

Bits	Name	Memory Access	Description
x:8	AOFF	R/W	Address Offset This field is used in indirect access of MAC_(#MSEL)_IndReg(#AOFF) registers. This field must be set along with initiation of read/write to MAC_(#MSEL)_IndReg(#AOFF) registers and must not be changed until the OB is reset. **Note:** Supported maximum value of AOFF are specific to MSEL. Based on maximum value remaining bits are not implemented and are reserved. AOFF Values: ■ 0x0 : IndReg0 (Indirect register 0) ■ ... ■ 0xFFFF : IndReg65535 **Value After Reset:** 0x0 **Exists:** ([<functionof> max(DWCXG_DDUP_IACC_ADDR,DWCXG_PC_IACC_ADDR,DWCXG_L2EXT_DAH_TBL_DCNT,DWCXG_L2EXT_ROVTL_DCNT,DWCXG_L2EXT_RIVTL_DCNT)])>0 **Range Variable[x]:** DWCXG_MAC_IACC_ADDR + 7
7:6	MIAEIEC	R/W	MAC Indirect Access ECC Inject Error Control This field is valid only when EXT_DAH, EXT_ROVTL or EXT_RIVTL is selected in the MSEL field of this register. **Note:** IP does error injection to respective entry's bit location based on the programming. For more detail, see the Automotive Safety Features section. Following errors are inserted based on the value encoded in this field while reading respective registers. **Values:** ■ 0x0 (NO_ERR): No error injection ■ 0x1 (D_1BIT): Insert 1 bit ECC error in data or ecc bits ■ 0x2 (D_2BIT): Insert 2 bit ECC error in data or ecc bits ■ 0x3 (A_1BIT): Insert 1 bit ECC error in address **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ECC&&(DWCXG_EXT_DAH_EN||DWCXG_EXT_RVTL_EN)

Table 11-115 Fields for Register: MAC_Indir_Access_Ctrl (continued)

Bits	Name	Memory Access	Description
5	AUTO	R/W	Auto increment ■ When Auto increment is set, AOFF is incremented by 1 and care must be taken by software not to cause a wrap condition. ■ Byte wise read/write is not supported when auto increment is enabled. **Value After Reset:** 0x0 **Exists:** Always
4:2	Reserved_4_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	COM	R/W	Command type This bit indicates if the current register access is a read or a write. When set, it indicates a read operation. When reset, it indicates a write operation. **Values:** ■ 0x0 (WRITE): Write operation ■ 0x1 (READ): Read operation **Value After Reset:** 0x0 **Exists:** Always
0	OB	R/W	Operation Busy This bit is set along with a read or write command for initiating the indirect access to MAC_(#MSEL)_IndReg(#AOFF) registers. This bit is reset when the read or write command to MAC_(#MSEL)_IndReg(#AOFF) register is complete. The next indirect register access can be initiated only after this bit is reset. During a write operation, the bit is reset only after the data has been written into MAC_(#MSEL)_IndReg(#AOFF) register. During a read operation, the data must be read from MAC_Indir_Access_Data register only after this bit is reset. To initiate an indirect access, this bit must be set at last. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.112 MAC_Indir_Access_Data

- **Description:** This register holds the read/write data for Indirect Access of MAC_(#MSEL)_IndReg(#AOFF) registers.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** DWCXG_DDUP||DWCXG_PC||DWCXG_EXT_DAH_EN||DWCXG_EXT_ROVTL_EN||DWCXG_EXT_RIVTL_EN

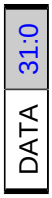


Table 11-116 Fields for Register: MAC_Indir_Access_Data

Bits	Name	Memory Access	Description
31:0	DATA	R/W	This field contains data to read/write for Indirect address access associated with MAC_Indir_Access_Ctrl register. **Value After Reset:** 0x0 **Exists:** Always

11.1.113 MAC_DChSel_IndReg(#i) (for i = 0; i <= DWCXG_ADD_MAC_ADDR_REG+1)

- **Description:** This register contains DCS (XDCS) one hot encoded value of receive channel selection for multicast/broadcast.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** (DWCXG_NUM_DMA_RX_CH>1)&&(DWCXG_DDUP)&&(!SNPS_RSVDPARAM_50)&&(DWCXG_ADD_MAC_ADDR_REG+1>0)

Table 11-117 Fields for Register: MAC_DChSel_IndReg(#i) (for i = 0; i <= DWCXG_ADD_MAC_ADDR_REG+1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 32-((DWCXG_NUM_DMA_RX_CH>16)?16:DWCXG_NUM_DMA_RX_CH) + ((DWCXG_NUM_DMA_RX_CH>16)?16:DWCXG_NUM_DMA_RX_CH) - 1 **Range Variable[y]:** ((DWCXG_NUM_DMA_RX_CH>16)?16:DWCXG_NUM_DMA_RX_CH)
x:0	XDCS	R/W	DMA Channel Select This field contains one hot encoded DMA receive channel selection for MAC Address registers to forward the received MC/BC packet. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_NUM_DMA_RX_CH>16)?16:DWCXG_NUM_DMA_RX_CH) - 1

11.1.114 MAC_PCCtrl_IndReg#(i) (for i = 0; i <= @DWCXG_NUM_PC-1)

- **Description:** This register contains control information for policing counters associated with MAC_(#MSEL)_(#AOFF) registers.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** DWCXG_PC && (DWCXG_NUM_PC > 0)

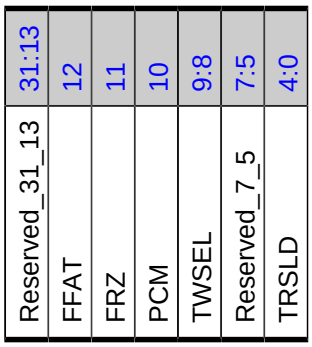


Table 11-118 Fields for Register: MAC_PCCtrl_IndReg#(i) (for i = 0; i <= @DWCXG_NUM_PC-1)

Bits	Name	Memory Access	Description
31:13	Reserved_31_13	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
12	FFAT	R/W	Forced Filter fail After Threshold When this bit is set and the corresponding counter crosses the threshold, all the filter events mapped to that counter is treated as fail. Packets are treated as fail from the time threshold is crossed, until the counter is cleared. **Value After Reset:** 0x0 **Exists:** Always
11	FRZ	R/W	Freeze Enable This field indicates counter freeze when end of time window is encountered **Values:** ■ 0x0: Restarts the counter on next start of time window Free running counter ■ 0x1: Retains counter value till corresponding counter is read, counts restart from the start of the next time window. **Value After Reset:** 0x0 **Exists:** Always

Table 11-118 Fields for Register: MAC_PCCtrl_IndReg#(i) (for i = 0; i <= @DWCXG_NUM_PC-1) (continued)

Bits	Name	Memory Access	Description
10	PCM	R/W	Policing Counter Mode **Values:** ■ 0x0: Free running counter. In this mode, counter is incremented irrespective of time window. ■ 0x1: Time window-based counter. In this mode, counter is incremented within the provided time window. **Value After Reset:** 0x0 **Exists:** Always
9:8	TWSEL	R/W	Time window Select This field is used to select any one of the time window from sbd_pc_tim_wdw_i[3:0] input, for counter increment. ■ 00: Time window 0 ■ 01: Time window 1 ■ 10: Time window 2 ■ 11: Time window 3 **Value After Reset:** 0x0 **Exists:** Always
7:5	Reserved_7_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4:0	TRSLD	R/W	Threshold set for Policing Counter This field indicates programmed filter event count threshold set for the policing counter. ■ 00000: 1(2^0) ■ 00001: 2(2^1) ■ ..... ■ 11111: 32(2^31) **Value After Reset:** 0x0 **Exists:** Always

11.1.115 MAC_PCntr_IndReg#(i) (for i = 0; i <= @DWCXG_NUM_PC-1)

- **Description:** This register contains count value of policing counters.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** DWCXG_PC && (DWCXG_NUM_PC > 0)

Table 11-119 Fields for Register: MAC_PCntr_IndReg#(i) (for i = 0; i <= @DWCXG_NUM_PC-1)

Bits	Name	Memory Access	Description
31:0	PCNT	R/W	Policing Count Value This field contains count value for policing counters. The counter is incremented within the start and end of the time window. Filter events results in increment of count. ■ Time window mode: When FRZ is not set, the counter value is cleared when start of the time window is detected. When FRZ is set, the counter is cleared on read. ■ Free running mode: Counter is cleared on read irrespective of the value of the FRZ bit **Value After Reset:** 0x0 **Exists:** Always

11.1.116 MAC_DPCSel_IndReg(#i) (for i = 0; i <= max((DWCXG_ADD_MAC_ADDR_REG+1),DWCXG_NUM_PC))

- **Description:** This register contains policing counter number corresponding to each DA or SA filter event.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** DWCXG_PC||((DWCXG_RCHLIST_CNT>0)&&SNPS_RSVDPARAM_50)

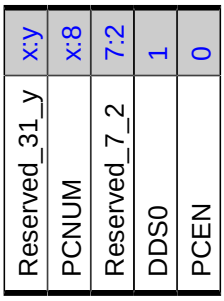


Table 11-120 Fields for Register: MAC_DPCSel_IndReg(#i) (for i = 0; i <= max((DWCXG_ADD_MAC_ADDR_REG+1),DWCXG_NUM_PC))

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_PC==16?20:19 + 8+DWCXG_LOG2_NUM_PC - 1 **Range Variable[y]:** DWCXG_LOG2_NUM_PC + 8
x:8	PCNUM	R/W	Policing Counter Number This field specifies the counter number selected for corresponding DA/SA address filter event. **Value After Reset:** 0x0 **Exists:** DWCXG_PC&&(DWCXG_NUM_PC>0) **Range Variable[x]:** DWCXG_LOG2_NUM_PC + 7
7:2	Reserved_7_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-120 Fields for Register: MAC_DPCSel_IndReg(#i) (for i = 0; i <= max((DWCXG_ADD_MAC_ADDR_REG+1),DWCXG_NUM_PC)) (continued)

Bits	Name	Memory Access	Description
1	DDS0	R/W	DA Duplication Select. This bit controls whether the DCS field in MAC_Address${i}_High register directly or indirectly gives the destination DMA Channels to which the ingress packet is directed to, when there is a DA match in this MAC_Address${i} register. ■ 1: The DCS value in MAC_Address${i}_High register, is the pointer to the entry in the Receive Channel List lookup table (MTL_RCHLIST_IndReg(#j)) containing the destination Receive DMA Channels to which this packet is directed to. ■ 0: The DCS value in MAC_Address${i}_High register directly gives the destination Receive DMA Channel number to which this packet is directed to. **Value After Reset:** 0x0 **Exists:** (DWCXG_RCHLIST_CNT>0)&&SNPS_RSVDPARAM_50&&((DWCXG_ADD_MAC_ADDR_REG+1)>0)
0	PCEN	R/W	Policing Counter Enable When set enables the policing counter indicated by the PCNUM field, to be incremented due to the corresponding DA/SA address filter event. **Value After Reset:** 0x0 **Exists:** DWCXG_PC&&(DWCXG_NUM_PC>0)

11.1.117 MAC_VPCSel_IndReg(#i) (for i = 0; i <= DWCXG_NRVF_PC-1)

- **Description:** This register contains policing counter number corresponding to each VLAN filter event.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** DWCXG_PC&&(DWCXG_NRVF_PC> 0)

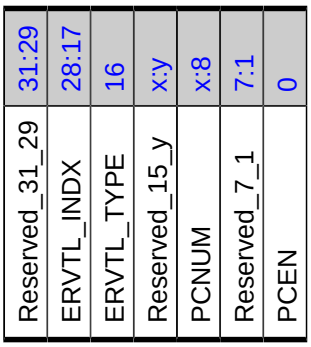


Table 11-121 Fields for Register: MAC_VPCSel_IndReg(#i) (for i = 0; i <= DWCXG_NRVF_PC-1)

Bits	Name	Memory Access	Description
31:29	Reserved_31_29	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
28:17	ERVTL_INDX	R/W	External Rx VLAN Lookup index This field specifies the Rx VLAN Lookup index for ERVTL_TYPE, where filtering result is available. **Value After Reset:** 0x0 **Exists:** DWCXG_ERVFE&&(DWCXG_NRVF==4096)
16	ERVTL_TYPE	R/W	External Rx VLAN Lookup Type This field specifies the type of Rx VLAN lookup where filtering result is available. **Values:** ■ 0x0: Outer Rx VLAN Lookup ■ 0x1: Inner Rx VLAN Lookup **Value After Reset:** 0x0 **Exists:** (DWCXG_EXT_RVTL_EN&&DWCXG_DOUBLE_VLAN_EN)

Table 11-121 Fields for Register: MAC_VPCSel_IndReg(#i) (for i = 0; i <= DWCXG_NRVF_PC-1) (continued)

Bits	Name	Memory Access	Description
x:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_PC==16?4:3 + 8+DWCXG_LOG2_NUM_PC - 1 **Range Variable[y]:** DWCXG_LOG2_NUM_PC + 8
x:8	PCNUM	R/W	Policing Counter Number This field specifies the counter number selected for corresponding VLAN filter event. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_LOG2_NUM_PC + 7
7:1	Reserved_7_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	PCEN	R/W	Policing Counter Enable When set enables the policing counter indicated by the PCNUM field **Value After Reset:** 0x0 **Exists:** Always

11.1.118 MAC_LPCSel_IndReg(#i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM_PC+1)

- **Description:** This register contains policing counter number corresponding to each L3L4 filter event.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** DWCXG_PC && DWCXG_L3_L4_FILTER_EN && (DWCXG_L3_L4_FILTER_NUM_PC > 0)

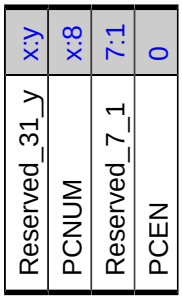


Table 11-122 Fields for Register: MAC_LPCSel_IndReg(#i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM_PC+1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_PC==16?20:19 + 8+DWCXG_LOG2_NUM_PC - 1 **Range Variable[y]:** DWCXG_LOG2_NUM_PC + 8
x:8	PCNUM	R/W	Policing Counter Number This field specifies the counter number selected for corresponding L3_L4 filter event. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_LOG2_NUM_PC + 7
7:1	Reserved_7_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	PCEN	R/W	Policing Counter Enable When set enables the policing counter indicated by the PCNUM field **Value After Reset:** 0x0 **Exists:** Always

11.1.119 MAC_EXT_DAH_IndReg(#i)_(#j) (for j = 0; j <= EXT_DAH_ENT_DCNT-1)

- **Description:** The MAC_EXT_DAH_IndReg0_0 registers, which are accessible through MAC_Indir_Access_* register. Each register is used to access 32-bit data word of respective DA Hash table entry implemented in external RXF Memory. Each 32-bit data word of the DA Hash table entry is accessible with ((m * Number of 32 bit data word counts of External DA Hashtable Entries) + n) address. Where m, 0 to (Destination Address Hash Table Size -1) represent DA Hash table entry number and n, 0 to (Number of 32 bit Dataword Counts of External DA Hashtable Entries -1) represent respective DA Hash table entry's 32-bit data word number for LSB.
  Note: If the RXF memory is configured to have ECC safety protection then, on every 32-bit data word write, IP reads all other data words of the same entry, update requested data word, recalculate ECC bits and update it to same entry.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:**
  DWCXG_EXT_DAH_EN&&(DWCXG_HASH_TABLE>0)&&(DWCXG_EXT_DAH_ENT_DCNT>0)

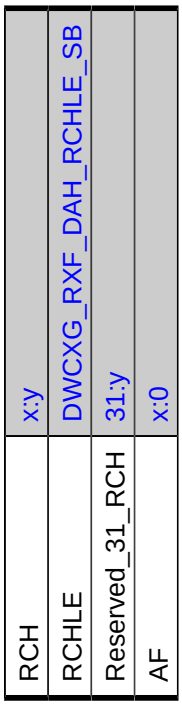


Table 11-123 Fields for Register: MAC_EXT_DAH_IndReg(#i)_(#j) (for j = 0; j <= EXT_DAH_ENT_DCNT-1)

Bits	Name	Memory Access	Description
x:y	RCH	R/W	Receive Channel This field has dual interpretation based on the value of the RCHLE field. It represents either receive-channel number or receive-channel list number for packet routing. If RCHLE field is not exist, then solely this field represent receive-channel number for routing. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RXF_DAH_RCH_SZ + DWCXG_RXF_DAH_RCH_SB - 1 **Range Variable[y]:** DWCXG_RXF_DAH_RCH_SB
DWCXG_RXF_DAH_RCHLE_SB	RCHLE	R/W	Receive Channel List Enable This field is used for RCH field interpretation. **Values:** ■ 0x0 (RCH_NO): RCH field represent Receive Channel number ■ 0x1 (RCHLIST_NO): RCH field represent Receive Channel List number of MTL_RCHLIST_IndReg registers **Value After Reset:** 0x0 **Exists:** DWCXG_RCHLIST_CNT>0
31:y	Reserved_31_RCH	R	Reserved **Value After Reset:** 0x0 **Exists:** (DWCXG_EXT_DAH_EN ? [<functionof> (DWCXG_RXF_DAH_RCH_SB + DWCXG_RXF_DAH_RCH_SZ)] : 0)<32 **Range Variable[y]:** DWCXG_RXF_DAH_DW
x:0	AF	R/W	Accept Frame This field decides whether to accept or reject packet. **Values:** ■ 0x0 (DROP): Drop Frame ■ 0x1 (ACPT): Accept Frame **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (DWCXG_RXF_DAH_AF_SZ) - 1

11.1.120 MAC_EXT_ROVTL_IndReg(#i)_(#j) (for j = 0; j <= EXT_RVTL_ENT_DCNT-1)

- **Description:** The MAC_EXT_ROVTL_IndReg0_0 registers, which are accessible through MAC_Indir_Access_* register. Each register is used to access 32-bit data word of respective Receive Outer VLAN Tag Lookup's entry implemented in external RXF Memory. Each 32-bit data word of Receive Outer VLAN Tag Lookup's entry is accessible with ((m * Number of 32 bit Dataword Counts of External Rx VLAN Lookup Entries) + n) address. Where m, 0 to (Number of VLAN Tag Filters-1) represent Receive Outer VLAN Tag Lookup entry number and n, 0 to (Number of 32 bit Dataword Counts of External Rx VLAN Lookup Entries -1) represent respective Receive Outer VLAN Tag Lookup entry's 32-bit data word number from LSB.
  Note: If the RXF memory is configured to have ECC safety protection then, on every 32-bit data word write, IP reads all other data words of same entry, update requested data word, recalculate ECC bits and update it to same entry.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** DWCXG_EXT_ROVTL_EN&&(DWCXG_NRVF>0)&&(DWCXG_EXT_RVTL_ENT_DCNT>0)

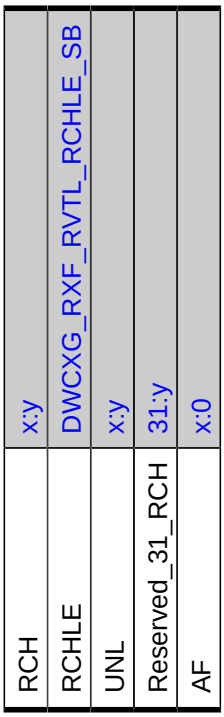


Table 11-124 Fields for Register: MAC_EXT_ROVTL_IndReg(#i)_(#j) (for j = 0; j <= EXT_RVTL_ENT_DCNT-1)

Bits	Name	Memory Access	Description
x:y	RCH	R/W	Receive Channel This field has dual interpretation based on the value of the RCHLE field. It represents either receive-channel number or receive-channel list number for packet routing. If RCHLE field is not exist, then solely this field represent receive-channel number for routing. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RXF_RVTL_RCH_SZ + DWCXG_RXF_RVTL_RCH_SB - 1 **Range Variable[y]:** DWCXG_RXF_RVTL_RCH_SB
DWCXG_RXF_RVTL_RCHLE_SB	RCHLE	R/W	Receive Channel List Enable This field is used for RCH field interpretation. **Values:** ■ 0x0 (RCH_NO): RCH field represent Receive Channel number ■ 0x1 (RCHLIST_NO): RCH field represent Receive Channel List number of MTL_RCHLIST_IndReg registers **Value After Reset:** 0x0 **Exists:** DWCXG_RCHLIST_CNT>0
x:y	UNL	R/W	Use Next Lookup This field is only valid when AF is 0, to indicate whether to use next lookup or not. **Note:** Refer respective filter section for detailed information. **Values:** ■ 0x0 (DUNL): don't use next lookup ■ 0x1 (UNL): use next (inner vlan) lookup if EDVLP is 1 **, otherwise bypass external VLAN Tag lookup-based filtering & routing. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RXF_RVTL_UNL_SZ + DWCXG_RXF_RVTL_UNL_SB - 1 **Range Variable[y]:** DWCXG_RXF_RVTL_UNL_SB

Table 11-124 Fields for Register: MAC_EXT_ROVTL_IndReg(#i)_(#j) (for j = 0; j <= EXT_RVTL_ENT_DCNT-1) (continued)

Bits	Name	Memory Access	Description
31:y	Reserved_31_RCH	R	Reserved **Value After Reset:** 0x0 **Exists:** (DWCXG_EXT_RVTL_EN ? [<functionof> (DWCXG_RXF_RVTL_RCH_SB + DWCXG_RXF_RVTL_RCH_SZ)] : 0)<32 **Range Variable[y]:** DWCXG_RXF_RVTL_DW
x:0	AF	R/W	Accept Frame This field decides whether to accept or reject packet. **Values:** ■ 0x0 (DROP): Drop Frame if UNL is 0 ■ 0x1 (ACPT): Accept Frame **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RXF_RVTL_AF_SZ - 1

11.1.121 MAC_EXT_RIVTL_IndReg(#i)_(#j) (for j = 0; j <= EXT_RVTL_ENT_DCNT-1)

- **Description:** The MAC_EXT_RIVTL_IndReg0_0 registers, which are accessible through MAC_Indir_Access_* register. Each register is used to access 32-bit data word of respective Receive Inner VLAN Tag Lookup's entry implemented in external RXF Memory. Each 32-bit data word of Receive Inner VLAN Tag Lookup's entry is accessible with ((m * Number of 32 bit Dataword Counts of External Rx VLAN Lookup Entries) + n) address. Where m, 0 to (Number of VLAN Tag Filters-1) represent Receive Inner VLAN Tag Lookup entry number and n, 0 to (Number of 32 bit Dataword Counts of External Rx VLAN Lookup Entries -1) represent respective Receive Inner VLAN Tag Lookup entry's 32-bit data word number from LSB.
  Note: If the RXF memory is configured to have ECC safety protection then, on every 32-bit data word write, IP reads all other data words of same entry, update requested data word, recalculate ECC bits and update it to same entry.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** DWCXG_EXT_RIVTL_EN&&(DWCXG_NRVF>0)&&(DWCXG_EXT_RVTL_ENT_DCNT>0)

x:y	DWCXG_RXF_RVTL_RCHLE_SB	x:y	31:y	x:0
RCH	RCHLE	UNL	Reserved_31_RCH	AF

Table 11-125 Fields for Register: MAC_EXT_RIVTL_IndReg(#i)_(#j) (for j = 0; j <= EXT_RVTL_ENT_DCNT-1)

Bits	Name	Memory Access	Description
x:y	RCH	R/W	Receive Channel This field has dual interpretation based on the value of the RCHLE field. It represents either receive-channel number or receive-channel list number for packet routing. If RCHLE field is not exist, then solely this field represent receive-channel number for routing. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RXF_RVTL_RCH_SZ + DWCXG_RXF_RVTL_RCH_SB - 1 **Range Variable[y]:** DWCXG_RXF_RVTL_RCH_SB
DWCXG_RXF_RVTL_RCHLE_SB	RCHLE	R/W	Receive Channel List Enable This field is used for RCH field interpretation. **Values:** ■ 0x0 (RCH_NO): RCH field represent Receive Channel number ■ 0x1 (RCHLIST_NO): RCH field represent Receive Channel List number of MTL_RCHLIST_IndReg registers **Value After Reset:** 0x0 **Exists:** DWCXG_RCHLIST_CNT>0
x:y	UNL	R/W	Use Next Lookup This field is only valid when AF is 0, to indicate whether to use next lookup or not. **Values:** ■ 0x0 (DUNL): don't use next lookup ■ 0x1 (UNL): use next lookup means bypass external VLAN Tag lookup-based filtering & routing. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RXF_RVTL_UNL_SZ + DWCXG_RXF_RVTL_UNL_SB - 1 **Range Variable[y]:** DWCXG_RXF_RVTL_UNL_SB
31:y	Reserved_31_RCH	R	Reserved **Value After Reset:** 0x0 **Exists:** (DWCXG_EXT_RVTL_EN ? [<functionof> (DWCXG_RXF_RVTL_RCH_SB + DWCXG_RXF_RVTL_RCH_SZ)] : 0)<32 **Range Variable[y]:** DWCXG_RXF_RVTL_DW

Table 11-125 Fields for Register: MAC_EXT_RIVTL_IndReg(#i)_(#j) (for j = 0; j <= EXT_RVTL_ENT_DCNT-1) (continued)

Bits	Name	Memory Access	Description
x:0	AF	R/W	Accept Frame This field decides whether to accept or reject packet. **Values:** ■ 0x0 (DROP): Drop Frame if UNL is 0 ■ 0x1 (ACPT): Accept Frame **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RXF_RVTL_AF_SZ - 1

11.1.122 MAC_FPCSel_IndReg0

- **Description:** This register contains policing counter number for events other than the Filter Pass events which is given below
  1. Filter Fail
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** (DWCXG_PC)

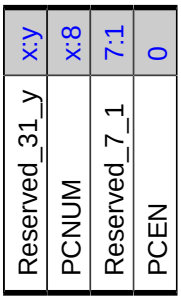


Table 11-126 Fields for Register: MAC_FPCSel_IndReg0

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_PC==16?20:19 + 8+DWCXG_LOG2_NUM_PC - 1 **Range Variable[y]:** DWCXG_LOG2_NUM_PC + 8
x:8	PCNUM	R/W	Policing Counter Number This field specifies the counter number selected for "Filter fail" event. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_LOG2_NUM_PC + 7
7:1	Reserved_7_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	PCEN	R/W	Policing Counter Enable When set enables the policing counter indicated by the PCNUM field **Value After Reset:** 0x0 **Exists:** Always

11.1.123 MAC_PCStatus_DA_IndReg#(i) (for i = 0; i <= DWCXG_ADD_MAC_ADDR_REG_PC_DCNT-1)

- **Description:** This register contains pass status for up to 32 DA/SA filter where each bit corresponds to specific DA/SA filter event.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** (DWCXG_PC&&((DWCXG_ADD_MAC_ADDR_REG_PC+1)>(32*0)))

Table 11-127 Fields for Register: MAC_PCStatus_DA_IndReg#(i) (for i = 0; i <= DWCXG_ADD_MAC_ADDR_REG_PC_DCNT-1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (((DWCXG_ADD_MAC_ADDR_REG>63) ? 63 : DWCXG_ADD_MAC_ADDR_REG+1)<(32*(0+1))) **Range Variable[x]:** (32*(0+1))-DWCXG_ADD_MAC_ADDR_REG_PC-1 + ((DWCXG_ADD_MAC_ADDR_REG_PC+1)-(32*0)) - 1 **Range Variable[y]:** ((DWCXG_ADD_MAC_ADDR_REG_PC+1)-(32*0))
x:0	FEPS	R/W	Filter Event Pass Status When set, indicates at least one DA/SA filter event is passed. Each bit represents individual DA/SA filter event. Access restriction applies. This field is cleared when read (or written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_ADD_MAC_ADDR_REG_PC+1)>(32*(0+1)))?32:(((DWCXG_ADD_MAC_ADDR_REG_PC+1)>(32*0))?((DWCXG_ADD_MAC_ADDR_REG_PC+1)-(32*0)):1) - 1

11.1.124 MAC_PCStatus_VLAN_IndReg#(i) (for i = 0; i <= DWCXG_NRVF_PC_DCNT-1)

- **Description:** This register contains pass status for up to 32 VLAN filter where each bit corresponds to specific VLAN filter event.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** (DWCXG_PC&&((DWCXG_NRVF_PC)>(32*0)))

Table 11-128 Fields for Register: MAC_PCStatus_VLAN_IndReg#(i) (for i = 0; i <= DWCXG_NRVF_PC_DCNT-1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (((DWCXG_NRVF>64) ? 64 : DWCXG_NRVF)<(32*(0+1))) **Range Variable[x]:** (32*(0+1))-DWCXG_NRVF_PC + ((DWCXG_NRVF_PC)-(32*0)) - 1 **Range Variable[y]:** ((DWCXG_NRVF_PC)-(32*0))
x:0	FEPS	R/W	Filter Event Pass Status When set, indicates at-least one VLAN filter event is passed. Each bit represents individual VLAN filters. Access restriction applies. This field is cleared when read (or written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_NRVF_PC)>(32*(0+1)))?32: (((DWCXG_NRVF_PC)>(32*0))?((DWCXG_NRVF_PC)-(32*0)):1) - 1

11.1.125 MAC_PCStatus_L3L4_IndReg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM_PC_DCNT-1)

- **Description:** This register contains pass status for up to 32 L3_L4 filter where each bit corresponds to specific L3_L4 address filter event.
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** (DWCXG_PC&&DWCXG_L3_L4_FILTER_EN&&((DWCXG_L3_L4_FILTER_NUM_PC)>(32*0)))

Table 11-129 Fields for Register: MAC_PCStatus_L3L4_IndReg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM_PC_DCNT-1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (((DWCXG_L3_L4_FILTER_NUM>64) ? 64 : DWCXG_L3_L4_FILTER_NUM)<(32*(0+1))) **Range Variable[x]:** (32*(0+1))-DWCXG_L3_L4_FILTER_NUM_PC + ((DWCXG_L3_L4_FILTER_NUM_PC)-(32*0)) - 1 **Range Variable[y]:** ((DWCXG_L3_L4_FILTER_NUM_PC)-(32*0))
x:0	FEPS	R/W	Filter Event Pass Status When set, indicates at least one L3_L4 filter event is passed. Each bit represents individual L3_L4 filter. Access restriction applies. This field is cleared when read (or written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_L3_L4_FILTER_NUM_PC)>(32*(0+1)))?32: (((DWCXG_L3_L4_FILTER_NUM_PC)>(32*0))?((DWCXG_L3_L4_FILTER_NUM_PC)-(32*0)):1) - 1

11.1.126 MAC_PCStatus_Filter_Event_IndReg

- **Description:** This register contains status for the events monitored using policing counters apart from Filter Pass Events which is given below
  1. Filter_Fail
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** (DWCXG_PC)

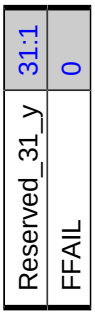


Table 11-130 Fields for Register: MAC_PCStatus_Filter_Event_IndReg

Bits	Name	Memory Access	Description
31:1	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	FFAIL	R/W	Filter Fail This bit is asserted when "Filter fail" event is set/triggered at least once. Access restriction applies. This bit is cleared when read (or written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always

11.1.127 MAC_L2TLSel_IndReg#(i) (for i = 0; i <= @SNPS_RSVDPARAM_28-1)

- **Description:** This set of registers forms the entries of the L2 2-Tuple Lookup Table for DA/SA & VLAN tuple match and option to link to FRP
- **Size:** 32 bits
- **Offset:** 0x704
- **Exists:** SNPS_RSVDPARAM_27

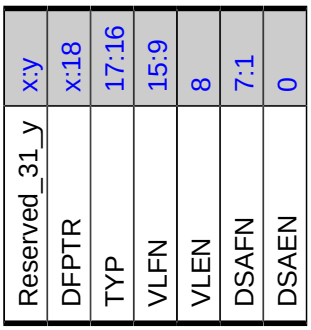


Table 11-131 Fields for Register: MAC_L2TLSel_IndReg#(i) (for i = 0; i <= @SNPS_RSVDPARAM_28-1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 32-(18+DWCXG_EFF_RXCHW) + 18+DWCXG_EFF_RXCHW - 1 **Range Variable[y]:** DWCXG_EFF_RXCHW + 18
x:18	DFPTR	R/W	DMA/FRP Pointer This field directly or indirectly provides the RX DMA number to which the ingress packet is to be directed to or the index of the FRP table which further process the packet header, as per the setting of TYP field. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_EFF_RXCHW + 17
17:16	TYP	R/W	Tuple Type This field initiates the action to be taken when the filters indicated by DSAFN & VLFN fields of this register (when enabled by DSAEN & VLEN respectively) generates a PASS result. **Values:** ■ 00: The ingress packet is PASS only if the DA/SA filter indicated by DSAFN and the VLAN filter indicated by VLFN gives a PASS result.

Table 11-131 Fields for Register: MAC_L2TLSel_IndReg#(i) (for i = 0; i <= @SNPS_RSVDPARAM_28-1) (continued)

Bits	Name	Memory Access	Description
			If both DSAEN and VLEN are set but only one of the filters indicated by DSAFN and VLFN gives a PASS, then the packet is considered to FAIL the 2-tuple filter. ■ 01: Same action as described for "00" plus further processing in FRP. The FRP processing starts from the entry indicated by DFPTR value. The valid range of DFPTR = 0 to (DWCXG_FRP_ENTRIES -1). This action/programming is valid only if DWCXG_FRP_EN is true ■ 10: Same action as described for "00" plus the additional function of classification. The 2-tuple PASS packet is forwarded to the RX DMA number indicated by DFPTR. The valid range of DFPTR = 0 to (DWCXG_NUM_DMA_RX_CH-1). This action/programming is valid only in DWCXG-AXI configuration. ■ 11: Same action as described for "00" plus the additional function of classification. The 2-tuple PASS packet is forwarded to the RX DMA(s) as indicated by the entry in MTL_RCHLIST_IndReg(#j) where j = DFPTR. The valid range of DFPTR = 0 to (DWCXG_RCHLIST_CNT-1). This action/programming is valid only in DWCXG-AXI configuration. **Value After Reset:** 0x0 **Exists:** Always
15:9	VLFN	R/W	L2 MAC VLAN Filter Number This field indicates the MAC_VLAN_Tag_Filter number whose match/filter result is to be evaluated further in this lookup table. **Value After Reset:** 0x0 **Exists:** Always
8	VLEN	R/W	L2 MAC VLAN Filter Enable When this bit is set to 1, VLFN gives the filter number (0 to DWCXG_NRVF-1) of the VLAN filter-set. When this bit is set to 0, the result of VLAN filter & VLFN is ignored in this entry. **Value After Reset:** 0x0 **Exists:** Always

Table 11-131 Fields for Register: MAC_L2TLSel_IndReg#(i) (for i = 0; i <= @SNPS_RSVDPARAM_28-1) (continued)

Bits	Name	Memory Access	Description
7:1	DSAFN	R/W	L2 MAC DA Filter Number This field indicates the MAC_Address DA Filter number whose match/filter result is to be evaluated further in this lookup table. **Value After Reset:** 0x0 **Exists:** Always
0	DSAEN	R/W	L2 MAC DA Filter Enable When this bit is set to 1, DSAFN field gives the filter number (0 to DWCXG_ADD_MAC_ADDR_REG) of the DA/SA filter-set. When this bit is reset, the result of DA/SA filter & DSAFN is ignored in this entry. **Value After Reset:** 0x0 **Exists:** Always

11.1.128 MAC_PCTH_Intr_Enable

- **Description:** This register contains threshold based interrupt enable for corresponding policing counters.
- **Size:** 32 bits
- **Offset:** 0x708
- **Exists:** (DWCXG_PC)

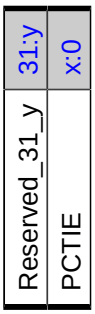


Table 11-132 Fields for Register: MAC_PCTH_Intr_Enable

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_PC<32 **Range Variable[y]:** DWCXG_NUM_PC
x:0	PCTIE	R/W	Policing Counter Threshold Interrupt Enable When set indicates the assertion of the interrupt is enabled for corresponding policing counters. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_PC - 1

11.1.129 MAC_PCTH_Intr_Status

- **Description:** This register contains threshold based interrupts for corresponding policing counters.
- **Size:** 32 bits
- **Offset:** 0x70c
- **Exists:** (DWCXG_PC)

Table 11-133 Fields for Register: MAC_PCTH_Intr_Status

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_PC<32 **Range Variable[y]:** DWCXG_NUM_PC
x:0	PCTIS	R/W	Policing Counter Threshold Interrupt Status This field is set when corresponding threshold-based interrupt is generated in corresponding policing counter Access restriction applies. This field is cleared when read (or written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_PC - 1

11.1.130 MAC_PCTW_Intr_Enable

- **Description:** This register contains enable bits for Time based interrupt.
- **Size:** 32 bits
- **Offset:** 0x730
- **Exists:** (DWCXG_PC)

Table 11-134 Fields for Register: MAC_PCTW_Intr_Enable

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3:0	TWIE	R/W	Time window Interrupt Enable When set indicates the assertion of the interrupt is enabled for corresponding Time window. **Value After Reset:** 0x0 **Exists:** Always

11.1.131 MAC_PCTW_Intr_Status

- **Description:** This register contains Time window based interrupt status.
- **Size:** 32 bits
- **Offset:** 0x734
- **Exists:** (DWCXG_PC)

Table 11-135 Fields for Register: MAC_PCTW_Intr_Status

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3:0	TWIS	R/W	Time window Interrupt Status This field is set when corresponding end of time window is detected. Access restriction applies. This field is cleared when read (or written to 1 when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always

11.1.132 MMC_Control

- **Description:** The MMC Control register establishes the operating mode of the management counters.
- **Size:** 32 bits
- **Offset:** 0x800
- **Exists:** (DWCXG_MMC_EN)

31	30:19	18:16	15:8	7	6	5:4	3	2	1	0
DRCHM	Reserved_30_19	PRMMCSEL	Reserved_15_8	CNTPRST	Reserved_6	MCT	MCF	RSTONRD	CNTSTOPRO	CNTRST

Table 11-136 Fields for Register: MMC_Control

Bits	Name	Memory Access	Description
31	DRCHM	R/W	This field disables receive channels mapping for certain set of Receive Frame/Priority/IPC MAC Management Counters on specific scenarios. By default, such mapping is enabled. Application should set this field to 1 to disable it. **Values:** ■ 0x0 (ENABLE): Receive Channels Mapping for RMMC-RCHM counters is enabled ■ 0x1 (DISABLE): Receive Channels Mapping for RMMC-RCHM counters is disabled **Value After Reset:** 0x0 **Exists:** DWCXG_RMMC_RCH_MAP
30:19	Reserved_30_19	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
18:16	PRMMCSEL	R/W	Priority Select This field selects the Priority Group. ■ 0: All the registers accesses between 0x08D0 to 0x08FC and 0x09D0 to 0x09FC correspond to Priority 0 ■ 1: All the registers accesses between 0x08D0 to 0x08FC and 0x09D0 to 0x09FC correspond to Priority 1 ■ 2: All the registers accesses between 0x08D0 to 0x08FC and 0x09D0 to 0x09FC correspond to Priority 2 ■ 3: All the registers accesses between 0x08D0 to 0x08FC and 0x09D0 to 0x09FC correspond to Priority 3 ■ 4: All the registers accesses between 0x08D0 to 0x08FC and 0x09D0 to 0x09FC correspond to Priority 4

Table 11-136 Fields for Register: MMC_Control (continued)

Bits	Name	Memory Access	Description
			■ 5: All the registers accesses between 0x08D0 to 0x08FC and 0x09D0 to 0x09FC correspond to Priority 5 ■ 6: All the registers accesses between 0x08D0 to 0x08FC and 0x09D0 to 0x09FC correspond to Priority 6 ■ 7: All the registers accesses between 0x08D0 to 0x08FC and 0x09D0 to 0x09FC correspond to Priority 7 **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_PRIORITY_EN
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7	CNTPRST	R/W	Counters Preset When this bit is set, all counters are initialized or preset to a value depending on Bits [5:4]. This bit is cleared after 1 clock cycle. This bit, along with Bits [5:4], is useful for debugging and testing the assertion of interrupts because of MMC counter becoming one-quarter-full/half-full/three-quarter-full. If this bit is set along with CNTRST bit, CNTRST bit has precedence. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
6	Reserved_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:4	MCT	R/W	MMC Counter Threshold Threshold value for the counter which, when crossed, causes the interrupt to be scheduled ■ 2'b00: Reserved ■ 2'b01: One-quarter the maximum value - X ■ 2'b10: One-half of the maximum value - X ■ 2'b11: Three-quarters of the maximum value - X Here, ■ X = 16 for all the Packet counters and ■ X = 2048 for all the Octet/Byte counters **Value After Reset:** 0x2 **Exists:** Always

Table 11-136 Fields for Register: MMC_Control (continued)

Bits	Name	Memory Access	Description
3	MCF	R/W	MMC Counter Freeze When set, this bit freezes the MMC counters to their current value. No MMC counter is updated for subsequent packet transmission or reception until this bit is reset to 0. If any MMC counter is read with the Reset on Read bit set, that counter is also cleared when this bit is set. **Value After Reset:** 0x0 **Exists:** Always
2	RSTONRD	R/W	Reset on Read When this bit is set, the MMC counters are reset to zero after a read. The counters are cleared when the least significant byte lane (bits[7:0]) is read. When the MMC module is enabled for the 64-bit counter mode, the lower and upper counters are cleared only when the least significant byte lane (bits[7:0]) of the corresponding counter is read. **Note:** When 64-bit counter mode is selected and RSTONRD is set, read the lower counter first followed by the corresponding upper counter. This is to ensure that only the lower 32-bit counter is updated after the first read operation and the software gets the original higher 32-bit counter and gets a consistent value. **Value After Reset:** 0x0 **Exists:** Always
1	CNTSTOPRO	R/W	Counter Stop Rollover When this bit is set, no MMC counter rolls over to zero after reaching the maximum count value. The counters are cleared when the least significant byte lane (bits[7:0]) is read after reaching the maximum count value. When the MMC module is enabled for the 64-bit counter mode, the lower and upper counters are cleared only when the least significant byte lane (bits[7:0]) of the upper counter is read after reaching the maximum count value. **Value After Reset:** 0x0 **Exists:** Always
0	CNTRST	R/W	Counters Reset When this bit is set, all counters are reset. This bit is cleared automatically after 1 clock cycle. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.133 MMC_Rx_Interrupt

- **Description:** The MMC Receive Interrupt Status register maintains the interrupts generated when receive statistic counters reach the threshold programmed in the MMC control register. When the MMC counter that caused the interrupt is read, the corresponding interrupt status bit is cleared. The least significant byte lane (bits[7:0]) of the counter must be read to clear the interrupt status bit. When the MMC module is enabled for 64-bit counter mode, the least significant byte lane (bits[7:0]) of the upper counter must be read to clear the interrupt status bit.
- **Size:** 32 bits
- **Offset:** 0x804
- **Exists:**
  ((DWCXG_MMC_EN_RX_ALL||DWCXG_MMC_RXFRMGB_CNT_EN||DWCXG_MMC_RXOCTG_CNT_EN||DWCXG_MMC_RXOCTGB_CNT_EN||DWCXG_MMC_RXBCASTG_CNT_EN||DWCXG_MMC_RXMCASTG_CNT_EN||DWCXG_MMC_RXCRCERR_CNT_EN||DWCXG_MMC_RXRUNTERR_CNT_EN||DWCXG_MMC_RXJABERR_CNT_EN||DWCXG_MMC_RXUNDERSZG_CNT_EN||DWCXG_MMC_RXOVERSZG_CNT_EN||DWCXG_MMC_RXADDR_RNG_CNT_EN||DWCXG_MMC_RXUCASTG_CNT_EN||DWCXG_MMC_RXLENERR_CNT_EN||DWCXG_MMC_RXOUTOFRNG_TYP_CNT_EN||DWCXG_MMC_RXPAUSEFRM_CNT_EN
  ||
  DWCXG_MMC_RXFIFOOVFL_CNT_EN||DWCXG_MMC_RXVLANFRGB_CNT_EN||DWCXG_MMC_RXWDGERR_CNT_EN||DWCXG_MMC_RXFRMDISGB_CNT_EN||DWCXG_MMC_RXOCTDISGB_CNT_EN||DWCXG_MMC_RXLPI_USEC_EN||DWCXG_MMC_RXLPI_TRANS_CNTR_EN||DWCXG_MMC_RXALGNERR_CNT_EN))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RXPRMMCIS	RRSCIS	SGFPIS	SGPPIS	RXALEPIS	RXLPITRCIS	RXLPIUSCIS	RXDISOCGBIS	RXDISPCGBIS	RXWDOGPIS	RXVLANGBPIS	RXFOVPIS	RXPAUSPIS	RXORANGEPIS	RXLENERPIS	RXUCGPIS	RX1024TMAXOCTGBPIS	RX512T1023OCTGBPIS	RX256T511OCTGBPIS	RX128T255OCTGBPIS	RX65T127OCTGBPIS	RX64OCTGBPIS	RXOSIZEGPIS	RXUSIZEGPIS	RXJABERPIS	RXRUNTPIS	RXCRCERPIS	RXMCGPIS	RXBCGPIS	RXGOCTIS	RXGBOCTIS	RXGBPKTIS

Table 11-137 Fields for Register: MMC_Rx_Interrupt

Bits	Name	Memory Access	Description
31	RXPRMMCIS	R	Rx Priority MMC Counter Interrupt Status The bit is set when the any of the Rx Per Priority MMC counter (0x09D0 to 0x09FC) reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-137 Fields for Register: MMC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
30	RRSCIS	R	Rx Channel Mapped Rx Frame/Priority MMC Skip Counter Interrupt Status This bit is set when the Rx Channel Mapped Rx Frame/Priority MMC Skip Counter (RMMC_RCHM_SKP_CNTR_*) reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_RMMC_RCH_MAP&&(DWCXG_MMC_RX_EN_AT LEAST_ONE||DWCXG_MMC_PRIORITY_EN)
29	SGFPIS	R	SGF Fail Packet Counter Interrupt Status The bit is set when the and when the SGF Fail Packet Counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
28	SGPPIS	R	SGF Pass Packet Counter Interrupt Status The bit is set when the SGF Pass Packet Counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
27	RXALEPIS	R	Rx Alignment Error Packet Counter Interrupt Status This bit is set when the Rx_Alignment_Error_Packets counter reaches the threshold value programed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
26	RXLPITRCIS	R	Rx LPI Transition Counter Interrupt Status This bit is set when the Rx_LPI_Tran_Cntr reaches the threshold value programed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
25	RXLPIUSCIS	R	Rx LPI microsecond Counter Interrupt Status This bit is set when the Rx_LPI_USEC_Cntr reaches the threshold value programed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-137 Fields for Register: MMC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
24	RXDISOCGBIS	R	Rx Good Bad Discarded Octet Counter Interrupt Status This bit is set when the Rx_Discard_Octets_Good_Bad_High and Rx_Discard_Octets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
23	RXDISPCGBIS	R	Rx Good Bad Discarded Packet Counter Interrupt Status This bit is set when the 64-bit Rx_Discard_Packets_Good_Bad_High and Rx_Discard_Packets_Good_Bad_Low counter or 32-bit Rx_Discard_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
22	RXWDOGPIS	R	Rx Watchdog Error Packet Counter Interrupt Status The bit is set when the Rx_Watchdog_Error_Packets counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
21	RXVLANGBPIS	R	Rx Good Bad VLAN Packet Counter Interrupt Status The bit is set when the 64-bit Rx_VLAN_Packets_Good_Bad_High and Rx_VLAN_Packets_Good_Bad_Low counter or 32-bit Rx_VLAN_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
20	RXFOVPIS	R	Rx Queue Overflow Packet Counter Interrupt Status The bit is set when the 64-bit Rx_FIFOOverflow_Packets_High and Rx_FIFOOverflow_Packets_Low counter or 32-bit Rx_FIFOOverflow_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-137 Fields for Register: MMC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
19	RXPAUSPIS	R	Rx Pause Packet Counter Interrupt Status The bit is set when the 64-bit Rx_Pause_Packets_High and Rx_Pause_Packets_Low counter or 32-bit Rx_Pause_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
18	RXORANGEPIS	R	Rx Out-of-Range Packet Counter Interrupt Status The bit is set when the 64-bit Rx_OutofRange_Packets_High and Rx_OutofRange_Packets_Low counter or 32-bit Rx_OutofRange_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
17	RXLENERPIS	R	Rx Length Error Packet Counter Interrupt Status The bit is set when the 64-bit Rx_Length_Error_Packets_High and Rx_Length_Error_Packets_Low counter or 32-bit Rx_Length_Error_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
16	RXUCGPIS	R	Rx Good Unicast Packet Counter Interrupt Status The bit is set when the 64-bit Rx_Unicast_Packets_Good_High and Rx_Unicast_Packets_Good_Low counter or 32-bit Rx_Unicast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
15	RX1024TMAXOCTGBPIS	R	Rx Good Bad 1024-to-Max Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Rx_1024ToMaxOctets_Packets_Good_Bad_High and Rx_1024ToMaxOctets_Packets_Good_Bad_Low counter or 32-bit Rx_1024ToMaxOctets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-137 Fields for Register: MMC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
14	RX512T1023OCTGBPIS	R	Rx Good Bad 512-to-1023 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Rx_512To1023Octets_Packets_Good_Bad_High and Rx_512To1023Octets_Packets_Good_Bad_Low counter or 32-bit Rx_512To1023Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
13	RX256T511OCTGBPIS	R	Rx Good Bad 256-to-511 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Rx_256To511Octets_Packets_Good_Bad_High and Rx_256To511Octets_Packets_Good_Bad_Low counter or 32-bit Rx_256To511Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
12	RX128T255OCTGBPIS	R	Rx Good Bad 128-to-255 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Rx_128To255Octets_Packets_Good_Bad_High and Rx_128To255Octets_Packets_Good_Bad_Low counter or 32-bit Rx_128To255Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
11	RX65T127OCTGBPIS	R	Rx Good Bad 65-to-127 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Rx_65To127Octets_Packets_Good_Bad_High and Rx_65To127Octets_Packets_Good_Bad_Low counter or 32-bit Rx_65To127Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-137 Fields for Register: MMC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
10	RX64OCTGBPIS	R	Rx Good Bad 64 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Rx_64Octets_Packets_Good_Bad_High and Rx_64Octets_Packets_Good_Bad_Low counter or 32-bit Rx_64Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
9	RXOSIZEGPIS	R	Rx Good Oversize Packet Counter Interrupt Status The bit is set when the Rx_Oversize_Packets_Good counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
8	RXUSIZEGPIS	R	Rx Good Undersize Packet Counter Interrupt Status The bit is set when the Rx_Undersize_Packets_Good counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
7	RXJABERPIS	R	Rx Jabber Error Packet Counter Interrupt Status The bit is set when the Rx_Jabber_Error_Packets counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
6	RXRUNTPIS	R	Rx Runt Error Packet Counter Interrupt Status This bit is set when the Rx_Runt_Error_Packets counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
5	RXCRCERPIS	R	Rx CRC Error Packet Counter Interrupt Status The bit is set when the 64-bit Rx_CRC_Error_Packets_High and Rx_CRC_Error_Packets_Low counter or 32-bit Rx_CRC_Error_Packets_Low reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-137 Fields for Register: MMC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
4	RXMCGPIS	R	Rx Good Multicast Packet Counter Interrupt Status The bit is set when the 64-bit Rx_Multicast_Packets_Good_High and Rx_Multicast_Packets_Good_Low counter or 32-bit Rx_Multicast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
3	RXBCGPIS	R	Rx Good Broadcast Packet Counter Interrupt Status The bit is set when the 64-bit Rx_Broadcast_Packets_Good_High and Rx_Broadcast_Packets_Good_Low counter or 32-bit Rx_Broadcast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
2	RXGOCTIS	R	Rx Good Octet Counter Interrupt Status The bit is set when the Rx_Octet_Count_Good_High and Rx_Octet_Count_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
1	RXGBOCTIS	R	Rx Good Bad Octet Counter Interrupt Status The bit is set when the Rx_Octet_Count_Good_Bad_High and Rx_Octet_Count_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
0	RXGBPKTIS	R	Rx Good Bad Packet Counter Interrupt Status The bit is set when the 64-bit Rx_Packet_Count_Good_Bad_High and Rx_Packet_Count_Good_Bad_Low counter or 32-bit Rx_Packet_Count_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

11.1.134 MMC_Tx_Interrupt

- **Description:** The MMC Transmit Interrupt Status register maintains the interrupts generated when the transmit statistic counters reach the threshold value programmed in MMC control register (for example, the counter's MSB is set). This register is 32 bits wide. When the MMC counter that caused the interrupt is read, the corresponding interrupt status bit is cleared. The least significant byte lane (bits[7:0]) of the corresponding counter must be read to clear the interrupt status bit. When the MMC module is enabled for 64-bit counter mode, the least significant byte lane (bits[7:0]) of the upper counter must be read to clear the interrupt status bit.
- **Size:** 32 bits
- **Offset:** 0x808
- **Exists:**
((DWCXG_MMC_EN_TX_ALL||DWCXG_MMC_HD_EN||DWCXG_MMC_TXADDR_RNG_CNT_EN||DWCXG_MMC_TXOCTGB_CNT_EN||DWCXG_MMC_TXFRMGB_CNT_EN||DWCXG_MMC_TXBCASTG_CNT_EN||DWCXG_MMC_TXMCASTG_CNT_EN||DWCXG_MMC_TXUCASTGB_CNT_EN||DWCXG_MMC_TXMCASTGB_CNT_EN||DWCXG_MMC_TXBCASTGB_CNT_EN||DWCXG_MMC_TXUNDRFLW_CNT_EN||DWCXG_MMC_TXOCTG_CNT_EN||DWCXG_MMC_TXFRMG_CNT_EN||DWCXG_MMC_TXPAUSE_CNT_EN||DWCXG_MMC_TXVLAN_CNT_EN||DWCXG_MMC_TXLPI_USEC_EN||DWCXG_MMC_TXLPI_TRANS_CNTR_EN))

31	30:27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TXPRMMCIS	Reserved_30_27	TXEXDEFPIS	TXCARERPIS	TXEXCOLPIS	TXLATCOLPIS	TXDEFPIS	TXMCOLGPIS	TXSCOLGPIS	TXLPITRCIS	TXLPIUSCIS	TXVLANGPIS	TXPAUSPIS	TXGPKTIS	TXGOCTIS	TXUFLOWERPIS	TXBCGBPIS	TXMCGBPIS	TXUCGBPIS	TX1024TMAXOCTGBPIS	TX512T1023OCTGBPIS	TX256T511OCTGBPIS	TX128T255OCTGBPIS	TX65T127OCTGBPIS	TX64OCTGBPIS	TXMCGPIS	TXBCGPIS	TXGBPKTIS	TXGBOCTIS

Table 11-138 Fields for Register: MMC_Tx_Interrupt

Bits	Name	Memory Access	Description
31	TXPRMMCIS	R	Tx Priority MMC Counter Interrupt Status The bit is set when the any of the Tx Per Priority MMC counter (0x08D0 to 0x08FC) reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
30:27	Reserved_30_27	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-138 Fields for Register: MMC_Tx_Interrupt (continued)

Bits	Name	Memory Access	Description
26	TXEXDEFPIS	R	MMC Transmit Excessive Deferral Packet Counter Interrupt Status This bit is set when the txexcessdef counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Transmit Excessive Deferral Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Transmit Excessive Deferral Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
25	TXCARERPIS	R	MMC Transmit Carrier Error Packet Counter Interrupt Status This bit is set when the txcarriererror counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Transmit Carrier Error Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Transmit Carrier Error Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
24	TXEXCOLPIS	R	MMC Transmit Excessive Collision Packet Counter Interrupt Status This bit is set when the txexesscol counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Transmit Excessive Collision Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Transmit Excessive Collision Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY

Table 11-138 Fields for Register: MMC_Tx_Interrupt (continued)

Bits	Name	Memory Access	Description
23	TXLATCOLPIS	R	MMC Transmit Late Collision Packet Counter Interrupt Status This bit is set when the txlatecol counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Transmit Late Collision Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Transmit Late Collision Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
22	TXDEFPIS	R	MMC Transmit Deferred Packet Counter Interrupt Status This bit is set when the txdeferred counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Transmit Deferred Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Transmit Deferred Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
21	TXMCOLGPIS	R	MMC Transmit Multiple Collision Good Packet Counter Interrupt Status This bit is set when the txmulticol_g counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Transmit Multiple Collision Good Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Transmit Multiple Collision Good Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY

Table 11-138 Fields for Register: MMC_Tx_Interrupt (continued)

Bits	Name	Memory Access	Description
20	TXSCOLGPIS	R	MMC Transmit Single Collision Good Packet Counter Interrupt Status This bit is set when the txsinglecol_g counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Transmit Single Collision Good Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Transmit Single Collision Good Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
19	TXLPITRCIS	R	Tx LPI Transition Counter Interrupt Status This bit is set when the Tx_LPI_Tran_Cntr counter reaches the threshold value programmed in the MMC counter register. **Value After Reset:** 0x0 **Exists:** Always
18	TXLPIUSCIS	R	Tx LPI microsecond Counter Interrupt Status This bit is set when the Tx_LPI_USEC_Cntr counter reaches the threshold value programmed in the MMC counter register. **Value After Reset:** 0x0 **Exists:** Always
17	TXVLANGPIS	R	Tx Good VLAN Packet Counter Interrupt Status The bit is set when the 64-bit Tx_VLAN_Packets_Good_High and Tx_VLAN_Packets_Good_Low counter or 32-bit Tx_VLAN_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
16	TXPAUSPIS	R	Tx Pause Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Pause_Packets_High and Tx_Pause_Packets_Low counter or 32-bit Tx_Pause_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-138 Fields for Register: MMC_Tx_Interrupt (continued)

Bits	Name	Memory Access	Description
15	TXGPKTIS	R	Tx Good Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Packet_Count_Good_High and Tx_Packet_Count_Good_Low counter or 32-bit Tx_Packet_Count_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
14	TXGOCTIS	R	Tx Good Octet Counter Interrupt Status The bit is set when the Tx_Octet_Count_Good_High and Tx_Octet_Count_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
13	TXUFLOWERPIS	R	Tx Underflow Error Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Underflow_Error_Packets_High and Tx_Underflow_Error_Packets_Low counter or 32-bit Tx_Underflow_Error_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
12	TXBCGBPIS	R	Tx Good Bad Broadcast Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Broadcast_Packets_Good_Bad_High and Tx_Broadcast_Packets_Good_Bad_Low counter or 32-bit Tx_Broadcast_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
11	TXMCGBPIS	R	Tx Good Bad Multicast Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Multicast_Packets_Good_Bad_High and Tx_Multicast_Packets_Good_Bad_Low counter or 32-bit Tx_Multicast_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-138 Fields for Register: MMC_Tx_Interrupt (continued)

Bits	Name	Memory Access	Description
10	TXUCGBPIS	R	Tx Good Bad Unicast Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Unicast_Packets_Good_Bad_High and Tx_Unicast_Packets_Good_Bad_Low counter or 32-bit Tx_Unicast_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
9	TX1024TMAXOCTGBPIS	R	Tx Good Bad 1024-to-Max Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Tx_1024ToMaxOctets_Packets_Good_Bad_High and Tx_1024ToMaxOctets_Packets_Good_Bad_Low counter or 32-bit Tx_1024ToMaxOctets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
8	TX512T1023OCTGBPIS	R	Tx Good Bad 512-to-1023 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Tx_512To1023Octets_Packets_Good_Bad_High and Tx_512To1023Octets_Packets_Good_Bad_Low counter or 32-bit Tx_512To1023Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
7	TX256T511OCTGBPIS	R	Tx Good Bad 256-to-511 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Tx_256To511Octets_Packets_Good_Bad_High and Tx_256To511Octets_Packets_Good_Bad_Low counter or 32-bit Tx_256To511Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-138 Fields for Register: MMC_Tx_Interrupt (continued)

Bits	Name	Memory Access	Description
6	TX128T255OCTGBPIS	R	Tx Good Bad 128-to-255 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Tx_128To255Octets_Packets_Good_Bad_High and Tx_128To255Octets_Packets_Good_Bad_Low counter or 32-bit Tx_128To255Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
5	TX65T127OCTGBPIS	R	Tx Good Bad 65-to-127 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Tx_65To127Octets_Packets_Good_Bad_High and Tx_65To127Octets_Packets_Good_Bad_Low counter or 32-bit Tx_65To127Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
4	TX64OCTGBPIS	R	Tx Good Bad 64 Octet Size Packet Counter Interrupt Status The bit is set when the 64-bit Tx_64Octets_Packets_Good_Bad_High and Tx_64Octets_Packets_Good_Bad_Low counter or 32-bit Tx_64Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
3	TXMCGPIS	R	Tx Good Multicast Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Multicast_Packets_Good_High and Tx_Multicast_Packets_Good_Low counter or 32-bit Tx_Multicast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-138 Fields for Register: MMC_Tx_Interrupt (continued)

Bits	Name	Memory Access	Description
2	TXBCGPIS	R	Tx Good Broadcast Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Broadcast_Packets_Good_High and Tx_Broadcast_Packets_Good_Low counter or 32-bit Tx_Broadcast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
1	TXGBPKTIS	R	Tx Good Bad Packet Counter Interrupt Status The bit is set when the 64-bit Tx_Packet_Count_Good_Bad_High and Tx_Packet_Count_Good_Bad_Low counter or 32-bit Tx_Packet_Count_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
0	TXGBOCTIS	R	Tx Good Bad Octet Counter Interrupt Status The bit is set when the Tx_Octet_Count_Good_Bad_High and Tx_Octet_Count_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

11.1.135 MMC_Receive_Interrupt_Enable

- **Description:** The MMC Receive Interrupt Enable register maintains the enables for the interrupts generated when receive statistic counters reach the threshold value programmed in the MMC Control register. This register is 32 bits wide.
- **Size:** 32 bits
- **Offset:** 0x80c
- **Exists:**
  ((DWCXG_MMC_EN_RX_ALL||DWCXG_MMC_RXFRMGB_CNT_EN||DWCXG_MMC_RXOCTG_CNT_EN||DWCXG_MMC_RXOCTGB_CNT_EN||DWCXG_MMC_RXBCASTG_CNT_EN||DWCXG_MMC_RXMCASTG_CNT_EN||DWCXG_MMC_RXCRCERR_CNT_EN||DWCXG_MMC_RXRUNTERR_CNT_EN||DWCXG_MMC_RXJABERR_CNT_EN||DWCXG_MMC_RXUNDERSZG_CNT_EN||DWCXG_MMC_RXOVERSZG_CNT_EN||DWCXG_MMC_RXADDR_RNG_CNT_EN||DWCXG_MMC_RXUCASTG_CNT_EN||DWCXG_MMC_RXLENERR_CNT_EN||DWCXG_MMC_RXOUTOFRNG_TYP_CNT_EN||DWCXG_MMC_RXPAUSEFRM_CNT_EN
  ||
  DWCXG_MMC_RXFIFOOVFL_CNT_EN||DWCXG_MMC_RXVLANFRGB_CNT_EN||DWCXG_MMC_RXWDGERR_CNT_EN||DWCXG_MMC_RXFRMDISGB_CNT_EN||DWCXG_MMC_RXOCTDISGB_CNT_EN||DWCXG_MMC_RXLPI_USEC_EN||DWCXG_MMC_RXLPI_TRANS_CNTR_EN||DWCXG_MMC_RXALGNERR_CNT_EN))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RXPRMMCIE	RRSCIE	SGFPIE	SGPPIE	RXALEPIE	RXLPITRCIE	RXLPIUSCIE	RXDISOCGBIE	RXDISPCGBIE	RXWDOGPIE	RXVLANGBPIE	RXFOVPIE	RXPAUSPIE	RXORANGEPIE	RXLENERPIE	RXUCGPIE	RX1024TMAXOCTGBPIE	RX512T1023OCTGBPIE	RX256T511OCTGBPIE	RX128T255OCTGBPIE	RX65T127OCTGBPIE	RX64OCTGBPIE	RXOSIZEGPIE	RXUSIZEGPIE	RXJABERPIE	RXRUNTPIE	RXCRCERPIE	RXMCGPIE	RXBCGPIE	RXGOCTIE	RXGBOCTIE	RXGBPKTIE

Table 11-139 Fields for Register: MMC_Receive_Interrupt_Enable

Bits	Name	Memory Access	Description
31	RXPRMMCIE	R/W	Rx Priority MMC Counter Interrupt Enable Setting this bit enables the interrupt when the any of Rx Per Priority Counters (0x09D0 to 0x09FC) reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_PRIORITY_EN

Table 11-139 Fields for Register: MMC_Receive_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
30	RRSCIE	R/W	Rx Channel Mapped Rx Frame/Priority MMC Skip Counter Interrupt Enable Setting this bit enables the interrupt when Rx Channel Mapped Rx Frame/Priority MMC Skip Counter (RMMC_RCHM_SKP_CNTR_*) reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_RMMC_RCH_MAP&&(DWCXG_MMC_RX_EN_AT LEAST_ONE||DWCXG_MMC_PRIORITY_EN)
29	SGFPIE	R/W	SGF Fail Packet Counter Interrupt Enable Setting this bit enables the interrupt when SGF Fail Packet Counter counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_SGF
28	SGPPIE	R/W	SGF Pass Packet Counter Interrupt Enable Setting this bit enables the interrupt when SGF Pass Packet Counter counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_SGF
27	RXALEPIE	R/W	Rx Alignment Error Packet Counter Interrupt Enable Setting this bit enables the interrupt when Rx_Alignment_Error_Packets counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXALGNERR_CNT_EN
26	RXLPITRCIE	R/W	Rx LPI Transition Counter Interrupt Enable Setting this bit enables the interrupt when Rx_LPI_Tran_Cntr reaches the threshold value programed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXLPI_TRANS_CNTR_EN
25	RXLPIUSCIE	R/W	Rx LPI microsecond Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_LPI_USEC_Cntr reaches the threshold value programed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXLPI_USEC_EN

Table 11-139 Fields for Register: MMC_Receive_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
24	RXDISOCGBIE	R/W	Rx Discard Octet Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_Discard_Octets_Good_Bad_High and Rx_Discard_Octets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXOCTDISGB_CNT_EN
23	RXDISPCGBIE	R/W	Rx Discard Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_Discard_Packets_Good_Bad_High and Rx_Discard_Packets_Good_Bad_Low counter or 32-bit Rx_Discard_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXFRMDISGB_CNT_EN
22	RXWDOGPIE	R/W	Rx Watchdog Error Packet Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_Watchdog_Error_Packets counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXWDGERR_CNT_EN
21	RXVLANGBPIE	R/W	Rx Good Bad VLAN Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_VLAN_Packets_Good_Bad_High and Rx_VLAN_Packets_Good_Bad_Low counter or 32-bit Rx_VLAN_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXVLANFRGB_CNT_EN
20	RXFOVPIE	R/W	Rx Queue Overflow Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_FIFOOverflow_Packets_High and Rx_FIFOOverflow_Packets_Low counter or 32-bit Rx_FIFOOverflow_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXFIFOOVFL_CNT_EN

Table 11-139 Fields for Register: MMC_Receive_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
19	RXPAUSPIE	R/W	Rx Pause Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_Pause_Packets_High and Rx_Pause_Packets_Low counter or 32-bit Rx_Pause_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXPAUSEFRM_CNT_EN
18	RXORANGEPIE	R/W	Rx Out-of-Range Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_OutofRange_Packets_High and Rx_OutofRange_Packets_Low counter or 32-bit Rx_OutofRange_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXOUTOFRNG_TYP_CNT_EN
17	RXLENERPIE	R/W	Rx Length Error Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_Length_Error_Packets_High and Rx_Length_Error_Packets_Low counter or 32-bit Rx_Length_Error_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXLENERR_CNT_EN
16	RXUCGPIE	R/W	Rx Good Unicast Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_Unicast_Packets_Good_High and Rx_Unicast_Packets_Good_Low counter or 32-bit Rx_Unicast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXUCASTG_CNT_EN
15	RX1024TMAXOCTGBPIE	R/W	Rx Good Bad 1024-to-Max Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_1024ToMaxOctets_Packets_Good_Bad_High and Rx_1024ToMaxOctets_Packets_Good_Bad_Low counter or 32-bit Rx_1024ToMaxOctets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXADDR_RNG_CNT_EN

Table 11-139 Fields for Register: MMC_Receive_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
14	RX512T1023OCTGBPIE	R/W	Rx Good Bad 512-to-1023 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_512To1023Octets_Packets_Good_Bad_High and Rx_512To1023Octets_Packets_Good_Bad_Low counter or 32-bit Rx_512To1023Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXADDR_RNG_CNT_EN
13	RX256T511OCTGBPIE	R/W	Rx Good Bad 256-to-511 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_256To511Octets_Packets_Good_Bad_High and Rx_256To511Octets_Packets_Good_Bad_Low counter or 32-bit Rx_256To511Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXADDR_RNG_CNT_EN
12	RX128T255OCTGBPIE	R/W	Rx Good Bad 128-to-255 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_128To255Octets_Packets_Good_Bad_High and Rx_128To255Octets_Packets_Good_Bad_Low counter or 32-bit Rx_128To255Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXADDR_RNG_CNT_EN
11	RX65T127OCTGBPIE	R/W	Rx Good Bad 65-to-127 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_65To127Octets_Packets_Good_Bad_High and Rx_65To127Octets_Packets_Good_Bad_Low counter or 32-bit Rx_65To127Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXADDR_RNG_CNT_EN

Table 11-139 Fields for Register: MMC_Receive_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
10	RX64OCTGBPIE	R/W	Rx Good Bad 64 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_64Octets_Packets_Good_Bad_High and Rx_64Octets_Packets_Good_Bad_Low counter or 32-bit Rx_64Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXADDR_RNG_CNT_EN
9	RXOSIZEGPIE	R/W	Rx Good Oversize Packet Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_Oversize_Packets_Good counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXOVERSZG_CNT_EN
8	RXUSIZEGPIE	R/W	Rx Good Undersize Packet Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_Undersize_Packets_Good counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXUNDERSZG_CNT_EN
7	RXJABERPIE	R/W	Rx Jabber Error Packet Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_Jabber_Error_Packets counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXJABERR_CNT_EN
6	RXRUNTPIE	R/W	Rx Runt Error Packet Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_Runt_Error_Packets counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXRUNTERR_CNT_EN
5	RXCRCERPIE	R/W	Rx CRC Error Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_CRC_Error_Packets_High and Rx_CRC_Error_Packets_Low counter or 32-bit Rx_CRC_Error_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXCRCERR_CNT_EN

Table 11-139 Fields for Register: MMC_Receive_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
4	RXMCGPIE	R/W	Rx Good Multicast Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_Multicast_Packets_Good_High and Rx_Multicast_Packets_Good_Low counter or 32-bit Rx_Multicast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXMCASTG_CNT_EN
3	RXBCGPIE	R/W	Rx Good Broadcast Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_Broadcast_Packets_Good_High and Rx_Broadcast_Packets_Good_Low counter or 32-bit Rx_Broadcast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXBCASTG_CNT_EN
2	RXGOCTIE	R/W	Rx Good Octet Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_Octet_Count_Good_High and Rx_Octet_Count_Good_Lo counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXOCTG_CNT_EN
1	RXGBOCTIE	R/W	Rx Good Bad Octet Counter Interrupt Enable Setting this bit enables the interrupt when the Rx_Octet_Count_Good_Bad_High and Rx_Octet_Count_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXOCTGB_CNT_EN
0	RXGBPKTIE	R/W	Rx Good Bad Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Rx_Packet_Count_Good_Bad_High and Rx_Packet_Count_Good_Bad_Low counter or 32-bit Rx_Packet_Count_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RXFRMGB_CNT_EN

11.1.136 MMC_Transmit_Interrupt_Enable

- **Description:** The MMC Transmit Interrupt Enable register maintains the enables for the interrupts generated when transmit statistic counters reach the threshold value programmed in the MMC Control register. This register is 32 bits wide.
- **Size:** 32 bits
- **Offset:** 0x810
- **Exists:**
((DWCXG_MMC_EN_TX_ALL||DWCXG_MMC_HD_EN||DWCXG_MMC_TXADDR_RNG_CNT_EN||DWCXG_MMC_TXOCTGB_CNT_EN||DWCXG_MMC_TXFRMGB_CNT_EN||DWCXG_MMC_TXBCASTG_CNT_EN||DWCXG_MMC_TXMCASTG_CNT_EN||DWCXG_MMC_TXUCASTGB_CNT_EN||DWCXG_MMC_TXMCASTGB_CNT_EN||DWCXG_MMC_TXBCASTGB_CNT_EN||DWCXG_MMC_TXUNDRFLW_CNT_EN||DWCXG_MMC_TXOCTG_CNT_EN||DWCXG_MMC_TXFRMG_CNT_EN||DWCXG_MMC_TXPAUSE_CNT_EN||DWCXG_MMC_TXVLAN_CNT_EN||DWCXG_MMC_TXLPI_USEC_EN||DWCXG_MMC_TXLPI_TRANS_CNTR_EN))

31	30:27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TXPRMMCIE	Reserved_30_27	TXEXDEFPIE	TXCARERPIE	TXEXCOLPIE	TXLATCOLPIE	TXDEFPIE	TXMCOLGPIE	TXSCOLGPIE	TXLPITRCIE	TXLPIUSCIE	TXVLANGPIE	TXPAUSPIE	TXGPKTIE	TXGOCTIE	TXUFLOWERPIE	TXBCGBPIE	TXMCGBPIE	TXUCGBPIE	TX1024TMAXOCTGBPIE	TX512T1023OCTGBPIE	TX256T511OCTGBPIE	TX128T255OCTGBPIE	TX65T127OCTGBPIE	TX64OCTGBPIE	TXMCGPIE	TXBCGPIE	TXGBPKTIE	TXGBOCTIE

Table 11-140 Fields for Register: MMC_Transmit_Interrupt_Enable

Bits	Name	Memory Access	Description
31	TXPRMMCIE	R/W	Tx Priority MMC Counter Interrupt Enable Setting this bit enables the interrupt when the any of Tx Per Priority Counters (0x08D0 to 0x08FC) reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_PRIORITY_EN
30:27	Reserved_30_27	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-140 Fields for Register: MMC_Transmit_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
26	TXEXDEFPIE	R/W	MMC Transmit Excessive Deferral Packet Counter Interrupt Enable Setting this bit masks the interrupt when the txexcessdef counter reaches the threshold value programmed in the MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Excessive Deferral Packet Counter Interrupt Enable is disabled ■ 0x1 (ENABLE): MMC Transmit Excessive Deferral Packet Counter Interrupt Enable is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
25	TXCARERPIE	R/W	MMC Transmit Carrier Error Packet Counter Interrupt Enable Setting this bit masks the interrupt when the txcarriererror counter reaches the threshold value programmed in the MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Carrier Error Packet Counter Interrupt Enable is disabled ■ 0x1 (ENABLE): MMC Transmit Carrier Error Packet Counter Interrupt Enable is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
24	TXEXCOLPIE	R/W	MMC Transmit Excessive Collision Packet Counter Interrupt Enable Setting this bit masks the interrupt when the txexcesscol counter reaches the threshold value programmed in the MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Excessive Collision Packet Counter Interrupt Enable is disabled ■ 0x1 (ENABLE): MMC Transmit Excessive Collision Packet Counter Interrupt Enable is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY

Table 11-140 Fields for Register: MMC_Transmit_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
23	TXLATCOLPIE	R/W	MMC Transmit Late Collision Packet Counter Interrupt Enable Setting this bit masks the interrupt when the txlatecol counter reaches the threshold value programmed in the MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Late Collision Packet Counter Interrupt Enable is disabled ■ 0x1 (ENABLE): MMC Transmit Late Collision Packet Counter Interrupt Enable is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
22	TXDEFPIE	R/W	MMC Transmit Deferred Packet Counter Interrupt Enable Setting this bit masks the interrupt when the txdeferred counter reaches the threshold value programmed in the MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Deferred Packet Counter Interrupt Enable is disabled ■ 0x1 (ENABLE): MMC Transmit Deferred Packet Counter Interrupt Enable is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
21	TXMCOLGPIE	R/W	MMC Transmit Multiple Collision Good Packet Counter Interrupt Enable Setting this bit masks the interrupt when the txmulticol_g counter reaches the threshold value programmed in the MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Multiple Collision Good Packet Counter Interrupt Enable is disabled ■ 0x1 (ENABLE): MMC Transmit Multiple Collision Good Packet Counter Interrupt Enable is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY

Table 11-140 Fields for Register: MMC_Transmit_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
20	TXSCOLGPIE	R/W	MMC Transmit Single Collision Good Packet Counter Interrupt Enable Setting this bit masks the interrupt when the txsinglecol_g counter reaches the threshold value programmed in the MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Single Collision Good Packet Counter Interrupt Enable is disabled ■ 0x1 (ENABLE): MMC Transmit Single Collision Good Packet Counter Interrupt Enable is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY
19	TXLPITRCIE	R/W	Tx LPI Transition Counter Interrupt Enable Setting this bit enables the interrupt when Tx_LPI_Tran_Cntr counter reaches the threshold value programmed in the MMC counter register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXLPI_TRANS_CNTR_EN
18	TXLPIUSCIE	R/W	Tx LPI microsecond Counter Interrupt Enable Setting this bit enables the interrupt when Tx_LPI_USEC_Cntr counter reaches the threshold value programmed in the MMC counter register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXLPI_USEC_EN
17	TXVLANGPIE	R/W	Tx Good VLAN Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_VLAN_Packets_Good_High and Tx_VLAN_Packets_Good_Low counter or 32-bit Tx_VLAN_Packets_Good_Low reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXVLAN_CNT_EN
16	TXPAUSPIE	R/W	Tx Pause Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Pause_Packets_High and Tx_Pause_Packets_Low counter or 32-bit Tx_Pause_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXPAUSE_CNT_EN

Table 11-140 Fields for Register: MMC_Transmit_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
15	TXGPKTIE	R/W	Tx Good Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Packet_Count_Good_High and Tx_Packet_Count_Good_Low counter or 32-bit Tx_Packet_Count_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXFRMG_CNT_EN
14	TXGOCTIE	R/W	Tx Good Octet Counter Interrupt Enable Setting this bit enables the interrupt when the Tx_Octet_Count_Good_High and Tx_Octet_Count_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXOCTG_CNT_EN
13	TXUFLOWERPIE	R/W	Tx Underflow Error Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Underflow_Error_Packets_High and Tx_Underflow_Error_Packets_Low counter or 32-bit Tx_Underflow_Error_Packets_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXUNDRFLW_CNT_EN
12	TXBCGBPIE	R/W	Tx Good Bad Broadcast Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Broadcast_Packets_Good_Bad_High and Tx_Broadcast_Packets_Good_Bad_Low counter or 32-bit Tx_Broadcast_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXBCASTGB_CNT_EN
11	TXMCGBPIE	R/W	Tx Good Bad Multicast Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Multicast_Packets_Good_Bad_High and Tx_Multicast_Packets_Good_Bad_Low counter or 32-bit Tx_Multicast_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXMCASTGB_CNT_EN

Table 11-140 Fields for Register: MMC_Transmit_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
10	TXUCGBPIE	R/W	Tx Good Bad Unicast Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Unicast_Packets_Good_Bad_High and Tx_Unicast_Packets_Good_Bad_Low counter or 32-bit Tx_Unicast_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXUCASTGB_CNT_EN
9	TX1024TMAXOCTGBPIE	R/W	Tx Good Bad 1024-to-Max Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_1024ToMaxOctets_Packets_Good_Bad_High and Tx_1024ToMaxOctets_Packets_Good_Bad_Low counter or 32-bit Tx_1024ToMaxOctets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXADDR_RNG_CNT_EN
8	TX512T1023OCTGBPIE	R/W	Tx Good Bad 512-to-1023 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_512To1023Octets_Packets_Good_Bad_High and Tx_512To1023Octets_Packets_Good_Bad_Low counter or 32-bit Tx_512To1023Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXADDR_RNG_CNT_EN
7	TX256T511OCTGBPIE	R/W	Tx Good Bad 256-to-511 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_256To511Octets_Packets_Good_Bad_High and Tx_256To511Octets_Packets_Good_Bad_Low counter or 32-bit Tx_256To511Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXADDR_RNG_CNT_EN

Table 11-140 Fields for Register: MMC_Transmit_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
6	TX128T255OCTGBPIE	R/W	Tx Good Bad 128-to-255 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_128To255Octets_Packets_Good_Bad_High and Tx_128To255Octets_Packets_Good_Bad_Low counter or 32-bit Tx_128To255Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXADDR_RNG_CNT_EN
5	TX65T127OCTGBPIE	R/W	Tx Good Bad 65-to-127 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_65To127Octets_Packets_Good_Bad_High and Tx_65To127Octets_Packets_Good_Bad_Low counter or 32-bit Tx_65To127Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXADDR_RNG_CNT_EN
4	TX64OCTGBPIE	R/W	Tx Good Bad 64 Octet Size Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_64Octets_Packets_Good_Bad_High and Tx_64Octets_Packets_Good_Bad_Low counter or 32-bit Tx_64Octets_Packets_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXADDR_RNG_CNT_EN
3	TXMCGPIE	R/W	Tx Good Multicast Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Multicast_Packets_Good_High and Tx_Multicast_Packets_Good_Low counter or 32-bit Tx_Multicast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXMCASTG_CNT_EN

Table 11-140 Fields for Register: MMC_Transmit_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
2	TXBCGPIE	R/W	Tx Good Broadcast Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Broadcast_Packets_Good_High and Tx_Broadcast_Packets_Good_Low counter or 32-bit Tx_Broadcast_Packets_Good_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXBCASTG_CNT_EN
1	TXGBPKTIE	R/W	Tx Good Bad Packet Counter Interrupt Enable Setting this bit enables the interrupt when the 64-bit Tx_Packet_Count_Good_Bad_High and Tx_Packet_Count_Good_Bad_Low counter or 32-bit Tx_Packet_Count_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXFRMGB_CNT_EN
0	TXGBOCTIE	R/W	Tx Good Bad Octet Counter Interrupt Enable Setting this bit enables the interrupt when the Tx_Octet_Count_Good_Bad_High and Tx_Octet_Count_Good_Bad_Low counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_TXOCTGB_CNT_EN

11.1.137 Tx_Octet_Count_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted octet count, excluding preamble, in good and bad packets.
- **Size:** 32 bits
- **Offset:** 0x814
- **Exists:** (DWCXG_MMC_TXOCTGB_CNT_EN)

Table 11-141 Fields for Register: Tx_Octet_Count_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TXOCTGBLO	R	Tx Good Bad Octet Count Low This field indicates the lower 32 bits of transmitted octet count, where octet count is the number of bytes transmitted, excluding preamble, in good and bad packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.138 Tx_Octet_Count_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted octet count, excluding preamble, in good and bad packets.
- **Size:** 32 bits
- **Offset:** 0x818
- **Exists:** (DWCXG_MMC_TXOCTGB_CNT_EN)

Table 11-142 Fields for Register: Tx_Octet_Count_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TXOCTGBHI	R	Tx Good Bad Octet Count High This field indicates upper 32 bits of transmitted octet count, where octet count is the number of bytes transmitted, excluding preamble, in good and bad packets (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.139 Tx_Packet_Count_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted good and bad packet count.
- **Size:** 32 bits
- **Offset:** 0x81c
- **Exists:** (DWCXG_MMC_TXFRMGB_CNT_EN)

Table 11-143 Fields for Register: Tx_Packet_Count_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TXPKTGBLO	R	Tx Good Bad Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and bad packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.140 Tx_Packet_Count_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted good and bad packet count.
- **Size:** 32 bits
- **Offset:** 0x820
- **Exists:** (DWCXG_MMC_TXFRMGB_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-144 Fields for Register: Tx_Packet_Count_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TXPKTGBHI	R	Tx Good Bad Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and bad packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.141 Tx_Broadcast_Packets_Good_Low

- **Description:** This register provides lower 32 bits of transmitted good broadcast packets count.
- **Size:** 32 bits
- **Offset:** 0x824
- **Exists:** (DWCXG_MMC_TXBCASTG_CNT_EN)

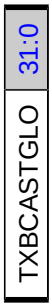


Table 11-145 Fields for Register: Tx_Broadcast_Packets_Good_Low

Bits	Name	Memory Access	Description
31:0	TXBCASTGLO	R	Tx Good Broadcast Packet Count Low. This field indicates lower 32 bits of the counter that counts the number of good broadcast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.142 Tx_Broadcast_Packets_Good_High

- **Description:** This register provides upper 32 bits of transmitted good broadcast packets count.
- **Size:** 32 bits
- **Offset:** 0x828
- **Exists:** (DWCXG_MMC_TXBCASTG_CNT_EN&&DWCXG_EN_64BIT_CNT)

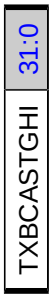


Table 11-146 Fields for Register: Tx_Broadcast_Packets_Good_High

Bits	Name	Memory Access	Description
31:0	TXBCASTGHI	R	Tx Good Broadcast Packet Count High This field indicates upper 32 bits of the counter that counts the number of good broadcast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.143 Tx_Multicast_Packets_Good_Low

- **Description:** This register provides lower 32 bits of transmitted good multicast packets count.
- **Size:** 32 bits
- **Offset:** 0x82c
- **Exists:** (DWCXG_MMC_TXMCASTG_CNT_EN)

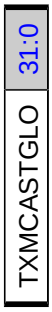


Table 11-147 Fields for Register: Tx_Multicast_Packets_Good_Low

Bits	Name	Memory Access	Description
31:0	TXMCASTGLO	R	Tx Good Multicast Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good multicast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.144 Tx_Multicast_Packets_Good_High

- **Description:** This register provides upper 32 bits of transmitted good multicast packets count.
- **Size:** 32 bits
- **Offset:** 0x830
- **Exists:** (DWCXG_MMC_TXMCASTG_CNT_EN&&DWCXG_EN_64BIT_CNT)

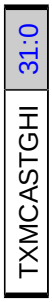


Table 11-148 Fields for Register: Tx_Multicast_Packets_Good_High

Bits	Name	Memory Access	Description
31:0	TXMCASTGHI	R	Tx Good Multicast Packet Count High This field indicates upper 32 bits of the counter that counts the number of good multicast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.145 Tx_64Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted 64 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x834
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN)

Table 11-149 Fields for Register: Tx_64Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TX64OCTGBLO	R	Tx Good Bad 64 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and bad packets transmitted that are of size 64 bytes, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.146 Tx_64Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted 64 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x838
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

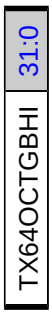


Table 11-150 Fields for Register: Tx_64Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TX64OCTGBHI	R	Tx Good Bad 64 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and bad packets transmitted that are of size 64 bytes, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.147 Tx_65To127Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted 65-to-127 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x83c
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN)

Table 11-151 Fields for Register: Tx_65To127Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TX65_127OCTGBLO	R	Tx Good Bad 65-to-127 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 65-to-127-byte good and bad packets transmitted, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.148 Tx_65To127Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted 65-to-127 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x840
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-152 Fields for Register: Tx_65To127Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TX65_127OCTGBHI	R	Tx Good Bad 65-to-127 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 65-to-127-byte good and bad packets transmitted, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.149 Tx_128To255Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted 128-to-255 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x844
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN)

Table 11-153 Fields for Register: Tx_128To255Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TX128_255OCTGBLO	R	Tx Good Bad 128-to-255 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 128-to-255-byte good and bad packets transmitted, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.150 Tx_128To255Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted 128-to-255 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x848
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-154 Fields for Register: Tx_128To255Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TX128_255OCTGBHI	R	Tx Good Bad 128-to-255 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 128-to-255-byte good and bad packets transmitted, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.151 Tx_256To511Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted 256-to-511 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x84c
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN)

Table 11-155 Fields for Register: Tx_256To511Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TX256_511OCTGBLO	R	Tx Good Bad 256-to-511 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 256-to-511-byte good and bad packets transmitted, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.152 Tx_256To511Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted 256-to-511 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x850
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-156 Fields for Register: Tx_256To511Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TX256_511OCTGBHI	R	Tx Good Bad 256-to-511 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 256-to-511-byte good and bad packets transmitted, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.153 Tx_512To1023Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted 512-to-1023 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x854
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN)

Table 11-157 Fields for Register: Tx_512To1023Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TX512_1023OCTGBLO	R	Tx Good Bad 512-to-1023 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 512-to-1023-byte good and bad packets transmitted, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.154 Tx_512To1023Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted 512-to-1023 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x858
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-158 Fields for Register: Tx_512To1023Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TX512_1023OCTGBHI	R	Tx Good Bad 512-to-1023 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 512-to-1023-byte good and bad packets transmitted, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.155 Tx_1024ToMaxOctets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted 1024-tomaxsize octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x85c
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN)

Table 11-159 Fields for Register: Tx_1024ToMaxOctets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TX1024_MAXOCTGBLO	R	Tx Good Bad 1024-tomaxsize Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 1024-tomaxsize-byte (inclusive) good and bad packets transmitted, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.156 Tx_1024ToMaxOctets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted 1024-tomaxsize octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x860
- **Exists:** (DWCXG_MMC_TXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-160 Fields for Register: Tx_1024ToMaxOctets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TX1024_MAXOCTGBHI	R	Tx Good Bad 1024-tomaxsize Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 1024-tomaxsize-byte (inclusive) good and bad packets transmitted, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.157 Tx_Unicast_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted good bad unicast packets count.
- **Size:** 32 bits
- **Offset:** 0x864
- **Exists:** (DWCXG_MMC_TXUCASTGB_CNT_EN)

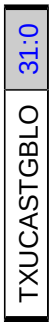


Table 11-161 Fields for Register: Tx_Unicast_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TXUCASTGBLO	R	Tx Good Bad Unicast Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and bad unicast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.158 Tx_Unicast_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted good bad unicast packets count.
- **Size:** 32 bits
- **Offset:** 0x868
- **Exists:** (DWCXG_MMC_TXUCASTGB_CNT_EN&&DWCXG_EN_64BIT_CNT)

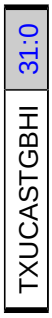


Table 11-162 Fields for Register: Tx_Unicast_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TXUCASTGBHI	R	Tx Good Bad Unicast Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and bad unicast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.159 Tx_Multicast_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted good bad multicast packets count.
- **Size:** 32 bits
- **Offset:** 0x86c
- **Exists:** (DWCXG_MMC_TXMCASTGB_CNT_EN)

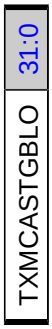


Table 11-163 Fields for Register: Tx_Multicast_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TXMCASTGBLO	R	Tx Good Bad Multicast Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and bad multicast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.160 Tx_Multicast_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted good bad multicast packets count.
- **Size:** 32 bits
- **Offset:** 0x870
- **Exists:** (DWCXG_MMC_TXMCASTGB_CNT_EN&&DWCXG_EN_64BIT_CNT)

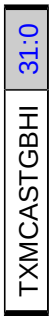


Table 11-164 Fields for Register: Tx_Multicast_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TXMCASTGBHI	R	Tx Good Bad Multicast Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and bad multicast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.161 Tx_Broadcast_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted good bad broadcast packets count.
- **Size:** 32 bits
- **Offset:** 0x874
- **Exists:** (DWCXG_MMC_TXBCASTGB_CNT_EN)

Table 11-165 Fields for Register: Tx_Broadcast_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TXBCASTGBLO	R	Tx Good Bad Broadcast Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and bad broadcast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.162 Tx_Broadcast_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted good bad broadcast packets count.
- **Size:** 32 bits
- **Offset:** 0x878
- **Exists:** (DWCXG_MMC_TXBCASTGB_CNT_EN&&DWCXG_EN_64BIT_CNT)

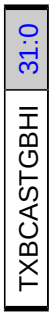


Table 11-166 Fields for Register: Tx_Broadcast_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TXBCASTGBHI	R	Tx Good Bad Broadcast Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and bad broadcast packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.163 Tx_Underflow_Error_Packets_Low

- **Description:** This register provides lower 32 bits of transmitted packets aborted due to packet underflow error count.
- **Size:** 32 bits
- **Offset:** 0x87c
- **Exists:** (DWCXG_MMC_TXUNDRFLW_CNT_EN)

Table 11-167 Fields for Register: Tx_Underflow_Error_Packets_Low

Bits	Name	Memory Access	Description
31:0	TXUNDRFLWLO	R	Tx Underflow Error Packet Count Low This field indicates lower 32 bits of the counter that counts the number of packets aborted due to packet underflow error. **Value After Reset:** 0x0 **Exists:** Always

11.1.164 Tx_Underflow_Error_Packets_High

- **Description:** This register provides upper 32 bits of transmitted packets aborted due to packet underflow error count.
- **Size:** 32 bits
- **Offset:** 0x880
- **Exists:** (DWCXG_MMC_TXUNDRFLW_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-168 Fields for Register: Tx_Underflow_Error_Packets_High

Bits	Name	Memory Access	Description
31:0	TXUNDRFLWHI	R	Tx Underflow Error Packet Count High This field indicates upper 32 bits of the counter that counts the number of packets aborted due to packet underflow error. **Value After Reset:** 0x0 **Exists:** Always

11.1.165 Tx_Octet_Count_Good_Low

- **Description:** This register provides lower 32 bits of transmitted good octets count.
- **Size:** 32 bits
- **Offset:** 0x884
- **Exists:** (DWCXG_MMC_TXOCTG_CNT_EN)

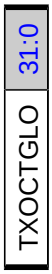


Table 11-169 Fields for Register: Tx_Octet_Count_Good_Low

Bits	Name	Memory Access	Description
31:0	TXOCTGLO	R	Tx Good Octet Count Low This field indicates lower 32 bits of the transmitted octet count, where the octet count is the number of bytes transmitted, excluding preamble, in good packets only. **Value After Reset:** 0x0 **Exists:** Always

11.1.166 Tx_Octet_Count_Good_High

- **Description:** This register provides upper 32 bits of transmitted good octets count.
- **Size:** 32 bits
- **Offset:** 0x888
- **Exists:** (DWCXG_MMC_TXOCTG_CNT_EN)

Table 11-170 Fields for Register: Tx_Octet_Count_Good_High

Bits	Name	Memory Access	Description
31:0	TXOCTGHI	R	Tx Good Octet Count High This field indicates upper 32 bits of the transmitted octet count, where the octet count is the number of bytes transmitted, excluding preamble, in good packets only (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.167 Tx_Packet_Count_Good_Low

- **Description:** This register provides lower 32 bits of transmitted good packets count.
- **Size:** 32 bits
- **Offset:** 0x88c
- **Exists:** (DWCXG_MMC_TXFRMG_CNT_EN)

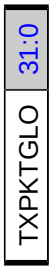


Table 11-171 Fields for Register: Tx_Packet_Count_Good_Low

Bits	Name	Memory Access	Description
31:0	TXPKTGLO	R	Tx Good Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.168 Tx_Packet_Count_Good_High

- **Description:** This register provides upper 32 bits of transmitted good packets count.
- **Size:** 32 bits
- **Offset:** 0x890
- **Exists:** (DWCXG_MMC_TXFRMG_CNT_EN&&DWCXG_EN_64BIT_CNT)

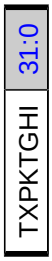


Table 11-172 Fields for Register: Tx_Packet_Count_Good_High

Bits	Name	Memory Access	Description
31:0	TXPKTGHI	R	Tx Good Packet Count High This field indicates upper 32 bits of the counter that counts the number of good packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.169 Tx_Pause_Packets_Low

- **Description:** This register provides lower 32 bits of transmitted Pause packets count.
- **Size:** 32 bits
- **Offset:** 0x894
- **Exists:** (DWCXG_MMC_TXPAUSE_CNT_EN)

Table 11-173 Fields for Register: Tx_Pause_Packets_Low

Bits	Name	Memory Access	Description
31:0	TXPAUSEGLO	R	Tx Pause Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good PAUSE packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.170 Tx_Pause_Packets_High

- **Description:** This register provides upper 32 bits of transmitted Pause packets count.
- **Size:** 32 bits
- **Offset:** 0x898
- **Exists:** (DWCXG_MMC_TXPAUSE_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-174 Fields for Register: Tx_Pause_Packets_High

Bits	Name	Memory Access	Description
31:0	TXPAUSEGHI	R	Tx Pause Packet Count High This field indicates upper 32 bits of the counter that counts the number of good PAUSE packets transmitted (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.171 Tx_VLAN_Packets_Good_Low

- **Description:** This register provides lower 32 bits of transmitted good VLAN packets count.
- **Size:** 32 bits
- **Offset:** 0x89c
- **Exists:** (DWCXG_MMC_TXVLAN_CNT_EN)

Table 11-175 Fields for Register: Tx_VLAN_Packets_Good_Low

Bits	Name	Memory Access	Description
31:0	TXVLANGLO	R	Tx Good VLAN Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good VLAN packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.172 Tx_VLAN_Packets_Good_High

- **Description:** This register provides upper 32 bits of transmitted good VLAN packets count.
- **Size:** 32 bits
- **Offset:** 0x8a0
- **Exists:** (DWCXG_MMC_TXVLAN_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-176 Fields for Register: Tx_VLAN_Packets_Good_High

Bits	Name	Memory Access	Description
31:0	TXVLANGHI	R	Tx Good VLAN Packet Count High This field indicates upper 32 bits of the counter that counts the number of good VLAN packets transmitted. **Value After Reset:** 0x0 **Exists:** Always

11.1.173 Tx_LPI_USEC_Cntr

- **Description:** This register provides duration in microsecond for which MAC was in Tx LPI.
- **Size:** 32 bits
- **Offset:** 0x8a4
- **Exists:** (DWCXG_MMC_TXLPI_USEC_EN)

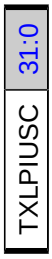


Table 11-177 Fields for Register: Tx_LPI_USEC_Cntr

Bits	Name	Memory Access	Description
31:0	TXLPIUSC	R	Tx LPI microsecond Count This field indicates the number of microseconds DWC_25gmac is in Tx LPI state. **Value After Reset:** 0x0 **Exists:** Always

11.1.174 Tx_LPI_Tran_Cntr

- **Description:** This register provides number of times MAC transitioned to Tx LPI state.
- **Size:** 32 bits
- **Offset:** 0x8a8
- **Exists:** (DWCXG_MMC_TXLPI_TRANS_CNTR_EN)

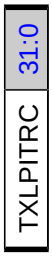


Table 11-178 Fields for Register: Tx_LPI_Tran_Cntr

Bits	Name	Memory Access	Description
31:0	TXLPITRC	R	Tx LPI Transition Count This field indicates the number of times DWC_25gmac entered to Tx LPI state. **Value After Reset:** 0x0 **Exists:** Always

11.1.175 Priority_Interrupt_Status

- **Description:** This register provides the Per Priority MMC Interrupt Status information.
- **Size:** 32 bits
- **Offset:** 0x8cc
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

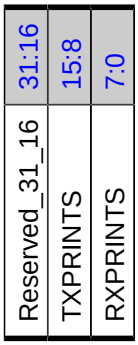


Table 11-179 Fields for Register: Priority_Interrupt_Status

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:8	TXPRINTS	R	Tx Per Priority MMC Counter Interrupt Status The interrupt status corresponding to Per Priority Tx MMC Counters (0x08D0 to 0x08FC). Each bit indicates whether the corresponding priority has an Interrupt. **Value After Reset:** 0x0 **Exists:** Always
7:0	RXPRINTS	R	Rx Per Priority MMC Counter Interrupt Status The interrupt status corresponding to Per Priority Rx MMC Counters (0x09D0 to 0x09FC). Each bit indicates whether the corresponding priority has an Interrupt. **Value After Reset:** 0x0 **Exists:** Always

11.1.176 Tx_Per_Priority_Status

- **Description:** This register indicates which Tx Per Priority Counters of the selected priority (PRMMCSEL field of MMC Control register) had an interrupt.
- **Size:** 32 bits
- **Offset:** 0x8d0
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

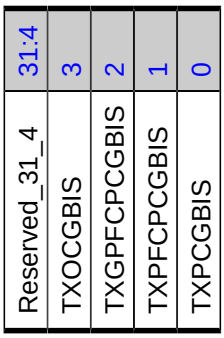


Table 11-180 Fields for Register: Tx_Per_Priority_Status

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	TXOCGBIS	R	Tx Good Bad Per Priority Octet Counter Interrupt Status The bit is set when the Tx_Per_Priority_Octet_Good_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
2	TXGPFCPCGBIS	R	Tx Good Bad Per Priority GPFC Packet Counter Interrupt Status The bit is set when the Tx_Per_Priority_GPFC_Pkt_Good_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
1	TXPFCPCGBIS	R	Tx Good Bad Per Priority PFC Packet Counter Interrupt Status The bit is set when the Tx_Per_Priority_PFC_Pkt_Good_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-180 Fields for Register: Tx_Per_Priority_Status (continued)

Bits	Name	Memory Access	Description
0	TXPCGBIS	R	Tx Good Bad Per Priority Packet Counter Interrupt Status The bit is set when the Tx_Per_Priority_Pkt_Good_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

11.1.177 Tx_Per_Priority_Pkt_Good_Bad

- **Description:** This register provides transmitted per priority good bad packets count.
- **Size:** 32 bits
- **Offset:** 0x8d4
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

Table 11-181 Fields for Register: Tx_Per_Priority_Pkt_Good_Bad

Bits	Name	Memory Access	Description
31:0	TXPPPKTGB	R	Tx Good Bad Per Priority Packet Count This field indicates the number of packets transmitted for each priority. This field is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.178 Tx_Per_Priority_PFC_Pkt_Good_Bad

- **Description:** This register provides transmitted per priority good bad PFC packets count.
- **Size:** 32 bits
- **Offset:** 0x8d8
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

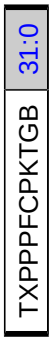


Table 11-182 Fields for Register: Tx_Per_Priority_PFC_Pkt_Good_Bad

Bits	Name	Memory Access	Description
31:0	TXPPPFCPKTGB	R	Tx Good Bad Per Priority PFC Packet Count This field indicates the number of PFC packets transmitted for each priority. This is the PFC packets generated by DWC_25gmac (either by hardware trigger). If transmitted PFC packet is for multiple priorities, then all the priority counters is incremented. **Value After Reset:** 0x0 **Exists:** Always

11.1.179 Tx_Per_Priority_GPFC_Pkt_Good_Bad

- **Description:** This register provides transmitted per priority good bad generated PFC packets count.
- **Size:** 32 bits
- **Offset:** 0x8dc
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

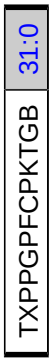


Table 11-183 Fields for Register: Tx_Per_Priority_GPFC_Pkt_Good_Bad

Bits	Name	Memory Access	Description
31:0	TXPPGPFCPKTGB	R	Tx Good Bad Per Priority Generated PFC Packet Count This field indicates the number of PFC packets transmitted for each priority. This is the PFC packets generated by DWC_25gmac (either by software trigger). If transmitted PFC packet is for multiple priorities, then all the priority counters is incremented. **Value After Reset:** 0x0 **Exists:** Always

11.1.180 Tx_Per_Priority_Octet_Good_Bad_Low

- **Description:** This register provides lower 32 bits of transmitted per priority good bad octets count.
- **Size:** 32 bits
- **Offset:** 0x8e0
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

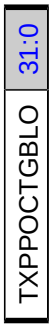


Table 11-184 Fields for Register: Tx_Per_Priority_Octet_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	TXPPOCTGBLO	R	Tx Good Bad Per Priority Octet Count Low This field indicates the lower 32 bits of number of octets transmitted for each priority. This field is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.181 Tx_Per_Priority_Octet_Good_Bad_High

- **Description:** This register provides upper 32 bits of transmitted per priority good bad octets count.
- **Size:** 32 bits
- **Offset:** 0x8e4
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

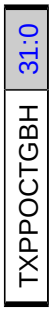


Table 11-185 Fields for Register: Tx_Per_Priority_Octet_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	TXPPOCTGBH	R	Tx Good Bad Per Priority Octet Count High This field indicates the upper 32 bits of number of octets transmitted for each priority. This field is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.182 Rx_Packet_Count_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x900
- **Exists:** (DWCXG_MMC_RXFRMGB_CNT_EN)

Table 11-186 Fields for Register: Rx_Packet_Count_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RXPKTGBLO	R	Rx Good Bad Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and bad packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.183 Rx_Packet_Count_Good_Bad_High

- **Description:** This register provides upper 32 bits of received good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x904
- **Exists:** (DWCXG_MMC_RXFRMGB_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-187 Fields for Register: Rx_Packet_Count_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RXPKTGBHI	R	Rx Good Bad Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and bad packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.184 Rx_Octet_Count_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received octet count, excluding preamble, in good and bad packets.
- **Size:** 32 bits
- **Offset:** 0x908
- **Exists:** (DWCXG_MMC_RXOCTGB_CNT_EN)

Table 11-188 Fields for Register: Rx_Octet_Count_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RXOCTGBLO	R	Rx Good Bad Octet Count Low This field indicates the lower 32 bits of received octet count, where octet count is the number of bytes received, excluding preamble, in good and bad packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.185 Rx_Octet_Count_Good_Bad_High

- **Description:** This register provides upper 32 bits of received octet count, excluding preamble, in good and bad packets.
- **Size:** 32 bits
- **Offset:** 0x90c
- **Exists:** (DWCXG_MMC_RXOCTGB_CNT_EN)

Table 11-189 Fields for Register: Rx_Octet_Count_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RXOCTGBHI	R	Rx Good Bad Octet Count High This field indicates upper 32 bits of received octet count, where octet count is the number of bytes received, excluding preamble, in good and bad packets (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.186 Rx_Octet_Count_Good_Low

- **Description:** This register provides lower 32 bits of received good octets count.
- **Size:** 32 bits
- **Offset:** 0x910
- **Exists:** (DWCXG_MMC_RXOCTG_CNT_EN)

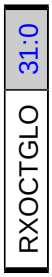


Table 11-190 Fields for Register: Rx_Octet_Count_Good_Low

Bits	Name	Memory Access	Description
31:0	RXOCTGLO	R	Rx Good Octet Count Low This field indicates lower 32 bits of the received octet count, where the octet count is the number of bytes received, excluding preamble, in good packets only. **Value After Reset:** 0x0 **Exists:** Always

11.1.187 Rx_Octet_Count_Good_High

- **Description:** This register provides upper 32 bits of received good octets count.
- **Size:** 32 bits
- **Offset:** 0x914
- **Exists:** (DWCXG_MMC_RXOCTG_CNT_EN)

Table 11-191 Fields for Register: Rx_Octet_Count_Good_High

Bits	Name	Memory Access	Description
31:0	RXOCTGHI	R	Rx Good Octet Count High This field indicates upper 32 bits of the received octet count, where the octet count is the number of bytes received, excluding preamble, in good packets only (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.188 Rx_Broadcast_Packets_Good_Low

- **Description:** This register provides lower 32 bits of received good broadcast packets count.
- **Size:** 32 bits
- **Offset:** 0x918
- **Exists:** (DWCXG_MMC_RXBCASTG_CNT_EN)

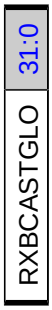


Table 11-192 Fields for Register: Rx_Broadcast_Packets_Good_Low

Bits	Name	Memory Access	Description
31:0	RXBCASTGLO	R	Rx Good Broadcast Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good broadcast packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.189 Rx_Broadcast_Packets_Good_High

- **Description:** This register provides upper 32 bits of received good broadcast packets count.
- **Size:** 32 bits
- **Offset:** 0x91c
- **Exists:** (DWCXG_MMC_RXBCASTG_CNT_EN&&DWCXG_EN_64BIT_CNT)

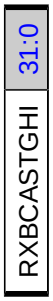


Table 11-193 Fields for Register: Rx_Broadcast_Packets_Good_High

Bits	Name	Memory Access	Description
31:0	RXBCASTGHI	R	Rx Good Broadcast Packet Count High This field indicates upper 32 bits of the counter that counts the number of good broadcast packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.190 Rx_Multicast_Packets_Good_Low

- **Description:** This register provides lower 32 bits of received good multicast packets count.
- **Size:** 32 bits
- **Offset:** 0x920
- **Exists:** (DWCXG_MMC_RXMCASTG_CNT_EN)

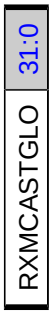


Table 11-194 Fields for Register: Rx_Multicast_Packets_Good_Low

Bits	Name	Memory Access	Description
31:0	RXMCASTGLO	R	Rx Good Multicast Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good multicast packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.191 Rx_Multicast_Packets_Good_High

- **Description:** This register provides upper 32 bits of received good multicast packets count.
- **Size:** 32 bits
- **Offset:** 0x924
- **Exists:** (DWCXG_MMC_RXMCASTG_CNT_EN&&DWCXG_EN_64BIT_CNT)

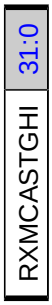


Table 11-195 Fields for Register: Rx_Multicast_Packets_Good_High

Bits	Name	Memory Access	Description
31:0	RXMCASTGHI	R	Rx Good Multicast Packet Count High This field indicates upper 32 bits of the counter that counts the number of good multicast packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.192 Rx_CRC_Error_Packets_Low

- **Description:** This register provides lower 32 bits of received packets with CRC error count.
- **Size:** 32 bits
- **Offset:** 0x928
- **Exists:** (DWCXG_MMC_RXCRCERR_CNT_EN)

Table 11-196 Fields for Register: Rx_CRC_Error_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXCRCERLO	R	Rx CRC Error Packet Count Low This field indicates lower 32 bits of the counter that counts the number of packets received with a CRC error. **Value After Reset:** 0x0 **Exists:** Always

11.1.193 Rx_CRC_Error_Packets_High

- **Description:** This register provides upper 32 bits of received packets with CRC error count.
- **Size:** 32 bits
- **Offset:** 0x92c
- **Exists:** (DWCXG_MMC_RXCRCERR_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-197 Fields for Register: Rx_CRC_Error_Packets_High

Bits	Name	Memory Access	Description
31:0	RXCRCERHI	R	Rx CRC Error Packet Count High This field indicates upper 32 bits of the counter that counts the number of packets received with a CRC error (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.194 Rx_Runt_Error_Packets

- **Description:** This register provides received packets with Runt error count.
- **Size:** 32 bits
- **Offset:** 0x930
- **Exists:** (DWCXG_MMC_RXRUNTERR_CNT_EN)

Table 11-198 Fields for Register: Rx_Runt_Error_Packets

Bits	Name	Memory Access	Description
31:0	RXRUNTER	R	Rx Runt Error Packet Count This field indicates number of shorter-than-64-byte packets received with a CRC error. **Value After Reset:** 0x0 **Exists:** Always

11.1.195 Rx_Jabber_Error_Packets

- **Description:** This register provides received packets with Jabber error count.
- **Size:** 32 bits
- **Offset:** 0x934
- **Exists:** (DWCXG_MMC_RXJABERR_CNT_EN)

Table 11-199 Fields for Register: Rx_Jabber_Error_Packets

Bits	Name	Memory Access	Description
31:0	RXJABERER	R	Rx Jabber Error Packet Count This field indicates number of giant (including CRC) packets received with a CRC error. The giant packet status is generated as per the programming described in MAC_Rx_Configuration. **Value After Reset:** 0x0 **Exists:** Always

11.1.196 Rx_Undersize_Packets_Good

- **Description:** This register provides count of received good undersize packets.
- **Size:** 32 bits
- **Offset:** 0x938
- **Exists:** (DWCXG_MMC_RXUNDERSZG_CNT_EN)

Table 11-200 Fields for Register: Rx_Undersize_Packets_Good

Bits	Name	Memory Access	Description
31:0	RXUSIZEG	R	Rx Good Undersize Packet Count This field indicates number of shorter-than-64-byte packets received without any errors. **Value After Reset:** 0x0 **Exists:** Always

11.1.197 Rx_Oversize_Packets_Good

- **Description:** This register provides count of received good oversize packets.
- **Size:** 32 bits
- **Offset:** 0x93c
- **Exists:** (DWCXG_MMC_RXOVERSZG_CNT_EN)

Table 11-201 Fields for Register: Rx_Oversize_Packets_Good

Bits	Name	Memory Access	Description
31:0	RXOSIZEG	R	Rx Good Oversize Packet Count This field indicates number of greater than the maxsize packets received without errors. In other words, the number of good giant packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.198 Rx_64Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received 64 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x940
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN)

Table 11-202 Fields for Register: Rx_64Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RX64OCTGBLO	R	Rx Good Bad 64 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and bad packets received that are of size 64 bytes, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.199 Rx_64Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received 64 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x944
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-203 Fields for Register: Rx_64Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RX64OCTGBHI	R	Rx Good Bad 64 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and bad packets received that are of size 64 bytes, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.200 Rx_65To127Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received 65-to-127 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x948
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN)

Table 11-204 Fields for Register: Rx_65To127Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RX65_127OCTGBLO	R	Rx Good Bad 65-to-127 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 65-to-127-byte good and bad packets received, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.201 Rx_65To127Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received 65-to-127 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x94c
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-205 Fields for Register: Rx_65To127Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RX65_127OCTGBHI	R	Rx Good Bad 65-to-127 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 65-to-127-byte good and bad packets received, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.202 Rx_128To255Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received 128-to-255 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x950
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN)

Table 11-206 Fields for Register: Rx_128To255Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RX128_255OCTGBLO	R	Rx Good Bad 128-to-255 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 128-to-255-byte good and bad packets received, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.203 Rx_128To255Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received 128-to-255 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x954
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-207 Fields for Register: Rx_128To255Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RX128_255OCTGBHI	R	Rx Good Bad 128-to-255 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 128-to-255-byte good and bad packets received, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.204 Rx_256To511Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received 256-to-511 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x958
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN)

Table 11-208 Fields for Register: Rx_256To511Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RX256_511OCTGBLO	R	Rx Good Bad 256-to-511 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 256-to-511-byte good and bad packets received, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.205 Rx_256To511Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received 256-to-511 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x95c
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-209 Fields for Register: Rx_256To511Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RX256_511OCTGBHI	R	Rx Good Bad 256-to-511 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 256-to-511-byte good and bad packets received, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.206 Rx_512To1023Octets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received 512-to-1023 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x960
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN)

Table 11-210 Fields for Register: Rx_512To1023Octets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RX512_1023OCTGBLO	R	Rx Good Bad 512-to-1023 Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 512-to-1023-byte good and bad packets received, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.207 Rx_512To1023Octets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received 512-to-1023 octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x964
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-211 Fields for Register: Rx_512To1023Octets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RX512_1023OCTGBHI	R	Rx Good Bad 512-to-1023 Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 512-to-1023-byte good and bad packets received, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.208 Rx_1024ToMaxOctets_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received 1024-tomaxsize octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x968
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN)

Table 11-212 Fields for Register: Rx_1024ToMaxOctets_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RX1024_MAXOCTGBLO	R	Rx Good Bad 1024-tomaxsize Octet Size Packet Count Low This field indicates lower 32 bits of the counter that counts the number of 1024-tomaxsize-byte (inclusive) good and bad packets received, excluding preamble. **Value After Reset:** 0x0 **Exists:** Always

11.1.209 Rx_1024ToMaxOctets_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received 1024-tomaxsize octet size good and bad packets count.
- **Size:** 32 bits
- **Offset:** 0x96c
- **Exists:** (DWCXG_MMC_RXADDR_RNG_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-213 Fields for Register: Rx_1024ToMaxOctets_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RX1024_MAXOCTGBHI	R	Rx Good Bad 1024-tomaxsize Octet Size Packet Count High This field indicates upper 32 bits of the counter that counts the number of 1024-tomaxsize-byte (inclusive) good and bad packets received, excluding preamble (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.210 Rx_Unicast_Packets_Good_Low

- **Description:** This register provides lower 32 bits of received good unicast packets count.
- **Size:** 32 bits
- **Offset:** 0x970
- **Exists:** (DWCXG_MMC_RXUCASTG_CNT_EN)

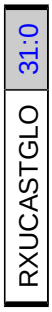


Table 11-214 Fields for Register: Rx_Unicast_Packets_Good_Low

Bits	Name	Memory Access	Description
31:0	RXUCASTGLO	R	Rx Good Unicast Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good unicast packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.211 Rx_Unicast_Packets_Good_High

- **Description:** This register provides upper 32 bits of received good unicast packets count.
- **Size:** 32 bits
- **Offset:** 0x974
- **Exists:** (DWCXG_MMC_RXUCASTG_CNT_EN&&DWCXG_EN_64BIT_CNT)

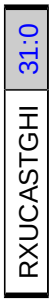


Table 11-215 Fields for Register: Rx_Unicast_Packets_Good_High

Bits	Name	Memory Access	Description
31:0	RXUCASTGHI	R	Rx Good Unicast Packet Count High This field indicates upper 32 bits of the counter that counts the number of good unicast packets received (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.212 Rx_Length_Error_Packets_Low

- **Description:** This register provides lower 32 bits of received packets with Length error count.
- **Size:** 32 bits
- **Offset:** 0x978
- **Exists:** (DWCXG_MMC_RXLENERR_CNT_EN)

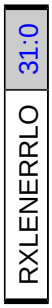


Table 11-216 Fields for Register: Rx_Length_Error_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXLENERRLO	R	Rx Length Error Packet Count Low This field indicates lower 32 bits of the counter that counts the number of packets received with a Length error (Length field not equal to packet size), for all packets with valid Length field. **Value After Reset:** 0x0 **Exists:** Always

11.1.213 Rx_Length_Error_Packets_High

- **Description:** This register provides upper 32 bits of received packets with Length error count.
- **Size:** 32 bits
- **Offset:** 0x97c
- **Exists:** (DWCXG_MMC_RXLENERR_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-217 Fields for Register: Rx_Length_Error_Packets_High

Bits	Name	Memory Access	Description
31:0	RXLENERRHI	R	Rx Length Error Packet Count High This field indicates upper 32 bits of the counter that counts the number of packets received with a Length error (Length field not equal packet size), for all packets with valid Length field (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.214 Rx_OutofRange_Packets_Low

- **Description:** This register provides lower 32 bits of received packets with out of range length field count.
- **Size:** 32 bits
- **Offset:** 0x980
- **Exists:** (DWCXG_MMC_RXOUTOFRNG_TYP_CNT_EN)

Table 11-218 Fields for Register: Rx_OutofRange_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXORANGELO	R	Rx Out of Range Packet Count Low This field indicates lower 32 bits of the counter that counts the number of packets received with Length field not equal to the valid packet size (greater than 1500). **Value After Reset:** 0x0 **Exists:** Always

11.1.215 Rx_OutofRange_Packets_High

- **Description:** This register provides upper 32 bits of received packets with out of range length field count.
- **Size:** 32 bits
- **Offset:** 0x984
- **Exists:** (DWCXG_MMC_RXOUTOFRNG_TYP_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-219 Fields for Register: Rx_OutofRange_Packets_High

Bits	Name	Memory Access	Description
31:0	RXORANGEHI	R	Rx Out of Range Packet Count High This field indicates upper 32 bits of the counter that counts the number of packets received with Length field not equal to the valid packet size (>1500) (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.216 Rx_Pause_Packets_Low

- **Description:** This register provides lower 32 bits of received Pause packets count.
- **Size:** 32 bits
- **Offset:** 0x988
- **Exists:** (DWCXG_MMC_RXPAUSEFRM_CNT_EN)

Table 11-220 Fields for Register: Rx_Pause_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXPAUSELO	R	Rx Pause Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and valid PAUSE packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.217 Rx_Pause_Packets_High

- **Description:** This register provides upper 32 bits of received Pause packets count.
- **Size:** 32 bits
- **Offset:** 0x98c
- **Exists:** (DWCXG_MMC_RXPAUSEFRM_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-221 Fields for Register: Rx_Pause_Packets_High

Bits	Name	Memory Access	Description
31:0	RXPAUSEHI	R	Rx Pause Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and valid PAUSE packets received (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.218 Rx_FIFOOverflow_Packets_Low

- **Description:** This register provides lower 32 bits of received packets count that had Rx Queue overflow.
- **Size:** 32 bits
- **Offset:** 0x990
- **Exists:** (DWCXG_MMC_RXFIFOOVFL_CNT_EN)

Table 11-222 Fields for Register: Rx_FIFOOverflow_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXFOVFLO	R	Rx FIFO Overflow Packet Count Low This field indicates lower 32 bits of the counter that counts the number of receive packets dropped due to FIFO overflow. **Value After Reset:** 0x0 **Exists:** Always

11.1.219 Rx_FIFOOverflow_Packets_High

- **Description:** This register provides upper 32 bits of received packets count that had Rx Queue overflow.
- **Size:** 32 bits
- **Offset:** 0x994
- **Exists:** (DWCXG_MMC_RXFIFOOVFL_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-223 Fields for Register: Rx_FIFOOverflow_Packets_High

Bits	Name	Memory Access	Description
31:0	RXFOVFHI	R	Rx FIFO Overflow Packet Count High This field indicates upper 32 bits of the counter that counts the number of receive packets dropped due to FIFO overflow (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.220 Rx_VLAN_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received good bad VLAN packets count.
- **Size:** 32 bits
- **Offset:** 0x998
- **Exists:** (DWCXG_MMC_RXVLANFRGB_CNT_EN)

Table 11-224 Fields for Register: Rx_VLAN_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RXVLANGBLO	R	Rx Good Bad VLAN Packet Count Low This field indicates lower 32 bits of the counter that counts the number of good and bad VLAN packets received. **Value After Reset:** 0x0 **Exists:** Always

11.1.221 Rx_VLAN_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received good bad VLAN packets count.
- **Size:** 32 bits
- **Offset:** 0x99c
- **Exists:** (DWCXG_MMC_RXVLANFRGB_CNT_EN&&DWCXG_EN_64BIT_CNT)

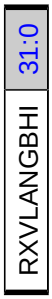


Table 11-225 Fields for Register: Rx_VLAN_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RXVLANGBHI	R	Rx Good Bad VLAN Packet Count High This field indicates upper 32 bits of the counter that counts the number of good and bad VLAN packets received (optional). **Value After Reset:** 0x0 **Exists:** Always

11.1.222 Rx_Watchdog_Error_Packets

- **Description:** This register has a count of packets that are received with watchdog error.
- **Size:** 32 bits
- **Offset:** 0x9a0
- **Exists:** (DWCXG_MMC_RXWDGERR_CNT_EN)

Table 11-226 Fields for Register: Rx_Watchdog_Error_Packets

Bits	Name	Memory Access	Description
31:0	RXWDOGERR	R	Rx Watchdog Error Packet Count This field indicates number of packets received with a watchdog time-out error (packets of length greater than the watchdog time-out limit as described in MAC_WD_JB_Timeout). **Value After Reset:** 0x0 **Exists:** Always

11.1.223 Rx_LPI_USEC_Cntr

- **Description:** This register provides duration in microsecond for which MAC was in Rx LPI.
- **Size:** 32 bits
- **Offset:** 0x9a4
- **Exists:** (DWCXG_MMC_RXLPI_USEC_EN)

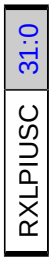


Table 11-227 Fields for Register: Rx_LPI_USEC_Cntr

Bits	Name	Memory Access	Description
31:0	RXLPIUSC	R	Rx LPI microsecond Count This field indicates the number of microseconds DWC_25gmac is in Rx LPI state. **Value After Reset:** 0x0 **Exists:** Always

11.1.224 Rx_LPI_Tran_Cntr

- **Description:** This register provides number of times MAC transitioned to Rx LPI state.
- **Size:** 32 bits
- **Offset:** 0x9a8
- **Exists:** (DWCXG_MMC_RXLPI_TRANS_CNTR_EN)

Table 11-228 Fields for Register: Rx_LPI_Tran_Cntr

Bits	Name	Memory Access	Description
31:0	RXLPITRC	R	Rx LPI Transition Count This field indicates the number of times DWC_25gmac entered the Rx LPI state. **Value After Reset:** 0x0 **Exists:** Always

11.1.225 Rx_Discard_Packets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received good bad packets count that were discarded due to filter fail.
- **Size:** 32 bits
- **Offset:** 0x9ac
- **Exists:** (DWCXG_MMC_RXFRMDISGB_CNT_EN)

Table 11-229 Fields for Register: Rx_Discard_Packets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RXDPCNTGBLO	R	Rx Good Bad Discard Packet Count Low This field indicates the number of packets received and discarded due to Filter Fail. This represents the lower 32 bits of the counter. When a packet is dropped due to filtering and not counted in the normal RMON counters, this counter is incremented. This counter is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.226 Rx_Discard_Packets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received good bad packets count that were discarded due to filter fail.
- **Size:** 32 bits
- **Offset:** 0x9b0
- **Exists:** (DWCXG_MMC_RXFRMDISGB_CNT_EN&&DWCXG_EN_64BIT_CNT)

Table 11-230 Fields for Register: Rx_Discard_Packets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RXDPCNTGBHI	R	Rx Good Bad Discard Packet Count High This field indicates the number of packets received and discarded due to Filter Fail. This represents the upper 32 bits of the counter. When a packet is dropped due to filtering and not counted in the normal RMON counters, this counter is incremented. This counter is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.227 Rx_Discard_Octets_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received good bad octets count that were discarded due to filter fail.
- **Size:** 32 bits
- **Offset:** 0x9b4
- **Exists:** (DWCXG_MMC_RXOCTDISGB_CNT_EN)

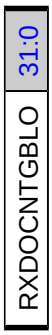


Table 11-231 Fields for Register: Rx_Discard_Octets_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RXDOCNTGBLO	R	Rx Good Bad Discard Octet Count Low This field indicates the number of octets received and discarded due to Filter Fail. This represents the lower 32 bits of the counter. When a packet is dropped due to filtering and not counted in the normal RMON counters, this octet counter is incremented by packet size. This counter is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.228 Rx_Discard_Octets_Good_Bad_High

- **Description:** This register provides upper 32 bits of received good bad octets count that were discarded due to filter fail.
- **Size:** 32 bits
- **Offset:** 0x9b8
- **Exists:** (DWCXG_MMC_RXOCTDISGB_CNT_EN)

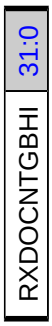


Table 11-232 Fields for Register: Rx_Discard_Octets_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RXDOCNTGBHI	R	Rx Good Bad Discard Octet Count High This field indicates the number of octets received and discarded due to Filter Fail. This represents the upper 32 bits of the counter. Any packet dropped due to filtering and not counted in the normal RMON counters then this octet counter is incremented by packet size. This counter is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.229 Rx_Alignment_Error_Packets

- **Description:** This register provides the number of packets received by DWC_25gmac with alignment (dribble) error. It is valid only in 10/100 mode.
- **Size:** 32 bits
- **Offset:** 0x9bc
- **Exists:** (DWCXG_MMC_RXALGNERR_CNT_EN)

Table 11-233 Fields for Register: Rx_Alignment_Error_Packets

Bits	Name	Memory Access	Description
31:0	RXALIGNERR	R	Rx Alignment Error Packet Count This field indicates the number of packets received with alignment (dribble) error. It is valid only in 10/100 mode. **Value After Reset:** 0x0 **Exists:** Always

11.1.230 RMMC_RCHM_SKP_CNTR_LOW

- **Description:** This register provides lower 32 bits of skip counts of the Rx Channel Mapped Rx Frame/Priority MMC. Basically it counts number of times, when such MMC are forbidden to update due to non-mapped receive-channel frame routing on specific scenarios.
- **Size:** 32 bits
- **Offset:** 0x9c8
- **Exists:**
  DWCXG_RMMC_RCH_MAP&&(DWCXG_MMC_RX_EN_ATLEAST_ONE||DWCXG_MMC_PRIORITY_EN)

Table 11-234 Fields for Register: RMMC_RCHM_SKP_CNTR_LOW

Bits	Name	Memory Access	Description
31:0	RRSCL	R	Rx Channel Mapped Rx Frame/Priority MMC Skip Count Low This field indicates the number of times Rx Channel Mapped Rx Frame/Priority MMC are forbidden to update due to non-mapped Rx Channel frame routing on specific scenarios. This represents the lower 32 bits of the counter. For more detail refer, feature specific functional description. **Value After Reset:** 0x0 **Exists:** Always

11.1.231 RMMC_RCHM_SKP_CNTR_HIGH

- **Description:** This register provides upper 32 bits of skip counts of the Rx Channel Mapped Rx Frame/Priority MMC. Basically it counts number of times, when such MMC are forbidden to update due to non-mapped receive-channel frame routing on specific scenarios.
- **Size:** 32 bits
- **Offset:** 0x9cc
- **Exists:** DWCXG_RMMC_RCHM_SKP_CNTR_EN&&DWCXG_EN_64BIT_CNT

Table 11-235 Fields for Register: RMMC_RCHM_SKP_CNTR_HIGH

Bits	Name	Memory Access	Description
31:0	RRSCH	R	Rx Channel Mapped Rx Frame/Priority MMC Skip Count High This field indicates the number of times Rx Channel Mapped Rx Frame/Priority MMC are forbidden to update due to non-mapped Rx Channel frame routing on specific scenarios. This represents the upper 32 bits of the counter. For more detail refer, feature specific functional description. **Value After Reset:** 0x0 **Exists:** Always

11.1.232 Rx_Per_Priority_Status

- **Description:** This register indicates which Rx Per Priority Counters of the selected priority (PRMMCSEL field of MMC Control register) had an interrupt.
- **Size:** 32 bits
- **Offset:** 0x9d0
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

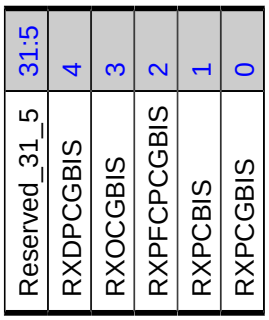


Table 11-236 Fields for Register: Rx_Per_Priority_Status

Bits	Name	Memory Access	Description
31:5	Reserved_31_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	RXDPCGBIS	R	Rx Good Bad Per Priority Discard Packet Counter Interrupt Status The bit is set when the Rx_Per_Priority_Discard_Good_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
3	RXOCGBIS	R	Rx Good Bad Per Priority Octet Counter Interrupt Status The bit is set when the Rx_Per_Priority_Octet_Good_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
2	RXPFCPCGBIS	R	Rx Good Bad Per Priority PFC Packet Counter Interrupt Status The bit is set when the Rx_Per_Priority_PFC_Pkt_Good_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-236 Fields for Register: Rx_Per_Priority_Status (continued)

Bits	Name	Memory Access	Description
1	RXPCBIS	R	Rx Bad Per Priority Packet Counter Interrupt Status The bit is set when the Rx_Per_Priority_Pkt_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always
0	RXPCGBIS	R	Rx Good Bad Per Priority Packet Counter Interrupt Status The bit is set when the Rx_Per_Priority_Pkt_Good_Bad counter reaches the threshold value programmed in the MMC control register. **Value After Reset:** 0x0 **Exists:** Always

11.1.233 Rx_Per_Priority_Pkt_Good_Bad

- **Description:** This register provides received per priority good bad packets count.
- **Size:** 32 bits
- **Offset:** 0x9d4
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

Table 11-237 Fields for Register: Rx_Per_Priority_Pkt_Good_Bad

Bits	Name	Memory Access	Description
31:0	RXPPPKTGB	R	Rx Good Bad Per Priority Packet Count This field indicates the number of tagged packets received for each priority. This field is incremented irrespective of whether the packet is good or bad. **Value After Reset:** 0x0 **Exists:** Always

11.1.234 Rx_Per_Priority_Pkt_Bad

- **Description:** This register provides received per priority bad packets count.
- **Size:** 32 bits
- **Offset:** 0x9d8
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

Table 11-238 Fields for Register: Rx_Per_Priority_Pkt_Bad

Bits	Name	Memory Access	Description
31:0	RXPPPKTB	R	Rx Bad Per Priority Packet Count This field indicates the number of bad packets received for each priority. This field is incremented only for bad packets (CRC Error or giant packets). Giant packets is determined by MAC_Rx_Configuration register. **Value After Reset:** 0x0 **Exists:** Always

11.1.235 Rx_Per_Priority_PFC_Pkt_Good_Bad

- **Description:** This register provides received per priority good bad PFC packets count.
- **Size:** 32 bits
- **Offset:** 0x9dc
- **Exists:** (DWCXG_MMC_PRIORITY_EN&&DWCXG_PFC_EN)

Table 11-239 Fields for Register: Rx_Per_Priority_PFC_Pkt_Good_Bad

Bits	Name	Memory Access	Description
31:0	RXPPPFCPKTGB	R	Rx Good Bad Per Priority PFC Packet Count This field indicates the number of PFC packets received for each priority. If the received PFC packet is for multiple priorities, then all the priority counters are incremented. This counter is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.236 Rx_Per_Priority_Octet_Good_Bad_Low

- **Description:** This register provides lower 32 bits of received per priority good bad octets count.
- **Size:** 32 bits
- **Offset:** 0x9e0
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

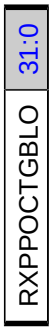


Table 11-240 Fields for Register: Rx_Per_Priority_Octet_Good_Bad_Low

Bits	Name	Memory Access	Description
31:0	RXPPOCTGBLO	R	Rx Good Bad Per Priority Octet Count Low This field indicates the lower 32 bits of number of octets received for each priority. This field is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.237 Rx_Per_Priority_Octet_Good_Bad_High

- **Description:** This register provides upper 32 bits of received per priority good bad octets count.
- **Size:** 32 bits
- **Offset:** 0x9e4
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

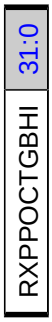


Table 11-241 Fields for Register: Rx_Per_Priority_Octet_Good_Bad_High

Bits	Name	Memory Access	Description
31:0	RXPPOCTGBHI	R	Rx Good Bad Per Priority Octet Count High This field indicates the upper 32 bits of number of octets received for each priority. This field is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.238 Rx_Per_Priority_Discard_Good_Bad

- **Description:** This register provides received per priority discarded good bad packets count.
- **Size:** 32 bits
- **Offset:** 0x9e8
- **Exists:** (DWCXG_MMC_PRIORITY_EN)

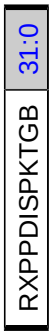


Table 11-242 Fields for Register: Rx_Per_Priority_Discard_Good_Bad

Bits	Name	Memory Access	Description
31:0	RXPPDISPKTGB	R	Rx Good Bad Per Priority Discarded Packets Count This field indicates the packets received and discarded (due to filter fail) for each priority. This field is incremented irrespective of whether it is a good or bad packet. **Value After Reset:** 0x0 **Exists:** Always

11.1.239 SGF_Pass_PktCnt

- **Description:** This register provides the number of Stream-Gate Filter (SGF) Passed Packets
- **Size:** 32 bits
- **Offset:** 0x9f0
- **Exists:** (DWCXG_SGF)

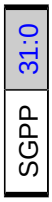


Table 11-243 Fields for Register: SGF_Pass_PktCnt

Bits	Name	Memory Access	Description
31:0	SGPP	R	Stream-Gate Filter Pass Packets Count This field indicates the number of SGF Passed Packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.240 SGF_Fail_PktCnt

- **Description:** This register provides the number of SGF Failed Packets
- **Size:** 32 bits
- **Offset:** 0x9f4
- **Exists:** (DWCXG_SGF)

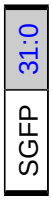


Table 11-244 Fields for Register: SGF_Fail_PktCnt

Bits	Name	Memory Access	Description
31:0	SGFP	R	Stream-Gate Filter Failed Packets Count This field indicates the number of SGF Failed Packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.241 MMC_FPE_Tx_Interrupt

- **Description:** This register maintains the interrupts generated from all FPE related Transmit statistics counters. The MMC FPE Transmit Interrupt register maintains the interrupts generated when the transmit statistic counters reach the threshold value programmed in MMC control register (for example, the counter's MSB is set). This register is 32 bits wide. An interrupt bit is cleared when the respective MMC counter that caused the interrupt is read. The least significant byte lane (bits[7:0]) of the corresponding counter must be read to clear the interrupt status bit.
- **Size:** 32 bits
- **Offset:** 0xa00
- **Exists:** ((DWCXG_FPE||DWCXG_AV_EST)&&DWCXG_MMC_FPE_EN)

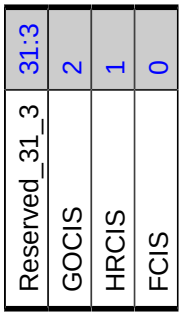


Table 11-245 Fields for Register: MMC_FPE_Tx_Interrupt

Bits	Name	Memory Access	Description
31:3	Reserved_31_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	GOCIS	R	MMC Tx Gate/Window Overrun Counter Interrupt Status This bit is set when the Tx_Gate_Orun_Cntr_Low/High counter reaches the threshold value programmed in MMC control register. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Tx Gate/Window Overrun Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Tx Gate/Window Overrun Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always

Table 11-245 Fields for Register: MMC_FPE_Tx_Interrupt (continued)

Bits	Name	Memory Access	Description
1	HRCIS	R	MMC Tx Hold Request Counter Interrupt Status This bit is set when the Tx_Hold_Req_Cntr counter reaches the threshold value programmed in MMC control register. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Tx Hold Request Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Tx Hold Request Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_FPE)
0	FCIS	R	MMC Tx FPE Fragment Counter Interrupt status This bit is set when the Tx_FPE_Fragment_Cntr counter reaches the threshold value programmed in MMC control register. Access restriction applies. Clears on read. Self-set to 1 on internal event. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (INACTIVE): MMC Tx FPE Fragment Counter Interrupt status not detected ■ 0x1 (ACTIVE): MMC Tx FPE Fragment Counter Interrupt status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_FPE)

11.1.242 MMC_FPE_Tx_Interrupt_Mask

- **Description:** This register maintains the masks for interrupts generated from all FPE related Transmit statistics counters. The MMC Receive Interrupt Mask register maintains the masks for the interrupts generated when FPE related receive statistic counters reach the threshold value programmed in MMC control register. This register is 32 bit wide.
- **Size:** 32 bits
- **Offset:** 0xa04
- **Exists:** ((DWCXG_FPE||DWCXG_AV_EST)&&DWCXG_MMC_FPE_EN)

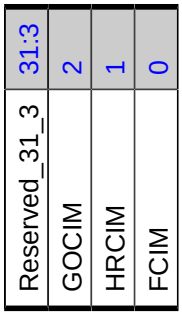


Table 11-246 Fields for Register: MMC_FPE_Tx_Interrupt_Mask

Bits	Name	Memory Access	Description
31:3	Reserved_31_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	GOCIM	R/W	MMC Transmit Gate/Window Overrun Counter Interrupt Mask Setting this bit masks the interrupt when the Tx_Gate_Orun_Cntr_Low/High counter the threshold value programmed in MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Gate/Window Overrun Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Transmit Gate/Window Overrun Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** Always

Table 11-246 Fields for Register: MMC_FPE_Tx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
1	HRCIM	R/W	MMC Transmit Hold Request Counter Interrupt Mask Setting this bit masks the interrupt when the Tx_Hold_Req_Cntr counter the threshold value programmed in MMC control register. **Values:** ■ 0x0 (DISABLE): MMC Transmit Hold Request Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Transmit Hold Request Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_FPE)
0	FCIM	R/W	MMC Transmit Fragment Counter Interrupt Mask Setting this bit masks the interrupt when the Tx_FPE_Fragment_Cntr counter reaches the threshold value programmed in MMC control register. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (DISABLE): MMC Transmit Fragment Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Transmit Fragment Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_FPE)

11.1.243 MMC_Tx_FPE_Fragment_Cntr

- **Description:** This register provides the number of additional mPackets transmitted due to preemption.
- **Size:** 32 bits
- **Offset:** 0xa08
- **Exists:** (DWCXG_FPE&&DWCXG_MMC_FPE_EN)

Table 11-247 Fields for Register: MMC_Tx_FPE_Fragment_Cntr

Bits	Name	Memory Access	Description
31:0	TXFFC	R	Tx FPE Fragment counter This field indicates the number of additional mPackets transmitted due to preemption Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Value After Reset:** 0x0 **Exists:** Always

11.1.244 MMC_Tx_Hold_Req_Cntr

- **Description:** This register provides the count of number of times a hold request is given to MAC
- **Size:** 32 bits
- **Offset:** 0xa0c
- **Exists:** (DWCXG_FPE&&DWCXG_MMC_FPE_EN)

Table 11-248 Fields for Register: MMC_Tx_Hold_Req_Cntr

Bits	Name	Memory Access	Description
31:0	TXHRC	R	Tx Hold Request Counter This field indicates count of number of a hold request is given to MAC. **Value After Reset:** 0x0 **Exists:** Always

11.1.245 MMC_Tx_Gate_Orun_Cntr_Low

- **Description:** This register provides the lower 32-bits of counter that counts the number of times packets overrun the transmission gate/window.
- **Size:** 32 bits
- **Offset:** 0xa10
- **Exists:** ((DWCXG_AV_EST||DWCXG_FPE)&&DWCXG_MMC_FPE_EN)

Table 11-249 Fields for Register: MMC_Tx_Gate_Orun_Cntr_Low

Bits	Name	Memory Access	Description
31:0	TXGOCL	R	Tx Gate/Window Overrun Counter Low This field indicates the lower 32-bits of counter that counts the number of times packets overrun the transmission gate/window. In FPE mode, the counter is updated based on pre-empt packet overrunning in Hold window. In EST mode, the counter is updated based on any packet of the queue overrunning after gate closed for the Tx Queue. When both enabled together, the counter is updated based on both the conditions. **Value After Reset:** 0x0 **Exists:** Always

11.1.246 MMC_Tx_Gate_Orun_Cntr_High

- **Description:** This register provides the upper 32-bits of counter that counts the number of times packets overrun the transmission gate/window.
- **Size:** 32 bits
- **Offset:** 0xa14
- **Exists:** ((DWCXG_AV_EST||DWCXG_FPE)&&DWCXG_MMC_FPE_EN)

Table 11-250 Fields for Register: MMC_Tx_Gate_Orun_Cntr_High

Bits	Name	Memory Access	Description
31:0	TXGOCH	R	Tx Gate/Window Overrun Counter High This field indicates the upper 32-bits of counter that counts the number of times packets overrun the transmission gate/window. In FPE mode, the counter is updated based on pre-empt packet overrunning in Hold window. In EST mode, the counter is updated based on any packet of the queue overrunning after gate closed for the Tx Queue. When both enabled together, the counter is updated based on both the conditions. **Value After Reset:** 0x0 **Exists:** Always

11.1.247 MMC_FPE_Rx_Interrupt

- **Description:** This register maintains the interrupts generated from all FPE related Receive statistics counters. The MMC FPE Receive Interrupt register maintains the interrupts generated when receive statistic counters reach the threshold value programmed in MMC control register (for example, the counter's MSB is set). The MMC FPE Receive Interrupt register is a 32 bit register. An interrupt bit is cleared when the respective MMC counter that caused the interrupt is read. The least significant byte lane (Bits[7:0]) of the respective counter must be read to clear the interrupt bit.
- **Size:** 32 bits
- **Offset:** 0xa20
- **Exists:** (DWCXG_FPE&&DWCXG_MMC_FPE_EN)

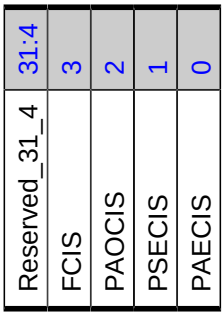


Table 11-251 Fields for Register: MMC_FPE_Rx_Interrupt

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	FCIS	R	MMC Rx FPE Fragment Counter Interrupt Status This bit is set when the Rx_FPE_Fragment_Cntr counter reach the threshold value programmed in MMC control register. Access restriction applies. Clears on read. Self-set to 1 on internal event. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (INACTIVE): MMC Rx FPE Fragment Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Rx FPE Fragment Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always

Table 11-251 Fields for Register: MMC_FPE_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
2	PAOCIS	R	MMC Rx Packet Assembly OK Counter Interrupt Status This bit is set when the Rx_Packet_Assemble_Ok_Cntr counter reach the threshold value programmed in MMC control register. Access restriction applies. Clears on read. Self-set to 1 on internal event. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (INACTIVE): MMC Rx Packet Assembly OK Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Rx Packet Assembly OK Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always
1	PSECIS	R	MMC Rx Packet SMD Error Counter Interrupt Status This bit is set when the Rx_Packet_SMD_Err_Cntr counter reaches the threshold value programmed in MMC control register. Access restriction applies. Clears on read. Self-set to 1 on internal event. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (INACTIVE): MMC Rx Packet SMD Error Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Rx Packet SMD Error Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always

Table 11-251 Fields for Register: MMC_FPE_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
0	PAECIS	R	MMC Rx Packet Assembly Error Counter Interrupt Status This bit is set when the Rx_Packet_Assemble_Err_Cntr counter reaches the threshold value programmed in MMC control register. Access restriction applies. Clears on read. Self-set to 1 on internal event. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (INACTIVE): MMC Rx Packet Assembly Error Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Rx Packet Assembly Error Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always

11.1.248 MMC_FPE_Rx_Interrupt_Mask

- **Description:** This register maintains the masks for interrupts generated from all FPE related Receive statistics counters. The MMC Receive Interrupt Mask register maintains the masks for the interrupts generated when FPE related receive statistic counters reach the threshold value programmed in MMC control register. This register is 32 bit wide.
- **Size:** 32 bits
- **Offset:** 0xa24
- **Exists:** (DWCXG_FPE&&DWCXG_MMC_FPE_EN)

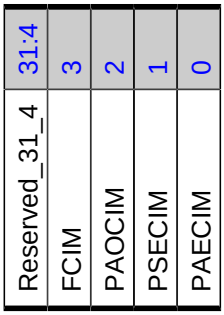


Table 11-252 Fields for Register: MMC_FPE_Rx_Interrupt_Mask

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	FCIM	R/W	MMC Rx FPE Fragment Counter Interrupt Mask Setting this bit masks the interrupt when the Tx_FPE_Fragment_Cntr counter reaches the threshold value programmed in MMC control register. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (DISABLE): MMC Rx FPE Fragment Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Rx FPE Fragment Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** Always

Table 11-252 Fields for Register: MMC_FPE_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
2	PAOCIM	R/W	MMC Rx Packet Assembly OK Counter Interrupt Mask Setting this bit masks the interrupt when the Rx_Packet_Assemble_Ok_Cntr counter reaches the threshold value programmed in MMC control register. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (DISABLE): MMC Rx Packet Assembly OK Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Rx Packet Assembly OK Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** Always
1	PSECIM	R/W	MMC Rx Packet SMD Error Counter Interrupt Mask Setting this bit masks the interrupt when the Rx_Packet_SMD_Err_Cntr counter reaches the threshold value programmed in MMC control register. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (DISABLE): MMC Rx Packet SMD Error Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Rx Packet SMD Error Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** Always
0	PAECIM	R/W	MMC Rx Packet Assembly Error Counter Interrupt Mask Setting this bit masks the interrupt when the Rx_Packet_Assemble_Err_Cntr counter reaches the threshold value programmed in MMC control register. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Values:** ■ 0x0 (DISABLE): MMC Rx Packet Assembly Error Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Rx Packet Assembly Error Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** Always

11.1.249 MMC_Rx_Packet_Assembly_Err_Cntr

- **Description:** This register provides the number of MAC frames with reassembly errors on the Receiver, due to mismatch in the Fragment Count value.
- **Size:** 32 bits
- **Offset:** 0xa28
- **Exists:** (DWCXG_FPE&&DWCXG_MMC_FPE_EN)

Table 11-253 Fields for Register: MMC_Rx_Packet_Assembly_Err_Cntr

Bits	Name	Memory Access	Description
31:0	PAEC	R	Rx Packet Assembly Error Counter This field indicates the number of MAC frames with reassembly errors on the Receiver, due to mismatch in the Fragment Count value. Exists when any one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Value After Reset:** 0x0 **Exists:** Always

11.1.250 MMC_Rx_Packet_SMD_Err_Cntr

- **Description:** This register provides the number of received MAC frames rejected due to unknown SMD value and MAC frame fragments rejected due to arriving with an SMD-C when there was no preceding preempted frame.
- **Size:** 32 bits
- **Offset:** 0xa2c
- **Exists:** (DWCXG_FPE&&DWCXG_MMC_FPE_EN)

Table 11-254 Fields for Register: MMC_Rx_Packet_SMD_Err_Cntr

Bits	Name	Memory Access	Description
31:0	PSEC	R	Rx Packet SMD Error Counter This field indicates the number of MAC frames rejected due to unknown SMD value and MAC frame fragments rejected due to arriving with an SMD-C when there was no preceding preempted frame. Exists when at least one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Value After Reset:** 0x0 **Exists:** Always

11.1.251 MMC_Rx_Packet_Assembly_OK_Cntr

- **Description:** This register provides the number of MAC frames that were successfully reassembled and delivered to MAC.
- **Size:** 32 bits
- **Offset:** 0xa30
- **Exists:** (DWCXG_FPE&&DWCXG_MMC_FPE_EN)

Table 11-255 Fields for Register: MMC_Rx_Packet_Assembly_OK_Cntr

Bits	Name	Memory Access	Description
31:0	PAOC	R	Rx Packet Assembly OK Counter This field indicates the number of MAC frames that were successfully reassembled and delivered to MAC. Exists when at least one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Value After Reset:** 0x0 **Exists:** Always

11.1.252 MMC_Rx_FPE_Fragment_Cntr

- **Description:** This register provides the number of additional mPackets received due to preemption.
- **Size:** 32 bits
- **Offset:** 0xa34
- **Exists:** (DWCXG_FPE&&DWCXG_MMC_FPE_EN)

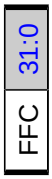


Table 11-256 Fields for Register: MMC_Rx_FPE_Fragment_Cntr

Bits	Name	Memory Access	Description
31:0	FFC	R	Rx FPE Fragment Counter This field indicates the number of additional mPackets received due to preemption Exists when at least one of the Rx/Tx MMC counters are enabled during FPE Enabled configuration. **Value After Reset:** 0x0 **Exists:** Always

11.1.253 Tx_Single_Collision_Good_Packets

- **Description:** This register provides the number of successfully transmitted packets by DWC_25gmac after a single collision in the half-duplex mode.
- **Size:** 32 bits
- **Offset:** 0xa40
- **Exists:** ((DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY))

Table 11-257 Fields for Register: Tx_Single_Collision_Good_Packets

Bits	Name	Memory Access	Description
31:0	TXSNGLCOLG	R	Tx Single Collision Good Packets This field indicates the number of successfully transmitted packets after a single collision in the half-duplex mode. **Value After Reset:** 0x0 **Exists:** Always

11.1.254 Tx_Multiple_Collision_Good_Packets

- **Description:** This register provides the number of successfully transmitted packets by DWC_25gmac after multiple collisions in the half-duplex mode.
- **Size:** 32 bits
- **Offset:** 0xa44
- **Exists:** ((DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY))

Table 11-258 Fields for Register: Tx_Multiple_Collision_Good_Packets

Bits	Name	Memory Access	Description
31:0	TXMULTCOLG	R	Tx Multiple Collision Good Packets This field indicates the number of successfully transmitted packets after multiple collisions in the half-duplex mode. **Value After Reset:** 0x0 **Exists:** Always

11.1.255 Tx_Deferred_Packets

- **Description:** This register provides the number of successfully transmitted by DWC_25gmac after a deferral in the half-duplex mode.
- **Size:** 32 bits
- **Offset:** 0xa48
- **Exists:** ((DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY))

Table 11-259 Fields for Register: Tx_Deferred_Packets

Bits	Name	Memory Access	Description
31:0	TXDEFRD	R	Tx Deferred Packets This field indicates the number of successfully transmitted packets after a deferral, in the half-duplex mode. **Value After Reset:** 0x0 **Exists:** Always

11.1.256 Tx_Late_Collision_Packets

- **Description:** This register provides the number of packets aborted by DWC_25gmac because of late collision error.
- **Size:** 32 bits
- **Offset:** 0xa4c
- **Exists:** ((DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY))

Table 11-260 Fields for Register: Tx_Late_Collision_Packets

Bits	Name	Memory Access	Description
31:0	TXLATECOL	R	Tx Late Collision Packets This field indicates the number of packets aborted because of late collision error. **Value After Reset:** 0x0 **Exists:** Always

11.1.257 Tx_Excessive_Collision_Packets

- **Description:** This register provides the number of packets aborted by DWC_25gmac because of excessive (16) collision errors.
- **Size:** 32 bits
- **Offset:** 0xa50
- **Exists:** ((DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY))

Table 11-261 Fields for Register: Tx_Excessive_Collision_Packets

Bits	Name	Memory Access	Description
31:0	TXEXSCOL	R	Tx Excessive Collision Packets This field indicates the number of packets aborted because of excessive (16) collision errors. **Value After Reset:** 0x0 **Exists:** Always

11.1.258 Tx_Carrier_Error_Packets

- **Description:** This register provides the number of packets aborted by DWC_25gmac because of carrier sense error (no carrier or loss of carrier).
- **Size:** 32 bits
- **Offset:** 0xa54
- **Exists:** ((DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY))

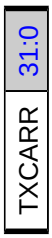


Table 11-262 Fields for Register: Tx_Carrier_Error_Packets

Bits	Name	Memory Access	Description
31:0	TXCARR	R	Tx Carrier Error Packets This field indicates the number of packets aborted because of carrier sense error (no carrier or loss of carrier). **Value After Reset:** 0x0 **Exists:** Always

11.1.259 Tx_Excessive_Deferral_Error

- **Description:** This register provides the number of packets aborted by DWC_25gmac because of excessive deferral error (deferred for more than two max-size packet times).
- **Size:** 32 bits
- **Offset:** 0xa58
- **Exists:** ((DWCXG_MMC_EN&&!DWCXG_FDUPLX_ONLY))

Table 11-263 Fields for Register: Tx_Excessive_Deferral_Error

Bits	Name	Memory Access	Description
31:0	TXEXSDEF	R	Tx Excessive Deferral Error This field indicates the number of packets aborted because of excessive deferral error (deferred for more than two max-sized packet times). **Value After Reset:** 0x0 **Exists:** Always

11.1.260 MMC_IPC_Rx_Interrupt_Mask

- **Description:** This register maintains the mask for the interrupt generated from the receive IPC statistic counters.
  The MMC Receive Checksum Off load Interrupt Mask register maintains the masks for the interrupts generated when the receive IPC (Checksum Off load) statistic counters reach half their maximum value, and when they reach their maximum values. This register is 32 bits wide.
- **Size:** 32 bits
- **Offset:** 0xa5c
- **Exists:**
  ((DWCXG_MMC_IPC_RX_PKTS_ATLEAST_ONE||DWCXG_MMC_IPC_RX_DGRAM_BCNT_ATLEAST_ONE||DWCXG_MMC_IPC_RX_PAYLD_BCNT_ATLEAST_ONE))

31:30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved_31_30	RXICMPEROIM	RXICMPGOIM	RXTCPEROIM	RXTCPGOIM	RXUDPEROIM	RXUDPGOIM	RXIPV6NOPAYOIM	RXIPV6HEROIM	RXIPV6GOIM	RXIPV4UDSBLOIM	RXIPV4FRAGOIM	RXIPV4NOPAYOIM	RXIPV4HEROIM	RXIPV4GOIM	Reserved_15	RIRSCIM	RXICMPERPIM	RXICMPGPIM	RXTCPERPIM	RXTCPGPIM	RXUDPERPIM	RXUDPGPIM	RXIPV6NOPAYPIM	RXIPV6HERPIM	RXIPV6GPIM	RXIPV4UDSBLPIM	RXIPV4FRAGPIM	RXIPV4NOPAYPIM	RXIPV4HERPIM	RXIPV4GPIM

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask

Bits	Name	Memory Access	Description
31:30	Reserved_31_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29	RXICMPEROIM	R/W	MMC Receive ICMP Error Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxICMP_Error_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive ICMP Error Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive ICMP Error Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_ICMP_ERR_OCTET_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
28	RXICMPGOIM	R/W	MMC Receive ICMP Good Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxICMP_Good_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive ICMP Good Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive ICMP Good Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_ICMP_GD_OCTET_CNT_EN
27	RXTCPEROIM	R/W	MMC Receive TCP Error Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxTCP_Error_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive TCP Error Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive TCP Error Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_TCP_ERR_OCTET_CNT_EN
26	RXTCPGOIM	R/W	MMC Receive TCP Good Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxTCP_Good_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive TCP Good Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive TCP Good Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_TCP_GD_OCTET_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
25	RXUDPEROIM	R/W	MMC Receive UDP Good Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxUDP_Error_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive UDP Good Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive UDP Good Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_UDP_ERR_OCTET_CNT_EN
24	RXUDPGOIM	R/W	MMC Receive IPv6 No Payload Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxUDP_Good_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV6 No Payload Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV6 No Payload Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_UDP_GD_OCTET_CNT_EN
23	RXIPV6NOPAYOIM	R/W	MMC Receive IPv6 Header Error Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv6_No_Payload_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV6 Header Error Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV6 Header Error Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_NOPAY_OCTET_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
22	RXIPV6HEROIM	R/W	MMC Receive IPv6 Good Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv6_Header_Error_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV6 Good Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV6 Good Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_HDRERR_OCTET_CNT_EN
21	RXIPV6GOIM	R/W	MMC Receive IPv6 Good Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv6_Good_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV6 Good Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV6 Good Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_GD_OCTET_CNT_EN
20	RXIPV4UDSBLOIM	R/W	MMC Receive IPv4 UDP Checksum Disabled Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_UDP_Checksum_Disable_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 UDP Checksum Disabled Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 UDP Checksum Disabled Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_UDSBL_OCTET_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
19	RXIPV4FRAGOIM	R/W	MMC Receive IPv4 Fragmented Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_Fragmented_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 Fragmented Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 Fragmented Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_FRAG_OCTET_CNT_EN
18	RXIPV4NOPAYOIM	R/W	MMC Receive IPv4 No Payload Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_No_Payload_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 No Payload Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 No Payload Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_NOPAY_OCTET_CNT_EN
17	RXIPV4HEROIM	R/W	MMC Receive IPv4 Header Error Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_Header_Error_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 Header Error Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 Header Error Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_HDRERR_OCTET_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
16	RXIPV4GOIM	R/W	MMC Receive IPv4 Good Octet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_Good_Octets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 Good Octet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 Good Octet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_GD_OCTET_CNT_EN
15	Reserved_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
14	RIRSCIM	R/W	Rx Channel Mapped Rx IPC MMC Skip Counter Interrupt Mask Setting this bit masks the interrupt when the Rx Channel Mapped Rx IPC MMC Skip Counter (RMMC_IPC_RCHM_SKP_CNTR _*) reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Rx Channel Mapped Rx IPC MMC Skip Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Rx Channel Mapped Rx IPC MMC Skip Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_RMMC_RCH_MAP&&DWCXG_MMC_IPC_RX_EN_ATLEAST_ONE
13	RXICMPERPIM	R/W	MMC Receive ICMP Error Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxICMP_Error_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive ICMP Error Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive ICMP Error Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_ICMP_ERR_PKTS_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
12	RXICMPGPIM	R/W	MMC Receive ICMP Good Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxICMP_Good_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive ICMP Good Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive ICMP Good Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_ICMP_GD_PKTS_CNT_EN
11	RXTCPERPIM	R/W	MMC Receive TCP Error Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxTCP_Error_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive TCP Error Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive TCP Error Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_TCP_ERR_PKTS_CNT_EN
10	RXTCPGPIM	R/W	MMC Receive TCP Good Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxTCP_Good_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive TCP Good Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive TCP Good Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_TCP_GD_PKTS_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
9	RXUDPERPIM	R/W	MMC Receive UDP Error Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxUDP_Error_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive UDP Error Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive UDP Error Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_UDP_ERR_PKTS_CNT_EN
8	RXUDPGPIM	R/W	MMC Receive UDP Good Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxUDP_Good_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive UDP Good Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive UDP Good Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_UDP_GD_PKTS_CNT_EN
7	RXIPV6NOPAYPIM	R/W	MMC Receive IPv6 No Payload Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv6_No_Payload_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV6 No Payload Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV6 No Payload Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_NOPAY_PKTS_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
6	RXIPV6HERPIM	R/W	MMC Receive IPv6 Header Error Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv6_Header_Error_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV6 Header Error Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV6 Header Error Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_HDRERR_PKTS_CNT_EN
5	RXIPV6GPIM	R/W	MMC Receive IPv6 Good Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv6_Good_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV6 Good Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV6 Good Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_GD_PKTS_CNT_EN
4	RXIPV4UDSBLPIM	R/W	MMC Receive IPv4 UDP Checksum Disabled Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_UDP_Checksum_Disabled_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 UDP Checksum Disabled Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 UDP Checksum Disabled Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_UDSBL_PKTS_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
3	RXIPV4FRAGPIM	R/W	MMC Receive IPv4 Fragmented Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_Fragmented_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 Fragmented Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 Fragmented Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_FRAG_PKTS_CNT_EN
2	RXIPV4NOPAYPIM	R/W	MMC Receive IPv4 No Payload Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_No_Payload_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 No Payload Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 No Payload Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_NOPAY_PKTS_CNT_EN
1	RXIPV4HERPIM	R/W	MMC Receive IPv4 Header Error Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_Header_Error_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 Header Error Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 Header Error Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_HDRERR_PKTS_CNT_EN

Table 11-264 Fields for Register: MMC_IPC_Rx_Interrupt_Mask (continued)

Bits	Name	Memory Access	Description
0	RXIPV4GPIM	R/W	MMC Receive IPv4 Good Packet Counter Interrupt Mask Setting this bit masks the interrupt when the RxIPv4_Good_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (DISABLE): MMC Receive IPV4 Good Packet Counter Interrupt Mask is disabled ■ 0x1 (ENABLE): MMC Receive IPV4 Good Packet Counter Interrupt Mask is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_GD_PKTS_CNT_EN

11.1.261 MMC_IPC_Rx_Interrupt

- **Description:** This register maintains the interrupt that the receive IPC statistic counters generate. The MMC Receive Checksum Offload Interrupt register maintains the interrupts generated when receive IPC statistic counters reach half their maximum values (0x8000_0000_0000_0000 for 64 bit counter and 0x8000_0000 for 32 bit counter), and when they cross their maximum values (0xFFFF_FFFF_FFFF_FFFF for 64 bit counter and 0xFFFF_FFFF for 32 bit counter). When Counter Stop Rollover is set, the interrupts are set but the counter remains at all-ones.
  The MMC Receive Checksum Offload Interrupt register is 32 bit wide. When the MMC IPC counter that caused the interrupt is read, its corresponding interrupt bit is cleared. The counter's least-significant byte lane (Bits[7:0]) must be read to clear the interrupt bit.
- **Size:** 32 bits
- **Offset:** 0xa60
- **Exists:**
  ((DWCXG_MMC_IPC_RX_PKTS_ATLEAST_ONE||DWCXG_MMC_IPC_RX_DGRAM_BCNT_ATLEAST_ONE||DWCXG_MMC_IPC_RX_PAYLD_BCNT_ATLEAST_ONE))

31:30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved_31_30	RXICMPEROIS	RXICMPGOIS	RXTCPEROIS	RXTCPGOIS	RXUDPEROIS	RXUDPGOIS	RXIPV6NOPAYOIS	RXIPV6HEROIS	RXIPV6GOIS	RXIPV4UDSBLOIS	RXIPV4FRAGOIS	RXIPV4NOPAYOIS	RXIPV4HEROIS	RXIPV4GOIS	Reserved_15	RIRSCIS	RXICMPERPIS	RXICMPGPIS	RXTCPERPIS	RXTCPGPIS	RXUDPERPIS	RXUDPGPIS	RXIPV6NOPAYPIS	RXIPV6HERPIS	RXIPV6GPIS	RXIPV4UDSBLPIS	RXIPV4FRAGPIS	RXIPV4NOPAYPIS	RXIPV4HERPIS	RXIPV4GPIS

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt

Bits	Name	Memory Access	Description
31:30	Reserved_31_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29	RXICMPEROIS	R	MMC Receive ICMP Error Octet Counter Interrupt Status This bit is set when the RxICMP_Error_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive ICMP Error Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive ICMP Error Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_ICMP_ERR_OCTET_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
28	RXICMPGOIS	R	MMC Receive ICMP Good Octet Counter Interrupt Status This bit is set when the RxICMP_Good_Octets reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive ICMP Good Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive ICMP Good Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_ICMP_GD_OCTET_CNT_EN
27	RXTCPEROIS	R	MMC Receive TCP Error Octet Counter Interrupt Status This bit is set when the RxTCP_Error_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive TCP Error Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive TCP Error Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_TCP_ERR_OCTET_CNT_EN
26	RXTCPGOIS	R	MMC Receive TCP Good Octet Counter Interrupt Status This bit is set when the RxTCP_Good_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive TCP Good Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive TCP Good Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_TCP_GD_OCTET_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
25	RXUDPEROIS	R	MMC Receive UDP Error Octet Counter Interrupt Status This bit is set when the RxUDP_Error_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive UDP Error Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive UDP Error Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_UDP_ERR_OCTET_CNT_EN
24	RXUDPGOIS	R	MMC Receive UDP Good Octet Counter Interrupt Status This bit is set when the RxUDP_Good_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive UDP Good Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive UDP Good Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_UDP_GD_OCTET_CNT_EN
23	RXIPV6NOPAYOIS	R	MMC Receive IPv6 No Payload Octet Counter Interrupt Status This bit is set when the RxIPv6_No_Payload_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV6 No Payload Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV6 No Payload Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_NOPAY_OCTET_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
22	RXIPV6HEROIS	R	MMC Receive IPv6 Header Error Octet Counter Interrupt Status This bit is set when the RxIPv6_Header_Error_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV6 Header Error Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV6 Header Error Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_HDRERR_OCTET_CNT_EN
21	RXIPV6GOIS	R	MMC Receive IPv6 Good Octet Counter Interrupt Status This bit is set when the RxIPv6_Good_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV6 Good Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV6 Good Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_GD_OCTET_CNT_EN
20	RXIPV4UDSBLOIS	R	MMC Receive IPv4 UDP Checksum Disabled Octet Counter Interrupt Status This bit is set when the RxIPv4_UDP_Checksum_Disable_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 UDP Checksum Disabled Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 UDP Checksum Disabled Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_UDSBL_OCTET_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
19	RXIPV4FRAGOIS	R	MMC Receive IPv4 Fragmented Octet Counter Interrupt Status This bit is set when the RxIPv4_Fragmented_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 Fragmented Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 Fragmented Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_FRAG_OCTET_CNT_EN
18	RXIPV4NOPAYOIS	R	MMC Receive IPv4 No Payload Octet Counter Interrupt Status This bit is set when the RxIPv4_No_Payload_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 No Payload Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 No Payload Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_NOPAY_OCTET_CNT_EN
17	RXIPV4HEROIS	R	MMC Receive IPv4 Header Error Octet Counter Interrupt Status This bit is set when the RxIPv4_Header_Error_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 Header Error Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 Header Error Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_HDRERR_OCTET_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
16	RXIPV4GOIS	R	MMC Receive IPv4 Good Octet Counter Interrupt Status This bit is set when the RxIPv4_Good_Octets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 Good Octet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 Good Octet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_GD_OCTET_CNT_EN
15	Reserved_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
14	RIRSCIS	R	Rx Channel Mapped Rx IPC MMC Skip Counter Interrupt Status This bit is set when the Rx Channel Mapped Rx IPC MMC Skip Counter (RMMC_IPC_RCHM_SKP_CNTR _*) reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Rx Channel Mapped Rx IPC MMC Skip Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Rx Channel Mapped Rx IPC MMC Skip Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_RMMC_RCH_MAP&&DWCXG_MMC_IPC_RX_EN_ATLEAST_ONE

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
13	RXICMPERPIS	R	MMC Receive ICMP Error Packet Counter Interrupt Status This bit is set when the RxICMP_Error_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive ICMP Error Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive ICMP Error Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_ICMP_ERR_PKTS_CNT_EN
12	RXICMPGPIS	R	MMC Receive ICMP Good Packet Counter Interrupt Status This bit is set when the RxICMP_Good_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive ICMP Good Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive ICMP Good Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_ICMP_GD_PKTS_CNT_EN
11	RXTCPERPIS	R	MMC Receive TCP Error Packet Counter Interrupt Status This bit is set when the RxTCP_Error_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive TCP Error Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive TCP Error Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_TCP_ERR_PKTS_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
10	RXTCPGPIS	R	MMC Receive TCP Good Packet Counter Interrupt Status This bit is set when the RxTCP_Good_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive TCP Good Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive TCP Good Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_TCP_GD_PKTS_CNT_EN
9	RXUDPERPIS	R	MMC Receive UDP Error Packet Counter Interrupt Status This bit is set when the RxUDP_Error_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive UDP Error Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive UDP Error Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_UDP_ERR_PKTS_CNT_EN
8	RXUDPGPIS	R	MC Receive UDP Good Packet Counter Interrupt Status This bit is set when the RxUDP_Good_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive UDP Good Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive UDP Good Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_UDP_GD_PKTS_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
7	RXIPV6NOPAYPIS	R	MMC Receive IPv6 No Payload Packet Counter Interrupt Status This bit is set when the RxIPv6_No_Payload_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV6 No Payload Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV6 No Payload Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_NOPAY_PKTS_CNT_EN
6	RXIPV6HERPIS	R	MMC Receive IPv6 Header Error Packet Counter Interrupt Status This bit is set when the RxIPv6_Header_Error_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV6 Header Error Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV6 Header Error Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_HDRERR_PKTS_CNT_EN
5	RXIPV6GPIS	R	MMC Receive IPV6 Good Packet Counter Interrupt Status This bit is set when the RxIPv6_Good_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV6 Good Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV6 Good Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV6_GD_PKTS_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
4	RXIPV4UDSBLPIS	R	MMC Receive IPv4 UDP Checksum Disabled Packet Counter Interrupt Status This bit is set when the RxIPv4_UDP_Checksum_Disabled_Packets counter reaches half of the maximum value or the maximum value. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 UDP Checksum Disabled Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 UDP Checksum Disabled Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_UDSBL_PKTS_CNT_EN
3	RXIPV4FRAGPIS	R	MMC Receive IPv4 Fragmented Packet Counter Interrupt Status This bit is set when the RxIPv4_Fragmented_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 Fragmented Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 Fragmented Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_FRAG_PKTS_CNT_EN
2	RXIPV4NOPAYPIS	R	MMC Receive IPv4 No Payload Packet Counter Interrupt Status This bit is set when the RxIPv4_No_Payload_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 No Payload Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 No Payload Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_NOPAY_PKTS_CNT_EN

Table 11-265 Fields for Register: MMC_IPC_Rx_Interrupt (continued)

Bits	Name	Memory Access	Description
1	RXIPV4HERPIS	R	MMC Receive IPv4 Header Error Packet Counter Interrupt Status This bit is set when the RxIPv4_Header_Error_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 Header Error Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 Header Error Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_HDRERR_PKTS_CNT_EN
0	RXIPV4GPIS	R	MMC Receive IPv4 Good Packet Counter Interrupt Status This bit is set when the RxIPv4_Good_Packets counter reaches half of the maximum value or the maximum value. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): MMC Receive IPV4 Good Packet Counter Interrupt Status not detected ■ 0x1 (ACTIVE): MMC Receive IPV4 Good Packet Counter Interrupt Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_MMC_RX_IPV4_GD_PKTS_CNT_EN

11.1.262 RxIPv4_Good_Packets_Low

- **Description:** This register provides lower 32 bits of good IPv4 datagrams received by DWC_25gmac with the TCP, UDP, or ICMP payload.
- **Size:** 32 bits
- **Offset:** 0xa64
- **Exists:** (DWCXG_MMC_RX_IPV4_GD_PKTS_CNT_EN)

Table 11-266 Fields for Register: RxIPv4_Good_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4GDPKTLO	R	RxIPv4 Good Packets This field indicates lower 32 bits of good IPv4 datagrams received with the TCP, UDP, or ICMP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.263 RxIPv4_Good_Packets_High

- **Description:** This register provides upper 32 bits of good IPv4 datagrams received by DWC_25gmac with the TCP, UDP, or ICMP payload.
- **Size:** 32 bits
- **Offset:** 0xa68
- **Exists:** (DWCXG_MMC_RX_IPV4_GD_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-267 Fields for Register: RxIPv4_Good_Packets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4GDPKTHI	R	RxIPv4 Good Packets This field indicates upper 32 bits of good IPv4 datagrams received with the TCP, UDP, or ICMP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.264 RxIPv4_Header_Error_Packets_Low

- **Description:** RxIPv4 Header Error Packets
  This register provides lower 32 bits of IPv4 datagrams received by DWC_25gmac with header (checksum, length, or version mismatch) errors.
- **Size:** 32 bits
- **Offset:** 0xa6c
- **Exists:** (DWCXG_MMC_RX_IPV4_HDRERR_PKTS_CNT_EN)

Table 11-268 Fields for Register: RxIPv4_Header_Error_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4HDRERRPKTLO	R	RxIPv4 Header Error Packets This field indicates lower 32 bits of IPv4 datagrams received with header (checksum, length, or version mismatch) errors. **Value After Reset:** 0x0 **Exists:** Always

11.1.265 RxIPv4_Header_Error_Packets_High

- **Description:** RxIPv4 Header Error Packets
  This register provides upper 32 bits of IPv4 datagrams received by DWC_25gmac with header (checksum, length, or version mismatch) errors.
- **Size:** 32 bits
- **Offset:** 0xa70
- **Exists:** (DWCXG_MMC_RX_IPV4_HDRERR_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-269 Fields for Register: RxIPv4_Header_Error_Packets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4HDRERRPKTHI	R	RxIPv4 Header Error Packets This field indicates upper 32 bits of IPv4 datagrams received with header (checksum, length, or version mismatch) errors. **Value After Reset:** 0x0 **Exists:** Always

11.1.266 RxIPv4_No_Payload_Packets_Low

- **Description:** This register provides lower 32 bits of IPv4 datagram packets received by DWC_25gmac that did not have a TCP, UDP, or ICMP payload.
- **Size:** 32 bits
- **Offset:** 0xa74
- **Exists:** (DWCXG_MMC_RX_IPV4_NOPAY_PKTS_CNT_EN)

Table 11-270 Fields for Register: RxIPv4_No_Payload_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4NOPAYPKTLO	R	RxIPv4 Payload Packets This field indicates lower 32 bits of IPv4 datagram packets received that did not have a TCP, UDP, or ICMP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.267 RxIPv4_No_Payload_Packets_High

- **Description:** This register provides upper 32 bits of IPv4 datagram packets received by DWC_25gmac that did not have a TCP, UDP, or ICMP payload.
- **Size:** 32 bits
- **Offset:** 0xa78
- **Exists:** (DWCXG_MMC_RX_IPV4_NOPAY_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-271 Fields for Register: RxIPv4_No_Payload_Packets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4NOPAYPKTHI	R	RxIPv4 Payload Packets This field indicates upper 32 bits of IPv4 datagram packets received that did not have a TCP, UDP, or ICMP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.268 RxIPv4_Fragmented_Packets_Low

- **Description:** This register provides lower 32 bits of good IPv4 datagrams received by DWC_25gmac with fragmentation.
- **Size:** 32 bits
- **Offset:** 0xa7c
- **Exists:** (DWCXG_MMC_RX_IPV4_FRAG_PKTS_CNT_EN)

Table 11-272 Fields for Register: RxIPv4_Fragmented_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4FRAGPKTLO	R	RxIPv4 Fragmented Packets This field indicates lower 32 bits of good IPv4 datagrams received with fragmentation. **Value After Reset:** 0x0 **Exists:** Always

11.1.269 RxIPv4_Fragmented_Packets_High

- **Description:** This register provides upper 32 bits of good IPv4 datagrams received by DWC_25gmac with fragmentation.
- **Size:** 32 bits
- **Offset:** 0xa80
- **Exists:** (DWCXG_MMC_RX_IPV4_FRAG_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-273 Fields for Register: RxIPv4_Fragmented_Packets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4FRAGPKTHI	R	RxIPv4 Fragmented Packets This field indicates upper 32 bits of good IPv4 datagrams received with fragmentation. **Value After Reset:** 0x0 **Exists:** Always

11.1.270 RxIPv4_UDP_Checksum_Disabled_Packets_Low

- **Description:** This register provides lower 32 bits of good IPv4 datagrams received by DWC_25gmac that had a UDP payload with checksum disabled.
- **Size:** 32 bits
- **Offset:** 0xa84
- **Exists:** (DWCXG_MMC_RX_IPV4_UDSBL_PKTS_CNT_EN)

Table 11-274 Fields for Register: RxIPv4_UDP_Checksum_Disabled_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4UDSBLPKTLO	R	RxIPv4 UDP Checksum Disabled Packets This field indicates lower 32 bits of good IPv4 datagrams received that had a UDP payload with checksum disabled. **Value After Reset:** 0x0 **Exists:** Always

11.1.271 RxIPv4_UDP_Checksum_Disabled_Packets_High

- **Description:** This register provides upper 32 bits of good IPv4 datagrams received by DWC_25gmac that had a UDP payload with checksum disabled.
- **Size:** 32 bits
- **Offset:** 0xa88
- **Exists:** (DWCXG_MMC_RX_IPV4_UDSBL_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-275 Fields for Register: RxIPv4_UDP_Checksum_Disabled_Packets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4UDSBLPKTHI	R	RxIPv4 UDP Checksum Disabled Packets This field indicates upper 32 bits of good IPv4 datagrams received that had a UDP payload with checksum disabled. **Value After Reset:** 0x0 **Exists:** Always

11.1.272 RxIPv6_Good_Packets_Low

- **Description:** This register provides lower 32 bits of good IPv6 datagrams received by DWC_25gmac with the TCP, UDP, or ICMP payload.
- **Size:** 32 bits
- **Offset:** 0xa8c
- **Exists:** (DWCXG_MMC_RX_IPV6_GD_PKTS_CNT_EN)

Table 11-276 Fields for Register: RxIPv6_Good_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV6GDPKTLO	R	RxIPv6 Good Packets This field indicates lower 32 bits of good IPv6 datagrams received with the TCP, UDP, or ICMP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.273 RxIPv6_Good_Packets_High

- **Description:** This register provides upper 32 bits of good IPv6 datagrams received by DWC_25gmac with the TCP, UDP, or ICMP payload.
- **Size:** 32 bits
- **Offset:** 0xa90
- **Exists:** (DWCXG_MMC_RX_IPV6_GD_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-277 Fields for Register: RxIPv6_Good_Packets_High

Bits	Name	Memory Access	Description
31:0	RXIPV6GDPKTHI	R	RxIPv6 Good Packets This field indicates upper 32 bits of good IPv6 datagrams received with the TCP, UDP, or ICMP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.274 RxIPv6_Header_Error_Packets_Low

- **Description:** This register provides lower 32 bits of IPv6 datagrams received by DWC_25gmac with header (length or version mismatch) errors.
- **Size:** 32 bits
- **Offset:** 0xa94
- **Exists:** (DWCXG_MMC_RX_IPV6_HDRERR_PKTS_CNT_EN)

Table 11-278 Fields for Register: RxIPv6_Header_Error_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV6HDRERRPKTLO	R	RxIPv6 Header Error Packets This field indicates lower 32 bits of IPv6 datagrams received with header (length or version mismatch) errors. **Value After Reset:** 0x0 **Exists:** Always

11.1.275 RxIPv6_Header_Error_Packets_High

- **Description:** This register provides upper 32 bits of IPv6 datagrams received by DWC_25gmac with header (length or version mismatch) errors.
- **Size:** 32 bits
- **Offset:** 0xa98
- **Exists:** (DWCXG_MMC_RX_IPV6_HDRERR_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-279 Fields for Register: RxIPv6_Header_Error_Packets_High

Bits	Name	Memory Access	Description
31:0	RXIPV6HDRERRPKTHI	R	RxIPv6 Header Error Packets This field indicates upper 32 bits of IPv6 datagrams received with header (length or version mismatch) errors. **Value After Reset:** 0x0 **Exists:** Always

11.1.276 RxIPv6_No_Payload_Packets_Low

- **Description:** This register provides lower 32 bits of IPv6 datagram packets received by DWC_25gmac that did not have a TCP, UDP, or ICMP payload. This includes all IPv6 datagrams with fragmentation or security extension headers.
- **Size:** 32 bits
- **Offset:** 0xa9c
- **Exists:** (DWCXG_MMC_RX_IPV6_NOPAY_PKTS_CNT_EN)

Table 11-280 Fields for Register: RxIPv6_No_Payload_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV6NOPAYPKTLO	R	RxIPv6 Payload Packets This field indicates lower 32 bits of IPv6 datagram packets received that did not have a TCP, UDP, or ICMP payload. This includes all IPv6 datagrams with fragmentation or security extension headers. **Value After Reset:** 0x0 **Exists:** Always

11.1.277 RxIPv6_No_Payload_Packets_High

- **Description:** This register provides upper 32 bits of IPv6 datagram packets received by DWC_25gmac that did not have a TCP, UDP, or ICMP payload. This includes all IPv6 datagrams with fragmentation or security extension headers.
- **Size:** 32 bits
- **Offset:** 0xaa0
- **Exists:** (DWCXG_MMC_RX_IPV6_NOPAY_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-281 Fields for Register: RxIPv6_No_Payload_Packets_High

Bits	Name	Memory Access	Description
31:0	RXIPV6NOPAYPKTHI	R	RxIPv6 Payload Packets This field indicates upper 32 bits of IPv6 datagram packets received that did not have a TCP, UDP, or ICMP payload. This includes all IPv6 datagrams with fragmentation or security extension headers. **Value After Reset:** 0x0 **Exists:** Always

11.1.278 RxUDP_Good_Packets_Low

- **Description:** This register provides lower 32 bits of good IP datagrams received by DWC_25gmac with a good UDP payload. This counter is not updated when the RxIPv4_UDP_Checksum_Disabled_Packets counter is incremented.
- **Size:** 32 bits
- **Offset:** 0xaa4
- **Exists:** (DWCXG_MMC_RX_UDP_GD_PKTS_CNT_EN)

Table 11-282 Fields for Register: RxUDP_Good_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXUDPGDPKTLO	R	RxUDP Good Packets This field indicates lower 32 bits of good IP datagrams received with a good UDP payload. This counter is not updated when the RxIPv4_UDP_Checksum_Disabled_Packets counter is incremented. **Value After Reset:** 0x0 **Exists:** Always

11.1.279 RxUDP_Good_Packets_High

- **Description:** This register provides upper 32 bits of good IP datagrams received by DWC_25gmac with a good UDP payload. This counter is not updated when the RxIPv4_UDP_Checksum_Disabled_Packets counter is incremented.
- **Size:** 32 bits
- **Offset:** 0xaa8
- **Exists:** (DWCXG_MMC_RX_UDP_GD_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-283 Fields for Register: RxUDP_Good_Packets_High

Bits	Name	Memory Access	Description
31:0	RXUDPGDPKTHI	R	RxUDP Good Packets This field indicates upper 32 bits of good IP datagrams received with a good UDP payload. This counter is not updated when the RxIPv4_UDP_Checksum_Disabled_Packets counter is incremented. **Value After Reset:** 0x0 **Exists:** Always

11.1.280 RxUDP_Error_Packets_Low

- **Description:** This register provides lower 32 bits of good IP datagrams received by DWC_25gmac whose UDP payload has a checksum error.
- **Size:** 32 bits
- **Offset:** 0xaac
- **Exists:** (DWCXG_MMC_RX_UDP_ERR_PKTS_CNT_EN)

Table 11-284 Fields for Register: RxUDP_Error_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXUDPERRPKTLO	R	RxUDP Error Packets This field indicates lower 32 bits of good IP datagrams received whose UDP payload has a checksum error. **Value After Reset:** 0x0 **Exists:** Always

11.1.281 RxUDP_Error_Packets_High

- **Description:** This register provides upper 32 bits of good IP datagrams received by DWC_25gmac whose UDP payload has a checksum error.
- **Size:** 32 bits
- **Offset:** 0xab0
- **Exists:** (DWCXG_MMC_RX_UDP_ERR_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-285 Fields for Register: RxUDP_Error_Packets_High

Bits	Name	Memory Access	Description
31:0	RXUDPERRPKTHI	R	RxUDP Error Packets This field indicates upper 32 bits of good IP datagrams received whose UDP payload has a checksum error. **Value After Reset:** 0x0 **Exists:** Always

11.1.282 RxTCP_Good_Packets_Low

- **Description:** This register provides lower 32 bits of good IP datagrams received by DWC_25gmac with a good TCP payload.
- **Size:** 32 bits
- **Offset:** 0xab4
- **Exists:** (DWCXG_MMC_RX_TCP_GD_PKTS_CNT_EN)

Table 11-286 Fields for Register: RxTCP_Good_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXTCPGDPKTLO	R	RxTCP Good Packets This field indicates lower 32 bits of good IP datagrams received with a good TCP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.283 RxTCP_Good_Packets_High

- **Description:** This register provides upper 32 bits of good IP datagrams received by DWC_25gmac with a good TCP payload.
- **Size:** 32 bits
- **Offset:** 0xab8
- **Exists:** (DWCXG_MMC_RX_TCP_GD_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-287 Fields for Register: RxTCP_Good_Packets_High

Bits	Name	Memory Access	Description
31:0	RXTCPGDPKTHI	R	RxTCP Good Packets This field indicates upper 32 bits of good IP datagrams received with a good TCP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.284 RxTCP_Error_Packets_Low

- **Description:** This register provides lower 32 bits of good IP datagrams received by DWC_25gmac whose TCP payload has a checksum error.
- **Size:** 32 bits
- **Offset:** 0xabc
- **Exists:** (DWCXG_MMC_RX_TCP_ERR_PKTS_CNT_EN)

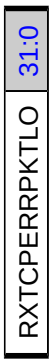


Table 11-288 Fields for Register: RxTCP_Error_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXTCPERRPKTLO	R	RxTCP Error Packets This field indicates lower 32 bits of good IP datagrams received whose TCP payload has a checksum error. **Value After Reset:** 0x0 **Exists:** Always

11.1.285 RxTCP_Error_Packets_High

- **Description:** This register provides upper 32 bits of good IP datagrams received by DWC_25gmac whose TCP payload has a checksum error.
- **Size:** 32 bits
- **Offset:** 0xac0
- **Exists:** (DWCXG_MMC_RX_TCP_ERR_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-289 Fields for Register: RxTCP_Error_Packets_High

Bits	Name	Memory Access	Description
31:0	RXTCPERRPKTHI	R	RxTCP Error Packets This field indicates upper 32 bits of good IP datagrams received whose TCP payload has a checksum error. **Value After Reset:** 0x0 **Exists:** Always

11.1.286 RxICMP_Good_Packets_Low

- **Description:** This register provides lower 32 bits of good IP datagrams received by DWC_25gmac with a good ICMP payload.
- **Size:** 32 bits
- **Offset:** 0xac4
- **Exists:** (DWCXG_MMC_RX_ICMP_GD_PKTS_CNT_EN)

Table 11-290 Fields for Register: RxICMP_Good_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXICMPGDPKTLO	R	RxICMP Good Packets This field indicates lower 32 bits of good IP datagrams received with a good ICMP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.287 RxICMP_Good_Packets_High

- **Description:** This register provides upper 32 bits of good IP datagrams received by DWC_25gmac with a good ICMP payload.
- **Size:** 32 bits
- **Offset:** 0xac8
- **Exists:** (DWCXG_MMC_RX_ICMP_GD_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-291 Fields for Register: RxICMP_Good_Packets_High

Bits	Name	Memory Access	Description
31:0	RXICMPGDPKTHI	R	RxICMP Good Packets This field indicates upper 32 bits of good IP datagrams received with a good ICMP payload. **Value After Reset:** 0x0 **Exists:** Always

11.1.288 RxICMP_Error_Packets_Low

- **Description:** This register provides lower 32 bits of good IP datagrams received by DWC_25gmac whose ICMP payload has a checksum error.
- **Size:** 32 bits
- **Offset:** 0xacc
- **Exists:** (DWCXG_MMC_RX_ICMP_ERR_PKTS_CNT_EN)

Table 11-292 Fields for Register: RxICMP_Error_Packets_Low

Bits	Name	Memory Access	Description
31:0	RXICMPERRPKTLO	R	RxICMP Error Packets This field indicates lower 32 bits of good IP datagrams received whose ICMP payload has a checksum error. **Value After Reset:** 0x0 **Exists:** Always

11.1.289 RxICMP_Error_Packets_High

- **Description:** This register provides upper 32 bits of good IP datagrams received by DWC_25gmac whose ICMP payload has a checksum error.
- **Size:** 32 bits
- **Offset:** 0xad0
- **Exists:** (DWCXG_MMC_RX_ICMP_ERR_PKTS_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-293 Fields for Register: RxICMP_Error_Packets_High

Bits	Name	Memory Access	Description
31:0	RXICMPERRPKTHI	R	RxICMP Error Packets This field indicates upper 32 bits of good IP datagrams received whose ICMP payload has a checksum error. **Value After Reset:** 0x0 **Exists:** Always

11.1.290 RxIPv4_Good_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in good IPv4 datagrams encapsulating TCP, UDP, or ICMP data. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xad4
- **Exists:** (DWCXG_MMC_RX_IPV4_GD_OCTET_CNT_EN)

Table 11-294 Fields for Register: RxIPv4_Good_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4GDOCTLO	R	RxIPv4 Good Octets This field indicates lower 32 bits of bytes received in good IPv4 datagrams encapsulating TCP, UDP, or ICMP data. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.291 RxIPv4_Good_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in good IPv4 datagrams encapsulating TCP, UDP, or ICMP data. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xad8
- **Exists:** (DWCXG_MMC_RX_IPV4_GD_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-295 Fields for Register: RxIPv4_Good_Octets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4GDOCTHI	R	RxIPv4 Good Octets This field indicates upper 32 bits of bytes received in good IPv4 datagrams encapsulating TCP, UDP, or ICMP data. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.292 RxIPv4_Header_Error_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in IPv4 datagrams with header errors (checksum, length, version mismatch). The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xadc
- **Exists:** (DWCXG_MMC_RX_IPV4_HDRERR_OCTET_CNT_EN)

Table 11-296 Fields for Register: RxIPv4_Header_Error_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4HDRERROCTLO	R	RxIPv4 Header Error Octets This field indicates lower 32 bits of bytes received in IPv4 datagrams with header errors (checksum, length, version mismatch). The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.293 RxIPv4_Header_Error_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in IPv4 datagrams with header errors (checksum, length, version mismatch). The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xae0
- **Exists:** (DWCXG_MMC_RX_IPV4_HDRERR_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-297 Fields for Register: RxIPv4_Header_Error_Octets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4HDRERROCTHI	R	RxIPv4 Header Error Octets This field indicates upper 32 bits of bytes received in IPv4 datagrams with header errors (checksum, length, version mismatch). The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.294 RxIPv4_No_Payload_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in IPv4 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xae4
- **Exists:** (DWCXG_MMC_RX_IPV4_NOPAY_OCTET_CNT_EN)

Table 11-298 Fields for Register: RxIPv4_No_Payload_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4NOPAYOCTLO	R	RxIPv4 Payload Octets This field indicates lower 32 bits of bytes received in IPv4 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.295 RxIPv4_No_Payload_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in IPv4 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xae8
- **Exists:** (DWCXG_MMC_RX_IPV4_NOPAY_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-299 Fields for Register: RxIPv4_No_Payload_Octets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4NOPAYOCTHI	R	RxIPv4 Payload Octets This field indicates upper 32 bits of bytes received in IPv4 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.296 RxIPv4_Fragmented_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in fragmented IPv4 datagrams. The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xaec
- **Exists:** (DWCXG_MMC_RX_IPV4_FRAG_OCTET_CNT_EN)

Table 11-300 Fields for Register: RxIPv4_Fragmented_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4FRAGOCTLO	R	RxIPv4 Fragmented Octets This field indicates lower 32 bits of bytes received in fragmented IPv4 datagrams. The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.297 RxIPv4_Fragmented_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in fragmented IPv4 datagrams. The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xaf0
- **Exists:** (DWCXG_MMC_RX_IPV4_FRAG_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-301 Fields for Register: RxIPv4_Fragmented_Octets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4FRAGOCTHI	R	RxIPv4 Fragmented Octets This field indicates upper 32 bits of bytes received in fragmented IPv4 datagrams. The value in the Length field of IPv4 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.298 RxIPv4_UDP_Checksum_Disable_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in a UDP segment that had the UDP checksum disabled. This counter does not count IP Header bytes. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xaf4
- **Exists:** (DWCXG_MMC_RX_IPV4_UDSBL_OCTET_CNT_EN)

Table 11-302 Fields for Register: RxIPv4_UDP_Checksum_Disable_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV4UDSBLOCTLO	R	RxIPv4 UDP Checksum Disable Octets This field indicates lower 32 bits of bytes received in a UDP segment that had the UDP checksum disabled. This counter does not count IP Header bytes. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.299 RxIPv4_UDP_Checksum_Disable_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in a UDP segment that had the UDP checksum disabled. This counter does not count IP Header bytes. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xaf8
- **Exists:** (DWCXG_MMC_RX_IPV4_UDSBL_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-303 Fields for Register: RxIPv4_UDP_Checksum_Disable_Octets_High

Bits	Name	Memory Access	Description
31:0	RXIPV4UDSBLOCTHI	R	RxIPv4 UDP Checksum Disable Octets This field indicates upper 32 bits of bytes received in a UDP segment that had the UDP checksum disabled. This counter does not count IP Header bytes. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.300 RxIPv6_Good_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in good IPv6 datagrams encapsulating TCP, UDP, or ICMP data. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xafc
- **Exists:** (DWCXG_MMC_RX_IPV6_GD_OCTET_CNT_EN)

Table 11-304 Fields for Register: RxIPv6_Good_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV6GDOCTLO	R	RxIPv6 Good Octets This field indicates lower 32 bits of bytes received in good IPv6 datagrams encapsulating TCP, UDP, or ICMP data. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.301 RxIPv6_Good_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in good IPv6 datagrams encapsulating TCP, UDP, or ICMP data. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xb00
- **Exists:** (DWCXG_MMC_RX_IPV6_GD_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-305 Fields for Register: RxIPv6_Good_Octets_High

Bits	Name	Memory Access	Description
31:0	RXIPV6GDOCTHI	R	RxIPv6 Good Octets This field indicates upper 32 bits of bytes received in good IPv6 datagrams encapsulating TCP, UDP, or ICMP data. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.302 RxIPv6_Header_Error_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in IPv6 datagrams with header errors (length, version mismatch). The value in the Length field of IPv6 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xb04
- **Exists:** (DWCXG_MMC_RX_IPV6_HDRERR_OCTET_CNT_EN)

Table 11-306 Fields for Register: RxIPv6_Header_Error_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV6HDRERROCTLO	R	RxIPv6 Header Error Octets This field indicates lower 32 bits of bytes received in IPv6 datagrams with header errors (length, version mismatch). The value in the Length field of IPv6 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.303 RxIPv6_Header_Error_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in IPv6 datagrams with header errors (length, version mismatch). The value in the Length field of IPv6 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xb08
- **Exists:** (DWCXG_MMC_RX_IPV6_HDRERR_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-307 Fields for Register: RxIPv6_Header_Error_Octets_High

Bits	Name	Memory Access	Description
31:0	RXIPV6HDRERROCTHI	R	RxIPv6 Header Error Octets This field indicates upper 32 bits of bytes received in IPv6 datagrams with header errors (length, version mismatch). The value in the Length field of IPv6 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.304 RxIPv6_No_Payload_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in IPv6 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the Length field of IPv6 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xb0c
- **Exists:** (DWCXG_MMC_RX_IPV6_NOPAY_OCTET_CNT_EN)

Table 11-308 Fields for Register: RxIPv6_No_Payload_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXIPV6NOPAYOCTLO	R	RxIPv6 Payload Octets This field indicates lower 32 bits of bytes received in IPv6 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the Length field of IPv6 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.305 RxIPv6_No_Payload_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in IPv6 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the Length field of IPv6 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter.
- **Size:** 32 bits
- **Offset:** 0xb10
- **Exists:** (DWCXG_MMC_RX_IPV6_NOPAY_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-309 Fields for Register: RxIPv6_No_Payload_Octets_High

Bits	Name	Memory Access	Description
31:0	RXIPV6NOPAYOCTHI	R	RxIPv6 Payload Octets This field indicates upper 32 bits of bytes received in IPv6 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the Length field of IPv6 header is used to update this counter. (Ethernet header, FCS, pad, or IP pad bytes are not included in this counter. **Value After Reset:** 0x0 **Exists:** Always

11.1.306 RxUDP_Good_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in a good UDP segment. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb14
- **Exists:** (DWCXG_MMC_RX_UDP_GD_OCTET_CNT_EN)

Table 11-310 Fields for Register: RxUDP_Good_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXUDPGDOCTLO	R	RxUDP Good Octets This field indicates lower 32 bits of bytes received in a good UDP segment. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.307 RxUDP_Good_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in a good UDP segment. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb18
- **Exists:** (DWCXG_MMC_RX_UDP_GD_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-311 Fields for Register: RxUDP_Good_Octets_High

Bits	Name	Memory Access	Description
31:0	RXUDPGDOCTHI	R	RxUDP Good Octets This field indicates upper 32 bits of bytes received in a good UDP segment. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.308 RxUDP_Error_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in a UDP segment that had checksum errors. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb1c
- **Exists:** (DWCXG_MMC_RX_UDP_ERR_OCTET_CNT_EN)

Table 11-312 Fields for Register: RxUDP_Error_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXUDPERROCTLO	R	RxUDP Error Octets This field indicates lower 32 bits of bytes received in a UDP segment that had checksum errors. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.309 RxUDP_Error_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in a UDP segment that had checksum errors. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb20
- **Exists:** (DWCXG_MMC_RX_UDP_ERR_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-313 Fields for Register: RxUDP_Error_Octets_High

Bits	Name	Memory Access	Description
31:0	RXUDPERROCTHI	R	RxUDP Error Octets This field indicates upper 32 bits of bytes received in a UDP segment that had checksum errors. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.310 RxTCP_Good_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in a good TCP segment. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb24
- **Exists:** (DWCXG_MMC_RX_TCP_GD_OCTET_CNT_EN)

Table 11-314 Fields for Register: RxTCP_Good_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXTCPGDOCTLO	R	RxTCP Good Octets This field indicates lower 32 bits of bytes received in a good TCP segment. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.311 RxTCP_Good_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in a good TCP segment. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb28
- **Exists:** (DWCXG_MMC_RX_TCP_GD_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-315 Fields for Register: RxTCP_Good_Octets_High

Bits	Name	Memory Access	Description
31:0	RXTCPGDOCTHI	R	RxTCP Good Octets This field indicates upper 32 bits of bytes received in a good TCP segment. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.312 RxTCP_Error_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in a TCP segment that had checksum errors. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb2c
- **Exists:** (DWCXG_MMC_RX_TCP_ERR_OCTET_CNT_EN)

Table 11-316 Fields for Register: RxTCP_Error_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXTCPERROCTLO	R	RxTCP Error Octets This field indicates lower 32 bits of bytes received in a TCP segment that had checksum errors. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.313 RxTCP_Error_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in a TCP segment that had checksum errors. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb30
- **Exists:** (DWCXG_MMC_RX_TCP_ERR_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-317 Fields for Register: RxTCP_Error_Octets_High

Bits	Name	Memory Access	Description
31:0	RXTCPERROCTHI	R	RxTCP Error Octets This field indicates upper 32 bits of bytes received in a TCP segment that had checksum errors. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.314 RxICMP_Good_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in a good ICMP segment. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb34
- **Exists:** (DWCXG_MMC_RX_ICMP_GD_OCTET_CNT_EN)

Table 11-318 Fields for Register: RxICMP_Good_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXICMPGDOCTLO	R	RxICMP Good Octets This field indicates lower 32 bits of bytes received in a good ICMP segment. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.315 RxICMP_Good_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in a good ICMP segment. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb38
- **Exists:** (DWCXG_MMC_RX_ICMP_GD_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-319 Fields for Register: RxICMP_Good_Octets_High

Bits	Name	Memory Access	Description
31:0	RXICMPGDOCTHI	R	RxICMP Good Octets This field indicates upper 32 bits of bytes received in a good ICMP segment. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.316 RxICMP_Error_Octets_Low

- **Description:** This register provides lower 32 bits of bytes received by DWC_25gmac in a ICMP segment that had checksum errors. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb3c
- **Exists:** (DWCXG_MMC_RX_ICMP_ERR_OCTET_CNT_EN)

Table 11-320 Fields for Register: RxICMP_Error_Octets_Low

Bits	Name	Memory Access	Description
31:0	RXICMPERROCTLO	R	RxICMP Error Octets This field indicates lower 32 bits of bytes received in a ICMP segment that had checksum errors. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.317 RxICMP_Error_Octets_High

- **Description:** This register provides upper 32 bits of bytes received by DWC_25gmac in a ICMP segment that had checksum errors. This counter does not count IP header bytes.
- **Size:** 32 bits
- **Offset:** 0xb40
- **Exists:** (DWCXG_MMC_RX_ICMP_ERR_OCTET_CNT_EN)&&DWCXG_EN_64BIT_CNT

Table 11-321 Fields for Register: RxICMP_Error_Octets_High

Bits	Name	Memory Access	Description
31:0	RXICMPERROCTHI	R	RxICMP Error Octets This field indicates upper 32 bits of bytes received in a ICMP segment that had checksum errors. This counter does not count IP header bytes. **Value After Reset:** 0x0 **Exists:** Always

11.1.318 RMMC_IPC_RCHM_SKP_CNTR_LOW

- **Description:** This register provides lower 32 bits of skip counts of the Rx Channel Mapped Rx IPC MMC. Basically it counts number of times, when such MMC are forbidden to update due to non-mapped receive-channel frame routing on specific scenarios.
- **Size:** 32 bits
- **Offset:** 0xb44
- **Exists:** DWCXG_RMMC_RCH_MAP&&DWCXG_MMC_IPC_RX_EN_ATLEAST_ONE

Table 11-322 Fields for Register: RMMC_IPC_RCHM_SKP_CNTR_LOW

Bits	Name	Memory Access	Description
31:0	RIRSCL	R	Rx Channel Mapped Rx IPC MMC Skip Count Low This field indicates the number of times Rx Channel Mapped Rx IPC MMC are forbidden to update due to non-mapped Rx Channel frame routing on specific scenarios. This represents the lower 32 bits of the counter. For more detail refer, feature specific functional description. **Value After Reset:** 0x0 **Exists:** Always

11.1.319 RMMC_IPC_RCHM_SKP_CNTR_HIGH

- **Description:** This register provides upper 32 bits of skip counts of the Rx Channel Mapped Rx IPC MMC. Basically it counts number of times, when such MMC are forbidden to update due to non-mapped receive-channel frame routing on specific scenarios.
- **Size:** 32 bits
- **Offset:** 0xb48
- **Exists:** DWCXG_RMMC_IPC_RCHM_SKP_CNTR_EN&&DWCXG_EN_64BIT_CNT

Table 11-323 Fields for Register: RMMC_IPC_RCHM_SKP_CNTR_HIGH

Bits	Name	Memory Access	Description
31:0	RIRSCH	R	Rx Channel Mapped Rx Frame/Priority MMC Skip Count High This field indicates the number of times Rx Channel Mapped Rx IPC MMC are forbidden to update due to non-mapped Rx Channel frame routing on specific scenarios. This represents the upper 32 bits of the counter. For more detail refer, feature specific functional description. **Value After Reset:** 0x0 **Exists:** Always

11.1.320 MAC_L3_L4_Address_Control

- **Description:** This register has the indirect address and control for accessing the Layer 3 and Layer 4 Filter registers.
- **Size:** 32 bits
- **Offset:** 0xc00
- **Exists:** (DWCXG_L3_L4_FILTER_EN)

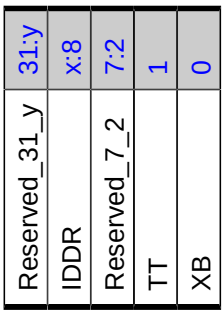


Table 11-324 Fields for Register: MAC_L3_L4_Address_Control

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_L3_L4_ADDR + 8
x:8	IDDR	R/W	L3_L4 Filter Address. This field contains the indirect address to the L3_L4 Filter registers. The IDDR[3:0] indicates the type of L3_L4 Filter register while bits[8:4] indicate the Filter number (0 to 31). IDDR[3:0] Values: - 0x0: MAC_L3_L4_Control - 0x1: MAC_Layer4_Address - 0x2 and 0x3: Reserved - 0x4: MAC_Layer3_Addr0_Reg - 0x5: MAC_Layer3_Addr1_Reg - 0x6: MAC_Layer3_Addr2_Reg - 0x7: MAC_Layer3_Addr3_Reg - 0x8 to 0xF: Reserved **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_L3_L4_ADDR + 7
7:2	Reserved_7_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-324 Fields for Register: MAC_L3_L4_Address_Control (continued)

Bits	Name	Memory Access	Description
1	TT	R/W	Transfer Type. When set, a read transfer from the L3_L4 Filter register addressed by IDDR is executed. The data read from the register is available in the MAC_L3_L4_Data register. When reset, a write transfer to the L3_L4 Filter register addressed by IDDR is executed. The data is transferred from the MAC_L3_L4_Data register and written to the respective L3_L4 Filter register. You must write the MAC_L3_L4_Data register before initiating the write transaction. **Value After Reset:** 0x0 **Exists:** Always
0	XB	R/W	Transfer Busy. When set, initiates a read or write transaction (as indicated by TT) is initiated to the L3_L4 Filter register. When reset, indicates that read or write transaction to L3-L4 Filter register is complete. You must read and confirm that this bit is 0 (previous transfer is completed) before setting this bit again. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.321 MAC_L3_L4_Data

- **Description:** This register has the indirect data and control for accessing the Layer 3 and Layer 4 Filter registers.
- **Size:** 32 bits
- **Offset:** 0xc04
- **Exists:** (DWCXG_L3_L4_FILTER_EN)

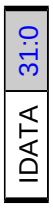


Table 11-325 Fields for Register: MAC_L3_L4_Data

Bits	Name	Memory Access	Description
31:0	IDATA	R/W	Indirect Data. The data written to or read from the L3_L4 Filter register. **Value After Reset:** 0x0 **Exists:** Always

11.1.322 MAC_L3_L4_Control#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

- **Description:** The Layer 3 and Layer 4 Control register controls the operations of filter 0 of Layer 3 and Layer 4.
- **Size:** 32 bits
- **Offset:** 0xc04
- **Exists:** DWCXG_L3_L4_FILTER_EN && (DWCXG_L3_L4_FILTER_NUM > 0)

31	30:y	x:24	23	22	21	20	19	18	17	16	15:11	10:6	5	4	3	2	1	0
DMCHEN	Reserved_27_y	DMCHN	DMCHLE0	Reserved_22	L4DPIM0	L4DPM0	L4SPIM0	L4SPM0	L5TEN0	L4PEN0	L3HDBM0	L3HSBM0	L3DAIM0	L3DAM0	L3SAIM0	L3SAM0	Reserved_1	L3PEN0

Table 11-326 Fields for Register: MAC_L3_L4_Control#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

Bits	Name	Memory Access	Description
31	DMCHEN	R/W	DMA Channel Select Enable. When set, this bit enables the selection of the DMA channel number for the packet that is passed by this L3_L4 filter. The DMA channel is indicated by the DMCHN bits. When the bit is reset, the DMA channel is not decided by the filter. **Value After Reset:** 0x0 **Exists:** (([<functionof> max(DWCXG_NUM_RX_CH_EFFW,DWCXG_L2RCHLIST_CNT)])>0)
30:y	Reserved_27_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_BN_RCH_BN_RCHL_W>0)&&(DWCXG_BN_RCH_BN_RCHL_W<7) **Range Variable[y]:** DWCXG_BN_RCH_BN_RCHL_W + 24

Table 11-326 Fields for Register: MAC_L3_L4_Control#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1) (continued)

Bits	Name	Memory Access	Description
x:24	DMCHN	R/W	DMA Channel Number. When DMCHEN is set high, this field selects the DMA Channel number to which the packet passed by this filter is routed. The width of this field depends on the number of the DMA channels present in your configuration. **Value After Reset:** 0x0 **Exists:** (([<functionof> max(DWCXG_NUM_RX_CH_EFFW,DWCXG_L2RCHLIST_CNT)])>0) **Range Variable[x]:** (DWCXG_BN_RCH_BN_RCHL_W) + 23
23	DMCHLE0	R/W	DMA Channel List Enable This field is used for DMCHN field (of this register) interpretation. When set to 1, IP does packet duplication based on the selected Receive Channel List number configured in the DMCHN field. Values: 0 - DMCHN field represent DMA Channel number 1 - DMCHN field represent Receive Channel List number of MTL_RCH_LST_IndReg registers. **Value After Reset:** 0x0 **Exists:** (DWCXG_RCHLIST_CNT>0)
22	Reserved_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
21	L4DPIM0	R/W	Layer 4 Destination Port Inverse Match Enable. When this bit is set, the Layer 4 Destination Port number field is enabled for inverse matching. When this bit is reset, the Layer 4 Destination Port number field is enabled for perfect matching. This bit is valid and applicable only when the L4DPM0 bit is set high. **Value After Reset:** 0x0 **Exists:** Always
20	L4DPM0	R/W	Layer 4 Destination Port Match Enable. When this bit is set, the Layer 4 Destination Port number field is enabled for matching. When this bit is reset, the MAC ignores the Layer 4 Destination Port number field for matching. **Value After Reset:** 0x0 **Exists:** Always

Table 11-326 Fields for Register: MAC_L3_L4_Control#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1) (continued)

Bits	Name	Memory Access	Description
19	L4SPIM0	R/W	Layer 4 Source Port Inverse Match Enable. When this bit is set, the Layer 4 Source Port number field is enabled for inverse matching. When this bit is reset, the Layer 4 Source Port number field is enabled for perfect matching. This bit is valid and applicable only when the L4SPM0 bit is set high. **Value After Reset:** 0x0 **Exists:** Always
18	L4SPM0	R/W	Layer 4 Source Port Match Enable. When this bit is set, the Layer 4 Source Port number field is enabled for matching. When this bit is reset, the MAC ignores the Layer 4 Source Port number field for matching. **Value After Reset:** 0x0 **Exists:** Always
17	L5TEN0	R/W	L3L4 5-Tuple Enable. When this bit is set, then the results of L3 filter and L4 filter enabled in this group (L3PEN & L4PEN = 1) is combined as a single result, that is, the match result is PASS only if both the L3 match and L4 match occurs. When this bit is reset, the results of L3 filter and L4 filter match are independently given and are not associated. **Value After Reset:** 0x0 **Exists:** Always
16	L4PEN0	R/W	Layer 4 Protocol Enable. When this bit is set, the Source and Destination Port number fields of UDP packets are used for matching. When this bit is reset, the Source and Destination Port number fields of TCP packets are used for matching. The Layer 4 matching is done only when the L4SPM0 or L4DPM0 bit is set. **Value After Reset:** 0x0 **Exists:** Always

Table 11-326 Fields for Register: MAC_L3_L4_Control#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1) (continued)

Bits	Name	Memory Access	Description
15:11	L3HDBM0	R/W	Layer 3 IP DA Higher Bits Match. **IPv4 Packets:** This field contains the number of higher bits of IP Destination Address that are matched in the IPv4 packets. The following list describes the values of this field: ■ 0: No bits are masked. ■ 1: LSB[0] is masked ■ 2: Two Lsbs [1:0] are masked ■ ... ■ 31: All bits except Msb are masked. **IPv6 Packets:** Bits[12:11] of this field correspond to Bits[6:5] of L3HSBM0 which indicate the number of lower bits of IP Source or Destination Address that are masked in the IPv6 packets. The following list describes the concatenated values of the L3HDBM0[1:0] and L3HSBM0 bits: ■ 0: No bits are masked. ■ 1: LSB[0] is masked. ■ 2: Two LSBs [1:0] are masked ■ ... ■ 127: All bits except Msb are masked. This field is valid and applicable only when the L3DAM0 or L3SAM0 bit is set. **Value After Reset:** 0x0 **Exists:** Always
10:6	L3HSBM0	R/W	Layer 3 IP SA Higher Bits Match. **IPv4 Packets:** This field contains the number of lower bits of IP Source Address that are masked for matching in the IPv4 packets. The following list describes the values of this field: ■ 0: No bits are masked. ■ 1: Lsb[0] is masked ■ 2: Two Lsbs [1:0] are masked ■ ... ■ 31: All bits except Msb are masked. **IPv6 Packets:** This field contains Bits[4:0] of L3HSBM0. These bits indicate the number of higher bits of IP Source or Destination Address matched in the IPv6 packets. This field is valid and applicable only when the L3DAM0 or L3SAM0 bit is set high. **Value After Reset:** 0x0 **Exists:** Always

Table 11-326 Fields for Register: MAC_L3_L4_Control#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1) (continued)

Bits	Name	Memory Access	Description
5	L3DAIM0	R/W	Layer 3 IP DA Inverse Match Enable. When this bit is set, the Layer 3 IP Destination Address field is enabled for inverse matching. When this bit is reset, the Layer 3 IP Destination Address field is enabled for perfect matching. This bit is valid and applicable only when the L3DAM0 bit is set high. **Value After Reset:** 0x0 **Exists:** Always
4	L3DAM0	R/W	Layer 3 IP DA Match Enable. When this bit is set, the Layer 3 IP Destination Address field is enabled for matching. When this bit is reset, the MAC ignores the Layer 3 IP Destination Address field for matching. **Note:** When the L3PEN0 bit is set, you must set either this bit or the L3SAM0 bit because either IPv6 DA or SA can be checked for filtering. **Value After Reset:** 0x0 **Exists:** Always
3	L3SAIM0	R/W	Layer 3 IP SA Inverse Match Enable. When this bit is set, the Layer 3 IP Source Address field is enabled for inverse matching. When this bit reset, the Layer 3 IP Source Address field is enabled for perfect matching. This bit is valid and applicable only when the L3SAM0 bit is set. **Value After Reset:** 0x0 **Exists:** Always
2	L3SAM0	R/W	Layer 3 IP SA Match Enable. When this bit is set, the Layer 3 IP Source Address field is enabled for matching. When this bit is reset, the MAC ignores the Layer 3 IP Source Address field for matching. **Note:** When the L3PEN0 bit is set, you must set either this bit or the L3DAM0 bit because either IPv6 SA or DA can be checked for filtering. **Value After Reset:** 0x0 **Exists:** Always
1	Reserved_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-326 Fields for Register: MAC_L3_L4_Control#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1) (continued)

Bits	Name	Memory Access	Description
0	L3PEN0	R/W	Layer 3 Protocol Enable. When this bit is set, the Layer 3 IP Source or Destination Address matching is enabled for IPv6 packets. When this bit is reset, the Layer 3 IP Source or Destination Address matching is enabled for IPv4 packets. The Layer 3 matching is done only when the L3SAM0 or L3DAM0 bit is set. **Value After Reset:** 0x0 **Exists:** Always

11.1.323 MAC_Layer4_Address#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

- **Description:** Layer 4 Address 0 register has the Layer 4 (TCP, UDP) Source and Destination Port fields checked against the received packet.
- **Size:** 32 bits
- **Offset:** 0xc04
- **Exists:** DWCXG_L3_L4_FILTER_EN && (DWCXG_L3_L4_FILTER_NUM > 0)

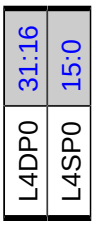


Table 11-327 Fields for Register: MAC_Layer4_Address#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

Bits	Name	Memory Access	Description
31:16	L4DP0	R/W	Layer 4 Destination Port Number Field. When the L4PEN0 bit is reset and the L4DPM0 bit is set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with the TCP Destination Port Number field in the IPv4 or IPv6 packets. When the L4PEN0 and L4DPM0 bits are set in MAC_L3_L4_Control0 register, this field contains the value to be matched with the UDP Destination Port Number field in the IPv4 or IPv6 packets. **Value After Reset:** 0x0 **Exists:** Always
15:0	L4SP0	R/W	Layer 4 Source Port Number Field. When the L4PEN0 bit is reset and the L4DPM0 bit is set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with the TCP Source Port Number field in the IPv4 or IPv6 packets. When the L4PEN0 and L4DPM0 bits are set in MAC_L3_L4_Control0 register, this field contains the value to be matched with the UDP Source Port Number field in the IPv4 or IPv6 packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.324 MAC_Layer3_Addr0_Reg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

- **Description:** For IPv4 packets, the Layer 3 Address 0 Register 0 register contains the 32-bit IP Source Address field. For IPv6 packets, it contains Bits[31:0] of the 128-bit IP Source Address or Destination Address field.
- **Size:** 32 bits
- **Offset:** 0xc04
- **Exists:** DWCXG_L3_L4_FILTER_EN && (DWCXG_L3_L4_FILTER_NUM > 0)

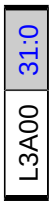


Table 11-328 Fields for Register: MAC_Layer3_Addr0_Reg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

Bits	Name	Memory Access	Description
31:0	L3A00	R/W	Layer 3 Address 0 Field. When the L3PEN0 and L3SAM0 bits are set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with Bits[31:0] of the IP Source Address field in the IPv6 packets. When the L3PEN0 and L3DAM0 bits are set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with Bits[31:0] of the IP Destination Address field in the IPv6 packets. When the L3PEN0 bit is reset and the L3SAM0 bit is set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with the IP Source Address field in the IPv4 packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.325 MAC_Layer3_Addr1_Reg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

- **Description:** For IPv4 packets, the Layer 3 Address 1 Register 0 register contains the 32-bit IP Destination Address field. For IPv6 packets, it contains Bits[63:32] of the 128-bit IP Source Address or Destination Address field.
- **Size:** 32 bits
- **Offset:** 0xc04
- **Exists:** DWCXG_L3_L4_FILTER_EN && (DWCXG_L3_L4_FILTER_NUM > 0)

Table 11-329 Fields for Register: MAC_Layer3_Addr1_Reg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

Bits	Name	Memory Access	Description
31:0	L3A10	R/W	Layer 3 Address 1 Field. When the L3PEN0 and L3SAM0 bits are set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with Bits[63:32] of the IP Source Address field in the IPv6 packets. When the L3PEN0 and L3DAM0 bits are set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with Bits[63:32] of the IP Destination Address field in the IPv6 packets. When the L3PEN0 bit is reset and the L3SAM0 bit is set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with the IP Destination Address field in the IPv4 packets. **Value After Reset:** 0x0 **Exists:** Always

11.1.326 MAC_Layer3_Addr2_Reg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

- **Description:** The Layer 3 Address 2 Register 0 register is reserved for IPv4 packets. For IPv6 packets, it contains Bits[95:64] of 128-bit IP Source Address or Destination Address field.
- **Size:** 32 bits
- **Offset:** 0xc04
- **Exists:** DWCXG_L3_L4_FILTER_EN && (DWCXG_L3_L4_FILTER_NUM > 0)

Table 11-330 Fields for Register: MAC_Layer3_Addr2_Reg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

Bits	Name	Memory Access	Description
31:0	L3A20	R/W	Layer 3 Address 2 Field. When the L3PEN0 and L3SAM0 bits are set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with Bits[95:64] of the IP Source Address field in the IPv6 packets. When the L3PEN0 and L3DAM0 bits are set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with Bits[95:64] of the IP Destination Address field in the IPv6 packets. When the L3PEN0 bit is reset in the MAC_L3_L4_Control0 register, this field is not used. **Value After Reset:** 0x0 **Exists:** Always

11.1.327 MAC_Layer3_Addr3_Reg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

- **Description:** The Layer 3 Address 3 Register 0 register is reserved for IPv4 packets. For IPv6 packets, it contains Bits[127:96] of 128-bit IP Source Address or Destination Address field.
- **Size:** 32 bits
- **Offset:** 0xc04
- **Exists:** DWCXG_L3_L4_FILTER_EN && (DWCXG_L3_L4_FILTER_NUM > 0)

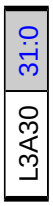


Table 11-331 Fields for Register: MAC_Layer3_Addr3_Reg#(i) (for i = 0; i <= DWCXG_L3_L4_FILTER_NUM-1)

Bits	Name	Memory Access	Description
31:0	L3A30	R/W	Layer 3 Address 3 Field. When the L3PEN0 and L3SAM0 bits are set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with Bits[127:96] of the IP Source Address field in the IPv6 packets. When the L3PEN0 and L3DAM0 bits are set in the MAC_L3_L4_Control0 register, this field contains the value to be matched with Bits[127:96] of the IP Destination Address field in the IPv6 packets. When the L3PEN0 bit is reset in the MAC_L3_L4_Control0 register, this field is not used. **Value After Reset:** 0x0 **Exists:** Always

11.1.328 MAC_ARP_Address

- **Description:** The ARP Address register contains the IPv4 Destination Address of the MAC. This register is available only when the Enable IPv4 ARP Offload option is selected.
- **Size:** 32 bits
- **Offset:** 0xc10
- **Exists:** (DWCXG_ARP_EN)

Table 11-332 Fields for Register: MAC_ARP_Address

Bits	Name	Memory Access	Description
31:0	ARPPA	R/W	ARP Protocol Address. This field contains the IPv4 Destination Address of the MAC. This address is used for perfect match with the Protocol Address of Target field in the received ARP packet. **Value After Reset:** 0xffffffff **Exists:** Always

11.1.329 MAC_RSS_Control

- **Description:** This register controls the RSS Hash function. This register is present when Enable support for Virtualized network or RSS Feature is selected.
- **Size:** 32 bits
- **Offset:** 0xc80
- **Exists:** (DWCXG_RSS_EN||DWCXG_OOS_EN)

31:20	x:17	16	15:10	9	8	7:4	3	2	1	0
Reserved_31_20	RSSLSZ	VN_IVALN_HASH_EN	Reserved_15_10	PSHE	OOSE	Reserved_7_4	UDP4TE	TCP4TE	IP2TE	RSSE

Table 11-333 Fields for Register: MAC_RSS_Control

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-333 Fields for Register: MAC_RSS_Control (continued)

Bits	Name	Memory Access	Description
x:17	RSSLSZ	R/W	RSS Lookup size. When external RSS Lookup is selected, this field configures RSS lookup size from 0th entry. Based on configuration RSS function uses RSS lookup. Values: ■ 0x0: 16 entries ■ 0x1: 32 entries ■ 0x2: 64 entries ■ 0x3: 128 entries ■ 0x4: 256 entries ■ 0x5: 512 entries ■ 0x6: 1024 entries ■ 0x7: 2048 entries **Value After Reset:** (DWCXG_RSS_TABLE_SIZE==2048?\"7\":DWCXG_RSS_TABLE_SIZE==1024?\"6\":DWCXG_RSS_TABLE_SIZE==512?\"5\":DWCXG_RSS_TABLE_SIZE==256?\"4\":DWCXG_RSS_TABLE_SIZE==128?\"3\":DWCXG_RSS_TABLE_SIZE==64?\"2\":DWCXG_RSS_TABLE_SIZE==32?\"1\":\"0\") **Exists:** (DWCXG_RSS_TABLE_SIZE>256) **Range Variable[x]:** ((DWCXG_RSS_TABLE_SIZE>128)?\"3\":(DWCXG_RSS_TABLE_SIZE>32)?\"2\":\"1\") + 16
16	VN_IVALN_HASH_EN	R/W	VxLAN Inner VLAN Hash Enable. When this bit is set to 1 the Inner Packet's VLAN is considered for the non-standard RSS Hashing. Otherwise Inner Packet VLAN is not considered for the non-standard RSS hashing. **Value After Reset:** 0x0 **Exists:** DWCXG_VXN_ENA
15:10	Reserved_15_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
9	PSHE	R/W	PSH Flag control enable. When set, the OoS function sets the NCP status bit when a TCP packet is received with the PSH flag set. **Value After Reset:** 0x0 **Exists:** DWCXG_RSS_EN&&DWCXG_SPLIT_HDR_EN

Table 11-333 Fields for Register: MAC_RSS_Control (continued)

Bits	Name	Memory Access	Description
8	OOSE	R/W	OoS function enable. When set, OoS function is enabled. This bit must be set only when RSSE and TCP4TE bits are set. **Value After Reset:** 0x0 **Exists:** DWCXG_RSS_EN&&DWCXG_SPLIT_HDR_EN
7:4	Reserved_7_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	UDP4TE	R/W	4-Tuple UDP Hash enable. When set, RSS HASH is calculated using the IPv4/v6 SA/DA and UDP SP/DP fields for IP packets received and recognized as with UDP payload **Value After Reset:** 0x0 **Exists:** Always
2	TCP4TE	R/W	4-Tuple TCP Hash enable. When set, RSS HASH is calculated using the IPv4/v6 SA/DA and TCP SP/DP fields for IP packets received and recognized as with TCP payload. **Value After Reset:** 0x0 **Exists:** Always
1	IP2TE	R/W	2-Tuple Hashing Enable. When set, RSS HASH is calculated using the IPv4/v6 SA/DA fields for received packets when TCP4TE/UDP4TE is disabled for TCP/UDP packets or the parser engine is not able to identify the L4 payload. **Value After Reset:** 0x0 **Exists:** Always
0	RSSE	R/W	RSS function enable. When set, RSS based HASH function is enabled. **Value After Reset:** 0x0 **Exists:** Always

11.1.330 MAC_RSS_Address

- **Description:** This register contains the address control bits to indirectly access the RSS Hash Key or the RSS Lookup Table.
- **Size:** 32 bits
- **Offset:** 0xc88
- **Exists:** (DWCXG_RSS_EN)

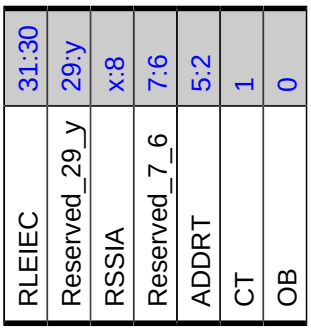


Table 11-334 Fields for Register: MAC_RSS_Address

Bits	Name	Memory Access	Description
31:30	RLEIEC	R/W	RSS Lookup ECC Inject Error Control. This field is valid only when external RXF memory-based RSS lookup is implemented and RSS_LKUP is selected in the ADDRT field of this register. Following errors are inserted based on the value encoded in this field while reading the respective registers. Values: ■ 0x0 (NO_ERR): No error injection ■ 0x1 (D_1BIT): Insert 1 bit ECC error in data or ECC bits ■ 0x2 (D_2BIT): Insert 2 bit ECC error in data or ECC bits ■ 0x3 (A_1BIT): Insert 1 bit ECC error in address **Note:** IP does error injection to respective entry's bit location based on programming. For more detail refer Automotive Safety Features section. **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ALL&&DWCXG_EXT_RSSL_EN
29:y	Reserved_29_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_RSS_TABLE_SIZE_W + 8

Table 11-334 Fields for Register: MAC_RSS_Address (continued)

Bits	Name	Memory Access	Description
x:8	RSSIA	R/W	RSS Indirect Address. The address of the Lookup Table or the index to the 40-byte RSS Hash Key. The valid range is 0 to 0x9 for accessing the RSS Hash Key. The upper limit of the address for accessing the RSS Lookup Table depends on the size of the table. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RSS_TABLE_SIZE_W + 7
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:2	ADDRT	R/W	RSS Address Type. This filed is used to select RSS indirect register block. Values: ■ 0x0 : RSS_LKUP, RSS Lookup Table (RSS_Lookup_Data#(RSSIA)) ■ 0x1 : RSS_HKEY, RSS Hash Key (MAC_RSS_Hash_Key#(RSSIA)) ■ 0x2 : I4UDP_4TMB, RSS IPv4 UDP 4-Tuple Mask Bits (MAC_RSS_IPV4UDP_4TMBIT#(RSSIA)) ■ 0x3 : I6UDP_4TMB, RSS IPv6 UDP 4-Tuple Mask Bits (MAC_RSS_IPV6UDP_4TMBIT#(RSSIA)) ■ 0x4 : I4TCP_4TMB, RSS IPv4 TCP 4-Tuple Mask Bits (MAC_RSS_IPV4TCP_4TMBIT#(RSSIA)) ■ 0x5 : I6TCP_4TMB, RSS IPv6 TCP 4-Tuple Mask Bits (MAC_RSS_IPV6TCP_4TMBIT#(RSSIA)) ■ 0x6 : I4UP_2TMB, RSS IPv4 Unsupported Payload 2-Tuple Mask Bits (MAC_RSS_IPV4UP_2TMBIT#(RSSIA)) ■ 0x7 : I6UP_2TMB, RSS IPv6 Unsupported Payload 2-Tuple Mask Bits (MAC_RSS_IPV6UP_2TMBIT#(RSSIA)) ■ 0x8 : TNL_TMB, RSS Tunnel Packet Tuples Mask Bits (MAC_RSS_TUNNEL_TMBIT#(RSSIA)) ■ 0x9 to 0xF: Reserved **Value After Reset:** 0x0 **Exists:** Always

Table 11-334 Fields for Register: MAC_RSS_Address (continued)

Bits	Name	Memory Access	Description
1	CT	R/W	Command Type. ■ When set, a read transfer is done from the location addressed by RSSIA. The data read from that location is available in the MAC_RSS_Data register. ■ When reset, a write transfer to the location addressed by RSSIA is executed. The data is transferred from the MAC_RSS_Data register. You must write the MAC_RSS_Data register before initiating the write transaction. **Value After Reset:** 0x0 **Exists:** Always
0	OB	R/W	Operation Busy. ■ When set, initiates read or write command (as indicated by CT) to the RSS function register. ■ When reset, indicates that read or write command to RSS function registers is complete. ■ For write operations, the application must program the MAC_RSS_Data register before setting this bit, and wait for this bit to be cleared to initiate the next access. ■ For read operations, the application must read the MAC_RSS_Data register after this bit is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.331 MAC_RSS_Data

- **Description:** This register has the data to be written into or read from the RSS Lookup Table or the RSS Hash Key.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** (DWCXG_RSS_EN)

Table 11-335 Fields for Register: MAC_RSS_Data

Bits	Name	Memory Access	Description
31:0	RSSD	R/W	RSS Data. The data to be written or read from the RSS Lookup Table or Hash Key. **Value After Reset:** 0x0 **Exists:** Always

11.1.332 RSS_Lookup_Data#(i) (for i = 0; i <= DWCXG_RSS_TABLE_SIZE-1)

- **Description:** The format of each location in the RSS Lookup Table is given in the following table. The size of the table is selected (16, 32, 64, 128, 256, and so on, locations) during RTL configuration.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** (DWCXG_RSS_EN)&&(DWCXG_RSS_TABLE_SIZE > 0)

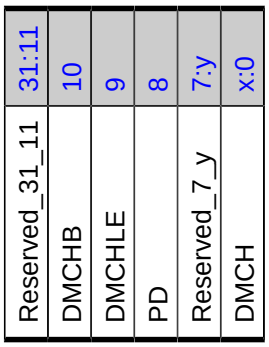


Table 11-336 Fields for Register: RSS_Lookup_Data#(i) (for i = 0; i <= DWCXG_RSS_TABLE_SIZE-1)

Bits	Name	Memory Access	Description
31:11	Reserved_31_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
10	DMCHB	R/W	DMA Channel Bypass When set to ■ 1'b1: MAC receiver does not consider DMCH field of this register. ■ 1'b0: MAC receiver considers DMCH field of this register. This bit can be used to bypass RSS Lookup DMA selection. **Note:** Final DMA selection is based on the priority among all the receive filters. **Value After Reset:** 0x0 **Exists:** (DWCXG_RSS_TABLE_SIZE>256)

Table 11-336 Fields for Register: RSS_Lookup_Data#(i) (for i = 0; i <= DWCXG_RSS_TABLE_SIZE-1) (continued)

Bits	Name	Memory Access	Description
9	DMCHLE	R/W	DMA List Enable This field is used for DMCH field (of this register) interpretation. When set to 1, IP does packet duplication based on the selected Receive Channel List number configured in the DMCH field. Values: ■ 0: DMCH field represent DMA Channel number ■ 1: DMCH field represent Receive Channel List number of MTL_RCH_LST_IndReg registers. **Value After Reset:** 0x0 **Exists:** DWCXG_RCHLIST_CNT>0&&DWCXG_EXT_RSSL_EN
8	PD	R/W	Packet Drop. When set, the MAC receiver drops the current received packet. In other words, the current packet with 2-Tuple or 4-Tuple fields that are used to determine the RSS Hash and generate the lookup of this location-must be dropped by the MAC Receiver. **Value After Reset:** 0x0 **Exists:** Always
7:y	Reserved_7_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_BN_RCH_BN_RCHL_W
x:0	DMCH	R/W	DMA Channel Number or Receive Channel List Number. This gives the RxDMA Channel or Receive Channel List Number if DMCHLE is exist, which transfers the received packet to the host memory. Selection between DMA Channel number or receive channel list number is decided based on DMCHLE field. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_BN_RCH_BN_RCHL_W - 1

11.1.333 RSS_Hash_Key(#i) (for i = 0; i <= 9)

- **Description:** The RSS Hash Key to be used in the RSS Hash calculation is 40 bytes. This is organized as 10 registers of 4 bytes each as shown in the following table. You can access these registers indirectly through the MAC_RSS_Data viewport register.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** DWCXG_RSS_EN

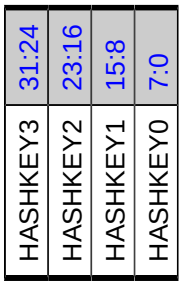


Table 11-337 Fields for Register: RSS_Hash_Key(#i) (for i = 0; i <= 9)

Bits	Name	Memory Access	Description
31:24	HASHKEY3	R/W	Hash Key Byte 3 **Value After Reset:** 0x0 **Exists:** Always
23:16	HASHKEY2	R/W	Hash Key Byte 2 **Value After Reset:** 0x0 **Exists:** Always
15:8	HASHKEY1	R/W	Hash Key Byte 1 **Value After Reset:** 0x0 **Exists:** Always
7:0	HASHKEY0	R/W	Hash Key Byte 0 **Value After Reset:** 0x0 **Exists:** Always

11.1.334 MAC_RSS_IPV4UDP_4TMBIT#(i) (for i = 0; i <= 2)

- **Description:** This is an indirect register. Each 32-bit data word of the register is accessible through MAC_RSS_* register. It contains mask bits for RSS IPv4 UDP 4-Tuples. These tuples are used for RSS Hash calculation.
  RSS function uses actual receive tuple bit, if its mask bit is set to 0. And if its mask bit is set to 1 then RSS function ignores actual receive tuple bit value and uses 0 value for that tuple bit.
  Each 32-bit data word of this register is accessible with 0 to RSSIA address, where, RSSIA 0 to 2 represents respective 32-bit data word number from least significant side.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** ((DWCXG_RSS_TABLE_SIZE>256))

Table 11-338 Fields for Register: MAC_RSS_IPV4UDP_4TMBIT#(i) (for i = 0; i <= 2)

Bits	Name	Memory Access	Description
31:0	IPV4UDPSAM	R/W	IPv4 UDP Source Address Mask bits This field has per bit mask for IPv4 UDP Source Address tuple which is used for RSS hash calculation. **Value After Reset:** 0x0 **Exists:** Always

11.1.335 MAC_RSS_IPV6UDP_4TMBIT#(i) (for i = 0; i <= 8)

- **Description:** This is an indirect register. Each 32-bit data word of the register is accessible through MAC_RSS_* register. It contains ${MSB_mask}:${LSB_mask} mask bits for RSS IPv6 UDP 4-Tuples UDP Source Address mask. These tuples are used for RSS Hash calculation.
  RSS function uses actual receive tuple bit, if its mask bit is set to 0. And if its mask bit is set to 1 then RSS function ignores actual receive tuple bit value and uses 0 value for that tuple bit.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** ((DWCXG_RSS_TABLE_SIZE>256))

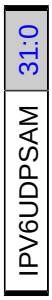


Table 11-339 Fields for Register: MAC_RSS_IPV6UDP_4TMBIT#(i) (for i = 0; i <= 8)

Bits	Name	Memory Access	Description
31:0	IPV6UDPSAM	R/W	IPv6 UDP Source Address Mask bits This field has per bit mask for IPv6 UDP Source Address tuple which is used for RSS hash calculation. **Value After Reset:** 0x0 **Exists:** Always

11.1.336 MAC_RSS_IPV4TCP_4TMBIT#(i) (for i = 0; i <= 2)

- **Description:** This is an indirect register, accessible through MAC_RSS_* register. It contains mask bits for RSS IPv4 TCP 4-Tuples Source Address. These tuples are used for RSS Hash calculation.
  RSS function uses actual receive tuple bit, if its mask bit is set to 0. And if its mask bit is set to 1 then RSS function ignores actual receive tuple bit value and uses 0 value for that tuple bit.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** ((DWCXG_RSS_TABLE_SIZE>256))

Table 11-340 Fields for Register: MAC_RSS_IPV4TCP_4TMBIT#(i) (for i = 0; i <= 2)

Bits	Name	Memory Access	Description
31:0	IPV4TCPSAM	R/W	IPv4 TCP Source Address Mask bits This field has per bit mask for IPv4 TCP Source Address tuple which is used for RSS hash calculation. **Value After Reset:** 0x0 **Exists:** Always

11.1.337 MAC_RSS_IPV6TCP_4TMBIT#(i) (for i = 0; i <= 8)

- **Description:** This is an indirect register. Each 32-bit data word of the register is accessible through MAC_RSS_* register. It contains ${MSB_mask}:${LSB_mask} mask bits for RSS IPv6 TCP 4-Tuples TCP Source Address mask. These tuples are used for RSS Hash calculation.
  RSS function uses actual receive tuple bit, if its mask bit is set to 0. And if its mask bit is set to 1 then RSS function ignores actual receive tuple bit value and uses 0 value for that tuple bit.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** ((DWCXG_RSS_TABLE_SIZE>256))

Table 11-341 Fields for Register: MAC_RSS_IPV6TCP_4TMBIT#(i) (for i = 0; i <= 8)

Bits	Name	Memory Access	Description
31:0	IPV6TCPSAM	R/W	IPv6 TCP Source Address Mask bits This field has per bit mask for IPv6 TCP Source Address tuple which is used for RSS hash calculation. **Value After Reset:** 0x0 **Exists:** Always

11.1.338 MAC_RSS_IPV4UP_2TMBIT#(i) (for i = 0; i <= 1)

- **Description:** This is an indirect register, accessible through MAC_RSS_* register. It contains mask bits for RSS IPv4 UP (Unsupported Payload) 2-Tuples for Source Address. These tuples are used for RSS Hash calculation. Where Unsupported Payload, which is not TCP/UDP or the parser engine is not able to identify the payload.
  RSS function uses actual receive tuple bit, if its mask bit is set to 0. And if its mask bit is set to 1 then RSS function ignores actual receive tuple bit value and uses 0 value for that tuple bit.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** ((DWCXG_RSS_TABLE_SIZE>256))

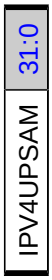


Table 11-342 Fields for Register: MAC_RSS_IPV4UP_2TMBIT#(i) (for i = 0; i <= 1)

Bits	Name	Memory Access	Description
31:0	IPV4UPSAM	R/W	IPv4 Unsupported Payload Source Address Mask bits This field has per bit mask for IPv4 UP Source Address tuple which is used for RSS hash calculation. **Value After Reset:** 0x0 **Exists:** Always

11.1.339 MAC_RSS_IPV6UP_2TMBIT#(i) (for i = 0; i <= 7)

- **Description:** This is an indirect register, accessible through MAC_RSS_* register. It contains ${MSB_mask}:${LSB_mask} mask bits of RSS IPv6 UP (Unsupported Payload) 2-Tuples. These tuples are used for RSS Hash calculation. Where Unsupported Payload, which is not TCP/UDP or the parser engine is not able to identify the payload.
  RSS function uses actual receive tuple bit, if its mask bit is set to 0. And if its mask bit is set to 1 then RSS function ignores actual receive tuple bit value and uses 0 value for that tuple bit.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** ((DWCXG_RSS_TABLE_SIZE>256))

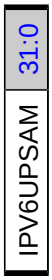


Table 11-343 Fields for Register: MAC_RSS_IPV6UP_2TMBIT#(i) (for i = 0; i <= 7)

Bits	Name	Memory Access	Description
31:0	IPV6UPSAM	R/W	IPv6 Unsupported Payload Source Address Mask bits This field has per bit mask for IPv6 TCP Source Address tuple which is used for RSS hash calculation. **Value After Reset:** 0x0 **Exists:** Always

11.1.340 MAC_RSS_TUNNEL_TMBIT#(i) (for i = 0; i <= 2)

- **Description:** This is an indirect register. Each 32-bit data word of the register is accessible through MAC_RSS_* register. It contains mask bits for RSS tunnel (VxLAN or NVGRE) packet tuples. These tuples are used for RSS Hash calculation.
  RSS function uses actual receive tuple bit, if its mask bit is set to 0. And if its mask bit is set to 1 then RSS function ignores actual receive tuple bit value and uses 0 value for that tuple bit.
- **Size:** 32 bits
- **Offset:** 0xc8c
- **Exists:** ((DWCXG_RSS_TABLE_SIZE>256))

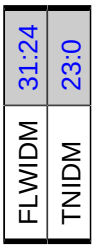


Table 11-344 Fields for Register: MAC_RSS_TUNNEL_TMBIT#(i) (for i = 0; i <= 2)

Bits	Name	Memory Access	Description
31:24	FLWIDM	R/W	NVGRE Flow ID Mask bits This field has per bit mask for NVGRE Flow ID tuple which is used for RSS hash calculation. This is only for NVGRE tunnel packet. **Value After Reset:** 0x0 **Exists:** Always
23:0	TNIDM	R/W	Tunnel ID Mask bits This field has per bit mask for Tunnel ID tuple which is used for RSS hash calculation. Based on VxLAN or NVGRE packet this tuple mask bits are used for following respective receive packet field: Values: ■ VxLAN packet: VNID ■ NVGRE packet: VSID **Value After Reset:** 0x0 **Exists:** Always

11.1.341 MAC_Timestamp_Control

- **Description:** This register controls the operation of the System Time generator and processing of PTP packets for timestamping in the Receiver.
- **Size:** 32 bits
- **Offset:** 0xd00
- **Exists:** (DWCXG_TIME_STAMPING)

31:30	29	28	27	26	25	24	23:21	20	19	18	17:16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved_31_30	TTSFSC	AV8021ASMEN	Reserved_27	ECPD	EPCSL	TXTSSTSM	Reserved_23_21	ESTI	CSC	TSENMACADDR	SNAPTYPSEL	TSMSTRENA	TSEVNTENA	TSIPV4ENA	TSIPV6ENA	TSIPENA	TSVER2ENA	TSCTRLSSR	TSENALL	Reserved_7	PTGE	TSADDREG	TSTRIG	TSUPDT	TSINIT	TSCFUPDT	TSENA

Table 11-345 Fields for Register: MAC_Timestamp_Control

Bits	Name	Memory Access	Description
31:30	Reserved_31_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29	TTSFSC	R/W	Transmit Timestamp Status FIFO Streaming Control . When this bit is set,Tx Timestamp Status FIFO output ((pkt ID, nanoseconds and seconds)) can be read from Streaming Interface. When this bit is reset, Tx Timestamp Status FIFO output can be read from CSR Subordinate Interface. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_31)
28	AV8021ASMEN	R/W	AV IEEE 802.1AS Mode Enable. When this bit is set, the MAC processes only untagged PTP over Ethernet packets for providing PTP status and capturing timestamp snapshots, that is, IEEE 802.1AS mode of operation. **Value After Reset:** 0x0 **Exists:** Always
27	Reserved_27	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-345 Fields for Register: MAC_Timestamp_Control (continued)

Bits	Name	Memory Access	Description
26	ECPD	R/W	Enable Timestamp Capturing in PTP Clock Domain. When this bit is set along with EPCSL bit, the timestamp for the packet is captured in PTP Clock Domain for higher accuracy. The value of this bit is ignored when EPCSL bit is reset. When this bit is reset, the timestamp for the packet is captured in respective MAC Receiver or Transmitter Clock Domain. **Note:** This feature must not be enabled when operating in One Step Timestamping mode, as the CDC delay from PTP clock domain to MAC Transmitter clock domain may be larger causing incorrect update in the OriginTimestamp and correctionField fields of egress PTP packet. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_37)
25	EPCSL	R/W	Enable PCS Latencies When set, Timestamp captured at the egress and ingress ports will include the Ingress and Egress latencies provided by XPCS layer through xpcs_rx_latency_i and xpcs_tx_latency_i input ports. When reset, the XPCS latencies are not considered in the captured Timestamp. **Value After Reset:** 0x0 **Exists:** (DWCXG_ETA)
24	TXTSSTSM	R/W	Transmit Timestamp Status Mode. ■ When this bit is set, the MAC overwrites the earlier transmit timestamp status even if it is not read by the software. The MAC indicates this by setting the TXTSSTSMIS bit of the MAC_TxTimestamp_Status_Nanoseconds register. ■ When this bit is reset, the MAC ignores the timestamp status of current packet if the timestamp status of previous packet is not read by the software. The MAC indicates this by setting the TXTSSTSHI bit of the MAC_TxTimestamp_Status_Seconds register. **Value After Reset:** 0x0 **Exists:** ((DWCXG>0)||DWCXG_PTO_EN)
23:21	Reserved_23_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-345 Fields for Register: MAC_Timestamp_Control (continued)

Bits	Name	Memory Access	Description
20	ESTI	R/W	External System Time Input. When this bit is set, the MAC uses the external 64-bit reference System Time input for the following: ■ To take the timestamp provided as status ■ To insert the timestamp in transmit PTP packets when One-step Timestamp or Timestamp Offload feature is enabled. When this bit is reset, the MAC uses the internal reference System Time. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE==2)
19	CSC	R/W	Enable checksum correction during OST for PTP over UDP/IPv4 packets When this bit is set, the last two bytes of PTP message sent over UDP/IPv4 is updated to keep the UDP checksum correct, for changes made to origin timestamp and/or correction field as part of one step timestamp operation. The application must form the packet with these two placeholder bytes. When reset, no updates are done to keep the UDP checksum correct. The application must form the packet with UDP checksum set to 0. **Value After Reset:** 0x0 **Exists:** DWCXG_POU_OST_EN
18	TSENMACADDR	R/W	Enable MAC Address for PTP Packet Filtering. When this bit is set, the DA MAC address (that matches any MAC Address register) is used to filter the PTP packets when PTP is directly sent over Ethernet. **Note:** When this bit is set, the PTP Offload function qualifies received PTP packet with unicast DA matching MAC Address 0 register (in addition to received PTP packet with unicast DA matching PTP special multicast addresses) for PTP response generation. **Value After Reset:** 0x0 **Exists:** Always
17:16	SNAPTYPSEL	R/W	Select PTP packets for Taking Snapshots. These bits, along with Bits 15 and 14, decide the set of PTP packet types for which snapshot needs to be taken. **Value After Reset:** 0x0 **Exists:** Always

Table 11-345 Fields for Register: MAC_Timestamp_Control (continued)

Bits	Name	Memory Access	Description
15	TSMSTRENA	R/W	Enable Snapshot for Messages Relevant to Manager. When this bit is set, the snapshot is taken only for the messages that are relevant to the manager node. Otherwise, the snapshot is taken for the messages relevant to the subordinate node. **Value After Reset:** 0x0 **Exists:** Always
14	TSEVNTENA	R/W	Enable Timestamp Snapshot for Event Messages. When this bit is set, the timestamp snapshot is taken only for event messages (SYNC, Delay_Req, Pdelay_Req, or Pdelay_Resp). When this bit is reset, the snapshot is taken for all messages except Announce, Management, and Signaling. **Value After Reset:** 0x0 **Exists:** Always
13	TSIPV4ENA	R/W	Enable Processing of PTP Packets Sent over IPv4-UDP. When this bit is set, the MAC receiver processes the PTP packets encapsulated in IPv4-UDP packets. When this bit is reset, the MAC ignores the PTP transported over IPv4-UDP packets. This bit is set by default. **Value After Reset:** 0x1 **Exists:** Always
12	TSIPV6ENA	R/W	Enable Processing of PTP Packets Sent over IPv6-UDP. When this bit is set, the MAC receiver processes the PTP packets encapsulated in IPv6-UDP packets. When this bit is clear, the MAC ignores the PTP transported over IPv6-UDP packets. **Value After Reset:** 0x0 **Exists:** Always
11	TSIPENA	R/W	Enable Processing of PTP over Ethernet Packets. When this bit is set, the MAC receiver processes the PTP packets encapsulated directly in the Ethernet packets. When this bit is reset, the MAC ignores the PTP over Ethernet packets. **Value After Reset:** 0x0 **Exists:** Always

Table 11-345 Fields for Register: MAC_Timestamp_Control (continued)

Bits	Name	Memory Access	Description
10	TSVER2ENA	R/W	Enable PTP Packet Processing for Version 2 Format. When this bit is set, the IEEE 1588 version 2 format is used to process the PTP packets. When this bit is reset, the IEEE 1588 version 1 format is used to process the PTP packets. The IEEE 1588 formats are described in "PTP Processing and Control" section. **Value After Reset:** 0x0 **Exists:** Always
9	TSCTRLSSR	R/W	Timestamp Digital or Binary Rollover Control. When this bit is set, the Timestamp Low register rolls over after 0x3B9A_C9FF value (that is, 1 nanosecond accuracy) and increments the timestamp (High) seconds. When this bit is reset, the rollover value of sub-second register is 0x7FFF_FFFF. The sub-second increment must be programmed correctly depending on the PTP reference clock frequency and the value of this bit. **Value After Reset:** 0x0 **Exists:** Always
8	TSENALL	R/W	Enable Timestamp for All Packets. When this bit is set, the timestamp snapshot is enabled for all packets received by the MAC. **Value After Reset:** 0x0 **Exists:** Always
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6	PTGE	R/W	Presentation Time Generation Enable When this bit is set the Presentation Time generation is enabled. **Value After Reset:** 0x0 **Exists:** DWCXG_FLEXI_PPS_OUT_EN&&DWCXG_TIME_STAMPING==1&&DWCXG_SYSTIME_SOURCE!=1

Table 11-345 Fields for Register: MAC_Timestamp_Control (continued)

Bits	Name	Memory Access	Description
5	TSADDREG	R/W	Update Addend Register. When this bit is set, the content of the Timestamp Addend register is updated in the PTP block for fine correction. This bit is cleared when the update is complete. This bit must be zero before it is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE!=1) **Testable:** untestable
4	TSTRIG	R/W	Enable Timestamp Interrupt Trigger. When this bit is set, the timestamp interrupt is generated when the System Time becomes greater than the value written in the Target Time register. This bit is reset after the Timestamp Trigger Interrupt is generated. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE!=1&&!DWCXG_FLEXI_PPS_OUT_EN) **Testable:** untestable
3	TSUPDT	R/W	Update Timestamp. When this bit is set, the system time is updated (added or subtracted) with the value specified in MAC_System_Time_Seconds_Update and MAC_System_Time_Nanoseconds_Update. This bit must be zero before updating it. This bit is reset when the update is complete in hardware. The Timestamp Higher Word register (if enabled during controller configuration) is not updated. When Media Clock Generation and Recovery is configured (DWCXG_FLEXI_PPS_OUT_EN) and enabled MAC_Presn_Time_Updt should also be updated before setting this field. Access restriction applies. Setting 1 sets. Self-cleared. Setting 0 has no effect. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE!=1) **Testable:** untestable

Table 11-345 Fields for Register: MAC_Timestamp_Control (continued)

Bits	Name	Memory Access	Description
2	TSINIT	R/W	Initialize Timestamp. When this bit is set, the system time is initialized (overwritten) with the value specified in the MAC_System_Time_Seconds_Update and MAC_System_Time_Nanoseconds_Update registers. This bit must be zero before it is updated. This bit is reset when the initialization is complete. The Timestamp Higher Word register (if enabled during controller configuration) can only be initialized. When Media Clock Generation and Recovery is configured (DWCXG_FLEXI_PPS_OUT_EN) and enabled MAC_Presn_Time_Updt should also be updated before setting this field. Access restriction applies. Setting 1 sets. Self-cleared. Setting 0 has no effect. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE!=1) **Testable:** untestable
1	TSCFUPDT	R/W	Fine or Coarse Timestamp Update. When this bit is set, the Fine method is used to update system timestamp. When this bit is reset, Coarse method is used to update the system timestamp. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE!=1)
0	TSENA	R/W	Enable Timestamp. When this bit is set, the timestamp is added for Transmit and Receive packets. When disabled, timestamp is not added for transmit and receive packets and the Timestamp Generator is also suspended. You need to initialize the Timestamp (system time) after enabling this mode. On the Receive side, the MAC processes the 1588 packets only if this bit is set. **Value After Reset:** 0x0 **Exists:** Always

11.1.342 MAC_Sub_Second_Increment

- **Description:** The Sub-Second Increment register is present only when the IEEE 1588 timestamp feature is selected without an external timestamp input. In Coarse Update mode [Bit 1 in MAC_Timestamp_Control register], the value in this register is added to the system time every clock cycle of clk_ptp_ref_i. In Fine Update mode, the value in this register is added to the system time when the Accumulator gets an overflow.
- **Size:** 32 bits
- **Offset:** 0xd04
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)

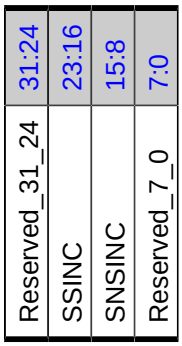


Table 11-346 Fields for Register: MAC_Sub_Second_Increment

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:16	SSINC	R/W	Sub-second Increment Value. The value programmed in this field is accumulated with the contents of the sub-second register. For example, when the PTP clock is 50 MHz (period is 20 ns), you must program 20 (0x14) when the System Time- Nanoseconds register has an accuracy of 1 ns [Bit 9 (TSCTRLSSR) is set in MAC_Timestamp_Control]. When TSCTRLSSR is clear, the Nanoseconds register has a resolution of ~0.465 ns. In this case, you must program a value of 43 (0x2B) which is derived by 20 ns/0.465. **Value After Reset:** 0x0 **Exists:** Always

Table 11-346 Fields for Register: MAC_Sub_Second_Increment (continued)

Bits	Name	Memory Access	Description
15:8	SNSINC	R/W	Sub-nanosecond Increment Value. This field contains the sub-nanosecond increment value, represented in nanoseconds multiplied by 28. This value is accumulated with the sub-nanoseconds field of the sub-second register. For example, when TSCTRLSSR field in the MAC_Timestamp_Control register is set. and if the required increment is 5.3ns, then SSINC must be 0x05 and SNSINC must be 0x4C. **Value After Reset:** 0x0 **Exists:** DWCXG_ADV_SUBNSEC_SUPPORT
7:0	Reserved_7_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.343 MAC_System_Time_Seconds

- **Description:** The System Time-Seconds register, along with System Time-Nanoseconds register, indicates the current value of the system time maintained by the MAC. Though it is updated on a continuous basis, there is some delay from the actual time because of clock domain transfer latencies (from clk_ptp_ref_i to CSR clock).
  This register is present only when the IEEE 1588 Timestamp feature is selected without external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xd08
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)

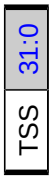


Table 11-347 Fields for Register: MAC_System_Time_Seconds

Bits	Name	Memory Access	Description
31:0	TSS	R	Timestamp Second. The value in this field indicates the current value in seconds of the System Time maintained by the MAC. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.344 MAC_System_Time_Nanoseconds

- **Description:** The System Time-Nanoseconds register, along with System Time-Seconds register, indicates the current value of the system time maintained by the MAC.
  This register is present only when the IEEE 1588 Timestamp feature is selected without external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xd0c
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)

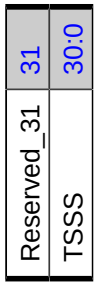


Table 11-348 Fields for Register: MAC_System_Time_Nanoseconds

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30:0	TSSS	R	Timestamp Sub Seconds. The value in this field has the sub-second representation of time, with an accuracy of 0.46 ns. When Bit 9 is set in MAC_Timestamp_Control, each bit represents 1 ns. The maximum value is 0x3B9A_C9FF after which it rolls-over to zero. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.345 MAC_System_Time_Seconds_Update

- **Description:** The System Time-Seconds Update register, along with the System Time-Nanoseconds Update register, initializes or updates the system time maintained by the MAC. You must write both registers before setting the TSINIT or TSUPDT bits in MAC_Timestamp_Control register.
  This register is present only when the IEEE 1588 Timestamp feature is selected without external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xd10
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)

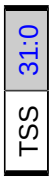


Table 11-349 Fields for Register: MAC_System_Time_Seconds_Update

Bits	Name	Memory Access	Description
31:0	TSS	R/W	Timestamp Second. Timestamp Seconds The value in this field is the seconds part of the update. When ADDSUB is reset, this field must be programmed with the seconds part of the update value. When ADDSUB is set, this field must be programmed with the complement of the seconds part of the update value. For example, if 2.000000001 seconds need to be subtracted from the system time, the TSS field in the MAC_Timestamp_Seconds_Update register must be 0xFFFF_FFFE (that is, 2^32 - 2). **Value After Reset:** 0x0 **Exists:** Always

11.1.346 MAC_System_Time_Nanoseconds_Update

- **Description:** The System Time-Nanoseconds Update register, along with the System Time-Seconds Update register, initializes or updates the system time maintained by the MAC. You must write both registers before setting the TSINIT or TSUPDT bits in MAC_Timestamp_Control registerThis register is present only when the IEEE 1588 timestamp feature is selected without external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xd14
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)

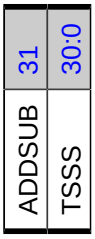


Table 11-350 Fields for Register: MAC_System_Time_Nanoseconds_Update

Bits	Name	Memory Access	Description
31	ADDSUB	R/W	Add or Subtract Time. When this bit is set, the time value is subtracted with the contents of the update register. When this bit is reset, the time value is added with the contents of the update register. **Value After Reset:** 0x0 **Exists:** Always
30:0	TSSS	R/W	Timestamp Sub Seconds. The value in this field is the sub-second part of the update. ■ When ADDSUB is reset, this field must be programmed with the sub-second part of the update value, with an accuracy based on the TSCTRLSSR bit of the MAC Timestamp Control register. ■ When ADDSUB is set, this field must be programmed with the complement of the sub-second part of the update value, as follows. ■ When TSCTRLSSR is set, the programmed value must be 10^9 - (sub-second value). When TSCTRLSSR is reset, the programmed value must be 2^31 - (sub-second_value). For example, when TSCTRLSSR bit is set and if 2.000000001 second needs to be subtracted with the system time, the TSS field in the MAC Timestamp Seconds update register must be 0xFFFF_FFFE (for instance, 2^32 - 2), ADDSUB bit in this register must be set, and the TSSS field must be 0x3B9A_C9FF (for instance, 10^9 - 1). **Value After Reset:** 0x0 **Exists:** Always

11.1.347 MAC_Timestamp_Addend

- **Description:** The Timestamp Addend register is present only when the IEEE 1588 Timestamp feature is selected without external timestamp input. This register value is used only when the system time is configured for Fine Update mode (TSCFUPDT bit in the MAC_Timestamp_Control register). The content of this register is added to a 32-bit accumulator in every clock cycle (of clk_ptp_ref_i) and the system time is updated when the accumulator overflows.
- **Size:** 32 bits
- **Offset:** 0xd18
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)

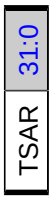


Table 11-351 Fields for Register: MAC_Timestamp_Addend

Bits	Name	Memory Access	Description
31:0	TSAR	R/W	Timestamp Addend Register. This field indicates the 32-bit time value to be added to the Accumulator register to achieve time synchronization. **Value After Reset:** 0x0 **Exists:** Always

11.1.348 MAC_System_Time_Higher_Word_Seconds

- **Description:** This register contains the most-significant 16-bits of timestamp seconds value.The System Time - Higher Word Seconds register is present only when the IEEE 1588 Timestamp feature is selected without an external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xd1c
- **Exists:** (DWCXG_ADV_TIME_HIGH_WORD)

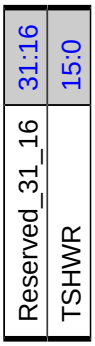


Table 11-352 Fields for Register: MAC_System_Time_Higher_Word_Seconds

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:0	TSHWR	R/W	Timestamp Higher Word Register. This field contains the most-significant 16-bits of timestamp seconds value. This register is optional. You can add this register by selecting the Add IEEE 1588 Higher Word Register option. This register is directly written to initialize the value and it is incremented when there is an overflow from 32-bits of the System Time - Seconds register. **Value After Reset:** 0x0 **Exists:** Always

11.1.349 MAC_Timestamp_Status

- **Description:** The Timestamp Status registeris present only when the IEEE 1588 Timestamp feature is selected. All bits except Bits[29:25] gets cleared when the application reads this register.
- **Size:** 32 bits
- **Offset:** 0xd20
- **Exists:** (DWCXG_TIME_STAMPING)

31	30	29:25	24	23:20	19:16	15	14:10	9	8	7	6	5	4	3	2	1	0
LTSSOVF	Reserved_30	ATSNS	ATSSTM	Reserved_23_20	ATSSTN	TXTSC	TTSNS	TSTRGTERR3	TSTARGT3	TSTRGTERR2	TSTARGT2	TSTRGTERR1	TSTARGT1	TSTRGTERR0	AUXTSTRIG	TSTARGT0	TSSOVF

Table 11-353 Fields for Register: MAC_Timestamp_Status

Bits	Name	Memory Access	Description
31	LTSSOVF	R	Local Timestamp Seconds Overflow. When this bit is set, it indicates that the seconds value of the local timestamp (when supporting version 2 format) has overflowed beyond 32'hFFFF_FFFF. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30	Reserved_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
29:25	ATSNS	R	Number of Auxiliary Timestamp Snapshots. This field indicates the number of Snapshots available in the FIFO. A value equal to the selected depth of FIFO (4, 8, or 16) indicates that the Auxiliary Snapshot FIFO is full. These bits are cleared (to 00000) when the Auxiliary snapshot FIFO clear bit is set. This bit is valid only if the Add IEEE 1588 Auxiliary Snapshot option is selected. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-353 Fields for Register: MAC_Timestamp_Status (continued)

Bits	Name	Memory Access	Description
24	ATSSTM	R	Auxiliary Timestamp Snapshot Trigger Missed. This bit is set when the Auxiliary timestamp snapshot FIFO is full and external trigger was set. This indicates that the latest snapshot is not stored in the FIFO. This bit is valid only if the Add IEEE 1588 Auxiliary Snapshot option is selected. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
23:20	Reserved_23_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
19:16	ATSSTN	R	Auxiliary Timestamp Snapshot Trigger Identifier. These bits identify the Auxiliary trigger inputs for which the timestamp available in the Auxiliary Snapshot register is applicable. When more than one bit is set at the same time, it means that corresponding auxiliary triggers are sampled at the same clock. These bits are applicable only if the number of Auxiliary snapshots is more than one. One bit is assigned for each trigger as shown in the following list: ■ Bit 16: Auxiliary trigger 0 ■ Bit 17: Auxiliary trigger 1 ■ Bit 18: Auxiliary trigger 2 ■ Bit 19: Auxiliary trigger 3 The software can read this register to find the triggers that are set when the timestamp is taken. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15	TXTSC	R	Tx Timestamp Captured. When set, this bit indicates that a timestamp of the current packet being transmitted by the MAC is available in MAC_TxTimestamp_Status_Seconds and MAC_TxTimestamp_Status_Nanoseconds registers. This bit is cleared on reading (byte 3 or byte 0 in little or big endian mode respectively) the MAC_TxTimestamp_Status_Seconds register. When Tx Timestamp Status FIFO depth selected is more than 1, this bit is set as long as Tx Timestamp status FIFO has Tx Timestamp status entries. It is cleared when Seconds field corresponding to last entry is read (byte 3

Table 11-353 Fields for Register: MAC_Timestamp_Status (continued)

Bits	Name	Memory Access	Description
			or byte 0 in little or big endian mode respectively) from MAC_TxTimestamp_Status_Seconds register. Access restriction applies. This bit is cleared on reading (byte 3 or byte 0 in little or big endian mode respectively) the MAC_TxTimestamp_Status_Seconds register (or write to corresponding byte of MAC_Tx_Timestamp_Status_Seconds register when RCWE bit of MAC_CSR_SW_Ctrl register is set). **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14:10	TTSNS	R	Number of Tx Timestamp Snapshots. This field indicates the number of Tx Timestamp Snapshots available in the FIFO. A value equal to the selected depth of FIFO (2, 4, 8, or 16) indicates that the Tx Timestamp Snapshot FIFO is full. The Tx Timestamp Snapshot FIFO is implemented as synchronous FIFO in the CSR clock domain. This field is incremented along with the Tx Timestamp Snapshot write command to the FIFO. This field is decremented along with the Tx Timestamp Snapshot read command (issued when byte 3 or byte 0 of MAC_TxTimestamp_Status_Seconds register is read in little or big endian mode respectively) to the FIFO. If both write and read command is issued simultaneously, this field is not changed. This field is valid only if the "Enable IEEE 1588 Timestamp Support" option is selected. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
9	TSTRGTERR3	R	Timestamp Target Time Error. This bit is set when the latest target time programmed in the MAC_PPS3_Target_Time_Seconds and MAC_PPS3_Target_Time_Nanoseconds registers has already elapsed. The bit is also set when the PPS command programmed cannot be executed as the earlier programmed target time has already elapsed. This bit is cleared when the application reads this bit. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-353 Fields for Register: MAC_Timestamp_Status (continued)

Bits	Name	Memory Access	Description
8	TSTARGT3	R	Timestamp Target Time Reached for Target Time PPS3 When this bit is set and MCGREN3 of MAC_PPS_Control register is reset, it indicates that the value of system time is greater than or equal to the value specified in the MAC_PPS3_Target_Time_Seconds and MAC_PPS3_Target_Time_Nanoseconds registers. When this bit is set and MCGREN3 of MAC_PPS_Control register is set, it indicates that mcgr_dma_req_o[3] is asserted. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
7	TSTRGTERR2	R	Timestamp Target Time Error. This bit is set when the latest target time programmed in the MAC_PPS2_Target_Time_Seconds and MAC_PPS2_Target_Time_Nanoseconds registers has already elapsed. The bit is also set when the PPS command programmed cannot be executed as the earlier programmed target time has already elapsed. This bit is cleared when the application reads this bit. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
6	TSTARGT2	R	Timestamp Target Time Reached for Target Time PPS2. When this bit is set and MCGREN2 of MAC_PPS_Control register is reset, it indicates that the value of system time is greater than or equal to the value specified in the MAC_PPS2_Target_Time_Seconds and MAC_PPS2_Target_Time_Nanoseconds registers. When this bit is set and MCGREN2 of MAC_PPS_Control register is set, it indicates that mcgr_dma_req_o[2] is asserted. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-353 Fields for Register: MAC_Timestamp_Status (continued)

Bits	Name	Memory Access	Description
5	TSTRGTERR1	R	Timestamp Target Time Error. This bit is set when the latest target time programmed in the MAC_PPS1_Target_Time_Seconds and MAC_PPS1_Target_Time_Nanoseconds registers has already elapsed. The bit is also set when the PPS command programmed cannot be executed as the earlier programmed target time has already elapsed. This bit is cleared when the application reads this bit. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
4	TSTARGT1	R	Timestamp Target Time Reached for Target Time PPS1. When this bit is set and MCGREN1 of MAC_PPS_Control register is reset, it indicates that the value of system time is greater than or equal to the value specified in the MAC_PPS1_Target_Time_Seconds and MAC_PPS1_Target_Time_Nanoseconds registers. When this bit is set and MCGREN1 of MAC_PPS_Control register is set, it indicates that mcgr_dma_req_o[1] is asserted. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
3	TSTRGTERR0	R	Timestamp Target Time Error. This bit is set when the latest target time programmed in the MAC_PPS0_Target_Time_Seconds and MAC_PPS0_Target_Time_Nanoseconds registers has already elapsed. The bit is also set when the PPS command programmed cannot be executed as the earlier programmed target time has already elapsed. This bit is cleared when the application reads this bit. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-353 Fields for Register: MAC_Timestamp_Status (continued)

Bits	Name	Memory Access	Description
2	AUXTSTRIG	R	Auxiliary Timestamp Trigger Snapshot. This bit is set high when the auxiliary snapshot is written to the FIFO. This bit is valid only if the Add IEEE 1588 Auxiliary Snapshot option is selected. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	TSTARGT0	R	Timestamp Target Time Reached. When this bit is set and MCGREN0 of MAC_PPS_Control register is reset, it indicates that the value of system time is greater than or equal to the value specified in the MAC_PPS0_Target_Time_Seconds and MAC_PPS0_Target_Time_Nanoseconds registers. When this bit is set and MCGREN0 of MAC_PPS_Control register is set, it indicates that mcgr_dma_req_o[0] is asserted. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	TSSOVF	R	Timestamp Seconds Overflow. When this bit is set, it indicates that the seconds value of the timestamp (when supporting version 2 format) has overflowed beyond 32'hFFFF_FFFF. Access restriction applies. Clears on read (or write of 1 when RCWE bit in MAC_CSR_SW_Ctrl register is set). Self-set to 1 on internal event. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.350 MAC_Rx_Domain_Time_Incr

- **Description:** This register indicates the increment value of the PTP time maintained in Receive clock domain that is used for timestamping.
- **Size:** 32 bits
- **Offset:** 0xd24
- **Exists:** (DWCXG_TIME_STAMPING)

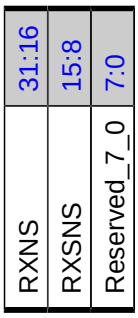


Table 11-354 Fields for Register: MAC_Rx_Domain_Time_Incr

Bits	Name	Memory Access	Description
31:16	RXNS	R/W	Receive Domain Time Increment Value in Nanoseconds. This field indicates the Nanosecond part of the increment value of the PTP time maintained in Receive clock domain that is used for timestamping. For example, when operating in 10G speed the clock period is 3.2ns, so for Digital Rollover mode the RXNS and RXSNS fields must be programmed with 3 and (0.2*256)=51 respectively. In Binary Rollover mode, 3/0.466=6.44 and 0.2+0.44=0.64, so RXNS and RXSNS fields must be programmed with 6 and (0.64*256)=164 respectively. When both RXSNS and RXNS fields are all-zeros, the fixed value based on operating speed is used for PTP time increment in Receive clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	RXSNS	R/W	Receive Domain Time Increment Value in Sub-Nanoseconds. This field indicates the Sub-Nanosecond part of the increment value of the PTP time maintained in Receive clock domain that is used for timestamping. This field has granularity of 1/256 ns. When both RXSNS and RXNS fields are all-zeros, the fixed value based on operating speed is used for PTP time increment in Receive clock domain. **Value After Reset:** 0x0 **Exists:** DWCXG_ADV_SUBNSEC_SUPPORT

Table 11-354 Fields for Register: MAC_Rx_Domain_Time_Incr (continued)

Bits	Name	Memory Access	Description
7:0	Reserved_7_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.351 MAC_Tx_Domain_Time_Incr

- **Description:** This register indicates the increment value of the PTP time maintained in Transmit clock domain that is used for timestamping.
- **Size:** 32 bits
- **Offset:** 0xd28
- **Exists:** (DWCXG_TIME_STAMPING)

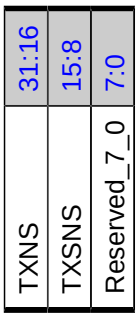


Table 11-355 Fields for Register: MAC_Tx_Domain_Time_Incr

Bits	Name	Memory Access	Description
31:16	TXNS	R/W	Transmit Domain Time Increment Value in Nanoseconds. This field indicates the Nanosecond part of the increment value of the PTP time maintained in Transmit clock domain that is used for timestamping. For example, when operating in 2.5G XGMII speed the clock period is 12.8ns, so for Digital Rollover mode the TXNS and TXSNS fields must be programmed with 12 and (0.8*256)=205 respectively. In Binary Rollover mode, 12/0.466=25.75 and 0.8+0.75=1.55, so TXNS and TXSNS fields must be programmed with (25+1)=26 and (0.55*256)=141 respectively. When both TXSNS and TXNS fields are all-zeros, the fixed value based on operating speed is used for PTP time increment in Transmit clock domain. **Value After Reset:** 0x0 **Exists:** Always
15:8	TXSNS	R/W	Transmit Domain Time Increment Value in Sub-Nanoseconds. This field indicates the Sub-Nanosecond part of the increment value of the PTP time maintained in Transmit clock domain that is used for timestamping. This field has granularity of 1/256 ns. When both TXSNS and TXNS fields are all-zeros, the fixed value based on operating speed is used for PTP time increment in Transmit clock domain. **Value After Reset:** 0x0 **Exists:** DWCXG_ADV_SUBNSEC_SUPPORT

Table 11-355 Fields for Register: MAC_Tx_Domain_Time_Incr (continued)

Bits	Name	Memory Access	Description
7:0	Reserved_7_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.352 MAC_Rx_PCH_CRC_Mismatch_Counter

- **Description:** The PCH CRC Mismatch Counter for Debugging purpose.
- **Size:** 32 bits
- **Offset:** 0xd2c
- **Exists:** (SNPS_RSVDPARAM_45)

Table 11-356 Fields for Register: MAC_Rx_PCH_CRC_Mismatch_Counter

Bits	Name	Memory Access	Description
31:0	RPCMC	R	Receive PCH CRC Mismatch. This indicates the number of packets received with CRC error in the PCH bytes. It is cleared when this register is read. This register is reserved in this release. **Value After Reset:** 0x0 **Exists:** Always

11.1.353 MAC_Tx_Timestamp_Status_Nanoseconds

- **Description:** The MAC_TxTimestamp_Status_Nanoseconds register, along with MAC_TxTimestamp_Status_Seconds, gives the 64-bit timestamp captured for the PTP packet successfully transmitted by the MAC. This value is considered to be read by the application when the last byte of MAC_Tx_Timestamp_Status_Seconds is read. In the little-endian mode, this means bits[31:24] are read; in big-endian mode, bits[7:0] are read.
  If the application does not read these registers and timestamp of another packet is captured, then either the current timestamp is lost (overwritten) or the new timestamp is lost (dropped), depending on the setting of the TXTSSTSM bit of the MAC_Timestamp_Control register. The status bit TXTSC bit [15] in MAC_Timestamp_Status register is set when the MAC transmitter captures the timestamp.
- **Size:** 32 bits
- **Offset:** 0xd30
- **Exists:** ((DWCXG>0&&DWCXG_TIME_STAMPING)||DWCXG_PTO_EN)

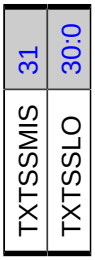


Table 11-357 Fields for Register: MAC_Tx_Timestamp_Status_Nanoseconds

Bits	Name	Memory Access	Description
31	TXTSSMIS	R	Transmit Timestamp Status Missed. When this bit is set, it indicates one of the following: ■ The timestamp of the current packet is ignored if TXTSSTSM bit of the MAC_Timestamp_Control register is reset ■ The timestamp of the previous packet is overwritten with timestamp of the current packet if TXTSSTSM bit of the MAC_Timestamp_Control register is set. **Value After Reset:** 0x0 **Exists:** Always
30:0	TXTSSLO	R	Transmit Timestamp Status Low. This field contains the 31 bits of the Nanoseconds field of the Transmit packet's captured timestamp. **Value After Reset:** 0x0 **Exists:** Always

11.1.354 MAC_Tx_Timestamp_Status_Seconds

- **Description:** The register contains the higher 32 bits of the timestamp (in seconds) captured when a PTP packet is transmitted.
- **Size:** 32 bits
- **Offset:** 0xd34
- **Exists:** ((DWCXG>0&&DWCXG_TIME_STAMPING)||DWCXG_PTO_EN)

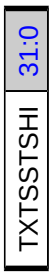


Table 11-358 Fields for Register: MAC_Tx_Timestamp_Status_Seconds

Bits	Name	Memory Access	Description
31:0	TXTSSTSHI	R	Transmit Timestamp Status High. This field contains the lower 32 bits of the Seconds field of Transmit packet's captured timestamp. **Value After Reset:** 0x0 **Exists:** Always

11.1.355 MAC_Tx_Timestamp_Status_PktID

- **Description:** This register contains the PTP packet ID corresponding to the Tx Timestamp Status in MAC_TxTimestamp_Status_Nanoseconds and MAC_TxTimestamp_Status_Seconds registers. This register is available when IEEE 1588 Tx Timestamp Status FIFO Depth is greater than 1.
  The correct reading sequence of these registers is as follows:
  - MAC_TxTimestamp_Status_PktID register
  - MAC_TxTimestamp_Status_Nanoseconds register
  - MAC_TxTimestamp_Status_Seconds register

  This is because, all the 3 registers are updated to a new value when the application reads the last byte of the MAC_TxTimestamp_Status_Seconds register.
- **Size:** 32 bits
- **Offset:** 0xd38
- **Exists:** ((DWCXG>0&&DWCXG_TIME_STAMPING)||DWCXG_PTO_EN)

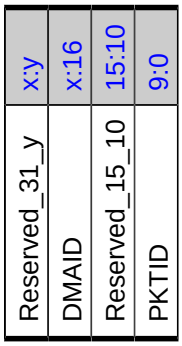


Table 11-359 Fields for Register: MAC_Tx_Timestamp_Status_PktID

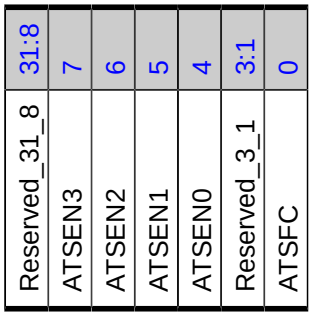
Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50 **Range Variable[x]:** (16-DWCXG_NUM_DMA_TX_CHW) + 16+DWCXG_NUM_DMA_TX_CHW - 1 **Range Variable[y]:** DWCXG_NUM_DMA_TX_CHW + 16
x:16	DMAID	R	Transmit VDMA ID. The field contains the Identifier/number of the Transmit VDMA that transferred the packet for which the Tx Timestamp status is captured in MAC_TxTimestamp_Status_Nanoseconds and MAC_TxTimestamp_Status_Seconds registers. **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50 **Range Variable[x]:** DWCXG_NUM_DMA_TX_CHW + 15

Table 11-359 Fields for Register: MAC_Tx_Timestamp_Status_PktID (continued)

Bits	Name	Memory Access	Description
15:10	Reserved_15_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50
9:0	PKTID	R	Packet ID. The field contains the packet ID corresponding to the Tx Timestamp status in MAC_TxTimestamp_Status_Nanoseconds and MAC_TxTimestamp_Status_Seconds registers. **Note:** The value of 0 for Packet ID is reserved, it is used by DWC_25gmac for indicating timestamp status of packets for which Packet ID was not provided by Software or for PTP response packets generated by the hardware PTP Offload engine. **Value After Reset:** 0x0 **Exists:** Always

11.1.356 MAC_Auxiliary_Control

- **Description:** The Auxiliary Timestamp Control register controls the Auxiliary Timestamp snapshot.
- **Size:** 32 bits
- **Offset:** 0xd40
- **Exists:** (DWCXG_ADV_TIME_AUX_SNAP)



31:8	7	6	5	4	3:1	0
Reserved_31_8	ATSEN3	ATSEN2	ATSEN1	ATSEN0	Reserved_3_1	ATSFC

Table 11-360 Fields for Register: MAC_Auxiliary_Control

Bits	Name	Memory Access	Description
31:8	Reserved_31_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7	ATSEN3	R/W	Auxiliary Snapshot 3 Enable. This bit controls the capturing of Auxiliary Snapshot Trigger 3. When this bit is set, the auxiliary snapshot of the event on ptp_aux_trig_i[3] input is enabled. When this bit is reset, the events on this input are ignored. **Value After Reset:** 0x0 **Exists:** (DWCXG_AUX_SNAP_IN_NUM>3)
6	ATSEN2	R/W	Auxiliary Snapshot 2 Enable. This bit controls the capturing of Auxiliary Snapshot Trigger 2. When this bit is set, the auxiliary snapshot of the event on ptp_aux_trig_i[2] input is enabled. When this bit is reset, the events on this input are ignored. **Value After Reset:** 0x0 **Exists:** (DWCXG_AUX_SNAP_IN_NUM>2)
5	ATSEN1	R/W	Auxiliary Snapshot 1 Enable. This bit controls the capturing of Auxiliary Snapshot Trigger 1. When this bit is set, the auxiliary snapshot of the event on ptp_aux_trig_i[1] input is enabled. When this bit is reset, the events on this input are ignored. **Value After Reset:** 0x0 **Exists:** (DWCXG_AUX_SNAP_IN_NUM>1)

Table 11-360 Fields for Register: MAC_Auxiliary_Control (continued)

Bits	Name	Memory Access	Description
4	ATSEN0	R/W	Auxiliary Snapshot 0 Enable. This bit controls the capturing of Auxiliary Snapshot Trigger 0. When this bit is set, the auxiliary snapshot of the event on ptp_aux_trig_i[0] input is enabled. When this bit is reset, the events on this input are ignored. **Value After Reset:** 0x0 **Exists:** (DWCXG_AUX_SNAP_IN_NUM>0)
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	ATSFC	R/W	Auxiliary Snapshot FIFO Clear. When set, this bit resets the pointers of the Auxiliary Snapshot FIFO. This bit is cleared when the pointers are reset and the FIFO is empty. When this bit is high, the auxiliary snapshots are stored in the FIFO. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.357 MAC_Auxiliary_Timestamp_Nanoseconds

- **Description:** The Auxiliary Timestamp - Nanoseconds register, along with MAC_Auxiliary_Timestamp_Seconds, gives the 64-bit timestamp stored as auxiliary snapshot. These two registers form the read port of a 64-bit wide FIFO with a depth of 4, 8, or 16 as selected while configuring the controller.
  You can store multiple snapshots in this FIFO. Bits[29:25] in MAC_Timestamp_Status indicate the fill-level of the FIFO. The top of the FIFO is removed only when the last byte of MAC_Auxiliary_Timestamp_Seconds register is read. In the Little-endian mode, this means Bits[31:24] are read and in Big-endian mode, Bits[7:0] are read.
  This register and MAC_Auxiliary_Timestamp_Seconds register are present only when you select the Add IEEE 1588 Auxiliary Snapshot option while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xd48
- **Exists:** (DWCXG_ADV_TIME_AUX_SNAP)

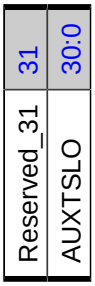


Table 11-361 Fields for Register: MAC_Auxiliary_Timestamp_Nanoseconds

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:0	AUXTSLO	R	Auxiliary Timestamp Low. Contains the lower 31 bits (nanoseconds field) of the auxiliary timestamp. **Value After Reset:** 0x0 **Exists:** Always

11.1.358 MAC_Auxiliary_Timestamp_Seconds

- **Description:** The Auxiliary Timestamp - Seconds register contains the lower 32 bits of the Seconds field of the auxiliary timestamp register.
- **Size:** 32 bits
- **Offset:** 0xd4c
- **Exists:** (DWCXG_ADV_TIME_AUX_SNAP)

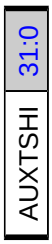


Table 11-362 Fields for Register: MAC_Auxiliary_Timestamp_Seconds

Bits	Name	Memory Access	Description
31:0	AUXTSHI	R	Auxiliary Timestamp High. Contains the lower 32 bits of the Seconds field of the auxiliary timestamp. **Value After Reset:** 0x0 **Exists:** Always

11.1.359 MAC_Timestamp_Ingress_Asym_Corr

- **Description:** The MAC Timestamp Ingress Asymmetry Correction register contains the Ingress Asymmetry Correction value to be used while updating correction field in some PTP messages. Ideally, Ingress Asymmetry correction needs to be done in the receive side, when the corresponding packet is received. But when one-step time stamping is enabled, the controller performs ingress asymmetry correction on the transmit (egress) side, which still holds good for packets that are received and transmitted on the same port (for example, Pdelay Resp, which is sent for a Pdelay Request received in the same port).
  In other cases (for example, transparent bridge), this feature can still be used if all receive ports have the same ingress correction value. If not, then OSTIAC must be programmed to 0x0.
- **Size:** 32 bits
- **Offset:** 0xd50
- **Exists:** (DWCXG_OST_EN||DWCXG_PTO_EN)

Table 11-363 Fields for Register: MAC_Timestamp_Ingress_Asym_Corr

Bits	Name	Memory Access	Description
31:0	OSTIAC	R/W	One-Step Timestamp Ingress Asymmetry Correction. This field contains the ingress path asymmetry value to be added to correctionField of Sync and Pdelay_Resp PTP packet. The programmed value must be in units of nanoseconds and multiplied by 2^16. For example, 2.5 ns is represented as 0x00028000. The value can also be negative and then is represented in twos complement form with bit 31 representing the sign bit. **Value After Reset:** 0x0 **Exists:** Always

11.1.360 MAC_Timestamp_Egress_Asym_Corr

- **Description:** The MAC Timestamp Egress Asymmetry Correction register contains the Egress Asymmetry Correction value to be used while updating the correction field in for some PTP message types.
- **Size:** 32 bits
- **Offset:** 0xd54
- **Exists:** (DWCXG_OST_EN)

Table 11-364 Fields for Register: MAC_Timestamp_Egress_Asym_Corr

Bits	Name	Memory Access	Description
31:0	OSTEAC	R/W	One-Step Timestamp Egress Asymmetry Correction. This field contains the egress path asymmetry value to be subtracted from correctionField of Delay_Req and Pdelay_Req PTP packet. The programmed value must be the negated value in units of nanoseconds multiplied by 2^16. For example, if the required correction is +2.5 ns, then the programmed value must be 0xFFFD_8000, which is twos complement of 0x0002_8000 (2.5 * 2^16). Similarly, if the required correction is -3.3 ns, then the programmed value is 0x0003_4CCC (3.3 * 2^16). **Value After Reset:** 0x0 **Exists:** Always

11.1.361 MAC_Timestamp_Ingress_Corr_Nanosecond

- **Description:** The MAC Timestamp Ingress Correction Nanosecond register contains the correction value to be used with the captured timestamp value in the ingress path. This is used to compensate the errors due to timestamp synchronization from the PTP clock domain to the MAC Rx clock domain, and the latency incurred between the message timestamp point and the point at which the timestamp snapshot is taken.
- **Size:** 32 bits
- **Offset:** 0xd58
- **Exists:** DWCXG_TIME_STAMPING

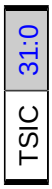


Table 11-365 Fields for Register: MAC_Timestamp_Ingress_Corr_Nanosecond

Bits	Name	Memory Access	Description
31:0	TSIC	R/W	Timestamp Ingress Correction. This field contains the ingress path correction value in nanoseconds as defined by the "Ingress Correction" expression. **Value After Reset:** 0x0 **Exists:** Always

11.1.362 MAC_Timestamp_Ingress_Corr_Subnanosecond

- **Description:** The MAC Timestamp Ingress Correction Sub-Nanosecond register contains the sub-nanosecond part of the correction value to be used with the captured timestamp value, for ingress directions.
- **Size:** 32 bits
- **Offset:** 0xd5c
- **Exists:** DWCXG_ADV_SUBNSEC_SUPPORT

Table 11-366 Fields for Register: MAC_Timestamp_Ingress_Corr_Subnanosecond

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:8	TSICSNS	R/W	Timestamp Ingress Correction, sub-nanoseconds. This field contains the sub-nanoseconds part of the ingress path correction value as defined by the "Ingress Correction" expression. **Value After Reset:** 0x0 **Exists:** Always
7:0	Reserved_7_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.363 MAC_Timestamp_Egress_Corr_Nanosecond

- **Description:** The MAC Egress Timestamp Correction Nanosecond register contains the correction value to be used with the captured timestamp value in the egress path. This is used to compensate the errors due to timestamp synchronization from PTP clock domain to MAC Tx clock domain, and the latency incurred between the point at which the timestamp snapshot is taken and the message timestamp point.
- **Size:** 32 bits
- **Offset:** 0xd60
- **Exists:** DWCXG_TIME_STAMPING

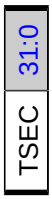


Table 11-367 Fields for Register: MAC_Timestamp_Egress_Corr_Nanosecond

Bits	Name	Memory Access	Description
31:0	TSEC	R/W	Timestamp Egress Correction. This field contains the nanoseconds part of the egress path correction value as defined by the "Egress Correction" expression, **Value After Reset:** 0x0 **Exists:** Always

11.1.364 MAC_Timestamp_Egress_Corr_Subnanosecond

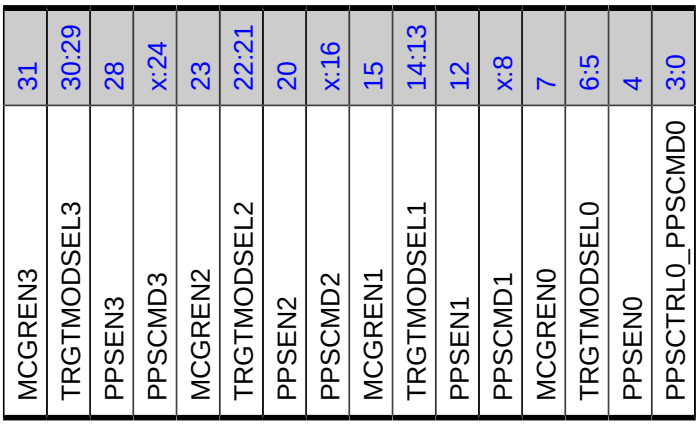
- **Description:** The MAC Timestamp Correction Egress Sub-nanosecond register contains the sub-nanosecond part of the correction value to be used with the captured timestamp value, for egress directions.
- **Size:** 32 bits
- **Offset:** 0xd64
- **Exists:** DWCXG_ADV_SUBNSEC_SUPPORT

Table 11-368 Fields for Register: MAC_Timestamp_Egress_Corr_Subnanosecond

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:8	TSECSNS	R/W	Timestamp Egress Correction, sub-nanoseconds. This field contains the sub-nanoseconds part of the egress path correction value as defined by the "Egress Correction" expression. **Value After Reset:** 0x0 **Exists:** Always
7:0	Reserved_7_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.365 MAC_PPS_Control

- **Description:** The PPS Control register is present only when the Timestamp feature is selected and External Timestamp is not enabled.
  Bits[30:24] of this register are valid only when four Flexible PPS outputs are selected. Bits[22:16] are valid only when three or more Flexible PPS outputs are selected. Bits[14:8] are valid only when two or more Flexible PPS outputs are selected. Bits[6:4] are valid. only when Flexible PPS feature is selected.
- **Size:** 32 bits
- **Offset:** 0xd70
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)



31	30:29	28	x:24	23	22:21	20	x:16	15	14:13	12	x:8	7	6:5	4	3:0
MCGREN3	TRGTMODSEL3	PPSEN3	PPSCMD3	MCGREN2	TRGTMODSEL2	PPSEN2	PPSCMD2	MCGREN1	TRGTMODSEL1	PPSEN1	PPSCMD1	MCGREN0	TRGTMODSEL0	PPSEN0	PPSCTRL0_PPSCMD0

Table 11-369 Fields for Register: MAC_PPS_Control

Bits	Name	Memory Access	Description
31	MCGREN3	R/W	MCGR Mode Enable for PPS3 Output This field enables the 3rd PPS instance to operate in PPS or MCGR mode. ■ 1: Operates in MCGR mode ■ 0: Operates in PPS mode **Value After Reset:** 0x0 **Exists:** DWCXG_MCGR&&DWCXG_PPS_OUT_3_EN **Testable:** untestable
30:29	TRGTMODSEL3	R/W	Target Time Register Mode for PPS3 Output. This field indicates the Target Time registers mode for PPS3 output signal. This field is similar to the TRGTMODSEL0 field. **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>3) **Testable:** untestable

Table 11-369 Fields for Register: MAC_PPS_Control (continued)

Bits	Name	Memory Access	Description
28	PPSEN3	R/W	Flexible PPS Output Mode Enable. When this bit is set, Bits[3:0] function as PPSCMD. When this bit is reset, Bits[3:0] function as PPSCTRL (Fixed PPS mode). **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>3) **Testable:** untestable
x:24	PPSCMD3	R/W	Flexible PPS3 Output Control. This field controls the flexible PPS3 output (ptp_pps_o[3]) signal. This field is similar to the PPSCMD0[2:0] field. ■ When MCGREN3 is 1, PPSCMD3 indicated by the 4 bits [27:24] are taken as Presentation Time Control bits for media clock generation and recovery for comparator instance 3. This field is similar to the PPSCMD0 Presentation Time Control bits. ■ When MCGREN3 is 0, only 3 bits from [26:24] are used as PPSCMD3 and the 4th bit must be set as 0. Access restriction applies. Setting 1 sets. Self-cleared. Setting 0 has no effect. **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>3) **Testable:** untestable **Range Variable[x]:** DWCXG_MCGR?4:3 + 23
23	MCGREN2	R/W	MCGR Mode Enable for PPS2 Output This field enables the 2nd PPS instance to operate in PPS or MCGR mode. ■ 1: Operates in MCGR mode ■ 0: Operates in PPS mode **Value After Reset:** 0x0 **Exists:** DWCXG_MCGR&&DWCXG_PPS_OUT_2_EN **Testable:** untestable
22:21	TRGTMODSEL2	R/W	Target Time Register Mode for PPS2 Output. This field indicates the Target Time registers mode for PPS2 output signal. This field is similar to the TRGTMODSEL0 field. **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>2) **Testable:** untestable

Table 11-369 Fields for Register: MAC_PPS_Control (continued)

Bits	Name	Memory Access	Description
20	PPSEN2	R/W	Flexible PPS Output Mode Enable. When this bit is set, Bits[3:0] function as PPSCMD. When this bit is reset, Bits[3:0] function as PPSCTRL (Fixed PPS mode). **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>2) **Testable:** untestable
x:16	PPSCMD2	R/W	Flexible PPS2 Output Control. This field controls the flexible PPS2 output (ptp_pps_o[2]) signal. This field is similar to the PPSCMD0 field. ■ When MCGREN2 is 1, PPSCMD2 indicated by the 4 bits [19:16] are taken as Presentation Time Control bits for media clock generation and recovery for comparator instance 2. This field is similar to the PPSCMD0 Presentation Time Control bits. ■ When MCGREN2 is 0, only 3 bits from [18:16] are used as PPSCMD2 and the 4th bit must be set as 0. Access restriction applies. Setting 1 sets. Self-cleared. Setting 0 has no effect. **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>2) **Testable:** untestable **Range Variable[x]:** DWCXG_MCGR?4:3 + 15
15	MCGREN1	R/W	MCGR Mode Enable for PPS1 Output This field enables the 1st PPS instance to operate in PPS or MCGR mode. ■ 1: Operates in MCGR mode ■ 0: Operates in PPS mode **Value After Reset:** 0x0 **Exists:** DWCXG_MCGR&&DWCXG_PPS_OUT_1_EN **Testable:** untestable
14:13	TRGTMODSEL1	R/W	Target Time Register Mode for PPS1 Output. This field indicates the Target Time registers mode for PPS1 output signal. This field is similar to the TRGTMODSEL0 field. **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>1) **Testable:** untestable

Table 11-369 Fields for Register: MAC_PPS_Control (continued)

Bits	Name	Memory Access	Description
12	PPSEN1	R/W	Flexible PPS Output Mode Enable. When this bit is set, Bits[3:0] function as PPSCMD. When this bit is reset, Bits[3:0] function as PPSCTRL (Fixed PPS mode). **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>1) **Testable:** untestable
x:8	PPSCMD1	R/W	Flexible PPS1 Output Control. This field controls the flexible PPS1 output (ptp_pps_o[1]) signal. This field is similar to the PPSCMD0 field. ■ When MCGREN1 is 1, PPSCMD1 indicated by the 4 bits [11:8] are taken as Presentation Time Control bits for media clock generation and recovery for comparator instance 1. This field is similar to the PPSCMD0 Presentation Time Control bits. ■ When MCGREN1 is 0, only 3 bits from [10:8] are used as PPSCMD1 and the 4th bit must be set as 0. Access restriction applies. Setting 1 sets. Self-cleared. Setting 0 has no effect. **Value After Reset:** 0x0 **Exists:** (DWCXG_PPS_OUT_NUM>1) **Testable:** untestable **Range Variable[x]:** DWCXG_MCGR?4:3 + 7
7	MCGREN0	R/W	MCGR Mode Enable for PPS0 Output This field enables the 0th PPS instance to operate in PPS or MCGR mode. When set it operates in MCGR mode and on reset it operates in PPS mode. **Value After Reset:** 0x0 **Exists:** DWCXG_FLEXI_PPS_OUT_EN&&DWCXG_TIME_STAMPING==1&&DWCXG_SYSTIME_SOURCE!=1 **Testable:** untestable

Table 11-369 Fields for Register: MAC_PPS_Control (continued)

Bits	Name	Memory Access	Description
6:5	TRGTMODSEL0	R/W	Target Time Register Mode for PPS0 Output. This field indicates the Target Time registers mode for PPS0 output signal: ■ 00: Target Time registers are programmed only for generating the Target Time Reached interrupt event. The Flexible PPS function must not be enabled in this mode, otherwise spurious transitions might be observed on the corresponding ptp_pps_o output port ■ 01: Reserved ■ 10: Target Time registers are programmed for generating the Target Time Reached interrupt event and starting or stopping the PPS0 output signal generation. ■ 11: Target Time registers are programmed only for starting or stopping the PPS0 output signal generation. No Target Time Reached interrupt is asserted. **Value After Reset:** 0x0 **Exists:** DWCXG_FLEXI_PPS_OUT_EN **Testable:** untestable
4	PPSEN0	R/W	Flexible PPS Output Mode Enable. When this bit is set, Bits[3:0] function as PPSCMD. When this bit is reset, Bits[3:0] function as PPSCTRL (Fixed PPS mode). **Value After Reset:** 0x0 **Exists:** DWCXG_FLEXI_PPS_OUT_EN **Testable:** untestable
3:0	PPSCTRL0_PPSCMD0	R/W	PPSCMD0: Flexible PPS Output (ptp_pps_o[0]) Control Programming these bits with a non-zero value instructs the MAC to initiate an event. When the command is transferred or synchronized to the PTP clock domain, these bits get cleared automatically. The software must ensure that these bits are programmed only when they are all-zero. The following list describes the values of PPSCMD0: ■ 0000: No Command ■ 0001: START Single Pulse This command generates a single pulse rising at the start point defined in Target Time registers and of a duration defined in the PPS0 Width register. ■ 0010: START Pulse Train This command generates the train of pulses rising at the start point defined in the Target Time registers and of a duration defined in the PPS0 Width register and repeated at interval defined in the PPS Interval register. By default, the PPS pulse

Table 11-369 Fields for Register: MAC_PPS_Control (continued)

Bits	Name	Memory Access	Description
			train is free-running unless stopped by the "Stop Pulse train at time" or "Stop Pulse Train immediately" commands. ■ 0011: Cancel START This command cancels the START Single Pulse and START Pulse Train commands if the system time has not crossed the programmed start time. ■ 0100: STOP Pulse train at time This command stops the train of pulses initiated by the START Pulse Train command (PPSCMD = 0010) after the time programmed in the Target Time registers elapses. ■ 0101: STOP Pulse Train immediately This command immediately stops the train of pulses initiated by the START Pulse Train command (PPSCMD = 0010). ■ 0110: Cancel STOP Pulse train
3:0...(cont.)	PPSCTRL0_PPSCMD0.	R/W	This command cancels the STOP pulse train at time command if the programmed stop time has not elapsed. The PPS pulse train becomes free-running on the successful execution of this command. ■ 0111-1111: Reserved *or* **Presentation Time Control** When MCGREN0 is 1, these bits are treated as Presentation time control bits. The following list describes the values of PPSCMD0: ■ 0000: MCGR operation is not carried out. If set to this value in the mid of clock recovery or generation, all the processing inputs are flushed ■ 0001: Capture the Presentation time at the rising edge of mcg_pst_trig_i[0] into the MAC_PPS0_Target_Time_Seconds register ■ 0010: Capture the Presentation time at the falling edge of mcg_pst_trig_i[0] into the MAC_PPS0_Target_Time_Seconds register ■ 0011: Capture the Presentation time at both edges of mcg_pst_trig_i[0] into the MAC_PPS0_Target_Time_Seconds register ■ 0100-1000: Reserved ■ 1001: Toggle output on compare ■ 1010: Pulse output low on compare for one PTP-clock cycle ■ 1011: Pulse output high on compare for one PTP-clock cycle ■ 1100-1111: Reserved

Table 11-369 Fields for Register: MAC_PPS_Control (continued)

Bits	Name	Memory Access	Description
			This field can be the PPS Output Frequency Control (PPSCTRL0) or the Flexible PPS Output (ptp_pps_o[0]) Control (PPSCMD0) PPSCTRL0: PPS Output Frequency Control This field controls the frequency of the PPS0 output (ptp_pps_o[0]) signal. The default value of PPSCTRL is 0000, and the PPS output is 1 pulse (of width clk_ptp_ref_i) every second. For other values of PPSCTRL, the PPS output becomes a generated clock of following frequencies: ■ 0001: The binary rollover is 2 Hz, and the digital rollover is 1 Hz. ■ 0010: The binary rollover is 4 Hz, and the digital rollover is 2 Hz. ■ 0011: The binary rollover is 8 Hz, and the digital rollover is 4 Hz.
3:0...(cont..)	PPSCTRL0_PPSCMD0..	R/W	■ 0100: The binary rollover is 16 Hz, and the digital rollover is 8 Hz. ■ ... ■ 1111: The binary rollover is 32.768 KHz and the digital rollover is 16.384 KHz. In the binary rollover mode, the PPS output (ptp_pps_o) has a duty cycle of 50 percent with these frequencies. In the digital rollover mode, the PPS output frequency is an average number. The actual clock is of different frequency that gets synchronized every second. For example: When PPSCTRL = 0001, the PPS (1 Hz) has a low period of 537 ms and a high period of 463 ms When PPSCTRL = 0010, the PPS (2 Hz) is a sequence of: ■ One clock of 50 percent duty cycle and 537 ms period ■ Second clock of 463 ms period (268 ms low and 195 ms high) When PPSCTRL = 0011, the PPS (4 Hz) is a sequence of: ■ Three clocks of 50 percent duty cycle and 268 ms period ■ Fourth clock of 195 ms period (134 ms low and 61 ms high) This behavior is because of the non-linear toggling of bits in the digital rollover mode in the MAC_System_Time_Nanoseconds register. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.366 MAC_PPS_Extnd_Ctrl

- **Description:** PPS Extended Control register.
  This register contains the additional controls for the PPS feature.
- **Size:** 32 bits
- **Offset:** 0xd74
- **Exists:**
  ((DWCXG_FLEXI_PPS_OUT_EN&&DWCXG_TIME_STAMPING==1&&DWCXG_SYSTIME_SOURCE!=1))

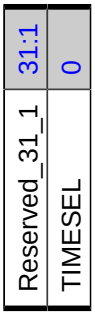


Table 11-370 Fields for Register: MAC_PPS_Extnd_Ctrl

Bits	Name	Memory Access	Description
31:1	Reserved_31_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	TIMESEL	R/W	Time Select When this bit is set, 64 bit PTP time is used to capture time at MCGR trigger input. When this bit is reset, presentation time is used to capture time at trigger input, maintaining backward compatibility. **Value After Reset:** 0x0 **Exists:** Always

11.1.367 MAC_PPS0_Target_Time_Seconds

- **Description:** The PPS0 Target Time Seconds register, along with PPS0 Target Time Nanoseconds register, is used to schedule an interrupt event [Bit 1 of MAC_Timestamp_Status] when the system time exceeds the value programmed in these registers.
  This register is present only when more than one Flexible PPS output is selected while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xd80
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)

Table 11-371 Fields for Register: MAC_PPS0_Target_Time_Seconds

Bits	Name	Memory Access	Description
31:0	TSTRH0	R/W	PPS0 Target Time Seconds Register. This field stores the time in seconds. When the timestamp value matches or exceeds both Target Timestamp registers, the MAC starts or stops the PPS signal output and generates an interrupt (if enabled) based on Bits[6:5] of MAC_PPS_Control register. If DWCXG_FLEXI_PPS_OUT_EN is enabled in the configuration and PTGE field of MAC_Timestamp_Control Register is set with Presentation time control set in recovery mode, then these bits indicate the TPT being programmed by the application and in generation mode it indicates the CPT generated at the sampled trigger. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.368 MAC_PPS0_Target_Time_Nanoseconds

- **Description:** The PPS0 Target Time Nanoseconds register is present only when more than one Flexible PPS output is selected.
- **Size:** 32 bits
- **Offset:** 0xd84
- **Exists:** (DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1)

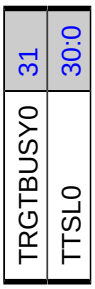


Table 11-372 Fields for Register: MAC_PPS0_Target_Time_Nanoseconds

Bits	Name	Memory Access	Description
31	TRGTBUSY0	R/W	PPS0 Target Time Register Busy. The MAC sets this bit when the PPSCMD0 field in the MAC_PPS_Control register is programmed to 010 or 011. Programming the PPSCMD0 field to 010 or 011 instructs the MAC to synchronize the Target Time registers to the PTP clock domain. The MAC clears this bit after synchronizing the Target Time registers to the PTP clock domain The application must not update the Target Time registers when this bit is read as 1. Otherwise, the synchronization of the previous programmed time gets corrupted. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-372 Fields for Register: MAC_PPS0_Target_Time_Nanoseconds (continued)

Bits	Name	Memory Access	Description
30:0	TTSL0	R/W	Target Time Low for PPS0 Register. This register stores the time in (signed) nanoseconds. When the value of the timestamp matches the value in both Target Timestamp registers, the MAC starts or stops the PPS signal output or generates a Target Time Reached interrupt (if enabled) based on the TRGTMODSEL0 field (Bits [6:5]) in MAC_PPS_Control. When the TSCTRLSSR bit is set in the MAC_Timestamp_Control register, this value must not exceed 0x3B9A_C9FF. The actual start or stop time of the PPS signal output might have an error margin up to one unit of sub-second increment value. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.369 MAC_PPS0_Interval

- **Description:** The PPS0 Interval register contains the number of units of sub-second increment value between the rising edges of PPS0 signal output (ptp_pps_o[0]).
- **Size:** 32 bits
- **Offset:** 0xd88
- **Exists:** (DWCXG_FLEXI_PPS_OUT_EN)

Table 11-373 Fields for Register: MAC_PPS0_Interval

Bits	Name	Memory Access	Description
31:0	PPSINT0	R/W	PPS0 Output Signal Interval. These bits store the interval between the rising edges of PPS0 signal output. The interval is stored in terms of number of units of sub-second increment value. You need to program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns), and desired interval between the rising edges of PPS0 signal output is 100 ns (that is, five units of sub-second increment value), you must program value 4 (5 - 1) in this register. **Value After Reset:** 0x0 **Exists:** Always

11.1.370 MAC_PPS0_Width

- **Description:** The PPS0 Width register contains the number of units of sub-second increment value between the rising and corresponding falling edges of PPS0 signal output (ptp_pps_o[0]).
- **Size:** 32 bits
- **Offset:** 0xd8c
- **Exists:** (DWCXG_FLEXI_PPS_OUT_EN)

Table 11-374 Fields for Register: MAC_PPS0_Width

Bits	Name	Memory Access	Description
31:0	PPSWIDTH0	R/W	PPS0 Output Signal Width. These bits store the width between the rising edge and corresponding falling edge of PPS0 signal output. The width is stored in terms of number of units of sub-second increment value. You need to program one value less than the required interval. For example, if PTP reference clock is 50 MHz (period of 20 ns), and width between the rising and corresponding falling edges of PPS0 signal output is 80 ns (that is, four units of sub-second increment value), you must program value 3 (4 - 1) in this register. **Note:** The value programmed in this register must be lesser than the value programmed in MAC_PPS0_Interval. **Value After Reset:** 0x0 **Exists:** Always

11.1.371 MAC_PPS1_Target_Time_Seconds

- **Description:** The PPS1 Target Time Seconds register, along with PPS1 Target Time Nanoseconds register, is used to schedule an interrupt event [Bit 1 of MAC_Timestamp_Status] when the system time exceeds the value programmed in these registers.
  This register is present only when more than one Flexible PPS output is selected while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xd90
- **Exists:** (DWCXG_PPS_OUT_NUM>1)

Table 11-375 Fields for Register: MAC_PPS1_Target_Time_Seconds

Bits	Name	Memory Access	Description
31:0	TSTRH1	R/W	PPS1 Target Time Seconds. This field stores the time in seconds. When the timestamp value matches or exceeds both Target Timestamp registers, the MAC starts or stops the PPS signal output and generates an interrupt (if enabled) based on Bits[14:13] of MAC_PPS_Control register. If DWCXG_FLEXI_PPS_OUT_EN is enabled in the configuration and PTGE field of MAC_Timestamp_Control Register is set with Presentation time control set in recovery mode, then these bits indicate the TPT being programmed by the application and in generation mode it indicates the CPT generated at the sampled trigger. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.372 MAC_PPS1_Target_Time_Nanoseconds

- **Description:** The PPS1 Target Time Nanoseconds register, along with PPS1 Target Time Seconds register, is used to schedule an interrupt event [Bit 1 of MAC_Timestamp_Status] when the system time exceeds the value programmed in these registers.This register is present only when more than one Flexible PPS output is selected while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xd94
- **Exists:** (DWCXG_PPS_OUT_NUM>1)

Table 11-376 Fields for Register: MAC_PPS1_Target_Time_Nanoseconds

Bits	Name	Memory Access	Description
31	TRGTBUSY1	R/W	PPS1 Target Time Register Busy. The MAC sets this bit when the PPSCMD1 field in MAC_PPS_Control register is programmed to 010 or 011. Programming the PPSCMD1 field to 010 or 011 instructs the MAC to synchronize the Target Time registers to the PTP clock domain. The MAC clears this bit after synchronizing the Target Time registers to the PTP clock domain The application must not update the Target Time registers when this bit is read as 1. Otherwise, the synchronization of the previous programmed time gets corrupted. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-376 Fields for Register: MAC_PPS1_Target_Time_Nanoseconds (continued)

Bits	Name	Memory Access	Description
30:0	TTSL1	R/W	Target Time Low for PPS1 Register. This register stores the time in (signed) nanoseconds. When the value of the timestamp matches the both Target Timestamp registers, then based on the TRGTMODSEL1 field in the MAC_PPS_Control register, the MAC starts or stops the PPS signal output or generates a Target Time Reached interrupt (if enabled). This value must not exceed 0x3B9A_C9FF when the TSCTRLSSR field is set in the MAC_Timestamp_Control register. The actual start or stop time of the PPS signal output might have an error margin up to one unit of sub-second increment value. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.373 MAC_PPS1_Interval

- **Description:** The PPS1 Interval register contains the number of units of sub-second increment value between the rising edges of PPS1 signal output (ptp_pps_o[1]).
- **Size:** 32 bits
- **Offset:** 0xd98
- **Exists:** (DWCXG_PPS_OUT_NUM>1)

Table 11-377 Fields for Register: MAC_PPS1_Interval

Bits	Name	Memory Access	Description
31:0	PPSINT1	R/W	PPS1 Output Signal Interval. These bits store the interval between the rising edges of PPS1 signal output. The interval is stored in terms of number of units of sub-second increment value. You need to program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns), and desired interval between the rising edges of PPS1 signal output is 100 ns (that is, five units of sub-second increment value), you must program value 4 (5 - 1) in this register. **Value After Reset:** 0x0 **Exists:** Always

11.1.374 MAC_PPS1_Width

- **Description:** The PPS1 Width register contains the number of units of sub-second increment value between the rising and corresponding falling edges of PPS1 signal output (ptp_pps_o[1]).
- **Size:** 32 bits
- **Offset:** 0xd9c
- **Exists:** (DWCXG_PPS_OUT_NUM>1)

Table 11-378 Fields for Register: MAC_PPS1_Width

Bits	Name	Memory Access	Description
31:0	PPSWIDTH1	R/W	PPS1 Output Signal Width. These bits store the width between the rising edge and corresponding falling edge of PPS1 signal output. The width is stored in terms of number of units of sub-second increment value. You need to program one value less than the required interval. For example, if PTP reference clock is 50 MHz (period of 20 ns), and width between the rising and corresponding falling edges of PPS1 signal output is 80 ns (that is, four units of sub-second increment value), you must program value 3 (4 - 1) in this register. **Note:** The value programmed in this register must be lesser than the value programmed in MAC_PPS1_Interval. **Value After Reset:** 0x0 **Exists:** Always

11.1.375 MAC_PPS2_Target_Time_Seconds

- **Description:** The PPS2 Target Time Seconds register, along with PPS2 Target Time Nanoseconds register, is used to schedule an interrupt event [Bit 1 of MAC_Timestamp_Status] when the system time exceeds the value programmed in these registers.
  This register is present only when more than one Flexible PPS output is selected while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xda0
- **Exists:** (DWCXG_PPS_OUT_NUM>2)

Table 11-379 Fields for Register: MAC_PPS2_Target_Time_Seconds

Bits	Name	Memory Access	Description
31:0	TSTRH2	R/W	PPS2 Target Time Seconds. This field stores the time in seconds. When the timestamp value matches or exceeds both Target Timestamp registers, the MAC starts or stops the PPS signal output and generates an interrupt (if enabled) based on Bits[14:13] of MAC_PPS_Control register. If DWCXG_FLEXI_PPS_OUT_EN is enabled in the configuration and PTGE field of MAC_Timestamp_Control Register is set with Presentation time control set in recovery mode, then these bits indicate the TPT being programmed by the application and in generation mode it indicates the CPT generated at the sampled trigger. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.376 MAC_PPS2_Target_Time_Nanoseconds

- **Description:** This register is present only when more than two Flexible PPS output is selected while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xda4
- **Exists:** (DWCXG_PPS_OUT_NUM>2)

Table 11-380 Fields for Register: MAC_PPS2_Target_Time_Nanoseconds

Bits	Name	Memory Access	Description
31	TRGTBUSY2	R/W	PPS2 Target Time Register Busy. The MAC sets this bit when the PPSCMD1 field in MAC_PPS_Control register is programmed to 010 or 011. Programming the PPSCMD1 field to 010 or 011 instructs the MAC to synchronize the Target Time registers to the PTP clock domain. The MAC clears this bit after synchronizing the Target Time registers to the PTP clock domain The application must not update the Target Time registers when this bit is read as 1. Otherwise, the synchronization of the previous programmed time gets corrupted. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30:0	TTSL2	R/W	Target Time Low for PPS2 Register. This register stores the time in (signed) nanoseconds. When the value of the timestamp matches the both Target Timestamp registers, then based on the TRGTMODSEL2 field in the MAC_PPS_Control register, the MAC starts or stops the PPS signal output or generates a Target Time Reached interrupt (if enabled). This value must not exceed 0x3B9A_C9FF when the TSCTRLSSR field is set in the MAC_Timestamp_Control register. The actual start or stop time of the PPS signal output might have an error margin up to one unit of sub-second increment value. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.377 MAC_PPS2_Interval

- **Description:** The PPS2 Interval register contains the number of units of sub-second increment value between the rising edges of PPS2 signal output (ptp_pps_o[2]).
- **Size:** 32 bits
- **Offset:** 0xda8
- **Exists:** (DWCXG_PPS_OUT_NUM>2)

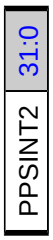


Table 11-381 Fields for Register: MAC_PPS2_Interval

Bits	Name	Memory Access	Description
31:0	PPSINT2	R/W	PPS2 Output Signal Interval. These bits store the interval between the rising edges of PPS2 signal output. The interval is stored in terms of number of units of sub-second increment value. You need to program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns), and desired interval between the rising edges of PPS2 signal output is 100 ns (that is, five units of sub-second increment value), you must program value 4 (5 - 1) in this register. **Value After Reset:** 0x0 **Exists:** Always

11.1.378 MAC_PPS2_Width

- **Description:** The PPS2 Width register contains the number of units of sub-second increment value between the rising and corresponding falling edges of PPS2 signal output (ptp_pps_o[2]).
- **Size:** 32 bits
- **Offset:** 0xdac
- **Exists:** (DWCXG_PPS_OUT_NUM>2)

Table 11-382 Fields for Register: MAC_PPS2_Width

Bits	Name	Memory Access	Description
31:0	PPSWIDTH2	R/W	PPS2 Output Signal Width. These bits store the width between the rising edge and corresponding falling edge of PPS2 signal output. The width is stored in terms of number of units of sub-second increment value. You need to program one value less than the required interval. For example, if PTP reference clock is 50 MHz (period of 20 ns), and width between the rising and corresponding falling edges of PPS2 signal output is 80 ns (that is, four units of sub-second increment value), you must program value 3 (4 - 1) in this register. **Note:** The value programmed in this register must be lesser than the value programmed in MAC_PPS2_Interval. **Value After Reset:** 0x0 **Exists:** Always

11.1.379 MAC_PPS3_Target_Time_Seconds

- **Description:** The PPS3 Target Time Seconds register, along with PPS3 Target Time Nanoseconds register, is used to schedule an interrupt event [Bit 1 of MAC_Timestamp_Status] when the system time exceeds the value programmed in these registers.
  This register is present only when more than one Flexible PPS output is selected while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xdb0
- **Exists:** (DWCXG_PPS_OUT_NUM>3)

Table 11-383 Fields for Register: MAC_PPS3_Target_Time_Seconds

Bits	Name	Memory Access	Description
31:0	TSTRH3	R/W	PPS3 Target Time Seconds. This field stores the time in seconds. When the timestamp value matches or exceeds both Target Timestamp registers, the MAC starts or stops the PPS signal output and generates an interrupt (if enabled) based on Bits[14:13] of MAC_PPS_Control register. If DWCXG_FLEXI_PPS_OUT_EN is enabled in the configuration and PTGE field of MAC_Timestamp_Control Register is set with Presentation time control set in recovery mode, then these bits indicate the TPT being programmed by the application and in generation mode it indicates the CPT generated at the sampled trigger. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.380 MAC_PPS3_Target_Time_Nanoseconds

- **Description:** This register is present only when more than three Flexible PPS output is selected while configuring the controller.
- **Size:** 32 bits
- **Offset:** 0xdb4
- **Exists:** (DWCXG_PPS_OUT_NUM>3)

Table 11-384 Fields for Register: MAC_PPS3_Target_Time_Nanoseconds

Bits	Name	Memory Access	Description
31	TRGTBUSY3	R/W	PPS3 Target Time Register Busy. The MAC sets this bit when the PPSCMD1 field in MAC_PPS_Control register is programmed to 010 or 011. Programming the PPSCMD1 field to 010 or 011 instructs the MAC to synchronize the Target Time registers to the PTP clock domain. The MAC clears this bit after synchronizing the Target Time registers to the PTP clock domain The application must not update the Target Time registers when this bit is read as 1. Otherwise, the synchronization of the previous programmed time gets corrupted. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30:0	TTSL3	R/W	Target Time Low for PPS3 Register. This register stores the time in (signed) nanoseconds. When the value of the timestamp matches the both Target Timestamp registers, then based on the TRGTMODSEL3 field in the MAC_PPS_Control register, the MAC starts or stops the PPS signal output or generates a Target Time Reached interrupt (if enabled). This value must not exceed 0x3B9A_C9FF when the TSCTRLSSR field is set in the MAC_Timestamp_Control register. The actual start or stop time of the PPS signal output might have an error margin up to one unit of sub-second increment value. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.1.381 MAC_PPS3_Interval

- **Description:** The PPS3 Interval register contains the number of units of sub-second increment value between the rising edges of PPS3 signal output (ptp_pps_o[3]).
- **Size:** 32 bits
- **Offset:** 0xdb8
- **Exists:** (DWCXG_PPS_OUT_NUM>3)

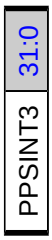


Table 11-385 Fields for Register: MAC_PPS3_Interval

Bits	Name	Memory Access	Description
31:0	PPSINT3	R/W	PPS3 Output Signal Interval. These bits store the interval between the rising edges of PPS3 signal output. The interval is stored in terms of number of units of sub-second increment value. You need to program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns), and desired interval between the rising edges of PPS3 signal output is 100 ns (that is, five units of sub-second increment value), you must program value 4 (5 - 1) in this register. **Value After Reset:** 0x0 **Exists:** Always

11.1.382 MAC_PPS3_Width

- **Description:** The PPS3 Width register contains the number of units of sub-second increment value between the rising and corresponding falling edges of PPS3 signal output (ptp_pps_o[3]).
- **Size:** 32 bits
- **Offset:** 0xdbc
- **Exists:** (DWCXG_PPS_OUT_NUM>3)

Table 11-386 Fields for Register: MAC_PPS3_Width

Bits	Name	Memory Access	Description
31:0	PPSWIDTH3	R/W	PPS3 Output Signal Width. These bits store the width between the rising edge and corresponding falling edge of PPS3 signal output. The width is stored in terms of number of units of sub-second increment value. You need to program one value less than the required interval. For example, if PTP reference clock is 50 MHz (period of 20 ns), and width between the rising and corresponding falling edges of PPS3 signal output is 80 ns (that is, four units of sub-second increment value), you must program value 3 (4 - 1) in this register. **Note:** The value programmed in this register must be lesser than the value programmed in MAC_PPS3_Interval. **Value After Reset:** 0x0 **Exists:** Always

11.1.383 MAC_PTO_Control

- **Description:** This register controls the PTP Offload Engine operation. This register is available only when the Enable PTP Timestamp Offload feature is selected.
- **Size:** 32 bits
- **Offset:** 0xdc0
- **Exists:** (DWCXG_PTO_EN)

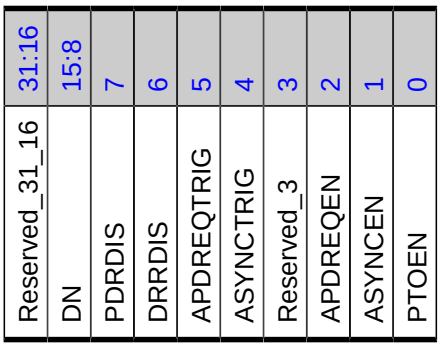


Table 11-387 Fields for Register: MAC_PTO_Control

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:8	DN	R/W	Domain Number. This field indicates the domainNumber in which the PTP node operates. **Value After Reset:** 0x0 **Exists:** Always
7	PDRDIS	R/W	Disable Peer Delay Response response generation When this bit is set, the Peer Delay Response (Pdelay_Resp) response is not generated for received Peer Delay Request (Pdelay_Req) request packet, as required by the programmed mode. **Note:** Setting this bit to 1 affects the normal PTP Offload operation and the time synchronization. So, this bit must be set only if there is problem with Pdelay_Resp generation in hardware and/or Pdelay_Resp generation is handled by Software. **Value After Reset:** 0x0 **Exists:** Always

Table 11-387 Fields for Register: MAC_PTO_Control (continued)

Bits	Name	Memory Access	Description
6	DRRDIS	R/W	Disable PTO Delay Request/Response response generation. When this bit is set, the Delay Request and Delay response is not generated for received SYNC and Delay request packet respectively, as required by the programmed mode. **Value After Reset:** 0x0 **Exists:** Always
5	APDREQTRIG	R/W	Automatic PTP Pdelay_Req message Trigger. When this bit is set, one PTP Pdelay_Req message is transmitted. This bit is automatically cleared after the PTP Pdelay_Req message is transmitted. The application must set the APDREQEN bit for this operation. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
4	ASYNCTRIG	R/W	Automatic PTP SYNC message Trigger. When this bit is set, one PTP SYNC message is transmitted. This bit is automatically cleared after the PTP SYNC message is transmitted. The application must set the ASYNCEN bit for this operation. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	APDREQEN	R/W	Automatic PTP Pdelay_Req message Enable. When this bit is set, PTP Pdelay_Req message is generated periodically based on interval programmed or trigger from application, when the MAC is programmed to be in Peer-to-Peer Transparent mode. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-387 Fields for Register: MAC_PTO_Control (continued)

Bits	Name	Memory Access	Description
1	ASYNCEN	R/W	Automatic PTP SYNC message Enable. When this bit is set, PTP SYNC message is generated periodically based on interval programmed or trigger from application, when the MAC is programmed to be in Clock Manager mode. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	PTOEN	R/W	PTP Offload Enable. When this bit is set, the PTP Offload feature is enabled. **Value After Reset:** 0x0 **Exists:** Always

11.1.384 MAC_Source_Port_Identity0

- **Description:** This register along with MAC_Source_Port_Identity1 and MAC_Source_Port_Identity2 contains the 80-bit Source Port Identity of the PTP node. These registers are available only when the Enable PTP Timestamp Offload feature is selected.
- **Size:** 32 bits
- **Offset:** 0xdc4
- **Exists:** (DWCXG_PTO_EN)

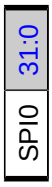


Table 11-388 Fields for Register: MAC_Source_Port_Identity0

Bits	Name	Memory Access	Description
31:0	SPI0	R/W	Source Port Identity 0. This field indicates bits [31:0] of sourcePortIdentity of PTP node. **Value After Reset:** 0x0 **Exists:** Always

11.1.385 MAC_Source_Port_Identity1

- **Description:** This register along with MAC_Source_Port_Identity0 and MAC_Source_Port_Identity2 contains the 80-bit Source Port Identity of the PTP node. These registers are available only when the Enable PTP Timestamp Offload feature is selected.
- **Size:** 32 bits
- **Offset:** 0xdc8
- **Exists:** (DWCXG_PTO_EN)

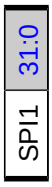


Table 11-389 Fields for Register: MAC_Source_Port_Identity1

Bits	Name	Memory Access	Description
31:0	SPI1	R/W	Source Port Identity 1. This field indicates bits [63:32] of sourcePortIdentity of PTP node. **Value After Reset:** 0x0 **Exists:** Always

11.1.386 MAC_Source_Port_Identity2

- **Description:** This register along with MAC_Source_Port_Identity0 and MAC_Source_Port_Identity1 contains the 80-bit Source Port Identity of the PTP node. These registers are available only when the Enable PTP Timestamp Offload feature is selected.
  Note: The values written in MAC_Source_Port_Identity0, MAC_Source_Port_Identity1 and MAC_Source_Port_Identity2 registers is updated in the PTP Offload logic when MAC_Source_Port_Identity2 register is written, so MAC_Source_Port_Identity2 register must be written after MAC_Source_Port_Identity0 and MAC_Source_Port_Identity1 registers are written.
- **Size:** 32 bits
- **Offset:** 0xdcc
- **Exists:** (DWCXG_PTO_EN)

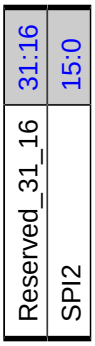


Table 11-390 Fields for Register: MAC_Source_Port_Identity2

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:0	SPI2	R/W	Source Port Identity 2. This field indicates bits [79:64] of sourcePortIdentity of PTP node. **Value After Reset:** 0x0 **Exists:** Always

11.1.387 MAC_Log_Message_Interval

- **Description:** This register contains the periodic intervals for automatic PTP packet generation. This register is available only when the Enable PTP Timestamp Offload feature is selected.
- **Size:** 32 bits
- **Offset:** 0xdd0
- **Exists:** (DWCXG_PTO_EN)

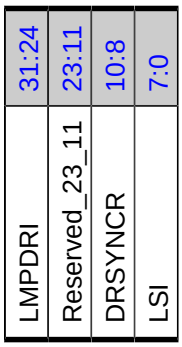


Table 11-391 Fields for Register: MAC_Log_Message_Interval

Bits	Name	Memory Access	Description
31:24	LMPDRI	R/W	Log Min Pdelay_Req Interval. This field indicates logMinPdelayReqInterval of PTP node. This is used to schedule the periodic Pdelay request packet transmission. Allowed values are -15 to 15. Negative value must be represented in twos complement form. For example, if the required value is -1, the value programmed must be 0xFF. **Value After Reset:** 0x0 **Exists:** Always
23:11	Reserved_23_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
10:8	DRSYNCR	R/W	Delay_Req to SYNC Ratio. In Subordinate mode, this is used to control the frequency of Delay_Req messages transmitted. ■ 'x0: DelayReq generated for every received SYNC messages ■ 'x1: DelayReq generated for every alternate reception of SYNC messages ■ 'x2: DelayReq generated for every 4 SYNC messages ■ 'x3: DelayReq generated for every 8 SYNC messages ■ 'x4: DelayReq generated for every 16 SYNC messages ■ 'x5: DelayReq generated for every 32 SYNC messages ■ 'x6-'x7: Reserved

Table 11-391 Fields for Register: MAC_Log_Message_Interval (continued)

Bits	Name	Memory Access	Description
			The manager sends this information (logMinDelayReqInterval) in the DelayResp PTP messages to the subordinate. The DWC_25gmac Receiver processes this value from received DelayResp messages and updates this field accordingly. In Subordinate mode, the host must not write/update this register unless it wants to override the received value. In Manager mode, the sum of this field and logSyncInterval (LSI) field is provided in the logMinDelayReqInterval field of the generated multicast Delay_Resp PTP message. **Value After Reset:** 0x0 **Exists:** Always
7:0	LSI	R/W	Log Sync Interval. This field indicates the periodicity of the automatically generated SYNC message when the PTP node is Manager. Allowed values are -15 to 15. Negative value must be represented in twos complement form. For example, if the required value is -1, the value programmed must be 0xFF. **Value After Reset:** 0x0 **Exists:** Always

11.1.388 MAC_Presn_Time_ns

- **Description:** This register contains the 32-bit binary rollover equivalent time of the PTP System Time in ns.
- **Size:** 32 bits
- **Offset:** 0xde0
- **Exists:** ((DWCXG_FLEXI_PPS_OUT_EN&&DWCXG_TIME_STAMPING==1&&DWCXG_SYSTIME_SOURCE!=1))

Table 11-392 Fields for Register: MAC_Presn_Time_ns

Bits	Name	Memory Access	Description
31:0	MPTN	R	MAC 1722 Presentation Time in ns These bits indicate the value of the 32-bit binary rollover equivalent time of the PTP System Time in ns. **Value After Reset:** 0x0 **Exists:** Always

11.1.389 MAC_Presn_Time_Updt

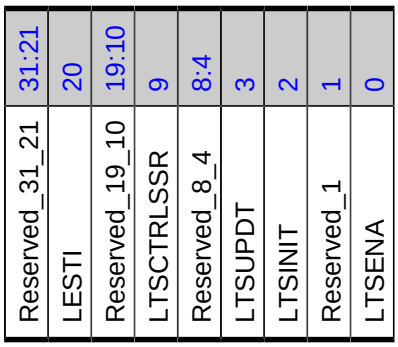
- **Description:** This field holds the 32-bit value of MAC 1722 Presentation Time in ns, that should be added to the Current Presentation Time Counter value. Init happens when TSINIT is set, and update happens when the TSUPDT bit is set (TSINIT and TSINIT defined in MAC_Timestamp_ Control register).
- **Size:** 32 bits
- **Offset:** 0xde4
- **Exists:** ((DWCXG_FLEXI_PPS_OUT_EN&&DWCXG_TIME_STAMPING==1&&DWCXG_SYSTIME_SOURCE!=1))

Table 11-393 Fields for Register: MAC_Presn_Time_Updt

Bits	Name	Memory Access	Description
31:0	MPTU	R/W	MAC 1722 Presentation Time Update This field holds the init value or the update value for the presentation time. When used for update, this field holds the 32-bit value in ns, that should be added to the Current Presentation Time Counter value. Init happens when TSINIT is set, and update happens when the TSUPDT bit is set (TSINIT and TSINIT defined in MAC_Timestamp_ Control register). When ADDSUB field of MAC_System_Time_Nanoseconds_Update is set, this value is directly used for subtraction. **Value After Reset:** 0x0 **Exists:** Always

11.1.390 MAC_Local_Timestamp_Control

- **Description:** This register controls the operation of the Local System Time generator and processing of PTP packets for timestamping in the Receiver.
- **Size:** 32 bits
- **Offset:** 0xe00
- **Exists:** (DWCXG_DSTS)



31:21	20	19:10	9	8:4	3	2	1	0
Reserved_31_21	LESTI	Reserved_19_10	LTSCTRLSSR	Reserved_8_4	LTSUPDT	LTSINIT	Reserved_1	LTSENA

Table 11-394 Fields for Register: MAC_Local_Timestamp_Control

Bits	Name	Memory Access	Description
31:21	Reserved_31_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
20	LESTI	R/W	External Local System Time Input. When this bit is set, the MAC uses the external 64-bit reference Local System Time input for the following: ■ To take the timestamp provided as status ■ To insert the timestamp in transmit PTP packets when One-step Timestamp or Timestamp Offload feature is enabled. When this bit is reset, the MAC uses the internal reference System Time. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE==2)
19:10	Reserved_19_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-394 Fields for Register: MAC_Local_Timestamp_Control (continued)

Bits	Name	Memory Access	Description
9	LTSCTRLSSR	R/W	Local Timestamp Digital or Binary Rollover Control. When this bit is set, the Local Timestamp Low register rolls over after 0x3B9A_C9FF value (that is, 1 nanosecond accuracy) and increments the local timestamp (High) seconds. When this bit is reset, the rollover value of sub-second register is 0x7FFF_FFFF. The sub-second increment must be programmed correctly depending on the PTP reference clock frequency and the value of this bit. **Value After Reset:** 0x0 **Exists:** Always
8:4	Reserved_8_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	LTSUPDT	R/W	Update Local Timestamp. When this bit is set, the system time is updated (added or subtracted) with the value specified in MAC_Local_System_Time_Seconds_Update and MAC_Local_System_Time_Nanoseconds_Update. This bit must be zero before updating it. This bit is reset when the update is complete in hardware. The Timestamp Higher Word register (if enabled during controller configuration) is not updated. When Media Clock Generation and Recovery is configured (DWCXG_FLEXI_PPS_OUT_EN) and enabled MAC_Local_Presn_Time_Updt should also be updated before setting this field. Access restriction applies. Setting 1 sets. Self-cleared. Setting 0 has no effect. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE!=1) **Testable:** untestable

Table 11-394 Fields for Register: MAC_Local_Timestamp_Control (continued)

Bits	Name	Memory Access	Description
2	LTSINIT	R/W	Initialize Local Timestamp. When this bit is set, the local system time is initialized (overwritten) with the value specified in the MAC_Local_System_Time_Seconds and MAC_Local_System_Time_Nanoseconds registers. This bit must be zero before it is updated. This bit is reset when the initialization is complete. The Timestamp Higher Word register (if enabled during controller configuration) can only be initialized. When Media Clock Generation and Recovery is configured (DWCXG_FLEXI_PPS_OUT_EN) and enabled MAC_Local_Presn_Time_Updt should also be updated before setting this field. Access restriction applies. Setting 1 sets. Self-cleared. Setting 0 has no effect. **Value After Reset:** 0x0 **Exists:** (DWCXG_SYSTIME_SOURCE!=1) **Testable:** untestable
1	Reserved_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	LTSENA	R/W	Enable Local Timestamp. When this bit is set, the timestamp is added for Transmit and Receive packets. When disabled, timestamp is not added for transmit and receive packets and the Timestamp Generator is also suspended. You need to initialize the Timestamp (local system time) after enabling this mode. On the Receive side, the MAC processes the 1588 packets only if this bit is set. **Value After Reset:** 0x0 **Exists:** Always

11.1.391 MAC_Local_Sub_Second_Increment

- **Description:** The Local Sub-Second Increment register is present only when the Dual Time Source Domain Capability is selected without an external timestamp input. In Coarse Update mode [Bit 1 in MAC_Timestamp_Control register], the value in this register is added to the system time every clock cycle of clk_ptp_ref_i. In Fine Update mode, the value in this register is added to the system time when the Accumulator gets an overflow.
- **Size:** 32 bits
- **Offset:** 0xe04
- **Exists:** (DWCXG_DSTS&&DWCXG_SYSTIME_SOURCE!=1)

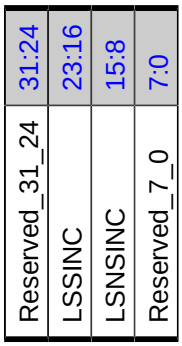


Table 11-395 Fields for Register: MAC_Local_Sub_Second_Increment

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:16	LSSINC	R/W	Local Sub-second Increment Value. The value programmed in this field is accumulated with the contents of the sub-second register. For example, when the PTP clock is 50 MHz (period is 20 ns), you must program 20 (0x14) when the Local System Time- Nanoseconds register has an accuracy of 1 ns [Bit 9 (LTSCTRLSSR) is set in MAC_Local_Timestamp_Control]. When LTSCTRLSSR is clear, the Nanoseconds register has a resolution of ~0.465 ns. In this case, you must program a value of 43 (0x2B) which is derived by 20 ns/0.465. **Value After Reset:** 0x0 **Exists:** Always

Table 11-395 Fields for Register: MAC_Local_Sub_Second_Increment (continued)

Bits	Name	Memory Access	Description
15:8	LSNSINC	R/W	Local Sub-nanosecond Increment Value. This field contains the sub-nanosecond increment value, represented in nanoseconds multiplied by 28. This value is accumulated with the sub-nanoseconds field of the sub-second register. For example, when LTSCTRLSSR field in the MAC_Local_Timestamp_Control register is set. and if the required increment is 5.3ns, then LSSINC must be 0x05 and LSNSINC must be 0x4C. **Value After Reset:** 0x0 **Exists:** DWCXG_ADV_SUBNSEC_SUPPORT
7:0	Reserved_7_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.1.392 MAC_Local_System_Time_Seconds

- **Description:** The Local System Time-Seconds register, along with Local System Time-Nanoseconds register, indicates the current value of the local system time maintained by the MAC. Though it is updated on a continuous basis, there is some delay from the actual time because of clock domain transfer latencies (from clk_ptp_ref_i to CSR clock).
  This register is present only when the Dual Time Source Domain Capability is selected without external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xe08
- **Exists:** (DWCXG_DSTS&&DWCXG_SYSTIME_SOURCE!=1)

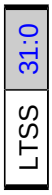


Table 11-396 Fields for Register: MAC_Local_System_Time_Seconds

Bits	Name	Memory Access	Description
31:0	LTSS	R	Local Timestamp Second. The value in this field indicates the current value in seconds of the Local System Time maintained by the MAC. **Value After Reset:** 0x0 **Exists:** Always

11.1.393 MAC_Local_System_Time_Nanoseconds

- **Description:** The Local System Time-Nanoseconds register, along with Local System Time-Seconds register, indicates the current value of the local system time maintained by the MAC.
  This register is present only when the Dual Time Source Domain Capability is selected without external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xe0c
- **Exists:** (DWCXG_DSTS&&DWCXG_SYSTIME_SOURCE!=1)

Table 11-397 Fields for Register: MAC_Local_System_Time_Nanoseconds

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:0	LTSSS	R	Local Timestamp Sub Seconds. The value in this field has the sub-second representation of time, with an accuracy of 0.46 ns. When Bit 9 is set in MAC_Local_Timestamp_Control, each bit represents 1 ns. The maximum value is 0x3B9A_C9FF after which it rolls-over to zero. **Value After Reset:** 0x0 **Exists:** Always

11.1.394 MAC_Local_System_Time_Seconds_Update

- **Description:** The Local System Time-Seconds Update register, along with the Local System Time-Nanoseconds Update register, initializes or updates the system time maintained by the MAC. You must write both registers before setting the LTSINIT or LTSUPDT bits in MAC_Local_Timestamp_Control register.
  This register is present only when the Dual Time Source Domain Capability is selected without external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xe10
- **Exists:** (DWCXG_DSTS&&DWCXG_SYSTIME_SOURCE!=1)

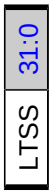


Table 11-398 Fields for Register: MAC_Local_System_Time_Seconds_Update

Bits	Name	Memory Access	Description
31:0	LTSS	R/W	Local Timestamp Second. The value in this field is the seconds part of the update. When LADDSUB is reset, this field must be programmed with the seconds part of the update value. When LADDSUB is set, this field must be programmed with the complement of the seconds part of the update value. For example, if 2.000000001 seconds need to be subtracted from the local system time, the LTSS field in the MAC_Local_Timestamp_Seconds_Update register must be 0xFFFF_FFFE (that is, 2^32 - 2). **Value After Reset:** 0x0 **Exists:** Always

11.1.395 MAC_Local_System_Time_Nanoseconds_Update

- **Description:** The Local System Time-Nanoseconds Update register, along with the Local System Time-Seconds Update register, initializes or updates the local system time maintained by the MAC. You must write both registers before setting the LTSINIT or LTSUPDT bits in MAC_Local_Timestamp_Control registerThis register is present only when the Dual Time Source Domain Capability is selected without external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xe14
- **Exists:** (DWCXG_DSTS&&DWCXG_SYSTIME_SOURCE!=1)

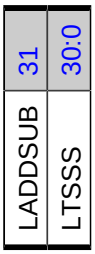


Table 11-399 Fields for Register: MAC_Local_System_Time_Nanoseconds_Update

Bits	Name	Memory Access	Description
31	LADDSUB	R/W	Add or Subtract Local Time. When this bit is set, the time value is subtracted with the contents of the update register. When this bit is reset, the time value is added with the contents of the update register. **Value After Reset:** 0x0 **Exists:** Always
30:0	LTSSS	R/W	Local Timestamp Sub Seconds. The value in this field is the sub-second part of the update. ■ When LADDSUB is reset, this field must be programmed with the sub-second part of the update value, with an accuracy based on the LTSCTRLSSR bit of the MAC Local Timestamp Control register. ■ When LADDSUB is set, this field must be programmed with the complement of the sub-second part of the update value, as follows. ■ When LTSCTRLSSR is set, the programmed value must be 10^9 - (sub-second value). When TSCTRLSSR is reset, the programmed value must be 2^31 - (sub-second_value). For example, when LTSCTRLSSR bit is set and if 2.000000001 second needs to be subtracted with the local system time, the LTSS field in the MAC Local Timestamp Seconds update register must be 0xFFFF_FFFE (for instance, 2^32 - 2), LADDSUB bit in this register must be set,

Table 11-399 Fields for Register: MAC_Local_System_Time_Nanoseconds_Update (continued)

Bits	Name	Memory Access	Description
			and the LTSSS field must be 0x3B9A_C9FF (for instance, 10^9 - 1). **Value After Reset:** 0x0 **Exists:** Always

11.1.396 MAC_Local_System_Time_Higher_Word_Seconds

- **Description:** This register contains the most-significant 16-bits of local timestamp seconds value.The Local System Time - Higher Word Seconds register is present only when the Dual Time Source Domain Capability is selected without an external timestamp input.
- **Size:** 32 bits
- **Offset:** 0xe1c
- **Exists:** (DWCXG_DSTS&&DWCXG_ADV_TIME_HIGH_WORD)

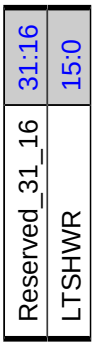


Table 11-400 Fields for Register: MAC_Local_System_Time_Higher_Word_Seconds

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:0	LTSHWR	R/W	Local Timestamp Higher Word Register. This field contains the most-significant 16-bits of local timestamp seconds value. This register is optional. You can add this register by selecting the Add IEEE 1588 Higher Word Register option. This register is directly written to initialize the value and it is incremented when there is an overflow from 32-bits of the Local System Time - Seconds register. **Value After Reset:** 0x0 **Exists:** Always

11.1.397 MAC_Dual_TS_Src_Ctrl

- **Description:** MAC Dual Timestamp Source Control.
  This register contains the per timestamp function clock selection control bits, whether Local or Correctable/Adjustable clock connected for a timestamp function.
- **Size:** 32 bits
- **Offset:** 0xe24
- **Exists:** (DWCXG_DSTS)

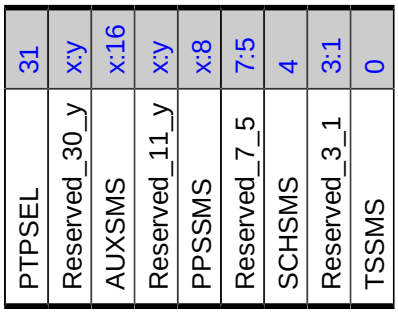


Table 11-401 Fields for Register: MAC_Dual_TS_Src_Ctrl

Bits	Name	Memory Access	Description
31	PTPSEL	R/W	Asynchronous Traffic Shaper (ATS) Source Mode Select When this bit is set to 1, Asynchronous Traffic Shaper (ATS) function works with Local Time Source. When this bit is set to 0, Asynchronous Traffic Shaper (ATS) function works with Correctable/Adjustable Time Source. **Value After Reset:** 0x0 **Exists:** DWCXG_ATS
x:y	Reserved_30_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_ADV_TIME_AUX_SNAP?15-DWCXG_AUX_SNAP_IN_NUM:15 + DWCXG_ADV_TIME_AUX_SNAP?16+DWCXG_AUX_SNAP_IN_NUM:16 - 1 **Range Variable[y]:** DWCXG_ADV_TIME_AUX_SNAP?16+DWCXG_AUX_SNAP_IN_NUM:16

Table 11-401 Fields for Register: MAC_Dual_TS_Src_Ctrl (continued)

Bits	Name	Memory Access	Description
x:16	AUXSMS	R/W	Auxiliary Timestamp Source Mode Select When this bit is set to 1, Auxiliary Timestamp function works with Local Time Source. When this bit is set to 0, Auxiliary Timestamp function works with Correctable/Adjustable Time Source. **Value After Reset:** 0x0 **Exists:** DWCXG_ADV_TIME_AUX_SNAP **Range Variable[x]:** DWCXG_AUX_SNAP_IN_NUM + 15
x:y	Reserved_11_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_FLEXI_PPS_OUT_EN?8-DWCXG_PPS_OUT_NUM:8 + DWCXG_FLEXI_PPS_OUT_EN?8+DWCXG_PPS_OUT_NUM:8 - 1 **Range Variable[y]:** DWCXG_FLEXI_PPS_OUT_EN?8+DWCXG_PPS_OUT_NUM:8
x:8	PPSSMS	R/W	Pulse Per Second (PPS) Source Mode Select When this bit is set to 1, Pulse Per Second (PPS) function works with Local Time Source. When this bit is set to 0, Pulse Per Second (PPS) function works with Correctable/Adjustable Time Source. **Value After Reset:** 0x0 **Exists:** DWCXG_FLEXI_PPS_OUT_EN **Range Variable[x]:** DWCXG_PPS_OUT_NUM + 7
7:5	Reserved_7_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	SCHSMS	R/W	Transmit Scheduler Source Mode Select When this bit is set to 1, Transmit Scheduler function works with Local Time Source. When this bit is set to 0, Transmit Scheduler function works with Correctable/Adjustable Time Source. **Value After Reset:** 0x0 **Exists:** ((DWCXG_TX_AV_ENABLE&&!DWCXG_CORE)||DWCXG_AV_EST||DWCXG_TBS||DWCXG_SGF)

Table 11-401 Fields for Register: MAC_Dual_TS_Src_Ctrl (continued)

Bits	Name	Memory Access	Description
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	TSSMS	R/W	Timestamp Capturing Source Mode Select When this bit is set to 1, Timestamp Capturing function works with Local Time Source. When this bit is set to 0, Timestamp Capturing function works with Correctable/Adjustable Time Source. **Value After Reset:** 0x0 **Exists:** Always

11.1.398 MAC_Local_Presn_Time_ns

- **Description:** This register contains the 32-bit binary rollover equivalent time of the PTP Local System Time in ns.
- **Size:** 32 bits
- **Offset:** 0xe30
- **Exists:** (DWCXG_MCGR&&DWCXG_DSTS)

Table 11-402 Fields for Register: MAC_Local_Presn_Time_ns

Bits	Name	Memory Access	Description
31:0	MLPTN	R	MAC Local 1722 Presentation Time in ns These bits indicate the value of the 32-bit binary rollover equivalent time of the PTP Local System Time in ns. **Value After Reset:** 0x0 **Exists:** Always

11.1.399 MAC_Local_Presn_Time_Updt

- **Description:** This field holds the 32-bit value of MAC Local 1722 Presentation Time in ns, that should be added to the Current Presentation Time Counter value. Init happens when LTSINIT is set, and update happens when the LTSUPDT bit is set (LTSINIT and LTSINIT defined in MAC_Local_Timestamp_ Control register).
- **Size:** 32 bits
- **Offset:** 0xe34
- **Exists:** (DWCXG_MCGR&&DWCXG_DSTS)

Table 11-403 Fields for Register: MAC_Local_Presn_Time_Updt

Bits	Name	Memory Access	Description
31:0	MLPTU	R/W	MAC Local 1722 Presentation Time Update This field holds the init value or the update value for the local presentation time. When used for update, this field holds the 32-bit value in ns, that should be added to the Current Presentation Time Counter value. Init happens when LTSINIT is set, and update happens when the LTSUPDT bit is set (LTSINIT and LTSINIT defined in MAC_Local_Timestamp_Control register). When LADDSUB field of MAC_Local_System_Time_Nanoseconds_Update is set, this value is directly used for subtraction. **Value After Reset:** 0x0 **Exists:** Always

11.2 DWCXG_MTL Registers

11.2.1 MTL_Operation_Mode

- **Description:** The Operation Mode register establishes the Transmit and Receive operating modes and commands.
- **Size:** 32 bits
- **Offset:** 0x1000
- **Exists:** Always

31:16	15	x:y	6	14:7	6:5	4	3	2	1	0
Rsvd	FRPE	Reserved_31_y	Rsvd	Reserved_14_7	ETSALG	Reserved_4	EATSP	RAA	FTS	Reserved_0

Table 11-404 Fields for Register: MTL_Operation_Mode

Bits	Name	Memory Access	Description
31:16			**Reserved Field:** Yes
15	FRPE	R/W	Flexible Receive Parser Enable. When this bit is set, Flexible Receive Parser is enabled for packet filtering and classification. When this bit is reset, Flexible Receive Parser is disabled for packet filtering and classification. **Note:** If the flexible receive parser is in the middle of the parsing then, FRPE programming change is applicable from next immediate packet. **Values:** ■ 0x0 (DISABLE): Flexible Rx parser is disabled ■ 0x1 (ENABLE): Flexible Rx parser is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_FRP_EN
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_FRP_EN?16:25 + DWCXG_FRP_EN?16:7 - 1 **Range Variable[y]:** DWCXG_FRP_EN?16:7
6			**Reserved Field:** Yes

Table 11-404 Fields for Register: MTL_Operation_Mode (continued)

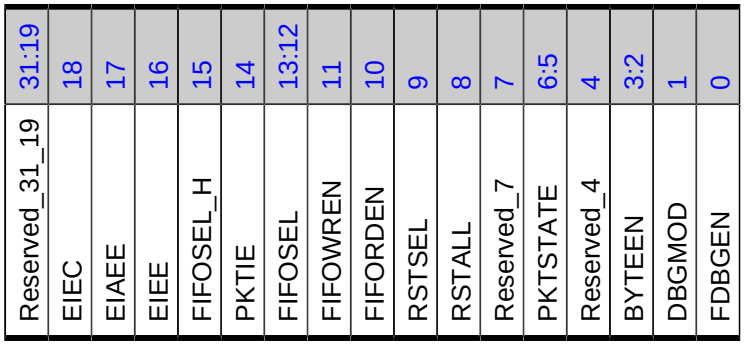
Bits	Name	Memory Access	Description
14:7	Reserved_14_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** DWCXG_FRP_EN
6:5	ETSALG	R/W	ETS Algorithm. This field selects the type of ETS algorithm to be applied for traffic classes whose transmission selection algorithm (TSA) is set to ETS: ■ 2'b00: WRR algorithm ■ 2'b01: WFQ algorithm when Data Center Bridging feature is selected. Otherwise, reserved ■ 2'b10: DWRR algorithm when Data Center Bridging feature is selected. Otherwise, reserved. ■ 2'b11: Reserved **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1)
4	Reserved_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	EATSP	R/W	Enhance ATS Priority. When this bit is set to 0, numerically highest TC has highest priority. When this bit is set to 1, ATS enabled TC is prioritized above ETS, SP. **Value After Reset:** 0x0 **Exists:** DWCXG_ATS
2	RAA	R/W	Receive Arbitration Algorithm. This field is used to select the arbitration algorithm for the Rx side. ■ 0: Strict Priority (SP) ■ 1: Weighted Strict Priority (WSP) Queue 0 has the lowest priority and the last queue has the highest priority. RXQ_WEGT field of MTL_RxQ(#i)_Control register controls the weight assigned to receive queue. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1)

Table 11-404 Fields for Register: MTL_Operation_Mode (continued)

Bits	Name	Memory Access	Description
1	FTS	R/W	Forward Transmit Status When this bit is set, the Tx packet status received from the MAC is forwarded by the MTL on to the ATI. When this bit is reset, the Tx packet status received from the MAC is dropped by the MTL. **Value After Reset:** 0x0 **Exists:** DWCXG_TXSTSE
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.2.2 MTL_Debug_Control

- **Description:** The MTL Debug Control register controls the operation mode of debug access to the TXFIFO, RXFIFO and Descriptor Cache memories connected to the DWC_25gmac.
  Note: Consecutive write or read to this register must be performed after at least 16 clock cycles of the slowest clock among MAC Transmitter/Receiver clock and Application/CSR clock.
- **Size:** 32 bits
- **Offset:** 0x1008
- **Exists:** (DWCXG_DBG_FIFO_ACCESS)



31:19	18	17	16	15	14	13:12	11	10	9	8	7	6:5	4	3:2	1	0
Reserved_31_19	EIEC	EIAEE	EIEE	FIFOSEL_H	PKTIE	FIFOSEL	FIFOWREN	FIFORDEN	RSTSEL	RSTALL	Reserved_7	PKTSTATE	Reserved_4	BYTEEN	DBGMOD	FDBGEN

Table 11-405 Fields for Register: MTL_Debug_Control

Bits	Name	Memory Access	Description
31:19	Reserved_31_19	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
18	EIEC	R/W	ECC Inject Error Control for Tx, Rx ,TSO, Descriptor Cache memories and AIS When EIEE bit of this register is set, errors are inserted in databus to memory based on the value encoded in this field. **Values:** ■ 0x0 (M_1BIT): Insert 1 bit error ■ 0x1 (M_2BIT): Insert 2 bit errors **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ECC
17	EIAEE	R/W	ECC Inject Address Error Enable for Tx, Rx ,TSO and Descriptor Cache memories When set, indicates that the ECC Debug error should be injected in Address Bits. When reset, indicates that the ECC Debug error should be injected in Data/Check Bits. **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ECC **Testable:** untestable

Table 11-405 Fields for Register: MTL_Debug_Control (continued)

Bits	Name	Memory Access	Description
16	EIEE	R/W	ECC Inject Error Enable for Tx, Rx ,TSO and Descriptor Cache memories When set, enables the ECC error injection function in Debug mode. If EIAEE is set, enables the ECC Address Error Injection, otherwise, enables the ECC Data/Check Bits Error Injection (See EIM and BLEI field in MTL_DPP_ECC_EIC Register). When reset, disables the ECC error injection function. **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ECC **Testable:** untestable
15	FIFOSEL_H	R/W	**Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50
14	PKTIE	R/W	Receive Packet Available Interrupt Status Enable. When this bit is set, an interrupt is generated when EOP of received packet is written by the MAC to the Rx FIFO. This bit is applicable only when Subordinate mode is active. **Value After Reset:** 0x0 **Exists:** Always
13:12	FIFOSEL	R/W	FIFO Selected for Access. This field indicates the FIFO selected for debug access: ■ 00: Tx FIFO ■ 01: Descriptor cache (only when DBGMOD = 1) ■ 10: TSO FIFO (only when DBGMOD = 1) ■ 11: Rx FIFO **Value After Reset:** 0x0 **Exists:** Always
11	FIFOWREN	R/W	FIFO Write Enable. When this bit is set, it enables the Write operation on the memory selected by FIFOSEL. If both FIFOWREN and FIFORDEN are set to 1 together, the behaviour is unpredictable. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-405 Fields for Register: MTL_Debug_Control (continued)

Bits	Name	Memory Access	Description
10	FIFORDEN	R/W	FIFO Read Enable. When this bit is set, it enables the Read operation on the memory selected by FIFOSEL. If both FIFOWREN and FIFORDEN are set to 1 together, the behaviour is unpredictable. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
9	RSTSEL	R/W	Reset Pointers of Selected FIFO. When this bit is set, the address pointer of the memory selected by FIFOSEL is reset in both Debug and Subordinate modes. This bit is self cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
8	RSTALL	R/W	Reset All Pointers. When this bit is set, the address pointers to all FIFO memory in both Debug and Subordinate modes are reset. This bit is self cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-405 Fields for Register: MTL_Debug_Control (continued)

Bits	Name	Memory Access	Description
6:5	PKTSTATE	R/W	Write Control Type. This field is used to decide the tag bits along with the data being written into the FIFO. When the TxFIFO is written in Subordinate mode or Debug mode, this field marks the type of data written as follows: ■ 00: Packet Data ■ 01: Control Word ■ 10: SOP Data ■ 11: EOP Data When the Rx FIFO is being written in Subordinate mode, this field marks the type of data written as follows: ■ 00: Packet Data ■ 01: Normal Status ■ 10: Last Status ■ 11: EOP **Value After Reset:** 0x0 **Exists:** Always
4	Reserved_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3:2	BYTEEN	R/W	Write Data Byte Enable. This field indicates the number of data bytes valid in the MTL_Debug_Data register during Write operation. This is valid only when PKTSTATE is 2'b11 (EOP) and Tx FIFO or Rx FIFO is selected. ■ 00: Byte 0 valid ■ 01: Byte 0 and Byte 1 are valid ■ 10: Byte 0, Byte 1, and Byte 2 are valid ■ 11: All four bytes are valid **Value After Reset:** 0x0 **Exists:** Always
1	DBGMOD	R/W	Debug Mode Access to FIFO. When this bit is set and FDBGEN is set, direct access is enabled to the memories selected by FIFOSEL. The transfer of packets between the host interface and the MAC through the TXFIFO & RXFIFO memory are disabled. The CPU/software can perform direct write/read operations to the memories as determined by programming of FIFOWREN/FIFORDEN bits.

Table 11-405 Fields for Register: MTL_Debug_Control (continued)

Bits	Name	Memory Access	Description
			When this bit is reset and FDBGEN is set, the TXFIFO and RXFIFO memory are put in Subordinate mode. In this mode, packets can be directly written into the TXFIFO memory for transmission by MAC as well as the packets received by the MAC can be read directly. The transfer of packets between the FIFO memories and the host interface is stopped. When this bit is reset, it indicates the Tx and Rx FIFO operate in subordinate mode. The software can directly write packets to Tx FIFO for transmission and read received packets from Rx FIFO. In this mode, only the following access are allowed: Packet Write access to the Tx FIFO Packet Read access to the Rx FIFO **Value After Reset:** 0x0 **Exists:** Always
0	FDBGEN	R/W	FIFO Debug Access Enable. When this bit is set, access is available to the memories selected by FIFOSEL. The type of access (Debug mode or Subordinate mode) is determined by DBGMOD. When this bit is reset, the memories are in mission mode and direct access to these memories are disabled.. **Value After Reset:** 0x0 **Exists:** Always

11.2.3 MTL_Debug_Status

- **Description:** The Debug Status register contains the state of the transactions to FIFO memory in Debug or Subordinate modes.
  Note: Consecutive write or read to this register must be performed after at least 16 clock cycles of the slowest clock among MAC Transmitter/Receiver clock and Application/CSR clock.
- **Size:** 32 bits
- **Offset:** 0x100c
- **Exists:** (DWCXG_DBG_FIFO_ACCESS)

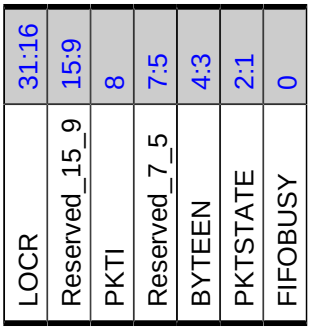


Table 11-406 Fields for Register: MTL_Debug_Status

Bits	Name	Memory Access	Description
31:16	LOCR	R	Remaining Locations in the FIFO. Subordinate Access Mode: This field indicates the space available in selected FIFO. Debug Access Mode: This field contains the Write or Read pointer value of the selected FIFO during Write or Read operation, respectively. **Value After Reset:** (DWCXG_NUM_TXQ==1)?(DWCXG_TXFIFO_SIZE/(DWCXG_DATAWIDTH/8)):\"0x0\" **Exists:** Always **Testable:** untestable
15:9	Reserved_15_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	PKTI	R/W	Receive Packet Available Interrupt Status. When set, this bit indicates that MAC layer has written the EOP of received packet to the Rx FIFO. This bit is reset when 1 is written to this bit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-406 Fields for Register: MTL_Debug_Status (continued)

Bits	Name	Memory Access	Description
7:5	Reserved_7_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4:3	BYTEEN	R	Read Data Byte Enable. This field indicates the number of data bytes valid in the MTL_FIFO_Debug_Data register during Read operation. This is valid only when PKTSTATE is 2'b11 (EOP). ■ 00: Byte 0 valid ■ 01: Byte 0 and Byte 1 are valid ■ 10: Byte 0, Byte 1, and Byte 2 are valid ■ 11: All four bytes are valid **Value After Reset:** 0x3 **Exists:** Always
2:1	PKTSTATE	R	Read Control Type. This field indicates the tag bits read along with the data being read from the selected FIFO. When the TxFIFO is read in Debug mode, this field marks the type of data read as follows: ■ 00: Packet Data ■ 01: Control Word ■ 10: SOP Data ■ 11: EOP Data When the RxFIFO is read in Debug or Subordinate mode, this field marks the type of data read as listed as follows: ■ 00: Packet Data ■ 01: Normal Status ■ 10: Last Status ■ 11: EOP **Value After Reset:** 0x0 **Exists:** Always
0	FIFOBUSY	R	FIFO Busy. When set, this bit indicates that a FIFO operation is in progress in the MAC and content of the following fields is not valid: ■ All other fields of this register ■ All fields of the MTL_FIFO_Debug_Data register **Value After Reset:** 0x0 **Exists:** Always

11.2.4 MTL_FIFO_Debug_Data

- **Description:** The FIFO Debug Data register contains the data to be written to or read from the FIFOs.
  Note: Consecutive write or read to this register must be performed after at least 16 clock cycles of the slowest clock among MAC Transmitter/Receiver clock and Application/CSR clock.
- **Size:** 32 bits
- **Offset:** 0x1010
- **Exists:** (DWCXG_DBG_FIFO_ACCESS)

Table 11-407 Fields for Register: MTL_FIFO_Debug_Data

Bits	Name	Memory Access	Description
31:0	FDBGDATA	R/W	FIFO Debug Data. During debug or subordinate access write operation, this field contains the data to be written to the Tx FIFO, Rx FIFO, or Descriptor Cache memories. During debug or subordinate access read operation, this field contains the data read from the Tx FIFO, Rx FIFO, or Descriptor Cache memories. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.5 MTL_Interrupt_Status

- **Description:** The software driver (application) reads this register during interrupt service routine or polling to determine the interrupt status of MTL queues and the MAC.
- **Size:** 32 bits
- **Offset:** 0x1020
- **Exists:** Always

31:24	23	22:21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved_31_24	MTLPIS	Reserved_22_21	SGFIS	TINS	ESTIS	DBGIS	MACIS	Q15IS	Q14IS	Q13IS	Q12IS	Q11IS	Q10IS	Q9IS	Q8IS	Q7IS	Q6IS	Q5IS	Q4IS	Q3IS	Q2IS	Q1IS	Q0IS

Table 11-408 Fields for Register: MTL_Interrupt_Status

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
23	MTLPIS	R	MTL Rx-Parser Interrupt Status This bit indicates that there is an interrupt from Rx Parser Block. To reset this bit, the application must read the MTL_RXP_Interrupt_Control_Status register to get the exact cause of the interrupt and clear its source. **Values:** ■ 0x0 (INACTIVE): MTL Rx Parser Interrupt status not activated. ■ 0x1 (ACTIVE): MTL Rx Parser Interrupt event activated. **Value After Reset:** 0x0 **Exists:** DWCXG_FRP_EN **Testable:** untestable
22:21	Reserved_22_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-408 Fields for Register: MTL_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
20	SGFIS	R	SGF Interrupt Status This bit indicates an interrupt event during the operation of SGF. To reset this bit, the application must clear the source of the error/event that has caused the interrupt. The status bits that are set in MTL_SGF_Status registers should be written to 1, to clear the SGFIS bit. **Values:** ■ 0x0 (INACTIVE): SGF Interrupt status not activated ■ 0x1 (ACTIVE): SGF Interrupt event activated **Value After Reset:** 0x0 **Exists:** DWCXG_SGF **Testable:** untestable
19	TINS	R	TBS Interrupt Status This bit is set when a frame is dropped in any of the TX queues in TBS mode due to the expiry of Launch time or transmission window. Access restriction applies. Clears on write of 1. **Values:** ■ 0x0 (INACTIVE): TBS Interrupt status not activated ■ 0x1 (ACTIVE): TBS Interrupt event activated **Value After Reset:** 0x0 **Exists:** DWCXG_TBS **Testable:** untestable
18	ESTIS	R	EST (TAS- IEEE 802.1Qbv) Interrupt Status This bit indicates an interrupt event during the operation of Time Aware Shaper for transmit traffic. To reset this bit, the application must clear the error/event that has caused the interrupt. The status bits that are set in MTL_EST_Status registers should be written to 1, to clear the ESTIS bit **Values:** ■ 0x0 (INACTIVE): EST (TAS- 802.1Qbv) Interrupt status not activated ■ 0x1 (ACTIVE): EST (TAS- 802.1Qbv) Interrupt event activated **Value After Reset:** 0x0 **Exists:** DWCXG_AV_EST **Testable:** untestable

Table 11-408 Fields for Register: MTL_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
17	DBGIS	R	Debug Interrupt Status. This bit indicates an interrupt event during the subordinate access. To reset this bit, the software must read the MTL_Debug_Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
16	MACIS	R	MAC Interrupt Status. This bit indicates an interrupt event in the MAC. To reset this bit to 1'b0, the software must read the corresponding MAC_Interrupt_Status in the MAC to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15	Q15IS	R	Queue 15 Interrupt Status. This bit indicates that there is an interrupt from Queue 15. To reset this bit, the software must read Queue 15 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>15||DWCXG_NUM_TXQ>15 **Testable:** untestable
14	Q14IS	R	Queue 14 Interrupt Status. This bit indicates that there is an interrupt from Queue 14. To reset this bit, the software must read Queue 14 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>14||DWCXG_NUM_TXQ>14 **Testable:** untestable

Table 11-408 Fields for Register: MTL_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
13	Q13IS	R	Queue 13 Interrupt Status. This bit indicates that there is an interrupt from Queue 13. To reset this bit, the software must read Queue 13 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>13||DWCXG_NUM_TXQ>13 **Testable:** untestable
12	Q12IS	R	Queue 12 Interrupt Status. This bit indicates that there is an interrupt from Queue 12. To reset this bit, the software must read Queue 12 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>12||DWCXG_NUM_TXQ>12 **Testable:** untestable
11	Q11IS	R	Queue 11 Interrupt Status. This bit indicates that there is an interrupt from Queue 11. To reset this bit, the software must read Queue 11 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>11||DWCXG_NUM_TXQ>11 **Testable:** untestable
10	Q10IS	R	Queue 10 Interrupt Status. This bit indicates that there is an interrupt from Queue 10. To reset this bit, the software must read Queue 10 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>10||DWCXG_NUM_TXQ>10 **Testable:** untestable

Table 11-408 Fields for Register: MTL_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
9	Q9IS	R	Queue 9 Interrupt Status. This bit indicates that there is an interrupt from Queue 9. To reset this bit, the software must read Queue 9 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>9||DWCXG_NUM_TXQ>9 **Testable:** untestable
8	Q8IS	R	Queue 8 Interrupt Status. This bit indicates that there is an interrupt from Queue 8. To reset this bit, the software must read Queue 8 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>8||DWCXG_NUM_TXQ>8 **Testable:** untestable
7	Q7IS	R	Traffic Class/Queue 7 Interrupt Status. This bit indicates that there is an interrupt from Traffic Class/Queue 7. To reset this bit, the software must read Traffic Class/Queue 7 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>7||DWCXG_NUM_TXQ>7 **Testable:** untestable
6	Q6IS	R	Traffic Class/Queue 6 Interrupt Status. This bit indicates that there is an interrupt from Traffic Class/Queue 6. To reset this bit, the software must read Traffic Class/Queue 6 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>6||DWCXG_NUM_TXQ>6 **Testable:** untestable

Table 11-408 Fields for Register: MTL_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
5	Q5IS	R	Traffic Class/Queue 5 Interrupt Status. This bit indicates that there is an interrupt from traffic Class/Queue 5. To reset this bit, the software must read traffic Class/Queue 5 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>5||DWCXG_NUM_TXQ>5 **Testable:** untestable
4	Q4IS	R	Traffic Class/Queue 4 Interrupt Status. This bit indicates that there is an interrupt from Traffic Class/Queue 4. To reset this bit, the software must read Traffic Class/Queue 4 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>4||DWCXG_NUM_TXQ>4 **Testable:** untestable
3	Q3IS	R	Traffic Class/Queue 3 Interrupt Status. This bit indicates that there is an interrupt from Traffic Class/Queue 3. To reset this bit, the software must read Traffic Class/Queue 3 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>3||DWCXG_NUM_TXQ>3 **Testable:** untestable
2	Q2IS	R	Traffic Class/Queue 2 Interrupt Status. This bit indicates that there is an interrupt from Traffic Class/Queue 2. To reset this bit, the software must read Traffic Class/Queue 2 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>2||DWCXG_NUM_TXQ>2 **Testable:** untestable

Table 11-408 Fields for Register: MTL_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
1	Q1IS	R	Traffic Class/Queue 1 Interrupt Status. This bit indicates that there is an interrupt from Traffic Class/Queue 1. To reset this bit, the software must read Traffic Class/Queue 1 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>1||DWCXG_NUM_TXQ>1 **Testable:** untestable
0	Q0IS	R	Traffic Class/Queue 0 Interrupt Status. This bit indicates that there is an interrupt from Traffic Class/Queue 0. To reset this bit, the software must read Traffic Class/Queue 0 Interrupt Control and Status register to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.6 MTL_RxQ_DMA_Map0

- **Description:** The Receive Queue and DMA Channel Mapping 0 register controls the static or dynamic mapping of Receive Queues 0-3 to Receive DMA Channels.
- **Size:** 32 bits
- **Offset:** 0x1030
- **Exists:** (DWCXG_NUM_DMA_RX_CH>1&&DWCXG_NUM_RXQ>(0*4))

31	30:y	x:24	23	22:y	x:16	15	14:y	x:8	7	6:y	x:0
Q3DDMACH	Reserved_30_y	Q3MDMACH	Q2DDMACH	Reserved_22_y	Q2MDMACH	Q1DDMACH	Reserved_14_y	Q1MDMACH	Q0DDMACH	Reserved_6_y	Q0MDMACH

Table 11-409 Fields for Register: MTL_RxQ_DMA_Map0

Bits	Name	Memory Access	Description
31	Q3DDMACH	R/W	Receive Queue 3 Enabled for Dynamic DMA Channel Selection When set, each packet received in Receive Queue 3 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 3 are routed to the DMA Channel programmed in the Q3MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>3)
30:y	Reserved_30_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 24

Table 11-409 Fields for Register: MTL_RxQ_DMA_Map0 (continued)

Bits	Name	Memory Access	Description
x:24	Q3MDMACH	R/W	Receive Queue 3 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 3 to the DMA channel. This field is valid when the Q3DDMACH field is reset. ■ 4'b0000: DMA Channel 0 ■ 4'b0001: DMA Channel 1 ■ 4'b0010: DMA Channel 2 ■ 4'b0011: DMA Channel 3 ■ 4'b0100: DMA Channel 4 ■ 4'b0101: DMA Channel 5 ■ 4'b0110: DMA Channel 6 ■ 4'b0111: DMA Channel 7 ■ 4'b1000: DMA Channel 8 ■ 4'b1001: DMA Channel 9 ■ 4'b1010: DMA Channel 10 ■ 4'b1011: DMA Channel 11 ■ 4'b1100: DMA Channel 12 ■ 4'b1101: DMA Channel 13 ■ 4'b1110: DMA Channel 14 ■ 4'b1111: DMA Channel 15 The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>3) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 23
23	Q2DDMACH	R/W	Receive Queue 2 Enabled for Dynamic MA Channel Selection When set, this bit indicates that each packet received in Receive Queue 2 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 2 are routed to the DMA Channel programmed in the Q2MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>2)

Table 11-409 Fields for Register: MTL_RxQ_DMA_Map0 (continued)

Bits	Name	Memory Access	Description
22:y	Reserved_22_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 16
x:16	Q2MDMACH	R/W	Receive Queue 2 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 2 to the DMA channel. This field is valid when the Q2DDMACH field is reset. The encoding is the same as Q3MDMACH of this register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>2) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 15
15	Q1DDMACH	R/W	Receive Queue 1 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 1 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 1 are routed to the DMA Channel programmed in the Q1MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1)
14:y	Reserved_14_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 8

Table 11-409 Fields for Register: MTL_RxQ_DMA_Map0 (continued)

Bits	Name	Memory Access	Description
x:8	Q1MDMACH	R/W	Receive Queue 1 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 1 to the DMA channel. This field is valid when the Q1DDMACH field is reset. The encoding is the same as Q3MDMACH of this register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 7
7	Q0DDMACH	R/W	Receive Queue 0 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 0 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 0 are routed to the DMA Channel programmed in the Q0MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>0)
6:y	Reserved_6_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW
x:0	Q0MDMACH	R/W	Receive Queue 0 Mapped to DMA Channel This field controls the routing of the packet received in Receive Queue 0 to the DMA channel. This field is valid when the Q0DDMACH field is reset. The encoding is the same as Q3MDMACH of this register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>0) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW - 1

11.2.7 MTL_RxQ_DMA_Map1

- **Description:** The Receive Queue and DMA Channel Mapping 1 register controls the static or dynamic mapping of Receive Queues 4-7 to Receive DMA Channels.
- **Size:** 32 bits
- **Offset:** 0x1034
- **Exists:** (DWCXG_NUM_DMA_RX_CH>1&&DWCXG_NUM_RXQ>(1*4))

31	30:y	x:24	23	22:y	x:16	15	14:y	x:8	7	6:y	x:0
Q7DDMACH	Reserved_30_y	Q7MDMACH	Q6DDMACH	Reserved_22_y	Q6MDMACH	Q5DDMACH	Reserved_14_y	Q5MDMACH	Q4DDMACH	Reserved_6_y	Q4MDMACH

Table 11-410 Fields for Register: MTL_RxQ_DMA_Map1

Bits	Name	Memory Access	Description
31	Q7DDMACH	R/W	Receive Queue 7 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 7 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 7 are routed to the DMA Channel programmed in the Q7MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>7)
30:y	Reserved_30_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 24

Table 11-410 Fields for Register: MTL_RxQ_DMA_Map1 (continued)

Bits	Name	Memory Access	Description
x:24	Q7MDMACH	R/W	Receive Queue 7 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 7 to the DMA channel. This field is valid when the Q7DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>7) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 23
23	Q6DDMACH	R/W	Receive Queue 6 Enabled for Dynamic MA Channel Selection When set, this bit indicates that each packet received in Receive Queue 6 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 6 are routed to the DMA Channel programmed in the Q6MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>6)
22:y	Reserved_22_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 16
x:16	Q6MDMACH	R/W	Receive Queue 6 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 6 to the DMA channel. This field is valid when the Q6DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>6) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 15

Table 11-410 Fields for Register: MTL_RxQ_DMA_Map1 (continued)

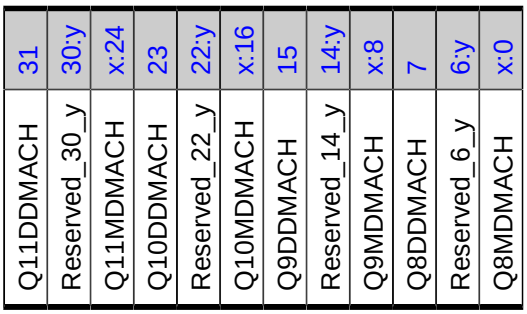
Bits	Name	Memory Access	Description
15	Q5DDMACH	R/W	Receive Queue 5 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 5 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 5 are routed to the DMA Channel programmed in the Q5MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>5)
14:y	Reserved_14_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 8
x:8	Q5MDMACH	R/W	Receive Queue 5 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 5 to the DMA channel. This field is valid when the Q5DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>5) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 7
7	Q4DDMACH	R/W	Receive Queue 4 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 4 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 4 are routed to the DMA Channel programmed in the Q4MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>4)

Table 11-410 Fields for Register: MTL_RxQ_DMA_Map1 (continued)

Bits	Name	Memory Access	Description
6:y	Reserved_6_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW
x:0	Q4MDMACH	R/W	Receive Queue 4 Mapped to DMA Channel This field controls the routing of the packet received in Receive Queue 4 to the DMA channel. This field is valid when the Q4DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>4) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW - 1

11.2.8 MTL_RxQ_DMA_Map2

- **Description:** The Receive Queue and DMA Channel Mapping 2 register controls the static or dynamic mapping of Receive Queues 8-11 to Receive DMA Channels.
- **Size:** 32 bits
- **Offset:** 0x1038
- **Exists:** (DWCXG_NUM_DMA_RX_CH>1&&DWCXG_NUM_RXQ>(2*4))



31	30:y	x:24	23	22:y	x:16	15	14:y	x:8	7	6:y	x:0
Q11DDMACH	Reserved_30_y	Q11MDMACH	Q10DDMACH	Reserved_22_y	Q10MDMACH	Q9DDMACH	Reserved_14_y	Q9MDMACH	Q8DDMACH	Reserved_6_y	Q8MDMACH

Table 11-411 Fields for Register: MTL_RxQ_DMA_Map2

Bits	Name	Memory Access	Description
31	Q11DDMACH	R/W	Receive Queue 11 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 11 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 11 are routed to the DMA Channel programmed in the Q11MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>11)
30:y	Reserved_30_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 24

Table 11-411 Fields for Register: MTL_RxQ_DMA_Map2 (continued)

Bits	Name	Memory Access	Description
x:24	Q11MDMACH	R/W	Receive Queue 11 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 11 to the DMA channel. This field is valid when the Q11DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>11) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 23
23	Q10DDMACH	R/W	Receive Queue 10 Enabled for Dynamic MA Channel Selection When set, this bit indicates that each packet received in Receive Queue 10 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 10 are routed to the DMA Channel programmed in the Q10MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>10)
22:y	Reserved_22_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 16
x:16	Q10MDMACH	R/W	Receive Queue 10 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 10 to the DMA channel. This field is valid when the Q10DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>10) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 15

Table 11-411 Fields for Register: MTL_RxQ_DMA_Map2 (continued)

Bits	Name	Memory Access	Description
15	Q9DDMACH	R/W	Receive Queue 9 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 9 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 9 are routed to the DMA Channel programmed in the Q9MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>9)
14:y	Reserved_14_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 8
x:8	Q9MDMACH	R/W	Receive Queue 9 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 9 to the DMA channel. This field is valid when the Q9DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>9) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 7
7	Q8DDMACH	R/W	Receive Queue 8 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 8 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 8 are routed to the DMA Channel programmed in the Q8MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>8)

Table 11-411 Fields for Register: MTL_RxQ_DMA_Map2 (continued)

Bits	Name	Memory Access	Description
6:y	Reserved_6_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW
x:0	Q8MDMACH	R/W	Receive Queue 8 Mapped to DMA Channel This field controls the routing of the packet received in Receive Queue 8 to the DMA channel. This field is valid when the Q8DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>8) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW - 1

11.2.9 MTL_RxQ_DMA_Map3

- **Description:** The Receive Queue and DMA Channel Mapping 3 register controls the static or dynamic mapping of Receive Queues 12-15 to Receive DMA Channels.
- **Size:** 32 bits
- **Offset:** 0x103c
- **Exists:** (DWCXG_NUM_DMA_RX_CH>1&&DWCXG_NUM_RXQ>(3*4))

31	30:y	x:24	23	22:y	x:16	15	14:y	x:8	7	6:y	x:0
Q15DDMACH	Reserved_30_y	Q15MDMACH	Q14DDMACH	Reserved_22_y	Q14MDMACH	Q13DDMACH	Reserved_14_y	Q13MDMACH	Q12DDMACH	Reserved_6_y	Q12MDMACH

Table 11-412 Fields for Register: MTL_RxQ_DMA_Map3

Bits	Name	Memory Access	Description
31	Q15DDMACH	R/W	Receive Queue 15 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 15 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 15 are routed to the DMA Channel programmed in the Q15MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>15)
30:y	Reserved_30_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 24

Table 11-412 Fields for Register: MTL_RxQ_DMA_Map3 (continued)

Bits	Name	Memory Access	Description
x:24	Q15MDMACH	R/W	Receive Queue 15 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 15 to the DMA channel. This field is valid when the Q15DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>15) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 23
23	Q14DDMACH	R/W	Receive Queue 14 Enabled for Dynamic MA Channel Selection When set, this bit indicates that each packet received in Receive Queue 14 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 14 are routed to the DMA Channel programmed in the Q14MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>14)
22:y	Reserved_22_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 16
x:16	Q14MDMACH	R/W	Receive Queue 14 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 14 to the DMA channel. This field is valid when the Q14DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>14) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 15

Table 11-412 Fields for Register: MTL_RxQ_DMA_Map3 (continued)

Bits	Name	Memory Access	Description
15	Q13DDMACH	R/W	Receive Queue 13 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 13 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address registers or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 13 are routed to the DMA Channel programmed in the Q13MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>13)
14:y	Reserved_14_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW + 8
x:8	Q13MDMACH	R/W	Receive Queue 13 Mapped to DMA Channel This field controls the routing of the received packet in Receive Queue 13 to the DMA channel. This field is valid when the Q13DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>13) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW + 7
7	Q12DDMACH	R/W	Receive Queue 12 Enabled for Dynamic DMA Channel Selection When set, this bit indicates that each packet received in Receive Queue 12 is routed to a DMA channel as decided in the MAC Receiver based on the DMA channel number programmed in the L3-L4 filter registers, RSS lookup table, the Ethernet DA address or VLAN filter registers. When reset, this bit indicates that all packets received in Receive Queue 12 are routed to the DMA Channel programmed in the Q12MDMACH field. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>12)

Table 11-412 Fields for Register: MTL_RxQ_DMA_Map3 (continued)

Bits	Name	Memory Access	Description
6:y	Reserved_6_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_DMA_RX_CHW
x:0	Q12MDMACH	R/W	Receive Queue 12 Mapped to DMA Channel This field controls the routing of the packet received in Receive Queue 12 to the DMA channel. This field is valid when the Q12DDMACH field is reset. The encoding is the same as Q3MDMACH of MTL_RxQ_DMA_Map0 register. The width of this field depends on the number of Receive DMA channels, therefore all values might not be valid in certain configurations. For example, if the number of Rx DMA channels selected is 2, the only valid values are 0 and 1. Other values are reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>12) **Range Variable[x]:** DWCXG_NUM_DMA_RX_CHW - 1

11.2.10 MTL_TC_Prty_Map0

- **Description:** This register contains the priority values assigned to traffic classes 0 to 3.
- **Size:** 32 bits
- **Offset:** 0x1040
- **Exists:** (DWCXG_PFC_EN)

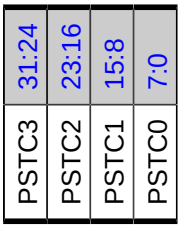


Table 11-413 Fields for Register: MTL_TC_Prty_Map0

Bits	Name	Memory Access	Description
31:24	PSTC3	R/W	Priorities Mapped to Traffic Class 3. This field is similar to the PSTC0. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>3)
23:16	PSTC2	R/W	Priorities Mapped to Traffic Class 2. This field is similar to the PSTC0. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>2)
15:8	PSTC1	R/W	Priorities Mapped to Traffic Class 1. This field is similar to the PSTC0. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1)
7:0	PSTC0	R/W	Priorities Mapped to Traffic Class 0. This field holds the priorities assigned to traffic class 0, programmed by the application. This field determines if the transmit queues associated with the traffic class 0 must be blocked from transmitting for the specified pause time when a PFC packet is received with priorities matching the priorities programmed in this field. Each bit corresponds to a priority. For example, PSTC0[7] corresponds to priority 7 and so on. If the content of this field is not mutually exclusive to the corresponding fields of other traffic classes, that is, if the same priority is mapped to multiple traffic classes, DWC_25gmac blocks all queues with matching priority for a specified time. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>0)

11.2.11 MTL_TC_Prty_Map1

- **Description:** This register contains the priority values assigned to traffic classes 4 to 7.
- **Size:** 32 bits
- **Offset:** 0x1044
- **Exists:** (DWCXG_PFC_EN&&DWCXG_NUM_TC>4)

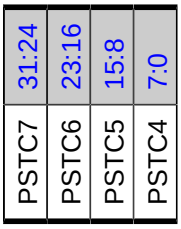


Table 11-414 Fields for Register: MTL_TC_Prty_Map1

Bits	Name	Memory Access	Description
31:24	PSTC7	R/W	Priorities Mapped to Traffic Class 7. This field is similar to the PSTC0 field of the MTL_TC_Prty_Map0 register. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>7)
23:16	PSTC6	R/W	Priorities Mapped to Traffic Class 6. This field is similar to the PSTC0 field of the MTL_TC_Prty_Map0 register. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>6)
15:8	PSTC5	R/W	Priorities Mapped to Traffic Class 5. This field is similar to the PSTC0 field of the MTL_TC_Prty_Map0 register. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>5)
7:0	PSTC4	R/W	Priorities Mapped to Traffic Class 4. This field is similar to the PSTC0 field of the MTL_TC_Prty_Map0 register. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>4)

11.2.12 MTL_TBS_CTRL

- **Description:** This register controls the operation of Time Based Scheduling.
- **Size:** 32 bits
- **Offset:** 0x1048
- **Exists:** (DWCXG_TBS)

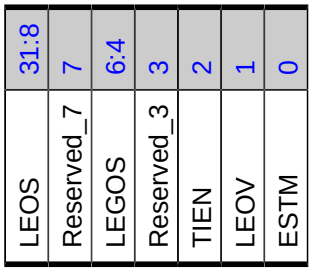


Table 11-415 Fields for Register: MTL_TBS_CTRL

Bits	Name	Memory Access	Description
31:8	LEOS	R/W	Launch Expiry Offset The value in units of 256 nanoseconds that has to be added to the Launch time to compute the Launch Expiry time. A value of 0 means 256ns, 1 means 512ns and so on. Value valid only when LEOV is set. Max value: 999,999,999 ns, additionally must be smaller than CTR-1 value when ESTM mode is set, as this value is a modulo CTR value. **Value After Reset:** 0x0 **Exists:** Always
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	LEGOS	R/W	Launch Expiry GSN Offset The number of GSN slots that has to be added to the Launch GSN to compute the Launch Expiry time. Value valid only when LEOV is set. **Value After Reset:** 0x0 **Exists:** DWCXG_AV_EST
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-415 Fields for Register: MTL_TBS_CTRL (continued)

Bits	Name	Memory Access	Description
2	TIEN	R/W	TBS Interrupt Enable When set indicates that TBS interrupt (occurs due to frame drop) is enabled. Value valid only when TIEN is set. **Values:** ■ 0x0 (INVALID): TBS interrupt is enabled ■ 0x1 (VALID): TBS interrupt is disabled **Value After Reset:** 0x0 **Exists:** Always
1	LEOV	R/W	Launch Expiry Offset Valid When set indicates the LEOS field is valid. When not set, indicates the Launch Expiry Offset is not valid and the MTL must not check for Launch expiry time. **Values:** ■ 0x0 (INVALID): LEOS field is invalid ■ 0x1 (VALID): LEOS field is valid **Value After Reset:** 0x0 **Exists:** Always
0	ESTM	R/W	EST offset Mode When this bit is set, the Launch Time value used in Time Based Scheduling is interpreted as an EST offset value and is added to the Base Time Register (BTR) of the current list. When reset, the Launch Time value is used as an absolute value that must be compared with the System time [39:8]. **Values:** ■ 0x0 (DISABLE): EST offset Mode is disabled ■ 0x1 (ENABLE): EST offset Mode is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_AV_EST

11.2.13 MTL_TBS_STATS

- **Description:** This register provides the One Hot encoded Queue numbers that have the frame drop related error
- **Size:** 32 bits
- **Offset:** 0x104c
- **Exists:** (DWCXG_TBS)

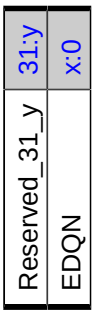


Table 11-416 Fields for Register: MTL_TBS_STATS

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_TXQ
x:0	EDQN	R/W	Expiry drop Queue number EDQN can have multiple bits set corresponding to the Queue numbers that have experienced error described in TINS field of MTL_Status register. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** DWCXG_NUM_TXQ - 1

11.2.14 MTL_EST_Control

- **Description:** This register controls the operation of Time Aware Shaper conforming to Enhancements to Scheduled Transmission (IEEE 802.1Qbv) standard.
- **Size:** 32 bits
- **Offset:** 0x1050
- **Exists:** (DWCXG_AV_EST)

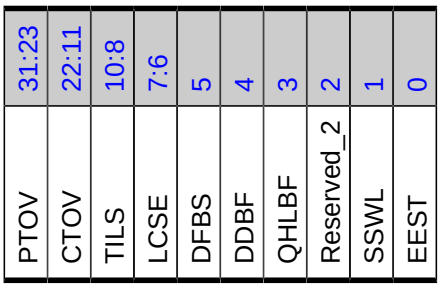


Table 11-417 Fields for Register: MTL_EST_Control

Bits	Name	Memory Access	Description
31:23	PTOV	R/W	PTP Time Offset Value The value of PTP Clock period multiplied by 9 in nanoseconds. This value is needed to avoid transmission overruns at the beginning of the installation of a new GCL. **Value After Reset:** 0x0 **Exists:** Always
22:11	CTOV	R/W	Current Time Offset Value Provides a 12 bit time offset value in nanosecond that is added to the current time to compensate for all the implementation pipeline delays such as the CDC sync delay, buffering delays, data path delays, and so on. This offset helps to ensure that the impact of gate controls is visible on the line exactly at the pre-determined schedule (or as close to the schedule as possible). **Value After Reset:** 0x0 **Exists:** Always

Table 11-417 Fields for Register: MTL_EST_Control (continued)

Bits	Name	Memory Access	Description
10:8	TILS	R/W	Time Interval Left Shift Amount This field provides the left shift amount for the programmed Time Interval values used in the Gate Control Lists. ■ 000: No left shift needed (equal to x1ns) ■ 001: Left shift TI by 1 bit (equal to x2ns) ■ 010: Left shift TI by 2 bits (equal to x4ns) ■ ... ■ 111: Left shift TI by 7 bits (equal to x128ns) Based on the configuration, one or more bits of this field must be treated as Reserved/Read-Only. **Value After Reset:** 0x0 **Exists:** Always
7:6	LCSE	R/W	Loop Count to report Scheduling Error Programmable number of GCL list iterations before reporting an HLBS error defined in EST_Status register. **Values:** ■ 0x0 (M_4_ITERNS): 4 iterations ■ 0x1 (M_8_ITERNS): 8 iterations ■ 0x2 (M_16_ITERNS): 16 iterations ■ 0x3 (M_32_ITERNS): 32 iterations **Value After Reset:** 0x0 **Exists:** Always
5	DFBS	R/W	Drop Frames causing Scheduling Error When this bit is set, frames that do not get scheduled for transmission after the number of GCL iterations as set in LCSE, are dropped. When this bit is reset, frames are not dropped due to any TAS scheduling conflicts or delays (HLBS). This can create HOL (head-of-line) blocking in the transmit queue. **Values:** ■ 0x0 (DONT_DROP): Do not Drop Frames causing Scheduling Error ■ 0x1 (DROP): Drop Frames causing Scheduling Error **Value After Reset:** 0x0 **Exists:** Always

Table 11-417 Fields for Register: MTL_EST_Control (continued)

Bits	Name	Memory Access	Description
4	DDBF	R/W	Do not Drop frames due to HLBF error When this bit is set, frames that are not scheduled for transmission because its size is larger than the Gate Open time window, are not dropped. This can result in head-of-line blocking in the corresponding TX Queue. When this bit is reset, frames that do not get scheduled for transmission due to HLBF error after the completion of the number of GCL Table iterations as set in QHLBF. **Values:** ■ 0x0 (DROP): Drop frames due to HLBF error ■ 0x1 (DONT_DROP): Do not Drop frames due to HLBF error **Value After Reset:** 0x0 **Exists:** Always
3	QHLBF	R/W	Quick Assertion of HLBF Error When set, HLBF error is reported after the packet is not transmitted for 1 to 2 iterations of GCL list. When this bit is reset, the HLBF error is reported after the packet is not transmitted for 2 to 3 iterations of GCL list. **Values:** ■ 0x0 (DISABLE): Disable Quick assertion of HLBF error ■ 0x1 (ENABLE): Quick Assertion of HLBF Error **Value After Reset:** 0x0 **Exists:** Always
2	Reserved_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	SSWL	R/W	Switch to software owned list When this bit is set, the DWC_25gmac is instructed to switch to the new GCL list (SWOL) programmed in the GCL memory, at the start of new BTR. Hardware clears this bit when the switch to the SWOL is completed or an non-recoverable BTR error (BTRL = "11111111" indicates the iterations exceeded the value of 512 and DWC_25gmac was not able to update New BTR to be equal to or greater than Current Time.) is set. When BTRE is set this bit is cleared but SWOL is not updated as the switch was not successful. The write to this bit is effective only when EEST bit is set to 1 along with it or EEST bit is previously set to 1. Access restriction applies. Writing 1 sets the bit. Self-cleared. Writing 0 has no effect.

Table 11-417 Fields for Register: MTL_EST_Control (continued)

Bits	Name	Memory Access	Description
			Values: ■ 0x0 (DISABLE): Switch to S/W owned list is disabled ■ 0x1 (ENABLE): Switch to S/W owned list is enabled **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	EEST	R/W	Enable EST When this bit is set, the Time Aware Shaper algorithm gets activated in the Transmit scheduler. During the toggle from 0 to 1, the gate control list processing starts only after the SSWL bit is set. When this bit is reset, the EST function is disabled and all gates of the GCL table are considered to be in Open state. When ECC is enabled for GCL memory access: If an uncorrectable error is detected in the GCL memory, DWC_25gmac resets this bit and disables the EST function. **Values:** ■ 0x0 (DISABLE): EST is disabled ■ 0x1 (ENABLE): EST is enabled **Value After Reset:** 0x0 **Exists:** Always

11.2.15 MTL_EST_Overhead

- **Description:** This register indicates the value of Overhead in bytes to be considered for EST Scheduling.
- **Size:** 32 bits
- **Offset:** 0x1054
- **Exists:** (DWCXG_AV_EST)

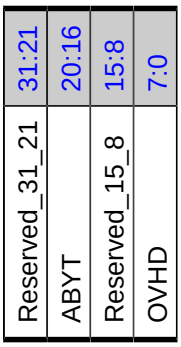


Table 11-418 Fields for Register: MTL_EST_Overhead

Bits	Name	Memory Access	Description
31:21	Reserved_31_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
20:16	ABYT	R/W	Bytes transmitted in 1 application clock This field should be programmed with the number of bytes transmitted on XGMII/GMII interface in the time window corresponding to 1 application clock period. For Example: if Time period of Application clock is 6.4 ns and speed mode is XGMII 10G, the programmed value must be, ((6.4)/(3.2)* (Number of Bytes in 1 Tx clock =4) = 8. (for values greater 0.5, round it off to 1) **Value After Reset:** 0x0 **Exists:** DWCXG_TX_SDPR||DWCXG_SPRAM
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-418 Fields for Register: MTL_EST_Overhead (continued)

Bits	Name	Memory Access	Description
7:0	OVHD	R/W	Overhead Bytes Value This field should be programmed with the delay in bytes between the point at which a packet is selected for transmission by the EST scheduler and time at which the Start of Packet appears on the XGMII/GMII TX interface. This value differs based on the operating speed and configuration. The value to be programmed is provided in Table "Overheads to be Accounted in the EST Overhead Field". **Value After Reset:** 0x0 **Exists:** Always

11.2.16 MTL_EST_Status

- **Description:** This register reports the events related to Enhancements to Scheduled Transmission (IEEE 802.1Qbv).
- **Size:** 32 bits
- **Offset:** 0x1058
- **Exists:** (DWCXG_AV_EST)

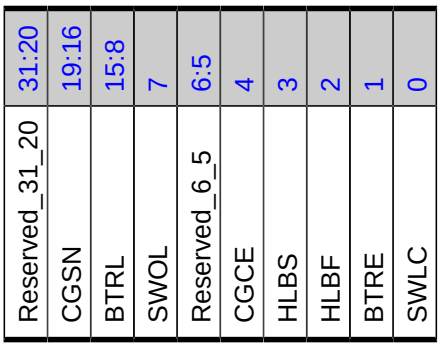


Table 11-419 Fields for Register: MTL_EST_Status

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19:16	CGSN	R	Current GCL Slot Number. Indicates the slot number of the GCL list. Slot number is a modulo 16 count of the GCL List loops executed so far. Even if a new GCL list is installed, the count is incremental. **Value After Reset:** 0x0 **Exists:** DWCXG_TBS
15:8	BTRL	R	BTR Loop Count This field provides the number of GCL cycle times after which the switch to new SWOL GCL occurs after SSWL is set. It is determined by the value N which satisfies the equation "Current Time is less than or equal to New BTR + (N * 4 * New Cycle Time)". N = "11111111" indicates that the iterations exceeded the value of 512 and DWC_25gmac is unable to update New BTR to be equal to or greater than Current Time. Software intervention is needed to update the New BTR. This field is cleared when BTRE field of this register is cleared. **Value After Reset:** 0x0 **Exists:** Always

Table 11-419 Fields for Register: MTL_EST_Status (continued)

Bits	Name	Memory Access	Description
7	SWOL	R	Software Owned List When this field is 0, indicates that the Gate control list number 0 is owned by software. When this field is 1, indicates that the Gate Control list 1 is owned by the software. Any reads/writes by the software (using indirect access through GCL_Control) is directed to the list indicated by this value. The inverse of this value is treated as HWOL. R/W operations performed by DWC_25gmac are directed to the list pointed by HWOL. Values: ■ 0x0: Gate control list number "0" is owned by software ■ 0x1: Gate control list number "1" is owned by software **Value After Reset:** 0x0 **Exists:** Always
6:5	Reserved_6_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	CGCE	R/W	Constant Gate Control Error This error occurs when the programmed Time Interval (TI) value after the optional Left Shifting is more than or equal to the Cycle Time (CTR). Programming a GCL row's time interval TI greater than CTR is equivalent of always closing or opening of certain Transmit queues which can be done by disabling or enabling the required open or closed queues without enabling EST. Due to this reason, such a programming is treated as Error. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. Writing 0 has no effect. **Values:** ■ 0x0 (INACTIVE): Constant Gate Control Error not detected ■ 0x1 (ACTIVE): Constant Gate Control Error detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
3	HLBS	R	Head-Of-Line Blocking due to Scheduling When set, it indicates that a packet is not able to get scheduled by EST even after LCSE number of GCL iterations has elapsed. Indicates to software a potential programming error. The one-hot encoded values of the Queue Numbers that have such packets blocking the queue, are indicated in the MTL_EST_Sch_Error register. Bit cleared when MTL_EST_Sch_Error register is all zeros.

Table 11-419 Fields for Register: MTL_EST_Status (continued)

Bits	Name	Memory Access	Description
			Note: A single fixed reference or tic is generated at integral multiple of MTL_EST_Control.LCSE loops/iterations. The HLBS is generated after (MTL_EST_Control.LCSE + N) loops/iterations, where N is non-integral number of MTL_EST_Control.LCSE loops/iterations the packet could not be scheduled before the last 2 tics. **Values:** ■ 0x0 (INACTIVE): Head-Of-Line Blocking due to Scheduling not detected ■ 0x1 (ACTIVE): Head-Of-Line Blocking due to Scheduling detected **Value After Reset:** 0x0 **Exists:** Always
2	HLBF	R	Head-Of-Line Blocking due to Frame Size This field is set when HOL Blocking is noticed on one or more Queues when none of the Time Intervals of open-gates in the GCL is greater than or equal to the duration needed for frame (or frame fragment size when preemption is enabled) transmission. The one-hot encoded Queue numbers that are experiencing HLBF are indicated in the MTL_EST_Frm_Size_Error register. Additionally, the first Queue number that experienced HLBF along with the frame size is captured in MTL_EST_Frm_Size_Capture register. Bit cleared when MTL_EST_Frame_Size_Error register is all zeros. **Values:** ■ 0x0 (INACTIVE): Head-Of-Line Blocking due to Frame Size not detected ■ 0x1 (ACTIVE): Head-Of-Line Blocking due to Frame Size detected **Value After Reset:** 0x0 **Exists:** Always

Table 11-419 Fields for Register: MTL_EST_Status (continued)

Bits	Name	Memory Access	Description
1	BTRE	R/W	BTR Error When this field is set, indicates a programming error in the BTR of SWOL where the programmed value is less than current time. If the BTRL = "11111111", SWOL is not updated and software must reprogram the BTR to a value greater than current time and set the SSWL bit to re-initiate the switch to SWOL. Otherwise, if the value of BTRL < "11111111", SWOL is updated and this field indicates the number of iterations (of + CycleTime) taken by DWC_25gmac to update the BTR to a value greater than Current Time. Access restriction applies. Self-set to 1 on internal event. Self clear or clears on write of 1. Writing 0 has no effect. **Values:** ■ 0x0 (INACTIVE): BTR Error not detected ■ 0x1 (ACTIVE): BTR Error detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	SWLC	R/W	Switch to Software Owned List Complete When this field is set, indicates that DWC_25gmac has successfully switched to the SWOL, and the SWOL bit is updated to that effect. This field is cleared when the SSWL field of EST_Control register transitions from 0 to 1, or when a 1 is written to this bit. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. Writing 0 has no effect. **Values:** ■ 0x0 (INACTIVE): Switch to S/W owned list Complete not detected ■ 0x1 (ACTIVE): Switch to S/W owned list Complete detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.17 MTL_EST_Sch_Error

- **Description:** This register provides the Traffic Class numbers that have the GCL scheduling related error.
- **Size:** 32 bits
- **Offset:** 0x1060
- **Exists:** (DWCXG_AV_EST)

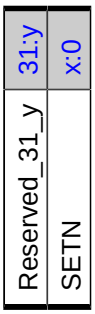


Table 11-420 Fields for Register: MTL_EST_Sch_Error

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_TC
x:0	SETN	R/W	Schedule Error TC Number SETN can have multiple bits set corresponding to the TC numbers that have experienced error described in HLBS field of status register. Bit 0 corresponds to TC0, Bit 1 corresponds to TC1, and so on. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. Writing 0 has no effect. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** DWCXG_NUM_TC - 1

11.2.18 MTL_EST_Frm_Size_Error

- **Description:** This register provides the Traffic Class numbers that have the Frame Size related scheduling error.
- **Size:** 32 bits
- **Offset:** 0x1064
- **Exists:** (DWCXG_AV_EST)

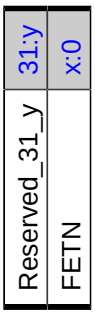


Table 11-421 Fields for Register: MTL_EST_Frm_Size_Error

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_NUM_TC
x:0	FETN	R/W	Frame Size Error TC Number FETN can have multiple bits set corresponding to the TC numbers that have experienced error described in HLBF field of status register. Bit 0 corresponds to TC0, Bit 1 corresponds to TC1, and so on. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. Writing 0 has no effect. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** DWCXG_NUM_TC - 1

11.2.19 MTL_EST_Frm_Size_Capture

- **Description:** This register captures the Frame Size and Traffic Class Number of the first occurrence of the Frame Size related error. Upon clearing, this register captures the data of the next occurrence of a similar error.
- **Size:** 32 bits
- **Offset:** 0x1068
- **Exists:** (DWCXG_AV_EST)

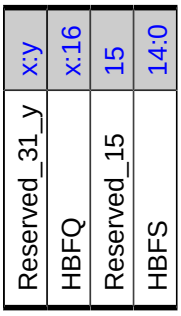


Table 11-422 Fields for Register: MTL_EST_Frm_Size_Capture

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_TC>4?13:DWCXG_NUM_TC>2?14:15 + DWCXG_NUM_TC>4?19:DWCXG_NUM_TC>2?18:17 - 1 **Range Variable[y]:** DWCXG_NUM_TC>4?19:DWCXG_NUM_TC>2?18:17
x:16	HBFQ	R	Transmit Queue Number of HLBF Captures the binary value of the first Transmit Queue experiencing HLBF error (see HLBF field of MTL_EST_status register). Value once written is not altered by any subsequent queue errors of similar nature. Once cleared, the queue number of the next occurring HLBF error is captured. Width is based on the number of Tx Queues configured; remaining bits are Read-Only. Cleared when MTL_EST_Frm_Size_Error register is all zeros. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_TC>4?3:DWCXG_NUM_TC>2??2:1 + 15
15	Reserved_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-422 Fields for Register: MTL_EST_Frm_Size_Capture (continued)

Bits	Name	Memory Access	Description
14:0	HBFS	R	Frame Size of HLBF Captures the Frame Size of the frame present in the queue number indicated by HBFQ field of this register. Contents of this register must be considered invalid, if this field is zero. Cleared when MTL_EST_Frm_Size_Error register is all zeros. **Value After Reset:** 0x0 **Exists:** Always

11.2.20 MTL_EST_Intr_Enable

- **Description:** This register implements the Interrupt Enable bits for the various events in the EST scheduler. Bit positions have a 1- to-1 correspondence with the status bit positions in MTL_EST_Status register.
- **Size:** 32 bits
- **Offset:** 0x1070
- **Exists:** (DWCXG_AV_EST)

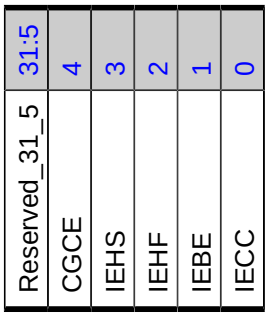


Table 11-423 Fields for Register: MTL_EST_Intr_Enable

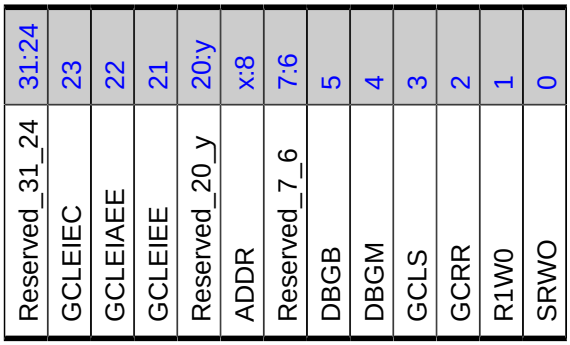
Bits	Name	Memory Access	Description
31:5	Reserved_31_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	CGCE	R/W	Interrupt Enable for CGCE When set, generates interrupt when the Constant Gate Control Error occurs and is indicated in the status. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Interrupt for CGCE is disabled ■ 0x1 (ENABLE): Interrupt for CGCE is enabled **Value After Reset:** 0x0 **Exists:** Always
3	IEHS	R/W	Interrupt Enable for HLBS When set, generates interrupt when HLBS status bit is set. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Interrupt for HLBS is disabled ■ 0x1 (ENABLE): Interrupt for HLBS is enabled **Value After Reset:** 0x0 **Exists:** Always

Table 11-423 Fields for Register: MTL_EST_Intr_Enable (continued)

Bits	Name	Memory Access	Description
2	IEHF	R/W	Interrupt Enable for HLBF When set, generates interrupt when HLBS status bit is set. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Interrupt for HLBF is disabled ■ 0x1 (ENABLE): Interrupt for HLBF is enabled **Value After Reset:** 0x0 **Exists:** Always
1	IEBE	R/W	Interrupt Enable for BTR Error When set, generates interrupt when the BTR Error occurs and is indicated in the status. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Interrupt for BTR Error is disabled ■ 0x1 (ENABLE): Interrupt for BTR Error is enabled **Value After Reset:** 0x0 **Exists:** Always
0	IECC	R/W	Interrupt Enable for Switch List When set, generates interrupt when the GCL Table switch is successful. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Interrupt for Switch List is disabled ■ 0x1 (ENABLE): Interrupt for Switch List is enabled **Value After Reset:** 0x0 **Exists:** Always

11.2.21 MTL_GCL_Control

- **Description:** This register provides the control for reading/writing to the Gate Control lists.
- **Size:** 32 bits
- **Offset:** 0x1080
- **Exists:** (DWCXG_AV_EST||DWCXG_SGF)



31:24	23	22	21	20:y	x:8	7:6	5	4	3	2	1	0
Reserved_31_24	GCLEIEC	GCLEIAEE	GCLEIEE	Reserved_20_y	ADDR	Reserved_7_6	DBGB	DBGM	GCLS	GCRR	R1W0	SRWO

Table 11-424 Fields for Register: MTL_GCL_Control

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23	GCLEIEC	* Varies	ECC Inject Error Control for GCL Memory When GCLEIEE bit of this register is set, ECC error is injected based on the value encoded in this field. **Values:** ■ 0x0 (M_1BIT): Insert 1 bit error ■ 0x1 (M_2BIT): Insert 2 bit errors **Value After Reset:** 0x0 **Exists:** Always **Memory Access:** ((DWCXG_ASP_ECC)?\"read-write\":\"read-only\")
22	GCLEIAEE	* Varies	GCL ECC Inject Address Error When this bit is set, error injection (when triggered by GCLEIEE) is done in Address bus to GCL memory. When this bit is reset, error is injected in the data bus to GCL memory. **Value After Reset:** 0x0 **Exists:** Always **Memory Access:** ((DWCXG_ASP_ECC)?\"read-write\":\"read-only\")

Table 11-424 Fields for Register: MTL_GCL_Control (continued)

Bits	Name	Memory Access	Description
21	GCLEIEE	* Varies	GCL ECC Inject Error Enable When set along with EEST bit of MTL_EST_Control register, enables the ECC error injection feature. If GCLEIAEE is set, enables the ECC Address Error Injection, otherwise, error is injected in address bus, otherwise error is injected in Data/ECC bits to the GCL memory. When reset, disables the ECC error injection feature. **Values:** ■ 0x0 (DISABLE): GCL ECC Inject Error is disabled ■ 0x1 (ENABLE): GCL ECC Inject Error is enabled **Value After Reset:** 0x0 **Exists:** Always **Memory Access:** ((DWCXG_ASP_ECC)?\"read-write\":\"read-only\")
20:y	Reserved_20_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_IMAW + 8
x:8	ADDR	R/W	Gate Control List Address: (GCLA when GCRR is 0). Provides the address (row number) of the Gate Control List at which the R/W operation has to be performed. By default the Gate Control List pointed by SWOL of MTL_EST_Status is selected for R/W. However, if the DBGM bit of this register is set, a debug mode access is given to R/W from DBGB. The width of this field is dependent on GCL Depth selected; unused bits must be treated as read-only. Gate Control list Related Registers Address: (GCRA when GCRR is 1). By default, the GCL related register set pointed by SWOL of MTL_EST_Status is selected for R/W. However, if the DBGM bit of this register is set, a debug mode access is given to R/W from DBGB. Only the lower 3 bits are used in this mode. Higher order bits are treated as don't care values. ■ 000: BTR Low (31:0) ■ 001: BTR High (63:31) ■ 010: CTR Low (31:0) ■ 011: CTR High (39:32) ■ 100: TER (m:0)(m=GCL Width selected+7-1) ■ 101: LLR (n:0) (where n is (log{GCL Depth selected} / log2)) ■ Others: Reserved

Table 11-424 Fields for Register: MTL_GCL_Control (continued)

Bits	Name	Memory Access	Description
			Range Variable[x]:GCL Memory Address Width of One logical partition + 7 **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_IMAW + 7
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5	DBGB	R/W	Debug Mode Bank Select When this field is set to 0, indicates R/W in debug mode must be directed to Bank 0 (GCL0 and corresponding Time related registers). When set to 1, indicates R/W in debug mode must be directed to Bank 1 (GCL1 and corresponding Time related registers). This value is used when DBGM is set and overrides the value of SWOL, which is normally used. **Values:** ■ 0x0 (BANK0): R/W in debug mode should be directed to Bank 0 ■ 0x1 (BANK1): R/W in debug mode should be directed to Bank 1 **Value After Reset:** 0x0 **Exists:** Always
4	DBGM	R/W	Debug Mode When this field is set to 1, indicates R/W in debug mode where the memory bank (for GCL and Time related registers) is explicitly provided by DBGB value. When set to 0, SWOL bit is used to determine which bank to access. **Values:** ■ 0x0 (DISABLE): Debug Mode is disabled ■ 0x1 (ENABLE): Debug Mode is enabled **Value After Reset:** 0x0 **Exists:** Always
3	GCLS	R/W	Gate Control/Address Select When set to "1" indicates the SGF R/W access for the GCL related registers/GCL memory. When set to "0" indicates the EST R/W access for the GCL related registers/GCL memory. **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST&&DWCXG_SGF)

Table 11-424 Fields for Register: MTL_GCL_Control (continued)

Bits	Name	Memory Access	Description
2	GCRR	R/W	Gate Control Related Registers When this field is set to 1, indicates that the R/W access is for the GCL related registers (BTR, CTR, TER, LLR) whose address is provided by GCRA. When 0, indicates that R/W must be directed to GCL from the address provided by GCLA. **Value After Reset:** 0x0 **Exists:** Always
1	R1W0	R/W	Read 1, Write 0: ■ When set to 1: Read Operation ■ When set to 0: Write Operation. **Values:** ■ 0x0 (WRITE): Write Operation ■ 0x1 (READ): Read Operation **Value After Reset:** 0x0 **Exists:** Always
0	SRWO	R/W	Start Read/Write Op ■ When this field is set, indicates that a Read/Write operation has started and is in progress. ■ When this field is reset by DWC_25gmac, indicates that the R/W operation is complete or an error has occurred. ■ Reads: Data can be read from MTL_GCL_Data register after this bit is reset. ■ Writes: MTL_GCL_Data must be programmed with write data before setting SRWO bit. Access restriction applies. Writing 1 sets the bit. Self-cleared. Writing 0 has no effect. **Values:** ■ 0x0 (DISABLE): Start Read/Write Op disabled ■ 0x1 (ENABLE): Start Read/Write Op enabled **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.22 MTL_GCL_Data

- **Description:** This register holds the read data or write data in case of reads and writes respectively.
- **Size:** 32 bits
- **Offset:** 0x1084
- **Exists:** (DWCXG_AV_EST||DWCXG_SGF)

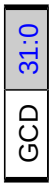


Table 11-425 Fields for Register: MTL_GCL_Data

Bits	Name	Memory Access	Description
31:0	GCD	R/W	Gate Control Data The data corresponding to the address selected in the GCL_Control register. Used for both Read and Write operations. **Value After Reset:** 0x0 **Exists:** Always

11.2.23 MTL_FPE_CTRL_STS

- **Description:** This register controls the operation of, and provides status for Frame Preemption (IEEE 802.1Qbu/802.3br).
- **Size:** 32 bits
- **Offset:** 0x1090
- **Exists:** (DWCXG_FPE)

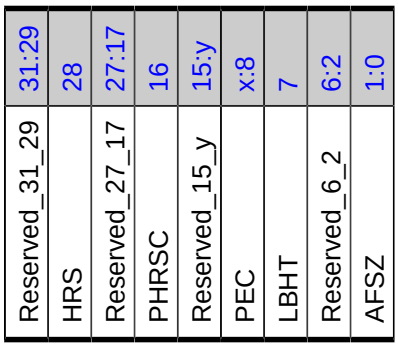


Table 11-426 Fields for Register: MTL_FPE_CTRL_STS

Bits	Name	Memory Access	Description
31:29	Reserved_31_29	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
28	HRS	R	Hold/Release Status ■ 1: Indicates a Set-and-Hold-MAC operation was executed and the pMAC is in Hold State. ■ 0: Indicates a Set-and-Release-MAC operation was executed and the pMAC is in Release State. **Values:** ■ 0x0 (SET_REL): Indicates a Set-and-Release-MAC operation was executed and the pMAC is in Release State ■ 0x1 (SET_HOLD): Indicates a Set-and-Hold-MAC operation was executed and the pMAC is in Hold State **Value After Reset:** 0x0 **Exists:** Always
27:17	Reserved_27_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-426 Fields for Register: MTL_FPE_CTRL_STS (continued)

Bits	Name	Memory Access	Description
16	PHRSC	R/W	Preemption MAC Hold/Release Sideband Control Select ■ 1: Indicates that Set-Hold-and-Release-MAC operation is controlled by Sideband Signal sbd_pmac_hold_i. ■ 0: Indicates that Set-Hold-and-Release-MAC operation is controlled by IEEE 802.1Qbv Enhancement to Scheduled Traffic (EST) Gate Control List (GCL) entries. **Value After Reset:** 0x0 **Exists:** DWCXG_AV_EST
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_TC!=8 **Range Variable[y]:** DWCXG_NUM_TC + 8
x:8	PEC	R/W	Preemption Classification This controls whether a traffic class (TC) is classified as preemptable or express. Each bit corresponds to a TC (PEC[0] is TC0, PEC[1] is TC1, and so on). When a bit is set, the packets in corresponding TC and all Tx Queues mapped to it are classified as preemptable. When set to 0, the corresponding TC and all Tx Queues mapped to it are classified as having express packets. When both EST (Qbv) and Preemption are enabled, TC0 is always assumed to be preemptable irrespective of the value of PEC[0] and is always assumed to be in "Open" state in the Gate Control List. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_NUM_TC + 7
7	LBHT	R/W	Level Based Hold Transition When set to 1, enables hold operation in second row of EST GCL during non-first loop even when hold request is set in first row, that is there is no release request to hold request transition. When set to 0, enables hold operation in second row of EST GCL only on transition from release request to hold request. **Value After Reset:** 0x0 **Exists:** DWCXG_AV_EST
6:2	Reserved_6_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-426 Fields for Register: MTL_FPE_CTRL_STS (continued)

Bits	Name	Memory Access	Description
1:0	AFSZ	R/W	Additional Fragment Size Used to indicate, in units of 64 bytes, the minimum number of bytes over 64 bytes required in non-final fragments of preempted frames. The minimum non-final fragment size is (AFSZ +1) * 64 bytes **Value After Reset:** 0x0 **Exists:** Always

11.2.24 MTL_FPE_Advance

- **Description:** This register holds the Hold and Release Advance time.
- **Size:** 32 bits
- **Offset:** 0x1094
- **Exists:** (DWCXG_FPE&&DWCXG_AV_EST)

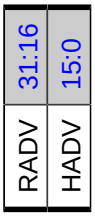


Table 11-427 Fields for Register: MTL_FPE_Advance

Bits	Name	Memory Access	Description
31:16	RADV	R/W	Release Advance The maximum time in nanoseconds that can elapse between issuing a RELEASE to the MAC and the MAC being ready to resume transmission of preemptable frames, in the absence of there being any express frames available for transmission. **Value After Reset:** 0x0 **Exists:** Always
15:0	HADV	R/W	Hold Advance The maximum time in nanoseconds that can elapse between issuing a HOLD to the MAC and the MAC ceasing to transmit any preemptable frame that is in the process of transmission or any preemptable frames that are queued for transmission. **Value After Reset:** 0x0 **Exists:** Always

11.2.25 MTL_RXP_Control_Status

- **Description:** This is the Control and Status register for Flexible Receive Parser.
- **Size:** 32 bits
- **Offset:** 0x10a0
- **Exists:** (DWCXG_FRP_EN)

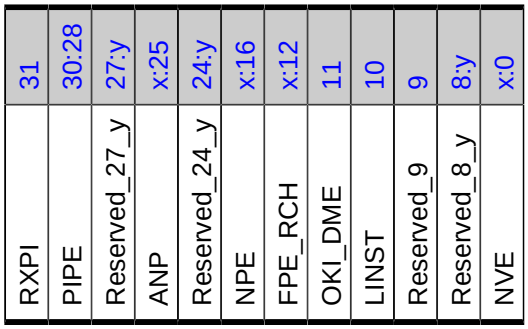


Table 11-428 Fields for Register: MTL_RXP_Control_Status

Bits	Name	Memory Access	Description
31	RXPI	R	Rx Parser is Active This status bit is set to 1 when the Rx parser is in Active state. This bit can be used as a handshake with the software when the parser gets disabled/enabled. **Values:** ■ 0x0 (INACTIVE): Rx Parser is not Active. ■ 0x1 (ACTIVE): Rx Parser is Active **Value After Reset:** 0x0 **Exists:** Always
30:28	PIPE	R	Number of Parallel Instruction Processor Engines. This field reflects the Number of Parallel Instruction Processor Engines selected in the configuration. This field is read-only. **Value After Reset:** DWCXG_FRP_PIPE **Exists:** Always
27:y	Reserved_27_y	R	Reserved **Value After Reset:** 0x0 **Exists:** ((DWCXG_FRP_PIPE>3) ? 3 : (DWCXG_FRP_PIPE>1) ? 2 : 1<3) **Range Variable[y]:** DWCXG_L2RXP_INPR + 25

Table 11-428 Fields for Register: MTL_RXP_Control_Status (continued)

Bits	Name	Memory Access	Description
x:25	ANP	R/W	Active Number of PIPEs. This field controls the number of PIPEs activated in the Rx Parser. This value must be equal to or less than the value read in PIPE field and always greater than 0. **Value After Reset:** DWCXG_FRP_PIPE **Exists:** ((DWCXG_FRP_PIPE>3) ? 3 : (DWCXG_FRP_PIPE>1) ? 2 : 1>0) **Range Variable[x]:** DWCXG_L2RXP_INPR + 24
24:y	Reserved_24_y	R	Reserved **Value After Reset:** 0x0 **Exists:** (([<functionof> (log(DWCXG_RXP_INST)/log(2))])<9) **Range Variable[y]:** DWCXG_RXP_INST_ADDR + 16
x:16	NPE	R/W	Number of Parsable Entries in the Instruction Table. This field indicates the number of parsable entries in the Instruction Table. This field is used in Rx-Parser logic to detect programming error. If the number of parsed entries for a packet is more than this value, NPEOVIS bit in the MTL_RXP_Interrupt_Control_Status register is set. **Note:** ■ A value of 0 value implies 1 Instruction. Value of 255 implies 256 instructions. ■ Program this field with a value between 0 and Maximum Entries of Parser Instruction Table-1.Read the value after programming this field. **Value After Reset:** DWCXG_FRP_ENTRIES-1 **Exists:** (([<functionof> (log(DWCXG_RXP_INST)/log(2))])>0) **Range Variable[x]:** DWCXG_RXP_INST_ADDR + 15
x:12	FPE_RCH	R/W	Frame Preemption Residue Receive DMA Channel This field holds the Rx Channel number to which the residual preemption frames must be forwarded. Preemption frames which pass Rx Parser filtering, are routed based on Instruction Table. All other frames are treated as residual frames and are routed to the channel number mentioned in this field. Channel-0 is used as a default Channel for express residual frames, so this field must not be programmed to a value 0. **Value After Reset:** 0x0 **Exists:** DWCXG_FPE&&(DWCXG_RXP_FPE_RCH_FW>0) **Range Variable[x]:** DWCXG_RXP_FPE_RCH_FW + 11

Table 11-428 Fields for Register: MTL_RXP_Control_Status (continued)

Bits	Name	Memory Access	Description
11	OKI_DME	R/W	Instruction's OK Index Dual Meaning Enable When this bit is set to 1, the OKI field in the Instruction can be interpreted also as Policing Counter Number/Valid or Gate ID/Valid when SGF or PC functions are enabled. For more detail refer respective feature section. When this bit is reset, the OKI field interpretation is always "OK Index". **Value After Reset:** 0x0 **Exists:** ((DWCXG_PSF&&!DWCXG_PC)||(! DWCXG_PSF&&DWCXG_PC))
10	LINST	R/W	Last Parsed Instruction Offset in Rx Status When this bit is set to 1, the MAC provides Flexible Receive Parser last instruction number/index/offset in RSSH/FRPLI field of RDES1 of Normal Receive Descriptor. **Value After Reset:** 0x0 **Exists:** (DWCXG>1)&&DWCXG_RSS_EN&&(DWCXG_DATAWIDTH ==64)
9	Reserved_9	R	Reserved **Value After Reset:** 0x0 **Exists:** Always
8:y	Reserved_8_y	R	Reserved **Value After Reset:** 0x0 **Exists:** (([<functionof> (log(DWCXG_RXP_INST)/log(2))])<9) **Range Variable[y]:** DWCXG_RXP_INST_ADDR

Table 11-428 Fields for Register: MTL_RXP_Control_Status (continued)

Bits	Name	Memory Access	Description
x:0	NVE	R/W	Number Valid Entries in the Instruction table. This field indicates the last valid entry in the Instruction Table. This is used by the Rx-Parser logic to detect programming errors. While parsing the table, if the address becomes greater than this entry, NVEOVIS bit in the MTL_RXP_Interrupt_Control_Status register is set. **Note:** ■ Value of 0 implies, 1st Instruction; a value of 255 implies 256th instruction. ■ Program this field with a value between 0 and Maximum Entries of Parser Instruction Table-1. Read the value after programming this field. **Value After Reset:** DWCXG_FRP_ENTRIES-1 **Exists:** (([<functionof> (log(DWCXG_RXP_INST)/log(2))])>0) **Range Variable[x]:** DWCXG_RXP_INST_ADDR - 1

11.2.26 MTL_RXP_Interrupt_Control_Status

- **Description:** This is Interrupt Control and Status register for Flexible Receive Parser.
- **Size:** 32 bits
- **Offset:** 0x10a4
- **Exists:** (DWCXG_FRP_EN)

31:20	19	18	17	16	15:4	3	2	1	0
Reserved_31_20	PDRFIE	FOOVIE	NPEOVIE	NVEOVIE	Reserved_15_4	PDRFIS	FOOVIS	NPEOVIS	NVEOVIS

Table 11-429 Fields for Register: MTL_RXP_Interrupt_Control_Status

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved **Value After Reset:** 0x0 **Exists:** Always
19	PDRFIE	R/W	Packet Drop due to RF Interrupt Enable When this bit is set, the interrupt is asserted when PDRFIS bit is set. When this bit is reset, the interrupt is not asserted when PDRFIS is set. **Values:** ■ 0x0 (DISABLE): Packet Drop due to RF Interrupt is disabled ■ 0x1 (ENABLE): Packet Drop due to RF Interrupt is enabled **Value After Reset:** 0x0 **Exists:** Always
18	FOOVIE	R/W	Frame Offset Overflow Interrupt Enable When this bit is set, the interrupt is asserted when FOOVIS bit is set. When this bit is reset, the interrupt is not asserted when FOOVIS is set. **Values:** ■ 0x0 (DISABLE): Frame Offset Overflow Interrupt is disabled ■ 0x1 (ENABLE): Frame Offset Overflow Interrupt is enabled **Value After Reset:** 0x0 **Exists:** Always

Table 11-429 Fields for Register: MTL_RXP_Interrupt_Control_Status (continued)

Bits	Name	Memory Access	Description
17	NPEOVIE	R/W	Number of Parsable Entries Overflow Interrupt Enable When this bit is set, the interrupt is asserted when NPEOVIS bit is set. When this bit is reset, the interrupt is not asserted when NPEOVIS is set. **Values:** ■ 0x0 (DISABLE): Number of Parsable Instructions Overflow Interrupt is disabled ■ 0x1 (ENABLE): Number of Parsable Instructions Overflow Interrupt is enabled **Value After Reset:** 0x0 **Exists:** Always
16	NVEOVIE	R/W	Number of Valid Entries Overflow Interrupt Enable When this bit is set, the interrupt is asserted when NVEOVIS bit is set. When this bit is reset, the interrupt is not asserted when NVEOVIS is set. **Values:** ■ 0x0 (DISABLE): Last valid instruction Overflow Interrupt is disabled ■ 0x1 (ENABLE): Last valid instruction Overflow Interrupt is enabled **Value After Reset:** 0x0 **Exists:** Always
15:4	Reserved_15_4	R	Reserved **Value After Reset:** 0x0 **Exists:** Always
3	PDRFIS	R/W	Packet Dropped due to RF Interrupt Status When the Rx-Parser requests a packet-drop (by setting RF=1 in the instruction table), this bit is set to 1. This bit is cleared when the software writes 1 to this bit. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. Writing 0 has no effect. **Values:** ■ 0x0 (INACTIVE): Packet Dropped due to RF Interrupt Status not detected ■ 0x1 (ACTIVE): Packet Dropped due to RF Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-429 Fields for Register: MTL_RXP_Interrupt_Control_Status (continued)

Bits	Name	Memory Access	Description
2	FOOVIS	R/W	Frame Offset Overflow Interrupt Status During parsing, if the Instruction table entry's 'Frame Offset' is found to be more than EOF offset, this bit is set. This bit is cleared when the software writes 1 to this bit. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. Writing 0 has no effect. **Values:** ■ 0x0 (INACTIVE): Frame Offset Overflow Interrupt Status not detected ■ 0x1 (ACTIVE): Frame Offset Overflow Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	NPEOVIS	R/W	Number of Parsable Entries Overflow Interrupt Status While parsing, if the number of parsed entries is found to be more than NPE[] (Number of Parsable Entries in MTL_RXP_Control register), this bit is set to 1. This bit is cleared when the application writes 1 to this bit. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. Writing 0 has no effect. **Values:** ■ 0x0 (INACTIVE): Number of Parsable Entries Overflow Interrupt Status not detected ■ 0x1 (ACTIVE): Number of Parsable Entries Overflow Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-429 Fields for Register: MTL_RXP_Interrupt_Control_Status (continued)

Bits	Name	Memory Access	Description
0	NVEOVIS	R/W	Number of Valid Entries Overflow Interrupt Status While parsing, if the instruction address is found to be more than NVE (Number of Valid Entries in MTL_RXP_Control register), this bit is set to 1. This bit is cleared when the software writes 1 to this bit. Access restriction applies. Self-set to 1 on internal event. Clears on write of 1. Writing 0 has no effect. **Values:** ■ 0x0 (INACTIVE): Number of Valid Entries Overflow Interrupt Status not detected ■ 0x1 (ACTIVE): Number of Valid Entries Overflow Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.27 MTL_Rx_SPkt_Ctrl

- **Description:** This register controls the processing of the Header fields of the IEEE1722 special packets.
- **Size:** 32 bits
- **Offset:** 0x10a8
- **Exists:** ((DWCXG==2)&&SNPS_RSVDPARAM_48)

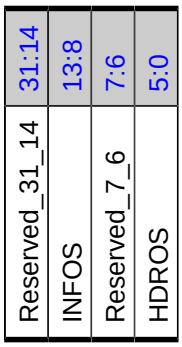
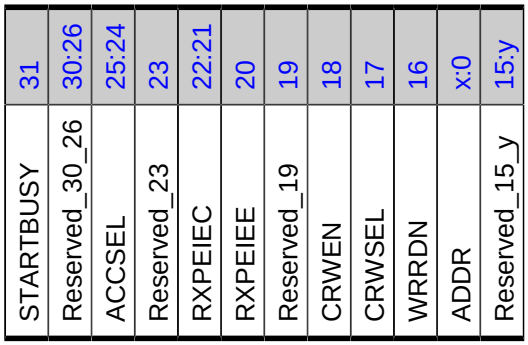


Table 11-430 Fields for Register: MTL_Rx_SPkt_Ctrl

Bits	Name	Memory Access	Description
31:14	Reserved_31_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:8	INFOS	R/W	Information Offset his field indicates the offset (in bytes) before the start of AVTP image/Payload data, where 4 byte information is located which will be part of the packetinfo as described in 2.7. **Value After Reset:** 0x0 **Exists:** Always
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:0	HDROS	R/W	Header Offset This field indicates the offset (in bytes) before the start of AVTP Image/Payload data, where the special 8Byte header is located. The DWC_25gmac will process this header as described in 2.7. Note that Bit[1:0] is Read-Only and always 0 as the header is aligned to 4Byte intervals. **Value After Reset:** 0x0 **Exists:** Always

11.2.28 MTL_Indirect_Acc_Control_Status

- **Description:** This is Control register to access the MTL indirect registers/tables.
- **Size:** 32 bits
- **Offset:** 0x10b0
- **Exists:** ((DWCXG_MTL_RCLK_IACC_EN||DWCXG_MTL_RRC_IACC_EN))



31	30:26	25:24	23	22:21	20	19	18	17	16	x:0	15:y
STARTBUSY	Reserved_30_26	ACCSEL	Reserved_23	RXPEIEC	RXPEIEE	Reserved_19	CRWEN	CRWSEL	WRRDN	ADDR	Reserved_15_y

Table 11-431 Fields for Register: MTL_Indirect_Acc_Control_Status

Bits	Name	Memory Access	Description
31	STARTBUSY	R/W	Indirect Access Busy When this bit is set to 1 by the software, it indicates to start the Read/Write operation from/to selected ACCSEL. Software must read this bit as 0 before issuing read or write request to access the selected ACCSEL. This bit is cleared by DWC_25gmac when the read/write transaction is completed. Software must not issue the next read or write operation, until this bit is cleared. **Values:** ■ 0x0 (INACTIVE): Read/Write transaction complete and Idle. ■ 0x1 (ACTIVE): Read/Write transaction is active **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30:26	Reserved_30_26	R	Reserved **Value After Reset:** 0x0 **Exists:** Always

Table 11-431 Fields for Register: MTL_Indirect_Acc_Control_Status (continued)

Bits	Name	Memory Access	Description
25:24	ACCSEL	R/W	Indirect Access Selection This field is used to select the indirect table/register block to access. Software can select what to access from following selection option. **Values:** ■ 0x0 (ACC_IT): Access FRP Instruction Table. ■ 0x1 (ACC_IFR): Access Indirect FRP Register block. ■ 0x2 (ACC_RCHLIST): Access Indirect receive channel list array used for packet duplication. ■ 0x3 (ACC_MMC): Access Indirect MMC Register block. **Value After Reset:** 0x0 **Exists:** Always
23	Reserved_23	R	Reserved **Value After Reset:** 0x0 **Exists:** Always
22:21	RXPEIEC	R/W	ECC Inject Error Control. This field is valid only when Instruction Table (ACC_IT) is selected in the ACCSEL field. When RXPEIEE bit of this register is set, following errors are inserted based on the value encoded in this field. **Values:** ■ 0x0 (D_1BIT): Insert 1 bit ECC error in data or parity ■ 0x1 (D_2BIT): Insert 2 bit ECC error in data or parity ■ 0x2 (A_1BIT): Insert 1 bit ECC error in address ■ 0x3 (RSVD): Reserved **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ECC
20	RXPEIEE	R/W	ECC Inject Error Enable. This field is valid only when Instruction Table (ACC_IT) is selected in ACCSEL field. When set, enables the ECC error injection feature. When reset, disables the ECC error injection feature. **Note:** Hardware injects error when this bit is set and software initiates read access to Rx Parser Instruction Table. **Values:** ■ 0x0 (DISABLE): ECC Inject Error for ACCSEL is disabled ■ 0x1 (ENABLE): ECC Inject Error for ACCSEL is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_ECC

Table 11-431 Fields for Register: MTL_Indirect_Acc_Control_Status (continued)

Bits	Name	Memory Access	Description
19	Reserved_19	R	Reserved **Value After Reset:** 0x0 **Exists:** Always
18	CRWEN	R/W	Continuous Read/Write Mode Enable This field is valid only when Instruction Table (ACC_IT) or ACC_RCHLIST is selected in the ACCSEL field. When this bit is set, continuous read/write mode is enabled. CRWSEL selects either Read or Write continuous mode. In this mode, software has to set ADDR, STARTBUSY once. For subsequent write/read operations, it is not required to set ADDR and STARTBUSY. The hardware automatically triggers the read or write access to the next address when the MTL_Indirect_Acc_Data register is read or written. To disable this mode, set this bit to 0. **Note:** Byte-wise Write/Read is not supported. **Values:** ■ 0x0 (DISABLE): Disabled Continuous Read/Write Mode ■ 0x1 (ENABLE): Enabled Continuous Read/Write Mode **Value After Reset:** 0x0 **Exists:** Always
17	CRWSEL	R/W	Continuous Read/Write Mode Select This field is valid only when Instruction Table (ACC_IT), ACC_RCHLIST or ACC_MMC is selected in ACCSEL field. When CRWEN is set, CRWSEL is valid. Software can select which continuous mode to use, either Write or Read. **Values:** ■ 0x0 (READ): Continuous Read ■ 0x1 (WRITE): Continuous Write **Value After Reset:** 0x0 **Exists:** Always
16	WRRDN	R/W	Read Write Control When this bit is set to 1, indicates the write operation is selected for ACCSEL. When this bit is set to 0, indicates the read operation is selected for ACCSEL. **Values:** ■ 0x0 (READ): Read operation to ACCSEL. ■ 0x1 (WRITE): Write operation to ACCSEL. **Value After Reset:** 0x0 **Exists:** Always

Table 11-431 Fields for Register: MTL_Indirect_Acc_Control_Status (continued)

Bits	Name	Memory Access	Description
x:0	ADDR	R/W	Indirect Access Write/Read Address This field indicates the address of each 32-bit data/register of the selected ACCSEL. ACCSEL Value Usage of this Field ■ ACC_IT: used for Instruction Table's 32 bits data word address. ■ ACC_IFR: used for Indirect FRP Register Block. Indirect FRP Register Block Address Map. 1) 0x0 : For MTL_RXP_Drop_Cnt. 2) 0x1 : For MTL_RXP_Error_Cnt. 3) 0x2 : For MTL_RXP_Bypass_Cnt. 4) 0x40 + (0x10 * i + 0x0): For MTL_FRP_ACNTR, where i = 0 to DWCXG_FRP_NUM_AC-1. 5) 0x40 + (0x10 * i + 0x2): For MTL_FRP_ACNTR_CNFG, where i = 0 to DWCXG_FRP_NUM_AC-1. Software can write this before issuing any Read/Write command. ■ ACC_RCHLIST: used for receive channel list indirect register array. Refer respective register description for more detail about addressing. 0x00: MTL_RCHLIST_IndReg0_0 0x01: MTL_RCHLIST_IndReg0_1 ... 0x0F: MTL_RCHLIST_IndReg0_15 ... 0xF0: MTL_RCHLIST_IndReg15_0 0xF1: MTL_RCHLIST_IndReg15_1 ... 0xFF: MTL_RCHLIST_IndReg15_15 ■ ACC_MMC: used for Indirect MMC (MAC Management Counters) Register Block. 0x00: MTL_RMMC_RCH_MAP_IndReg0 0x01: MTL_RMMC_RCH_MAP_IndReg1 ... 0x0F: MTL_RMMC_RCH_MAP_IndReg15 **Note:** Use valid address according to the ACCSEL selection. Software should write this before issuing any Read/Write command. **Value After Reset:** 0x0 **Exists:** ([<functionof> max(DWCXG_MTL_RCLK_IACC_AW,DWCXG_MTL_RRC_IACC_ADDR)])>0 **Range Variable[x]:** DWCXG_MTL_IACC_ADDR - 1

Table 11-431 Fields for Register: MTL_Indirect_Acc_Control_Status (continued)

Bits	Name	Memory Access	Description
15:y	Reserved_15_y	R	Reserved **Value After Reset:** 0x0 **Exists:** ([<functionof> max(DWCXG_MTL_RCLK_IACC_AW,DWCXG_MTL_RRC_IACC_ADDR)])<16 **Range Variable[y]:** DWCXG_MTL_IACC_ADDR

11.2.29 MTL_Indirect_Acc_Data

- **Description:** This is the data register to Access MTL indirect registers/tables.
- **Size:** 32 bits
- **Offset:** 0x10b4
- **Exists:** ((DWCXG_MTL_RCLK_IACC_EN||DWCXG_MTL_RRC_IACC_EN))

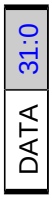


Table 11-432 Fields for Register: MTL_Indirect_Acc_Data

Bits	Name	Memory Access	Description
31:0	DATA	R/W	Indirect Access Write/Read Data Software must write this register before issuing any write command. DWC_25gmac provides the read data from selected ACCSEL for read operation when STARTBUSY =0 after read command. **Value After Reset:** 0x0 **Exists:** Always

11.2.30 MTL_RXP_Drop_Cnt

- **Description:** This register provides the number of packets dropped by the Flexible Receive Parser. This is an indirect register which can be accessed through MTL_Indirect_Acc_Control_Status and MTL_Indirect_Acc_Data registers.
- **Size:** 32 bits
- **Offset:** 0x10b4
- **Exists:** (DWCXG_FRP_EN)

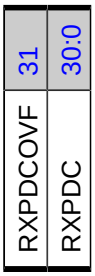


Table 11-433 Fields for Register: MTL_RXP_Drop_Cnt

Bits	Name	Memory Access	Description
31	RXPDCOVF	R	Rx Parser Packet Drop Counter Overflow. When set, this bit indicates that the MTL_RXP_Drop_cnt (RXPDC) counter field has crossed the maximum limit. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Rx Parser Drop count overflow not occurred ■ 0x1 (ACTIVE): Rx Parser Drop count overflow occurred **Value After Reset:** 0x0 **Exists:** Always
30:0	RXPDC	R	Rx Parser Packet Drop Count. This 31-bit counter is incremented when a Rx Parser drops a packet due to AF = 0 and RF = 1. The counter is cleared when the register is read. **Value After Reset:** 0x0 **Exists:** Always

11.2.31 MTL_RXP_Error_Cnt

- **Description:** This register provides Error Counts of the Flexible Receive Parser. This is an indirect register which can be access through MTL_Indirect_Acc_Control_Status and MTL_Indirect_Acc_Data registers.
- **Size:** 32 bits
- **Offset:** 0x10b4
- **Exists:** (DWCXG_FRP_EN)

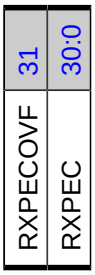


Table 11-434 Fields for Register: MTL_RXP_Error_Cnt

Bits	Name	Memory Access	Description
31	RXPECOVF	R	Rx Parser Error Counter Overflow Bit When set, this bit indicates that the MTL_RXP_Error_cnt (RXPEC) counter field has crossed the maximum limit. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Rx Parser Error count overflow not occurred ■ 0x1 (ACTIVE): Rx Parser Error count overflow occurred **Value After Reset:** 0x0 **Exists:** Always
30:0	RXPEC	R	Rx Parser Error count This 31-bit counter is incremented when the Rx Parser encounters the following error scenarios: ■ Current Instruction Address is greater than NVE[] ■ Number of Parsed Instructions is greater than (NPE + 1) ■ Current Instruction's Frame offset is greater than EOF frame offset or Last Parsable Byte ■ Parsing is not complete even after receiving the next to next packet SOF The counter is cleared when the register is read. **Value After Reset:** 0x0 **Exists:** Always

11.2.32 MTL_RXP_Bypass_Cnt

- **Description:** The MTL_RXP_Bypass_Cnt registers provides the count of the number of frames bypassed by Rx Parser. This is an indirect register which can be access through MTL_Indirect_Acc_Control_Status and MTL_Indirect_Acc_Data registers.
- **Size:** 32 bits
- **Offset:** 0x10b4
- **Exists:** (DWCXG_FRP_EN)

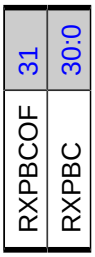


Table 11-435 Fields for Register: MTL_RXP_Bypass_Cnt

Bits	Name	Memory Access	Description
31	RXPBCOF	R	Rx-Parser Bypass Counter Overflow Bit When set, this bit indicates that the RXPBC Counter field crossed the maximum limit. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Rx Parser Bypass Counter overflow not occurred ■ 0x1 (ACTIVE): Rx Parser Bypass Counter overflow occurred **Value After Reset:** 0x0 **Exists:** Always
30:0	RXPBC	R	Rx-Parser Bypass Counter This 31-bit counter is implemented whenever a Rx Parser Bypass a packet due to AF = 1 and RF = 1. The counter is cleared when the register is read. **Value After Reset:** 0x0 **Exists:** Always

11.2.33 MTL_FRP_ACNTR(#i) (for i = 0; i <= DWCXG_FRP_NUM_AC-1)

- **Description:** The MTL_FRP_ACNTR(#i) registers are indirect and provide the count of the number of frames accepted by FRP particular to channel. Such accept counters mapping with receive channel is static, one to one, if supported number of receive channels are less than or equal to 16. Otherwise if it is more than 16 then, FRP implements fix number of accept counters and provides programmable option to application to map it to application-specific receive channel via corresponding MTL_FRP_ACNTR_CNFG(#i) indirect registers. These indirect registers can be accessible through MTL_Indirect_Acc_Control_Status and MTL_Indirect_Acc_Data registers.
- **Size:** 32 bits
- **Offset:** 0x10b4
- **Exists:** (DWCXG_FRP_EN)&&DWCXG_FRP_AC_EN&&(DWCXG_FRP_NUM_AC>0)

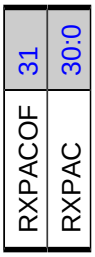


Table 11-436 Fields for Register: MTL_FRP_ACNTR(#i) (for i = 0; i <= DWCXG_FRP_NUM_AC-1)

Bits	Name	Memory Access	Description
31	RXPACOF	R	Rx-Parser Accept Counter Overflow Bit When set, this bit indicates that the RXPAC Counter field crossed the maximum limit. Access restriction applies. Clears on read. Self-set to 1 on internal event. **Values:** ■ 0x0 (INACTIVE): Rx Parser Accept Counter overflow not occurred ■ 0x1 (ACTIVE): Rx Parser Accept Counter overflow occurred **Value After Reset:** 0x0 **Exists:** Always
30:0	RXPAC	R	Rx-Parser Accept Counter This 31-bit counter is incremented whenever the Rx Parser accepts a packet due to AF = 1 and RF = 0. The packet must match the Rx Parser acceptance criteria irrespective of its type (length or type) and size (undersize/runt or oversize). The counter is cleared when the register is read. **Value After Reset:** 0x0 **Exists:** Always

11.2.34 MTL_FRP_ACNTR_CNFG(#i) (for i = 0; i <= DWCXG_FRP_NUM_AC-1)

- **Description:** The MTL_FRP_ACNTR_CNFG(#i) registers are indirect and allows application to configure corresponding FRP accept counter. Such indirect register is available for each supported FRP accept counter, when accept counter configuration is supported. These indirect registers can be accessible through MTL_Indirect_Acc_Control_Status and MTL_Indirect_Acc_Data registers. Application should configure such accept counter well before FRP get enabled.
- **Size:** 32 bits
- **Offset:** 0x10b4
- **Exists:** (DWCXG_FRP_EN)&&DWCXG_FRP_CFG_AC&&(DWCXG_FRP_NUM_AC>0)

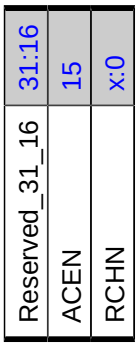


Table 11-437 Fields for Register: MTL_FRP_ACNTR_CNFG(#i) (for i = 0; i <= DWCXG_FRP_NUM_AC-1)

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved **Value After Reset:** 0x0 **Exists:** Always
15	ACEN	R/W	Accept Counter Enable Set this bit to 1, to enable corresponding MTL_FRP_ACNTR(#i) FRP accept counter. **Values:** ■ 0x0 (DISABLE): Accept counter is disabled. ■ 0x1 (ENABLE): Accept counter is enabled. **Value After Reset:** 0x0 **Exists:** Always
x:0	RCHN	R/W	Receive Channel Number Program Rx DMA channel number to map it with corresponding MTL_FRP_ACNTR(#i) FRP accept counter. Note: The application must program a valid Rx DMA Channel number, otherwise the corresponding Accept Counter is not updated. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (DWCXG_NUM_RX_CH_EFFW) - 1

11.2.35 MTL_RCHLIST_IndReg(#i)_(#j) (for j = 0; j <= DWCXG_RCHLIST_DCNT-1)

- **Description:** The MTL_RCHLIST_IndReg0_0 registers, which are accessible through MTL_RXP_Indirect_Acc_* register. Each register contains receive channel list's one hot encoded receive channel selection. These receive channel list are used in receive filters for packet duplication. Each receive channel list's 32-bit data word is accessible with ((m * Number of 32 bit Dataword Counts in Receive Channel List) + n) address. Where m, 0 to (Number of Entries in Receive Channel List for Packet Duplication-1) represent receive channel list number and n, 0 to (Number of 32 bit Dataword Counts in Receive Channel List-1) represent respective receive channel list's 32-bit data word number. Number of receive channel list's 32-bit data word depends on receive channel list's width.
- **Size:** 32 bits
- **Offset:** 0x10b4
- **Exists:** (DWCXG_RCHLIST_CNT>0)&&(DWCXG_RCHLIST_DCNT>0)

Table 11-438 Fields for Register: MTL_RCHLIST_IndReg(#i)_(#j) (for j = 0; j <= DWCXG_RCHLIST_DCNT-1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_RCHS	R	Reserved **Value After Reset:** 0x0 **Exists:** ((DWCXG_NUM_RX_CH_EFF>(0*32))&&(DWCXG_NUM_RX_CH_EFF<((0+1)*32))) **Range Variable[x]:** (32-(DWCXG_NUM_RX_CH_EFF-0)) + (DWCXG_NUM_RX_CH_EFF-0) - 1 **Range Variable[y]:** DWCXG_NUM_RX_CH_EFF

Table 11-438 Fields for Register: MTL_RCHLIST_IndReg(#i)_(#j) (for j = 0; j <= DWCXG_RCHLIST_DCNT-1) (continued)

Bits	Name	Memory Access	Description
x:0	RCHS	R/W	Receive Channel Select This field contains one hot encoded receive channel selection for duplicated packet routing. **Note:** Last data word can have reserved bit based on enabled number of receive channel. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (((DWCXG_NUM_RX_CH_EFF>0)&&(DWCXG_NUM_RX_CH_EFF<32))?(DWCXG_NUM_RX_CH_EFF-0):32) - 1

11.2.36 MTL_RMMC_RCH_MAP_IndReg(#i) (for i = 0; i <= DWCXG_RCHLIST_DCNT-1)

- **Description:** The MTL_RMMC_RCH_MAP_IndReg(#i) registers, which are accessible through MTL_Indirect_Acc_* register. Each register contains 32-bit dataword of one hot encoded receive channel selection. Such receive channel selection is used to map certain set of Rx Frame/Priority/IPC MMC with application specific receive channel. For specific scenarios, such RxMMC are allowed to update only, when corresponding frame is routed to receive channels which are programmed in these register by setting corresponding receive channel's bit to 1. Each receive channel selection's 32-bit dataword register is accessible with i address. Where i, 0 to ((number of 32bits dataword counts in receive channel selection)-1), represent respective receive channel selection's 32-bit dataword number. Number of receive channel selection's 32-bit dataword depends on selected total number of receive channels. Maximum 32-bit dataword counts can be 16.
- **Size:** 32 bits
- **Offset:** 0x10b4
- **Exists:** DWCXG_RMMC_RCH_MAP&&(DWCXG_RCHLIST_DCNT>0)

Table 11-439 Fields for Register: MTL_RMMC_RCH_MAP_IndReg(#i) (for i = 0; i <= DWCXG_RCHLIST_DCNT-1)

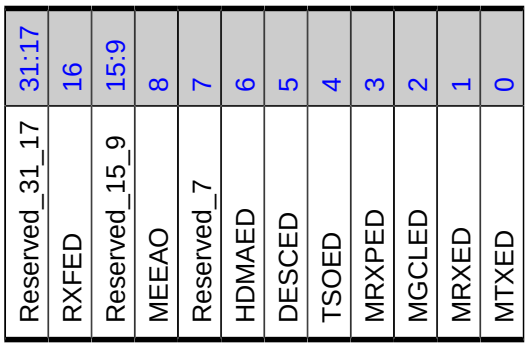
Bits	Name	Memory Access	Description
x:y	Reserved_31_RCHM	R	Reserved **Value After Reset:** 0x0 **Exists:** ((DWCXG_NUM_RX_CH_EFF>(0*32))&&(DWCXG_NUM_RX_CH_EFF<((0+1)*32))) **Range Variable[x]:** (32-(DWCXG_NUM_RX_CH_EFF-0)) + (DWCXG_NUM_RX_CH_EFF-0) - 1 **Range Variable[y]:** DWCXG_NUM_RX_CH_EFF
x:0	RCHM	R/W	Receive Channel Mapping for RxMMC This field contains one hot encoded receive channel selection for certain set of Rx Frame/Priority/IPC MAC Management Counters (RxMMC). For specific scenarios, such RxMMC are allowed to update only, when corresponding frame is routed to receive channels which are programmed in this field by setting corresponding receive channel's bit to 1.

Table 11-439 Fields for Register: MTL_RMMC_RCH_MAP_IndReg(#i) (for i = 0; i <= DWCXG_RCHLIST_DCNT-1) (continued)

Bits	Name	Memory Access	Description
			Note1: Last data word can have reserved bit based on selected number of receive channel. **Note2:** It is recommended to program receive channel mapping well before normal packet traffic reception. The application must program at the least one valid receive channel, otherwise the RMMC-RCHM counters are not allowed for update on specific scenarios.**Note3:** Programming all bits of this field to 1, does not mean DWC_25gmac bypasses receive channels mapping for RMMC-RCHM counters. In order to bypass it, application should disable such receive channels mapping by programming DRCHM field of MMC_Control register to 1. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (((DWCXG_NUM_RX_CH_EFF>0)&&(DWCXG_NUM_RX_CH_EFF<32))?(DWCXG_NUM_RX_CH_EFF-0):32) - 1

11.2.37 MTL_ECC_Control

- **Description:** The MTL_ECC_Control register establishes the operating mode of ECC related to MAC and MTL memories.
- **Size:** 32 bits
- **Offset:** 0x10c0
- **Exists:** (DWCXG_ASP_ECC)



31:17	16	15:9	8	7	6	5	4	3	2	1	0
Reserved_31_17	RXFED	Reserved_15_9	MEEAO	Reserved_7	HDMAED	DESCED	TSOED	MRXPED	MGCLED	MRXED	MTXED

Table 11-440 Fields for Register: MTL_ECC_Control

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	RXFED	R/W	MAC RXF Memory ECC Checker Disable ■ 1: Disables the ECC Checker for RXF Memory. ■ 0: Enables the ECC Checker for RXF Memory. **Value After Reset:** 0x0 **Exists:** DWCXG_RXF_MEM_EN
15:9	Reserved_15_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	MEEAO	R/W	MAC/MTL ECC Error Address Status Override When set to 1, the address of location in which the latest ECC error is detected, is stored in the MTL_ECC_Err_Addr_Status register. When reset to 0, the address of the location in which the ECC error is reported for the first time is stored in this register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-440 Fields for Register: MTL_ECC_Control (continued)

Bits	Name	Memory Access	Description
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6	HDMAED	R/W	**Values:** ■ 0x0 (ENABLE): HDMA CSR memory ECC Checker is enabled ■ 0x1 (DISABLE): HDMA CSR memory ECC Checker is disabled **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50
5	DESCED	R/W	DCACHE memory ECC Checker Disable ■ 1: Disables the ECC Checker for DCACHE memory in DMA. ■ 0: Enables the ECC Checker for DCACHE memory in DMA. **Value After Reset:** 0x0 **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN)
4	TSOED	R/W	TSO memory ECC Checker Disable ■ 1: Disables the ECC Checker for TSO memory in DMA. ■ 0: Enables the ECC Checker for TSO memory in DMA. **Value After Reset:** 0x0 **Exists:** DWCXG_TSO_MEM_EN
3	MRXPED	R/W	MTL Rx Parser ECC Checker Disable ■ 1: Disables the ECC Checker for Rx Parser memory. ■ 0: Enables the ECC Checker for Rx Parser memory. **Value After Reset:** 0x0 **Exists:** DWCXG_FRP_EN
2	MGCLED	R/W	MTL GCL ECC Checker Disable ■ 1: Disables the ECC Checker for GCL memory. ■ 0: Enables the ECC Checker for GCL memory. **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST||DWCXG_SGF)

Table 11-440 Fields for Register: MTL_ECC_Control (continued)

Bits	Name	Memory Access	Description
1	MRXED	R/W	MTL Rx FIFO ECC Checker Disable ■ 1: Disables the ECC Checker for MTL Rx FIFO memory. ■ 0: Enables the ECC Checker for MTL Rx FIFO memory. **Value After Reset:** 0x0 **Exists:** Always
0	MTXED	R/W	MTL Tx FIFO ECC Checker Disable ■ 1: Disables the ECC Checker for MTL Tx FIFO memory. ■ 0: Enables the ECC Checker for MTL Tx FIFO memory. **Value After Reset:** 0x0 **Exists:** Always

11.2.38 MTL_Safety_Interrupt_Status

- **Description:** The MTL_Safety_Interrupt_Status registers indicates the occurrence of various safety related events.
- **Size:** 32 bits
- **Offset:** 0x10c4
- **Exists:** (DWCXG_ASP_ECC)

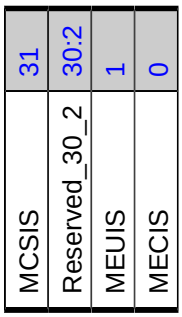


Table 11-441 Fields for Register: MTL_Safety_Interrupt_Status

Bits	Name	Memory Access	Description
31	MCSIS	R	MAC Safety Unrecoverable Interrupt Status This bit is set when the interrupt due to Unrecoverable Status is set in the MAC_DPP_FSM_Interrupt_Status register. MAC_DPP_FSM_Interrupt_Status register must be read when this bit is set, to get the cause of the safety interrupt in MAC. **Value After Reset:** 0x0 **Exists:** Always
30:2	Reserved_30_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	MEUIS	R	MAC/MTL ECC Uncorrectable error Interrupt Status This bit is set when the interrupt due to Unrecoverable Status is set in the MTL_ECC_Interrupt_Status register. To get the exact cause of the interrupt, the application must read the MTL_ECC_Interrupt_Status register. **Value After Reset:** 0x0 **Exists:** Always
0	MECIS	R	MAC/MTL ECC Correctable error Interrupt Status This bit is set when the interrupt due to Correctable Status is set in the MTL_ECC_Interrupt_Status register. To get the exact cause of the interrupt, the application must read the MTL_ECC_Interrupt_Status register. **Value After Reset:** 0x0 **Exists:** Always

11.2.39 MTL_ECC_Interrupt_Enable

- **Description:** The MTL_ECC_Interrupt_Enable register provides enable bits for the ECC interrupts.
- **Size:** 32 bits
- **Offset:** 0x10c8
- **Exists:** (DWCXG_ASP_ECC)

31:17	16	15:13	12	11:9	8	7:5	4	3:1	0
Reserved_31_17	RXFCEIE	Reserved_15_13	RPCEIE	Reserved_11_9	GCEIE	Reserved_7_5	RXCEIE	Reserved_3_1	TXCEIE

Table 11-442 Fields for Register: MTL_ECC_Interrupt_Enable

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	RXFCEIE	R/W	RXF Memory Correctable Error Interrupt Enable When set to 1, generates an interrupt when an correctable error is detected at RXF Memory interface. It is indicated in RXFCES status bit of MTL_ECC_Interrupt_Status register. When set to 0, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): RXF Memory Correctable Error Interrupt is disabled ■ 0x1 (ENABLE): RXF Memory Correctable Error Interrupt is enabled **Value After Reset:** 0x1 **Exists:** DWCXG_RXF_MEM_EN
15:13	Reserved_15_13	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-442 Fields for Register: MTL_ECC_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
12	RPCEIE	R/W	Rx Parser memory Correctable Error Interrupt Enable When set, generates an interrupt when an correctable error is detected at the Rx Parser memory interface. It is indicated in RPCES status bit of MTL_ECC_Interrupt_Status register. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Rx Parser memory Correctable Error Interrupt is disabled ■ 0x1 (ENABLE): Rx Parser memory Correctable Error Interrupt is enabled **Value After Reset:** 0x1 **Exists:** DWCXG_FRP_EN
11:9	Reserved_11_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	GCEIE	R/W	GCL memory Correctable Error Interrupt Enable When set, generates an interrupt when a correctable error is detected at the MTL GCL memory interface(EST/SGF). It is indicated in the GCES and SCES bits of MTL_ECC_Interrupt_Status register. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): GCL memory Correctable Error Interrupt is disabled ■ 0x1 (ENABLE): GCL memory Correctable Error Interrupt is enabled **Value After Reset:** 0x1 **Exists:** (DWCXG_AV_EST||DWCXG_SGF)
7:5	Reserved_7_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-442 Fields for Register: MTL_ECC_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
4	RXCEIE	R/W	Rx memory Correctable Error Interrupt Enable When set, generates an interrupt when a correctable error is detected at the MTL Rx memory interface. It is indicated in the RXCES bit of MTL_ECC_Interrupt_Status register. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Rx memory Correctable Error Interrupt is disabled ■ 0x1 (ENABLE): Rx memory Correctable Error Interrupt is enabled **Value After Reset:** 0x1 **Exists:** Always
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	TXCEIE	R/W	Tx memory Correctable Error Interrupt Enable When set, generates an interrupt when a correctable error is detected at the MTL Tx memory interface. It is indicated in the TXCES bit of MTL_ECC_Interrupt_Status register. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Tx memory Correctable Error Interrupt is disabled ■ 0x1 (ENABLE): Tx memory Correctable Error Interrupt is enabled **Value After Reset:** 0x1 **Exists:** Always

11.2.40 MTL_ECC_Interrupt_Status

- **Description:** The MTL_ECC_Interrupt_Status register provides MAC/MTL ECC Interrupt Status.
- **Size:** 32 bits
- **Offset:** 0x10cc
- **Exists:** (DWCXG_ASP_ECC)

31:23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved_31_23	RXFUES	RXFAMS	RXFCES	Reserved_19	SUES	SAMS	SCES	Reserved_15	RPUES	RPAMS	RPCES	Reserved_11	GUES	GAMS	GCES	Reserved_7	RXUES	RXAMS	RXCES	Reserved_3	TXUES	TXAMS	TXCES

Table 11-443 Fields for Register: MTL_ECC_Interrupt_Status

Bits	Name	Memory Access	Description
31:23	Reserved_31_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
22	RXFUES	R/W	MAC RXF Memory Uncorrectable Error Status ■ 1: Indicates that an uncorrectable error is detected at RXF Memory interface. **Values:** ■ 0x0 (INACTIVE): MAC RXF Memory Uncorrectable Error is not detected ■ 0x1 (ACTIVE): MAC RXF Memory Uncorrectable Error is detected **Value After Reset:** 0x0 **Exists:** DWCXG_RXF_MEM_EN **Testable:** untestable

Table 11-443 Fields for Register: MTL_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
21	RXFAMS	R/W	MAC RXF Memory Address Mismatch Status ■ 1: Indicates that address mismatch is found for address bus of RXF Memory. **Values:** ■ 0x0 (INACTIVE): MAC RXF Memory Address Mismatch is not detected ■ 0x1 (ACTIVE): MAC RXF Memory Address Mismatch is detected **Value After Reset:** 0x0 **Exists:** DWCXG_RXF_MEM_EN **Testable:** untestable
20	RXFCES	R/W	MAC RXF Memory Correctable Error Status ■ 1: Indicates that correctable error is detected at RXF Memory interface. **Values:** ■ 0x0 (INACTIVE): MAC RXF Memory Correctable Error is not detected ■ 0x1 (ACTIVE): MAC RXF Memory Correctable Error is detected **Value After Reset:** 0x0 **Exists:** DWCXG_RXF_MEM_EN **Testable:** untestable
19	Reserved_19	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
18	SUES	R/W	MTL SGF GCL memory Uncorrectable Error Status When this bit is set, it indicates that an uncorrectable error is detected at MTL SGF GCL memory interface. **Values:** ■ 0x0 (INACTIVE): MTL SGF memory Uncorrectable Error Status not detected ■ 0x1 (ACTIVE): MTL SGF memory Uncorrectable Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_SGF) **Testable:** untestable

Table 11-443 Fields for Register: MTL_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
17	SAMS	R/W	MTL SGF GCL memory Address Mismatch Status When this bit is set, it indicates that address mismatch is found for address bus of MTL SGF GCL memory. **Values:** ■ 0x0 (INACTIVE): MTL SGF memory Address Mismatch Status not detected ■ 0x1 (ACTIVE): MTL SGF memory Address Mismatch Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_SGF) **Testable:** untestable
16	SCES	R/W	MTL SGF GCL memory Correctable Error Status When this bit is set, it indicates that correctable error is detected at the MTL SGF GCL memory. The Correctable safety interrupt is generated when this bit and GCEIE bit of MTL_ECC_Interrupt_Enable register are set. **Values:** ■ 0x0 (INACTIVE): MTL SGF memory Correctable Error Status not detected ■ 0x1 (ACTIVE): MTL SGF memory Correctable Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_SGF) **Testable:** untestable
15	Reserved_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
14	RPUES	R/W	Rx Parser memory Uncorrectable Error Status ■ 1: Indicates that an uncorrectable error is detected at Rx Parser memory interface. **Values:** ■ 0x0 (INACTIVE): Rx Parser memory Uncorrectable Error Status not detected ■ 0x1 (ACTIVE): Rx Parser memory Uncorrectable Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_FRP_EN **Testable:** untestable

Table 11-443 Fields for Register: MTL_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
13	RPAMS	R/W	MTL Rx Parser memory Address Mismatch Status When this bit is set, it indicates that address mismatch is found for address bus of Rx Parser memory. **Values:** ■ 0x0 (INACTIVE): MTL Rx Parser memory Address Mismatch Status not detected ■ 0x1 (ACTIVE): MTL Rx Parser memory Address Mismatch Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_FRP_EN **Testable:** untestable
12	RPCES	R/W	MTL Rx Parser memory Correctable Error Status When this bit is set, it indicates that correctable error is detected at Rx Parser memory interface. **Values:** ■ 0x0 (INACTIVE): MTL Rx Parser memory Correctable Error Status not detected ■ 0x1 (ACTIVE): MTL Rx Parser memory Correctable Error Status detected **Value After Reset:** 0x0 **Exists:** DWCXG_FRP_EN **Testable:** untestable
11	Reserved_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
10	GUES	R/W	MTL EST GCL memory Uncorrectable Error Status When this bit is set, it indicates that an uncorrectable error is detected at MTL EST GCL memory interface. **Values:** ■ 0x0 (INACTIVE): MTL EST memory Uncorrectable Error Status not detected ■ 0x1 (ACTIVE): MTL EST memory Uncorrectable Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST) **Testable:** untestable

Table 11-443 Fields for Register: MTL_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
9	GAMS	R/W	MTL EST GCL memory Address Mismatch Status When this bit is set, it indicates that address mismatch is found for address bus of MTL EST GCL memory. **Values:** ■ 0x0 (INACTIVE): MTL EST memory Address Mismatch Status not detected ■ 0x1 (ACTIVE): MTL EST memory Address Mismatch Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST) **Testable:** untestable
8	GCES	R/W	MTL EST GCL memory Correctable Error Status When this bit is set, it indicates that correctable error is detected at the MTL EST GCL memory. The Correctable safety interrupt is generated when this bit and GCEIE bit of MTL_ECC_Interrupt_Enable register are set. **Values:** ■ 0x0 (INACTIVE): MTL EST memory Correctable Error Status not detected ■ 0x1 (ACTIVE): MTL EST memory Correctable Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST) **Testable:** untestable
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6	RXUES	R/W	MTL Rx memory Uncorrectable Error Status When set, indicates that an uncorrectable error is detected at the MTL Rx memory interface. **Values:** ■ 0x0 (INACTIVE): MTL Rx memory Uncorrectable Error Status not detected ■ 0x1 (ACTIVE): MTL Rx memory Uncorrectable Error Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-443 Fields for Register: MTL_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
5	RXAMS	R/W	MTL Rx memory Address Mismatch Status When this bit is set, it indicates that address mismatch is found for address bus of the MTL Rx memory. **Values:** ■ 0x0 (INACTIVE): MTL Rx memory Address Mismatch Status not detected ■ 0x1 (ACTIVE): MTL Rx memory Address Mismatch Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
4	RXCES	R/W	MTL Rx memory Correctable Error Status When this bit is set, it indicates that correctable error is detected at the MTL Rx memory. **Values:** ■ 0x0 (INACTIVE): MTL Rx memory correctable Error Status not detected ■ 0x1 (ACTIVE): MTL Rx memory correctable Error Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	TXUES	R/W	MTL Tx memory Uncorrectable Error Status When set, indicates that an uncorrectable error is detected at the MTL Tx memory interface. **Values:** ■ 0x0 (INACTIVE): MTL Tx memory Uncorrectable Error Status not detected ■ 0x1 (ACTIVE): MTL Tx memory Uncorrectable Error Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-443 Fields for Register: MTL_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
1	TXAMS	R/W	MTL Tx memory Address Mismatch Status When this bit is set, it indicates that address mismatch is found for address bus of the MTL Tx memory. **Values:** ■ 0x0 (INACTIVE): MTL Tx memory Address Mismatch Status not detected ■ 0x1 (ACTIVE): MTL Tx memory Address Mismatch Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	TXCES	R/W	MTL Tx memory Correctable Error Status When this bit is set, it indicates that a correctable error is detected at the MTL Tx memory. **Values:** ■ 0x0 (INACTIVE): MTL Tx memory Correctable Error Status not detected ■ 0x1 (ACTIVE): MTL Tx memory Correctable Error Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.41 MTL_ECC_Err_Sts_Rctl

- **Description:** The MTL_ECC_Err_Sts_Rctl register establishes the control for ECC Error status capture.
- **Size:** 32 bits
- **Offset:** 0x10d0
- **Exists:** (DWCXG_ASP_ECC)

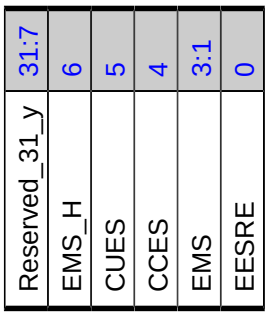


Table 11-444 Fields for Register: MTL_ECC_Err_Sts_Rctl

Bits	Name	Memory Access	Description
31:7	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6	EMS_H	R/W	**Values:** ■ 0x1 (HDMA_MEM): HDMA CSR memory **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
5	CUES	R/W	Clear Uncorrectable Error Status When this bit is set along with EESRE bit, the uncorrectable error address and uncorrectable error count of the memory indicated by EMS field are cleared upon reading. Hardware resets this bit when all the error status values are cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-444 Fields for Register: MTL_ECC_Err_Sts_Rctl (continued)

Bits	Name	Memory Access	Description
4	CCES	R/W	Clear Correctable Error Status When this bit is set along with EESRE bit, correctable error address and correctable error count of the memory indicated by EMS field are cleared upon reading. Hardware resets this bit when all the error status values are cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
3:1	EMS	R/W	MAC/MTL ECC Memory Selection When EESRE bit of this register is set, this field indicates which memory's error status value to be read. Values: ■ 0x0 (TX_MEM): MTL Tx Memory ■ 0x1 (RX_MEM): MTL Rx Memory ■ 0x2 (EST_MEM): MTL GCL Memory ■ 0x3 (RXP_MEM): MTL Rx Parser Memory ■ 0x4 (TSO_MEM): DMA TSO Memory ■ 0x5 (DCACHE_MEM): DMA DCACHE Memory ■ 0x6 (SGF_MEM): MTL SGE GCL Memory ■ 0x7 (RXF_MEM): MTL RXF Memory **Value After Reset:** 0x0 **Exists:** Always
0	EESRE	R/W	MAC/MTL ECC Error Status Read Enable When this bit is set, the error status of the memory selected by EMS is captured as follows: ■ The correctable and uncorrectable error count values are captured in the MTL_ECC_Err_Cnt_Status register. ■ The address locations of correctable and uncorrectable errors are captured in the MTL_ECC_Err_Addr_Status register. Hardware resets this bit when all the status values are captured in the MTL_ECC_Err_Cnt_Status and MTL_ECC_Err_Addr_Status registers. **Values:** ■ 0x0 (DISABLE): MAC/MTL ECC Error Status Read is disabled ■ 0x1 (ENABLE): MAC/MTL ECC Error Status Read is enabled **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.42 MTL_ECC_Err_Addr_Status

- **Description:** The MTL_ECC_Err_Addr_Status register provides the memory addresses for the correctable and uncorrectable errors.
- **Size:** 32 bits
- **Offset:** 0x10d4
- **Exists:** (DWCXG_ASP_ECC)

Table 11-445 Fields for Register: MTL_ECC_Err_Addr_Status

Bits	Name	Memory Access	Description
31:16	EUEAS	R	MAC/MTL ECC Uncorrectable Error Address Status Based on the EMS field of MTL_ECC_Err_Sts_Rctl register, this field holds the respective memory's address locations for which an uncorrectable error or address mismatch is detected. When MEEAO bit of MTL_ECC_Control register is set, this field holds the address of memory for which either an uncorrectable error or an address mismatch is detected most recently. When MEEAO bit of MTL_ECC_Control register is reset, this field holds the address of the memory for which either an uncorrectable error or address mismatch is detected for the first time. **Value After Reset:** 0x0 **Exists:** Always
15:0	ECEAS	R	MAC/MTL ECC Correctable Error Address Status Based on the EMS field of MTL_ECC_Err_Sts_Rctl register, this field holds the respective memory's address locations for which a correctable error is detected. When MEEAO bit of MTL_ECC_Control register is set, this field holds the address of memory for which correctable error or address mismatch is detected most recently. When MEEAO bit of MTL_ECC_Control register is reset, this field holds the address of the memory for which correctable error is detected for the first time. **Value After Reset:** 0x0 **Exists:** Always

11.2.43 MTL_ECC_Err_Cntr_Status

- **Description:** The MTL_ECC_Err_Cntr_Status register provides Error count for Correctable and uncorrectable ECC errors.
- **Size:** 32 bits
- **Offset:** 0x10d8
- **Exists:** (DWCXG_ASP_ECC)

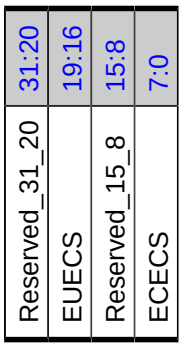
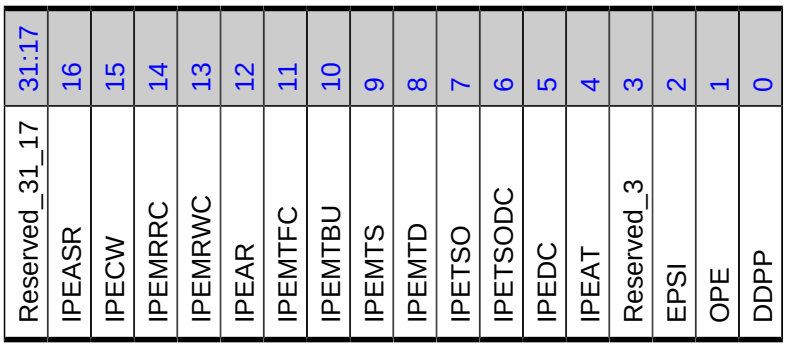


Table 11-446 Fields for Register: MTL_ECC_Err_Cntr_Status

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19:16	EUECS	R	MAC/MTL ECC Uncorrectable Error Counter Status Based on the EMS field of MTL_ECC_Err_Cntr_Rctl register, this field holds the respective memory's uncorrectable error count value. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7:0	ECECS	R	MAC/MTL ECC Correctable Error Counter Status Based on the EMS field of MTL_ECC_Err_Cntr_Rctl register, this field holds the respective memory's correctable error count value. **Value After Reset:** 0x0 **Exists:** Always

11.2.44 MTL_DPP_Control

- **Description:** The MTL_DPP_Control establishes the operating mode of Data Parity protection and error injection.
- **Size:** 32 bits
- **Offset:** 0x10e0
- **Exists:** ((DWCXG_ASP_ALL))



31:17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved_31_17	IPEASR	IPECW	IPEMRRC	IPEMRWC	IPEAR	IPEMTFC	IPEMTBU	IPEMTS	IPEMTD	IPETSO	IPETSODC	IPEDC	IPEAT	Reserved_3	EPSI	OPE	DDPP

Table 11-447 Fields for Register: MTL_DPP_Control

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	IPEASR	R/W	Insert Parity Error in AXI Subordinate Read data path Parity Checker When set to 1, parity/data bit of first valid input parity/data of the AXI Subordinate read data interface is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity error in AXI Subordinate Read data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity error in AXI Subordinate Read data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)) **Testable:** untestable

Table 11-447 Fields for Register: MTL_DPP_Control (continued)

Bits	Name	Memory Access	Description
15	IPECW	R/W	Insert Parity Error in CSR Write data path Parity Checker When set to 1, parity/data bit of first valid input parity/data of the CSR write data interface is flipped. When EPSI bit of MTL_DPP_Control register is set and this bit is set, parity/data bit of first valid input parity/data of the input subordinate parity ports (or at PC-axi) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity error in CSR Write data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity error in CSR Write data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&((DWCXG_ASP_PPE==1)||((DWCXG_ASP_PPE==2)&&(DWCXG_APB_SLAVE||DWCXG_MCI_SLAVE))||DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)) **Testable:** untestable
14	IPEMRRC	R/W	Insert Parity Error in MTL RRC data path Parity Checker When set to 1, parity/data bit of first valid input parity/data of the MAC RRC output data (or at PC7) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity error in MTL RRC data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity error in MTL RRC data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>1)) **Testable:** untestable

Table 11-447 Fields for Register: MTL_DPP_Control (continued)

Bits	Name	Memory Access	Description
13	IPEMRWC	R/W	Insert Parity error in MTL RWC data path Parity Checker When set to 1, parity/data bit of first valid input parity/data of the MTL RWC input data (or at PC6) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity error in MTL RWC data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity error in MTL RWC data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>0)) **Testable:** untestable
12	IPEAR	R/W	Insert Parity error in Insert Parity Error in Application Receive interface data path Parity Checker When set to 1, parity/data bit of first valid input parity/data of Application Receive Interface Parity Checker is flipped. Following checkers based on the system configuration as described below ■ In MTL configuration (DWCXG = 1), parity checker (PC-b) at ARI interface. ■ In AXI configuration (DWCXG = 2), parity checker (PC-a) at AXI manager interface. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity error in Application Receive interface data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity error in Application Receive interface data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>0)) **Testable:** untestable

Table 11-447 Fields for Register: MTL_DPP_Control (continued)

Bits	Name	Memory Access	Description
11	IPEMTFC	R/W	Insert Parity error in MAC TFC data path Parity checker When set to 1, parity/data bit of first valid input parity/data of the MAC TFC input data (or at PC5) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity error in MAC TFC data path Parity Checker is disabled ■ 0x1 (ENABLE): Insert Parity error in MAC TFC data path Parity Checker is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_DPP&&(DWCXG>=0) **Testable:** untestable
10	IPEMTBU	R/W	Insert Parity error in MAC TBU data path Parity checker When set to 1, parity/data bit of first valid input parity/data of the MAC TBU input data on mti interface (or at PC4) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity Error in MAC TBU data path Parity Checker is disabled ■ 0x1 (ENABLE): Insert Parity Error in MAC TBU data path Parity Checker is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_DPP&&(DWCXG>=0) **Testable:** untestable

Table 11-447 Fields for Register: MTL_DPP_Control (continued)

Bits	Name	Memory Access	Description
9	IPEMTS	R/W	Insert Parity Error in MTL TX Status data path Parity checker When set to 1, parity/data bit of first valid input parity/data of the MTL TX Status data on ati interface (or at PC3) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity Error in MTL TX Status data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity Error in MTL TX Status data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>0)&&DWCXG_TXSTSE) **Testable:** untestable
8	IPEMTD	R/W	Insert Parity Error in MTL data path Parity checker When set to 1, parity/data bit of first valid input parity/data of the MTL transmit write controller parity checker (or at PC2) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity Error in MTL data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity Error in MTL data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG>0)) **Testable:** untestable

Table 11-447 Fields for Register: MTL_DPP_Control (continued)

Bits	Name	Memory Access	Description
7	IPETSO	R/W	Insert Parity Error in TSO header (before replacement) data path Parity checker When set to 1, parity/data bit of first valid input parity/data of the DMA TSO parity checker (or at PC1-hdr) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity Error in DMA TSO Header (before replacement) data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity Error in DMA TSO Header (before replacement) data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&DWCXG_TSO_EN&&DWCXG>1) **Testable:** untestable
6	IPETSODC	R/W	Insert Parity Error in TSO or Descriptor Cache data path Parity checker When set to 1, parity/data bit of first valid input parity/data of the DMA TSO parity checker (or at PC-hdr) is flipped. When set to 1, parity/data bit of first valid input data of the DMA TSO parity checker (or at PC-hdr/PC-desc) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity Error in DMA TSO Header or Descriptor Cache (shared same memory with TSO) write data path Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity Error in DMA TSO Header or Descriptor Cache (shared same memory with TSO) write data path Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&DWCXG_TSO_MEM_EN&&DWCXG>1) **Testable:** untestable

Table 11-447 Fields for Register: MTL_DPP_Control (continued)

Bits	Name	Memory Access	Description
5	IPEDC	R/W	Insert Parity Error in Descriptor Cache data path Parity checker When set to 1, parity/data bit of first valid input parity/data of the Descriptor Cache parity checker (or at PC-desc) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Hardware clears this bit after the respective parity bit is flipped. **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&DWCXG_DCACHE_MEM_EN&&DWCXG>1) **Testable:** untestable
4	IPEAT	R/W	Insert Parity Error in Application Transmit Interface Parity checker When set to 1, parity/data bit of first valid input parity/data of the Interface data parity checker is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. Following are the locations of the checkers based on the system configuration, ■ In MTL configuration (DWCXG = 1), parity checker (PC-e) at ATI interface. ■ In AXI configuration (DWCXG = 2), parity checker (PC-e) at AXI manager interface. Hardware clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity Error in Application Transmit Interface Parity checker is disabled ■ 0x1 (ENABLE): Insert Parity Error in Application Transmit Interface Parity checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_ASP_DPP&&(DWCXG_ASP_PPE==1)&&(DWCXG>0)) **Testable:** untestable
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-447 Fields for Register: MTL_DPP_Control (continued)

Bits	Name	Memory Access	Description
2	EPSI	R/W	Enable Parity on Subordinate Interface port When set to 1, enables the parity check for the subordinate interface ports and disables the internal parity generation for the input subordinate data port (axi/apb/mci subordinate port). When set to 0, disables the parity check for the subordinate interface ports and enables the internal parity generation for the input subordinate data port. **Values:** ■ 0x0 (DISABLE): Parity on Register Bus Interface port is disabled ■ 0x1 (ENABLE): Parity on Register Bus Interface port is enabled **Value After Reset:** 0x0 **Exists:** DWCXG_ASP_PPE==1
1	OPE	R/W	Odd Parity Enable ■ 1: Enables odd parity protection on all the external interfaces. ■ 0: Enables even parity protection on all the external interfaces. OPE default value (up on reset) is based on the Enable Odd Parity ports on External Application interface feature selection, which checks if external interface parity must be odd or even). **Values:** ■ 0x0 (DISABLE): Odd Parity is disabled ■ 0x1 (ENABLE): Odd Parity is enabled **Value After Reset:** DWCXG_ASP_PPE_ODD **Exists:** Always
0	DDPP	R/W	Disable Data path Parity Protection When set to 0, enables the parity protection for 25GMAC datapath by generating and checking the parity on 25GMAC datapath. When set to 1, disables the parity protection for 25GMAC datapath. **Values:** ■ 0x0 (DISABLE): Data path Parity Protection is enabled ■ 0x1 (ENABLE): Data path Parity Protection is disabled **Value After Reset:** 0x0 **Exists:** Always

11.2.45 MTL_DPP_ECC_EIC

- **Description:** The MTL_DPP_ECC_EIC establishes the operating mode of ECC/DPP error injection.
- **Size:** 32 bits
- **Offset:** 0x10e4
- **Exists:** ((DWCXG_ASP_ALL||DWCXG_ASP_ECC)&&(DWCXG>=1))

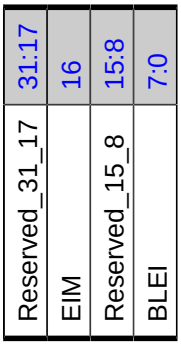


Table 11-448 Fields for Register: MTL_DPP_ECC_EIC

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	EIM	R/W	Error Injection Mode ■ 0: Indicates error injection on data ■ 1: Indicates error injection on ECC/Parity bits (Check that the address error injection mode is disabled). **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-448 Fields for Register: MTL_DPP_ECC_EIC (continued)

Bits	Name	Memory Access	Description
7:0	BLEI	R/W	Bit Location of error injection This field indicates the bit location of DPP/ECC error injection, determination of error in Parity/ECC bits or Data (being protected) depends on the Error Injection Mode (EIM field). Depending on the interface being used for error injection not all bits of this field are valid. Example, for error injection on a 64 bit data interface this field must be programmed to a value between 63 and 0. In case of 2-bit error injection bit 0 is always included in error injection and this field must represent the second bit selection for error injection. If the second bit is programed at bit 0 when 2-bit error injection is enabled, DUT inserts only a 1-bit error at bit 0. **Value After Reset:** 0x0 **Exists:** Always

11.2.46 MTL_SGF_Control

- **Description:** This register controls the operation of Stream-Gate Filtering.
- **Size:** 32 bits
- **Offset:** 0x10e8
- **Exists:** (DWCXG_SGF)

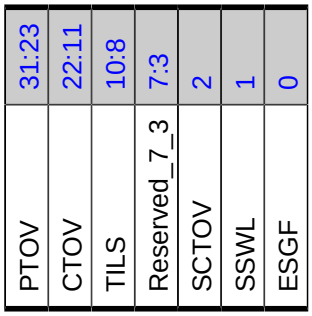


Table 11-449 Fields for Register: MTL_SGF_Control

Bits	Name	Memory Access	Description
31:23	PTOV	R/W	PTP Time Offset Value This field must be programmed with a value of PTP Clock period multiplied by 9 in nanoseconds. This value is needed to avoid transmission overruns at the beginning of the installation of a new GCL due to sync delays between PTP clock and MAC transmitter or Application clock. **Value After Reset:** 0x0 **Exists:** Always
22:11	CTOV	R/W	Current Time Offset Value Provides a 12 bit time offset value in nanoseconds that is added to the current time, to compensate for all the implementation pipeline delays such as the CDC sync delay, buffering delays, data path delays, and so on. **Value After Reset:** 0x0 **Exists:** Always
10:8	TILS	R/W	Time Interval Left Shift Amount This field provides the left shift amount for the programmed Time Interval values used in the Gate Control Lists. ■ 000: No left shift needed (equal to x1ns) ■ 001: Left shift TI by 1 bit (equal to x2ns) ■ 010: Left shift TI by 2 bits (equal to x4ns) ■ ... ■ 111: Left shift TI by 7 bits (equal to x128ns) **Value After Reset:** 0x0 **Exists:** Always

Table 11-449 Fields for Register: MTL_SGF_Control (continued)

Bits	Name	Memory Access	Description
7:3	Reserved_7_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	SCTOV	R/W	Sign bit of SGF CTOV When reset indicates that the SGF CTOV is a negative value. When set that the SGF CTOV is a positive value. **Values:** ■ 0x0 (DISABLE): Indicates that the SGF CTOV is a negative value ■ 0x1 (ENABLE): Indicates that the SGF CTOV is a positive value **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	SSWL	R/W	Switch to software owned list ■ 1: Indicates that the software has programmed the list that it currently owns (SWOL) and the DWC_25gmac must switch to the new list based on the new BTR. DWC_25gmac clears this bit when it switches the SWOL, to indicate the completion of the switch or when a non-recoverable BTR error occurs (BTRL = "11111111" indicates the iterations exceeded the value of 512 and DWC_25gmac is unable to update New BTR to be equal to or greater than Current Time). When BTRE == 1, this bit is cleared. However, SWOL is not updated as the switch is unsuccessful. The write to this bit is effective only when ESGF bit is set to 1 along with it or ESGF bit is previously set to 1. Access restriction applies. Setting 1 sets. Self-cleared. Setting 0 has no effect. **Values:** ■ 0x0 (DISABLE): Switch to S/W owned list is disabled ■ 0x1 (ENABLE): Switch to S/W owned list is triggered **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-449 Fields for Register: MTL_SGF_Control (continued)

Bits	Name	Memory Access	Description
0	ESGF	R/W	Enable Stream-Gate Filtering ■ 0: Indicates that the gate control list processing is halted and all gates are assumed to be in Open state. ■ 1: DWC_25gmac starts processing the gate control lists, only when this bit is 1. During the toggle from 0 to 1, the gate control list processing starts only after the SSWL bit is set. When ECC is selected during the configuration, if an uncorrectable error is detected in the SGF memory, DWC_25gmac resets this bit and disables the SGF function. **Values:** ■ 0x0 (DISABLE): SGF is disabled ■ 0x1 (ENABLE): SGF is enabled **Value After Reset:** 0x0 **Exists:** Always

11.2.47 MTL_SGF_Extended_Control

- **Description:** This register controls the extended operation of Stream-Gate Filtering.
- **Size:** 32 bits
- **Offset:** 0x10ec
- **Exists:** (DWCXG_SGF)

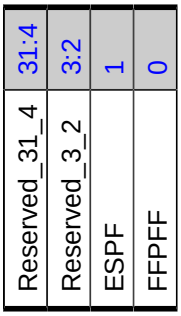


Table 11-450 Fields for Register: MTL_SGF_Extended_Control

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	ESPF	R/W	Enable Stream-Gate filtering of Preemption Frames ■ 1: Stream-Gate filtering is enabled for preemption frames. ■ 0: Stream-Gate filtering is disabled for preemption frames. **Values:** ■ 0x0 (DISABLE): Preemption frame of SGF filter is disabled ■ 0x1 (ENABLE): Preemption frame of SGF filter is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG_FPE) **Testable:** untestable

Table 11-450 Fields for Register: MTL_SGF_Extended_Control (continued)

Bits	Name	Memory Access	Description
0	FFPFF	R/W	Forward Frame due to SGF filter failed ■ 1: Flexible Receive Parser forwards the frame to the residual DMA Channel if stream-gate filter fails. ■ 0: Flexible Receive Parser drops the frame if stream-gate filter fails. **Values:** ■ 0x0 (DISABLE): When reset, Rx Parser drops the frame directly if Stream-Gate filter fails. ■ 0x1 (ENABLE): When set, Rx Parser forwards the frame to residual channel if Stream-Gate filter fails. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.48 MTL_SGF_Status

- **Description:** This register provides Status related to Stream-Gate Filtering.
- **Size:** 32 bits
- **Offset:** 0x10f0
- **Exists:** (DWCXG_SGF)

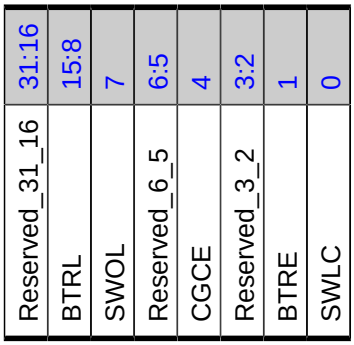


Table 11-451 Fields for Register: MTL_SGF_Status

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:8	BTRL	R	BTR Loop Count This field provides the number of GCL cycle times after which the switch to new SWOL GCL occurs after SSWL is set. It is determined by the value N which satisfies the equation "Current Time less than or equal to New BTR + (N * 4 * New Cycle Time)". N = "11111111" indicates that the iterations exceeded the value of 512 and DWC_25gmac is unable to update New BTR to be equal to or greater than Current Time. Software intervention is needed to update the New BTR. This field is cleared when BTRE field of this register is cleared. **Value After Reset:** 0x0 **Exists:** Always

Table 11-451 Fields for Register: MTL_SGF_Status (continued)

Bits	Name	Memory Access	Description
7	SWOL	R	Software Owned List When this field is 0, indicates that the Gate control list number 0 is owned by software. When this field is 1, indicates that the Gate Control list 1 is owned by the software. Any reads/writes by the software (using indirect access through GCL_Control) is directed to the list indicated by this value. The inverse of this value is treated as HWOL. R/W operations performed by DWC_25gmac are directed to the list pointed by HWOL. Values: ■ 0x0: Gate control list number "0" is owned by software ■ 0x1: Gate control list number "1" is owned by software **Value After Reset:** 0x0 **Exists:** Always
6:5	Reserved_6_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	CGCE	R/W	Constant Gate Control Error This error occurs when the list length (LLR) is 1 and the programmed Time Interval (TI) value after the optional Left Shifting is more than or equal to the Cycle Time (CTR). These programming guidelines imply that Gates are either always Closed or always Open based on the Gate Control values; the same effect can be achieved by other simpler (non TSN) programming mechanisms. As the implementation does not support such a programming mechanism, an error is reported. Access restriction applies. Self-set to 1 on internal event. Setting 1 clears. Setting 0 has no effect. **Values:** ■ 0x0 (INACTIVE): Constant Gate Control Error not detected ■ 0x1 (ACTIVE): Constant Gate Control Error detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-451 Fields for Register: MTL_SGF_Status (continued)

Bits	Name	Memory Access	Description
1	BTRE	R/W	BTR Error When this field is set, indicates a programming error in the BTR of SWOL where the programmed value is less than current time. If the BTRL = "11111111", SWOL is not updated and software must reprogram the BTR to a value greater than current time and set the SSWL bit to re-initiate the switch to SWOL. Otherwise, if the value of BTRL is less than "11111111", SWOL is updated and this field indicates the number of iterations (of + CycleTime) taken by DWC_25gmac to update the BTR to a value greater than Current Time. Access restriction applies. Self-set to 1 on internal event. Setting 1 clears. Setting 0 has no effect. **Values:** ■ 0x0 (INACTIVE): BTR Error not detected ■ 0x1 (ACTIVE): BTR Error detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	SWLC	R/W	Switch to Software Owned List Complete When this field is set, indicates that DWC_25gmac has successfully switched to the SWOL, and the SWOL bit is updated to that effect. This field is cleared when the SSWL field of SGF_Control register transitions from 0 to 1, or on a software write. Access restriction applies. Self-set to 1 on internal event. Setting 1 clears this field. Setting 0 has no effect. **Values:** ■ 0x0 (INACTIVE): Switch to S/W owned list not Complete ■ 0x1 (ACTIVE): Switch to S/W owned list Complete **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.2.49 MTL_SGF_Interrupt_Enable

- **Description:** This register implements the Interrupt Enable bits for the various events that generate an interrupt. Bit positions have a 1- to-1 correspondence with the status bit positions in MTL_SGF_Status register.
- **Size:** 32 bits
- **Offset:** 0x10f4
- **Exists:** (DWCXG_SGF)

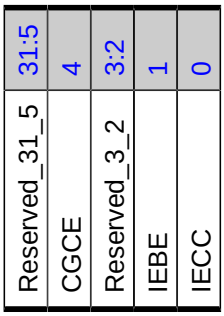


Table 11-452 Fields for Register: MTL_SGF_Interrupt_Enable

Bits	Name	Memory Access	Description
31:5	Reserved_31_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	CGCE	R/W	Interrupt Enable for CGCE ■ 1: Generates interrupt when the Constant Gate Control Error occurs. Error is indicated in the status. ■ 0: This event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Interrupt for CGCE is disabled ■ 0x1 (ENABLE): Interrupt for CGCE is enabled **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-452 Fields for Register: MTL_SGF_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
1	IEBE	R/W	Interrupt Enable for BTR Error ■ 1: Generates interrupt when the BTR Error occurs. Error is indicated in the status. ■ 0: This event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Interrupt for BTR Error is disabled ■ 0x1 (ENABLE): Interrupt for BTR Error is enabled **Value After Reset:** 0x0 **Exists:** Always
0	IECC	R/W	Interrupt Enable for Switch List ■ 1: Generates interrupt when the configuration change is successful and DWC_25gmac has switched to the new list. ■ 0: This event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Interrupt for Switch List is disabled ■ 0x1 (ENABLE): Interrupt for Switch List is enabled **Value After Reset:** 0x0 **Exists:** Always

11.3 DWCXG_MTL_TCQ0 Registers

11.3.1 MTL_TxQ0_Operation_Mode

- **Description:** The Queue 0 Transmit Operation Mode register establishes the Transmit queue operating modes and commands.
- **Size:** 32 bits
- **Offset:** 0x1100
- **Exists:** DWCXG_NUM_TXQ>0

31	30:y	x:16	15:y	x:8	7	6:4	3:2	1	0
EPAT	Reserved_30_y	TQS	Reserved_15_y	Q2TCMAP	Reserved_7	TTC	TXQEN	TSF	FTQ

Table 11-453 Fields for Register: MTL_TxQ0_Operation_Mode

Bits	Name	Memory Access	Description
31	EPAT	R/W	Externally Provided Arrival Time. When this bit is set, ATS uses software provided frame arrival time through Enhanced descriptor/control-word. Otherwise, ATS uses internally captured IEEE 1588 reference system time when particular frame crosses its end on ATI interface. **Value After Reset:** 0x0 **Exists:** DWCXG_ATS
30:y	Reserved_30_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_L256_TXMS + 16

Table 11-453 Fields for Register: MTL_TxQ0_Operation_Mode (continued)

Bits	Name	Memory Access	Description
x:16	TQS	* Varies	Transmit Queue Size. This field indicates the size of the allocated Transmit queues in blocks of 256 bytes. The sixteenth bit is the starting bit of this field. The width of this field depends on the Tx memory size selected in your configuration. For example, if the memory size is 2048, the width of this field is 3 bits: LOG2(2048/256) = LOG2(8) = 3 bits A value of 0 indicates 256 bytes. When this queue is enabled (TXQEN=10), a minimum value of 7 (size = 2048) is recommended so that the queue can store more than a complete max-sized Ethernet packet. This field is reserved and RO in single queue configuration, and its reset value corresponds to the transmit FIFO size **Value After Reset:** (DWCXG_NUM_TXQ>1)?0:((DWCXG_TXFIFO_SIZE/256)-1) **Exists:** Always **Range Variable[x]:** DWCXG_L256_TXMS + 15 **Memory Access:** (DWCXG_NUM_TXQ>1)?\"read-write\":\"read-only\"
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_TCW + 8

Table 11-453 Fields for Register: MTL_TxQ0_Operation_Mode (continued)

Bits	Name	Memory Access	Description
x:8	Q2TCMAP	R/W	Queue to Traffic Class Mapping. This field is used to map this Tx Queue to a traffic class (TC). The actual width of this field depends on the number of traffic classes selected during configuration. ■ 3'b000: TC0 ■ 3'b001: TC1 ■ 3'b010: TC2 ■ 3'b011: TC3 ■ 3'b100: TC4 ■ 3'b101: TC5 ■ 3'b110: TC6 ■ 3'b111: TC7 **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1)&&(!SNPS_RSVDPARAM_50) **Range Variable[x]:** DWCXG_TCW + 7
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	TTC	R/W	Transmit Threshold Control. These bits control the threshold level of the MTL Tx Queue. Transmission starts when the packet size within the MTL Tx Queue is larger than the threshold. In addition, full packets with length less than the threshold are also transmitted. These bits are used only when the TSF bit is reset. ■ 3'b000: 64 ■ 3'b001: Reserved ■ 3'b010: 96 ■ 3'b011: 128 ■ 3'b100: 192 ■ 3'b101: 256 ■ 3'b110: 384 ■ 3'b111: 512 **Value After Reset:** 0x0 **Exists:** Always

Table 11-453 Fields for Register: MTL_TxQ0_Operation_Mode (continued)

Bits	Name	Memory Access	Description
3:2	TXQEN	R/W	Transmit Queue Enable. This field is used to enable/disable the transmit queue. ■ 2'b00: Not enabled ■ 2'b01: Audio Video Bridging ■ 2'b01: Reserved ■ 2'b10: Enabled ■ 2'b11: Reserved The queue is enabled by default in single queue configuration. Therefore, this field is reserved when the number of transmit queues selected is one and has a reset value of 2'b10. In multiple Tx queues configuration, as the queue is disabled by default, enable the Tx Queue by programming this field in the AVB mode or default mode as desired. **Value After Reset:** (DWCXG_NUM_TXQ>1)?0x0:0x2 **Exists:** (DWCXG_NUM_TXQ>1)
1	TSF	R/W	Transmit Store and Forward. When this bit is set, the transmission starts when a full packet resides in the MTL Tx Queue. When this bit is set, the values specified in the TTC field are ignored. This bit must be changed only when the transmission is stopped. **Value After Reset:** 0x0 **Exists:** Always
0	FTQ	R/W	Flush Transmit Queue. When this bit is set, the Tx Queue Controller logic is reset to its default values. Therefore, all the data in the Tx queue is lost or flushed. This bit is internally reset when the flushing operation is complete. You must not write to this register until this bit is reset. The data which is already accepted by the MAC transmitter is not flushed. It is scheduled for transmission and results in underflow and runt packet transmission. **Note:** The flush operation is complete only when the Tx queue is empty. To complete this flush operation, the PHY Tx clock (clk_tx_312pt5_i) must be active. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.3.2 MTL_TxQ0_Underflow

- **Description:** The Queue 0 Underflow Counter register contains the number of packets aborted because of Transmit queue underflow.
- **Size:** 32 bits
- **Offset:** 0x1104
- **Exists:** DWCXG_NUM_TXQ>0

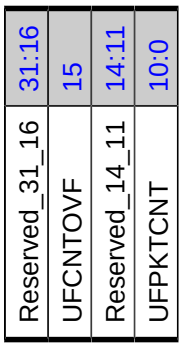


Table 11-454 Fields for Register: MTL_TxQ0_Underflow

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15	UFCNTOVF	R	Overflow Bit for Underflow Packet Counter. This bit is set every time the Tx Queue Underflow Packet Counter field overflows, that is, it has crossed the maximum count. In such a scenario, the overflow packet counter is reset to all-zeros and this bit indicates that the rollover happened. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14:11	Reserved_14_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-454 Fields for Register: MTL_TxQ0_Underflow (continued)

Bits	Name	Memory Access	Description
10:0	UFPKTCNT	R	Underflow Packet Counter. This field indicates the number of packets aborted by the controller because of Transmit Queue Underflow. This counter is incremented each time the MAC aborts an outgoing packet because of underflow. The counter is reset when this register is read with LSB of the Byte Enable for CSR access signal at 1'b1 (mci_be_i[0] at 1'b1). **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.3.3 MTL_TxQ0_Debug

- **Description:** The Queue 0 Transmit Debug register gives the debug status of various blocks related to the Transmit Queue 0 and Traffic Class 0.
- **Size:** 32 bits
- **Offset:** 0x1108
- **Exists:** DWCXG_NUM_TXQ>0

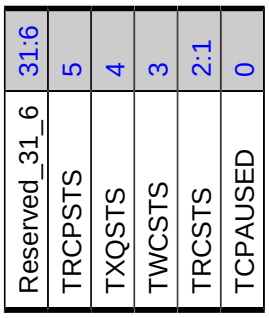


Table 11-455 Fields for Register: MTL_TxQ0_Debug

Bits	Name	Memory Access	Description
31:6	Reserved_31_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5	TRCPSTS	R	MTL TRC Pending Status When this bit is high, it indicates that the MTL Tx Read Controller (TRC) has a pending status from MAC. **Value After Reset:** 0x0 **Exists:** Always
4	TXQSTS	R	MTL Tx Queue Not Empty Status. When this bit is high, it indicates that the MTL Tx Queue is not empty and some data is left for transmission. **Value After Reset:** 0x0 **Exists:** Always
3	TWCSTS	R	MTL Tx Queue Write Controller Status. When high, this bit indicates that the MTL Tx Queue Write Controller is active, and it is transferring the data to the Tx Queue. **Value After Reset:** 0x0 **Exists:** Always

Table 11-455 Fields for Register: MTL_TxQ0_Debug (continued)

Bits	Name	Memory Access	Description
2:1	TRCSTS	R	MTL Tx Queue Read Controller Status. This field indicates the state of the Tx Queue Read Controller: ■ 2'b00: Idle state ■ 2'b01: Read state (transferring data to the MAC transmitter) ■ 2'b10: Waiting for pending Tx Status from the MAC transmitter ■ 2'b11: Flushing the Tx queue because of the Packet Terminate request from the MAC **Value After Reset:** 0x0 **Exists:** Always
0	TCPAUSED	R	Traffic Class in Pause. When this bit is high and the Rx flow control is enabled, it indicates that the Traffic Class 0 is in the Pause condition because of the following: ■ Reception of the PFC packet for the priorities assigned to the Traffic Class 0 when PFC is enabled ■ Reception of IEEE 802.3-2018 Pause packet when Priority Based Flow Control is disabled **Value After Reset:** 0x0 **Exists:** Always

11.3.4 MTL_TC0_ETS_Control

- **Description:** This register configures the transmission selection algorithm, for Traffic Class 0.This register is not available in single traffic class configuration.
- **Size:** 32 bits
- **Offset:** 0x1110
- **Exists:** DWCXG_NUM_TC>1

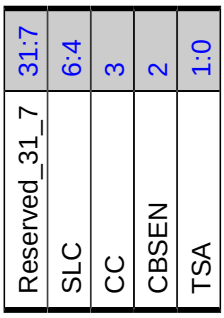


Table 11-456 Fields for Register: MTL_TC0_ETS_Control

Bits	Name	Memory Access	Description
31:7	Reserved_31_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	SLC	R/W	Slot Count. If the credit-based shaper algorithm is enabled, the software can program the number of slots (of 125 us duration) over which the average transmitted bits per slot, provided in the MTL_TC(#n)_ETS_Status register, need to be computed for TC(#n). The encoding is as follows: ■ 3'b000: 1 Slot ■ 3'b001: 2 Slots ■ 3'b010: 4 Slots ■ 3'b011: 8 Slots ■ 3'b100: 16 Slots ■ 3'b101-3'b111: Reserved **Value After Reset:** 0x0 **Exists:** (0)

Table 11-456 Fields for Register: MTL_TC0_ETS_Control (continued)

Bits	Name	Memory Access	Description
3	CC	R/W	Credit Control. When this bit is set, the accumulated credit parameter in the credit-based shaper algorithm logic is not reset to zero when there is positive credit and no packet to transmit in TC(#n). The credit accumulates even when there is no packet waiting in TC(#n) and another TC is transmitting. When this bit is reset, the accumulated credit parameter in the credit-based shaper algorithm logic is set to zero when there is positive credit and no packet to transmit in TC(#n). When there is no packet waiting in TC(#n) and other TC is transmitting, no credit is accumulated. **Value After Reset:** 0x0 **Exists:** (0)
2	CBSEN	R	Credit Based Shaper Enable This field indicates that TSA field of this register is enabled for CBS **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1:0	TSA	R/W	Transmission Selection Algorithm. This field is used to assign a transmission selection algorithm for this traffic class. ■ 2'b00: Strict priority ■ 2'b01: CBS ■ 2'b10: ETS ■ 2'b11: ATS **Value After Reset:** 0x0 **Exists:** Always

11.3.5 MTL_TC0_ETS_Status

- **Description:** This register contains the information of average bits transmitted over a slot interval of 10 million bit times, for Traffic Class 0.
- **Size:** 32 bits
- **Offset:** 0x1114
- **Exists:** (DWCXG_DCB_EN&&DWCXG_NUM_TC>1)

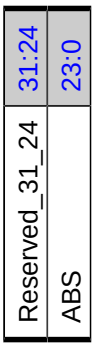


Table 11-457 Fields for Register: MTL_TC0_ETS_Status

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:0	ABS	R	Average Bits per Slot. This field contains the average transmitted bits per slot computed over every slot of 10 million bit times (1ms in 10G mode, 10ms in 1G mode). The maximum value is 0x989680. When AVB mode is enabled, this field contains the average transmitted bits over number of slots indicated by SLC field of MTL_TC0_ETS_Control register. **Value After Reset:** 0x0 **Exists:** Always

11.3.6 MTL_TC0_Quantum_Weight

- **Description:** The TC1 Quantum or Weights register controls the average traffic transmitted in Traffic Class 0. For TCn, where n = 1 to 7, this register is not available when the number of traffic classes selected is less than n + 1
  This register is not available in corresponding Queues 8-15 registers.
- **Size:** 32 bits
- **Offset:** 0x1118
- **Exists:** (DWCXG_NUM_TC>1)

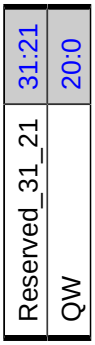


Table 11-458 Fields for Register: MTL_TC0_Quantum_Weight

Bits	Name	Memory Access	Description
31:21	Reserved_31_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
20:0	QW	R/W	idleSlopeCredit, Quantum or Weights. ■ **idleSlopeCredit** ❑ When this TC is in AVB mode, this field contains the idleSlopeCredit value required for the credit-based shaper algorithm for Traffic Class 0. This is the rate of change of credit in bits per cycle (3.2 ns for 2.5/10 Gbps; 8 ns for 1000 Mbps) when the credit is increasing. The software must program this field with computed credit in bits per cycle scaled by 1024. The maximum value is portTransmitRate, that is, 0x2000 in 1000/2500 Mbps mode and 0x8000 in 10 Gbps mode. Bits[20:16] must be written to zero. This field is not valid for Traffic Class 0. ■ **Quantum** ❑ When the TSA is set to ETS and ETSALG is set to DWRR algorithm for Traffic Class 1 traffic, this field contains the quantum value in bytes to be added to credit during every queue scanning cycle. The maximum value is 0x1312D0 bytes. ■ **Weights**

Table 11-458 Fields for Register: MTL_TC0_Quantum_Weight (continued)

Bits	Name	Memory Access	Description
			❑ When TSA is set to ETS and ETSALG is set to WFQ algorithm for Traffic Class1 traffic, this field contains the weight for this queue. The maximum value is 0x3FFF where weight of 0 indicates 100% bandwidth. Bits [20:14] must be written to zero. ❑ When TSA is set to ETS and ETSALG is set to WRR algorithm for Traffic Class 1 traffic, this field contains the weight for this queue. The maximum value is 0x64. Bits[20:7] must be written to zero. This field is reserved and RO when the number of traffic classes selected is less than two. This field is reserved in corresponding Queues 8-15 register. **Value After Reset:** 0x0 **Exists:** Always

11.3.7 MTL_TxQ(#i)_Bucket_Empty_2_Full (for i = 0; i <= min(7,DWCXG_NUM_TXQ-1))

- **Description:** The MTL Queue 0 ATS bucket empty 2 full register controls the Asynchronous Traffic shaper scheduler and transmission selection operation.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1128
- **Exists:** DWCXG_ATS

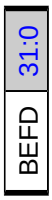


Table 11-459 Fields for Register: MTL_TxQ(#i)_Bucket_Empty_2_Full (for i = 0; i <= min(7,DWCXG_NUM_TXQ-1))

Bits	Name	Memory Access	Description
31:0	BEFD	R/W	Bucket Empty to Full Duration. This field is programmed with time duration that is required for token bucket to fill from empty to full. Maximum bucket size per queue is its own size (in bits), hence BEFD should not be greater than Bucket Size/CIR. **Value After Reset:** 0x0 **Exists:** Always

11.3.8 MTL_TxQ(#i)_Committed_Info_Rate (for i = 0; i <= min(7,DWCXG_NUM_TXQ-1))

- **Description:** The MTL Queue 0 ATS committed info rate register controls the Asynchronous Traffic Shaper scheduler & transmission selection operation.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x112C
- **Exists:** DWCXG_ATS

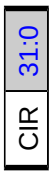
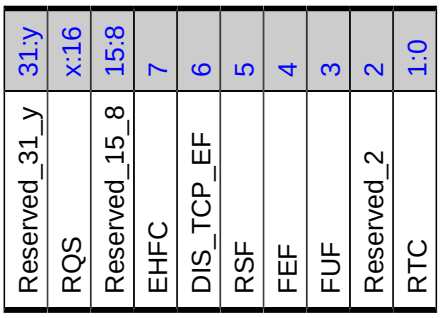


Table 11-460 Fields for Register: MTL_TxQ(#i)_Committed_Info_Rate (for i = 0; i <= min(7,DWCXG_NUM_TXQ-1))

Bits	Name	Memory Access	Description
31:0	CIR	R/W	Committed Information Rate in bits per second. The committed information rate of the ATS scheduler instance, in bits per second. It defines the rate at which the token bucket is refilled with tokens until the maximum token capacity of the token bucket is reached. CIR has granularity of 1% of the selected speed mode. CIR of all the ATS enabled queues must not exceed the data rate. **Value After Reset:** 0x0 **Exists:** Always

11.3.9 MTL_RxQ0_Operation_Mode

- **Description:** The Queue 0 Receive Operation Mode register establishes the Receive queue 0 operating modes and command.
- **Size:** 32 bits
- **Offset:** 0x1140
- **Exists:** DWCXG_NUM_RXQ>0



31:y	x:16	15:8	7	6	5	4	3	2	1:0
Reserved_31_y	RQS	Reserved_15_8	EHFC	DIS_TCP_EF	RSF	FEF	FUF	Reserved_2	RTC

Table 11-461 Fields for Register: MTL_RxQ0_Operation_Mode

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_L256_RXMS + 16
x:16	RQS	* Varies	Receive Queue Size. This field indicates the size of the allocated Receive queues in blocks of 256 bytes. The sixteenth bit is the starting bit of this field. The width of this field depends on the Rx memory size selected in your configuration. For example, if the memory size is 2048, the width of this field is 3 bits: LOG2(2048/256) = LOG2(8) = 3 bits A value of 0 indicates 256 bytes. This field is reserved and RO in single queue configuration, and its reset value corresponds to the receive FIFO size. When this queue is used, a minimum value of 7 (size = 2048) is recommended so that the queue can store more than a complete max-sized Ethernet packet. **Value After Reset:** (DWCXG_NUM_RXQ>1)?0: ((DWCXG_RXFIFO_SIZE/256)-1) **Exists:** Always **Range Variable[x]:** DWCXG_L256_RXMS + 15 **Memory Access:** (DWCXG_NUM_RXQ>1)?\"read-write\":\"read-only\"

Table 11-461 Fields for Register: MTL_RxQ0_Operation_Mode (continued)

Bits	Name	Memory Access	Description
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7	EHFC	R/W	Enable Hardware Flow Control. When this bit is set, the flow control signal operation, based on the fill-level of Rx queue, is enabled. When reset, the flow control operation is disabled. This bit is not used (reserved and always reset) when the Rx queue is less than 4 KB. **Value After Reset:** 0x0 **Exists:** (DWCXG_RXFIFO_SIZE>=4096)
6	DIS_TCP_EF	R/W	Disable Dropping of TCP/IP Checksum Error Packets. ■ When this bit is set, the MAC does not drop the packets which only have the errors detected by the Receive Checksum Offload engine. Such packets have errors only in the encapsulated payload. There are no errors (including FCS error) in the Ethernet packet received by the MAC. ■ When this bit is reset, all error packets are dropped if the FEF bit is reset. **Value After Reset:** 0x0 **Exists:** (DWCXG_RX_COE)
5	RSF	R/W	Receive Queue Store and Forward. ■ 1: DWC_25gmac reads a packet from the Rx queue only after the complete packet has been written to it, ignoring the RTC field of this register. ■ 0: Rx queue operates in the Threshold (cut-through) mode, subject to the threshold specified by the RTC field of this register. **Value After Reset:** 0x0 **Exists:** Always

Table 11-461 Fields for Register: MTL_RxQ0_Operation_Mode (continued)

Bits	Name	Memory Access	Description
4	FEF	R/W	Forward Error Packets. ■ 0: Rx queue drops packets with error status (CRC error, GMII_ER, watchdog timeout, or overflow). However, if the start byte (write) pointer of a packet is already transferred to the read controller side (in Threshold mode), the packet is not dropped. ■ 1: All packets except the runt error packets are forwarded to the application or DMA. If the RSF bit is set and the Rx queue overflows when a partial packet is written, the packet is dropped irrespective of the setting of this bit. However, if the RSF bit is reset and the Rx queue overflows when a partial packet is written, a partial packet might be forwarded to the application or DMA. **Value After Reset:** 0x0 **Exists:** Always
3	FUF	R/W	Forward Undersized Good Packets. ■ 1: Rx queue forwards the undersized good packets (with no Invalid-code/GMII/CRC/Length errors and length less than 64 bytes at Rx (X)GMII interface), including pad-bytes and CRC. ■ 0: Rx queue drops all packets of length less than 64 bytes. **Value After Reset:** 0x0 **Exists:** Always
2	Reserved_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-461 Fields for Register: MTL_RxQ0_Operation_Mode (continued)

Bits	Name	Memory Access	Description
1:0	RTC	R/W	Receive Queue Threshold Control. These bits control the threshold level of the MTL Rx queue (in bytes): ■ 2'b00: 64 ■ 2'b01: Reserved ■ 2'b10: 96 ■ 2'b11: 128 The received packet is transferred to the application or DMA when the packet size within the MTL Rx queue is larger than the threshold. In addition, full packets with length less than the threshold are automatically transferred. The value of 11 is not applicable if the size of the configured Rx Queue is 128 bytes. This field is valid only when the RSF bit is zero. This field is ignored when the RSF bit is set to 1. **Value After Reset:** 0x0 **Exists:** Always

11.3.10 MTL_RxQ0_Missed_Pkt_Overflow_Cnt

- **Description:** The Queue 0 Overflow Counter register contains the counter for packets missed because of Receive queue packet flush and packets discarded because of Receive queue overflow.
- **Size:** 32 bits
- **Offset:** 0x1144
- **Exists:** DWCXG_NUM_RXQ>0

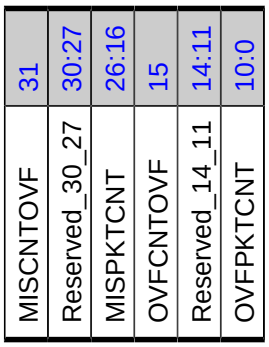


Table 11-462 Fields for Register: MTL_RxQ0_Missed_Pkt_Overflow_Cnt

Bits	Name	Memory Access	Description
31	MISCNTOVF	R	Missed Packet Counter Overflow Bit. When set, this bit indicates that the MISPKTCNT field crossed the maximum limit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30:27	Reserved_30_27	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
26:16	MISPKTCNT	R	Missed Packet Counter. This field indicates the number of packets missed by DWC_25gmac because the host asserted packet-flush for this queue. This counter is incremented each time a packet is flushed for this queue. This counter is reset when this register's bits[23:16] is read by the software. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-462 Fields for Register: MTL_RxQ0_Missed_Pkt_Overflow_Cnt (continued)

Bits	Name	Memory Access	Description
15	OVFCNTOVF	R	Overflow Counter Overflow Bit. When set, this bit indicates that OVFPKTCNT field crossed the maximum limit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14:11	Reserved_14_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
10:0	OVFPKTCNT	R	Overflow Packet Counter. This field indicates the number of packets discarded by DWC_25gmac because of Receive queue overflow. This counter is incremented each time DWC_25gmac discards an incoming packet because of overflow. This counter is reset when this register bits[7:0] is read by software. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.3.11 MTL_RxQ0_Debug

- **Description:** The Queue 0 Receive Debug register gives the debug status of various blocks related to the Receive queue.
- **Size:** 32 bits
- **Offset:** 0x1148
- **Exists:** DWCXG_NUM_RXQ>0

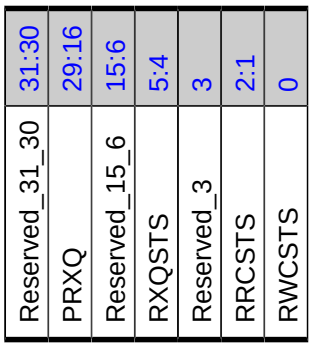


Table 11-463 Fields for Register: MTL_RxQ0_Debug

Bits	Name	Memory Access	Description
31:30	Reserved_31_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29:16	PRXQ	R	Number of Packets in Receive Queue. This field indicates the current number of packets in the Rx Queue. The theoretical maximum value for this field is 256KB/16B = 16K Packets, that is, Max_Queue_Size/Min_Packet_Size. **Value After Reset:** 0x0 **Exists:** Always
15:6	Reserved_15_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:4	RXQSTS	R	MTL Rx Queue Fill-Level Status. This field gives the status of the fill-level of the Rx Queue: ■ 2'b00: Rx Queue empty ■ 2'b01: Rx Queue fill-level below flow-control deactivate threshold ■ 2'b10: Rx Queue fill-level above flow-control activate threshold ■ 2'b11: Rx Queue full **Value After Reset:** 0x0 **Exists:** Always

Table 11-463 Fields for Register: MTL_RxQ0_Debug (continued)

Bits	Name	Memory Access	Description
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2:1	RRCSTS	R	MTL Rx Queue Read Controller State. This field gives the state of the Rx queue Read controller: ■ 2'b00: Idle state ■ 2'b01: Reading packet data ■ 2'b10: Reading packet status (or timestamp) ■ 2'b11: Flushing the packet data and status **Value After Reset:** 0x0 **Exists:** Always
0	RWCSTS	R	MTL Rx Queue Write Controller Active Status. When high, this bit indicates that the MTL Rx queue Write controller is active, and it is transferring a received packet to the Rx Queue. **Value After Reset:** 0x0 **Exists:** Always

11.3.12 MTL_RxQ0_Control

- **Description:** The Queue 0 Receive Control register controls the receive arbitration and passing of received packets to the application.
- **Size:** 32 bits
- **Offset:** 0x114c
- **Exists:** DWCXG_NUM_RXQ>1

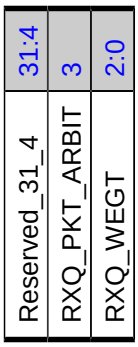


Table 11-464 Fields for Register: MTL_RxQ0_Control

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	RXQ_PKT_ARBIT	R/W	Receive Queue Packet Arbitration Control. ■ 1: MTL enables the packet-level arbitration. ■ 0: MTL enables the PBL based arbitration. **Value After Reset:** DWCXG_MTL_SUBSYS **Exists:** ((DWCXG_MTL_SUBSYS)||(! DWCXG_MTL_SUBSYS&&DWCXG_NUM_DMA_RX_CH> 1))
2:0	RXQ_WEGT	R/W	Receive Queue Weight. This field indicates the weight assigned to the Rx Queue 0. The weight is used as the number of continuous PBL requests or contiguous packets (depending on the RXQ_PKT_ARBIT) allocated to the queue in one arbitration cycle. **Value After Reset:** 0x0 **Exists:** Always

11.3.13 MTL_RxQ0_Flow_Control

- **Description:** This register controls the activation and de-activation threshold in the queue for triggering the generation of Flow Control packets by MAC transmitter.
- **Size:** 32 bits
- **Offset:** 0x1150
- **Exists:** (DWCXG_RXFIFO_SIZE>=4096&&DWCXG_NUM_RXQ>0)

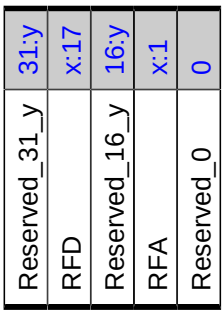


Table 11-465 Fields for Register: MTL_RxQ0_Flow_Control

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_FLCW + 17
x:17	RFD	R/W	Threshold for deactivating Flow Control. These bits control the threshold (fill-level of RxQ0) at which the flow control is de-asserted once it is activated. ■ 6'b000000: RQS value minus 1 KB, that is, FULL - 1024 bytes ■ 6'b000001: RQS value minus 1.5 KB, that is, FULL - 1536 bytes ■ 6'b000010: RQS value minus 2 KB, that is, FULL - 2048 bytes. ■ ... ■ 6'b111111: RQS value minus 32.5 KB, that is, FULL - 33280 bytes The de-assertion is effective only after flow control is asserted as per the setting of RFA field. Note that the value to be programmed must be such that the threshold is positive number. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_FLCW + 16

Table 11-465 Fields for Register: MTL_RxQ0_Flow_Control (continued)

Bits	Name	Memory Access	Description
16:y	Reserved_16_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_FLCW + 1
x:1	RFA	R/W	Threshold for activating Flow Control. These bits control the threshold (fill-level of RxQ0) at which the flow control is activated ■ 6'b000000: RQS value minus 1 KB, that is, FULL - 1024 bytes ■ 6'b000001: RQS value minus 1.5 KB, that is, FULL - 1536 bytes ■ 6'b000010: RQS value minus 2 KB, that is, FULL - 2048 bytes. ■ ... ■ 6'b111111: RQS value minus 32.5 KB, that is, FULL - 33280 bytes The flow control is triggered when EHFC=1 and the queue size (RQS value in MTL_RxQ0_Operation Mode register) is equal to or more than 4KB. Note that the value to be programmed in RFA must be such that the threshold is positive number. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_FLCW
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.3.14 MTL_Q0_Interrupt_Enable

- **Description:** This register contains the interrupt enable bits for Traffic class/Queue 0.
- **Size:** 32 bits
- **Offset:** 0x1170
- **Exists:** Always

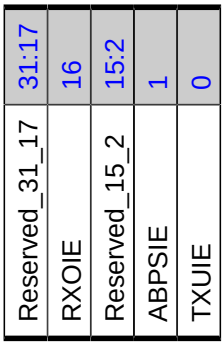


Table 11-466 Fields for Register: MTL_Q0_Interrupt_Enable

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	RXOIE	R/W	Receive Queue Overflow Interrupt Enable. ■ 1: Receive Queue Overflow interrupt is enabled. ■ 0: Receive Queue Overflow interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>0)
15:2	Reserved_15_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	ABPSIE	R/W	Average Bits Per Slot Interrupt Enable. When this bit is set, average bits per slot interrupt is enabled for Traffic Class 0. An interrupt is generated when ABPSIS is set for this traffic class. This bit is reserved and RO when the number of traffic classes selected is less than 0. This field in the corresponding Queues 8 - 15 registers is always reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1)

Table 11-466 Fields for Register: MTL_Q0_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
0	TXUIE	R/W	Transmit Queue Underflow Interrupt Enable. ■ 1: Transmit Queue Underflow interrupt is enabled. ■ 0: Transmit Queue Underflow interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TXQ>0)

11.3.15 MTL_Q0_Interrupt_Status

- **Description:** This register contains the interrupt status bits for Traffic class/Queue 0.
- **Size:** 32 bits
- **Offset:** 0x1174
- **Exists:** Always

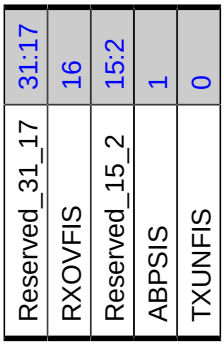


Table 11-467 Fields for Register: MTL_Q0_Interrupt_Status

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
16	RXOVFIS	R/W	Receive Queue Overflow Interrupt Status. This bit indicates that the Receive Queue had an Overflow during packet reception. If partial packet is transferred to the application, the overflow status is set in the corresponding Rx status. This bit is cleared when the application writes a 1 to this bit. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>0) **Testable:** untestable
15:2	Reserved_15_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-467 Fields for Register: MTL_Q0_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
1	ABPSIS	R/W	Average Bits Per Slot Interrupt Status. When set, this bit indicates that the MAC has updated the ABS value for this traffic class. This bit is cleared when the application writes a 1 to this bit. This bit is reserved and RO when the number of traffic classes selected is less than 0. This field in corresponding Queues 8 - 15 registers is reserved and RO. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1) **Testable:** untestable
0	TXUNFIS	R/W	Transmit Queue Underflow Interrupt Status. This bit indicates that the Transmit Queue had an Underflow during packet transmission. Transmission is suspended and an Underflow Error TDES3[2] is set. This bit is cleared when the application writes a 1 to this bit. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TXQ>0) **Testable:** untestable

11.4 DWCXG_MTL_TCQ(#i) Registers

11.4.1 MTL_TxQ1_Operation_Mode

- **Description:** The Queue 1 Transmit Operation Mode register establishes the Transmit queue operating modes and commands.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1100
- **Exists:** DWCXG_NUM_TXQ>1

31	30:y	x:16	15:y	x:8	7	6:4	3:2	1	0
EPAT	Reserved_30_y	TQS	Reserved_15_y	Q2TCMAP	Reserved_7	TTC	TXQEN	TSF	FTQ

Table 11-468 Fields for Register: MTL_TxQ1_Operation_Mode

Bits	Name	Memory Access	Description
31	EPAT	R/W	Externally Provided Arrival Time. When this bit is set, ATS uses software provided frame arrival time through Enhanced descriptor/control-word. Otherwise, ATS uses internally captured IEEE 1588 reference system time when particular frame crosses its end on ATI interface. **Value After Reset:** 0x0 **Exists:** DWCXG_ATS
30:y	Reserved_30_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_L256_TXMS + 16

Table 11-468 Fields for Register: MTL_TxQ1_Operation_Mode (continued)

Bits	Name	Memory Access	Description
x:16	TQS	R/W	Transmit Queue Size. This field indicates the size of the allocated Transmit queues in blocks of 256 bytes. The sixteenth bit is the starting bit of this field. The width of this field depends on the Tx memory size selected in your configuration. For example, if the memory size is 2048, the width of this field is 3 bits: LOG2(2048/256) = LOG2(8) = 3 bits A value of 0 indicates 256 bytes. When this queue is enabled (TXQEN=10), a minimum value of 7 (size = 2048) is recommended so that the queue can store more than a complete max-sized Ethernet packet. This field is reserved and RO in single queue configuration, and its reset value corresponds to the transmit FIFO size **Value After Reset:** (DWCXG_NUM_TXQ>1)?0:((DWCXG_TXFIFO_SIZE/256)-1) **Exists:** Always **Range Variable[x]:** DWCXG_L256_TXMS + 15
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_TCW + 8
x:8	Q2TCMAP	R/W	Queue to Traffic Class Mapping. This field is used to map this Tx Queue to a traffic class (TC). The actual width of this field depends on the number of traffic classes selected during configuration. ■ 3'b000: TC0 ■ 3'b001: TC1 ■ 3'b010: TC2 ■ 3'b011: TC3 ■ 3'b100: TC4 ■ 3'b101: TC5 ■ 3'b110: TC6 ■ 3'b111: TC7 **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1)&&(!SNPS_RSVDPARAM_50) **Range Variable[x]:** DWCXG_TCW + 7

Table 11-468 Fields for Register: MTL_TxQ1_Operation_Mode (continued)

Bits	Name	Memory Access	Description
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	TTC	R/W	Transmit Threshold Control. These bits control the threshold level of the MTL Tx Queue. Transmission starts when the packet size within the MTL Tx Queue is larger than the threshold. In addition, full packets with length less than the threshold are also transmitted. These bits are used only when the TSF bit is reset. ■ 3'b000: 64 ■ 3'b001: Reserved ■ 3'b010: 96 ■ 3'b011: 128 ■ 3'b100: 192 ■ 3'b101: 256 ■ 3'b110: 384 ■ 3'b111: 512 **Value After Reset:** 0x0 **Exists:** Always
3:2	TXQEN	R/W	Transmit Queue Enable. This field is used to enable/disable the transmit queue. ■ 2'b00: Not enabled ■ 2'b01: Audio Video Bridging ■ 2'b01: Reserved ■ 2'b10: Enabled ■ 2'b11: Reserved The queue is enabled by default in single queue configuration. Therefore, this field is reserved when the number of transmit queues selected is one and has a reset value of 2'b10. In multiple Tx queues configuration, as the queue is disabled by default, enable the Tx Queue by programming this field in the AVB mode or default mode as desired. **Value After Reset:** (DWCXG_NUM_TXQ>1)?0x0:0x2 **Exists:** Always

Table 11-468 Fields for Register: MTL_TxQ1_Operation_Mode (continued)

Bits	Name	Memory Access	Description
1	TSF	R/W	Transmit Store and Forward. When this bit is set, the transmission starts when a full packet resides in the MTL Tx Queue. When this bit is set, the values specified in the TTC field are ignored. This bit must be changed only when the transmission is stopped. **Value After Reset:** 0x0 **Exists:** Always
0	FTQ	R/W	Flush Transmit Queue. When this bit is set, the Tx Queue Controller logic is reset to its default values. Therefore, all the data in the Tx queue is lost or flushed. This bit is internally reset when the flushing operation is complete. You must not write to this register until this bit is reset. The data which is already accepted by the MAC transmitter is not flushed. It is scheduled for transmission and results in underflow and runt packet transmission. **Note:** The flush operation is complete only when the Tx queue is empty. To complete this flush operation, the PHY Tx clock (clk_tx_312pt5_i) must be active. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.4.2 MTL_TxQ1_Underflow

- **Description:** The Queue 1 Underflow Counter register contains the number of packets aborted because of Transmit queue underflow.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1104
- **Exists:** DWCXG_NUM_TXQ>1

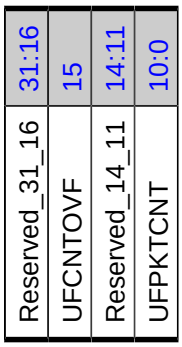


Table 11-469 Fields for Register: MTL_TxQ1_Underflow

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15	UFCNTOVF	R	Overflow Bit for Underflow Packet Counter. This bit is set every time the Tx Queue Underflow Packet Counter field overflows, that is, it has crossed the maximum count. In such a scenario, the overflow packet counter is reset to all-zeros and this bit indicates that the rollover happened. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14:11	Reserved_14_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-469 Fields for Register: MTL_TxQ1_Underflow (continued)

Bits	Name	Memory Access	Description
10:0	UFPKTCNT	R	Underflow Packet Counter. This field indicates the number of packets aborted by the controller because of Transmit Queue Underflow. This counter is incremented each time the MAC aborts an outgoing packet because of underflow. The counter is reset when this register is read with LSB of the Byte Enable for CSR access signal at 1'b1 (mci_be_i[0] at 1'b1). **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.4.3 MTL_TxQ1_Debug

- **Description:** The Queue 1 Transmit Debug register gives the debug status of various blocks related to the Transmit Queue 1 and Traffic Class 1.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1108
- **Exists:** DWCXG_NUM_TXQ>1

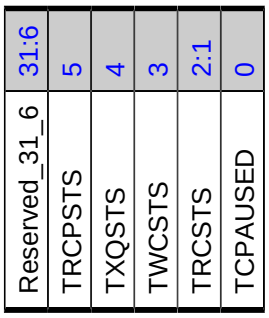


Table 11-470 Fields for Register: MTL_TxQ1_Debug

Bits	Name	Memory Access	Description
31:6	Reserved_31_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5	TRCPSTS	R	MTL TRC Pending Status When this bit is high, it indicates that the MTL Tx Read Controller (TRC) has a pending status from MAC. **Value After Reset:** 0x0 **Exists:** Always
4	TXQSTS	R	MTL Tx Queue Not Empty Status. When this bit is high, it indicates that the MTL Tx Queue is not empty and some data is left for transmission. **Value After Reset:** 0x0 **Exists:** Always
3	TWCSTS	R	MTL Tx Queue Write Controller Status. When high, this bit indicates that the MTL Tx Queue Write Controller is active, and it is transferring the data to the Tx Queue. **Value After Reset:** 0x0 **Exists:** Always

Table 11-470 Fields for Register: MTL_TxQ1_Debug (continued)

Bits	Name	Memory Access	Description
2:1	TRCSTS	R	MTL Tx Queue Read Controller Status. This field indicates the state of the Tx Queue Read Controller: ■ 2'b00: Idle state ■ 2'b01: Read state (transferring data to the MAC transmitter) ■ 2'b10: Waiting for pending Tx Status from the MAC transmitter ■ 2'b11: Flushing the Tx queue because of the Packet Terminate request from the MAC **Value After Reset:** 0x0 **Exists:** Always
0	TCPAUSED	R	Traffic Class in Pause. When this bit is high and the Rx flow control is enabled, it indicates that the Traffic Class 1 is in the Pause condition because of the following: ■ Reception of the PFC packet for the priorities assigned to the Traffic Class 1 when PFC is enabled ■ Reception of IEEE 802.3-2018 Pause packet when Priority Based Flow Control is disabled **Value After Reset:** 0x0 **Exists:** Always

11.4.4 MTL_TC1_ETS_Control

- **Description:** This register configures the transmission selection algorithm, for Traffic Class 1.This register is not available in single traffic class configuration.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1110
- **Exists:** DWCXG_NUM_TC>1

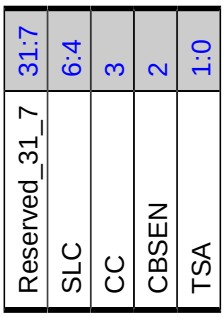


Table 11-471 Fields for Register: MTL_TC1_ETS_Control

Bits	Name	Memory Access	Description
31:7	Reserved_31_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	SLC	R/W	Slot Count. If the credit-based shaper algorithm is enabled, the software can program the number of slots (of 125 us duration) over which the average transmitted bits per slot, provided in the MTL_TC(#n)_ETS_Status register, need to be computed for TC(#n). The encoding is as follows: ■ 3'b000: 1 Slot ■ 3'b001: 2 Slots ■ 3'b010: 4 Slots ■ 3'b011: 8 Slots ■ 3'b100: 16 Slots ■ 3'b101-3'b111: Reserved **Value After Reset:** 0x0 **Exists:** (DWCXG_TC1_AV_EN)

Table 11-471 Fields for Register: MTL_TC1_ETS_Control (continued)

Bits	Name	Memory Access	Description
3	CC	R/W	Credit Control. When this bit is set, the accumulated credit parameter in the credit-based shaper algorithm logic is not reset to zero when there is positive credit and no packet to transmit in TC(#n). The credit accumulates even when there is no packet waiting in TC(#n) and another TC is transmitting. When this bit is reset, the accumulated credit parameter in the credit-based shaper algorithm logic is set to zero when there is positive credit and no packet to transmit in TC(#n). When there is no packet waiting in TC(#n) and other TC is transmitting, no credit is accumulated. **Value After Reset:** 0x0 **Exists:** (DWCXG_TC1_AV_EN)
2	CBSEN	R	Credit Based Shaper Enable This field indicates that TSA field of this register is enabled for CBS **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1:0	TSA	R/W	Transmission Selection Algorithm. This field is used to assign a transmission selection algorithm for this traffic class. ■ 2'b00: Strict priority ■ 2'b01: CBS ■ 2'b10: ETS ■ 2'b11: ATS **Value After Reset:** 0x0 **Exists:** Always

11.4.5 MTL_TC1_ETS_Status

- **Description:** This register contains the information of average bits transmitted over a slot interval of 10 million bit times, for Traffic Class 1.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1114
- **Exists:** (DWCXG_DCB_EN&&DWCXG_NUM_TC>1)

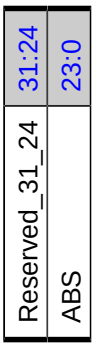


Table 11-472 Fields for Register: MTL_TC1_ETS_Status

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:0	ABS	R	Average Bits per Slot. This field contains the average transmitted bits per slot computed over every slot of 10 million bit times (1ms in 10G mode, 10ms in 1G mode). The maximum value is 0x989680. When AVB mode is enabled, this field contains the average transmitted bits over number of slots indicated by SLC field of MTL_TC1_ETS_Control register. **Value After Reset:** 0x0 **Exists:** Always

11.4.6 MTL_TC1_Quantum_Weight

- **Description:** The TC1 Quantum or Weights register controls the average traffic transmitted in Traffic Class 1. For TCn, where n = 1 to 7, this register is not available when the number of traffic classes selected is less than n + 1
  This register is not available in corresponding Queues 8-15 registers.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1118
- **Exists:** (DWCXG_NUM_TC>1)

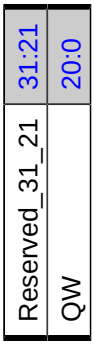


Table 11-473 Fields for Register: MTL_TC1_Quantum_Weight

Bits	Name	Memory Access	Description
31:21	Reserved_31_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
20:0	QW	R/W	idleSlopeCredit, Quantum or Weights. ■ **idleSlopeCredit** ❑ When this TC is in AVB mode, this field contains the idleSlopeCredit value required for the credit-based shaper algorithm for Traffic Class 1. This is the rate of change of credit in bits per cycle (3.2 ns for 2.5/10 Gbps; 8 ns for 1000 Mbps) when the credit is increasing. The software must program this field with computed credit in bits per cycle scaled by 1024. The maximum value is portTransmitRate, that is, 0x2000 in 1000/2500 Mbps mode and 0x8000 in 10 Gbps mode. Bits[20:16] must be written to zero. This field is not valid for Traffic Class 0. ■ **Quantum** ❑ When the TSA is set to ETS and ETSALG is set to DWRR algorithm for Traffic Class 1 traffic, this field contains the quantum value in bytes to be added to credit during every queue scanning cycle. The maximum value is 0x1312D0 bytes. ■ **Weights**

Table 11-473 Fields for Register: MTL_TC1_Quantum_Weight (continued)

Bits	Name	Memory Access	Description
			❑ When TSA is set to ETS and ETSALG is set to WFQ algorithm for Traffic Class1 traffic, this field contains the weight for this queue. The maximum value is 0x3FFF where weight of 0 indicates 100% bandwidth. Bits [20:14] must be written to zero. ❑ When TSA is set to ETS and ETSALG is set to WRR algorithm for Traffic Class 1 traffic, this field contains the weight for this queue. The maximum value is 0x64. Bits[20:7] must be written to zero. This field is reserved and RO when the number of traffic classes selected is less than two. This field is reserved in corresponding Queues 8-15 register. **Value After Reset:** 0x0 **Exists:** Always

11.4.7 MTL_TC1_SendSlopeCredit

- **Description:** MTL registers for traffic classes and queues 1 registers.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x111c
- **Exists:** (DWCXG_TC1_AV_EN)

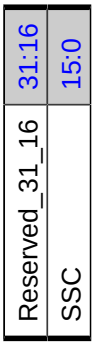


Table 11-474 Fields for Register: MTL_TC1_SendSlopeCredit

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15:0	SSC	R/W	sendSlopeCredit Value. ■ When Audio Video Bridging operation is enabled, this field contains the sendSlopeCredit value required for credit-based shaper algorithm for TC(#n). The software must program this field with computed credit in bits per cycle scaled by 1024. This field must be programmed with absolute sendSlopeCredit value. The credit-based shaper logic subtracts it from the accumulated credit when TC(#n) is selected for transmission. See the application note for more details on programming this value. **Value After Reset:** 0x0 **Exists:** Always

11.4.8 MTL_TC1_HiCredit

- **Description:** This register contains the hiCredit value required for the credit-based shaper algorithm for TC1.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1120
- **Exists:** (DWCXG_TC1_AV_EN)

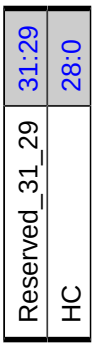


Table 11-475 Fields for Register: MTL_TC1_HiCredit

Bits	Name	Memory Access	Description
31:29	Reserved_31_29	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
28:0	HC	R/W	hiCredit Value. ■ When the Audio Video Bridging operation is enabled, this field contains the hiCredit value required for the credit-based shaper algorithm. This is the maximum value that can be accumulated in the credit parameter. This is specified in bits scaled by 1024. The maximum value is maxInterferenceSize, that is, best-effort maximum packet size (16384 bytes or 131072 bits). The value to be specified is 131072 * 1024 = 134217728 or 0x0800_0000 **Value After Reset:** 0x0 **Exists:** Always

11.4.9 MTL_TC1_LoCredit

- **Description:** This register contains the loCredit value required for the credit-based shaper algorithm for TC1.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1124
- **Exists:** (DWCXG_TC1_AV_EN)

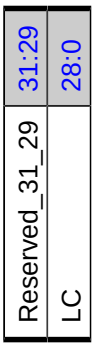


Table 11-476 Fields for Register: MTL_TC1_LoCredit

Bits	Name	Memory Access	Description
31:29	Reserved_31_29	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
28:0	LC	R/W	loCredit Value. ■ When Audio Video Bridging operation is enabled, this field contains the loCredit value required for the credit-based shaper algorithm. This is the minimum value that can be accumulated in the credit parameter. This is specified in bits scaled by 1024. The maximum value is maxPacketSize transmitted from this queue, that is, 16384 * 8 * 1024 = 134217728 or 0x0800_0000. Because it is a negative value, the programmed value is twos complement of the value, that is, 0x1800_0000. **Value After Reset:** 0x0 **Exists:** Always

11.4.10 MTL_RxQ1_Operation_Mode

- **Description:** The Queue 1 Receive Operation Mode register establishes the Receive queue 1 operating modes and command.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1140
- **Exists:** DWCXG_NUM_RXQ>1

31:y	x:16	15:8	7	6	5	4	3	2	1:0
Reserved_31_y	RQS	Reserved_15_8	EHFC	DIS_TCP_EF	RSF	FEF	FUF	Reserved_2	RTC

Table 11-477 Fields for Register: MTL_RxQ1_Operation_Mode

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_L256_RXMS + 16
x:16	RQS	R/W	Receive Queue Size. This field indicates the size of the allocated Receive queues in blocks of 256 bytes. The sixteenth bit is the starting bit of this field. The width of this field depends on the Rx memory size selected in your configuration. For example, if the memory size is 2048, the width of this field is 3 bits: LOG2(2048/256) = LOG2(8) = 3 bits A value of 0 indicates 256 bytes. This field is reserved and RO in single queue configuration, and its reset value corresponds to the receive FIFO size. When this queue is used, a minimum value of 7 (size = 2048) is recommended so that the queue can store more than a complete max-sized Ethernet packet. **Value After Reset:** (DWCXG_NUM_RXQ>1)?0: ((DWCXG_RXFIFO_SIZE/256)-1) **Exists:** Always **Range Variable[x]:** DWCXG_L256_RXMS + 15
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-477 Fields for Register: MTL_RxQ1_Operation_Mode (continued)

Bits	Name	Memory Access	Description
7	EHFC	R/W	Enable Hardware Flow Control. When this bit is set, the flow control signal operation, based on the fill-level of Rx queue, is enabled. When reset, the flow control operation is disabled. This bit is not used (reserved and always reset) when the Rx queue is less than 4 KB. **Value After Reset:** 0x0 **Exists:** (DWCXG_RXFIFO_SIZE>=4096)
6	DIS_TCP_EF	R/W	Disable Dropping of TCP/IP Checksum Error Packets. ■ When this bit is set, the MAC does not drop the packets which only have the errors detected by the Receive Checksum Offload engine. Such packets have errors only in the encapsulated payload. There are no errors (including FCS error) in the Ethernet packet received by the MAC. ■ When this bit is reset, all error packets are dropped if the FEF bit is reset. **Value After Reset:** 0x0 **Exists:** (DWCXG_RX_COE)
5	RSF	R/W	Receive Queue Store and Forward. ■ 1: DWC_25gmac reads a packet from the Rx queue only after the complete packet has been written to it, ignoring the RTC field of this register. ■ 0: Rx queue operates in the Threshold (cut-through) mode, subject to the threshold specified by the RTC field of this register. **Value After Reset:** 0x0 **Exists:** Always
4	FEF	R/W	Forward Error Packets. ■ 0: Rx queue drops packets with error status (CRC error, GMII_ER, watchdog timeout, or overflow). However, if the start byte (write) pointer of a packet is already transferred to the read controller side (in Threshold mode), the packet is not dropped. ■ 1: All packets except the runt error packets are forwarded to the application or DMA. If the RSF bit is set and the Rx queue overflows when a partial packet is written, the packet is dropped irrespective of the setting of this bit. However, if the RSF bit is reset and the Rx queue overflows when a partial packet is written, a partial packet might be forwarded to the application or DMA. **Value After Reset:** 0x0 **Exists:** Always

Table 11-477 Fields for Register: MTL_RxQ1_Operation_Mode (continued)

Bits	Name	Memory Access	Description
3	FUF	R/W	Forward Undersized Good Packets. ■ 1: Rx queue forwards the undersized good packets (with no Invalid-code/GMII/CRC/Length errors and length less than 64 bytes at Rx (X)GMII interface), including pad-bytes and CRC. ■ 0: Rx queue drops all packets of length less than 64 bytes. **Value After Reset:** 0x0 **Exists:** Always
2	Reserved_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1:0	RTC	R/W	Receive Queue Threshold Control. These bits control the threshold level of the MTL Rx queue (in bytes): ■ 2'b00: 64 ■ 2'b01: Reserved ■ 2'b10: 96 ■ 2'b11: 128 The received packet is transferred to the application or DMA when the packet size within the MTL Rx queue is larger than the threshold. In addition, full packets with length less than the threshold are automatically transferred. The value of 11 is not applicable if the size of the configured Rx Queue is 128 bytes. This field is valid only when the RSF bit is zero. This field is ignored when the RSF bit is set to 1. **Value After Reset:** 0x0 **Exists:** Always

11.4.11 MTL_RxQ1_Missed_Pkt_Overflow_Cnt

- **Description:** The Queue 1 Overflow Counter register contains the counter for packets missed because of Receive queue packet flush and packets discarded because of Receive queue overflow.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1144
- **Exists:** DWCXG_NUM_RXQ>1

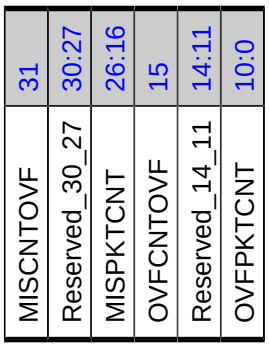


Table 11-478 Fields for Register: MTL_RxQ1_Missed_Pkt_Overflow_Cnt

Bits	Name	Memory Access	Description
31	MISCNTOVF	R	Missed Packet Counter Overflow Bit. When set, this bit indicates that the MISPKTCNT field crossed the maximum limit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30:27	Reserved_30_27	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
26:16	MISPKTCNT	R	Missed Packet Counter. This field indicates the number of packets missed by DWC_25gmac because the host asserted packet-flush for this queue. This counter is incremented each time a packet is flushed for this queue. This counter is reset when this register's bits[23:16] is read by the software. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-478 Fields for Register: MTL_RxQ1_Missed_Pkt_Overflow_Cnt (continued)

Bits	Name	Memory Access	Description
15	OVFCNTOVF	R	Overflow Counter Overflow Bit. When set, this bit indicates that OVFPKTCNT field crossed the maximum limit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14:11	Reserved_14_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
10:0	OVFPKTCNT	R	Overflow Packet Counter. This field indicates the number of packets discarded by DWC_25gmac because of Receive queue overflow. This counter is incremented each time DWC_25gmac discards an incoming packet because of overflow. This counter is reset when this register bits[7:0] is read by software. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.4.12 MTL_RxQ1_Debug

- **Description:** The Queue 1 Receive Debug register gives the debug status of various blocks related to the Receive queue.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1148
- **Exists:** DWCXG_NUM_RXQ>1

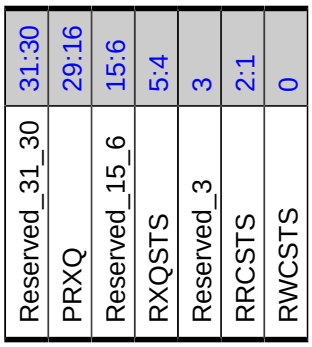


Table 11-479 Fields for Register: MTL_RxQ1_Debug

Bits	Name	Memory Access	Description
31:30	Reserved_31_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29:16	PRXQ	R	Number of Packets in Receive Queue. This field indicates the current number of packets in the Rx Queue. The theoretical maximum value for this field is 256KB/16B = 16K Packets, that is, Max_Queue_Size/Min_Packet_Size. **Value After Reset:** 0x0 **Exists:** Always
15:6	Reserved_15_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:4	RXQSTS	R	MTL Rx Queue Fill-Level Status. This field gives the status of the fill-level of the Rx Queue: ■ 2'b00: Rx Queue empty ■ 2'b01: Rx Queue fill-level below flow-control deactivate threshold ■ 2'b10: Rx Queue fill-level above flow-control activate threshold ■ 2'b11: Rx Queue full **Value After Reset:** 0x0 **Exists:** Always

Table 11-479 Fields for Register: MTL_RxQ1_Debug (continued)

Bits	Name	Memory Access	Description
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2:1	RRCSTS	R	MTL Rx Queue Read Controller State. This field gives the state of the Rx queue Read controller: ■ 2'b00: Idle state ■ 2'b01: Reading packet data ■ 2'b10: Reading packet status (or timestamp) ■ 2'b11: Flushing the packet data and status **Value After Reset:** 0x0 **Exists:** Always
0	RWCSTS	R	MTL Rx Queue Write Controller Active Status. When high, this bit indicates that the MTL Rx queue Write controller is active, and it is transferring a received packet to the Rx Queue. **Value After Reset:** 0x0 **Exists:** Always

11.4.13 MTL_RxQ1_Control

- **Description:** The Queue 1 Receive Control register controls the receive arbitration and passing of received packets to the application.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x114c
- **Exists:** DWCXG_NUM_RXQ>1

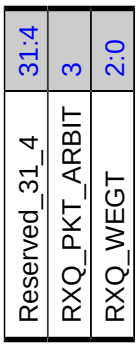


Table 11-480 Fields for Register: MTL_RxQ1_Control

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	RXQ_PKT_ARBIT	R/W	Receive Queue Packet Arbitration Control. ■ 1: MTL enables the packet-level arbitration. ■ 0: MTL enables the PBL based arbitration. **Value After Reset:** DWCXG_MTL_SUBSYS **Exists:** ((DWCXG_MTL_SUBSYS)||(! DWCXG_MTL_SUBSYS&&DWCXG_NUM_DMA_RX_CH> 1))
2:0	RXQ_WEGT	R/W	Receive Queue Weight. This field indicates the weight assigned to the Rx Queue 1. The weight is used as the number of continuous PBL requests or contiguous packets (depending on the RXQ_PKT_ARBIT) allocated to the queue in one arbitration cycle. **Value After Reset:** 0x0 **Exists:** Always

11.4.14 MTL_RxQ1_Flow_Control

- **Description:** This register controls the activation and de-activation threshold in the queue for triggering the generation of Flow Control packets by MAC transmitter.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1150
- **Exists:** (DWCXG_RXFIFO_SIZE>=4096&&DWCXG_NUM_RXQ>1)

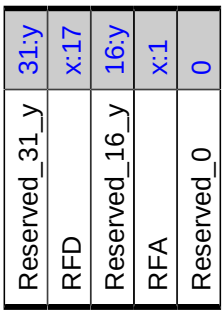


Table 11-481 Fields for Register: MTL_RxQ1_Flow_Control

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_FLCW + 17
x:17	RFD	R/W	Threshold for deactivating Flow Control. These bits control the threshold (fill-level of RxQ0) at which the flow control is de-asserted once it is activated. ■ 6'b000000: RQS value minus 1 KB, that is, FULL - 1024 bytes ■ 6'b000001: RQS value minus 1.5 KB, that is, FULL - 1536 bytes ■ 6'b000010: RQS value minus 2 KB, that is, FULL - 2048 bytes. ■ ... ■ 6'b111111: RQS value minus 32.5 KB, that is, FULL - 33280 bytes The de-assertion is effective only after flow control is asserted as per the setting of RFA field. Note that the value to be programmed must be such that the threshold is positive number. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_FLCW + 16

Table 11-481 Fields for Register: MTL_RxQ1_Flow_Control (continued)

Bits	Name	Memory Access	Description
16:y	Reserved_16_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_FLCW + 1
x:1	RFA	R/W	Threshold for activating Flow Control. These bits control the threshold (fill-level of RxQ0) at which the flow control is activated ■ 6'b000000: RQS value minus 1 KB, that is, FULL - 1024 bytes ■ 6'b000001: RQS value minus 1.5 KB, that is, FULL - 1536 bytes ■ 6'b000010: RQS value minus 2 KB, that is, FULL - 2048 bytes. ■ ... ■ 6'b111111: RQS value minus 32.5 KB, that is, FULL - 33280 bytes The flow control is triggered when EHFC=1 and the queue size (RQS value in MTL_RxQ1_Operation Mode register) is equal to or more than 4KB. Note that the value to be programmed in RFA must be such that the threshold is positive number. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_FLCW
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.4.15 MTL_Q1_Interrupt_Enable

- **Description:** This register contains the interrupt enable bits for Traffic class/Queue 1.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1170
- **Exists:** Always

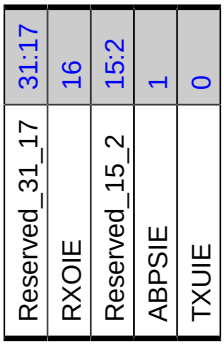


Table 11-482 Fields for Register: MTL_Q1_Interrupt_Enable

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	RXOIE	R/W	Receive Queue Overflow Interrupt Enable. ■ 1: Receive Queue Overflow interrupt is enabled. ■ 0: Receive Queue Overflow interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1)
15:2	Reserved_15_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	ABPSIE	R/W	Average Bits Per Slot Interrupt Enable. When this bit is set, average bits per slot interrupt is enabled for Traffic Class 1. An interrupt is generated when ABPSIS is set for this traffic class. This bit is reserved and RO when the number of traffic classes selected is less than 1. This field in the corresponding Queues 8 - 15 registers is always reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1)

Table 11-482 Fields for Register: MTL_Q1_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
0	TXUIE	R/W	Transmit Queue Underflow Interrupt Enable. ■ 1: Transmit Queue Underflow interrupt is enabled. ■ 0: Transmit Queue Underflow interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TXQ>1)

11.4.16 MTL_Q1_Interrupt_Status

- **Description:** This register contains the interrupt status bits for Traffic class/Queue 1.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x1174
- **Exists:** Always

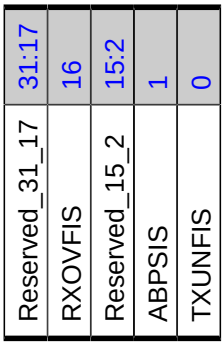


Table 11-483 Fields for Register: MTL_Q1_Interrupt_Status

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
16	RXOVFIS	R/W	Receive Queue Overflow Interrupt Status. This bit indicates that the Receive Queue had an Overflow during packet reception. If partial packet is transferred to the application, the overflow status is set in the corresponding Rx status. This bit is cleared when the application writes a 1 to this bit. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>1) **Testable:** untestable
15:2	Reserved_15_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

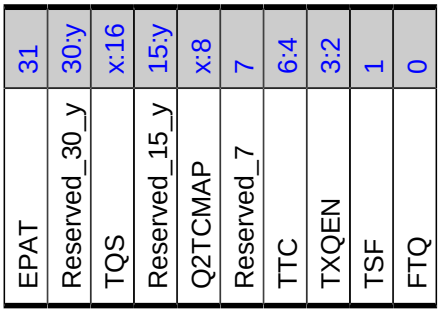
Table 11-483 Fields for Register: MTL_Q1_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
1	ABPSIS	R/W	Average Bits Per Slot Interrupt Status. When set, this bit indicates that the MAC has updated the ABS value for this traffic class. This bit is cleared when the application writes a 1 to this bit. This bit is reserved and RO when the number of traffic classes selected is less than 1. This field in corresponding Queues 8 - 15 registers is reserved and RO. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1) **Testable:** untestable
0	TXUNFIS	R/W	Transmit Queue Underflow Interrupt Status. This bit indicates that the Transmit Queue had an Underflow during packet transmission. Transmission is suspended and an Underflow Error TDES3[2] is set. This bit is cleared when the application writes a 1 to this bit. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TXQ>1) **Testable:** untestable

11.5 DWCXG_MTL_Q(#i) Registers

11.5.1 MTL_TxQ8_Operation_Mode

- **Description:** The Queue 8 Transmit Operation Mode register establishes the Transmit queue operating modes and commands.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1100
- **Exists:** DWCXG_NUM_TXQ>8



31	30:y	x:16	15:y	x:8	7	6:4	3:2	1	0
EPAT	Reserved_30_y	TQS	Reserved_15_y	Q2TCMAP	Reserved_7	TTC	TXQEN	TSF	FTQ

Table 11-484 Fields for Register: MTL_TxQ8_Operation_Mode

Bits	Name	Memory Access	Description
31	EPAT	R/W	Externally Provided Arrival Time. When this bit is set, ATS uses software provided frame arrival time through Enhanced descriptor/control-word. Otherwise, ATS uses internally captured IEEE 1588 reference system time when particular frame crosses its end on ATI interface. **Value After Reset:** 0x0 **Exists:** DWCXG_ATS
30:y	Reserved_30_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_L256_TXMS + 16

Table 11-484 Fields for Register: MTL_TxQ8_Operation_Mode (continued)

Bits	Name	Memory Access	Description
x:16	TQS	R/W	Transmit Queue Size. This field indicates the size of the allocated Transmit queues in blocks of 256 bytes. The sixteenth bit is the starting bit of this field. The width of this field depends on the Tx memory size selected in your configuration. For example, if the memory size is 2048, the width of this field is 3 bits: LOG2(2048/256) = LOG2(8) = 3 bits A value of 0 indicates 256 bytes. When this queue is enabled (TXQEN=10), a minimum value of 7 (size = 2048) is recommended so that the queue can store more than a complete max-sized Ethernet packet. This field is reserved and RO in single queue configuration, and its reset value corresponds to the transmit FIFO size **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_L256_TXMS + 15
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_TCW + 8
x:8	Q2TCMAP	R/W	Queue to Traffic Class Mapping. This field is used to map this tx Queue to a traffic class (TC). The actual width of this field depends on the number of traffic classes selected during configuration. ■ 3'b000: TC0 ■ 3'b001: TC1 ■ 3'b010: TC2 ■ 3'b011: TC3 ■ 3'b100: TC4 ■ 3'b101: TC5 ■ 3'b110: TC6 ■ 3'b111: TC7 **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1)&&(!SNPS_RSVDPARAM_50) **Range Variable[x]:** DWCXG_TCW + 7

Table 11-484 Fields for Register: MTL_TxQ8_Operation_Mode (continued)

Bits	Name	Memory Access	Description
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	TTC	R/W	Transmit Threshold Control. These bits control the threshold level of the MTL Tx Queue. Transmission starts when the packet size within the MTL Tx Queue is larger than the threshold. In addition, full packets with length less than the threshold are also transmitted. These bits are used only when the TSF bit is reset. ■ 000: 64 ■ 001: Reserved ■ 010: 96 ■ 011: 128 ■ 100: 192 ■ 101: 256 ■ 110: 384 ■ 111: 512 **Value After Reset:** 0x0 **Exists:** Always
3:2	TXQEN	R/W	Transmit Queue Enable. This field is used to enable/disable the transmit queue. ■ 2'b00: Not enabled ■ 2'b01: Audio Video Bridging ■ 2'b01: Reserved ■ 2'b10: Enabled ■ 2'b11: Reserved The queue is disabled by default, enable the Tx Queue by programming this field in the AVB mode or default mode as desired. **Value After Reset:** 0x0 **Exists:** Always
1	TSF	R/W	Transmit Store and Forward. When this bit is set, the transmission starts when a full packet resides in the MTL Tx Queue. When this bit is set, the values specified in the TTC field are ignored. This bit must be changed only when the transmission is stopped. **Value After Reset:** 0x0 **Exists:** Always

Table 11-484 Fields for Register: MTL_TxQ8_Operation_Mode (continued)

Bits	Name	Memory Access	Description
0	FTQ	R/W	Flush Transmit Queue. When this bit is set, the Tx Queue Controller logic is reset to its default values. Therefore, all the data in the Tx queue is lost or flushed. This bit is internally reset when the flushing operation is complete. You must not write to this register until this bit is reset. The data which is already accepted by the MAC transmitter is not flushed. It is scheduled for transmission and results in underflow and runt packet transmission. **Note:** The flush operation is complete only when the Tx queue is empty. To complete this flush operation, the PHY Tx clock (clk_tx_312pt5_i) must be active. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.5.2 MTL_TxQ8_Underflow

- **Description:** The Queue 8 Underflow Counter register contains the counter for packets aborted because of Transmit queue underflow.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1104
- **Exists:** DWCXG_NUM_TXQ>8

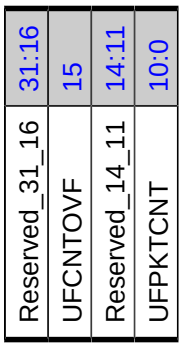


Table 11-485 Fields for Register: MTL_TxQ8_Underflow

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15	UFCNTOVF	R	Overflow Bit for Underflow Packet Counter. This bit is set every time the Tx Queue Underflow Packet Counter field overflows, that is, it has crossed the maximum count. In such a scenario, the overflow packet counter is reset to all-zeros and this bit indicates that the rollover happened. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14:11	Reserved_14_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-485 Fields for Register: MTL_TxQ8_Underflow (continued)

Bits	Name	Memory Access	Description
10:0	UFPKTCNT	R	Underflow Packet Counter. This field indicates the number of packets aborted by the controller because of Transmit Queue Underflow. This counter is incremented each time the MAC aborts an outgoing packet because of underflow. The counter is cleared when UFPKTCNT[7:0] is read by software. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.5.3 MTL_TxQ8_Debug

- **Description:** The Transmit Debug register gives the debug status of various blocks related to this Transmit Queue 8 and Traffic Class 8.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1108
- **Exists:** DWCXG_NUM_TXQ>8

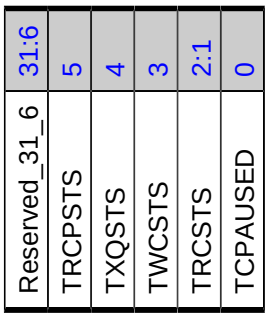


Table 11-486 Fields for Register: MTL_TxQ8_Debug

Bits	Name	Memory Access	Description
31:6	Reserved_31_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5	TRCPSTS	R	MTL TRC Pending Status When this bit is high, it indicates that the MTL Tx Read Controller (TRC) has a pending status from MAC. **Value After Reset:** 0x0 **Exists:** Always
4	TXQSTS	R	MTL Tx Queue Not Empty Status. When this bit is high, it indicates that the MTL Tx Queue is not empty and some data is left for transmission. **Value After Reset:** 0x0 **Exists:** Always
3	TWCSTS	R	MTL Tx Queue Write Controller Status. When high, this bit indicates that the MTL Tx Queue Write Controller is active, and it is transferring the data to the Tx Queue. **Value After Reset:** 0x0 **Exists:** Always

Table 11-486 Fields for Register: MTL_TxQ8_Debug (continued)

Bits	Name	Memory Access	Description
2:1	TRCSTS	R	MTL Tx Queue Read Controller Status. This field indicates the state of the Tx Queue Read Controller: ■ 00: Idle state ■ 01: Read state (transferring data to the MAC transmitter) ■ 10: Waiting for pending Tx Status from the MAC transmitter ■ 11: Flushing the Tx queue because of the Packet Terminate request from the MAC **Value After Reset:** 0x0 **Exists:** Always
0	TCPAUSED	R	Traffic Class in Pause. When this bit is high and the Rx flow control is enabled, it indicates that Traffic Class is in the Pause condition because of the following: ■ Reception of the PFC packet for the priorities assigned to the Traffic Class 1 when PFC is enabled ■ Reception of IEEE 802.3-2018 Pause packet when PFC is disabled For TCQn, where n is 1 to 7, this bit is reserved and RO when the number of traffic class selected is less than n + 1 This field is reserved and RO in the corresponding Queues 8-15 registers **Value After Reset:** 0x0 **Exists:** Always

11.5.4 MTL_TxQ(#j)_Bucket_Empty_2_Full (for j = 8; j <= DWCXG_NUM_TXQ-1)

- **Description:** The MTL Queue 8 ATS bucket empty 2 full register controls the Asynchronous Traffic shaper scheduler and transmission selection operation.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1128
- **Exists:** DWCXG_ATS

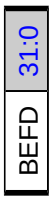


Table 11-487 Fields for Register: MTL_TxQ(#j)_Bucket_Empty_2_Full (for j = 8; j <= DWCXG_NUM_TXQ-1)

Bits	Name	Memory Access	Description
31:0	BEFD	R/W	Bucket Empty to Full Duration. This field is programmed with time duration that is required for token bucket to fill from empty to full. Maximum bucket size per queue is its own size (in bits), hence BEFD should not be greater than Bucket Size/CIR. **Value After Reset:** 0x0 **Exists:** Always

11.5.5 MTL_TxQ(#j)_Committed_Info_Rate (for j = 8; j <= DWCXG_NUM_TXQ-1)

- **Description:** The MTL Queue 8 ATS committed info rate register controls the Asynchronous Traffic Shaper scheduler & transmission selection operation.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x112C
- **Exists:** DWCXG_ATS

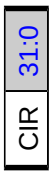


Table 11-488 Fields for Register: MTL_TxQ(#j)_Committed_Info_Rate (for j = 8; j <= DWCXG_NUM_TXQ-1)

Bits	Name	Memory Access	Description
31:0	CIR	R/W	Committed Information Rate in bits per second. The committed information rate of the ATS scheduler instance, in bits per second. It defines the rate at which the token bucket is refilled with tokens until the maximum token capacity of the token bucket is reached. **Value After Reset:** 0x0 **Exists:** Always

11.5.6 MTL_RxQ8_Operation_Mode

- **Description:** The Queue 8 Receive Operation Mode register establishes the Receive queue operating modes and command.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1140
- **Exists:** DWCXG_NUM_RXQ>8

31:y	x:16	15:8	7	6	5	4	3	2	1:0
Reserved_31_y	RQS	Reserved_15_8	EHFC	DIS_TCP_EF	RSF	FEF	FUF	Reserved_2	RTC

Table 11-489 Fields for Register: MTL_RxQ8_Operation_Mode

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_L256_RXMS + 16
x:16	RQS	R/W	Receive Queue Size. This field indicates the size of the allocated Receive queues in blocks of 256 bytes. The sixteenth bit is the starting bit of this field. The width of this field depends on the Rx memory size selected in your configuration. For example, if the memory size is 2048, the width of this field is 3 bits: LOG2(2048/256) = LOG2(8) = 3 bits A value of 0 indicates 256 bytes. This field is reserved and RO in single queue configuration, and its reset value corresponds to the receive FIFO size. When this queue is used, a minimum value of 7 (size = 2048) is recommended so that the queue can store more than a complete max-sized Ethernet packet. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_L256_RXMS + 15
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-489 Fields for Register: MTL_RxQ8_Operation_Mode (continued)

Bits	Name	Memory Access	Description
7	EHFC	R/W	Enable Hardware Flow Control. When this bit is set, the flow control signal operation, based on the fill-level of Rx queue, is enabled. When reset, the flow control operation is disabled. This bit is not used (reserved and always reset) when the Rx queue is less than 4 KB. **Value After Reset:** 0x0 **Exists:** (DWCXG_RXFIFO_SIZE>=4096)
6	DIS_TCP_EF	R/W	Disable Dropping of TCP/IP Checksum Error Packets. When this bit is set, the MAC does not drop the packets which only have the errors detected by the Receive Checksum Offload engine. Such packets have errors only in the encapsulated payload. There are no errors (including FCS error) in the Ethernet packet received by the MAC. When this bit is reset, all error packets are dropped if the FEF bit is reset. **Value After Reset:** 0x0 **Exists:** (DWCXG_RX_COE)
5	RSF	R/W	Receive Queue Store and Forward. When this bit is set, DWC_25gmac reads a packet from the Rx queue only after the complete packet has been written to it, ignoring the RTC field of this register. When this bit is reset, the Rx queue operates in the Threshold (cut-through) mode, subject to the threshold specified by the RTC field of this register. **Value After Reset:** 0x0 **Exists:** Always
4	FEF	R/W	Forward Error Packets. When this bit is reset, the Rx queue drops packets with error status (CRC error, GMII_ER, watchdog timeout, or overflow). However, if the start byte (write) pointer of a packet is already transferred to the read controller side (in Threshold mode), the packet is not dropped. When this bit is set, all packets except the runt error packets are forwarded to the application or DMA. If the RSF bit is set and the Rx queue overflows when a partial packet is written, the packet is dropped irrespective of the setting of this bit. However, if the RSF bit is reset and the Rx queue overflows when a partial packet is written, a partial packet might be forwarded to the application or DMA. **Value After Reset:** 0x0 **Exists:** Always

Table 11-489 Fields for Register: MTL_RxQ8_Operation_Mode (continued)

Bits	Name	Memory Access	Description
3	FUF	R/W	Forward Undersized Good Packets. When this bit is set, the Rx queue forwards the undersized good packets (with no Invalid-code/GMII/CRC/Length errors and length less than 64 bytes at Rx (X)GMII interface), including pad-bytes and CRC. When this bit is reset, the Rx queue drops all packets of length less than 64 bytes, unless a packet is already transferred because of the lower value of Rx Threshold, for example, RTC = 01. **Value After Reset:** 0x0 **Exists:** Always
2	Reserved_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1:0	RTC	R/W	Receive Queue Threshold Control. These bits control the threshold level of the MTL Rx queue (in bytes): ■ 00: 64 ■ 01: Reserved ■ 10: 96 ■ 11: 128 The received packet is transferred to the application or DMA when the packet size within the MTL Rx queue is larger than the threshold. In addition, full packets with length less than the threshold are automatically transferred. The value of 11 is not applicable if the size of the configured Rx Queue is 128 bytes. This field is valid only when the RSF bit is zero. This field is ignored when the RSF bit is set to 1. **Value After Reset:** 0x0 **Exists:** Always

11.5.7 MTL_RxQ8_Missed_Pkt_Overflow_Cnt

- **Description:** The Queue 8 Overflow Counter register contains the counter for packets missed because of Receive queue packet flush and packets discarded because of Receive queue overflow.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1144
- **Exists:** DWCXG_NUM_RXQ>8

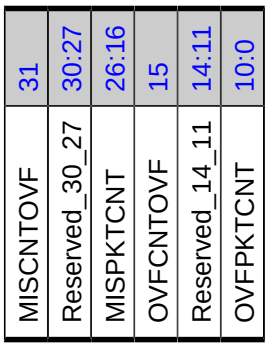


Table 11-490 Fields for Register: MTL_RxQ8_Missed_Pkt_Overflow_Cnt

Bits	Name	Memory Access	Description
31	MISCNTOVF	R	Missed Packet Counter Overflow Bit. When set, this bit indicates that the Rx Queue Missed Packet Counter crossed the maximum limit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30:27	Reserved_30_27	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
26:16	MISPKTCNT	R	Missed Packet Counter. This field indicates the number of packets missed by DWC_25gmac because the application asserted ari_pkt_flush_i[] for this queue. This counter is incremented each time the application issues ari_pkt_flush_i[] for this queue. This counter is reset when this register is read with mci_be_i[0] at 1'b1. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-490 Fields for Register: MTL_RxQ8_Missed_Pkt_Overflow_Cnt (continued)

Bits	Name	Memory Access	Description
15	OVFCNTOVF	R	Overflow Counter Overflow Bit. When set, this bit indicates that the Rx Queue Overflow Packet Counter field crossed the maximum limit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
14:11	Reserved_14_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
10:0	OVFPKTCNT	R	Overflow Packet Counter. This field indicates the number of packets discarded by DWC_25gmac because of Receive queue overflow. This counter is incremented each time DWC_25gmac discards an incoming packet because of overflow. This counter is reset when this register is read with mci_be_i[0] at 1'b1. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.5.8 MTL_RxQ8_Debug

- **Description:** The Queue 8 Receive Debug register gives the debug status of various blocks related to the Receive queue.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1148
- **Exists:** DWCXG_NUM_RXQ>8

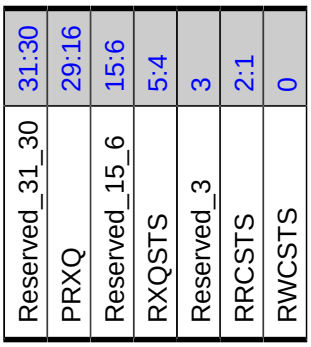


Table 11-491 Fields for Register: MTL_RxQ8_Debug

Bits	Name	Memory Access	Description
31:30	Reserved_31_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29:16	PRXQ	R	Number of Packets in Receive Queue. This field indicates the current number of packets in the Rx Queue. The theoretical maximum value for this field is 256KB/16B = 16K Packets, that is, Max_Queue_Size/Min_Packet_Size. **Value After Reset:** 0x0 **Exists:** Always
15:6	Reserved_15_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:4	RXQSTS	R	MTL Rx Queue Fill-Level Status. This field gives the status of the fill-level of the Rx Queue: ■ 00: Rx Queue empty ■ 01: Rx Queue fill-level below flow-control deactivate threshold ■ 10: Rx Queue fill-level above flow-control activate threshold ■ 11: Rx Queue full **Value After Reset:** 0x0 **Exists:** Always

Table 11-491 Fields for Register: MTL_RxQ8_Debug (continued)

Bits	Name	Memory Access	Description
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2:1	RRCSTS	R	MTL Rx Queue Read Controller State. This field gives the state of the Rx queue Read controller: ■ 00: Idle state ■ 01: Reading packet data ■ 10: Reading packet status (or timestamp) ■ 11: Flushing the packet data and status **Value After Reset:** 0x0 **Exists:** Always
0	RWCSTS	R	MTL Rx Queue Write Controller Active Status. When high, this bit indicates that the MTL Rx queue Write controller is active, and it is transferring a received packet to the Rx Queue. **Value After Reset:** 0x0 **Exists:** Always

11.5.9 MTL_RxQ8_Control

- **Description:** The Queue 8 Receive Control register controls the receive arbitration and passing of received packets to the application.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x114c
- **Exists:** (DWCXG_NUM_RXQ>8)

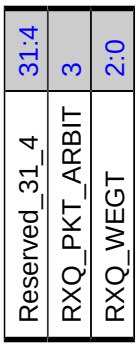


Table 11-492 Fields for Register: MTL_RxQ8_Control

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	RXQ_PKT_ARBIT	R/W	Receive Queue Packet Arbitration. When this bit is set, DWC_25gmac drives the packet data to the ARI interface such that the entire packet data of currently-selected queue is transmitted before switching to other queue. When this bit is reset, DWC_25gmac drives the packet data to the ARI interface such that the following amount of data of currently-selected queue is transmitted before switching to other queue: ■ PBL amount of data (indicated by ari_qN_pbl_i[]) or ■ complete data of a packet The status and the timestamp are not a part of the PBL data. Therefore, DWC_25gmac drives the complete status (including timestamp status) during first PBL request for the packet (in store-and-forward mode) or the last PBL request for the packet (in Threshold mode). **Value After Reset:** DWCXG_MTL_SUBSYS **Exists:** ((DWCXG_MTL_SUBSYS)||(! DWCXG_MTL_SUBSYS&&DWCXG_NUM_DMA_RX_CH> 1))

Table 11-492 Fields for Register: MTL_RxQ8_Control (continued)

Bits	Name	Memory Access	Description
2:0	RXQ_WEGT	R/W	Receive Queue Weight. This field indicates the weight assigned to the Rx Queue 8. The weight is used as the number of continuous PBL requests or contiguous packets (depending on the RXQ_PKT_ARBIT) allocated to the queue in one arbitration cycle. **Value After Reset:** 0x0 **Exists:** Always

11.5.10 MTL_RxQ8_Flow_Control

- **Description:** This register controls the activation and de-activation threshold in the queue for PAUSE/PFC flow control.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1150
- **Exists:** (DWCXG_RXFIFO_SIZE>=4096&&DWCXG_NUM_RXQ>8)

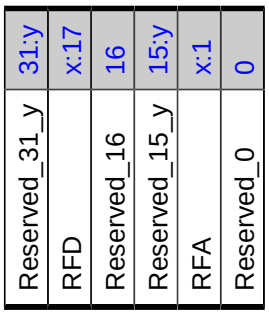


Table 11-493 Fields for Register: MTL_RxQ8_Flow_Control

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_FLCW + 17
x:17	RFD	R/W	Threshold for deactivating Flow Control. These bits control the threshold (fill-level of RxQ0) at which the flow control is de-asserted once it is activated. ■ 000000: RQS value minus 1 KB, that is, FULL - 1024 bytes ■ 000001: RQS value minus 1.5 KB, that is, FULL - 1536 bytes ■ 000010: RQS value minus 2 KB, that is, FULL - 2048 bytes. ■ ... ■ 111111: RQS value minus 32.5 KB, that is, FULL - 33280 bytes The de-assertion is effective only after flow control is asserted as per the setting of RFA field. Note that the value to be programmed must be such that the threshold is positive number. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_FLCW + 16
16	Reserved_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-493 Fields for Register: MTL_RxQ8_Flow_Control (continued)

Bits	Name	Memory Access	Description
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_FLCW + 1
x:1	RFA	R/W	Threshold for activating Flow Control. These bits control the threshold (fill-level of RxQ0) at which the flow control is activated ■ 000000: RQS value minus 1 KB, that is, FULL - 1024 bytes ■ 000001: RQS value minus 1.5 KB, that is, FULL - 1536 bytes ■ 000010: RQS value minus 2 KB, that is, FULL - 2048 bytes. ■ ... ■ 111111: RQS value minus 32.5 KB, that is, FULL - 33280 bytes The flow control is triggered when EHFC=1 and the queue size (RQS value in MTL_RxQ0_Operation Mode register) is equal to or more than 4KB. **Note:** The value to be programmed in RFA must be such that the threshold is a positive number. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_FLCW
0	Reserved_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.5.11 MTL_Q8_Interrupt_Enable

- **Description:** This register contains the interrupt enable bits for Traffic class/Queue 8.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1170
- **Exists:** Always

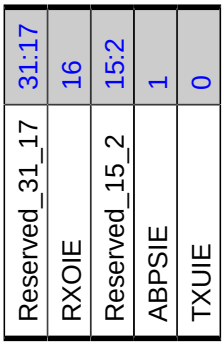


Table 11-494 Fields for Register: MTL_Q8_Interrupt_Enable

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	RXOIE	R/W	Receive Queue Overflow Interrupt Enable. When this bit is set, the Receive Queue Overflow interrupt is enabled. When this bit is reset, the Receive Queue Overflow interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>8)
15:2	Reserved_15_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	ABPSIE	R/W	Average Bits Per Slot Interrupt Enable. When this bit is set, average bits per slot interrupt is enabled for Traffic Class 8. An interrupt is generated when ABPSIS is set for this traffic class. This bit is reserved and RO when the number of traffic classes selected is less than two. This field in corresponding Queues 8-15 registers is always reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>8)

Table 11-494 Fields for Register: MTL_Q8_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
0	TXUIE	R/W	Transmit Queue Underflow Interrupt Enable. When this bit is set, the Transmit Queue Underflow interrupt is enabled. When this bit is reset, the Transmit Queue Underflow interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TXQ>8)

11.5.12 MTL_Q8_Interrupt_Status

- **Description:** This register contains the interrupt status bits for Traffic class/Queue 8.
- **Size:** 32 bits
- **Offset:** (0x0080*j)+0x1174
- **Exists:** Always

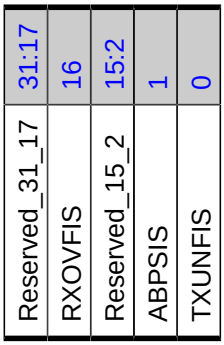


Table 11-495 Fields for Register: MTL_Q8_Interrupt_Status

Bits	Name	Memory Access	Description
31:17	Reserved_31_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	RXOVFIS	R/W	Receive Queue Overflow Interrupt Status. This bit indicates that the Receive Queue had an Overflow during packet reception. If partial packet is transferred to the application, the overflow status is set in the corresponding Rx status. This bit is cleared when the application writes a 1 to this bit. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_RXQ>8)
15:2	Reserved_15_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	ABPSIS	R/W	Average Bits Per Slot Interrupt Status. When set, this bit indicates that the MAC has updated the ABS value for this traffic class. This bit is cleared when the application writes a 1 to this bit. This bit is reserved when the number of traffic classes selected is less than two. This field in corresponding Queues 8-15 registers is always reserved and RO. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>8)

Table 11-495 Fields for Register: MTL_Q8_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
0	TXUNFIS	R/W	Transmit Queue Underflow Interrupt Status. This bit indicates that the Transmit Queue had an Underflow during packet transmission. This bit is cleared when the application writes a 1 to this bit. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TXQ>8)

11.6 DWCXG_DMA Registers

11.6.1 DMA_Mode

- **Description:** The Bus Mode register establishes the bus operating modes for the DMA.
- **Size:** 32 bits
- **Offset:** 0x3000
- **Exists:** Always

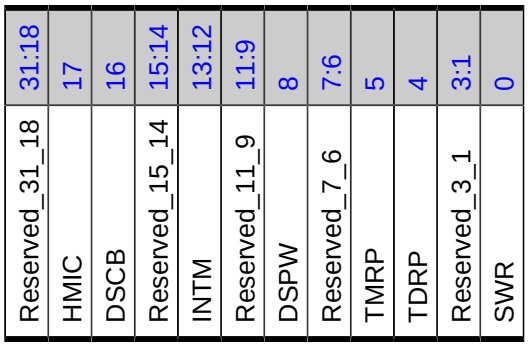


Table 11-496 Fields for Register: DMA_Mode

Bits	Name	Memory Access	Description
31:18	Reserved_31_18	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
17	HMIC	R	HDMA External CSR Memory initialization Complete. Design sets this bit to one to indicate External CSR Memory initialization is complete **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-496 Fields for Register: DMA_Mode (continued)

Bits	Name	Memory Access	Description
16	DSCB	R/W	DCACHE Size Computation Busy. When set, this field signifies that descriptor cache size for all the eligible DMAs are programmed to required values and design is computing base addresses for the corresponding VDMAs This bit will be reset to 0 by the Design after completing calculation of DCACHE memory base addresses for Tx/Rx DMAs. Application should monitor the value of this field after it sets it to 1, and once the value is reset to 0, application has a go ahead for starting Tx/Rx DMAs Once set to 1 by the application, this field must be cleared by the design upon base address calculation completion and new value must not be written into this field by the application **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15:14	Reserved_15_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:12	INTM	R/W	Interrupt Mode. This field controls the behavior of the interrupt signal (sbd_intr_o, sbd_perch_tx_intr_o, sbd_perch_rx_intr_o) outputs for packet transfer completion events in TxDMA and RxDMA. ■ 00: sbd_perch_* are pulse signals for each completion events. sbd_intr_o is also asserted and cleared only when software clears the corresponding RI/TI status bits. ■ 01: sbd_perch_* are level signals asserted on the corresponding event and de-asserted when the software clears the corresponding RI/TI status bits. The sbd_intr_o is not asserted for these packet transfer completion events. ■ 10: sbd_perch_* are level signals asserted on the corresponding event and de-asserted when the software clears the corresponding RI/TI status bits. However, the signal is asserted again if the same event occurred again before it was cleared. The sbd_intr_o is not asserted for these packet transfer completion events. ■ 11: Reserved For more details see Table, "DWC_25gmac Transfer Complete Interrupt Behavior" **Value After Reset:** 0x0 **Exists:** Always

Table 11-496 Fields for Register: DMA_Mode (continued)

Bits	Name	Memory Access	Description
11:9	Reserved_11_9	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
8	DSPW	R/W	Descriptor Posted Write. ■ 0: By default, all Rx DMA descriptor writes are non-posted and any Rx interrupt generation mechanism can be used, that is, Descriptor IOC, Watchdog timer, Byte Counter based or any combination of these. ■ 1: All Rx DMA data and descriptor writes are posted and Descriptor IOC based Rx interrupt generation does not function properly and must not be used. Only Watchdog timer, Byte Counter based or any combination of these Rx interrupt generation mechanisms must be used. The Tx/Rx DMA Descriptor IOC based Tx/Rx interrupt generation mechanism does not function properly and must not be used. **Value After Reset:** 0x0 **Exists:** Always
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5	TMRP	R/W	TSO Memory Read Priority. This bit controls the arbitration priority between the descriptor fetches and the TSO header fetches from the common TSO memory. By default, when this bit is 0, the DMA Descriptor read requests get higher priority than any of the TSO Header read requests of TxDMA. When this bit is set to 1, TSO header reads are serviced first by the arbiter before the DMA descriptor read requests. **Value After Reset:** 0x0 **Exists:** (DWCXG_EDMA&&!DWCXG_DIFF_CACHE_MEM_EN)

Table 11-496 Fields for Register: DMA_Mode (continued)

Bits	Name	Memory Access	Description
4	TDRP	R/W	Tx Descriptor Read Priority. This bit controls the arbitration priority of descriptor fetches from the local cache when simultaneous requests are generated from RxDMA and TxDMA channels. By default, when this bit is 0, the RxDMA requests get higher priority. When this bit is set to 1, TxDMA descriptor fetches are serviced first by the arbiter before the RxDMA requests. **Value After Reset:** 0x0 **Exists:** (DWCXG == 2) || SNPS_RSVDPARAM_50
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	SWR	R/W	Software Reset. When this bit is set, the MAC and the DMA controller reset the logic and all internal registers of the DMA, MTL, and MAC. This bit is automatically cleared after the reset operation is complete in all active clock domains. Before reprogramming any DWC_25gmac register, read a value of 0 from this bit. **Note:** The reset operation is complete only when all resets in all active clock domains are de-asserted. Therefore, it is essential that all PHY inputs clocks (applicable for the selected PHY interface) are present for software reset completion. The time to complete the software reset operation depends on the frequency of the slowest active clock. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.6.2 DMA_SysBus_Mode

- **Description:** The System Bus mode register controls the behavior of the AXI manager. It mainly controls burst splitting and number of outstanding requests.
- **Size:** 32 bits
- **Offset:** 0x3004
- **Exists:** Always

31:y	x:24	23:y	x:16	15	14	13	12	11	10	9:8	7	6	5	4	3	2	1	0
Reserved_31_y	WR_OSR_LMT	Reserved_23_22	RD_OSR_LMT	EN_LPI	LPI_XIT_PKT	ONEKBBE	AAL	EAME	AALE	Reserved_9_8	BLEN256	BLEN128	BLEN64	BLEN32	BLEN16	BLEN8	BLEN4	UBL

Table 11-497 Fields for Register: DMA_SysBus_Mode

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_WOSRLW + 24
x:24	WR_OSR_LMT	R/W	AXI Maximum Write Outstanding Request Limit. This value limits the maximum outstanding request on the AXI write interface. Maximum outstanding requests = WR_OSR_LMT + 1 **Note:** ■ Bit 29 is reserved and RO if AXI Maximum Outstanding Write Requests selected are less than 64 ■ Bit 28 is reserved and RO if AXI Maximum Outstanding Write Requests selected are less than 32 ■ Bit 27 is reserved and RO if AXI Maximum Outstanding Write Requests selected are less than 16 **Value After Reset:** 0x1 **Exists:** Always **Range Variable[x]:** DWCXG_WOSRLW + 23
23:y	Reserved_23_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_ROSRLW + 16

Table 11-497 Fields for Register: DMA_SysBus_Mode (continued)

Bits	Name	Memory Access	Description
x:16	RD_OSR_LMT	R/W	AXI Maximum Read Outstanding Request Limit. This value limits the maximum outstanding request on the AXI read interface. Maximum outstanding requests = RD_OSR_LMT + 1 **Note:** ■ Bit 21 is reserved and RO if AXI Maximum Outstanding Read Requests selected are less than 64 ■ Bit 20 is reserved and RO if AXI Maximum Outstanding Read Requests selected are less than 32 ■ Bit 19 is reserved and RO if AXI Maximum Outstanding Read Requests selected are less than 16 **Value After Reset:** 0x1 **Exists:** Always **Range Variable[x]:** DWCXG_ROSRLW + 15
15	EN_LPI	R/W	Enable Low Power Interface (LPI). ■ 1: Enables the LPI mode supported by the AXI interface and accepts the LPI request from the AXI System Clock controller. ■ 0: Disables the LPI mode and always denies the LPI request from the AXI System Clock controller. **Value After Reset:** 0x0 **Exists:** Always
14	LPI_XIT_PKT	R/W	Unlock on Magic Packet or Remote Wake-Up Packet. ■ 1: Enables the AXI manager to come out of the LPI mode only when the magic packet or remote wake-up packet is received. ■ 0: Enables the AXI manager to come out of the LPI mode when any packet is received. **Value After Reset:** 0x0 **Exists:** Always
13	ONEKBBE	R/W	1 KB Boundary Crossing Enable for the AXI Manager. ■ 1: The burst transfers performed by the AXI manager do not cross 1 KB boundary. ■ 0: The burst transfers performed by the AXI manager do not cross 4 KB boundary. **Value After Reset:** 0x0 **Exists:** Always

Table 11-497 Fields for Register: DMA_SysBus_Mode (continued)

Bits	Name	Memory Access	Description
12	AAL	R/W	Address-Aligned Beats. When this bit is set to 1, the AXI manager performs address-aligned burst transfers on Read and Write channels. **Value After Reset:** 0x0 **Exists:** Always
11	EAME	R/W	Enhanced Address Mode Enable. ■ 1: DMA manager enables the enhanced address mode (40-bit or 48-bit addressing mode). In this mode, the DMA engine uses either the 40-bit or 48-bit address, depending on the configuration. **Value After Reset:** 0x0 **Exists:** ((DWCXG_ADDRWIDTH != 32))
10	AALE	R/W	Automatic AXI LPI enable. ■ 1: Enables the AXI manager to enter into LPI state when there is no activity in DWC_25gmac for the number of system clock cycles programmed in the LPIEI field of AXI_LPI_Entry_Interval register. **Value After Reset:** 0x0 **Exists:** (DWCXG==2)
9:8	Reserved_9_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7	BLEN256	R/W	AXI Burst Length 256. When this bit is set to 1, the AXI manager can select a burst length of 256 on the AXI interface. **Value After Reset:** 0x0 **Exists:** DWCXG_AXI_BL > 128
6	BLEN128	R/W	AXI Burst Length 128. When this bit is set to 1, the AXI manager can select a burst length of 128 on the AXI interface. **Value After Reset:** 0x0 **Exists:** DWCXG_AXI_BL > 64
5	BLEN64	R/W	AXI Burst Length 64. When this bit is set to 1, the AXI manager can select a burst length of 64 on the AXI interface. **Value After Reset:** 0x0 **Exists:** DWCXG_AXI_BL > 32

Table 11-497 Fields for Register: DMA_SysBus_Mode (continued)

Bits	Name	Memory Access	Description
4	BLEN32	R/W	AXI Burst Length 32. When this bit is set to 1, the AXI manager can select a burst length of 32 on the AXI interface. **Value After Reset:** 0x0 **Exists:** DWCXG_AXI_BL > 16
3	BLEN16	R/W	AXI Burst Length 16. When this bit and the UBL bit is set to 0, the AXI manager can select a burst length of 16 on the AXI interface. When UBL is set to 1, this bit has no effect as the AXI manager can always select a burst length up to 16 on the AXI interface. **Value After Reset:** 0x0 **Exists:** Always
2	BLEN8	R/W	AXI Burst Length 8. When this bit and the UBL bit is set to 0, the AXI manager can select a burst length of 8 on the AXI interface. When the UBL bit is set to 1, setting this bit has no effect. **Value After Reset:** 0x0 **Exists:** Always
1	BLEN4	R/W	AXI Burst Length 4. When this bit and the UBL bit is set to 0, the AXI manager can select a burst length of 4 on the AXI interface. When the UBL bit is set to 1, setting this bit has no effect. **Value After Reset:** 0x0 **Exists:** Always
0	UBL	R/W	AXI Undefined Burst Length. ■ 1: The AXI manager can perform burst transfers that are equal to or less than the maximum allowed burst length programmed in Bits[7:4]. ■ 0: The AXI manager performs one of the following burst transfers: ❑ Burst transfers of fixed burst lengths as indicated by the BLEN256, BLEN128, BLEN64, BLEN32, BLEN16, BLEN8, or BLEN4 field ❑ Burst transfers of length 1 When this bit is set and no BLEN bit is set, the AXI manager performs bursts transfers of any length up to 16. **Value After Reset:** 0x0 **Exists:** Always

11.6.3 DMA_Interrupt_Status

- **Description:** The application reads this Interrupt Status register during interrupt service routine or polling to determine the interrupt status of DMA channels, MTL queues, and the MAC.
- **Size:** 32 bits
- **Offset:** 0x3008
- **Exists:** Always

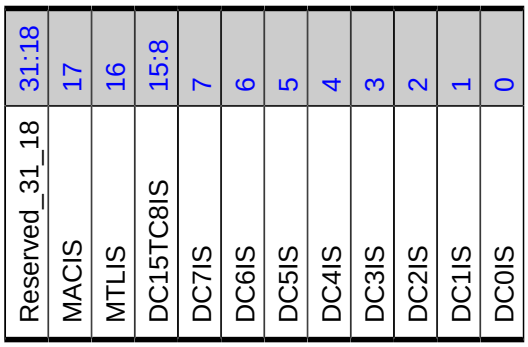


Table 11-498 Fields for Register: DMA_Interrupt_Status

Bits	Name	Memory Access	Description
31:18	Reserved_31_18	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
17	MACIS	R	MAC Interrupt Status. This bit indicates an interrupt event in the MAC. To reset this bit to 1'b0, the software must read the corresponding register in the MAC to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
16	MTLIS	R	MTL Interrupt Status. This bit indicates an interrupt event in the MTL. To reset this bit to 1'b0, the software must read the corresponding register in the MTL to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-498 Fields for Register: DMA_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
15:8	DC15TC8IS	R	DMA Channel 15 Interrupt Status to DMA Channel 8 Interrupt Status. To reset a bit to 1'b0, the software must read the Channel Status register in the corresponding DMA Channel (15 to 8) to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
7	DC7IS	R	DMA Channel 7 Interrupt Status. This bit indicates an interrupt event in DMA Channel 7. To reset this bit to 1'b0, the software must read the corresponding register in DMA Channel 7 to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
6	DC6IS	R	DMA Channel 6 Interrupt Status. This bit indicates an interrupt event in DMA Channel 6. To reset this bit to 1'b0, the software must read the corresponding register in DMA Channel 6 to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
5	DC5IS	R	DMA Channel 5 Interrupt Status. This bit indicates an interrupt event in DMA Channel 5. To reset this bit to 1'b0, the software must read the corresponding register in DMA Channel 5 to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-498 Fields for Register: DMA_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
4	DC4IS	R	DMA Channel 4 Interrupt Status. This bit indicates an interrupt event in DMA Channel 4. To reset this bit to 1'b0, the software must read the corresponding register in DMA Channel 4 to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
3	DC3IS	R	DMA Channel 3 Interrupt Status. This bit indicates an interrupt event in DMA Channel 3. To reset this bit to 1'b0, the software must read the corresponding register in DMA Channel 3 to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
2	DC2IS	R	DMA Channel 2 Interrupt Status. This bit indicates an interrupt event in DMA Channel 2. To reset this bit to 1'b0, the software must read the corresponding register in DMA Channel 2 to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	DC1IS	R	DMA Channel 1 Interrupt Status. This bit indicates an interrupt event in DMA Channel 1. To reset this bit to 1'b0, the software must read the corresponding register in DMA Channel 1 to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-498 Fields for Register: DMA_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
0	DC0IS	R	DMA Channel 0 Interrupt Status. This bit indicates an interrupt event in DMA Channel 0. To reset this bit to 1'b0, the software must read the corresponding register in DMA Channel 0 to get the exact cause of the interrupt and clear its source. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.6.4 AXI_Tx_AR_ACE_Control

- **Description:** This register is used to control the AXI4 Cache Coherency Signals for read transactions by all the Transmit DMA channels. The following signals of the AXI4 interface are driven with different values as programmed for corresponding type (descriptor, buffer1, buffer2) of access.
  - AXI Manager read control channel's cache type (arcache_m_o[3:0])
  - AXI Manager read control channel's Domain indicator signal (ardomain_m_o[1:0))
- **Size:** 32 bits
- **Offset:** 0x3010
- **Exists:** Always

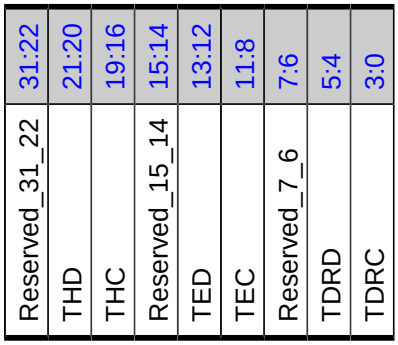


Table 11-499 Fields for Register: AXI_Tx_AR_ACE_Control

Bits	Name	Memory Access	Description
31:22	Reserved_31_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
21:20	THD	R/W	Transmit DMA First Packet Buffer or TSO Header Domain Control. For TSO not enabled : This field is used to drive ardomain_o[1:0] signal when Transmit DMA is accessing First Buffer of the Packet (First valid buffer with FD being set in the TDES3 of the Descriptor). For TSO enabled: This field is used to drive ardomain_o[1:0] signal when the Transmit DMA is accessing the TSO Header data. **Value After Reset:** 0x0 **Exists:** Always

Table 11-499 Fields for Register: AXI_Tx_AR_ACE_Control (continued)

Bits	Name	Memory Access	Description
19:16	THC	R/W	Transmit DMA First Packet Buffer or TSO Header Cache Control. For TSO not enabled : This field is used to drive arcache_o[3:0] signal when Transmit DMA is accessing First Buffer of the Packet (First valid buffer with FD being set in the TDES3 of the Descriptor.) For TSO enabled : This field is used to drive arcache_o[3:0] signal when the Transmit DMA is accessing the TSO Header data. **Value After Reset:** 0x0 **Exists:** Always
15:14	Reserved_15_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:12	TED	R/W	Transmit DMA Extended Packet Buffer or TSO Payload Domain Control. For TSO not enabled : This field is used to drive ardomain_o[1:0] signal when Transmit DMA is accessing the extended buffers (when packet is distributed across multiple buffers). For TSO enabled : This field is used to drive ardomain_o[1:0] signal when the Transmit DMA is accessing the TSO payload data. **Value After Reset:** 0x0 **Exists:** Always
11:8	TEC	R/W	Transmit DMA Extended Packet Buffer or TSO Payload Cache Control. For TSO not enabled : This field is used to drive arcache o[3:0] signal when Transmit DMA is accessing the extended buffers (when packet is distributed across multiple buffers). For TSO enabled : This field is used to drive arcache_o[3:0] signal when the Transmit DMA is accessing the TSO payload data. **Value After Reset:** 0x0 **Exists:** Always
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-499 Fields for Register: AXI_Tx_AR_ACE_Control (continued)

Bits	Name	Memory Access	Description
5:4	TDRD	R/W	Transmit DMA Read Descriptor Domain Control. This field is used to drive ardomain_o[1:0] signal when Transmit DMA engines access the Descriptor. **Value After Reset:** 0x0 **Exists:** Always
3:0	TDRC	R/W	Transmit DMA Read Descriptor Cache Control. This field is used to drive arcache_o[3:0] signal when Transmit DMA engines access the Descriptor. **Value After Reset:** 0x0 **Exists:** Always

11.6.5 AXI_Rx_AW_ACE_Control

- **Description:** This register is used to control the Cache Coherency Signals of the AXI4-ACE Lite on the Receive DMA. The programmable controllability is supported on the following signals of the AXI Write channel.
  - AXI Manager write control channel's cache type (awcache_m_o[3:0])
  - AXI Manager write control channel's Domain indicator signal (awdomain_m_o[1:0])
- **Size:** 32 bits
- **Offset:** 0x3018
- **Exists:** Always

31:30	29:28	27:24	23:22	21:20	19:16	15:14	13:12	11:8	7:6	5:4	3:0
Reserved_31_30	RDD	RDC	Reserved_23_22	RHD	RHC	Reserved_15_14	RPD	RPC	Reserved_7_6	RDWD	RDWC

Table 11-500 Fields for Register: AXI_Rx_AW_ACE_Control

Bits	Name	Memory Access	Description
31:30	Reserved_31_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29:28	RDD	R/W	Receive DMA Buffer Domain Control. This field is used to drive the awdomain_o[1:0] signal when Receive DMA is accessing the Buffer when Header and payload are not separated. **Value After Reset:** 0x0 **Exists:** Always
27:24	RDC	R/W	Receive DMA Buffer Cache Control. This field is used to drive awcache_o[3:0] signal when Receive DMA is accessing the Buffer when Header and payload are not separated. **Value After Reset:** 0x0 **Exists:** Always
23:22	Reserved_23_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-500 Fields for Register: AXI_Rx_AW_ACE_Control (continued)

Bits	Name	Memory Access	Description
21:20	RHD	R/W	Receive DMA Header Domain Control. This field is used to drive awdomain_o[1:0] and signal when Receive DMA is accessing the header Buffer when Header and payload are separated. **Value After Reset:** 0x0 **Exists:** Always
19:16	RHC	R/W	Receive DMA Header Cache Control. This field is used to drive awcache_o[3:0] and signal when Receive DMA is accessing the header Buffer when Header and payload are separated. **Value After Reset:** 0x0 **Exists:** Always
15:14	Reserved_15_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:12	RPD	R/W	Receive DMA Payload Domain Control. This field is used to drive awdomain_o[1:0] signal when Receive DMA is accessing the Payload Buffer when Header and payload are separated. **Value After Reset:** 0x0 **Exists:** Always
11:8	RPC	R/W	Receive DMA Payload Cache Control. This field is used to drive awcache_o[3:0] signal when Receive DMA is accessing the Payload Buffer when Header and payload are separated. **Value After Reset:** 0x0 **Exists:** Always
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:4	RDWD	R/W	Receive DMA Write Descriptor Domain Control. This field is used to drive awdomain_o[1:0] signal when Receive DMA access the Descriptor. **Value After Reset:** 0x0 **Exists:** Always

Table 11-500 Fields for Register: AXI_Rx_AW_ACE_Control (continued)

Bits	Name	Memory Access	Description
3:0	RDWC	R/W	Receive DMA Write Descriptor Cache Control. This field is used to drive awcache_o[3:0] signal when Receive DMA access the Descriptor. **Value After Reset:** 0x0 **Exists:** Always

11.6.6 AXI_TxRx_AWAR_ACE_Control

- **Description:** This register is used to control the AXI4 Cache Coherency Signals for Descriptor write transactions by all the TxDMA channels and Descriptor read transactions by all the RxDMA channels. It also controls the values to be driven on AXI Manager write control channel's protection control signal and AXI Manager read control channel's protection control signal (awprot_m_o and arprot_m_o).
- **Size:** 32 bits
- **Offset:** 0x301c
- **Exists:** Always

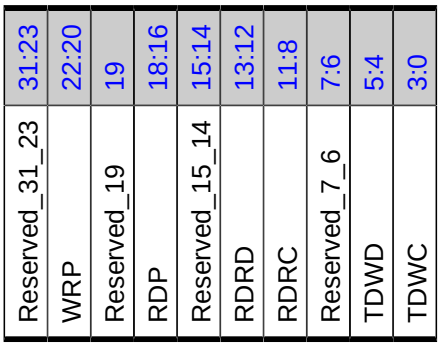


Table 11-501 Fields for Register: AXI_TxRx_AWAR_ACE_Control

Bits	Name	Memory Access	Description
31:23	Reserved_31_23	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
22:20	WRP	R/W	DMA Write Protection control. This field is used to drive the AXI Manager write control channel's protection control signal (awprot_m_o[2:0]) during all write Requests. **Value After Reset:** 0x0 **Exists:** Always
19	Reserved_19	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
18:16	RDP	R/W	DMA Read Protection control. This field is used to drive the AXI Manager read control channel's protection control signal (arprot_m_o[2:0]) during all read requests. **Value After Reset:** 0x0 **Exists:** Always

Table 11-501 Fields for Register: AXI_TxRx_AWAR_ACE_Control (continued)

Bits	Name	Memory Access	Description
15:14	Reserved_15_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:12	RDRD	R/W	Receive DMA Read Descriptor Domain control. This field is used to drive ardomain_o[1:0] signal when Receive DMA engine is reading the Descriptor. **Value After Reset:** 0x0 **Exists:** Always
11:8	RDRC	R/W	Receive DMA Read Descriptor Cache control. This field is used to drive arcache_o[3:0] signal when Receive DMA engine is reading the Descriptor. **Value After Reset:** 0x0 **Exists:** Always
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:4	TDWD	R/W	Transmit DMA Write Descriptor Domain control. This field is used to drive awdomain_o[1:0] signal when Transmit DMA is writing the Descriptor. **Value After Reset:** 0x0 **Exists:** Always
3:0	TDWC	R/W	Transmit DMA Write Descriptor Cache control. This field is used to drive awcache_o[3:0] signal when Transmit DMA is writing the Descriptor. **Value After Reset:** 0x0 **Exists:** Always

11.6.7 DMA_Debug_Status0

- **Description:** The Debug Status 0 register gives the Receive and Transmit process status for DMA Channel 0 - Channel 2 for debugging purpose.
- **Size:** 32 bits
- **Offset:** 0x3020
- **Exists:** Always

Table 11-502 Fields for Register: DMA_Debug_Status0

Bits	Name	Memory Access	Description
31:2	Reserved_31_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	AXRHSTS	R	AXI Manager Read Channel Status. When high, this bit indicates that the read channel of the AXI manager is active, and it is transferring the data. **Value After Reset:** 0x0 **Exists:** Always
0	AXWHSTS	R	AXI Manager Write Channel Status. ■ 1: Indicates that the write channel of the AXI manager is active, and it is transferring data. **Value After Reset:** 0x0 **Exists:** Always

11.6.8 DMA_Debug_Status1

- **Description:** The Debug Status1 register gives the per Transmit DMA active status.
- **Size:** 32 bits
- **Offset:** 0x3024
- **Exists:** ((DWCXG_NUM_DMA_CSR_CH>3&&DWCXG_EDMA==0)||DWCXG_EDMA==1)

Table 11-503 Fields for Register: DMA_Debug_Status1

Bits	Name	Memory Access	Description
x:y	Reserved_31_x	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 32-((DWCXG_NUM_DMA_TX_CH>16)?16:DWCXG_NUM_DMA_TX_CH) + (DWCXG_NUM_DMA_TX_CH>16)?16:DWCXG_NUM_DMA_TX_CH - 1 **Range Variable[y]:** (DWCXG_NUM_DMA_TX_CH>16)?16:DWCXG_NUM_DMA_TX_CH
x:0	TDAS	R	Transmit DMA Active Status. ■ 1: The corresponding Transmit DMA FSMs are not idle, that is, the FSMs are actively processing the descriptors or packet data. ■ 0: The corresponding Transmit DMA FSMs are in idle state. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (DWCXG_NUM_DMA_TX_CH>16)?16:DWCXG_NUM_DMA_TX_CH - 1

11.6.9 DMA_Debug_Status3

- **Description:** The Debug Status3 register gives the per Receive DMA active status.
- **Size:** 32 bits
- **Offset:** 0x302c
- **Exists:** ((DWCXG_NUM_DMA_CSR_CH>11&&DWCXG_EDMA==0)||DWCXG_EDMA==1)

Table 11-504 Fields for Register: DMA_Debug_Status3

Bits	Name	Memory Access	Description
x:y	Reserved_31_x	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 32-((DWCXG_NUM_DMA_RX_CH>16)?16:DWCXG_NUM_DMA_RX_CH) + (DWCXG_NUM_DMA_RX_CH>16)?16:DWCXG_NUM_DMA_RX_CH - 1 **Range Variable[y]:** (DWCXG_NUM_DMA_RX_CH>16)?16:DWCXG_NUM_DMA_RX_CH
x:0	RDAS	R	Receive DMA Active Status. When the bit is set to 1, the corresponding Receive DMA FSMs are not idle, that is, actively processing the descriptors or packet data. When the bit is set to 0, the corresponding Receive DMA FSMs are in idle state. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (DWCXG_NUM_DMA_RX_CH>16)?16:DWCXG_NUM_DMA_RX_CH - 1

11.6.10 DMA_Tx_EDMA_Control

- **Description:** The DMA_Tx_EDMA_Control register controls the sequence of all TxDMA operations with the DMA engine.
- **Size:** 32 bits
- **Offset:** 0x3040
- **Exists:** ((DWCXG == 2) || SNPS_RSVDPARAM_50==1)

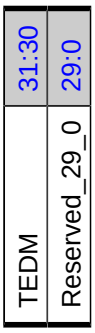


Table 11-505 Fields for Register: DMA_Tx_EDMA_Control

Bits	Name	Memory Access	Description
31:30	TEDM	R/W	Tx DMA Mode. This field controls the sequence of descriptor and data transfers in the DMA engine for all TxDMA channels is as follows: ■ 2'b00: Default mode of DMA, in which all transfers are independently triggered and can occur in parallel. This gives you the best performance. Use the other modes only for debug purposes. ■ 2'b01: The value of TDPS field is ignored and only one Descriptor is fetched at a time from the System Memory. The next descriptor is fetched only after the current descriptor is accepted by Data Transfer Engine. The Data transfer, Descriptor close and the next descriptor fetch can still occur in parallel. ■ 2'b10: The value of TDPS field is ignored and only one Descriptor is fetched at a time from the System Memory. The next descriptor is fetched only after the data transfer corresponding to current descriptor is completed. The Descriptor close and the next descriptor fetch can still happen in parallel. ■ 2'b11: All operations are sequential. Descriptor is fetched, data transfer is completed, descriptor is closed and then only the next descriptor is fetched. The value of TDPS is ignored in this mode. **Value After Reset:** 0x0 **Exists:** Always

Table 11-505 Fields for Register: DMA_Tx_EDMA_Control (continued)

Bits	Name	Memory Access	Description
29:0	Reserved_29_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.6.11 DMA_Rx_EDMA_Control

- **Description:** The DMA_Rx_EDMA_Control register controls the sequence of all RxDMA operations with the DMA engine.
- **Size:** 32 bits
- **Offset:** 0x3044
- **Exists:** ((DWCXG == 2) || SNPS_RSVDPARAM_50==1)

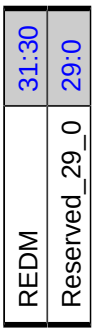


Table 11-506 Fields for Register: DMA_Rx_EDMA_Control

Bits	Name	Memory Access	Description
31:30	REDM	R/W	Rx DMA Mode. This field controls the sequence of descriptor and data transfers in the DMA engine for all RxDMA channels: ■ 2'b00: Default mode of DMA as explained in section 2.4.8 in which all transfers are independently triggered and can occur in parallel. This gives you the best performance. Use the other modes only for debug purposes. ■ 2'b01: The value of RDPS field is ignored and only 1 Descriptor is fetched at a time from the System Memory. The next descriptor is fetched only after the current descriptor is accepted by Data Transfer Engine. The Data transfer, Descriptor close and the next descriptor fetch can still happen in parallel. ■ 2'b10: The value of RDPS field is ignored and only 1 Descriptor is fetched at a time from the System Memory. The next descriptor is fetched only after the data transfer corresponding to current descriptor is completed. The Descriptor close and the next descriptor fetch can still happen in parallel. ■ 2'b11: All operations are sequential. Descriptor is fetched, data transfer is completed, descriptor is closed; then the next descriptor is fetched. The value of RDPS is ignored in this mode. **Value After Reset:** 0x0 **Exists:** Always

Table 11-506 Fields for Register: DMA_Rx_EDMA_Control (continued)

Bits	Name	Memory Access	Description
29:0	Reserved_29_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.6.12 AXI_LPI_Entry_Interval

- **Description:** This register is used to control the AXI LPI entry interval.
- **Size:** 32 bits
- **Offset:** 0x3050
- **Exists:** (DWCXG==2)

Table 11-507 Fields for Register: AXI_LPI_Entry_Interval

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3:0	LPIEI	R/W	LPI Entry Interval This field contains the number of system clock cycles, multiplied by 256, to wait for an activity in DWC_25gmac to enter into the AXI low-power state, 0 indicates 256 clock cycles. **Value After Reset:** 0x0 **Exists:** Always

11.6.13 DMA_TBS_CTRL0

- **Description:** This register is used to control the TBS0 attributes.
- **Size:** 32 bits
- **Offset:** 0x3054
- **Exists:** (DWCXG>1)&&DWCXG_TBS

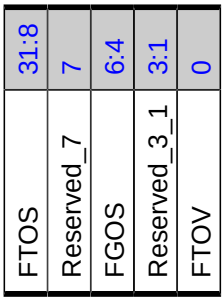


Table 11-508 Fields for Register: DMA_TBS_CTRL0

Bits	Name	Memory Access	Description
31:8	FTOS	R/W	Fetch Time Offset 0 ■ The value in units of 256 ns must be deducted from the Launch time to compute the Fetch Time. ■ Max value: 999,999,999 ns. Also, this must be smaller than CTR-1 value when ESTM mode is set, as this value is a modulo CTR value. **Value After Reset:** 0x0 **Exists:** Always
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	FGOS	R/W	Fetch GSN Offset 0 The number of GSN slots that must be deducted from the Launch GSN to compute the Fetch GSN. Value valid only when FTOV is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST)
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-508 Fields for Register: DMA_TBS_CTRL0 (continued)

Bits	Name	Memory Access	Description
0	FTOV	R/W	Fetch Time Offset 0 Valid When set indicates the FTOS field is valid. When not set, indicates the Fetch Offset is not valid and the DMA engine can fetch the frames from host memory without any time restrictions. **Values:** ■ 0x0 (INVALID): Fetch Time Offset is invalid ■ 0x1 (VALID): Fetch Time Offset is valid **Value After Reset:** 0x0 **Exists:** Always

11.6.14 DMA_TBS_CTRL1

- **Description:** This register is used to control the TBS1 attributes.
- **Size:** 32 bits
- **Offset:** 0x3058
- **Exists:** (DWCXG>1)&&DWCXG_TBS&&(DWCXG_NUM_DMA_TX_CH>1)

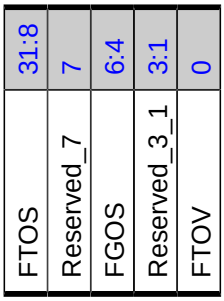


Table 11-509 Fields for Register: DMA_TBS_CTRL1

Bits	Name	Memory Access	Description
31:8	FTOS	R/W	Fetch Time Offset 1 ■ The value in units of 256 ns, that must be deducted from the Launch time to compute the Fetch Time. ■ Max value: 999,999,999 ns. Also, this must be smaller than CTR-1 value when ESTM mode is set, as this value is a modulo CTR value. **Value After Reset:** 0x0 **Exists:** Always
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	FGOS	R/W	Fetch GSN Offset 1 The number of GSN slots that must be deducted from the Launch GSN to compute the Fetch GSN. Value valid only when FTOV is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST)
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-509 Fields for Register: DMA_TBS_CTRL1 (continued)

Bits	Name	Memory Access	Description
0	FTOV	R/W	Fetch Time Offset 1 Valid When set indicates the FTOS field is valid. When not set, indicates the Fetch Offset is not valid and the DMA engine can fetch the frames from host memory without any time restrictions. **Values:** ■ 0x0 (INVALID): Fetch Time Offset is invalid ■ 0x1 (VALID): Fetch Time Offset is valid **Value After Reset:** 0x0 **Exists:** Always

11.6.15 DMA_TBS_CTRL2

- **Description:** This register is used to control the TBS2 attributes.
- **Size:** 32 bits
- **Offset:** 0x305c
- **Exists:** (DWCXG>1)&&DWCXG_TBS&&(DWCXG_NUM_DMA_TX_CH>2)

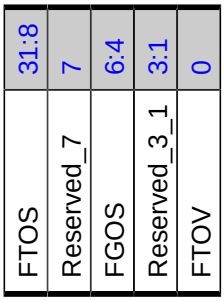


Table 11-510 Fields for Register: DMA_TBS_CTRL2

Bits	Name	Memory Access	Description
31:8	FTOS	R/W	Fetch Time Offset 2 The value in units of 256 ns, that must be deducted from the Launch time to compute the Fetch Time. Max value: 999,999,999 ns. This must be smaller than CTR-1 value when ESTM mode is set, as this value is a modulo CTR value. **Value After Reset:** 0x0 **Exists:** Always
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	FGOS	R/W	Fetch GSN Offset 2 The number of GSN slots that must be deducted from the Launch GSN to compute the Fetch GSN. Value valid only when FTOV is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST)
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-510 Fields for Register: DMA_TBS_CTRL2 (continued)

Bits	Name	Memory Access	Description
0	FTOV	R/W	Fetch Time Offset 2 Valid When set indicates the FTOS field is valid. When not set, indicates the Fetch Offset is not valid and the DMA engine can fetch the frames from host memory without any time restrictions. **Values:** ■ 0x0 (INVALID): Fetch Time Offset is invalid ■ 0x1 (VALID): Fetch Time Offset is valid **Value After Reset:** 0x0 **Exists:** Always

11.6.16 DMA_TBS_CTRL3

- **Description:** This register is used to control the TBS3 attributes.
- **Size:** 32 bits
- **Offset:** 0x3060
- **Exists:** (DWCXG>1)&&DWCXG_TBS&&(DWCXG_NUM_DMA_TX_CH>3)

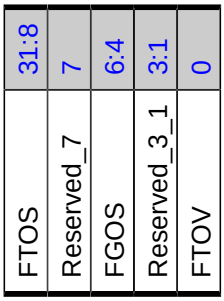


Table 11-511 Fields for Register: DMA_TBS_CTRL3

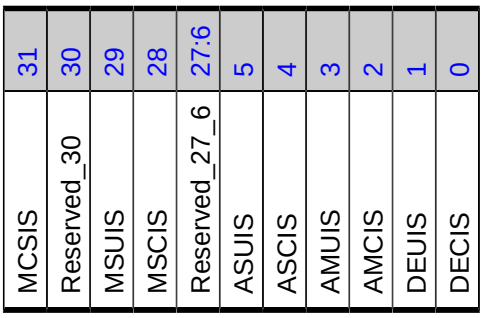
Bits	Name	Memory Access	Description
31:8	FTOS	R/W	Fetch Time Offset 3 The value in units of 256 ns, that has to be deducted from the Launch time to compute the Fetch Time. Max value: 999,999,999 ns. Also, this must be smaller than CTR-1 value when ESTM mode is set, as this value is a modulo CTR value. **Value After Reset:** 0x0 **Exists:** Always
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:4	FGOS	R/W	Fetch GSN Offset 3 The number GSN slots that must be deducted from the Launch GSN to compute the Fetch GSN. Value valid only when FTOV is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_AV_EST)
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-511 Fields for Register: DMA_TBS_CTRL3 (continued)

Bits	Name	Memory Access	Description
0	FTOV	R/W	Fetch Time Offset 3 Valid When set indicates the FTOS field is valid. When not set, indicates the Fetch Offset is not valid and the DMA engine can fetch the frames from host memory without any time restrictions. **Values:** ■ 0x0 (INVALID): Fetch Time Offset is invalid ■ 0x1 (VALID): Fetch Time Offset is valid **Value After Reset:** 0x0 **Exists:** Always

11.6.17 DMA_Safety_Interrupt_Status

- **Description:** This register indicates summary (whether error occurred in DMA/MTL/MAC and correctable/uncorrectable) of the Automotive Safety related error interrupts.
- **Size:** 32 bits
- **Offset:** 0x3064
- **Exists:** ((DWCXG_ASP_ALL||DWCXG_ASP_ECC)&&(DWCXG>1))



31	30	29	28	27:6	5	4	3	2	1	0
MCSIS	Reserved_30	MSUIS	MSCIS	Reserved_27_6	ASUIS	ASCIS	AMUIS	AMCIS	DEUIS	DECIS

Table 11-512 Fields for Register: DMA_Safety_Interrupt_Status

Bits	Name	Memory Access	Description
31	MCSIS	R	MAC Safety Unrecoverable Interrupt Status Indicates an unrecoverable Safety related Interrupt is set in the MAC module. When this bit is set, to get the cause of the safety interrupt in the MAC, read the MAC_DPP_FSM_Interrupt_Status and DMA_DPP_Interrupt_Status registers. **Values:** ■ 0x0 (INACTIVE): MAC Safety Uncorrectable Interrupt Status not detected ■ 0x1 (ACTIVE): MAC Safety Uncorrectable Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
30	Reserved_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-512 Fields for Register: DMA_Safety_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
29	MSUIS	R	MTL Safety Unrecoverable error Interrupt Status This bit indicates an uncorrectable error interrupt event in MTL. To get exact cause of the interrupt, the software must read the MTL_Safety_Interrupt_Status register. **Values:** ■ 0x0 (INACTIVE): MTL Safety Uncorrectable error Interrupt Status not detected ■ 0x1 (ACTIVE): MTL Safety Uncorrectable error Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
28	MSCIS	R	MTL Safety Correctable error Interrupt Status This bit indicates a correctable error interrupt event in MTL. To get exact cause of the interrupt, the software must read the MTL_Safety_Interrupt_Status register. **Values:** ■ 0x0 (INACTIVE): MTL Safety Correctable error Interrupt Status not detected ■ 0x1 (ACTIVE): MTL Safety Correctable error Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
27:6	Reserved_27_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-512 Fields for Register: DMA_Safety_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
5	ASUIS	R	AXI Subordinate interface Safety Uncorrectable error Interrupt Status Indicates an interrupt event in the AXI safety feature. To get the cause of the interrupt, read the DMA_AS_Sfty_ECC_Intr_Sts and DMA_AS_Sfty_DPP_Intr_Sts. **Values:** ■ 0x0 (INACTIVE): AXI Manager Interface Safety Uncorrectable error Interrupt Status not detected ■ 0x1 (ACTIVE): AXI Manager Interface Safety Uncorrectable error Interrupt Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)) **Testable:** untestable
4	ASCIS	R	AXI Subordinate interface Safety Correctable error Interrupt Status Indicates an interrupt event in the AXI safety feature. To get the cause of the interrupt, read the DMA_AS_Sfty_ECC_Intr_Sts register. **Values:** ■ 0x0 (INACTIVE): AXI Manager Interface Safety Correctable error Interrupt Status not detected ■ 0x1 (ACTIVE): AXI Manager Interface Safety Correctable error Interrupt Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)) **Testable:** untestable
3	AMUIS	R	AXI Manager interface Safety Uncorrectable error Interrupt Status This bit indicates an interrupt event in the AXI safety feature. To get the exact cause of the interrupt, the application must read DMA_AM_Sfty_ECC_Intr_Sts and DMA_AM_Sfty_DPP_Intr_Sts. **Values:** ■ 0x0 (INACTIVE): AXI Manager Interface Safety Uncorrectable error Interrupt Status not detected ■ 0x1 (ACTIVE): AXI Manager Interface Safety Uncorrectable error Interrupt Status detected **Value After Reset:** 0x0 **Exists:** ((DWCXG_ASP_PPE==2)) **Testable:** untestable

Table 11-512 Fields for Register: DMA_Safety_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
2	AMCIS	R	AXI Manager interface Safety Correctable error Interrupt Status This bit indicates an interrupt event in the AXI safety feature. To get the exact cause of the interrupt, the application must read DMA_AM_Sfty_ECC_Intr_Sts. **Values:** ■ 0x0 (INACTIVE): AXI Manager Interface Safety Correctable error Interrupt Status not detected ■ 0x1 (ACTIVE): AXI Manager Interface Safety Correctable error Interrupt Status detected **Value After Reset:** 0x0 **Exists:** ((DWCXG_ASP_PPE==2)) **Testable:** untestable
1	DEUIS	R	DMA ECC Uncorrectable error Interrupt Status This bit indicates an interrupt event in the DMA ECC safety feature. To get the exact cause of the interrupt, the application must read the DMA_ECC_Interrupt_Status register. **Values:** ■ 0x0 (INACTIVE): DMA ECC Uncorrectable error Interrupt Status not detected ■ 0x1 (ACTIVE): DMA ECC Uncorrectable error Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
0	DECIS	R	DMA ECC Correctable error Interrupt Status This bit indicates an interrupt event in the DMA ECC safety feature. To get the exact cause of the interrupt, the application must read the DMA_ECC_Interrupt_Status register. **Values:** ■ 0x0 (INACTIVE): DMA ECC Correctable error Interrupt Status not detected ■ 0x1 (ACTIVE): DMA ECC Correctable error Interrupt Status detected **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.6.18 DMA_ECC_Interrupt_enable

- **Description:** This register is used to enable the Automotive Safety related TSO memory ECC error interrupt.
- **Size:** 32 bits
- **Offset:** 0x3068
- **Exists:** (DWCXG_ASP_ECC&&(DWCXG_TSO_MEM_EN||DWCXG_DCACHE_MEM_EN)&&(DWCXG>1))

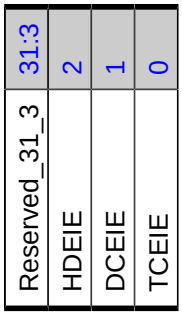


Table 11-513 Fields for Register: DMA_ECC_Interrupt_enable

Bits	Name	Memory Access	Description
31:3	Reserved_31_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	HDEIE	R/W	**Values:** ■ 0x0 (DISABLE): HDMA CSR Memory Correctable Error Interrupt is disabled ■ 0x1 (ENABLE): HDMA CSR Memory Correctable Error Interrupt is enabled **Value After Reset:** 0x1 **Exists:** SNPS_RSVDPARAM_50

Table 11-513 Fields for Register: DMA_ECC_Interrupt_enable (continued)

Bits	Name	Memory Access	Description
1	DCEIE	R/W	DCACHE memory Correctable Error Interrupt Enable When set, generates an interrupt when a correctable error is detected at the DMA DCACHE memory interface. It is indicated in the DCES bit of DMA_ECC_Interrupt_Status register. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): Descriptor Cache Memory Correctable Error Interrupt is disabled ■ 0x1 (ENABLE): Descriptor Cache Memory Correctable Error Interrupt is enabled **Value After Reset:** 0x1 **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN)
0	TCEIE	R/W	TSO memory Correctable Error Interrupt Enable When set, generates an interrupt when a correctable error is detected at the DMA TSO memory interface. It is indicated in the TCES bit of DMA_ECC_Interrupt_Status register. When reset, this event does not generate an interrupt. **Values:** ■ 0x0 (DISABLE): TSO memory Correctable Error Interrupt is disabled ■ 0x1 (ENABLE): TSO memory Correctable Error Interrupt is enabled **Value After Reset:** 0x1 **Exists:** DWCXG_TSO_MEM_EN

11.6.19 DMA_ECC_Interrupt_Status

- **Description:** This register indicates the Automotive Safety related TSO memory ECC error interrupt status.
- **Size:** 32 bits
- **Offset:** 0x306c
- **Exists:** (DWCXG_ASP_ECC&&(DWCXG_TSO_MEM_EN||DWCXG_DCACHE_MEM_EN)&&(DWCXG>1))

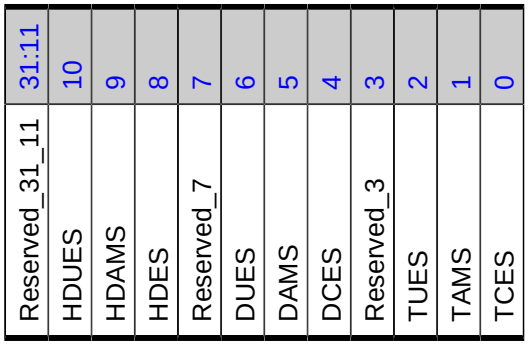


Table 11-514 Fields for Register: DMA_ECC_Interrupt_Status

Bits	Name	Memory Access	Description
31:11	Reserved_31_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
10	HDUES	R/W	**Values:** ■ 0x0 (INACTIVE): HDMA memory Uncorrectable Error status not detected ■ 0x1 (ACTIVE): HDMA memory Uncorrectable Error status detected **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50 **Testable:** untestable
9	HDAMS	R/W	**Values:** ■ 0x0 (INACTIVE): HDMA memory Address Mismatch status not detected ■ 0x1 (ACTIVE): HDMA memory Address Mismatch status detected **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50 **Testable:** untestable

Table 11-514 Fields for Register: DMA_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
8	HDES	R/W	**Values:** ■ 0x0 (INACTIVE): HDMA memory Correctable Error status not detected ■ 0x1 (ACTIVE): HDMA memory Correctable Error status detected **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50 **Testable:** untestable
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6	DUES	R/W	DMA DCACHE memory Uncorrectable Error Status ■ 1: Indicates that an uncorrectable error is detected at DMA DCACHE memory interface. **Values:** ■ 0x0 (INACTIVE): DMA DCACHE memory Uncorrectable Error status not detected ■ 0x1 (ACTIVE): DMA DCACHE memory Uncorrectable Error status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Testable:** untestable
5	DAMS	R/W	DMA DCACHE memory Address Mismatch Status ■ 1: Indicates that the address mismatch is found for address bus of DMA DCACHE memory. **Values:** ■ 0x0 (INACTIVE): DMA DCACHE memory Address Mismatch status not detected ■ 0x1 (ACTIVE): DMA DCACHE memory Address Mismatch status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Testable:** untestable

Table 11-514 Fields for Register: DMA_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
4	DCES	R/W	DMA DCACHE memory Correctable Error Status When this bit is set, it indicates that correctable error is detected at DMA DCACHE memory interface. **Values:** ■ 0x0 (INACTIVE): DMA DCACHE memory Correctable Error status not detected ■ 0x1 (ACTIVE): DMA DCACHE memory Correctable Error status detected **Value After Reset:** 0x0 **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Testable:** untestable
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	TUES	R/W	DMA TSO memory Uncorrectable Error Status When set, indicates that an uncorrectable error is detected at DMA TSO memory interface. **Values:** ■ 0x0 (INACTIVE): DMA TSO memory Uncorrectable Error status not detected ■ 0x1 (ACTIVE): DMA TSO memory Uncorrectable Error status detected **Value After Reset:** 0x0 **Exists:** DWCXG_TSO_EN **Testable:** untestable
1	TAMS	R/W	DMA TSO memory Address Mismatch Status When this bit is set, it indicates that address mismatch is found for address bus of DMA TSO memory. **Values:** ■ 0x0 (INACTIVE): DMA TSO memory Address Mismatch status not detected ■ 0x1 (ACTIVE): DMA TSO memory Address Mismatch status detected **Value After Reset:** 0x0 **Exists:** DWCXG_TSO_EN **Testable:** untestable

Table 11-514 Fields for Register: DMA_ECC_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
0	TCES	R/W	DMA TSO memory Correctable Error Status When this bit is set, it indicates that correctable error is detected at DMA TSO memory interface. **Values:** ■ 0x0 (INACTIVE): DMA TSO memory Correctable Error status not detected ■ 0x1 (ACTIVE): DMA TSO memory Correctable Error status detected **Value After Reset:** 0x0 **Exists:** DWCXG_TSO_EN **Testable:** untestable

11.6.20 DMA_DPP_Control

- **Description:** The DMA_DPP_Control establishes the operating mode of Data Parity protection and error injection per channel in DMA.
- **Size:** 32 bits
- **Offset:** 0x3070
- **Exists:** DWCXG_ASP_DPP&&(DWCXG>1)

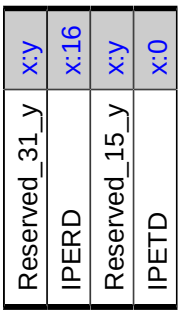


Table 11-515 Fields for Register: DMA_DPP_Control

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 16-(SNPS_RSVDPARAM_50?DWCXG_NUM_RXQ:DWCXG_NUM_DMA_RX_CH) + 16+(SNPS_RSVDPARAM_50?DWCXG_NUM_RXQ:DWCXG_NUM_DMA_RX_CH) - 1 **Range Variable[y]:** 16+(SNPS_RSVDPARAM_50?DWCXG_NUM_RXQ:DWCXG_NUM_DMA_RX_CH)

Table 11-515 Fields for Register: DMA_DPP_Control (continued)

Bits	Name	Memory Access	Description
x:16	IPERD	R/W	Insert Parity Error in DMA RD parity checker ■ 1: Parity/data bit of the first valid input parity/data of DMA Read Rx descriptor parity checker (or at PC1#rd) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. DWC_25gmac clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity Error in DMA RD Parity Checker is disabled ■ 0x1 (ENABLE): Insert Parity Error in DMA RD Parity Checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG>=2) **Testable:** untestable **Range Variable[x]:** (SNPS_RSVDPARAM_50?DWCXG_NUM_RXQ:DWCXG_NUM_DMA_RX_CH) + 15
x:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 16-(SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH) + (SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH) - 1 **Range Variable[y]:** (SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH)

Table 11-515 Fields for Register: DMA_DPP_Control (continued)

Bits	Name	Memory Access	Description
x:0	IPETD	R/W	Insert Parity Error in DMA TD parity checker ■ 1: Parity/data bit of the first valid input parity/data of DMA Read Tx descriptor parity checker (or at PC1#td) is flipped. Based on the EIM/BLEI field of MTL_DPP_ECC_EIC register, software can corrupt any one bit of parity/data. DWC_25gmac clears this bit after the respective parity bit is flipped. **Values:** ■ 0x0 (DISABLE): Insert Parity Error in DMA TD Parity Checker is disabled ■ 0x1 (ENABLE): Insert Parity Error in DMA TD Parity Checker is enabled **Value After Reset:** 0x0 **Exists:** (DWCXG>=2) **Testable:** untestable **Range Variable[x]:** (SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH) - 1

11.6.21 DMA_DPP_Interrupt_Status

- **Description:** This register contains the status of Automotive Safety related Data Path Parity Errors per channel in DMA.
- **Size:** 32 bits
- **Offset:** 0x3074
- **Exists:** DWCXG_ASP_DPP&&(DWCXG>1)

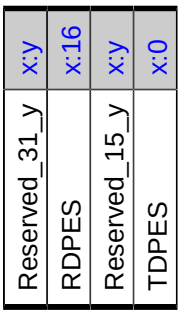


Table 11-516 Fields for Register: DMA_DPP_Interrupt_Status

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 16-(SNPS_RSVDPARAM_50?DWCXG_NUM_RXQ:DWCXG_NUM_DMA_RX_CH) + 16+(SNPS_RSVDPARAM_50?DWCXG_NUM_RXQ:DWCXG_NUM_DMA_RX_CH) - 1 **Range Variable[y]:** 16+(SNPS_RSVDPARAM_50?DWCXG_NUM_RXQ:DWCXG_NUM_DMA_RX_CH)

Table 11-516 Fields for Register: DMA_DPP_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
x:16	RDPES	R/W	Read Rx Descriptor Parity checker Error Status ■ 1: Indicates that a parity error is detected at the DMA Read Rx descriptor per channel parity checker (or at PC2#rd) is flipped. It is cleared when written to 1. **Values:** ■ 0x0 (INACTIVE): Read Receive Descriptor(Per Channel) Parity checker Error Status not detected ■ 0x1 (ACTIVE): Read Receive Descriptor(Per Channel) Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG>=2) **Testable:** untestable **Range Variable[x]:** (SNPS_RSVDPARAM_50?DWCXG_NUM_RXQ:DWCXG_NUM_DMA_RX_CH) + 15
x:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 16-(SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH) + (SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH) - 1 **Range Variable[y]:** (SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH)

Table 11-516 Fields for Register: DMA_DPP_Interrupt_Status (continued)

Bits	Name	Memory Access	Description
x:0	TDPES	R/W	Read Tx Descriptor Parity checker Error Status ■ 1: Indicates that a parity error is detected at the DMA Read Tx descriptor per channel parity checker (or at PC2#td) is flipped. It is cleared when written to 1. **Values:** ■ 0x0 (INACTIVE): Read Transmit Descriptor(Per Channel) Parity checker Error Status not detected ■ 0x1 (ACTIVE): Read Transmit Descriptor(Per Channel) Parity checker Error Status detected **Value After Reset:** 0x0 **Exists:** (DWCXG>=2) **Testable:** untestable **Range Variable[x]:** (SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH) - 1

11.6.22 DMA_CH_Ind_Ctrl

- **Description:** This register is used to program Indirect Access DMA configuration registers
- **Size:** 32 bits
- **Offset:** 0x3080
- **Exists:** DWCXG == 2

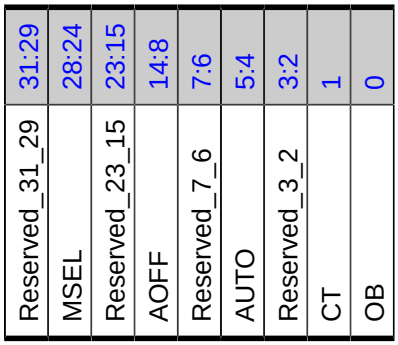


Table 11-517 Fields for Register: DMA_CH_Ind_Ctrl

Bits	Name	Memory Access	Description
31:29	Reserved_31_29	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-517 Fields for Register: DMA_CH_Ind_Ctrl (continued)

Bits	Name	Memory Access	Description
28:24	MSEL	R/W	Mode Select This field selects a group of identical registers. ■ 00000: TxExtCfg (Tx Extended Config) ■ 00001: RxExtCfg (Rx Extended Config) ■ 00010: TxDbgSts ■ 00011: RxDbgSts ■ 00100: TxDescCtrl ■ 00101: RxDescCtrl ■ 00110-00111: reserved for future use ■ 01000: RxPktInfoBuf_BaseAddr_Hi ■ 01001: RxPktInfoBuf_BaseAddr_Lo ■ 01010: RxPktInfoBuf_CurrOffset ■ 01011-01110: reserved for future use ■ 01111: RxFrameBuf_BaseAddr_Hi ■ 10000: RxFrameBuf0_BaseAddr_Lo ■ 10001: RxFrameBuf1_BaseAddr_Lo ■ 10010: RxFrameBuf2_BaseAddr_Lo ■ 10011: RxFrameBuf3_BaseAddr_Lo ■ 10100-11111: reserved for future use MSEL field is used in the indirect access of the DMA_(#MSEL)_(#AOFF) This field must be set along with initiation of read/write to the DMA_(#MSEL)_(#AOFF) register and must not be changed until the OB is reset. **Value After Reset:** 0x0 **Exists:** Always
23:15	Reserved_23_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
14:8	AOFF	R/W	Address Offset This field selects the offset address of per DMA Channel registers specified by MSEL field. ■ 0000000: CH0 ■ 0000001: CH1 ■ ---- ■ 1111111: CH127 Refer the valid range of DMA Channel registers specified by MSEL field to program this field. AOFF field is used in the indirect access of the DMA_(#MSEL)_(#AOFF) This field must be set along with initiation of read/write to the DMA_(#MSEL)_(#AOFF) register and must not be changed until the OB is reset. **Value After Reset:** 0x0 **Exists:** Always

Table 11-517 Fields for Register: DMA_CH_Ind_Ctrl (continued)

Bits	Name	Memory Access	Description
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
5:4	AUTO	R/W	Auto Increment ■ 00: No auto increment ■ 01: Auto increment MSEL only ■ 10: Auto increment AOFF only ■ 11: No auto increment **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	CT	R/W	Command Type This bit indicates if the current register access is a read or a write operation. ■ 1: Indicates a read operation. ■ 0: Indicates a write operation. **Value After Reset:** 0x0 **Exists:** Always
0	OB	R/W	Operation Busy This bit is set along with a read or write command for initiating the indirect access to DMA_(#MSEL)_(#AOFF) register. This bit is reset when the read or write command to DMA_(#MSEL)_(#AOFF) register is complete. The next indirect register access can be initiated only after this bit is reset. During a write operation, the bit is reset only after the data has been written into the DMA_(#MSEL)_(#AOFF) register. During a read operation, the data must be read from the DMA_CH_Ind_Data register, only after this bit is reset. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.6.23 DMA_CH_Ind_Data

- **Description:** This register holds the read/write data for Indirect Access of the registers indicated by DMA_(#MSEL)_(#AOFF). During the read access, this field contains valid read data only after the OB bit is reset. During the write access, this field must be valid prior setting the OB bit in the DMA_CH_Ind_Data register. Once auto increment is set, application is allowed to write without setting OB bit of DMA_CH_Ind_Ctrl register.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** DWCXG == 2

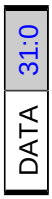


Table 11-518 Fields for Register: DMA_CH_Ind_Data

Bits	Name	Memory Access	Description
31:0	DATA	R/W	DMA Indirect Address Data **Value After Reset:** 0x0 **Exists:** Always

11.6.24 PDMA_CH(#i)_TxExtCfg (for i = 0; i <= DWCXG_NUM_TXQ-1)

- **Description:** This register controls the parameters (like PBL, ORRQ, etc) that corresponds to this PDMA/Hardware Engine including the mapping of this TDMA to a specific TrafficClass.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_TXQ> 0)

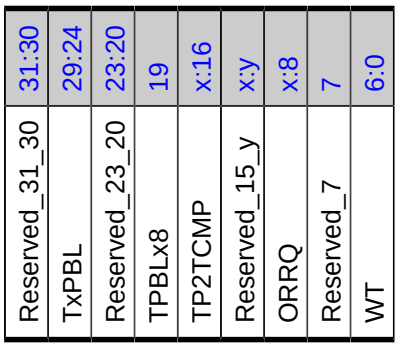


Table 11-519 Fields for Register: PDMA_CH(#i)_TxExtCfg (for i = 0; i <= DWCXG_NUM_TXQ-1)

Bits	Name	Memory Access	Description
31:30	Reserved_31_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29:24	TxPBL	R/W	Transmit Programmable Burst Length. This field controls the maximum size of burst transaction that can be initiated by the TX DMA channel for packet data transfers. The TX DMA channel attempts to fetch the data of this burst size if available, from the Transmit Buffer pointed by the descriptor. You can program PBL with any of the following values: 1, 2, 4, 8, 16, or 32. Any other value results in undefined behavior. To transfer more than 32 beats, perform the following steps: ■ Set the PBLx8 mode in VDMA_CH${i}_Control register. ■ Set the PBL. **Note-1:** This PBL controls the maximum burst size of the requests from this internal DMA engine. The AXI Interface block can split this PBL requests from DMA engine into multiple AXI requests based on the programmed values in DMA_SysBus_Mode register. **Note-2:** The maximum limit programmed must be less than or equal to half the size of the corresponding Tx Queue depth (Queue Depth = Queue Size / Datawidth) **Value After Reset:** 0x0 **Exists:** Always

Table 11-519 Fields for Register: PDMA_CH(#i)_TxExtCfg (for i = 0; i <= DWCXG_NUM_TXQ-1) (continued)

Bits	Name	Memory Access	Description
23:20	Reserved_23_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19	TPBLx8	R/W	8xPBL mode. When this bit is set, the PBL value programmed in Bits[21:16] in VDMA_CH${i}_Tx_Control is multiplied eight times. Therefore, the DMA transfers the data in 8, 16, 32, 64, 128, and 256 beats depending on the PBL value. **Value After Reset:** 0x0 **Exists:** Always
x:16	TP2TCMP	R/W	Tx PDMA to TC Mapping This field indicates the TC number that has been mapped to the corresponding Tx PDMA. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_TC>1) **Range Variable[x]:** DWCXG_TCW + 15
x:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (16-8-(log(DWCXG_MAX_RDBREQ)/log(2))) + (8+(log(DWCXG_MAX_RDBREQ)/log(2))) - 1 **Range Variable[y]:** (8+(log(DWCXG_MAX_RDBREQ)/log(2)))
x:8	ORRQ	R/W	Read Outstanding Requests When set to a value greater than zero, acts as the maximum number of Read outstanding requests that can be serviced per DMA. When DWCXG_MAX_RDBREQ is - 64: Up to 63 Outstanding requests can be programmed - 32: Up to 31 outstanding requests can be programmed - 16: Up to 15 outstanding requests can be programmed - 8: Up to 7 outstanding requests can be programmed - 4: Up to 3 outstanding requests can be programmed - 2: Up to 1 outstanding requests can be programmed When set to 0, outstanding requests are configured by the parameter DWCXG_MAX_RDBREQ. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (log(DWCXG_MAX_RDBREQ)/log(2)) + 7

Table 11-519 Fields for Register: PDMA_CH(#i)_TxExtCfg (for i = 0; i <= DWCXG_NUM_TXQ-1) (continued)

Bits	Name	Memory Access	Description
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6:0	WT	R/W	Weights This field contains the weights of corresponding Tx DMA used for Weighted Round Robin Arbitration of Tx DMA Data Reads. The maximum value is 0x64. **Value After Reset:** 0x0 **Exists:** Always

11.6.25 PDMA_CH(#i)_TxDbgSts (for i = 0; i <= DWCXG_NUM_TXQ-1)

- **Description:** This register provides the active TX DMA numbers which is being processed by the different functions (Descriptor-Fetch, Descriptor-process, Data-Fetch, etc) of this PDMA/Hardware Engine.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_TXQ> 0)

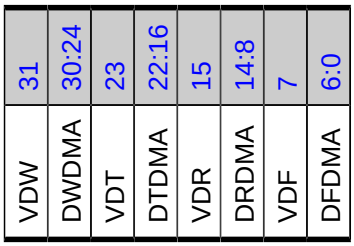


Table 11-520 Fields for Register: PDMA_CH(#i)_TxDbgSts (for i = 0; i <= DWCXG_NUM_TXQ-1)

Bits	Name	Memory Access	Description
31	VDW	R/W	Valid Descriptor Writeback When set to 1, indicates that no DMA is accessing the Data Transfer Virtual Function of Tx PDMA#i. When set to 0 indicates that the virtual function is accessed by a DMA indicated by DTDMA field **Value After Reset:** 0x0 **Exists:** Always
30:24	DWDMA	R/W	Descriptor Write DMA Indicates the DMA number currently accessing the Descriptor Writeback Virtual Function of Tx PDMA#i **Value After Reset:** 0x0 **Exists:** Always
23	VDT	R/W	Valid Data Transfer When set to 1, indicates that no DMA is accessing the Data Transfer Virtual Function of Tx PDMA#i. When set to 0 indicates that the virtual function is accessed by a DMA indicated by DTDMA field **Value After Reset:** 0x0 **Exists:** Always
22:16	DTDMA	R/W	Data Transfer DMA Indicates the DMA number currently accessing the Data Transfer Virtual Function of Tx PDMA#i **Value After Reset:** 0x0 **Exists:** Always

Table 11-520 Fields for Register: PDMA_CH(#i)_TxDbgSts (for i = 0; i <= DWCXG_NUM_TXQ-1) (continued)

Bits	Name	Memory Access	Description
15	VDR	R/W	Valid Descriptor Read When set to 1, indicates that no DMA is accessing the Descriptor Read Virtual Function of Tx PDMA#i. When set to 0 indicates that the virtual function is accessed by a DMA indicated by DRDMA field **Value After Reset:** 0x0 **Exists:** Always
14:8	DRDMA	R/W	Descriptor Read DMA number This field indicates the DMA number currently accessing the Descriptor Read Virtual Function of Tx PDMA#i **Value After Reset:** 0x0 **Exists:** Always
7	VDF	R/W	Valid Descriptor Fetch When set to 1, indicates that no DMA is accessing the Descriptor Fetch Virtual Function of Tx PDMA#i. When set to 0 indicates that the virtual function is accessed by a DMA indicated by DFDMA field **Value After Reset:** 0x0 **Exists:** Always
6:0	DFDMA	R/W	Descriptor Fetch DMA number This field indicates the DMA number currently accessing the Descriptor Fetch Virtual Function of Tx PDMA#i **Value After Reset:** 0x0 **Exists:** Always

11.6.26 VDMA_CH(#i)_TxDescCtrl (for i = 0; i <= DWCXG_NUM_DMA_TX_CH-1)

- **Description:** This register contains the per channel programming field for Tx Descriptor Cache size.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_DMA_TX_CH>0)

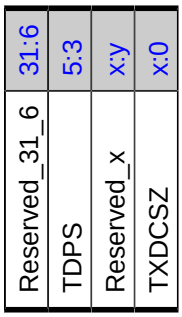


Table 11-521 Fields for Register: VDMA_CH(#i)_TxDescCtrl (for i = 0; i <= DWCXG_NUM_DMA_TX_CH-1)

Bits	Name	Memory Access	Description
31:6	Reserved_31_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
5:3	TDPS	R/W	Tx Descriptor Pre-fetch threshold Size. This field sets the, descriptor pre-fetching threshold for Tx channels. The DMA engine, initiates the request for descriptor pre fetched for a Tx DMA channel, only after the number of descriptors in the cache memory for that particular Tx DMA channel, falls below or equal to the number of descriptors provided by the below encoded value of this field. ■ 3'b000: 0 ■ 3'b001: For enhanced descriptor 1/8* (Transmit Channel Descriptor Cache Size as per TXDCSZ/32) Otherwise 1/8*(Transmit Channel Descriptor Cache Size as per TXDCSZ/16) ■ 3'b010: For enhanced descriptor 1/4* (Transmit Channel Descriptor Cache Size as per TXDCSZ/32) Otherwise 1/4*(Transmit Channel Descriptor Cache Size as per TXDCSZ/16) ■ 3'b011: For enhanced descriptor 1/2* (Transmit Channel Descriptor Cache Size as per TXDCSZ/32) Otherwise 1/2*(Transmit Channel Descriptor Cache Size as per TXDCSZ/16) ■ 3'b100: For enhanced descriptor 3/4* (Transmit Channel Descriptor Cache Size as per TXDCSZ/32) Otherwise

Table 11-521 Fields for Register: VDMA_CH(#i)_TxDescCtrl (for i = 0; i <= DWCXG_NUM_DMA_TX_CH-1) (continued)

Bits	Name	Memory Access	Description
			3/4*(Transmit Channel Descriptor Cache Size as per TXDCSZ/16) ■ 3'b101 - 3'b111: Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
x:y	Reserved_x	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** ((16*DWCXG_MAX_RDBREQ*2)>256?0:1) + ((16*DWCXG_MAX_RDBREQ*2)>256?3:2) - 1 **Range Variable[y]:** ((16*DWCXG_MAX_RDBREQ*2)>256?3:2)
x:0	TXDCSZ	R/W	This field indicates the programmable size of Descriptor Cache per Tx Channel. The maximum value is, 16 when DWCXG_MAX_RDBREQ<=8 otherwise 16* DWCXG_MAX_RDBREQ*2. This field must be programmed to a required value before starting the VDMA. The encoding is given as follows. **Note:** Bit 2 is reserved and RO when "Maximum Outstanding Read Burst Requests for each Tx DMA Channel" is less than or equal to 8, that is TXDCSZ value of greater than 2'b11 is not valid. - 000: No Space allocated - 001: 64 bytes - 010: 128 bytes - 011: 256 bytes - 100: 512bytes - 101: 1024bytes - 110: 2048bytes - 111: Invalid **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** ((16*DWCXG_MAX_RDBREQ*2)>256?3:2) - 1

11.6.27 PDMA_CH(#i)_RxExtCfg (for i = 0; i <= DWCXG_NUM_RXQ-1)

- **Description:** This register controls the parameters (like PBL, ORRQ, etc) that corresponds to this PDMA/Hardware Engine including the mapping of this PDMA to a specific TrafficClass.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&DWCXG_AXI_SUBSYS&&(DWCXG_NUM_RXQ> 0)

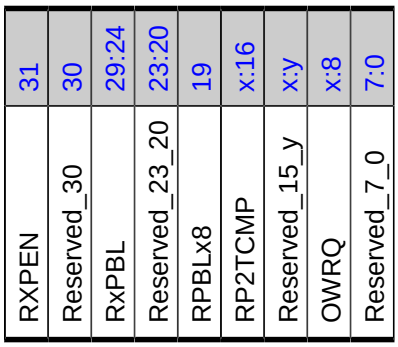


Table 11-522 Fields for Register: PDMA_CH(#i)_RxExtCfg (for i = 0; i <= DWCXG_NUM_RXQ-1)

Bits	Name	Memory Access	Description
31	RXPEN	R/W	Rx PDMA Enable When this bit is set, corresponding Rx PDMA is enabled and can be used for data transfer through it When this bit is reset, corresponding Rx PDMA should be considered as disabled and can not be considered for data transfer through it **Value After Reset:** 0x0 **Exists:** Always
30	Reserved_30	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
29:24	RxPBL	R/W	Receive Programmable Burst Length. This field controls the maximum size of burst transaction that can be initiated by the RX DMA channel for packet data transfers. The Rx DMA channel attempts to transfer the data of this burst size if available, to the Receive Buffer pointed by the descriptor. You can program RxPBL with any of the following values: 1, 2, 4, 8, 16, or 32. Any other value results in undefined behavior. It is recommended to use higher values (8 and higher) for better performance. A value of 0 might result in undefined behavior. To transfer more than 32 beats, perform the following steps: ■ Set the RPBLx8 mode in the PDMA_CH0_RxExtCfg register. ■ Set the RxPBL.

Table 11-522 Fields for Register: PDMA_CH(#i)_RxExtCfg (for i = 0; i <= DWCXG_NUM_RXQ-1) (continued)

Bits	Name	Memory Access	Description
			Note-1: This RxPBL controls the maximum burst size of the requests from this internal Rx DMA engine. The AXI Interface block can split this RxPBL requests from Rx DMA engine into multiple AXI requests based on the programmed values in DMA_SysBus_Mode register. **Note-2:**The maximum limit programmed must be less than or equal to half the size of the corresponding Rx Queue Depth (Queue Depth = Queue Size / Datawidth in bytes) of all the Rx Queues from which packets can be services by this Rx DMA . For example, in 64-bit configuration, there are 8 bytes per location. So, when Rx Queue Size is 256 bytes, the Rx Queue Depth = 256/8 = 32. The RxPBL must be programmed to less than or equal to total no. of locations in Rx Queue/2, so RxPBL must be programmed less than 32. **Value After Reset:** 0x0 **Exists:** Always
23:20	Reserved_23_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
19	RPBLx8	R/W	RPBLx8 mode. When this bit is set, the RxPBL value programmed in Bits[29:24] in PDMA_CH0_RxExtCfg is multiplied eight times. Therefore, the PDMA transfers the data in 8, 16, 32, 64, 128, and 256 beats depending on the RxPBL value. **Value After Reset:** 0x0 **Exists:** Always
x:16	RP2TCMP	R/W	Rx PDMA to TC Mapping This field indicates the TC number that has been mapped to the corresponding Rx PDMA. **Value After Reset:** 0x0 **Exists:** DWCXG_NUM_RXQ>1 **Range Variable[x]:** DWCXG_TCW + 15
x:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (16-8-(log(DWCXG_MAX_WRBREQ)/log(2))) + (8+(log(DWCXG_MAX_WRBREQ)/log(2))) - 1 **Range Variable[y]:** (8+(log(DWCXG_MAX_WRBREQ)/log(2)))

Table 11-522 Fields for Register: PDMA_CH(#i)_RxExtCfg (for i = 0; i <= DWCXG_NUM_RXQ-1) (continued)

Bits	Name	Memory Access	Description
x:8	OWRQ	R/W	Write Outstanding Requests When set to a value greater than zero, acts as maximum number of Write outstanding requests that can be serviced per DMA. When DWCXG_MAX_WRBREQ is - 64: Up to 63 Outstanding requests can be programmed - 32: Up to 31 outstanding requests can be programmed - 16: Up to 15 outstanding requests can be programmed - 8: Up to 7 outstanding requests can be programmed - 4: Up to 3 outstanding requests can be programmed - 2: Up to 1 outstanding requests can be programmed When set to 0, outstanding requests are configured by the parameter DWCXG_MAX_WRBREQ. When set to 0, backward compatibility is maintained. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** (log(DWCXG_MAX_WRBREQ)/log(2)) + 7
7:0	Reserved_7_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.6.28 PDMA_CH(#i)_RxDbgSts (for i = 0; i <= DWCXG_NUM_RXQ-1)

- **Description:** This register provides the active RX DMA numbers which is being processed by the different functions (Descriptor-Fetch, Descriptor-process, Data-Transfer, etc) of this PDMA/Hardware Engine.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_RXQ> 0)

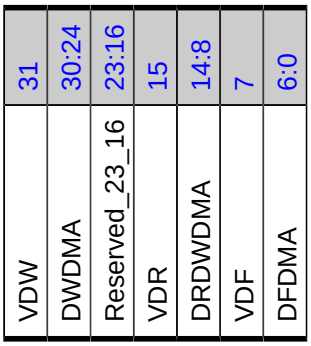


Table 11-523 Fields for Register: PDMA_CH(#i)_RxDbgSts (for i = 0; i <= DWCXG_NUM_RXQ-1)

Bits	Name	Memory Access	Description
31	VDW	R/W	Valid Descriptor Writeback When set to 1, indicates that no DMA is accessing the Data Transfer Virtual Function of Rx PDMA#i. When set to 0 indicates that the virtual function is accessed by a DMA indicated by DTDMA field **Value After Reset:** 0x0 **Exists:** Always
30:24	DWDMA	R/W	Descriptor Write DMA Indicates the DMA number currently accessing the Descriptor Writeback Virtual Function of Rx PDMA#i **Value After Reset:** 0x0 **Exists:** Always
23:16	Reserved_23_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15	VDR	R/W	Valid Descriptor Read and Data Write When set to 1, indicates that no DMA is accessing the Descriptor Read and Data write Virtual Function of Rx PDMA#i. When set to 0 indicates that the virtual function is accessed by a DMA indicated by DRDMA field **Value After Reset:** 0x0 **Exists:** Always

Table 11-523 Fields for Register: PDMA_CH(#i)_RxDbgSts (for i = 0; i <= DWCXG_NUM_RXQ-1) (continued)

Bits	Name	Memory Access	Description
14:8	DRDWDMA	R/W	Descriptor Read Descriptor Write DMA Number Indicates the DMA number currently accessing the Descriptor Read and Data write Virtual Function of Rx PDMA#i **Value After Reset:** 0x0 **Exists:** Always
7	VDF	R/W	Valid Descriptor Fetch When set to 1, indicates that no DMA is accessing the Descriptor Fetch Virtual Function of Rx PDMA#i. When set to 0 indicates that the virtual function is accessed by a DMA indicated by DFDMA field **Value After Reset:** 0x0 **Exists:** Always
6:0	DFDMA	R/W	Descriptor Fetch Number Indicates the DMA number currently accessing the Descriptor Fetch Virtual Function of Rx PDMA#i **Value After Reset:** 0x0 **Exists:** Always

11.6.29 VDMA_CH(#i)_RxDescCtrl (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

- **Description:** This register contains the per channel programming field for Rx Descriptor Cache size.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_DMA_RX_CH>0)

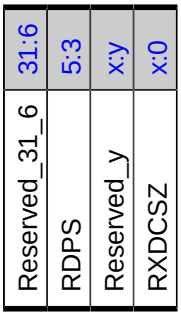


Table 11-524 Fields for Register: VDMA_CH(#i)_RxDescCtrl (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

Bits	Name	Memory Access	Description
31:6	Reserved_31_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
5:3	RDPS	R/W	Rx Descriptor Pre-fetch threshold Size. This field sets the, descriptor pre-fetching threshold for Rx channels. The DMA engine, initiates the request for descriptor pre fetched for a Rx DMA channel, only after the number of descriptors in the cache memory for that particular Rx DMA channel, falls below or equal to the number of descriptors provided by the below encoded value of this field. ■ 3'b000: 0 ■ 3'b001: For enhanced descriptor 1/8* (Receive Channel Descriptor Cache Size as per RXDCSZ/32) Otherwise 1/8*(Receive Channel Descriptor Cache Size as per RXDCSZ/16) ■ 3'b010: For enhanced descriptor 1/4* (Receive Channel Descriptor Cache Size as per RXDCSZ/32) Otherwise 1/4*(Receive Channel Descriptor Cache Size as per RXDCSZ/16) ■ 3'b011: For enhanced descriptor 1/2* (Receive Channel Descriptor Cache Size as per RXDCSZ/32) Otherwise 1/2*(Receive Channel Descriptor Cache Size as per RXDCSZ/16) ■ 3'b100: For enhanced descriptor 3/4* (Receive Channel Descriptor Cache Size as per RXDCSZ/32) Otherwise

Table 11-524 Fields for Register: VDMA_CH(#i)_RxDescCtrl (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1) (continued)

Bits	Name	Memory Access	Description
			3/4*(Receive Channel Descriptor Cache Size as per RXDCSZ/16) ■ 3'b101 - 3'b111: Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
x:y	Reserved_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** ((16*DWCXG_MAX_WRBREQ*2)>256?0:1) + ((16*DWCXG_MAX_WRBREQ*2)>256?3:2) - 1 **Range Variable[y]:** ((16*DWCXG_MAX_WRBREQ*2)>256?3:2)
x:0	RXDCSZ	R/W	This field indicates the programmable size of Descriptor Cache per Rx Channel. The maximum value is, 16 when DWCXG_MAX_WRBREQ<=8 otherwise 16* DWCXG_MAX_WRBREQ*2. This field must be programmed to a required value before starting the VDMA. The encoding is given as follows. **Note:** Bit 2 is reserved and RO when "Maximum Outstanding Write Burst Requests for each Rx DMA Channel" is less than or equal to 8, that is RXDCSZ value of greater than 2'b11 is not valid. - 000- No Space allocated - 001: 64 bytes - 010: 128 bytes - 011: 256 bytes - 100: 512 bytes - 101: 1024 bytes - 110: 2048 bytes - 111: Invalid **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** ((16*DWCXG_MAX_WRBREQ*2)>256?3:2) - 1

11.6.30 VDMA_CH(#i)_RxFrameBuf_BaseAddr_Hi (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

- **Description:** This register stores the MS bits of base address of the FrameBuffer[n], whose lower address bits are given in VDMA_CH(#i)_RxFrameBuf[n]_BaseAddr_Lo registers. The upper bits of Base Address is common to all the 4 Frame buffers and hence provided in this common register.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_DMA_RX_CH>0)&&SNPS_RSVDPARAM_48

Table 11-525 Fields for Register: VDMA_CH(#i)_RxFrameBuf_BaseAddr_Hi (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** (64-DWCXG_ADDRWIDTH) + (DWCXG_ADDRWIDTH-32) - 1 **Range Variable[y]:** DWCXG_ADDRWIDTH - 32
x:0	FB_BAH	R/W	Frame Buffer Base Address High This field provides the upper bits of Base Address [x:32] to all the Frame Buffer[n]. The LS Address bits[31:0] of the corresponding FrameBuffer[n] address is given by FB_BA[n]L field. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** (DWCXG_ADDRWIDTH-32) - 1

11.6.31 VDMA_CH(#i)_RxFrameBuf0_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

- **Description:** This register stores the LS 32bits of base address of the FrameBuffer0, whose upper address bits are given in VDMA_CH(#i)_RxFrameBuf_BaseAddr_Hi register. The base address along with the Offset provided in the Packet header forms the actual start address to which special packet payload is written in application memory.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_DMA_RX_CH>0)&&SNPS_RSVDPARAM_48

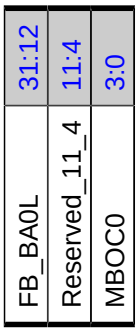


Table 11-526 Fields for Register: VDMA_CH(#i)_RxFrameBuf0_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

Bits	Name	Memory Access	Description
31:12	FB_BA0L	R/W	Frame Buffer Base Address0 Low This field provides the Base Address [31:12] to the Frame Buffer[n]; The LS 12 bits of Base Address is all-zero as this buffer size is aligned to 4KB. The Byte_Offset extracted from the Packet header is added- to the Base Address to determine the full Address where the payload is to be written/stored. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
11:4	Reserved_11_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
3:0	MBOC0	R/W	Mask Byte Offset Control This field controls the number of MS (most Significant) bits of ByteOffset field to be masked while calculating the Frame Buffer address. - 0: None of ByteOffset bits are masked - 1: ByteOffset[27] is masked - 2: ByteOffset27:26] are masked ... - 15: ByteOffset[27:12] are masked. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)

11.6.32 VDMA_CH(#i)_RxFrameBuf1_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

- **Description:** This register stores the LS 32bits of base address of the FrameBuffer1, whose upper address bits are given in VDMA_CH(#i)_RxFrameBuf_BaseAddr_Hi register. The base address along with the Offset provided in the Packet header forms the actual start address to which special packet payload is written in application memory.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_DMA_RX_CH>0)&&SNPS_RSVDPARAM_48

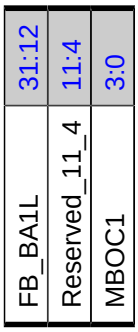


Table 11-527 Fields for Register: VDMA_CH(#i)_RxFrameBuf1_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

Bits	Name	Memory Access	Description
31:12	FB_BA1L	R/W	Frame Buffer Base Address1 Low This field provides the Base Address [31:12] to the Frame Buffer[n]; The LS 12 bits of Base Address is all-zero as this buffer size is aligned to 4KB. The Byte_Offset extracted from the Packet header is added- to the Base Address to determine the full Address where the payload is to be written/stored. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
11:4	Reserved_11_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
3:0	MBOC1	R/W	Mask Byte Offset Control This field controls the number of MS (most Significant) bits of ByteOffset field to be masked while calculating the Frame Buffer address. - 0: None of ByteOffset bits are masked - 1: ByteOffset[27] is masked - 2: ByteOffset27:26] are masked ... - 15: ByteOffset[27:12] are masked. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)

11.6.33 VDMA_CH(#i)_RxFrameBuf2_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

- **Description:** This register stores the LS 32bits of base address of the FrameBuffer2, whose upper address bits are given in VDMA_CH(#i)_RxFrameBuf_BaseAddr_Hi register. The base address along with the Offset provided in the Packet header forms the actual start address to which special packet payload is written in application memory.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_DMA_RX_CH>0)&&SNPS_RSVDPARAM_48

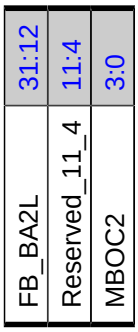


Table 11-528 Fields for Register: VDMA_CH(#i)_RxFrameBuf2_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

Bits	Name	Memory Access	Description
31:12	FB_BA2L	R/W	Frame Buffer Base Address2 Low This field provides the Base Address [31:12] to the Frame Buffer[n]; The LS 12 bits of Base Address is all-zero as this buffer size is aligned to 4KB. The Byte_Offset extracted from the Packet header is added- to the Base Address to determine the full Address where the payload is to be written/stored. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
11:4	Reserved_11_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
3:0	MBOC2	R/W	Mask Byte Offset Control This field controls the number of MS (most Significant) bits of ByteOffset field to be masked while calculating the Frame Buffer address. - 0: None of ByteOffset bits are masked - 1: ByteOffset[27] is masked - 2: ByteOffset27:26] are masked ... - 15: ByteOffset[27:12] are masked. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)

11.6.34 VDMA_CH(#i)_RxFrameBuf3_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

- **Description:** This register stores the LS 32bits of base address of the FrameBuffer3, whose upper address bits are given in VDMA_CH(#i)_RxFrameBuf_BaseAddr_Hi register. The base address along with the Offset provided in the Packet header forms the actual start address to which special packet payload is written in application memory.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_DMA_RX_CH>0)&&SNPS_RSVDPARAM_48

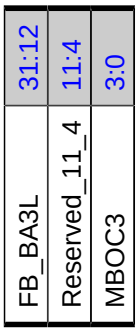


Table 11-529 Fields for Register: VDMA_CH(#i)_RxFrameBuf3_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_DMA_RX_CH-1)

Bits	Name	Memory Access	Description
31:12	FB_BA3L	R/W	Frame Buffer Base Address3 Low This field provides the Base Address [31:12] to the Frame Buffer[n]; The LS 12 bits of Base Address is all-zero as this buffer size is aligned to 4KB. The Byte_Offset extracted from the Packet header is added- to the Base Address to determine the full Address where the payload is to be written/stored. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
11:4	Reserved_11_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
3:0	MBOC3	R/W	Mask Byte Offset Control This field controls the number of MS (most Significant) bits of ByteOffset field to be masked while calculating the Frame Buffer address. - 0: None of ByteOffset bits are masked - 1: ByteOffset[27] is masked - 2: ByteOffset27:26] are masked ... - 15: ByteOffset[27:12] are masked. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)

11.6.35 PDMA_CH(#i)_RxPktInfoBuf_BaseAddr_Hi (for i = 0; i <= DWCXG_NUM_RXQ-1)

- **Description:** This register contains the MS/Upper bits of the Start address of the PacketInfo Buffer associated with the PDMA. This is concatenated with address value given in DMA_TC(n)_Rx_PktInfoBuff_Addr_Lo to form the full address.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_RXQ> 0)&&SNPS_RSVDPARAM_48

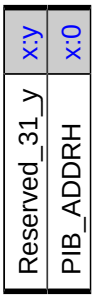


Table 11-530 Fields for Register: PDMA_CH(#i)_RxPktInfoBuf_BaseAddr_Hi (for i = 0; i <= DWCXG_NUM_RXQ-1)

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** (32-DWCXG_ADDRWIDTH_MNS32) + DWCXG_ADDRWIDTH_MNS32 - 1 **Range Variable[y]:** DWCXG_ADDRWIDTH_MNS32
x:0	PIB_ADDRH	R/W	PacketInfo Buffer Start Address High This field provides the MS bits of [x:32] of the Packet_Info Buffer Pointer/Address. LS bits[31:0] of the Buffer address is given by PIB_AADDRL. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50) **Range Variable[x]:** DWCXG_ADDRWIDTH_MNS32 - 1

11.6.36 PDMA_CH(#i)_RxPktInfoBuf_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_RXQ-1)

- **Description:** This register contains the Start address of the PacketInfo Buffer associated with the PDMA. All the VDMAs associated/mapped to this PDMA will write the PacketInfo to this buffer and increment the pointer. This pointer wraps around when the last location of this buffer (as indicated by PIB_SZ field) is written.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_RXQ> 0)&&SNPS_RSVDPARAM_48

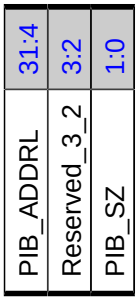


Table 11-531 Fields for Register: PDMA_CH(#i)_RxPktInfoBuf_BaseAddr_Lo (for i = 0; i <= DWCXG_NUM_RXQ-1)

Bits	Name	Memory Access	Description
31:4	PIB_ADDRL	R/W	PacketInfo Buffer Start Address Low This field provides the Base Address [31:4] of the Packet_Info Buffer Pointer/Address. LS bits[3:0] of the Buffer address = 0x0 as the buffer is aligned to the datawidth of 128b. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)
1:0	PIB_SZ	R/W	PacketInfo Buffer Size This field provides the size of the PacketInfo Buffer depth/size in terms of 128-bit locations. - 00: 256 locations - 01: 512 locations - 01: 2048 locations - 11: 4096 locations **Value After Reset:** 0x0 **Exists:** (SNPS_RSVDPARAM_50)

11.6.37 PDMA_CH(#i)_RxPktInfoBuf_CurrOffset (for i = 0; i <= DWCXG_NUM_RXQ-1)

- **Description:** This register indicates the offset from the start of the Packet Info Buffer to which the last Packet Information capsule was written.
- **Size:** 32 bits
- **Offset:** 0x3084
- **Exists:** SNPS_RSVDPARAM_50&&(DWCXG_NUM_RXQ> 0)&&SNPS_RSVDPARAM_48

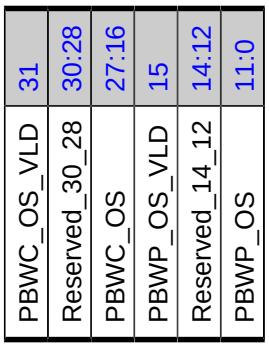


Table 11-532 Fields for Register: PDMA_CH(#i)_RxPktInfoBuf_CurrOffset (for i = 0; i <= DWCXG_NUM_RXQ-1)

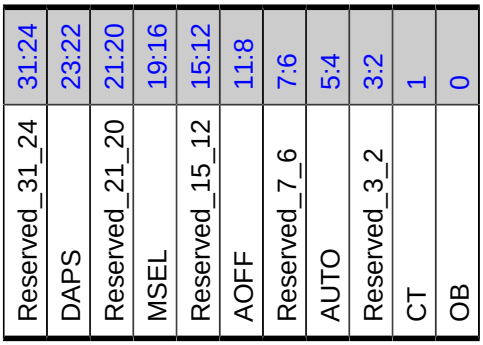
Bits	Name	Memory Access	Description
31	PBWC_OS_VLD	R/W	PacketInfo Buffer Write Completed Offset valid. **Value After Reset:** 0x0 **Exists:** Always
30:28	Reserved_30_28	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
27:16	PBWC_OS	R/W	PacketInfo Buffer Write Completed Offset This field indicates the ring offset to the location in Rx PktInfoBuffer to which a write-back request is completed. **Value After Reset:** 0x0 **Exists:** Always
15	PBWP_OS_VLD	R/W	PacketInfo Buffer Write Posted Offset valid. **Value After Reset:** 0x0 **Exists:** Always
14:12	Reserved_14_12	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-532 Fields for Register: PDMA_CH(#i)_RxPktInfoBuf_CurrOffset (for i = 0; i <= DWCXG_NUM_RXQ-1) (continued)

Bits	Name	Memory Access	Description
11:0	PBWP_OS	R/W	PacketInfo Buffer Write Posted Offset This field indicates the ring offset to the location in Rx PktInfoBuffer to which a write-back transaction was posted. **Value After Reset:** 0x0 **Exists:** Always

11.6.38 DMA_Sfty_Ind_Ctrl

- **Description:** This a control register to access DMA Safety related indirect registers.
- **Size:** 32 bits
- **Offset:** 0x3088
- **Exists:** (DWCXG_ASP_PPE==2)



31:24	23:22	21:20	19:16	15:12	11:8	7:6	5:4	3:2	1	0
Reserved_31_24	DAPS	Reserved_21_20	MSEL	Reserved_15_12	AOFF	Reserved_7_6	AUTO	Reserved_3_2	CT	OB

Table 11-533 Fields for Register: DMA_Sfty_Ind_Ctrl

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:22	DAPS	R/W	Data or Address Part Select This field indicates which 32-bit data or address word is written to AXI Manager Read Data channel data, AXI Subordinate Write Data channel data, AXI Subordinate Read Address channel address or AXI Subordinate Write Address channel address, in the debug mode. ■ 00: data[31:0] ■ 01: data[63:32] ■ 10: data[95:64] ■ 11: data[127:96] **Value After Reset:** 0x0 **Exists:** Always
21:20	Reserved_21_20	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-533 Fields for Register: DMA_Sfty_Ind_Ctrl (continued)

Bits	Name	Memory Access	Description
19:16	MSEL	R/W	Mode Select This field indicates which set of DMA Safety indirect registers is accessed. ■ 0000: Common Safety Registers ■ 0001: AXI Manager Interface Safety Registers ■ 0010: AXI Manager Interface Safety Debug Error Injection Registers ■ 0011: AXI Subordinate Interface Safety Registers ■ 0100: AXI Subordinate Interface Safety Debug Error Injection Registers ■ 0101-1111: reserved for future use **Value After Reset:** 0x0 **Exists:** Always
15:12	Reserved_15_12	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-533 Fields for Register: DMA_Sfty_Ind_Ctrl (continued)

Bits	Name	Memory Access	Description
11:8	AOFF	R/W	Address Offset This field indicates offset address of DMA Safety indirect registers within the set of registers. If MSEL is 0000: ■ 0000: DMA_Sfty_Dbg_EIC ■ 0101 to 1111 - reserved If MSEL is 0001: ■ 0000: DMA_AM_Sfty_Ctrl ■ 0001: DMA_AM_Sfty_Intr_En ■ 0010: DMA_AM_Sfty_ECC_Intr_Sts ■ 0011: DMA_AM_Sfty_DPP_Intr_Sts ■ 0100: DMA_AM_Sfty_RDC_ECC_Err_Cntr_Sts ■ 0101 to 1111 - reserved If MSEL is 0010: ■ 0000: DMA_AM_Sfty_RDC_Dbg_Data ■ 0001: DMA_AM_Sfty_RDC_Dbg_Ctrl ■ 0010:DMA_AM_Sfty_WRC_Dbg_Ctrl If MSEL is 0011: ■ 0000: DMA_AS_Sfty_Ctrl ■ 0001: DMA_AS_Sfty_Intr_En ■ 0010: DMA_AS_Sfty_ECC_Intr_Sts ■ 0011: DMA_AS_Sfty_DPP_Intr_Sts ■ 0100: DMA_AS_Sfty_WDC_ECC_Err_Cntr_Sts If MSEL is 0100: ■ 0000: DMA_AS_Sfty_WDC_Dbg_Data ■ 0001: DMA_AS_Sfty_WDC_Dbg_Ctrl ■ 0010: DMA_AS_Sfty_RAC_Dbg_Addr ■ 0011: DMA_AS_Sfty_RAC_Dbg_Ctrl ■ 0100: DMA_AS_Sfty_WAC_Dbg_Addr ■ 0101: DMA_AS_Sfty_WAC_Dbg_Ctrl ■ 0110-1111: Reserved **Value After Reset:** 0x0 **Exists:** Always
7:6	Reserved_7_6	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-533 Fields for Register: DMA_Sfty_Ind_Ctrl (continued)

Bits	Name	Memory Access	Description
5:4	AUTO	R/W	Auto Increment This field indicates that the next access is for incremental MSEL or AOFF, the application need not program the MSEL or AOFF for subsequent accesses. ■ 00: No Auto increment ■ 01: Auto increment MSEL only ■ 10: Auto increment AOFF only ■ 11: No Auto increment **Value After Reset:** 0x0 **Exists:** Always
3:2	Reserved_3_2	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
1	CT	R/W	Command Type ■ 0: Indicates a read operation. ■ 1: Indicates a write operation. **Value After Reset:** 0x0 **Exists:** Always
0	OB	R/W	Operation Busy ■ 1: Indicates the initiation of access to DMA Safety indirect register space. The DWC_25gmac clears this bit when the access to DMA Safety indirect register is complete. The requested data from indirect register is available in DMA_Sfty_Ind_Data register for read operation. The application must not change content of this register while this bit is set. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.6.39 DMA_Sfty_Ind_Data

- **Description:** This field contains data to read/write from DMA Safety indirect registers.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

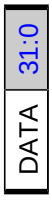
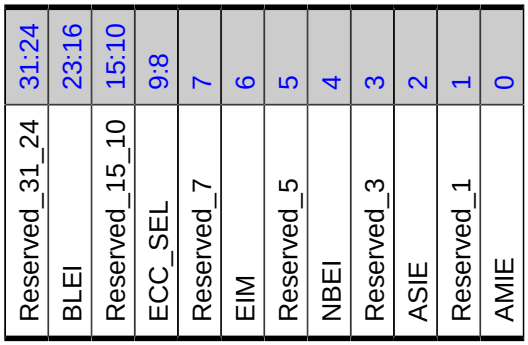


Table 11-534 Fields for Register: DMA_Sfty_Ind_Data

Bits	Name	Memory Access	Description
31:0	DATA	R/W	Read or Write Data This field contains data to read/write from DMA Safety indirect register. ■ Read access: This field contains valid read data after the OB bit in the DMA_Sfty_Ind_Data register is reset. ■ Write access: This field must be valid prior to setting the OB bit in the DMA_Sfty_Ind_Data register. When Auto Increment bit is set, the application can write new data without setting OB bit of DMA_Sfty_Ind_Data register. **Value After Reset:** 0x0 **Exists:** Always

11.6.40 DMA_Sfty_Dbg_EIC

- **Description:** This register contains control signals for Debug Error Injection.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)



31:24	23:16	15:10	9:8	7	6	5	4	3	2	1	0
Reserved_31_24	BLEI	Reserved_15_10	ECC_SEL	Reserved_7	EIM	Reserved_5	NBEI	Reserved_3	ASIE	Reserved_1	AMIE

Table 11-535 Fields for Register: DMA_Sfty_Dbg_EIC

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:16	BLEI	R/W	Bit Location of error injection This field indicates the bit location of DPP/ECC error injection. The EIM bit of DMA_Sfty_Dbg_EIC register indicates whether the error is injected in Data or Parity/ECC Check-bits. The channel/interface selected for debug error injection determines the number of valid bits in this field, for example, for debug error injection on a 64 bit data bus this field can be programmed to a value between 0 and 63, so only bits[21:16] are valid. When 2-bit error injection is enabled, the error is always injected in bit 0 and second error bit is indicated by this field. If the field is also programed to 0 when 2-bit error injection is enabled, DWC_25gmac inserts single bit error at bit 0. **Value After Reset:** 0x0 **Exists:** Always
15:10	Reserved_15_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-535 Fields for Register: DMA_Sfty_Dbg_EIC (continued)

Bits	Name	Memory Access	Description
9:8	ECC_SEL	R/W	ECC Select This field selects the ECC to insert error. Depending on the configuration (input data width, ECC granularity) there can be maximum of 4 granules available.**Values:** ■ 2'b00: Insert ECC Error in 0th granule ■ 2'b01: Insert ECC Error in 1st granule ■ 2'b10: Insert ECC Error in 2nd granule ■ 2'b11: Insert ECC Error in 3rd granule **Value After Reset:** 0x0 **Exists:** Always
7	Reserved_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
6	EIM	R/W	Debug Error Injection Mode ■ 0: Indicates debug error injected on data ■ 1: Indicates debug error injected on ECC/Parity Check-bits **Value After Reset:** 0x0 **Exists:** Always
5	Reserved_5	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
4	NBEI	R/W	Number of Bit Error Injection in ECC for AXI Interface When ASIE or AMIE bit is set, following errors are inserted based on the value encoded in this field: ■ 0: Inserts 1-bit error ■ 1: Inserts 2-bit error **Value After Reset:** 0x0 **Exists:** Always
3	Reserved_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-535 Fields for Register: DMA_Sfty_Dbg_EIC (continued)

Bits	Name	Memory Access	Description
2	ASIE	R/W	AXI Subordinate Insert Error ■ 1: Parity/ECC/data bit at the input of selected AXI Subordinate channel is flipped. The error type and location is based on the EIM/BLEI field of this register. DWC_25gmac clears this bit after the respective bit is flipped. **Value After Reset:** 0x0 **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE
1	Reserved_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	AMIE	R/W	AXI Manager Insert Error When this bit is set to 1, parity/ECC/data bit at input of selected AXI Manager channel is flipped. The error type and location is based on the EIM/BLEI field of this register. DWC_25gmac clears this bit after the respective bit is flipped. **Value After Reset:** 0x0 **Exists:** Always

11.6.41 DMA_AM_Sfty_Ctrl

- **Description:** This register contains control signal for AXI Manager interface safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

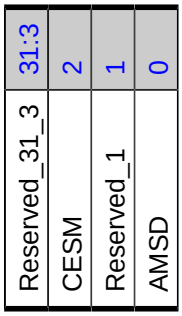


Table 11-536 Fields for Register: DMA_AM_Sfty_Ctrl

Bits	Name	Memory Access	Description
31:3	Reserved_31_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	CESM	R/W	Clear Error Status for AXI Manager Interface Safety ■ 1: AXI Manager Interface Safety ECC uncorrectable and correctable error counters are cleared upon reading by application. The DWC_25gmac clears this bit when all the error counters are cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	Reserved_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	AMSD	R/W	AXI Manager Safety Disable ■ 1: Disables the safety features for all the channels of AXI Manager Interface. ■ 0: Enables the safety features for all the channels of AXI Manager Interface. **Value After Reset:** 0x0 **Exists:** Always

11.6.42 DMA_AM_Sfty_Intr_En

- **Description:** This register contains the interrupt enables for AXI Manager Interface Safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

Table 11-537 Fields for Register: DMA_AM_Sfty_Intr_En

Bits	Name	Memory Access	Description
31:3	Reserved_31_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	RDCEIE	R/W	AXI Manager Read Data channel Data Correctable Error Interrupt Enable ■ 1: DWC_25gmac generates an interrupt when a correctable error is detected in the AXI Manager Read Data channel data and RDCES bit of DMA_AM_Sfty_ECC_Intr_Sts register is set. ■ 0: DWC_25gmac does not generate an interrupt when a correctable error is detected in the AXI Manager Read Data channel data and RDCES bit of DMA_AM_Sfty_ECC_Intr_Sts register is set. **Value After Reset:** 0x0 **Exists:** Always
1:0	Reserved_1_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.6.43 DMA_AM_Sfty_ECC_Intr_Sts

- **Description:** This register contains the ECC interrupt status for the AXI Manager Interface Safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

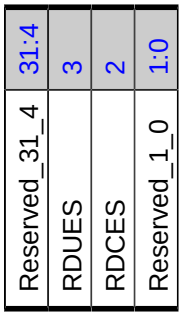


Table 11-538 Fields for Register: DMA_AM_Sfty_ECC_Intr_Sts

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	RDUES	R/W	AXI Manager Read Data channel data Uncorrectable Error Status ■ 1: Indicates that an uncorrectable error is detected at AXI Manager Read Data channel data. **Value After Reset:** 0x0 **Exists:** Always
2	RDCES	R/W	AXI Manager Read Data channel data Correctable Error Status ■ 1: Indicates that a correctable error is detected at AXI Manager Read Data channel data. **Value After Reset:** 0x0 **Exists:** Always
1:0	Reserved_1_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.6.44 DMA_AM_Sfty_DPP_Intr_Sts

- **Description:** This register contains the DPP interrupt status for AXI Manager Interface safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

31:8	7	6	5	4	3	2	1	0
Reserved_31_8	ARRPES	AWRPES	WRPES	Reserved_4	RCPES	RVPES	BCPES	BVPES

Table 11-539 Fields for Register: DMA_AM_Sfty_DPP_Intr_Sts

Bits	Name	Memory Access	Description
31:8	Reserved_31_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7	ARRPES	R/W	AXI Manager Read Address channel Ready Parity Error Status ■ 1: Indicates that a parity error is detected on the AXI Manager Read Address channel Ready signal. **Value After Reset:** 0x0 **Exists:** Always
6	AWRPES	R/W	AXI Manager Write Address channel Ready Parity Error Status ■ 1: Indicates that a parity error is detected on the AXI Manager AW channel Ready signal. **Value After Reset:** 0x0 **Exists:** Always
5	WRPES	R/W	AXI Manager Write Data channel Ready Parity Error Status ■ 1: Indicates that a parity error is detected on AXI Manager W channel Ready signal. **Value After Reset:** 0x0 **Exists:** Always
4	Reserved_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-539 Fields for Register: DMA_AM_Sfty_DPP_Intr_Sts (continued)

Bits	Name	Memory Access	Description
3	RCPES	R/W	AXI Manager Read Data channel Control Parity Error Status ■ 1: Indicates that a parity error is detected on the AXI Manager Read Data channel control signals. **Value After Reset:** 0x0 **Exists:** Always
2	RVPES	R/W	AXI Manager Read Data channel Valid Parity Error Status ■ 1: Indicates that a parity error is detected on the AXI Manager Read Data channel valid signal. **Value After Reset:** 0x0 **Exists:** Always
1	BCPES	R/W	AXI Manager Write Response channel Control Parity Error Status ■ 1: Indicates that a parity error is detected at the AXI Manager Write Response channel control signals. **Value After Reset:** 0x0 **Exists:** Always
0	BVPES	R/W	AXI Manager Write Response Channel Valid Parity Error Status ■ 1: Indicates that a parity error is detected at the AXI Manager Write Response channel valid signal. **Value After Reset:** 0x0 **Exists:** Always

11.6.45 DMA_AM_Sfty_RDC_ECC_Err_Cntr_Sts

- **Description:** This register contains the ECC Error counters for read data of AXI Manager Interface safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

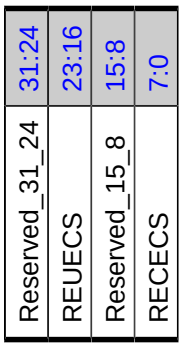


Table 11-540 Fields for Register: DMA_AM_Sfty_RDC_ECC_Err_Cntr_Sts

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:16	REUECS	R/W	AXI Manager Read Data channel ECC Uncorrectable Error Counter This field provides the number of ECC uncorrectable errors detected in AXI Manager Read Data channel. This field is cleared when the CESM bit is set to 1 and the application reads this field. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7:0	RECECS	R/W	AXI Manager Read Data channel ECC Correctable Error Counter This field provides the number of ECC correctable errors detected in AXI Manager Read Data channel. This field is cleared when the CESM bit is set to 1 and the application reads this field. **Value After Reset:** 0x0 **Exists:** Always

11.6.46 DMA_AM_Sfty_RDC_Dbg_Data

- **Description:** This register contains the data to be driven on AXI Manager read data in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

Table 11-541 Fields for Register: DMA_AM_Sfty_RDC_Dbg_Data

Bits	Name	Memory Access	Description
31:0	RD	R/W	AXI Manager Read Data channel Debug Data This field contains the 32-bit debug data to be driven on AXI Manager Read Data bus. The DAPS field of DMA_Sfty_Ind_Ctl register indicates which 32-bit word of read data bus this correspond to. **Value After Reset:** 0x0 **Exists:** Always

11.6.47 DMA_AM_Sfty_RDC_Dbg_Ctrl

- **Description:** This register contains the data to be driven on AXI Manager Read Data channel control signals in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

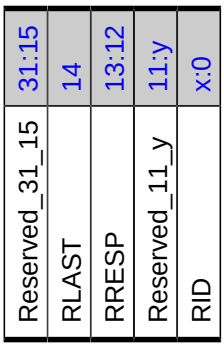


Table 11-542 Fields for Register: DMA_AM_Sfty_RDC_Dbg_Ctrl

Bits	Name	Memory Access	Description
31:15	Reserved_31_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
14	RLAST	R/W	AXI Manager Read Data channel RLAST Control Signal This bit contains the debug data to be driven on AXI Manager RLAST signal. **Value After Reset:** 0x0 **Exists:** Always
13:12	RRESP	R/W	AXI Manager Read Data channel RRESP Control Signal This field contains the debug data to be driven on AXI Manager RRESP signal. **Value After Reset:** 0x0 **Exists:** Always
11:y	Reserved_11_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_AXI_GM_AXI_ID_WIDTH)!=12) **Range Variable[y]:** DWCXG_AXI_IDW

Table 11-542 Fields for Register: DMA_AM_Sfty_RDC_Dbg_Ctrl (continued)

Bits	Name	Memory Access	Description
x:0	RID	R/W	AXI Manager Read Data channel RID Control Signal This field contains the debug data to be driven on AXI Manager RID signal. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_AXI_IDW - 1

11.6.48 DMA_AM_Sfty_WRC_Dbg_Ctrl

- **Description:** This register contains the data to be driven on AXI Manager write response channel control signals in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** (DWCXG_ASP_PPE==2)

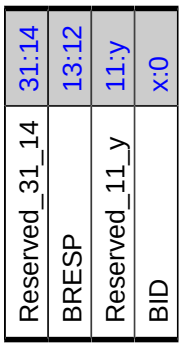


Table 11-543 Fields for Register: DMA_AM_Sfty_WRC_Dbg_Ctrl

Bits	Name	Memory Access	Description
31:14	Reserved_31_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:12	BRESP	R/W	AXI Manager Response Channel BRESP Control Signal This field contains the debug data to be driven on AXI Manager BRESP signal. **Value After Reset:** 0x0 **Exists:** Always
11:y	Reserved_11_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_AXI_GM_AXI_ID_WIDTH)!=12) **Range Variable[y]:** DWCXG_AXI_IDW
x:0	BID	R/W	AXI Manager Response Channel BID Control Signal This field contains the debug data to be driven on AXI Manager BID signal. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_AXI_IDW - 1

11.6.49 DMA_AS_Sfty_Ctrl

- **Description:** This register contains control signal for AXI Subordinate interface safety module.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

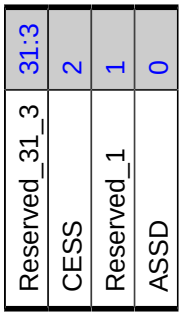


Table 11-544 Fields for Register: DMA_AS_Sfty_Ctrl

Bits	Name	Memory Access	Description
31:3	Reserved_31_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	CESS	R/W	Clear Error Status for AXI Subordinate Interface Safety ■ 1: AXI Subordinate Interface Safety ECC uncorrectable and correctable error counters are cleared upon reading by application. The DWC_25gmac clears this bit when all the error counters are cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	Reserved_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	ASSD	R/W	AXI Subordinate Safety Disable ■ 1: Disables the safety features for all the channels of AXI Subordinate Interface. ■ 0: Enables the safety features for all the channels of AXI Subordinate Interface. **Value After Reset:** 0x0 **Exists:** Always

11.6.50 DMA_AS_Sfty_Intr_En

- **Description:** This register contains the interrupt enables for AXI Subordinate Interface Safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

Table 11-545 Fields for Register: DMA_AS_Sfty_Intr_En

Bits	Name	Memory Access	Description
31:3	Reserved_31_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	WDCEIE	R/W	AXI Subordinate Write Data channel Data Correctable Error Interrupt Enable ■ 1: DWC_25gmac generates an interrupt when a correctable error is detected in the AXI Subordinate Write Data channel data and WDCES bit of DMA_AS_Sfty_ECC_Intr_Sts register is set. ■ 0: DWC_25gmac does not generate an interrupt when a correctable error is detected in the AXI Subordinate Write Data channel data and WDCES bit of DMA_AS_Sfty_ECC_Intr_Sts register is set. **Value After Reset:** 0x0 **Exists:** Always
1:0	Reserved_1_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.6.51 DMA_AS_Sfty_ECC_Intr_Sts

- **Description:** This register contains the ECC interrupt status for the AXI Subordinate Interface Safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

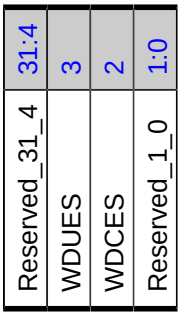


Table 11-546 Fields for Register: DMA_AS_Sfty_ECC_Intr_Sts

Bits	Name	Memory Access	Description
31:4	Reserved_31_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	WDUES	R/W	AXI Subordinate Write Data channel Data Uncorrectable Error Status ■ 1: Indicates that an uncorrectable error is detected at AXI Subordinate Write Data channel data. **Value After Reset:** 0x0 **Exists:** Always
2	WDCES	R/W	AXI Subordinate Write Data channel Data Correctable Error Status When this bit is set to 1, it indicates that a correctable error is detected at AXI Subordinate Write Data channel data. **Value After Reset:** 0x0 **Exists:** Always
1:0	Reserved_1_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

11.6.52 DMA_AS_Sfty_DPP_Intr_Sts

- **Description:** This register contains the DPP interrupt status for AXI Subordinate Interface safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

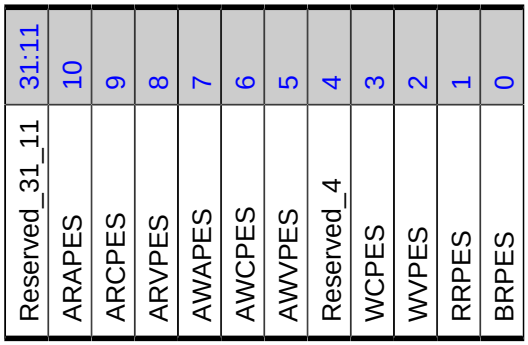


Table 11-547 Fields for Register: DMA_AS_Sfty_DPP_Intr_Sts

Bits	Name	Memory Access	Description
31:11	Reserved_31_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
10	ARAPES	R/W	AXI Subordinate Read Address channel Address Parity Error Status When this bit is set to 1, it indicates that a parity error is detected on the AXI Subordinate Read Address channel address signal. **Value After Reset:** 0x0 **Exists:** Always
9	ARCPES	R/W	AXI Subordinate Read Address channel Control Parity Error Status When this bit is set to 1, it indicates that a parity error is detected on the AXI Subordinate Read Address channel control signal. **Value After Reset:** 0x0 **Exists:** Always
8	ARVPES	R/W	AXI Subordinate Read Address channel Valid Parity Error Status When this bit is set to 1, it indicates that a parity error is detected on the AXI Subordinate Read Address channel valid signal. **Value After Reset:** 0x0 **Exists:** Always

Table 11-547 Fields for Register: DMA_AS_Sfty_DPP_Intr_Sts (continued)

Bits	Name	Memory Access	Description
7	AWAPES	R/W	AXI Subordinate Write Address channel Address Parity Error Status When this bit is set to 1, it indicates that a parity error is detected on the AXI Subordinate Write Address channel address signal. **Value After Reset:** 0x0 **Exists:** Always
6	AWCPES	R/W	AXI Subordinate Write Address channel Control Parity Error Status When this bit is set to 1, it indicates that a parity error is detected on the AXI Subordinate Write Address channel control signal. **Value After Reset:** 0x0 **Exists:** Always
5	AWVPES	R/W	AXI Subordinate Write Address channel Valid Parity Error Status When this bit is set to 1, it indicates that a parity error is detected on the AXI Subordinate Write Address channel valid signal. **Value After Reset:** 0x0 **Exists:** Always
4	Reserved_4	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
3	WCPES	R/W	AXI Subordinate Write Data channel Control Parity Error Status ■ 1: Indicates that a parity error is detected on the AXI Subordinate Write Data channel control signal. **Value After Reset:** 0x0 **Exists:** Always
2	WVPES	R/W	AXI Subordinate Write Data channel Valid Parity Error Status ■ 1: Indicates that a parity error is detected on the AXI Subordinate Write Data channel valid signal. **Value After Reset:** 0x0 **Exists:** Always

Table 11-547 Fields for Register: DMA_AS_Sfty_DPP_Intr_Sts (continued)

Bits	Name	Memory Access	Description
1	RRPES	R/W	AXI Subordinate Read Data channel Ready Parity Error Status ■ 1: Indicates that a parity error is detected on the AXI Subordinate Read Data channel ready signal. **Value After Reset:** 0x0 **Exists:** Always
0	BRPES	R/W	AXI Subordinate Write Response channel Ready Parity Error Status ■ 1: Indicates that a parity error is detected on the AXI Subordinate Write Response channel ready signal. **Value After Reset:** 0x0 **Exists:** Always

11.6.53 DMA_AS_Sfty_WDC_ECC_Err_Cntr_Sts

- **Description:** This register contains the ECC Error counters for write data of AXI Subordinate Interface safety.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

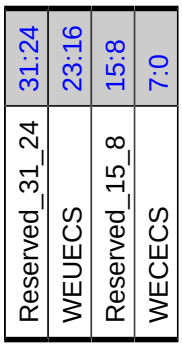


Table 11-548 Fields for Register: DMA_AS_Sfty_WDC_ECC_Err_Cntr_Sts

Bits	Name	Memory Access	Description
31:24	Reserved_31_24	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
23:16	WEUECS	R/W	AXI Subordinate Write Data channel ECC Uncorrectable Error Counter This field provides the number of ECC uncorrectable errors detected in AXI Subordinate Write Data channel. This field is cleared when the CESS bit is set to 1 and the application reads this field. **Value After Reset:** 0x0 **Exists:** Always
15:8	Reserved_15_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7:0	WECECS	R/W	AXI Subordinate Write Data channel ECC Correctable Error Counter This field provides the number of ECC correctable errors detected in AXI Subordinate Write Data channel. This field is cleared when the CESS bit is set to 1 and the application reads this field. **Value After Reset:** 0x0 **Exists:** Always

11.6.54 DMA_AS_Sfty_WDC_Dbg_Data

- **Description:** This register contains the data to be driven on AXI Subordinate write data in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

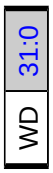


Table 11-549 Fields for Register: DMA_AS_Sfty_WDC_Dbg_Data

Bits	Name	Memory Access	Description
31:0	WD	R/W	AXI Subordinate Write Data Channel Debug Data This field contains the 32-bit debug data to be driven on AXI Subordinate write Data bus. The DAPS field of DMA_Sfty_Ind_Ctl register indicates which 32-bit word of write data bus this correspond to. **Value After Reset:** 0x0 **Exists:** Always

11.6.55 DMA_AS_Sfty_WDC_Dbg_Ctrl

- **Description:** This register contains the data to be driven on AXI Subordinate Write Data channel control signals in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

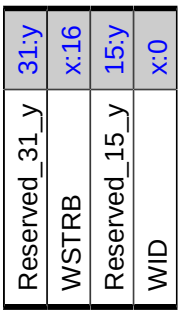


Table 11-550 Fields for Register: DMA_AS_Sfty_WDC_Dbg_Ctrl

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_CSR_DATAWIDTH)!=16) **Range Variable[y]:** DWCXG_GS_DWB + 16
x:16	WSTRB	R/W	AXI Subordinate Write Data channel WSTRB Control Signal This field contains the debug data to be driven on AXI Subordinate WSTRB signal. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_GS_DWB + 15
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_GS_ID!=16) **Range Variable[y]:** DWCXG_GS_ID
x:0	WID	R/W	AXI Subordinate Write Data channel WID Control Signal This field contains the debug data to be driven on AXI Subordinate WID signal. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_GS_ID - 1

11.6.56 DMA_AS_Sfty_RAC_Dbg_Addr

- **Description:** This register contains the data to be driven on address bus of AXI Subordinate read address channel in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

Table 11-551 Fields for Register: DMA_AS_Sfty_RAC_Dbg_Addr

Bits	Name	Memory Access	Description
31:0	ADDR	R/W	AXI Subordinate Read Address Channel Address This field contains the 32-bit debug data to be driven on AXI Subordinate Read address bus. The DAPS field of DMA_Sfty_Ind_Ctl register indicates which 32-bit word of Read address bus this correspond to. **Value After Reset:** 0x0 **Exists:** Always

11.6.57 DMA_AS_Sfty_RAC_Dbg_Ctrl

- **Description:** This register contains the data to be driven on control signals of AXI Subordinate read address channel in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

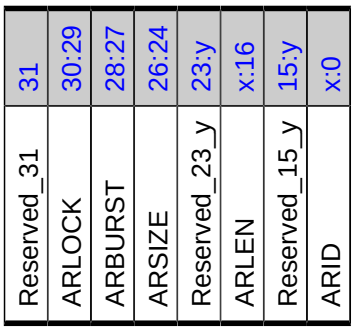


Table 11-552 Fields for Register: DMA_AS_Sfty_RAC_Dbg_Ctrl

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:29	ARLOCK	R/W	AXI Subordinate Address Read Address channel ARLOCK Control Signal This field contains the debug data to be driven on AXI Subordinate ARLOCK signal. **Value After Reset:** 0x0 **Exists:** Always
28:27	ARBURST	R/W	AXI Subordinate Address Read Address channel ARBURST Control Signal This field contains the debug data to be driven on AXI Subordinate ARBURST signal. **Value After Reset:** 0x0 **Exists:** Always
26:24	ARSIZE	R/W	AXI Subordinate Address Read Address channel ARSIZE Control Signal This field contains the debug data to be driven on AXI Subordinate ARSIZE signal. **Value After Reset:** 0x0 **Exists:** Always

Table 11-552 Fields for Register: DMA_AS_Sfty_RAC_Dbg_Ctrl (continued)

Bits	Name	Memory Access	Description
23:y	Reserved_23_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_AXI4_LITE_SLAVE ? 4 : DWCXG_BLW)!=8) **Range Variable[y]:** DWCXG_GS_BW + 16
x:16	ARLEN	R/W	AXI Subordinate Address Read Address channel ARLEN Control Signal This field contains the debug data to be driven on AXI Subordinate ARLEN signal. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_GS_BW + 15
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_GS_ID!=16) **Range Variable[y]:** DWCXG_GS_ID
x:0	ARID	R/W	AXI Subordinate Address Read Address channel ARID Control Signal This field contains the debug data to be driven on AXI Subordinate ARID signal. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_GS_ID - 1

11.6.58 DMA_AS_Sfty_WAC_Dbg_Addr

- **Description:** This register contains the data to be driven on address bus of AXI Subordinate write address channel in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

Table 11-553 Fields for Register: DMA_AS_Sfty_WAC_Dbg_Addr

Bits	Name	Memory Access	Description
31:0	ADDR	R/W	AXI Subordinate Address Write Address channel address This field contains the 32-bit debug data to be driven on AXI Subordinate Write address bus. The DAPS field of DMA_Sfty_Ind_Ctl register indicates which 32-bit word of Write address bus this correspond to. **Value After Reset:** 0x0 **Exists:** Always

11.6.59 DMA_AS_Sfty_WAC_Dbg_Ctrl

- **Description:** This register contains the data to be driven on control signals of AXI Subordinate write address channel in Debug Error Injection mode.
- **Size:** 32 bits
- **Offset:** 0x308c
- **Exists:** DWCXG_AIS_EN&&(DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)

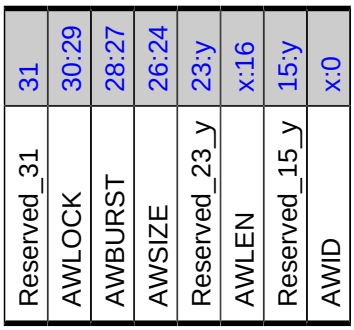


Table 11-554 Fields for Register: DMA_AS_Sfty_WAC_Dbg_Ctrl

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:29	AWLOCK	R/W	AXI Subordinate Address Write Address channel AWLOCK Control Signal This field contains the debug data to be driven on AXI Subordinate AWLOCK signal. **Value After Reset:** 0x0 **Exists:** Always
28:27	AWBURST	R/W	AXI Subordinate Address Write Address channel AWBURST Control Signal This field contains the debug data to be driven on AXI Subordinate AWBURST signal. **Value After Reset:** 0x0 **Exists:** Always
26:24	AWSIZE	R/W	AXI Subordinate Address Write Address channel AWSIZE Control Signal This field contains the debug data to be driven on AXI Subordinate AWSIZE signal. **Value After Reset:** 0x0 **Exists:** Always

Table 11-554 Fields for Register: DMA_AS_Sfty_WAC_Dbg_Ctrl (continued)

Bits	Name	Memory Access	Description
23:y	Reserved_23_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_AXI4_LITE_SLAVE ? 4 : DWCXG_BLW)!=8) **Range Variable[y]:** DWCXG_GS_BW + 16
x:16	AWLEN	R/W	AXI Subordinate Address Write Address channel AWLEN Control Signal This field contains the debug data to be driven on AXI Subordinate AWLEN signal. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_GS_BW + 15
15:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** (DWCXG_GS_ID!=16) **Range Variable[y]:** DWCXG_GS_ID
x:0	AWID	R/W	AXI Subordinate Address Write Address channel AWID Control Signal This field contains the debug data to be driven on AXI Subordinate AWID signal. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_GS_ID - 1

11.6.60 DMA_Interrupt_Status16_47

- **Description:** The application reads this Interrupt Status register during interrupt service routine or polling to determine the interrupt status of DMA channels numbered from 16 to 47
- **Size:** 32 bits
- **Offset:** 0x3090
- **Exists:** (SNPS_RSVDPARAM_50)&&((DWCXG_NUM_DMA_TX_CH>16)||(DWCXG_NUM_DMA_RX_CH>16))

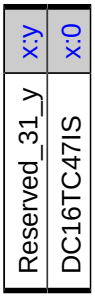


Table 11-555 Fields for Register: DMA_Interrupt_Status16_47

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_NUM_DMA_TX_CH<=47)&&(DWCXG_NUM_DMA_RX_CH<=47)) **Range Variable[x]:** ((DWCXG_NUM_DMA_CSR_CH>47)?0:DWCXG_NUM_DMA_CSR_CH>16?(32-(DWCXG_NUM_DMA_CSR_CH-16)):0) + (DWCXG_NUM_DMA_CSR_CH>47?0:DWCXG_NUM_DMA_CSR_CH>16?(DWCXG_NUM_DMA_CSR_CH-16):0) - 1 **Range Variable[y]:** (DWCXG_NUM_DMA_CSR_CH>47?0:DWCXG_NUM_DMA_CSR_CH>16?(DWCXG_NUM_DMA_CSR_CH-16):0)

Table 11-555 Fields for Register: DMA_Interrupt_Status16_47 (continued)

Bits	Name	Memory Access	Description
x:0	DC16TC47IS	R	DMA Channel 16 Interrupt Status to DMA Channel 47 Interrupt Status To reset a bit to 1'b0, the software must read the Channel Status register in the corresponding DMA Channel (15 to 8) to get the exact cause of the interrupt and clear its source **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** ((DWCXG_NUM_DMA_CSR_CH>47)?32: (DWCXG_NUM_DMA_CSR_CH>16)?(DWCXG_NUM_DMA_CSR_CH-16):0) - 1

11.6.61 DMA_Interrupt_Status48_79

- **Description:** The application reads this Interrupt Status register during interrupt service routine or polling to determine the interrupt status of DMA channels numbered from 48 to 79
- **Size:** 32 bits
- **Offset:** 0x3094
- **Exists:** (SNPS_RSVDPARAM_50)&&((DWCXG_NUM_DMA_TX_CH>48)||(DWCXG_NUM_DMA_RX_CH>48))

Table 11-556 Fields for Register: DMA_Interrupt_Status48_79

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_NUM_DMA_TX_CH<=79)&&(DWCXG_NUM_DMA_RX_CH<=79)) **Range Variable[x]:** (DWCXG_NUM_DMA_CSR_CH>79?0:DWCXG_NUM_DMA_CSR_CH>48?(32-(DWCXG_NUM_DMA_CSR_CH-48)):0) + (DWCXG_NUM_DMA_CSR_CH>79?0:DWCXG_NUM_DMA_CSR_CH>48?(DWCXG_NUM_DMA_CSR_CH-48):0) - 1 **Range Variable[y]:** (DWCXG_NUM_DMA_CSR_CH>79?0:DWCXG_NUM_DMA_CSR_CH>48?(DWCXG_NUM_DMA_CSR_CH-48):0)
x:0	DC48TC79IS	R	DMA Channel 48 Interrupt Status to DMA Channel 79 Interrupt Status To reset a bit to 1'b0, the software must read the Channel Status register in the corresponding DMA Channel (48 to 79) to get the exact cause of the interrupt and clear its source **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** (DWCXG_NUM_DMA_CSR_CH>79?32:DWCXG_NUM_DMA_CSR_CH>48?(DWCXG_NUM_DMA_CSR_CH-48):0) - 1

11.6.62 DMA_Interrupt_Status80_111

- **Description:** The application reads this Interrupt Status register during interrupt service routine or polling to determine the interrupt status of DMA channels numbered from 80 to 111
- **Size:** 32 bits
- **Offset:** 0x3098
- **Exists:** (SNPS_RSVDPARAM_50)&&((DWCXG_NUM_DMA_TX_CH>80)||(DWCXG_NUM_DMA_RX_CH>80))

Table 11-557 Fields for Register: DMA_Interrupt_Status80_111

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_NUM_DMA_TX_CH<=111)&&(DWCXG_NUM_DMA_RX_CH<=111)) **Range Variable[x]:** (DWCXG_NUM_DMA_CSR_CH>111?0:DWCXG_NUM_DMA_CSR_CH>80?(32-(DWCXG_NUM_DMA_CSR_CH-80)):0) + (DWCXG_NUM_DMA_CSR_CH>111?0:DWCXG_NUM_DMA_CSR_CH>80?(DWCXG_NUM_DMA_CSR_CH-80):0) - 1 **Range Variable[y]:** (DWCXG_NUM_DMA_CSR_CH>111?0:DWCXG_NUM_DMA_CSR_CH>80?(DWCXG_NUM_DMA_CSR_CH-80):0)

Table 11-557 Fields for Register: DMA_Interrupt_Status80_111 (continued)

Bits	Name	Memory Access	Description
x:0	DC80TC111IS	R	DMA Channel 80 Interrupt Status to DMA Channel 111 Interrupt Status To reset a bit to 1'b0, the software must read the Channel Status register in the corresponding DMA Channel (80 to 111) to get the exact cause of the interrupt and clear its source **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** (DWCXG_NUM_DMA_CSR_CH>111)?32: (DWCXG_NUM_DMA_CSR_CH>80)?(DWCXG_NUM_DMA_CSR_CH-80):0 - 1

11.6.63 DMA_Interrupt_Status112_127

- **Description:** The application reads this Interrupt Status register during interrupt service routine or polling to determine the interrupt status of DMA channels numbered from 112 to 127
- **Size:** 32 bits
- **Offset:** 0x309c
- **Exists:** (SNPS_RSVDPARAM_50)&&((DWCXG_NUM_DMA_TX_CH>112)||(DWCXG_NUM_DMA_RX_CH>112))

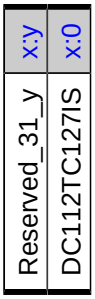


Table 11-558 Fields for Register: DMA_Interrupt_Status112_127

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((DWCXG_NUM_DMA_TX_CH<=143)&&(DWCXG_NUM_DMA_RX_CH<=143)) **Range Variable[x]:** (DWCXG_NUM_DMA_CSR_CH>143?0:DWCXG_NUM_DMA_CSR_CH>112?(32-(DWCXG_NUM_DMA_CSR_CH-112)):0) + (DWCXG_NUM_DMA_CSR_CH>143?0:DWCXG_NUM_DMA_CSR_CH>112?(DWCXG_NUM_DMA_CSR_CH-112):0) - 1 **Range Variable[y]:** (DWCXG_NUM_DMA_CSR_CH>143?0:DWCXG_NUM_DMA_CSR_CH>112?(DWCXG_NUM_DMA_CSR_CH-112):0)

Table 11-558 Fields for Register: DMA_Interrupt_Status112_127 (continued)

Bits	Name	Memory Access	Description
x:0	DC112TC127IS	R	DMA Channel 112 Interrupt Status to DMA Channel 127 Interrupt Status To reset a bit to 1'b0, the software must read the Channel Status register in the corresponding DMA Channel (112 to 127) to get the exact cause of the interrupt and clear its source **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable **Range Variable[x]:** (DWCXG_NUM_DMA_CSR_CH>143?32:DWCXG_NUM_DMA_CSR_CH>112?(DWCXG_NUM_DMA_CSR_CH-112):0) - 1

11.7 DWCXG_DMA_CH(#i) Registers

11.7.1 VDMA_CH0_Control

- **Description:** The VDMA Channel0 Control register specifies the MSS value for segmentation, length to skip between two descriptors, and also the features such as header splitting and 8xPBL mode.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3100
- **Exists:** Always

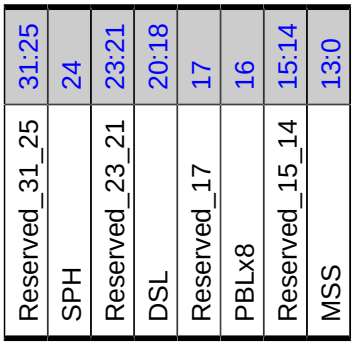


Table 11-559 Fields for Register: VDMA_CH0_Control

Bits	Name	Memory Access	Description
31:25	Reserved_31_25	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
24	SPH	R/W	Header-Payload Split. When this bit is set, the DMA splits the header and payload in the Receive path. The DMA writes the header to the Buffer Address1. The DMA writes the payload to the buffer to which the Buffer Address2 is pointing. The software must ensure that the header fits into the Receive buffers. If the header length exceeds the receive buffer size, the DMA does not split the header and payload. **Value After Reset:** 0x0 **Exists:** DWCXG_SPLIT_HDR_EN
23:21	Reserved_23_21	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-559 Fields for Register: VDMA_CH0_Control (continued)

Bits	Name	Memory Access	Description
20:18	DSL	R/W	Descriptor Skip Length. This bit specifies the number of Word , DWord or LWord (depending on the 32-bit, 64-bit, or 128-bit bus) to skip between two descriptors. The address skipping starts from the end of the current descriptor to the start of the next descriptor. When the DSL value is equal to zero, the DMA takes the descriptor list as contiguous. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}
17	Reserved_17	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
16	PBLx8	R/W	8xPBL mode. When this bit is set, the PBL value programmed in Bits[21:16] in VDMA_CH0_Tx_Control is multiplied eight times. Therefore, the DMA transfers the data in 8, 16, 32, 64, 128, and 256 beats depending on the PBL value. **Value After Reset:** 0x0 **Exists:** !SNPS_RSVDPARAM_50
15:14	Reserved_15_14	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
13:0	MSS	R/W	Maximum Segment Size. This field specifies the maximum segment size that must be used while segmenting the Transmit packet. This field is valid only if the TSE bit of VDMA_CH0_Tx_Control register is set. **Value After Reset:** 0x0 **Exists:** (DWCXG_TSO_EN&&(SNPS_RSVDPARAM_50?1:DWCXG_NUM_DMA_TSO_CH>0))

11.7.2 VDMA_CH0_Tx_Control

- **Description:** The VDMA Channel0 Transmit Control register controls the Tx features such as PBL, TCP segmentation, and Tx Channel weights.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3104
- **Exists:** (DWCXG_NUM_DMA_TX_CH>0)

31	30:29	28	27:24	23:22	21:16	15	14:13	12	11:7	6:4	3:1	0
Reserved_31	TFSEL	EDSE	TQOS	Reserved_23_22	TxPBL	IPBL	Reserved_14_13	TSE	Reserved_11_7	TD2TCMP	Reserved_3_1	ST

Table 11-560 Fields for Register: VDMA_CH0_Tx_Control

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:29	TFSEL	R/W	TBS Fetch Time Select. Selects the DMA_TBS_CTRLn register used to determine the settings of the TBS function applicable for the packets transferred by this TX DMA channel. **Values:** ■ 0x0 (DMA_TBS_CTRL0) ■ 0x1 (DMA_TBS_CTRL1) ■ 0x2 (DMA_TBS_CTRL2) ■ 0x3 (DMA_TBS_CTRL3) **Value After Reset:** 0x0 **Exists:** DWCXG_TBS&&DWCXG_NUM_DMA_TX_CH>4

Table 11-560 Fields for Register: VDMA_CH0_Tx_Control (continued)

Bits	Name	Memory Access	Description
28	EDSE	R/W	Enhanced Descriptor Enable. When this bit is set, the corresponding channel uses enhanced Descriptors with increased size of 32 Bytes for both Normal and Context Descriptors. When reset, the corresponding channel uses the descriptors with default size of 16 Bytes. **Values:** ■ 0x0 (DISABLE): Enhanced Descriptor is disabled. ■ 0x1 (ENABLE): Enhanced Descriptor is enabled. **Value After Reset:** 0x0 **Exists:** ((DWCXG_TBS&&(SNPS_RSVDPARAM_50?1:((DWCXG_NUM_DMA_TX_CH>0)&&(((DWCXG_NUM_DMA_TX_CH)-(DWCXG_NUM_DMA_TBS_CH))<(0+1)))))||DWCXG_ATS)
27:24	TQOS	R/W	Transmit QOS. This field is used to drive the values on AXI Manager read control channel's QoS indicator or AXI Manager write control channel's QoS indicator (arqos_m_o[3:0] or awqos_m_o[3:0]) output signals of the AXI bus for all transactions initiated by this Tx DMA channel **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}
23:22	Reserved_23_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
21:16	TxPBL	R/W	Transmit Programmable Burst Length. This field controls the maximum size of burst transaction that can be initiated by the TX DMA channel for packet data transfers. The TX DMA channel attempts to fetch the data of this burst size if available, from the Transmit Buffer pointed by the descriptor. You can program PBL with any of the following values: 1, 2, 4, 8, 16, or 32. Any other value results in undefined behavior. To transfer more than 32 beats, perform the following steps: ■ Set the PBLx8 mode in VDMA_CH0_Control register. ■ Set the PBL. **Note-1:** This PBL controls the maximum burst size of the requests from this internal DMA engine. The AXI Interface block can split this PBL requests from DMA engine into

Table 11-560 Fields for Register: VDMA_CH0_Tx_Control (continued)

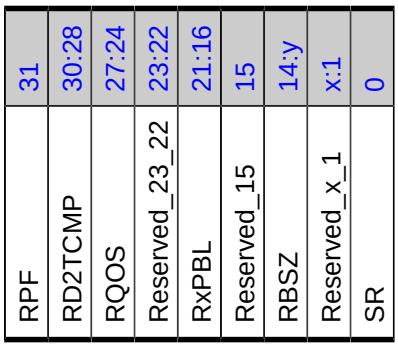
Bits	Name	Memory Access	Description
			multiple AXI requests based on the programmed values in DMA_SysBus_Mode register. **Note-2:** The maximum limit programmed must be less than or equal to half the size of the corresponding Tx Queue depth (Queue Depth = Queue Size / Datawidth) **Value After Reset:** 0x0 **Exists:** (!SNPS_RSVDPARAM_50)
15	IPBL	R/W	Ignore PBL Requirement. When this bit is set, the DMA does not check for PBL number of locations availability in the MTL Transmit Queue before initiating a transfer. If space is not available, and transfer on the AXI is initiated, then the arvalid_m_o signal on the AXI is de-asserted when MTL is not ready to accept the data. If this bit is not set, the arvalid_m_o signal on the AXI is not de-asserted. **Value After Reset:** 0x0 **Exists:** !(DWCXG == 2) || SNPS_RSVDPARAM_50
14:13	Reserved_14_13	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
12	TSE	R/W	TCP Segmentation Enabled. When this bit is set, the DMA performs the TCP segmentation for packets in Channel 0. The TCP segmentation is done only for those packets for which the TSE bit (TDES0[19]) is set in the Tx Normal descriptor. **Value After Reset:** 0x0 **Exists:** (DWCXG_TSO_EN&&(SNPS_RSVDPARAM_50?1:DWCXG_NUM_DMA_TSO_CH>0))
11:7	Reserved_11_7	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always

Table 11-560 Fields for Register: VDMA_CH0_Tx_Control (continued)

Bits	Name	Memory Access	Description
6:4	TD2TCMP	R/W	Tx DMA to TC Mapping This field is used to map Transit DMA to a traffic class (TC). The actual width of this field depends on the number of traffic classes selected during configuration. 3'b000: TC0 3'b001: TC1 3'b010: TC2 3'b011: TC3 3'b100: TC4 3'b101: TC5 3'b110: TC6 3'b111: TC7 **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50 **Testable:** {1?{untestable}:{unconstrained}}
3:1	Reserved_3_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
0	ST	R/W	Start or Stop Transmission Command. When this bit is set, transmission is placed in the Running state. The DMA checks the Transmit list at the current position for a packet to be transmitted. The DMA tries to acquire descriptor from either of the following positions: ■ The current position in the list This is the base address of the Transmit list set by the VDMA_CH0_TxDesc_List_LAddress register. ■ The position at which the transmission was previously stopped If the DMA does not own the current descriptor, the transmission enters the Suspended state and the TBU bit of the VDMA_CH0_Status register is set. The Start Transmission command is effective only when the transmission is stopped. If the command is issued before setting the VDMA_CH0_TxDesc_List_LAddress register, the DMA behavior is unpredictable. When this bit is reset, the transmission process is placed in the Stopped state after completing the transmission of the current packet. The Next Descriptor position in the Transmit list is saved, and it becomes the current position when the transmission is restarted. To change the list address, program DMA_CH0_TxDesc_List_Address register with a new value when this bit is reset. The new value is considered when this bit is set again. The stop transmission command is effective only when the transmission of the current packet is complete or the transmission is in the Suspended state. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.7.3 VDMA_CH0_Rx_Control

- **Description:** The VDMA Channel0 Receive Control register controls the Rx features such as PBL, buffer size, and extended status.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3108
- **Exists:** (DWCXG_NUM_DMA_RX_CH>0)



31	30:28	27:24	23:22	21:16	15	14:y	x:1	0
RPF	RD2TCMP	RQOS	Reserved_23_22	RxPBL	Reserved_15	RBSZ	Reserved_x_1	SR

Table 11-561 Fields for Register: VDMA_CH0_Rx_Control

Bits	Name	Memory Access	Description
31	RPF	R/W	Rx VDMA Channel0 Packet Flush. When this bit is set to 1, then DWC_25gmac automatically flushes the packet from the Rx Queues destined to this DMA Rx Channel, when it is stopped or suspended due to unavailability of descriptors. When this bit remains set and the DMA is re-started by the software driver the packets residing in the Rx Queues that were received when the DMA is stopped, still gets flushed out. The packets received by the MAC only after the DMA is re-started is routed to the Rx DMA. The flushing is done on the Read side of the Rx Queue. When the packet is duplicated to multiple Rx DMA, it is flushed by the Rx DMA to which it is routed only when it is stopped or suspended due to unavailability of descriptors to prevent head-of-line blocking. The packet duplicated to multiple Rx DMA is not flushed when the Rx DMA is re-started, irrespective of whether the packet was received during one or more Rx DMAs were stopped. When this bit is set to 0, DWC_25gmac does not flush the packet in the Rx Queue destined to this Rx VDMA Channel when it is in STOP state. This might cause head-of-line blocking in the corresponding RxQueue. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}

Table 11-561 Fields for Register: VDMA_CH0_Rx_Control (continued)

Bits	Name	Memory Access	Description
30:28	RD2TCMP	R/W	Receive DMA to TC Mapping This field is used to map Receive DMA to priorities. The actual width of this field depends on the number of traffic classes selected during configuration. 3'b000: TC0 3'b001: TC1 3'b010: TC2 3'b011: TC3 3'b100: TC4 3'b101: TC5 3'b110: TC6 3'b111: TC7 On Rx side, Priorities are equivalent to TCs. **Value After Reset:** 0x0 **Exists:** SNPS_RSVDPARAM_50 **Testable:** {1?{untestable}:{unconstrained}}
27:24	RQOS	R/W	Receive QOS. This field is used to drive values on AXI Manager read control channel's QoS indicator or AXI Manager write control channel's QoS indicator (arqos_m_o[3:0] or awqos_m_o[3:0]) output signals of AXI bus for all transactions initiated by this Rx VDMA Channel. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}
23:22	Reserved_23_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
21:16	RxPBL	R/W	Receive Programmable Burst Length. This field controls the maximum size of burst transaction that can be initiated by the RX DMA channel for packet data transfers. The RX DMA channel attempts to transfer the data of this burst size if available, to the Receive Buffer pointed by the descriptor. You can program PBL with any of the following values: 1, 2, 4, 8, 16, or 32. Any other value results in undefined behavior. It is recommended to use higher values (8 and higher) for better performance. A value of 0 might result in undefined behavior. To transfer more than 32 beats, perform the following steps: ■ Set the PBLx8 mode in the VDMA_CH0_Control register. ■ Set the PBL. **Note-1:**This PBL controls the maximum burst size of the requests from this internal DMA engine. The AXI Interface block can split this PBL requests from DMA engine into multiple AXI requests based on the programmed values in DMA_SysBus_Mode register. **Note-2:**The maximum limit programmed must be less than or equal to half the size of the corresponding Rx Queue Depth (Queue Depth = Queue Size / Datawidth in bytes) of all the

Table 11-561 Fields for Register: VDMA_CH0_Rx_Control (continued)

Bits	Name	Memory Access	Description
			Rx Queues from which packets can be services by this RX DMA . For example, in 64-bit configuration, there are 8 bytes per location. So, when Rx Queue Size is 256 bytes, the Rx Queue Depth = 256/8 = 32. The RxPBL must be programmed is less that or equal to total no. of locations in Rx Queue/2, so RxPBL must be programmed less than 32. **Value After Reset:** 0x0 **Exists:** !SNPS_RSVDPARAM_50
15	Reserved_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
14:y	RBSZ	R/W	Receive Buffer size. This field indicates the size of the Rx buffers specified in bytes allocated by the software to store the packets the Rx DMA transfers to the host memory. The maximum buffer size is limited to 16K bytes. The buffer size is applicable to payload buffers when split headers are enabled. **Note-1:** The buffer size must be a multiple of 8 or 16 depending on the bus widths (64-bit or 128-bit respectively). This is required even if the value of buffer address pointer is not aligned to bus width. If the buffer size is not a multiple of 8 or 16, it might result into undefined behavior. The LSB bits (2:0 or 3:0) for 64-bit or 128-bit bus width are ignored and the DMA internally takes the LSB bits as all-zero. Therefore, these LSB bits are read-only (RO). Therefore, the maximum values that can be programmed into this field are 0x3FF8 (16376) and 0x3FF0 (16368) in 64-bit and 128-bit configurations, respectively. **Note-2:** It is recommended to program a minimum size of 256 bytes for better performance. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}} **Range Variable[y]:** DWCXG_BSA + 1
x:1	Reserved_x_1	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_BSA
0	SR	R/W	Start or Stop Receive. ■ 1: DMA tries to acquire the descriptor from the Receive list and processes the incoming packets.

Table 11-561 Fields for Register: VDMA_CH0_Rx_Control (continued)

Bits	Name	Memory Access	Description
			The DMA tries to acquire descriptor from either of the following positions: -- The current position in the list This is the address set by the VDMA_CH0_RxDesc_List_LAddress register. -- The position at which the Rx process was previously stopped If the DMA does not own the current descriptor, the reception is suspended and the RBU bit of the VDMA_CH0_Status register is set. The Start Receive command is effective only when the reception is stopped. If the command is issued before setting the VDMA_CH0_RxDesc_List_LAddress register, the DMA behavior is unpredictable. ■ 0: Rx DMA operation is stopped after the transfer of the current packet. The next descriptor position in the Receive list is saved, and it becomes the current position after the Rx process is restarted. The Stop Receive command is effective only when the Rx process is in the Running (waiting for Rx packet) or Suspended state. Similarly, Start Receive command is effective only when the Receive process is in the Stopped state. So, read the RD*S fields in the corresponding channel's DMA_CH0_Debug Status register before writing the SR bit (Start or Stop Receive) in DMA_CH0_RX_Control register. Additionally, when RSE and AIE bits of VDMA_CH0_Interrupt_Enable register are set, an interrupt is generated when the Rx DMA is stopped. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.7.4 VDMA_CH0_Slot_Function_Control_Status

- **Description:** This register contains the control bits for slot function and its status for Transmit path.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x310c
- **Exists:** !SNPS_RSVDPARAM_50&&(DWCXG_AV_SLOT_EN&&DWCXG_NUM_DMA_TX_CH>1)

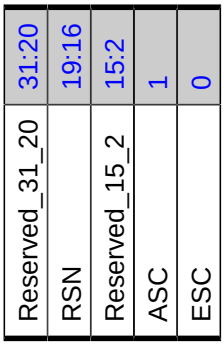


Table 11-562 Fields for Register: VDMA_CH0_Slot_Function_Control_Status

Bits	Name	Memory Access	Description
31:20	Reserved_31_20	R	Reserved **Value After Reset:** 0x0 **Exists:** Always
19:16	RSN	R	Reference Slot Number This field gives the current value of the reference slot number active in the DMA. It is used for slot comparison with the value programmed in SLOTNUM field of TX Descriptor before the data is fetched. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15:2	Reserved_15_2	R	Reserved **Value After Reset:** 0x0 **Exists:** Always
1	ASC	R/W	Advance Slot Check When this bit is set, the TX DMA fetches data from the buffer when the RSN value is equal to or less than the SLOTNUM value programmed in that descriptor by up to 2 slots. When this bit is reset, the DMA will start fetching the data when the RSN value is equal to or less the SLOTNUM value by one slot. This bit is applicable only when the ESC bit is set. **Value After Reset:** 0x0 **Exists:** Always

Table 11-562 Fields for Register: VDMA_CH0_Slot_Function_Control_Status (continued)

Bits	Name	Memory Access	Description
0	ESC	R/W	Enable Slot Comparison When set, this bit enables the checking of the slot numbers programmed in the Tx descriptor with the current reference given in the RSN field. The DMA fetches the data from the corresponding buffer only when the RSN value is equal to or less the SLOTNUM value programmed in the descriptor as determined by ASC. When reset, this bit disables the checking of the slot numbers. The DMA fetches the data immediately after the descriptor is processed. **Note:** Do not enable TSO along with TBS/Audio Video Bridging Slot number check. The TSO involves transmission of multiple segments for a single packet received from application and the slot number check is applicable for fetching only the first segment/fragment. So, it might be difficult for the software to specify slot number for subsequent packets. **Values:** ■ 0x0 (DISABLE): Slot Comparison is disabled ■ 0x1 (ENABLE): Slot Comparison is enabled **Value After Reset:** 0x0 **Exists:** Always

11.7.5 VDMA_CH0_TxDesc_List_HAddress

- **Description:** The Channel0 Tx Descriptor List HAddress register has the higher 8 or 16 bits of the start address of the Transmit descriptor list. This register is present only when DWC_25gmac is configured for 40-bit or 48-bit addressing mode.
  You can write to this register only when the Tx DMA has stopped, that is, the ST bit is set to zero in VDMA_CH0_Tx_Control register. When stopped, this register can be written with a new descriptor list address. When you set the ST bit to 1, the DMA takes the newly-programmed descriptor base address. If this register is not changed when the ST bit is set to 0, the DMA takes the descriptor address where it was stopped earlier. This register must be programmed with desired higher address before programming the VDMA_CH0_TxDesc_List_LAddress register.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3110
- **Exists:** (DWCXG_ADDRWIDTH_64&&DWCXG_NUM_DMA_TX_CH>0)

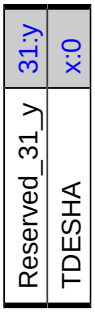


Table 11-563 Fields for Register: VDMA_CH0_TxDesc_List_HAddress

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_ADDRWIDTH_MNS32
x:0	TDESHA	R/W	Start of Transmit List. This field contains the most-significant 8 or 16 bits of the 40 or 48-bit base address of the first descriptor in the Transmit descriptor list. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_ADDRWIDTH_MNS32 - 1

11.7.6 VDMA_CH0_TxDesc_List_LAddress

- **Description:** The Channel0 Tx Descriptor List LAddress register has the lower 32 bits of the start address of the Transmit descriptor list. The descriptor lists reside in the physical memory space of the application and must be Dword, or Lword-aligned (for 64-bit or 128-bit data bus). The DMA internally converts it to bus width aligned address by making the corresponding LSB to low.
  You can write to this register only when the Tx DMA has stopped, that is, the ST bit is set to zero in VDMA_CH0_Tx_Control register. When stopped, this register can be written with a new descriptor list address. When you set the ST bit to 1, the DMA takes the newly-programmed descriptor base address. If this register is not changed when the ST bit is set to 0, the DMA takes the descriptor address where it was stopped earlier.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3114
- **Exists:** (DWCXG_NUM_DMA_TX_CH>0)

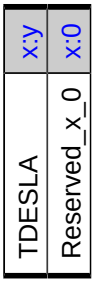


Table 11-564 Fields for Register: VDMA_CH0_TxDesc_List_LAddress

Bits	Name	Memory Access	Description
x:y	TDESLA	R/W	Start of Transmit List. This field contains the base address of the first descriptor in the Transmit descriptor list. The 32b address is formed by extending the TDESLA field with all-zero in the LSB bits below which are RO. The width of this field depends on the configuration: ■ 31:3 for 64-bit configuration ■ 31:4 for 128-bit configuration **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}} **Range Variable[x]:** (DWCXG_ADDRWIDTH==32)?(1+DWCXG_DESC_ADDR_MSB):((1+DWCXG_DESC_ADDR_MSB)-DWCXG_ADDRWIDTH_MNS32) + DWCXG_BSA - 1 **Range Variable[y]:** DWCXG_BSA

Table 11-564 Fields for Register: VDMA_CH0_TxDesc_List_LAddress (continued)

Bits	Name	Memory Access	Description
x:0	Reserved_x_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_BSA - 1

11.7.7 VDMA_CH0_RxDesc_List_HAddress

- **Description:** The Channel0 Rx Descriptor List HAddress register has the higher 8 or 16 bits of the start address of the Receive descriptor list. This register is present only when DWC_25gmac is configured for 40-bit or 48-bit addressing mode.
  You can write to this register only when the Rx DMA has stopped, that is, the SR bit is set to zero in VDMA_CH0_Rx_Control register. When stopped, this register can be written with a new descriptor list address. When you set the SR bit to 1, the DMA takes the newly-programmed descriptor base address. If this register is not changed when the SR bit is set to 0, the DMA takes the descriptor address where it was stopped earlier. This register must be programmed with desired higher address before programming the VDMA_CH0_RxDesc_List_LAddress register.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3118
- **Exists:** (DWCXG_ADDRWIDTH_64&&DWCXG_NUM_DMA_RX_CH>0)

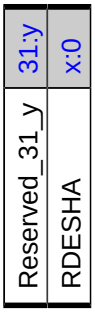


Table 11-565 Fields for Register: VDMA_CH0_RxDesc_List_HAddress

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_ADDRWIDTH_MNS32
x:0	RDESHA	R/W	Start of Receive List. This field contains the most-significant 8 or 16 bits of the 40- or 48-bit base address of the first descriptor in the Receive Descriptor list. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_ADDRWIDTH_MNS32 - 1

11.7.8 VDMA_CH0_RxDesc_List_LAddress

- **Description:** The Channel0 Rx Descriptor List LAddress register has the lower 32 bits of the start address of the Receive descriptor list.
  This register points to the start of the Receive Descriptor List. The descriptor lists reside in the physical memory space of the application and must be Dword or Lword-aligned (for 64-bit or 128-bit data bus). The DMA internally converts it to bus width aligned address by making the corresponding LS bits low. Writing to this register is permitted only when reception is stopped. When stopped, this register must be written to before the receive Start command is given. You can write to this register only when Rx DMA has stopped, that is, SR bit is set to zero in VDMA_CH0_Rx_Control register. When stopped, this register can be written with a new descriptor list address.
  When you set the SR bit to 1, the DMA takes the newly programmed descriptor base address.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x311c
- **Exists:** (DWCXG_NUM_DMA_RX_CH>0)

Table 11-566 Fields for Register: VDMA_CH0_RxDesc_List_LAddress

Bits	Name	Memory Access	Description
x:y	RDESLA	R/W	Start of Receive List. This field contains the base address of the first descriptor in the Rx Descriptor list. The 32b address is formed by extending the RDESLA field with all-zero in the LSB bits below which are RO. The width of this field depends on the configuration: ■ 31:3 for 64-bit configuration ■ 31:4 for 128-bit configuration **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}} **Range Variable[x]:** (DWCXG_ADDRWIDTH==32)?(1+DWCXG_DESC_ADDR_MSB):((1+DWCXG_DESC_ADDR_MSB)-DWCXG_ADDRWIDTH_MNS32) + DWCXG_BSA - 1 **Range Variable[y]:** DWCXG_BSA

Table 11-566 Fields for Register: VDMA_CH0_RxDesc_List_LAddress (continued)

Bits	Name	Memory Access	Description
x:0	Reserved_x_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_BSA - 1

11.7.9 VDMA_CH0_TxDesc_Tail_LPointer

- **Description:** The Channel0 Tx Descriptor Tail LPointer register has the 32 bits end address of the Transmit descriptor list.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3124
- **Exists:** (DWCXG_NUM_DMA_TX_CH>0)

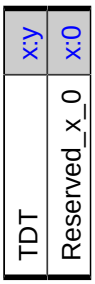


Table 11-567 Fields for Register: VDMA_CH0_TxDesc_Tail_LPointer

Bits	Name	Memory Access	Description
x:y	TDT	R/W	Transmit Descriptor Tail Pointer. This field contains the tail pointer for the Tx descriptor ring. The software writes the tail pointer to add more descriptors to the Transmit List. DWC_25gmac tries to transmit all packets referenced by the descriptors between the head and the tail pointer registers. The 32b address is formed by extending the TDT field with all-zero in the LSB bits below which are RO. The width of this field depends on the configuration: ■ 31:3 for 64-bit configuration ■ 31:4 for 128-bit configuration **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (DWCXG_ADDRWIDTH==32)?(1+DWCXG_DESC_ADDR_MSB):((1+DWCXG_DESC_ADDR_MSB)-DWCXG_ADDRWIDTH_MNS32) + DWCXG_BSA - 1 **Range Variable[y]:** DWCXG_BSA
x:0	Reserved_x_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_BSA - 1

11.7.10 VDMA_CH0_RxDesc_Tail_LPointer

- **Description:** The Channel0 Rx Descriptor Tail LPointer register has the 32 bits end address of the Receive descriptor list.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x312c
- **Exists:** (DWCXG_NUM_DMA_RX_CH>0)

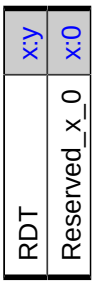


Table 11-568 Fields for Register: VDMA_CH0_RxDesc_Tail_LPointer

Bits	Name	Memory Access	Description
x:y	RDT	R/W	Receive Descriptor Tail Pointer. This field contains the tail pointer for the Rx descriptor ring. The software writes the tail pointer to add more descriptors to the Rx channel. DWC_25gmac tries to write all the received packets to the descriptors referenced between the head and the tail pointer registers. The 32b address is formed by extending the RDT field with all-zero in the LSB bits below which are RO. The width of this field depends on the configuration: ■ 31:3 for 64-bit configuration ■ 31:4 for 128-bit configuration **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (DWCXG_ADDRWIDTH==32)?(1+DWCXG_DESC_ADDR_MSB):((1+DWCXG_DESC_ADDR_MSB)-DWCXG_ADDRWIDTH_MNS32) + DWCXG_BSA - 1 **Range Variable[y]:** DWCXG_BSA
x:0	Reserved_x_0	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_BSA - 1

11.7.11 VDMA_CH0_Tx_Control2

- **Description:** The Tx_Control2 register contains programmable control to Outstanding read requests per DMA and the length of the Transmit descriptor ring.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3130
- **Exists:** (DWCXG_NUM_DMA_TX_CH>0)

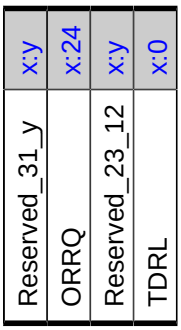


Table 11-569 Fields for Register: VDMA_CH0_Tx_Control2

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (32-24-(log(DWCXG_MAX_RDBREQ)/log(2))) + (24+(log(DWCXG_MAX_RDBREQ)/log(2))) - 1 **Range Variable[y]:** (24+(log(DWCXG_MAX_RDBREQ)/log(2)))
x:24	ORRQ	R/W	Read Outstanding Requests When this is set to 0, the DMA can initiate the maximum number of outstanding read transactions of packet data up to DWCXG_MAX_RDBREQ for each Transmit DMA channel. When this field is set to a value greater than zero, the DMA limits the number of outstanding read transactions to the value of ORRQ. **Value After Reset:** 0x0 **Exists:** (!SNPS_RSVDPARAM_50) **Range Variable[x]:** (log(DWCXG_MAX_RDBREQ)/log(2)) + 23

Table 11-569 Fields for Register: VDMA_CH0_Tx_Control2 (continued)

Bits	Name	Memory Access	Description
x:y	Reserved_23_12	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (24-((DWCXG_RINGLEN_16B==1)?16:12)) + ((DWCXG_RINGLEN_16B==1)?16:12) - 1 **Range Variable[y]:** ((DWCXG_RINGLEN_16B==1)?16:12)
x:0	TDRL	R/W	Transmit Descriptor Ring Length. This field sets the maximum number of Tx descriptors in the circular descriptor ring. . ■ 0 implies ring length of 1 descriptor. ■ 0xFFFF implies a ring length of 65536 descriptors. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}} **Range Variable[x]:** ((DWCXG_RINGLEN_16B==1)?16:12) - 1

11.7.12 VDMA_CH0_Rx_Control2

- **Description:** The Channel0 Rx Control2 register contains programmable control to Outstanding Write requests and the length of the Receive descriptor circular ring.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3134
- **Exists:** (DWCXG_NUM_DMA_RX_CH>0)

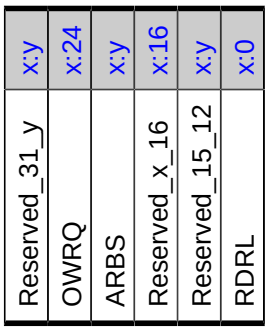


Table 11-570 Fields for Register: VDMA_CH0_Rx_Control2

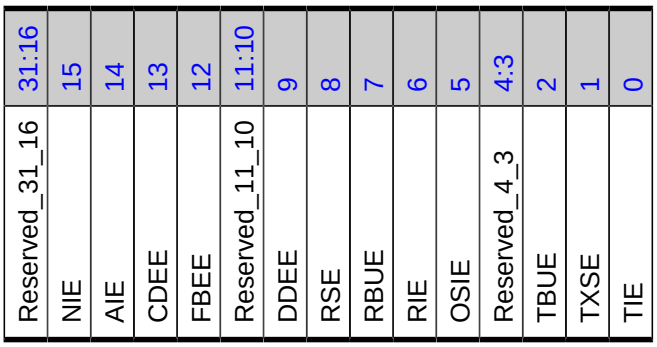
Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (32-24-(log(DWCXG_MAX_WRBREQ)/log(2))) + (24+(log(DWCXG_MAX_WRBREQ)/log(2))) - 1 **Range Variable[y]:** (24+(log(DWCXG_MAX_WRBREQ)/log(2)))
x:24	OWRQ	R/W	Write Outstanding Requests When this is set to 0, the DMA can initiate the maximum number of outstanding write transactions of packet data, up to DWCXG_MAX_WRBREQ for each Rx DMA channel. When this field is set to a value greater than zero, the DMA limits the number of outstanding write transactions to the value of OWRQ. **Value After Reset:** 0x0 **Exists:** (!SNPS_RSVDPARAM_50) **Range Variable[x]:** (log(DWCXG_MAX_WRBREQ)/log(2)) + 23

Table 11-570 Fields for Register: VDMA_CH0_Rx_Control2 (continued)

Bits	Name	Memory Access	Description
x:y	ARBS	R/W	Alternate Receive Buffer Size Indicates size in bytes for Buffer 1 when ARBS is programmed to a non-zero value (when split header feature is not enabled). When split header feature is enabled, ARBS indicates the buffer size for header data. The maximum alternate buffer is limited to 1016 or 1008-bytes depending on the data bus widths (64-bit or 128-bit respectively). When ARBS=0, Rx Buffer1 and Rx Buffer2 sizes are based on RBSZ field of DMA_CH(#i)_Rx_Control. Width of ARBS field is 7 or 6-bits depending on the data bus widths (64-bit or 128-bit respectively) **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}} **Range Variable[x]:** (DWCXG_DATAWIDTH==64)?7:6 + (DWCXG_DATAWIDTH==64)?17:18 - 1 **Range Variable[y]:** (DWCXG_DATAWIDTH==64)?17:18
x:16	Reserved_x_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (DWCXG_DATAWIDTH==64)?1:2 + 15
x:y	Reserved_15_12	R	Reserved. **Value After Reset:** 0x0 **Exists:** 0 **Range Variable[x]:** (16-((DWCXG_RINGLEN_16B==1)?16:12)) + ((DWCXG_RINGLEN_16B==1)?16:12) - 1 **Range Variable[y]:** ((DWCXG_RINGLEN_16B==1)?16:12)
x:0	RDRL	R/W	Receive Descriptor Ring Length. This field sets the maximum number of Rx descriptors in the circular descriptor ring. ■ 0 implies ring length of 1 descriptor. ■ 0xFFFF implies a ring length of 65536 descriptors. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}} **Range Variable[x]:** ((DWCXG_RINGLEN_16B==1)?16:12) - 1

11.7.13 VDMA_CH0_Interrupt_Enable

- **Description:** The Channel0 Interrupt Enable register enables the interrupts reported by the Status register.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3138
- **Exists:** Always



31:16	15	14	13	12	11:10	9	8	7	6	5	4:3	2	1	0
Reserved_31_16	NIE	AIE	CDEE	FBEE	Reserved_11_10	DDEE	RSE	RBUE	RIE	OSIE	Reserved_4_3	TBUE	TXSE	TIE

Table 11-571 Fields for Register: VDMA_CH0_Interrupt_Enable

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15	NIE	R/W	Normal Interrupt Summary Enable. When this bit is set, the normal interrupt summary is enabled. This bit enables the following interrupts in the VDMA_CH0_Status register: ■ Bit 0: Transmit Interrupt ■ Bit 2: Transmit Buffer Unavailable ■ Bit 6: Receive Interrupt When this bit is reset, the normal interrupt summary is disabled. **Value After Reset:** 0x0 **Exists:** Always

Table 11-571 Fields for Register: VDMA_CH0_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
14	AIE	R/W	Abnormal Interrupt Summary Enable. When this bit is set, the abnormal interrupt summary is enabled. This bit enables the following interrupts in the VDMA_CH0_Status register: ■ Bit 1: Transmit Process Stopped ■ Bit 7: Rx Buffer Unavailable ■ Bit 8: Receive Process Stopped ■ Bit 9: Descriptor Definition Error ■ Bit 12: Fatal Bus Error ■ Bit 13: Context Descriptor Error When this bit is reset, the abnormal interrupt summary is disabled. **Value After Reset:** 0x0 **Exists:** Always
13	CDEE	R/W	Context Descriptor Error Enable. When this bit is set along with the AIE bit, the Context Descriptor error interrupt is enabled. When this bit is reset, the Context Descriptor error interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_TX_CH>0)
12	FBEE	R/W	Fatal Bus Error Enable. When this bit is set along with the AIE bit, the Fatal Bus error interrupt is enabled. When this bit is reset, the Fatal Bus Error error interrupt is disabled. **Value After Reset:** 0x0 **Exists:** Always
11:10	Reserved_11_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
9	DDEE	R/W	Descriptor Definition Error Enable. When this bit is set along with the AIE bit, the Descriptor Definition error interrupt is enabled. When this bit is reset, the Descriptor Definition Error error interrupt is disabled. **Value After Reset:** 0x0 **Exists:** Always

Table 11-571 Fields for Register: VDMA_CH0_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
8	RSE	R/W	Receive Stopped Enable. When this bit is set along with the AIE bit, the Receive Stopped Interrupt is enabled. When this bit is reset, the Receive Stopped interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_RX_CH>0)
7	RBUE	R/W	Receive Buffer Unavailable Enable. When this bit is set along with the AIE bit, the Receive Buffer Unavailable interrupt is enabled. When this bit is reset, the Receive Buffer Unavailable interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_RX_CH>0)
6	RIE	R/W	Receive Interrupt Enable. When this bit is set along with the NIE bit, the Receive Interrupt is enabled. When this bit is reset, the Receive Interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_RX_CH>0)
5	OSIE	R/W	Out of Sequence Interrupt Enable. When this bit is set, the RI bit in VDMA_CH_Status register is set to high when OSI becomes high. When this bit is reset, the setting of OSI does not affect the RI bit. **Value After Reset:** 0x0 **Exists:** ((DWCXG_RSS_EN&&DWCXG_SPLIT_HDR_EN))
4:3	Reserved_4_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
2	TBUE	R/W	Transmit Buffer Unavailable Enable. When this bit is set along with the NIE bit, the Transmit Buffer Unavailable interrupt is enabled. When this bit is reset, the Transmit Buffer Unavailable interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_TX_CH>0)

Table 11-571 Fields for Register: VDMA_CH0_Interrupt_Enable (continued)

Bits	Name	Memory Access	Description
1	TXSE	R/W	Transmit Stopped Enable. When this bit is set along with the AIE bit, the Transmission Stopped interrupt is enabled. When this bit is reset, the Transmission Stopped interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_TX_CH>0)
0	TIE	R/W	Transmit Interrupt Enable. When this bit is set along with the NIE bit, the Transmit Interrupt is enabled. When this bit is reset, the Transmit Interrupt is disabled. **Value After Reset:** 0x0 **Exists:** (DWCXG_NUM_DMA_TX_CH>0)

11.7.14 VDMA_CH0_Rx_Interrupt_Watchdog_Timer

- **Description:** The Receive Interrupt Watchdog Timer register indicates the watchdog/Idle timeout for Receive Interrupt (RI) from the DMA. This can be in terms of number of clock cycles (RWT field) or number of transferred bytes or number of packets received (RBCT) based on the value of programmable bit PSEL for the watchdog timer. When either of these fields are written with a non-zero value, the interrupt timer is enabled when a packet transfer is completed for which RI is not generated. For Idle timer the timeout is given based on number of AXI clock cycles (ITW*ITCU fields), and the timer is enabled when a non-zero value is written in the ITW field. The timers are cleared when the RI is generated by that RxDMA channel.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x313c
- **Exists:** (DWCXG_NUM_DMA_RX_CH>0)

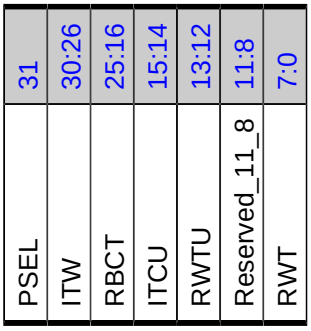


Table 11-572 Fields for Register: VDMA_CH0_Rx_Interrupt_Watchdog_Timer

Bits	Name	Memory Access	Description
31	PSEL	R/W	Packet Count Interrupt Select. When this bit is set, RBCT indicates the number of packet threshold at which the Receive Interrupt (RI) is activated and the watchdog timer reset. When this bit is low, RBCT indicates the number of data byte threshold at which the Receive Interrupt (RI) is activated and the watchdog timer reset. ■ 1: RBCT field is used to generate interrupt based on number of packets received. ■ 0: RBCT field is used to generate interrupt based on number of bytes received. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}

Table 11-572 Fields for Register: VDMA_CH0_Rx_Interrupt_Watchdog_Timer (continued)

Bits	Name	Memory Access	Description
30:26	ITW	R/W	Idle Timer Window. This field when programmed with a non-zero value enables the operation of RX Idle Timer. The timer window is determined in number of AXI clock cycles multiplied by the ITCU unit. The idle timer is triggered with the programmed value when RIE is set for the Rx DMA and RI bit is not set in the VDMA_CH0_Status register. When the idle timer runs out, the RI bit is set and the timer is restarted. The idle timer is reset when the RI bit is set high. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}
25:16	RBCT	R/W	Receive Byte Count Threshold. This field sets the limit in number of bytes or number of RX packets (based on PSEL) transferred without the assertion of Receive Interrupt. Once this limit is crossed, Receive interrupt is asserted, and the watchdog timer (corresponding to RWT) is also reset. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}
15:14	ITCU	R/W	Idle Timer Count Units. This fields indicates the number of AXI clock cycles corresponding to one unit in ITW field. ■ 2'b00: 256 ■ 2'b01: 512 ■ 2'b10: 1024 ■ 2'b11: 2048 For example, when ITW=01010 and ITCU=1, the idle timer is set for 10*512=5120 AXI clock cycles. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}

Table 11-572 Fields for Register: VDMA_CH0_Rx_Interrupt_Watchdog_Timer (continued)

Bits	Name	Memory Access	Description
13:12	RWTU	R/W	Receive Interrupt Watchdog Timer Count Units. This fields indicates the number of system clock cycles corresponding to one unit in RWT field. ■ 2'b00: 256 ■ 2'b01: 512 ■ 2'b10: 1024 ■ 2'b11: 2048 For example, when RWT=2 and RWTU=1, the watchdog timer is set for 2*512=1024 system clock cycles. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}
11:8	Reserved_11_8	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
7:0	RWT	R/W	Receive Interrupt Watchdog Timer Count. This field control the number of AXI clock cycles, multiplied by factor indicated in RWTU field, for which the watchdog timer is set. The watchdog timer is triggered with the programmed value after the Rx DMA completes the transfer of a packet for which the RI bit is not set in the VDMA_CH0_Status register, because the Interrupt Enable bit in the corresponding descriptor RDES3[30] is reset. When the watchdog timer runs out, the RI bit is set and the timer is stopped. The watchdog timer is reset when the RI bit is set high whenever the Interrupt Enable bit (RDES3[30]) is set in descriptor, corresponding to the RX packet transfer or due to the triggering of RBCT counter threshold. **Value After Reset:** 0x0 **Exists:** Always **Testable:** {1?{untestable}:{unconstrained}}

11.7.15 VDMA_CH0_Current_App_TxDesc_H

- **Description:** The Channel0 Current Application Transmit Descriptor High register has the higher 32 bits of the current address of the Transmit descriptor read by the DMA.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3140
- **Exists:** 0

Table 11-573 Fields for Register: VDMA_CH0_Current_App_TxDesc_H

Bits	Name	Memory Access	Description
31:0	CURTDESAPTRH	R	Application Transmit Descriptor Address Pointer. The DMA updates this pointer during Tx operation. This pointer is cleared on reset. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.7.16 VDMA_CH0_Current_App_TxDesc_L

- **Description:** The Channel0 Current Application Transmit Descriptor High register has the lower 32 bits of the current address of the Transmit descriptor read by the DMA. The upper 32 address bits of the Current Transmit Descriptor are equal to the value written in the VDMA_CH0_TxDesc_List_HAddress. Therefore, the upper address bits are not given in the register map.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3144
- **Exists:** (DWCXG_NUM_DMA_TX_CH>0)

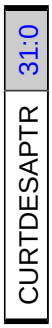


Table 11-574 Fields for Register: VDMA_CH0_Current_App_TxDesc_L

Bits	Name	Memory Access	Description
31:0	CURTDESAPTR	R	Application Transmit Descriptor Address Pointer. This gives the address of the Descriptor of the next descriptor that is going to be fetched from system memory. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.7.17 VDMA_CH0_Current_App_RxDesc_H

- **Description:** The Channel0 Current Application Receive Descriptor High register has the higher 32 bits of the current address of the Receive descriptor read by the DMA.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3148
- **Exists:** 0

Table 11-575 Fields for Register: VDMA_CH0_Current_App_RxDesc_H

Bits	Name	Memory Access	Description
31:0	CURRDESAPTRH	R	Application Receive Descriptor Address Pointer. The DMA updates this pointer during Rx operation. This pointer is cleared on reset. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.7.18 VDMA_CH0_Current_App_RxDesc_L

- **Description:** The Channel0 Current Application Receive Descriptor Low register has the lower 32 bits of the current address of the Receive descriptor read by the DMA. The upper 32 address bits of the Current Receive Descriptor are equal to the value written in the VDMA_CH0_RxDesc_List_HAddress. Therefore, the upper address bits are not given in the register map.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x314c
- **Exists:** (DWCXG_NUM_DMA_RX_CH>0)

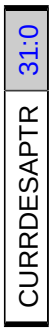


Table 11-576 Fields for Register: VDMA_CH0_Current_App_RxDesc_L

Bits	Name	Memory Access	Description
31:0	CURRDESAPTR	R	Application Receive Descriptor Address Pointer. This gives the address of the next descriptor that is going to be fetched from system memory. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.7.19 VDMA_CH0_Current_App_TxBuffer_H

- **Description:** The Channel0 Current Application Transmit Buffer Address High register has the higher 8 or 16 bits of the current address of the Transmit buffer address read by the DMA. This register is present only when DWC_25gmac is configured for 40-bit or 48-bit addressing mode.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3150
- **Exists:** (DWCXG_ADDRWIDTH_64&&DWCXG_NUM_DMA_TX_CH>0)

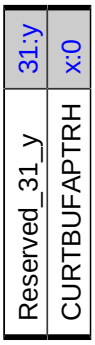


Table 11-577 Fields for Register: VDMA_CH0_Current_App_TxBuffer_H

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_ADDRWIDTH_MNS32
x:0	CURTBUFAPTRH	R	Application Receive Buffer Address Pointer. This field is set to the value of the last data buffer address specified in Transmit Descriptor that DMA put read request on to the AXI interface. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_ADDRWIDTH_MNS32 - 1

11.7.20 VDMA_CH0_Current_App_TxBuffer_L

- **Description:** The Channel0 Current Application Transmit Buffer Address Low register has the lower 32 bits of the current address of the Transmit buffer address read by the DMA.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3154
- **Exists:** (DWCXG_NUM_DMA_TX_CH>0)

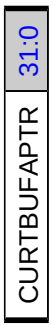


Table 11-578 Fields for Register: VDMA_CH0_Current_App_TxBuffer_L

Bits	Name	Memory Access	Description
31:0	CURTBUFAPTR	R	Application Transmit Buffer Address Pointer. This field is set to the value of the last data buffer address specified in Transmit Descriptor that DMA put read request on to the AXI interface. **Value After Reset:** 0x0 **Exists:** Always

11.7.21 VDMA_CH0_Current_App_RxBuffer_H

- **Description:** The Channel0 Current Application Receive Buffer Address High register has the higher 8 or 16 bits of the current address of the Receive buffer address read by the DMA. This register is present only when DWC_25gmac is configured for 40-bit or 48-bit addressing mode.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3158
- **Exists:** (DWCXG_ADDRWIDTH_64&&DWCXG_NUM_DMA_RX_CH>0)

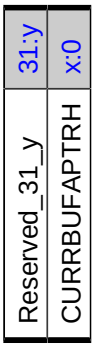


Table 11-579 Fields for Register: VDMA_CH0_Current_App_RxBuffer_H

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_ADDRWIDTH_MNS32
x:0	CURRBUFAPTRH	R	Application Receive Buffer Address Pointer. This field is set to the value of the address of the first of memory block allocated by software (the address in RDESC) and DMA transferred data from that memory block in normal DMA. This field is set to the value of the last address DMA put request on AXI interface in DMA. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_ADDRWIDTH_MNS32 - 1

11.7.22 VDMA_CH0_Current_App_RxBuffer_L

- **Description:** The Channel0 Current Application Receive Buffer Address Low register has the lower 32 bits of the current address of the Receive buffer address read by the DMA.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x315c
- **Exists:** (DWCXG_NUM_DMA_RX_CH>0)

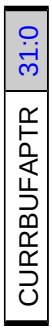


Table 11-580 Fields for Register: VDMA_CH0_Current_App_RxBuffer_L

Bits	Name	Memory Access	Description
31:0	CURRBUFAPTR	R	Application Receive Buffer Address Pointer. This field is set to the value of the last data buffer address specified in Receive Descriptor that DMA put write request on to the AXI interface. **Value After Reset:** 0x0 **Exists:** Always

11.7.23 VDMA_CH0_Status

- **Description:** The software driver (application) reads the Status register during interrupt service routine or polling to determine the status of the DMA.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3160
- **Exists:** Always

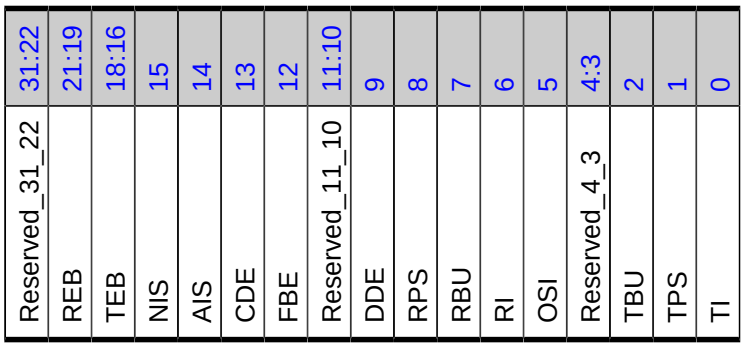


Table 11-581 Fields for Register: VDMA_CH0_Status

Bits	Name	Memory Access	Description
31:22	Reserved_31_22	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
21:19	REB	R/W	Rx DMA Error Bits. This field indicates the type of error that caused a Bus Error. For example, error response on the AXI interface. Bit 21 ■ 1'b1: Error during data transfer by the Rx DMA ■ 1'b0: No error during data transfer by the Rx DMA Bit 20 ■ 1'b1: Error during descriptor access ■ 1'b0: Error during data buffer access Bit 19 ■ 1'b1: Error during read transfer ■ 1'b0: Error during write transfer This field is valid only when the FBE bit is set. This field does not generate an interrupt. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-581 Fields for Register: VDMA_CH0_Status (continued)

Bits	Name	Memory Access	Description
18:16	TEB	R/W	Tx DMA Error Bits. This field indicates the type of error that caused a Bus Error. For example, error response on the AXI interface. Bit 18 ■ 1'b1: Error during data transfer by the Tx DMA ■ 1'b0: No Error during data transfer by the Tx DMA Bit 17 ■ 1'b1: Error during descriptor access ■ 1'b0: Error during data buffer access Bit 16 ■ 1'b1: Error during read transfer ■ 1'b0: Error during write transfer This field is valid only when the FBE bit is set. This field does not generate an interrupt. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
15	NIS	R/W	Normal Interrupt Summary. Normal Interrupt Summary bit value is the logical OR of the following bits when the corresponding interrupt bits are enabled in the VDMA_CH0_Interrupt_Enable register: ■ Bit 0: Transmit Interrupt ■ Bit 2: Transmit Buffer Unavailable ■ Bit 6: Receive Interrupt Only unmasked bits (interrupts for which interrupt enable is set in VDMA_CH0_Interrupt_Enable register) affect the Normal Interrupt Summary bit. This is a sticky bit. You must clear this bit (by writing 1 to this bit) each time a corresponding bit which causes NIS to be set is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-581 Fields for Register: VDMA_CH0_Status (continued)

Bits	Name	Memory Access	Description
14	AIS	R/W	Abnormal Interrupt Summary. Abnormal Interrupt Summary bit value is the logical OR of the following when the corresponding interrupt bits are enabled in the VDMA_CH0_Interrupt_Enable register: ■ Bit 1: Transmit Process Stopped ■ Bit 7: Receive Buffer Unavailable ■ Bit 8: Receive Process Stopped ■ Bit 9: Descriptor Definition Error ■ Bit 12: Fatal Bus Error ■ Bit 13: Context Descriptor Error Only unmasked bits affect the Abnormal Interrupt Summary bit. This is a sticky bit. You must clear this bit (by writing 1 to this bit) each time a corresponding bit, which causes AIS to be set, is cleared. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
13	CDE	R/W	Context Descriptor Error. This bit when set, indicates that the DMA Tx engine received a context descriptor in the middle of a packet (in an intermediate descriptor), and the DMA Tx engine ignored it. This is a sticky bit and is cleared only when a "1" is written to this bit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
12	FBE	R/W	Fatal Bus Error. This bit when set, indicates that a bus error occurred (as described in the REB and TEB fields). When this bit is set, the corresponding DMA channel engine disables all subsequent transactions and proceeds to STOP state. This is a sticky bit and is cleared only when a "1" is written to this bit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
11:10	Reserved_11_10	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-581 Fields for Register: VDMA_CH0_Status (continued)

Bits	Name	Memory Access	Description
9	DDE	R/W	Descriptor Definition Error. This bit when set, indicates that a Descriptor Definition error (Descriptor content is all-ones) occurred. When this bit is set, the corresponding DMA channel engine stops Descriptor Fetch. This is a sticky bit and is cleared only when a "1" is written to this bit. The recovery is only by issuing a Software Reset, that is setting SWR bit of DMA_Mode register, followed by reconfiguration and restarting. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
8	RPS	R/W	Receive Process Stopped. This bit is asserted when the Rx DMA process enters the Stopped state. This is a sticky bit and is cleared only when a "1" is written to this bit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
7	RBU	R/W	Receive Buffer Unavailable. This bit indicates that the application owns the next descriptor in the Receive list, and the DMA cannot acquire it. The Rx process is suspended. To resume the Rx process, the application must change the ownership of the descriptor and advance the VDMA_CH0_RxDesc_Tail_LPointer register. If this is not done, the RxDMA remains in SUSPEND state and results in overflow of the corresponding RxQueue. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
6	RI	R/W	Receive Interrupt. This bit indicates that the packet reception is complete. When packet reception is complete, Bit 31 of RDES1 is reset in the last descriptor, and the specific packet status information is updated in the descriptor. The reception remains in the Running state. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-581 Fields for Register: VDMA_CH0_Status (continued)

Bits	Name	Memory Access	Description
5	OSI	R/W	Out of Sequence Interrupt. This bit when set, indicates that the Rx-DMA has determined an out-of-sequence TCP/IP packet transfer to the host memory. This is a sticky bit and is cleared when 1 is written into it. **Value After Reset:** 0x0 **Exists:** ((DWCXG_RSS_EN&&DWCXG_SPLIT_HDR_EN)) **Testable:** untestable
4:3	Reserved_4_3	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
2	TBU	R/W	Transmit Buffer Unavailable. This bit when set, indicates that the application owns the next descriptor in the Transmit Descriptor list, and the DMA cannot acquire it and hence is suspended. To resume the transmission, the application must do the following: ■ Change the ownership of the descriptor by setting Bit 31 of TDES0. ■ Issue a Transmit Poll Demand command by performing a write to the Transmit Descriptor Tail Pointer register. This is a sticky bit and is cleared only when a '1' is written to this bit. **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable
1	TPS	R/W	Transmit Process Stopped. This bit when set, indicates that the Transmit DMA process has entered STOP state. This is a sticky bit and is cleared only when a "1" is written to this bit **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

Table 11-581 Fields for Register: VDMA_CH0_Status (continued)

Bits	Name	Memory Access	Description
0	TI	R/W	Transmit Interrupt. This bit when set, indicates that transfer of packet to the TxQueue is completed. This is set only when the corresponding Tx packet descriptor's IOC bit (TDES2[31]) is set. This is a sticky bit and is cleared only when a "1" is written to this bit **Value After Reset:** 0x0 **Exists:** Always **Testable:** untestable

11.7.24 VDMA_CH0_Debug_Status

- **Description:** DMA Channe0 Debug Status register
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3164
- **Exists:** ((DWCXG == 2) || SNPS_RSVDPARAM_50==1)

31	30:28	27:19	18:16	15	14:12	11:8	7:6	5:3	2:0
Reserved_31	RDWS	RDTS	RDFS	Reserved_15	TDWS	TDTS	TDRS	TDXS	TDFS

Table 11-582 Fields for Register: VDMA_CH0_Debug_Status

Bits	Name	Memory Access	Description
31	Reserved_31	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
30:28	RDWS	R	RxDMA Descriptor Write FSM state ■ 'x0: STOP/IDLE state ■ 'x1: Request to Arbiter for Descriptor Write state ■ 'x2: Descriptor Write Transfer state ■ 'x3: Waiting for AXI Write Response state ■ 'x4: ERROR state ■ 'x5-'x7: Reserved **Value After Reset:** 0x0 **Exists:** Always

Table 11-582 Fields for Register: VDMA_CH0_Debug_Status (continued)

Bits	Name	Memory Access	Description
27:19	RDTS	R	RxDMA Data Transfer FSM State ■ 'x0: STOP state ■ 'x1: waiting for the New Descriptor from RDF state ■ 'x2: Waiting for Packet Header Status from ARI state ■ 'x3: Waiting for new intermediate Descriptor from RDF state ■ 'x4: Waiting for Watermark on ARI state ■ 'x5: Request for Data Transfer on AXI state ■ 'x6: Write Data Transfer on AXI state ■ 'x7: Write Placeholder Data after receiving AXI Error state ■ 'x8: Waiting for previous descriptor to closed by RDW state ■ 'x9: Error state ■ 'xA-'xF: Reserved **Value After Reset:** 0x0 **Exists:** Always
18:16	RDFS	R	RxDMA Descriptor Fetch FSM State ■ 'x0: STOP/IDLE State ■ 'x1: Requesting for Descriptor Read ■ 'x2: Reading the Descriptor and writing into Descriptor Cache ■ 'x3: SUSPEND state ■ 'x4: ERROR state ■ 'x5: Waiting for Descriptor data transfer complete after OWN bit error ■ 'x6: Start Read descriptor with OWN bit not set again ■ 'x7: Reserved **Value After Reset:** 0x0 **Exists:** Always
15	Reserved_15	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
14:12	TDWS	R	TxDMA Descriptor Write FSM state ■ 'x0: STOP/IDLE state ■ 'x1: Request to Arbiter for Descriptor Write state ■ 'x2: Descriptor Write Transfer state ■ 'x3: Waiting for AXI Write Response state ■ 'x4: ERROR state ■ 'x5-'x7: Reserved **Value After Reset:** 0x0 **Exists:** Always

Table 11-582 Fields for Register: VDMA_CH0_Debug_Status (continued)

Bits	Name	Memory Access	Description
11:8	TDTS	R	TxDMA Data Transfer Control Module FSM State ■ 'x0: STOP state ■ 'x1: IDLE state ■ 'x2: Buffer Reading request ■ 'x3: Internal normal control word writing ■ 'x4: ERROR state ■ 'x5: Internal context control word writing ■ 'x6: Internal Timestamp correction updating ■ 'x7: TSO header Reading request ■ 'x8: TSO payload Reading request ■ 'x9: Writing pended TSO header descriptor ■ 'xA: Waiting for TSO memory writing complete ■ 'xB: TSO header cache Reading request ■ 'xC-'xF: TDT in reset, or Some unknown error (must not be here) **Value After Reset:** 0x0 **Exists:** Always
7:6	TDRS	R	TxDMA Data Request FSM State ■ 'x0: IDLE state ■ 'x1: Checking Tx FIFO space state ■ 'x2: AXI or TSO cache request state ■ 'x3: Internal Request information updating state **Value After Reset:** 0x0 **Exists:** Always
5:3	TDXS	R	TxDMA Data Transfer FSM State ■ 'x0: IDLE state ■ 'x1: Control word writing state ■ 'x2: Data transferring state ■ 'x3: Descriptor Writing Back state ■ 'x4: Writing Timestamp correction field state (Reserved if Data Width is greater than 64) ■ 'x5-'x7: Reserved **Value After Reset:** 0x0 **Exists:** Always

Table 11-582 Fields for Register: VDMA_CH0_Debug_Status (continued)

Bits	Name	Memory Access	Description
2:0	TDFS	R	TxDMA Descriptor Fetch FSM State ■ 'x0: STOP/IDLE State ■ 'x1: Requesting for Descriptor Read state ■ 'x2: Reading the Descriptor and writing into Descriptor Cache state ■ 'x3: SUSPEND state ■ 'x4: ERROR state ■ 'x5: Waiting for Descriptor data transfer complete after OWN bit error ■ 'x6: Start Read descriptor with OWN bit not set again ■ 'x7: Reserved **Value After Reset:** 0x0 **Exists:** Always

11.7.25 VDMA_CH0_Desc_Mem_Cache_Fill_Level

- **Description:** The VDMA Channel0 Descriptor Cache Fill Level Status register.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3168
- **Exists:** ((DWCXG == 2) || SNPS_RSVDPARAM_50==1)

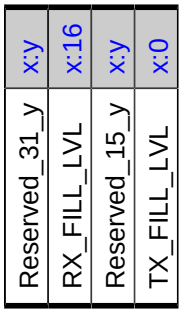


Table 11-583 Fields for Register: VDMA_CH0_Desc_Mem_Cache_Fill_Level

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 16-((DWCXG_DATAWIDTH==64)?DWCXG_PCHRDC_PW+3:DWCXG_PCHRDC_PW+2) + 16+((DWCXG_DATAWIDTH==64)?DWCXG_PCHRDC_PW+3:DWCXG_PCHRDC_PW+2) - 1 **Range Variable[y]:** 16+((DWCXG_DATAWIDTH==64)?DWCXG_PCHRDC_PW+3:DWCXG_PCHRDC_PW+2)
x:16	RX_FILL_LVL	R	RxDMA Descriptor Cache Fill Level This field gives the fill level of the Descriptor Cache for Rx VDMA Channel. A value of 0 implies empty and a value equal to size of Descriptor Cache implies it is full. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_DATAWIDTH==64)?DWCXG_PCHRDC_PW+3:DWCXG_PCHRDC_PW+2) + 15

Table 11-583 Fields for Register: VDMA_CH0_Desc_Mem_Cache_Fill_Level (continued)

Bits	Name	Memory Access	Description
x:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** 16-((DWCXG_DATAWIDTH==64)?DWCXG_PCHTDC_PW+3:DWCXG_PCHTDC_PW+2) + ((DWCXG_DATAWIDTH==64)?DWCXG_PCHTDC_PW+3:DWCXG_PCHTDC_PW+2) - 1 **Range Variable[y]:** ((DWCXG_DATAWIDTH==64)?DWCXG_PCHTDC_PW+3:DWCXG_PCHTDC_PW+2)
x:0	TX_FILL_LVL	R	TxDMA Descriptor Cache Fill Level This field gives the fill level of the Descriptor Cache for TxDMA channel. A value of 0 implies empty and a value equal to size of Descriptor Cache implies it is full. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_DATAWIDTH==64)?DWCXG_PCHTDC_PW+3:DWCXG_PCHTDC_PW+2) - 1

11.7.26 VDMA_CH0_Miss_Packet_Cnt

- **Description:** This register has the number of the packets that are dropped by the DMA either due to Bus Error or due to programming the RPF field in the VDMA_CH0_Rx_Control register.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x316c
- **Exists:** (DWCXG_NUM_DMA_RX_CH>0)

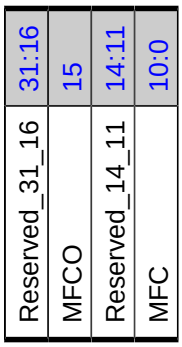


Table 11-584 Fields for Register: VDMA_CH0_Miss_Packet_Cnt

Bits	Name	Memory Access	Description
31:16	Reserved_31_16	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
15	MFCO	R	Overflow status of MFC Counter When this bit is set, the MFC counter reached the maximum value and wrapped around to 0. The MFC counter does not get incremented further. This bit gets cleared when this register is read. **Value After Reset:** 0x0 **Exists:** Always
14:11	Reserved_14_11	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always
10:0	MFC	R	Missed Packet Counter This counter indicates the number of packet counter that are dropped by the DMA either due to Bus Error or due to programming RPF field in VDMA_CH0_Rx_Control register. (Refer RPF field of VDMA_CH0_Rx_Control register for more details.) The counter gets cleared when this register is read. **Value After Reset:** 0x0 **Exists:** Always

11.7.27 VDMA_CH0_Tx_Data_Xfer_Ring_Offset

- **Description:** This register indicates the Offset of the Tx Descriptor List, for which the DMA engine is fetching the data.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3170
- **Exists:** ((DWCXG == 2) || SNPS_RSVDPARAM_50==1)

Table 11-585 Fields for Register: VDMA_CH0_Tx_Data_Xfer_Ring_Offset

Bits	Name	Memory Access	Description
31:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[y]:** DWCXG_RING_LENGTHW
x:0	DESC_OFFSET	R	TxDMA channel0 Data Transfer ring Offset In DMA, this field indicates the ring offset of the Descriptor DMA currently working on data transmission. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** DWCXG_RING_LENGTHW - 1

11.7.28 VDMA_CH0_Rx_Data_Xfer_Ring_Offset

- **Description:** The VDMA Channel0 Receive Data Transfer Ring Offset register indicates the Ring Offset of the Rx Descriptor List, for which the DMA engine is transferring the data.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3174
- **Exists:** ((DWCXG == 2) || SNPS_RSVDPARAM_50==1)

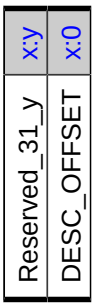


Table 11-586 Fields for Register: VDMA_CH0_Rx_Data_Xfer_Ring_Offset

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** (32-((DWCXG_RINGLEN_16B==1)?16:12)) + ((DWCXG_RINGLEN_16B==1)?16:12) - 1 **Range Variable[y]:** ((DWCXG_RINGLEN_16B==1)?16:12)
x:0	DESC_OFFSET	R	Rx VDMA Channel0 Data Transfer Ring Offset In DMA, this field indicates the ring offset of the Descriptor DMA currently working on data reception. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_RINGLEN_16B==1)?16:12) - 1

11.7.29 VDMA_CH0_Tx_Desc_Write_Ring_Offset

- **Description:** This register indicates the Offset of the Tx Descriptor List, for which the DMA engine is closing the descriptor.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x3178
- **Exists:** Always

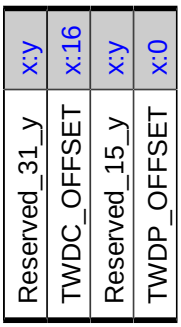


Table 11-587 Fields for Register: VDMA_CH0_Tx_Desc_Write_Ring_Offset

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((1)==0) **Range Variable[x]:** (16-((DWCXG_RINGLEN_16B==1)?16:12)) + (16+((DWCXG_RINGLEN_16B==1)?16:12)) - 1 **Range Variable[y]:** (16+((DWCXG_RINGLEN_16B==1)?16:12))
x:16	TWDC_OFFSET	R	Tx VDMA Channel0 Completed Descriptor Write Ring Offset This field indicates the ring offset of the last Tx VDMA descriptor for which a write-back request is completed. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_RINGLEN_16B==1)?16:12) + 15
x:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((1)==0) **Range Variable[x]:** (16-((DWCXG_RINGLEN_16B==1)?16:12)) + ((DWCXG_RINGLEN_16B==1)?16:12) - 1 **Range Variable[y]:** ((DWCXG_RINGLEN_16B==1)?16:12)

Table 11-587 Fields for Register: VDMA_CH0_Tx_Desc_Write_Ring_Offset (continued)

Bits	Name	Memory Access	Description
x:0	TWDP_OFFSET	R	Tx VDMA Channel0 Posted Descriptor Write Ring Offset This field indicates the ring offset of the last Tx VDMA descriptor for which a write-back request is initiated/posted. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_RINGLEN_16B==1)?16:12) - 1

11.7.30 VDMA_CH0_Rx_Desc_Write_Ring_Offset

- **Description:** This register indicates the Offset of the Rx Descriptor List, for which the DMA engine is closing the descriptor.
- **Size:** 32 bits
- **Offset:** (0x0080*i)+0x317c
- **Exists:** Always

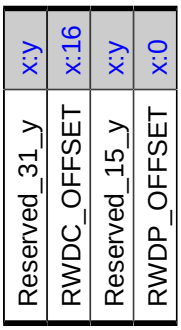


Table 11-588 Fields for Register: VDMA_CH0_Rx_Desc_Write_Ring_Offset

Bits	Name	Memory Access	Description
x:y	Reserved_31_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((1)==0) **Range Variable[x]:** (16-((DWCXG_RINGLEN_16B==1)?16:12)) + (16+((DWCXG_RINGLEN_16B==1)?16:12)) - 1 **Range Variable[y]:** (16+((DWCXG_RINGLEN_16B==1)?16:12))
x:16	RWDC_OFFSET	R	Rx VDMA Channel0 Completed Descriptor Write Ring Offset This field indicates the ring offset of the last Tx VDMA descriptor for which a write-back request is completed. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_RINGLEN_16B==1)?16:12) + 15
x:y	Reserved_15_y	R	Reserved. **Value After Reset:** 0x0 **Exists:** ((1)==0) **Range Variable[x]:** (16-((DWCXG_RINGLEN_16B==1)?16:12)) + ((DWCXG_RINGLEN_16B==1)?16:12) - 1 **Range Variable[y]:** ((DWCXG_RINGLEN_16B==1)?16:12)

Table 11-588 Fields for Register: VDMA_CH0_Rx_Desc_Write_Ring_Offset (continued)

Bits	Name	Memory Access	Description
x:0	RWDP_OFFSET	R	Rx VDMA Channel0 Descriptor Write Ring Offset This field indicates the ring offset of the descriptor DMA posted write-back request. **Value After Reset:** 0x0 **Exists:** Always **Range Variable[x]:** ((DWCXG_RINGLEN_16B==1)?16:12) - 1

12

Signal Descriptions

This chapter details all possible I/O signals in the IP. For configurable IP titles, your actual configuration might not contain all of these signals.

Inputs are on the left of the signal diagrams; outputs are on the right.

Attention: For configurable IP titles, do not use this document to determine the exact I/O footprint of the controller. It is for reference purposes only.

When you configure the controller in coreConsultant, you must access the I/O signals for your actual configuration at workspace/report/IO.html or workspace/report/IO.xml after you have completed the report creation activity. That report comes from the exact same source as this chapter but removes all the I/O signals that are not in your actual configuration. This does not apply to non-configurable IP titles. In addition, all parameter expressions are evaluated to actual values. Therefore, the widths might change depending on your actual configuration.

Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

In addition to describing the function of each signal, the signal descriptions in this chapter include the following information:

- **Active State:** Indicates whether the signal is active high or active low. When a signal is not intended to be used in a particular application, then this signal needs to be tied or driven to the inactive state (opposite of the active state).
- **Registered:** Indicates whether or not the signal is registered directly inside the IP boundary without intervening logic (excluding simple buffers). A value of *No* does not imply that the signal is not synchronous, only that there is some combinatorial logic between the signal's origin or destination register and the boundary of the controller. A value of N/A indicates that this information is not provided for this IP title.
- **Synchronous to:** Indicates which clocks in the IP sample this input (drive for an output). This clock might not be the same as the clock that your application logic should use to clock (sample/drive) this pin. For more details, consult the clock section in the databook. The presence of the postfix SuperList indicates a list of all possible clocks over all possible configs. Consult coreConsultant report for which clock applies to your specific configuration.

- **Exists:** Name of configuration parameter that populates this signal in your configuration.
- **Power Domain:** Name of power/voltage domain that this signal is part of when power/voltage islands are used. The SINGLE_DOMAIN value indicates that there are no islands.

The I/O signals are grouped as follows:

- "Clock and Reset Signals Signals" on page 1632
- "XGMII Clock and Reset Signals Signals" on page 1638
- "IEEE 1588 Timestamp Signals Signals" on page 1641
- "Power Management Signals Signals" on page 1652
- "AXI Manager Interface Signals Signals" on page 1655
- "RGMII Clock and Reset Signals Signals" on page 1672
- "Test Mode Signals Signals" on page 1673
- "Sideband Interface Signals Signals" on page 1674
- "External Lookup Interface Signals Signals" on page 1683
- "L2 Virtualized Network Over L3 Network Interface Signals Signals" on page 1687
- "G999_1 Signals Signals" on page 1689
- "GCL Memory Interface Signals Signals" on page 1690
- "AXI Subordinate Interface Signals Signals" on page 1693
- "AXI Low Power Interface Signals Signals" on page 1708
- "TSO Memory Interface (TMI) Signals Signals" on page 1709
- "Descriptor Cache Memory Interface (DMI) Signals Signals" on page 1712
- "HDMA CSR Memory Interface (CMI) Signals Signals" on page 1715
- "Application Transmit Interface (ATI) Signals Signals" on page 1718
- "Application Receive Interface (ARI) Signals Signals" on page 1726
- "Two Port SRAM Interface Signals Signals" on page 1732
- "Single Port SRAM Interface Signals Signals" on page 1738
- "MAC Tx Interface Signals Signals" on page 1747
- "Debug Interface Signals Signals" on page 1756
- "MAC Rx Interface Signals Signals" on page 1758
- "Energy Efficient Ethernet Signals Signals" on page 1761
- "General Purpose Signals Signals" on page 1762
- "APB3 Interface Signals Signals" on page 1763
- "APB Interface Signals Signals" on page 1766
- "APB4 Interface Signals Signals" on page 1767
- "MAC Control Interface Signals Signals" on page 1768
- "MDIO Interface Signals Signals" on page 1771

- "Operation Speed Signals Signals" on page 1773
- "Loopback Mode Signals Signals" on page 1774
- "GMII Interface Signals Signals" on page 1775
- "RGMII Interface Signals Signals" on page 1778
- "XGMII Interface Signals Signals" on page 1781
- "RXF Memory Interface Signals Signals" on page 1785
- "Tx Timestamp Status FIFO Streaming Interface Signals Signals" on page 1788
- "SSM Bus Interface Signals Signals" on page 1789
- "Filtering/Camera over Ethernet Offload Signals Signals" on page 1794

12.1 Clock and Reset Signals

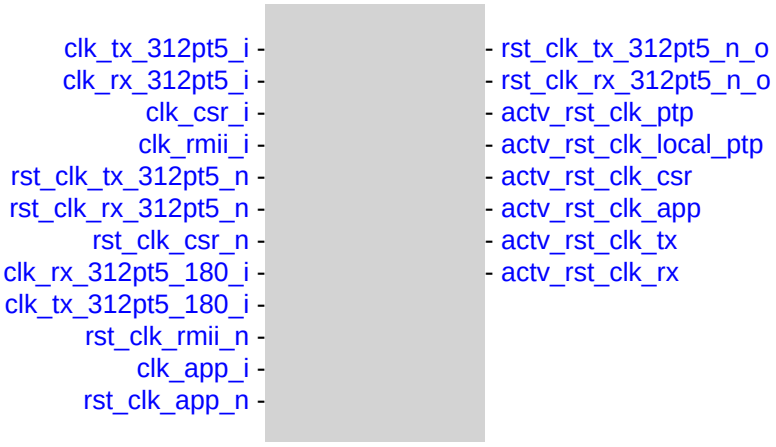


Table 12-1 Clock and Reset Signals

Port Name	I/O	Description
clk_tx_312pt5_i	I	This is the transmission clock provided by the external oscillator. All DWC_25gmac transmit modules operate synchronous to this clock. ■ Speed Interface Data width Interface Programming MAC Tx Configuration Clock Frequency {SS[2:0]} {mac_speed_o[2:0]} ■ 10Gbps XGMII 32-bit 312.5 MHz 3'b000 ■ 25Gbps XGMII 32-bit 781.25 MHz 3'b001 ■ 5Gbps XGMII 32-bit 156.25 MHz 3'b101 ■ 2.5Gbps XGMII 32-bit 78.125 MHz 3'b110 ■ 1Gbps GMII 8-bit 125 MHz 3'b011 ■ 2.5Gbps GMII 8-bit 312.5MHz 3'b010 ■ 100Mbps MII 4-bit 25 MHz 3'b100 ■ 10Mbps MII 4-bit 2.5 MHz 3'b111 **Exists:** Always **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A

Table 12-1 Clock and Reset Signals (continued)

Port Name	I/O	Description
clk_rx_312pt5_i	I	This is the reception clock (312.5 MHz in 10/2.5 Gbps or 125 MHz in 1 Gbps) provided by the external clock source. All DWC_25gmac receive modules operate synchronous to this clock. Speed Interface Data Width Interface Programming MAC Tx Configuration Clock Frequency {SS[2:0]} {mac_speed_o[2:0]} ■ 10Gbps XGMII 32-bit 312.5 MHz 3'b000 ■ 25Gbps XGMII 32-bit 781.25 MHz 3'b001 ■ 5Gbps XGMII 32-bit 156.25 MHz 3'b101 ■ 2.5Gbps XGMII 32-bit 78.125 MHz 3'b110 ■ 1Gbps GMII 8-bit 125 MHz 3'b011 ■ 2.5Gbps GMII 8-bit 312.5MHz 3'b010 ■ 100Mbps MII 4-bit 25 MHz 3'b100 ■ 10Mbps MII 4-bit 2.5 MHz 3'b111 **Exists:** Always **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
clk_csr_i	I	This is a free-running clock provided by the application. This clock is required in the XGMAC-CORE configuration, but is optional in the XGMAC-MTL and XGMAC-AXI configurations. The DWC_25gmac MAC control interface, CSR, station management agent, and MAC management counters run on this clock. This clock's minimum frequency is 87 MHz when Enable DWC_25gmac Management Counter (MMC) feature is selected in configuration, frequencies less than this might result in incorrect operation. **Exists:** DWCXG_CORE||DWCXG_CSR_SLV_CLK||SNPS_RSVDPARAM_24 **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
clk_rmii_i	I	RMII Clock. This is the 50-MHz clock used by the RMII interface. When the RMII interface is selected, the clk_rx_i clock must be 25 or 2.5 MHz and it must be derived from this clock. **Exists:** DWCXG_RMII_EN **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A

Table 12-1 Clock and Reset Signals (continued)

Port Name	I/O	Description
rst_clk_tx_312pt5_n	I	This is the active-low reset signal synchronous to clk_tx_312pt5_i. **Exists:** !(DWCXG == 2) **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_tx_3125pt5_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rst_clk_rx_312pt5_n	I	This is the active-low reset signal synchronous to clk_rx_312pt5_i. **Exists:** !(DWCXG == 2) **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_rx_312pt5_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** Low
rst_clk_csr_n	I	This is the active-low reset signal synchronous to clk_csr_i. All registers and counters inside the CSR go to their default state when rst_clk_csr_n is asserted. **Exists:** DWCXG_CORE||(DWCXG_CSR_SLV_CLK&&!DWCXG_AXI_SUBSYS) **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_csr_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rst_clk_tx_312pt5_n_o	O	Transmit Clock Reset Output. Reset signal. This output is available only in XGMAC-AXI configuration. This signal is generated synchronous to the clk_tx_312pt5_i clock in the following conditions: ■ Any change in the pwr_on_rst_n signal ■ For software reset **Exists:** DWCXG == 2 **Synchronous To:** clk_tx_312pt5_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** Low

Table 12-1 Clock and Reset Signals (continued)

Port Name	I/O	Description
rst_clk_rx_312pt5_n_o	O	Receive Clock Reset Output. Reset signal. This output is available only in XGMAC-AXI configuration. This signal is generated synchronous to the clk_rx_312pt5_i clock in the following conditions: ■ Any change in the pwr_on_rst_n signal ■ For software reset **Exists:** DWCXG == 2 **Synchronous To:** clk_rx_312pt5_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** Low
actv_rst_clk_ptp	O	Indicates rst_clk_ptp_ref_n is active; asserted 1 clock cycle earlier than rst_clk_ptp_ref_n assertion and de-asserted with rst_clk_ptp_ref_n. **Exists:** (DWCXG_TIME_STAMPING)&&(DWCXG_AXI_SUBSYS&&DWCXG_ASYNC_RST) **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
actv_rst_clk_local_ptp	O	Indicates rst_clk_local_ptp_ref_n is active; asserted 1 clock cycle earlier than rst_clk_local_ptp_ref_n assertion and de-asserted with rst_clk_local_ptp_ref_n. **Exists:** (DWCXG_DSTS)&&(DWCXG_AXI_SUBSYS&&DWCXG_ASYNC_RST) **Synchronous To:** clk_local_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
actv_rst_clk_csr	O	Indicates rst_clk_csr_n is active; asserted 1 clock cycle earlier than rst_clk_csr_n assertion and de-asserted with rst_clk_csr_n. **Exists:** (DWCXG_AXI_SUBSYS&&DWCXG_ASYNC_RST) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-1 Clock and Reset Signals (continued)

Port Name	I/O	Description
actv_rst_clk_app	O	Indicates rst_clk_app_n is active; asserted 1 clock cycle earlier than rst_clk_app_n assertion and de-asserted with rst_clk_app_n. **Exists:** (DWCXG_AXI_SUBSYS&&DWCXG_ASYNC_RST) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
actv_rst_clk_tx	O	Indicates rst_clk_tx_312pt5_n is active; asserted 1 clock cycle earlier than rst_clk_tx_312pt5_n assertion and de-asserted w with rst_clk_tx_312pt5_n. **Exists:** (DWCXG_AXI_SUBSYS&&DWCXG_ASYNC_RST) **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
actv_rst_clk_rx	O	Indicates rst_clk_rx_312pt5_n is active; asserted 1 clock cycle earlier than rst_clk_rx_312pt5_n assertion and de-asserted with rst_clk_rx_312pt5_n. **Exists:** (DWCXG_AXI_SUBSYS&&DWCXG_ASYNC_RST) **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
clk_rx_312pt5_180_i	I	This is the 180 degree shifted version of Rx(clk_rx_312pt5_i) clock (125 MHz in 1 Gbps or 25 MHz in 100 Mbps or 2.5MHz in 10 Mbps) provided by the external oscillator. The RGMII Rx modules work synchronous to this clock. **Exists:** DWCXG_RGMII_EN **Synchronous To:** clk_rx_312pt5_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A

Table 12-1 Clock and Reset Signals (continued)

Port Name	I/O	Description
clk_tx_312pt5_180_i	I	This is the 180 degree shifted version of Tx (clk_tx_312pt5_i) clock (125 MHz in 1 Gbps or 25 MHz in 100 Mbps or 2.5MHz in 10 Mbps) provided by the external oscillator. The RGMII Tx modules work synchronous to this clock. **Exists:** DWCXG_RGMII_EN **Synchronous To:** clk_tx_312pt5_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
rst_clk_rmii_n	I	RMII Clock Reset. This is a reset signal synchronous to the clk_rmii_i. **Exists:** DWCXG_RMII_EN&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "None" : "clk_rmii_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** Low
clk_app_i	I	This is a free-running clock provided by the application. In the XGMAC-MTL configurations, the ATI and ARI operate synchronous to this clock. The valid frequency range of this clock is ■ 200-400 MHz: In 64-bit non Synchronous two-port RAM configurations. ■ 100 MHz: Minimum frequency in 128-bit non Synchronous two-port RAM configurations, For more details about the Synchronous Two-Port RAM, see "Synchronous Two-Port RAM Feature" section in DWC_25gmac Databook. **Exists:** DWCXG == 1 **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rst_clk_app_n	I	This is the active-low reset signal synchronous to the application clock. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low

12.2 XGMII Clock and Reset Signals

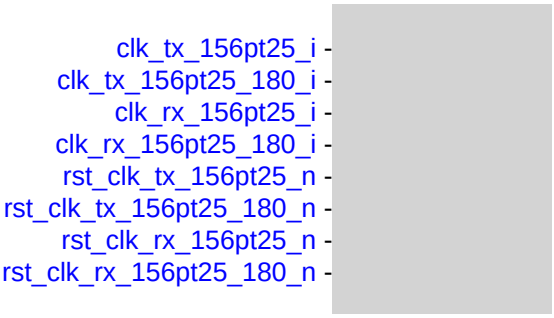


Table 12-2 XGMII Clock and Reset Signals

Port Name	I/O	Description
clk_tx_156pt25_i	I	This is the 156.25-MHz/78.125-MHz/39.065-MHz clock input from an external clock source in 10Gbps XGMII, 5Gbps XGMII, 2.5Gbps XGMII mode respectively. This clock is required only when the XGMII interface is configured for DDR or DDW mode. The clk_tx_312pt5_i clock must run synchronous to this clock. All XGMII transmit signals are synchronous to both the positive and negative edges of this clock. When you configure DWC_25gmac for a multiplexed GMII/XGMII interface (XGMII in DDR or DDW mode) and operating in GMII/MII mode, this input must be driven with the 312.5-MHz clock (for 2.5G mode) or the 125-MHz clock (for 1G mode) or the 25-MHz clock (for 100Mbps mode) or the 2.5-MHz clock (for 10Mbps mode), synchronous to clk_tx_312pt5_i. **Exists:** (!(DWCXG_XGMII == 0)) **Synchronous To:** clk_tx_312pt5_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
clk_tx_156pt25_180_i	I	This is the inverse of clk_tx_156pt25_i, required only when the XGMII is configured in DDR mode. The XGMII transmitter uses the rising edge of clk_tx_156pt25_180_i to represent the falling edge of clk_tx_156pt25_i. This input need not be driven when the DWC_25gmac is configured for a multiplexed GMII/XGMII interface (XGMII in DDR mode) and operating in GMII mode. **Exists:** DWCXG_XGMII == 1 **Synchronous To:** clk_tx_156pt25_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-2 XGMII Clock and Reset Signals (continued)

Port Name	I/O	Description
clk_rx_156pt25_i	I	This is the 156.25-MHz/78.125-MHz/39.065-MHz clock input for the 10Gbps XGMII, 5Gbps XGMII, 2.5Gbps XGMII mode respectively. This clock is only required when the XGMII Receive interface is configured for DDR or DDW mode. clk_rx_312pt5_i must run synchronous to this clock. All XGMII receive signals are synchronous to both the positive and negative edges of this clock. This input need not be driven when the DWC_25gmac is configured for a multiplexed GMII/XGMII interface (XGMII in DDR or DDW mode) and operating in GMII mode. **Exists:** (!(DWCXG_XGMII == 0)) **Synchronous To:** clk_rx_312pt5_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
clk_rx_156pt25_180_i	I	This is the inverse of clk_rx_156pt25_i, required only when the XGMII is configured in DDR mode. The XGMII receiver uses the rising edge of clk_rx_156pt25_180_i to represent the falling edge of clk_rx_156pt25_i. This input need not be driven when the DWC_25gmac is configured for a multiplexed GMII/XGMII interface (XGMII in DDR or DDW mode) and operating in GMII mode. **Exists:** DWCXG_XGMII == 1 **Synchronous To:** clk_rx_156pt25_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
rst_clk_tx_156pt25_n	I	This is the active-low reset signal synchronous to clk_tx_156pt25_i. **Exists:** !DWCXG_XGMII_SDR&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_tx_156pt25_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low

Table 12-2 XGMII Clock and Reset Signals (continued)

Port Name	I/O	Description
rst_clk_tx_156pt25_180_n	I	This is the active-low reset signal synchronous to clk_tx_156pt25_180_i. **Exists:** DWCXG_XGMII_DDR&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_tx_156pt25_180_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rst_clk_rx_156pt25_n	I	This is the active-low reset signal synchronous to clk_rx_156pt25_i. **Exists:** !DWCXG_XGMII_SDR&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_rx_156pt25_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** Low
rst_clk_rx_156pt25_180_n	I	This is the active-low reset signal synchronous to clk_rx_156pt25_180_i. **Exists:** DWCXG_XGMII_DDR&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_rx_156pt25_180_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** Low

12.3 IEEE 1588 Timestamp Signals

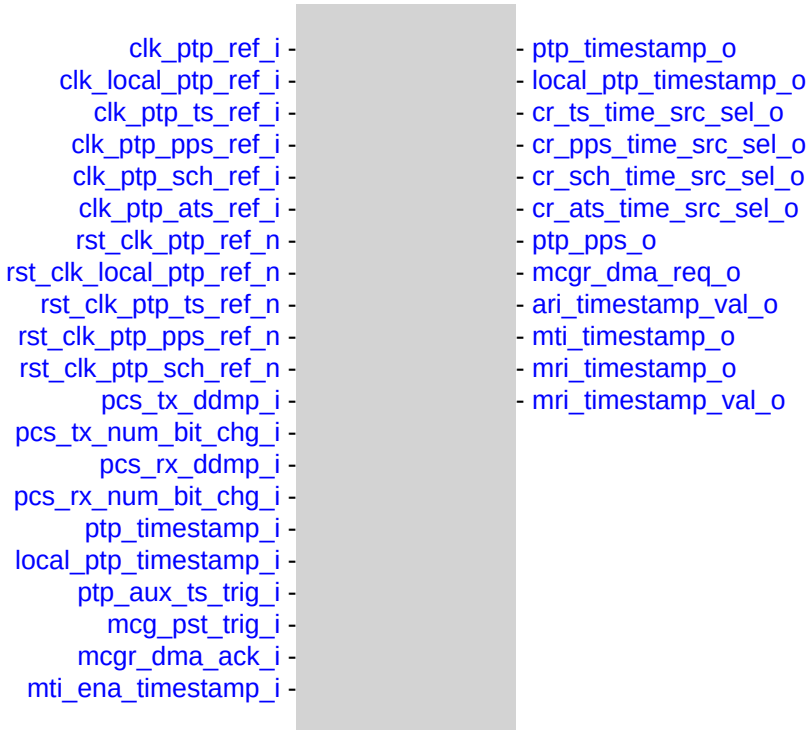


Table 12-3 IEEE 1588 Timestamp Signals

Port Name	I/O	Description
clk_ptp_ref_i	I	Reference Clock for the Timestamp Update Logic. This reference clock, provided by an external oscillator or a timing source, is used for the timestamp logic. The minimum clock frequency supported is provided in the table "Minimum PTP Clock Frequency Example". The maximum clock frequency depends on the frequency at which logic timing is met with the technology library used. **Exists:** DWCXG_TIME_STAMPING **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
clk_local_ptp_ref_i	I	Local Reference Clock for the Timestamp Update Logic. This reference clock, provided by an external oscillator or a timing source, is used for the timestamp logic. The minimum clock frequency supported is provided in the table "Minimum PTP Clock Frequency Example". The maximum clock frequency depends on the frequency at which logic timing is met with the technology library used. **Exists:** DWCXG_DSTS **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
clk_ptp_ts_ref_i	I	Reference Clock for the Timestamping Capturing Logic. This reference clock, provided by an external oscillator or a timing source, is used for the timestamp logic. The minimum clock frequency supported is provided in the table "Minimum PTP Clock Frequency Example". The maximum clock frequency depends on the frequency at which logic timing is met with the technology library used. **Exists:** DWCXG_DSTS **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
clk_ptp_pps_ref_i[(DWCXG_PPS_OUT_NUM-1):0]	I	Reference Clock for the Flexible PPS Logic. This reference clock, provided by an external oscillator or a timing source, is used for the timestamp logic. The minimum clock frequency supported is provided in the table "Minimum PTP Clock Frequency Example". The maximum clock frequency depends on the frequency at which logic timing is met with the technology library used. **Exists:** DWCXG_DSTS&&DWCXG_FLEXI_PPS_OUT_EN **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
clk_ptp_sch_ref_i	I	Reference Clock for the Scheduler Timestamp Logic. This reference clock, provided by an external oscillator or a timing source, is used for the timestamp logic. The minimum clock frequency supported is provided in the table "Minimum PTP Clock Frequency Example". The maximum clock frequency depends on the frequency at which logic timing is met with the technology library used. **Exists:** DWCXG_DSTS&&((DWCXG_TX_AV_ENABLE&&!DWCXG_CORE)||DWCXG_AV_EST||DWCXG_TBS||DWCXG_SGF) **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
clk_ptp_ats_ref_i	I	Reference Clock for the Scheduler Timestamp Logic. This reference clock, provided by an external oscillator or a timing source, is used for the timestamp logic. The minimum clock frequency supported is provided in the table "Minimum PTP Clock Frequency Example". The maximum clock frequency depends on the frequency at which logic timing is met with the technology library used. **Exists:** DWCXG_DSTS&&DWCXG_ATS **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
rst_clk_ptp_ref_n	I	Timestamp Logic Reset. Reset signal. This input is not present in the XGMAC-AXI configuration. **Exists:** DWCXG_TIME_STAMPING&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_ptp_ref_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
rst_clk_local_ptp_ref_n	I	Timestamp Logic Reset. Reset signal. This input is not present in the XGMAC-AXI configuration. **Exists:** DWCXG_DSTS&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_local_ptp_ref_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rst_clk_ptp_ts_ref_n	I	Timestamp Capture Logic Reset. Reset signal. This input is not present in the XGMAC-AXI configuration. **Exists:** DWCXG_DSTS&&!DWCXG_AXI_SUBSYS&&DWCXG_TIME_STAMPING **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_ptp_ref_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rst_clk_ptp_pps_ref_n[(DWCXG_PPS_OUT_NUM-1):0]	I	Flexible PPS Logic Reset. Reset signal. This input is not present in the XGMAC-AXI configuration. **Exists:** DWCXG_DSTS&&!DWCXG_AXI_SUBSYS&&DWCXG_FLEXI_PPS_OUT_EN **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_ptp_ref_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rst_clk_ptp_sch_ref_n	I	Scheduler Timestamp Logic Reset. Reset signal. This input is not present in the XGMAC-AXI configuration. **Exists:** DWCXG_DSTS&&!DWCXG_AXI_SUBSYS&&((DWCXG_TX_AV_ENABLE&&!DWCXG_CORE)||DWCXG_AV_EST||DWCXG_TBS||DWCXG_SGF) **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "Asynchronous" : "clk_ptp_ref_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
pcs_tx_ddmp_i	I	This signal acts as a valid signal for the PCS egress latency value in terms of bits put onto pcs_tx_num_bit_chg_i by the PCS layer. **Exists:** DWCXG_ETA **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
pcs_tx_num_bit_chg_i[15:0]	I	This signal indicates the Egress Latency value in terms of bit times driven by the PCS layer. The signal is sampled only when pcs_tx_ddmp_i is set. **Exists:** DWCXG_ETA **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
pcs_rx_ddmp_i	I	This signal acts as a valid signal for the PCS ingress latency value in terms of bits put onto pcs_rx_num_bit_chg_i by the PCS layer. **Exists:** DWCXG_ETA **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
pcs_rx_num_bit_chg_i[15:0]	I	This signal indicates the Ingress Latency value in terms of bit times driven by the PCS layer. The signal is sampled only when pcs_rx_ddmp_i is set. **Exists:** DWCXG_ETA **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
ptp_timestamp_i[63:0]	I	64-Bit Timestamp Input. This bus provides the system time used to timestamp the specific transmit packets or received packets. This signal is available only when the parameter DWCXG_SYSTIME_SOURCE = External or Both options. **Exists:** DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=0 **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
local_ptp_timestamp_i[63:0]	I	64-Bit Timestamp Input. This bus provides the system time used to timestamp the specific transmit packets or received packets. This signal is available only when the parameter DWCXG_SYSTIME_SOURCE = External or Both options. **Exists:** DWCXG_DSTS&&DWCXG_SYSTIME_SOURCE!=0 **Synchronous To:** clk_local_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ptp_timestamp_o[63:0]	O	64-Bit Reference Timestamp Output. This bus provides the system timestamp output when the timestamp is generated internally. This signal is available only when the parameter DWCXG_SYSTIME_SOURCE = Internal or Both options. **Exists:** DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1 **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
local_ptp_timestamp_o[63:0]	O	64-Bit Reference Timestamp Output. This bus provides the system timestamp output when the timestamp is generated internally. This signal is available only when the parameter DWCXG_SYSTIME_SOURCE = Internal or Both options. **Exists:** DWCXG_DSTS&&DWCXG_SYSTIME_SOURCE!=1 **Synchronous To:** clk_local_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
cr_ts_time_src_sel_o	O	Timestamping Time Source Select. This signal is high when Free Running Time Source is selected for One-Step and Two-Step Timestamping functions, otherwise Adjustable/Correctable Time Source is selected. **Exists:** DWCXG_DSTS **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
cr_pps_time_src_sel_o[(DWCXG_PPS_OUT_NUM-1):0]	O	PPS Time Source Select. This signal is high when Free Running Time Source is selected for PPS functions, otherwise Adjustable/Correctable Time Source is selected. **Exists:** DWCXG_DSTS&&DWCXG_FLEXI_PPS_OUT_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
cr_sch_time_src_sel_o	O	Time-Based Scheduler Time Source Select. This signal is high when Free Running Time Source is selected for Time-Based Scheduler functions, otherwise Adjustable/Correctable Time Source is selected. **Exists:** DWCXG_DSTS&&((DWCXG_TX_AV_ENABLE&&!DWCXG_CORE)||DWCXG_AV_EST||DWCXG_TBS||DWCXG_SGF) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
cr_ats_time_src_sel_o	O	ATS Time Source Select. This signal is high when Free Running Time Source is selected for ATS functions. **Exists:** DWCXG_DSTS&&DWCXG_ATS **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ptp_pps_o[(DWCXG_PPSOW-1):0]	O	Pulse Per Second. This signal is high based on the PPS mode selected in the MAC_PPS_Control register. This pulse signal is high every time the Seconds counter is incremented. This signal is available only when the parameter DWCXG_SYSTIME_SOURCE = Internal or Both options. **Exists:** DWCXG_TIME_STAMPING&&DWCXG_SYSTIME_SOURCE!=1 **Synchronous To:** clk_ptp_ref_i **Registered:** (DWCXG_AXI_SUBSYS==1 && DWCXG_ASYNC_RST==1)? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
ptp_aux_ts_trig_i[(DWCXG_AUX_SNAP_IN_NUM-1):0]	I	Auxiliary Timestamp Trigger. This signal goes high to take an auxiliary snapshot of the time and store it in the auxiliary timestamp FIFO. A rising edge of this port is used to trigger the auxiliary snapshot. **Exists:** DWCXG_ADV_TIME_AUX_SNAP **Synchronous To:** Asynchronous **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mcg_pst_trig_i[(DWCXG_MCGR_NUM-1):0]	I	Media Clock Generation Trigger Input Trigger Input to the DUT to capture presentation time. Based on presentation control value of MAC_PPS_Control register, this signal can be pulse or level signal. Width of this IO changes with the configured number of PPS instances. **Exists:** DWCXG_FLEXI_PPS_OUT_EN&&DWCXG_TIME_STAMPING==1&&DWCXG_SYSTIME_SOURCE!=1 **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
mcgr_dma_req_o[(DWCXG_MCGR_NUM-1):0]	O	MCGR DMA Read Request Used to Request a Target Presentation time value for one of the comparator modules when the instance is running in recovery mode. In Media Clock generation mode, this I/O signal is held high indicating a valid Presentation time is captured and updated in MAC_PPS(#i)_Target_Time_Seconds register. One or more DMA read requests can simultaneously active, it is up to the DMA logic to maintain priority. Bits have a 1-1 correspondence to the comparator instances (i.e. Bit-0 belongs to comparator 0, bit-1 belongs to comparator-1 and so on). Once set, will hold high and will be reset the cycle after receiving the mcgr_dma_ack for that comparator. Width of this IO changes with the configured number of PPS instances. **Exists:** DWCXG_FLEXI_PPS_OUT_EN&&DWCXG_TIME_STAMPING==1&&DWCXG_SYSTIME_SOURCE!=1 **Synchronous To:** clk_ptp_ref_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
mcgr_dma_ack_i[(DWCXG_MCGR_NUM -1):0]	I	MCGR DMA Ack This port indicate that the request indicated by mcgr_dma_req_o is accepted by the application . Should be an active high pulse for one cycle. Width of this IO changes with the configured number of PPS instances. **Exists:** DWCXG_FLEXI_PPS_OUT_EN&&DWCXG_TIME_STAMPING==1&& DWCXG_SYSTIME_SOURCE!=1 **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
ari_timestamp_val_o	O	Receive Timestamp Valid. This signal indicates that a valid timestamp value is available on the ARI data bus (ari_data_o). This signal is available only in the XGMAC-MTL configuration. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_TIME_STAMPING **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_ena_timestamp_i	I	Transmit Enable Timestamping. This signal directs the MAC to capture the timestamp for the Transmit packet. This signal is available only in the XGMAC-CORE configuration. The application must drive this signal high only for those PTP packets that require timestamping. This signal is sampled only when mti_sop_i and mti_val_i signals are high. **Exists:** DWCXG_TIME_STAMPING&&DWCXG_CORE **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-3 IEEE 1588 Timestamp Signals (continued)

Port Name	I/O	Description
mti_timestamp_o[63:0]	O	Transmit Packet Timestamping. This bus provides the timestamp of the packet transmitted by the MAC. This bus is available only in the XGMAC-CORE configuration. This bus is valid only when the mti_txstatus_val_o signal is high. **Exists:** DWCXG_CORE&&DWCXG_TIME_STAMPING **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mri_timestamp_o[63:0]	O	Receive Packet Timestamping. This bus provides the timestamp of received packet. This bus is available only in the XGMAC-CORE configuration. This bus is valid only when the mri_eop_o signal is high. **Exists:** DWCXG_TIME_STAMPING&&DWCXG_CORE **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mri_timestamp_val_o	O	Receive Timestamp Valid. This signal validates the presence of timestamp for the Receive packet. This signal is available only in the XGMAC-CORE configuration. **Exists:** DWCXG_TIME_STAMPING&&DWCXG_CORE **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.4 Power Management Signals

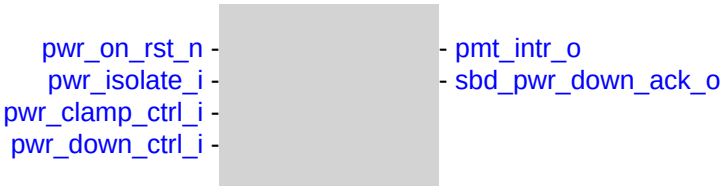


Table 12-4 Power Management Signals

Port Name	I/O	Description
pwr_on_rst_n	I	Application Clock Reset. This is a reset signal synchronous to the application clock domain in the XGMAC-DMA configuration. When you select the PMT feature, this is an asynchronous reset signal. **Exists:** DWCXG_PMT_EN&&!DWCXG_AXI_SUBSYS **Synchronous To:** Asynchronous **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
pmt_intr_o	O	PMT Interrupt. This signal indicates an interrupt event in the optional PMT module of DWC_25gmac. This interrupt pin is always present when the Enable Power Management option is selected. You can use it to wake the system from the Sleep mode. This interrupt has no masking control. **Exists:** DWCXG_PMT_EN **Synchronous To:** clk_rx_312pt5_i **Registered:** (DWCXG_PMT_MGK_EN==1 && DWCXG_PMT_RWK_EN==1) ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
sbd_pwr_down_ack_o	O	Power Down Acknowledge. This signal is acknowledgement to start the power-down sequence. **Exists:** DWCXG_PMT_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

Table 12-4 Power Management Signals (continued)

Port Name	I/O	Description
pwr_isolate_i	I	Power Isolate Control. This signal is the control input for clamping the isolation cells. **Exists:** DWCXG_PMT_EN **Synchronous To:** Asynchronous **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
pwr_clamp_ctrl_i	I	Power Clamp Control. This signal is the control input for clamping the reset signals to the always-on logic. **Exists:** DWCXG_PMT_EN **Synchronous To:** Asynchronous **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
pwr_down_ctrl_i	I	Power Down Control. This signal controls the VDD power down. **Exists:** DWCXG_PMT_EN **Synchronous To:** Asynchronous **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

12.5 AXI Manager Interface Signals

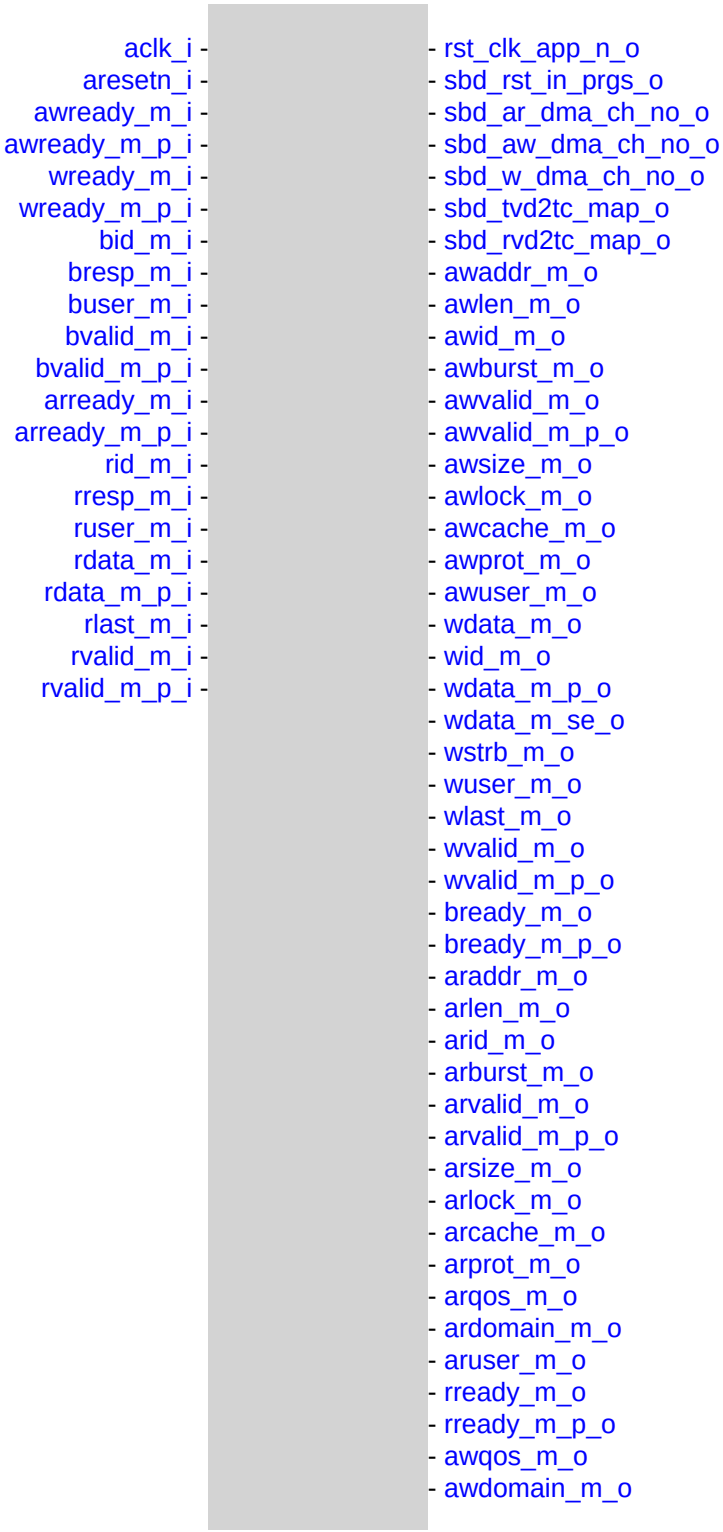


Table 12-5 AXI Manager Interface Signals

Port Name	I/O	Description
rst_clk_app_n_o	O	Application Clock Reset Output. Reset signal. This signal is available only in XGMAC-AXI configuration. This signal is generated synchronous to the application clock domain in any of the following conditions: ■ Any change in the pwr_on_rst_n signal ■ software reset **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
sbd_rst_in_prgs_o	O	Reset in progress (output). This signal is available only in the XGMAC-AXI configuration. This signal is generated synchronous to the CSR clock domain when reset is in progress. **Exists:** DWCXG == 2 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_ar_dma_ch_no_o[(DWCXG_AR_DMA_CHW-1):0]	O	AXI Manager Read Address DMA channel number. This is the DMA channel number sideband signal of read address channel. **Exists:** DWCXG_AXI_SUBSYS&&DWCXG_DMACHN **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sbd_aw_dma_ch_no_o[(DWCXG_AW_DMA_CHW-1):0]	O	AXI Manager Write Address DMA channel number. This is the DMA channel number sideband signal of write address channel. **Exists:** DWCXG_AXI_SUBSYS&&DWCXG_DMACHN **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
sbd_w_dma_ch_no_o[(DWCXG_W_DMA_CHW-1):0]	O	AXI Manager Write Data DMA channel number. This is the DMA channel number sideband signal of write data channel. **Exists:** DWCXG_AXI_SUBSYS&&DWCXG_DMACHN **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sbd_tvd2tc_map_o[((DWCXG_NUM_DMA_TX_CH*DWCXG_TCW)-1):0]	O	Tx VDMA to TC Map. This bus carries the TC number to which each Tx VDMA is mapped to **Exists:** SNPS_RSVDPARAM_50 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sbd_rvd2tc_map_o[((DWCXG_NUM_DMA_RX_CH*DWCXG_TCW)-1):0]	O	Rx VDMA to TC Map. This bus carries the TC number to which each Rx VDMA is mapped to **Exists:** SNPS_RSVDPARAM_50 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
aclk_i	I	AMBA AXI System Clock. This is the free-running AXI clock input provided by the application. The DMA and the MTL modules also operate with this clock. **Exists:** DWCXG == 2 **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
aresetn_i	I	AMBA AXI Power-on System Reset. This is the AMBA AXI power system reset signal. **Exists:** DWCXG == 2 **Synchronous To:** Asynchronous **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** Low

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
awaddr_m_o[(DWCXG_ADDRWIDTH-1):0]	O	Manager Write Address. The Write address bus gives the address of the first transfer in a Write burst transaction. The associated control signals are used to determine the addresses of the remaining transfers in the burst. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awlen_m_o[(DWCXG_BLW-1):0]	O	Manager Burst Length. This signal gives the exact number of transfers in a burst. This information determines the number of data transfers associated with the address. The default width is 4 to support 16-beats transfer but it can be configured to up to 8-bit to support 256 beats transfers. The value of all-zeros indicates a single transfer. The minimum width of awlen_m_o signal is 4 (DWC_XGMAC_AXI_BL = 16) and the maximum is 8 (DWC_XGMAC_AXI_BL = 256). **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awid_m_o[(DWCXG_AXI_IDW-1):0]	O	Manager Write Address ID. This signal is the identification tag for Write address group of signals. You can configure the width while configuring the controller. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awburst_m_o[1:0]	O	Manager Burst Type. The burst type, along with the size information, specifies how the address for each transfer within the burst is calculated. This is always hard wired to 2'b01 to indicate INCR burst type. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
awvalid_m_o	O	Manager Write Address Valid. This signal indicates that valid write address and control information are available: ■ 1: Address and control information available ■ 0: Address and control information not available. The address and control information remain stable until the address acknowledge signal awready_m_i goes high. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** DWCXG_AIS_EN ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
awvalid_m_p_o	O	Parity for Manager Write Address Valid. The address and control information remain stable until the address acknowledge signal awready_m_i goes high. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
awready_m_i	I	Manager Write Address Ready. This signal indicates that the AXI subordinate is ready to accept an address and associated control signals: ■ 1: Subordinate ready ■ 0: Subordinate not ready **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
awready_m_p_i	I	Parity for Manager Write Address Ready. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
awsize_m_o[2:0]	O	Manager Burst Length. This signal indicates the size of each transfer in the burst. Byte lane strobes indicate exactly which byte lanes to update. This is always driven to have the System Data Width transfers:. ■ 3'b011 for 64-bit ■ 3'b100 for 128-bit interface **Exists:** DWCXG == 2 **Synchronous To:** (DWCXG_ASP_PPE==2) ? "aclk_i" : "None" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awlock_m_o	O	Locked Access Indicator for Write Address Channel of AXI Manager. This signal indicates the type of locked access to the Write address channel of the AXI Manager. **Exists:** DWCXG == 2 **Synchronous To:** (DWCXG_ASP_PPE==2) ? "aclk_i" : "None" **Registered:** DWCXG_AIS_EN ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
awcache_m_o[3:0]	O	Manager Cache Type. This signal is used in AXI4 ACE-compliant systems. The value driven by this signal is controlled by programming the AXI_AWCache_control register. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awprot_m_o[2:0]	O	Manager Protection Type. This signal indicates the normal, privileged, or secure protection level of the transaction and whether the transaction is a data access or an instruction access. This is always hard wired to 3'b000 to indicate normal and secure data access. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
awuser_m_o[(DWCXG_M_AW_PYLD_USER_WIDTH-1):0]	O	AXI manager address read user signal. This signal gives parity for control word 1 and control word 0. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
wdata_m_o[(DWCXG_DATW-1):0]	O	Manager Write Data. The write data bus can be 32-bit, 64-bit, or 128-bit wide and takes the value of DWC_XGMAC_DATAWIDTH parameter. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** DWCXG_AIS_EN ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
wid_m_o[(DWCXG_AXI_IDW-1):0]	O	Manager Write ID Tag. This signal is the ID tag of the Write data transfer. The WID value must match the AWID value of the write transaction. Do not use this signal in AXI4-compliant systems. **Exists:** DWCXG_AXI_SUBSYS&&!DWCXG_AXI4_SUBSYS **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
wdata_m_p_o[(DWCXG_DPW-1):0]	O	AXI Manager Write Data Parity. The parity data bus width can be 4-bit, 8-bit, or 16-bit wide and takes the value of DWCXG_DPW parameter. **Exists:** DWCXG_AXI_SUBSYS&&(DWCXG_ASP_PPE==1) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
wdata_m_se_o	O	AXI Manager Write Data uncorrectable safety error. It indicates uncorrectable safety error on wdata_m_o. **Exists:** DWCXG_AXI_SUBSYS&&DWCXG_ASP_ALL&&SNPS_RSVDPARAM_33 **Synchronous To:** aclk_i **Registered:** (DWCXG_AIS_EN&&DWCXG_AISM_W_RS_EN) ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
wstrb_m_o[(DWCXG_STRW-1):0]	O	Manager Write Strobes. This signal indicates which byte lanes to update in memory. There is one Write strobe for each 8-bits of Write data bus. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
wuser_m_o[(DWCXG_M_W_PYLD_USER_WIDTH-1):0]	O	Manager Write Channel user signal. This provides ■ Parity for control word ■ ECC for data word **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
wlast_m_o	O	Manager Write Last. This signal indicates the last transfer in a Write burst. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
wvalid_m_o	O	Manager Write Valid. This signal indicates that valid Write data and strobes are available: ■ 1: Write data and strobes available ■ 0: Write data and strobes not available **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** DWCXG_AIS_EN ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
wvalid_m_p_o	O	Parity for Manager Write Valid. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
wready_m_i	I	Manager Write Ready. This signal indicates that the subordinate accepted the Write data: ■ 1: Subordinate has accepted ■ 0: Subordinate has not accepted **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
wready_m_p_i	I	Parity for Manager Write Ready. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
bid_m_i[(DWCXG_AXI_IDW-1):0]	I	Manager Response ID. This signal is the identification tag of Write response. The BID value must match the AWID value of the write transaction to which the subordinate is responding. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
bresp_m_i[1:0]	I	Manager Write Response. This signal indicates the status of the Write transaction. The valid responses are: ■ OKAY: 2'b00 ■ EXOKAY: 2'b01 ■ SLVERR: 2'b10 ■ DECERR: 2'b11 **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
buser_m_i[(DWCXG_M_B_PYLD_USER_WIDTH-1):0]	I	Manager Response Channel User signal. This signal gives parity of control word for response channel. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
bvalid_m_i	I	Manager Write Response Valid. This signal indicates that a valid Write response is available: ■ 1: Write response available ■ 0: Write response not available **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
bvalid_m_p_i	I	Parity for Manager Write Response Valid. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
bready_m_o	O	Manager Response Ready. This signal indicates that the manager can accept the response information. ■ 1: Manager ready ■ 0: Manager not ready. This signal is always high. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** DWCXG_AIS_EN ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
bready_m_p_o	O	Parity for Manager Response Ready. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
araddr_m_o[(DWCXG_ADDRWIDTH-1) :0]	O	Manager Read Address. The Read address bus gives the initial address of a Read burst transaction. It provides the start address of the burst and the control signals that are issued with the address. The control signals specify how the address is calculated for remaining transfers in the burst. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
arlen_m_o[(DWCXG_BLW-1):0]	O	Manager Burst Length. The burst length gives the exact number of transfers in a burst. This information determines the number of data transfers associated with the address. The default width is 4 but width of this signal can be increased while configuring the controller to support bigger bursts. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arid_m_o[(DWCXG_AXI_IDW-1):0]	O	Manager Read Address ID. This signal is the identification tag for the Read address group of signals. The width of this signal is configured during the controller configuration. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arburst_m_o[1:0]	O	Manager Burst Type. The burst type, along with the size information, specifies how the address for each transfer is calculated within the burst. This is always hard wired to 2'b01 to indicate INCR burst type. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arvalid_m_o	O	Manager Read Address Valid. When this signal is high, it indicates that the Read address and Control information is valid. This signal remains stable until the address acknowledge signal arready_m_i is high. ■ 1: Address and control information valid ■ 0: Address and control information not valid **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** DWCXG_AIS_EN ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
arvalid_m_p_o	O	Parity for Manager Read Address Valid. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
arready_m_i	I	Manager Read Ready. This signal indicates that the subordinate can accept the Read request: ■ 1: Subordinate ready ■ 0: Subordinate not ready **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
arready_m_p_i	I	Parity for Manager address Read Ready. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
arsize_m_o[2:0]	O	Manager Burst Length. This signal indicates the size of each transfer in the burst. This signal is always driven to have the System Data Width transfers (3'b011 for 64-bit). **Exists:** DWCXG == 2 **Synchronous To:** (DWCXG_ASP_PPE==2) ? "aclk_i" : "None" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
arlock_m_o	O	Locked Access Indicator for Read Address Channel AXI Manager. This signal indicates the type of locked access to the Read address channel of AXI manager. **Exists:** DWCXG == 2 **Synchronous To:** (DWCXG_ASP_PPE==2) ? "aclk_i" : "None" **Registered:** DWCXG_AIS_EN ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
arcache_m_o[3:0]	O	Manager Cache Type. This signal is used in AXI4 ACE-compliant systems. The value driven by this signal is controlled by programming the AXI_ARCache_control register. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arprot_m_o[2:0]	O	Manager Protection Type. This signal provides protection unit information for a transaction. This is always hard wired to 3'b000 to indicate normal and secure data access. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arqos_m_o[3:0]	O	Write Channel QOS. This signal is used in AXI-4 compliant QOS aware systems. The value driven by this is controlled by the DMA_CHx_Tx_control and DMA_CHx_Rx_Control register. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
ardomain_m_o[1:0]	O	AXI Domain Control. This signal is used in AXI4 ACE compliant systems. The value driven by this signal is controlled by programming the AXI_ARCache_Control register. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
aruser_m_o[(DWCXG_M_AR_PYLD_USER_WIDTH-1):0]	O	AXI Manager Address read user signal. This signal give parity for control word 1 and control word 0. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rid_m_i[(DWCXG_AXI_IDW-1):0]	I	Manager Read ID Tag. This signal is the ID tag of the read data group of signals. The rid value is generated by the subordinate and must match the arid value of Read transaction to which it is responding. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rresp_m_i[1:0]	I	Manager Read Response. This signal indicates the status of the Read transfer. The valid responses are: ■ OKAY: 2'b00 ■ EXOKAY: 2'b01 ■ SLVERR: 2'b10 ■ DECERR: 2'b11 **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
ruser_m_i[(DWCXG_M_R_PYLD_USER_WIDTH-1):0]	I	Manager Read User signal. This signal gives parity for control word 1 and ECC for control word 0 for AXI manager read **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rdata_m_i[(DWCXG_DATW-1):0]	I	Manager Read Data. The Read data bus takes the value of DWC_XGMAC_DATAWIDTH parameter. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rdata_m_p_i[(DWCXG_DPW-1):0]	I	AXI Manager Read Data Parity. The Read data bus takes the value of DWCXG_DPW parameter. **Exists:** DWCXG_AXI_SUBSYS&&(DWCXG_ASP_PPE==1) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rlast_m_i	I	Manager Read Last. This signal indicates the last transfer in a Read burst. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
rvalid_m_i	I	Manager Read Valid. This signal indicates that required Read data is available and the Read transfer can complete: ■ 1: Read data available ■ 0: Read data not available **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rvalid_m_p_i	I	Parity for Manager Read Valid. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rready_m_o	O	Manager Read Ready. This signal indicates that the manager can accept the read data and response information: ■ 1: Manager ready ■ 0: Manager not ready. This signal is always high. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** DWCXG_AIS_EN ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rready_m_p_o	O	Parity for Manager Read Ready. **Exists:** (DWCXG_ASP_PPE==2) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-5 AXI Manager Interface Signals (continued)

Port Name	I/O	Description
awqos_m_o[3:0]	O	Write Channel QOS. This signal is used in AXI4-compliant QOS-aware systems. The value driven by this is controlled by the DMA_CHx_Tx_control and DMA_CHx_Rx_Control register. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awdomain_m_o[1:0]	O	AXI Domain Control. This signal is used in AXI4 ACE-compliant systems. The value driven by this signal is controlled by programming the AXI_AWCache_Control register. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.6 RGMII Clock and Reset Signals

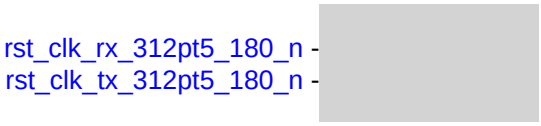


Table 12-6 RGMII Clock and Reset Signals

Port Name	I/O	Description
rst_clk_rx_312pt5_180_n	I	This is the active-low reset signal synchronous to clk_rx_312pt5_180_i. **Exists:** DWCXG_RGMII_EN&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "None" : "clk_rx_312pt5_180_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** Low
rst_clk_tx_312pt5_180_n	I	This is the active-low reset signal synchronous to clk_tx_312pt5_180_i. **Exists:** DWCXG_RGMII_EN&&!DWCXG_AXI_SUBSYS **Synchronous To:** (DWCXG_RESET_MODE == 1)==1 ? "None" : "clk_tx_312pt5_180_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low

12.7 Test Mode Signals

scan_mode -

Table 12-7 Test Mode Signals

Port Name	I/O	Description
scan_mode	I	Scan Mode. This input enables bypassing of internally-generated resets and directly connecting these resets to the external reset input (hreset_n, aresetn_i, or pwr_on_rst_n respectively). Generally, tie this input to 0. It must be tied to 1, only during scan testing. This input is available only in XGMAC-AXI configurations. **Exists:** (DWCXG_EEE_EN&&!DWCXG_XGMII_SDR)||DWCXG_PMT_EN||DWCXG_AXI_SUBSYS **Synchronous To:** None **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

12.8 Sideband Interface Signals

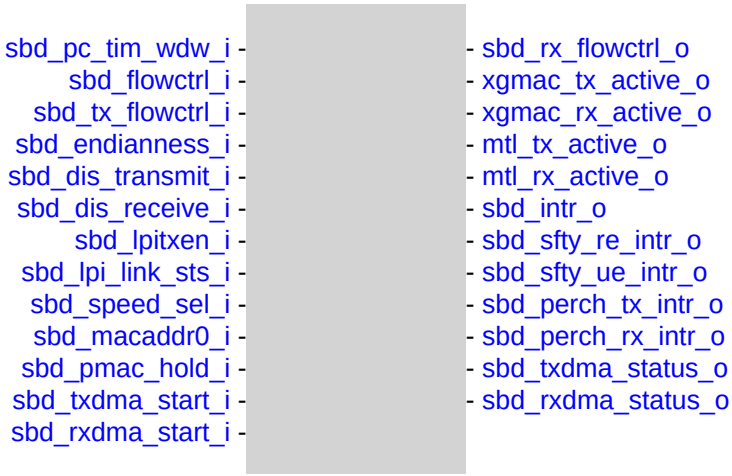


Table 12-8 Sideband Interface Signals

Port Name	I/O	Description
sbd_pc_tim_wdw_i[3:0]	I	Policing Time window signals. This input is used to enable the increment of Policing counters and must be synchronous to Rx clock. **Exists:** DWCXG_PC **Synchronous To:** clk_rx_312pt5_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_flowctrl_i[(DWCXG_SBDFC-1):0]	I	Sideband Flow Control. When set high, instructs the MAC to transmit Pause frames (or PFC frames when PFCE bit in MAC_Rx_Flow_Ctrl register is set). **Exists:** DWCXG_SBD_FC_EN **Synchronous To:** Asynchronous **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-8 Sideband Interface Signals (continued)

Port Name	I/O	Description
sbd_rx_flowctrl_o[(DWCXG_NUM_RXQ-1):0]	O	Sideband Rx Flow Status output. This signal is set to 1 whenever the corresponding Rx Queue space goes lower than the activate threshold (RFA) value programmed in MTL_RxQx_Flow_Control register. This signal set to 0 when the corresponding Rx Queue space goes higher than the de-activate threshold (RFD) value programmed in MTL_RxQx_Flow_Control register. **Exists:** SNPS_RSVDPARAM_10&&(DWCXG_RXFIFO_SIZE>=4096) **Synchronous To:** !DWCXG_RX_SRAM ? "clk_rx_312pt5_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_tx_flowctrl_i[(DWCXG_NUM_TC-1):0]	I	Sideband Tx Flow Control input. When this input signal is set to 1, the corresponding Traffic Class is paused for Transmission. (When either this signal is set to 1 or when internally generated PFC is set to 1, the corresponding Traffic Class is paused for Transmission). **Exists:** SNPS_RSVDPARAM_10 **Synchronous To:** !DWCXG_TX_SRAM ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** (DWCXG_PFC_EN&&SNPS_RSVDPARAM_10) ? No : ((DWCXG_NUM_TXQ>1&&SNPS_RSVDPARAM_10) ? Yes : No) **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_endianness_i	I	Sideband Endianness Control. When this signal is ■ High: Configures the Application Interface to transfer the data in the big-endian format. ■ Low: Configures the Application Interface to transfer the data in the little-endian format. This signal is sampled during active reset (including soft-reset) only and ignored after reset is low. In XGMAC-AXI configuration, sbd_endianess_i is also used for descriptor endianness. This signal is an optional port available only when you select BOTH_OPTIONS for the Endian Mode during controller configuration. **Exists:** DWCXG_ENDIAN_NESS==2 **Synchronous To:** Asynchronous **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-8 Sideband Interface Signals (continued)

Port Name	I/O	Description
sbd_dis_transmit_i	I	Sideband MAC Transmitter Disable Control. This signal instructs the DWC_25gmac transmitter to stop transmitting packets after completing current packet transfer. The DWC_25gmac transmitter restarts transmission only if this signal is reset to low and the TE bit of MAC_Tx_Configuration register is set high. This signal is an optional port applicable only in XGMAC-CORE and XGMAC-MTL configurations. **Exists:** (DWCXG_ADD_TXRX_DIS_IO) **Synchronous To:** Asynchronous **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_dis_receive_i	I	Sideband MAC Receiver Disable Control. This signal instructs the DWC_25gmac receiver to stop receiving packets after completing the current packet reception. The DWC_25gmac receiver restarts reception only if this signal is reset to low and the RE bit of MAC_Rx_Configuration register is set high. This signal is an optional port applicable only in XGMAC-CORE and XGMAC-MTL configurations. **Exists:** (DWCXG_ADD_TXRX_DIS_IO) **Synchronous To:** Asynchronous **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_lpitxen_i	I	Sideband EEE LPI Transmit enable signal. When this signal is ■ High: Instructs the MAC Transmitter to enter the LPI state. ■ Low: Instructs the MAC Transmitter to exit the LPI state and resume normal transmission. This signal is selected over LPITXEN bit of the MAC_LPI_Control_Status register when Sideband IO selection enable (Bit 8 of MAC_Extended_Configuration register) is set. This signal is used only when either LPITXA=1 & LPIATE=1 or LPITXA=0 & LPIATE=0 (in MAC_LPI_Control_Status register). **Exists:** DWCXG_ADD_SBD_IO&&DWCXG_EEE_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-8 Sideband Interface Signals (continued)

Port Name	I/O	Description
sbd_lpi_link_sts_i	I	Sideband LPI Link status indication in 1Gbps/100Mbps/10Mbps speeds in GMII/MII mode. This signal indicates the link status of the PHY in GMII/1G mode. When this signal is ■ High: Indicates that the link status is up. ■ Low: Indicates that the link status is down. The MAC transmitter asserts the LPI pattern only when the link status is up and the Sideband IO sel in Bit 8 of MAC_Extended_Configuration register is set. **Exists:** DWCXG_ADD_SBD_IO&&DWCXG_EEE_EN&&DWCXG_GMII_SUPPORT **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_speed_sel_i[2:0]	I	Sideband speed selection signal. Speed encoding is as follows: ■ 3'b000 10G - XGMII ■ 3'b001 - Reserved ■ 3'b010 2.5G - GMII ■ 3'b011 1G - GMII ■ 3'b100 100M - MII ■ 3'b101 5G - XGMII ■ 3'b110 2.5G - XGMII ■ 3'b111 10M - MII This signal is allowed to change only when MAC Transmitter and MAC Receiver are disabled and inactive. In other words, only when sbd_dis_transmit_i, sbd_dis_receive_i = 1 AND xgmac_tx_active_o, mtl_tx_active_o = 0 AND xgmac_rx_active_o, mtl_rx_active_o = 0 **Exists:** DWCXG_ADD_SBD_IO&&(DWCXG_GMII_SUPPORT||DWCXG_TPRE) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sbd_macaddr0_i[55:0]	I	Sideband MAC_Address0 register input signal. ■ [47:0]: 48-bit MAC Destination Address to be updated in the MAC_Address0_Low and MAC_Address0_High registers. ■ [48]: Address Enable.

Table 12-8 Sideband Interface Signals (continued)

Port Name	I/O	Description
		■ [54:49]: Mask Byte Control. ■ [55]: Validates the sbd_macaddr0_i[54:0], toggle this bit along with new value driven on sbd_macaddr0_i[54:0]. The Sideband MAC_Address0 register input signal has higher priority over the software write to MAC_Address0_Low and MAC_Address0_High registers. This signal is allowed to change only when MAC Transmitter and MAC Receiver are disabled and inactive. In other words, only when all the following conditions are met ■ sbd_dis_transmit_i, sbd_dis_receive_i = 1 ■ xgmac_tx_active_o, mtl_tx_active_o = 0 ■ xgmac_rx_active_o, mtl_rx_active_o = 0 **Note:** This signal must have stable value for at least two CSR clock cycles so that DWC_25gmac updates the correct value in the MAC_Address0_Low and MAC_Address0_High registers. When "Synchronize CSR MAC Address to Tx/Rx Clock Domain" or "Enable Power Management (PMT) Block" features are selected the configuration, this signal must be stable for additional sync delay, as mentioned in the description of MAC_Address0_High register. **Exists:** SNPS_RSVDPARAM_15 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
xgmac_tx_active_o	O	MAC Transmitter active indication. When this signal is asserted, it indicates that the MAC Transmitter is in ACTIVE state. When this signal is de-asserted, it indicates that the MAC Transmitter is in IDLE state. **Exists:** DWCXG_ADD_SBD_IO **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-8 Sideband Interface Signals (continued)

Port Name	I/O	Description
xgmac_rx_active_o	O	MAC Receiver active indication. When this signal is asserted, it indicates that the MAC Receiver is in ACTIVE state. When this signal is de-asserted, it indicates that the MAC Receiver is in IDLE state. **Exists:** DWCXG_ADD_SBD_IO **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sbd_pmac_hold_i	I	Sideband Hold Release Control. This is Hold & Release Control for Frame Preemption specified in IEEE 802.3br Std. The MAC considers low to high and high to low transitions on this signal as start of Hold and Release windows, respectively. **Exists:** DWCXG_FPE **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** DWCXG_AV_EST ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mtl_tx_active_o	O	MTL Transmitter active indication. When this signal is asserted, it indicates that the MTL Transmitter is in ACTIVE state. When this signal is de-asserted, it indicates that the MTL Transmitter is in IDLE state. **Exists:** !DWCXG_CORE&&DWCXG_ADD_SBD_IO **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mtl_rx_active_o	O	MTL Receiver active indication. When this signal is asserted, it indicates that the MTL Receiver is in ACTIVE state. When this signal is de-asserted, it indicates that the MTL Receiver is in IDLE state. **Exists:** !DWCXG_CORE&&DWCXG_ADD_SBD_IO **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-8 Sideband Interface Signals (continued)

Port Name	I/O	Description
sbd_intr_o	O	Common Interrupt Output. This signal is generated by DWC_25gmac as a level to indicate the interrupt events. For more details, see 'Interrupts'. **Exists:** Always **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_sfty_re_intr_o	O	Subsystem Interrupt. When this signal is high, it indicates a correctable error interrupt event to the application from Automotive Safety features. This signal is available in non-core and DWCXG_ASP_ECC enabled configurations. **Exists:** DWCXG_ASP_ECC **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sbd_sfty_ue_intr_o	O	Subsystem Interrupt. When this signal is high, it indicates an un-recoverable interrupt event to the application from Automotive Safety features. This signal is available in non-core and "DWCXG_ASP_ECC or DWCXG_ASP_ALL" enabled configurations. **Exists:** DWCXG_ASP_ALL||DWCXG_ASP_ECC **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-8 Sideband Interface Signals (continued)

Port Name	I/O	Description
sbd_perch_tx_intr_o[(DWCXG_NTXCH-1):0]	O	Per Channel Transfer Complete Interrupt Pulse of Transmit Channel. This is generated when the corresponding Transmit Channel closes the descriptor with LD bit and IOC bit set. For more details, see 'Per Channel Interrupt Support'. **Exists:** !DWCXG_CORE&&!DWCXG_MTL_SUBSYS **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_perch_rx_intr_o[(DWCXG_NRXCH-1):0]	O	Per Channel Transfer Complete Interrupt Pulse of Receive Channel. This is generated when the corresponding Receive Channel closes the descriptor with LD bit and has IOC bit set or when the Receive Interrupt Watchdog Timer expires. For more details, see 'Per Channel Interrupt Support'. **Exists:** !DWCXG_CORE&&!DWCXG_MTL_SUBSYS **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
sbd_txdma_start_i[(DWCXG_NTXCH-1):0]	I	Sideband Tx DMA Start/Stop Control. When this signal goes high, rising edge indicates that Tx DMA must be started. When this signal goes low (default state), falling edge indicates that the Tx DMA must be stopped. This signal must be tied low when this function is not used. **Exists:** DWCXG_DMA_STRTSTP **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-8 Sideband Interface Signals (continued)

Port Name	I/O	Description
sbd_rxdma_start_i[(DWCXG_NRXCH-1):0]	I	Sideband Rx DMA Start/Stop Control. When this signal goes high, rising edge indicates that Rx DMA must be started. When this signal goes low (default state), falling edge indicates that the Rx DMA must be stopped. This signal must be tied low when this function is not used. **Exists:** DWCXG_DMA_STRTSTP **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sbd_txdma_status_o[(DWCXG_NTXCH-1):0]	O	Sideband Tx DMA Status Output. This signal is set to '1' when Tx DMA is in Start State, and '0' when Tx DMA is Stopped. **Exists:** DWCXG_DMA_STRTSTP **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sbd_rxdma_status_o[(DWCXG_NRXCH-1):0]	O	Sideband Rx DMA Status Output. This signal is set to '1' when Rx DMA is in Start State, and '0' when Rx DMA is Stopped. **Exists:** DWCXG_DMA_STRTSTP **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.9 External Lookup Interface Signals

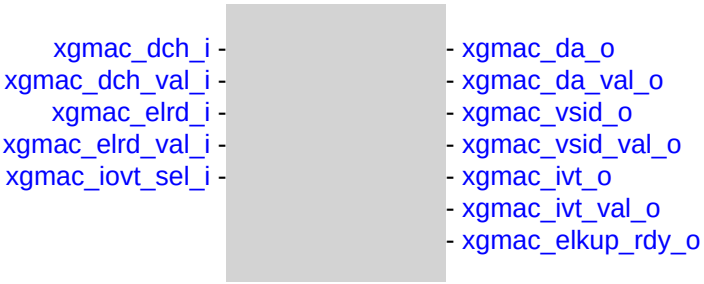


Table 12-9 External Lookup Interface Signals

Port Name	I/O	Description
xgmac_da_o[47:0]	O	This signal provides the 48 bit destination address of the received packet. This is valid only when xgmac_da_val_o is high. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
xgmac_da_val_o	O	A pulse that indicates xgmac_da_o is valid. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_vsid_o[15:0]	O	When G999.1 is enabled in the configuration, this signal provides the 16-bit SID of the received packet. When G999.1 is disabled in the configuration, this signal provides the outer VLAN tag of the received packet, when available. This signal is valid only when xgmac_vsid_val_o is high. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-9 External Lookup Interface Signals (continued)

Port Name	I/O	Description
xgmac_vsid_val_o	O	A pulse that indicates xgmac_vsid_o is valid. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_ivt_o[15:0]	O	When double VLAN processing is enabled, this signal provides the inner VLAN tag of the received packet, when available. This signal is valid only when xgmac_ivt_val_o is high. This signal is available only when Enable Double VLAN processing is selected in the configuration. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1&&DWCXG_DOUBLE_VLAN_EN **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_ivt_val_o	O	A pulse that indicates xgmac_ivt_o is valid. This signal is available only when Enable Double VLAN processing is selected in the configuration. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1&&DWCXG_DOUBLE_VLAN_EN **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_elkup_rdy_o	O	A level signal which indicates that the MAC receiver is ready to accept the lookup results xgmac_dch_i. This signal is asserted after all the possible fields of the received frame is given out. It is de-asserted upon receiving the lookup result, or if the packet is aborted, or after the timeout occurs. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-9 External Lookup Interface Signals (continued)

Port Name	I/O	Description
xgmac_dch_i[(DWCXG_NUM_DMA_RX_CH-1):0]	I	This signal contains the DMA channel number selected by the external lookup logic. This signal is valid only when xgmac_dch_val_i is high. Each bit corresponds to one DMA channel. Bit[0] for DMA channel 0, Bit[1] for channel 1, and so on. One or more bits can be set simultaneously. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_dch_val_i	I	A pulse that indicates xgmac_dch_i, xgmac_elrd_i, xgmac_elrd_val_i, and xgmac_iovt_sel_i are valid. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_elrd_i[15:0]	I	This signal provides the lookup result data, that must be included in the OVT or IVT field of the receive status. This result data replaces the IVT field when xgmac_iovt_sel_i is high, OVT field when xgmac_iovt_sel_i is low. When xgmac_elrd_val_i is low, then no modification is done in the Receive status. This is valid when xgmac_dch_val_i is high. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
xgmac_elrd_val_i	I	This signal indicates xgmac_elrd_i is valid, and is valid only when xgmac_dch_val_i is high. **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-9 External Lookup Interface Signals (continued)

Port Name	I/O	Description
xgmac_iovt_sel_i	I	This signal when set to 1, indicates that the IVT field is replaced with lookup result data. Otherwise, OVT field to be replaced with look up data. The lookup data is provided on xgmac_elrd_i[15:0] when xgmac_elrd_val_i and xgmac_dch_val_i are high **Exists:** DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.10 L2 Virtualized Network Over L3 Network Interface Signals

- xgmac_tnl_pkt_o
- xgmac_tnl_vsegid_val_o
- xgmac_tnl_vsegid_o
- xgmac_tnl_ihda_o
- xgmac_tnl_ihda_val_o
- xgmac_tnl_ihvlan_o
- xgmac_tnl_ihvlan_val_o

Table 12-10 L2 Virtualized Network Over L3 Network Interface Signals

Port Name	I/O	Description
xgmac_tnl_pkt_o	O	This signal when set, indicates that the current packet received is a Tunneled packet. **Exists:** DWCXG_VXN_ENA&&DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_tnl_vsegid_val_o	O	This signal when set, indicates that xgmac_tnl_vsegid_o contains valid value. **Exists:** DWCXG_VXN_ENA&&DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_tnl_vsegid_o[23:0]	O	This signal contains the VNI or TNI field of the Tunneled packet. **Exists:** DWCXG_VXN_ENA&&DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-10 L2 Virtualized Network Over L3 Network Interface Signals (continued)

Port Name	I/O	Description
xgmac_tnl_ihda_o[47:0]	O	This signal contains DA field of Inner packet in Tunneled packet. **Exists:** DWCXG_VXN_ENA&&DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
xgmac_tnl_ihda_val_o	O	This signal when set, indicates that xgmac_innerpkt_da_o contains valid value. **Exists:** DWCXG_VXN_ENA&&DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmac_tnl_ihvlan_o[15:0]	O	This signal contains VLAN field of Inner packet in Tunneled packet. **Exists:** DWCXG_VXN_ENA&&DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
xgmac_tnl_ihvlan_val_o	O	This signal when set, indicates that xgmac_tnl_ihvlan_o contains valid value. **Exists:** DWCXG_VXN_ENA&&DWCXG_EXT_LKUP_EN&&DWCXG_NUM_DMA_RX_CH>1 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.11 G999_1 Signals

Table 12-11 G999_1 Signals

Port Name	I/O	Description
xgmac_dfc_i[(DWCXG_DFCW-1):0]	I	DFC field. This signal gives the DFC field of the PAUSE control frame generated by the MAC transmitter in G999_1 mode of operation. Any transition in any of the signal bits schedules a PAUSE frame transmission. In non-G999_1 modes, this input bus is ignored. **Exists:** DWCXG_G9991_EN **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
xgmac_rcvd_dfc_o[(DWCXG_DFCW-1):0]	O	DFC field in Received PAUSE Packet. This signal gives the DFC field of the PAUSE control packet received by the MAC receiver in G999_1 mode of operation. It is updated with every PAUSE packet received. In non-G999_1 modes, this output bus can be ignored. **Exists:** DWCXG_G9991_EN **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.12 GCL Memory Interface Signals

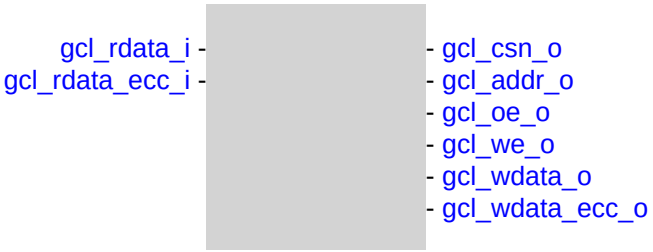


Table 12-12 GCL Memory Interface Signals

Port Name	I/O	Description
gcl_csn_o	O	GCL Memory Chip Select. Use this signal as a chip select to GCL Memory. **Exists:** (DWCXG_AV_EST||DWCXG_PSF) **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
gcl_addr_o[(DWCXG_GMAW-1):0]	O	GCL Memory Address Bus. GCL Memory Address bus to be connected to the External GCL Memory. **Exists:** (DWCXG_AV_EST||DWCXG_PSF) **Synchronous To:** clk_ptp_ref_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
gcl_oe_o	O	GCL Memory Output Enable. Use this signal as an output enable to the GCL Memory. Memory block must drive Read Data on gcl_rdata_i when this signal is high. **Exists:** (DWCXG_AV_EST||DWCXG_PSF) **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-12 GCL Memory Interface Signals (continued)

Port Name	I/O	Description
gcl_we_o	O	GCL Memory Write Enable. Use this signal as a write enable to the GCL Memory. Memory block must write data provided on the gcl_wdata_o to the address provided on gcl_addr_o. **Exists:** (DWCXG_AV_EST||DWCXG_PSF) **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
gcl_wdata_o[(DWCXG_GMDW-1):0]	O	GCL Memory Write Data. This is the write data bus to GCL Memory. **Exists:** (DWCXG_AV_EST||DWCXG_PSF) **Synchronous To:** clk_ptp_ref_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
gcl_wdata_ecc_o[(DWCXG_GCL_ECW-1):0]	O	GCL Memory Write ECC Data. This is the write data bus to GCL Memory. **Exists:** (DWCXG_AV_EST||DWCXG_PSF)&&DWCXG_ASP_ECC **Synchronous To:** clk_ptp_ref_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
gcl_rdata_i[(DWCXG_GMDW-1):0]	I	GCL Memory Read Data. This is the read data bus from GCL Memory. **Exists:** (DWCXG_AV_EST||DWCXG_PSF) **Synchronous To:** clk_ptp_ref_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-12 GCL Memory Interface Signals (continued)

Port Name	I/O	Description
gcl_rdata_ecc_i[(DWCXG_GCL_ECW-1):0]	I	GCL Memory Read ECC Data. This is the read data bus from GCL Memory. **Exists:** (DWCXG_AV_EST||DWCXG_PSF)&&DWCXG_ASP_ECC **Synchronous To:** clk_ptp_ref_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.13 AXI Subordinate Interface Signals

Table 12-13 AXI Subordinate Interface Signals

Port Name	I/O	Description
awid_s_i[(DWCXG_GS_ID-1):0]	I	Subordinate Write Address ID. This signal is the identification tag for Write address group of signals. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
awaddr_s_i[(DWCXG_GS_AW-1):0]	I	Subordinate Write Address. The Write address bus gives the address of the first transfer in a Write burst transaction. The associated control signals are used to determine the addresses of the remaining transfers in the burst. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awlen_s_i[(DWCXG_GS_BW-1):0]	I	Subordinate Burst Length. The burst length gives the exact number of transfers in a burst for the Write channel. This information determines the number of data transfers associated with the address. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awsize_s_i[2:0]	I	Subordinate Burst Length. This signal indicates the size of each transfer in the burst. Byte lane strobes indicate exactly which byte lanes to update. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awburst_s_i[1:0]	I	Subordinate Burst Type. The burst type, along with the size information, specifies how the address is calculated for each transfer within the burst for the Write channel. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
awlock_s_i	I	Locked Access Indicator for Write Address Channel of the AXI Subordinate. This signal indicates the type of locked access to the Write address channel of the AXI Subordinate. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
awcache_s_i[3:0]	I	Subordinate Cache Type. This signal indicates the bufferable, cacheable, write-through, write-back, and allocate attributes of the transaction. DWC_25gmac does not use this input because it supports only normal data transactions. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awprot_s_i[2:0]	I	Subordinate Protection Type. This signal indicates the normal, privileged, or secure protection level of the transaction and whether the transaction is a data access or an instruction access. DWC_25gmac does not use this input because it supports only normal data transactions. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
awuser_s_i[(DWCXG_S_AW_PYLD_USER_WIDTH_PORT-1):0]	I	AXI subordinate address write user signal. This provides parity for control word 1 and control word 0. **Exists:** (DWCXG_ASP_PPE==2)&&(DWCXG_AXI4_LITE_SLAVE||DWCXG_AXI_SLAVE) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
awvalid_s_i	I	Subordinate Write Address Valid. This signal indicates that valid write address and control information are available: ■ 1: Address and control information available ■ 0: Address and control information not available. The address and control information remain stable until the address acknowledge signal awready_s_o goes high. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
awvalid_s_p_i	I	Parity for Subordinate Write Address Valid. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
awready_s_o	O	Subordinate Write Address Ready. This signal indicates that the subordinate is ready to accept an address and associated control signals: ■ 1: Subordinate ready ■ 0: Subordinate not ready **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
awready_s_p_o	O	Parity for Subordinate Write Address Ready. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
wid_s_i[(DWCXG_GS_ID-1):0]	I	Subordinate Write ID Tag. This signal is the ID tag of the Write data transfer. The wid value must match the awid value of the Write transaction. This input is not used because the AXI subordinate does not accept more than one Write request at a time. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
wdata_s_i[(DWCXG_GS_DW-1):0]	I	Subordinate Write Data. The Write data bus takes the value of the DWC_XGMAC_DATAWIDTH parameter. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
wdata_s_p_i[(DWCXG_GS_PW-1):0]	I	AXI Subordinate Write Parity Data. The Write parity data bus takes the value of the DWCXG_CSR_DATAWIDTH/8 parameter. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==1) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
wstrb_s_i[((DWCXG_GS_DW/8)-1):0]	I	Subordinate Write Strobes. This signal indicates the byte lanes that are updated in the memory. One write strobe is equal to the eight bits of the Write data bus. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** DWCXG_AIS_EN ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
wlast_s_i	I	Subordinate Write Last. This signal indicates the last transfer in a Write burst. This input is not used because the AXI manager always provides the input about the required number of the Write data transfers to the subordinate. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
wuser_s_i[(DWCXG_S_W_PYLD_USER_WIDTH_PORT-1):0]	I	AXI subordinate write channel user signal. This provides parity for control word and ECC for data word. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
wvalid_s_i	I	Subordinate Write Valid. This signal indicates that valid Write data and strobes are available: ■ 1: Write data and strobes available ■ 0: Write data and strobes not available **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
wvalid_s_p_i	I	Parity for Subordinate Write Valid. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
wready_s_o	O	Subordinate Write Ready. This signal indicates that the subordinate can accept the Write data: ■ 1: Subordinate ready ■ 0: Subordinate not ready **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
wready_s_p_o	O	Parity for Subordinate Write Ready. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
bid_s_o[(DWCXG_GS_ID-1):0]	O	Subordinate Response ID. The identification tag of the Write response. The bid value is equal to the awid value of the write transaction to which the subordinate is responding. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** DWCXG_AIS_EN ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
bresp_s_o[1:0]	O	Subordinate Write Response. This signal indicates the status of the write transaction. The valid responses are: ■ OKAY: 2'b00 ■ EXOKAY: 2'b01 ■ SLVERR: 2'b10 ■ DECERR: 2'b11 DWC_25gmac provides only OKAY and SLVERR response for the bresp_s_o signal. It does not provide the EXOKAY and DECERR responses. It includes uncorrectable safety error on subordinate path as SLVERR(2'b10), if it is configured. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** DWCXG_AIS_EN ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
buser_s_o[(DWCXG_S_B_PYLD_USER_WIDTH-1):0]	O	AXI Subordinate response user signal. This signal provides parity for control word **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
bvalid_s_o	O	Subordinate Write Response valid. This signal indicates that a valid Write response is available: ■ 1: Write response available ■ 0: Write response not available **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
bvalid_s_p_o	O	Parity for Subordinate Write Response valid. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
bready_s_i	I	Subordinate Response Ready. This signal indicates that the manager can accept the response information: ■ 1: Subordinate ready ■ 0: Subordinate not ready **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
bready_s_p_i	I	Parity for Subordinate Response Ready. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
arid_s_i[(DWCXG_GS_ID-1):0]	I	Subordinate Read Address ID. This signal is the identification tag for the Read address group of signals. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
araddr_s_i[(DWCXG_GS_AW-1):0]	I	Subordinate Read Address. The Read address bus gives the initial address of a read burst transaction. It provides the start address of the burst and the control signals that are issued with the address. The control signals specify how the address is calculated for remaining transfers in the burst. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arlen_s_i[(DWCXG_GS_BW-1):0]	I	Subordinate Burst Length. The burst length gives the exact number of transfers in a burst for the Read channel. This information determines the number of data transfers associated with the address. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arsize_s_i[2:0]	I	Subordinate Burst Length. This signal indicates the size of each transfer in the burst. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arburst_s_i[1:0]	I	Subordinate Burst Type. The burst type, along with the size information, specifies how the address is calculated for each transfer within the burst for the Read channel. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
arlock_s_i	I	Locked Access Indicator for Read Address Channel of the AXI Subordinate. This signal indicates the type of locked access to the Read address channel of the AXI Subordinate. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
arcache_s_i[3:0]	I	Manager Cache Type. This signal provides additional information about the cacheable characteristics of the transfer. DWC_25gmac does not use this input because it supports only normal data transactions. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
arprot_s_i[2:0]	I	Subordinate Protection Type. This signal provides protection unit information for the transaction. The DWC_25gmac AXI subordinate does not support Exclusive access. DWC_25gmac does not use this input because it supports only normal data transactions. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
aruser_s_i[(DWCXG_S_AR_PYLD_USER_WIDTH_PORT-1):0]	I	AXI subordinate address read user signal. This provides parity for control word 1 and control word 0 **Exists:** (DWCXG_ASP_PPE==2)&&(DWCXG_AXI4_LITE_SLAVE||DWCXG_AXI_SLAVE) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
arvalid_s_i	I	Subordinate Read Address Valid. When this signal is high, it indicates that the read address and control information is valid. This signal remains stable until the address acknowledge signal aready_s_o is high. ■ 1: Address and control information valid ■ 0: Address and control information not valid **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
arvalid_s_p_i	I	Parity for Subordinate Read Address Valid. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
aready_s_o	O	Subordinate Read Address Ready. This signal indicates that the subordinate is ready to accept an address and associated control signals: ■ 1: Subordinate ready ■ 0: Subordinate not ready **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
aready_s_p_o	O	Parity for Subordinate Read Address Ready. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
rid_s_o[(DWCXG_GS_ID-1):0]	O	Subordinate Read ID Tag. This signal is the ID tag of the Read data group of signals. The rid value is generated by the subordinate. The rid value is equal to the arid value of the Read transaction to which it is responding. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** DWCXG_AIS_EN ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rdata_s_o[(DWCXG_GS_DW-1):0]	O	Subordinate Read Data. The Read data bus takes the value of the DWC_XGMAC_DATAWIDTH parameter. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rdata_s_p_o[(DWCXG_GS_PW-1):0]	O	AXI Subordinate Read Parity Data. The Read parity data bus takes the value of the DWCXG_CSR_DATAWIDTH/8 parameter. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==1) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
rresp_s_o[1:0]	O	Subordinate Read Response. This signal indicates the status of the read transfer. The valid responses are: ■ OKAY: 2'b00 ■ EXOKAY: 2'b01 ■ SLVERR: 2'b10 ■ DECERR: 2'b11 DWC_25gmac provides only OKAY and SLVERR response for the rresp_s_o signal. It does not provide the EXOKAY and DECERR responses. It includes uncorrectable safety error on subordinate path as SLVERR(2'b10), if it is configured. **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** DWCXG_AIS_EN ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rlast_s_o	O	Subordinate Read Last. This signal indicates the last transfer in a Read burst. **Exists:** (DWCXG_CSR_PORT==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** DWCXG_AIS_EN ? Yes : No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ruser_s_o[(DWCXG_S_R_PYLD_USER_WIDTH-1):0]	O	AXI subordinate read user signal. This signal provides Parity for control word and ECC for data word. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-13 AXI Subordinate Interface Signals (continued)

Port Name	I/O	Description
rvalid_s_o	O	Subordinate Read Valid. This signal indicates that the required Read data is available and the Read transfer can be completed: ■ 1: Read data available ■ 0: Read data not available **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rvalid_s_p_o	O	Parity for Subordinate Read Valid. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rready_s_i	I	Subordinate Read Ready. This signal indicates that the subordinate can accept the read data and response information: ■ 1: AXI manager ready ■ 0: AXI manager not ready **Exists:** DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rready_s_p_i	I	Parity for Subordinate Read Ready. **Exists:** (DWCXG_AXI_SLAVE||DWCXG_AXI4_LITE_SLAVE)&&(DWCXG_ASP_PPE==2) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : "aclk_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.14 AXI Low Power Interface Signals

Table 12-14 AXI Low Power Interface Signals

Port Name	I/O	Description
csysreq_i	I	Clock Controller System Low-Power Request. When this signal is low, it indicates that the system clock controller requested the AXI low-power interface to enter the low-power state. When high, this signal indicates that the AXI low-power interface must come out of the low-power state. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
csysack_o	O	Low-Power Request Acknowledgement. This signal is the acknowledgement to the low-power request from the system. **Exists:** DWCXG == 2 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
cactive_o	O	Clock Active. This signal indicates that the AXI low-power interface requires clock signal: ■ 1: Clock required ■ 0: Clock not required **Note:** This clock is synchronous to clk_rx_312pt5_i when self-initiating request to come out of low-power mode. **Exists:** DWCXG == 2 **Synchronous To:** DWCXG_CSR_SLV_CLK==1 ? "aclk_i,clk_csr_i,clk_rx_312pt5_i" : "aclk_i,clk_rx_312pt5_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.15 TSO Memory Interface (TMI) Signals

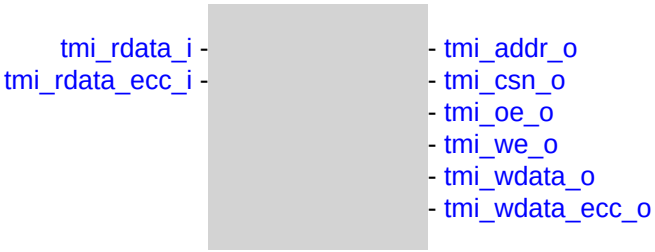


Table 12-15 TSO Memory Interface (TMI) Signals

Port Name	I/O	Description
tmi_addr_o[(DWCXG_CACHE_MEM_ADDR_WIDTH-1):0]	O	Address. This bus gives the address of the data transfer. The address width depends upon the size of the selected memory. **Exists:** DWCXG_TSO_MEM_EN **Synchronous To:** aclk_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tmi_csn_o	O	Chip Select. This signal is used as a chip select to select the TSO memory. **Exists:** DWCXG_TSO_MEM_EN **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
tmi_oe_o	O	Output Enable. The DMA drives this signal to enable the output data from the TSO memory. The TSO memory drives the valid Read data the tmi_rdata_i signal only when this signal is high. The DMA samples the Read data, with this signal high, one clock after the address is given. **Exists:** DWCXG_TSO_MEM_EN **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-15 TSO Memory Interface (TMI) Signals (continued)

Port Name	I/O	Description
tmi_we_o	O	Write Enable. When this signal is high, it indicates that a Write operation request is intended for TSO memory. **Exists:** DWCXG_TSO_MEM_EN **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
tmi_wdata_o[(DWCXG_TSOMEM_DW-1):0]	O	Write Data. This bus carries the data to be written in the Tx FIFO. It is valid when the tmi_csn_o signal is high and the tmi_oe_o signal is low. **Exists:** DWCXG_TSO_MEM_EN **Synchronous To:** aclk_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tmi_wdata_ecc_o[(DWCXG_CACHE_ECW-1):0]	O	TSO ECC Write Data. This bus carries the ECC data (computed on {tmi_addr_o,tmi_wdata_o}) to be written in the TSO Memory. It is valid when tmi_we_o is high along with tmi_csn_o low. **Exists:** DWCXG_TSO_MEM_EN&&DWCXG_ASP_ECC **Synchronous To:** aclk_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tmi_rdata_i[(DWCXG_TSOMEM_DW-1):0]	I	Read Data. This bus carries the data read from the TSO Memory. It is valid when the tmi_csn_o and tmi_oe_o signals are high. **Exists:** DWCXG_TSO_MEM_EN **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-15 TSO Memory Interface (TMI) Signals (continued)

Port Name	I/O	Description
tmi_rdata_ecc_i[(DWCXG_CACHE_ECW-1):0]	I	TSO ECC Read Data. This bus carries the ECC data read from the TSO Memory. It is valid when the tmi_oe_o signal is high. **Exists:** DWCXG_TSO_MEM_EN&&DWCXG_ASP_ECC **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.16 Descriptor Cache Memory Interface (DMI) Signals

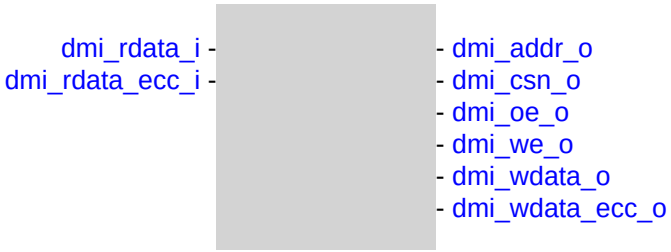


Table 12-16 Descriptor Cache Memory Interface (DMI) Signals

Port Name	I/O	Description
dmi_addr_o[(DWCXG_DCACHEAW-1):0]	O	Address. This bus gives the address of the descriptor transfer. **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Synchronous To:** aclk_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
dmi_csn_o	O	Chip Select. This signal is used as a chip select to select the Descriptor memory. **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
dmi_oe_o	O	Output Enable. The DMA drives this signal to enable the output data from the Descriptor memory. The Descriptor memory drives the valid Read data the dmi_rdata_i signal only when this signal is high. The DMA samples the Read data, with this signal high, one clock after the address is given. **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-16 Descriptor Cache Memory Interface (DMI) Signals (continued)

Port Name	I/O	Description
dmi_we_o	O	Write Enable. When this signal is high, it indicates that a Write operation request is intended for Descriptor memory. **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
dmi_wdata_o[(DWCXG_DCACHE_DW-1):0]	O	Write Data. This bus carries the pre-fetched descriptor. It is valid when the dmi_csn_o signal is high and the dmi_oe_o signal is low. **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Synchronous To:** aclk_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
dmi_wdata_ecc_o[(DWCXG_DCACHE_ECW-1):0]	O	DCACHE ECC Write Data. This bus carries the ECC data (computed on {dmi_addr_o,dmi_wdata_o}) to be written in the Descriptor Cache Memory. It is valid when dmi_we_o is high along with dmi_csn_o low. **Exists:** DWCXG_DCACHE_MEM_EN&&DWCXG_ASP_ECC **Synchronous To:** aclk_i **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
dmi_rdata_i[(DWCXG_DCACHE_DW-1):0]	I	Read Data. This bus carries the descriptor read from the Descriptor Memory. It is valid when the dmi_csn_o and dmi_oe_o signals are high. **Exists:** (DWCXG_EDMA&&DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&&!DWCXG_TSO_MEM_EN) **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-16 Descriptor Cache Memory Interface (DMI) Signals (continued)

Port Name	I/O	Description
dmi_rdata_ecc_i[(DWCXG_DCACHE_ECW-1):0]	I	DCACHE ECC Read Data. This bus carries the ECC data read from the Descriptor Cache Memory. It is valid when the dmi_oe_o signal is high. **Exists:** DWCXG_DCACHE_MEM_EN&&DWCXG_ASP_ECC **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.17 HDMA CSR Memory Interface (CMI) Signals

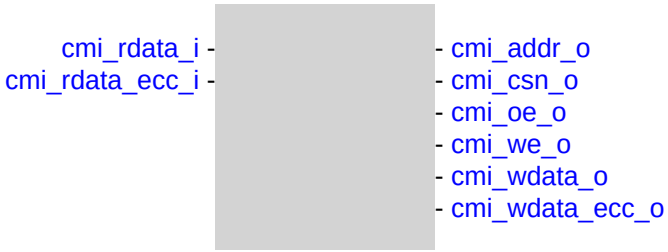


Table 12-17 HDMA CSR Memory Interface (CMI) Signals

Port Name	I/O	Description
cmi_addr_o[(DWCXG_DCSRAW-1):0]	O	Address. This bus gives the address of the CSR data transfer. The address width depends upon the size of the selected memory. **Exists:** SNPS_RSVDPARAM_50 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
cmi_csn_o	O	Chip Select. This signal is used as a chip select to select the DMA CSR memory. **Exists:** SNPS_RSVDPARAM_50 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
cmi_oe_o	O	Output Enable. The DMA drives this signal to enable the output data from the DMA CSR memory. The DMA CSR memory drives the valid Read data on the cmi_rdata_i signal only when this signal is high. The Controller samples the Read data, with this signal high, one clock after the address is given. **Exists:** SNPS_RSVDPARAM_50 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-17 HDMA CSR Memory Interface (CMI) Signals (continued)

Port Name	I/O	Description
cmi_we_o	O	Write Enable. When this signal is high, it indicates that a Write operation request is intended. **Exists:** SNPS_RSVDPARAM_50 **Synchronous To:** aclk_i **Registered:** N/A **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
cmi_wdata_o[(DWCXG_DCSRDW-1):0]	O	Write Data. This bus carries the data to be written in the DMA CSR external memory. It is valid when cmi_we_o is high along with cmi_csn_o low. **Exists:** SNPS_RSVDPARAM_50 **Synchronous To:** aclk_i **Registered:** (DWCXG_HDMA_MEM_DBG_ACCESS) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
cmi_rdata_i[(DWCXG_DCSRDW-1):0]	I	Read Data. This bus carries the DMA CSR data read from the HDMA Memory. It is valid when the cmi_csn_o and cmi_oe_o signals are high. **Exists:** SNPS_RSVDPARAM_50 **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
cmi_wdata_ecc_o[(DWCXG_HDMA_ECW-1):0]	O	HDMA ECC Write Data. This bus carries the ECC data (computed on {cmi_addr_o,cmi_wdata_o}) to be written in the HDMA Memory. It is valid when cmi_we_o is high along with cmi_csn_o low **Exists:** SNPS_RSVDPARAM_50&&DWCXG_ASP_ECC **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-17 HDMA CSR Memory Interface (CMI) Signals (continued)

Port Name	I/O	Description
cmi_rdata_ecc_i[(DWCXG_HDMA_ECW-1):0]	I	HDMA ECC Read Data. This bus carries the ECC data read from the HDMA Memory. It is valid when the cmi_oe_o signal is high. **Exists:** SNPS_RSVDPARAM_50&&DWCXG_ASP_ECC **Synchronous To:** aclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.18 Application Transmit Interface (ATI) Signals

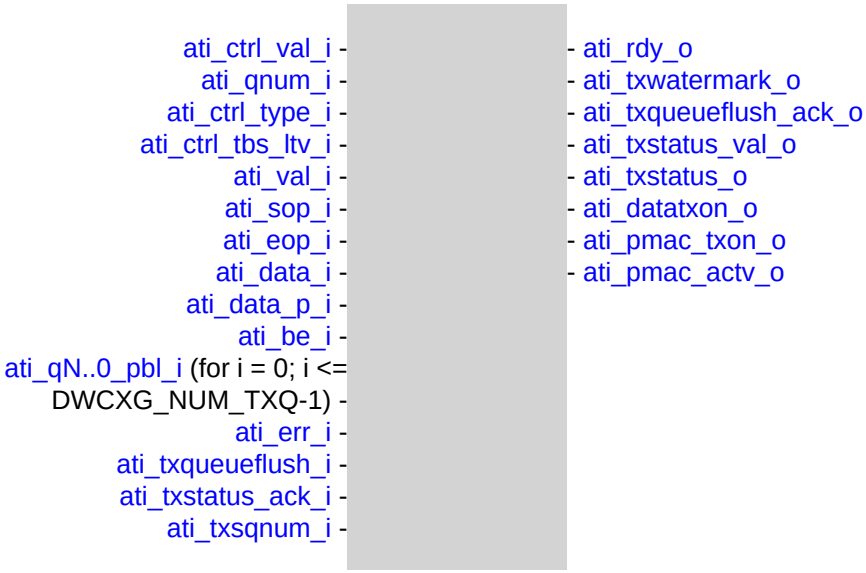


Table 12-18 Application Transmit Interface (ATI) Signals

Port Name	I/O	Description
ati_ctrl_val_i	I	Packet Control Data Valid. Assert this signal before the ati_sop_i and ati_val_i signals are asserted for packet transmission. When this signal is high, it qualifies the ati_data_i signal as Packet Control Word in association with the ati_ctrl_type_i signal. The type of the control word is indicated through the ati_ctrl_type_i signal. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_qnum_i[(DWCXG_TXQW-1):0]	I	Transmit Queue Number. This signal indicates the Tx queue number with which the current ATI data or control transfer is associated. This signal is not present in configurations with only one Tx queue. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_NUM_TXQ!=1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-18 Application Transmit Interface (ATI) Signals (continued)

Port Name	I/O	Description
ati_ctrl_type_i[(DWCXG_ATI_CTL_WIDTH-1):0]	I	Packet Control Data Type. This signal determines the type of the control word driven the ati_data_i signal when the ati_ctrl_val_i signal is high. This signal is present if one-step timestamp feature is selected in 64-bit configurations. This signal is not present in 128-bit data width configuration because entire Packet Control field can fit into 128-bit width when the ati_ctrl_val_i signal is high. The following are the encoded values: ■ 0: Packet Type Control Word ■ 1: Timestamp Control (Both upper and Lower) **Exists:** DWCXG_MTL_SUBSYS&&(DWCXG_DATAWIDTH_64&&(DWCXG_TIME_STAMPING||DWCXG_TBS)) **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_ctrl_tbs_ltv_i	I	TBS Launch Time Valid. Used when ati_ctrl_val_i is high and ati_ctrl_type_i(if available) is zero. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_TBS **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_val_i	I	ATI Valid. This signal qualifies all packet transmission control signals and the data the application transmit interface. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_sop_i	I	Start of Packet. This signal indicates that the application is ready to transfer the first data of packet. This signal is valid only when the ati_val_i signal is high. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-18 Application Transmit Interface (ATI) Signals (continued)

Port Name	I/O	Description
ati_eop_i	I	End of Packet. This signal indicates that the current data transferred to the MTL is the last data. This signal is valid only when the ati_val_i signal is high. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_data_i[(DWCXG_DW-1):0]	I	Application Data. This is the Packet data from the application when the ati_val_i signal is high. In little-endian format, the LSB lane ati_data_i[7:0] is the first byte transmitted by DWC_25gmac. In big-endian format, the MSB lane ati_data_i[DW-1:DW-8] is the first byte transmitted by DWC_25gmac. This signal indicates the control information when the ati_ctrl_val_i signal is high along with the type of control indicated by the ati_ctrl_type_i signal. When the control word is being transmitted, there is no change in the format of the control word based the endianness. The endianness is applicable only for the data. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ati_data_p_i[(DWCXG_DPW-1):0]	I	Application Parity Data. This is the Parity data computed on ati_data_i bus. **Exists:** DWCXG_MTL_SUBSYS&&(DWCXG_ASP_PPE==1) **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-18 Application Transmit Interface (ATI) Signals (continued)

Port Name	I/O	Description
ati_be_i[(DWCXG_BEW-1):0]	I	Number of Valid Byte Lanes. When the ari_eop_o signal is high, this signal indicates the number of valid bytes. The total number of valid bytes is equal to ati_be_o + 1 bytes the ati_data_i signal. This signal is valid only when the ati_val_i and ati_eop_i signals are high. When the ati_eop_i signal is low, the MTL considers all bytes of the ati_data_i signal as valid. When the ati_be_i signal width is 3, the following values indicate the lanes that have the data:. ■ 111: All 8 byte lanes have data ■ 110: LS 7 byte lanes have data ■ ... ■ 001: LS 2 byte lanes have data ■ 000: LS 1 byte lane have data **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** DWCXG_TX_COE ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ati_rdy_o	O	ATI Ready. When this signal is high, it indicates that the MTL is ready to accept additional data or control from the application. This signal is low in the following conditions: ■ When Tx Queue is being flushed ■ When Tx Queue is full ■ When internally-generated checksum data is being written to Tx Queue **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_qN..0_pbl_i[(DWCXG_ATI_PBL_WIDTH-1):0] (for i = 0; i <= DWCXG_NUM_TXQ-1)	I	Number of Beats Requested By the Data Application to Transmit Tx Queue. This signal indicates the space requested by the application the Tx Queue. This space is in terms of number of beats depending on the configurations: 8 or 16 bytes in 64-bit or 128-bit data bus mode, respectively. The maximum limit of the number of beats is equal to half of the Tx Queue depth. For configurations with multiple Tx Queues, you must not drive values that are more than the half of the programmed values of Tx Queue depth. For example, if the Tx Queue size is programed to be 4K and data width is 64, the Tx Queue depth is equal to 512. Therefore, you must not drive any value which requests more than 256 beats transfer, that is, you must not drive any value which is more than 255. **Note:** While ati_q0_pbl_i is always present, the ati_q1_pbl_i to ati_q15_pbl_i signals are present depending upon the number of Tx

Table 12-18 Application Transmit Interface (ATI) Signals (continued)

Port Name	I/O	Description
		queues selected while configuring the controller. The number of beats for each value of ati_qN_pbl_i is as follows: ■ 0: 1 ■ 1: 2 ■ ... ■ N: N+1 **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ati_txwatermark_o[(DWCXG_NUM_TXQ-1):0]	O	Transmit Watermark. This signal is available for each Tx queue. When this signal is high, it indicates that the corresponding Tx queue in MTL has enough space for the number of data beats as indicated by ati_q_pbl_i (in previous clock cycle). **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_err_i	I	Transmit Error. This signal indicates an error in the packet that is being transmitted. The MAC must corrupt the packet with CRC error. The application must drive this signal high along with the ati_val_i and ati_eop_i signals. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_txqueueflush_i[(DWCXG_NUM_TXQ-1):0]	I	TX Queue flush. This signal is available for each Tx queue. A pulse on this signal flushes the contents of the queue. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-18 Application Transmit Interface (ATI) Signals (continued)

Port Name	I/O	Description
ati_txqueueflush_ack_o[(DWCXG_NUM_TXQ-1):0]	O	Acknowledgement for TX Queue flush. This signal is available for each Tx queue. A pulse on this signal indicates that the transmit queue flush request through the ati_txqueueflush_i is completed. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_txstatus_val_o[(DWCXG_NUM_TXQ-1):0]	O	Valid Status Available on the Transmit Queue. This signal is available for each Tx queue. This signal indicates that the packet status is available for corresponding Tx queue. When this signal is high, the Tx Status can be read by driving the ati_ack_i and ati_txsqnum_i signals high. The ati_txsqnum_i signal indicates the Tx queue number for which the status is being read by the application. This signal is not available when the Disable Transmit Status in MTL option is selected in the XGMAC-MTL configuration. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_TXSTSE **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_txstatus_o[(DWCXG_TX_STSW-1):0]	O	Transmit Status Word of the Selected Queue Indicated through ati_txsqnum_i. This signal indicates the packet status of the Tx queue. This is valid only when the ati_txstatus_ack_i signal is high for the Tx queue indicated through the ati_txsqnum_i signal. This signal is not available when the Disable Transmit Status in MTL option is selected in the XGMAC-MTL configuration. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_TXSTSE **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-18 Application Transmit Interface (ATI) Signals (continued)

Port Name	I/O	Description
ati_txstatus_ack_i	I	Acknowledgement for the Tx Status. The application drives this signal high to accept the valid (indicated by ati_txstatus_val_o) Tx status available on ati_txstatus_o signal. When multiple Tx queues are present in your configuration, the ati_txsqnum_i signal indicates the acknowledged queue number. This signal is not available when the Disable Transmit Status in MTL option is selected in XGMAC-MTL configuration. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_TXSTSE **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_txsqnum_i[(DWCXG_TXQW-1):0]	I	Queue Selection for Transmit Status Read. This signal is used to select the Tx queue number to read the status when the ati_txstatus_ack_i signal is high. This signal is not present when you have only one Tx Queue. This signal is not available when the Disable Transmit Status in MTL option is selected in the XGMAC-MTL configuration. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_TXSTSE&&DWCXG_NUM_TXQ!=1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ati_datatxon_o	O	Packet Transmission Indicator. This signal indicates that the packet transmission is ongoing on the line. **Exists:** !DWCXG_CORE&&DWCXG_FPE **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-18 Application Transmit Interface (ATI) Signals (continued)

Port Name	I/O	Description
ati_pmac_txon_o	O	Preemption Fragment Transmission Indicator. This signal indicates that the Preemption fragment transmission is ongoing on the line. This signal combined with ati_datatxon_o indicates following: {ati_datatxon_o,ati_pmac_txon_o} 2'b0X: No transmission 2'b10: eMAC is transmitting 2'b11: pMAC is transmitting **Exists:** !DWCXG_CORE&&DWCXG_FPE **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ati_pmac_actv_o	O	Preemption Packet Transmission Indicator. This signal indicates that the Preemption packet transmission is ongoing on the line. High when pMAC starts the first segment and becomes low when the pMAC finishes the last segment. This indicates that pMAC is activated for a preemption packet. Note that this signal will remain high during IPG between preempt & express packets or when the pMAC transmission is preempted by eMAC i.e., it will be high from start of first fragment to end of final fragment for a preemption packet, irrespective of IPG and express packet transmission. **Exists:** !DWCXG_CORE&&DWCXG_FPE **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.19 Application Receive Interface (ARI) Signals

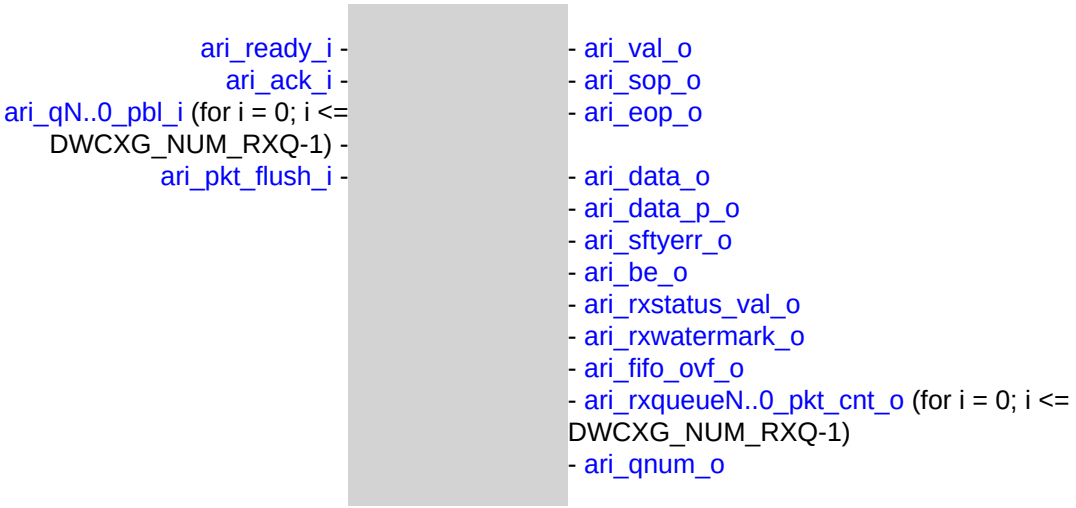


Table 12-19 Application Receive Interface (ARI) Signals

Port Name	I/O	Description
ari_val_o	O	ARI Valid. When this signal is high, it qualifies the data and other control signals the MTL receive application interface. In configurations with multiple queues, this signal corresponds to the currently-selected queue number indicated by the ari_qnum_o signal. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ari_ready_i[(DWCXG_NUM_RXQ-1):0]	I	Application Queue Ready Signal. This signal is available for each Rx queue. The application drives this signal high when it is ready to accept a single burst of data transfer (in PBL based arbitration) or a complete packet (in frame based arbitration) the corresponding Rx queue. The burst length is as indicated by the application through ari_q_pbl_i signal. **Note:** This signal is not present in configurations with only one Rx queue. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_NUM_RXQ>1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-19 Application Receive Interface (ARI) Signals (continued)

Port Name	I/O	Description
ari_sop_o	O	Start of Packet. This signal indicates that the current data transferred to the application corresponds to the first data of the packet. In configurations with multiple queues, this signal corresponds to the currently-selected queue number indicated by the ari_qnum_o signal. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ari_eop_o	O	End of Packet. This signal indicates that the current data transferred to the application corresponds to the last data of the packet. In configurations with multiple queues, this signal corresponds to the currently-selected queue number indicated by the ari_qnum_o signal. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ari_data_o[(DWCXG_DATAWIDTH-1):0]	O	Packet Data. This is the packet data from the Rx Queue to the application. In little-endian format, the LSB lane ari_data_o[7:0] is the first byte received by DWC_25gmac. In big-endian format, the MSB lane ari_data_o[DW-1:DW-8] is the first byte received by DWC_25gmac. In configurations with multiple queues, this signal corresponds to the currently-selected queue number indicated by the ari_qnum_o signal. This signal gives the status when the ari_rxstatus_val_i signal is high. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-19 Application Receive Interface (ARI) Signals (continued)

Port Name	I/O	Description
ari_data_p_o[(DWCXG_DPW-1):0]	O	Parity of Packet Data. This is the packet data parity from the Rx Queue to the application. ■ Little-endian: The LSB lane ari_data_p_o[0] is the first byte received by the DWC_25gmac. ■ Big-endian: The MSB lane ari_data_p_o[DWCXG_DATAWIDTH/8 - 1 : DWCXG_DATAWIDTH/8 - 8] is the first byte received by DWC_25gmac. In configurations with multiple queues, this signal corresponds to the currently-selected queue number indicated by the ari_qnum_o signal. This signal gives the status when the ari_rxstatus_val_i signal is high. **Exists:** DWCXG_MTL_SUBSYS&&(DWCXG_ASP_PPE==1) **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ari_sftyerr_o	O	ARI Safety Error This is a pulse or level signal. Its high value indicates that there is safety error for the respective current packet data word, which is being read by the application. This signal must be valid when ari_val_o, ari_rxstatus_val_o, or ari_timestamp_val_o is set. For a specific packet, this signal can be asserted multiple times as per the detected safety error. Normally IP drops such a safety erroneous packet. But sometimes, full or partial packet can be forwarded to the application. If required, application can initiate packet flush request in such cases. For some cases Rx status indicates the error status to be set. **Exists:** DWCXG_MTL_SUBSYS&&(DWCXG_ASP_ALL||DWCXG_ASP_ECC) &&SNPS_RSVDPARAM_33 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-19 Application Receive Interface (ARI) Signals (continued)

Port Name	I/O	Description
ari_be_o[(DWCXG_BEW-1):0]	O	Number of Valid Byte Lanes. This signal indicates the number of valid byte lanes in the last data phase transfer (when the ari_eop_o signal is high) the MTL receive application interface. In configurations with multiple queues, this signal corresponds to the currently-selected queue number indicated by the ari_qnum_o signal. This signal is valid only when the ari_val_o and ari_eop_o signals are high. When the ari_eop_o signal is low, the ari_be_o signal is always driven with all-ones by the MTL. When the width of this signal is 3, the following values indicate the lanes that have the data:. ■ 111: All 8 byte lanes have data ■ 110: LS 7 byte lanes have data ■ ... ■ 001: LS 2 byte lanes have data ■ 000: LS 1 byte lane have data **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ari_rxstatus_val_o	O	Valid Receive Status. This signal indicates that the ari_data_o bus contains the Receive status to the application. In configurations with multiple queues, this signal corresponds to the currently-selected queue number indicated by the ari_qnum_o signal. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ari_ack_i	I	ARI acknowledgement. This is the acknowledgement signal from the application. When the ari_val_o or ari_rxstatus_val_o signal is high, this signal indicates that the application has accepted the current data or status the ari_data_o signal. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ari_qN..0_pbl_i[(DWCXG_ARI_PBL_WIDTH-1):0] (for i = 0; i <= DWCXG_NUM_RXQ-1)	I	Number of Beats Requested by the Application for Queue. This signal indicates the number of beats of the data requested by application from the MTL Rx queue. For configurations with multiple Rx Queues, you must not drive values that are more than the half of

Table 12-19 Application Receive Interface (ARI) Signals (continued)

Port Name	I/O	Description
		the programmed values of Rx queue depth. For example, if the Rx Queue size is programed to be 4K and Data width is 64, the Rx Queue depth is equal to 512. Therefore, you must not drive any value which requests more than 256 beats transfer, that is, you must not drive any value which is more than 255. The number of beats for each value of ari_qN.0_pbl_i is as follows:. ■ 0: 1 ■ 1: 2 ■ ... ■ N: N+1 **Note:** The ari_q0_pbl_i signal is not present in configurations with only one Rx queue. The ari_q1_pbl_i to ari_q15_pbl_i signals are present depending upon the number of Rx queues selected while configuring the controller. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ari_rxwatermark_o[(DWCXG_NUM_RXQ-1):0]	O	Receive Watermark. This signal is available for each Rx queue. When this signal is high, it indicates that the corresponding Rx queue in the MTL has enough data to generate the number of beats as requested through ari_q_pbl_i (in the previous clock cycle). **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ari_fifo_ovf_o[(DWCXG_NUM_RXQ-1):0]	O	Rx Queue Overflow. This signal is available for all Rx queues. This indicates that the Rx packet is lost because of overflow in the corresponding Rx queue. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-19 Application Receive Interface (ARI) Signals (continued)

Port Name	I/O	Description
ari_pkt_flush_i[(DWCXG_NUM_RXQ-1):0]	I	Packet Flush Request. This signal is available for all Rx queues. This signal indicates the packet flush request from the application to flush the packets at the head of the corresponding Rx queue. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
ari_rxqueueN..0_pkt_cnt_o[(RFCW-1):0] (for i = 0; i <= DWCXG_NUM_RXQ-1)	O	Rx Queue Packets Counter in Rx Queue. This status signal provides the number of packets present in the Rx queue. The width of the counter depends on the maximum number of packets that can be stored in the Rx queue. For example, if the total Rx memory size is 4,096 bytes and the minimum packet size is 15 bytes, the maximum number of packets that can be stored in the Rx Queue is 256. Therefore, the width RXPCW = 8. **Note:** While ari_rxqueue0_pkt_cnt_o is always present, the ari_rxqueue1_pkt_cnt_o to ari_rxqueue15_pkt_cnt_o signals are present depending upon the number of queues selected while configuring the controller. **Exists:** DWCXG == 1 **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
ari_qnum_o[(QNUMW-1):0]	O	Currently Selected Rx Queue Number. This signal indicates the queue that is selected by the Rx Arbiter in the MTL for current burst/packet transfer. This signal qualifies the following signals: ■ ari_val_o ■ ari_sop_o ■ ari_eop_o ■ ari_data_o ■ ari_be_o ■ ari_rxstatus_val_o **Note:** This signal is not present in configurations with only one Rx queue. **Exists:** DWCXG_MTL_SUBSYS&&DWCXG_NUM_RXQ>1 **Synchronous To:** clk_app_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.20 Two Port SRAM Interface Signals

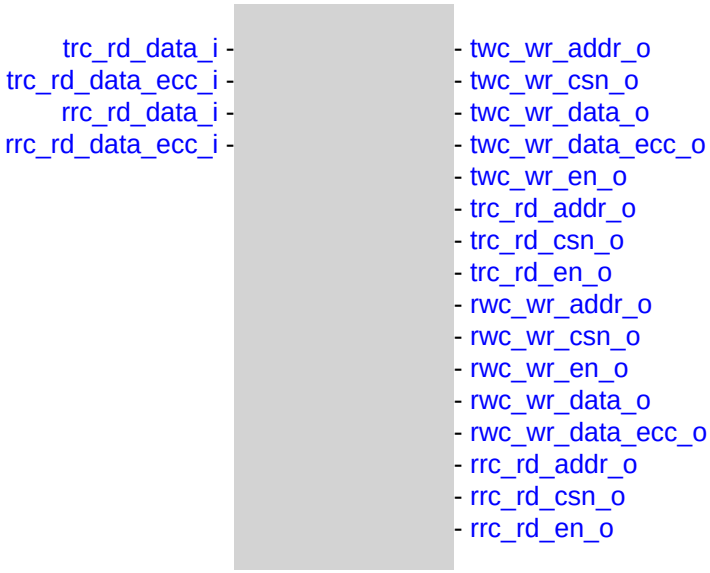


Table 12-20 Two Port SRAM Interface Signals

Port Name	I/O	Description
twc_wr_addr_o[(DWCXG_TX_FIFO_PTR_WIDTH-1):0]	O	Transmit FIFO Write Address. This signal gives the address to the Tx FIFO write port. It is valid when the twc_wr_csn_o signal is high. The address width is displayed in coreConsultant after you configure the Tx FIFO size. The MTL Tx FIFO Address Bus Width field of the Buffer Management dialog box displays the address width. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
twc_wr_csn_o	O	Transmit FIFO Write Port Chip Select. This signal qualifies the twc_wr_addr_o, twc_wr_data_o, and twc_wr_en_o signals. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low

Table 12-20 Two Port SRAM Interface Signals (continued)

Port Name	I/O	Description
twc_wr_data_o[(DWCXG_MEMW-1):0]	O	Transmit FIFO Write Data. This bus carries the data to be written in the Tx FIFO. It is valid when the twc_wr_csn_o and twc_wr_en_o signals are high. The width of this data bus depends on the data bus width selected during configuration. The additional bits (4 or 5 in 64-bit or 128-bit data) are used for storing the control information (byte enable, SOP, EOP, and so on). **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
twc_wr_data_ecc_o[(DWCXG_TX_ECW-1):0]	O	Transmit FIFO ECC Write Data. This bus carries the ECC data (computed on {twc_wr_addr_o,twc_wr_data_o}) to be written in the Tx FIFO. It is valid when the twc_wr_csn_o signal is low and twc_wr_en_o signal is high. The width of this data bus depends on the data bus width and the depth (address width) selected during configuration. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
twc_wr_en_o	O	Transmit FIFO Write Enable. When this signal is high, it indicates that the twc_wr_data_o data must be written to the memory. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-20 Two Port SRAM Interface Signals (continued)

Port Name	I/O	Description
trc_rd_addr_o[(DWCXG_TX_FIFO_PTR_WIDTH-1):0]	O	Transmit FIFO Read Address. This bus gives the address to the Tx FIFO read port. It is valid when the trc_rd_csn_o signal is high. The width of address bus depends on the Tx FIFO depth and the data bus width. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
trc_rd_csn_o	O	Transmit FIFO Read Port Chip Select. This signal qualifies the trc_rd_addr_o signal. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
trc_rd_data_i[(DWCXG_MEMW-1):0]	I	Transmit FIFO Read Data. This bus carries the data read from the Tx FIFO. It is valid when the trc_rd_en_o signal is high. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
trc_rd_data_ecc_i[(DWCXG_TX_ECW-1):0]	I	Transmit FIFO ECC Read Data. This bus carries the ECC data read from the Tx FIFO. It is valid when the trc_rd_en_o signal is high. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM&&DWCXG_ASP_ECC **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-20 Two Port SRAM Interface Signals (continued)

Port Name	I/O	Description
trc_rd_en_o	O	Transmit FIFO Read Enable. When this signal is high, it indicates that the trc_rd_data_i signal must contain valid data. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_tx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1&&DWCXG_ASYNC_RAM==0) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rwc_wr_addr_o[(DWCXG_RX_FIFO_PTR_WIDTH-1):0]	O	Receive FIFO Write Address. This bus gives the address to the Rx FIFO write port. It is valid when the rwc_wr_csn_o signal is high. The address width is displayed in the coreConsultant after you configure the Rx FIFO size. The MTL Rx FIFO Address Bus Width field of the Buffer Management dialog box displays the address width. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_rx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rwc_wr_csn_o	O	Receive FIFO Write Port Chip Select. This signal qualifies the rwc_wr_addr_o, rwc_wr_data_o, and rwc_wr_en_o signals. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_rx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rwc_wr_en_o	O	Receive FIFO Write Enable. When this signal is high, it indicates that the rwc_wr_data_o signal must be written into the memory. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_rx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-20 Two Port SRAM Interface Signals (continued)

Port Name	I/O	Description
rwc_wr_data_o[(DWCXG_MEMW-1):0]	O	Receive FIFO Write Data. This bus carries the data to be written in the Rx FIFO. It is valid when the rwc_wr_csn_o and rwc_wr_en_o signals are high. The width of the FIFO write data bus depends on the data bus width selected while configuring the controller. The additional bits (4 or 5 in 64-bit or 128-bit data) are used for storing the control information (byte enable, SOP, EOP, and so on). **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_rx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rwc_wr_data_ecc_o[(DWCXG_RX_ECW-1):0]	O	Receive FIFO ECC Write Data. This bus carries the ECC data (computed on {rwc_wr_addr_o,rwc_wr_data_o}) to be written in the Rx FIFO. It is valid when the rwc_wr_csn_o is low and rwc_wr_en_o signals are high. The width of the FIFO write data bus depends on the data bus width and FIFO depth (address width) selected while configuring the core. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM&&DWCXG_ASP_ECC **Synchronous To:** DWCXG_MTL_MEMTYP==0 ? "clk_rx_312pt5_i" : (DWCXG_AXI_SUBSYS==1 ? "aclk_i" : "clk_app_i") **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rrc_rd_addr_o[(DWCXG_RX_FIFO_PTR_WIDTH-1):0]	O	Receive FIFO Read Address. This bus gives the address to the Rx FIFO read port. It is valid when the rrc_rd_csn_o signal is high. The width of address bus depends on the Rx FIFO depth and the data bus width. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** (DWCXG_AXI_SUBSYS==1&&DWCXG_ASYNC_RST==1) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-20 Two Port SRAM Interface Signals (continued)

Port Name	I/O	Description
rrc_rd_csn_o	O	Receive Read Qualification. This signal qualifies the rrc_rd_addr_o signal. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rrc_rd_data_i[(DWCXG_MEMW-1):0]	I	Receive FIFO Read Data. This bus carries the data read from the Rx FIFO. It is valid when the rrc_rd_en_o signal is high. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rrc_rd_data_ecc_i[(DWCXG_RX_ECW-1):0]	I	Receive FIFO ECC Read Data. This bus carries the ECC data read from the Rx FIFO. It is valid when the rrc_rd_en_o signal is high. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rrc_rd_en_o	O	Receive Read Enable. This signal indicates that rrc_rd_data_i signal must contain valid data. **Exists:** !DWCXG_CORE&&!DWCXG_SPRAM **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** (DWCXG_ASYNC_RAM==1||(DWCXG_ASYNC_RST==1&&DWCXG_AXI_SUBSYS==1)) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.21 Single Port SRAM Interface Signals

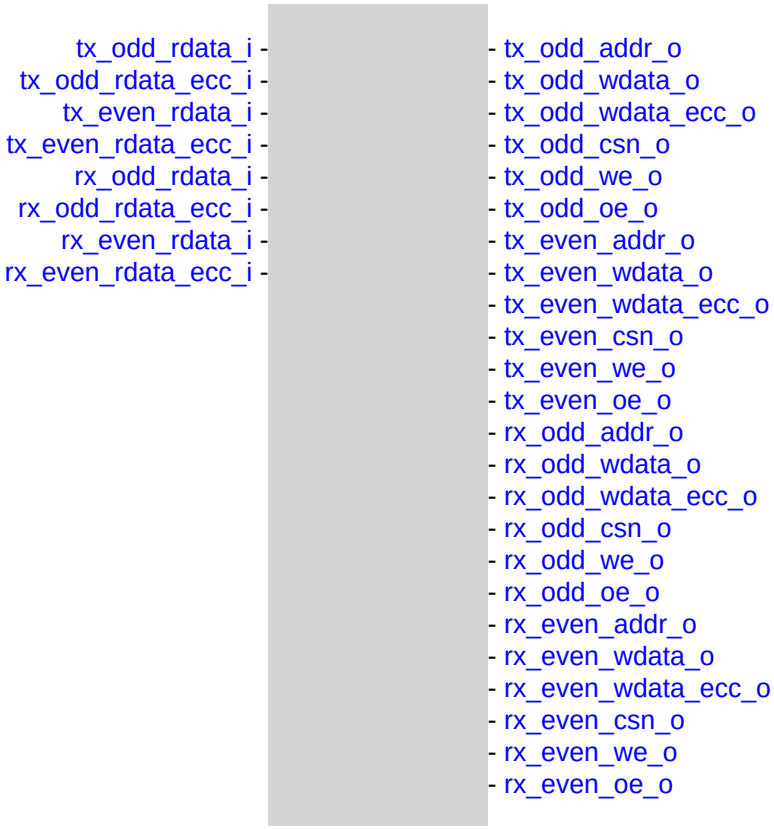


Table 12-21 Single Port SRAM Interface Signals

Port Name	I/O	Description
tx_odd_addr_o[(DWCXG_TX_FIFO_PTR_WIDTH-2):0]	O	Transmit FIFO Address, for odd memory. It is valid when tx_odd_csn_o is low. The address width is displayed in coreConsultant after you configure the Tx FIFO size. The MTL Tx FIFO Address Bus Width field of the Buffer Management dialog box displays the address width. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-21 Single Port SRAM Interface Signals (continued)

Port Name	I/O	Description
tx_odd_wdata_o[(((DWCXG_DATAWIDTH+DWCXG_BEW)+1)-1):0]	O	Transmit FIFO write data, for odd memory. It is valid when tx_odd_csn_o is low and tx_odd_we_o is high. The width of this signal depends on the data width selected during configuration. The additional bits (3, 4, or 5 in 32-bit, 64-bit, or 128-bit data) are used for storing the tag or byte enable bits for the valid data lanes. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tx_odd_wdata_ecc_o[(DWCXG_TX_ECW-1):0]	O	Transmit FIFO ECC write data, for odd memory. It is valid when tx_odd_csn_o is low and tx_odd_we_o is high. The width of this signal depends on the data width and depth of FIFO (address width) selected during configuration. **Exists:** DWCXG_SPRAM&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tx_odd_rdata_i[(((DWCXG_DATAWIDTH+DWCXG_BEW)+1)-1):0]	I	Transmit FIFO read data, for odd memory. It is valid when tx_odd_oe_o is asserted. The width of this signal depends on the data width selected during configuration. The additional bits (3, 4, or 5 in 32-bit, 64-bit, or 128-bit data) are used for storing the tag or byte enable bits for the valid data lanes. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tx_odd_rdata_ecc_i[(DWCXG_TX_ECW-1):0]	I	Transmit FIFO ECC read data, for odd memory. It is valid when tx_odd_oe_o is asserted. The width of this signal depends on the data width and depth of FIFO (address width) selected during configuration. **Exists:** DWCXG_SPRAM&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-21 Single Port SRAM Interface Signals (continued)

Port Name	I/O	Description
tx_odd_csn_o	O	Transmit FIFO chip select, for odd memory. This signal qualifies tx_odd_addr_o and tx_odd_we_o **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
tx_odd_we_o	O	Transmit FIFO write enable for odd memory. This signal controls the read/write operation. Indicates write operation when high and read operation when low. It is valid when tx_odd_csn_o is low. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
tx_odd_oe_o	O	Transmit FIFO output enable, for odd memory. This signal enables the output port of the memory **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
tx_even_addr_o[(DWCXG_TX_FIFO_PTR_WIDTH-2):0]	O	Transmit FIFO Address for even memory. This signal is valid when tx_even_csn_o is low. The address width is displayed in coreConsultant after you configure the Tx FIFO size. The MTL Tx FIFO Address Bus Width field of the Buffer Management dialog box displays the address width. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-21 Single Port SRAM Interface Signals (continued)

Port Name	I/O	Description
tx_even_wdata_o[(((DWCXG_DATAWIDTH+DWCXG_BEW)+1)-1):0]	O	Transmit FIFO write data, for even memory. This signal is valid when tx_even_csn_o is low and tx_even_we_o is high. The width of this signal depends on the data width selected during configuration. The additional bits (3, 4, or 5 in 32-bit, 64-bit, or 128-bit data) are used for storing the tag or byte enable bits for the valid data lanes. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tx_even_wdata_ecc_o[(DWCXG_TX_ECW-1):0]	O	Transmit FIFO ECC write data, for even memory. This signal is valid when tx_even_csn_o is low and tx_even_we_o is high. The width of this signal depends on the data width and depth of the FIFO (address width) selected during configuration. **Exists:** DWCXG_SPRAM&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tx_even_rdata_i[(((DWCXG_DATAWIDTH+DWCXG_BEW)+1)-1):0]	I	Transmit FIFO read data, for even memory. This signal is valid when tx_even_oe_o is asserted. The width of this signal depends on the data width selected during configuration. The additional bits (3, 4, or 5 in 32-bit, 64-bit, or 128-bit data) are used for storing the tag or byte enable bits for the valid data lanes. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tx_even_rdata_ecc_i[(DWCXG_TX_ECW-1):0]	I	Transmit FIFO ECC read data, for even memory. This signal is valid when tx_even_oe_o is asserted. The width of this signal depends on the data width and depth of FIFO (address width) selected during configuration. **Exists:** DWCXG_SPRAM&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-21 Single Port SRAM Interface Signals (continued)

Port Name	I/O	Description
tx_even_csn_o	O	Transmit FIFO chip select, for even memory. This signal qualifies tx_even_addr_o and tx_even_we_o. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
tx_even_we_o	O	Transmit FIFO write enable for even memory. This signal controls the read/write operation. ■ High: Write operation ■ Low: Read operation This signal is valid when tx_even_csn_o is low. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
tx_even_oe_o	O	Transmit FIFO output enable for even memory. This signal enables the output port of the memory. **Exists:** DWCXG_MTL_MEMTYP==2 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rx_odd_addr_o[(DWCXG_RX_FIFO_PTR_WIDTH-2):0]	O	Receive FIFO Address, for odd memory. It is valid when rx_odd_csn_o is low. The address width is displayed in coreConsultant after you configure the Rx FIFO size. The MTL Rx FIFO Address Bus Width field of the Buffer Management dialog box displays the address width. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-21 Single Port SRAM Interface Signals (continued)

Port Name	I/O	Description
rx_odd_wdata_o[(((DWCXG_DATAWIDTH+DWCXG_BEW)+1)-1):0]	O	Receive FIFO write data, for odd memory. It is valid when rx_odd_csn_o is low and rx_odd_we_o is high. The width of this signal depends on the data width selected during configuration. The additional bits (3, 4, or 5 in 32-bit, 64-bit, or 128-bit data) are used for storing the tag or byte enable bits for the valid data lanes. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rx_odd_wdata_ecc_o[(DWCXG_RX_ECW-1):0]	O	Receive FIFO ECC write data, for odd memory. It is valid when rx_odd_csn_o is low and rx_odd_we_o is high. The width of this signal depends on the data width and depth of FIFO (address width) selected during configuration. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rx_odd_rdata_i[(((DWCXG_DATAWIDTH+DWCXG_BEW)+1)-1):0]	I	Receive FIFO read data, for odd memory. It is valid when rx_odd_oe_o is asserted. The width of this signal depends on the data width selected during configuration. The additional bits (3, 4, or 5 in 32-bit, 64-bit, or 128-bit data) are used for storing the tag or byte enable bits for the valid data lanes. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rx_odd_rdata_ecc_i[(DWCXG_RX_ECW-1):0]	I	Receive FIFO ECC read data, for odd memory. It is valid when rx_odd_oe_o is asserted. The width of this signal depends on the data width and depth (address width) selected during configuration. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-21 Single Port SRAM Interface Signals (continued)

Port Name	I/O	Description
rx_odd_csn_o	O	Receive FIFO chip select, for odd memory. This signal qualifies rx_odd_addr_o and rx_odd_we_o **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rx_odd_we_o	O	Receive FIFO write enable, for odd memory. This signal controls the read/write operation. Indicates write operation when high and read operation when low. It is valid when rx_odd_csn_o is low. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rx_odd_oe_o	O	Receive FIFO output enable, for odd memory. This signal enables the output port of the memory **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rx_even_addr_o[(DWCXG_RX_FIFO_PTR_WIDTH-2):0]	O	Receive FIFO Address, for even memory. It is valid when rx_even_csn_o is low. The address width is displayed in coreConsultant after you configure the Rx FIFO size. The MTL Rx FIFO Address Bus Width field of the Buffer Management dialog box displays the address width. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-21 Single Port SRAM Interface Signals (continued)

Port Name	I/O	Description
rx_even_wdata_o[(((DWCXG_DATAWIDTH+DWCXG_BEW)+1)-1):0]	O	Receive FIFO write data, for even memory. It is valid when rx_even_csn_o is low and rx_even_we_o is high. The width of this signal depends on the data width selected during configuration. The additional bits (3, 4, or 5 in 32-bit, 64-bit, or 128-bit data) are used for storing the tag or byte enable bits for the valid data lanes. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rx_even_wdata_ecc_o[(DWCXG_RX_ECW-1):0]	O	Receive FIFO ECC write data, for even memory. It is valid when rx_even_csn_o is low and rx_even_we_o is high. The width of this signal depends on the data width and depth of FIFO (address width) selected during configuration. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rx_even_rdata_i[(((DWCXG_DATAWIDTH+DWCXG_BEW)+1)-1):0]	I	Receive FIFO read data, for even memory. It is valid when rx_even_oe_o is asserted. The width of this signal depends on the data width selected during configuration. The additional bits (3, 4, or 5 in 32-bit, 64-bit, or 128-bit data) are used for storing the tag or byte enable bits for the valid data lanes. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rx_even_rdata_ecc_i[(DWCXG_RX_ECW-1):0]	I	Receive FIFO ECC read data, for even memory. It is valid when rx_even_oe_o is asserted. The width of this signal depends on the data width and depth of FIFO (address width) selected during configuration. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE&&DWCXG_ASP_ECC **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-21 Single Port SRAM Interface Signals (continued)

Port Name	I/O	Description
rx_even_csn_o	O	Receive FIFO chip select, for even memory. This signal qualifies rx_even_addr_o and rx_even_we_o **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rx_even_we_o	O	Receive FIFO write enable, for even memory. This signal controls the read/write operation. Indicates write operation when high and read operation when low. It is valid when rx_even_csn_o is low. **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rx_even_oe_o	O	Receive FIFO output enable, for even memory. This signal enables the output port of the memory **Exists:** DWCXG_SPRAM&&!DWCXG_CORE **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.22 MAC Tx Interface Signals

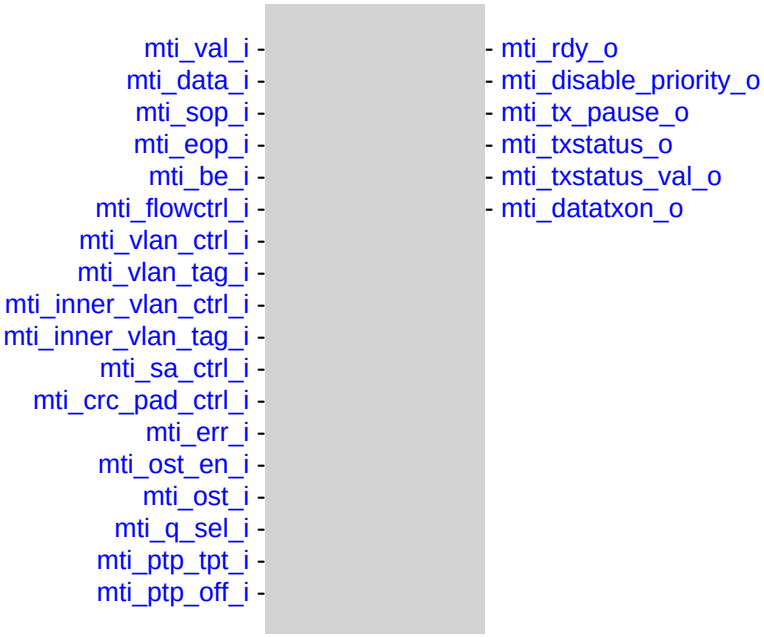


Table 12-22 MAC Tx Interface Signals

Port Name	I/O	Description
mti_val_i	I	Transmit Data Valid. The application drives this signal high to indicate to the MAC that data is available for transmission the mti_data_i bus. This signal also qualifies the validity of mti_be_i, mti_sop_i, and mti_eop_i signals. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_data_i[(DWCXG_DATAWIDTH-1):0]	I	Transmit Data Bus. The data for transmission is input to the MAC this bus. You can configure the MAC to process the data in the little-endian or big-endian format. In the little-endian format, the LSB lane mti_data_i[7:0] is the first byte transmitted by the MAC. In the big-endian format, the MSB lane mti_data_i[DWC_XGMAC_DATAWIDTH ' 1: DWC_XGMAC_ DATAWIDTH ' 8] is the first byte transmitted by the MAC. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-22 MAC Tx Interface Signals (continued)

Port Name	I/O	Description
mti_sop_i	I	Transmit Start of Packet. This signal indicates that the start of an Ethernet packet is being transferred to the MAC. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_eop_i	I	Transmit End of Packet. This signal indicates that the end of an Ethernet packet is being transferred to the MAC. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_be_i[((DWCXG_DATAWIDTH/64)+1) :0]	I	Transmit Byte Lanes. This signal indicates the number of byte lanes valid the mti_data_i bus when the mti_eop_i signal is high. Total number of valid bytes = mti_be_i bytes + 1 byte When the mti_eop_i signal is low, the MAC always considers all byte lanes to have valid data. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mti_rdy_o	O	Transmit Data Ready. When this signal is high, it indicates that the MAC is ready to accept the data input the mti_data_i bus. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-22 MAC Tx Interface Signals (continued)

Port Name	I/O	Description
mti_flowctrl_i[(DWCXG_NTXFC-1):0]	I	Transmit Flow Control. When this signal is high, it instructs the MAC to transmit the Pause packets in the full-duplex mode. When Pause Flow Control (PFC) is enabled, the corresponding bit indicates the MAC to transmit PFC packet with pause quanta for programmed priorities of the corresponding Rx queue. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_vlan_ctrl_i[1:0]	I	VLAN Insertion Control. This encoded signal requests the MAC to perform VLAN tagging or untagging before transmitting the packets. If the packet is modified for VLAN tags, the MAC automatically recalculates and replaces the CRC bytes. This signal is sampled only when the mti_sop_i signal is high. The following list describes the values: ■ 2'b00: Do not add a VLAN tag. ■ 2'b01: Remove the VLAN tag from the packets before transmission. Use this option only with the VLAN packets. ■ 2'b10: Insert a VLAN tag with the tag value programmed in the MAC_VLAN_Incl register. Use this option only with the VLAN packets. ■ 2'b11: Replace the VLAN tag in packets with the Tag value programmed in the MAC_VLAN_Incl register. Use this option only with the VLAN packets. This signal is available in the XGMAC-CORE configuration when the Enable SA and VLAN Insertion Tx option is selected. **Exists:** DWCXG_SA_VLAN_INS_CTRL_EN&&DWCXG_CORE&&!SNPS_RSVDPARAM_49 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-22 MAC Tx Interface Signals (continued)

Port Name	I/O	Description
mti_vlan_tag_i[15:0]	I	VLAN Tag for Insertion. This signal contains the VLAN Tag for Insertion or Replacement in the Transmit packet based the mti_vlan_ctrl_i signal. This signal is available only in XGMAC-CORE configurations. **Exists:** DWCXG_SA_VLAN_INS_CTRL_EN&&DWCXG_CORE&&!SNPS_RSVDPARAM_49 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mti_inner_vlan_ctrl_i[1:0]	I	Inner VLAN Insertion Control. This encoded signal requests the MAC to perform inner VLAN tagging or untagging before transmitting the packets. The application must set appropriate mti_crc_pad_ctrl_i value when this control is enabled. This signal is sampled only when the mti_sop_i signal is asserted. The following list describes the values: ■ 2'b00: Do not add a inner VLAN tag. ■ 2'b01: Remove the inner VLAN tag from the packet before transmission. Use this option only with the Double VLAN packets. ■ 2'b10: Insert a inner VLAN tag with the tag value programmed in the MAC_Inner_VLAN_Incl register. Use this option only with the VLAN packets. ■ 2'b11: Replace the inner VLAN tag in packets with the tag value programmed in the MAC_Inner_VLAN_Incl register. Use this option only with the Double VLAN packets. This signal is available in the XGMAC-CORE configuration when the Enable SA and VLAN Insertion Tx and Enable Double VLAN Processing options are selected. **Exists:** DWCXG_SA_VLAN_INS_CTRL_EN&&DWCXG_DOUBLE_VLAN_EN&&DWCXG_CORE&&!SNPS_RSVDPARAM_49 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-22 MAC Tx Interface Signals (continued)

Port Name	I/O	Description
mti_inner_vlan_tag_i[15:0]	I	Inner VLAN Tag for Insertion. This signal contains the inner VLAN Tag for insertion or replacement in the Transmit packet based the mti_inner_vlan_ctrl_i signal. This signal is available only in the XGMAC-CORE configurations. **Exists:** DWCXG_SA_VLAN_INS_CTRL_EN&&DWCXG_DOUBLE_VLAN_EN&&DWCXG_CORE&&!SNPS_RSVDPARAM_49 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mti_sa_ctrl_i[2:0]	I	Source Address Insertion Control. This encoded signal requests the MAC to add or replace the Source Address field in the Ethernet packet with the value given in the MAC Address 0 register. This signal is sampled only when the mti_sop_i signal is high. If the Source Address field is modified in a packet, the MAC automatically recalculates and replaces the CRC bytes. Bit 2 specifies the MAC Address Register (0 or 1) that is used for Source Address insertion or replacement. The following list describes the values of Bits[1:0]:. ■ 2'b00: Do not include the source address. ■ 2'b01: Include or insert the source address. For reliable transmission, the application must provide packets without source addresses. ■ 2'b10: Replace the source address. For reliable transmission, the application must provide packets with source addresses. ■ 2'b11: Reserved. This signal is available in the XGMAC-CORE configuration when the Enable SA and VLAN Insertion Tx option is selected. **Exists:** DWCXG_SA_VLAN_INS_CTRL_EN&&DWCXG_CORE&&!SNPS_RSVDPARAM_49 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-22 MAC Tx Interface Signals (continued)

Port Name	I/O	Description
mti_disable_priority_o[(DWCXG_DISPRTY_W-1):0]	O	MTI Disable Priority. This signal indicates the pause state of a particular priority. Signal corresponding to a priority is asserted when a PFC packet containing nonzero pause time value for one or more priorities is received. **Exists:** DWCXG_PFC_EN **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_tx_pause_o	O	MTI Transmitter Paused. This signal indicates that the transmitter is paused due to reception of 802.3-2018 Pause packet containing nonzero pause time value. **Exists:** Always **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_crc_pad_ctrl_i[1:0]	I	Transmit Cyclic Redundancy Check (CRC) and PAD Control. This signal controls the insertion of the FCS bytes and PAD bits in the packet being transmitted by the MAC. This signal is sampled only when the mti_sop_i signal is high. The MAC decodes the signal as follows:. ■ 2'b00: CRC and Pad Insertion The MAC appends CRC at the end of transmitted packet having length greater than or equal to 60 bytes. The MAC automatically adds padding and CRC to a packet having length less than 60 bytes. ■ 2'b01: CRC Insertion (Disable Pad Insertion) The MAC appends CRC to the end of the transmitted packet, but it does not append padding. The application must ensure that the padding bytes are present in the packet being transferred. ■ 2'b10: Disable CRC Insertion The MAC does not append the CRC to the end of the transmitted packet. The application must ensure that the padding and CRC bytes are present in the packet being transferred. ■ 2'b11: CRC Replacement The MAC replaces the last four bytes of the transmitted packet with recalculated CRC bytes. The application must ensure that the padding and CRC bytes are present in the packet being transferred. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No

Table 12-22 MAC Tx Interface Signals (continued)

Port Name	I/O	Description
		Power Domain: {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mti_err_i	I	Transmit Error Input. When this signal is high, it indicates to the MAC to corrupt the packet currently being transmitted by forcing the CRC error. This signal is sampled along with the mti_eop_i signal. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_ost_en_i	I	Transmit Enable One-Step Time Correction. When this signal is high, it directs the MAC to perform one-step time correction for the Transmit packet. It also validates the mti_ost_i signal. This signal is available only when the Enable One-Step Timestamp Feature option is selected. The application must drive this signal high only for those PTP packets that require one-step time correction. This signal is sampled only when the mti_sop_i and mti_val_i signals are high. **Exists:** DWCXG_OST_EN&&DWCXG_CORE **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_ost_i[63:0]	I	Transmit One-Step Time Correction Input. This bus contains the timestamp required for one-step time correction. This is validated by the mti_ost_en_i signal. This signal is available only when the Enable One-Step Timestamp Feature option is selected. **Exists:** DWCXG_OST_EN&&DWCXG_CORE **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-22 MAC Tx Interface Signals (continued)

Port Name	I/O	Description
mti_q_sel_i[(DWCXG_TXQW-1):0]	I	Transmit Queue Select. This signal indicates the Tx queue from which the transmit packet arrives to MAC transmitter. This signal is sampled only when the mti_sop_i and mti_val_i signals are high and is valid when mti_ost_en_i is high. This signal is available only when Enable one step timestamp for PTP over UDP/IP feature option is selected. **Exists:** DWCXG_CBTI_EN&&DWCXG_CORE **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mti_ptp_tpt_i[1:0]	I	Transmit PTP transport protocol type. This signal indicates the type of transport protocol over which PTP messages are sent. The following list describes the values of these bits: ■ 2'b00: Reserved ■ 2'b01: PTP over Ethernet ■ 2'b10: PTP over UDP/IPv4 ■ 2'b11: PTP over UDP/IPv6 This signal is sampled only when the mti_sop_i and mti_val_i signals are high and is valid when mti_ost_en_i is high. This signal is available only when Enable one step timestamp for PTP over UDP/IP feature option is selected. **Exists:** DWCXG_POU_OST_EN&&DWCXG_CORE **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-22 MAC Tx Interface Signals (continued)

Port Name	I/O	Description
mti_ptp_off_i[13:0]	I	Transmit PTP offset. This signal indicates the starting byte location of the PTP message, for the packet sent on the MTI interface. This signal is sampled only when the signals mti_sop_i and mti_val_i are high and is valid when mti_ost_en_i is high and mti_ptp_tpt_i is 10 or 11. For example, for PTP message sent over UDP/IPv4 over untagged Ethernet packet, the offset is 42. This signal is available only when Enable one step timestamp for PTP over UDP/IP feature option is selected. **Exists:** DWCXG_POU_OST_EN&&DWCXG_CORE **Synchronous To:** clk_tx_312pt5_i **Registered:** yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mti_txstatus_o[31:0]	O	Transmit Status. This bus gives the transmission status of the last Ethernet packet sent to the MAC when the mti_txstatus_val_o signal is asserted. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mti_txstatus_val_o	O	Transmit Status Valid. This signal is set high for 1 clock cycle to indicate that mti_txstatus_o gives valid status for the last transmitted packet. **Exists:** DWCXG == 0 **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mti_datatxon_o	O	Transmit AV. This signal indicates that the MAC is transmitting a packet. **Exists:** DWCXG_CORE&&DWCXG_NUM_TXQ>1 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.23 Debug Interface Signals

Table 12-23 Debug Interface Signals

Port Name	I/O	Description
dbg_bus_tx_o[(DWCXG_DBGB_TXW-1):0]	O	Tx Clock Domain Debug Bus. This is the Debug Bus of signals from Tx Clock Domain. **Exists:** SNPS_RSVDPARAM_36 **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
dbg_bus_rx_o[(DWCXG_DBGB_RXW-1):0]	O	Rx Clock Domain Debug Bus. This is the Debug Bus of signals from Rx Clock Domain. **Exists:** SNPS_RSVDPARAM_36 **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
dbg_bus_csr_o[(DWCXG_DBGB_CSRW-1):0]	O	CSR Clock Domain Debug Bus. This is the Debug Bus of signals from CSR Clock Domain. **Exists:** SNPS_RSVDPARAM_36&&(DWCXG_CSR_SLV_CLK||DWCXG_CORE) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-23 Debug Interface Signals (continued)

Port Name	I/O	Description
dbg_bus_app_o[(DWCXG_DBGB_APPW-1):0]	O	Application Clock Domain Debug Bus. This is the Debug Bus of signals from Application Clock Domain. **Exists:** SNPS_RSVDPARAM_36&&DWCXG>0 **Synchronous To:** (DWCXG == 2) ? "aclk_i" : "clk_app_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
dbg_bus_ptp_o[(DWCXG_DBGB_PTPW-1):0]	O	PTP Clock Domain Debug Bus. This is the Debug Bus of signals from PTP Clock Domain. **Exists:** SNPS_RSVDPARAM_36&&DWCXG_TIME_STAMPING **Synchronous To:** clk_ptp_ref_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
dbg_bus_rmii_o[(DWCXG_DBGB_RMIIW-1):0]	O	RMII Clock Domain Debug Bus. This is the Debug Bus of signals from RMII Clock Domain. **Exists:** SNPS_RSVDPARAM_36&&DWCXG_RMII_EN **Synchronous To:** clk_rmii_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.24 MAC Rx Interface Signals

Table 12-24 MAC Rx Interface Signals

Port Name	I/O	Description
mri_val_o	O	Receive Data Valid. The MAC drives this signal high to indicate to the application (MTL) that the received packet data is available the mri_data_o bus. This signal also qualifies the mri_data_o, mri_be_o, mri_sop_o, and mri_eop_o signals. **Exists:** DWCXG == 0 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mri_data_o[(DWCXG_DATAWIDTH-1):0]	O	Receive Data Bus. This signal carries the received packet data from the MAC to the MTL. This signal is valid only when the mri_val_o signal is high. In big-endian format, the MSB lane mri_data_o[DWC_XGMAC_DATAWIDTH'1: DWC_XGMAC_DATAWIDTH ' 8] is the first byte received by DWC_25gmac. In little-endian format, the LSB lane mri_data_o[7:0] is the first byte received by DWC_25gmac. **Exists:** DWCXG == 0 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-24 MAC Rx Interface Signals (continued)

Port Name	I/O	Description
mri_be_o[((DWCXG_DATAWIDTH/64)+1):0]	O	Receive Byte Lanes. This signal indicates the number of byte lanes valid the mri_data_o signal when the mri_eop_o signal is high. The total number of valid bytes is equal to mri_be_o + 1 bytes. The width of this bus depends on the width of the data bus. When the mri_eop_i signal is low, the MAC always drives all-ones the mri_be_o signal. When the mri_be_o signal width is 2, the following values indicate the lanes that have the data: ■ 11: All 4-byte lanes have the data ■ 10: LS 3-byte lane has the data ■ 01: LS 2-byte lane has the data ■ 00: LS 1-byte lane has the data **Exists:** DWCXG == 0 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mri_sop_o	O	Receive Start of Packet. This signal indicates that the start of an Ethernet packet is being transferred from the MAC. **Exists:** DWCXG == 0 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mri_eop_o	O	Receive End of Packet. This signal indicates that the end of an Ethernet packet is being transferred from the MAC. **Exists:** DWCXG == 0 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-24 MAC Rx Interface Signals (continued)

Port Name	I/O	Description
mri_rxstatus_o[(DWCXG_CORE_STSW-1):0]	O	Receive Status. This bus gives the status of the received Ethernet packet from the MAC the mri_data_o signal. This signal is valid only when the mri_eop_o signal is high. The width of this signal is always 96-bits **Exists:** DWCXG == 0 **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mri_q_sel_o[(DWCXG_RXQW-1):0]	O	Receive Queue Select. This signal indicates the Receive queue to which a received packet goes. This output is available only when you select more than one Receive queues. **Exists:** DWCXG_NUM_RXQ>1&&DWCXG_CORE **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.25 Energy Efficient Ethernet Signals

- lpi_intr_o
- sbd_tx_clk_gating_ctrl_o

Table 12-25 Energy Efficient Ethernet Signals

Port Name	I/O	Description
lpi_intr_o	O	LPI Interrupt. This signal is high when the MAC receiver exits the LPI state. **Exists:** DWCXG_EEE_EN **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
sbd_tx_clk_gating_ctrl_o	O	LPI Tx Clock Gating Control. This signal is high after the MAC enters the Tx LPI mode. You can use this signal to control the Tx clock gating. **Exists:** DWCXG_EEE_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.26 General Purpose Signals

Table 12-26 General Purpose Signals

Port Name	I/O	Description
gpo_o[(DWCXG_GOW-1):0]	O	General Purpose Output. The MAC drives this output based the GPO field in the MAC_GPIO_Control register. **Exists:** DWCXG_GPIO_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
gpi_i[(DWCXG_GIW-1):0]	I	General Purpose Input. You can drive this input to generate an interrupt event. **Exists:** DWCXG_GPIO_EN **Synchronous To:** Asynchronous **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.27 APB3 Interface Signals

Table 12-27 APB3 Interface Signals

Port Name	I/O	Description
penable_i	I	APB Enable. This signal completes the Read or Write transaction cycle. **Exists:** DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
psel_i	I	APB Completer Select. This signal indicates the start of transaction and the validity of the control signals and the pwdata_i signal. **Exists:** DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
paddr_i[15:0]	I	APB Address. This signal carries the register address of the CSR module. The address is valid only when the psel_i signal is high. **Note:** paddr_i[13] must be connected to 0. **Exists:** DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-27 APB3 Interface Signals (continued)

Port Name	I/O	Description
pwrite_i	I	APB Read or Write. When this signal is high, it indicates a Write operation. When this signal is low, it indicates a Read operation. **Exists:** DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
pwdata_i[31:0]	I	Write Data. This signal carries the Write data from the application for Control registers of the CSR module. **Exists:** DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
prdata_o[31:0]	O	Read Data. This signal outputs the Read data from the CSR module to the APB bus. Valid data is output 1 clock cycle after the psel_i signal is high. If the psel_i signal continues to be high for burst transfers, valid data is output every 2 clock cycles during this period. **Exists:** DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
pready_o	O	Read Data Ready. This signal indicates that the data the prdata_o signal is valid and can be accepted by the application. **Exists:** DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-27 APB3 Interface Signals (continued)

Port Name	I/O	Description
pslverr_o	O	APB3 Completer Error Response. When this signal is high, it indicates that the application has accessed reserved register offset address. This feature is enabled when SEEN bit in MAC_CSR_SW_Ctrl register is programmed to 1. On detecting uncorrectable safety error on the subordinate path, IP asserts this signal to high (if configured), irrespective of the programming of the SEEN bit. **Exists:** DWCXG_APB3_SLAVE&&DWCXG_SLVERR **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.28 APB Interface Signals

Table 12-28 APB Interface Signals

Port Name	I/O	Description
pwdata_p_i[((4*(DWCXG_ASP_NIB_EN+1))-1):0]	I	APB Write Parity Data. This signal carries the Write Parity data from the application for Control registers of the CSR module. **Exists:** DWCXG_APB_SLAVE&&(DWCXG_ASP_PPE>0) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
prdata_p_o[((4*(DWCXG_ASP_NIB_EN+1))-1):0]	O	APB Parity Read Data. This signal outputs the Parity Read data from the CSR module to the APB bus. When APB Completer is selected, valid data is output 1 clock cycle after the psel_i signal is high. If the psel_i signal continues to be high for burst transfers, valid data is output every 2 clock cycles during this period. When APB3/APB4 Completer is selected, valid data is output with the pready_o signal 2 clock cycles after the psel_i signal is high. **Exists:** DWCXG_APB_SLAVE&&(DWCXG_ASP_PPE>0) **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.29 APB4 Interface Signals

Table 12-29 APB4 Interface Signals

Port Name	I/O	Description
pstrb_i[3:0]	I	APB4 Write strobes. **Exists:** DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
pprot_i[2:0]	I	APB4 Protection control. **Exists:** DWCXG_CSR_PORT == 4 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.30 MAC Control Interface Signals

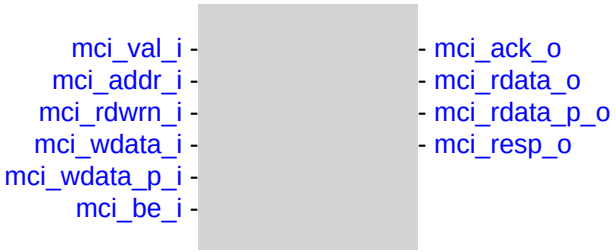


Table 12-30 MAC Control Interface Signals

Port Name	I/O	Description
mci_val_i	I	Data Valid. The application drives this signal high to access the MAC CSR module, indicating to the MAC that the following are valid:. ■ Address ■ Read/Write control ■ Data (for Write operation) **Exists:** DWCXG_MCI_SLAVE||SNPS_RSVDPARAM_24 **Synchronous To:** clk_csr_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
mci_addr_i[(DWCXG_MCI_ADDR_WIDTH-1):0]	I	Address. This signal carries the register address of the CSR from the application. The address is valid only when the mci_val_i signal is high. **Exists:** DWCXG_MCI_SLAVE||SNPS_RSVDPARAM_24 **Synchronous To:** clk_csr_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mci_rdwrn_i	I	Read or Write. This signal provides the Read or Write control. The application writes the data into the MAC Control registers by driving this signal low. When this signal is high, it indicates a Read operation. **Exists:** DWCXG_MCI_SLAVE||SNPS_RSVDPARAM_24 **Synchronous To:** clk_csr_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-30 MAC Control Interface Signals (continued)

Port Name	I/O	Description
mci_wdata_i[31:0]	I	Write Data. This signal carries the Write data from the application to the Control registers of the CSR module. The validity of each byte in this signal is qualified with the corresponding byte enables the mci_be_i signal. **Exists:** DWCXG_MCI_SLAVE||SNPS_RSVDPARAM_24 **Synchronous To:** clk_csr_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mci_wdata_p_i[((4*(DWCXG_ASP_NIB _EN+1))-1):0]	I	MCI Write Data Parity. This signal carries the Write Parity data from the application. The validity of each bit in this signal is qualified with the corresponding byte enables in the mci_be_i signal. **Exists:** DWCXG_MCI_SLAVE&&(DWCXG_ASP_PPE>0) **Synchronous To:** clk_csr_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mci_be_i[3:0]	I	Bytes Enabled. This signal indicates the bytes that are valid the mci_wdata_i signal. This signal is valid when the mci_val_i signal is high. **Exists:** DWCXG_MCI_SLAVE||SNPS_RSVDPARAM_24 **Synchronous To:** clk_csr_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mci_ack_o	O	Transfer Acknowledge. This is an acknowledgement from the MAC Control Interface indicating that the Write data the mci_wdata_i bus has been accepted during CSR write operation and valid read data is available the mci_rdata_o bus during CSR read operation. **Exists:** DWCXG_MCI_SLAVE||SNPS_RSVDPARAM_24 **Synchronous To:** clk_csr_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-30 MAC Control Interface Signals (continued)

Port Name	I/O	Description
mci_rdata_o[31:0]	O	Read Data. This signal carries the Read data from the MAC CSR to the application. The MAC supplies all bytes irrespective of the byte enable signal. **Exists:** DWCXG_MCI_SLAVE||SNPS_RSVDPARAM_24 **Synchronous To:** clk_csr_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mci_rdata_p_o[((4*(DWCXG_ASP_NIB_EN+1))-1):0]	O	MCI Parity Read Data. This signal carries the Parity Read data from the MAC CSR to the application. **Exists:** DWCXG_MCI_SLAVE&&(DWCXG_ASP_PPE>0) **Synchronous To:** clk_csr_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
mci_resp_o	O	MCI Subordinate Error Response. When this signal is high, it indicates that the application has accessed reserved register offset address. This feature is enabled when SEEN bit in MAC_CSR_SW_Ctrl register is programmed to 1. When an uncorrectable safety error is detected on the subordinate path, IP assert this signal to high, irrespective of the programming of the SEEN bit. **Exists:** DWCXG_MCI_SLAVE&&DWCXG_SLVERR&&SNPS_RSVDPARAM_33 **Synchronous To:** clk_csr_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.31 MDIO Interface Signals

Table 12-31 MDIO Interface Signals

Port Name	I/O	Description
sma_mdc_o	O	The DWC_25gmac provides a timing reference for the sma_mdi_i and sma_mdo_o signals through this aperiodic clock. The maximum frequency of this clock is 2.5 MHz. This clock is generated from the application clock through a clock divider. **Exists:** DWCXG_SMA_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sma_mdi_i	I	The PHY generates this signal to transfer register data during a read operation. This signal is driven synchronous to sma_mdc_o. **Exists:** DWCXG_SMA_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
sma_mdo_o	O	The DWC_25gmac uses this signal to transfer control and data information to the PHY. **Exists:** DWCXG_SMA_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-31 MDIO Interface Signals (continued)

Port Name	I/O	Description
sma_mdo_o_e	O	This enable signal drives the sma_mdo_o signal from an external three-state I/O buffer. This signal is asserted when a valid data is driven by the sma_mdo_o signal. **Exists:** DWCXG_SMA_EN **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.32 Operation Speed Signals

- mac_speed_o

Table 12-32 Operation Speed Signals

Port Name	I/O	Description
mac_speed_o[2:0]	O	MAC Operation Speed. This signal indicates the speed of the MAC operation according to the following encoding: ■ 3'b000 10G - XGMII ■ 3'b001 - Reserved ■ 3'b010 2.5G - GMII ■ 3'b011 1G - GMII ■ 3'b100 100M - MII ■ 3'b101 5G - XGMII ■ 3'b110 2.5G - XGMII ■ 3'b111 10M - MII **Exists:** Always **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.33 Loopback Mode Signals

- loopback_mode_o

Table 12-33 Loopback Mode Signals

Port Name	I/O	Description
loopback_mode_o	O	Loopback Test Mode. This signal indicates the Loopback mode of MAC operation, according to the following encoding: ■ 1'b0: Normal mode ■ 1'b1: Loopback mode **Exists:** Always **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.34 GMII Interface Signals

Table 12-34 GMII Interface Signals

Port Name	I/O	Description
gmii_crs_i	I	GMII/MII Carrier Sense. This signal is valid only in the GMII or MII mode. The PHY drives this signal high when the Transmit or Receive medium is not idle. The PHY drives this signal low when both Transmit and Receive mediums are idle. This signal is not synchronous to any clock. **Exists:** (DWCXG_XGMII_GMII==1) **Synchronous To:** Asynchronous **Registered:** DWCXG_MII_SUPPORT&&(DWCXG_RMII_EN||DWCXG_RGMII_EN) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
gmii_col_i	I	GMII/MII Collision. This signal is valid only in the GMII or MII mode. The PHY drives this signal high when a collision is detected on the medium. This signal is not synchronous to any clock. **Exists:** DWCXG_MII_SUPPORT&&!DWCXG_FDUPLX_ONLY **Synchronous To:** Asynchronous **Registered:** DWCXG_MII_SUPPORT&&!DWCXG_FDUPLX_ONLY&&(DWCXG_RMII_EN||DWCXG_RGMII_EN) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
gmii_rxd_i[(DWCXG_GMII_DATA_WIDTH-1):0]	I	GMII/MII Receive Data. This is a bundle of 8/4-bit data signals received from the PHY. All 8/4 bits provided on GMII/MII receive data byte. The validity of the data is qualified with gmii_rxdv_i and gmii_rxer_i. **Exists:** DWCXG_ADD_GMII_PORT **Synchronous To:** clk_rx_312pt5_i **Registered:** DWCXG_FDUPLX_ONLY==0 ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A

Table 12-34 GMII Interface Signals (continued)

Port Name	I/O	Description
gmii_rxdv_i	I	GMII/MII Receive Data Valid. When high, indicates that data the gmii_rxd_i bus is valid. It remains asserted continuously from the first recovered byte of the frame through the final recovered byte. **Exists:** DWCXG_ADD_GMII_PORT **Synchronous To:** clk_rx_312pt5_i **Registered:** DWCXG_FDUPLX_ONLY==0 ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
gmii_rxer_i	I	GMII/MII Receive Error. When high, indicates an error in the received frame data the gmii_rxd_i bus. **Exists:** DWCXG_ADD_GMII_PORT **Synchronous To:** clk_rx_312pt5_i **Registered:** DWCXG_FDUPLX_ONLY==0 ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
gmii_txd_o[(DWCXG_GMII_DATA_WIDTH-1):0]	O	GMII/MII Transmit Data. This is a bundle of eight transmit data signals driven by the GMAC. All 8/4 bits provided on the GMII/MII transmit data byte. The validity of the data is qualified with gmii_txen_o and gmii_txer_o. **Exists:** DWCXG_ADD_GMII_PORT **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
gmii_txen_o	O	GMII/MII Transmit Data Enable. This signal is driven by the GMAC. When high, it indicates that valid data is being transmitted the gmii_txd_o bus. **Exists:** DWCXG_ADD_GMII_PORT **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-34 GMII Interface Signals (continued)

Port Name	I/O	Description
gmii_txer_o	O	GMII/MII Transmit Error. This signal is driven by GMAC. When high, it indicates that error byte is being transmitted the gmii_txd_o bus. **Exists:** DWCXG_ADD_GMII_PORT **Synchronous To:** clk_tx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

12.35 RGMII Interface Signals

Table 12-35 RGMII Interface Signals

Port Name	I/O	Description
phy_intf_sel_i[1:0]	I	This signal selects the RGMII interface and is sampled only during reset assertion, and it is ignored after that. In XGMAC-CORE and XGMAC-MTL (with separate CSR Register Bus Interface Clock) configurations, the rst_clk_csr_n signal is used for sampling. In XGMAC-MTL configuration (without Separate CSR Register Bus Interface Clock) the pwr_on_rst_n signal is used for sampling. In XGMAC-AXI configuration aresetn_i or software initiated reset is used for sampling ■ 00: non-RGMII interface (XGMII/GMII/MII) ■ 01: RGMII interface ■ 10: RMII Interface ■ 11: Reserved **Exists:** DWCXG_RGMII_EN||DWCXG_RMII_EN **Synchronous To:** Asynchronous **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
phy_txen_o	O	PHY Transmit Data Enable. The MAC drives this signal. This signal has multiple functions depending on the selected PHY interface as described in the following list: ■ RMII: When high, indicates that valid data is being transmitted on the phy_txd_o bus. Synchronous to: clk_rmii_i ■ RGMII: This signal is the control signal (rgmii_tctl) for the Transmit data, and it is driven on both edges of the clock. Synchronous to: clk_tx_312pt5_i, clk_tx_312pt5_180_i **Exists:** DWCXG_RGMII_EN||DWCXG_RMII_EN **Synchronous To:** DWCXG_RMII_EN==1 ? "clk_rmii_i,clk_tx_312pt5_i,clk_tx_312pt5_180_i" : "clk_tx_312pt5_i,clk_tx_312pt5_180_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-35 RGMII Interface Signals (continued)

Port Name	I/O	Description
phy_txd_o[3:0]	O	PHY Transmit Data. This is a group of transmit data signals driven by the MAC. It has multiple functions depending on the selected PHY interface as described in the following list: ■ RGMII: Bits[3:0] provide the RGMII transmit data. The data bus changes with both rising and falling edges of the transmit clock (clk_tx_i). The data is valid only when the phy_txen_o signal is high. Synchronous to: clk_tx_312pt5_i and clk_tx_312pt5_180_i ■ RMII: Bits[1:0] provide the RMII transmit data. The data is valid only when the phy_txen_o signal is high. Synchronous to: clk_rmii_i **Exists:** DWCXG_RGMII_EN||DWCXG_RMII_EN **Synchronous To:** DWCXG_RMII_EN==1 ? "clk_rmii_i,clk_tx_312pt5_i,clk_tx_312pt5_180_i" : "clk_tx_312pt5_i,clk_tx_312pt5_180_i" **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
phy_rxd_i[3:0]	I	PHY Receive Data. This is a group of data signals received from the PHY. It has multiple functions depending on the selected PHY interface as described in the following list: ■ RGMII: Bits [3:0] provide the RGMII receive data. The data bus is sampled with both rising and falling edges of the receive clock (clk_rx_312pt5_i). The data is valid only when the phy_rxdv_i signal is high. Synchronous to: clk_rx_312pt5_i, clk_rx_312pt5_180_i ■ RMII: Bits [1:0] provide the RMII receive data. he data is valid only when the phy_rxdv_i signal is high. Synchronous to: clk_rmii_i **Exists:** DWCXG_RGMII_EN||DWCXG_RMII_EN **Synchronous To:** DWCXG_RMII_EN==1 ? "clk_rmii_i,clk_rx_312pt5_i,clk_rx_312pt5_180_i" : "clk_rx_312pt5_i,clk_rx_312pt5_180_i" **Registered:** (DWCXG_RMII_EN && DWCXG_RGMII_EN) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

Table 12-35 RGMII Interface Signals (continued)

Port Name	I/O	Description
phy_rxdv_i	I	PHY Receive Data Valid. The PHY drives this signal. It has multiple functions depending on the selected PHY interface as described in the following list: ■ RGMII: This is the Receive control signal that is used to qualify the data received phy_rxd_i. This signal is sampled both edges of the clock. Synchronous to: clk_rx_312pt5_i, clk_rx_312pt5_180_i ■ RMII: Contains the Carrier Sense and Data Valid information of the Receive interface. Synchronous to: clk_rmii_i **Exists:** DWCXG_RGMII_EN||DWCXG_RMII_EN **Synchronous To:** DWCXG_RMII_EN==1 ? "clk_rmii_i,clk_rx_312pt5_i,clk_rx_312pt5_180_i" : "clk_rx_312pt5_i,clk_rx_312pt5_180_i" **Registered:** (DWCXG_RMII_EN && DWCXG_RGMII_EN) ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

12.36 XGMII Interface Signals

Table 12-36 XGMII Interface Signals

Port Name	I/O	Description
xgmii_rxd_i[(DWCXG_XGMII_DATA_WIDTH-1):0]	I	XGMII Receive Data. This is a bundle of four, 8-bit data signals received from the PHY/XAUI. The data is synchronous to the rising and falling edge of clk_rx_156pt25_i. In DDW mode, a bundle of eight 8-bit signals (64 bits wide) is received at every rising edge of clk_rx_156pt25_i. When GMII/MII pins are shared with XGMII, xgmii_rxd_i[7:0]/xgmii_rxd_i[3:0] takes the function described for gmii_rxd_i in GMII mode. In the signal name, N is the bus width. It is equal to: ■ 32 in SDR or DDR mode ■ 64 in DDW mode **Note:** In DDR mode, this signal can be synchronous to clk_rx_156pt25_180_i or clk_rx_156pt25_i. In SDR mode, this signal is synchronous to clk_rx_312pt5_i. **Exists:** Always **Synchronous To:** DWCXG_XGMII==0 ? "clk_rx_312pt5_i" : "clk_rx_156pt25_i" **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A

Table 12-36 XGMII Interface Signals (continued)

Port Name	I/O	Description
xgmii_rxc_i[(DWCXG_XGMII_CTRL_WIDTH-1):0]	I	XGMII Receive Control. This PHY-driven signal indicates whether data or control characters are being received the XGMIII Receive Data (xgmii_rxd_i) lanes. If any of these bits is asserted, a control character is being received the corresponding XGMII Receive data lane. If any of them is de-asserted, data is being received the corresponding XGMII Receive data lane. When GMII pins are shared with XGMII, xgmii_rxc_i[1:0] takes the function described for gmii_rxer_i and gmii_rxdv_i in GMII mode. This signal is: ■ 4-bit wide in SDR or DDR mode ■ 8-bit wide in DDW mode **Note:** In DDR mode, this signal can be synchronous to clk_rx_156pt25_180_i or clk_rx_156pt25_i. In SDR mode, this signal is synchronous to clk_rx_312pt5_i. **Exists:** Always **Synchronous To:** DWCXG_XGMII==0 ? "clk_rx_312pt5_i" : "clk_rx_156pt25_i" **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** N/A
xgmii_rxval_i	I	XGMII Rx Valid. This signal is a sideband signal to enable backpressure the XGMII Rx interface. ■ High: Indicates that the data/control the xgmii_rxd_i and xgmii_rxc_i are valid. ■ Low: Indicates that the content of these xgmii_rxd/rxc_i signals are invalid and the DWC_25gmac must not sample the contents. This implies that the XGMII Rx interface is in 'suspend' mode. This signal is applicable/valid only in XGMII-SDR mode and must be set high for normal XGMII operation. **Exists:** DWCXG_XGMII_SDR_BCKPRES **Synchronous To:** clk_rx_312pt5_i **Registered:** DWCXG_GMII_SUPPORT==1 && !DWCXG_ADD_GMII_PORT==1 ? No : Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

Table 12-36 XGMII Interface Signals (continued)

Port Name	I/O	Description
xgmii_txrdy_i	I	XGMII Tx Ready. This signal is a sideband signal to enable backpressure the XGMII TX interface. When ■ High: Indicates that the data/control the xgmii_txd_o and xgmii_txc_o has been accepted by the PHY and the DWC_25gmac must update these signals in the next clock. ■ Low: DWC_25gmac maintains the values xgmii_txd_o and xgmii_txc_o and does not update the contents. In other words, the XGMII TX interface is in 'suspend' mode. This signal is applicable/valid only in XGMII-SDR mode and must be set high for normal XGMII operation. **Exists:** DWCXG_XGMII_SDR_BCKPRES **Synchronous To:** clk_tx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
xgmii_txd_o[(DWCXG_XGMII_DATA_WIDTH-1):0]	O	XGMII Transmit Data. This is a bundle of four 8-bit transmit data signals driven by DWC_25gmac its four lanes. The data is synchronous to the rising and falling edges of clk_tx_156pt25_i in DDR mode. In both DDR and SDR modes, Ethernet frames start (SOF character) in Lane 0 (Bits[7:0]). In DDW mode, a bundle of eight 8-bit signals (width is 64 bits) is transmitted at every rising edge of clk_tx_156pt25_i. In DDW mode, Ethernet frames start (SOF character) in either Lane 0 or Lane 4 (bits[39:32]). When GMII pins are shared with XGMII, xgmii_txd_o[7:0] takes the function described for gmii_txd_o in GMII mode. In the signal name, N is the bus width. It is equal to: ■ 32 in SDR or DDR mode ■ 64 in DDW mode **Note:** In DDR mode, this signal can be synchronous to clk_tx_156pt25_180_i or clk_tx_156pt25_i. In SDR mode, this signal is synchronous to clk_tx_312pt5_i. **Exists:** Always **Synchronous To:** DWCXG_XGMII==0 ? "clk_tx_312pt5_i" : "clk_tx_156pt25_i" **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-36 XGMII Interface Signals (continued)

Port Name	I/O	Description
xgmii_txc_o[(DWCXG_XGMII_CTRL_WIDTH-1):0]	O	XGMII Transmit Control. This signal indicates whether data or control characters are being driven the XGMIII Transmit Data (xgmii_txd_o) lanes. If any of these bits are de-asserted, data is driven to the corresponding XGMII Transmit data lane. If a bit remains asserted, control characters are driven to the corresponding XGMII Transmit data lane. When GMII pins are shared with XGMII, xgmii_txc_o[1:0] takes the function described for gmii_txer_o and gmii_txen_o in GMII mode. In the signal name, W is the signal width. This signal is: ■ 4-bit wide in SDR or DDR mode ■ 8-bit wide in DDW mode **Note:** In DDR mode, this signal can be synchronous to clk_tx_156pt25_180_i or clk_tx_156pt25_i. In SDR mode, this signal is synchronous to clk_tx_312pt5_i. **Exists:** Always **Synchronous To:** DWCXG_XGMII==0 ? "clk_tx_312pt5_i" : "clk_tx_156pt25_i" **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
tx_ts_data_val_o	O	TX TimeStamp FIFO Data Out Valid . When set high, the timestamp value in tx_ts_data_o is valid. **Exists:** SNPS_RSVDPARAM_31 **Synchronous To:** (DWCXG_CSR_SLV_CLK||DWCXG_CORE) ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.37 RXF Memory Interface Signals

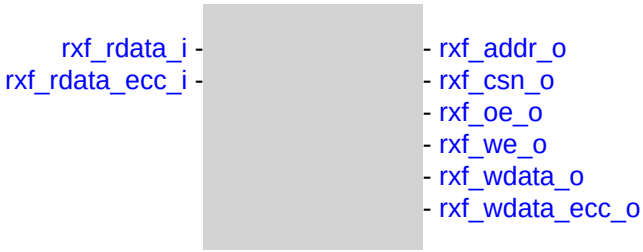


Table 12-37 RXF Memory Interface Signals

Port Name	I/O	Description
rxf_addr_o[(DWCXG_RXF_MEM_AW-1):0]	O	RXF Memory Address Bus. This bus gives the address for writing/reading to/from RXF Memory. It is valid when rxf_csn_o is low. **Exists:** (DWCXG_RXF_MEM_EN) **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rxf_csn_o	O	RXF Memory Chip Select. This signal is used as chip select to select the RXF Memory. This signal qualifies rxf_addr_o and rxf_we_o. **Exists:** (DWCXG_RXF_MEM_EN) **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** Low
rxf_oe_o	O	RXF Memory Output Enable. The MAC drives this signal to enable the output port of the RXF Memory. The RXF Memory drives the valid read data on rxf_rdata_i signal, only when this signal is high. The MAC samples the read data, with this signal high and one clock after the read address is given. **Exists:** (DWCXG_RXF_MEM_EN) **Synchronous To:** clk_rx_312pt5_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High

Table 12-37 RXF Memory Interface Signals (continued)

Port Name	I/O	Description
rxf_we_o	O	RXF Memory Write Enable. This signal controls the read/write operation. Indicates write operation when high and read operation when low. It is valid when rxf_csn_o is low. **Exists:** (DWCXG_RXF_MEM_EN) **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
rxf_wdata_o[(DWCXG_RXF_MEM_DW-1):0]	O	RXF Memory Write Data. This bus carries the data to be written in the RXF Memory. It is valid when the rxf_csn_o signal is low and rxf_we_o is high. **Exists:** (DWCXG_RXF_MEM_EN) **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rxf_wdata_ecc_o[(DWCXG_RXF_MEM_EW-1):0]	O	RXF Memory Write ECC Data. This bus carries the ECC bits to be written in the RXF Memory. It is valid when the rxf_csn_o signal is low and rxf_we_o is high. **Exists:** (DWCXG_RXF_MEM_EN&&DWCXG_ASP_ECC) **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A
rxf_rdata_i[(DWCXG_RXF_MEM_DW-1):0]	I	RXF Memory Read Data. This bus carries the data read from RXF Memory. It is valid when rxf_oe_o signal is high. **Exists:** (DWCXG_RXF_MEM_EN) **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

Table 12-37 RXF Memory Interface Signals (continued)

Port Name	I/O	Description
rxf_rdata_ecc_i[(DWCXG_RXF_MEM_EW-1):0]	I	RXF Memory Read ECC Data. This bus carries the ecc bits read from RXF Memory. It is valid when rxf_oe_o signal is high. **Exists:** (DWCXG_RXF_MEM_EN&&DWCXG_ASP_ECC) **Synchronous To:** clk_rx_312pt5_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.38 Tx Timestamp Status FIFO Streaming Interface Signals

Table 12-38 Tx Timestamp Status FIFO Streaming Interface Signals

Port Name	I/O	Description
tx_ts_data_rdy_i	I	TX TimeStamp FIFO Data Ready. This signal is a sideband signal to enable backpressure the XG TX interface. When ■ High: Indicates that the data/control the tx_ts_data_o and tx_ts_data_val_o has been accepted by the PHY and the Tx TS FIFO must update these signals in the next clock. ■ Low: Tx TS FIFO maintains the values tx_ts_data_o and tx_ts_data_val_o and does not update the contents. **Exists:** SNPS_RSVDPARAM_31 **Synchronous To:** (DWCXG_CSR_SLV_CLK||DWCXG_CORE) ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** High
tx_ts_data_o[((74+DWCXG_NUM_DMA_TX_CHW)-1):0]	O	TX TimeStamp FIFO Data Out. When tx_ts_data_val_o is high, the Tx Timestamp value is available in tx_ts_data_o signal. First 10 MSB bits contain packet id, next 32 bits represent nanoseconds and next 32 LSB bits represent seconds timestamp. **Exists:** SNPS_RSVDPARAM_31 **Synchronous To:** (DWCXG_CSR_SLV_CLK||DWCXG_CORE) ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_OFF" : "XGMAC_TOP"} **Active State:** N/A

12.39 SSM Bus Interface Signals

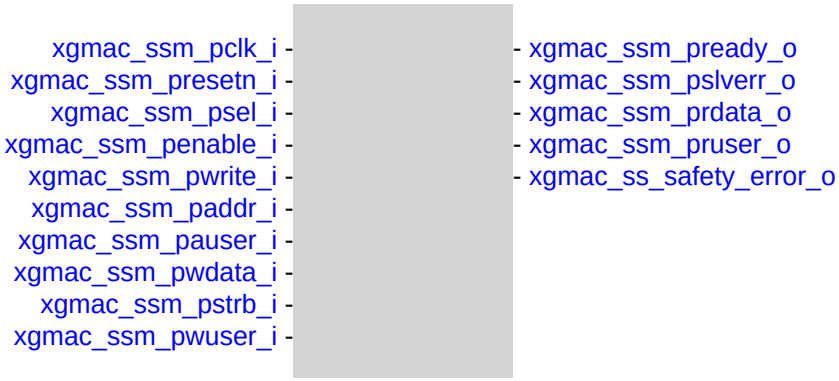


Table 12-39 SSM Bus Interface Signals

Port Name	I/O	Description
xgmac_ssm_pclk_i	I	SSM clock. The clock may be enabled only when accessing the safety subordinate. Must be asserted along with core_resetn and deasserted synchronously with ssm_pclk after core_resetn goes inactive. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** None **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_presetn_i	I	Safety Subordinate Reset. Must be asserted along with core_resetn and deasserted synchronously with ssm_pclk after core_resetn goes inactive. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

Table 12-39 SSM Bus Interface Signals (continued)

Port Name	I/O	Description
xgmac_ssm_psel_i	I	Safety Subordinate Select. When this signal is asserted, it indicates the start of transaction and validity of the xgmac_ssm_paddr_i[] and xgmac_ssm_pwrite_i signals. In addition, during a write operation, this signal indicates validity of the xgmac_ssm_pwdata_i[] write data. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_penable_i	I	Safety Subordinate Enable **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_pwrite_i	I	Safety Subordinate Write/Read Indicator. When high, this signal indicates a write operation. When low, it indicates a read operation. Safety Subordinate Address (configurable to 8 or 16 bits) **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_paddr_i[15:0]	I	Safety Subordinate Address. This signal carries the register address of the Control or Status Register of the safety_slave module. The address is valid only when the xgmac_ssm_psel_i signal is asserted. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

Table 12-39 SSM Bus Interface Signals (continued)

Port Name	I/O	Description
xgmac_ssm_pauser_i[5:0]	I	User defined address bus. This consists of the parity bit for the address and Antivalent bits of the control signals pwrite, psel and penable. {ssm_paddr_parity, 0, ssm_pwrite_av, ssm_penable_av, ssm_psel_av} **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_pwdata_i[31:0]	I	Write Data. This signal carries the write data from the host (Safety Manager Application) for the Control registers inside the safety_slave. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_pstrb_i[3:0]	I	Write Data Strobe. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_pwuser_i[4:0]	I	User defined write data bus. It contains the parity for write data and anti-valent bit write_strb. {ssm_pwdata_parity, ssm_pstrb_parity} **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

Table 12-39 SSM Bus Interface Signals (continued)

Port Name	I/O	Description
xgmac_ssm_pready_o	O	Safety Subordinate ready. When high, this signal indicates the following: - Data on the xgmac_ssm_prdata_o[15:0] signal is valid. The host can accept the data. - End of write transfer Parity error detected if enabled on xgmac_ssm_paddr_i or xgmac_ssm_pwdata_i or xgmac_ssm_pstrb_i. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_pslverr_o	O	Safety Subordinate Error. Parity error detected if enabled on xgmac_ssm_paddr_i or xgmac_ssm_pwdata_i or xgmac_ssm_pstrb_i. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ssm_prdata_o[31:0]	O	SSM Read Data. This port carries the read data from the Safety Subordinate Registers to the host (Safety Manager Application). This data is valid only when the xgmac_ssm_ pready_o signal is asserted. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

Table 12-39 SSM Bus Interface Signals (continued)

Port Name	I/O	Description
xgmac_ssm_pruser_o[5:0]	O	User defined read data bus. It contains the parity for read data and anti -valent bit for xgmac_ssm_pslverr_o, xgmac_ssm_pready_o. {ssm_prdata_parity, ssm_pslverr_av, ssm_pready_av}. It contains the parity for read data and read parity. {ssm_prdata_parity, ssm_pslverr_av, ssm_pready_av}. **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** xgmac_ssm_pclk_i **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
xgmac_ss_safety_error_o[1:0]	O	Safety Error. Sticky until reset. No Error: 2'b10, Error: 2'b01 **Exists:** SNPS_RSVDPARAM_43 **Synchronous To:** DWCXG_CSR_SLV_CLK ? "clk_csr_i" : (DWCXG_AXI_SUBSYS ? "aclk_i" : "clk_app_i") **Registered:** Yes **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

12.40 Filtering/Camera over Ethernet Offload Signals

- sbd_rx_pdma_pktinfo_done_o
- sbd_rx_dmaid_o

Table 12-40 Filtering/Camera over Ethernet Offload Signals

Port Name	I/O	Description
sbd_rx_pdma_pktinfo_done_o[(DWCXG_NUM_RXQ-1):0]	O	Packet-info Write Done. This signal (each bit corresponds to an Rx PDMA) is set high for 1 clock whenever the completion/acknowledgement of the Packet-Info write transaction to the host Buffer memory is received by the DMA. The VDMA-ID which initiated and completed this packet-Info write transfer is given in the sbd_rx_dmaid_o bus. Only 1 of the 8 bits will be high in any clock. **Exists:** SNPS_RSVDPARAM_48 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High
sbd_rx_dmaid_o[(DWCXG_RX_CHW-1):0]	O	RX DMA ID. This bus gives the RX DMA-ID initiated and completed the packet-Info write transfer to host memory. This is valid only when any of the bits of sbd_rx_tc_pktinfo_done_o[7:0] is set high. **Exists:** SNPS_RSVDPARAM_48 **Synchronous To:** aclk_i **Registered:** No **Power Domain:** {DWCXG_PMT_EN ? "MAC_ON" : "XGMAC_TOP"} **Active State:** High

13

Descriptors

This chapter describes the 25GMAC descriptors.

This chapter contains the following sections:

- "Overview" on page 1796
- "Descriptor Structure" on page 1797
- "Descriptor Endianness" on page 1799
- "Transmit Descriptor" on page 1800
- "Receive Descriptor" on page 1811
- "Enhanced Descriptor for Time-Based Scheduling" on page 1829
- "Header Payload Split Support" on page 1834

13.1 Overview

The DMA in the Ethernet subsystem transfers data based on a linked list of descriptors. The application creates the descriptors in the system memory. The 25GMAC supports the following two types of descriptors:

- Normal Descriptor: Normal descriptors are used for exchanging packet data and packet control or status information applicable between the application and the 25GMAC controller.
- Context Descriptor: Context descriptors are used to provide additional control information applicable to the packets to be transmitted or additional status information for received packets.
  - Tx side: The application prepares the context descriptors and puts the context descriptors in the linked list before the normal descriptor. The XGMAC-AXI interface processes the controls given in the context descriptor for the packets following the context descriptor.
  - Rx side: The application does not create context descriptors. It creates a linked-list of normal descriptors. When the RxDMA has additional status information for a received packet, it closes the next descriptor with this additional status and marks it as “context descriptor”.

Each normal descriptor contains two buffers and two address pointers. These buffers enable the adapter port to be compatible with various types of memory management schemes.

Note

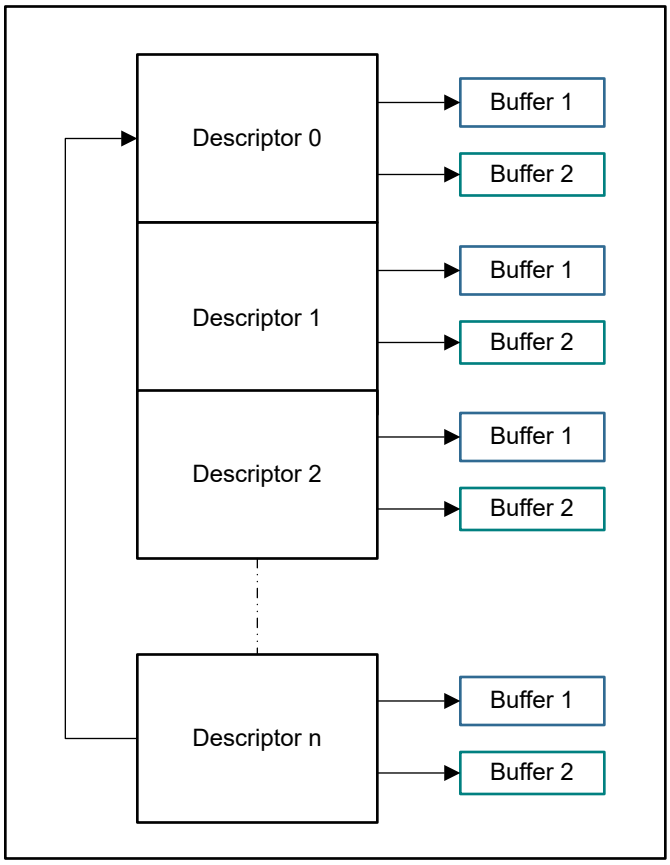
There is no limit for number of descriptors that can be used for a single packet except that the number of descriptors must be less than the ring length.

The reserved fields in the descriptor must be set to 0, otherwise the behavior is unpredictable.

13.2 Descriptor Structure

The 25GMAC supports the ring structure for DMA descriptor as shown in Figure 13-1.

Figure 13-1 Descriptor Ring Structure



In the ring structure, descriptors are separated by the Word, DWord, or LWord number programmed in the DSL field of the VDMA_CH(#i)_Control register. The application needs to program the total ring length, that is, the total number of descriptors in ring span in the following registers of a DMA channel:

- Transmit Descriptor Ring Length Register (VDMA_CH(#i)_Tx_Control2)
- Receive Descriptor Ring Length Register (VDMA_CH(#i)_Rx_Control2)

The Descriptor Tail Pointer Register contains the pointer to the descriptor address (i). The descriptors up to one location less than the one indicated by the descriptor tail pointer (i – 1) are owned by the DMA. The DMA continues to process the descriptors until the following condition occurs:

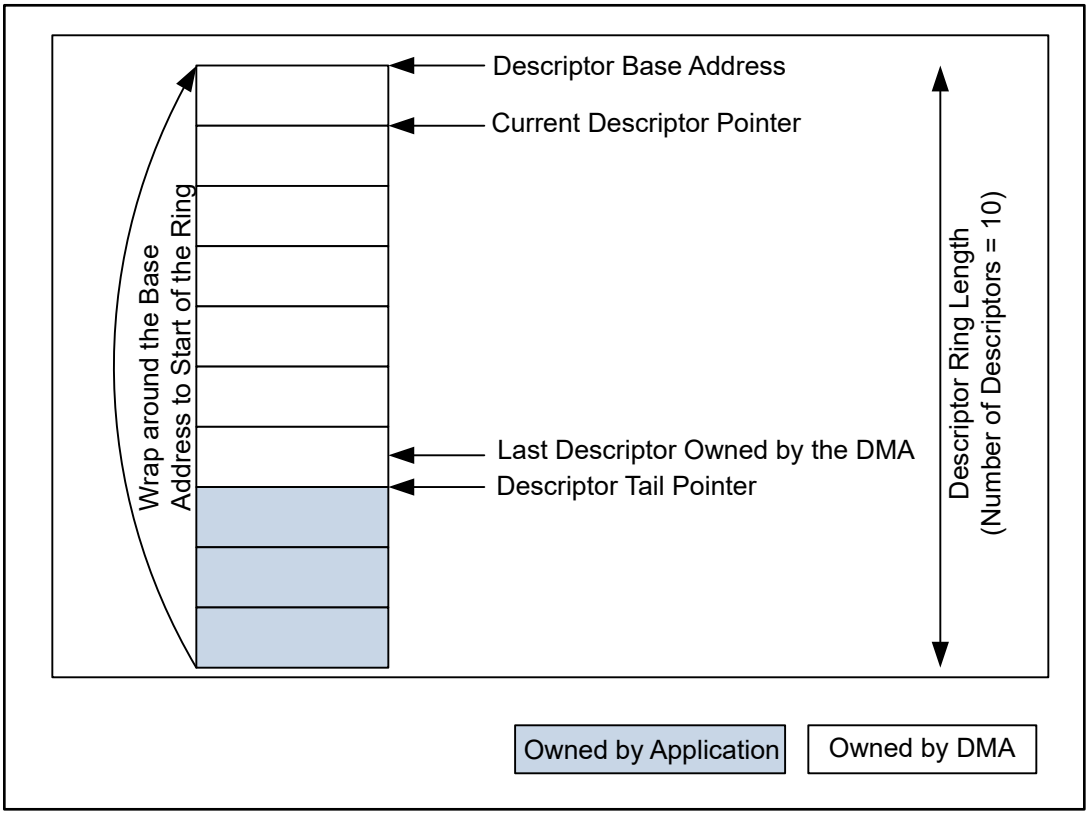
```
Current Descriptor Pointer == Descriptor Tail Pointer
```

The DMA goes into the Suspend mode when this condition occurs. The application must perform a write to the Descriptor Tail pointer register and increase the offset so that the following condition is true:

```
Current Descriptor < Pointer Descriptor Tail Pointer
```

The DMA automatically wraps around the base address when the last descriptor in the ring is processed, as shown in Figure 13-2.

Figure 13-2 DMA Descriptor Ring



For descriptors owned by the application, the OWN bit of TDES3 and RDES3 are reset to 0. For descriptors owned by the DMA, the OWN bit is set to 1. If the application has only two descriptors in the beginning, the application sets the last descriptor address (tail pointer) to Descriptor Base Address + 1. The DMA processes the first descriptor and then waits for the application to advance change the tail pointer back to Descriptor Base Address.

Note In case of 40-bit or 48-bit addressing, the higher 8 or 16 bits are fixed and not incremented. Therefore, the lower 32 bits of the descriptor list address (VDMA_CH(#i)_TxDesc_List_LAddress and VDMA_CH(#i)_RxDesc_List_LAddress) and descriptor ring lengths (VDMA_CH(#i)_Tx_Control2 and VDMA_CH(#i)_Rx_control2) must be programmed such that the descriptor address does not cross a 4 GB address boundary.

13.3 Descriptor Endianness

The descriptor addresses must be aligned to used bus width (Word, DWord, or LWord for 64-bit or 128-bit bus respectively). The data bus can be configured for little-endian format only. Table 13-1 provides the information about the figures that are provided to show the structure of normal descriptor with respect to the data bus endianness.

Table 13-1 Descriptor Endianness

Data Bus Endianness	Descriptor Endianness	Figure Number
Little-endian	Same-endian	■ Figure 13-3 (for 64-bit data bus) ■ Figure 13-4 (for 128-bit data bus)

Figure 13-3 shows the normal Receive and Transmit descriptors for 64-bit data bus when the endian mode of data bus and descriptors is little-endian.

Figure 13-3 Rx/Tx Descriptors in Same-Endian Mode for 64-Bit, Little-Endian Data Bus

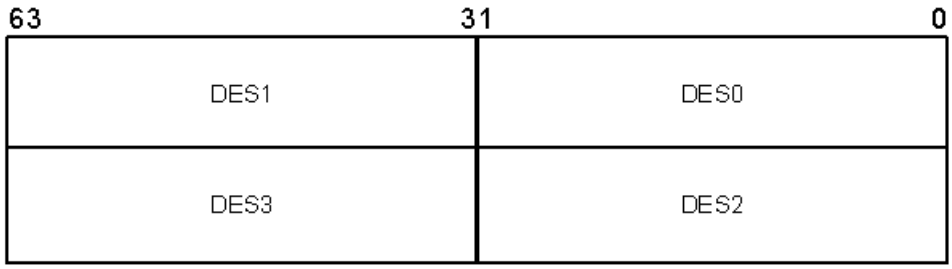
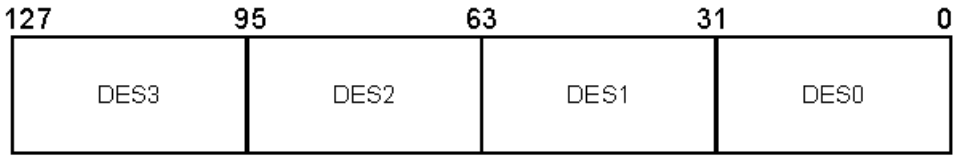


Figure 13-4 shows the normal Receive and Transmit descriptors for 128-bit data bus when the endian mode of data bus and descriptors is little-endian.

Figure 13-4 Rx/Tx Descriptors in Same-Endian Mode for 128-Bit, Little-Endian Format Data Bus



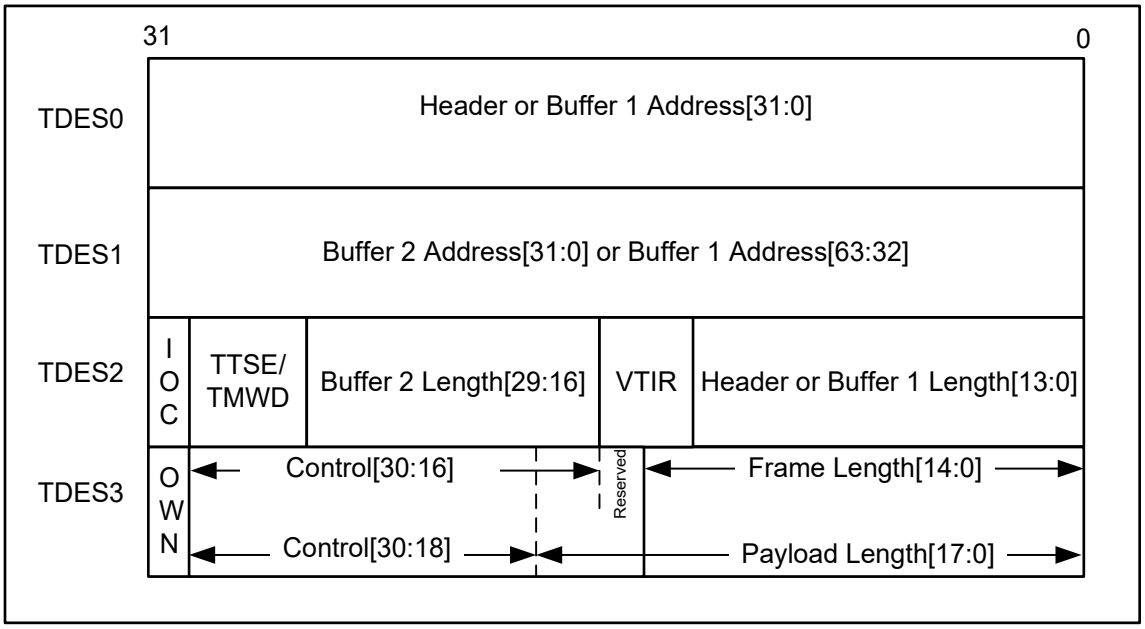
13.4 Transmit Descriptor

The DMA in 25GMAC requires at least one descriptor for a transmit packet. In addition to two buffer length fields, and two address pointers, the transmit descriptor has control fields which can be used to control the packet processing on per-transmit packet basis. The Transmit Normal descriptor has the following two formats: Read format and Write-Back format

13.4.1 Transmit Normal Descriptor (Read Format)

Figure 13-5 shows the Read Format for a Transmit normal descriptor. This is the format in which the software can prepare the descriptor contents and hand it over to the DMA Engines of the 25GMAC.

Figure 13-5 Transmit Descriptor Read Format



13.4.1.1 TDES0 Normal Descriptor (Read Format)

Table 13-2 TDES0 Normal Descriptor (Read Format) (Continued)

Bit	Name	Description
31:0	BUF1AP	Buffer 1 Address Pointer or TSO Header Address Pointer These bits indicate the physical address of Buffer 1. These bits indicate the TSO Header Address pointer when the following bits are set: ■ TSE bit of TDES3 ■ FD bit of TDES3

13.4.1.2 TDES1 Normal Descriptor (Read Format)

Table 13-3 TDES1 Normal Descriptor (Read Format)

Bit	Name	Description
31:0	BUF2AP	Buffer 2 or Buffer 1 Address Pointer This bit indicates the physical address of Buffer 2 when a descriptor ring structure is used. There is no limitation for the buffer address alignment. In 40- or 48-bit addressing mode, these bits indicate the most-significant 8- or 16-bits of the Buffer 1 Address Pointer.

13.4.1.3 TDES2 Normal Descriptor (Read Format)

Table 13-4 TDES2 Normal Descriptor (Read Format)

Bits	Name	Description
31	IOC	Interrupt on Completion This bit sets the TI bit in the VDMA_CH(#i)_Status register after the present packet has been transmitted.
30	TTSE/ TMWD	Transmit Timestamp Enable or External TSO Memory Write Disable. This bit enables the IEEE1588 time stamping for Transmit packet referenced by the descriptor, if TSE bit not set. If TSE bit is set and external TSO memory is enabled, setting this bit disables external TSO memory writing for this packet.
29:16	B2L	Buffer 2 Length The driver sets this field. When set, this field indicates Buffer 2 length.
15:14	VTIR	VLAN Tag Insertion or Replacement These bits request the MAC to perform VLAN tagging or untagging before transmitting the packets. If the packet is modified for VLAN tags, the MAC automatically recalculates and replaces the CRC bytes. The following list describes the values of these bits: ■ 2'b00: Do not add a VLAN tag. ■ 2'b01: Remove the VLAN tag from the packets before transmission. This option must be used only with the VLAN packets. ■ 2'b10: Insert a VLAN tag with the tag value programmed in the MAC_VLAN_Incl register or context descriptor. ■ 2'b11: Replace the VLAN tag in packets with the tag value programmed in the MAC_VLAN_Incl register or context descriptor. This option must be used only with the VLAN packets. These bits are valid when the Enable SA and VLAN Insertion on Tx option is selected while configuring the 25GMAC controller.
13:0	HL or B1L	Header Length or Buffer 1 Length

Table 13-4 TDES2 Normal Descriptor (Read Format) (continued)

Bits	Name	Description
		For Header length, only bits [9:0] are taken. The size 13:0 is applicable only when interpreting buffer 1 length. If the TCP Segmentation Offload feature is enabled through the TSE bit of TDES3, this field is equal to the header length. When the TSE bit is set in TDES3, the header length includes the length in bytes from Ethernet Source address till the end of the TCP header. The minimum value of Header length is 0x30 (48 bytes) when SA insertion is enabled, or 0x36 (54 bytes), otherwise. If the value is less than this, TCP Segmentation is not performed on the packet. The maximum header length supported for TSO feature is 1023 bytes. If SA insertion is performed on this packet, the header length must be no less than 48 bytes, otherwise, it must be no less than 54 bytes. If the TCP Segmentation Offload feature is not enabled, this field is equal to Buffer 1 length.

13.4.1.4 TDES3 Normal Descriptor (Read Format)

Table 13-5 TDES3 Normal Descriptor (Read Format)

Bits	Name	Description
31	OWN	Own Bit When this bit is set, it indicates that the DMA owns the descriptor. When this bit is reset, it indicates that the application owns the descriptor. The DMA clears this bit after it completes the transfer of data given in the associated buffers.
30	CTXT	Context Type This bit must be set to 1'b0 for normal descriptor.
29	FD	First Descriptor When this bit is set, it indicates that the buffer contains the first segment of a packet.
28	LD	Last Descriptor When this bit is set, it indicates that the buffer contains the last segment of the packet. When this bit is set, the B1L or B2L field must have a non-zero value.
27:26	CPC	CRC Pad Control This field controls the CRC and Pad Insertion for Tx packet. This field is valid only when the first descriptor bit (TDES3[29]) is set. The following list describes the values of Bits[27:26]: ■ 2'b00: CRC and Pad Insertion The MAC appends the cyclic redundancy check (CRC) at the end of the transmitted packet of length greater than or equal to 60 bytes. The MAC automatically adds padding and CRC to a frame shorter than 60 bytes. ■ 2'b01: CRC Insertion (Disable Pad Insertion)

Table 13-5 TDES3 Normal Descriptor (Read Format) (continued)

Bits	Name	Description
		The MAC appends the CRC at the end of the transmitted packet but it does not append padding. The application must ensure that the padding bytes are present in the packet being transferred from the Transmit Buffer, that is, the packet being transferred from the Transmit Buffer is of length greater than or equal to 60 bytes. ■ 2'b10: Disable CRC Insertion The MAC does not append the CRC at the end of the transmitted packet. The application must ensure that the padding and CRC bytes are present in the packet being transferred from the Transmit Buffer. ■ 2'b11: CRC Replacement The MAC replaces the last four bytes of the transmitted packet with recalculated CRC bytes. The application must ensure that the padding and CRC bytes are present in the packet being transferred from the Transmit Buffer. Note: When the TSE bit is set, the MAC ignores this field because the CRC and pad insertion is always done for segmentation.
25:23	SAIC / VNP	SA Insertion Control/Virtualized Tunnel Packet Control These bits request the MAC to add or replace the Source Address field in the Ethernet packet with the value given in the MAC Address 0 register. If the Source Address field is modified in a packet, the MAC automatically recalculates and replaces the CRC bytes. Bit 25 specifies the MAC Address Register (1 or 0) value that is used for Source Address insertion or replacement. The following list describes the values of Bits[24:23]: ■ 2'b00: Do not include the source address ■ 2'b01: Include or insert the source address. For reliable transmission, the application must provide frames without source addresses. ■ 2'b10: Replace the source address. For reliable transmission, the application must provide frames with source addresses. ■ 2'b11: Reserved These bits are valid in the XGMAC-AXI configurations when the Enable SA and VLAN Insertion on Tx option is selected while configuring the 25GMAC controller and when the First Segment control bit (TDES3 [29]) is set. Virtualized Tunnel Packet (VNP) When these bits are set to 3’b011, the current packet is treated as Tunneled packet by the TSO and TxCOE functions in the Transmit path. VNP is applicable only when “Enable support for VxLAN/NVGRE packets” is selected while configuring the 25GMAC controller and when bit FD is set. Note: SA Insertion Control feature is not supported for Tunneled packets.
22:19	SLOTNUM or THL	SLOTNUM: Slot Number Control Bits in AV Mode These bits indicate the slot interval in which the data must be fetched from the corresponding buffers addressed by TDES0 or TDES1.

Table 13-5 TDES3 Normal Descriptor (Read Format) (continued)

Bits	Name	Description
		When the Transmit descriptor is fetched, the DMA compares the slot number value in this field with the slot interval maintained in the RSN field `VDMA_CH(#i)_Slot_Function_Control_Status`. It fetches the data from the buffers only if a value matches. These bits are valid only for the AV channels. TCP Header Length: If the TSE bit is set, this field contains the length of the TCP header (in terms of 32-bit words). The minimum value of this field must be 5.
18	TSE	TCP Segmentation Enable 1: the DMA performs the TCP segmentation for a packet. This bit is valid only if the FD bit and TSE bit in the corresponding VDMA_CH(#i)_Tx_Control register = 1. Otherwise, this bit is ignored and considered as 0.
17:16	CIC/TPL	Checksum Insertion Control or TCP Payload Length These bits control the checksum calculation and insertion. The following list describes the bit encoding: ■ 2'b00: Checksum Insertion Disabled. ■ 2'b01: Only IP header checksum calculation and insertion are enabled (both Outer & Inner Headers in case of Tunneled packet). ■ 2'b10: IP header checksum and payload checksum calculation and insertion are enabled, but pseudo-header checksum is not calculated in hardware (both Outer & Inner Headers in case of Tunneled packet). ■ 2'b11: IP Header checksum and payload checksum calculation and insertion are enabled, and pseudo-header checksum is calculated in hardware (both Outer & Inner Headers in case of Tunneled packet). This field is valid when the Enable Transmit TCP/IP Checksum Offload option is selected and the TSE bit is reset. When the TSE bit is set, this field contains the upper bits [17:16] of the TCP Payload length. This allows the TCP packet length field to be spanned across TDES3[17:0] to provide 256 KB packet length support. When Timestamping is enabled without One Step Timestamping Feature and OSTC bit is set in the context Descriptor, the content of this field is not effective and Checksum is updated as though CIC field is set to 2'b11. Note: In case of tunneled packets (as indicated by SAIC/VNP = 3'b011 and the VxLAN mode is enabled in the MAC Transmit Configuration Register), the outer Header UDP Checksum is always 0 as per the VxLAN standard compliance.
15	TPL	TCP Payload Length When the TSE bit is reset, this bit is reserved. When the TSE bit is set, this is Bit 15 of the TCP payload length [17:0]. This field is valid only when the Enable TCP Segmentation Offloading for TCP/IP Packets option is selected while configuring the 25GMAC controller.

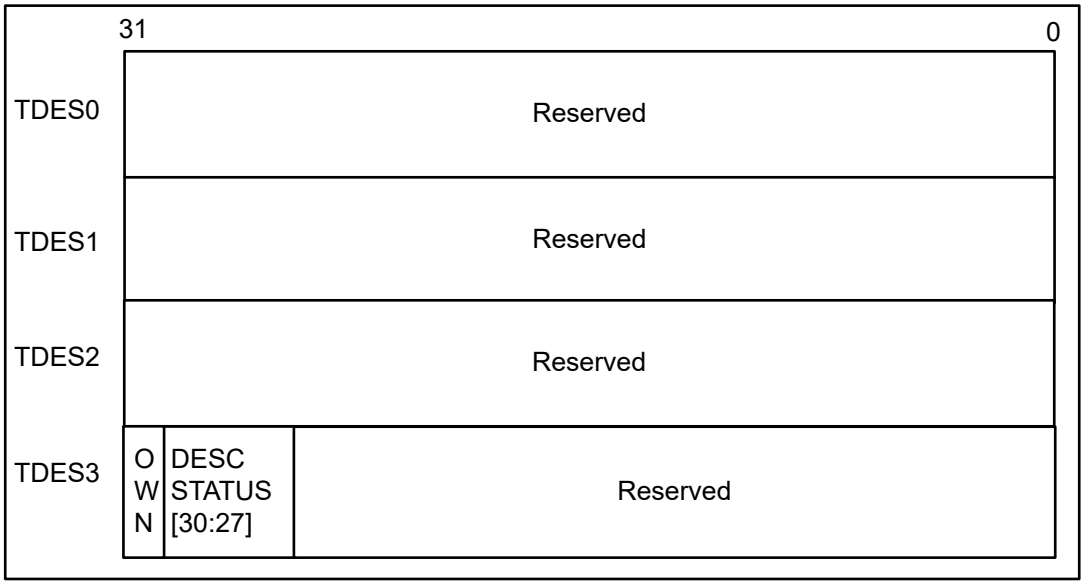
Table 13-5 TDES3 Normal Descriptor (Read Format) (continued)

Bits	Name	Description
14:0	FL/TPL	Packet Length or TCP Payload Length This field is equal to the length of the packet to be transmitted in bytes. When the TSE bit is not set, this field is equal to the total length of the packet to be transmitted: `Ethernet Header Length + TCP /IP Header Length – Preamble Length – SFD Length + Ethernet Payload Length` When the TSE bit is set, this field is equal to the lower 15 bits of the TCP payload length. This length does not include Ethernet header or TCP/IP header length.

13.4.2 Transmit Normal Descriptor (Write-Back Format)

The 25GMAC processes the descriptor, acts on it and then writes back with the status content. This operation indicates to the software that the descriptor is free and no longer required by the hardware. The25GMAC writes only the bits[31:24] of TDES3 during this operation and all the other fields of the descriptor in the host memory are not modified.

Figure 13-6 Transmit Descriptor Write-Back Format



13.4.2.1 TDES3 Normal Descriptor (Write-Back Format)

Table 13-6 TDES3 Normal Descriptor (Write-Back Format)

Bit	Name	Description
31	OWN	Own Bit

Table 13-6 TDES3 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		When this bit is set, it indicates that the 25GMAC DMA owns the descriptor. The DMA clears this bit when it writes back the descriptor. The DMA closes the descriptor when either of the following conditions are true: ■ The DMA completes the transfer of data indicated by the buffers from the host memory to the corresponding Tx Queue. ■ The B1L and B2L values are zero, indicating empty buffers.
30	CTXT	Context Type This bit is set to 'b0 for Normal descriptor.
29	FD	First Descriptor This bit indicates that the buffer contains the first segment of a packet.
28	LD	Last Descriptor This bit is set 'b1 for last descriptor of a packet. The DMA writes the status fields only in the last descriptor of the packet.
27	DERR	Descriptor Error When this bit is set, it indicates that the descriptor content is incorrect. The DMA sets this bit during write-back while closing the descriptor. The descriptor errors can be: ■ AXI Bus Error: This bit is asserted when AXI Bus Error response is received while processing the descriptor. This can be either when the descriptor is being fetched or when the packet data indicated by its buffer pointers are fetched. a. When a transmit descriptor read from descriptor memory has FD, LD and CTXT bits set to 1, the transmit DMA terminates the ongoing DMA transfer (forcing a CRC error when partial packet is transmitted on the line) and enters stop state. b. The DDE and TPS bits of the corresponding `VDMA_CH(#i)_Status` register are set to 1. **Recovery Mechanism:** Perform a software reset by programming the SWR field of the `DMA_Mode` register to 1. ■ Descriptor Definition Error: 1 This bit is asserted if CTXT and FD or LD bits are set to 1 (CTXT & (FD | LD)). ■ All 1s
26:0	Rsvd	Reserved

13.4.3 Transmit Context Descriptor

The Transmit Context descriptor can be provided any time before a packet descriptor. The context is valid for the current packet and subsequent packets. The context descriptor is used to provide the timestamps for one-step timestamp correction, VLAN Tag ID for VLAN insertion feature, and SA insertion bit for SA insertion. Write back is done on a context descriptor only to reset the OWN bit and to set the CDE bit if necessary.

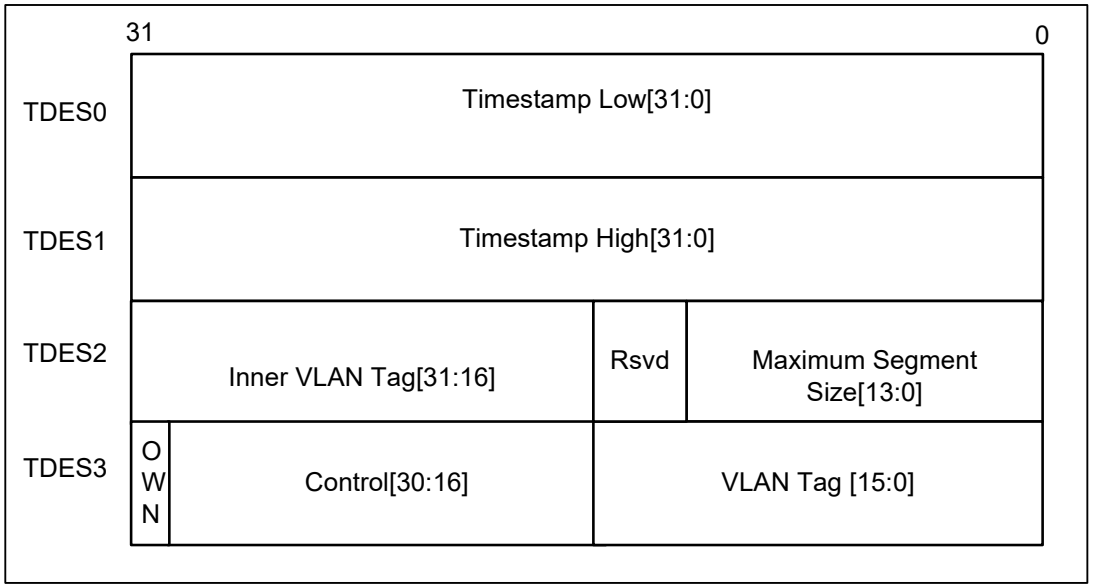
Note

The VLAN Tag IDs and MSS values, provided by the application in a context descriptor with their corresponding Valid bits set, are stored internally by the DMA.

When the outer or inner VLAN tag is provided with the Valid bit set, the DMA always passes the last valid VLAN tag to the MTL. The application cannot invalidate the valid VLAN tag stored by the DMA. The VLAN tag is inserted or replaced based on the control inputs provided for the packet.

The Inner VLAN Tag Control input is used only for the next packet that immediately follows the context descriptor. The application must provide a context descriptor before the normal descriptor of each packet for which the DMA must use the inner VLAN Tag control input.

Figure 13-7 Transmit Context Descriptor (Read Format)



13.4.3.1 TDES0 Context Descriptor

Table 13-7 TDES0 Context Descriptor

Bit	Name	Description
31:0	TTSL	Transmit Packet Timestamp Low or Packet ID For one-step correction, the driver can provide the lower 32 bits of timestamp in this descriptor word. The DMA uses this value as the low word during one-step timestamp correction. This field is valid only if the OSTC and TCMSSV bits of TDES3 context descriptor are set.

Table 13-7 TDES0 Context Descriptor (continued)

Bit	Name	Description
		When the OSTC bit is reset and PIDV is set, the bits [DWCXG_TX_CHW+10:10] represent the VDMAID and the lower 10-bits [9:0] indicate the packet ID which is used for 2-step time stamping. For more details, see “Details of the Operating Modes” on page 213

13.4.3.2 TDES1 Context Descriptor

Table 13-8 TDES1 Context Descriptor

Bit	Name	Description
31:0	TTSH	Transmit Packet Timestamp High For one-step correction, the driver can provide the upper 32 bits of timestamp in this descriptor. The DMA uses this value as the high word for doing one-step timestamp correction. This field is valid only if the OSTC and TCMSSV bits of TDES3 context descriptor are set.

13.4.3.3 TDES2 Context Descriptor

Table 13-9 TDES2 Context Descriptor

Bit	Name	Description
31:16	IVT	Inner VLAN Tag When the IVLTV bit of TDES3 context descriptor is set and the TCMSSV and OSTC bits of TDES3 context descriptor are reset, TDES2[31:16] contains the inner VLAN Tag to be inserted in the subsequent Transmit packets.
15:14	Rsvd	Reserved
13:0	MSS	Maximum Segment Size When the Enable TCP Segmentation Offloading for TCP/IP Packets option is selected, the driver can provide maximum segment size in this field. This segment size is used while segmenting the TCP/IP payload. This field is valid only if the TCMSSV bit of TDES3 context descriptor is set and the OSTC bit of the TDES3 context descriptor is reset.

13.4.3.4 TDES3 Context Descriptor (Read Format)

Table 13-10 TDES3 Context Descriptor

Bit	Name	Description
31	OWN	Own Bit

Table 13-10 TDES3 Context Descriptor (continued)

Bit	Name	Description
		When this bit is set, it indicates that the 25GMAC DMA owns the descriptor. When this bit is reset, it indicates that the application owns the descriptor. The DMA clears this bit after it accepts the context descriptor contents.
30	CTXT	Context Type This bit is set to 1'b1 for Context descriptor.
29:28	Rsvd	Reserved
27	OSTC	One-Step Timestamp Correction Enable When this bit is set, the DMA performs a one-step timestamp correction with reference to the timestamp values provided in TDES0 and TDES1.
26	TCMSSV	One-Step Timestamp Correction Input or MSS Valid When this bit and the OSTC bit are set, it indicates that the Timestamp Correction input provided in TDES0 and TDES1 is valid. When the OSTC bit is reset and this bit and the TSE bit of TDES3 are set in subsequent normal descriptor, it indicates that the MSS input in TDES2 is valid.
25	PIDV	Packet ID valid When this bit is set and OSTC bit is reset, the bits [9:0] of TDES0 Context Descriptor indicates the Packet ID for 2-step Time stamping.
24	Rsvd	Reserved
23:20	Rsvd	Reserved
19:18	IVTIR	Inner VLAN Tag Insert or Replace When this bit is set, these bits request the MAC to perform Inner VLAN tagging or untagging before transmitting the packets. If the packet is modified for VLAN tags, the MAC automatically recalculates and replaces the CRC bytes. The following list describes the values of these bits: ■ 2'b00: Do not add the inner VLAN tag. ■ 2'b01: Remove the inner VLAN tag from the packets before transmission. This option must be used only with the VLAN frames. ■ 2'b10: Insert an inner VLAN tag with the tag value programmed in the register or context descriptor. ■ 2'b11: Replace the inner VLAN tag in packets with the tag value programmed in the register or context descriptor. This option must be used only with the VLAN frames. These bits are valid when the Enable SA and VLAN Insertion on Tx and Enable Double VLAN Processing options are selected.
17	IVLTV	Inner VLAN Tag Valid When this bit is set, it indicates that the IVT field of TDES2 is valid.
16	VLTV	VLAN Tag Valid

Table 13-10 TDES3 Context Descriptor (continued)

Bit	Name	Description
		When this bit is set, it indicates that the VT field of TDES3 is valid.
15:0	VT	VLAN Tag This field contains the VLAN Tag to be inserted or replaced in the packet. This field is used as VLAN Tag only when the VLTI bit of the MAC_VLAN_Incl register is set.

13.4.3.5 TDES3 Context Descriptor (Write-Back Format)

Table 13-11 TDES3 Context Descriptor (Write-Back Format)

Bit	Name	Description
31	OWN	Own Bit When this bit is set, it indicates that the 25GMAC DMA owns the descriptor. When this bit is reset by the DMA, it indicates that the application owns the descriptor. The DMA clears this bit after it accepts the context descriptor contents.
30	CTXT	Context Type This bit is set to 1’b1 for Context descriptor.
29	CDE	Context Descriptor Error When this bit is set, it indicates that the context descriptor was provided in the incorrect sequence and the DMA ignored it. The DMA sets this bit during write-back while closing the context descriptor. When the context descriptor is read by DMA, this bit is reserved and must be having a value 1‘b0.
28:0	Rsvd	Reserved

13.5 Receive Descriptor

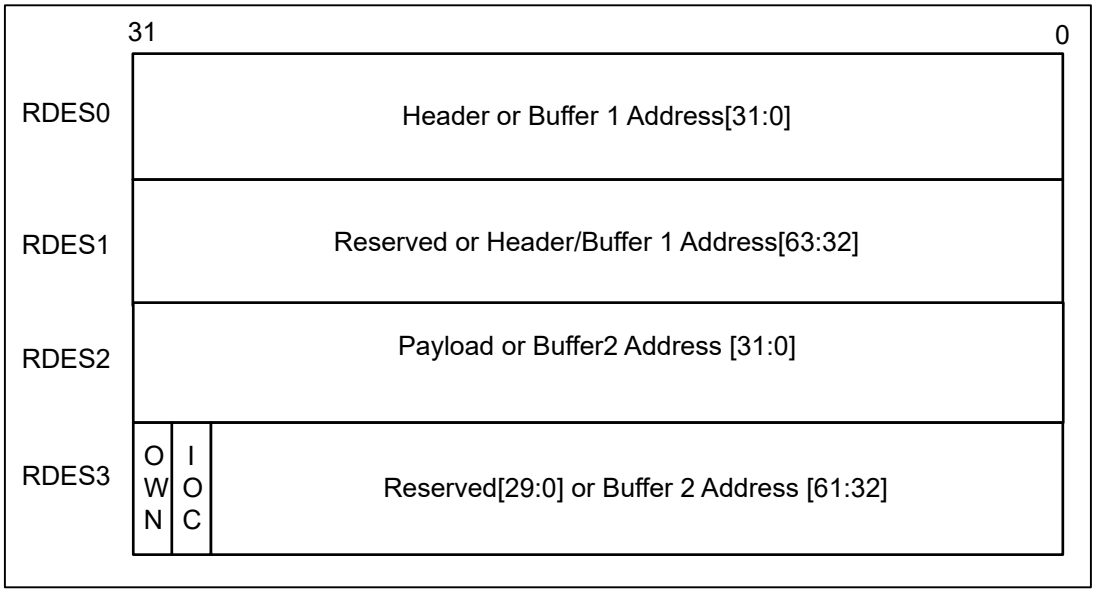
The DMA in the 25GMAC attempts to read a descriptor only if the Tail Pointer is different from the Base Pointer or current pointer. It is recommended to have a descriptor ring with a length that can accommodate at least two complete packets received by the MAC. Otherwise, the performance of the DMA is impacted greatly because of the unavailability of the descriptors. In such situations, the RxFIFO in MTL becomes full and starts dropping packets.

There are two receive descriptor type: Normal and Context descriptors. All RX descriptors are prepared by the software and given to the DMA as Normal Descriptors with the content as shown in Figure 13-8. The DMA reads this descriptor and after transferring a received packet (or part of) to the buffers indicated by the descriptor, the RxDMA closes the descriptor with the corresponding packet status. The format of this status is given in the "Receive Normal Descriptor (Write-Back Format)" on page 1813.

For some packets, the normal descriptor bits are not enough to write the complete status. For such packets, the RXDMA writes the extended status to the next descriptor (without processing or using the Buffers Pointers embedded in that descriptor). The format and content of this descriptor write back is described in "Receive Context Descriptor (Write-Back Format)" on page 1825.

13.5.1 Receive Normal Descriptor (Read Format)

Figure 13-8 Receive Normal Descriptor (Read Format)



Note In the Receive Descriptor (Read Format), if the Buffer Address field is all 0s, 25GMAC does not transfer data to that buffer and skips to the next buffer or next descriptor.

13.5.1.1 RDES0 Normal Descriptor (Read Format)

Table 13-12 RDES0 Normal Descriptor (Read Format)

Bit	Name	Description
31:0	BUF1AP	Header or Buffer 1 Address Pointer When the SPH bit of Control register of a channel is reset, these bits indicate the physical address of Buffer 1. When the SPH bit is set, these bits indicate the physical address of Header Buffer where the RxDMA writes the L2/L3/L4 header bytes of the received packet. The software can program a byte-aligned address for this buffer which means that the LS bits of this field can be non-zero. However, while transferring the start of packet, the DMA performs a Write operation with RDES0[3:0] (or RDES0[2:0] in case of 64-bit configuration) as zero. However, the packet data is shifted as per actual offset as given by buffer address pointer. If the address pointer points to a buffer where the middle or last part of the packet is stored, the DMA ignores the offset address and writes to the full location as indicated by the data-width.

13.5.1.2 RDES1 Normal Descriptor (Read Format)

Table 13-13 RDES1 Normal Descriptor (Read Format)

Bit	Name	Description
31:0	BUF1AP	In 64-bit addressing mode, this field contains the most-significant 32 bits of the Header/Buffer 1 Address Pointer. Otherwise, this field is reserved.

13.5.1.3 RDES2 Normal Descriptor (Read Format)

Table 13-14 RDES2 Normal Descriptor (Read Format)

Bit	Name	Description
31:0	BUF2AP	Buffer 2 Address Pointer These bits indicate the physical address of Buffer 2. When the SPH bit of the VDMA_CH(#i)_Control register is set, the buffer address pointer must be bus width-aligned, that is, RDES2[3:0 or 2:0] = 0 corresponding to 128 or 64 bus width. LSBs are ignored internally. When the SPH bit of the VDMA_CH(#i)_Control register is reset, the BUF2AP can be of any byte-aligned value. However, the RxDMA uses the LS Bits of the pointer address only while transferring the start bytes of a packet. If the BUF2AP is giving the address of a buffer in which the middle or last part of a packet is stored, the DMA ignores BUF2AP[3:0 or 2:0] (corresponding to 128- or 64-bit databus) and writes to the complete location.

13.5.1.4 RDES3 Normal Descriptor (Read Format)

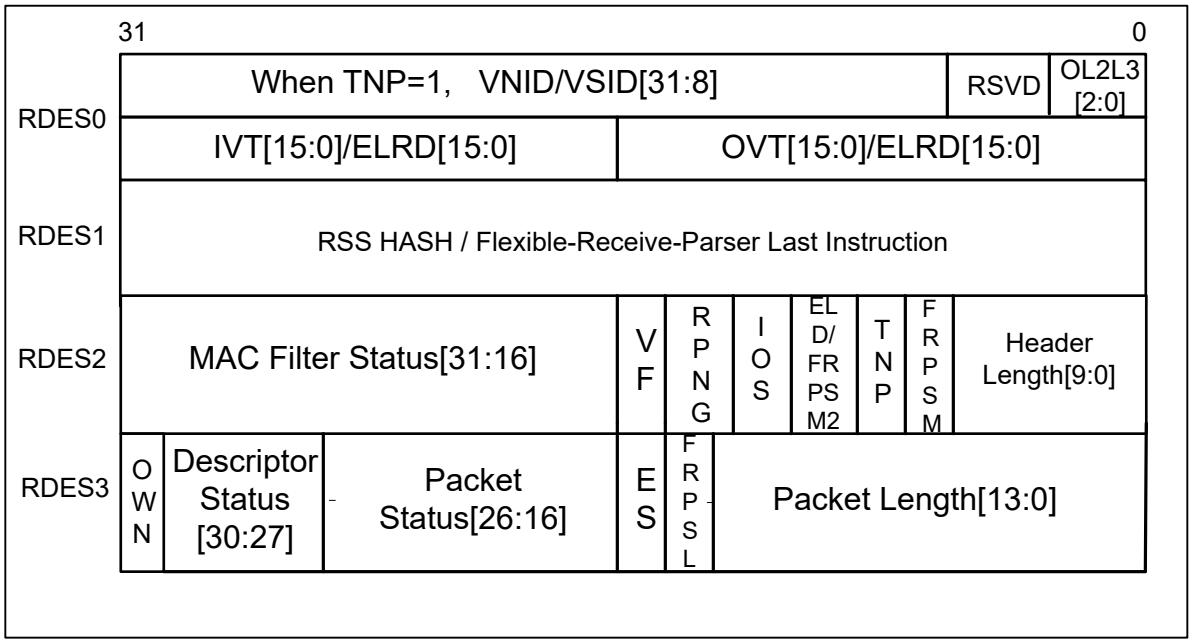
Table 13-15 RDES3 Normal Descriptor (Read Format)

Bit	Name	Description
31	OWN	Own Bit When this bit is set, it indicates that the 25GMAC DMA owns the descriptor. When this bit is reset, it indicates that the application owns the descriptor.
30	IOC	Interrupt on Completion When this bit is set, an interrupt is issued to the application when the DMA closes this descriptor.
29:0	BUF2AP	Higher 30-bits of the Buffer2 Address Pointer in 64-bit addressing mode. Otherwise it is reserved.

13.5.2 Receive Normal Descriptor (Write-Back Format)

The RxDMA frees up a descriptor by writing it back to the system memory after transferring the received packet contents to the buffers pointed by the descriptor (Read Format). At the same time, it also writes the status information of the current packet in the format as shown in Figure 13-9.

Figure 13-9 Receive Normal Descriptor (Write-Back Format)



13.5.2.1 RDES0 Normal Descriptor (Write-Back Format)

The status fields in the write-back format are valid only for the last descriptor (RDES3[28] is set).

Table 13-16 RDES0 Normal Descriptor (Write-Back Format)

Bit	Name	Description
For Non Tunneled Frames (TNP = 0)		
31:16	IVT/ELRD	■ Inner VLAN Tag This field contains the Inner VLAN tag of the received packet. This is valid only when Double VLAN tag processing and VLAN tag stripping are enabled. This field is valid only when the L2T field of RDES0 indicates that the received packet is a Doubled VLAN tagged Packet. Note: In case of external receive Inner VLAN lookup, this field shows the lookup index based on the RIVTBS field of the MAC_RVLAN_LKP_SIZE register programming. ■ External Lookup Result Data When you enable external lookup function, this field contains the data accepted from the External Lookup Engine (xgmac_elrd_i input) when the ELD=1 & IOS=1. When status bit ELD=0, then this field corresponds to IVT.
15:0	OVT/ELRD	■ Outer VLAN Tag This field contains the Outer VLAN tag of the received packet. This is valid only when VLAN tag stripping is enabled. This field is valid only when the L2T Field of RDES0 indicates that the received packet is a VLAN or Doubled VLAN tagged Packet. Note: In case of external receive outer VLAN lookup, this field shows the lookup index based on the ROVTBS field of the MAC_RVLAN_LKP_SIZE register programming. ■ External Lookup Result Data When you enable external lookup function, this field contains the data accepted from the External Lookup Engine (xgmac_elrd_i input) when the ELD=1 & IOS=0. When status bit ELD=0, then this field corresponds to OVT.
For Tunneled Frames (TNP = 1)		
31:8	VNID / VSID	VNID / VSID Tag This field contains VNID for VxLAN type tunneled packet and VSID for NVGRE type tunneled packets.
7:3	Rsvd	Reserved
2:0	OL2L3	Outer L2 L3 Type This field indicates the type of outer header of the received tunneled packet. ■ 001: unTagged IPv6 ■ 010: VLAN Tagged IPv4 ■ 011: VLAN Tagged IPv6 ■ 100: Double VLAN Tagged IPv4 ■ 101: Double VLAN Tagged IPv6 ■ 110-111: Reserved

13.5.2.2 RDES1 Normal Descriptor (Write-Back Format)

The Status fields in write-back format are valid only for the last descriptor (RDES3[28] is set).

Table 13-17 RDES1 Normal Descriptor (Write-Back Format)

Bit	Name	Description
31:0	RSSH/FRPLI	Received Side Scaling Hash / FRP last instruction When LINST field (of MTL_RXP_Control_Status) is present and it is set to 1 and FRP feature is enabled then, this field contains the FRP Last Instruction number where FRP result is generated. Otherwise, when RDES3[26] bit is set and RSS Hash feature is enabled, this field contains the 32-bit RSS Hash calculated with some of the L3-L4 header fields of the current packet. For a tunneled packet, this contains the non-standard RSS hash using some of the outer header and inner header fields. Note: In case of external RSS lookup, based on RSS lookup size, LSB of this field can be used to know the lookup index. When RDES3[26] bit is reset and FRP feature is enabled, this field contains the FRP Last Instruction number where FRP result is generated. For more details about FRP result, see Table 13-20 on page 1825. In case of non-receive channel list based frame duplication, when packet is duplicated to multiple Receive DMA channels, 25GMAC does not provide RSS Hash.

13.5.2.3 RDES2 Normal Descriptor (Write-Back Format)

The Status fields in the write-back format are valid only for the last descriptor (when RDES3[28] is set).

Table 13-18 RDES2 Normal Descriptor (Write-Back Format)

Bit	Name	Description
31:29	L3L4FM	Layer 3 and Layer 4 Filter Number Matched ■ When the DHLFRS field of the MAC_Packet_Filter register is set to ❑ 0x1: This field represents DA hash table Index [10:8] bits, if it is valid, based on HFDAF field. ❑ 0x2: This field is reserved. If bit 28 or bit 27 is set to 1, the MADRM field of the same status word indicates the Layer 3 and Layer 4 filter number that matched the received packet. ■ For all other cases, these bits indicate the number of the Layer 3 and Layer 4 Filter that matched the received packet: ❑ 000: Filter 0 ❑ 001: Filter 1 ❑ 010: Filter 2 ❑ 011: Filter 3 ❑ 100: Filter 4

Table 13-18 RDES2 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		❑ 101: Filter 5 ❑ 110: Filter 6 ❑ 111: Filter 7 This field is valid only when Bit 28 or Bit 27 is set to 1. When more than one filter matches, these bits give the lowest filter number. When more than one filter matches, these bits indicate the number of the lowest filter. For a tunneled packet, only the outer L3-L4 header fields are used for comparison.
28	L4FM	Layer 4 Filter Match When this bit is set, it indicates that the received packet matches one of the enabled Layer 4 Port Number fields. This status is given only when one of the following conditions is true: ■ Layer 3 fields are not enabled and all enabled Layer 4 fields match ■ All enabled Layer 3 and Layer 4 filter fields match When more than one filter matches, this bit gives the layer 4 filter status of filter indicated by the following bits of the status word. ■ MADRM[5:0]: When DHLFRS field of MAC_Packet_Filter register is set to 0x02 ■ Bits[31:29]: Otherwise For a tunneled packet, only the outer L3-L4 header fields are used for filter comparison.
27	L3FM	Layer 3 Filter Match When this bit is set, it indicates that the received packet matches one of the enabled Layer 3 IP Address fields. This status is given only when one of the following conditions is true: ■ All enabled Layer 3 fields match and all enabled Layer 4 fields are bypassed ■ All enabled filter fields match When more than one filter matches, this bit gives the layer 3 filter status of filter indicated by the following fields of the status word: ■ MADRM[5:0]: When DHLFRS field of MAC_Packet_Filter register is set to 0x02 ■ Bits[31:29]: Otherwise For a tunneled packet, only the outer L3-L4 header fields are used for filter comparison.
26:19	MADRM	MAC Address Match or Hash Value or L3L4 Filter Match ■ When the DHLFRS field of the MAC_Packet_Filter register is set to 0x0 or 0x1: ❑ When (DAIF = 0 & HFDAF = 2'b00) or (DAIF = 1 & HFDAF = 2'b01), this field contains the MAC address register number that matched the destination address of the received packet. ❑ When (DAIF = x & HFDAF = 2'b1x), this field contains the hash value computed by the MAC. A packet passes the hash filter when the bit corresponding to the hash value is set in the hash filter register. For other combinations of DAIF & HFDAF, this field is unused. ■ When the DHLFRS field of the MAC_Packet_Filter register is set to 0x2:

Table 13-18 RDES2 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		If Bit 28 or Bit 27 is set to 1, MADRM field indicates the Layer 3 and Layer 4 filter number that matched for the received packet. The number of lower bits of the MADRM (1, 2, 3, 4, or 5 bits) that are valid is based on the number of L3-L4 filters selected (2, 4, 8, 16, or 32); remaining bits are reserved. ■ When the DHLFRS field of the MAC_Packet_Filter register is set to 0x3: This field is reserved. For a tunneled packet, only the outer Ethernet header fields are used for filtering.
18:17	HFDAF	Hash Filter and Destination Address Filter Status This field gives combined result for both Hash DA & Perfect DA filter status. ■ When DAIF = 0 ❑ 2'b00: Perfect DA filter pass and don't care Hash DA filter. When DHLFRS is 0x0 or 0x1. MADRM = MAC address register number that matched the DA of the received packet. ❑ 2'b01: Both Perfect DA filter & Hash filter fail When DHLFRS is 0x0 or 0x1, MADRM = unused. ❑ 2'b1x: Perfect DA filter fail & Hash filter pass ● When DHLFRS = 0, x = 0 and MADRM = [7:0] bit of DA hash table index. ● When DHLFRS = 1, x = 11th bit, if DA hash table size>2048 x = 0, otherwise. L3L4M = [10:8] bit and MADRM = [7:0] bit of DA hash table index ● When DHLFRS != (0 or 1) x = 0 ■ When DAIF = 1 ❑ 2'b00: Perfect DA filter pass (no match for Perfect DA) and Hash DA filter pass (no match for Hash DA) When DHLFRS is 0x0 or 0x1, MADRM = unused ❑ 2'b01: Perfect DA filter fail (match for Perfect DA) and don't care Hash DA filter When DHLFRS is 0x0 or 0x1, MADRM = MAC address register number that matched the DA of the received packet ❑ 2'b1x: Perfect DA filter pass (no match for Perfect DA) & Hash DA filter fail (match for Hash DA). ● When DHLFRS = 0 x = 1 and MADRM = [7:0] bit of DA hash table index ● When DHLFRS = 1 x = 11th bit, if DA hash table size > 2048

Table 13-18 RDES2 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		x = 0, otherwise. L3L4M = [10:8] bit and MADRM = [7:0] bit of DA hash table index ● When DHLFRS != (0 or 1) x = 1 For a tunneled packet, only the outer Ethernet header fields are used for filtering
17	DAF	Destination Address Filter Fail When this bit is set, it indicates that the packet failed the DA Filter in the MAC. For a tunneled packet, only the outer Ethernet header fields are used for filtering.
16	SAF	SA Address Filter Fail When this bit is set, it indicates that the packet failed the SA Filter in the MAC. For a tunneled packet, only the outer Ethernet header fields are used for filtering.
15	VF	VLAN Filter Status When this bit is set, it indicates that the VLAN Tag of received packet passed or bypassed the VLAN filter. For a tunneled packet, only the outer Ethernet header fields are used for filtering.
14	RPNG	Response Packet Not Generated For ARP packets, this bit is set when the MAC is busy transmitting ARP reply to earlier ARP request (only one ARP request is processed at a time or when the Register bit ARPEN = 0). Similarly, for PTP packets, this bit is set when the MAC is busy transmitting a PTP response to an earlier PTP packet. This bit is reserved when the Enable IPv4 ARP Offload option or PTP Offload feature are not selected. RPNG field is reserved for tunneled packet.
13	IOS	IVT-OVT Select This bit when set to 1 and when ELD=1, indicates that RDES0[31:6] field is interpreted as ELRD and contains the data read from the External Lookup Interface (xgmac_elrd_i bus). When this bit is 0 and ELD=1, then RDES[15:0] contains the ELRD field. This bit is reserved when “Enable External Lookup Interface” is not selected or when current packet is a tunneled packet (TNP=1).
12	ELD/FRPSM2	ELRD valid / Flexible Receive Parser Status bit[2] When "Enable Flexible Receive Parser" is selected, it is Flexible Receive Parser Status bit[2].

Table 13-18 RDES2 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		When “Enable External Lookup Interface” is selected: ■ This bit when set to 1 indicates that either RDES0[31:6] field or RDES0[15:0] field is interpreted as ELRD and contains the data read from the External Lookup Interface (xgmac_elrd_i bus). ■ This bit when set to 0, RDES0 reverts back to the default IVT, OVT fields. ■ This bit is valid and set only when ELEN bit is set in `MAC_Rx_Configuration` register and the xgmac_elrd_val_i input signal from the External Lookup interface is active for this packet. This bit is reserved when any or both of the following conditions are true: ■ “Enable External Lookup Interface” is not selected or when the current packet is a tunneled packet (TNP=1). ■ "Enable Flexible Receive Parser" is not selected.
11	TNP	Tunnel Packet When this field is ‘1’, received packet is a tunneled packet. When this field is ‘0’, received packet is a non-tunneled packet. Tunneled packet recognition is enabled only when VNE of MAC_Tx_Configuration register is set. Note: This field exists only when the EDMA field of the MAC_HW_Feature0 register is selected; Otherwise this field is Reserved.
10	FRPSM	Flexible Receive Parser Status bit[1] For details see, Table 13-20 on page 1825.
9:2	HL	L3/L4 Header Length This field contains the length of the header of the packet split by the MAC at L3 or L4 header boundary as identified by the MAC receiver. This field is valid only when the first descriptor bit is set (FD = 1). The header data is written to the Buffer 1 address of corresponding descriptor. If header length is zero, it implies that the MAC did not identify and split the header and both Buffer 1 and Buffer 2 are used for storing the packet. For VxLAN tunneled packets, header length = (header length of outer packet + eight bytes of VxLAN header + header length of inner packet). For NVGRE tunneled packets, header length = (outer packet’s header length +inner packet’s header length). This field is valid when the Enable Split Header Structure option is selected. For details about the header length availability in RDES2, see Table 13-28 on page 1835.
1	AVTDP	AV Tagged Data Packet When L34T = 0000, this bit is set to 1 when AV Tagged Data packet is received When L34T is non-zero, this bit is HL[1] for packet split by the MAC at L3 or L4 header.
0	AVTCP	AV Tagged Control Packet When L34T = 0000, this bit is set to 1 when AV Tagged Control packet is received

Table 13-18 RDES2 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		When L34T is non-zero, this bit is HL[0] for packet split by the MAC at L3 or L4 header.

13.5.2.4 RDES3 Normal Descriptor (Write-Back Format)

In the write-back format, all the status fields (except bits[31:28]) are valid only for the last descriptor (when RDES3[28] is set).

Table 13-19 RDES3 Normal Descriptor (Write-Back Format)

Bit	Name	Description
31	OWN	Own Bit DMA clears this bit (to 0) to indicate that it has transfered the data to the buffers and the descriptor is now owned by the software. The DMA clears this bit when either of the following conditions is true: ■ The DMA completes the packet transfer ■ The buffers associated with the descriptor are full
30	CTXT	Receive Context Descriptor When this bit is set, it indicates that the current descriptor is a context type descriptor. The DMA writes 1'b0 to this bit for normal receive descriptor.
29	FD	First Descriptor 1: Indicates that this descriptor contains the first buffer of the packet. If the size of the first buffer is 0, the second buffer contains the beginning of the packet. If the size of the second buffer is also 0, the next descriptor contains the beginning of the packet. CTXT, FD, and LD bits together indicate Descriptor Definition Error. During write-back, all three bits are set to 1 {CTXT, FD, LD} = 3'b111, to indicate Descriptor Definition Error in the Rx Descriptor. Descriptor Definition Error for Rx Descriptor is indicated when both Buffer-1 and Buffer-2 are all 1s. On encountering this error, the Receive DMA terminates the ongoing DMA transfer, flushes the ongoing packet in the Receive Queue and enters stop state. The DEE and RPS fields of the corresponding `VDMA_CH(#i)_Status` register is set to 1. **Recovery Mechanism:** Perform a software reset by programming the SWR field of the DMA_Mode register to 1.
28	LD	Last Descriptor When this bit is 1, ■ Indicates that the buffers to which this descriptor is pointing. Contains end of packet. ■ Rest of the status fields in RDES0, RDES1, RDES2, and RDES3 provide information about the packet.
27	CDA	Context Descriptor Available

Table 13-19 RDES3 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		When this bit is set, the next descriptor following this descriptor is a Context Descriptor.
26	RSV	RSS Valid When LINST field (of MTL_RXP_Control_Status) is present and it is set tso 1 and FRP feature is enabled then, this field will be set to 0. Otherwise,when this bit is set, it indicates that valid RSS Hash is available in RDES1[31:0]. This bit is valid only when the RSS function is enabled MAC_RSS_Control register. In case of non-receive channel list based frame duplication, when packet is duplicated to multiple Receive DMA channels, this field is 0.
25	ISP	In Sequence Packet When this bit is reset, it indicates that this packet is not in sequence (TCP sequence number) with respect to the previous packet transferred by this DMA, or the RSS Hash value is different for this packet with respect to the RSS Hash calculated and given for the previous packet. When this bit is set, it indicates that this TCP packet is in sequence with respect to the previous packet transferred by this DMA and the RSS hash values also match. The application can do a TCP coalescing of this packet with the previous packet transferred by this DMA. This bit is valid only when the OoS function is enabled in MAC_RSS_Control register. This bit is not valid if it is a tunneled packet.(TNP=1)
24	E TM/NCP	Ether Type Match / No Coalescing Packet When L34T=000, this bit is set to 1 (ETM Status) when the “Type” field of the received packet matches the value programmed in the MAC_Rx_Eth_Type_Match register. When L34T has any value other than 000 and this bit is set to 1 (NCP Status), it indicates that the DMA has received a packet that must not be kept holding for TCP packet coalescing. It must be forwarded to the TCP/IP stack immediately even though it may be in sequence with respect to the previous packet. NCP Status is valid only when the OoS function is enabled in the MAC_RSS_Control register. For a tunneled packet, ETM refers to the Type Match of the Inner Ethernet Header. NCP is not supported for a tunneled packet.
23:20	L34T	Layer3 / Layer4 packet Type These bits indicate the type of payload encapsulated in the IP datagram and processed by the Receive Checksum Offload Engine (RxCOE). For a tunneled packet, the Layer3/Layer4 packet type refers to the L3/L4 type of the inner packet. This field is valid only when the LD field is set. ■ 0000: Not an IP packet ■ 0001: IPv4 packet with TCP Payload as Layer4 ■ 0010: IPv4 packet with UDP Payload as Layer4

Table 13-19 RDES3 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		■ 0011: IPv4 packet with ICMP Payload as Layer4 ■ 0100: IPv4 packet with IGMP Payload is Layer4 ■ 0101 to 0110: Reserved ■ 0111: IPv4 with unknown (unsupported) payload ■ 1000: Reserved ■ 1001: IPv6 packet with TCP Payload as Layer4 ■ 1010: IPv6 packet with UDP Payload as Layer4 ■ 1011: IPv6 packet with ICMP Payload as Layer4 ■ 1100 to 1110: Reserved ■ 1111: IPv6 with unknown (unsupported) payload
19:16	ET/LT	Error Type or L2 Type When the 15th bit of this descriptor (ES) is set to 1, this field indicates the Error Type. When the 15th bit of this descriptor (ES) is set to 0, this field indicates the L2 Packet Type. Following is the encoding: When ES = 1 (Error Type Encoding) ■ 0000: Reserved ■ 0001: Watchdog Timeout Error It indicates that the Receive Watchdog Timer has expired while receiving the current packet. The current packet is truncated after watchdog timeout ■ 0010: Invalid Code Error or GMII Error In XGMII mode, it indicates that the frame was terminated due to the reception of an invalid code character. In GMII mode, it indicates that the RX_ER and RX_DV signals were asserted for the frame, indicating a receive error signaled to the GMII. ■ 0011: CRC Error It indicates that a Cyclic Redundancy Check (CRC) Error occurred on the received packet. ■ 0100: Giant Packet Error It indicates that the packet length exceeds the specified maximum Ethernet size of 1518, 1522, or 2000 bytes (9018 or 9022 bytes if jumbo packet enable is set). Note: Giant packet indicates only the packet length. It does not cause any packet truncation. ■ 0101: IP Header Error It indicates that the 16-bit IPv4 Header checksum calculated by the 25GMAC controller does not match the received Checksum bytes or that the IP datagram version is not consistent with the Ethernet Type value. This error is possible only when you select the Enable Receive TCP/IP Checksum Check feature ■ 0110: Payload (L4) Checksum Error

Table 13-19 RDES3 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		It indicates that the 16-bit IP payload checksum (that is, the TCP, UDP, or ICMP checksum) calculated by the 25GMAC controller does not match the received TCP, UDP, or ICMP checksum field. ■ 0111: Overflow Error It indicates that the received packet is damaged because of buffer overflow in Rx FIFO. Note: This bit is set only when the DMA transfers a partial packet to the application. This happens only when the Rx FIFO is operating in the threshold mode. In the store-and-forward mode, all partial packets are dropped completely in Rx FIFO. ■ 1000: Bus Error It indicates that AXI Bus Error response is received while fetching this descriptor or when packet is being transferred to its buffers. ■ 1001: Length Error It indicates that the Length value of the current frame is inconsistent with the total number of bytes received in the current frame. This is applicable when Frame Length value is less than 1500. This error is reported when the following type of frames are received: ❑ Frames in which the value of the Length field is less than 46 bytes and the length of the received frame is less than the following: Length field + 18 bytes ❑ Frames in which the value of the Length field is equal to or greater than 46 bytes and the length of the received frame is not equal to the following: Length field + 18 bytes ■ 1010: Good runt packet. Packet less than 64 bytes but with good CRC. ■ 1011: Reserved ■ 1100: Dribble Error Indicates that, in the 10/100Mbps speed mode, the packet has ended without a byte boundary. ■ 1101 -1110: Reserved for non-tunneled packets ■ 1111: Safety Error In the case of tunneled packet (TNP = 1), the error status is extended to differentiate between the errors in outer Header or Inner Header. In the status types, the following values are applicable for the Outer Header fields: ■ 0101: Outer IP Header Error ■ 0110: Outer Header's Payload (L4) Checksum Error Additionally, the following Error Status types are applicable ■ 1001: Inner Packet IP Header Error ■ 1010: Inner Packet Payload (L4) Checksum Error ■ 1011: Invalid VxLAN Header Field

Table 13-19 RDES3 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		It indicates that the VxLAN Header Flag field did not have the value 0x08 ■ 1101-1110: Reserved for tunneled packets When ES = 0 (L2 Packet Type Encoding) ■ 0000: The packet is a length packet. ■ 0001: The packet is a MAC Control packet. ■ 0010: DCB Control Packet.(Note: Inner Header of a tunneled packet is not parsed for DCB Control type) ■ 0011: The packet is of ARP Request Packet Type. ■ 0100: The packet is of OAM packet type.(Note: Inner header of a tunneled packet is not parsed for OAM Control type) ■ 0101: The untagged packet has a Type field which matched the value in the MAC_Rx_Eth_Type_Match register ■ 0110 – Untagged AV Control packet ■ 0111: The packet is a other Type Packet. ■ 1000: The packet is type packet with Single SVLAN tag. ■ 1001: The packet is type packet with Single CVLAN tag. ■ 1010: The packet is type packet with Double VLAN tag with outer VLAN is CVLAN and inner is also CVLAN. ■ 1011: The packet is type packet with Double VLAN tag with outer VLAN is SVLAN and inner is SVLAN. ■ 1100: The packet is type packet with Double VLAN tag with outer VLAN is SVLAN and inner is CVLAN ■ 1101: The packet is type packet with Double VLAN tag with outer VLAN is CVLAN and inner is SVLAN. ■ 1110 - 1111: Reserved For a tunneled packet, the L2 packet type encoding refers to the L2 type of the Inner packet.
15	ES	Error Summary This bit indicates that the received packet has an error. The type of error is indicated by bits [19:16]. This field is valid only when the LD bit of RDES3 is set. When this field is 1, indicates safety error.
14	FRPSL	Flexible Receive Parser Status bit[0] For details see, Table 13-20 on page 1825.
13:0	PL	Packet Length

Table 13-19 RDES3 Normal Descriptor (Write-Back Format) (continued)

Bit	Name	Description
		These bits indicate the byte length of the received packet that was transferred to system memory. This does not include the VLAN/Pad/CRC bytes if VLAN/Pad/CRC stripping is enabled. This field is valid when the LD (bit[28]) field of RDES3 is set and there is no overflow error indication in ET/LT(bits[19:16]).

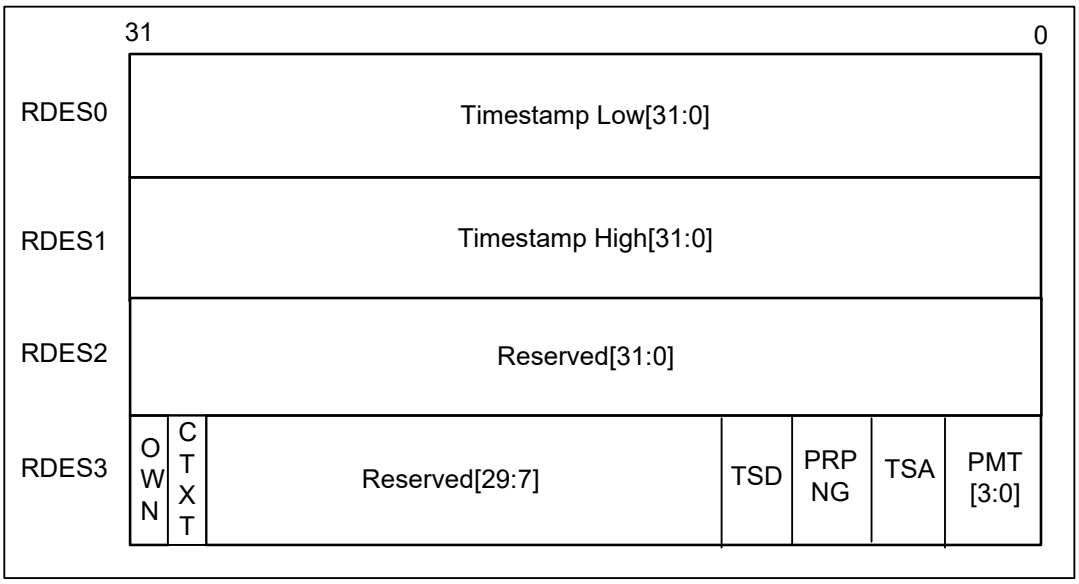
Table 13-20 Flexible Receive Parser Receive Status

RDES2 Bit 12 FRPSM2	RDES2 Bit 10 FRPSM	RDES3 Bit 14 FRPSL	FRP Status
1'b0	1'b0	1'b0	FRP accept, routing-bypass, or result-bypass
1'b0	1'b0	1'b1	FRP Reject/Drop or FRP safety error
1'b0	1'b1	1'b0	Found Parsing Error ■ Current Instruction Number > NVE[] ■ Number of Parsed Instructions > NPE[] + 1 ■ Current Instruction's Frame offset > EOF frame offset or last parsable frame offset.
1'b0	1'b1	1'b1	Incomplete Parsing ■ Parsing is not complete even after receiving the next to next packet's SOF.
1'b1	1'b0	1'b0	FRP routing by-pass
1'b1	1'b0	1'b1	Stream Gate Filter fail/drop
1'b1	1'b1	1'b0	Policing Counter fail/drop
1'b1	1'b1	1'b1	Safety Error

13.5.3 Receive Context Descriptor (Write-Back Format)

The Receive context descriptor provides information about the extended status of the last received packet. The Bit 30 of RDES3 indicates the context type descriptor.

Figure 13-10 Receive Context Descriptor



13.5.3.1 RDES0 Context Descriptor

Table 13-21 RDES0 Context Descriptor

Bit	Name	Description
31:0	RTSL	Receive Packet Timestamp Low The DMA updates this field with least significant 32 bits of the timestamp captured for corresponding Receive packet. When this field and the RTSH field of RDES1 show all-ones value, the timestamp must be considered as corrupt.

13.5.3.2 RDES1 Context Descriptor

Table 13-22 RDES1 Context Descriptor

Bit	Field	Description
31:0	RTSH	Receive Packet Timestamp High The DMA updates this field with most significant 32 bits of the timestamp captured for corresponding receive packet. When this field and the RTSL field of RDES0 show all-ones value, the timestamp must be considered as corrupt.

13.5.3.3 RDES2 Context Descriptor

Table 13-23 RDES2 Context Descriptor

Bit	Description
31:0	Reserved

13.5.3.4 RDES3 Context Descriptor

31	30	29:7	6	5	4	3:0
OWN	CTXT	Rsvd	TSD	PRPNG	TSA	PMT

Table 13-24 RDES3 Context Descriptor

Bit	Name	Description
31	OWN	Own Bit DMA clears this bit to 0, to indicate that the status is written in the descriptor and is now owned by the software.
30	CTXT	Receive Context Descriptor When this bit is set, it indicates that the current descriptor is a context descriptor. The DMA writes 1'b1 to this bit for context descriptor.
29:7	Rsvd	Reserved
6	TSD	Timestamp Dropped When set, it indicates that a timestamp was captured for this packet but was dropped due to the lack of space in Rx FIFO. The host must consider the RTSL and RTSH as valid only when TSD=0 and TSA=1
5	PRPNG	PTP response Packet Not Generated When set, it indicates that a PTP Response packet was not generated by the MAC for this received PTP Packet. This bit is valid only when the PTP Offload Feature is enabled in your configuration.
4	TSA	Timestamp Available When set, this bit indicates that the timestamp value is available in RDES0 and RDES1 of the context descriptor.
3:0	PMT	PTP Message Type: These bits are encoded to give the type of the PTP message received: ■ 0000: No PTP message received ■ 0001: SYNC (all clock types) ■ 0010: Follow-up (all clock types) ■ 0011: Delay_Req (all clock types) ■ 0100: Delay_Resp (all clock types) ■ 0101: Pdelay_Req (in peer-to-peer transparent clock) ■ 0110: Pdelay_Resp (in peer-to-peer transparent clock) ■ 0111: Pdelay_Resp_Follow_Up (in peer-to-peer transparent clock) ■ 1000: Announce ■ 1001: Management

Table 13-24 RDES3 Context Descriptor (continued)

Bit	Name	Description
		■ 1010: Signaling ■ 1011-1110: Reserved ■ 1111: PTP packet with Reserved message type

13.6 Enhanced Descriptor for Time-Based Scheduling

The time-based scheduling feature needs Enhanced Descriptors (that are 32 Bytes) to be enabled on all the required DMA channels that intend to use the feature

1. By configuring the number of channels that use the time-based scheduling feature.

   Use the DWCXG_NUM_DMA_TBS_CH parameter to specify the number of DMA channels that use the time-based scheduling feature. Specify the channels in the descending order

2. Setting the EDSE field of the VDMA_CH(#i)_TX_Control register

The structure of 32-byte Descriptor for the Context and the Normal Descriptor in Read and Write formats are described in the following sections:

13.6.1 Enhanced Normal Descriptor, Read Format

The fields present in the first 16 bytes of the Enhanced Descriptor format of normal descriptor are called TDES4, TDES5, TDES6, and TDES7 as shown in Figure 13-11. TDES0-TDES3 remain the same as in default Normal Descriptor format, as described in "Transmit Normal Descriptor (Read Format)" on page 1800.

Figure 13-11 Enhanced Normal Descriptor - Read Format

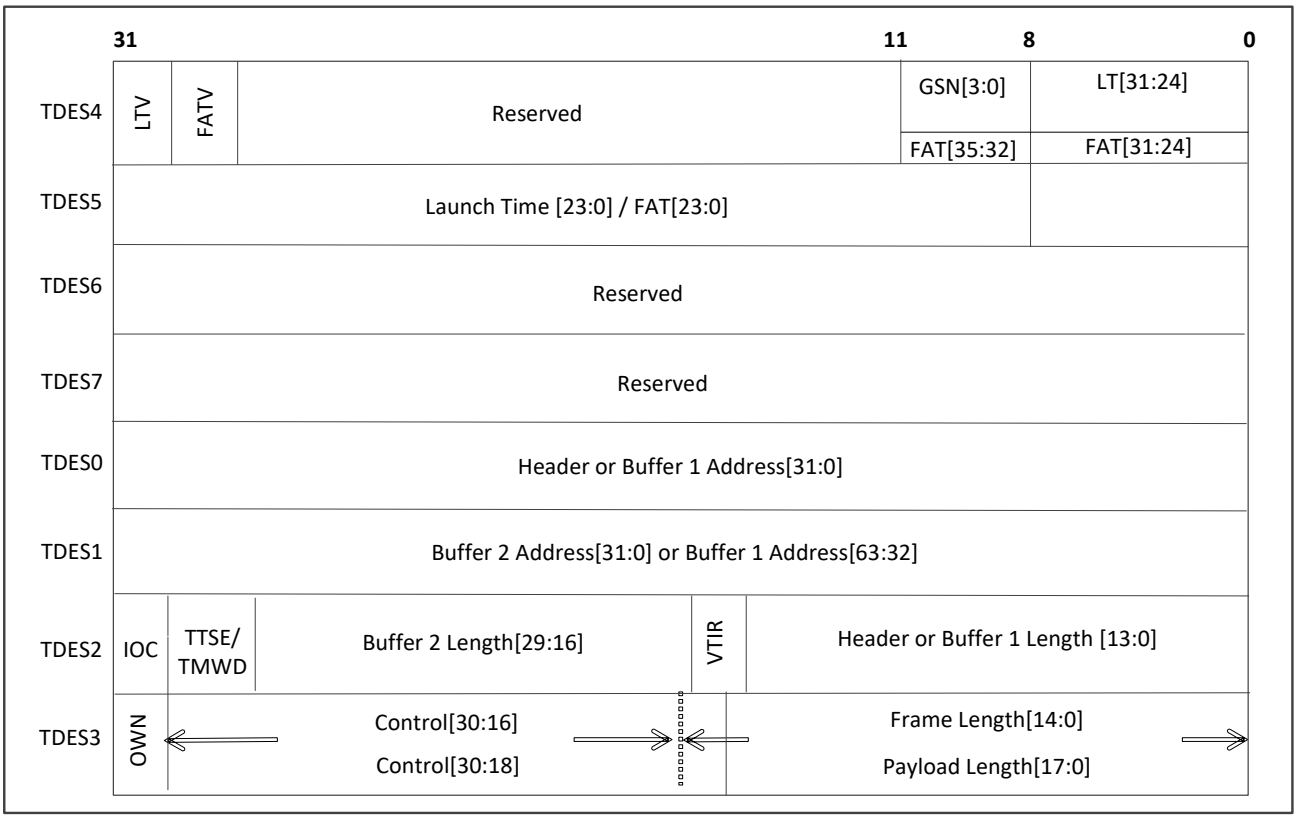


Table 13-25 TDES4 Enhanced Normal Descriptor - Read Format

Bit	Name	Description
31	LTV	Launch Time Valid

Table 13-25 TDES4 Enhanced Normal Descriptor - Read Format (continued)

Bit	Name	Description
		When FD = 0, this field indicates that the Launch Time (LT) and GSN fields present in the descriptor are valid. Set this field to 1, only if the FD field of the descriptor is 0.
30 :10	Rsvd	Reserved
11:8	GSN	GCL Slot Number This field indicates the GCL slot number associated with the packet.
7:0	LT	Launch Time[31:23] This field indicates the Launch Time in seconds, associated with the packet. This value is compared with the least significant 8 bits of the seconds field of the time maintained in 25GMAC, for PTP protocol.

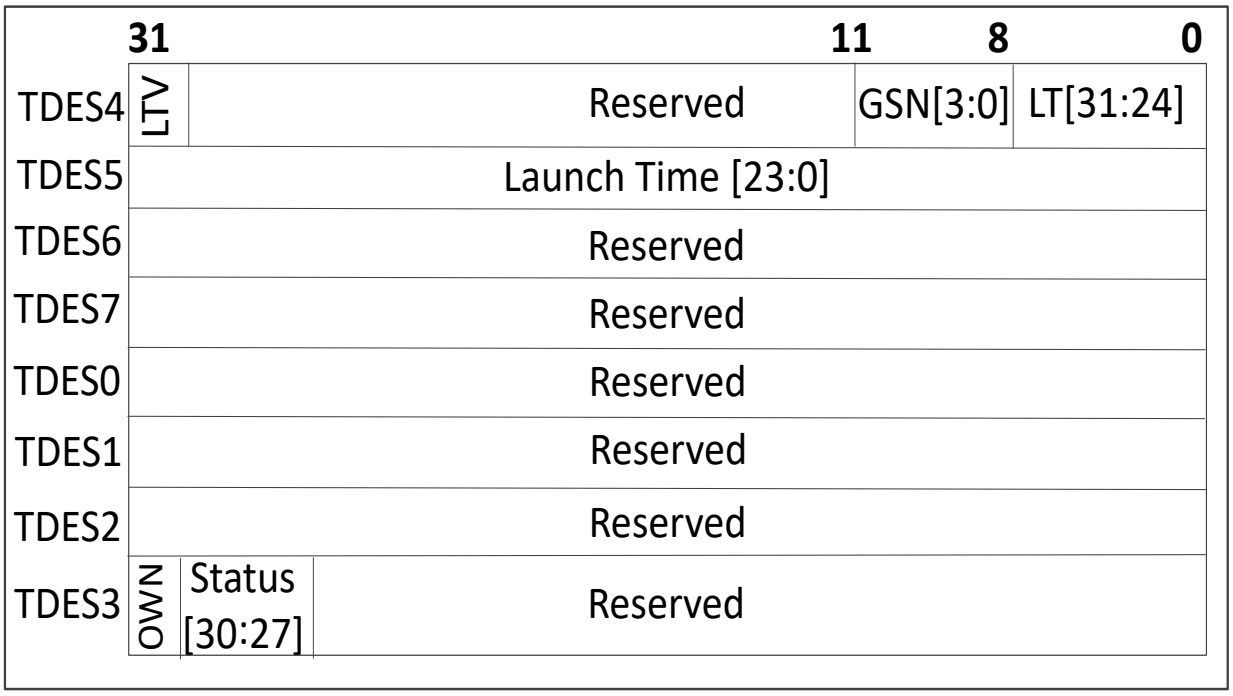
Table 13-26 TDES5 Enhanced Normal Descriptor - Read Format

Bit	Name	Description
31:8	LT	Launch Time (in nanoseconds) This indicates the Launch Time in nanoseconds, associated with the packet. This value is compared with the bits[31:8] of the time maintained in the controller, for PTP protocol. As the least significant 8 bits of the time in nanoseconds is ignored, the granularity of this value is 256ns.
7:0		Reserved

13.6.2 Enhanced Normal Descriptor, Write-Back Format

When the DMA closes the enhanced descriptor, it updates only the TDES0, TDES1, TDES2, and TDES3 fields. So, the values in the TDES4-TDES7 fields are retained in the host memory.

Figure 13-12 Enhanced Normal Descriptor - Write-Back Format



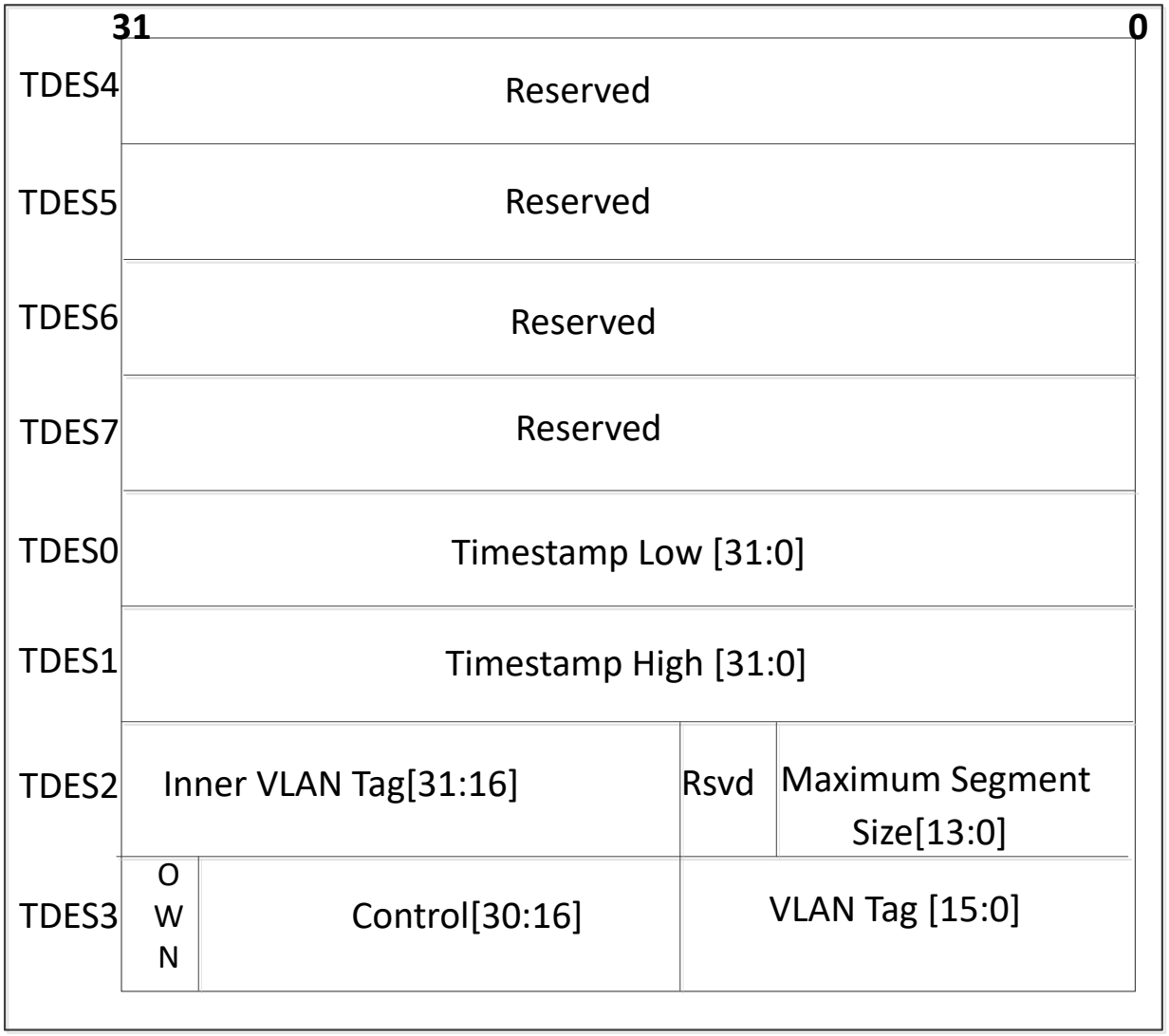
Note

- All the enhanced descriptor examples, assumes a 32-bit data structure. However, the same definitions are valid in 64 and 128 bit configurations.
- In the write back format, no modifications is done to the extended 16 bytes. The rest of the 16 bytes (TDES0 to TDES3) are written back as per the previous 16 bytes descriptor format as described in “Transmit Normal Descriptor (Write-Back Format)” on page 1805.
- Fetch time when enabled overrides the AV slot function
- When an unaligned new GCL list (with a different CTR) is installed it is recommended not to have traffic during the installation of the new list. Any traffic during the switching of the lists might have unpredictable behavior regarding Fetch, Launch, and Launch expiry as the CTR and BTR values get updated while the frame is being processed

13.6.3 Enhanced Context Descriptor, Read Format

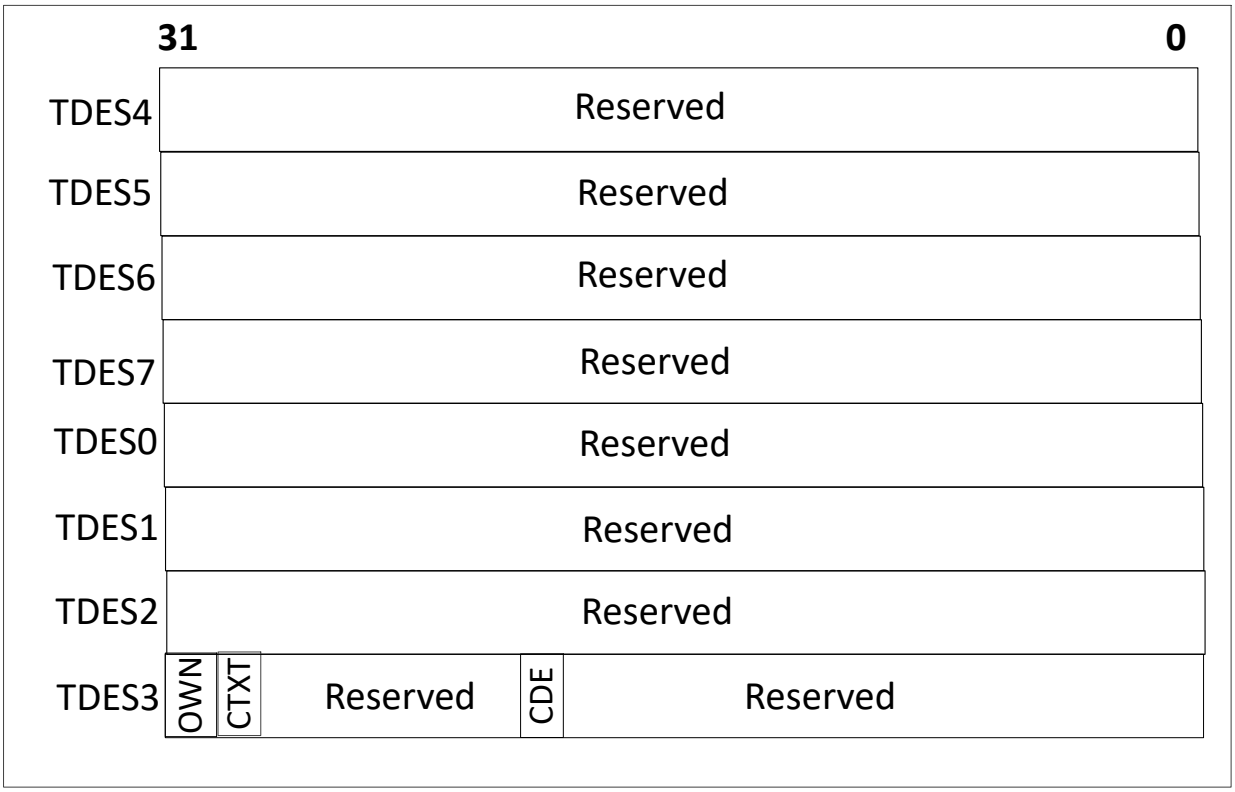
The first 16 bytes of the Enhanced context descriptor (TDESC4 to TDESC7) are reserved and must be zeros. The fields present in last bytes of the Descriptor (TDESC0 to TDESC3) are same as the context descriptor (TDESC0 to TDESC3) in 16 byte format. For details of the context descriptor format, see “Transmit Context Descriptor” on page 1807.

Figure 13-13 Enhanced Context Descriptor - Read



13.6.4 Enhanced Context Descriptor, Write

Figure 13-14 Enhanced Context Descriptor - Write



Note In the write-back format, no modifications are done to the extended 16 bytes in the TDES4, TDES5, TDES6, and TDES7 fields. The rest of the 16 bytes (TDES0 to TDES3) are written back as per the previous 16 bytes of the default normal descriptor format, as described in “Transmit Context Descriptor” on page 1807.

13.7 Header Payload Split Support

This feature supports splitting the packet header and the payload of received packets in separate buffers (buffer-1 and buffer-2). You can enable the split header feature by selecting the Enable Split Header Structure option and setting the SPH bit in the `VDMA_CH(#i)_Control` register.

HDSMS field of `MAC_Rx_Configuration` register indicates maximum header size allowed for splitting the header data in the received packet based on the value programmed. When the ARBS field of the `VDMA_CH(#i)_Rx_Control2` register is programmed to a non-zero value, ARBS field indicates Receive Buffer size for buffer-1 and the RBSZ field indicates the receive buffer size for buffer-2. The ARBS field width is:

- 7 bits for data width of 64 bits
- 6 bits for data width of 128 bits

The granularity is in data-width. The ARBS field can be set to 0, for backward compatibility.

The value in the HDSMS field must be less than or equal to the Buffer-1 size indicated by the ARBS field, for all the channels.

Table 13-27 shows how DMA processes a packet based on the packet type.

Table 13-27 Header Payload Split Support Depending on the Packet Type

Packet Type	SPLM	Description
TCP or UDP Packet	00(L3/L4 Split)	The DMA writes the Ethernet header + IP header + TCP or UDP header into the header buffer.
IP packet (not TCP/UDP)		The DMA writes the Ethernet header + IP header into the header buffer.
Non-IP packet		The DMA does not split the header and payload
Any packet	01 (L2 Split)	The DMA writes the Ethernet header based on split offset (SPLOFST) into the header buffer.
IP packet	10 (Combination of L2 or L3/L4 Split)	L3/L4 split
Non-IP packet		L2 split
NA	11	Reserved

The IP header includes IPv4 options in case of IPv4 packet and IPv6 extension headers in case of IPv6 frames. The points at which the header is split are shown in Figure 13-15.

Note

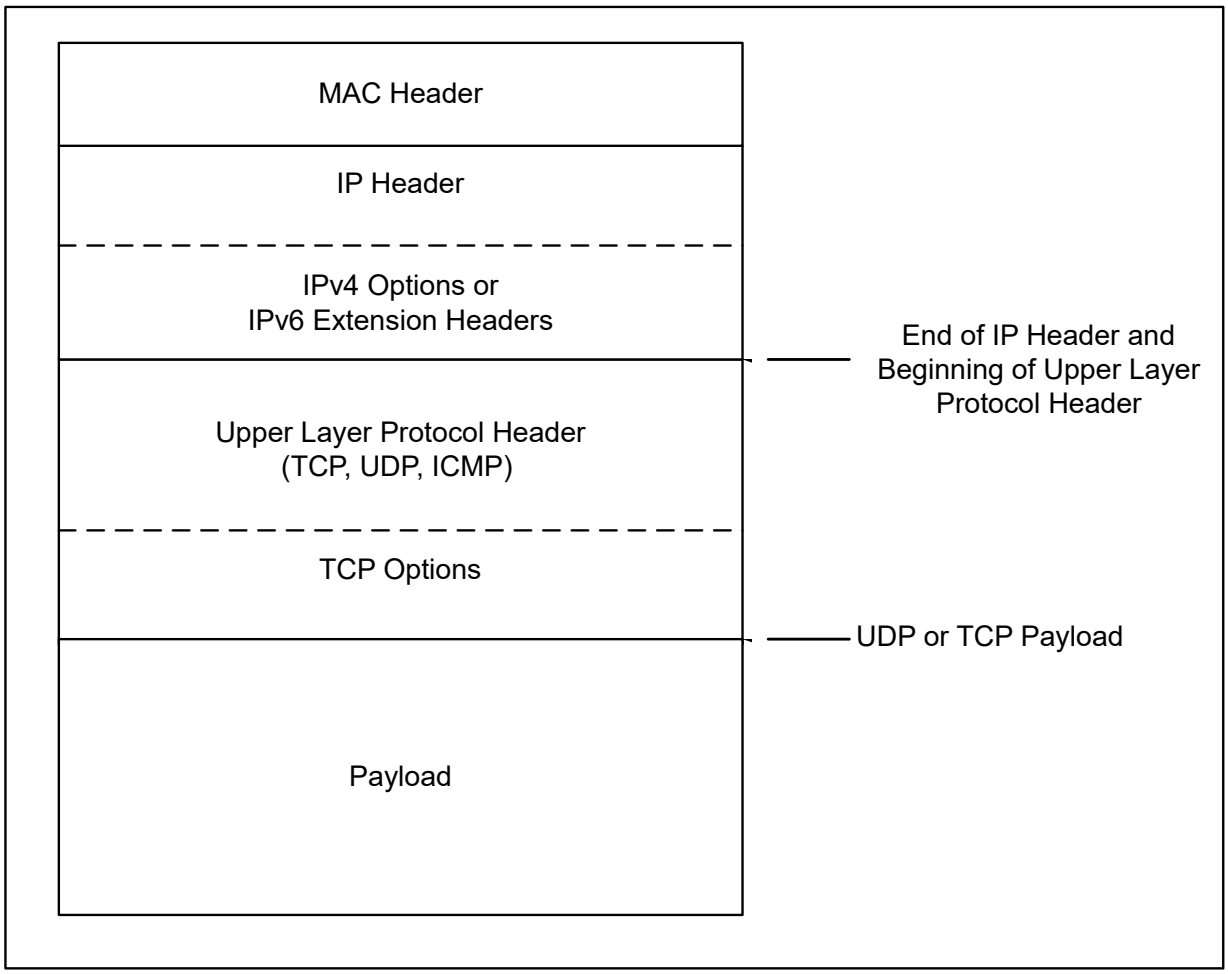
- The VLAN tag stripping must be set for the split function. For instance, the DMA separates the header and payload of an untagged packet only. So, when a tagged packet is received, the program 25GMAC such that the VLAN tags are deleted/stripped from the received packets.
- L3/L4 split is applicable for IP packets that are either untagged or VLAN stripped. If VLAN tag is retained in the packet forwarded to the DMA, L3-L4 split is not performed. However, if SPLM field is set to 2, L2 split is performed for VLAN tagged IP packets.
- For AV packets, to specify an alternative L2 split value, use the SAVE and SAVO fields of the `MAC_Ext_Cfg1` register.
- L2 Split is not supported for packets with variable preamble

Table 13-28 shows the header length availability (HL) in RDES2. This field is valid only when FD = 1 (First Descriptor).

Table 13-28 Header Length Availability in RDES2

Value of FD	Value of LD	Header Length (HL) Availability in RDES2
0	1	HL not available
1	0	HL is [9:0]
1	1	■ IP packets, HL is [9:0] ■ Non-IP packets, HL is [9:2]

Figure 13-15 Header Split Points



13.7.1 Descriptor Structure

Figure 13-16 shows the descriptor structure without the Split Header feature.

Figure 13-16 Descriptors without Split Header Feature

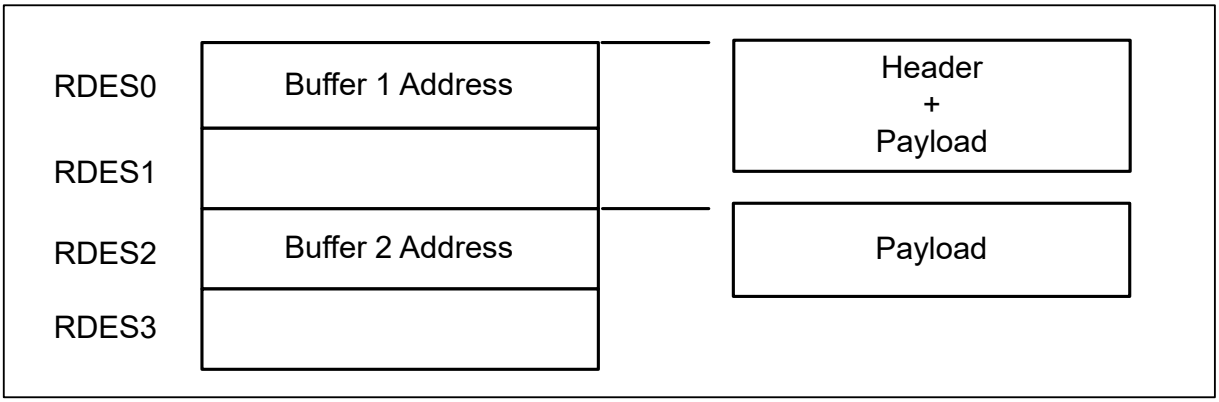
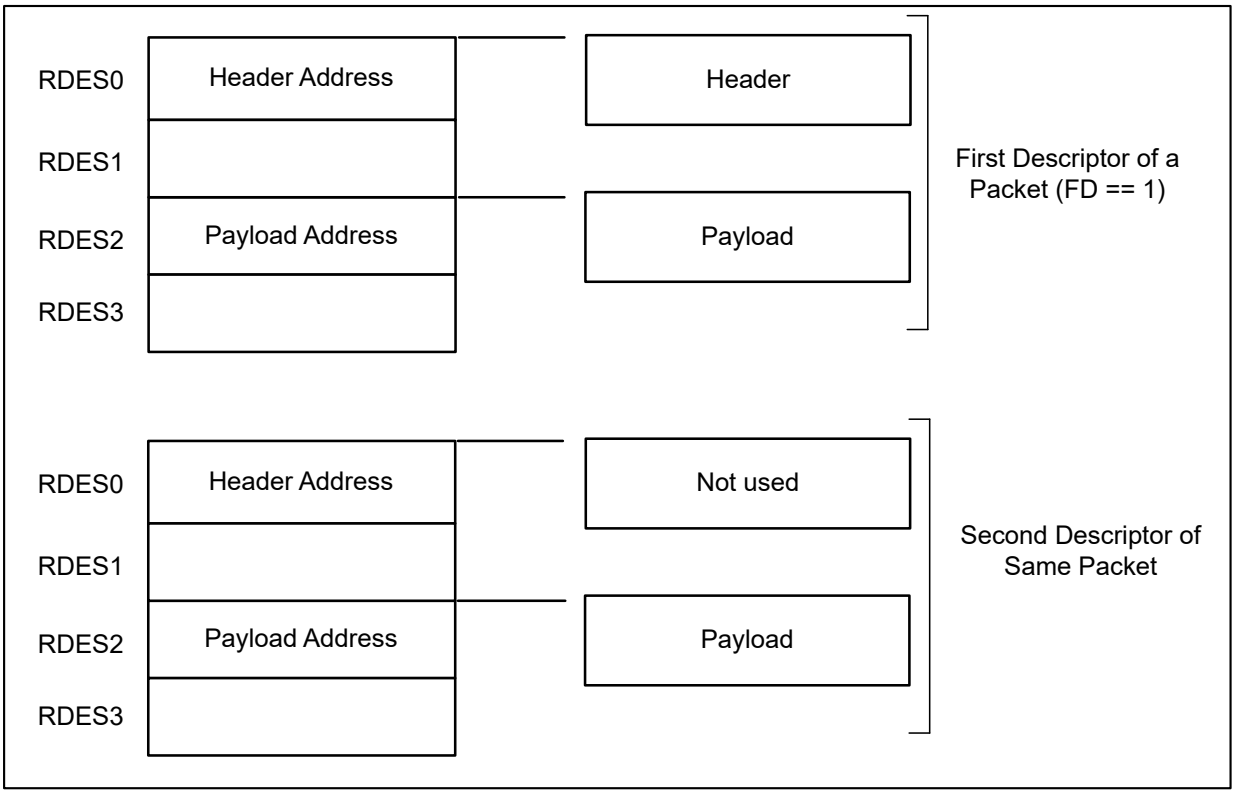


Figure 13-17 shows the descriptor structure with the Split Header feature.

Figure 13-17 Descriptors with Split Header Feature

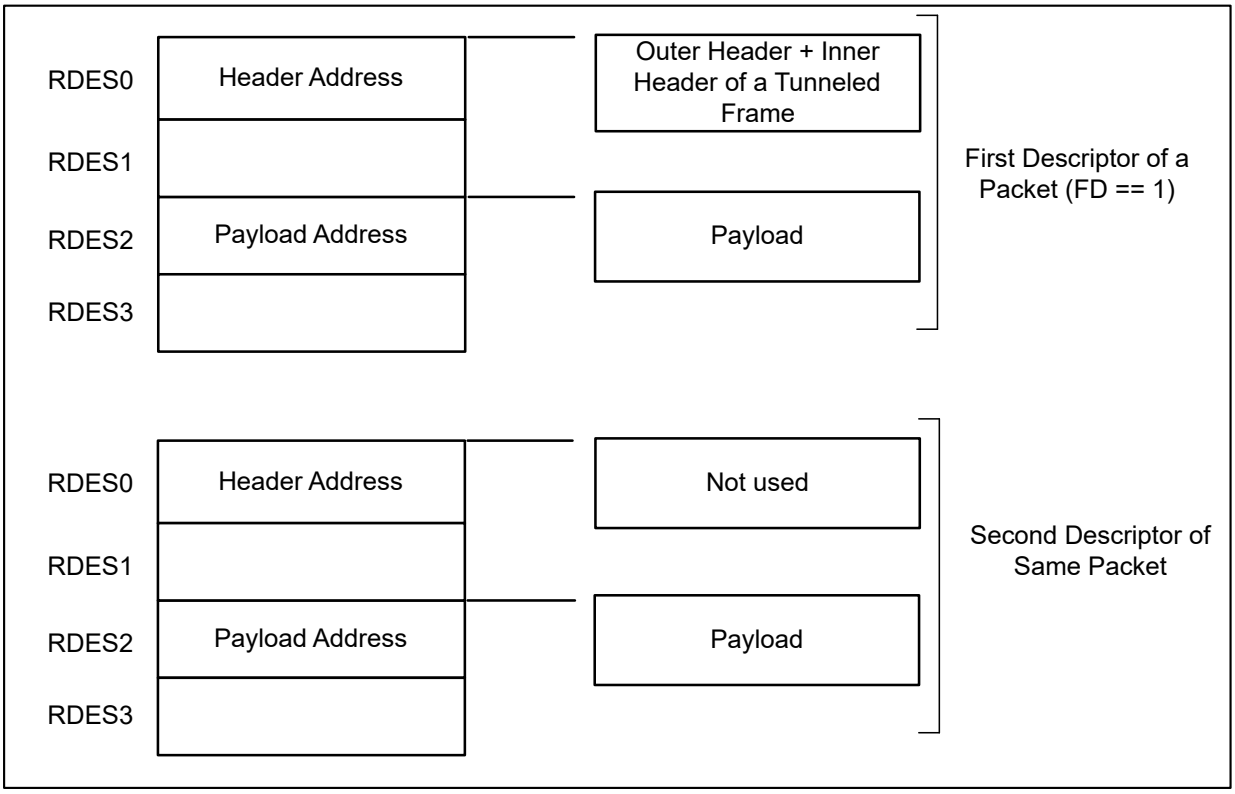


The DMA writes the header of the received packet by using the header address to which the RDES0 in the first descriptor is pointing (FD bit of RDES3 is set). The DMA writes the payload of the received packet into the buffer address to which the RDES2 is pointing. For subsequent descriptors (FD is set to 0), the address to which RDES0 (Header address) is pointing is not used. The payload is written only to buffers to which the RDES2 (payload address) is pointing.

The DMA writes the header length in RDES2 of the first receive descriptor (RDES3[29] (FD Bit) is set) for the packet. The packet length is written in RDES3 of the last receive descriptor (RDES3[28] (LD Bit) set). The buffer length for the payload is set by the driver through the RBSZ field in the corresponding DMA Channel Receive Control Register. The DMA fills receive buffers fully in all except the last descriptor. The maximum header length is limited by the value programmed in the HDSMS field of MAC_Rx_Configuration register.

Figure 13-18 shows the descriptor structure with the Split Header feature for a tunneled packet.

Figure 13-18 Descriptors with Split Header for Tunneled Packets



The header of Outer Packet and Inner Packet are part of data written in the Header buffer. Only the Payload of the Inner packet is written into the buffers pointed by RDES2.

14

Programming Guidelines

This chapter provides initialization sequences used for when the 25GMAC controller is configured with an AXI interface.

This chapter contains the following sections:

- "Initializing the MAC Registers" on page 1841
- "Initializing the MTL Registers " on page 1842
- "Initializing the DMA Registers" on page 1843
- "Programming Guidelines for DMA" on page 1845
- "Performing Receive and Transmit Operation" on page 1847
- "Stopping and Restarting Transmission" on page 1848
- "Reconfiguring the DMA Registers" on page 1849
- "Switching to New Descriptor List in Receive DMA" on page 1850
- "Handling Bus Errors and Recovery" on page 1851
- "Setting Up Multi-Channel and Multi-Queue Operations" on page 1853
- "Programming Guidelines for XGMII Link State Transitions" on page 1856
- "Enabling the Energy Efficient Ethernet" on page 1857
- "Setting up TCP Segmentation Offload (TSO)" on page 1859
- "Enabling the Header-Payload Splitting Offload" on page 1860
- "Enabling the VxLAN/NVGRE Tunneled Packet Processing " on page 1861
- "Programming Sequence for Queue/Channel Based VLAN Inclusion Register" on page 1862
- "Programming Guidelines for External DA Hash Table-Based Filtering and Routing" on page 1863
- "Setting up VLAN Filtering on Receive" on page 1864
- "Setting up Extended VLAN Filtering" on page 1865
- "Setting up External Rx VLAN Tag Lookup-Based Filtering and Routing" on page 1866
- "Setting up the L3-L4 Filtering" on page 1867
- "Setting Up the Flexible Receive Parser" on page 1868

- "MTL Indirect Read/Write Access " on page 1870
- "MAC Indirect Read/Write Access" on page 1872
- "Programming Guidelines for the Policing Counters" on page 1874
- "Setting up the IEEE 1588 PTP Timestamping" on page 1875
- "Setting up the Flexible Pulse-Per-Second (PPS) Output" on page 1877
- "Enabling the Audio Video Traffic Shaper" on page 1879
- "Programming the GCL and GCL Linked Registers" on page 1882
- "Programming Guidelines for EST" on page 1883
- "Enabling the Frame Preemption Function" on page 1885
- "Setting up the Time-Based Scheduling Function" on page 1886
- "Programming Guidelines for the Asynchronous Traffic Shaper (ATS)" on page 1887
- "Programming Guidelines for Media Clock Generation and Recovery" on page 1888
- "Programming Guidelines for ECC" on page 1890
- "Programming Guidelines for On-Chip Datapath Parity Protection" on page 1892
- "Programming Guidelines for FSM and Timeout" on page 1893
- "Programming Guidelines for Switching the AXI Clock Frequency" on page 1894
- "Programming Guidelines for Dual System Time Source Support" on page 1895
- "Programming Guidelines for 25G XGMII Speed Mode and Timestamping Accuracy" on page 1896
- "Programming Guidelines for Enabling Tx Timestamp Status FIFO Streaming Interface " on page 1897

14.1 Initializing the MAC Registers

The following MAC Initialization operations can be performed after DMA initialization. If the MAC initialization is completed before the DMA is configured, enable the MAC receiver (last step in the following sequence) only after the DMA is active. Otherwise, received frames fill the Rx FIFO and cause overflow.

1. Provide the MAC address registers: MAC_Address0_High and MAC_Address0_Low. If more than one MAC address is enabled in the configuration (during configuration in coreConsultant), program the MAC addresses appropriately.
2. Program the following fields to set the appropriate filters for the incoming frames in the MAC_Packet_Filter register:
   - Receive All
   - Promiscuous mode
   - Hash or Perfect Filter
   - Unicast, multicast, broadcast, and control frames filter settings
3. Program the following fields for proper flow control in the MAC_Q0_Tx_Flow_Ctrl register:
   - Pause time and other Pause frame control bits
   - Transmit Flow control bits
   - Flow Control Busy
4. Program the MAC_Interrupt_Enable register, as required, and if applicable, for your configuration.
5. Program the appropriate fields in the MAC_Tx_Configuration and MAC_Rx_Configuration registers. For example, Inter-packet gap while transmission and jabber disable.
6. Set bit 0 in both the MAC_Tx_Configuration and MAC_Rx_Configuration registers to start the MAC transmitter and receiver.

Note

To support Jumbo packets

- For Transmit side, program the JD field in the MAC_Tx_Configuration register to 1.
- For Receive side, program the WD field in the MAC_Rx_Configuration register to 1.

14.2 Initializing the MTL Registers

1. Program the following fields in `MTL_Operation_Mode` register to initialize the MTL operation in case of multiple Tx and Rx queues.
    a. ETS Algorithm (ETSALG) when DCB is enabled
    b. Receive Arbitration Algorithm (RAA) as required
2. Program the Receive Queue to DMA mapping in `MTL_RxQ_DMA_Map`0-3 registers.
3. Program the priorities mapped to the Selected Traffic Classes in MTL_TC_Prty_Map0-1 registers.
4. Program the Transmit Selection Algorithm (TSA) in `MTL_TC(#i)_ETS_Control` register for all the Selected Traffic Classes (n=0, 1, …, selected number of TCs - 1).

Note The ETS Algorithm (ETSALG) programmed in MTL_Operation_Mode takes effect only when the TSA bit for the corresponding Traffic Class is programmed to ETS (2'b10).

1. Program the following fields to initialize the mode of operation in the MTL_TxQ0_Operation_Mode register.
    a. Transmit Store And Forward (TSF)
    b. Transmit Threshold Control (TTC)
    c. Transmit Queue Enable (TXQEN) to value 2'b10 to enable Transmit Queue0
    d. Transmit Queue Size (TQS)
    e. Queue to Traffic Class Mapping (Q2TCMAP)
    Program which traffic class is mapped to the corresponding queue.
2. Program the following fields to initialize the mode of operation in the MTL_RxQ0_Operation_Mode register:
    - Receive Store and Forward (RSF)
    - Flow Control Activation and De-activation thresholds for MTL Receive and Transmit FIFO (RFA and RFD)
    - Error Packet and undersized good Packet forwarding enable (FEF and FUP)
    - Receive Queue Size (RQS)

Repeat previous two steps for all MTL Tx and Rx queues selected in the configuration.

14.3 Initializing the DMA Registers

1. Provide a software reset by programming the SWR (bit[0]) field of the DMA_Mode register. This resets all of the MAC internal registers and logic.
2. Wait for the completion of the reset process. Poll the SWR, bit[0] of the DMA_Mode register. This field is cleared only after the reset operation is complete.
3. Program the following fields to initialize the DMA_SysBus_Mode register:
   a. AAL
   b. UBL
   c. EAME
   d. If fixed burst length is enabled (UNDEF = 0), select the maximum burst length possible on the AXI bus (bits [7:1]).
4. Create a descriptor list for transmit and receive. In addition, ensure that the receive descriptors are owned by DMA (set bit 31 of descriptor TDES3/RDES3). For more information about descriptors, see "Descriptors" on page 1795.

> **Note** Descriptor address from start to end of the ring must not cross the 4GB boundary.

5. Periodically read the HMIC bit of DMA_Mode register, wait until the bit is read as 1.

> **Note** Any register implemented in DMA CSR Memory must not be accessed until the HMIC bit is read as 1.

6. Program the Transmit and Receive Ring length registers (VDMA_CH(#i)_Tx_Control2 and VDMA_CH(#i)_Rx_Control2). The ring length programmed must at the least be the size of the descriptor cache of the corresponding DMA.
7. Initialize receive and transmit descriptor list address with the base address of the transmit and receive descriptor (VDMA_CH(#i)_TxDesc_List_LAddress, VDMA_CH(#i)_RxDesc_List_LAddress).

   Also, program transmit and receive tail pointer registers indicating to the DMA about the available descriptors (VDMA_CH(#i)_TxDesc_Tail_LPointer and VDMA_CH(#i)_RxDesc_Tail_LPointer).

> **Note** For 40- or 48-bit addressing mode, program the higher address List registers (VDMA_CH(#i)_TxDesc_List_HAddress, VDMA_CH(#i)_RxDesc_List_HAddress).

8. Enable the interrupts by programming the VDMA_CH(#i)_Interrupt_Enable register.
9. Program DSCB bit to 1 and wait for the design to clear this bit after completing calculation of DCACHE memory base addresses for Tx/Rx DMAs.
10. Program the settings of the VDMA_CH(#i)_Control, VDMA_CH(#i)_Tx_Control and VDMA_CH(#i)_Rx_Control for the controls such as maximum burst-length (PBL) initiated by DMA, descriptor skip lengths, OSP in case of TxDMA; RBSZ in case of RxDMA, and so on.

11. Enable the required Receive PDMA by programming the RXPEN bit of respective PDMA_CH(#i)_RxExtCfg register by performing the indirect register access.
12. Start the Receive and Transmit DMA by bit 0 of the `VDMA_CH(#i)_Tx_Control` register and bit 0 of the `VDMA_CH(#i)_Rx_Control` register.

Repeat steps 4 to 9 for all DMA Tx and Rx channels selected in the configuration.

14.4 Programming Guidelines for DMA

Program the VDMA to traffic-channel mapping and PDMA to traffic channel mapping registers for both Tx and Rx. This mapping determines how the data transfer takes place to or from the software.

1. Program the VDMA to traffic-channel mapping per Tx and Rx DMA channels. Program the VDMA to traffic-channel mapping for both Tx and Rx using the following register fields:
   - TP2TCMP field of indirect `PDMA_CH(#i)_TxExtCfg` register
   - RP2TCMP field of indirect `PDMA_CH(#i)_RxExtCfg` register
2. Program the PDMA to traffic-channel mapping per PDMA, which is equal to the number of traffic channels for transmit side and number of Rx queues on the receive side. Program the PDMA to traffic-channel mapping for both Tx and Rx using the following register fields:
   - TD2TCMP field of PDMA_CH(#i)_Tx_Control register
   - RD2TCMP field of PDMA_CH(#i)_Rx_Control register

Note If you do not program these register fields, the corresponding VDMA and PDMA are mapped to traffic channel 0. This leads to unwanted mapping.

3. Program the descriptor cache size as follows:
   - Transmit side: TXDCSZ field of VDMA_CH(#i)_TxDescCtrl register.
   - Receive side: RXDCSZ field of VDMA_CH(#i)_RxDescCtrl register.

   Program the cache size field to 3'b000 encoding only if you require VDMA to remain disabled throughout the transaction.
4. Program the descriptor prefetch threshold sizes as follows:
   - Tx: TDPS field of the VDMA_CH(#i)_TxDescCtrl register.
   - Rx: RDPS field of the VDMA_CH(#i)_RxDescCtrl register.
5. Additionally, program the already existing per VDMA fields to valid values prior to starting the VDMA.
6. Program all the other register fields in DMA, MTL, and MAC to valid values before the traffic starts on the corresponding Tx or Rx side. Changing the values of register fields amidst ongoing traffic results in unknown behavior.

Program all the PDMAs mapped to the same traffic channel to have same properties. Table 14-1 shows the details of these register fields:

Table 14-1 Programming Guidelines for 25GMAC

Register	Field	Description
MTL_TxQ(#j)_Operation_Mod	TQS	Transmit Queue Size
	TTC	Transmit Threshold Control
	TSF	Transmit Store and Forward
	FTQ	Flush Transmit Queue

Table 14-1 Programming Guidelines for 25GMAC (continued)

Register	Field	Description
PDMA_CH(#)_TxExtcfg	TPBLx8	Tx 8xPBL mode
	TxPBL	Transmit Programmable Burst Length
	ORRQ	Read Outstanding Request
PDMA_CH(#)_RxExtcfg	RPBLx8	Rx 8xPBL mode
	RxPBL	Receive Programmable Burst Length
	OWRQ	Write Outstanding Request

14.4.1 Programming Guidelines for Idle Timer Function

Program a non-zero value in the register field ITW and the desired value in the ITCU field of `VDMA_CH(#i)_Rx_Interrupt_Watchdog_Timer` register, to enable Idle timer function in the corresponding Rx VDMA channel.

14.4.2 Programming Guidelines for Datapath Parity

When initializing the DMA, the number of TSO engines is based on number of Tx PDMAs. Also, the Tx and Rx descriptor read path from cache memory is common and falls under per PDMA engines. Therefore, while configuring the DMA, IPETSO and IPEDTC fields of DMA_DPP_Control register and TDPES and RDPES register fields of the DMA_DPP_Interrupt_Status registers are per Tx or Rx PDMAs. So, the parity error inserted by the application and the parity error status reported from the DUT, are per Tx or Rx PDMA.

14.5 Performing Receive and Transmit Operation

For normal operation, complete the following steps:

1. For normal transmit and receive interrupts, read the interrupt status. Then, poll the descriptors, reading the status of the descriptor owned by the Host (either transmit or receive).
2. Set appropriate values for the descriptors, ensuring that transmit and receive descriptors are owned by the DMA to resume the transmission and reception of data.
3. If the descriptors are not owned by the DMA (or no descriptor is available), the DMA goes into SUSPEND state. The transmission or reception can be resumed by freeing the descriptors and writing the descriptor tail pointer to the Tx/Rx tail pointer register (`VDMA_CH(#i)_TxDesc_Tail_LPointer` and `VDMA_CH(#i)_RxDesc_Tail_LPointer`).
4. The values of the current host transmitter or receiver descriptor address pointer can be read for the debug process:
   - `VDMA_CH(#i)_Current_App_TxDesc_L`
   - `VDMA_CH(#i)_Current_App_RxDesc_L`
5. The values of the current host transmit buffer address pointer and receive buffer address pointer can be read for the debug process:
   - `VDMA_CH(#i)_Current_App_TxBuffer_L`
   - `VDMA_CH(#i)_Current_App_TxBuffer_H`
   - `VDMA_CH(#i)_Current_App_RxBuffer_L`
   - `VDMA_CH(#i)_Current_App_RxBuffer_H`

14.6 Stopping and Restarting Transmission

Complete the following steps to pause the transmission for some period of time and then restart it:

1. Disable the Transmit DMA (if applicable) by clearing Bit 0 (ST) of the `VDMA_CH(#i)_Tx_Control` register.
2. Wait until the Transmit DMA generates the stopped interrupt (TPS field of the `VDMA_CH(#i)_Status` register).
3. Wait for any previous frame transmissions to complete. You can check this by reading the appropriate fields of the `MTL_TxQ0_Debug` register (`TRCSTS != 1` and `TXQSTS = 0`).
4. Ensure that both Tx Queue and Rx Queue are empty; the `TXQSTS = 0` in the `MTL_TxQ0_Debug` register and `RXQSTS = 0` in the `MTL_RxQ0_Debug` register).
5. Disable the MAC transmitter and MAC receiver by clearing Bit 0 (RE) of the `MAC_Rx_Configuration` and Bit 0 (TE) of the `MAC_Tx_Configuration` register.
6. Disable the Receive DMA (if applicable), after ensuring the data in the Rx FIFO is transferred to the system memory (by reading the appropriate fields of the `MTL_RxQ0_Debug` register; `PRXQ = 0` and `RXQSTS = 00`).
7. Wait until the Receive DMA generates the stopped interrupt, which is available in the RPS field of the `VDMA_CH(#i)_Status` register.
8. Restart the DMAs by programming the `SWR` (bit[0]) field of the `DMA_Mode` register.
9. Enable the MAC Transmitter and Receiver.

14.7 Reconfiguring the DMA Registers

The following reconfiguration flow must be followed if DMA report bus error, or software wants to write to registers whose value is supposed to change only during DMA initialization. For example, DMA_Mode DMA_SysBus_Mode, VDMA_CH(#i)_Tx_Control2 and VDMA_CH(#i)_Rx_Control2 registers.

1. Set DMA start to 1'b0 in the ST field of the VDMA_CH(#i)_Tx_Control or the VDMA_CH(#i)_Rx_Control register.
2. Wait for DMA channel to enter "Process Stop". MAC indicates this by setting the TPS or the RPS field of the VDMA_CH(#i)_Status register to 1.
3. Reconfigure the DMA channel as described in "Initializing the DMA Registers" on page 1843

Note

Follow this process only for registers which are programed in the "Initializing the DMA Registers" on page 1843 flow, such as DMA_SysBus_Mode, DMA_CH(#i)_TxDesc_List_LAddress, and DMA_CH(#i)_RxDesc_List_LAddress, or when "Fatal Bus Error" Handling.

DMA completes the current ongoing packet transmission or reception before entering "Process Stop". Software must make sure that no deadlock is caused by this process. One of the possible options are:

Before you run step 1, make sure that the last descriptor with LD is set for TX side. And RX descriptors enough for at least two maximum supported receive packet are allocated."

14.8 Switching to New Descriptor List in Receive DMA

Switching to a new descriptor list is different in RxDMA as compared to TxDMA. Switching to a new descriptor list is permitted when the RxDMA is in SUSPEND state. This is because:

- Generally, RxDMA prepares the descriptors in advance.
- If the RxDMA goes to SUSPEND state due to descriptors not being available, which is a major failure (software is not able to free the filled-up descriptors/buffers). If this is not rectified immediately, it leads to loss of frames due to RxFIFO Overflow. So, the software is allowed to create a new descriptor list and program the RxDMA to start using it immediately, without going into STOP state.

14.9 Handling Bus Errors and Recovery

When the DMA feature is selected, any DMA Channel receiving an AXI Bus Error goes into a fatal state and stops functioning. Optionally, the software can issue a soft reset and wait until the completion of the soft reset. The software can identify the root cause of the error, fix the issue, and re-initialize or restart the enabled DMA channels when required.

14.9.1 Transmit DMA Channel

On getting an AXI Error response, TxDMA channel sets its Fatal Bus Error Interrupt Status (FBE bits of `VDMA_CH(#i)_Status` register), go into ERROR state and gracefully terminate any ongoing packet transfer immediately, tag that packet to have CRC error. It also accepts and drops the rest of the packet or next packet scheduled for fetching from the system memory before the Bus Error event. It then goes into the STOP state after setting the DMA Stopped Interrupt status (TPS bits of `VDMA_CH(#i)_Status` register). The sbd_intr_o is generated if the corresponding interrupts are enabled in the `VDMA_CH(#i)_Interrupt_Enable` Register.

Driver Flow for TxDMA Channel Bus Error Recovery is as follows:

1. Wait for FBE Interrupt (FBE bit of the `VDMA_CH(#i)_Status` register)
2. Wait till the DMA is stopped (TPS bits of the `VDMA_CH(#i)_Status` register) Interrupt
3. Fix the root cause of the Bus Error
4. Service the FBE & TPS Interrupts & Clear those interrupts by writing the `VDMA_CH(#i)_Status` register
5. Issue a soft reset by setting the SWR field of the DMA_Mode register and wait until the SWR field is cleared.
6. You are allowed to re-configure the DMA new PBLs, New Ring Length
7. You must program the New List Address.
8. Start the DMA by setting ST of `VDMA_CH0_Tx_Control` register.

Note Step 6 initiates the Descriptor list. The descriptors that would have fetched before the Bus Error event is ignored by 25GMAC.

14.9.2 Receive DMA Channel

On getting an AXI Error response, RxDMA channel sets its Fatal Bus Error Interrupt Status (FBE bits of the `VDMA_CH(#i)_Status` register), go into ERROR state and gracefully terminate any ongoing packet transfer immediately. It also accepts and drops the rest of the packet from MTL RxQ. It goes into the STOP state after setting the DMA Stopped Interrupt status (RPS bits of the `VDMA_CH(#i)_Status` register). The sbd_intr_o is generated if the corresponding interrupts are enabled in the `VDMA_CH(#i)_Interrupt_Enable` register.

Driver Flow for RxDMA Channel Bus Error Recovery is as follows:

1. Wait for FBE Interrupt (FBE bit of `VDMA_CH(#i)_Status` register)
2. Wait till the DMA is stopped (RPS bits of `VDMA_CH(#i)_Status` register) Interrupt
3. You can program RPF (bit[31]) of the DMA_CH(#i)_Rx_Control register to prevent head-of-line blocking. That is instruct the 25GMAC to drop packets in any RxQ intended to this RxDMA Channel in STOP/ERROR state.

4. Fix the root cause of the Bus Error
5. Service the FBE & TPS/RPS Interrupts & Clear those interrupts by writing the `VDMA_CH(#i)_Status` register
6. Issue a soft reset by setting the SWR field of the DMA_Mode register and wait until the SWR field is cleared.
7. You can re-configure the DMA, new PBLs, and New Ring Length
8. You must program the New List Address
9. Start the DMA by setting SR (bit[0]) of `VDMA_CH(#i)_Rx_Control`

Note Step 6 initiates the Descriptor list. The descriptors that would have fetched before the Bus Error event are ignored by 25GMAC.

14.10 Setting Up Multi-Channel and Multi-Queue Operations

14.10.1 Programming Sequence for the Transmit Channel/Queue

1. Program the Transmit queue size in the TQS field, Transmit queue enable in the TXQEN field, Transmit queue to Traffic class mapping in Q2TCMAP field (when more than one traffic class is selected) of MTL_TxQ(#i)_Operation_Mode register. Based on the value programmed in TQS field, the size of the queue is determined. In the Transmit operation, the number of TxDMA channels is equal to the number of the queues. Due to this reason, the Channel to Queue mapping is fixed.
2. Program the TSA field in the MTL_TC(#i)_ETS_Control register when DCB feature is selected. When TSA field is set to ETS, the ETSALG of the MTL_Operation_Mode register is used for scheduling packets belonging to that traffic class.
3. The ETS Algorithm method needs to be programmed in ETSALG of MTL_Operation_Mode register.
4. If DCB is enabled program the MTL_TC(#i)_Quantum_Weight for DCB queue as per the selected algorithm.
5. If DCB is enabled and PFC function is required, program MTL_TC_Prty_Map(#i) Register to assign a priority to the queue. This priority assigned is used for determining if the corresponding queue must stop transmitting packet based on the received PFC packet.

14.10.2 Programming Sequence for the Receive Channel/Queue

1. Program the Receive queue size in the RQS field of MTL_RxQ(#i)_Operation_Mode register. Based on the value programmed in RQS field, the size of the queue is determined.
2. Enable the Receive Queues 0 to 11 in the fields RXQ0EN to RXQ11EN in MAC_RxQ_Ctrl0 register for DCB. In DMA configurations, SR bit of statically or dynamically mapped VDMA_CH(#i)_Rx_Control Register and corresponding RXQ(#i)_EN in MAC_RxQ_Ctrl0 Register needs to be enabled.
3. The MAC routes the Rx packets to the Rx Queues based on the following packet types:
   - Data Center Bridging (DCB) related Link Layer Discovery Protocol (LLDP) packets

     Program DCBCPQ in MAC_RxQ_Ctrl1 register to indicate to MAC which queue must get the DCB packets.
   - Multicast or Broadcast packets

     Program the MCBCQ field of the MAC_RxQ_Ctrl1 register to indicate which queue must receive Multicast and Broadcast packets.
   - VLAN Tag Priority field in VLAN Tagged packets

     Program PSRQ7-0 of the MAC_RxQ_Ctrl3 and MAC_RxQ_Ctrl2 registers for the routing of tagged packets based on the USP field (user Priority) of the received packets to the Rx Queues 0-7.
   - Untagged packets

     Program the UPQ field of the MAC_RxQ_Ctrl1 register to route untagged packets to the desired Rx Queue.

4. If multiple RXDMA channels are enabled, perform the following programming steps to ensure proper arbitration and mapping:
    a. Program the RAA field of MTL_Operation_Mode registers to select the arbitration algorithm to decide which RxQ is read out from the RxFIFO memory.
    b. Program the MTL_RxQ(#i)_Control to determine the weights and the packet arbitration for each RxQ.
    c. If static mapping is programmed in the MTL_RxQ_DMA_MAP(#i) register (Q(#i)DDMACH is reset to 0), program bits RXQx2DMA and others to select the channel for which each queue is mapped.
    d. Set Q(#i)DDMACH bit in the MTL_RxQ_DMA_Map0 register to select dynamic mapping of packets in each RxQueue.
        - If L3-L4 filtering feature is enabled, program the DMCHEN and DMCHN fields in the MAC_L3_L4_Control(#i) registers for each filter as required.
        - If RSS feature is enabled, program the DMCH field in the RSS_Lookup_Data register.
    e. In dynamic channel mapping, the routing of a packet to a specific RxDMA channel is decided by one of the following in the same priority order:
        - The DMCHN field in the MAC_L3_L4_Control(#i) register when L3-L4 filtering is selected and the packet passes one of the L3-L4 filters.
        - The DMCH field in the RSS_Lookup_Data
        - The value of DCS field in the lowest MAC Address register.

14.10.3 Programming Guidelines for Enabling Packet Duplication

1. Program the Multicast/Broadcast Destination Address in the respective `MAC_Address(#i)_Low` and `MAC_Address(#i)_High` registers for which the received Multicast/Broadcast packet with matched DA must be duplicated. The DCS field of register must be programmed to index of the Receive Channel List.
2. Program the DDS(#i) bit of the respective `MAC_DPCSel_IndReg(#i)` indirect register to 1 to enable Multicast/Broadcast packet duplication.

Note The received Multicast/Broadcast DA packet is not duplicated when it is queued to non-highest-numbered Receive Queue. These packets are routed to lowest bit set in the one-hot encoded value of Receive Channel List location pointed by DCS field, that is no duplication.

14.10.4 Programming Guidelines to Enable Broadcasting of Unicast Packet

Follows these steps to enable Tx timestamp status FIFO streaming interface.

1. Program the UDC field of the `MAC_RxQ_Ctrl4` register to 1.
2. Program Unicast Destination Address in the respective `MAC_Address(#i)_Low` and `MAC_Address(#i)_High` registers for which the received Unicast packet with matched DA must be duplicated. The DCS field of register must be programmed to index of the Receive Channel List.
3. Program the DDS(#i) bit of the respective `MAC_DPCSel_IndReg(#i)` indirect register to 1 to enable Unicast packet duplication.

4. The received Unicast DA packets is routed to lowest bit set in the one-hot encoded value of Receive Channel List location pointed by DCS field, that is no duplication when the:
   - DDS(#i) field of the `MAC_DPCSel_IndReg(#i)` indirect register = 1
   - UDC field of `MAC_RxQ_Ctrl4` = 0

Note	The received Unicast DA packet is not duplicated when it is queued to non-highest-numbered Receive Queue. These packets are routed to lowest bit set in the one-hot encoded value of Receive Channel List location pointed by DCS field, that is no duplication.

14.11 Programming Guidelines for XGMII Link State Transitions

Transmit and Receive Clocks Running when Link Down

Complete the following steps when the link is down but the Transmit and Receive clocks are running:

1. Disable the Transmit DMA by clearing Bit 0 (ST) of the VDMA_CH(#i)_Tx_Control register.
2. Disable the MAC receiver by clearing Bit 0 (RE) of the MAC_Rx_Configuration register.
3. Wait for any previous frame transmissions to complete.

   You can check this by reading the appropriate bits of the MTL_TxQ0_Debug register (TRCSTS is not 01).

   Alternatively, flush the Tx FIFO for faster empty operation.
4. Disable the MAC transmitter by clearing Bit 0(TE) of the MAC_Tx_Configuration register.
5. Make sure that both the Tx Queue and Rx Queue are empty (TXQSTS is 0 in the MTL_TxQ0_Debug register, and RXQSTS is 0 in the MTL_RxQ0_Debug register).
6. After the link is up, read the PHY registers to know the latest configuration and accordingly program the MAC registers.
7. Restart the operation by starting the TxDMA. You do not need to disable the RxDMA.
8. Enable the MAC Transmitter and Receiver. As the MAC Receiver is disabled, the FIFO does not get any data in the Rx FIFO.

Transmit and Receive Clocks Stopped when Link Down

Complete the following steps when the link is down and the Transmit and Receive clocks are stopped:

1. Disable the MAC Transmitter and Receiver by clearing the RE and TE bits of the MAC_Rx_Configuration and MAC_Tx_Configuration registers. This does not take effect immediately because the clocks are absent.
2. Wait until the link is up and the clocks are restored.
3. Wait for the completion of the transfer of any partial frame if any at time of stopping of Transmit/Receive clock. This can be checked by reading the MAC_Debug Register (must be all-zero). Some old packets may still remain in the TXFIFO as the MAC Transmitter is stopped.
4. Read the PHY registers to know the latest operating mode and accordingly program the MAC registers
5. Restart the MAC Transmitter and Receiver by setting RE and TE bits.

14.12 Enabling the Energy Efficient Ethernet

Entering and Exiting the Tx LPI Mode

You can configure the Energy Efficient Ethernet (EEE) feature in coreConsultant. Complete the following steps during 25GMAC controller initialization:

1. Read the PHY register through the MDIO interface, check if the remote end has the EEE capability, and then negotiate the timer values.
2. Program the PHY registers through the MDIO interface (including the RX_CLK_stoppable bit that indicates to the PHY whether to stop Rx clock in LPI mode.)
3. Program LST (bits[25:16]) and TWT (bits[15:0]) in MAC_LPI_Timers_Control Register
4. For GMII, read the link status of the PHY chip by using the MDIO interface and update Bit 17 of MAC_LPI_Control_Status register accordingly. This update must be done whenever the link status in the PHY chip changes. For XGMII, the update is automatic unless PLSDIS bit is set.
5. Program the MAC_1US_Tic_Counter as per the frequency of the clock used for accessing the CSR secondary port.
6. Program the MAC_LPI_Auto_Entry_Timer register (LPIET) with the IDLE time for which the MAC must wait before entering the LPI state on its own.
7. Set LPIATE and LPITXA (bit[20:19]) of MAC_LPI_Control_Status register to enable the auto-entry into LPI and auto-exit of MAC from LPI state.
8. Set LPITXEN (bit[16]) of MAC_LPI_Control_Status register to make the MAC Transmitter enter the LPI state. The MAC enters the LPI mode after completing all scheduled packets and remain IDLE for the time indicated by LPIET. It sets the TLPIEN (bit[0]) after entry to LPI state.
9. When a packet is scheduled for transmission (when the TxDMA comes out of IDLE state or when a packet is presented at ATI or MTI interface), the MAC Transmitter exits LPI state automatically. It waits for TWT time before setting the TLPIEX interrupt status bit and then resumes the packet transmission.
10. MAC Transmitter re-enters LPI state if it remains IDLE for LPIET time and set the TLPIEN bit and thus the entry-exit cycle continues.
11. Reset LPITXEN in case the software wants to override the auto-entry/exit modes and make the MAC Transmitter exit the LPI state directly.

Note

- To make the MAC enter the LPI state only after it completes the transmission of all queued frames in the Tx FIFO, you must set Bit 19 in MAC_LPI_Control_Status register.
- To switch off the GMII Transmit Clock during the LPI state, use the sbd_tx_clk_gating_ctrl_o signal for gating the clock input.
- To switch off the CSR clock or power to the rest of the system during the LPI state, you must wait for the TLPIEN interrupt of MAC_LPI_Control_Status register to be generated. Restore the clocks before performing step 6 when you want to come out of the LPI state.

Gating Off the CSR Clock in the LPI Mode

You can gate off the CSR clock to save the power when the MAC is in the Low-Power Idle (LPI) mode.

Gating Off the CSR Clock in the Rx LPI Mode

The following operations are performed when the MAC receives the LPI pattern from the PHY.

1. The MAC RX enters the LPI mode and the Rx LPI entry interrupt status [RLPIEN interrupt of MAC_LPI_Control_Status Register] is set.
2. The interrupt pin (sbd_intr_o) is asserted. The sbd_intr_o interrupt is cleared when the host reads the MAC_LPI_Control_Status register.

   After the sbd_intr_o interrupt is asserted and the MAC Tx is also in the LPI mode, you can gate-off the CSR clock. If the MAC TX is not in the LPI mode when you gate off the CSR clock, the events on the MAC transmitter do not get reported or updated in the CSR.

   For restoring the CSR clock, wait for the LPI exit indication from the PHY after which the MAC asserts the LPI exit interrupt on lpi_intr_o (synchronous to clk_rx_312pt5_i). The lpi_intr_o interrupt is cleared when MAC_LPI_Control_Status Register is read.

Gating Off the CSR Clock in the Transmit LPI Mode

The following operations are performed when Bit 16 (LPIEN) of MAC_LPI_Control_Status register is set:

1. The Transmit LPI Entry interrupt (TLPIEN bit of MAC_LPI_Control_Status register) is set.
2. The interrupt pin (sbd_intr_o) is asserted. The sbd_intr_o interrupt is cleared when the host reads the MAC_LPI_Control_Status Register.

   After the sbd_intr_o interrupt is asserted and the MAC RX is also in the LPI mode, you can gate off the CSR clock. If the MAC RX is not in the LPI mode when you gate off the CSR clock, the events on the MAC receiver do not get reported or updated in the CSR.

   For restoring the CSR clock, switch on the CSR clock when the MAC has to come out of the TX LPI mode. After the CSR clock is resumed, reset Bit 16 (LPIEN) of MAC_LPI_Control_Status register to bring the MAC out of the LPI mode.

14.13 Setting up TCP Segmentation Offload (TSO)

1. Program TSE bit of corresponding VDMA_CH(#i)_Tx_Control register to enable TCP packet segmentation in that DMA.
2. In addition to the normal transfer descriptor setting, the following descriptor fields must be programmed to enable TSO for the current packet:
   a. Enable TSE in Bit 18 of TDES3.
   b. Program the length of the un-segmented TCP/IP packet payload in bits [17:0] of TDES3 and the TCP header in bits [22:19] of TDES3.
   c. Program the maximum size of the segment in:
      - MSS of VDMA_CH(#i)_Control register (OR)
      - MSS in the context descriptor

      If MSS field is programmed in both VDMA_CH(#i)_Control register and in the context descriptor, the latest software programmed sequence is considered.
   d. TDES2 field
      - When TSE is set, program HL[9:0]
      - When TSE = 1 in TDES2, Program the length of the TCP/UDP Header field in [22:19]
      - Program External TSO Memory Write Enable
   e. Program TCMSSV in context descriptor CTDES3
3. The header of the unsegmented TCP/IP packet must be in Buffer 1 of the first descriptor and this buffer must not hold any payload bytes. The payload can be allocated starting from Buffer 2 of the first descriptor or any buffer of the subsequent descriptors.

14.14 Enabling the Header-Payload Splitting Offload

When split header support is enabled, the buffer 1 (RDES0) of the descriptors is used only for the header. Therefore, the DMA writes the header of the received packet in buffer 1 when FD bit of RDES3 is set. The payload of the received packet is directed to the buffer2 (RDES2).

For subsequent descriptors (FD is set to 0), buffer1 is not used and the payload is written only to buffer 2. When the host enables split header structure by setting the SPH Bit[24] of the VDMA_CH(#i)_Control register, the host must ensure that Buffer1 is always valid in all descriptors by creating Buffer1 and indicating this by providing a non-zero buffer1 address.

To configure Split mode control and offset for Header-Payload Split feature, you must program the SPLOFST, SAVE and SAVO fields of the MAC_Ext_Cfg1 register. For more information on how to program these fields, see “Header Payload Split Support” on page 1834.

14.15 Enabling the VxLAN/NVGRE Tunneled Packet Processing

1. Set VNE (bit[24]) of MAC_Tx_Configuration register to enable 25GMAC to recognize and process the tunneled packets.
2. Select tunneling mode (VxLAN or NVGRE) by programming VNM (bit[25]) of MAC_Tx_Configuration register.

14.15.1 Tx

To indicate a tunneled packet to TxDMA, set VNP field in corresponding TxDMA Descriptor (TDES). This enables TSO, TxCOE operations to be performed on tunneled packets during transmission.

14.15.2 Rx

1. As per the selected tunneling mode, program the MAC_Tunnel_Identifier register to enable 25GMAC to recognize the VxLAN Tunneled Packets based on Outer Packet Header UDP destination Port ID or NVGRE Tunneled Packets based on Protocol Type field. By default/reset, this register contains the UDP Destination Port ID value of 0x12B5 (4789) as defined by IANA for VxLAN.
2. Set VUCC and DNTU bits in the MAC_Packet_Filter register as per requirement for tunneled packets.
3. By default all received Tunneled packets are directed to RxDMA Channel 0. To classify the received packets and direct them to different RxDMA channels (in a multi RxDMA channel configuration), there are two options:
    a. External Lookup: Set ELEN (bit[30]) of MAC_Rx_Configuration register. The external lookup engine provides the RxDMA channel based on the information (header fields) provided by the MAC receiver signals outputs.
    b. Internal Lookup: Do not set ELEN but enable RSS hash Lookup. Program the RSS Lookup Table as required.
4. Set SPH bit in VDMA_CH(#i)_Control register to enable the 25GMAC to transfer the outer+inner header and inner payload of received tunneled packets into separate buffers.
5. The status/identification of tunneled packets are given in RxDMA Descriptor Status (RDES).

14.16 Programming Sequence for Queue/Channel Based VLAN Inclusion Register

Writing to the Per Queue/Channel VLAN Inclusion Indirect Registers

To write the queue/channel-based VLAN inclusion register, complete the following steps:

1. Program the CBTI bit of MAC_VLAN_Incl register to 1, to enable queue/channel based VLAN Tag insertion on all transmitted packets. This bit must be programmed before any indirect access to the queue/channel specific MAC_VLAN_Incl(#i) register.
2. Program the RDWR field of MAC_VLAN_Incl register to 1, to indicate write access.
3. Program the VLAN Tag and VLAN Type to be inserted in packets from a particular queue/channel in VLT and CSVL fields of MAC_VLAN_Incl register; corresponding offset address in ADDR field (0 for queue/channel 0, 1 for queue/channel 1, and so on) must be programmed. Program the RDWR bit to 1 to indicate write access.
4. The BUSY bit of MAC_VLAN_Incl register is programmed by DWC_ether_qos to indicate the progress of access to indirect access MAC_VLAN_Incl(#i) register. On completion of the access, the BUSY bit is cleared. The application must not attempt subsequent access to MAC_VLAN_Incl(#i) register when the BUSY bit is 1.
5. Repeat step 2 and step 3 to program VLAN Tag and VLAN Type to be inserted in packets from the remaining queues/channels. The application must ensure that the required VLAN Tag and VLAN Type for all the queues/channels are programmed; otherwise unintended VLAN Tag and VLAN Type might be inserted.

Read the Per Queue/Channel VLAN Inclusion Indirect Registers

To read the queue/channel-based VLAN inclusion register, complete the following steps:

1. Program the CBTI field of MAC_VLAN_Incl register to 1, to enable queue/channel based VLAN Tag insertion on all the transmitted packets. This bit must be programmed before indirect access to the queue/channel specific MAC_VLAN_Incl(#i) register.
2. Program the read offset address in the ADDR field (0 for queue/channel 0, 1 for queue/channel 1, and so on).
3. Program the RDWR bit to 0, to indicate read access.
4. The BUSY field of MAC_VLAN_Incl register is programmed by DWC_ether_qos to indicate the progress of access to the indirect access MAC_VLAN_Incl(#i) register. On completion of the access, the BUSY field is cleared. The bits [15:0] and bit [19] of MAC_VLAN_Incl register contains the VLAN Tag and VLAN Type respectively, from the corresponding queue/channel/offset address.
5. Repeat step 2 and step 3 to read VLAN Tag and VLAN Type from the remaining queues/channels.

14.17 Programming Guidelines for External DA Hash Table-Based Filtering and Routing

Follow these additional steps to configure the external DA hash table.

1. Configure the DA hash table size using the MAC_DAH_TBL_SIZE register.

   25GMAC supports dynamic selection of DA hash table size. You can select specific DA hash CRC bits to be used for has table size indexing. 25GMAC concatenates the selected CRC bits up to maximum count, based on the supported hash table size.

   For example, if your external DA hash table size is 4096, 25GMAC concatenates a total of 12 selected bits from LSB to MSB and uses them for indexing; remaining selected bits gets ignored.

2. Program the external DA has table using the MAC_Indir_Access_Ctrl and MAC_Indir_Access_Data indirect access registers.
3. Program the DHLFRS field along with other enable fields of the MAC_Packet_Filter register to get a good DA hash in the receive status.

14.18 Setting up VLAN Filtering on Receive

1. Program the MAC_VLAN_Tag register for the following bit to select the filtering method:
    a. ETV: Enable 12-Bit VLAN Tag Comparison or 16-bit VLAN Tag comparison.
    b. VTHM: VLAN Tag Hash Table Match Enable.
    c. ERIVLT: Enable inner VLAN Tag or outer VLAN Tag (to enable the inner or outer VLAN Tag filtering, Double VLAN Processing must be enabled by setting EDVLP)
    d. ERSVLM: Enable Receive S-VLAN Match or C-VLAN match (for S-VLAN processing to be enabled, set ESVL)
    e. DOVLTC: Ignores VLAN Type for Tag Match
    f. VTIM: to enable VLAN Tag Inverse Match instead of the normal VLAN Tag matching
2. Program VID of MAC_VLAN_Tag register for the 12 bit or 16bit VLAN tag.
3. If Hash filtering of VLAN tag is enabled, program MAC_VLAN_Hash_Table register. The upper four bits of the calculated CRC are used to index the contents of the VLAN Hash table. For example, a hash value of 4'b1000 selects Bit 8 of the VLAN Hash table.

14.19 Setting up Extended VLAN Filtering

For the indirect access of the per VLAN Tag registers, follow these steps:

- Write:
  - Write the required data into the MAC VLAN Tag Data register.
  - Program the VLAN Tag Control Register's OFS field with the required Filter Register's offset and command type to the CT field. For a write command, set this bit to 0.
  - Write 1 to the OB field and wait till the OB bit is reset to do the next write. This guarantees that the appropriate VLAN Tag Filter Register has been programmed.
- Read:
  - Program the VLAN Tag Control Register's OFS Field with the required register's offset and command type to the CT field. For a read command, set this bit to 1.
  - Write 1 to the OB field and wait till the OB bit is reset. The appropriate VLAN Tag Filter register's value is available in the MAC VLAN Tag Data Register.

14.20 Setting up External Rx VLAN Tag Lookup-Based Filtering and Routing

Follow these steps for external lookup-based filtering and routing

1. To configure the external receive outer/inner VLAN tag lookup size, use the `MAC_RVLAN_LKP_SIZE` register.

   25GMAC supports dynamic selection of Rx VLAN lookup size using the MAC_RVLAN_LKP_SIZE register. You can select the specific VLAN tag bits to be used for lookup indexing. 25GMAC concatenates the selected bits up to a maximum count based on the supported lookup size.

   For example, if your lookup size is 4096, 25GMAC concatenate a total of 12 selected bits from LSB to MSB; the remaining selected bits gets ignored. This is useful for logical masking of respective receive VLAN tag bits and programmed reduced lookup size.

2. Program the external receive outer/inner VLAN tag lookup using the `MAC_Indir_Access_Ctrl` and `MAC_Indir_Access_Data` indirect access registers.

3. Program the ERIVTL and EROVTL fields along with other enable fields of the MAC_VLAN_Tag_Ctrl register to setup and use as the external receive outer/inner VLAN tag lookup.

For more details, see the *Register Descriptions* chapter.

14.21 Setting up the L3-L4 Filtering

In 25GMAC, L3-L4 filter registers are implemented as indirect addressed registers. Follow these steps to access these registers.

14.21.1 Writing to the Indirect Addressed Registers

1. Write the data into the MAC_L3_L4_Data register
2. Program the following fields of the MAC_L3_L4_Address_Control register
   a. Set the IDDR field to the required register offset address of the L3-L4 Filter register
   b. Set the TT field to 0, to indicate write operation

Note

- IDDR[8:4] specifies the filter number;0 for L3-L4 filter 0, 1 for L3-L4 filter 1, and so on.
- IDDR[3:0] specifies the filter register of a specific L3-L4 register, as follows:
  - 0: MAC_L3_L4_Control#(i)
  - 1: MAC_Layer4_Address#(i)
  - 2-3: Reserved
  - 4: MAC_Layer3_Addr0_Reg#(i)
  - 5: MAC_Layer3_Addr1_Reg#(i)
  - 6: MAC_Layer3_Addr2_Reg#(i)
  - 7: MAC_Layer3_Addr3_Reg#(i)
  - 8-15: Reserved

3. Write 1 to the XB field of MAC_L3_L4_Address_Control register. Wait until the XB bit is reset, before initiating the next write operation. This ensures that appropriate L3-L4 filter registers is programmed correctly.

14.21.2 Reading the Indirect Addressed Registers

1. Set the TT field to 1, to indicate read operation.
2. Program the IDDR field of MAC_L3_L4_Address_Control register with the required L3-L4 Filter register offset address.
3. Write 1 to the XB bit of MAC_L3_L4_Address_Control register and wait until the XB bit is reset.
4. After the XB bit is reset, read the MAC_L3_L4_Data register; it must have the content of the requested L3-L4 filter register.

Note Program the DHLFRS field of MAC_Packet_Filter to get proper L3L4 filter number (if number of L3L4 are more than 8) in receive status due to space crunch.

14.22 Setting Up the Flexible Receive Parser

Following are the programing guidelines for Flexible Receive Parser.

1. Program NVE, NPE, and the required interrupts such as:
   - ❑ PDRFIE: Packet Drop due to RF Interrupt Enable
   - ❑ FOOVIE - Frame Offset Overflow Interrupt Enable
   - ❑ NPEOVIE - Number of Parsable Instructions Overflow Interrupt Enable
   - ❑ NVEOVIE - Number of valid Instructions Overflow Interrupt Enable
2. Program or build the Flexible Receive Parser Instruction Table with the format described in Table 4-15 on page 191, using the MTL indirect register accessing.
3. If more than 16 receive channels are supported, configure the accept counters by using the `MTL_FRP_ACNTR_CNFG(#i)` indirect registers. Application must enable the corresponding accept counter for valid application-specific Rx DMA channel number. Otherwise the corresponding accept counter is not updated. These indirect registers are accessible through the `MTL_Indirect_Acc_Control_Status` and `MTL_Indirect_Acc_Data` registers.

Note Application can configure the Error, Bypass, and Drop Count registers as well through the `MTL_Indirect_Acc_Control_Status` and `MTL_Indirect_Acc_Data` registers.

4. Enable Flexible Receive Parser

   For details, see "Enabling the Flexible Receive Parser" on page 202.
5. Optionally, check the status of the Flexible Receive Parser for each frame in the "Receive Normal Descriptor (Write-Back Format)" on page 1813.

14.22.1 Programming the Stream-Gate Filtering Registers

After completing the steps mentioned in "Programming the GCL and GCL Linked Registers" on page 1882, program the MTL_SGF_Control register.

1. Set the Current Time Offset Value and Time Internal Left Shift fields of the MTL_SGF_Control register appropriately. Also, set the Enable SGF (ESGF) and Switch to SWOL (SWOL) fields.

Note
- ❑ Program the STOV field of the MTL_SGF_Control register to 0.
- ❑ Program the CTOV field of the MTL_SGF_Control register as follows:
- ❑ ● 2 * Receive clock period + 2 * PTP clock period

2. This enables 25GMAC to own and process the new GCL and switch to the new GCL at the BTR value. If interrupt is enabled, 25GMAC generates an interrupt when it switches to the new GCL.
3. Software must address any other interrupts received from 25GMAC during the execution of the GCL.

When the software sets the SSWL (Switch to Software Owned List) field in the MTL_SGF_Control register, the GCL list is handed-off to 25GMAC. After successfully switching to the new GCL list, 25GMAC resets/clears the SSWL field. 25GMAC also flips the SWOL field to indicate the new GCL that the software now owns.

Software is not allowed to write to the GCL and GCL linked registers when the SSWL field is set, because the new GCL might be in use by 25GMAC.

To install a new GCL, software must program the GCL it owns (indicated by the SWOL field) as described in "Programming the GCL and GCL Linked Registers" on page 1882. Then program the MTL_SGF_Control register. Ensure that the new BTR is set to an appropriate value to avoid BTR Error that might require software intervention.

To avoid transmission overruns, the packet length (frame size) information must be available at all times. Therefore, in the DMA configurations, program the packet length in the first descriptor of every Tx frame. Similarly, in the MTL configuration, provide the packet length in the control word.

Note To avoid transmission overruns, the Packet Length provided in the Transmit Descriptor must account for the SA and VLAN insertion, if applicable, that is, Packet Length must represent the actual packet length on the Ethernet line.

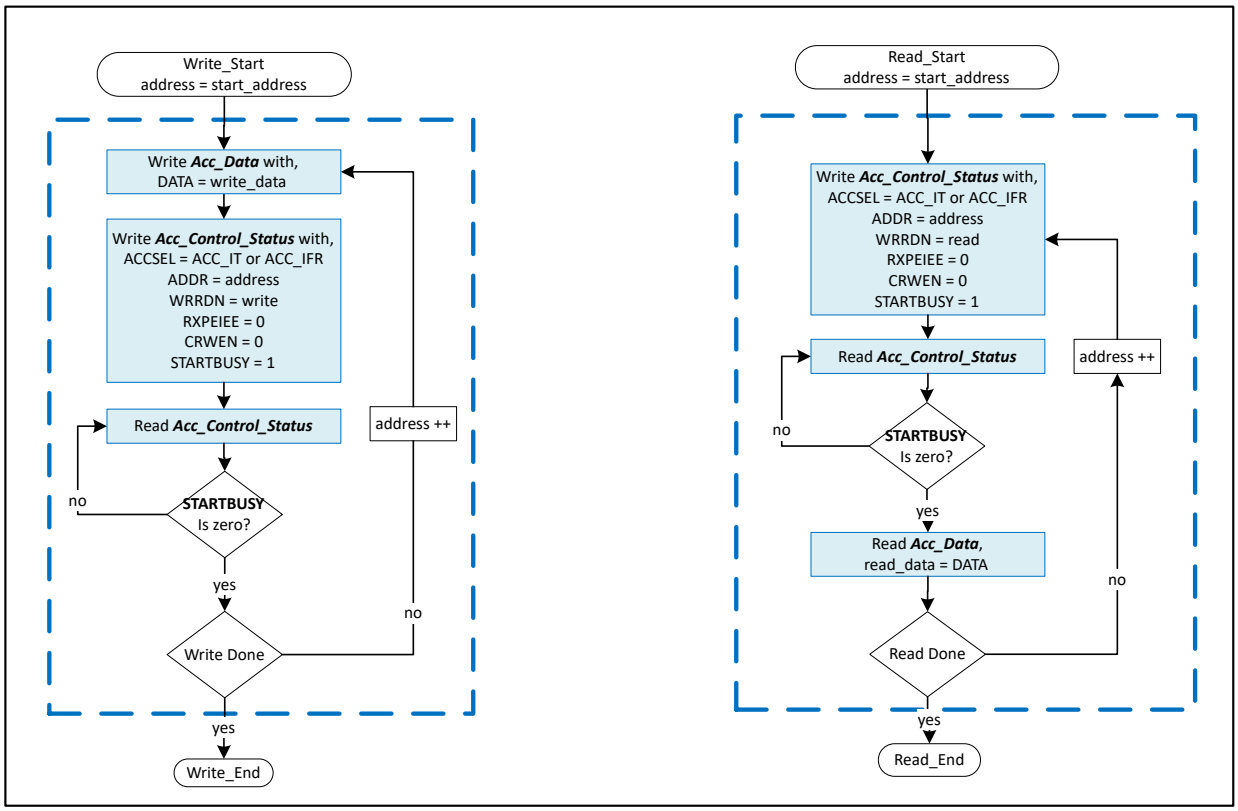
14.23 MTL Indirect Read/Write Access

This section describes the MTL indirect read/write access.

14.23.1 MTL Single Indirect Read/Write Access

Figure 14-1 shows the flow chart for single indirect read/write access.

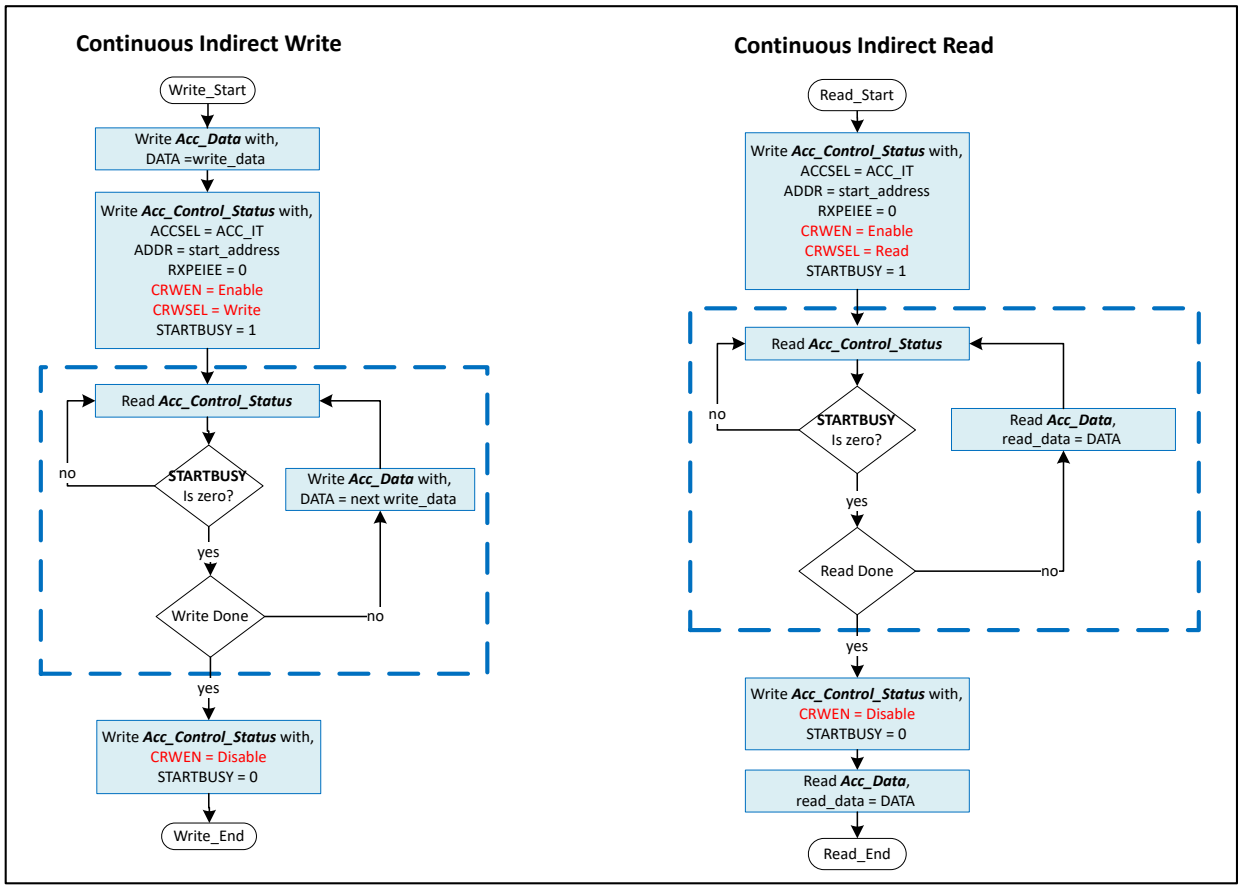
Figure 14-1 Flow Chart for MTL Single Indirect Read/Write Access



14.23.2 MTL Continuous Indirect Read/Write Access

Figure 14-4 shows the flow chart for continuous indirect read/write access

Figure 14-2 Flow Chart for MTL Continuous Indirect Read/Write Access



Note

- Byte wise Write/Read is not supported when Continuous Read/Write Mode is enable.
- Indirect Flexible Receive Parser register does not support continuous indirect read/write accessing.

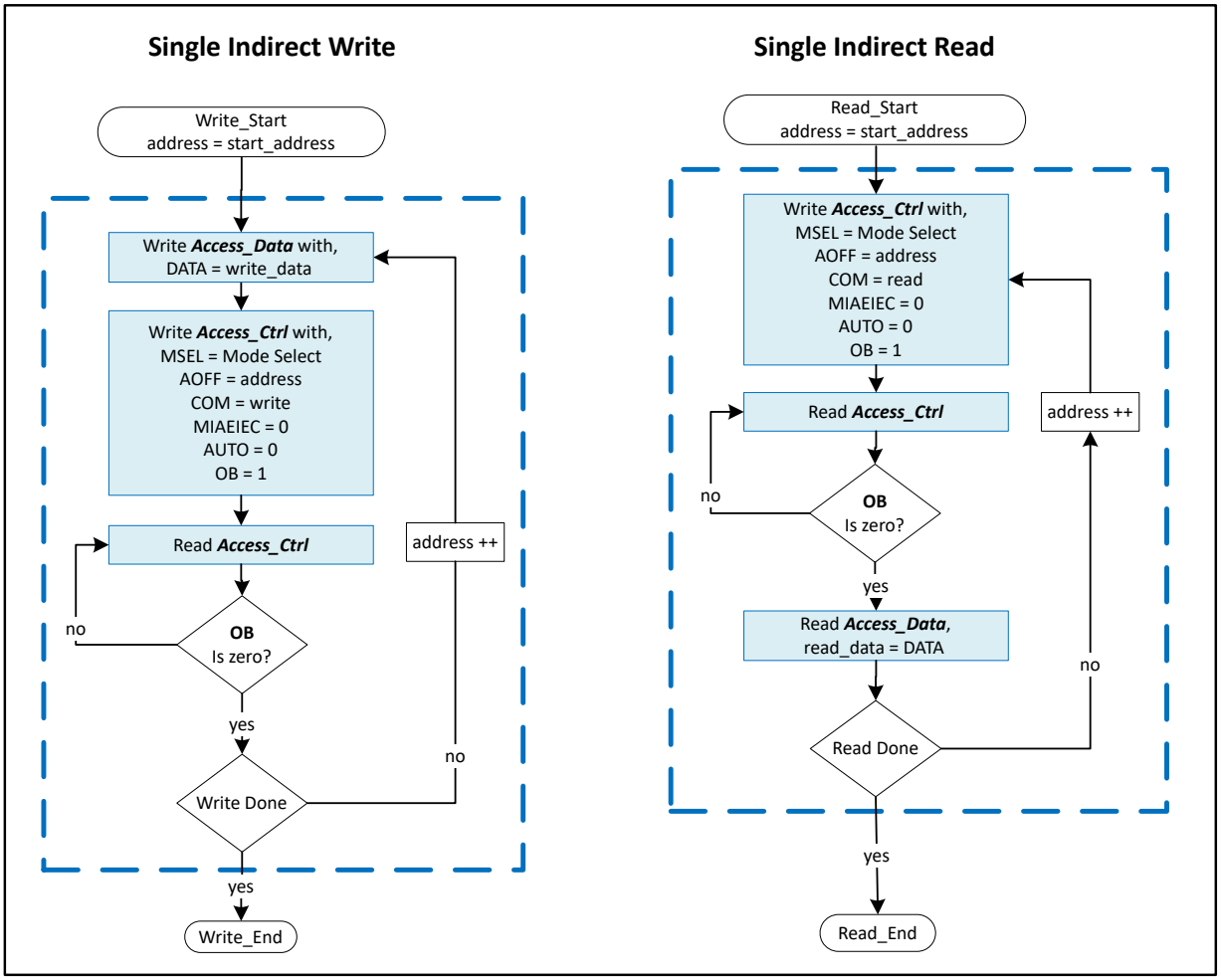
14.24 MAC Indirect Read/Write Access

The following sections discuss the MAC single indirect read and write and MAC continuous indirect read and write accesses.

14.24.1 MAC Single Indirect Read/Write Access

Figure 14-3 shows the flow chart for MAC single indirect read/write access.

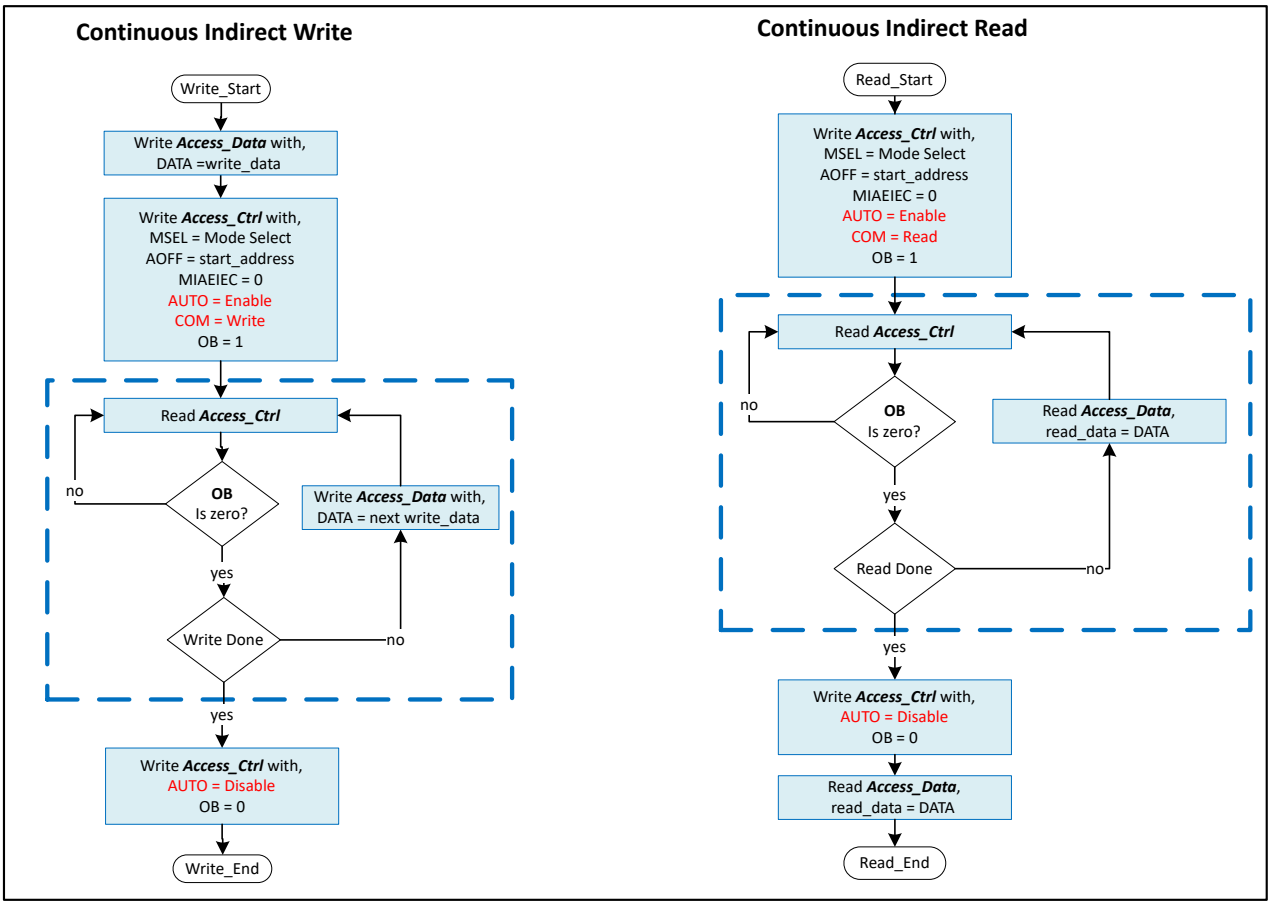
Figure 14-3 Flow Chart for MAC Single Indirect Read/Write Access



14.24.2 MAC Continuous Indirect Read/Write Access

Figure 14-4 shows the flow chart for MAC continuous indirect read/write access.

Figure 14-4 Flow Chart for MAC Continuous Indirect Read/Write Access



Note

- Byte wise Write/Read is not supported when Continuous Read/Write Mode is enable.
- Indirect Flexible Receive Parser register does not support continuous indirect read/write accessing.

14.25 Programming Guidelines for the Policing Counters

Follow these steps to program the policing counter registers:

1. Programming the policing counter control and status is through the indirect address registers, MAC_Indir_Access_Ctrl and MAC_Indir_Access_Data registers.
2. Enable the policing counter by setting the PCEN field of the MAC_*PCSel_IndReg register. Program the PCNUM field of the MAC_*PCSel_IndReg to select one of the DWCXG_NUM_PC counter selection register.

   The policing counter number and policing counter validity is programmed using the following fields of the Flexible Receive Parser instruction table.

   These fields overlap with the OKI field of the Flexible Receive Parser instruction table. So, these fields can be used only when all of the following conditions are met:

   - The NIC field of the Flexible Receive Parser instruction table is 0
   - (AF == 1 && RF == 0) || (AF == 1 && RF == 1)
   - Software has set the OKI_DME field of the MTL_RXP_Control_Status register

Table 14-2 Policing Counter Fields in the Flexible Receive Parser Instruction Table

Bit Position	Field Name	Description
80	PCV	Policing Counter Valid
((log2 DWCXG_NUM_PC) - 1):81	PCN	Policing Counter Number

3. Program the TWSEL, FRZ, and TRSLD fields of the MAC_PCCtrl_IndReg register to select the time window, enable freeze, and set threshold respectively.
4. The policing counter status registers have a status field corresponding to the each filter event to track the filter event. The status is asserted when at the least one filter passes.
5. To enable threshold interrupt, program the MAC_PCTH_Intr_Enable register. When a threshold interrupt is asserted in the MAC_PCTH_Intr_Status register, writing 1 to the corresponding field clears the interrupt.
6. To enable time-window based interrupts, program the MAC_PCTW_Intr_Enable register. The status of the interrupt is provided in MAC_PCTW_Intr_Status register.
7. Filter fail and count value are tracked in the MAC_PCntr_IndReg register. The status for the filter fail is indicated through the FFAIL field of the MAC_PCStatus_Filter_Event register.

For more details, see the *"Register Description"* chapter.

14.26 Setting up the IEEE 1588 PTP Timestamping

14.26.1 Initialization Guideline for System Time Generation

You can enable the timestamp feature by setting Bit 0 of the MAC_Timestamp_Control register. However, it is essential that the timestamp counter must be initialized after this bit is set. Complete the following steps during 25GMAC initialization:

1. Mask the Timestamp Trigger interrupt by clearing the bit 12 of MAC_Interrupt_Enable register.
2. Set Bit 0 of MAC_Timestamp_Control register to enable timestamping.
3. Program MAC_Sub_Second_Increment register based on the PTP clock frequency.
4. You can use either the Coarse Correction Method or the Fine-Correction Method to update the time.
5. Program MAC_System_Time_Seconds_Update register and MAC_System_Time_Nanoseconds_Update register with the appropriate time value.
6. Set Bit 2 in MAC_Timestamp_Control register. The timestamp counter starts operation as soon as it is initialized with the value written in the Timestamp Update registers. If one-step timestamping is enabled
    a. To enable one-step timestamping, program Bit 27 of the TDES3 Context Descriptor.
    b. Program registers MAC_Timestamp_Ingress_Asym_Corr and MAC_Timestamp_Egress_Asym_Corr to update the correction field in PDelay_Req PTP messages.
7. Enable the MAC receiver and transmitter for proper timestamping.

Note	If timestamp operation is disabled by clearing Bit 0 of MAC_Timestamp_Control register, you need to repeat all these steps to restart the timestamp operation.

Coarse Correction Method

To synchronize or update the system time in one process (coarse correction method), complete the following steps:

1. Set the offset (positive or negative) in the Timestamp Update registers (MAC_System_Time_Seconds_Update and MAC_System_Time_Nanoseconds_Update).
2. Set Bit 3 (TSUPDT) of the MAC_Timestamp_Control register. The value in the Timestamp Update registers is added to or subtracted from the system time when the TSUPDT bit is cleared. To synchronize or update the system time to reduce system-time jitter (fine correction method), complete the following steps:
    a. With the help of the algorithm explained in “System Time Register Module”, calculate the rate by which you want to make the system time increments slower or faster.
    b. Update the MAC_Timestamp_Addend register with the new value and set Bit 5 of the MAC_Timestamp_Control register.
    c. Wait for the time for which you want the new value of the Addend register to be active. You can do this by enabling the Timestamp Trigger interrupt after the system time reaches the target value.
    d. Program the required target time in MAC_PPS(#i)_Target_Time_Seconds Register and MAC_PPS(#i)_Target_Time_Nanoseconds Register.

e. Enable the Timestamp interrupt in bit 12 of MAC_Interrupt_Enable register.

f. Set bit 4 in MAC_Timestamp_Control register.

g. When this trigger causes an interrupt, read MAC_Interrupt_Status register.

h. Reprogram MAC_Timestamp_Addend register with the old value and set bit 5 again.

Fine-Correction Method

To synchronize or update the system time over a period of time (fine-correction method), complete the following steps:

1. Program Bit 1 of MAC_Timestamp_Control register to select the Fine Update method.
2. Program MAC_Timestamp_Addend register and set Bit 5 of MAC_Timestamp_Control register.
3. Poll the MAC_Timestamp_Control register until Bit 5 is cleared.

14.27 Setting up the Flexible Pulse-Per-Second (PPS) Output

Generating Single Pulse on PPS

To generate single Pulse on PPS:

1. Program 11 or 10 (for interrupt) in Bits [6:5], TRGTMODSEL, of MAC_PPS_Control register. This instructs the MAC to use the Target Time registers for start time of PPS signal output.
2. Program the start time value in the Target Time registers.
3. Program the width of the PPS signal output in MAC_PPS0_Width register.
4. Program Bits [3:0], PPSCMD, of MAC_PPS_Control register to 0001. This instructs the MAC to generate single pulse on the PPS signal output at the time programmed in the Target Time registers.

When the PPSCMD is executed (PPSCMD bits = 0), you can terminate the pulse generation by giving the Terminate Start Command (PPSCMD=0011) before the programmed start time elapses. You can also program the behavior of the next pulse in advance. To program the next pulse:

1. Program the start time for the next pulse in the Target Time registers. This time must be more than the time at which the falling edge occurs for the previous pulse.
2. Program the width of the next PPS signal output in MAC_PPS0_Width register.
3. Program Bits [3:0], PPSCMD, of MAC_PPS_Control to generate a single pulse after the time at which the previous pulse is de-asserted. This instructs the MAC to generate single pulse on the PPS signal output, at the time programmed in Target Time registers.

   If you give this command before the previous pulse becomes low, then the new command overwrites the previous command and the 25GMAC may generate only 1 extended pulse.

Generating a Pulse Train on PPS

To generate a pulse train on PPS:

1. Program 11 or 10 (for interrupt) in Bits [6:5], TRGTMODSEL, of MAC_PPS_Control register. This instructs the MAC to use the Target Time registers for start time of the PPS signal output.
2. Program the start time value in the Target Time registers.
3. Program the interval value between the train of pulses on the PPS signal output in MAC_PPS0_Interval register.
4. Program the width of the PPS signal output in MAC_PPS0_Width register.
5. Program Bits[3:0], PPSCMD, of MAC_PPS_Control register to 0010. This instructs the MAC to generate train of pulses on the PPS signal output with start time programmed in Target Time registers. By default, the PPS pulse train is free-running unless stopped by 'STOP Pulse train at time' or 'STOP Pulse Train immediately' commands.
6. Program the stop value in the Target Time registers. Ensure that Bit 31 (TRGTBUSY0) of MAC_PPS0_Target_Time_Nanoseconds Register is reset before programming the Target Time registers again.

7. Program the PPSCMD field (bit 3:0) of MAC_PPS_Control register to 0100. This stops the train of pulses on PPS signal output after the programmed stop time specified in Step 6 elapses.

   You can stop the pulse train at any time by programming 0101 in the PPSCMD field. Similarly, you can terminate the Stop Pulse train command (given in Step 7) by programming 0110 in the PPSCMD field before the time (programmed in Step 6) elapses. You can terminate the pulse train generation by programming 0011 in the PPSCMD field before the programmed start time (in Step 2) elapses.

Generating an Interrupt without Affecting the PPS

The Bits [6:5], TRGTMODSEL, of the MAC_PPS_Control register enable you to program the Target Time registers (register 736 and 737) to do any one of the following:

- Generate only interrupts.
- Generate interrupts and the PPS start and stop time.
- Generate only PPS start and stop time.

To program the Target Time registers to generate only interrupt event:

1. Program 00 (for interrupt) in Bits [6:5], TRGTMODSEL, of MAC_PPS_Control register. This instructs the MAC to use the Target Time registers for target time interrupt.
2. Program a target time value in the Target Time registers. This instructs the MAC to generate an interrupt when the target time elapses.

   If Bits [6:5], TRGTMODSEL, are changed (for example, to control the PPS), then the interrupt generation is over-written with the new mode and new programmed Target Time register value.

14.28 Enabling the Audio Video Traffic Shaper

14.28.1 Initializing the DMA

This initialization sequence is used only for XGMAC-AXI configuration with AV feature. Complete the following steps to initialize the DMA:

1. Provide a software reset to reset all internal registers and logic (bit 0 in `DMA_Mode` register).
2. Wait for the completion of the reset process. Poll bit 0 of the `DMA_Mode` register), which is cleared only after the reset operation is completed.
3. Program the fields to initialize the DMA register by setting the values in `DMA_Mode` register.
4. Create a proper descriptor list for transmit and receive. In addition, ensure that the DMA owns the Transmit and Receive descriptors. When OSF mode is used, at least two TX descriptors are required.

   For more information about descriptors, see "Descriptors" on page 1795.
5. Make sure that your software creates three or more different transmit or receive descriptors in the list before reusing any of the descriptors.
6. Program the Transmit and Receive Ring length registers `VDMA_CH(#i)_Tx_Control2` register. The ring length programmed must be at least 4.
7. Initialize receive and transmit descriptor list address with the base address of the transmit and receive descriptor`VDMA_CH(#i)_TxDesc_List_HAddress` and `VDMA_CH(#i)_TxDesc_List_LAddress` register. In addition, you must program the Transmit and Receive tail pointer registers indicating to the DMA about the available descriptors (`VDMA_CH(#i)_Tx_Desc_Tail_Pointer` and `VDMA_CH(#i)_Rx_Desc_Tail_Pointer`) register.
8. Program the following fields to initialize the mode of operation in the `MTL_TxQ0_Operation_Mode` register:
   a. Transmit Store And Forward (TSF)
   b. Transmit Threshold Control (TTC)
   c. Transmit Queue Enable (TXQEN) to value 2'b10 to enable Transmit Queue0
   d. Transmit Queue Size (TQS)
9. Enable the interrupts by programming the `VDMA_CH(#i)_Interrupt_Enable` register.
10. Repeat steps4 through 9 for all additional channels of AV feature.
11. Program the CBS control register, idleSlope, sendSlope, hiCredit, and loCredit registers of the AV Queues.

    Program the TSA field in `MTL_tc#I_ETS_Control` to 2'b01 to select CBS algorithm.
12. Start the Receive and Transmit DMA by setting bit 0 of the `VDMA_CH(#i)_TX_Control` register and bit 0 of the `VDMA_CH(#i)_RX_Control` register.

14.28.2 Enabling Slot Number Checking

You can use the slot number check feature to specify the intervals at which the DMA Channels mapped to AV Queues fetches the frames from the AXI system bus. This feature is useful for a uniform and periodic transfer of the AV traffic from the host memory. The feature is available only when you enable timestamping and program the `MAC_Sub_Second_Increment` register. Complete the following steps to enable the slot number checking:

Note These steps must be completed after step 11 and before step 12 of "Initializing the DMA" on page 1879.

1. Enable timestamping by following the steps described in "Initialization Guideline for System Time Generation" on page 1875.
2. Make sure that the `SLOTNUM` field (bits 22:19) of TDES3 Normal Descriptor (Read Format) contains a valid slot number. You can read the current reference slot number from the `VDMA_CH(#i)_Slot_Function_Control_Status` Registers. For more details, see Table 13-4 on page 1801.
3. Set Bit 0 (`ESC`) of the Slot Function Control and Status register of a channel to enable the slot number checking.

14.28.3 Enabling Average Bits Per Slot Reporting

The CBS Status register of the additional AV channels provides information about the average bits that are transmitted in a slot. The software can asynchronously read this register to retrieve information about the average bits transmitted per slot. Complete the following steps to enable average bits per slot reporting:

1. Enable timestamping by following the steps described in "Initialization Guideline for System Time Generation" on page 1875.
2. Program Bits [6:4], SLC, of the `MTL_TxQ(#i)_ETS_Control` register of a channel with number of slots over which the average transmitted bits per slot need to be computed.
3. Enable Bit 9(`ABPSSIE`) of the `MTL_Q(#i)_Interrupt_Control_Status` register of a channel to generate the average bits per slot interrupt.

Note
- The frequency of this interrupt depends on the value programmed in the step2. For example, when you program value 0 in the SLC field, the interrupt is generated at every 125 microsecond.
- When not required, you can disable this interrupt to stop the interrupt flooding.

4. Read Bits [16:0], ABS, from the `MTL_TxQ(#i)_ETS_Status` register of a channel on each interrupt.

Note The software can read the ABS bits in polling mode even if the ABPSIE bit is not enabled. When high, bit 1 (ABPSIS) of the MTL_TxQ(#i)_ETS_Status register indicates that a new value is updated in the ABS field.

14.28.4 Disabling Flow Control for AV Enabled Queues

14.28.4.1 Transmit Flow Control

Program the `PSTC(#i)` field corresponding to AV enabled Tx Queue in MTL_TC_Prty_Map0/1

14.28.4.2 Receive Flow Control

Program the EHFC (Enable Hardware Flow Control) bit of corresponding Rx Queue's `MTL_RxQ(#i)_Operation_Mode` register to 0.

14.29 Programming the GCL and GCL Linked Registers

Follow these steps to program the Gate Control List (GCL) and the four other registers implemented per GCL.

1. The GCL and the four other GCL-linked registers must be accessed through indirect addressing using the `MTL_GCL_Control` and `MTL_GCL_Data` registers. The GCLS field of the `MTL_GCL_Control` register indicates whether the EST feature or the PSF feature gets access to the GCL memory/register. The SWOL field of the `MTL_SGF_Status` register and `MTL_EST_Status` register indicates if GCL0 or GCL1 is owned by software.
2. To program the GCL, write the 32-bit write data to `MTL_GCL_Data` register. Then program `MTL_GCL_Control` register with the write address and other control information.
3. In the `MTL_GCL_Data` register, Write Data consists of up to 8 bits (configurable) of Gate Controls and up to 24 bits (configurable) of Time Interval. Gate Close is indicated by programming a '0' and Gate Open is indicated by programming a '1'. For a 4-Gate ID/TC and 24-bit Time Interval Configuration, the data width is 28 bits and the remaining 4 bits are reserved/read-only. The data must be written in the following format.

   ```
   {8'h0, Gate ID3/TC3, Gate ID2/TC2, Gate ID1/TC1, Gate ID0/TC0, 24-bit Time Interval}
   ```
4. `MTL_GCL_Control` register: Program the SRWO field to 1 (to start a Write operation) and program the address and R/W fields appropriately.
5. Poll and check for the SRWO field to be cleared by hardware to indicate the completion of the previous operation before initiating a new R/W operation through the same indirect addressing mode.
6. Repeat steps 3, 4, 5 until the programming of the GCL is completed.
7. Using the same indirect addressing method, program the BTR, CTR, TER and LLR registers. Set the GCRR field in the `MTL_GCL_Control` register which indicates that the R/W access is for GCL related registers (BTR, CTR, TER, LLR). For more details, see the *Register Descriptions* chapter.

After programming of the GCL and the related registers, program the `MTL_SGF_Control` register or `MTL_EST_Control` register for 25GMAC controller to own and process the GCL. When the List Length (as indicated in LLR) is 1, the associated Time Interval must be greater than the value of the Cycle Time Register. Otherwise, an error is reported (as detailed in the Error handling section) as a single set of gate controls add no value in the TSN context.

Note

- The time unit in all the GCL related registers is seconds and nano-seconds. If internally generated PTP System Time is used, the nano-seconds field must be programmed to use the Digital Rollover mode. (TSCTRLSSR field of MAC_Timestamp_Control must be set to 1).
- When GCL gating is not active, all the stream gate instance state is considered open as BTR is not reached.

14.30 Programming Guidelines for EST

The following sub-sections provide step by step details for programming the GCL and the other EST related registers.

Synopsys recommends to use Strict Priority or Credit Based Shaper (CBS) scheduling algorithms.

14.30.1 Programming the EST Registers

After completing the steps mentioned in "Programming the GCL and GCL Linked Registers" on page 1882, program the `MTL_EST_Control` register.

1. Set the Current Time Offset Value and Time Internal Left Shift fields of the `MTL_EST_Control` register appropriately. Also, set the Enable EST (EEST) and Switch to SWOL (SWOL) fields.
2. This enables 25GMAC to own and process the new GCL and switch to the new GCL at the BTR value. If interrupt is enabled, 25GMAC generates an interrupt when it switches to the new GCL.
3. Software must address any other interrupts received from 25GMAC during the execution of the GCL.

When the software sets the SSWL (Switch to Software Owned List) field in the `MTL_EST_Control` register, the GCL list is handed-off to 25GMAC. When 25GMAC has successfully switched to the new GCL list, it resets/clears the SSWL field. 25GMAC also flips the `SWOL` field to indicate the new GCL that the software owns.

Software is not allowed to write to the GCL and GCL linked registers when the SSWL field is set, because the new GCL might be in use by 25GMAC.

To install a new GCL, software must program the GCL it owns (indicated by the SWOL field) as described in "Programming the GCL and GCL Linked Registers" on page 1882. Then program the `MTL_EST_Control` register. Ensure that the new BTR is set to an appropriate value to avoid BTR Error that might requires software intervention.

For more information on BTR error reporting and BTR programming conditions see "Installing a New GCL" on page 380.

To avoid transmission overruns, the packet length (frame size) information must be available at all times. Therefore, in the DMA configurations, program the packet length in the first descriptor of every Tx frame. Similarly, in the MTL configuration, provide the packet length in the control word.

The IPG or EIPG, preamble or SFD overheads are handled by 25GMAC. Only the scheduler delays must be included in the EST overhead field as shown in Table 14-3.

Table 14-3 Overheads to be Accounted in the EST Overhead Field

Speeds	PTOV (ns)	CTOV (ns)	Overhead (in Bytes)	Per Packet Slot Interval	Slot Time to be Programmed for n packets
10 Gbps, 2.5 Gbps, 1Gpbs, 100 Mbps(XGMII, GMII, MII)	10 PTP clock cycles	3 PTP + 3 Tx clock cycles	**Single Port SRAM (SPRAM) Configurations**		
			14 Tx + (12+X+Y+Z) application clock cycles	(Packet Size + Overhead + IPG + 8) * (Time to transmit 1 byte) + 4 Tx clock cycles	n * (Per Packet Slot Interval) + 1 PTP clock cycle
			Two Port Asynchronous RAM Configurations		

Table 14-3 Overheads to be Accounted in the EST Overhead Field (continued)

Speeds	PTOV (ns)	CTOV (ns)	Overhead (in Bytes)	Per Packet Slot Interval	Slot Time to be Programmed for n packets
			10 Tx + (12 +X+Y+Z) application clock cycles	(Packet Size + Overhead + IPG + 8) * (Time to transmit 1 byte) + 4 Tx clock cycles	n * (Per Packet Slot Interval) + 1 PTP clock cycle
			Two Port Synchronous RAM Configurations		
			14 Tx + (12+X+Y+Z) Application clock cycles	(Packet Size + Overhead + IPG + 8) * (Time to transmit 1 byte) + 4 Tx clock cycles	n * (Per Packet Slot Interval) + 1 PTP clock cycle

The value of X, Y, and Z in Table 14-3 are as follows:

- When Tx COE or TSO is selected in the configuration, X is
  - 4 for DW == 32 bits
  - 2 when DW != 32 bits

  Otherwise X = 0.
- When TBS or OST is selected in the configuration, Y is
  - 4 for DW == 32 bits
  - 2 for DW == 64 bits

  Otherwise Y = 0.
- When VLAN tag insertion is enabled, Z is
  - 2 for DW == 32 bits

  Otherwise Z = 0.

Note To avoid transmission overruns, the Packet Length provided in the Transmit Descriptor must account for the SA and VLAN insertion, if applicable, that is, Packet Length must represent the actual packet length on the Ethernet line.

14.31 Enabling the Frame Preemption Function

14.31.1 Receive Side Programming

1. Enable at least 2 Rx queues by programming appropriate values in the MAC_RxQ_Ctrl0 register; one queue for express and the other for preemption when preemption receive traffic is expected.
2. Program the `MAC_RxQ_Ctrl(#i)` registers to route the received packets to MTL Rx Queues. Ensure that Preemptable traffic and express traffic are not routed to the same Rx queue.
   - As the Queue mapping for tagged packets is based on VLAN user-priority field, the priority of preemptable and express packets must be exclusive to each other. In other words, packets of a certain priority (traffic class) are either express or preemptable but cannot be both
   - If the `MCBCQEN` field of the `MAC_RxQ_Ctrl1` register is set, preemptive Multicast/Broadcast packets are routed to the `PMCBCQ` field as programmed in the `MAC_RxQ_Ctrl4` register. Therefore, ensure that `PMCBCQ` field is not same as any of the queues programmed for express queues
3. Program the preemption residue queue (RQ field of the `MAC_RxQ_Ctrl1` register) to route all non-classified received preemptive packets, so that these packets are not routed to the default/residue Queue for express packets, which is Rx Queue 0.

14.31.2 Transmit Side Programming

1. Enable at least 2 Tx Queues by programming appropriate values in the `TXQEN` field of the `MTL_TxQ(#i)Operation_Mode` registers; one for express and the other for preemption (by setting the corresponding bit in the PEC field of `MTL_FPE_CTRL_STS` register) when preemption receive traffic is expected.
2. Program the `EFPE` field of the `MAC_FPE_CTRL_STS` register to 1 to enable preemption function in the transmitter
3. Program the `AFSZ` field of the `MTL FPE_CTRL_STS` register to set the minimum size of the non-final fragments
4. When 802.1Qbv/EST (Enhancement to Scheduled Traffic) is also enabled, follow the programming guidelines for EST and GCL.
5. Program the Hold Advance (HADV) and Release Advance (RADV) fields of `MTL_FPE_Advance` register with how much early the hold and release respectively must be initiated so that the hold and release happen at exactly the boundaries of GCL rows. The value of HADV must be equal to the time required to transmit minimum_fragment_size + IPG/EIPG + Preamble, in nanosecond, so that the packet can be preempted before the hold window start
6. Program the `LBHT` field of `MTL_FPE_CTRL_STS` register to 1, if the hold operation must start from second row of new GCL without a 0 to 1 transition on the Hold/Release bit in GCL, that is, Hold/Release bits of GCL row 1 and 2 are set to 1.

14.32 Setting up the Time-Based Scheduling Function

The Launch Time is programmed in the Context Transmit Descriptors in DMA configurations and is driven as a control word in MTL configurations as follows:

The OSTC, Launch Time (TBS) and Frame Arrival Time (ATS) features are mutually exclusive and should not be used together; in case of a scenario where OSTC = LTV = 1 in MTL configuration, priority is given to OSTC, and the TBS and ATS are ignored.

In DMA configuration, if a context descriptor is received with a valid OSTC, TBSV, and LTV values, the Launch Time/GSN is ignored.

Note

1. For a proper TBS usage, the application must update tail pointer only after adding context as well as the first normal descriptor in the transmit ring.
2. Use separate channel (without TBS enabled) for time critical traffic. Mixing such traffic with TBS enabled traffic can cause delays in transmitting time critical traffic.

For more information on TBS programming guidelines, see "Time-Based Scheduling" on page 399.

14.33 Programming Guidelines for the Asynchronous Traffic Shaper (ATS)

14.33.1 Initialization Flow

The ATS field must be set to 2’b11 to enable frame queuing based on eligibility time assigned by ATS scheduler machines.

The EPAT field must be programmed to a desired value prior to starting any TX DMAs to reflect Frame Arrival Time capturing source.

TxQs with TXQEN = 2'b10(Non AV enabled) can only be mapped to ATS enabled TCs.

When the EATSP bit of `MTL_Operation_Mode` is set to 0, the priority of traffic scheduling is in descending order from higher numbered TCs to lower numbered TCs, that is, positional ordering is the existing behavior.

When the EATSP bit of `MTL_Operation_Mode` is set to 1, the priority of TCs programmed for ATS, ETS, and SP is altered such that ATS gets priority over ETS and SP, irrespective of the position of these algorithms programmed in the TCs. There is no change in priority for TCs programmed for other scheduling algorithms.

The BEFD and CIR fields must be programmed to the desired values prior to enabling the corresponding TxQ.

14.34 Programming Guidelines for Media Clock Generation and Recovery

14.34.1 Common Guidelines for Media Clock Generation and Recovery

Following are the common steps for media clock generation and recovery

1. Enable the Current Presentation Time (CPT) Counter by setting the PTGE field of `MAC_Timestamp_Control` register. In addition to programming the initialization values for the System Time, update the `MAC_Presn_Time_Updt` register with the equivalent Presentation Time initial value. This is a requirement for the TSINIT field of `MAC_Timestamp_Control` register to be set.
2. The Increment values used for the System time is also used for the Current Presentation time. This is because, the increment value is in Sub-seconds and Sub-nanoseconds.
3. When an update is needed, along with programming the update values for the System Time, update the `MAC_Presn_Time_Updt` register with the equivalent Presentation Time update value (32-bit ns). Also, program the TSUPDT field of `MAC_Timestamp_Control` register. This is required for the update to occur.
4. Program the appropriate Presentation Time Control (supported recovery modes "0001 - 0011") to PPSCTRL_PPSCMD (for 0th instance) and PPSCMD(#i) (for 1, 2, 3 instances) of `MAC_PPS_Control` register. Program all the MCGREN(#i) of the `MAC_PPS_Control` register to enable MCGR mode.

14.34.2 Programming Guidelines for Media Clock Generation

1. Complete the steps in "Common Guidelines for Media Clock Generation and Recovery" on page 1888.
2. The application must drive the appropriate trigger signal to the corresponding `mcg_pst_trig_i(#i)` signal, for the instances enabled for Media Clock Generation.
3. Based on the programmed mode, MCGR instance#i captures the timestamp and programs it in the `MAC_PPS(#i)_Target_Time_Seconds` register. 25GMAC asserts the `mcgr_dma_req_o(#i)` signal.
4. For every request generated by the MCGR instance#i, Application reads the corresponding `MAC_PPS(#i)_Target_Time_Seconds` register. Application must acknowledge the read request by asserting the corresponding `mcgr_dma_ack_i(#i)` signal.

14.34.3 Programming Guidelines for Media Clock Recovery

1. Complete the steps in "Common Guidelines for Media Clock Generation and Recovery" on page 1888.
2. Enable the Current Presentation Time (CPT) Counter by setting the PTGE field of `MAC_Timestamp_Control` register. In addition to programming the initialization values for the System Time, update the `MAC_Presn_Time_Updt` register with the equivalent Presentation Time initial value. This is a requirement for the TSINIT field of `MAC_Timestamp_Control` register to be set.
3. The Increment values used for the System time is also used for the Current Presentation time. This is because, the increment value is in Sub-seconds and Sub-nanoseconds.

4. When an update is needed, along with programming the update values for the System Time, update the `MAC_Presn_Time_Updt` register with the equivalent Presentation Time update value (32-bit ns). Also, program the TSUPDT field of `MAC_Timestamp_Control` register. This is required for the update to occur.
5. Program the appropriate Presentation Time Control (supported recovery modes "0001 - 0011") to PPSCTRL_PPSCMD (for 0th instance) and PPSCMD(#i) (for 1, 2, 3 instances) of `MAC_PPS_Control` register. Program all the MCGREN(#i) of the `MAC_PPS_Control` register to enable MCGR mode.
6. For instances that you enable for Media Clock Recovery, 25GMAC asserts the corresponding `mcgr_dma_req_o(#i)` signal. For every request generated by 25GMAC, program the target presentation time in the `MAC_PPS(#i)_Target_Time_Seconds` register.

   Application must acknowledge the request by asserting the corresponding `mcgr_dma_ack_i(#i)` signal either before or after programming the Target presentation time.
7. Observe the recovered media clock on the corresponding `ptp_pps_o(#i)` signal.

Note In these steps, PPS instance#i operating in MCGR mode is mentioned as MCGR instance#i

14.35 Programming Guidelines for ECC

14.35.1 Programming Guidelines for ECC Protection for Memories

Programming guidelines for the ECC protection for memories:

- Enable the ECC feature for the respective memory by setting the appropriate bit in `MTL_ECC_Control` register.
- A correctable interrupt (`sbd_sfty_ce_intr_o`) or an unrecoverable interrupt (`sbd_sfty_ue_intr_o`) are generated to indicate to the application if any correctable/uncorrectable/address errors are detected. Appropriate status is indicated in the `DMA_ECC_Interrupt_Status/MTL_ECC_Interrupt_Status` register.
- Debug mode is supported for each non-flexible receive parser memory to insert errors. For more details, see "Error Correction Code (ECC) Protection for Memories" on page 429.
- For details of Flexible Receive Parser ECC Hardware Error Injection method, see "Programming Guidelines for ECC Hardware Error Injection for Flexible Receive Parser Memory" on page 1890.

Note Enable the ECC feature before the traffic is on the line (or after the reset). Otherwise false interrupts might be triggered.

14.35.2 Programming Guidelines for ECC Protection for AXI Read/Write Operations

- Enable the ECC feature for the AXI Manager and Subordinate interfaces by clearing the AMSD/ASSD field in the `DMA_AM_Sfty_Ctrl/DMA_AS_Sfty_Ctrl` register.
- Program the `DMA_AM_Sfty_Intr_En/DMA_AS_Sfty_Intr_En` register to enable the correctable error interrupts. Uncorrectable error interrupts are always enabled.
- Program the CESM/CESS field in the `DMA_AM_Sfty_Ctrl/DMA_AS_Sfty_Ctrl` register to clear the `DMA_AM_Sfty_RDC_ECC_Err_Cntr_Sts/ DMA_AS_Sfty_WDC_ECC_Err_Cntr_Sts` register on read.
- Read the `DMA_AM_Sfty_ECC_Intr_Sts/ DMA_AS_Sfty_ECC_Intr_Sts` register to check the status of correctable error interrupts or uncorrectable error interrupts. Each ECC has a separate uncorrectable/correctable interrupt.
- Read the `DMA_AM_Sfty_RDC_ECC_Err_Cntr_Sts/ DMA_AS_Sfty_WDC_ECC_Err_Cntr_Sts` register to get the ECC error count for correctable and uncorrectable errors.

14.35.3 Programming Guidelines for ECC Hardware Error Injection for Flexible Receive Parser Memory

Follow these steps for ECC hardware error injection for Flexible Receive parser Memory:

1. Enable the ECC error injection for Flexible Receive Parser memory by setting the appropriate bit in the `MTL_Indirect_Acc_Control_Status` register while reading instruction. To access Flexible Receive Parser memory in debug mode, disable the Flexible Receive Parser feature by setting the FRPE bit field of `MTL_Operation_Mode` register to 0.

Note Avoid error injection in the MSB, reserved bits. For details of the reserved bits, see "Flexible Receive Parser Instruction Table (IT) Format" on page 190.

2. 25GMAC can detect FRP ECC error without reading instruction from the application, in the periodic ECC check mode. To enable this mode,
    a. Set a non-zero value in the TMR field of the MAC_FSM_ACT_Timer register.
    b. Set a default value in the CTMR field of the MAC_FSM_ACT_Timer register.
    c. Enable TMOUTEN field of the MAC_FSM_Control register.

For every tick generated, 25GMAC checks ECC from 0th instruction to the last valid instruction.

14.35.4 Programming Guidelines for ECC Hardware Error Injection in Debug Mode for AXI Read/Write Operations

- Program the MSEL and AOFF fields of the DMA_Sfty_Ind_Ctrl to select the AXI Manager/Subordinate channel where you intend to insert error.
- Provide the debug data to be driven on the selected AXI Manager/Subordinate channel in DMA_Sfty_Ind_Data register. For more details, see the description of DMA_<AM/AS>_Sfty_<channel>_dbg_<Data/Ctrl/Addr> in the *Register Description* chapter.
- Program the following fields of the DMA_Sfty_Dbg_EIC register:
    - BLEI field to control the bit-location of the error
    - EIM field to control the mode of error injection; data/ECC/parity
    - NBEI field to control the type of error injection; 1-bit or 2-bit
    - ECC_SEL field to select the ECC granularity
- Program ASIE/AMIE field in DMA_Sfty_Dbg_EIC register to inject error on AXI Manager/Subordinate interface channels.
- After error insertion, AMIE/ASIE field of DMA_Sfty_Dbg_EIC register is cleared by 25GMAC.

14.35.5 Programming Guidelines for ECC Hardware Error Injection in Debug Mode

Follow these steps for ECC hardware error injection in the debug mode.

Note These steps are not applicable to Flexible Receive Parser Memory.

- Enable the ECC error injection feature for MTL Tx/Rx and DMA TSO, by setting the appropriate bit in the MTL_Debug_Control register. For more details, see “Debug Mode Access” on page 250.
- Enable the ECC error injection feature for the EST memory by setting the appropriate bit in the MTL_GCL_Control register.
- In the debug mode, software must ensure that no traffic is present on the line.
- Where multiple CSR writes are required, for writing single data word into the memory, the application must ensure that all the CSR writes corresponding to one memory write must have the same value for the error injection control word.

14.36 Programming Guidelines for On-Chip Datapath Parity Protection

14.36.1 Programming Guidelines for Parity Protection of AXI Control Signals

- Enable the ECC feature for the AXI interface by clearing the AMSD/ASSD bit in `DMA_AM_Sfty_Ctrl/ DMA_AS_Sfty_Ctrl` register.
- Read `DMA_AM_Sfty_DPP_Intr_Sts/DMA_AS_Sfty_DPP_Intr_Sts` register to check status for uncorrectable error interrupts. Each DPP has separate interrupt status bit.

14.36.2 Programming Guidelines for On-Chip Datapath Parity Protection

Follow these steps for on-chip datapath parity protection:

- Select the required parity checkers by enabling the corresponding bit in the `MTL_DPP_CONTROL` register.
- Set the DDPP field of the `MTL_DPP_Control` register to enable the generation of parity and detection of parity Mismatch on datapath. An odd parity generation/detection can be enabled by setting the OPE bit of `MTL_DPP_control` register; otherwise, by default, even-parity generation/detection is active.
- An unrecoverable interrupt (`sbd_sfty_ue_intr_o`) is generated to indicate to the application if any parity Mismatches are detected and the appropriate status is indicated in DMA/`MTL_Safety_Interrupt_status` and `MAC_DPP_FSM_Interrupt_Status` register.
- Debug mode is supported for each parity generator, to insert error. For details of how to insert errors, see, "Parity Error Injection" on page 449.

Note To avoid false safety interrupts, enable the datapath parity protection before the start of Rx or Tx traffic.

14.36.3 Programming Guidelines for Parity Hardware Error Injection in Debug Mode for AXI Control Operations

Follow the same steps as "Programming Guidelines for ECC Hardware Error Injection in Debug Mode for AXI Read/Write Operations" on page 1891.

14.37 Programming Guidelines for FSM and Timeout

Follow these steps to program the FSM parity and timeout

- Set the PRTYEN and TMOUTEN fields of MAC_FSM_Control register to enable the FSM parity and timeout feature respectively
- For parity error detection, force the FSMs to enter into a state with even number of 1s and wait for the safety unrecoverable interrupt to be set with FSMPES field of the MAC_DPP_FSM_Interrupt_Status register.
- For Error Injection mode in FSM parity, select the appropriate clock domain for which parity error injection must be set using the [23:16] bits of MAC_FSM_Control register. Wait for the safety unrecoverable interrupt to be set with FSMPES of MAC_DPP_FSM_Interrupt_Status.
- For FSM Timeouts, program the large and normal mode values indicated by LTMRMD and NTMRMD of MAC_FSM_ACT_Timer register.
- Select large or normal mode tic generation per clock domain by using the [31:24] bits of MAC_FSM_Control Register.
- Set the TMR field of the MAC_ACT_Timer field with the appropriate number of CSR clock cycles used to generate 1us tic.
- Force the FSM in a particular clock domain to be in an active state for Timeout duration (for example, T-2T). Check for the safety interrupt and the status bit for the corresponding clock domain getting set in [15:8] bits of MAC_DPP_FSM_Interrupt_Status register.
- For Error Injection mode in FSM timeout, select the appropriate clock domain for which timeout error injection must be set using the [15:8] bits of MAC_FSM_Control register. Wait for the safety uncorrectable interrupt along with the corresponding error status set in MAC_DPP_FSM_Interrupt_Status.
- For application /CSR interface timeout, only normal mode tic generation is used. Based on the timeouts of configured interface, appropriate fields (MSTTES, SLVTES) of MAC_DPP_FSM_interrupt_Status is set along with the safety interrupt.

14.38 Programming Guidelines for Switching the AXI Clock Frequency

To dynamically change the AXI clock frequency (without applying soft reset or hard reset) in AXI configuration, follow these steps:

1. Disable the Transmit DMA (if applicable) and wait for any previous frame transmissions to complete. When the frame transmissions is complete, the Tx FIFO becomes empty and the Tx DMA enters the STOP state. The Tx FIFO status is given in the `MTL_TxQ(#i)_Debug` register and the status of DMA is given in the `DMA_Debug_Status0` register.
2. Disable the MAC transmitter and the MAC receiver by clearing the appropriate bits in `the MAC_Tx_Configuration and the MAC_Rx_Configuration` registers.
3. Disable the Receive DMA (if applicable) after making sure that the data in the Rx FIFO is transferred to the system memory. The Rx FIFO empty status is given in `MTL_RxQ_(#i)_Debug` register.
4. Ensure that the application does not perform any register read or write operation.
5. Change the frequency of the AXI clock.
6. Enable the MAC Transmitter or the MAC Receiver and the Transmit or Receive DMA.

These steps ensure that no valid data is present in the Tx FIFO or Rx FIFO at the time of clock frequency switching and prevent any data corruption.

Note To avoid glitches or short pulses, use glitch-free multiplexes or other mechanisms for clock switching.

14.39 Programming Guidelines for Dual System Time Source Support

14.39.1 Initialization Guidelines for Local System Time Generation

You can enable the local timestamp by setting bit 0 of the `MAC_Local_Timestamp_Control` register. However, it is essential that the local timestamp counter must be initialized after this bit is set. Complete the following steps during 25GMAC initialization:

1. Set Bit 0 of `MAC_Local_Timestamp_Control` register to enable timestamping.
2. Program `MAC_Local_Sub_Second_Increment` register based on the PTP local clock frequency.
3. Program `MAC_Local_System_Time_Seconds_Update` register and `MAC_Local_System_Time_Nanoseconds_Update` register with the appropriate time value.
4. Set Bit 2 in `MAC_Local_Timestamp_Control` register. The local timestamp counter starts operation as soon as it is initialized with the value written in the Local Timestamp Update registers.
5. Program the respective fields in the `MAC_Dual_TS_Src_Ctrl` register to select the adjustable/correctable or free running time source for One-Step and Two-Step Timestamping, PPS, Auxiliary Timestamping, and Time-based Scheduler Functions.

Coarse Correction Method

To synchronize or update the local system time in one process (coarse correction method), complete the following steps:

1. Set the offset (positive or negative) in the Local Timestamp Update registers (`MAC_Local_System_Time_Seconds_Update` and `MAC_Local_System_Time_Nanoseconds_Update`).
2. Set Bit 3 (`LTSUPDT`) of the `MAC_Local_Timestamp_Control` register. The value in the Local Timestamp Update registers is added to or subtracted from the local system time when the `LTSUPDT` bit is cleared.

14.40 Programming Guidelines for 25G XGMII Speed Mode and Timestamping Accuracy

Follow these steps to operate in the 25G XGMII speed.

1. Program the SS field of the `MAC_Tx_Configuration` register to 3'b001.
2. Connect the 781.25MHz clock to the clk_tx_312pt5_i and clk_rx_312pt5_i ports when the programmed value of 3'b001 is seen on mac_speed_o output port.

Follow these steps to enable the Timestamp Accuracy using Pulse and Latency input from PCS:

1. Setup the IEEE 1588 PTP Timestamping.
2. Program EPCSL of `MAC_Timestamp_Control` register to 1.
3. Program ECPD with EPCSL to capture timestamp in PTP clock domain.
4. Program the `MAC_Rx_Domain_Time_Incr` and `MAC_Tx_Domain_Time_Incr` registers with the required values. These registers are used for latency computation along with the information on ingress ports pcs_tx/rx_num_bit_chg_i[15:0].

The Time is incremented by a fixed amount, equal to Tx/Rx clock period, in the Rx and Tx clock domains based on the:

- Programmed operating speed
- Operating format of time
- Value programmed in the TSCTRLSSR bit of the `MAC_Timestamp_Control` register.

For precise time increment, including jitter, clock variation, and so on, and accurate timestamp capturing, appropriately program the `MAC_Rx_Domain_Time_Incr` and `MAC_Tx_Domain_Time_Incr` registers.

Follow the steps in the following examples. This is for ideal case. If the receive clock has jitter, clock variation and so on, program the fields accordingly.

Example 1: When operating in the 10G speed and the clock period is 3.2 ns:

- In the digital rollover mode, program the `MAC_Rx_Domain_Time_Incr` register as follows:
  - RXNS = 3
  - RXSNS = (0.2 * 256) = 51.
- In the binary rollover mode, program the `MAC_Rx_Domain_Time_Incr` register fields as follows:
  - RXNS = (3/0.466) = 6
  - RXSNS = (0.64 * 256) = 164

Example 2: When operating in the 2.5G XGMII speed and the clock period is 12.8 ns:

- In the digital rollover mode, program the `MAC_Rx_Domain_Time_Incr` register as follows:
  - RXNS = 12
  - RXSNS = (0.8 * 256) = 205.
- In the binary rollover mode, program the `MAC_Rx_Domain_Time_Incr` register fields as follows:
  - RXNS = (12/0.466) = 26
  - RXSNS = (0.55 * 256) = 141

You can use these steps to program the `MAC_Tx_Domain_Time_Incr` register fields.

14.41 Programming Guidelines for Enabling Tx Timestamp Status FIFO Streaming Interface

Follows these steps to enable Tx timestamp status FIFO streaming interface.

1. Program the TTSFSC field of the MAC_Timestamp_Control register to 1.
2. When the tx_ts_data_val_o is high, drive the tx_ts_data_rdy_i signal. This signal can be kept high as long as application is ready to accept the consecutive status. Capture the status from the tx_ts_data_o bus.

A

Area and Power

This appendix provides the information about area and power.

Note	The IP meets logic timing for target AXI/Tx/Rx clock frequencies required for 25Gbps operating speed, only with 5nm, multi Vth (SVT, LVT, and ~10% ULVT), high speed tech library.

This appendix contains the following sections:

- "Area" on page 1900
- "Power" on page 1915

A.1 Area

To estimate the area for your target configuration,

1. Take the area of the minimum target configuration (XGMAC-CORE or XGMAC-MTL, and so on) with 128-bit in the first row.
2. Add the area of the optional features selected in your configuration.
3. For features such as multiple DMA, Queues, address filters and so on, multiply the per-item area (mentioned in bold) with the selected number.
4. If your target configuration is 64-bit, add the corresponding area change factor for relevant features as given in Table A-3 on page 1907.

Note

- The calculated estimated area might have about +/- 5% variation with respect to the actual area based on the target technology library/operating conditions.
- The area numbers provided are with clock-gating and scan.

A.1.1 DMA Area Estimates

Table A-1 on page 1900 shows the approximate area estimates

Table A-1 DMA Area Estimates

Number of Traffic Channels	Number of Rx Queues	Number of Tx and Rx Channels	Area in Gates
4	4	4	~44-64K
8	8	8	~80-100K
8	8	16	~98-118K
8	8	24	~96-114K

A.1.2 Gate Count Summary

This section summarizes the area in terms of gate count for the various modules and configurations listed in Table A-2 on page 1901. Gate count is calculated with the following parameters:

- Design Compiler Version: U-2022.12-SP5
- Technology synthesized for: industry standard 7 nm FFC
- Application clock: 500 MHz
- XGMII clock: 312.5 MHz
- Gate size: NAND2x1 (= 0.0768 lib Area)
- Data bus width: 128 bits (default)

Table A-2 Gate Count Summary for Various 128-Bit Data Bus Configurations and Optional Functions

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
Minimum Configuration	24.7	46.4	122.5
Application Data Path			
Both Endianness	0.3	0.3	0.8
CSR Port interface			
APB	0.3	0.3	0.3
AXI4-Lite	NA	NA	3.0
AXI (32, 64, 128-bit)	NA	NA	3.8, 4.7, 6.8
Different Clock for CSR (Per Tx + Rx)	NA	0.5	2.4
AXI Interface Options			
Address-width (40, 48-bit)	NA	NA	2.0
Per Transmit DMA Outstanding Read Request (per 2 requests)	NA	NA	2.3
Per Receive DMA Outstanding Write Request (per 2 requests)	NA	NA	2.3
Unique AXI ID for Descriptor Access	NA	NA	8
AXI Outstanding Read Request (per 8 requests)	NA	NA	0.5
AXI Outstanding Write Request (per 8 requests)	NA	NA	2.8
Sideband Signals			
GPIO (Each set of 4 inputs/outputs)	0.3	0.3	0.3
General Features			
10/100 half-duplex support	3.5	3.5	3.5
Double VLAN	0.7	0.7	0.7
SA-VLAN Insertion	1.5	1.5	1.5
SA-VLAN Insertion (additional with Double VLAN)	**0.7**	**0.7**	**0.7**
SA-VLAN Insertion (additional with MAC_Address1 register)	1.1	1.1	1.1
Queue/Channel Based VLAN Tag Insertion (2 TxQ)	0.6	0.6	0.6

Table A-2 Gate Count Summary for Various 128-Bit Data Bus Configurations and Optional Functions (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
Queue/Channel Based VLAN Tag Insertion (Per additional TxQ)	0.2	0.2	0.2
Tx Status: Every 2 packets (per TxQ)	**NA**	**0.3**	**NA**
Tx Status: Every 2 packets (per TxQ with 1588)	**NA**	**1.3**	**NA**
ITU-T G.999.1	2.1	2.1	2.1
CDC Synchronization			
CSR MAC Address 0 synchronous to Tx/Rx Clock	0.8	0.8	0.8
Every Additional CSR MAC Address to Rx Clock	1.4	1.4	1.4
Pause Time to Tx Clock	0.2	0.2	0.2
Buffer Management			
Maximum Rx FIFO Size (256KB)	NA	1.5	1.5
Maximum Tx FIFO Size (256KB)	NA	1.0	1.0
Single Port SRAM (SPRAM)	NA	38.3	38.3
Synchronous 2-Port SRAM	NA	28.0	28.0
Debug Memory Access	NA	3.4	3.4
Filtering Features			
Each Additional MAC Address Register (1 - 31)	0.9	0.9	0.9
DA Hash Filter (64bit, 128bit, 256bit)	0.8, 1.5, 3.0	0.8, 1.5, 3.0	0.8, 1.5, 3.0
DA Hash Filter (per 64-bit when CDC-synchronized to Rx clock)	0.6	0.6	0.6
VLAN Hash Filter	0.2	0.2	0.2
Extended VLAN Filter (set of 4 filters)	1.3	1.3	1.3
Extended VLAN Filter (every additional set of 4 filters)	**1.0**	**1.0**	**1.0**
Base logic of Extended VLAN/DAH Filter with External Memory	7.7	7.7	7.7
DA Hash Filter (512, 1024, 2048, 4096)	7.7	7.7	7.7
Extended VLAN Filter (4096)	3	3	3

Table A-2 Gate Count Summary for Various 128-Bit Data Bus Configurations and Optional Functions (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
Extended VLAN Filter with Double VLAN (4096)	4.3	4.3	4.3
L3-L4 Filter	6.8	6.8	6.8
Each additional L3-L4 Filter	4.9	4.9	4.9
Flexible Receiver Parser (64B Header parsing, 64 entry table, 2 Parallel Engines)	**NA**	**NA**	77.6
Each additional 64 byte parsing	NA	NA	0.5
Each additional 64 Instruction Table	NA	NA	62.3
Per additional Parallel engine in FRP	**NA**	NA	1.2
PHY Interface			
1G/100M support	1.5	1.5	1.5
Additional GMII/MII Port	0.5	0.5	0.5
RGMII	0.7	0.7	0.7
RMII	0.6	0.6	0.6
XGMII DDW	5.2	5.2	5.2
XGMII with Handshake Interface	0.2	0.2	0.2
Variable Preamble in XGMII	0.2	0.2	0.2
SMA (MDIO)	16.2	16.2	16.2
Multiple Channel, Multiple Queue Features			
Packet Duplication (32 MAC Addresses)	NA	NA	3.2
DCB (1 Tx + 2 RxQ)	1.4	6.2	31.8
DCB (2 Tx + 2 RxQ)	1.4	22.5	71.5
DCB: Every Additional RxQ	0.5	4.0	29.5
DCB: Every Additional TxQ	0.1	9.9	50.2
AVB (2 Tx + 1 RxQ)	NA	14.2	39.1
AVB: Every additional TxQ	NA	7.2	44.9
AVB: Every additional RxQ	0.2	7.5	32.7

Table A-2 Gate Count Summary for Various 128-Bit Data Bus Configurations and Optional Functions (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
For SPRAM/synchronous 2-port SRAM option, Reduce per TxQ	NA	- 1.2	- 1.2
For SPRAM/synchronous 2-port SRAM option, Reduce per RxQ	NA	- 0.8	- 0.8
DCB + AVB - 2 Tx + 2 RxQ	3.1	26.0	75.7
Each additional TxQ with AVB and DCB	0.1	10	50
Each additional TxQ without AVB or DCB	NA	7.2	44.8
Each additional RxDMA Channel	NA	NA	7.3
External Lookup Interface	NA	NA	1.3
PFC	8.7	9.0	9.0
VxLAN/NVGRE	NA	NA	62.1
RSS (16, 32, 64, 128, 256)	NA	NA	15
RSS with external memory (512, 1024, 2048, 4096)	NA	NA	36.7, 52.2, 84, 148
TCP/IP Offloading Features			
Receive COE	4.6	4.6	4.6
Transmit COE (Per TxQ)	NA	19	19
TSO (Per TxDMA)	NA	NA	7.2
TSO - Dedicated Memory	NA	NA	1.7
TSO: Dedicated Memory (Add Debug Access)	NA	NA	3.5
Split Header in Rx path	NA	NA	3.4
IPv4 ARP Offload	2.2	2.2	2.2
ARP Protocol Address synchronous to Tx/Rx Clock	0.6	0.6	0.6
IEEE 1588 Timestamping			
External System Time Source	10.6	10.6	10.6
Internal System Time Source	17.5	17.5	17.5
Both System Time Source	17.3	17.3	17.3

Table A-2 Gate Count Summary for Various 128-Bit Data Bus Configurations and Optional Functions (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
IEEE 1588 Timestamping: Higher Word	0.2	0.2	0.2
IEEE 1588 Timestamping: Sub-nanosecond	2.4	2.4	2.4
Aux Snapshot (FIFO = 4, 8,16)	3.7, 6.0, 10.5	3.7, 6.0, 10.5	3.7, 6.0, 10.5
Each Flexible PPS	7.2	7.2	7.2
PTP Offload	13.8	13.8	13.8
One-Step Timestamping for PTP over Ethernet	6.1	6.3	6.3
One-Step Timestamping PTP over UDP/IP	3.3	2.6	2.6
For SPRAM (with IEEE Timestamping)	NA	1.8	1.8
For SPRAM (with One-step Timestamping)	NA	1.8	1.8
Tx Timestamp Status in register (1,2,4,8,16)	1, 1.7, 3.1, 5.9, 11.5	1, 1.7, 3.1, 5.9, 11.5	1, 1.7, 3.1, 5.9, 11.5
Time Sensitive Networking Features			
MAC Frame Preemption	NA	26	26
802.1Qbv Time Aware Shaper (EST) - (2 Tx + 1 RxQ)	NA	26	26
802.1Qbv Time Aware Shaper (EST) - per additional TxQ	NA	8.3	46.5
Additional logic of FPE + EST	NA	9	11
Time Based Scheduling (TBS) without EST - per TxQ	NA	NA	6.5
Time Based Scheduling (TBS) with EST-2 TxQ	NA	NA	66.1
Time Based Scheduling (TBS) with EST (per additional TxQ)	NA	NA	8.4
Stream-Gate Filtering	NA	NA	7.1
Policing Counters (16, 32)	15.4, 28.5	15.4, 28.5	15.4, 28.5
RMON/MMC Counters			
RMON/MMC Counters - Tx Counter Common logic	2.2	2.2	2.2
RMON/MMC Counters - Each 64-bit Tx Octet Counter	0.9	0.9	0.9
RMON/MMC Counters - Each 32-bit Tx Packet Counter	0.5	0.5	0.5
Area Increase for each 64-bit Tx Packet Counter	0.9	0.9	0.9

Table A-2 Gate Count Summary for Various 128-Bit Data Bus Configurations and Optional Functions (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
RMON/MMC Counters - Rx Counter Common logic	2.7	2.7	2.7
RMON/MMC Counters - Each 64-bit Rx Octet Counter	1.0	1.0	1.0
RMON/MMC Counters - Each 32-bit Rx Packet Counter	0.5	0.5	0.5
Area Increase for each 64-bit Rx Packet Counter	0.8	0.8	0.8
RMON/MMC Counters - Rx IPC Counter Common logic	2.9	2.9	2.9
RMON/MMC Counters - Each 64-bit Rx IPC Octet Counter	0.6	0.6	0.6
RMON/MMC Counters - Each 32-bit Rx IPC Packet Counter	0.5	0.5	0.5
Area Increase for each 64-bit Rx IPC Packet Counter	0.5	0.5	0.5
Priority Counters	59.0	59.0	59.0
Priority Counters (additional with PFC)	8.7	8.7	8.7
Automotive Safety Features			
Safety features (ECC, DPP, FSM, Timeouts)	NA	26.5	44.5
AXI Interface Safety (ECC for Data and Parity for Controls)	NA	NA	NA
Low Power Management			
Energy Efficient Ethernet (EEE)	2	2	2
PMT: Magic packet	2	2	2
PMT: Remote Wakeup (4, 8, 16 filters)	10, 18.2, 35	10, 18.2, 35	10, 18.2, 35
PMT Remote Wakeup Packet Filter synchronous to Rx Clock (per set of 4 filters)	1.7	1.7	1.7
Maximum Configuration (1 Tx + 1 Rx) with SPRAM	666.2	817	1063.5
Maximum Configuration (16 Tx + 16 Rx) with SPRAM	747.6	1990	4209.3

Table A-3 on page 1907 summarizes the area in terms of gate count for the various modules and configurations. Gate count is calculated with the following parameters:

- Design Compiler Version: U-2022.12-SP5
- Technology synthesized for: Industry standard 7nm FFC
- Application clock: 500 MHz
- XGMII clock: 312.5 MHz

- Gate size: NAND2x1 (= 0.0768 lib Area)
- Data bus width: 64 bits

Table A-3 Area of Important Features for 64-Bit Data Bus Configurations

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
Minimum Configuration	23.7	38.8	105.8
Application Data Path			
Both Endianness	0.3	0.3	0.6
CSR Port interface			
APB	0.3	0.3	0.3
AXI4-Lite	NA	NA	2
AXI (32, 64, 128-bit)	NA	NA	4.6, 6.1, 6.1
Different Clock for CSR (Per Tx + Rx)	NA	0.5	2.9
AXI Interface Options			
Address-width (40, 48-bit)	NA	NA	3.3
Per Transmit DMA Outstanding Read Request (per 2 requests)	NA	NA	2.2
Per Receive DMA Outstanding Write Request (per 2 requests)	NA	NA	3
Unique AXI ID for Descriptor Access	NA	NA	8
AXI Outstanding Read Request (per 8 requests)	NA	NA	0.5
AXI Outstanding Write Request (per 8 requests)	NA	NA	3.7
Sideband Signals			
GPIO (Each set of 4 inputs/outputs)	0.3	0.3	0.3
General Features			
10/100 half-duplex support	3.6	3.6	3.6
Double VLAN	0.7	0.7	0.7
SA-VLAN Insertion	1.4	1.4	1.4
SA-VLAN Insertion (additional with Double VLAN)	**1.9**	**1.9**	**1.9**
SA-VLAN Insertion (additional with MAC_Address1 register)	1.1	1.1	1.1

Table A-3 Area of Important Features for 64-Bit Data Bus Configurations (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
Queue/Channel Based VLAN Tag Insertion (2 TxQ)	0.7	0.7	0.7
Queue/Channel Based VLAN Tag Insertion (Per additional TxQ)	0.2	0.2	0.2
Tx Status: Every 2 packets (per TxQ)	**NA**	**0.3**	**NA**
Tx Status: Every 2 packets (per TxQ with 1588)	**NA**	**1.3**	**NA**
ITU-T G.999.1	2.1	2.1	2.1
CDC Synchronization			
CSR MAC Address 0 synchronous to Tx/Rx Clock	0.8	0.8	0.8
Every Additional CSR MAC Address to Rx Clock	1.5	1.5	1.5
Pause Time to Tx Clock	0.2	0.2	0.2
Buffer Management			
Maximum Rx FIFO Size (256KB)	NA	1.2	1.2
Maximum Tx FIFO Size (256KB)	NA	1.2	1.2
Single Port SRAM (SPRAM)	NA	23.6	23.6
Synchronous 2-Port SRAM (S2PRAM)	NA	17.3	17.3
Debug Memory Access	NA	2.6	2.6
Filtering Features			
Each Additional MAC Address Register (1 - 31)	0.9	0.9	0.9
DA Hash Filter (64bit, 128bit, 256bit)	0.8, 1.5, 2.9	0.8, 1.5, 2.9	0.8, 1.5, 2.9
DA Hash Filter (per 64-bit when CDC-synchronized to Rx clock)	0.6	0.6	0.6
VLAN Hash Filter	0.2	0.2	0.2
Extended VLAN Filter (set of 4 filters)	1.3	1.3	1.3
Extended VLAN Filter (every additional set of 4 filters)	**1.0**	**1.0**	**1.0**
Base logic of Extended VLAN/DAH Filter with External Memory	7.7	7.7	7.7
DA Hash Filter (512, 1024, 2048, 4096)	7.7	7.7	7.7
Extended VLAN Filter (4096)	3	3	3

Table A-3 Area of Important Features for 64-Bit Data Bus Configurations (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
Extended VLAN Filter with Double VLAN (4096)	4.4	4.4	4.4
L3-L4 Filter	6.8	6.8	6.8
Each additional L3-L4 Filter	4.9	4.9	4.9
Flexible Receiver Parser (64-bit Header parsing, 64 entry table, 2 Parallel Engines)	**NA**	**NA**	74.8
Each additional 64 byte parsing	NA	NA	0.2
Each additional 64 Instruction Table	NA	NA	62.3
Per additional Parallel engine in FRP	**NA**	**NA**	1.2
PHY Interface			
1G/100M support	1.5	1.5	1.5
Additional GMII/MII Port	0.5	0.5	0.5
RGMII	0.7	0.7	0.7
RMII	0.6	0.6	0.6
XGMII DDW	5.2	5.2	5.2
XGMII with Handshake Interface	0.2	0.2	0.2
Variable Preamble in XGMII	0.2	0.2	0.2
SMA (MDIO)	16.2	16.2	16.2
Multiple Channels and Multiple Queues Features			
Packet Duplication (32 MAC Addresses)	NA	NA	3
DCB (1 Tx + 2 RxQ)	2.0	6.3	30
DCB (2 Tx + 2 RxQ)	2.0	20.7	66.2
DCB - Every Additional RxQ	0.5	3.9	25.4
DCB - Every Additional TxQ	0.2	9.0	43
AVB (2 Tx + 1 RxQ)	NA	12.5	35.4
AVB: Every additional TxQ	NA	6.2	38.5
AVB: Every additional RxQ	0.3	8.6	31.4

Table A-3 Area of Important Features for 64-Bit Data Bus Configurations (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
For SPRAM/synchronous 2-port SRAM option, Reduce per TxQ	NA	- 1.2	- 1.2
For SPRAM/synchronous 2-port SRAM option, Reduce per RxQ	NA	- 0.6	- 0.6
DCB + AVB - 2 Tx + 2 RxQ	4.5	26.3	71.5
Each additional TxQ with AVB and DCB	0.2	8	41.7
Each additional TxQ without AVB or DCB	NA	6.2	38.5
Each additional RxDMA Channel	NA	NA	8.9
External Lookup Interface	NA	NA	1.1
PFC	8.7	9.0	9.0
VxLAN/NVGRE	NA	NA	NA
RSS (16, 32, 64, 128, 256)	NA	NA	13.0, 13.6, 14.1, 15.4, 18.1
RSS with external memory (512, 1024, 2048, 4096)	NA	NA	29.2
TCP/IP Offloading Features			
Receive COE	4.6	4.6	4.6
Transmit COE (Per TxQ)	NA	11.7	11.7
TSO (Per TxDMA)	NA	NA	6.4
TSO: Dedicated Memory (Per TxDMA)	NA	NA	1.9
TSO: Dedicated Memory (Add Debug Access)	NA	NA	7.2
Split Header in Rx path	NA	NA	3.4
IPv4 ARP Offload	2.2	2.2	2.2
ARP Protocol Address synchronous to Tx/Rx Clock	0.6	0.6	0.6
IEEE 1588 Timestamping			
External System Time Source	10.6	10.6	10.6
Internal System Time Source	17.5	17.5	17.5
Both System Time Source	17.2	17.2	17.2
IEEE 1588 Timestamping - Higher Word	0.3	0.3	0.3

Table A-3 Area of Important Features for 64-Bit Data Bus Configurations (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
IEEE 1588 Timestamping - Sub-Nanosecond	2.7	2.7	2.7
Aux Snapshot (FIFO = 4, 8,16)	3.7, 6.0, 10.6	3.7, 6.0, 10.6	3.7, 6.0, 10.6
Each Flexible PPS	7.1	7.1	7.1
PTP Offload	13.8	13.8	13.8
One-Step Timestamping for PTP over Ethernet	6.2	6.8	6.8
One-Step Timestamping PTP over UDP/IP	3.0	3.0	3.0
For SPRAM (with IEEE Timestamping)	NA	1.7	1.7
For SPRAM (with One-step Timestamping)	NA	1.7	1.7
Tx Timestamp Status in register (1,2,4,8,16)	1, 1.7, 3.1, 5.9, 11.5	1, 1.7, 3.1, 5.9, 11.5	1, 1.7, 3.1, 5.9, 11.5
Time Sensitive Networking Features			
MAC Frame Preemption (FPE)	NA	22.0	22.0
802.1Qbv Time Aware Shaper (EST) - (2 Tx + 1RxQ)	NA	26.1	26.1
802.1Qbv Time Aware Shaper (EST) - per additional TxQ	NA	7.9	39.2
Additional logic of FPE + EST	NA	8.2	10
Time Based Scheduling (TBS) without EST - per TxQ	NA	NA	5.1
Time Based Scheduling (TBS) with EST-2 TxQ	NA	NA	64.9
Time Based Scheduling (TBS) with EST - per additional TxQ	NA	NA	5
Stream Gate Filtering	NA	NA	15.9
Policing Counters (16, 32)	15.4, 28.5	15.4, 28.5	15.4, 28.5
RMON/MMC Counters			
RMON/MMC Counters - Tx Counter Common logic	2.2	2.2	2.2
RMON/MMC Counters - Each 64-bit Tx Octet Counter	1.0	1.0	1.0
RMON/MMC Counters - Each 32-bit Tx Packet Counter	0.5	0.5	0.5
Area Increase for each 64-bit Tx Packet Counter	0.8	0.8	0.8
RMON/MMC Counters - Rx Counter Common logic	2.7	2.7	2.7

Table A-3 Area of Important Features for 64-Bit Data Bus Configurations (continued)

Features	Area in Kgates (NAND2)		
	XGMAC-CORE	XGMAC-MTL	XGMAC-AXI
RMON/MMC Counters - Each 64-bit Rx Octet Counter	1.0	1.0	1.0
RMON/MMC Counters - Each 32-bit Rx Packet Counter	0.7	0.7	0.7
Area Increase for each 64-bit Rx Packet Counter	0.8	0.8	0.8
RMON/MMC Counters - Rx IPC Counter Common logic	2.9	2.9	2.9
RMON/MMC Counters - Each 64-bit Rx IPC Octet Counter	0.6	0.6	0.6
RMON/MMC Counters - Each 32-bit Rx IPC Packet Counter	0.5	0.5	0.5
Area Increase for each 64-bit Rx IPC Packet Counter	0.5	0.5	0.5
Priority Counters	59.0	59.0	59.0
Priority Counters (additional with PFC)	8.7	8.7	8.7
Automotive Safety Features			
Safety features (ECC, DPP, FSM, Timeouts)	NA	21.8	34.3
AXI Interface Safety (ECC for Data and Parity for Controls)	NA	NA	9.5
Low Power Management			
Energy Efficient Ethernet (EEE)	2.0	2.0	2.0
PMT: Magic packet	2.0	2.0	2.0
PMT: Remote Wakeup (4, 8, 16 filters)	9.9, 18.3, 35.0	9.9, 18.3, 35.0	9.9, 18.3, 35.0
PMT Remote Wakeup Packet Filter synchronous to Rx Clock (per set of 4 filters)	1.7	1.7	1.7
Maximum Configuration (1 Tx + 1 Rx) with SPRAM	664.9	764.5	1002.5
Maximum Configuration (16 Tx + 16 Rx) with SPRAM	747.4	1806.9	3657.7

A.1.3 Area for Scan Ready and Clock Gating

The area for scan ready and clock gating for the two configurations described in Table A-4 on page 1913 are shown in Table A-5 on page 1913.

Table A-4 Configurations

Configurations	Configuration 1	Configuration 2
Main Parameters	■ XGMAC-CORE ■ PHY Interface: XGMII + GMII ■ APB3 CSR Interface ■ 64-bit, Little Endian ■ Asynchronous Reset ■ XGMI Interface = DDW ■ Optional Features: ❑ L3/L4 filter ❑ DA Hash Filter ❑ SMA ❑ DCB, G999.1 ❑ MMC ❑ OST ❑ PMT with 4 RWK	■ XGMAC-AXI ■ PHY Interface: XGMII + GMII ■ AXI CSR Interface ■ 128 bit, Big Endian ■ Asynchronous Reset ■ XGMI Interface = SDR ■ Optional Features: ❑ VLAN filter ❑ 32 MAC filters ❑ 2TxQ + 2 TxDMA Channel, ❑ 3RxQ + 2 RxDMA Channel ■ Optional Features: ❑ DCB ❑ TSO for 1 channel ❑ Split Header ❑ PMT with MGK, ❑ 8 RWK filters and EEE ❑ One Step Time stamp ❑ Time stamp with both sources
Technology	16 nm FFC	
Library	Industry Standard	
WLM	zero net area Wire load Model	
Gate size	1 NAND2x1 = 0.15552 lib Area	
Application clock frequency	500 MHz	

Table A-5 Gate Count to Scan Ready and Clock Gating

Name	Clock Gating	Scan Ready	Test Coverage (%)	Gated Registers (%)	Area (in K gates)
Configuration 1: XGMAC-CORE					
1	No	No	N/A	N/A	140

Table A-5 Gate Count to Scan Ready and Clock Gating (continued)

Name	Clock Gating	Scan Ready	Test Coverage (%)	Gated Registers (%)	Area (in K gates)
1b	No	Yes	99.63	N/A	167
1a	Yes	No	N/A	79.19	123
1c	Yes	Yes	99.43	79.19	152
Configuration 2: XGMAC-AXI					
1	No	No	N/A	N/A	254
1b	No	Yes	99.74	N/A	294
1a	Yes	No	N/A	88.31	234
1c	Yes	Yes	99.47	88.31	279

A.2 Power

Power dissipation estimates with and without clock gating are shown in Table A-6 on page 1915. The power estimates are for the configurations described in Table A-4 on page 1913 and are for simulations with continuous transmission and reception of around 1,600 frames with the application clock frequency given in the table.

Table A-6 Power Estimates and Gate Count With and Without Clock Gating

		Area (in K gates)	Combination/Sequential Cell Counts	Internal Power (mw)	Switch Power (mw)	Leakage Power (mw)	Total Power(mw)
Configuration: XGMAC-CORE							
Clock Gating	No	140	65/75	12.2	0.233	0.475	12.908
	Yes	123	48/75	3.76	0.336	0.449	4.54
Configuration: XGMAC-AXI							
Clock Gating	No	254	125/160	15.7	0.16	0.97	16.8
	Yes	234	123/160	3.2	0.33	0.88	4.5

B

Endian Support

This appendix provides information about the endian modes that 25GMAC supports for 32-bit, 64-bit, or 128-bit data bus.

This appendix contains the following sections:

- "Introduction" on page 1918
- "64-bit Data Bus Transfers" on page 1919
- "128-bit Data Bus Transfers" on page 1922

B.1 Introduction

The 25GMAC supports the following endian modes:

- "Little-Endian Mode" on page 1918
- "Big-Endian Mode" on page 1918
- "Byte-Invariant Big-Endian Mode" on page 1918

B.1.1 Little-Endian Mode

In the little-endian (LE) mode, the lowest storage address accesses the least significant byte (LSB) lane of the data bus and the highest address accesses the most significant byte (MSB) lane. This mode is supported in all 25GMAC configurations.

B.1.2 Big-Endian Mode

25GMAC supports byte-invariant big-endian mode as described in "Byte-Invariant Big-Endian Mode" on page 1918.

B.1.3 Byte-Invariant Big-Endian Mode

In the byte-invariant big-endian (BIBE) or address invariant mode, the address of each byte of memory remains unchanged when switching between the little-endian and big-endian modes. When a data item larger than a byte is loaded from or stored to the memory, the bytes making up that data item are arranged in the correct order depending on the endianness of the memory access. This mode is supported only in the XGMAC-AXI configuration on the AXI bus interface.

Note	Although the XGMAC-AXI manager port supports the byte-invariant big-endian mode, it works in the similar way as the conventional big-endian mode. This is because the XGMAC-AXI manager always uses transfers size equal to the AXI bus data width. Such transfers are identical in both conventional and byte-invariant big-endian modes.

B.2 64-bit Data Bus Transfers

This section provides information about the little-endian, conventional big-endian, and byte-invariant big-endian mode accesses on 64-bit data bus for Byte, HalfWord, Word, and DWord transfers. It contains the following sections:

- "Manager Interface" on page 1919
  - "Little-Endian Data Transfer" on page 1919
  - "Conventional Big-Endian/ Byte-Invariant Big-Endian Data Transfer" on page 1919
- "Register Bus Interface" on page 1919
  - "Little-Endian Data Transfer" on page 1920
  - "Byte-Invariant Big-Endian Data Transfer" on page 1921

B.2.1 Manager Interface

The XGMAC-AXI Manager interface initiates only DWord transfers. Therefore, this section does not describe the transfer types such as HalfWord, Byte, and Word at address offset 4.

B.2.1.0 Little-Endian Data Transfer

Table B-1 on page 1919 provides information about the data transfers in little-endian mode on the XGMAC-AXI Manager interface.

Table B-1 64-Bit Data Bus: Little Endian Support for Manager Interface

Transfer Size	Address Offset A[2:0]	Data [63:56]	Data [55:48]	Data [47:40]	Data [39:32]	Data [31:24]	Data [23:16]	Data [15:8]	Data [7:0]
DWord	0	MSB	MSB – 1	MSB – 2	MSB – 3	LSB + 3	LSB + 2	LSB + 1	LSB

B.2.1.0 Conventional Big-Endian/ Byte-Invariant Big-Endian Data Transfer

Table B-2 on page 1919 provides information about the data transfer in conventional big-endian and byte-invariant big-endian mode on the XGMAC-AXI Manager interface.

Table B-2 64-Bit Data Bus: Big-Endian/Byte-Invariant Big-Endian Support for Manager Interface

Transfer Size	Address Offset A[2:0]	Data [63:56]	Data [55:48]	Data [47:40]	Data [39:32]	Data [31:24]	Data [23:16]	Data [15:8]	Data [7:0]
DWord	0	LSB	LSB – 1	LSB + 2	LSB + 3	MSB – 3	MSB – 2	MSB – 1	MSB

B.2.2 Register Bus Interface

The 25GMAC registers and internal bus are 32-bit wide. Therefore, the DWord (64-bit) CSR access internally get converted into two 32-bit CSR accesses as shown in Table B-3 on page 1920. The DWord access through the register bus interface is supported only with the AXI subordinate port.

Note 64-bit Data Bus is not supported in APB3/MCI manager interface configurations.

Table B-3 64-Bit Data Bus: Reference CSR Register for Register Bus Interface

CSR Bits	31:24	23:16	15:8	7:0
First CSR Register	D3	D2	D1	D0
Second CSR Register	D7	D6	D5	D4
Address in LE	A3	A2	A1	A0
	A7	A6	A5	A4
Address in CBE or BIBE	A0	A1	A2	A3
	A4	A5	A6	A7

B.2.2.0 Little-Endian Data Transfer

Table B-4 on page 1920 provides information about data transfers in little-endian mode on the 25GMAC's AXI Subordinate interface.

Table B-4 64-Bit Data Bus: Little-Endian Support for Register Bus Interface

Transfer Size	Address Offset A[2:0]	Data [63:56]	Data [55:48]	Data [47:40]	Data [39:32]	Data [31:24]	Data [23:16]	Data [15:8]	Data [7:0]
DWord	0	D7	D6	D5	D4	D3	D2	D1	D0
Word	0					D3	D2	D1	D0
Word	4	D7	D6	D5	D4				
HalfWord	0							D1	D0
HalfWord	2					D3	D2		
HalfWord	4			D5	D4				
HalfWord	6	D7	D6						
Byte	0								D0
Byte	1							D1	
Byte	2						D2		
Byte	3					D3			
Byte	4				D4				
Byte	5			D5					
Byte	6		D6						
Byte	7	D7							

B.2.2.0 Byte-Invariant Big-Endian Data Transfer

Table B-5 on page 1921 provides information about the data transfers in byte-invariant big-endian mode on the 25GMAC's AXI Subordinate interface. For DWord access at offset 0, Bits[63:32] of the AXI bus get transferred to bits[31:0] of the Register 0 and bits[31:0] get transferred to bits[31:0] of Register 1. For Word access at offset 0, bits[31:0] of the AXI bus get transferred to Register 0.

For HalfWord access to Address A0, bits[15:0] of the AXI bus get transferred to bits[31:16] of the Control and Status Register.

Table B-5 64-Bit Data Bus: Big-Endian/Byte-Invariant Big-Endian Support for Register Bus Interface

Transfer Size	Address Offset A[2:0]	Data [63:56]	Data [55:48]	Data [47:40]	Data [39:32]	Data [31:24]	Data [23:16]	Data [15:8]	Data [7:0]
DWord	0	D3	D2	D1	D0	D7	D6	D5	D4
Word	0					D3	D2	D1	D0
Word	4	D7	D6	D5	D4				
HalfWord	0							D3	D2
HalfWord	2					D1	D0		
HalfWord	4			D7	D6				
HalfWord	6	D5	D4						
Byte	0								D3
Byte	1							D2	
Byte	2						D1		
Byte	3					D0			
Byte	4				D7				
Byte	5			D6					
Byte	6		D5						
Byte	7	D4							

B.3 128-bit Data Bus Transfers

This section provides information about the little-endian, conventional big-endian, and byte-invariant big-endian mode accesses on 128-bit data bus for byte, HalfWord, Word, DWord, and QWord transfers. It contains the following sections:

- "Manager Interface" on page 1922
  - "Little-Endian Data Transfer" on page 1922
  - "Conventional Big-Endian/Byte-Invariant Big-Endian Data Transfer" on page 1922
- "Register Bus Interface" on page 1923
  - "Little-Endian Data Transfer" on page 1923
  - "Byte-Invariant Big-Endian Data Transfer" on page 1925

B.3.1 Manager Interface

The XGMAC-AXI Manager interface initiates only QWord transfers. Therefore, this section does not describe the transfer types such as HalfWord and Byte.

B.3.1.0 Little-Endian Data Transfer

Table B-6 on page 1922 provides information about the data transfers in little-endian mode on the 25GMAC's AXI Manager interface.

Table B-6 128-Bit Data Bus: Little-Endian Support for Manager Interface

Transfer Size	Address Offset A[3:0]	Data															
		127:120	119:112	111:104	103:96	95:88	87:80	79:72	71:64	63:56	55:48	47:40	39:32	31:24	23:16	15:8	7:0
QWord	0	MSB	MSB – 1	MSB – 2					MSB – 7	LSB + 7						LSB +1	LSB

B.3.1.0 Conventional Big-Endian/Byte-Invariant Big-Endian Data Transfer

Table B-7 on page 1922 provides information about the data transfers in conventional big-endian or byte-invariant big-endian mode on the 25GMAC's AXI Manager interface. In byte-invariant big-endian mode for AXI, only QWord transfers are applicable.

Table B-7 128-Bit Data Bus: Conventional Big-Endian/Byte-Invariant Big-Endian Support for Manager Interface

Transfer Size	Address Offset A[3:0]	Data															
		127:120	119:112	111:104	103:96	95:88	87:80	79:72	71:64	63:56	55:48	47:40	39:32	31:24	23:16	15:8	7:0
QWord	0	LSB	LSB + 1	LSB + 2					LSB + 7	MSB – 7						MSB – 1	MSB

B.3.2 Register Bus Interface

The Control and Status Register are 32-bit wide. Therefore, the DWord or QWord CSR access internally gets converted into two/four 32-bit CSR accesses as shown in Table B-8 on page 1923. The DWord or QWord access through the register bus interface is supported only with the 25GMAC's AXI Subordinate interface.

Note 128-bit Data Bus is not supported in APB3/MCI manager interface configurations.

Table B-8 128-Bit Data Bus: Reference CSR Register for Register Bus Interface

CSR Bits	31:24	23:16	15:8	7:0
First CSR Register	D3	D2	D1	D0
Second CSR Register	D7	D6	D5	D4
Third CSR Register	D11	D10	D9	D8
Fourth CSR Register	D15	D14	D13	D12
Address in LE	A3	A2	A1	A0
	A7	A6	A5	A4
	AB	AA	A9	A8
	AF	AE	AD	AC
Address in CBE or BIBE	A0	A1	A2	A3
	A4	A5	A6	A7
	A8	A9	AA	AB
	AC	AD	AE	AF

B.3.2.0 Little-Endian Data Transfer

Table B-9 on page 1923 provides information about the data transfers in little-endian mode on the 25GMAC's AXI Subordinate interface.

Table B-9 128-Bit Data Bus: Little-Endian Support for Subordinate Interface

Transfer Size	Address Offset A[3:0]	Data															
		127:120	119:112	111:104	103:96	95:88	87:80	79:72	71:64	63:56	55:48	47:40	39:32	31:24	23:16	15:8	7:0
QWord	0	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
DWord	0									D7	D6	D5	D4	D3	D2	D1	D0

Table B-9 128-Bit Data Bus: Little-Endian Support for Subordinate Interface (continued)

Transfer Size	Address Offset A[3:0]	Data															
		127:120	119:112	111:104	103:96	95:88	87:80	79:72	71:64	63:56	55:48	47:40	39:32	31:24	23:16	15:8	7:0
DWord	8	D15	D14	D13	D12	D11	D10	D9	D8								
Word	0													D3	D2	D1	D0
Word	4									D7	D6	D5	D4				
Word	8					D11	D10	D9	D8								
Word	0xC	D15	D14	D13	D12												
HalfWord	0															D1	D0
HalfWord	2													D3	D2		
HalfWord	4											D5	D4				
HalfWord	6									D7	D6						
HalfWord	8							D9	D8								
HalfWord	0xA					D11	D10										
HalfWord	0xC			D13	D12												
HalfWord	0xE	D15	D14														
Byte	0																D0
Byte	1															D1	
Byte	x										...	...	...	...	...		
Byte	7									D7							
Byte	8								D8								
Byte	x			...	...	...	...	...									
Byte	0xE		D14														
Byte	0xF	D15															

In Table B-9 on page 1923, the ellipses (...) represent Dx where x is the byte offset.

B.3.2.0 Byte-Invariant Big-Endian Data Transfer

Table B-10 on page 1925 provides information about the data transfers in byte-invariant big-endian mode on the 25GMAC's AXI Subordinate interface. For a QWord access, Bits[127:96] get transferred to Bits[31:0] of Register 0 (at address offset 0) in the first internal cycle, and Bits[31:0] of the AXI bus get transferred to Bits[31:0] of Register 3 (at address offset 0xC) in the fourth internal cycle.

Table B-10 128-Bit Data Bus: Byte-Invariant Big-Endian Support for Subordinate Interface

Transfer Size	Address Offset A[3:0]	Data															
		127:120	119:112	111:104	103:96	95:88	87:80	79:72	71:64	63:56	55:48	47:40	39:32	31:24	23:16	15:8	7:0
QWord	0	D3	D2	D1	D0	D7	D6	D5	D4	D11	D10	D9	D8	D15	D14	D13	D12
DWord	0									D3	D2	D1	D0	D7	D6	D5	D4
DWord	8	D11	D10	D9	D8	D15	D14	D13	D12								
Word	0													D3	D2	D1	D0
Word	4									D7	D6	D5	D4				
Word	8					D11	D10	D9	D8								
Word	0xC	D15	D14	D13	D12												
HalfWord	0															D3	D2
HalfWord	2													D1	D0		
HalfWord	4											D7	D6				
HalfWord	6									D5	D4						
HalfWord	8							D11	D10								
HalfWord	0xA					D9	D8										
HalfWord	0xC			D15	D14												
HalfWord	0xE	D13	D12														
Byte	0																D3
Byte	1															D2	
Byte	x										...	...	...	...	...		
Byte	7									D4							
Byte	8								D11								

Table B-10 128-Bit Data Bus: Byte-Invariant Big-Endian Support for Subordinate Interface (continued)

Transfer Size	Address Offset A[3:0]	Data															
		127:120	119:112	111:104	103:96	95:88	87:80	79:72	71:64	63:56	55:48	47:40	39:32	31:24	23:16	15:8	7:0
Byte	x			...	...	...	...	...									
Byte	0xE		D13														
Byte	0xF	D12															

In Table B-10 on page 1925, the ellipses (...) represent Dx where x is the byte offset.

C

VxLAN / NVGRE Packet Format

Both VxLAN and NVGRE are overlay encapsulation protocols to support L2 Virtualization over L3 Network.

This appendix contains the following sections:

- "VxLAN Overlay Packet Structure" on page 1928
- "NVGRE Overlay Packet Structure" on page 1932

C.1 VxLAN Overlay Packet Structure

C.1.1 Outer Header Using IPV4

The structure of the VxLAN overlay packet when outer Header is using IPv4.

Figure C-1 Outer Ethernet Header

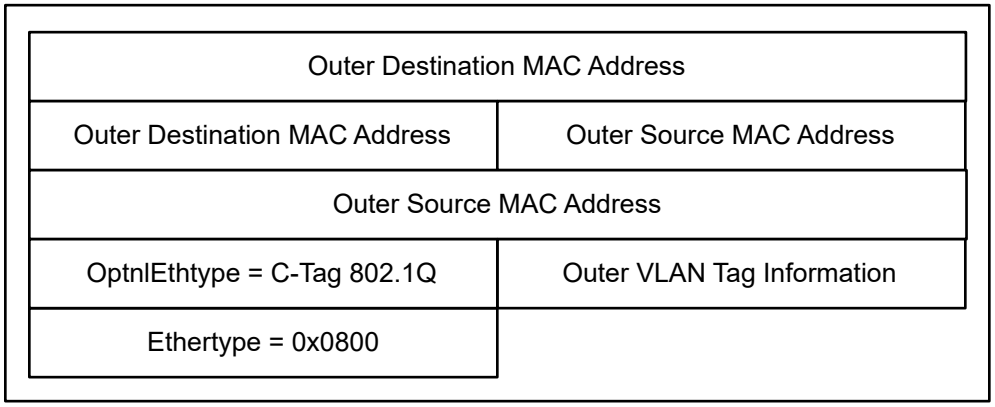


Figure C-2 Outer IPv4 Header

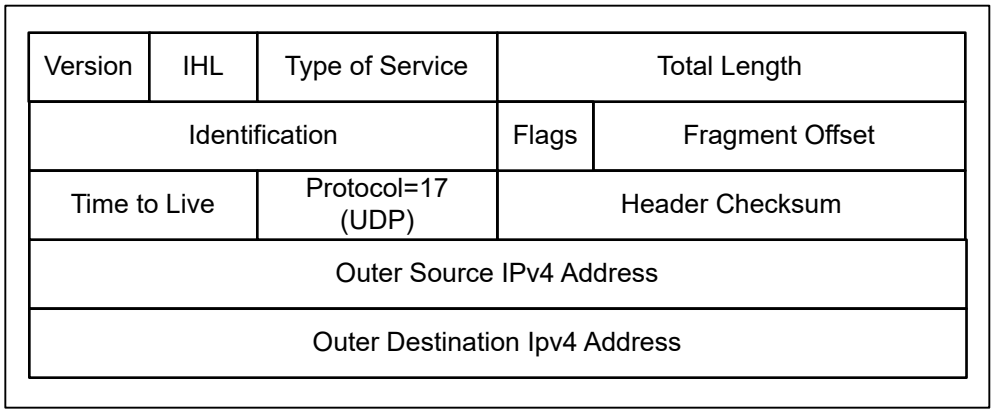


Figure C-3 Outer UDP Header

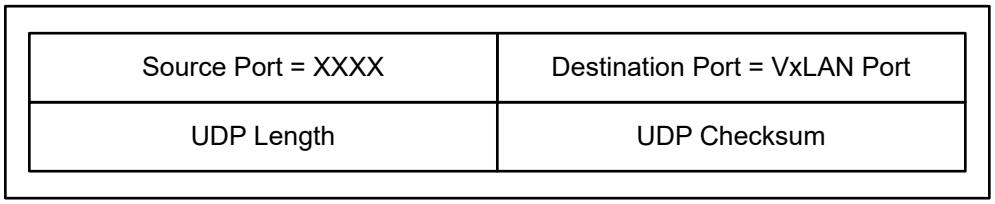


Figure C-4 VxLAN Header

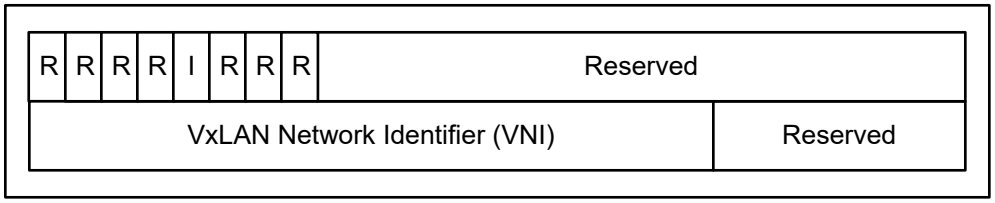


Figure C-5 Inner Ethernet Header

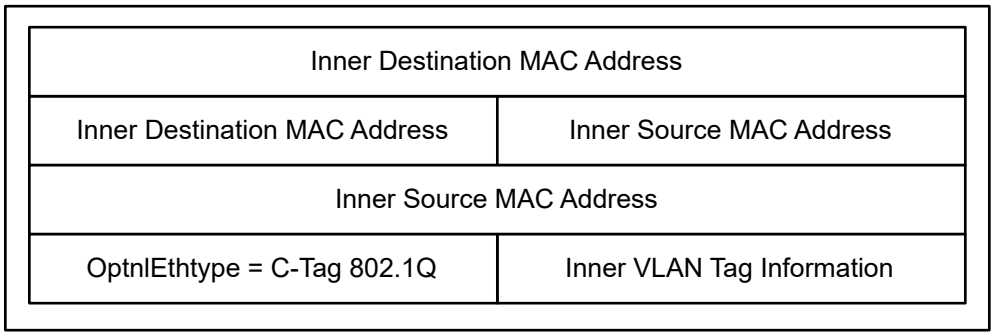


Figure C-6 Payload

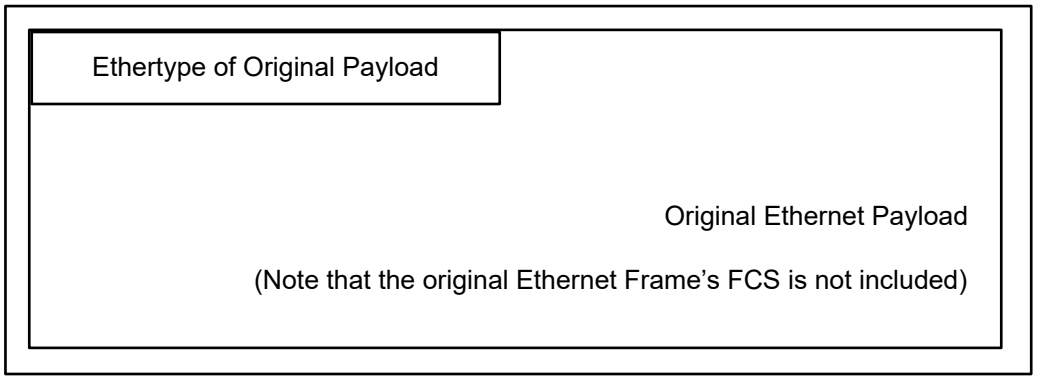
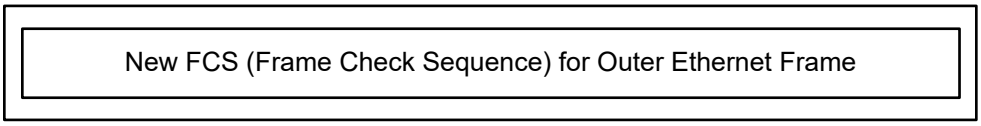


Figure C-7 Frame Check Sequence



Outer Header implies the following headers in the overlay packets:

- Outer Ethernet Header
- Outer IPv4 Header
- Outer UDP Header
- VxLAN Header
- FCS

Inner Header implies the following headers in the overlay packets:

- Inner Ethernet Header
- Inner Ethernet Payload Header (L3-L4)

Note

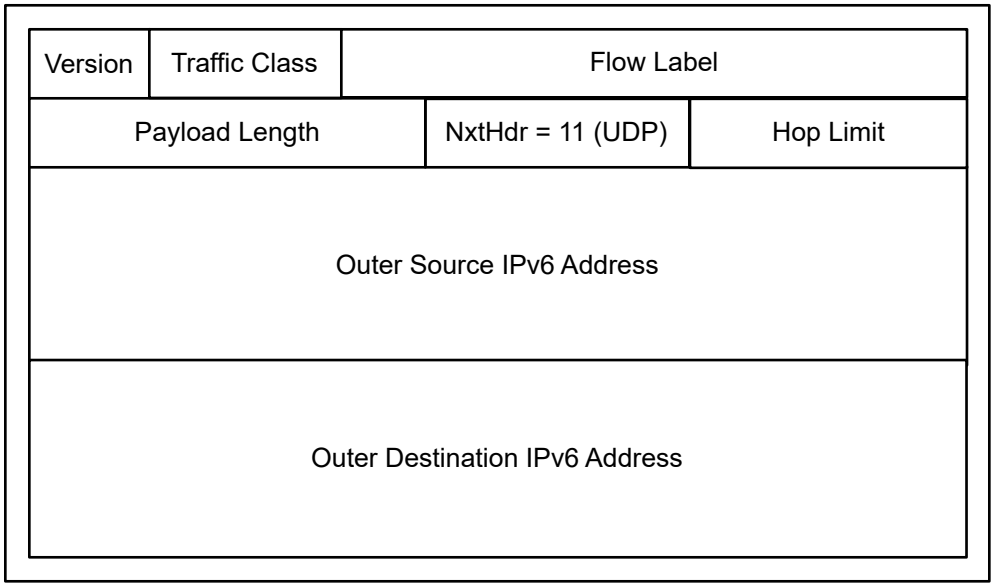
- FCS: Only Outer Header has FCS. This Outer Header FCS is corresponding to the Outer Ethernet Header + its entire Payload. There is no FCS field for the Inner Packet.
- IPv4 Header of the Outer Header must have UDP as its IPv4 Protocol field.
- UDP Header of the Outer Header must have a Unique Destination ID called VxLAN Port ID (16'd4789 or 16'h12B5).
- UDP Checksum of the Outer Header is always set to 0 (To indicate that Checksum is not calculated for this UDP segment).
- UDP of the Outer Header has the following Payload:
  - VxLAN Header
  - Inner Ethernet Header
  - Inner Ethernet Payload
- The VxLAN Network Identifier (VNI) in VxLAN Header field is the one which is used for Identifying a particular Virtual Network Segment within a VxLAN.
- The Inner Header's Ethernet Payload can be IPv4/IPv6/Any Ethernet Type. In case of IPv4/IPv6 it can also have UDP/TCP/ICMPv4/IPCMPv6 payload.

C.1.2 Outer Header Using IPv6

The Overlay structure of the VxLAN when outer Header is using IPv6.

It is same as that of IPv4 except that IPv6 header has different format as shown in Figure C-8 on page 1931. However, similar to IPv4 protocol field, the NxtHdr must have value corresponding to UDP. Also, note that the Outer Header must be only UDP as additional Next Headers is not supported for tunneled packets.

Figure C-8 Outer IPv6 Header



C.2 NVGRE Overlay Packet Structure

This section explains the structure of the NVGRE overlay packet:

C.2.1 Outer NVGRE Header

The following outer header format is same as that of VxLAN

- Outer Ethernet Header
- Outer IPv4/IPv6 Header
- FCS

However, the “Protocol” Field of the Outer Packet’s IPv4/IPv6 has a GRE (Value 8’h2F) instead of UDP. So, UDP Header is replaced with the GRE header:

Note that the GRE Header protocol field has 0x6558 (Transparent Ethernet Bridging). The Virtual Subnet ID (VSID) is the virtual network segment Identifier in a NVGRE. FlowID is the additional differentiator over the VSID whose scope is within a particular VSID.

The Inner Header format is as follows:

- The GRE header is followed by the Inner Ethernet Header.
- The Ethernet Type Field 0x0800/0x86DD (IP Type). (This could be either Tagged or Untagged packet). The Inner Ethernet Payload must be either IPv4 or IPv6
- The Inner IPv4/IPv6 payload can be anything.

Following is the reference NVGRE packet structure, when the Inner and Outer Packet has IPv4:

Figure C-9 GRE Header

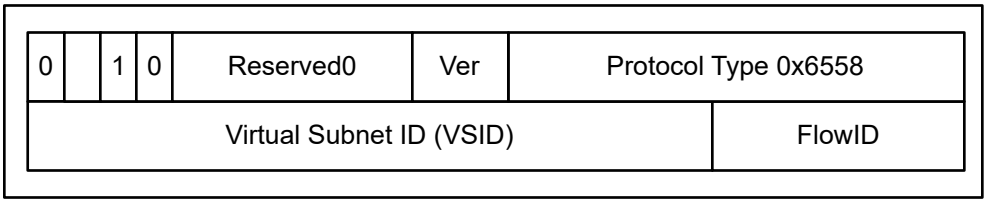


Figure C-10 Outer Ethernet Header

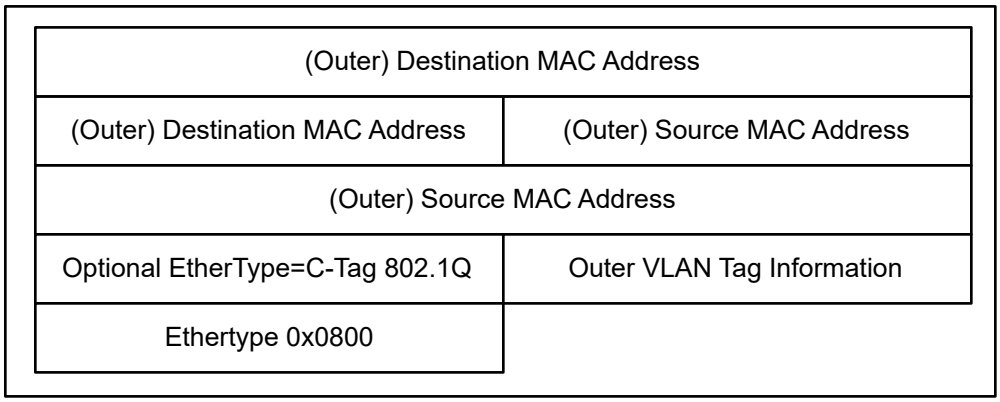


Figure C-11 Outer IPv4 Header

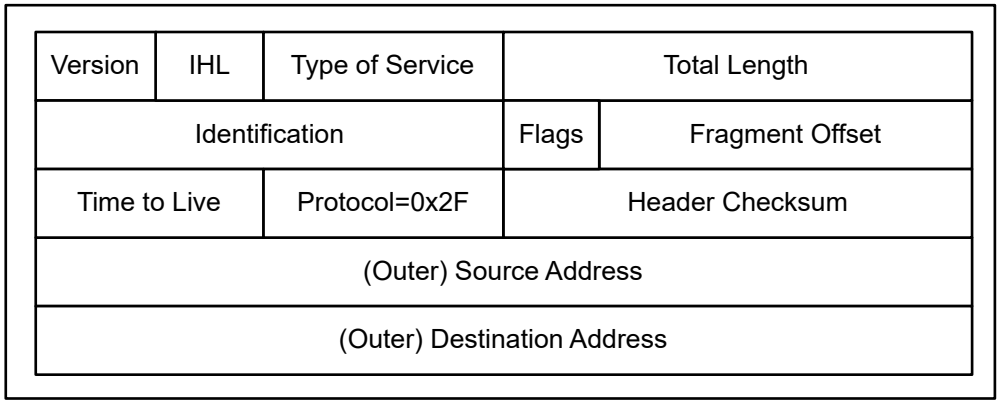


Figure C-12 GRE Header

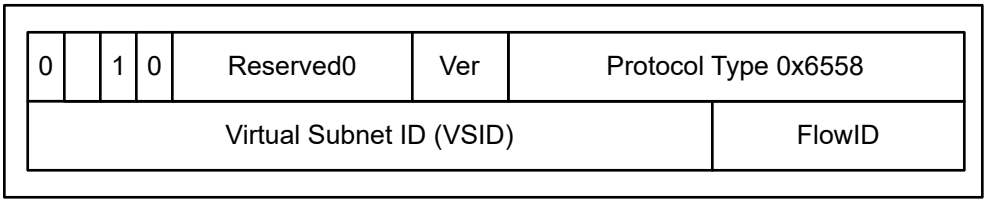


Figure C-13 Inner Ethernet Header

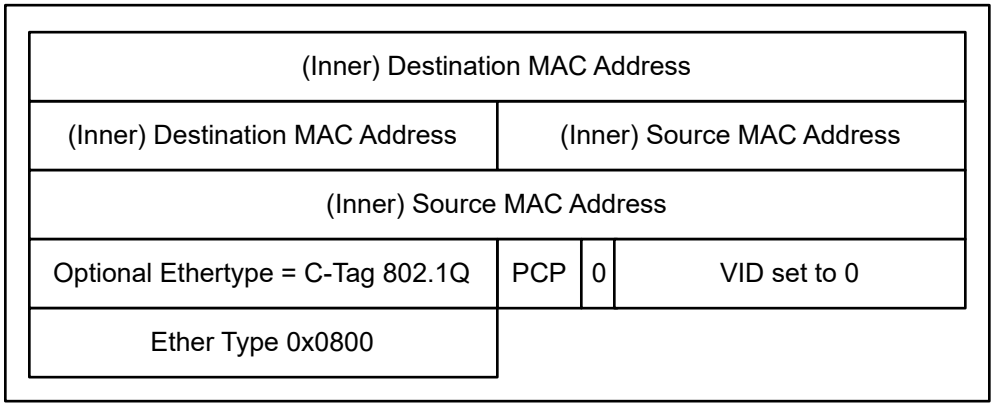


Figure C-14 Inner IPv4 Header

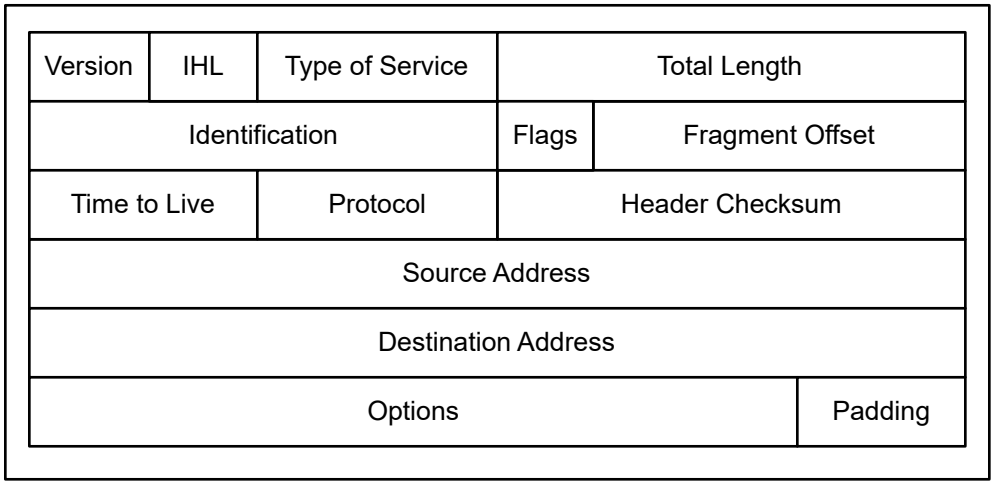


Figure C-15 Payload

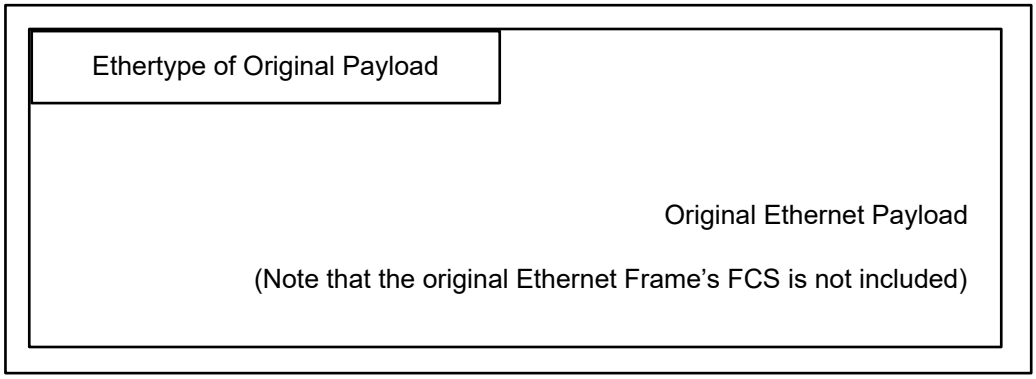
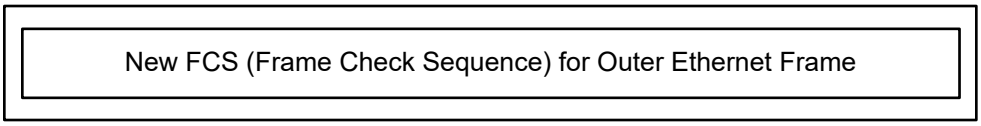


Figure C-16 Frame Check Sequence



Note the following important points:

- FCS: Only Outer Header has FCS. This Outer Packet FCS corresponds to the Outer Ethernet Header + its entire Payload. There is no FCS field for the Inner Packet.
- IPv4 Header of the Outer Header must have GRE as its IPv4 Protocol field.
- The GRE header of the Outer Header must have a Unique Destination ID called Protocol Type (16'h6558).
- The Virtual Subnet ID (VSID) in GRE Header field is used for Identifying a particular Virtual Network Segment within a NVGRE. Also, Flow ID is can be used as an additional differentiator within a particular VSID.

- The GRE of the Outer Header has the following Payload:
  - Inner Ethernet Header
  - Inner IPv4 Header and Payload

This example shows the NVGRE overlay packet format in case of IPv4 Inner and Outer Packets. In case of IPv6 it is same as that of IPv4 with only difference that the IPv4 header must be replaced with IPv6 Header.

D

25GMAC Automotive Safety

This appendix contains the following sections:

- "Overview" on page 1938
- "Automotive Safety Package Documents" on page 1940
- "Configuring the Automotive Safety Features " on page 1941

D.1 Overview

Functional safety is an important consideration throughout the IP development process to make it better suited for applications in the automotive space. ISO 26262 addresses the functional safety and requires analysis of the hardware components based on their ability to detect or detect and control the effects of random hardware faults.

Random hardware faults can occur at any point in time during the life cycle of the hardware component. They can take the form of permanent or transient faults and require safety mechanisms to be in place. Safety mechanisms ensure that severe consequences do not occur as a result of those faults.

The ISO 26262 also defines various systematic requirements to ensure design specifications have been precisely defined, executed, validated, and approved.

The IP includes a broad range of internal safety mechanisms which assist in handling random hardware faults. The internal safety mechanisms monitor the IP logic and report any detected errors to the application. The external safety mechanisms define procedures on the IP to aid in detecting faulty behaviors.

D.1.1 Functional Safety Assessment

The IP is assessed in the scope of ISO 26262, targeting ASIL-B random hardware fault metrics and applicable ASIL-D systematic requirements for a particular reference configuration. The accompanying Safety Manual describes safety mechanisms and assumptions that determine the ASIL-B random hardware fault status.

For more information on automotive safety package documents, see "Automotive Safety Package Documents" on page 1940.

D.1.2 Reference Configurations

The accompanying Safety Manual describes reference configurations supporting the automotive safety features.

D.1.3 Failure Mode, Effects, and Diagnostic Analysis (FMEDA)

Synopsys performs an FMEDA for the reference configurations described in the accompanying Safety Manual. The analysis relies on a per-transistor failure rate derived according to IEC TR 62380. FMEDA, in conjunction with area information from design synthesis trials, determines the failure rate for the IP.

Note Use the relative area of the design modules to distribute the failure rate.

There are defined safety goals for the IP. For FMEDA:

- Synopsys derives a list of failure modes and their effects for each module.

  Each failure mode is assigned a portion of the failure rate of the module to which they belong.
- Synopsys reviews failure modes in the context of the safety goals they can violate, whether the violation is of type single-point or multi-point.

  If there is a diagnostic capability for each failure mode, a Diagnostic Coverage number is assigned.
- Synopsys repeats the process for all failure modes, accumulates the results, and collects IP safety metrics in the FMEDA.

D.1.4 Safety Manual

The safety manual is a user guide to implementing the IP for functional safety. The safety manual contains descriptions of any internal and external safety mechanisms. The safety manual contains Assumptions of Use (AoU), programming requirements, and any other required information to use the IP for functional safety.

D.2 Automotive Safety Package Documents

The IP safety package contains the following documents:

- Quality Manual
- Design FMEA (DFMEA)
- Safety Manual
- Dependent Fault Analysis (DFA)
- Safety Case Report
- Functional Safety Assessment Report

After configuring the IP in coreConsultant, the automotive documents are available in the workspace directory at `<workspace>/automotive`.
Table D-1 describes the documents included in the 25GMAC safety package.

Table D-1 Description of Documents in Automotive Safety Package

File Name	Description	Contents
Certificate_ASIL-B_Random_25GMAC.pdf	Certificate	ASIL-B Random Ready Certificate
Certificate_Report_ASIL-B_Random_25GMAC.pdf	Certification Report	ASIL-B Random Ready Certification report
WKPR41_Safety_Quality_Manual_25GMAC.pdf	Safety Manual	Lists all Synopsys quality process documents; explains IP design flow and tools, verification flows (high-level), and code coverage concepts; describes FMEDA worksheet
SG_Quality_Manual.pdf	Quality Manual	Describes the internal Quality Management System and how its specific activities are related to the Synopsys quality process documents listed in the Safety Manual
WKPR22_FMEDA_25GMAC.xlsx	FMEDA worksheet	Calculates FIT rates for each module; enumerates diagnostic capabilities for failure modes.
WKPR17_DFMEA_25GMAC.xlsx	DFMEA worksheet	Interface DFMEA analysis

D.3 Configuring the Automotive Safety Features

When you have a valid automotive safety package license (DWC-Ethernet-XGMAC-ASP), you can select the safety features supported by 25GMAC, from the Automotive Safety Package pane in the Specify Configuration activity dialog box, while configuring the controller in the coreConsultant GUI.

For details of the various automotive safety features supported by 25GMAC, see "Automotive Safety Features" on page 419.

E

Internal Parameter Descriptions

Provides a description of the internal parameters that might be indirectly referenced in expressions in the Signals, Parameters, or Registers chapters. These parameters are not visible in the coreConsultant GUI and most of them are derived automatically from visible parameters.**You must not set any of these parameters directly.**

Some expressions might refer to TCL functions or procedures (sometimes identified as **function_of**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the core in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

Table E-1 Internal Parameters

Parameter Name	Equals To
DWCXG_ADD_MAC_ADDR_REG_PC	(DWCXG_ADD_MAC_ADDR_REG>63) ? 63 : DWCXG_ADD_MAC_ADDR_REG
DWCXG_ADD_MAC_ADDR_REG_PC_DCNT	[function_of: int(ceil((DWCXG_ADD_MAC_ADDR_REG_PC)/32.0))]
DWCXG_ADDRWIDTH_64	DWCXG_ADDRWIDTH != 32
DWCXG_ADDRWIDTH_MNS32	DWCXG_ADDRWIDTH==48 ? 16 : 8
DWCXG_AIS_ECC_GRAN	DWCXG_AISEG
DWCXG_AIS_EN	(DWCXG_ASP_PPE==2)
DWCXG_APB3_SLAVE	DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4
DWCXG_APB_SLAVE	DWCXG_CSR_PORT == 1||DWCXG_CSR_PORT == 4
DWCXG_AR_DMA_CHW	[function_of: (1+DWCXG_CHW)]
DWCXG_ARI_PBL_WIDTH	(DWCXG==1) ? (DWCXG_RX_FIFO_PTR_WIDTH-1) : 8
DWCXG_ASP_CSR	DWCXG_ASP_ALL
DWCXG_ASP_DPP	DWCXG_ASP_ALL
DWCXG_ASP_FSM	DWCXG_ASP_ALL
DWCXG_ASP_NIB	0
DWCXG_ASP_NIB_EN	DWCXG_ASP_NIB
DWCXG_ASYNC_RAM	0

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_ASYNC_RST	DWCXG_RESET_MODE == 1
DWCXG_ATI_CTL_WIDTH	(DWCXG_DATAWIDTH==32&& DWCXG_SA_VLAN_INS_CTRL_EN&& (DWCXG_OST_EN||DWCXG_TBS||DWCXG_ATS)) ? 2 : (DWCXG_DATAWIDTH==32&& (DWCXG_OST_EN||DWCXG_TBS||DWCXG_ATS)) ? 2 : (DWCXG_DATAWIDTH==32&& DWCXG_SA_VLAN_INS_CTRL_EN) ? 1 : (DWCXG_DATAWIDTH==64&& (DWCXG_TIME_STAMPING)) ? 1 : 1
DWCXG_ATI_PBL_WIDTH	(DWCXG==1) ? (DWCXG_TX_FIFO_PTR_WIDTH-1) : 8
DWCXG_AV_SLOT_EN	DWCXG>1&& DWCXG_TX_AV_ENABLE
DWCXG_AW_DMA_CHW	[function_of: (DWCXG_CHW)]
DWCXG_AXI4_LITE_SLAVE	(DWCXG_CSR_PORT==3)
DWCXG_AXI4_SUBSYS	DWCXG == 2
DWCXG_AXI_BL	256
DWCXG_AXI_GM_DATA_WIDTH	DWCXG_DATAWIDTH
DWCXG_AXI_IDW	DWCXG_AXI_GM_AXI_ID_WIDTH
DWCXG_AXI_SLAVE	(DWCXG_CSR_PORT==2)
DWCXG_AXI_SUBSYS	DWCXG == 2
DWCXG_BEW	(DWCXG_DATAWIDTH==64) ? 3 : (DWCXG_DATAWIDTH==128) ? 4 : 5
DWCXG_BLW	[function_of: (log(DWCXG_AXI_BL)/log(2))]
DWCXG_BN_RCH_BN_RCHL_W	[function_of: max(DWCXG_NUM_RX_CH_EFFW,DWCXG_L2RCHLIST_CNT)]
DWCXG_BSA	[function_of: (log(DWCXG_DATAWIDTH/8)/log(2))]
DWCXG_CACHE_MEM_EN	DWCXG_TSO_MEM_EN||DWCXG_EDMA
DWCXG_CACHE_MEM_SIZE_POW2	[function_of: (DWCXG_CACHE_MEM_SIZE < 64 ? 0 : DWCXG_CACHE_MEM_SIZE == 64 ? 64 : DWCXG_CACHE_MEM_SIZE <= 128 ? 128 : DWCXG_CACHE_MEM_SIZE <= 256 ? 256 : DWCXG_CACHE_MEM_SIZE <= 512 ? 512 : DWCXG_CACHE_MEM_SIZE <= 1024 ? 1024 : DWCXG_CACHE_MEM_SIZE <= 2048 ? 2048 : DWCXG_CACHE_MEM_SIZE <= 4096 ? 4096 : DWCXG_CACHE_MEM_SIZE <= 8192 ? 8192 : DWCXG_CACHE_MEM_SIZE <= 16384 ? 16384 : DWCXG_CACHE_MEM_SIZE <= 32768 ? 32768 : DWCXG_CACHE_MEM_SIZE <=65536 ? 65536 : DWCXG_CACHE_MEM_SIZE <= 131072 ? 131072: DWCXG_CACHE_MEM_SIZE <= 262144 ? 262144: DWCXG_CACHE_MEM_SIZE <= 524288 ? 524288:

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
	DWCXG_CACHE_MEM_SIZE <= 1048576 ?1048576 : 2097152)]
DWCXG_CHW	[function_of: (DWCXG_NUM_DMA_CSR_CH>64 ? 7 : DWCXG_NUM_DMA_CSR_CH>32 ? 6 : DWCXG_NUM_DMA_CSR_CH>16 ? 5 : DWCXG_NUM_DMA_CSR_CH>8 ? 4 : DWCXG_NUM_DMA_CSR_CH>4 ? 3 : ((DWCXG_NUM_DMA_CSR_CH>2) ? 2 : 1))]
DWCXG_CORE	DWCXG == 0
DWCXG_CORE_STSW	DWCXG_CORE ? 96 : 128
DWCXG_CSR_REG_ROWS	9
DWCXG_DATAWIDTH_64	DWCXG_DATAWIDTH == 64
DWCXG_DATW	DWCXG_AXI_GM_DATA_WIDTH
DWCXG_DBGB_APPW	[function_of: ((DWCXG_DBGB_ATI_INTFW) + (DWCXG_DBGB_ARI_INTFW) + (2+2) + (1+2) + (1+2) + (2+2) + (2+2) + (DWCXG_MDC_DAPPW) + (DWCXG_MTL_DAPPW) + (DWCXG_MAC_DAPPW) + 3)]
DWCXG_DBGB_ARI_INTFW	[function_of: ((16*15) + 128 + 7 + (16*73) + 73 + (8) + 1 + 1 + 1 + 1 + (DWCXG_DWM) + 4 + 1 + 1 + 128 + 128 + 1 + 1 + (8))]
DWCXG_DBGB_ATI_INTFW	[function_of: (4 + (2) + 4 + (2) + 1 + 2 + 1 + (DWCXG_DWM) + 4 + 1 + 1 + 1 + 1 + (16*14) + (8) + 1 + 16 + 16 + 16 + 82 + (8))]
DWCXG_DBGB_CSRW	[function_of: ((2+2) + (DWCXG_MDC_DCSRW) + (DWCXG_MTL_DCSRW) + (DWCXG_MAC_DCSRW) + 1)]
DWCXG_DBGB_MRI_INTFW	[function_of: (1 + (DWCXG_DWM) + ((DWCXG_DWM)/64+1+1) + 1 + 1 + 1 + 4 + (1) + 16 + (16) + 1 + 1 + 128 + (8) + 124 + (8))]
DWCXG_DBGB_MTI_INTFW	[function_of: (1 + (DWCXG_DWM) + 1 + 1 + ((DWCXG_DWM)/64+1+1) + 2 + 8 + 1 + 1 + 64 + 1 + 2 + 16 + 2 + 16 + 3 + 1 + 1 + 64 + 2 + 14 + 4 + (8) + 8 + 32 + 1 + (8))]
DWCXG_DBGB_PTPW	[function_of: ((DWCXG_MTL_DPTPW) + (DWCXG_MAC_DPTPW) + 1)]
DWCXG_DBGB_RMIIW	[function_of: ((DWCXG_DRMIIW) + 1)]
DWCXG_DBGB_RXW	[function_of: ((DWCXG_DBGB_MRI_INTFW) + (DWCXG_MTL_DRXW) + (DWCXG_MAC_DRXW) + 2)]
DWCXG_DBGB_TXW	((DWCXG_DBGB_MTI_INTFW) + (DWCXG_MTL_DTXW) + (DWCXG_MAC_DTXW) + 1)
DWCXG_DCACHEAW	DWCXG_DCACHE_MEM_ADDR_WIDTH

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_DCACHE_MEM_EN	(DWCXG_EDMA&& DWCXG_DIFF_CACHE_MEM_EN)||(DWCXG_EDMA&& ! DWCXG_TSO_MEM_EN)
DWCXG_DCACHE_MEM_SIZE_POW2	[function_of: (DWCXG_DCACHE_MEM_SIZE < 64 ? 0 : DWCXG_DCACHE_MEM_SIZE == 64 ? 64 : DWCXG_DCACHE_MEM_SIZE <= 128 ? 128 : DWCXG_DCACHE_MEM_SIZE <= 256 ? 256 : DWCXG_DCACHE_MEM_SIZE <= 512 ? 512 : DWCXG_DCACHE_MEM_SIZE <= 1024 ? 1024 : DWCXG_DCACHE_MEM_SIZE <= 2048 ? 2048 : DWCXG_DCACHE_MEM_SIZE <= 4096 ? 4096 : DWCXG_DCACHE_MEM_SIZE <= 8192 ? 8192 : DWCXG_DCACHE_MEM_SIZE <= 16384 ? 16384 : DWCXG_DCACHE_MEM_SIZE <= 32768 ? 32768 : DWCXG_DCACHE_MEM_SIZE <= 65536 ? 65536 : DWCXG_DCACHE_MEM_SIZE <= 131072 ? 131072 : DWCXG_DCACHE_MEM_SIZE <= 262144 ? 262144 : DWCXG_DCACHE_MEM_SIZE <= 524288 ? 524288 : 1048576)]
DWCXG_DDUP	(DWCXG>1)&& (((DWCXG_NUM_DMA_RX_CH>1)&& ! SNPS_RSVDPARAM_50)||((DWCXG_RCHLIST_CNT>0)&& SNPS_RSVDPARAM_50))
DWCXG_DDUP_IACC_ADDR	DWCXG_DDUP ? 8 : 0
DWCXG_DESC_ADDR_MSB	[function_of: (DWCXG_DATAWIDTH==32 ? DWCXG_ADDRWIDTH-2-1 : DWCXG_DATAWIDTH==64 ? DWCXG_ADDRWIDTH-3-1 : DWCXG_ADDRWIDTH-4-1)]
DWCXG_DFCW	64
DWCXG_DISPRTY_W	DWCXG_PFC_EN ? 8 : 1
DWCXG_DPW	[function_of: ((DWCXG_DATAWIDTH==32) ? 4*((DWCXG_ASP_NIB? 1 : 0)+1) : (DWCXG_DATAWIDTH==64) ? 8*((DWCXG_ASP_NIB? 1 : 0)+1) : (DWCXG_DATAWIDTH==128) ? 16*((DWCXG_ASP_NIB? 1 : 0)+1) : 32*((DWCXG_ASP_NIB? 1 : 0)+1))]
DWCXG_DRMIIW	[function_of: ((2+2) + (8))]
DWCXG_DW	DWCXG_DATAWIDTH
DWCXG_DWM	128
DWCXG_ECC_GRAN_WIDTH_M	((DWCXG_AIS_ECC_GRAN==1)||(DWCXG_DATW==32)) ? 32 : (DWCXG_AIS_ECC_GRAN==2) ? 64 : DWCXG_DATW
DWCXG_ECC_GRAN_WIDTH_S	((DWCXG_AIS_ECC_GRAN==1)||(DWCXG_DATW==32)) ? 32 : (DWCXG_AIS_ECC_GRAN==2) ? 64 : DWCXG_GS_DW

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_ECC_USER_WIDTH_M	[function_of:]
DWCXG_ECC_USER_WIDTH_S	[function_of:]
DWCXG_EDMA	(DWCXG == 2) || SNPS_RSVDPARAM_50
DWCXG_EFF_RXCHW	[function_of: max(DWCXG_NUM_DMA_RX_CHW,DWCXG_RXP_INST_ADDR,DWCXG_L2RCHLIST_CNT)]
DWCXG_EMDW	[function_of: ((DWCXG_GCL_WID)+(DWCXG_NUM_TC))]
DWCXG_EST_ECW	(DWCXG_AV_EST&& DWCXG_ASP_ECC)==1 ? (((DWCXG_EMDW+DWCXG_IMAW+1) <= 26) ? 6 : 7) : 0
DWCXG_EXT_DAH_EN	DWCXG_DA_HASH_EN&& (DWCXG_HASH_TABLE>256)
DWCXG_EXT_DAH_ENT_DCNT	[function_of: int(ceil((DWCXG_RXF_DAH_DW)/32.0))]
DWCXG_EXT_DAH_TBL_DCNT	[function_of: (DWCXG_RXF_DAH_SZ*DWCXG_EXT_DAH_ENT_DCNT)]
DWCXG_EXT_RIVTL_DCNT	[function_of: (DWCXG_RXF_RIVTL_SZ*DWCXG_EXT_RVTL_ENT_DCNT)]
DWCXG_EXT_RIVTL_EN	(DWCXG_EXT_RVTL_EN&& DWCXG_DOUBLE_VLAN_EN)
DWCXG_EXT_ROVTL_DCNT	[function_of: (DWCXG_RXF_ROVTL_SZ*DWCXG_EXT_RVTL_ENT_DCNT)]
DWCXG_EXT_ROVTL_EN	DWCXG_EXT_RVTL_EN
DWCXG_EXT_RSSL_EN	(DWCXG_RSS_TABLE_SIZE>256)
DWCXG_EXT_RVTL_EN	DWCXG_ERVFE&& (DWCXG_NRVF==4096)
DWCXG_EXT_RVTL_ENT_DCNT	[function_of: int(ceil((DWCXG_RXF_RVTL_DW)/32.0))]
DWCXG_FLCW	(DWCXG_RXFIFO_SIZE<=4096) ? 3 : (DWCXG_RXFIFO_SIZE<=8192) ? 4 : (DWCXG_RXFIFO_SIZE<=16384) ? 5 : 6
DWCXG_FRP_NUM_AC	[function_of: ((DWCXG_NUM_RX_CH_EFF<=16)?DWCXG_NUM_RX_CH_EFF:16)]
DWCXG_GCL_QDEP	[function_of: (DWCXG_AV_EST&& DWCXG_SGF ? (DWCXG_GCL_TDEP)*2 : DWCXG_GCL_TDEP)]
DWCXG_GCL_TDEP	[function_of: ((DWCXG_GCL_DEP)*2)]
DWCXG_GCL_WID	24
DWCXG_GMAW	[function_of: (log(DWCXG_GCL_QDEP)/log(2))]
DWCXG_GMDW	(DWCXG_AV_EST&& DWCXG_SGF ? ((DWCXG_EMDW > DWCXG_PMDW) ? DWCXG_EMDW : DWCXG_PMDW) : (DWCXG_AV_EST ?

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
	DWCXG_EMDW : (DWCXG_SGF ? DWCXG_PMDW : 25)))
DWCXG_GMII_DATA_WIDTH	8
DWCXG_GMII_SUPPORT	DWCXG_XGMII_GMII == 1
DWCXG_GS_AW	DWCXG_ADDRWIDTH
DWCXG_GS_BW	DWCXG_AXI4_LITE_SLAVE ? 4 : DWCXG_BLW
DWCXG_GS_DW	DWCXG_CSR_DATAWIDTH
DWCXG_GS_DWB	DWCXG_GS_DW/8
DWCXG_GS_PW	DWCXG_ASP_NIB ? (DWCXG_GS_DW/4) : (DWCXG_GS_DW/8)
DWCXG_HDMA_MEM_DBG_ACCESS	SNPS_RSVDPARAM_50&& DWCXG_DBG_FIFO_ACCESS
DWCXG_IMAW	[function_of: (log(DWCXG_GCL_DEP)/log(2))]
DWCXG_INT_DAH_EN	DWCXG_DA_HASH_EN&& (DWCXG_HASH_TABLE>0)&& (DWCXG_HASH_TABLE<=256)
DWCXG_INT_RVTL_EN	DWCXG_ERVFE&& (DWCXG_NRVF>1)&& (DWCXG_NRVF<4096)
DWCXG_L256_RXMS	DWCXG_RXFIFO_SIZE <= 512 ? 1 : DWCXG_RXFIFO_SIZE <= 1024 ? 2 : DWCXG_RXFIFO_SIZE <= 2048 ? 3 : DWCXG_RXFIFO_SIZE <= 4096 ? 4 : DWCXG_RXFIFO_SIZE <= 8192 ? 5 : DWCXG_RXFIFO_SIZE <= 16384 ? 6 : DWCXG_RXFIFO_SIZE <= 32768 ? 7 : DWCXG_RXFIFO_SIZE <= 65536 ? 8 : DWCXG_RXFIFO_SIZE <= 131072 ? 9 : 10
DWCXG_L256_TXMS	DWCXG_TXFIFO_SIZE <= 512 ? 1 : DWCXG_TXFIFO_SIZE <= 1024 ? 2 : DWCXG_TXFIFO_SIZE <= 2048 ? 3 : DWCXG_TXFIFO_SIZE <= 4096 ? 4 : DWCXG_TXFIFO_SIZE <= 8192 ? 5 : DWCXG_TXFIFO_SIZE <= 16384 ? 6 : DWCXG_TXFIFO_SIZE <= 32768 ? 7 : DWCXG_TXFIFO_SIZE <= 65536 ? 8 : DWCXG_TXFIFO_SIZE <= 131072 ? 9 : 10
DWCXG_L2EXT_DAH_TBL_DCNT	DWCXG_EXT_DAH_EN ? ((DWCXG_EXT_DAH_TBL_DCNT>0) ? [function_of: ceil(log(DWCXG_EXT_DAH_TBL_DCNT)/log(2))] : 0) : 0
DWCXG_L2EXT_RIVTL_DCNT	DWCXG_EXT_RIVTL_EN ? ((DWCXG_EXT_RIVTL_DCNT>0) ? [function_of: ceil(log(DWCXG_EXT_RIVTL_DCNT)/log(2))] : 0) : 0

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_L2EXT_ROVTL_DCNT	DWCXG_EXT_ROVTL_EN ? ((DWCXG_EXT_ROVTL_DCNT>0) ? [function_of: ceil(log(DWCXG_EXT_ROVTL_DCNT)/log(2))] : 0) : 0
DWCXG_L2RCHLIST_ARRAY_DCNT	(DWCXG_RCHLIST_ARRAY_DCNT>0) ? [function_of: ceil(log(DWCXG_RCHLIST_ARRAY_DCNT)/log(2))] : 0
DWCXG_L2RCHLIST_CNT	(DWCXG_RCHLIST_CNT>0) ? [function_of: ceil(log(DWCXG_RCHLIST_CNT)/log(2))] : 0
DWCXG_L2RXP_INPR	(DWCXG_FRP_PIPE>3) ? 3 : (DWCXG_FRP_PIPE>1) ? 2 : 1
DWCXG_L3_L4_ADDR	[function_of: ((DWCXG_L3_L4_FILTER_NUM > 0)?(4 + ((DWCXG_L3_L4_FILTER_NUM > 32)?6:(DWCXG_L3_L4_FILTER_NUM > 16)?5:(DWCXG_L3_L4_FILTER_NUM > 8)? (log(DWCXG_L3_L4_FILTER_NUM)/log(2)):(DWCXG_L3_L4_FILTER_NUM > 4)?3:(DWCXG_L3_L4_FILTER_NUM > 2)?2:1)): 8)]
DWCXG_L3_L4_FILTER_NUM_PC	(DWCXG_L3_L4_FILTER_NUM>64) ? 64 : DWCXG_L3_L4_FILTER_NUM
DWCXG_L3_L4_FILTER_NUM_PC_DCNT	[function_of: int(ceil((DWCXG_L3_L4_FILTER_NUM_PC)/32.0))]
DWCXG_LOG2_NUM_PC	(DWCXG_NUM_PC>2) ? [function_of: (log(DWCXG_NUM_PC)/log(2))] : 1
DWCXG_MA0IE_DFLT	0x0
DWCXG_MAC_DAPPW	[function_of: ((3+2) + (3+2) + (2+2) + (2+2) + (3+2) + 32 + 1 + (8))]
DWCXG_MAC_DCSRW	[function_of: ((3+2) + (3+2) + (2+2) + (2+2) + (3+2) + 32 + 1 + (8))]
DWCXG_MAC_DPTPW	[function_of: (((4*3)+4) + (8))]
DWCXG_MAC_DRXW	[function_of: ((3+2) + 1 + 1 + 1 + 1 + 1 + 8 + 1 + 1 + 1 + (DWCXG_NUM_DMA_RX_CH>16?DWCXG_RX_CHW:4) + (2) + 1 + (3+2) + (3+2) + (3+2) + 1 + 1 + (3+2) + (8))]
DWCXG_MAC_DTXW	[function_of: ((3+2) + 1 + (4+2) + (2+2) + ((2*8)+8) + (2+2) + (3+2) + (2+2) + 1 + (4+2) + 1 + (8))]
DWCXG_MAC_IACC_ADDR	[function_of: max(DWCXG_DDUP_IACC_ADDR,DWCXG_PC_IACC_ADDR,DWCXG_L2EXT_DAH_TBL_DCNT,DWCXG_L2EXT_ROVTL_DCNT,DWCXG_L2EXT_RIVTL_DCNT)]
DWCXG_M_AR_PYLD0_USER_WIDTH	4
DWCXG_M_AR_PYLD1_USER_WIDTH	5
DWCXG_M_AR_PYLD_USER_WIDTH	[function_of: (DWCXG_M_AR_PYLD0_USER_WIDTH + DWCXG_M_AR_PYLD1_USER_WIDTH)]
DWCXG_M_AW_PYLD0_USER_WIDTH	4

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_M_AW_PYLD1_USER_WIDTH	5
DWCXG_M_AW_PYLD_USER_WIDTH	[function_of: (DWCXG_M_AW_PYLD0_USER_WIDTH + DWCXG_M_AW_PYLD1_USER_WIDTH)]
DWCXG_M_B_PYLD0_USER_WIDTH	2
DWCXG_M_B_PYLD_USER_WIDTH	DWCXG_M_B_PYLD0_USER_WIDTH
DWCXG_MCGR	DWCXG_FLEXI_PPS_OUT_EN&& DWCXG_TIME_STAMPING==1&& DWCXG_SYSTIME_SOURCE!=1
DWCXG_MCGR_NUM	DWCXG_FLEXI_PPS_OUT_EN ? DWCXG_PPS_OUT_NUM : 1
DWCXG_MCI_ADDR_WIDTH	[function_of: ((SNPS_RSVDPARAM_44) ? 16 : SNPS_RSVDPARAM_50 ? 15 : 14)]
DWCXG_MCI_SLAVE	DWCXG_CSR_PORT == 0
DWCXG_MDC_CSRW	32
DWCXG_MDC_DAPPW	[function_of: (2 + 1 + 1 + 1 + 1 + 1 + (4+2) + (2+2) + (3+2) + (3+2) + (DWCXG_MDC_CSRW) + (DWCXG_MDC_CSRW) + (DWCXG_MDC_CSRW) + (DWCXG_MDC_CSRW) + ((16*3) + 16) + ((16*9) + 16) + ((16*3) + 16) + ((16*3) + 16) + ((16*9) + 16) + ((16*3) + 16) + (8))]
DWCXG_MDC_DCSRW	[function_of: ((DWCXG_MDC_CSRW) + (DWCXG_MDC_CSRW) + (DWCXG_MDC_CSRW) + (DWCXG_MDC_CSRW) + (8))]
DWCXG_MEMW	(DWCXG_DATAWIDTH==64) ? 68 : (DWCXG_DATAWIDTH==128) ? 133 : 262
DWCXG_MII_SUPPORT	(DWCXG_XGMII_GMII==1)
DWCXG_MMC_HD_EN	DWCXG_MMC_EN&& !DWCXG_FDUPLX_ONLY
DWCXG_MMC_IPC_RX_DGRAM_BCNT_ATLEAST_ONE	DWCXG_MMC_RX_IPV4_GD_OCTET_CNT_EN||DWCXG_MMC_RX_IPV4_HDRERR_OCTET_CNT_EN||DWCXG_MMC_RX_IPV4_NOPAY_OCTET_CNT_EN||DWCXG_MMC_RX_IPV4_FRAG_OCTET_CNT_EN||DWCXG_MMC_RX_IPV6_GD_OCTET_CNT_EN||DWCXG_MMC_RX_IPV6_HDRERR_OCTET_CNT_EN||DWCXG_MMC_RX_IPV6_NOPAY_OCTET_CNT_EN
DWCXG_MMC_IPC_RX_EN_ATLEAST_ONE	DWCXG_MMC_IPC_RX_PKTS_ATLEAST_ONE||DWCXG_MMC_IPC_RX_DGRAM_BCNT_ATLEAST_ONE||DWCXG_MMC_IPC_RX_PAYLD_BCNT_ATLEAST_ONE
DWCXG_MMC_IPC_RX_PAYLD_BCNT_ATLEAST_ONE	DWCXG_MMC_RX_UDP_GD_OCTET_CNT_EN||DWCXG_MMC_RX_IPV4_UDSBL_OCTET_CNT_EN||DWCXG_MMC_RX_UDP_ERR_OCTET_CNT_EN||DWCXG_MMC_RX_TCP_GD_OCTET_CNT_EN||DWCXG_MMC_RX_TCP_ERR_OCTET_CNT_EN||DWCXG_MMC_RX_ICMP_GD_OCTET_CNT_EN||DWCXG_MMC_RX_ICMP_ERR_OCTET_CNT_EN

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_MMC_IPC_RX_PKTS_ATLEAST_ONE	DWCXG_MMC_RX_IPV4_GD_PKTS_CNT_EN||DWCXG_MMC_RX_IPV4_HDRERR_PKTS_CNT_EN||DWCXG_MMC_RX_IPV4_NOPAY_PKTS_CNT_EN||DWCXG_MMC_RX_IPV4_FRAG_PKTS_CNT_EN||DWCXG_MMC_RX_IPV4_UDSBL_PKTS_CNT_EN||DWCXG_MMC_RX_IPV6_GD_PKTS_CNT_EN||DWCXG_MMC_RX_IPV6_HDRERR_PKTS_CNT_EN||DWCXG_MMC_RX_IPV6_NOPAY_PKTS_CNT_EN||DWCXG_MMC_RX_UDP_GD_PKTS_CNT_EN||DWCXG_MMC_RX_UDP_ERR_PKTS_CNT_EN||DWCXG_MMC_RX_TCP_GD_PKTS_CNT_EN||DWCXG_MMC_RX_TCP_ERR_PKTS_CNT_EN||DWCXG_MMC_RX_ICMP_GD_PKTS_CNT_EN||DWCXG_MMC_RX_ICMP_ERR_PKTS_CNT_EN
DWCXG_MMC_RX_EN_ATLEAST_ONE	(DWCXG_MMC_EN_RX_ALL||DWCXG_MMC_RXFRMGB_CNT_EN||DWCXG_MMC_RXOCTG_CNT_EN||DWCXG_MMC_RXOCTGB_CNT_EN||DWCXG_MMC_RXBCASTG_CNT_EN||DWCXG_MMC_RXMCASTG_CNT_EN||DWCXG_MMC_RXCRCERR_CNT_EN||DWCXG_MMC_RXRUNTERR_CNT_EN||DWCXG_MMC_RXJABERR_CNT_EN||DWCXG_MMC_RXUNDERSZG_CNT_EN||DWCXG_MMC_RXOVERSZG_CNT_EN||DWCXG_MMC_RXADDR_RNG_CNT_EN||DWCXG_MMC_RXUCASTG_CNT_EN||DWCXG_MMC_RXLENERR_CNT_EN||DWCXG_MMC_RXOUTOFRNG_TYP_CNT_EN||DWCXG_MMC_RXPAUSEFRM_CNT_EN || DWCXG_MMC_RXFIFOOVFL_CNT_EN||DWCXG_MMC_RXVLANFRGB_CNT_EN||DWCXG_MMC_RXWDGERR_CNT_EN||DWCXG_MMC_RXFRMDISGB_CNT_EN||DWCXG_MMC_RXOCTDISGB_CNT_EN||DWCXG_MMC_RXLPI_USEC_EN||DWCXG_MMC_RXLPI_TRANS_CNTR_EN||DWCXG_MMC_RXALGNERR_CNT_EN)
DWCXG_MMC_TX_EN_ATLEAST_ONE	DWCXG_MMC_EN_TX_ALL||DWCXG_MMC_HD_EN||DWCXG_MMC_TXADDR_RNG_CNT_EN||DWCXG_MMC_TXOCTGB_CNT_EN||DWCXG_MMC_TXFRMGB_CNT_EN||DWCXG_MMC_TXBCASTG_CNT_EN||DWCXG_MMC_TXMCASTG_CNT_EN||DWCXG_MMC_TXUCASTGB_CNT_EN||DWCXG_MMC_TXMCASTGB_CNT_EN||DWCXG_MMC_TXBCASTGB_CNT_EN||DWCXG_MMC_TXUNDRFLW_CNT_EN||DWCXG_MMC_TXOCTG_CNT_EN||DWCXG_MMC_TXFRMG_CNT_EN||DWCXG_MMC_TXPAUSE_CNT_EN||DWCXG_MMC_TXVLAN_CNT_EN||DWCXG_MMC_TXLPI_USEC_EN||DWCXG_MMC_TXLPI_TRANS_CNTR_EN
DWCXG_M_R_PYLD0_USER_WIDTH	[function_of: (DWCXG_NO_ECC_CHECKER_M*DWCXG_ECC_USER_WIDTH_M)]
DWCXG_M_R_PYLD1_USER_WIDTH	2

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_M_R_PYLD_USER_WIDTH	[function_of: (DWCXG_M_R_PYLD0_USER_WIDTH + DWCXG_M_R_PYLD1_USER_WIDTH)]
DWCXG_MTL_DAPPW	[function_of: ((3+2) + 1 + 1 + 4 + 1 + (2+2) + 1 + 1 + 1 + 1 + (3+2) + 1 + 1 + 1 + 1 + 1 + 1 + ((4*16)+16) + (16) + ((16*16)+16) + 16 + 16 + ((3*16)+16) + ((16*4)+16) + 1 + 16 + + (3+2) + 1 + (8))]
DWCXG_MTL_DCSRW	[function_of: (32 + (16*32) + (16*32) + (8))]
DWCXG_MTL_DPTPW	[function_of: ((3+2) + (2+2) + (8))]
DWCXG_MTL_DRXW	[function_of: ((3+2) + 1 + 1 + 1 + 1 + 1 + 1 + 1 + (8))]
DWCXG_MTL_DTXW	[function_of: ((3+2) + 1 + 1 + (4) + 1 + (2+1) + 1 + 1 + 1 + 1 + (8))]
DWCXG_MTL_IACC_ADDR	[function_of: max(DWCXG_MTL_RCLK_IACC_AW,DWCXG_MTL_RRC_IACC_ADDR)]
DWCXG_MTL_RCLK_IACC_AW	[function_of: max(DWCXG_RXP_IACC_ADDR,DWCXG_RMMC_RCHM_RCLK_IACC_AW)]
DWCXG_MTL_RCLK_IACC_EN	DWCXG_RXP||DWCXG_RMMC_RCHM_RCLK_IACC_EN
DWCXG_MTL_RRC_IACC_ADDR	[function_of: max(DWCXG_L2RCHLIST_ARRAY_DCNT,DWCXG_RMMC_RCHM_ACLK_IACC_AW)]
DWCXG_MTL_RRC_IACC_EN	(DWCXG_RCHLIST_CNT>0)||DWCXG_RMMC_RCHM_ACLK_IACC_EN
DWCXG_MTL_SUBSYS	DWCXG == 1
DWCXG_M_W_PYLD0_USER_WIDTH	[function_of: (DWCXG_NO_ECC_CHECKER_M*DWCXG_ECC_USER_WIDTH_M)]
DWCXG_M_W_PYLD1_USER_WIDTH	2
DWCXG_M_W_PYLD_USER_WIDTH	[function_of: (DWCXG_M_W_PYLD0_USER_WIDTH + DWCXG_M_W_PYLD1_USER_WIDTH)]
DWCXG_NO_ECC_CHECKER_M	[function_of: (DWCXG_DATW/DWCXG_ECC_GRAN_WIDTH_M)]
DWCXG_NO_ECC_CHECKER_S	[function_of: (DWCXG_GS_DW/DWCXG_ECC_GRAN_WIDTH_S)]
DWCXG_NRVF_PC	(DWCXG_NRVF>64) ? 64 : DWCXG_NRVF
DWCXG_NRVF_PC_DCNT	[function_of: int(ceil((DWCXG_NRVF_PC)/32.0))]
DWCXG_NRXCH	DWCXG_NUM_DMA_RX_CH
DWCXG_NTXCH	DWCXG_NUM_DMA_TX_CH
DWCXG_NTXFC	[function_of:]
DWCXG_NUM_CSR_PDMA	[function_of: (DWCXG_NUM_TXQ > DWCXG_NUM_RXQ ? DWCXG_NUM_TXQ : DWCXG_NUM_RXQ)]

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_NUM_DMA_CSR_CH	[function_of: (DWCXG_NUM_DMA_TX_CH > DWCXG_NUM_DMA_RX_CH ? DWCXG_NUM_DMA_TX_CH : DWCXG_NUM_DMA_RX_CH)]
DWCXG_NUM_DMA_RX_CHW	DWCXG_NUM_DMA_RX_CH<=2 ? 1 : DWCXG_NUM_DMA_RX_CH<=4 ? 2 : DWCXG_NUM_DMA_RX_CH<=8 ? 3 : DWCXG_NUM_DMA_RX_CH<=16 ? 4 : DWCXG_NUM_DMA_RX_CH<=32 ? 5 : DWCXG_NUM_DMA_RX_CH<=64 ? 6 : 7
DWCXG_NUM_DMA_TX_CHW	DWCXG_NUM_DMA_TX_CH<=2 ? 1 : DWCXG_NUM_DMA_TX_CH<=4 ? 2 : DWCXG_NUM_DMA_TX_CH<=8 ? 3 : DWCXG_NUM_DMA_TX_CH<=16 ? 4 : DWCXG_NUM_DMA_TX_CH<=32 ? 5 : DWCXG_NUM_DMA_TX_CH<=64 ? 6 : 7
DWCXG_NUM_GID	8
DWCXG_NUM_RX_CH_EFF	(DWCXG_MTL_SUBSYS) ? DWCXG_NUM_RXQ : DWCXG_NUM_DMA_RX_CH
DWCXG_NUM_RX_CH_EFFW	(DWCXG_NUM_RX_CH_EFF>0)?((DWCXG_NUM_RX_CH_EFF<=2)?1:(DWCXG_NUM_RX_CH_EFF<=4)?2: (DWCXG_NUM_RX_CH_EFF<=8)?3: (DWCXG_NUM_RX_CH_EFF<=16)?4: (DWCXG_NUM_RX_CH_EFF<=32)?5: (DWCXG_NUM_RX_CH_EFF<=64)?6:7):0
DWCXG_OOS_EN	DWCXG_RSS_EN&& DWCXG_SPLIT_HDR_EN
DWCXG_PCHRDC_PW	[function_of: (log(DWCXG_PCHRDC_SZ)/log(2))]
DWCXG_PCHTDC_PW	[function_of: (log(DWCXG_PCHTDC_SZ)/log(2))]
DWCXG_PC_IACC_ADDR	DWCXG_PC ? 8 : 0
DWCXG_PMDW	[function_of: ((DWCXG_GCL_WID)+(DWCXG_NUM_GID))]
DWCXG_PPSOW	[function_of: ((DWCXG_FLEXI_PPS_OUT_EN) ? DWCXG_PPS_OUT_NUM : 1)]
DWCXG_PSF	DWCXG_SGF
DWCXG_RCHLIST_ARRAY_DCNT	(DWCXG_RCHLIST_CNT>0)?[function_of: (DWCXG_RCHLIST_CNT*16)]:0
DWCXG_RCHLIST_DCNT	[function_of: int(ceil((DWCXG_NUM_RX_CH_EFF)/32.0))]
DWCXG_RINGLEN_16B	1
DWCXG_RING_LENGTHW	DWCXG_RINGLEN_16B?16:12
DWCXG_RMMC_IPC_RCHM_SKP_CNTR_EN	DWCXG_RMMC_RCH_MAP&& DWCXG_MMC_IPC_RX_EN_ATLEAST_ONE
DWCXG_RMMC_RCHM_ACLK_IACC_AW	DWCXG_RMMC_RCHM_ACLK_IACC_EN?DWCXG_RMMC_RCHM_IACC_AW:0

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_RMMC_RCHM_ACLK_IACC_EN	DWCXG_RMMC_RCH_MAP&& (DWCXG_RX_SRAM||((!DWCXG_RX_SRAM)&& (DWCXG_RCHLIST_CNT>0)))
DWCXG_RMMC_RCHM_IACC_AW	DWCXG_RMMC_RCH_MAP?((DWCXG_RCHLIST_DCNT >1)?[function_of: ceil(log(DWCXG_RCHLIST_DCNT)/log(2))]:1):0
DWCXG_RMMC_RCHM_RCLK_IACC_AW	DWCXG_RMMC_RCHM_RCLK_IACC_EN?DWCXG_RM MC_RCHM_IACC_AW:0
DWCXG_RMMC_RCHM_RCLK_IACC_EN	DWCXG_RMMC_RCH_MAP&& (!DWCXG_RMMC_RCHM_ACLK_IACC_EN)
DWCXG_RMMC_RCHM_SKP_CNTR_EN	DWCXG_RMMC_RCH_MAP&& (DWCXG_MMC_RX_EN_ATLEAST_ONE||DWCXG_MMC _PRIORITY_EN)
DWCXG_ROSRLW	[function_of: (log(DWCXG_AXI_GM_MAX_RD_REQUESTS)/log(2))]
DWCXG_RSS_TABLE_SIZE_W	(DWCXG_RSS_TABLE_SIZE>0) ? [function_of: (log(DWCXG_RSS_TABLE_SIZE)/log(2))] : 1
DWCXG_RXCHCNT	((DWCXG_NUM_DMA_RX_CH<=16)?[function_of: (DWCXG_NUM_DMA_RX_CH-1)]:((DWCXG_NUM_DMA _RX_CH==20)?16: (DWCXG_NUM_DMA_RX_CH==24)?17: (DWCXG_NUM_DMA_RX_CH==28)? 18 :(DWCXG_NUM_DMA_RX_CH==32)?19: (DWCXG_NUM_DMA_RX_CH==40)?20: (DWCXG_NUM_DMA_RX_CH==48)?21: (DWCXG_NUM_DMA_RX_CH==56)?22: (DWCXG_NUM_DMA_RX_CH==64)?23: (DWCXG_NUM_DMA_RX_CH==80)?24 : (DWCXG_NUM_DMA_RX_CH==96)?25: (DWCXG_NUM_DMA_RX_CH==112)?26: (DWCXG_NUM_DMA_RX_CH==128)?27:31))
DWCXG_RX_CHW	DWCXG_NUM_DMA_RX_CHW
DWCXG_RXF_DAH_AF_SB	0
DWCXG_RXF_DAH_AF_SZ	1
DWCXG_RXF_DAH_DW	DWCXG_EXT_DAH_EN ? [function_of: (DWCXG_RXF_DAH_RCH_SB + DWCXG_RXF_DAH_RCH_SZ)] : 0
DWCXG_RXF_DAH_RCHLE_SB	[function_of: (DWCXG_RXF_DAH_AF_SB + DWCXG_RXF_DAH_AF_SZ)]
DWCXG_RXF_DAH_RCHLE_SZ	(DWCXG_RCHLIST_CNT>0) ? 1 : 0
DWCXG_RXF_DAH_RCH_SB	[function_of: (DWCXG_RXF_DAH_RCHLE_SB + DWCXG_RXF_DAH_RCHLE_SZ)]
DWCXG_RXF_DAH_RCH_SZ	DWCXG_BN_RCH_BN_RCHL_W
DWCXG_RXF_DAH_SZ	(DWCXG_EXT_DAH_EN ? DWCXG_HASH_TABLE : 0)

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_RXF_MEM_EDW	[function_of: (DWCXG_RXF_MEM_DW+DWCXG_RXF_MEM_AW)]
DWCXG_RXF_MEM_ENT	[function_of: (DWCXG_RXF_DAH_SZ + DWCXG_RXF_ROVTL_SZ + DWCXG_RXF_RIVTL_SZ + DWCXG_RXF_RSSL_SZ)]
DWCXG_RXF_RIVTL_SZ	(DWCXG_EXT_RIVTL_EN ? DWCXG_NRVF : 0)
DWCXG_RXF_ROVTL_SZ	(DWCXG_EXT_ROVTL_EN ? DWCXG_NRVF : 0)
DWCXG_RXF_RSSL_DMCHB_SB	[function_of: (DWCXG_RXF_RSSL_PD_SB + DWCXG_RXF_RSSL_PD_SZ)]
DWCXG_RXF_RSSL_DMCHB_SZ	1
DWCXG_RXF_RSSL_DMCHLE_SB	[function_of: (DWCXG_RXF_RSSL_DMCHB_SB + DWCXG_RXF_RSSL_DMCHB_SZ)]
DWCXG_RXF_RSSL_DMCHLE_SZ	(DWCXG_RCHLIST_CNT>0) ? 1 : 0
DWCXG_RXF_RSSL_DMCH_SB	[function_of: (DWCXG_RXF_RSSL_DMCHLE_SB + DWCXG_RXF_RSSL_DMCHLE_SZ)]
DWCXG_RXF_RSSL_DMCH_SZ	DWCXG_BN_RCH_BN_RCHL_W
DWCXG_RXF_RSSL_DW	DWCXG_EXT_RSSL_EN ? [function_of: (DWCXG_RXF_RSSL_DMCH_SB + DWCXG_RXF_RSSL_DMCH_SZ)] : 0
DWCXG_RXF_RSSL_PD_SB	0
DWCXG_RXF_RSSL_PD_SZ	1
DWCXG_RXF_RSSL_SZ	(DWCXG_EXT_RSSL_EN ? DWCXG_RSS_TABLE_SIZE : 0)
DWCXG_RXF_RVTL_AF_SB	0
DWCXG_RXF_RVTL_AF_SZ	1
DWCXG_RXF_RVTL_DW	DWCXG_EXT_RVTL_EN ? [function_of: (DWCXG_RXF_RVTL_RCH_SB + DWCXG_RXF_RVTL_RCH_SZ)] : 0
DWCXG_RXF_RVTL_RCHLE_SB	[function_of: (DWCXG_RXF_RVTL_UNL_SB + DWCXG_RXF_RVTL_UNL_SZ)]
DWCXG_RXF_RVTL_RCHLE_SZ	(DWCXG_RCHLIST_CNT>0) ? 1 : 0
DWCXG_RXF_RVTL_RCH_SB	[function_of: (DWCXG_RXF_RVTL_RCHLE_SB + DWCXG_RXF_RVTL_RCHLE_SZ)]
DWCXG_RXF_RVTL_RCH_SZ	DWCXG_BN_RCH_BN_RCHL_W
DWCXG_RXF_RVTL_UNL_SB	[function_of: (DWCXG_RXF_RVTL_AF_SB + DWCXG_RXF_RVTL_AF_SZ)]
DWCXG_RXF_RVTL_UNL_SZ	1
DWCXG_RXP	DWCXG_FRP_EN
DWCXG_RXP_FPE_RCH_FW	DWCXG_FPE?((DWCXG_NUM_RX_CH_EFF>16)?4:DWCXG_NUM_RX_CH_EFFW):0

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_RXP_IACC_ADDR	[function_of: (DWCXG_RXP ? ((DWCXG_RXP_INST_DW_ADDR>=DWCXG_RXP_IFR_ADDR) ? DWCXG_RXP_INST_DW_ADDR : DWCXG_RXP_IFR_ADDR) : 0)]
DWCXG_RXP_IFR_ADDR	9
DWCXG_RXP_INST	DWCXG_FRP_ENTRIES
DWCXG_RXP_INST_ADDR	[function_of: (log(DWCXG_RXP_INST)/log(2))]
DWCXG_RXP_INST_DW_ADDR	[function_of: (log(DWCXG_RXP_INST * DWCXG_RXP_INST_DW_CNT)/log(2))]
DWCXG_RXP_INST_DW_CNT	[function_of: (DWCXG_RXP_INST_W/DWCXG_RXP_INST_DW_W)]
DWCXG_RXP_INST_DW_W	32
DWCXG_RXP_INST_W	128
DWCXG_RXQW	(DWCXG_NUM_RXQ<=2) ? 1 : (DWCXG_NUM_RXQ<=4) ? 2 : (DWCXG_NUM_RXQ<=8) ? 3 : 4
DWCXG_RX_SDPR	DWCXG_MTL_MEMTYP==1
DWCXG_RX_SRAM	DWCXG_RX_SDPR||DWCXG_SPRAM
DWCXG_S_AR_PYLD0_USER_WIDTH	4
DWCXG_S_AR_PYLD1_USER_WIDTH_PORT	3
DWCXG_S_AR_PYLD_USER_WIDTH_PORT	[function_of: (DWCXG_S_AR_PYLD0_USER_WIDTH + DWCXG_S_AR_PYLD1_USER_WIDTH_PORT)]
DWCXG_S_AW_PYLD0_USER_WIDTH	4
DWCXG_S_AW_PYLD1_USER_WIDTH_PORT	3
DWCXG_S_AW_PYLD_USER_WIDTH_PORT	[function_of: (DWCXG_S_AW_PYLD0_USER_WIDTH + DWCXG_S_AW_PYLD1_USER_WIDTH_PORT)]
DWCXG_SBDFC	DWCXG_PFC_EN ? 8 : 1
DWCXG_S_B_PYLD0_USER_WIDTH	2
DWCXG_S_B_PYLD_USER_WIDTH	DWCXG_S_B_PYLD0_USER_WIDTH
DWCXG_SGF_ECW	(DWCXG_SGF&& DWCXG_ASP_ECC)==1 ? (((DWCXG_PMDW+DWCXG_IMAW+1) <= 26) ? 6 : 7) : 0
DWCXG_SPRAM	DWCXG_MTL_MEMTYP==2
DWCXG_S_R_PYLD0_USER_WIDTH	[function_of: (DWCXG_NO_ECC_CHECKER_S*DWCXG_ECC_USER_WIDTH_S)]
DWCXG_S_R_PYLD1_USER_WIDTH	2
DWCXG_S_R_PYLD_USER_WIDTH	[function_of: (DWCXG_S_R_PYLD0_USER_WIDTH + DWCXG_S_R_PYLD1_USER_WIDTH)]
DWCXG_STRW	[function_of: (pow(2,DWCXG_BSA))]

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_S_W_PYLD0_USER_WIDTH	[function_of: (DWCXG_NO_ECC_CHECKER_S*DWCXG_ECC_USER_WIDTH_S)]
DWCXG_S_W_PYLD1_USER_WIDTH_PORT	2
DWCXG_S_W_PYLD_USER_WIDTH_PORT	[function_of: (DWCXG_S_W_PYLD0_USER_WIDTH + DWCXG_S_W_PYLD1_USER_WIDTH_PORT)]
DWCXG_TCW	(DWCXG_NUM_TC<=2) ? 1 : (DWCXG_NUM_TC<=4) ? 2 : (DWCXG_NUM_TC<=8) ? 3 : 4
DWCXG_TOT_TSO_MSIZE	(DWCXG_NUM_DMA_TSO_CH*DWCXG_TSO_MEM_SIZE)
DWCXG_TPRE	1
DWCXG_TTL_REG_ROWS	[function_of: (DWCXG_CSR_REG_ROWS*DWCXG_NUM_DMA_CSR_CH)]
DWCXG_TX_AV_ENABLE	DWCXG_AV_ENABLE&& DWCXG_NUM_TC>1&& (DWCXG_TC1_AV_EN||DWCXG_TC2_AV_EN||DWCXG_TC3_AV_EN||DWCXG_TC4_AV_EN||DWCXG_TC5_AV_EN||DWCXG_TC6_AV_EN||DWCXG_TC7_AV_EN)
DWCXG_TXCHCNT	((DWCXG_NUM_DMA_TX_CH<=16)?[function_of: (DWCXG_NUM_DMA_TX_CH-1)]:((DWCXG_NUM_DMA_TX_CH==20)?16: (DWCXG_NUM_DMA_TX_CH==24)?17: (DWCXG_NUM_DMA_TX_CH==28)? 18 :(DWCXG_NUM_DMA_TX_CH==32)?19: (DWCXG_NUM_DMA_TX_CH==40)?20: (DWCXG_NUM_DMA_TX_CH==48)?21: (DWCXG_NUM_DMA_TX_CH==56)?22: (DWCXG_NUM_DMA_TX_CH==64)?23: (DWCXG_NUM_DMA_TX_CH==80)?24 : (DWCXG_NUM_DMA_TX_CH==96)?25: (DWCXG_NUM_DMA_TX_CH==112)?26: (DWCXG_NUM_DMA_TX_CH==128)?27:31))
DWCXG_TX_PDMA	(SNPS_RSVDPARAM_50?DWCXG_NUM_TXQ:DWCXG_NUM_DMA_TX_CH)
DWCXG_TXQW	(DWCXG_NUM_TXQ<=2) ? 1 : (DWCXG_NUM_TXQ<=4) ? 2 : (DWCXG_NUM_TXQ<=8) ? 3 : 4
DWCXG_TX_SDPR	DWCXG_MTL_MEMTYP==1
DWCXG_TX_SRAM	DWCXG_TX_SDPR||DWCXG_SPRAM
DWCXG_TX_STSW	DWCXG_TIME_STAMPING ? 82 : 18
DWCXG_W_DMA_CHW	[function_of: (DWCXG_CHW)]
DWCXG_WOSRLW	[function_of: (log(DWCXG_AXI_GM_MAX_WR_REQUESTS)/log(2))]
DWCXG_XGMII_CTRL_WIDTH	DWCXG_XGMII_DDW ? 8 : 4

Table E-1 Internal Parameters (continued)

Parameter Name	Equals To
DWCXG_XGMII_DATA_WIDTH	DWCXG_XGMII_DDW ? 64 : 32
DWCXG_XGMII_DDR	DWCXG_XGMII == 1
DWCXG_XGMII_DDW	DWCXG_XGMII == 2
DWCXG_XGMII_SDR	DWCXG_XGMII == 0
MAX_RX_FRM	(DWCXG_RXFIFO_SIZE/16)
QNUMW	((DWCXG_NUM_RXQ<=2)?1: ((DWCXG_NUM_RXQ<=4)?2: ((DWCXG_NUM_RXQ<=8)?3:4)))
RFCW	((MAX_RX_FRM<=2)?1:((MAX_RX_FRM<=4)?2: ((MAX_RX_FRM<=8)?3:((MAX_RX_FRM<=16)?4: ((MAX_RX_FRM<=32)?5:((MAX_RX_FRM<=64)?6: ((MAX_RX_FRM<=128)?7:((MAX_RX_FRM<=256)?8: ((MAX_RX_FRM<=512)?9:((MAX_RX_FRM<=1024)?10: ((MAX_RX_FRM<=2048)?11: ((MAX_RX_FRM<=4096)?12: ((MAX_RX_FRM<=8192)?13: ((MAX_RX_FRM<=16384)?14:15))))))))))))))
SNPS_RSVDPARAM_10	DWCXG!=0&& DWCXG_SBD_FC_EN
SNPS_RSVDPARAM_15	DWCXG_ADD_SBD_IO
SNPS_RSVDPARAM_24	0
SNPS_RSVDPARAM_27	0
SNPS_RSVDPARAM_28	4
SNPS_RSVDPARAM_31	0
SNPS_RSVDPARAM_36	0
SNPS_RSVDPARAM_37	DWCXG_ETA
SNPS_RSVDPARAM_38	0
SNPS_RSVDPARAM_43	0
SNPS_RSVDPARAM_44	0
SNPS_RSVDPARAM_45	0
SNPS_RSVDPARAM_48	0
SNPS_RSVDPARAM_49	0
SNPS_RSVDPARAM_50	(DWCXG == 2) ? 1 : 0
SNPS_RSVDPARAM_52	0
SNPS_RSVDPARAM_53	0